

从 C++ 到计算系统：第二册

硬件原理、多核并行、高性能计算、同步、无锁结构与分布式计算

CPU Performance Study

本册接续第一册的程序执行基础，系统讲解现代 CPU 硬件、多核心并发、高性能计算、锁与无锁数据结构、并行算法、运行时和分布式计算。AI 模型、算子开发和推理引擎放入第三册。

前言

第一册已经回答了基础问题：一个 C++ 程序如何变成进程、地址空间、二进制接口、汇编指令和可测量的执行现象。它负责打牢源码到二进制、x86-64 汇编、System V ABI、Linux 基础工具、虚拟地址空间和可信 benchmark 的底座。第二册不再把这些内容重新讲一遍，而是把它们当作读者已经掌握的工具：能看汇编，能确认二进制身份，能写基本实验报告，能理解进程和虚拟地址空间的基本边界。

本册继续向下和向外展开，但不提前进入 AI。真正要理解高性能程序，必须先把“计算本身”讲清楚：核心怎样取指、译码、调度和提交，乱序执行为什么能隐藏延迟，分支预测为什么会失败，缓存和 TLB 为什么决定很多程序的上限，多核心之间为什么需要一致性协议，NUMA 为什么让同一块内存存在不同核心看来成本不同。第一册把单个程序放到机器上，第二册追问这台机器怎样扩展为多核心计算系统，再怎样扩展为多进程、多节点和分布式计算系统。

软件层面同样不能跳步。多线程不是简单地把循环切成几份；锁不是只有“慢”一个性质；原子操作不是更快的锁；无锁数据结构也不是不用同步。并行算法要面对数据依赖、负载均衡、局部性、归约顺序和任务粒度。分布式计算更进一步：一旦机器之间只能通过网络通信，延迟、带宽、故障、重试、超时、复制和一致性就会成为计算模型的一部分。

因此第二册的主题是计算系统，而不是某个应用领域。它从硬件原理开始，讲到单机高性能计算、多线程并发、锁与无锁结构、并行算法、运行时系统、异步 I/O 和分布式计算。AI 模型、张量、算子开发和量化推理引擎会放到第三册。这样安排不是延后重点，而是先把地基打牢：没有硬件和并发基础，后续任何 AI 算子优化都会变成经验堆砌。

核心思想

第二册的核心问题是：给定真实硬件、操作系统和网络边界，怎样组织数据、线程、同步、任务和通信，使计算能够正确、可测量、可扩展地运行。

本册仍然沿用第一册的写法：原理先行，代码作为证据；实验必须记录输入、环境、命令和误差；源码阅读抓数据结构、热路径、同步边界和性能瓶颈。不过，本册的证据重心会改变。第一册主要用反汇编、ELF、GDB、`readelf`、`objdump` 和基础 `perfstat` 建立“这段程序真的这样执行”的证据；第二册会更多使用拓扑、PMU、调度事件、锁等待、cache line 迁移、NUMA 放置、队列长度、吞吐曲线和失败注入来解释“系统为什么不能继续扩展”。

读者读完后，应当能解释一个多核程序为什么没有线性加速，能判断锁竞争和 false sharing 的差异，能写出可测试的并行归约、有界队列和环形队列，能读懂 Linux 调度、futex、RCU、内存管理和 perf 相关源码入口，也能把单机计算原则扩展到分布式系统。

怎样阅读本册

本册默认目标平台是本地 Linux 机器。普通 x86-64 Linux 台式机、笔记本或服务器适合完成完整实验；Termux 可以用于阅读、构建小实验和运行基础测试，但 perf、NUMA、线程亲和性、硬件计数器、futex 观察、eBPF 观测和分布式网络实验可能受权限、内核和设备能力限制。所有性能结论都必须写清环境。

阅读本册前，应默认已经完成第一册的最低要求：能把源码编译成二进制并确认产物身份，能读基本 x86-64 汇编，能解释调用约定和栈帧，能写出隔离热路径的 benchmark，能使用 Linux 常见工具观察进程、地址空间和基础性能计数。第二册不会反复解释这些背景；遇到这些内容时，本册只会说明它们怎样服务多核、同步、运行时或分布式问题。

阅读顺序建议严格按章节推进。前四章讲硬件执行模型，这是后面所有内容的解释基础。单机高性能计算部分讲测量、数据布局、SIMD 和典型内核，帮助读者把“程序慢”分解成计算、内存、分支、同步或 I/O 问题。并发部分讲线程、锁、条件变量、原子、内存序和无锁数据结构，重点不是背 API，而是理解共享状态怎样被保护、发布和回收。并行算法和分布式计算部分再把这些原则扩展到任务图、运行时、网络通信、复制和一致性。

本册的贯穿实验叫 Compute Systems Labs。它不追求一开始很大，而是从小而真实的模块开始：顺序归约、并行归约、带锁计数器、原子计数器、有界队列、任务切分和简单 benchmark。后续扩写会继续加入线程池、工作窃取、SPSC 环形队列、MPMC 队列、并行扫描、分布式 shuffle 和故障注入。

心智模型

读本册时始终追问四件事：数据在哪里，谁会读写它，硬件会怎样移动它，失败或竞争发生时系统怎样保持正确。

每章至少做一个实验。硬件章节要配合 `lscpu`、`numactl`、`perfstat`、拓扑图和内存访问 microbenchmark；并发章节要写出可重复触发竞争、阻塞、false sharing、CAS 失败或内存序问题的小程序；运行时章节要记录队列长度、任务粒度、worker 空闲时间和背压；分布式章节即使只在本机多进程模拟，也要记录延迟、重试、超时、重复请求和消息顺序。

常见误区

不要把高性能计算理解成“把线程数开大”。线程数只是资源请求，性能来自数据复用、负载均衡、同步控制和测量证据。没有这些证据，加线程常常只是更快地制造争用。

本册的章节之间有明确承接关系。第一部分回答硬件成本从哪里来，第二部分回答怎样把成本变成可测假设，第三部分回答共享内存里怎样保持正确，第四部分回答怎样把多个线程组织成可扩展运行时，第五部分回答当共享内存消失、网络和故障进入模型后，计算原则怎样变化。不要把某章当成孤立技巧阅读；每次做实验，都要把结论回连到这条链上。

全书结构

本册分为五个部分，围绕“计算系统如何正确且高效地扩展”展开。第一册已经承担源码、进程、虚拟地址、汇编、ABI、工具链和基础测量，本册只在需要建立新成本模型时短暂回扣这些内容，不再重复铺陈。这样处理的目的，是把篇幅留给第一册没有充分展开的主题：多核心硬件、缓存一致性、NUMA、SIMD 和内存带宽、Linux 调度和同步、无锁结构、并行运行时、异步 I/O、分布式计算和故障模型。

第一部分是硬件执行模型。它讲核心流水线、乱序执行、分支预测、寄存器重命名、缓存层级、TLB、页表、预取、NUMA、互连和缓存一致性。读者要在这里建立硬件成本模型：一条指令不只是语法，背后有取指、依赖、执行端口、访存、提交和一致性流量。

第二部分是单机高性能计算。第一册已经讲过 benchmark 纪律，所以本部分不再停留在“怎样计时”这类基础问题，而是进入多核进阶测量、Roofline、PMU 归因、cache 和 TLB 事件、数据布局、局部性、SIMD、指令级并行和典型计算内核。目标是把“优化”从口号变成可验证的分析：到底是算力不够、内存带宽不够、cache miss 太多、分支预测差、TLB 覆盖不足、同步等待太长，还是 benchmark 边界仍然写错。

第三部分是多线程、同步与内存模型。它讲线程生命周期、调度、亲和性、锁、条件变量、信号量、原子操作、内存序、false sharing、无锁队列和内存回收。这里的重点是正确性先于性能：如果可见性、互斥、进度保证和对象生命周期没有说清楚，所谓高性能就是不稳定的。

第四部分是并行算法与运行时。归约、扫描、分区、图遍历、任务图、线程池、工作窃取、异步 I/O 和背压，都是把多核硬件变成可用计算能力的桥梁。读者在这里要学会从算法依赖和数据移动角度设计并程序序。

第五部分是分布式计算。它把单机原则扩展到网络和集群：RPC、序列化、时间、故障、分片、复制、一致性、共识、shuffle、检查点和观测性。分布式不是“多几台机器”，而是把通信和失败纳入计算模型。

核心思想

第二册的主线是一条从硬件到集群的链：硬件决定局部成本，多线程决定共享内存内的协作，并行算法决定可扩展形状，分布式系统决定跨机器的通信和故障边界。

和第一册的边界

第一册已经讲清楚的内容，本册只作为前提使用：源码到目标文件和 ELF 的路径，进程和虚拟地址空间的基本概念，x86-64 汇编阅读，System V AMD64 ABI，C++ 与汇编互操作，基础 Linux 命令，基础 benchmark 设计，基础 **perfstat** 和 Linux 源码入口。第二册遇到这些内容时，重点不是重复定义，而是解释它们在可扩展计算中的新作用。

例如，第一册讲虚拟地址空间时，目标是让读者理解进程地址、映射和缺页；第二册讲 TLB 和页级性能时，目标是解释页大小、page walk、NUMA 放置和首次触页怎样改变吞吐。第一册讲 **perfstat** 时，目标是建立可信测量意识；第二册讲 PMU 时，目标是用事件组合区分前端、后端、内存、同步和调度瓶颈。第一册讲汇编控制流时，目标是读懂分支和跳转；第二册讲分支预测时，目标是理解输入分布、预测器状态和流水线回滚怎样限制单核吞吐。

本册扩展的重点

本册扩展不是简单增加术语，而是围绕可扩展计算的核心矛盾展开。硬件部分要能从 pipeline、ROB、load/store queue、store buffer、cache line、TLB、NUMA 和一致性协议解释成本。同步部分要能从不变量、等待谓词、futex、内存序、线性化点和内存回收解释正确性。运行时部分要能从任务粒度、局部性、工作窃取、背压和关闭语义解释工程稳定性。分布式部分要能从消息、重试、幂等、复制、共识、shuffle、检查点和观测性解释跨机器计算。

每个章节都要有三个出口：能画出机制图，能写出最小可运行例子，能设计一份可复查实验。只

有这样，第二册才不是第一册的复述，而是把第一册的基础能力推进到现代计算系统。

规模目标和章节配额

第二册按正式书籍标准写作，最终正文不少于 50 万单位。统计口径只看 `chapters/` 下的正文内容，以中文字符加英文单词计数；代码块、命令、反汇编、终端输出、配置、前言、目录、附录和 LaTeX 标记不计入。50 万只是最低验收线，不是上限；如果硬件、多核、运行时、无锁结构和分布式计算的知识链需要更大篇幅，就按完整性继续写，不能因为数字达到门槛而停在半成品。这个目标不是排版长度目标，而是知识密度目标：每一章都必须把原理、机制、案例、实验和源码阅读边界讲清楚。

按 18 章估算，平均每章需要 2.7 万到 2.9 万单位。考虑章节难度不同，硬件执行、缓存/TLB/NUMA、测量、锁与原子、无锁结构、任务运行时和分布式计算应承担更高篇幅；实验路线和源码索引只能辅助正文，不能替代正文。后续扩写时，每章都要围绕同一条链展开：先讲模型，再讲硬件或系统实现，再讲 C++ 代码形态，再讲测量证据，再讲反例和工程边界。

常见误区

本册不接受用重复定义、空泛口号、附录堆表、题目清单或模板化实验报告凑字数。若一个段落不能帮助读者理解现代计算系统的机制、判断或实践，就不应进入正文。

章节质量标准

每章必须按“机制、案例、代码、测量、源码、作业”六个维度写作。机制部分要解释底层原理，不只给术语定义；案例部分要从真实问题出发，展示为什么朴素方案不够；代码部分要给 baseline、改进版本和边界处理版本，不能只给最终答案；测量部分要说明实验矩阵、指标、预期现象和失败解释；源码部分要给 Linux 或生产系统的窄入口，并说明读源码要验证什么；作业部分要能训练读者独立复现、修改、测量和写报告。

本册的案例不能停留在“数组求和”“打印线程 ID”这种单点玩具上。简单例子可以作为入口，但必须继续推进到工程问题：线程数增加为什么不线性加速，锁竞争和 false sharing 怎样区分，TLB miss 和 cache miss 怎样分开，任务粒度怎样影响线程池，队列满时背压怎样传导，分布式 shuffle 为什么会被热点 key 拖垮。每个案例都要有问题背景、朴素方案、失败原因、改进方案、代价和实验验证。

贯穿大项目

第二册贯穿项目命名为 Compute Systems Engine。它不是一个孤立实验，而是整本书的工程主线。第一阶段实现顺序和并行归约、测量 harness、拓扑记录和结果校验；第二阶段加入数据布局实验、线程本地 partial、false sharing 对照和 NUMA 初始化策略；第三阶段加入 mutex 队列、条件变量、有界队列、原子计数器和内存序实验；第四阶段加入线程池、任务图、工作窃取和运行时指标；第五阶段加入 SPSC ring buffer、MPMC queue、内存回收实验和压力测试；第六阶段加入异步 I/O 模拟、背压策略、批处理和限流；第七阶段在本机多进程模拟分布式 shuffle、幂等提交、重试、检查点和故障注入。

每章扩写时都要回答两个问题：本章为 Compute Systems Engine 增加了什么能力；这个能力需要哪些测试和测量才能被认为可靠。若某章不能落到项目上，也必须说明它提供的是哪一种模型能力，例如分支预测帮助解释热循环，TLB 帮助解释大工作集，NUMA 帮助解释多 socket 扩展曲线。

目录

第一部分	硬件执行模型：计算为什么会快也会慢	
第 1 章	计算系统全景	2
第 2 章	核心流水线、乱序执行与分支预测	32
第 3 章	缓存、TLB 与页级性能	60
第 4 章	NUMA、互连与缓存一致性	84
第二部分	单机高性能计算：从测量到数据流	
第 5 章	可信测量、Roofline 与性能计数器	108
第 6 章	数据布局、局部性与预取	130
第 7 章	SIMD、指令级并行与向量化	157
第 8 章	典型计算内核模式	181
第三部分	多线程、同步与内存模型	
第 9 章	线程、调度与亲和性	207
第 10 章	锁、条件变量与信号量	229
第 11 章	原子操作与内存序	254
第 12 章	无锁数据结构	285
第四部分	并行算法与运行时	
第 13 章	归约、扫描与分区	311
第 14 章	任务运行时与工作窃取	337
第 15 章	I/O、异步与背压	363
第五部分	分布式计算：把单机原则扩展到集群	
第 16 章	分布式计算模型	389
第 17 章	分片、复制与共识	417
第 18 章	分布式计算工程实践	445
附录 A	实验路线	495
附录 B	源码阅读索引	496

第零部分

硬件执行模型：计算为什么会快也会慢

第 1 章

计算系统全景

1.1 问题与目标

第二册不能从一串名词开始。若第一章只是依次介绍多核、缓存、TLB、线程、锁、无锁队列、异步 I/O 和分布式协议，读者会得到一张很长的术语表，却仍然不知道真实系统为什么会变慢、为什么会出错、为什么某个概念必须出现。经典教材舒服的地方，不是标题漂亮，而是它能从读者已经会写的小程序出发，让每个新概念都从一个绕不开的问题里长出来。

本章选择一个足够朴素的任务：读取一批日志，按事件名统计次数，写出结果。这个任务小到可以用几十行 C++ 写出串行版本，大到会自然遇到第二册的全部核心矛盾。输入变大以后，数据搬运比加法更贵；线程变多以后，共享计数表会产生锁竞争和缓存行迁移；写出结果以后，临时文件存在并不代表结果已经有效；任务失败以后，重试可能让同一份数据被合并两次；扩展到多机以后，分片、shuffle、提交点和幂等性都会变成正面问题。

所以本章不是把全书知识提前压缩一遍，而是建立一条以后每章都要遵守的推导线：

一个具体任务

- > 一个朴素实现
- > 一个暴露出来的问题
- > 一个必须引入的系统概念
- > 一个可检查的接口或实验
- > 一个新的边界条件

这条线也解释了为什么本册要接在第一册后面。第一册已经讲过程序如何从源码变成进程，虚拟地址如何组织内存，汇编指令如何改变寄存器和内存，Linux 工具如何帮助我们观察运行结果。第二册不再重复这些基础，而是把一个进程放进更复杂的计算环境里：多个核心同时执行，多个缓存层级同时保存副本，多个线程共享状态，多个任务在队列中等待，多个节点通过网络交换数据，多个失败路径随时可能打断正常流程。

核心理想

第二册的主题不是“使用更多资源”，而是“让一个计算任务在更多资源、更复杂等待和更多失败边界下仍然正确、快速、可解释”。硬件、操作系统、运行时、算法和分布式协议，都是为这个目标服务的工具。

本章读完后，你不需要已经掌握所有优化技巧，但应当能回答六个基础问题。第一，什么结果算正确，reference 应该怎样定义。第二，数据从输入到输出经过哪些形态，哪些字段属于热路径，哪些字段只是诊断信息。第三，CPU、缓存、TLB 和内存系统为什么会影响一个看似普通的循环。第四，多个执行实体怎样协作，等待点怎样从隐式调用变成可观察状态。第五，结果何时对外有效，临时状态和提交事实怎样区分。第六，后续章节怎样沿同一条逻辑继续展开，而不是变成互不相干的概念堆叠。

1.2 从一个朴素日志统计程序开始

先不要急着谈多线程。一个系统如果连单线程语义都讲不清，后面的并行优化只会把错误放大。假设每行日志采用非常简单的格式：

```
timestamp user_id event_name value
```

本章先只统计 `event_name` 出现次数。朴素思路是逐行读取，解析字段，把事件名放入哈希表计数。这个版本不追求极致速度，而是作为 reference 的雏形：它要尽量直接、容易审查、错误策略清楚，方便后面所有优化版本对照。

Listing 1.1 串行 reference 的最小形态

```

1 #include <stdint>
2 #include <string>
3 #include <string_view>
4 #include <unordered_map>
5 #include <vector>
6
7 struct ParseResult {
8     bool ok = false;
9     std::string_view event_name;
10    std::uint32_t error_code = 0;
11 };
12
13 ParseResult parse_line(std::string_view line);
14
15 using EventCountMap = std::unordered_map<std::string, std::uint64_t>;
16
17 EventCountMap run_reference(const std::vector<std::string>& lines)
18 {
19     EventCountMap counts;
20
21     for (const std::string& line : lines) {
22         const ParseResult parsed = parse_line(line);
23         if (!parsed.ok) {
24             continue;
25         }
26
27         ++counts[std::string(parsed.event_name)];
28     }
29
30     return counts;
31 }

```

这段代码看起来简单，却已经暴露出许多系统问题。第一，错误行被跳过，这是否符合业务语义。第二，`std::string(parsed.event_name)` 每次可能分配内存，热路径上会有额外成本。第三，`std::unordered_map` 的节点通常分散在堆上，查询和插入可能产生随机访存。第四，哈希表扩容会造成突发成本。第五，如果多个线程直接共享这个 map，需要锁或原子协调。第六，函数返回时只有内存结果，没有说明结果如何写到磁盘、如何恢复、如何防止重复提交。

这些不是要马上修复的缺陷，而是教材叙事的入口。一本合格的系统书不能在这里直接跳到“使用线程池”或“使用 SIMD 解析”。它必须先把朴素程序拆开，说明哪些问题属于语义，哪些问题属于数据布局，哪些问题属于硬件执行，哪些问题属于并发协作，哪些问题属于持久化和恢复。

一、reference 的作用不是慢版本 很多人把 reference 理解成“一个慢但容易写的版本”。这个说法只说对了一半。Reference 的核心作用是定义正确性，而不是定义速度。后续的多线程版本、SIMD 版本、异步 I/O 版本、分布式版本，都必须在同一批输入上与 reference 对齐。只要 reference 语义含糊，任何性能数字都没有意义。

例如错误行到底计入错误计数，还是直接跳过；大小写是否敏感；空事件名是否合法；最后一行没有换行符是否有效；同一个输入文件重复处理一次是否允许；输出顺序是否必须稳定。这些问题不写清楚，后续优化版本就可能各自做出不同解释。表面上它们都在“统计日志”，实际得到的是不同程序。

二、朴素程序帮我们找到第一批边界 从代码结构看，最小统计程序包含四个阶段：读取、解析、聚合、输出。读取阶段把字节放进内存；解析阶段把文本字段变成结构化字段；聚合阶段把字段

变成计数；输出阶段把计数对外公布。每个阶段都有自己的边界。读取边界关心文件、offset、buffer 和行分隔；解析边界关心字段规则和错误分类；聚合边界关心 key 的身份和计数溢出；输出边界关心格式、排序、临时文件和提交点。

如果这些边界没有提前分清，后续章节就会混乱。缓存章节会不清楚热数据在哪里，线程章节会不清楚共享状态是谁，锁章节会不清楚临界区保护什么，I/O 章节会不清楚写出完成是否等于结果有效，分布式章节会不清楚重复执行是否安全。因此第一章要把简单程序里的边界讲透，而不是急着扩展。

三、第一版代码为什么故意不完美 第一版 reference 仍然把事件名复制进 `std::string`，仍然使用通用哈希表，仍然跳过错误行。这是教学上有意保留的粗糙度。若第一段代码就塞进字符串池、冷热分离、分片聚合、批量写出和 manifest，读者会看到一堆设计，却看不到设计从哪里来。正确的顺序是先接受一个朴素版本，再让它在更大的输入、更高的并发和更真实的失败路径中暴露问题，最后再引入必要机制。

1.3 规模放大：朴素程序为什么会长成系统

一个几十行日志统计程序，在一百行输入上看不出系统味道。它读文件很快，哈希表很小，错误可以人工检查，写出结果也只是一瞬间。很多教材写得散乱，原因之一就是没有先解释规模怎样改变问题：小输入上不存在的问题，在大输入、长时间运行、多线程和失败恢复中会变成主线。第二册必须从这里把逻辑接起来，否则后面讲缓存、队列、锁和分布式都会像突然冒出来的概念。

先把输入放大到一亿行。哪怕每行平均只有几十字节，程序也要处理数 GB 的文本。此时，读入字节、扫描分隔符、计算哈希、比较字符串、分配 key、更新计数和写出 partial，每一步都可能占据可观时间。再把事件名分布改成高度倾斜：百分之九十的记录都是同一个事件。此时，全局计数器会成为热点。再把 worker 增加到多个核心：热点计数所在 cache line 会在核心之间迁移。再加入故障：某个 worker 写出 partial 后崩溃，重试又写出一份 partial。此时，目录里有两个看起来都像结果的文件，reduce 如果扫描目录就会重复合并。

这些变化不是在增加“高级功能”，而是在逐步去掉朴素程序默认享受的假设。小输入默认内存够用，大输入不一定。单线程默认顺序固定，多线程不一定。一次运行默认不会失败，长时间作业不一定。内存结果默认立即有效，外部文件和分布式消息不一定。系统概念就是从这些假设被打破的地方长出来的。

一、小输入隐藏固定成本 每个系统都有固定成本和可变成本。启动线程、打开文件、分配哈希表、初始化队列、提交 manifest、写 trace，这些成本不完全随记录数线性增加。输入很小时，固定成本可能比实际计算还大；输入很大时，每条记录上的可变成本又会主导。若只用小输入测量，很容易得出错误结论：某个批处理优化看起来没用，可能只是因为输入太小；某个线程池版本看起来更慢，可能只是因为任务太少，线程调度成本盖过了并行收益。

日志统计里，批量解析就是固定成本和可变成本的取舍。一次处理一行，接口简单，但每行都要走一遍函数调用、边界检查和容器操作。一次处理一批，批次头部、队列入出和 trace 只发生一次，可以摊薄固定成本；但批次太大，又会增加尾延迟和失败后重做范围。于是 batch size 不是随便选的常量，而是由输入规模、缓存容量、队列等待、I/O 写出和恢复粒度共同决定。

二、工作负载不是一个数字 说“输入有一亿行”仍然不够。系统性能取决于工作负载形状。总字节数决定数据移动下限，平均行长影响解析，最大行长影响 buffer 策略，事件种类数影响聚合结构，最高频事件占比影响热点，错误率影响冷路径，split 大小影响任务粒度，输入是否已在 page cache 中影响 I/O 结论。一个工程报告如果只写输入规模，不写这些形状，就很难解释为什么同一程序在另一批数据上表现不同。

例如事件种类少时，线程本地 partial 的收益通常明显，因为共享热点可以被推迟到合并阶段。事件种类极多时，partial 自身可能很大，合并阶段会成为新的瓶颈，字符串存储和哈希表局部性更重要。若错误行集中在一个 split，取消和重试策略会被放大；若错误均匀分布，错误路径可能只是轻微开销。工作负载不同，正确的系统形状就不同。

三、复杂度不能替代机器成本 朴素扫描、批量解析和本地聚合，从算法复杂度看都可能是 $O(n)$ 。但机器成本并不相同。一个版本顺序扫描连续 buffer，另一个版本每条记录都创建字符串并插入节点式哈希表；它们的复杂度相同，cache、TLB、分配器和分支行为却完全不同。复杂度回答规模增长趋势，机器成本回答真实硬件怎样执行。第二册不能丢掉复杂度，但也不能停在复杂度。

同样是常数时间操作，成本也可能差几个数量级。一次本地整数加法、一次原子加法、一次互斥锁竞争、一次系统调用、一次磁盘写、一次跨机 RPC，都可以被粗略看成某种常数操作，但系统设计不能把它们当成同一种常数。日志统计里，减少一条本地加法的收益，可能抵不过一次额外字符串分配；减少一次字符串复制的收益，可能抵不过一次错误的跨节点 shuffle。成本尺度决定优化优先级。

四、批量化和流水线从问题中出现 当 reader、parser、worker 和 writer 的速度不同，单个大循环就不再合适。Reader 可能被磁盘或 page cache 影响，parser 消耗 CPU，worker 聚合时受哈希和内存影响，writer 受文件系统影响。若所有阶段串行执行，任何一个慢阶段都会让整个程序停住；若阶段之间用有界队列连接，就可以形成流水线，让不同阶段同时工作。但流水线不是免费午餐，它引入了队列容量、关闭语义、错误传播和背压。

批量化也是同一个推导。逐行传递会让队列节点和调度成本过高；整文件传递又会让内存压力和失败重做范围过大。批次是折中：它把若干记录作为一个处理单位，让固定成本被摊薄，同时让错误定位和恢复仍然可控。这个折中会在后面的 SIMD、任务运行时、异步 I/O 和分布式 shuffle 中反复出现。

五、瓶颈迁移不是异常 优化一个阶段后，瓶颈经常迁移。最初的串行程序可能慢在字符串分配；引入字符串视图和批量 buffer 后，慢点可能变成哈希表随机访问；改成线程本地 partial 后，慢点可能变成合并阶段；加入更多线程后，慢点可能变成内存带宽或 NUMA；异步写出后，慢点可能变成 manifest 提交；扩展到多机后，慢点可能变成 shuffle 和热点 key。若报告只写“优化后快了多少”，没有说明瓶颈是否迁移，就不算完整分析。

瓶颈迁移也能解释为什么同一个优化在不同阶段结论不同。在全局锁严重竞争时，线程本地 partial 提升巨大；当内存带宽已经打满时，同样的 partial 结构再精细也可能收益有限。在 parser 分支预测严重时，SIMD 扫描有价值；当 I/O 阻塞主导时，改 parser 只会让队列更快积压。高质量教材要教读者重新判断瓶颈，而不是记住一个固定答案。

心智模型

规模放大不是把小程序运行更多次，而是逐步打破小程序隐含的假设。每打破一个假设，就会逼出一个新的系统边界：工作负载、批次、队列、共享状态、提交点、恢复和证据。

1.4 第一道边界：什么结果算正确

系统设计的第一个问题不是“怎样更快”，而是“怎样判断没有算错”。对于日志统计，正确性至少包括输入合同、解析规则、错误策略、计数语义和输出格式。输入合同说明每行有哪些字段、字段用什么分隔、最大行长是多少、编码是什么。解析规则说明事件名从哪里取、是否允许空字段、是否允许转义。错误策略说明坏行是跳过、终止、记录到错误输出，还是进入特殊计数。计数语义说明重复行、重复文件和重试是否可能造成重复计数。输出格式说明 key 的排序、数值宽度和校验方式。

这些内容看起来像业务细节，其实是系统优化的基础。没有输入合同，就不知道 split 能不能从任意字节切开。没有错误策略，就不知道 parser 是否能把错误局部化。没有计数语义，就不知道并行 partial 能不能无序合并。没有输出格式，就不知道结果能不能稳定比较。性能工程不是绕过这些细节，而是把它们变成可验证的接口。

Listing 1.2 把语义写成可检查的结构

```
1 #include <cstdint>
2 #include <string>
3 #include <string_view>
4
5 enum class ParseError : std::uint32_t {
6     none = 0,
7     empty_line = 1,
8     missing_field = 2,
9     event_too_long = 3,
10    malformed_value = 4
```

```

11 };
12
13 struct RecordId {
14     std::uint32_t input_id = 0;
15     std::uint32_t split_id = 0;
16     std::uint64_t byte_offset = 0;
17     std::uint32_t line_index = 0;
18 };
19
20 struct ParsedRecord {
21     RecordId id;
22     std::string_view event_name;
23     ParseError error = ParseError::none;
24 };

```

这里的重点不是多写几个结构体，而是让系统以后能回答“这一条记录是谁”。**RecordId** 把记录和输入位置绑定起来。解析成功时，热路径只需要事件名和必要标识；解析失败时，错误报告可以通过 **RecordId** 回到原始输入。这样做会增加一点控制信息，但换来的是可追踪性。没有可追踪性，后续并发和分布式错误会很难复盘。

一、为什么要有数据身份 单线程小程序可以靠调用栈理解当前处理哪一行。多线程之后，记录会被切成批次，放入队列，被不同 worker 处理，再合并成 partial。分布式之后，记录还会跨进程、跨节点、跨重试 attempt 迁移。若没有稳定身份，一条错误记录可能只留下“worker 3 解析失败”这种无用信息。更糟的是，重复执行时系统无法判断某个 partial 是否来自同一份输入。

数据身份不是为了让代码看起来正式，而是为了让优化后仍能解释数据。缓存优化可能改变对象布局，线程池可能改变执行顺序，I/O 批处理可能改变写出时机，分布式调度可能改变 worker 位置。只要 **RecordId**、**SplitId**、**AttemptId** 和 **BlockId** 这些身份保持清楚，系统就能在执行顺序变化后仍然追踪事实。

二、错误策略必须早于并行化 假设输入中某一行缺少事件字段。朴素程序直接跳过，另一个程序把它计入 **bad_record**，第三个程序遇到错误立即终止。三者都可能有道理，但它们不是同一个语义。如果先并行化再讨论错误策略，会遇到更难的问题：一个 worker 发现错误时，其他 worker 已经写出 partial 怎么办；错误是否需要取消整批任务；已经提交的结果是否撤销；错误报告是否保留输入位置。

因此本册要求每个模块先写错误出口。错误出口至少说明谁观察到错误，哪些资源需要释放，哪些结果可以保留，哪些结果必须撤销，是否允许重试。这样后面讲锁、原子、I/O 和分布式时，错误不是额外补丁，而是正常路径的一部分。

三、reference 要覆盖边界输入 Reference 不只跑正常输入。第一章的 reference 至少要覆盖空文件、只有一行、最后一行没有换行符、超长行、字段缺失、事件名重复、事件名极多、某个事件名占据大部分记录、输入被拆成多个 split 后跨边界。这些输入不是为了凑测试数量，而是为了逼迫语义变清楚。

例如跨 split 的半行问题会迫使我们定义 split 的边界。若 split 可以从任意字节开始，reader 必须找到第一条完整记录；若 split 必须由上游按行边界生成，调度器要保存这个合同。再比如热点事件名会迫使我们思考共享计数表是否会变成热点。边界输入越早出现，后续优化越不容易走偏。

常见误区

不要把“能跑完一个正常样例”当成正确。系统越往后扩展，越多错误来自边界输入、重复执行、迟到完成和局部失败。第一章就把这些条件写进 reference，后面才能放心谈性能。

1.5 第二道边界：数据怎样穿过机器

当输入从几百行变成几亿行，程序的主要成本通常不再是几次整数加法，而是数据如何移动。日志字节从存储设备进入内核页缓存，进入用户态 buffer，进入解析器，进入聚合结构，最后进入输出文件。每一次复制、解析、哈希、分配和写回，都可能比 `++count` 本身贵得多。第二册的硬件部分正是从这里开始，而不是凭空介绍缓存名词。

朴素 reference 的热路径至少包含三类数据。第一类是输入字节，它们连续、体积大、只读，适合顺序扫描。第二类是热字段，例如事件名的起止位置、事件名哈希值和计数，它们在聚合时反复访问。第三类是冷信息，例如原始行文本、错误诊断、输入文件名、详细 trace，它们很重要，但不应该被每次计数操作一起搬进缓存。

如果把所有信息塞进一个大对象，热循环就会不断搬动不需要的字段。硬件按 cache line 搬数据，而不是按 C++ 字段的“重要程度”搬数据。一个对象里只要有热字段和冷字段混在一起，CPU 读取热字段时就可能顺便把冷字段所在的字节也带进缓存。小输入看不出问题，大输入就会变成缓存容量和带宽浪费。

Listing 1.3 把热字段和冷字段分开

```
1 #include <cstdint>
2 #include <string>
3 #include <string_view>
4 #include <vector>
5
6 struct HotRecord {
7     std::uint64_t event_hash = 0;
8     std::uint32_t event_begin = 0;
9     std::uint32_t event_size = 0;
10    std::uint32_t local_line_index = 0;
11 };
12
13 struct ColdRecordInfo {
14     RecordId id;
15     std::string original_line;
16     ParseError error = ParseError::none;
17 };
18
19 struct ParsedBatch {
20     std::string input_bytes;
21     std::vector<HotRecord> hot_records;
22     std::vector<ColdRecordInfo> cold_infos;
23 };
```

这段代码不是最终实现，只是表达一个原则：热路径应尽量携带计算必须反复使用的最小信息，冷路径保存诊断和恢复需要的上下文。后续章节讲数据布局、预取、SIMD 和 cache 时，都会回到这个原则。高性能不是把所有信息删掉，而是让不同频率的信息走不同路径。

一、从字节流到结构化记录 读取文件时，程序看到的是字节流；解析之后，程序才知道哪些字节构成时间戳、用户、事件名和值。这个转换看似简单，却决定了后面的并行性。若每条记录独立，多个 worker 可以处理不同 split；若格式允许引号、转义、多行字段，解析状态可能跨字符传播，切分就不再随意。输入格式越复杂，reader 和 parser 的合同越重要。

日志统计的教学版本可以先限制格式，让每行独立。这个限制不是偷懒，而是为后续推导建立清晰起点。等理解了独立记录如何切分、聚合和提交，再放宽格式，读者就能看清额外复杂性来自哪里。经典教材不会一开始就给最复杂的工业格式，而是先给能暴露本质的模型。

二、为什么连续访问通常更友好 连续数组让硬件更容易工作。CPU 读取某个地址时，缓存通

常会以 cache line 为单位搬入一段连续字节；硬件预取器也更容易识别顺序访问模式。相反，链表节点、哈希表节点和散落的小对象会让访问地址难以预测，导致更多 cache miss 和 TLB 压力。两个算法复杂度都可以是 $O(n)$ ，但连续扫描和随机追指针的机器成本可能相差很大。

这正是朴素 `std::unordered_map` 版本的问题之一。它语义方便，但每个 key 可能是独立分配的字符串，每个桶和节点也可能分散在内存中。对于事件种类较少的日志，局部聚合可以把多个记录先聚到线程本地结构，再批量合并；对于事件种类很多的日志，可以考虑字符串池、排序归约或分片哈希。选择哪一种方案，取决于 workload，而不是取决于某个容器名字是否高级。

三、硬件章节要回答的不是术语，而是代价 后面讲流水线时，要解释一个循环为什么可能受依赖链限制。讲缓存时，要解释热字段为什么应当紧凑，随机访问为什么会慢。讲 TLB 时，要解释大工作集和大量小对象为什么会击穿地址转换缓存。讲 NUMA 时，要解释同一个虚拟地址背后的物理页可能靠近某个 socket，远端访问为什么会拖慢多线程程序。所有硬件概念都要回到“这批日志字节怎样穿过机器”这个问题。

心智模型

先画数据路径，再谈硬件优化。若不知道字节从哪里来、变成什么结构、被谁反复访问、在哪里写出，就无法判断缓存、TLB、预取、SIMD 或 NUMA 优化到底作用在哪一段。

1.6 第三道边界：多个执行实体怎样协作

单线程 reference 清楚以后，最自然的想法是把输入切成多个 split，让多个 worker 并行处理。表面上这只是把循环拆开，实际会立刻引入执行实体、队列、共享状态、等待和背压。多线程不是把同一段代码复制几份那么简单，因为多个执行实体一旦共享资源，就必须回答谁拥有状态、谁能修改状态、谁等待谁、等待多久、失败如何传播。

先看一个常见但危险的设计：所有 worker 共享一个全局计数 map，每解析到一条记录就加锁更新。小输入上它能工作，线程数上来后却可能变慢。原因不是“锁一定慢”这么粗，而是每条记录都要进入同一个临界区，哈希表内部状态和锁所在 cache line 会在核心之间迁移。worker 越多，争用越严重，最后程序的主成本可能从解析和聚合变成等待锁。

更好的第一步通常是线程本地聚合。每个 worker 处理自己的 split，把结果写入本地 partial；最后再合并 partial。这个设计牺牲了一点内存，换来热路径上更少共享写。它也把问题拆成两个阶段：map 阶段并行处理输入，reduce 阶段合并 partial。这个结构以后会自然扩展到分布式计算。

Listing 1.4 线程本地 partial 的接口形状

```

1 #include <stdint>
2 #include <string>
3 #include <unordered_map>
4 #include <vector>
5
6 struct Split {
7     std::uint32_t input_id = 0;
8     std::uint32_t split_id = 0;
9     std::uint64_t begin_offset = 0;
10    std::uint64_t end_offset = 0;
11 };
12
13 struct PartialCount {
14     std::uint32_t split_id = 0;
15     std::unordered_map<std::string, std::uint64_t> counts;
16 };
17
18 PartialCount process_split(const Split& split);
19
20 EventCountMap merge_partials(const std::vector<PartialCount>& partials)

```



```

21 {
22     EventCountMap merged;
23
24     for (const PartialCount& partial : partials) {
25         for (const auto& [event_name, count] : partial.counts) {
26             merged[event_name] += count;
27         }
28     }
29
30     return merged;
31 }

```

这段代码仍然不是最终高性能版本，但它把共享写从每条记录一次，降低到每个 partial 合并时发生。这个变化背后的思想很重要：并发优化的第一原则通常不是马上使用更复杂的同步原语，而是减少共享、延迟共享、分片共享，让大部分工作在本地完成。

一、任务队列为什么必须有边界 如果 reader 不断产生 split，worker 处理速度又跟不上，队列会不断增长。无界队列看起来提高了吞吐，实际可能把内存耗尽，或者把延迟藏起来。系统需要有界队列：队列满时，上游必须等待或降速；队列关闭时，下游必须能醒来退出；任务失败时，等待者必须能收到错误。这个机制叫背压，但它不是一个独立术语，而是从“生产者可能快于消费者”这个问题中推导出来的。

有界队列的合同至少包括容量、push 失败条件、pop 等待条件、关闭语义和错误传播。若这些合同不清楚，线程池很容易卡死。比如 producer 在队列满时等待，consumer 因错误提前退出，队列没有关闭通知，producer 就可能永远睡眠。后续讲 mutex、condition variable 和 semaphore 时，必须围绕这些具体不变量展开，而不是只背 API。

二、线程池不是为了把代码异步化 线程池的核心价值是控制执行资源。没有线程池时，每个任务都创建线程，成本高且数量失控；有线程池后，系统可以限制 worker 数量，复用线程，集中处理任务生命周期。线程池也带来新的责任：任务提交后是否一定执行，shutdown 是否等待正在运行的任务，任务抛出异常如何传播，worker 内部等待另一个任务是否可能死锁。

这些问题都可以在日志统计项目里具体出现。Reader 提交 split，worker 解析并聚合，writer 写出 partial。如果 writer 阻塞，worker 是否继续产生结果；如果某个 split 失败，其他 split 是否取消；如果主线程等待所有 future，而 worker 又等待主线程提交的任务，是否可能形成等待环。线程池不是语法糖，而是执行图的一部分。

三、等待点必须显式化 并发程序最难解释的时间通常不是计算时间，而是等待时间。等待队列空间、等待任务、等待锁、等待 I/O、等待另一个线程完成、等待重试预算，都会让吞吐和延迟变化。若代码没有记录等待点，系统报告只能说“变慢了”，无法判断是上游太快、下游太慢、队列太小、锁太粗，还是任务粒度太碎。

第一章要求每个任务至少有几个时间点：创建、入队、开始运行、结束运行、提交结果。后续章节可以把这些时间点展开成更细指标，例如锁等待、futex、context switch、I/O completion 和 RPC deadline。显式等待不是为了多写日志，而是为了让性能瓶颈有证据。

1.7 队列、状态机和背压

上一节说到队列，但队列不只是一个容器。一个普通 `std::queue` 只能保存元素，不能表达生产者什么时候该停、消费者什么时候该醒、系统失败时谁负责通知等待者。真正进入系统设计后，队列必须带状态机。状态机的意义不是把代码写复杂，而是把隐式控制流变成可讨论的合同。

在日志统计项目里，reader 向 split queue 放任务，worker 从 split queue 取任务并生成 partial，writer 从 partial queue 取结果并写出 block。只要 writer 慢于 worker，partial queue 就会增长；只要 worker 慢于 reader，split queue 就会增长。若队列无界，系统会把速度不匹配伪装成内存增长，直到延迟失控或进程被杀。若队列有界，上游在队列满时等待，速度不匹配会立刻暴露出来。这就是背压。

一、队列状态先于实现技巧 一个有界队列至少有四种状态。Open 表示可以 push，也可以 pop。

Closing 表示不再接受新任务,但已有任务仍可被消费。Closed 表示队列为空,消费者应该退出。Failed 表示队列因为错误关闭,等待者应收到失败原因。很多初学者只写一个布尔变量 `done`,结果无法区分“正常没有更多任务”和“发生错误必须取消”。这两种情况对上游和下游完全不同。

例如 reader 读完整个文件后关闭 split queue,这是正常结束。Worker 解析时发现不可恢复的输入合同错误,应该让下游知道作业失败,并让其他等待者醒来退出。Writer 写 manifest 失败,也可能需要阻止 reducer 继续读取 partial。若队列只有空和非空,没有失败状态,错误只能藏在日志里,控制流仍然可能继续向前走。

二、等待谓词必须写成不变量 条件变量最容易写错的地方,是把等待当成“睡一会儿”,而不是等待某个谓词变真。对有界队列来说,producer 等待的谓词通常是“队列未满,或者队列已经关闭,或者队列已经失败”;consumer 等待的谓词通常是“队列非空,或者队列已经关闭,或者队列已经失败”。只要谓词没有写清楚,虚假唤醒、关闭竞态和错误传播都会出问题。

这就是为什么后面讲锁和条件变量时,不能只写 API。必须先写队列的不变量:元素数量不超过容量;open 状态允许 push;closing 状态拒绝新 push;closed 状态没有剩余元素;failed 状态唤醒全部等待者;任何等待都必须能被关闭或失败打断。API 只是实现这些不变量的工具。

三、任务状态帮助解释等待 队列里保存的不应只是一个函数对象。教学项目里,每个任务至少应有 `TaskId`、输入 split、attempt、状态、创建时间、入队时间、开始时间、结束时间和错误码。这样当吞吐下降时,报告能区分任务还没入队、已经入队但没有 worker、正在运行、等待写出、已经失败还是已经提交。没有状态字段,线程池内部就会成为黑箱。

Listing 1.5 任务状态记录的教学形态

```

1 #include <cstdint>
2
3 enum class TaskState : std::uint32_t {
4     created = 0,
5     queued = 1,
6     running = 2,
7     finished = 3,
8     committed = 4,
9     failed = 5,
10    cancelled = 6
11 };
12
13 struct TaskTrace {
14     std::uint32_t task_id = 0;
15     std::uint32_t split_id = 0;
16     std::uint32_t attempt_id = 0;
17     TaskState state = TaskState::created;
18     std::uint64_t timestamp_ns = 0;
19     std::uint32_t worker_id = 0;
20 };

```

这段代码看起来像控制面信息,但它会直接影响性能分析。若大量任务停在 queued,说明 worker 数量、任务粒度或上游提交方式有问题;若大量任务 running 时间很长,说明计算或 I/O 本身慢;若 finished 到 committed 间隔很长,说明写出或 manifest 是瓶颈;若 cancelled 很多,说明错误传播或重试策略需要检查。状态机让等待可见。

四、背压不是简单把队列变小 有界队列的容量需要设计。容量太小,生产者和消费者频繁互相等待,吞吐可能下降。容量太大,系统延迟和内存占用升高,失败时需要处理的在途任务也更多。合理容量来自成本账本:每个 split 大小、每个 ParsedBatch 大小、每个 partial 大小、worker 数量、writer 吞吐和允许的内存预算。队列容量应该能解释,而不是写成随手的常量。

在日志统计中,split queue 可以按 worker 数量设置为若干倍,让 reader 有少量预读空间,但不要整个文件都塞进队列。partial queue 要考虑 writer 速度,若 writer 被磁盘阻塞,上游应当降

速，而不是无限制制造 partial。分布式系统中的限流和 backpressure，本质上就是同一个问题扩大到跨节点消息。

五、关闭路径是并发程序的试金石 很多并发代码在正常输入上能工作，在关闭时暴露问题。Reader 结束后，worker 如何知道没有新 split；worker 失败后，reader 是否还继续读取；writer 失败后，partial queue 中已有数据如何处理；主线程收到取消后，正在运行的 worker 能否安全结束；某个等待 push 的 producer 能否醒来。这些问题不解决，程序就会出现偶发卡死。

因此，第一章的样板要求每个并发模块都写关闭路径。不是只写 happy path 的“处理完所有任务”，还要写 reader 失败、worker 失败、writer 失败、主线程取消、队列满时取消、队列空时关闭。后面讲线程池和锁时，这些关闭路径会成为测试用例，而不是口头描述。

核心思想

队列的本质不是保存元素，而是管理速度差、等待关系和关闭语义。没有状态机的队列，只能演示并发；有状态机和背压的队列，才开始像系统组件。

1.8 第四道边界：结果何时对外有效

单线程程序返回一个 map 时，结果有效性比较直观。系统变大后，结果往往先进入临时状态，再通过某个提交动作对外有效。这个区分非常关键。一个 worker 写出了临时 partial 文件，不代表 reduce 阶段应该读取它；一个异步写请求返回，不代表数据已经进入最终 manifest；一个 attempt 告诉 driver 成功，不代表它一定是被接受的那一次成功。

考虑一个失败场景。Worker 处理 split 7，写出文件 `part-7.tmp`，随后进程崩溃。Driver 没收到成功响应，于是重试 split 7，新 attempt 写出 `part-7-retry.tmp`。如果 reduce 阶段只是扫描目录，把两个文件都合并，split 7 就被统计了两次。这个错误不是性能问题，而是提交语义问题。正确做法是让 manifest 成为唯一有效事实：临时文件可以存在，但只有 manifest 接受的 block 才能被下游读取。

Listing 1.6 用 manifest 表达提交事实

```

1 #include <stdint>
2 #include <string>
3 #include <vector>
4
5 enum class BlockState : std::uint32_t {
6     temporary = 0,
7     committed = 1,
8     discarded = 2
9 };
10
11 struct ShuffleBlock {
12     std::uint32_t split_id = 0;
13     std::uint32_t attempt_id = 0;
14     std::uint32_t block_id = 0;
15     std::uint64_t record_count = 0;
16     std::uint64_t checksum = 0;
17     std::string path;
18     BlockState state = BlockState::temporary;
19 };
20
21 struct ManifestEntry {
22     std::uint32_t manifest_id = 0;
23     std::uint32_t split_id = 0;
24     std::uint32_t accepted_attempt_id = 0;
25     std::uint32_t block_id = 0;

```

```

26     std::uint64_t checksum = 0;
27 };
28
29 using Manifest = std::vector<ManifestEntry>;

```

Manifest 的意义在于把“产生了文件”和“结果已经被系统接受”分开。临时文件是事实，但不是提交事实。Reduce 只读取 manifest 中出现的 block，因此重复 attempt 不会造成重复合并。这个思想后面会扩展到 I/O 完成、分布式提交、检查点和恢复。

一、提交点是正确性和性能的分界线 提交点越早，下游越快看到结果，但失败恢复可能更复杂。提交点越晚，系统更容易一次性校验和提交，但延迟可能增加，临时状态也更多。批量提交可以减少 I/O 和锁开销，却扩大崩溃后重做范围；逐条提交恢复简单，却可能吞吐很低。性能和正确性不是两条线，而是在提交点上同时出现。

日志统计项目的第一版可以选择按 split 提交 partial。每个 split 完成后，worker 写出临时 block，校验 checksum，向 manifest 提交一条记录。这个粒度足够小，失败后只重做一个 split；又足够大，避免每条记录都提交。后续章节讲批处理、异步 I/O 和分布式 checkpoint 时，可以继续讨论粒度如何调整。

二、幂等性来自身份和提交事实 幂等不是一句“重复执行也没事”。它需要身份和提交事实支撑。若每个 attempt 都有 `AttemptId`，每个输出 block 都有 `BlockId`，manifest 又能记录某个 split 已经接受了哪个 attempt，那么重复消息、迟到成功和重复扫描都可以被处理。若没有这些身份，系统只能依赖时间顺序和文件名猜测，失败路径很容易出错。

一个简单规则是：可重试的操作必须有请求身份，可对外有效的结果必须有提交记录，可重复到达的响应必须能被判定为旧响应或已接受响应。单机任务队列、异步 I/O、分布式 RPC 都遵守这个规则。第二册后面不会把幂等性当成分布式章节才出现的概念，而是从第一章的本地 manifest 就开始训练。

三、恢复不是失败后再补 恢复能力要在状态设计时出现。若写出文件时没有 checksum，恢复时无法判断文件是否完整；若 manifest 没有输入 split 范围，恢复时无法判断哪些 split 已经完成；若任务状态没有 attempt，恢复时无法区分旧执行和新执行；若错误日志没有 record id，恢复后无法定位坏输入。失败发生后再补信息，通常已经太晚。

这也是第一章反复强调身份、时间线和提交点的原因。它们不是管理系统的花活，而是恢复路径的最低信息量。后续任何优化，只要破坏这些信息，就必须给出新的证明方式。

1.9 失败时间线：从一次崩溃推导恢复协议

为了避免把恢复讲成概念，本节只分析一次具体崩溃。输入 split 7 被提交给 worker 2，attempt id 为 1。Worker 解析 split，生成 local partial，写出临时文件，计算 checksum，然后准备向 manifest 提交。就在写出临时文件之后、manifest 提交之前，进程崩溃。重启后，driver 看不到 split 7 的 manifest 条目，于是提交 attempt id 为 2。新 attempt 成功写出并提交。此时目录里可能同时存在两个临时文件，但 manifest 只有 attempt 2 的条目。

这个时间线解释了为什么“扫描目录找文件”是错误恢复方法。目录记录的是文件系统事实，不是作业语义事实。临时文件可以来自失败 attempt、旧 attempt、部分写入、调试输出或清理失败；它们不能直接进入 reduce。Manifest 记录的是提交事实：哪个 split 的哪个 attempt 被接受，checksum 是什么，record count 是多少，下游可以读取哪个 block。恢复协议的核心，就是让语义事实不依赖偶然留下的临时状态。

一、正常时间线 正常情况下，split 从 created 进入 queued，再进入 running。Worker 完成解析和本地聚合后，生成 temporary block。Writer 把 block 写入临时路径，计算 checksum，确认记录数，然后请求 manifest commit。Manifest 检查这个 split 是否尚未接受其他 attempt；若尚未接受，就写入条目并返回 committed；若已经接受，就拒绝这个迟到 attempt。Worker 收到 committed 后，任务状态才进入 committed。

这条时间线里有两个容易混淆的点。第一，worker finished 不等于作业结果有效。Finished 只是计算完成，committed 才表示外部可见。第二，文件写完不等于 block 有效。Block 必须通过校验并被 manifest 引用。把这两个点分清，后面异步 I/O 和分布式响应才不会误读。

二、崩溃后的扫描顺序 恢复程序启动后，不应该先相信临时目录，而应该先读取 manifest。Manifest 告诉系统哪些 split 已经有有效输出。随后再扫描临时目录，把不在 manifest 中的旧临时文件标记为 orphan，按策略清理或保留诊断。接着 driver 重新提交未完成 split。这个顺序很重要：先看 manifest，再处理临时文件；不要先扫描目录再拼出 manifest。

如果反过来，系统会把失败 attempt 留下的文件当成候选输出，恢复逻辑就会变得复杂。它必须猜测文件是否完整、是否来自旧运行、是否已经被 reduce 读过。让 manifest 成为唯一提交事实，可以把猜测变成查询。

三、校验和不是装饰 Checksum 的作用不是防止所有数据损坏，而是给恢复一个最低限度的完整性证据。Writer 写出 block 后，计算 checksum 并把它写进 manifest。Reducer 读取 block 时重新计算，若不一致，就拒绝这个 block 并触发恢复。没有 checksum，短写、截断、旧文件覆盖和路径错误都可能变成静默错误。

教学项目里的 checksum 可以先很简单，但接口要保留它的位置。后续章节可以讨论更强的校验、压缩块校验、分段校验和端到端校验。重要的是读者从第一章就知道：结果有效不只取决于控制状态，也取决于数据本身能被验证。

四、重试预算防止恢复变成放大器 重试可以恢复临时失败，也可以放大故障。若磁盘已满，writer 失败后无限重试只会消耗 CPU 和日志空间；若输入合同错误，重试同一个 split 也不会成功；若网络拥塞，所有 worker 同时重试可能造成更大拥塞。因此 attempt 必须有重试预算和错误分类。可恢复错误进入重试，不可恢复错误进入 failed，超出预算进入作业失败。

这条规则在单机和分布式中一致。单机 writer 短暂失败可以重试，输入格式错误不该重试；分布式 RPC 超时可以重试，业务语义冲突不该盲目重试。恢复协议不是“遇错再跑一遍”，而是根据错误类型、attempt 身份和提交事实决定下一步。

五、迟到响应必须可判定 分布式系统里经常出现迟到响应，单机异步系统里也一样。Attempt 1 可能在 driver 判定超时后仍然完成，并在 attempt 2 提交后才返回成功。如果 driver 只看“成功”两个字，就可能用旧结果覆盖新结果。正确做法是让响应携带 split id、attempt id、block id 和 checksum，manifest 根据当前已接受 attempt 判断它是新提交、重复提交还是迟到提交。

这就是为什么第一章要引入身份。身份不是为了日志好看，而是让迟到事件可判定。没有 attempt id，系统无法区分“这是当前运行的成功”还是“这是旧运行的迟到成功”。没有 block id，系统无法关联文件和提交。没有 manifest id，系统无法说明 reducer 读取的是哪个版本的提交事实。

六、恢复协议也要有 reference 恢复逻辑同样需要 reference。最简单的 reference 可以不关心性能，只根据输入 split 和已接受 manifest 重新计算最终结果。然后对照优化版本：无论 worker 崩溃多少次、临时文件留下多少个、attempt 顺序怎样变化，只要最终 manifest 接受的 split 集合相同，reduce 结果就必须与 reference 一致。这样故障注入才有判断标准。

很多系统教材把恢复放在最后讲，导致读者把它看成额外章节。本册从第一章就把恢复放进主线，是因为失败会改变接口。没有恢复需求，临时文件和最终文件可以混在一起；有恢复需求，就必须有 manifest。没有重试需求，任务可以没有 attempt；有重试需求，就必须有 attempt id。恢复不是附加功能，而是塑造系统结构的力量。

深入理解

一次崩溃就足以推出四个概念：临时状态、提交事实、attempt 身份和恢复扫描顺序。若教材直接给出这些名词，读者会觉得抽象；从崩溃时间线推出来，它们就是不得不有的工程边界。

1.10 第五道边界：硬件为什么会改变软件设计

到这里，我们已经从语义、数据、执行和提交几个角度拆开了日志统计。现在回到第二册第一部分的标题：硬件执行模型。硬件不是背景知识，而是会反过来改变软件接口和数据结构的约束。一个数据结构在源码层看起来清楚，不代表在机器上便宜；一个算法在复杂度上最优，不代表它适合缓存和多核；一个同步原语在 API 上简单，不代表它不会造成 cache line 所有权来回迁移。

先看聚合。若事件种类很少，全局哈希表里某几个 key 会成为热点。多个线程更新同一个计数，即使用原子加法，也会让包含计数的 cache line 在多个核心之间移动。每次移动都需要一致性协议协调，代价远高于本地寄存器加法。于是线程本地 partial 不是一种“高级并发技巧”，而是从硬件

缓存一致性中推导出的设计：让大部分更新在本地发生，把跨核心共享推迟到合并阶段。

再看解析。若每条记录都创建 `std::string`，程序会频繁分配小对象。小对象不仅带来分配器成本，还会让事件名散落在堆上，增加随机访存和 TLB 压力。于是字符串视图、字符串池、批量分配和冷热分离就不是过度设计，而是为了让热路径更连续，让硬件更容易预取和缓存。

一、流水线让依赖链变重要 CPU 核心可以同时处理多条指令，但这种并行需要独立工作。如果一个循环每次迭代都依赖上一次结果，例如单个累加器连续相加，依赖链可能限制吞吐。多累加器版本把一条长依赖链拆成几条短依赖链，让乱序执行更容易重叠工作。后面讲核心流水线和乱序执行时，会用这种例子说明为什么“同样的加法次数”也可能有不同速度。

日志统计里也有类似问题。若所有记录都更新同一个计数器，依赖和共享都会集中；若先在线程本地或分片结构里计数，依赖被拆散，合并阶段再处理全局结果。这里的思想与多累加器相通：不要把所有工作压在一条不可并行的链上。

二、缓存让布局变重要 缓存按块搬运数据。热字段越集中，一次缓存行搬运能服务的有效工作越多；冷字段混入热对象，缓存就会搬来很多暂时不用的字节。后面讲 cache 和 TLB 时，会详细分析数组、结构体数组、结构体拆分、对齐、padding、false sharing 和随机访问。本章只先建立一个判断：数据布局不是代码风格，而是机器执行成本的一部分。

对日志统计来说，`HotRecord` 可以连续存放，解析循环顺序写入，聚合循环顺序读取。错误详情和原始行文本进入冷路径，只在出错或诊断时访问。这个设计不会让所有情况都最快，但它让主路径更清楚，也让后续测量有对象可测。

三、TLB 让页级行为变重要 程序使用虚拟地址，CPU 访问内存前需要地址转换。TLB 缓存最近的虚拟页到物理页映射。若工作集跨越大量页面，或者随机访问分散小对象，TLB miss 会增加。大数据顺序扫描通常比大量小节点随机访问更友好，不只是因为 cache，也因为页级局部性更好。

这会影响容器选择。通用哈希表方便，但节点式布局可能让访问跨越许多页面。排序后归约、开放寻址哈希、分片数组或批量 arena 分配，有时能以更好的局部性换取更高吞吐。是否值得这样做，要看事件种类、输入规模、内存预算和错误诊断需求，不能只凭容器名下结论。

四、NUMA 让“同一台机器”不再简单 多 socket 机器上，内存物理位置可能靠近某个 socket。一个线程访问本地 NUMA 节点内存较快，访问远端内存较慢。如果主线程先分配并初始化所有 buffer，其他 socket 上的 worker 可能频繁访问远端页面。于是 first-touch、线程绑定和按 NUMA 节点分块就会进入软件设计。后面讲 NUMA 时，我们会把日志统计的输入 split 和 partial buffer 放到这个背景下分析。

这里先记住一件事：多线程性能差，不一定是锁；可能是内存放置、线程迁移、共享缓存争用、带宽上限或远端访问。没有硬件证据，就不要急着改同步结构。

1.11 第六道边界：证据怎样支撑判断

系统性能分析最忌讳把猜测写成结论。看到多线程不加速，就说“锁太慢”；看到 CPU 占用高，就说“计算密集”；看到一次运行很快，就说“优化有效”。这些说法都缺少证据。第二册每章都要训练一种写法：先给现象，再给假设，再给可验证指标，再给对照实验，最后承认边界。

日志统计项目的性能报告不需要一开始就很复杂，但至少包含输入规模、事件分布、错误比例、线程数、机器信息、编译参数、运行次数、热身策略、reference 校验和主要指标。主要指标可以从简单的吞吐、总耗时、CPU 时间、最大队列长度、错误行数开始；后面再逐步加入 PMU 事件、cache miss、TLB miss、context switch、futex、I/O 等待和 trace。

一、输入分布决定结论能否外推 同样是一亿行日志，事件名只有十种和事件名有一千万种，是两种完全不同的 workload。前者容易出现热点 key，局部聚合收益大；后者哈希表体积大，字符串存储和随机访存更重要。同样是错误率千分之一，错误均匀分布和错误集中在某个 split，也会影响取消和重试。报告不写输入分布，就无法解释性能差异。

因此每个实验都应保存输入摘要：总行数、字节数、平均行长、最大行长、事件种类数、最高频事件占比、错误行数、split 数量。摘要不是装饰，而是让后来的人知道这个结论适用在哪里。

二、对照实验要一次只改变一个主因素 如果把字符串池、线程池、分片哈希、SIMD 解析和异步写出一次性全加进去，速度变快也很难解释原因。经典教材的推导通常一次只改变一个主因素：先比较串行 reference 和线程本地 partial，观察共享写减少后的变化；再比较不同 split 大小，观察

任务粒度；再比较冷热分离，观察缓存和分配；再比较异步写出，观察 I/O 等待。这样每个概念都有证据，而不是堆功能。

这并不意味着真实工程只能小步提交。真实工程可以合并多个优化，但教学和复盘必须能拆开原因。第二册后续每章都会尽量保持这种节奏。

三、测量边界要诚实 在手机或普通云主机上，可能没有完整 PMU 权限；在单 socket 机器上，无法真实验证 NUMA；在本机模拟网络时，无法代表真实跨机延迟；文件系统缓存也可能掩盖磁盘成本。报告要把这些边界写出来。诚实写边界不是降低质量，而是提高结论可信度。

例如本章实验如果只在本地文件系统上运行，就不要声称证明了分布式 shuffle 的性能；如果只用小输入，就不要声称缓存优化可以外推到大数据；如果没有固定 CPU 频率，就要承认运行时间可能受频率变化影响。专业的系统报告不会把所有结论说满。

深入理解

证据链的最低形状是：reference 证明结果，输入摘要说明 workload，版本和编译参数说明构建，指标说明现象，对照实验支持或反驳假设，边界说明哪些结论还不能外推。后续章节只是在这条链上增加更专业的指标。

1.12 三张图：身份、执行和成本

面对一个计算系统，第一反应不应是写更多代码，而是画三张图。第一张是数据身份图，回答每份数据从哪里来、现在叫什么、何时有效。第二张是执行等待图，回答哪些执行实体在工作，哪些地方可能等待。第三张是成本账本，回答每个阶段主要消耗什么资源，证据在哪里。这三张图能把硬件、运行时、算法和恢复连接到同一张工作台上。

一、数据身份图 日志统计的数据身份可以这样画：

```
input file
-> split_id
-> record_id
-> parsed hot record
-> local partial
-> shuffle block
-> manifest entry
-> final result
```

每一次箭头都表示数据形态变化。输入文件切成 split，split 内的行获得 record id，解析后生成热字段，本地聚合后生成 partial，写出后生成 block，manifest 接受后成为有效输出，最终 reduce 读 manifest 得到结果。若某个箭头没有身份，恢复时就会断链。若某个状态没有有效条件，下游就可能读取临时数据。

这张图以后会出现在不同章节。缓存章节会问 hot record 是否连续；线程章节会问 partial 是否线程本地；I/O 章节会问 block 是否完整；分布式章节会问 manifest 是否能处理重复 attempt。图没有变，只是每章放大不同边。

二、执行等待图 同一个作业的执行等待图可以这样画：

```
reader
-> bounded split queue
-> worker threads
-> partial queue
-> writer
-> manifest commit
-> reducer
```

箭头不是简单调用，而是可能等待的边。Reader 可能等待 split queue 有空间；worker 可能等

待任务，也可能等待 partial queue 有空间；writer 可能等待 I/O 完成；manifest commit 可能等待锁或持久化；reducer 必须等待所需 block 被提交。任何一个等待点都可能成为瓶颈。若只看源码调用顺序，就看不到队列积压和异步完成。

执行图还要标出关闭路径。Reader 完成后如何通知 worker 没有新 split；worker 完成后如何通知 writer 没有新 partial；某个 worker 失败后，队列中的任务是否取消；writer 失败后，已经完成的 worker 如何退出。没有关闭路径的并发设计，在正常输入上可能没问题，在错误路径上容易卡死。

三、成本账本 成本账本不是财务表，而是把每个阶段的主要资源写清楚：

```
read bytes      -> I/O, page cache, copy
parse fields    -> branch, SIMD opportunity, input bandwidth
aggregate local -> hash, allocation, cache locality
merge partials  -> key cardinality, memory bandwidth, contention
write blocks    -> buffering, syscall, filesystem
commit manifest -> serialization, fsync policy, retry rule
reduce output   -> data movement, sort or hash, checksum
```

成本账本的作用是防止错配优化。若主成本是本地聚合的随机访存，先改异步 I/O 未必有效；若主成本是 writer 阻塞导致队列积压，给 parser 加 SIMD 可能只会更快制造等待；若主成本是 manifest 锁，增加 worker 可能让尾延迟更差。成本账本让讨论回到证据，而不是回到感觉。

心智模型

数据身份图保证系统知道“这是谁”；执行等待图保证系统知道“谁在等谁”；成本账本保证系统知道“时间花在哪里”。三张图缺一，系统分析都会失去一条主线。

1.13 代码演进：从一行日志到可恢复作业

前面已经把语义、数据、执行、提交和证据讲成几条边界。现在需要把这些边界落到代码演进上。高质量教材不应该一上来给一个很大的框架，然后要求读者接受它的复杂性；更好的写法是先解决一个小问题，观察它暴露出的缺口，再扩展一点代码。每一次扩展都要能说清：它解决了上一版的什么问题，又留下了什么问题。

下面的代码不是最终工程实现，而是一组教学骨架。它们故意从一行日志开始，逐步扩展到 reference、记录身份、批处理、局部聚合和 manifest 提交。每一步都很小，但连起来就是 Compute Systems Engine 的第一条主线。

一、先只解决一行：解析 event 字段 第一步不要写线程池，也不要写哈希表。先回答最小问题：给一行日志，能不能稳定取出 event_name。这一版的输入合同很窄：字段用单个空格分隔，格式是 timestampuser_idevent_namevalue，事件名最长不超过固定字节数，不处理引号和转义。限制写清楚以后，代码就容易审查；以后要支持更复杂格式，也知道复杂性从哪里增加。

Listing 1.7 第一步：只解析一行日志的事件字段

```
1 #include <cstdint>
2 #include <string_view>
3
4 struct EventField {
5     bool ok = false;
6     std::string_view event_name;
7     ParseError error = ParseError::none;
8 };
9
10 EventField parse_event_field(std::string_view line)
11 {
12     constexpr char FIELD_DELIMITER = ' ';
```

```

13 constexpr std::size_t MAX_EVENT_NAME_BYTES = 64;
14
15 const std::size_t first_space = line.find(FIELD_DELIMITER);
16 if (first_space == std::string_view::npos) {
17     return {false, {}, ParseError::missing_field};
18 }
19
20 const std::size_t second_space =
21     line.find(FIELD_DELIMITER, first_space + 1);
22 if (second_space == std::string_view::npos) {
23     return {false, {}, ParseError::missing_field};
24 }
25
26 const std::size_t third_space =
27     line.find(FIELD_DELIMITER, second_space + 1);
28 if (third_space == std::string_view::npos) {
29     return {false, {}, ParseError::missing_field};
30 }
31
32 const std::size_t event_begin = second_space + 1;
33 const std::size_t event_size = third_space - event_begin;
34 if (event_size == 0) {
35     return {false, {}, ParseError::missing_field};
36 }
37 if (event_size > MAX_EVENT_NAME_BYTES) {
38     return {false, {}, ParseError::event_too_long};
39 }
40
41 return {
42     true,
43     line.substr(event_begin, event_size),
44     ParseError::none
45 };
46 }

```

这一版解决了“字段在哪里”的问题，同时暴露出第一个限制：它只返回 `std::string_view`，这个 view 指向原始行。只要原始行还活着，它就有效；如果调用者把原始行释放了，event view 就会悬垂。因此下一步不能只考虑解析，还要考虑输入数据的生命周期。

二、再解决多行：写出可对照 reference 第二步把单行解析放进循环，得到最小 reference。这里仍然不优化字符串复制，因为 reference 的任务是定义语义和输出，而不是追求速度。它把成功行和失败行分开统计，让后续优化版本有明确对照。

Listing 1.8 第二步：把单行解析扩展成 reference

```

1 #include <cstdint>
2 #include <string>
3 #include <vector>
4
5 struct ReferenceResult {
6     EventCountMap counts;
7     std::uint64_t accepted_records = 0;
8     std::uint64_t rejected_records = 0;

```



```

9 };
10
11 ReferenceResult run_reference_checked(
12     const std::vector<std::string>& lines)
13 {
14     ReferenceResult result;
15
16     for (const std::string& line : lines) {
17         const EventField field = parse_event_field(line);
18         if (!field.ok) {
19             ++result.rejected_records;
20             continue;
21         }
22
23         ++result.counts[std::string(field.event_name)];
24         ++result.accepted_records;
25     }
26
27     return result;
28 }

```

这一版解决了“怎么算正确”的问题，但还不能回答“哪一行错了”。如果输入只有十行，人工看日志就行；如果输入有一亿行，错误报告只写 rejected count 完全不够。于是下一步必须给每条记录加身份。

三、给记录加身份：错误能够回到输入 第三步不改变算法，只改变记录接口。解析函数接收 `RecordId`，返回 `ParsedRecord`。这样错误不再只是一个计数，而能回到 input、split、offset 和行号。注意，这一步没有让程序更快，但让后续并发和恢复变得可解释。

Listing 1.9 第三步：把解析结果和 RecordId 绑定

```

1 ParsedRecord parse_record(RecordId id, std::string_view line)
2 {
3     const EventField field = parse_event_field(line);
4     if (!field.ok) {
5         return {id, {}, field.error};
6     }
7
8     return {id, field.event_name, ParseError::none};
9 }
10
11 bool is_valid_record(const ParsedRecord& record)
12 {
13     return record.error == ParseError::none;
14 }

```

这一版解决了“错误能否定位”的问题，但仍然是一行一行处理。每行都单独进入聚合，无法表达批次生命周期，也很难把热字段和冷诊断分开。下一步要把若干行组成一个 batch。

四、引入 batch：让生命周期和冷热数据清楚 第四步把 split 中的一批行解析成 batch。热路径只保留聚合需要的事件名和哈希，错误行进入冷路径。这里仍然使用 `std::string_view`，因此 batch 的生命周期必须覆盖这些 view 的使用期。真实工程里可以让 `ParsedBatch` 拥有输入字节 buffer，或者让上游保证输入块在下游完成前不释放。

Listing 1.10 第四步：把逐行解析扩展成批处理

```
1 #include <span>
2
3 std::uint64_t stable_hash(std::string_view text);
4
5 struct EventView {
6     RecordId id;
7     std::string_view event_name;
8     std::uint64_t event_hash = 0;
9 };
10
11 struct BatchParseResult {
12     std::vector<EventView> events;
13     std::vector<ColdRecordInfo> errors;
14 };
15
16 BatchParseResult parse_split_lines(
17     std::uint32_t input_id,
18     std::uint32_t split_id,
19     std::span<const std::string> lines)
20 {
21     BatchParseResult batch;
22
23     for (std::size_t index = 0; index < lines.size(); ++index) {
24         const RecordId id{
25             input_id,
26             split_id,
27             0,
28             static_cast<std::uint32_t>(index)
29         };
30         const ParsedRecord record = parse_record(id, lines[index]);
31
32         if (!is_valid_record(record)) {
33             batch.errors.push_back({
34                 id,
35                 lines[index],
36                 record.error
37             });
38             continue;
39         }
40
41         batch.events.push_back({
42             id,
43             record.event_name,
44             stable_hash(record.event_name)
45         });
46     }
47
48     return batch;
49 }
```

这一版解决了“批次和冷热路径”的问题，也暴露出两个新问题。第一，`byte_offset` 这里暂时填了 0，说明这段代码还不是完整 reader；真正按文件切 split 时，reader 必须给出准确 offset。第二，`static_cast<std::uint32_t>(index)` 假设一个 split 的行数不会超过 `std::uint32_t` 可表示范围；真实工程要么限制 split 大小，要么把行号字段改成更宽类型。教材代码必须把这些边界说出来，不能假装不存在。

五、局部聚合：先减少共享，再谈锁 第五步把 batch 聚合成 partial。这里先不让多个线程写同一个全局 map，而是每个 split 产生自己的 `PartialCount`。这样热路径没有跨线程共享写。后面要优化容器布局、字符串池或排序归约，都可以在这个局部边界内替换实现。

Listing 1.11 第五步：把 batch 聚合成局部 partial

```
1 PartialCount aggregate_batch(
2     std::uint32_t split_id,
3     const BatchParseResult& batch)
4 {
5     PartialCount partial;
6     partial.split_id = split_id;
7
8     for (const EventView& event : batch.events) {
9         ++partial.counts[std::string(event.event_name)];
10    }
11
12    return partial;
13 }
```

这一版解决了“多个 worker 是否必须共享全局 map”的问题。它仍然复制字符串，仍然使用节点式哈希表，但这些是下一阶段的数据结构优化问题，而不是并发语义问题。教材写法要把问题分层：先把共享边界改清楚，再优化局部容器；不要一段代码同时引入线程、字符串池、SIMD 和 manifest，让读者分不清每个变化解决了什么。

六、提交 partial：从内存结果走向可恢复输出 第六步把 partial 写成 `ShuffleBlock`，再提交到 manifest。下面只展示提交判断，不展示文件写入细节。关键规则是：同一个 split 只能接受一个 attempt；迟到 attempt 可以存在，但不能成为有效结果。

Listing 1.12 第六步：manifest 只接受一个 attempt

```
1 struct CommitDecision {
2     bool accepted = false;
3     ManifestEntry entry;
4 };
5
6 CommitDecision commit_block(
7     Manifest& manifest,
8     const ShuffleBlock& block)
9 {
10    for (const ManifestEntry& entry : manifest) {
11        if (entry.split_id == block.split_id) {
12            return {false, entry};
13        }
14    }
15
16    ManifestEntry entry;
17    entry.manifest_id =
18        static_cast<std::uint32_t>(manifest.size());
```

```

19     entry.split_id = block.split_id;
20     entry.accepted_attempt_id = block.attempt_id;
21     entry.block_id = block.block_id;
22     entry.checksum = block.checksum;
23
24     manifest.push_back(entry);
25     return {true, entry};
26 }

```

这一版解决了“重复 attempt 是否会重复计数”的问题。它仍然是单进程内存 manifest，不具备崩溃持久化能力，也没有并发提交保护。下一章之后继续扩展时，读者就能清楚地区分：这里的规则是提交语义，后面需要补的是持久化、锁保护、并发安全和分布式复制，而不是重新发明提交事实。

七、每一步都要有验收 代码渐进的价值在于每一步都能验收。解析一行时，验收正常行、缺字段、空事件名、超长事件名。Reference 阶段，验收 accepted 和 rejected 计数。RecordId 阶段，验收错误能定位到 split 和 line。Batch 阶段，验收 hot events 和 cold errors 数量。Partial 阶段，验收合并结果等于 reference。Manifest 阶段，验收重复 attempt 不会重复计数。验收不是附录，它是下一步扩展的地基。

核心思想

从一行日志到可恢复作业，中间每一步都只解决一个新问题：字段解析、正确性对照、错误定位、批次生命周期、减少共享、提交事实。以后改后续章节，也按这个节奏写：先小问题，再代码，再暴露限制，再扩展。

1.14 三个实际案例：问题怎样推动代码继续长大

上一节给出了代码演进的骨架，但真正学习系统时，还要看到具体输入怎样把问题逼出来。本节补三组实际案例。每组都先给一个很小的现象，再说明朴素实现为什么不够，最后给出下一步代码形态和验收方式。这样读者不会觉得 RecordId、线程本地 partial 和 manifest 是为了“架构好看”才出现，而会看到它们各自解决了哪类真实问题。

一、案例一：一条坏日志为什么逼出 RecordId 假设输入只有六行，其中第五行缺少 event_name：

```

10:00 u001 login 1
10:01 u002 view 1
10:02 u001 click 1
10:03 u003 logout 1
10:04 u004
10:05 u001 login 1

```

最早的 reference 只会把 rejected_records 加一。这个结果在六行输入上还能人工定位；一旦输入变成数千万行，错误报告只有“拒绝一行”就没有工程价值。运维想知道坏行在哪个文件、哪个 split、哪个 offset；开发想知道是字段缺失、事件名过长，还是数值格式错误；恢复程序想知道这个错误是否影响整个 split。于是错误计数不够，必须把错误和数据身份绑定。

这一步的代码变化不大，但思想很关键。解析函数不能只返回 bool，而要返回带身份的结果。错误路径也不能只写日志，因为日志只是旁路文本，后续程序无法可靠查询。更稳的做法是把错误变成结构化对象，和 RecordId 一起进入 cold path。

Listing 1.13 把错误行变成结构化诊断

```

1 struct ParseDiagnostic {
2     RecordId id;

```



```

3   ParseError error = ParseError::none;
4   std::uint32_t column = 0;
5   };
6
7   struct CheckedRecord {
8       RecordId id;
9       std::string_view event_name;
10      ParseDiagnostic diagnostic;
11  };
12
13  CheckedRecord parse_checked(RecordId id, std::string_view line)
14  {
15      const EventField field = parse_event_field(line);
16      if (!field.ok) {
17          return {
18              id,
19              {},
20              {id, field.error, 0}
21          };
22      }
23
24      return {
25          id,
26          field.event_name,
27          {id, ParseError::none, 0}
28      };
29  }

```

这里没有追求 column 的完整实现，因为第一章重点不是写一个完美 parser，而是展示诊断对象的位置。等后续 parser 更复杂时，column 可以来自扫描状态；错误也可以带上字段名、原始片段和恢复建议。关键是接口已经承认：错误是系统语义的一部分，不是打印到 stderr 就结束。

这个案例的验收很具体。给定上面的六行输入，reference 输出应包含 **login=2**、**view=1**、**click=1**、**logout=1**，accepted 为五，rejected 为一；诊断中应有一条记录，指向 input id、split id 和第五行的 line index。若把输入切成两个 split，诊断仍应指向同一条逻辑记录，而不是只写“第二个 worker 出错”。这就说明 RecordId 解决的是错误定位和并发后可追踪性。

二、案例二：热点事件为什么逼出线程本地 partial 再看第二组输入。假设一千万行日志中，九百万行都是 **view**，其余事件分散。朴素多线程方案可能让每个 worker 直接更新全局 map：

Listing 1.14 危险版本：所有 worker 共享一个全局 map

```

1  #include <mutex>
2
3  struct SharedCounts {
4      std::mutex mutex;
5      EventCountMap counts;
6  };
7
8  void update_shared_counts(
9      SharedCounts& shared,
10     std::string_view event_name)
11  {
12     const std::lock_guard<std::mutex> lock(shared.mutex);

```

```

13     ++shared.counts[std::string(event_name)];
14 }

```

这段代码语义上很容易理解，也许在两千行输入上没有明显问题。但热点输入会把它的结构缺陷放大：每条记录都要进入同一把锁，每次更新 **view** 都会修改同一个计数节点，多个核心在同一小块状态上排队。此时增加线程不一定提高吞吐，反而可能增加上下文切换、锁等待和 cache line 迁移。问题不是“mutex 这个 API 不好”，而是“共享发生得太频繁、太早、太集中”。

下一步不是立刻改成无锁结构。若把全局锁换成全局 atomic counter，只能解决一个特殊 key 的计数，无法处理任意事件名；若把哈希表每个桶都加锁，热点 key 仍然集中在同一个桶附近；若一上来写复杂无锁哈希表，读者会被实现细节淹没。更清晰的第一步是线程本地 partial：让 worker 在自己的局部 map 中计数，处理完 split 后再合并。

Listing 1.15 把共享更新推迟到合并阶段

```

1 PartialCount build_partial_from_events(
2     std::uint32_t split_id,
3     std::span<const EventView> events)
4 {
5     PartialCount partial;
6     partial.split_id = split_id;
7
8     for (const EventView& event : events) {
9         ++partial.counts[std::string(event.event_name)];
10    }
11
12    return partial;
13 }
14
15 void merge_one_partial(
16     EventCountMap& merged,
17     const PartialCount& partial)
18 {
19     for (const auto& [event_name, count] : partial.counts) {
20         merged[event_name] += count;
21     }
22 }

```

这个版本没有消灭合并成本，只是把共享从“每条记录一次”改成“每个 partial 的每个 key 一次”。如果一个 split 内有一百万条 **view**，局部 map 对 **view** 的更新仍然很多，但它发生在单个 worker 的本地状态里，不需要跨线程协调。合并时只把 **view** 的局部总数加到全局结果一次。对于热点 key，这个结构变化通常比微调锁 API 更有价值。

这个案例的验收也要清楚。先用串行 reference 计算结果，再用共享全局 map 版本和线程本地 partial 版本分别计算，三者输出必须一致。然后记录每个版本的任务时间和等待时间：共享版本如果锁等待占比高，说明问题确实在共享更新；partial 版本如果合并阶段变成主成本，说明瓶颈已经迁移。教学重点不是宣布 partial 永远最快，而是让读者看到共享边界怎样改变瓶颈位置。

三、案例三：一次写后崩溃为什么逼出 manifest 第三组案例是失败恢复。假设 split 3 的 worker 已经完成 partial，并写出临时文件 **tmp-s3-a1.block**，但还没来得及告诉 driver 成功，进程就崩溃了。Driver 重启后没有看到 split 3 的提交事实，于是启动 attempt 2，写出 **tmp-s3-a2.block** 并成功提交。现在目录里有两个文件。如果 reduce 只是扫描 ***.block**，就会把 split 3 统计两次。

朴素实现常写成这样：

Listing 1.16 危险版本：reduce 扫描所有 block

```

1 std::vector<ShuffleBlock> scan_all_blocks(std::string_view directory);
2
3 EventCountMap reduce_by_scanning_directory(std::string_view directory)
4 {
5     EventCountMap result;
6     const std::vector<ShuffleBlock> blocks =
7         scan_all_blocks(directory);
8
9     for (const ShuffleBlock& block : blocks) {
10         merge_block_into_result(result, block);
11     }
12
13     return result;
14 }

```

这段代码的问题在于，它把“文件存在”误认为“结果有效”。文件系统目录不知道哪个 attempt 被接受，也不知道某个 block 是临时输出、旧输出还是诊断残留。要修复这个问题，不是在文件名里多加几个 if，而是改变 reduce 的输入来源：reduce 只能从 manifest 读取已提交条目，再根据条目找到对应 block。

Listing 1.17 安全版本：reduce 只读 manifest 接受的 block

```

1 std::vector<ShuffleBlock> load_committed_blocks(
2     const Manifest& manifest);
3
4 EventCountMap reduce_from_manifest(const Manifest& manifest)
5 {
6     EventCountMap result;
7     const std::vector<ShuffleBlock> blocks =
8         load_committed_blocks(manifest);
9
10    for (const ShuffleBlock& block : blocks) {
11        merge_block_into_result(result, block);
12    }
13
14    return result;
15 }

```

这一步解决的是外部可见性。Worker 产生 block，只说明它完成了一次尝试；manifest 接受 block，才说明这个 block 成为作业结果的一部分。Attempt 1 留下的临时文件可以被清理，也可以保留用于诊断，但不能被 reduce 当成输入。这样的设计让恢复规则简单得多：先读取 manifest，确定已经提交的 split；再决定哪些 split 需要重试；最后清理不在 manifest 中的临时文件。

这个案例的验收必须包含故障注入。先运行 attempt 1，让它写出临时 block 后模拟崩溃；再运行 attempt 2 并提交；最后检查目录中确实可以存在两个 block，但 manifest 只有一个条目，reduce 输出与串行 reference 完全一致。若把 reduce 改回扫描目录，测试应当失败。这种测试比口头说“我们支持恢复”更有价值。

四、三个案例串起来看 这三个案例看似分属不同主题：坏日志是解析问题，热点事件是并发性能问题，写后崩溃是恢复问题。它们共同说明同一件事：系统结构不是凭空设计出来的，而是被具体输入、具体等待和具体失败推出来的。坏日志推动结构化诊断和 RecordId；热点事件推动减少共享和 partial；写后崩溃推动提交点和 manifest。每个概念都对应一个小反例。

后续章节也按这个方法写。讲缓存时，不是先列 L1、L2、L3，而是拿连续数组和随机节点比较；

讲 TLB 时，不是先背页表层级，而是拿大工作集和小对象分配比较；讲锁时，不是先列 mutex 种类，而是拿共享计数器的等待时间比较；讲无锁时，不是先贴复杂算法，而是从“锁保护的不变量能否拆成单写者状态”开始；讲分布式时，不是先列 CAP 和共识名词，而是从迟到响应和重复提交开始。这样的写法才有学习价值。

五、代码扩展要保留旧版本 教学项目中，每次扩展都应保留旧版本作为对照。保留单行 parser，是为了测试边界字段；保留串行 reference，是为了校验并行结果；保留共享 map 版本，是为了展示锁等待；保留扫描目录的失败测试，是为了证明 manifest 的必要性。旧版本不是垃圾代码，而是反例和基线。没有旧版本，读者只看到最终结构，看不到为什么需要它。

工程仓库可以把这些版本放在不同实验或测试里，而不是在生产路径中到处保留分支。教材正文则要明确告诉读者：最终写法不是天生正确，它是在一个个反例中长出来的。这个习惯会影响整本书的质量，也会影响读者以后写系统的方式。

六、把案例变成可运行实验 案例只有文字还不够，必须能运行。第一章的实验组织可以很朴素：一个目录保存输入样例，一个 reference 程序生成黄金输出，一个实验程序生成候选输出，一个比较程序检查结果和诊断。不要一开始就引入复杂测试框架；先让读者看到最小闭环。闭环跑通后，再把同一批输入交给多线程版本、批处理版本和 manifest 版本。这样每个新版本都不是“重新写一个程序”，而是在同一条验证链上增加能力。

最小实验可以按三类输入组织。第一类是正常输入，用来证明基础路径正确。第二类是边界输入，用来验证空行、缺字段、超长事件名、最后一行没有换行符。第三类是故障输入或故障脚本，用来触发写后崩溃、重复 attempt、迟到响应和 orphan block。每类输入都要有说明，不是简单把文件堆在目录里。读者看到一个失败时，应当知道它是在验证哪条系统边界。

```
experiments/ch01/
  inputs/normal.log
  inputs/dirty.log
  inputs/hot-key.log
  inputs/retry-split-3.log
  expected/reference-counts.txt
  expected/reference-errors.txt
  run-reference
  run-partial
  run-manifest-failure
```

这张目录图的意义在于把“读书”变成“复盘”。正常输入对应 reference，dirty 输入对应 RecordId 和诊断，hot-key 输入对应共享 map 与 partial 对比，retry 输入对应 manifest。以后第二章讲流水线，可以继续用 hot-key 或普通输入观察单核执行；第三章讲缓存，可以继续用同一批 parsed batch 观察布局；第十章讲锁，可以继续用共享 map 反例；第十五章讲 I/O，可以继续用 manifest 和临时 block。实验材料复用，学习主线就不会断。

七、比较程序比打印输出更重要 很多初学者写实验，只看程序打印结果。这样很容易漏掉错误。更好的做法是让比较程序明确检查三个东西：计数是否一致，错误诊断是否一致，提交事实是否一致。计数一致证明最终结果对；错误诊断一致证明坏输入没有被悄悄吞掉；提交事实一致证明恢复路径没有把临时状态当成有效状态。只比较最终计数，有时会掩盖严重 bug。例如某个 split 被重复计数，另一个 split 又丢失，最终总数可能偶然相同，但 manifest 和 RecordId 会暴露问题。

比较程序的输出也要面向学习。不要只写 **failed**，而要写出哪一个 key 计数不同，哪一个 record id 的错误分类不同，哪一个 split 被重复提交。错误信息越具体，读者越容易把现象和章节概念连起来。教材里的实验不是竞赛判题器，而是调试材料。它应该帮助读者回答“为什么失败”，而不只是宣布“失败了”。

Listing 1.18 比较计数结果的最小骨架

```
1 struct CompareMismatch {
2     std::string key;
```



```

3     std::uint64_t expected = 0;
4     std::uint64_t actual = 0;
5 };
6
7 std::vector<CompareMismatch> compare_counts(
8     const EventCountMap& expected,
9     const EventCountMap& actual)
10 {
11     std::vector<CompareMismatch> mismatches;
12
13     for (const auto& [key, expected_count] : expected) {
14         const auto found = actual.find(key);
15         const std::uint64_t actual_count =
16             found == actual.end() ? 0 : found->second;
17         if (actual_count != expected_count) {
18             mismatches.push_back({
19                 key,
20                 expected_count,
21                 actual_count
22             });
23         }
24     }
25
26     for (const auto& [key, actual_count] : actual) {
27         if (!expected.contains(key)) {
28             mismatches.push_back({key, 0, actual_count});
29         }
30     }
31
32     return mismatches;
33 }

```

这段代码仍然只是骨架。真实比较还要检查错误诊断、accepted records、rejected records、manifest 条目和 checksum。这里先展示计数比较，是为了让读者理解 reference 的使用方式：reference 不是拿来人工看的，而是进入自动比较链。后续任何优化只要改变输出，就必须被比较程序抓住。

八、从小实验扩展到压力实验 小实验负责证明语义，压力实验负责暴露成本。两者不能互相替代。三十行 dirty 输入很适合检查 RecordId，却无法说明哈希表布局；一亿行 hot-key 输入很适合暴露锁等待，却不适合人工审查每条错误。第一章要求读者把实验分层：先用小输入保证每条边界正确，再用大输入观察瓶颈迁移。若语义小实验不过，不允许进入压力实验；若压力实验没有 reference 校验，性能数据也不可信。

压力实验也要一步一步扩展。先跑串行 reference，得到基线吞吐和输出。再跑共享 map 多线程版本，记录线程数增加时吞吐是否线性增长、锁等待是否出现。再跑线程本地 partial，观察共享等待是否下降、合并阶段是否上升。再加入 manifest，观察写出和提交开销。每一步只改一个主因素，报告才有解释力。一次性加入所有优化，只能得到一个数字，不能得到知识。

九、实验报告要写出限制 实验报告最后必须写限制。比如输入在 page cache 中，不能代表冷盘读取；机器只有单 socket，不能验证 NUMA；没有 PMU 权限，不能给出 cache miss 事件；manifest 只是内存 vector，不能证明崩溃后持久化；网络用本机队列模拟，不能代表真实 RTT。写限制不是认输，而是说明结论的适用范围。一个诚实的限制，比一个说满的结论更接近工程事实。

这个习惯也会影响后续章节。讲缓存时要说明输入规模是否超过 LLC；讲 SIMD 时要说明编译器参数和目标指令集；讲锁时要说明线程是否绑定核心；讲分布式时要说明故障模型。第一章把报告限制写清楚，后面每章就有统一标准。

十、从案例到全书写作规则 本章后续打磨全书时，每章都应该能找到类似三组材料：一个小输入或小代码暴露语义问题，一个中等规模实验暴露性能问题，一个失败或边界案例暴露恢复问题。不是每章都必须三者完全一样，但至少要有“问题、反例、代码、验收”这条链。没有代码的概念要补代码，没有反例的结论要补反例，没有验收的优化要补验收，没有限制的报告要补限制。

这也是第一章现在和旧稿最大的区别。旧稿里许多内容是正确的，但读者会感觉它们并列摆放；新写法把它们放回同一个学习过程：先解析一行，再统计多行，再定位错误，再批处理，再减少共享，再提交结果，再恢复失败。后面修改第二章、第三章和分布式章节时，也按这个过程处理。这样整本书才会像经典教材，而不是像资料库。

这个规则还会约束代码篇幅。代码多不等于质量高，代码少也不等于讲得清。真正有价值的代码，应当让读者看到一个边界怎样被对象和函数承接。比如 `parse_event_field` 承接字段合同，`run_reference_checked` 承接正确性对照，`parse_checked` 承接错误定位，`parse_split_lines` 承接批次和生命周期，`build_partial_from_events` 承接减少共享，`commit_block` 承接提交事实。若一段代码不能说明自己承接了哪条边界，就应该删掉或改写；若一段文字讲了很重要的边界，却没有任何代码、实验或反例承接，就应该补上。

后续章节也要保持这种“先窄后宽”的节奏。讲核心流水线时，先从一个整数累加循环开始，不要一上来铺满取指、译码、重命名、发射、执行、提交等名词；等读者看到单累加器受依赖链限制，再引出多累加器和乱序窗口。讲缓存时，先从两个同为 $O(n)$ 的循环开始，一个连续访问，一个随机追指针；等读者看到复杂度相同但机器成本不同，再引出 cache line、预取和 TLB。讲锁时，先从共享计数器的等待时间开始；等读者看到所有线程排队，再引出临界区、不变量、条件变量和背压。讲分布式时，先从 split 重试导致重复 partial 开始；等读者看到扫描目录会算错，再引出 manifest、attempt、幂等和提交协议。

第一章还要提醒读者：最终版本不是学习的起点。很多工业系统的源码一打开就是成熟结构，里面已经有线程池、任务图、缓冲区、状态机、重试、压缩、校验和指标。直接阅读这种源码，初学者很容易只记住对象名，却不知道它们为什么存在。本册的写法要反过来：先让读者经历缺失这些对象时会发生什么，再给出对象。读者以后读 Linux、数据库、计算框架或推理引擎源码时，就能把复杂结构还原成问题链，而不是把源码当成不可理解的工程堆叠。

这种写法对作者也有约束。每章写完后，都要自查四个问题。第一，本章第一个案例是否足够小，小到读者能手工推导。第二，本章扩展是否每一步只引入一个主概念。第三，本章代码是否能被 reference 或实验验收。第四，本章是否写出了限制，说明哪些结论还不能外推。若四个问题回答不上来，就说明章节还在堆材料，需要继续重写。第一章就是这个检查表的样板。

十一、最小闭环的日常运行顺序 真正写项目时，不要等到全章代码都写完再验证。每天最小闭环可以很固定：先跑单行 parser 的边界输入，再跑串行 reference，再跑 batch 解析，再跑 partial 合并，再跑 manifest 故障注入，最后跑一次小压力输入。前四步证明语义没有坏，第五步证明恢复边界没有坏，最后一步只观察性能趋势。若前面任意一步失败，就不要看压力数据，因为那时程序可能只是更快地算错。

这个顺序看起来保守，其实能节省大量调试时间。比如某次修改 parser 后，hot-key 压力测试吞吐下降。若没有最小闭环，作者可能直接怀疑锁、缓存或线程调度；但如果单行 parser 边界测试已经失败，就说明问题还在语义层。再比如 manifest 故障注入失败，而普通 reference 仍然通过，说明计算本身没错，错误在提交事实。分层验证能防止调试方向乱跳。

读者以后阅读大型源码，也可以用同样方法反推。看到一个复杂模块时，先找它的最小输入是什么，reference 或基线在哪里，错误如何定位，批次生命周期由谁保证，局部结果怎样合并，提交事实在哪里记录。若源码没有这些边界，就要格外警惕；若源码有这些边界，就能顺着边界阅读，而不是按文件名乱翻。这种阅读方式会贯穿后面的 Linux 数据结构、线程运行时、I/O 管线和分布式章节。

十二、扩写也必须服务学习路径 本书后面会继续扩充，但扩充不能变成把材料往章节里塞。每新增一段，都要能回答它服务哪一步学习路径：是让小问题更具体，还是让反例更真实，还是让代码边界更清楚，还是让实验验收更可靠。若一段文字只是换个说法重复概念，它会增加字数，却降低密度；若一段代码只是展示最终答案，却没有说明从哪里推出来，它会增加篇幅，却降低理解。第一章宁可围绕日志统计反复深入，也不要跳到十个不相干的例子上。后面每章也一样：深度来自把一个问题拆穿，而不是把许多名词排满。

因此，第一章的质量标准不是“介绍了多少概念”，而是读者能否沿着同一条线复述出来：一行

日志怎样被解析，坏行怎样定位，很多行怎样形成 reference，批次怎样承载生命周期，热点 key 怎样推动局部聚合，临时输出怎样通过 manifest 变成有效结果，失败后怎样恢复，实验怎样证明这些边界没有坏。能复述这条线，才说明章节真的讲清楚了。

1.15 从单机原则到分布式计算

第二册后面会讲分布式计算，但第一章不能突然把单机和分布式切成两门课。更好的方式是从日志统计的单机 pipeline 自然推出分布式版本。单机里，多个 worker 线程处理多个 split；分布式里，多个 worker 进程或节点处理多个 split。单机里，线程本地 partial 最后合并；分布式里，map worker 产生 shuffle block，reduce worker 按 key 合并。单机里，manifest 防止重复 attempt；分布式里，manifest 或提交协议防止重复输出。尺度变大了，但问题类型没有变。

区别在于成本和失败模式。线程之间通信通常是共享内存，节点之间通信要经过网络。线程崩溃通常会带着进程一起失败，节点失败可能只影响一部分任务。单机时间大致由同一台机器观察，分布式时间没有全局可靠顺序。共享内存里的指针不能跨节点，必须序列化成消息。正因为这些差异，分布式系统需要更严格的身份、重试、超时和提交协议。

一、分片来自并行，也来自恢复 把输入切成 split，一开始是为了并行：多个 worker 可以同时处理不同范围。后来它也服务恢复：一个 split 失败，只需要重做这个 split，而不是重做整个输入。分布式系统的 partition 也是同样道理。按输入范围分片可以让读取并行，按 key 分区可以让 reduce 并行，按时间窗口分片可以让恢复边界清楚。分片不是单纯为了负载均衡，它同时决定数据本地性、失败粒度和提交粒度。

分片过小，调度和 manifest 条目过多，固定成本上升；分片过大，负载不均和失败重做成本上升。若某个事件名占据大部分记录，按输入范围平均切分也可能在 reduce 阶段形成热点 key。于是分布式章节要讲的负载均衡，并不是凭空出现，而是从第一章的 split 粒度和 key 分布继续推导。

二、shuffle 是跨边界的数据重排 单机线程本地 partial 合并时，数据还在同一进程或同一台机器里。分布式 map-reduce 中，map worker 需要把相同 key 的 partial 发送到同一个 reduce 分区，这就是 shuffle。Shuffle 的本质是为了让“同一个 key 的所有贡献”在某个地方相遇。这个需求来自聚合语义，不来自框架名字。

Shuffle 之所以昂贵，是因为它把数据从本地内存和本地磁盘推向网络、远端内存和远端磁盘。第一章的数据移动原则在这里被放大：能在 map 端先合并，就不要把每条记录都发出去；能压缩，就用 CPU 换网络字节；能按 key 分区稳定输出，就让 reduce 少做随机查找；能把热点 key 单独处理，就不要让一个 reduce 被拖垮。后面讲分布式时，这些策略都应从 shuffle 的成本中推出来。

三、慢节点把平均速度变成尾部问题 单机多线程里，一个 worker 慢会让最终合并等待它。分布式里，一个节点慢会让整个 stage 等它，这个慢任务常被称为 straggler。它可能因为硬件差异、远端读取、数据倾斜、重试、后台负载或网络拥塞产生。若报告只看平均任务时间，就看不到最后几个任务拖住全局进度。执行等待图必须从平均时间扩展到尾部时间。

日志统计的例子很直观。若某个 split 包含异常长的行，parser 慢；若某个 split 包含热点 key，reduce 慢；若某个 worker 所在节点磁盘慢，写出慢；若某个 block 需要跨机拉取，shuffle 慢。分布式优化不是简单增加节点，而是让慢任务可见、可重试、可拆分，并确保重试不会破坏提交语义。

四、一致性从提交点继续长出来 第一章的 manifest 是单机提交事实。到了分布式环境，manifest 本身可能需要复制，driver 可能失败，多个节点可能同时尝试提交同一个 split。此时就会引出更强的复制、租约、epoch、共识或事务日志。注意，这些概念不是突然从分布式教材里掉下来，而是从“谁有权宣布结果有效”这个问题继续长出来。

如果只有一个可靠 driver，manifest 可以是本地文件。若 driver 可能失败，就要把 manifest 放到可靠存储或复制服务里。若多个 driver 可能同时存在，就要有 leader 或 epoch 防止旧 leader 继续提交。若多个副本要对提交顺序达成一致，就会进入共识协议。第一章不展开这些协议细节，但要让读者看到它们的源头：提交事实必须唯一、可恢复、可被下游信任。

五、单机实验也能训练分布式思想 即使没有多机环境，也可以在本机模拟分布式的关键问题。用多个进程或线程模拟 worker，用消息队列模拟 RPC，用随机延迟模拟网络抖动，用重复消息模拟重试，用临时目录模拟远端 block，用 manifest 模拟提交事实。这个模拟不能证明真实性能，但可以验证语义：重复请求不会重复计数，迟到响应不会覆盖新提交，失败 attempt 不会被 reduce 读取。

这很适合本册读者。手机 Termux 或普通 Linux 机器未必能提供真实集群，但完全可以训练分

布式系统最重要的语义边界。等以后进入真实集群，网络成本、调度系统和存储系统会更复杂，但身份、提交、幂等、重试和恢复这些原则不会变。

核心理想

分布式不是另一套孤立概念，而是把单机的切分、等待、数据移动和提交事实放大到网络和故障更多的环境中。先把单机边界讲清，分布式才不会变成术语堆。

1.16 贯穿项目：Compute Systems Engine

本册需要一个贯穿全书的大项目，否则章节会散成很多局部技巧。这个项目命名为 Compute Systems Engine。它不是要做商业级大数据框架，而是用一个足够小、可测试、可测量、可失败注入的工程，把现代计算系统的核心问题串起来。第一章只定义项目的骨架，后面每章都在这个骨架上增加一个可解释的能力。

项目的输入是一批日志 split，输出是按事件名聚合的结果。第一阶段实现串行 reference 和输入合同。第二阶段实现批量解析、冷热分离和局部聚合，用来连接缓存、TLB 和数据布局。第三阶段实现有界队列、线程池和线程本地 partial，用来连接调度、锁和背压。第四阶段实现异步写出、manifest 和恢复，用来连接 I/O、提交点和故障处理。第五阶段实现本机分布式模拟：多个 worker 进程或线程按分片处理，shuffle 按 key 重分布，reduce 读取 manifest，故障注入验证幂等。

一、项目不是功能堆砌 每加一个模块，都必须回答三件事。语义上，它改变了什么结果合同；性能上，它试图移动哪个瓶颈；恢复上，它新增了哪个失败路径。若一个模块只能让架构图更复杂，却说不清这三件事，就不应该加入教材。比如线程池不是为了“看起来高级”，而是为了控制执行资源和显式化等待；manifest 不是为了“模拟大数据系统”，而是为了防止重复 attempt 污染结果；分片聚合不是为了“多写一层”，而是为了减少共享写和改善局部性。

二、项目的最小接口 第一章后的项目可以只有几个核心对象：InputSplit 描述输入范围，RecordId 描述记录身份，ParsedBatch 描述解析结果，PartialCount 描述局部聚合，ShuffleBlock 描述临时输出，ManifestEntry 描述提交事实，TraceEvent 描述状态变化。对象数量不多，但每个对象都有明确责任。

后面章节可以替换实现，但不轻易改变责任边界。比如聚合容器可以从 std::unordered_map 换成开放寻址哈希或排序归约，但 PartialCount 的语义仍然是某个输入范围的局部计数。队列可以从 mutex 版本换成无锁 ring buffer，但队列合同仍然包含容量、关闭和等待谓词。这样全书既能深入实现，又不会失去主线。

三、项目要能故意失败 只会跑 happy path 的项目不能教授系统。Compute Systems Engine 从第一章就要支持故障注入：解析到某个 record id 时返回错误，worker 写出 block 后崩溃，writer 短写，manifest commit 前后分别失败，重复提交同一个 attempt，迟到响应覆盖新响应。每个失败都必须有 reference 或恢复验证。这样后面讲并发和分布式时，读者不会只看到理论概念，而能看到错误怎样被接口吸收。

四、项目要能测量 每个阶段都要输出最小指标。Reader 输出字节数、split 数和队列水位；parser 输出记录数、错误数和处理时间；worker 输出运行时间和等待时间；writer 输出 block 数、写入字节和失败次数；manifest 输出提交条目和重复 attempt；reducer 输出最终 checksum。指标不需要一开始就完美，但要能支撑下一步问题。后面加入 PMU 和 trace 时，这些基础身份会让指标可以关联。

1.17 本章实验

第一章的实验不追求性能极限，而是把系统边界跑通。建议准备一个包含三十到一百行的小输入，再准备几个破坏朴素假设的输入：最后一行没有换行符，某行缺少事件字段，某个事件名非常长，同一个 split 被重复执行，worker 在写出临时 block 后模拟崩溃。实验规模故意小，是为了让读者能手工检查每条记录的命运。

一、实验一：写出串行 reference 先实现 run_reference，输入是内存中的行数组，输出是事件计数 map 和错误统计。要求把错误策略写清楚，并为每条错误记录保存 RecordId。验收方式不是看程序打印了什么，而是用固定输入得到固定输出，并能解释每个错误行为为什么被归类。

这个实验训练的是语义边界。不要在这里急着优化字符串分配，也不要急着并行。若 reference 还在变化，后续所有性能实验都会失去锚点。

二、实验二：引入 split 和 record id 把输入切成两个或多个 split，每个 split 记录起止 offset 或行范围。要求每条记录获得稳定 **RecordId**，并且无论 split 处理顺序如何，最终 reference 输出相同。若某条记录跨 split，要么在合同中禁止这种切分，要么由 reader 修正到行边界。实验报告必须说明你选择了哪一种合同。

这个实验训练的是数据身份。只要身份稳定，执行顺序就可以改变；身份不稳定，后续线程池和重试都会失控。

三、实验三：线程本地 partial 实现多个 split 的局部聚合，每个 split 生成 **PartialCount**，最后合并。要求最终结果与串行 reference 相同。先不使用共享全局 map，避免一开始就把锁竞争引入主路径。报告要记录 split 数量、每个 partial 的记录数、每个 partial 的 key 数量，以及合并后的 checksum。

这个实验训练的是减少共享。线程本地 partial 不是最终答案，但它让读者看到并行化的基本形状：先本地计算，再合并结果。

四、实验四：临时 block 和 manifest 把每个 partial 写成临时 block，再通过 manifest 提交。Reduce 只能读取 manifest，不允许扫描目录中所有临时文件。故意模拟一次 worker 写出临时文件后崩溃，再重试同一个 split。验收条件是目录里可以有多个临时文件，但 manifest 只接受一个 attempt，最终结果不重复计数。

这个实验训练的是提交点。它比性能数字更重要，因为没有提交点，分布式和异步 I/O 后面一定会错。

五、实验五：写一页系统报告 报告只需要一页，但必须包含输入合同、reference 输出摘要、数据身份图、执行等待图、成本账本、故障注入结果和未验证边界。未验证边界可以写得很朴素，例如没有真实磁盘故障、没有多 socket NUMA、没有 PMU 权限、没有真实网络。重点是养成诚实复盘的习惯。

习题与作业

习题一：为日志统计定义三组输入：正常输入、边界输入和破坏朴素假设的输入。说明每组输入验证哪条语义。

习题二：把共享全局 map 版本和线程本地 partial 版本画成执行等待图。指出两者主要等待点有什么不同。

习题三：构造一个重复 attempt 导致重复计数的反例，再用 manifest 修复它。要求写出每个 attempt 的 **AttemptId**、临时路径和最终 manifest 条目。

习题四：为项目增加最小 trace。每个任务至少记录 created、queued、started、finished、committed 五个状态，并说明这些状态怎样帮助定位等待。

1.18 后续章节的统一规则

读者学习这本书，不是为了背一个又一个名词，而是为了形成能处理真实计算系统的能力。因此从第一章开始，后续章节都按同一套规则打磨。

一、每章先从具体问题开始 讲流水线，就从一个循环为什么不能每周完成一次开始；讲缓存，就从连续数组和随机指针为什么同为线性复杂度却速度不同开始；讲锁，就从共享计数器为什么线程越多越慢开始；讲原子，就从一个无锁计数器为什么仍然可能成为热点开始；讲分布式，就从一个 split 重试为什么会重复提交开始。概念必须从问题中出现，而不是先列概念再找例子。

二、每个概念都要说明为什么需要 引入 cache line，要说明硬件按块搬运数据；引入 TLB，要说明虚拟地址需要转换；引入 acquire 和 release，要说明线程之间的发布和观察需要顺序；引入 backpressure，要说明生产者可能快于消费者；引入 consensus，要说明多个副本在故障下仍要同意一个提交事实。只给定义不解释需求，就是低质量写法。

三、代码必须服务原理 代码不是答案展示，而是推导材料。每段代码要说明它暴露了什么问题，解决了什么边界，仍然留下什么限制。示例使用位宽清晰的整数类型，例如 **std::uint32_t** 和 **std::uint64_t**。如果代码只是为了堆长度，应该删掉；如果代码能把抽象概念落到对象、状态和不变量上，就应该写完整、排版清楚。

四、案例必须有反例和验收 只给正确版本不够。好的教材要先给朴素版本，再给它失败的输入或性能现象，然后分析结构，最后给改进版本和验收证据。比如 manifest 不是直接端出来，而是

从重复 `partial` 的反例中推导；有界队列不是直接端出来，而是从无界队列内存膨胀和下游阻塞中推导；线程本地聚合不是直接端出来，而是从全局锁竞争中推导。

五、章节之间要串起来 第一部分讲硬件，不是为了停在硬件；它要解释第二部分的数据布局 and 测量。第二部分讲单机高性能，不是为了停在 `microbenchmark`；它要支撑第三部分的多线程设计。第三部分讲同步，不是为了炫耀锁和原子；它要支撑第四部分的任务运行时。第四部分讲并行算法和 I/O，不是为了停在单机；它要自然过渡到第五部分的分布式计算。每章都要告诉读者：上一章解决了什么，本章新增了什么，下一章为什么必须继续。

核心理想

本册后续修改以第一章为样板：问题驱动、反例驱动、原理先行、代码落地、实验验收、边界诚实。没有这六点，内容再多也只是材料堆积。

1.19 本章小结

本章只讨论了一个日志统计作业，却已经建立了第二册的主线。我们没有从概念列表出发，而是从一个朴素 `reference` 出发，依次看到正确性、数据路径、硬件成本、执行协作、提交事实、证据链和贯穿项目。每个概念都不是孤立出现的：`RecordId` 来自追踪记录的需要，冷热分离来自数据搬运成本，线程本地 `partial` 来自减少共享写，有界队列来自生产者和消费者速度不匹配，`manifest` 来自重复 `attempt` 和恢复，三张图来自系统复盘的需要。

后面第二章会把视角缩进 CPU 核心内部，讨论流水线、乱序执行、分支预测和依赖链。读者不应把它看成脱离项目的硬件知识。相反，它要回答本章留下的第一个性能问题：同样处理一批记录，为什么某些循环能被核心高吞吐执行，某些循环却被依赖、分支或取数等待卡住。只要记住这个问题，硬件章节就不会变成术语背诵，而会成为解释系统性能的第一块地基。

第 2 章

核心流水线、乱序执行与分支预测

先看一个线上现象：日志解析器在固定格式输入上很快，换成用户自定义字段后没有改一行代码，吞吐却下降，尾延迟也明显抖动。源码层面仍然只是一个循环：读一个字节，判断分隔符、数字、引号、转义或非法字符，更新状态。若只看 C++，很容易把问题归因成“字符串处理慢”；但把输入分布、反汇编和计数器放在一起，就会看到另一条链路：随机分支让预测失败变多，错误路径混入热循环让前端供给变差，状态变量形成循环携带依赖，某些字段统计又把加法串成一条长链。

因此本章不从流水线部件名开始，而从这个热循环为什么变慢开始。流水线回答“为什么一次猜错分支会丢掉已经取入的工作”，乱序执行回答“为什么多累加器能拆开一条依赖链”，前端和执行端口回答“为什么源码行数相同，机器看到的工作却不同”。这些硬件概念只有能解释一段具体代码的性能异常，才真正有用。

2.1 从流水线到乱序执行

流水线把取指、译码、执行、访存和提交分成阶段。理想情况下，每个周期都有新指令进入，每个阶段都在工作。但真实程序有数据依赖、控制依赖和资源冲突。后一条指令需要前一条指令结果时，执行端口即使空闲也不能使用；分支目标未知时，前端可能取错指令；访存 miss 时，后端会等待数据。

流水线的关键不是“越深越快”，而是吞吐和延迟的权衡。深流水线可以提高频率，却放大分支预测失败的代价。并发程序中的短临界区也会受流水线影响：一个看似只有几条指令的锁操作，背后可能包含原子读改写、cache line 独占获取和内存序约束。

前端：取指、译码和微操作 核心前端负责把指令流喂给后端。它要从 instruction cache 取指，预测下一条指令地址，译码复杂指令，把它们转成内部微操作，并放入队列。前端跟不上时，后端即使有空闲执行端口也没有工作可做。大函数、过多分支、间接跳转、指令 cache miss 和译码压力都可能让前端成为瓶颈。

x86-64 指令长度可变，译码比定长指令集复杂。现代核心通常会使用 micro-op cache 或类似结构缓存译码结果，让热循环不用每次重新译码。这个机制解释了为什么热循环体大小、函数内联和代码布局会影响性能。内联可以减少调用开销，但过度内联会膨胀代码，伤害 instruction cache 和前端供给。

第一册已经教读者读汇编。第二册读汇编时要多看一个维度：这段机器码对前端是否友好。短小热循环、稳定分支、连续代码布局和可预测调用目标，通常更容易让前端持续供给后端。反过来，大量模板实例化、过度内联、异常路径混在热路径、虚调用目标多变、解释器式大 switch，都可能让前端成本超过源码直觉。

深入理解

前端瓶颈的一个典型现象是：数据已经在缓存中，算术也不复杂，但 IPC 仍然低，并且改变数据布局帮助不大。这时应检查代码体积、分支目标、间接跳转和译码压力，而不是继续只改数组排列。

后端：执行端口和调度 后端接收微操作后，要等待操作数准备好，并把它们发射到合适执行端口。整数 ALU、浮点乘加、加载、存储、分支和地址生成都有各自资源。一个循环如果每轮需要太多 load，可能受加载端口限制；如果每轮地址计算复杂，可能受地址生成单元限制；如果依赖链很长，端口空着也不能发射。

因此分析单核性能时，要区分“端口吞吐不够”和“依赖导致发不出去”。多累加器能改善后者；SIMD 能提高每条指令处理的数据量；简化地址表达式和连续访问能减轻加载和地址生成压力。不同 CPU 微架构端口配置不同，严肃优化必须结合具体机器。

后端不是一个抽象黑箱。一个 load 通常需要地址生成、TLB 查询、缓存访问和可能的 miss 处

理；一个 store 通常拆成地址和数据路径，并进入 store buffer；一个浮点 FMA 会占用特定向量执行资源；一个整数分支会占用分支执行资源。循环的性能取决于这些微操作怎样分布到端口上，也取决于它们之间是否互相等待。

这解释了为什么源码层面的“操作次数”常常误导。两个循环都执行同样数量的加法，一个可能因为单累加器依赖链而慢，另一个可能因为多累加器让后端并行发射而快。两个循环都做同样数量的 load，一个可能连续访问被预取器支持，另一个可能指针追踪导致每次 load 等上一次 load 的地址。

乱序执行的边界 乱序执行允许核心在不改变单线程可观察结果的前提下，提前执行已经准备好的指令。寄存器重命名消除了很多假依赖，保留站和重排序缓冲区让多个指令同时等待资源。这样做可以隐藏一部分延迟，例如在一次乘法等待时执行另一个独立加法。

但是乱序执行不能越过所有边界。真实数据依赖不能消除，cache miss 太多时可并行等待的 miss 数量有限，内存屏障会限制重排，原子操作会带来更强的顺序约束。写高性能代码时，增加独立累加器、减少循环携带依赖、让数据访问可预测，都是在帮助乱序核心找到可调度空间。

重排序缓冲区和执行窗口 乱序执行不是无限的。核心只能观察一个有限窗口内的指令。重排序缓冲区保存尚未提交的指令，load buffer 保存未完成加载，store buffer 保存尚未对外可见的存储。如果窗口被长延迟 load 占住，后续可执行工作又不够，核心会停顿。

链表遍历就是典型例子。每次读取下一个指针之前，必须等当前节点加载完成。乱序核心很难提前知道下一步地址，所以内存级并行度很低。相比之下，遍历多个独立数组或同时处理多条链，可以让多个 load 同时飞行，更容易隐藏延迟。

执行窗口可以被理解为硬件能同时“看见”的未来。窗口越大，越有机会在一个长延迟操作等待时找到其他独立工作。但如果代码中充满循环携带依赖，窗口再大也找不到可提前执行的操作；如果 cache miss 太多，load buffer 被占满，后续加载也发不出去；如果分支预测失败，窗口里的错误路径工作会被丢弃。

软件能做的事情，是把独立工作显式放进窗口。多累加器、分块处理、批量处理多个请求、一次处理多个链表游标、把检查从热循环中外提，都属于给乱序核心提供更多可调度空间。相反，频繁调用不可内联函数、隐藏别名、每步都依赖共享原子结果，会压缩这个空间。

Load、Store 和内存消歧 内存操作比寄存器操作复杂。核心要判断一个 load 是否可能读取前面尚未完成的 store。如果地址关系不明确，硬件会做内存消歧预测。预测错误时，需要回滚并重新执行相关操作。高性能循环中，减少指针别名、使用清晰的数组边界、避免复杂交叉读写，可以帮助硬件和编译器更大胆地调度。

store buffer 也影响并发语义。普通 store 可以先进入 store buffer，稍后再对其他核心可见。内存屏障、原子操作和锁释放会约束这种行为。C++ 内存模型的 acquire/release 最终要落到这类硬件机制上。

Load/store queue 还是很多微妙性能问题的来源。若一个循环中既写数组又读另一个可能别名的数组，编译器和硬件都必须保守处理读写顺序。若 store 地址很晚才算出来，后面的 load 就难以确认是否可以提前。若 store buffer 堆积，后续 store 可能被阻塞，锁释放和原子操作也可能放大这个问题。

并发程序中，store buffer 让“本线程已经执行了写”不等于“其他线程已经能按你想象的顺序看到写”。这不是说硬件不可靠，而是说可见性需要同步语义。后面讲 release/acquire 时，会把这条硬件直觉变成 C++ 层面的 happens-before 关系。

分支预测和控制流 分支预测让核心在条件结果出来之前先猜路径。预测正确时，流水线保持饱满；预测失败时，错误路径上的工作要被丢弃。随机分支、数据相关分支和复杂间接跳转都会破坏预测。很多高性能循环会把条件变成掩码、查表或分段处理，就是为了减少不可预测控制流。

并发程序也受分支预测影响。锁的 fast path 通常期望“不竞争”，如果竞争比例上升，分支行为 and cache 行为都会改变。无锁结构的 CAS 循环在低竞争时很短，在高竞争时会反复失败，控制流和内存系统同时恶化。

深入理解

判断一段代码是不是受分支预测影响，可以比较不同输入分布下的 cycles、branches 和 branch-misses。如果随机数据显著慢于有序数据，而内存访问量没有明显变化，分支预

测通常就是主要嫌疑。

SMT 和共享核心资源 SMT 让一个物理核心暴露多个逻辑 CPU。它可以在一个线程等待内存或前端供给不足时，让另一个线程使用空闲资源。但 SMT 不是免费翻倍。两个逻辑线程会共享前端、执行端口、缓存、TLB 和乱序资源的一部分。如果两个线程都在做同样高强度向量计算，它们可能互相抢执行资源；如果一个线程经常等内存，另一个线程可能填补空档。

因此 benchmark 中不能只写“8 个 CPU”。要区分物理核心和 SMT 线程。对于 CPU-bound 数值内核，先扩展到物理核心往往比立刻使用所有逻辑 CPU 更稳定；对于延迟隐藏明显的工作负载，SMT 可能提高吞吐。结论必须来自本机测量，而不是固定经验。

微架构和 ISA 的边界 第一册强调 ISA 和 ABI：同一条 `add` 在 x86-64 语义上做什么，函数参数怎样传递。第二册强调微架构：同一个 ISA 指令在不同 CPU 上可能拆成不同微操作，使用不同端口，拥有不同延迟和吞吐。ISA 给出程序可见语义，微架构决定实现成本。

这条边界非常重要。写教材时不能把某个 CPU 的端口配置说成 x86-64 的永久性质；写性能报告时也不能只说“x86 上这个指令很快”。更准确的说法是：在某个 CPU 型号、某个编译选项、某个输入分布下，观察到某个指令序列的瓶颈更像前端、端口、依赖、内存或分支。

典型性能症状 前端瓶颈常表现为 IPC 低、branch miss 或 instruction cache 相关事件高，代码布局 and 热循环大小值得检查。后端执行瓶颈常表现为数据在缓存中、分支也稳定，但 IPC 仍受端口或依赖限制，适合检查 SIMD、多累加器和指令混合。内存瓶颈常表现为 cache miss、TLB miss 或带宽接近平台上限。同步瓶颈则可能表现为 IPC 低、cache line 迁移、context switch、futex 等待或原子热点。

这些症状不会自动给答案，但能排除方向。不要在内存带宽瓶颈上执着减少几条整数指令，也不要锁竞争瓶颈上只调整循环展开。性能工程的价值在于把“慢”拆成可以证伪的假设。

从硬件回到代码 核心流水线给软件三个直接启发。第一，减少长依赖链，例如把单个累加器拆成多个局部累加器。第二，减少不可预测分支，让热路径尽量直。第三，减少会强制顺序的操作，例如不必要的原子和屏障。后续章节讨论 SIMD、锁、无锁队列和并行归约时，都会回到这些原则。

2.2 从代码形状到硬件瓶颈

本章不能只停留在“流水线很深、乱序很强”这种抽象描述上。我们从 Compute Systems Engine 的第一条主线开始：对一个很大的 `std::int32_t` 数组求和。这个问题看起来太简单，甚至像入门练习，但正因为它简单，才适合暴露单核执行的几个基本矛盾：每次循环都有加载、扩展、加法、循环控制和分支；总和变量形成循环携带依赖；数组访问通常可被预取器识别；输入足够大时又会被内存带宽限制。一个求和循环可以同时讲清“依赖、前端、后端、内存和测量”这几条线。

朴素版本通常写成一个累加器。

Listing 2.1 baseline: 单累加器顺序归约

```
1 std::int64_t sum_baseline(std::span<const std::int32_t> values) {
2     std::int64_t sum = 0;
3     for (std::size_t i = 0; i < values.size(); ++i) {
4         sum += values[i];
5     }
6     return sum;
7 }
```

这个版本的好处是清楚、正确、容易作为 reference。它的弱点也很明显：每一轮的 `sum` 都依赖上一轮的 `sum`。处理器可以提前加载后面的数组元素，也可以预测循环分支，但加法结果本身是一条长链。只要下一次加法必须等上一次加法结果，乱序执行就不能把这些加法任意并行起来。此时瓶颈不一定是加法器数量，而可能是依赖链延迟。

改进版本把一个长依赖链拆成多条短依赖链。

Listing 2.2 改进版本：四个独立累加器


```

1 std::int64_t sum_four_accumulators(std::span<const std::int32_t> values) {
2     std::int64_t sum0 = 0;
3     std::int64_t sum1 = 0;
4     std::int64_t sum2 = 0;
5     std::int64_t sum3 = 0;
6
7     std::size_t i = 0;
8     const std::size_t limit = values.size() / 4 * 4;
9     for (; i < limit; i += 4) {
10        sum0 += values[i];
11        sum1 += values[i + 1];
12        sum2 += values[i + 2];
13        sum3 += values[i + 3];
14    }
15    for (; i < values.size(); ++i) {
16        sum0 += values[i];
17    }
18    return (sum0 + sum1) + (sum2 + sum3);
19 }

```

这个版本不是为了“手动展开看起来高级”，而是给乱序核心更多可调度空间。四个累加器之间没有真实数据依赖，核心可以在一个累加器等待结果时推进另一个累加器。循环体变大，前端压力略增，但依赖链压力下降。若输入在缓存中，这种改法常能提高 IPC；若输入远大于 LLC 并且已经接近内存带宽上限，提升可能很小，因为瓶颈已经从依赖链迁移到数据移动。

还要注意边界处理。第二个循环处理不能被四整除的尾部元素。很多性能示例为了简短直接假设长度是四的倍数，这是教材不应采用的写法。真实系统里，输入规模经常来自文件、网络或上层分片，不能为了微优化丢掉边界正确性。Compute Systems Engine 的 reference 测试必须覆盖空输入、一个元素、非对齐长度、负数和大规模输入，不能只测漂亮输入。

优化不是越展开越好 既然四个累加器能拆依赖链，是否八个、十六个更好？答案取决于机器、编译器和 workload。展开过少，依赖链仍然限制后端；展开过多，会增加寄存器压力、代码体积和前端供给压力。寄存器不够时，编译器可能把临时值溢出到栈上，反而引入额外 load/store。代码体积变大时，热循环可能更容易超过微操作缓存或 instruction cache 的舒适范围。

这就是本册反复强调的“瓶颈会迁移”。一个版本慢，是因为单累加器依赖；另一个版本慢，可能因为寄存器压力；再另一个版本慢，可能因为已经受内存带宽限制。优化不能只问“这个技巧有没有用”，要问“在当前证据下，它针对的是哪个瓶颈，改完后瓶颈会迁到哪里”。

教材里常见的坏写法是直接给“最佳版本”。这种写法让读者只会背答案，不会分析。正确写法应保留 baseline，因为 baseline 是 correctness oracle；给出有限改进版本，因为它暴露优化动机；再说明它的边界，因为工程中没有永久最优版本。对求和来说，baseline 用于验证结果，多累加器用于解释依赖链，SIMD 版本用于解释向量执行，线程本地 partial 用于解释共享写和 false sharing，多线程版本用于解释带宽和 NUMA。这些版本不是重复代码，而是同一个问题在不同系统层面的投影。

不要把共享原子当作并行归约 很多初学者把并行求和写成多个线程同时对一个原子变量做加法。这个版本结果可能是正确的，却几乎一定不是好的归约实现。

Listing 2.3 反例：所有 worker 争用同一个原子和同一条 cache line

```

1 std::int64_t sum_with_hot_atomic(std::span<const std::int32_t> values,
2     std::int32_t worker_count) {
3     std::atomic<std::int64_t> global_sum{0};
4     std::vector<std::thread> workers;
5     workers.reserve(static_cast<std::size_t>(worker_count));
6

```

```

7  for (std::int32_t worker = 0; worker < worker_count; ++worker) {
8  const std::size_t begin =
9  values.size() * static_cast<std::size_t>(worker) /
10 static_cast<std::size_t>(worker_count);
11 const std::size_t end =
12 values.size() * static_cast<std::size_t>(worker + 1) /
13 static_cast<std::size_t>(worker_count);
14 workers.emplace_back([&values, &global_sum, begin, end]() {
15     for (std::size_t i = begin; i < end; ++i) {
16         global_sum.fetch_add(values[i], std::memory_order_relaxed);
17     }
18 });
19 }
20
21 for (std::thread& worker : workers) {
22     worker.join();
23 }
24 return global_sum.load(std::memory_order_relaxed);
25 }

```

这里使用 `memory_order_relaxed` 并没有错，因为这个原子只用于统计自身数值，不发布其他对象。但 `relaxed` 只能减少顺序约束，不能消除一致性争用。所有线程仍在更新同一个 `cache line`。每一次 `fetch_add` 都要求对应 `cache line` 获得独占修改权，多个核心会反复争夺所有权。结果是用户以为自己写了“无锁高性能”，硬件看到的是一个极热的共享写点。

正确方向通常是线程本地 `partial`。

Listing 2.4 工程版本：线程本地 `partial`，再串行合并

```

1  struct alignas(64) PartialSum {
2  std::int64_t value = 0;
3  };
4
5  std::int64_t sum_with_partials(std::span<const std::int32_t> values,
6  std::int32_t worker_count) {
7  std::vector<PartialSum> partials(static_cast<std::size_t>(worker_count));
8  std::vector<std::thread> workers;
9  workers.reserve(static_cast<std::size_t>(worker_count));
10
11 for (std::int32_t worker = 0; worker < worker_count; ++worker) {
12 const std::size_t begin =
13 values.size() * static_cast<std::size_t>(worker) /
14 static_cast<std::size_t>(worker_count);
15 const std::size_t end =
16 values.size() * static_cast<std::size_t>(worker + 1) /
17 static_cast<std::size_t>(worker_count);
18 workers.emplace_back([&values, &partials, worker, begin, end]() {
19     std::int64_t local = 0;
20     for (std::size_t i = begin; i < end; ++i) {
21         local += values[i];
22     }
23     partials[static_cast<std::size_t>(worker)].value = local;
24 });
25 }

```

```

26
27 for (std::thread& worker : workers) {
28     worker.join();
29 }
30
31 std::int64_t total = 0;
32 for (const PartialSum& partial : partials) {
33     total += partial.value;
34 }
35 return total;
36 }

```

这个版本把热路径写共享变成线程本地写。每个 worker 在自己的寄存器里累加，结束时只写一次 partial。对齐槽位用于避免多个 partial 落在同一条 cache line 上形成 false sharing。它的代价也要写清楚：需要额外 partial 数组，需要合并阶段，线程创建和 join 也有成本。如果输入很小，线程成本可能超过计算成本；如果输入很大，局部顺序扫描加一次合并通常比热原子好得多。

从源码到汇编应该看什么 第一册已经讲过如何读汇编。第二册读汇编不是为了炫耀指令名，而是为了验证源码是否产生了符合预期的执行形态。看求和循环时，应至少关注四件事。

第一，看编译器是否保留了循环。若结果没有被使用，优化器可能删除整个循环，benchmark 就变成空测。第二，看循环是否形成单累加器依赖，还是被编译器展开成多个累加器。现代编译器在优化级别较高时可能自动做循环展开和向量化，因此手写版本和编译器生成版本要放在同一编译选项下比较。第三，看是否有意外的内存访问。例如寄存器压力过大导致 spill，会在热循环中出现额外栈访问。第四，看是否有分支和调用留在热路径里。例如调试检查、虚调用、异常路径或不可内联函数都可能改变前端压力。

下面是一份实验报告可以采用的观察顺序。先用 **objdump** 或编译器的汇编输出确认热循环；再用 **perfstat** 比较 cycles、instructions 和 IPC；然后改变输入规模，让数据分别处在 L1、L2、LLC 和内存范围；最后再改变版本，从 baseline 到多累加器到线程本地 partial。若只看一个规模，很容易把依赖瓶颈和内存瓶颈混在一起。

深入理解

同一个源码版本，在不同编译器和不同优化级别下可能生成不同机器码。教材里的结论必须写成“分析方法”，不能写成“某段源码永远会生成某条指令”。严肃报告要记录编译器版本、目标架构、优化选项和关键汇编片段。

别名会限制编译器和硬件 处理器的乱序执行受真实依赖限制，编译器的优化也受别名限制。若函数接收两个指针，一个用于读，一个用于写，编译器必须考虑它们可能指向同一块内存。只要存在别名可能，编译器就不能随意重排 load 和 store，也不容易做激进向量化。硬件层面也类似，load/store queue 要判断后面的 load 是否可能读取前面尚未完成的 store。

看一个简化例子。

Listing 2.5 别名不清会让优化更保守

```

1 void add_scaled(std::span<const std::int32_t> input,
2   std::span<std::int32_t> output,
3   std::int32_t scale) {
4     const std::size_t count = std::min(input.size(), output.size());
5     for (std::size_t i = 0; i < count; ++i) {
6         output[i] += input[i] * scale;
7     }
8 }

```

这个函数语义清晰，但它没有说明 **input** 和 **output** 是否重叠。若调用方传入同一数组的重

叠视图，循环每一轮的写可能影响后续读。编译器为了保持 C++ 语义，必须保守。工程中解决别名问题不是靠随口告诉编译器“相信我”，而是靠 API 设计、数据所有权和测试约束。例如把输入输出设计成不同对象，文档声明不允许重叠，调试版本检查地址范围，或者把算法拆成先读后写的两阶段结构。底层性能问题常常来自上层接口没有表达清楚所有权。

Compute Systems Engine 的数据流也要这样设计。输入数组应是只读 span，输出 partial 数组由运行时创建并按 worker 独占写入，合并阶段再读取 partial。这样 API 本身表达了“读输入”和“写 partial”不会互相别名，后续章节讨论线程池和任务图时也能延续这个边界。

分支预测案例：过滤计数 分支预测不是只影响 if 语句的理论概念。考虑一个常见内核：统计数组中大于阈值的元素数量。最直观版本如下。

Listing 2.6 baseline：带条件分支的过滤计数

```
1 std::int64_t count_greater_branch(std::span<const std::int32_t> values,
2   std::int32_t threshold) {
3   std::int64_t count = 0;
4   for (const std::int32_t value : values) {
5     if (value > threshold) {
6       ++count;
7     }
8   }
9   return count;
10 }
```

如果输入已经排序，或者绝大多数值都落在同一侧，分支预测器很容易猜对。若输入接近随机，且一半大于阈值，一半不大于阈值，预测会困难得多。错误预测导致流水线清空错误路径工作，短循环中的成本会被放大。

可以写一个无显式分支版本。

Listing 2.7 改进方向：把条件变成整数贡献

```
1 std::int64_t count_greater_branchless(std::span<const std::int32_t> values,
2   std::int32_t threshold) {
3   std::int64_t count = 0;
4   for (const std::int32_t value : values) {
5     count += value > threshold ? 1 : 0;
6   }
7   return count;
8 }
```

这段源码仍然写了条件表达式，编译器可能生成条件移动、集合指令或向量比较，也可能仍生成分支，具体取决于目标架构和优化器。教材重点不是保证某种机器码，而是教读者提出可验证假设：随机输入下 branch miss 是否更高；无分支版本 instructions 是否增加；总时间是否下降；当输入分布从随机变成有序后，两个版本差距是否消失。如果这些现象吻合，就说明分支预测是主要因素之一。

分支优化也有代价。把复杂 if 改成掩码运算，可能让所有路径都执行，增加算术或内存访问；把数据分组处理，可能增加预处理成本；把虚调用改成分发表，可能降低可读性。不要把“branchless”当作普遍更快。只有当分支不可预测且热路径短，减少分支才经常有收益。

前端瓶颈案例：热循环里的调用和大 switch 很多性能分析只盯着数据 cache，却忽略了前端供给。前端要取指、预测、译码和提供微操作。若热路径里有过多不可内联调用、异常边、模板膨胀后的大代码体、解释器式大 switch 或间接跳转，后端可能一直等不到足够微操作。

一个典型案例是记录处理器。每条记录包含类型字段，不同类型执行不同处理逻辑。朴素实现可能把所有逻辑放进一个巨大的 switch，每个 case 又调用复杂函数。若输入类型分布随机，间接

分支或多路分支预测困难；若每个 case 代码很大，instruction cache 和微操作缓存压力上升。优化方向可能不是“少做几次整数加法”，而是重排数据：先按类型分桶，再批量处理同类记录。这样做改变了控制流形状，让分支更稳定，也让热代码更集中。

这类优化有一个重要边界：分桶会增加一次额外遍历和写入，还可能破坏原始顺序。若语义要求严格保持输入顺序，就需要额外索引或稳定输出；若记录数量很少，分桶成本超过收益；若每个记录处理非常重，分支成本可能不是主导。系统工程的结论永远要回到工作负载。

内存级并行度和指针追踪 乱序执行能隐藏一部分内存延迟，前提是窗口里有多个独立 load。顺序数组扫描虽然会 cache miss，但预取器能提前请求后续 cache line，多个 miss 可以同时飞行。链表遍历则不同：下一步地址存放在当前节点里，必须等当前节点加载完成后才知道下一次 load 地址。这种指针追踪几乎把内存访问串成依赖链。

下面的代码形态看起来只是遍历节点，硬件看到的是一条地址依赖链。

Listing 2.8 指针追踪：下一次加载地址依赖当前加载结果

```
1 struct Node {
2     std::int32_t value = 0;
3     Node* next = nullptr;
4 };
5
6 std::int64_t sum_list(const Node* head) {
7     std::int64_t sum = 0;
8     const Node* current = head;
9     while (current != nullptr) {
10        sum += current->value;
11        current = current->next;
12    }
13    return sum;
14 }
```

这段代码不适合作为热路径大规模数据扫描的默认结构。它的每个节点可能来自不同页面，cache miss 和 TLB miss 都可能很高；下一节点地址不可提前知道，内存级并行度低；分配器还可能把节点分散到不同 NUMA 节点。若业务语义允许，把节点压缩成连续数组或索引数组，通常更适现代 CPU。若必须使用链式结构，可以考虑批量处理多条独立链，让核心同时等待多个地址链；也可以使用池化分配，让节点尽量连续；还可以把热字段和冷字段分离，减少每个 cache line 带入的无用数据。

和编译器优化的边界 有些读者会问：既然编译器能展开、向量化、调度指令，为什么还要学习微架构？答案是，编译器非常强，但它不是业务语义的读心器。编译器不知道某个输入分布是否随机，不知道两个 span 在业务上是否必不重叠，不知道一把锁竞争是否来自热点 key，不知道任务粒度是否会造成线程池空转，也不知道分布式 shuffle 是否会遇到倾斜。它只能在 C++ 语义、目标架构和优化启发式内工作。

软件工程师要做的是把真实语义表达给编译器和硬件。用连续容器表达顺序访问，用只读 span 表达输入不可修改，用线程本地 partial 表达写入独占，用稳定的数据分块表达可预测访问，用测试保证边界输入正确。高性能代码不是处处和编译器对抗，而是提供更清晰的结构，让编译器能安全优化，让硬件能持续执行。

2.3 观察、实验与源码阅读

本章的 Linux 源码阅读不应从调度器全局策略开始，而应从性能证据的来源开始。若使用 `perfstat` 观察 cycles、instructions、branches 和 branch-misses，可以阅读 Linux 内核的 perf event 子系统入口，理解用户态工具如何请求硬件计数器、内核如何复用和调度计数器、权限如何限制某些事件。体系结构相关事件通常在 `arch/x86/events/` 一类目录下，通用框架在 `kernel/events/` 附近。不同内核版本路径会变化，读源码时必须记录版本。

编译器源码阅读可以从优化报告开始，而不是直接钻进后端。GCC 和 Clang 都能输出向量化或优化诊断。读者可以先写两个循环，一个容易向量化，一个因为别名或分支不容易向量化；观察诊断后，再选择性阅读对应编译器文档或源码。目标不是成为编译器开发者，而是理解“为什么这段循环没被优化”的证据链。

流水线不是简单并行流水 很多入门图把 CPU 流水线画成取指、译码、执行、访存、写回五段，这有助于入门，但现代核心远比五段复杂。前端要取指、预测分支、解码指令、使用微操作缓存；中间要重命名寄存器、分配 reorder buffer、调度微操作；后端要在多个执行端口、load/store queue 和缓存层级之间执行；提交阶段还要按程序顺序退休结果。理解现代性能时，不能停留在五段流水线的静态图。

乱序执行的核心思想是：只要不违反数据依赖和内存依赖，后面的独立操作可以先执行，用来隐藏前面操作的等待。若一个 load 等待内存，核心可以执行窗口中其他独立加法、乘法或 load。若窗口里全是依赖这个 load 的操作，核心就只能等。软件优化常做的多累加器、循环展开、数据连续化，本质上是在给乱序窗口提供更多独立工作。

乱序执行不是无限的。窗口大小有限，load buffer、store buffer、物理寄存器、调度队列和执行端口都有限。若程序有太多未完成 miss，load buffer 会满；若分支预测错，错误路径上的工作会被丢弃；若指令前端供不上微操作，后端再强也会饿。性能分析要问的是哪一部分资源限制了当前循环。

寄存器重命名和假依赖 寄存器重命名让硬件把架构寄存器映射到更多物理寄存器，从而消除某些假依赖。比如两次写同一个架构寄存器，如果后一次不需要等待前一次的值，硬件可以给它们分配不同物理寄存器。这样写后写和写后读的某些约束被解除，更多指令可以并行执行。

但真正的数据依赖无法靠重命名消除。一个累加循环中，下一次加法需要上一次 sum 的结果，这是真依赖。硬件不能凭空知道最终和可以分成多个 partial 再合并，除非编译器或程序写出多个累加器。多累加器优化的意义就在这里：把一条长依赖链拆成几条短依赖链，给乱序执行和 SIMD 留空间。

Listing 2.9 多个累加器拆开真依赖链

```
1 std::int64_t sum_four_lanes(std::span<const std::int32_t> values) {
2     std::int64_t a = 0;
3     std::int64_t b = 0;
4     std::int64_t c = 0;
5     std::int64_t d = 0;
6     std::size_t i = 0;
7     for (; i + 3 < values.size(); i += 4) {
8         a += values[i];
9         b += values[i + 1];
10        c += values[i + 2];
11        d += values[i + 3];
12    }
13    for (; i < values.size(); ++i) {
14        a += values[i];
15    }
16    return a + b + c + d;
17 }
```

这段代码不是保证一定比编译器自动优化更快，而是帮助读者看见依赖形状。实验时要检查编译器是否已经自动展开；若自动展开，手写版本可能没有收益。教材强调机制，不鼓励无证据地手写微优化。

前端：取指、译码和微操作缓存 前端负责持续给后端提供可执行微操作。若热代码太大、分支太乱、间接跳转太多、函数无法内联或模板膨胀严重，前端可能成为瓶颈。此时后端执行端口并未打满，但核心仍然无法退休更多有用指令。解释器、大型状态机、复杂虚调用层和异常路径都可

能遇到前端压力。

微操作缓存能缓存已经解码的微操作，减少重复译码成本。但它容量有限，代码布局和热路径大小会影响命中。把冷错误处理、日志格式化和调试路径混在热循环附近，可能增加指令 cache 和微操作缓存压力。冷热分离不仅适用于数据，也适用于代码路径。业务代码中常见的早期返回、错误分支和虚调度，也会影响前端行为。

优化前端瓶颈时，常见方法包括内联真正热的小函数、把冷路径拆出去、按类型分桶减少随机分支、使用表驱动但避免巨大不可预测间接跳转、减少模板生成的大量重复代码。每种方法都有可维护性成本，必须有证据支持。

后端：端口和执行单元 后端有多个执行端口，不同端口连接整数 ALU、乘法、浮点、加载、存储地址生成等执行单元。一个循环可能总指令数不多，却被某个端口限制。例如每轮需要两次 load 和一次 store，加载端口可能比整数加法更早成为瓶颈。另一个循环乘法很多，乘法吞吐可能限制整体速度。

端口分析通常需要反汇编、架构手册或工具辅助。教材阶段不要求读者手算每条指令端口，但要形成意识：减少指令数不一定减少瓶颈端口压力；增加一条看似便宜的地址计算，可能挤占关键端口；把复杂寻址改成简单指针递增，有时能减轻地址生成压力。SIMD 章节会继续讨论执行端口和向量吞吐。

Load/store queue 和内存依赖 内存操作也需要调度。Load queue 记录未完成加载，store queue 记录尚未提交或尚未写回的存储。处理器要判断一个 load 是否可能依赖前面的 store。如果地址不确定，硬件可能保守等待；如果预测错，还要回滚。这就是为什么别名信息会影响性能：编译器和硬件都需要知道读写是否可能指向同一位置。

C++ 中的指针、引用和 span 都可能表达别名。一个函数如果接收两个可写 span，编译器很难证明它们不重叠。即使硬件能做一些内存依赖预测，语言语义仍限制编译器重排。高性能 API 应尽量表达只读输入和独占输出，必要时用调试检查保证不重叠。不要把别名问题完全推给编译器。

分支预测器的学习对象 分支预测器不是读懂业务逻辑，它根据历史模式、地址和上下文猜测分支方向和目标。循环分支通常容易预测，数据相关随机分支难预测，间接分支目标多时更困难。分支预测器也会被不同分支之间的历史干扰，输入分布变化时可能短暂失效。

一个判断是否需要 branchless 的方法是改变输入分布。若有序输入和随机输入差距巨大，branch miss 也随之变化，分支预测很可能参与瓶颈。若两者差距不大，优化分支可能不是重点。另一个方法是按类型分桶：先把同类记录聚在一起，再批量处理，让分支模式稳定。这个方法用一次数据重排换更稳定控制流，是否划算要看记录数和每条记录处理成本。

Speculation 的成本 预测执行让核心在分支结果未知时先沿着猜测路径工作。猜对时节省等待，猜错时丢弃错误路径的结果并从正确路径重新开始。错误路径虽然不提交架构状态，但它消耗了取指、译码、执行端口和缓存资源。短循环里频繁错误预测，会让有效工作比例下降。

Speculation 还影响安全边界。现代 CPU 的投机执行曾引发多类侧信道问题。第二册不是安全专著，但要提醒读者：硬件为了性能做的预测和缓存，会让“没有提交的路径”仍可能留下微架构痕迹。高性能系统和安全系统并不是完全独立的领域。

SMT 的收益和风险 Simultaneous Multithreading 让一个物理核心同时运行多个硬件线程，共享前端、执行端口、缓存和队列资源。它能在一个线程等待内存或分支时，让另一个线程利用空闲资源；但如果两个线程都争用同一资源，例如都在做带宽密集扫描，SMT 可能降低单线程性能，甚至降低总效率。

性能实验要区分物理核心数和逻辑 CPU 数。若 8 物理核心 16 逻辑 CPU 的机器上，线程数从 8 增加到 16 没有提升或变慢，不一定是程序错误，可能是 SMT 资源争用。若任务有大量内存等待，SMT 可能有收益；若任务已经打满执行端口或内存带宽，SMT 可能无益。报告里写“使用 16 线程”时，必须说明这是逻辑线程还是物理核心。

频率、功耗和温度 CPU 频率不是固定常数。Turbo boost、功耗限制、温度、核心数和负载类型都会影响实际频率。单线程运行时，核心可能提升到较高频率；全核高负载时，频率可能下降；长时间 AVX 或宽向量负载也可能触发频率调整。若 benchmark 没记录频率和运行时间，就可能把频率变化误判为算法变化。

这对手机和笔记本尤其明显。短测试可能跑在高频，长测试因发热降频；后台充电状态、电源模式和温度都会影响结果。严肃实验应预热、重复运行、记录样本分布，并在报告中说明设备限制。

不能把一次短跑数字当作稳定吞吐。

代码布局 and I-cache 数据 cache 经常被讲得很多, instruction cache 却容易被忽略。大型模板代码、过度内联、巨型 switch 和大量错误处理混在热路径里, 都会增加 I-cache 压力。若前端 miss 增加, 后端会等待指令供应。对解释器、正则引擎、数据库执行器和网络协议解析器, I-cache 可能很重要。

优化代码布局时, 可以把冷路径标记或拆分出去, 把常见 case 放在紧凑路径, 把罕见错误处理放到单独函数。也可以通过 profile-guided optimization 让编译器根据真实运行频率布局代码。教学项目不必一开始使用 PGO, 但要让读者知道, 代码也是被缓存的对象。

反汇编阅读方法 阅读汇编时, 不要试图逐条记住所有指令。先看循环主体: 每轮有几次 load、几次 store、几次算术、几次分支, 是否有函数调用, 是否使用 SIMD, 是否展开, 是否有原子指令或锁前缀。再看循环边界: 尾部如何处理, 是否有额外检查, 是否因为别名生成运行时版本分派。最后结合 perf 指标判断瓶颈。

编译器输出不同不代表其中一个一定更好。更短的汇编可能因为少做了必要检查而不正确, 也可能因为使用了更强假设。读汇编的目的不是炫技, 而是验证源码结构是否被编译成预期形状。若你希望循环向量化, 却看到标量 load 和分支, 就要回到源码检查别名、对齐、控制流和编译选项。

案例: 哈希查找为什么难快 哈希表查找常被认为平均 $O(1)$, 但在核心流水线里, 它可能很难快。一次查找要计算 hash, 访问 bucket, 比较 key, 可能追踪链表或探测多个槽位, 遇到分支和随机内存。若 key 是字符串, 还要读取字符串字节。每一步都可能产生 cache miss、分支 miss 或依赖链。平均复杂度没有表达这些硬件成本。

开放寻址哈希表通常比链式哈希更容易利用 cache, 因为探测槽位连续; 但高负载因子会增加探测长度。链式哈希删除简单, 但每个节点可能分散在堆上。若业务 key 可以 intern 成整数, 查找会大幅简化。这个案例连接第2章的流水线、第3章的 cache、第6章的数据布局和第8章的聚合内核。

案例: 虚函数和间接分支 面向对象设计常用虚函数表达多态。热路径里每个元素调用虚函数, 硬件看到的是间接分支和不可内联调用。若对象类型分布随机, 分支目标预测困难; 若每种类型代码很大, I-cache 压力增加。优化方向可能是按类型分桶, 或者把热操作改成数据驱动的 switch 或 function table, 并用 profile 证明收益。

这不是说虚函数不好。冷路径、控制面和插件接口中, 虚函数的清晰性很有价值。问题是热循环里每元素虚调用可能成为前端和分支瓶颈。系统工程要区分热路径和冷路径, 不要把所有代码都按同一个风格写。

案例: 异常路径和热路径 C++ 异常在未抛出时通常不直接执行大量代码, 但异常边、栈展开信息和不可见控制流可能影响优化和代码布局。更重要的是, 热路径中把错误处理逻辑写得很大, 会增加 I-cache 和分支复杂度。高性能代码常把快速路径写得紧凑, 把错误处理拆到冷函数。

例如解析日志时, 正常行占 99.9%, 错误行很少。热循环应快速识别字段并写入 hot batch; 错误详情、格式化消息和上下文收集应进入冷路径。这样既保持错误可诊断, 也不让冷逻辑污染热指令流。代码冷热分离和数据冷热分离是同一思想。

微架构实验清单 本章建议增加四个小实验。第一, 单累加器与多累加器, 观察依赖链。第二, 可预测分支与随机分支, 观察 branch-misses。第三, 虚函数调用与按类型分桶, 观察前端和分支影响。第四, 链表遍历与数组遍历, 观察内存级并行度和 TLB。每个实验都保留相同语义和 correctness check。

实验报告不要只写时间。要写输入分布、循环形状、预期瓶颈、可用计数器和结果解释。微架构知识如果不落到实验, 很容易变成背术语。

案例: 解释器循环 解释器或字节码执行器是前端和分支预测的典型压力场景。一个大 switch 根据 opcode 分派, 每个 case 执行不同逻辑。若 opcode 分布随机, 分支目标预测困难; 若每个 case 很小, 分派成本占比高; 若每个 case 很大, I-cache 压力上升。优化方向包括 direct threading、按 opcode 分组、热点 opcode 专门化、JIT 或把多个简单 opcode 融合成 superinstruction。

这个案例对第二册也有启发。任务运行时、数据处理管线和未来 AI 推理引擎都可能有“解释器式”控制流: 根据算子类型、消息类型或记录类型分派。若热路径每条记录都做复杂分派, 前端成本会很高。把控制流批量化, 让一批同类数据走同一条路径, 是通用优化。

案例：函数指针表和预测 函数指针表可以替代大 switch，但它不自动更快。间接调用目标如果很多且随机，预测仍然困难；函数无法内联，还会增加调用开销。若目标集合很小且稳定，函数指针表可能足够；若目标频繁变化，按类型分桶或生成专门化代码可能更好。性能报告应比较 switch、函数指针和分桶三种形态，而不是凭风格选择。

这个案例也提醒接口设计。过早把热路径抽象成大量虚接口，会让编译器难以内联；完全去抽象又会让代码难维护。常见折中是控制面使用抽象，数据面热内核使用模板、函数对象或枚举派发，并用 benchmark 验证。

微架构章节总结 现代核心的性能来自前端供给、乱序窗口、寄存器重命名、执行端口、load/store 队列、分支预测和缓存层级共同作用。源码层面的一个 for 循环，硬件看到的是依赖图、指令流和内存请求。优化时先问依赖是否可拆、控制流是否可预测、数据是否连续、前端是否供得上、后端哪个资源受限。这个问题清单比记住某个 CPU 名词更有价值。

循环优化决策树 优化一个热循环，可以按决策树走。第一，结果是否正确且有 reference；没有 reference，先补 reference。第二，循环是否真的热；如果 profile 不在这里，先别优化。第三，循环主要是计算、访存、分支还是调用；用源码、输入分布和计数器判断。第四，若是依赖链，考虑多累加器、分块或改变算法；若是访存，考虑布局、连续访问、预取友好和减少字节；若是分支，考虑数据分桶、掩码和输入分布；若是调用，考虑内联、冷热拆分或批量接口。第五，优化后重新测，因为瓶颈可能迁移。

决策树的价值是防止跳步。很多人看到循环就先展开，看到分支就写 branchless，看到数组就写 SIMD。真正的工程优化应先定位受限资源。比如一个循环已经受内存带宽限制，再展开可能只增加代码大小；一个分支高度可预测，branchless 可能更慢；一个函数调用不在热路径，内联只会让代码膨胀。决策树让优化从证据开始，而不是从偏好开始。

微架构和可维护性 微架构优化常会牺牲可读性，因此必须设边界。标量 reference 必须保留，复杂优化必须有注释说明语义和适用条件，benchmark 要覆盖小输入和大输入，回退路径要可用。若某个优化只在一台机器上提升一点，却让代码难以维护，可能不值得。性能工程不是把每段代码都写到底层，而是把最热、最稳定、最有证据的路径优化到合适程度。

2.4 从朴素循环推导微架构概念

这一章不要从“前端、后端、乱序执行、分支预测”这些词开始。我们还是从一个最普通的问题开始：对一个很大的整数数组求和。最朴素的写法只有一个循环和一个变量。代码简单到几乎没有设计空间，所以它特别适合教学：如果这么简单的代码都可能慢，我们就必须问硬件到底在等什么。

第一步，先假设每次迭代只是加载一个整数，加到 sum，再进入下一轮。初学者容易以为 CPU 会把很多加法同时做完，因为现代处理器很强。但仔细看 sum 变量就会发现，第 i+1 次加法必须等第 i 次加法产生新的 sum。这里不是处理器不努力，而是程序自己给了硬件一条长链：后一步依赖前一步。于是“循环携带依赖”这个概念出现了。它不是术语游戏，而是对“为什么每轮看起来独立，硬件却不能完全并行”的解释。

第二步，朴素优化会想到循环展开。展开本身不一定有用，如果展开后仍然只写同一个 sum，依赖链还在。真正起作用的是多累加器：把一条 sum 链拆成 sum0、sum1、sum2、sum3 四条链。这样硬件在等待 sum0 的下次结果时，可以去执行 sum1、sum2、sum3 的加法。于是“乱序执行”和“指令级并行”也不是凭空概念，它们是从“多给硬件一些互不依赖的工作，硬件才能填满执行空隙”推出来的。

第三步，继续优化会遇到新问题。多累加器拆掉了加法依赖，循环可能仍然不快，因为每个元素都要从内存加载。数据在 L1、L2、LLC、DRAM 中的位置不同，成本不同。如果数组很大，CPU 可能主要在等数据，而不是等加法。于是瓶颈从“依赖链”迁移到“加载和内存带宽”。这就是为什么优化报告不能只说某个版本快了多少，还要说明快之前卡在哪里，快之后又卡在哪里。概念之间的顺序也由此清楚：先看依赖链，拆开后再看加载和内存层级。

第四步，我们给循环加一个条件：只统计大于阈值的元素。现在问题不只是加法和加载，还多了分支。若输入几乎都大于阈值，分支很好预测；若输入随机，一半大于一半小于，预测就困难。这里才需要“分支预测”这个概念。它回答的是：CPU 为了不断流水执行，必须提前猜下一步走哪条路；猜对了流水线继续，猜错了就要丢掉一部分已经做的工作。分支预测不是孤立知识点，而是从“同一段 if 为什么在不同输入上速度差很多”这个现象推出来的。

第五步,我们尝试把分支改成 branchless 写法。它可能更快,也可能更慢。为什么?因为 branchless 减少错误预测,但可能增加指令数量,执行本来可以跳过的计算,还可能占用更多寄存器。于是我们得到一个重要原则:每个优化都在交换成本。减少分支成本,可能增加算术成本;减少依赖链,可能暴露内存成本;增加内联,可能增加 I-cache 压力。微架构学习的主线不是背名词,而是追踪成本怎样迁移。

从求和推广到解析器 现在把求和循环换成日志解析。每个字符都要判断是不是数字、逗号、换行或引号。朴素解析器可能写成一个状态机,里面有很多 if 和 switch。输入格式稳定时,分支预测还可以;输入混杂时,分支目标变得难猜;错误处理如果写在热路径里,正常记录也要背着错误格式化和日志输出的指令。于是我们自然推出“热路径”和“冷路径”的区别。正常行占绝大多数,就让热路径只做快速判断和必要写入;错误详情放到冷路径,等真的出错再构造。

这不是为了追求代码花样,而是从“每条正常记录都为罕见错误付出前端成本”这个失败推出来的。CPU 前端要取指、解码和预测分支,热函数越大、分支目标越杂,前端越容易成为瓶颈。把错误处理拆冷,就是让最常见路径更短、更稳定。这个原则会在后面的 I/O、分布式重试和恢复章节反复出现:慢路径必须存在,但不应该污染每条数据的热路径。

从编译器报告回到源码结构 当我们希望编译器自动向量化求和或过滤循环时,编译器可能拒绝。它拒绝的原因通常不是“编译器笨”,而是源码没有给出足够清楚的结构。输入输出可能别名,函数调用不可内联,循环里有数据相关写位置,控制流太复杂,或者成本模型认为向量化不划算。于是“优化报告”不是神秘工具,而是把编译器的疑问显示给人看。

读报告也要按推导来。若报告说可能别名,说明编译器不知道写 output 会不会影响读 input;我们就要回到接口设计,明确输入只读、输出独占,必要时在 debug 版检查重叠。若报告说函数调用阻碍向量化,说明热循环里的抽象边界太厚;我们可以拆出纯函数、让函数可内联,或者把单元接口改成批量接口。若报告说控制流复杂,可能需要先用 selection vector 或分桶把数据变规则。每一种报告原因,都应该推回一个源码结构问题。

反例为什么必须存在 微架构实验如果只证明一个技巧有效,就容易变成新的教条。多累加器在依赖链明显时有用,但在内存带宽已经饱和时收益会变小;branchless 在随机分支上可能有用,在高度可预测分支上可能更慢;数组通常比链表局部性好,但如果数组元素很大且只访问少数字段,也可能需要改成列式布局;内联能减少调用开销,但过度内联会让热代码膨胀。

所以每个实验都要有反例。多累加器实验要测小数组、大数组和不同优化级别;分支实验要测全真、全假、规律交替、随机和真实输入;链表实验要测连续分配和随机打乱;虚函数实验要测单一类型和多类型混合。反例不是为难读者,而是让读者看到概念的适用边界。真正掌握微架构,不是知道“某技巧快”,而是知道它为什么快、何时不快、如何证明当前场景属于哪一类。

本章的回归阅读要盯住热循环证据包。以日志分隔符扫描为例,先保留最朴素的逐字节版本,再分别构造三组输入:分隔符密集、分隔符稀疏、非法字符集中在早期位置。不同输入会改变分支预测、前端供给和后端等待,不能用一组随机数据代表所有结论。热循环不是“写得短就快”,而是要看依赖、分支和取指是否同时顺畅。

证据包至少包含四类材料。第一,源码版本和编译选项,说明是否允许展开、内联和向量化。第二,反汇编片段,标出关键 load、compare、branch 和累加依赖。第三,计数器摘要,记录 cycles、instructions、branch-misses 和 IPC。第四,对照解释,例如多累加器为什么打断依赖链,branchless 版本为什么在某些输入上更稳,在另一些输入上反而做了无用工作。

本章的质量标准不是把流水线术语背完,而是能解释一个具体改动为什么有效或无效。如果四累加器版本变快,要说清瓶颈从串行依赖转向加载或带宽;如果随机分支版本变慢,要说明输入分布怎样破坏预测;如果反汇编显示编译器已经自动改写循环,就不要把收益误归因给手工小改。这样的分析才配得上“微架构”三个字。

热循环实验还要区分前端和后端。前端问题通常表现为指令供给不足、分支预测失败、代码体过大或间接跳转难预测;后端问题通常表现为 load 等待、执行端口饱和、长依赖链或 store buffer 压力。一个解释器循环可能前端受限,因为每次 dispatch 都有间接分支;一个数组归约可能后端受限,因为累加依赖和加载端口限制吞吐;一个日志解析循环可能两者都有,分支由输入分布决定,加载由字段位置决定。

本章可以要求读者写两份反汇编注释。第一份注释标出循环体中的关键指令,说明哪些指令在依赖链上,哪些可以并行发射。第二份注释标出分支位置和 fall-through 路径,说明输入分布如何影响预测。注释不需要覆盖所有指令,但必须把“源码中的一行”翻译成“硬件要经历的取指、比较、

跳转、加载和提交”。这种翻译能力会在 SIMD、锁热点和任务调度章节继续使用。

分支预测案例要避免只讲“预测错很贵”。真正要写清的是分布如何改变控制流。若日志字段大多合法，错误检查分支很少 taken，预测器容易形成稳定模式；若合法和非法随机交替，分支可能频繁失败；若输入按类型分桶，分支又重新变得可预测。这个观察会连接到后面的 partition：有时先按类型重排数据，不是为了算法复杂度，而是为了让硬件看到更规则的控制流。

乱序执行案例也要说明窗口不是无限的。多累加器可以暴露更多独立操作，但 load miss、store buffer、寄存器压力和前端指令供给都会限制收益。若一个循环把状态拆成太多临时变量，编译器可能产生更多寄存器占用甚至 spill；若展开过度，代码体变大，前端压力上升。微架构优化的高级感在于看见这些边界，而不是把“展开循环”和“多累加器”当作永远正确的口诀。

2.5 从一个热循环拆出微架构因果链

流水线、乱序执行、分支预测和前端供给如果单独讲，很容易变成硬件名词表。更好的入口是一个热循环：遍历一段日志字节，找到分隔符，解析字段，并把 event type 累加到本地桶。这个循环表面上只是 for、if 和数组访问，实际会同时触发取指、译码、分支预测、load、store、地址生成、依赖等待和提交。读者要学会从源码里画出依赖图，再用测量去验证哪一条边正在卡住执行。

第一版循环通常只写一个累加器或一个状态变量。它的优点是语义清楚，缺点是容易形成循环携带依赖：下一次迭代必须等上一次结果出来。乱序核心可以同时处理许多独立指令，但不能违反真依赖。如果每次迭代都依赖前一次的累加值，硬件窗口再大也只能等待。多累加器不是魔法，它只是把一条长依赖链拆成几条短链，让乱序窗口里能放进更多可并行工作。这里的推导顺序必须写清：先证明合并函数可结合，再拆累加器；若浮点误差或顺序语义不能接受，就不能随便重排。

前端问题：机器没有足够指令可发射

前端负责把指令送入后端。热循环如果包含过多间接调用、虚函数、复杂模板实例或难以内联的函数边界，前端可能供给不足。此时后端执行单元并不忙，程序慢在取指、译码、指令缓存和分支目标预测。很多 C++ 抽象在冷路径很好，在热路径可能让编译器看不见具体类型，无法内联和展开循环。教材不能简单说“抽象不好”，而要说清控制面和数据面的区别：控制面可以保留清晰接口，数据面热内核要让编译器看清连续范围、类型、别名关系和循环边界。

验证前端瓶颈不能靠猜。可以比较四个版本：虚接口逐条处理、函数对象批量处理、枚举分派批量处理、完全内联的专用内核。若完全内联版本指令数下降、分支目标更稳定、IPC 提高，说明调用边界确实参与成本。若四个版本差异很小，就不要为了“可能更快”牺牲接口可维护性。高质量教材要让读者看到，去抽象是一种成本很高的优化，必须由 profile 证明。

前端还和代码体积有关。为了消除分支，把每种输入模式都展开成一大段专用代码，可能让 I-cache 压力上升；为了模板特化，把许多类型组合都实例化，可能让热路径附近代码变大。一次优化如果减少了后端分支，却造成前端缺指令，瓶颈就迁移了。报告应同时记录指令数、分支事件、I-cache 或前端 stall 相关指标，至少要通过反汇编确认热循环没有被意外膨胀。

后端问题：执行单元、端口和依赖

后端问题从依赖图开始。数组求和的地址顺序可预测，load 可以排队，多个累加器能提高并行度；指针追踪的下次地址取决于当前 load 返回值，内存级并行度很低；哈希表查询同时有随机 load、比较、分支和可能的链式冲突。三者都是循环，但硬件看到的形状完全不同。章节要反复强调：源码复杂度和硬件复杂度不是同一个维度。

执行端口也会形成瓶颈。一个循环可能不是被算术延迟卡住，而是被加载端口、存储端口或地址生成单元限制。若每次迭代需要多次随机 load，增加整数 ALU 优化没有意义；若每次迭代都有 store、store buffer 和写分配也会进入成本；若循环中混入除法、复杂浮点或类型转换，少数高延迟指令可能压住吞吐。这里适合让读者写一个小实验：固定输入规模，只改变每次迭代的 load 数、store 数和算术数，观察吞吐如何变化。

乱序窗口不是无限资源。它能隐藏一部分 cache miss，但需要足够独立请求。顺序扫描数组时，硬件预取和多个 outstanding miss 可以让带宽接近上限；链表遍历时，下一个地址没有返回前无法发起下次 load，乱序窗口没有足够独立工作。这个差异解释了为什么同样访问相同字节数，连续数组远快于链表。后续数据布局章会继续展开，但本章要先让读者从流水线角度理解：局部性不仅是缓存命中率，也是硬件能否提前知道下一步地址。

分支问题：预测、数据分布和改写边界

分支预测不是“有 if 就慢”。若输入分布稳定，分支方向容易预测，条件分支成本可能很低；若输入随机，方向接近均匀，误预测会清空流水线，成本很高。日志解析中，分隔符出现位置、字段合法性、事件类型分布都会影响分支行为。同一段代码在真实日志和随机生成数据上的结论可能完全相反，所以实验必须写清输入分布。

把分支改成 branchless 也不是固定答案。branchless 版本可能多做计算、多访问内存、引入掩码和选择指令。若原分支高度可预测，改写后反而变慢；若原分支导致严重误预测，改写可能有效。教材应给出推导：先测 branch-misses 和输入分布，再写保持语义的 branchless reference，再比较指令数、吞吐和结果。不要直接把“无分支”当作高级写法。

错误处理分支尤其要谨慎。热路径可以把错误记录放到旁路缓冲，减少每条正常记录的分支和状态污染；但错误语义不能丢。若解析失败需要报告行号、字段和原因，就必须保存足够上下文。高性能不是把错误处理删掉，而是把低频错误路径和高频正常路径分离，同时保留可诊断性。这个原则会在 I/O 错误、任务异常和分布式失败中反复出现。

编译器、反汇编和测量闭环

本章的 Linux 和工具入口包括 `perfstat`、`perfrecord`、`perfannotate`、编译器优化报告和 `objdump`。优化报告告诉你编译器是否向量化、是否因为别名或依赖拒绝优化；反汇编告诉你热循环实际有多少 load、store、branch 和调用；perf 告诉你运行时事件是否支持假设。三个证据缺一不可。

一个规范实验可以这样写。先准备串行 reference，确保输出 checksum 固定。然后实现单累加器、多累加器、branchless 计数、批量接口四个版本。每个版本运行多轮，固定 CPU 频率和线程绑定，输入规模从 L1 能容纳扩到内存级。报告中写出：小输入是否受依赖和前端影响，大输入是否迁移到内存带宽，branchless 是否只在随机分布下有效，批量接口是否让编译器内联和展开。若某台设备没有硬件计数器权限，至少保留反汇编和耗时曲线，并明确证据边界。

贯穿项目在本章的交付物不是“把循环优化到最快”，而是建立热循环分析模板。每个热内核都要有 scalar reference、输入分布说明、反汇编摘录、计数器摘要和瓶颈迁移记录。后续 SIMD、线程、队列和分布式章节都要复用这个模板，否则项目很容易变成每章单独写一个 demo，无法积累判断力。

案例走查：日志分隔符扫描的三次改写

第一版逐字节扫描，遇到逗号或换行就进入分支。它清楚，但在随机字段长度和多分隔符输入下，分支预测可能不稳定。第二版把条件结果转成整数贡献，减少控制流，但会增加一些指令。第三版用批量接口一次处理一段字节，为后续 SIMD 或自动向量化留下空间。三版都必须返回同样的分隔符位置列表，不能只比较耗时。

实验时准备三组输入：固定长度字段、随机长度字段、含错误行字段。固定长度验证分支可预测时 branchless 不一定有收益；随机长度验证误预测成本；错误行验证旁路错误处理是否破坏热路径。报告应摘出热循环反汇编，记录 instructions、cycles、branches、branch-misses 和吞吐。若硬件事件不可用，就用输入分布和多轮耗时作为替代，并明确不能确认误预测比例。

这个案例训练的是推导过程：不是先说“分支慢”，而是先问输入分布会不会让分支难预测；不是先说“SIMD 快”，而是先把输出位置和错误语义写清；不是先把代码写复杂，而是保留标量 reference 和每一步证据。

微架构实验的代码边界

本章实验代码要故意小而干净。一个文件只放输入生成、reference、被测内核和计时，不要同时接入线程池、I/O 和日志系统。原因很简单：流水线实验要隔离核心循环，若把读取文件、分配内存、任务调度混进去，cycles 和 IPC 就无法归因。高质量教材不是每个例子都写成完整应用，而是在不同层次选择合适的最小系统。

输入生成要避免被编译器优化掉。结果必须写入 checksum，checksum 要在计时后被使用。数组指针和 span 要明确只读或可写，避免别名让编译器保守。若要比对虚调用和批量接口，就保持算法完全相同，只改变调用边界。若要比对分支和 branchless，就保持输入分布相同。每个变量都要有名字和目的。

代码中的计时也要谨慎。不要把随机数生成、内存分配和打印放进热循环。使用 steady clock 计端到端热循环时，要多轮运行并丢弃明显异常；使用 perf 时，命令行要保存。若用 `perfstat`，至少记录 cycles、instructions、branches、branch-misses、cache-misses；若没有权限，就保存反汇编和时间序列。报告里不能写“应该是分支预测问题”，只能写“当前证据支持或不支持这个假设”。

源码阅读从编译产物开始

微架构章的源码阅读入口不是 Linux 内核，而是编译产物。读 C++ 源码、优化报告、汇编三者之间的差异。一个简单循环可能被展开、向量化、消除、内联或保持标量；一个看起来小的函数调用可能阻止优化；一个分支可能被编译成条件移动。读者要学会尊重机器实际执行的代码。

具体路线是：先用 `-O0` 和 `-O3` 编译同一内核，比较反汇编；再打开向量化报告，看循环是否被识别；再用 `perfannotator` 找热指令；最后回到源码判断是否能够通过接口改写帮助编译器。这个过程会让读者明白，性能优化不是在源码里凭肉眼数语句，而是源码、编译器和硬件之间的闭环。

本章的验收不是“写出最快版本”，而是能解释四种情况：为什么多累加器有效，为什么指针追踪无效，为什么 `branchless` 有时有效有时无效，为什么抽象接口可能影响热路径。能解释这四种情况，读者才真正理解流水线和乱序执行。

综合练习：为热循环建立证据包

本章综合题要求读者选择一个热循环，例如日志分隔符扫描或本地桶计数，提交一份证据包。证据包第一部分是语义说明：输入范围、输出、错误处理、是否允许重排、是否允许 `branchless` 改写。第二部分是源码版本：baseline、一个只改变依赖链的版本、一个只改变分支形态的版本、一个只改变调用边界的版本。第三部分是编译证据：优化报告、反汇编热循环和是否发生内联或向量化。

证据包第四部分是运行证据。输入至少要有可预测分支和不可预测分支两类，规模至少覆盖缓存内和内存级两类。每个版本运行多轮，记录 checksum、耗时和可用的 perf 事件。若没有事件权限，也要记录不可用原因，并用反汇编和输入分布解释当前只能支持哪些结论。证据包第五部分是瓶颈迁移：某个优化生效后，新的主成本是什么，下一步该看前端、后端、分支还是内存。

这个题严禁直接写“优化版代码”。没有 baseline，读者不知道改进来自哪里；没有只改变一个因素的版本，读者无法归因；没有 checksum，读者无法证明语义保持；没有反汇编，读者无法知道机器实际执行什么。热循环优化看似是底层技巧，本质上是证据组织能力。

验收时可以让另一个读者只看证据包复述结论。如果他能说清“这个版本快是因为依赖链拆开，而不是因为少读了数据”，说明报告合格；如果只能说“版本 B 比版本 A 快”，说明证据不足。这样的训练比背乱序窗口、分支预测器、执行端口更能形成工程判断。

容易出错的边界：微优化为什么会误导系统判断

微架构章需要专门说明误导性优化。第一个误导是“减少源码行数”。源码行数和指令数没有稳定关系，一行模板调用可能展开成大量代码，一段看起来复杂的循环可能被编译器优化得很紧。第二个误导是“减少一条指令就一定快”。如果程序受内存带宽限制，少几条整数指令可能没有任何端到端收益。第三个误导是“`branchless` 一定优于 `branch`”。可预测分支很便宜，`branchless` 可能增加算术和寄存器压力。

第四个误导是“多累加器只是在骗编译器”。多累加器改变的是依赖图，不是语法技巧。它让硬件有更多独立工作可发射，但前提是合并语义允许重排。第五个误导是“反汇编太底层，不影响 C++ 设计”。如果热路径 API 隐藏别名关系、每个元素走虚调用、输出和输入可能重叠，编译器就会保守；这说明接口设计本身就是性能设计。第六个误导是“只要开最高优化级就够”。优化级不能替代正确的数据布局、清晰语义和可预测访问。

项目中的代码审查应把这些误导写成问题。这个热循环是否有 reference？是否有 checksum 防止被优化掉？是否知道编译器实际生成什么？是否只改变了一个因素？是否说明小输入和大输入下瓶颈是否迁移？是否把错误路径和热路径分开？这些问题比“有没有用某条指令”更重要。

最后还要承认微架构结论的局部性。某个 CPU 的分支预测器、缓存、SIMD 宽度和前端能力与另一台机器不同。书中实验给的是推导方法，不是永久数字。读者在自己的 Termux 或 Linux 设备上跑出不同结果时，不应认为书错了，而应回到输入、编译器、硬件和权限，解释差异从哪里来。

本章核心接口：HotKernelReport

微架构章给项目留下 `HotKernelReport`。它至少包含 kernel name、input bytes、records、checksum、version name、elapsed time、instructions、cycles、branch misses、cache misses、compiler flags 和 disassembly note。没有硬件计数器时，对应字段标记 unavailable，并写替代证据。报告对象不参与热路径，只服务实验和回归。

后续 SIMD、布局 and 内核章节都复用这个结构。这样一个优化提交不能只说“更快”，必须说明在哪个输入、哪个版本、哪个证据下更快。热内核报告也能防止性能回归：同样输入下，checksum 变了立即失败，耗时大幅波动则进入人工分析。

从流水线进入缓存层级

本章分析的是核心如何消费已经到手或正在等待的数据。下一章继续追问：数据为什么能及时到手，为什么有时迟迟不到。乱序窗口可以隐藏部分延迟，但它需要独立请求；预取器可以提前取数据，但它需要可预测地址；分支预测可以保持流水线不断，但它不能修复随机访存。于是微架构分析自然走向 cache line、TLB、页和地址序列。

项目中也要按这个顺序推进。先用热循环报告判断是否存在依赖、分支或前端问题；若优化后仍慢，再看数据布局和地址序列。不要一开始就同时改循环和布局，否则无法知道收益来自哪里。硬件两章的衔接，就是从“指令怎样执行”过渡到“数据怎样到达执行单元”。

2.6 项目落地

Compute Systems Engine 在本章至少要得到三个能力。第一，顺序 reference: `sequential_sum` 保持最直接写法，用它验证所有优化版本结果。第二，单核优化版本：增加多累加器或向量化实验，但每个版本都要保留 correctness check。第三，测量 harness：能以相同输入、相同编译选项、相同计时边界比较 baseline、多累加器、热原子和线程本地 partial。这里的目标不是立刻写最强归约，而是建立“同一问题跨层演化”的实验骨架。

实验报告建议按下面结构写。先说明输入规模、数据类型和正确性 oracle；再说明 CPU 型号、编译器、优化选项、线程数和是否绑定；然后列出版本：baseline、四累加器、热原子、多线程 partial；最后解释每个版本的预期瓶颈和实际指标。若四累加器没有变快，不能直接说技巧无效，要检查编译器是否已经自动展开、输入是否被内存带宽限制、计时是否包含线程创建、以及结果是否被优化掉。

指令流 处理器首先面对的是指令流。循环体太大、间接跳转太多、冷热路径混在一起，都会让前端把周期花在取指和译码上。分析时先看循环体大小、分支数量和反汇编形态，再谈算术指令本身。

依赖链 乱序执行能重排独立操作，却不能让真正的数据依赖消失。单累加器求和把每次 add 串在同一条链上，吞吐被延迟限制；多累加器把链拆开，才给后端提供并行空间。这个例子比抽象讲 ILP 更直观。

分支预测 分支不是一概慢，可预测分支成本很低，随机分支会让流水线频繁清空。教材应让读者构造两组输入：一组排序良好，一组随机分布，再看同一段代码的差异。这样才能理解分支成本来自不确定性。

load 延迟 load 指令的延迟取决于数据在 L1、L2、L3、内存还是被别的核心持有。流水线章不深入缓存细节，但要告诉读者：一条 load 可以让后续依赖操作停住，乱序只能用其他独立工作隐藏它。

端口和吞吐 同一类指令会竞争执行端口。代码里看起来有很多操作，硬件上可能集中打到少数端口。写报告时不要把“指令数减少”直接等价于“更快”，还要看关键资源是否被释放。

反汇编 反汇编不是炫技，它用于确认编译器实际生成了什么。源码中的循环、函数调用、条件判断和数据类型，经过优化后可能完全变形。热循环报告至少保存编译选项、目标架构和关键汇编片段。

计数器 PMU 计数器可以提示 cycles、instructions、branch-misses、cache-misses 等现象，但它不是自动答案。计数器要和输入、代码、反汇编一起解释；只截图一个 IPC 数字，无法说明瓶颈。

优化边界 微优化必须有退出条件。若展开循环让代码体积过大，前端可能更差；若消除分支需要额外内存写，瓶颈可能转到带宽；若手写 SIMD 破坏可读性却没有稳定收益，就不该进入主线。

案例：单累加器与多累加器 对同一数组写两个求和版本，先预测依赖链长度，再比较 cycles per element。要求报告说明为什么多累加器不改变算法复杂度，却改变了后端可并行度。

案例：冷热路径分离 把罕见错误处理从热循环中挪出去，观察前端和分支行为。重点不是让代码变花，而是让常见路径保持短、直、可预测。

案例：随机分支输入 用固定比例但不同顺序的输入测试 branch miss。排序输入和随机输入的逻辑结果相同，硬件路径不同，这能训练读者区分语义等价和执行成本不同。

源码阅读路线 Linux 源码阅读从 `tools/perf`、`kernel/events/core.c`、`arch/x86/events/core.c` 附近进入即可。阅读目标不是把整套子系统抄进书里，而是追一条和本章实验相

关的路径：用户态动作怎样变成内核对象，内核对象怎样排队、等待、唤醒、计数、提交或回收。

阅读报告要写证据等级。能直接测到的数字放在第一层；从源码控制流推导出的解释放在第二层；当前设备无法验证、只能作为合理猜测的内容放在第三层。这样读者会知道热循环证据包中哪些结论可复现，哪些结论需要换机器继续确认。

源码阅读还要克制。看到一个复杂函数，不要把每个分支都展开；先回到本章的问题，问它能否解释一个等待点、一次状态转移或一条性能曲线。解释不了的内容可以留到索引，不要打断正文主线。

单累加器与多累加器 对同一数组写两个求和版本，先预测依赖链长度，再比较 cycles per element。要求报告说明为什么多累加器不改变算法复杂度，却改变了后端可并行度。

冷热路径分离 把罕见错误处理从热循环中挪出去，观察前端和分支行为。重点不是让代码变花，而是让常见路径保持短、直、可预测。

随机分支输入 用固定比例但不同顺序的输入测试 branch miss。排序输入和随机输入的逻辑结果相同，硬件路径不同，这能训练读者区分语义等价和执行成本不同。

前端供给

热循环不是只由执行端口决定。取指、译码、分支目标预测和指令缓存供给不足时，后端执行单元会等不到微操作。

实验设计：把一个小函数过度拆成虚调用版本，再和内联版本比较指令数、分支事件和运行时间。

依赖链

单累加器归约把下一次加法依赖在上一次结果上，乱序窗口无法创造不存在的独立工作。多个累加器不是魔法，而是把一条长链拆成几条短链。

实验设计：比较一个累加器、四个累加器和手写展开版本，记录结果一致性和吞吐变化。

分支预测

可预测分支很便宜，随机分支很贵。把所有分支都改成 branchless 并不总是更快，因为它可能增加算术、加载和寄存器压力。

实验设计：构造全真、全假、周期性和随机四种输入，观察同一条件判断的成本变化。

内存级并行

指针追踪让下一次地址依赖当前加载结果，硬件难以并行发起多个 miss；连续数组扫描则能让预取和乱序执行发挥作用。

实验设计：用数组索引链和连续数组求和对照，解释为什么复杂度同为线性但等待形状不同。

别名信息

编译器不知道两个指针是否指向同一内存时，会保守地保持加载和存储顺序。接口把只读、只写和不重叠说清，优化空间才会出现。

实验设计：把函数参数从裸指针改成只读视图和独占输出，比较优化报告中向量化和重排提示。

反汇编阅读

读汇编不是数指令，而是找循环体、依赖、分支、加载、存储和调用边界。源码短不等于机器码短，模板展开和异常路径也可能进入热区。

实验设计：标出循环入口、回边、条件跳转和累加指令，再把它们对应回源码。

错误优化

减少源码行数、减少一条指令或消除一个分支都不是最终目标。若瓶颈已经在内存带宽，少几条整数指令可能没有收益。

实验设计：先测单线程标量瓶颈，再逐步加入多累加器和 SIMD，不跳过瓶颈迁移记录。

前端体积

过度内联和大量模板实例可能让指令缓存压力上升。控制面可以保持抽象，数据面热路径要让编译器看到具体类型和连续范围。

实验设计：为解析器写策略对象版本和热内核函数对象版本，比较代码体积与热循环布局。

计数器归因

IPC、branch misses、cycles 和 instructions 只能组合解释。单个计数器异常不等于根因，必须结合输入分布和代码形状。

实验设计：同一程序改变输入随机性，记录分支事件和总时间是否同向变化。

项目接口

热循环报告应保留 reference、输入形状、反汇编摘要、计数器摘要和仍未证明的边界。它是后续 SIMD 与多线程优化的底稿。

实验设计：每次优化只改一个因素，并把旧版本保留到报告中。

2.7 热循环不是源码行数，而是硬件供给和依赖形状

读者学习流水线时，最容易把它理解成一张硬件结构图：取指、译码、执行、访存、写回。结构图有用，但如果停在图上，遇到 C++ 热循环仍然不知道看什么。本章应从一个循环开始：遍历一列整数，把满足条件的元素累加。源码只有几行，机器执行时却包含取指、分支预测、加载、依赖链、端口竞争和提交。教材写法要把这几层按因果关系连起来，而不是先列一串微架构名词。

第一条因果链是前端供给。源码里一个简单判断，编译后可能包含加载、比较、条件跳转和回边；如果循环体里还有虚调用、异常路径或过度模板展开，指令缓存和分支目标预测都会进入成本。很多人只盯着数据 cache，却忽视了前端。判断方法不是猜，而是看反汇编里的热循环：循环体有多大，回边在哪里，是否有间接跳转，是否调用了无法内联的函数。若后端端口并不忙，分支事件和前端停顿却高，继续改算术表达式往往无效。

第二条因果链是依赖。单累加器求和写起来最自然，但每次加法依赖前一次结果，乱序执行无法凭空跳过这条链。四个累加器版本之所以可能更快，不是因为它“高级”，而是把一条长依赖拆成多条短依赖，让核心在等待一个结果时还能推进其他结果。这个例子也提醒读者不要把优化写成玄学：多累加器有寄存器压力，循环体变大，尾部处理也更复杂。它适合依赖链是瓶颈的场景，不适合已经被内存带宽限制的版本。

第三条因果链是分支。一个分支是否昂贵，取决于输入分布。全真、全假或周期性输入，预测器可能很快学会；随机输入会让预测失败。把分支改成算术掩码可以减少 mispredict，但可能增加指令数和寄存器压力。经典教材的舒服之处在于不把技巧绝对化，而是给出判断条件：先写 baseline，再构造不同分布的输入，再观察分支事件和总时间是否同向变化。若随机输入提速而全真输入变慢，说明技巧只在某类 workload 上成立。

第四条因果链是内存级并行。连续数组扫描能让预取器和乱序窗口提前工作；指针链表的下一次地址依赖当前节点，核心难以同时挂起足够多的请求。两个算法都可能是线性的，但等待形状完全不同。这里要把第一册的数据结构知识放到硬件里解释：`std::vector` 的连续性不只是语法便利，它让硬件看到规律；`std::list` 的插删稳定性也不是免费，它把遍历变成一串依赖加载。

实验设计可以围绕一个热循环证据包展开。第一组是单累加器、四累加器和向量化前的标量版本；第二组是可预测分支、随机分支和 branchless 版本；第三组是连续数组、索引数组和指针链。每组实验都要保留输入生成器和反汇编摘要。报告不要只写“某版本快百分之多少”，而要写“旧版本慢在什么因果链，新版本解除了哪条限制，瓶颈是否迁移到加载、带宽或前端”。这样读者才会形成微架构思维，而不是收集优化偏方。

源码阅读也要服务热循环。`perfrecord` 找热点，`perfanotate` 或 `objdump` 看循环，编译器优化报告看向量化失败原因。阅读内核 perf 入口时，只需明白事件采样如何把用户态地址和符号联系起来，不必在本章钻进所有 PMU 分支。高质量的本章结尾应让读者能面对一段 C++ 循环说出：它的输入分布是什么，热代码长什么样，关键依赖在哪里，证据能证明到哪一步。

案例推导：解析器热循环为什么不能只靠开线程解决

日志解析器常见旧实现是逐字符扫描，每遇到分隔符就进入一个分支，字段结束后调用一个小函数处理。小输入下这段代码很清楚；大输入上，profile 显示 parser 占主要时间。初学者可能立刻开更多线程，但如果单线程热循环被分支预测和依赖链限制，开线程只会把低效内核复制到更多核心，还可能更快撞上内存带宽。正确顺序是先把单核路径看清。

第一步看输入分布。若日志格式稳定，分隔符位置规律，分支预测可能很好，瓶颈未必在分支；若字段长度随机、错误行很多，预测失败会增多。第二步看循环体。每次处理一个字符都调用函数，函数无法内联时，前端供给和调用开销可能超过字符比较本身。第三步看依赖。若解析状态机每次都依赖上一个字符状态，编译器很难自动向量化。报告要把这些观察写成热循环证据包，而不是只贴一张火焰图。

优化可以按层次推进。先把冷错误路径移出热循环，让正常行走短路径；再把字段处理改成批

量接口，减少每字符调用；再尝试用多累加器或查表减少依赖；最后才讨论 SIMD 字节扫描。每一步都要保留 reference，因为解析器最容易在边界上出错：空字段、行尾无换行、连续分隔符、非法字节、超长字段、整数溢出。性能优化如果改变了错误分类，后续系统账本会全部失真。

反汇编阅读要具体。找到热循环后，标出加载字符、比较分隔符、条件跳转、状态更新和回边。如果优化后源码更复杂但机器码更短，说明抽象边界让编译器看清了模式；如果源码更短但机器码出现间接调用和大块冷路径，说明抽象放错了位置。这个案例的结论不是“手写解析器一定快”，而是“热路径要让硬件看见稳定模式，冷路径要让维护者看见清晰语义”。

实验：写两个求和版本。第一个使用单累加器，第二个使用四个独立累加器。用 `perfstat` 比较 cycles、instructions 和 IPC。再构造一个带随机分支的版本，观察 branch-misses 对时间的影响。

从一个解析热循环推导流水线问题 考虑贯穿项目中的日志解析循环。朴素写法通常是一边扫描字符，一边根据分隔符、数字、转义和字段位置更新状态。它看起来是普通业务代码，实际上会直接撞上前端取指、分支预测、乱序执行窗口和执行端口。若每个字符都进入一个复杂的 switch，分支预测器需要从输入分布里猜下一条路径；若每个字段都调用一个不可内联函数，前端会承受更多调用和返回；若每次解析都更新同一个统计对象，后端会形成长依赖链。微架构概念不是抽象背景，它们都能在这个循环中找到原因。

第一步要把循环拆成可观察事件：读取字节、判断类别、推进状态、写出字段、处理错误。读取字节是连续内存访问，通常容易被硬件预取；判断类别若写成多层 if，会产生分支；推进状态依赖上一个字符，形成循环携带依赖；写出字段可能触发分配或哈希；错误路径若和正常路径混在一起，会污染指令缓存和预测。这样拆开后，读者才能理解为什么一个“只扫字符串”的函数可能慢。

分支预测不是猜对率一个数字 分支预测要结合输入分布解释。若日志格式稳定，字段分隔符位置规律，某些分支会高度可预测；若用户字段包含任意文本，字符类别随机，预测失败会增多。预测失败的成本不只是一次分支本身，而是错误路径上已经进入流水线的指令被丢弃，前端重新取正确路径，后端可执行工作暂时减少。乱序执行能隐藏部分延迟，但不能跨过真正的数据依赖和控制依赖。

优化分支时不要只追求“少一个 if”。有时把错误路径拆到冷函数，让正常路径更紧凑，比把所有逻辑写成无分支表达式更好。有时把字符分类改成查表，能把多分支变成一次数组访问，但查表也会引入内存访问和缓存问题。有时分支高度可预测，强行改成掩码反而做了更多无用计算。实验要比较不同输入：规则日志、随机字段、错误密集输入和短行输入。只有一种输入得出的结论，不能指导系统设计。

乱序执行能隐藏什么，不能隐藏什么 乱序执行依赖两个条件：指令之间有足够独立性，窗口里有足够可调度工作。单累加器、链式哈希、逐字符状态机都会缩短可并行调度空间。把一个全局计数拆成多个局部计数器，把一个长状态机拆成先扫描结构位置再解析字段，把一个单元循环展开成多个独立累加器，都是在给乱序执行喂独立工作。这里的目标不是让代码看起来复杂，而是打断不必要的依赖链。

但乱序执行不能消除内存层级的根本问题。若每次循环都追一个随机指针，load miss 会让后续依赖指令等待；若任务访问远端 NUMA 内存，延迟更高；若分支预测频繁失败，窗口里装进来的指令本身就是错的。微架构优化必须和第 3、4 章一起读：流水线决定单核怎样消费指令，缓存和 TLB 决定数据能否及时送到，NUMA 决定访问是否跨拓扑。

实验：把热循环拆成四个版本 本章可以安排一个解析字段计数实验。版本一是直观 switch 状态机，作为 reference。版本二把字符类别判断改成小查表，观察分支和缓存变化。版本三把错误处理移出热路径，正常路径只记录错误标志，循环后统一处理。版本四把输入按块处理，先找分隔符位置，再解析字段内容。每个版本都要输出同样的字段计数和错误摘要，不能只测时间。

报告中至少记录 cycles、instructions、branch misses、cache misses 和代码大小变化。若没有 perf 权限，也可以用编译器优化报告、反汇编和时间分位数替代。关键是解释因果：版本二若更快，是因为少了预测失败，还是因为查表让代码变短；版本三若更快，是因为冷路径分离，还是因为错误输入变少；版本四若更快，是因为批处理暴露了 SIMD，还是因为减少了函数调用。每个结论都要能回到指令流。

Linux 源码阅读连接点 读 Linux 调度器时，本章关心的不是把所有函数名背下来，而是看

CPU 如何获得可运行任务，任务被唤醒后怎样进入运行队列，迁移如何影响 cache 热度。调度器的快路径必须极短，因为它会频繁运行；复杂策略往往拆到较低频路径。这个设计和业务热循环相同：热路径保持规则、短、可预测，慢路径承载异常和策略。

读内核里的字符串、校验和或拷贝相关代码时，也可以看到类似思想：对齐处理、批量处理、平台特化、尾部处理和 fallback。源码阅读的收获不是“内核也这么写”，而是理解成熟代码怎样把语义、平台差异和性能路径分层。项目中的解析内核也应该这样组织：reference、portable fast path、可选平台特化和一致的测试矩阵。

事故复盘：随机字段如何击穿流水线直觉 日志解析热循环最容易给人错觉：源码只有几十行，为什么还会慢。事故从一次输入变化开始。规则日志里字段长度稳定，分支预测器很快学到路径；换成用户文本随机、引号和转义密集的日志后，同一段 switch 状态机突然 branch miss 上升，IPC 下滑，开更多线程也只是把低效循环复制到更多核心。这里不是“代码行数”变多了，而是硬件看到的控制流变乱了。

分析顺序应从输入分布进入机器码。先把字符分成普通分隔符、数字、引号、转义、错误字节，统计每类比例；再看热循环里条件跳转数量、回边、函数调用和错误路径；最后用 perf 或反汇编判断瓶颈在前端取指、分支恢复、依赖链还是内存访问。可预测分支并不坏，随机分支才让流水线不断丢弃已经取入的错误路径。Branchless 写法也不是万能，它可能减少 miss，却增加算术、加载和寄存器压力。

优化要逐步保留反例。第一版是直观状态机，作为 reference；第二版把字符类别做成小表；第三版把冷错误路径移出热循环；第四版再讨论块扫描或 SIMD 候选。每一步都用规则、随机、错误密集和短行输入测。若只拿一种输入证明“更快”，结论就会误导后续章节。微架构学习的主线是成本迁移：分支成本能换成内存成本，调用成本能换成代码大小，依赖链能换成更多局部状态。报告必须说清这次交换是否值得。

热循环结果实验判读 拿到 cycles、instructions 和 branch misses 后，不要立刻宣布优化成功。先看输出 checksum 是否和 reference 一致，再看输入分布是否覆盖规则、随机和错误密集三类。若随机输入变慢但规则输入变快，说明优化依赖预测；若指令减少但 cycles 不降，可能瓶颈迁到内存或依赖；若 IPC 提高但端到端不变，说明热循环不是主瓶颈。判读重点是把数字放回流水线因果链。

这一段也服务于复习。读者合上书后，应该能凭本章的判读口径重新审查自己的代码：有没有最小正确版本，有没有能击穿旧方案的输入，有没有状态记录，有没有资源指标，有没有失败后的恢复动作。若五个问题缺两个以上，就说明实现还停在演示层，而不是工程层。

解析器四版本对照的读法 解析器实验可以设计四个版本。第一版是直观状态机，所有分支写在一个循环里；第二版把字符类别做成小表，减少多层判断；第三版把错误路径移出热循环，只记录错误位置；第四版先扫描结构位置，再交给字段解析器。四个版本都必须输出相同字段数和错误摘要，否则性能比较没有意义。

读结果时要看输入分布。规则日志下，第一版分支可能高度可预测，查表未必明显快；随机字段下，查表和冷路径分离可能更稳；错误密集输入下，第四版可能因为频繁回退而收益下降。若只用一种输入，就会把某个版本误判成永远最好。微架构分析真正训练的是“输入形状改变了硬件看到的工作形状”。

汇编阅读不要求读者背指令名，但要看几个事实：热循环是否短，是否有不可内联调用，是否有依赖链，是否把错误处理塞进主路径，是否出现额外边界检查。把这些事实和 perf 事件对应起来，读者就能从源码、指令和测量之间建立闭环。

实验课：解析循环从直观状态机到证据包 先写一个直观状态机，逐字节扫描日志行。状态包括 normal、in quote、escape 和 error。它把字段计数、非法字符记录、行号维护都放在同一个循环里。这个版本容易审查，也适合作为 correctness reference。然后准备四类输入：字段长度固定的规则输入，字段长度随机的输入，引号和转义很多的输入，错误字符密集的输入。四类输入必须产生可校验的字段数和错误数。

第二版只做一个改动：把字符类别判断从多层 if 改成小表。第三版把错误路径移出主循环，主循环只记录第一个错误位置和错误计数。第四版先批量找分隔符和换行候选，再让标量状态机确认复杂语法。每次改动只改变一个主要因素，报告要记录 cycles、instructions、branch misses、IPC、代码大小和输出 checksum。

判读时不要只看谁最快。若查表版在规则输入上没有提升，但在随机输入上更稳，说明分支分

布是关键变量。若冷路径分离后 instructions 下降不多但 branch misses 下降,说明前端路径更干净。若候选扫描版在错误密集输入上回退很多,说明 SIMD 或批处理只适合语义规则的子问题。每个结论都要能回到流水线:前端取指、分支预测、依赖链或后端等待。

最后读一小段反汇编。不要背指令名,只标出循环体范围、条件跳转数量、内联情况和是否存在函数调用。把汇编观察和 perf 数字放在一起,形成 ParseLoopReport。后续 SIMD 章节会复用这个报告:若标量语义和错误路径没有先稳定,就不允许写平台特化内核。

热循环练习验收重点 合格答案要给出输入分布、标量 reference、优化版本和证据。只说“减少分支”不够,必须说明分支原来在哪些输入上不可预测,冷路径如何拆出,汇编或优化报告显示了什么。答案还要写出反例:某个 branchless 版本为什么在可预测输入上可能更慢。

答案还要写“为什么不采用更复杂方案”。例如没有证据证明队列是瓶颈,就不应把锁队列换成无锁;没有证明输出顺序可变,就不应随意重排;没有证明错误路径能恢复,就不应只提交正常路径。能解释不做什么,说明读者开始具备工程判断。

最后,答案必须包含可复查材料。至少有输入摘要、关键状态、期望输出、实际输出、运行命令和未验证边界。没有这些材料,老师或同伴无法判断结论是否成立。第二册的作业不是让读者写漂亮段落,而是训练读者把系统问题变成可验证证据。

项目实现:核心流水线、乱序执行与分支预测 本章对应的贯穿项目实现不要求一次写成完整框架,但必须有最小可运行切片。这个切片包含三个部分:一个 reference,一个带本章机制的实现,一个能击穿 reference 之外隐含假设的测试。Reference 用来固定语义,机制实现用来展示本章知识,测试用来证明新增复杂度确实解决了问题。

代码组织上,不要把所有内容放进一个大文件。基础类型、输入生成、核心机制、报告输出和测试应分开。即使当前只是教学项目,也要让目录能表达责任边界。这样后续章节接入时,不会因为找不到状态归属而重新发明一套对象。

每个实现都要输出一段机器可读摘要。摘要至少包含输入规模、关键状态、错误数、输出校验、耗时和未验证边界。这里的机器可读不要求复杂格式,简单的 key value 文本也可以。关键是让脚本和读者都能复查,而不是只能看屏幕截图。

测试应包含正常路径、边界路径和失败路径。正常路径证明功能可用;边界路径证明长度、容量、尾部、空输入、关闭或超时没有遗漏;失败路径证明本章机制不是装饰。若失败路径写不出来,往往说明本章概念还没有落到工程对象上。

报告最后写一个取舍段。新增机制带来了什么好处,增加了什么成本,哪些场景不应该使用它,下一章会复用哪个字段或对象。这个取舍段非常重要,因为系统工程不是把所有高级机制都加进去,而是在证据和约束下选择合适的边界。

调试笔记:热循环变慢时先保存输入分布 热循环报告必须附输入分布摘要。行长固定还是随机,分隔符密度多大,错误字符比例多少,引号和转义是否常见,都会改变分支预测和依赖形状。若优化只在固定行长上快,却在随机行长上慢,就不能写成通用结论。调试时先冻结输入,再比较版本;否则两次运行的输入差异会被误当成微架构效果。

复查清单:核心流水线、乱序执行与分支预测 复查本章时,先检查 reference 是否仍然存在。没有 reference,后面的并发、优化、异步或分布式改造都无法证明语义没有变。再检查压力输入是否尖锐:它应当攻击本章最核心的边界,而不是只把数据量放大。第三,检查状态是否可观察:关键对象的身份、状态、错误和提交结果应能在报告中找到。

第四,检查代码示例是否说明禁止行为。调用者不能做什么,往往比能做什么更能体现工程质量。第五,检查实验是否写出未验证边界;当前设备、权限或输入无法证明的结论,必须标注。第六,检查本章对象是否能被下一章复用,不能复用就说明边界还不稳。

完成这六项后,再看文字是否连贯。每个概念都应从问题推出,每个案例都应从朴素方案走到改进方案,每个实验都应能解释一个判断。若某段话放到其他章节也成立,就把它改成本章的具体对象、具体输入和具体失败。

2.8 工程案例:把日志解析热循环拆到微架构层

流水线和乱序执行不应该从一堆部件名开始讲。读者最容易理解的入口,是一个看起来普通却很容易变慢的热循环:日志解析器逐字节扫描输入,识别分隔符、数字、引号、转义和错误行。源码只是一个循环和几个分支,但机器看到的是取指、译码、分支预测、依赖链、加载、执行端口和提

交。把这个例子拆清楚，后面讲 SIMD、缓存和任务切分才有根。

一、先把热循环里的动作分开 一个解析循环至少包含五类动作。第一，加载下一个字节。第二，判断字符类别，例如数字、逗号、换行、引号或其他字符。第三，推进解析状态，例如当前在普通字段、引号字段还是转义后。第四，把字段边界或数值写到输出。第五，处理错误路径。初学者常把这五类动作看成“几行代码”，但处理器会把它们变成不同的等待形状。

加载字节如果来自连续数组，预取器容易工作；如果输入是一串小字符串指针，load 会变成随机追踪。字符类别判断如果输入很规则，分支预测容易命中；如果用户字段随机，预测失败会频繁清空流水线。解析状态依赖上一个字符，形成循环携带依赖；错误路径如果混在正常路径里，会扩大热循环的指令体积。把动作分开后，读者才能知道该看分支、依赖、前端还是内存。

二、可预测分支和随机分支不是同一个问题 分支预测不是“分支慢”四个字。一个总是为真的边界检查可能很便宜，一个随机字符类别判断可能很贵。可预测分支让前端持续供应正确路径上的指令，后端有稳定工作；随机分支让错误路径上的微操作被丢弃，流水线重新取正确路径，窗口里的可执行工作突然变少。这个成本和源码行数没有直接关系。

实验要构造三类输入。第一类是规则日志，每行字段位置相似，分支方向稳定。第二类是随机字段，字符类别接近均匀，预测器难以学习。第三类是错误密集输入，冷路径频繁进入。三类输入用同一份解析代码，记录 checksum、耗时、branch misses、instructions 和 cycles。如果没有硬件计数器权限，也要保留输入分布、编译选项和反汇编，说明当前结论的证据边界。

不要一看到分支就改成 branchless。查表分类可能减少分支，却增加一次内存访问；掩码计算可能减少控制依赖，却增加算术和寄存器压力；把错误路径延迟到循环后处理，可能比完全无分支更有效。选择取决于输入分布和瓶颈证据。

三、依赖链解释多累加器为什么有效 单累加器求和是微架构教学的最小例子。每次加法都依赖上一次结果，乱序执行看见的是一条长链。多累加器把输入分成几条独立链，最后再合并，让后端同时调度更多加法。这个优化不改变算法复杂度，却改变了指令依赖图。理解这一点后，读者才能把它迁移到解析器：不是所有状态都能拆，但某些计数、校验和、字段长度统计可以拆成独立 partial。

解析状态机里也有真依赖和假依赖。引号状态必须按字符顺序推进，这是真依赖；错误计数可以按块局部统计再合并，这是假共享状态；输出字段数量可以先局部计数，后面用 scan 分配位置。第 13 章会把这个思想推广成并行算法。本章只要求读者看懂：硬件能并行的前提是程序给出独立工作。

四、前端供给会被接口形状影响 热循环不仅受循环内部代码影响，也受接口影响。若每个字符都通过虚函数读取，前端要面对间接调用；若每次读取都检查复杂对象边界，热路径会膨胀；若错误处理和正常处理写在同一个大函数里，指令缓存和分支预测都会受影响。一个看似“抽象良好”的接口，可能让编译器无法内联或无法看清别名关系。

C++ 接口可以帮助硬件。连续输入用 `std::span<constchar>` 表达，输出字段写到预分配数组或块级 buffer，错误记录走独立冷路径；只读输入和独占输出分开，让编译器更容易优化。这里不是为了追求底层风格，而是让类型和函数边界表达热路径语义。性能设计从 API 开始，不是最后才加几条特殊指令。

五、反汇编是源码和机器之间的校准 读反汇编不是为了炫技，而是确认机器实际执行什么。编译器可能把循环展开，可能把小函数内联，可能拒绝向量化，可能因为别名保守而反复加载。热循环报告至少要保存编译器版本、优化选项、目标架构和关键汇编片段。若报告只写“源码里少了一个 if”，没有反汇编证据，就无法判断机器是否真的少了分支。

读反汇编时不要求初学者理解每条指令的所有细节。先看几个问题：循环体有多大，是否有条件跳转，是否有间接跳转，是否出现向量指令，load 和 store 数量是否符合预期，函数调用是否留在循环内。再结合 `perfannotat` 或采样地址，定位哪些指令真正热。这样读者会把微架构学习变成可复查过程。

六、Linux 工具阅读的入口 本章读 Linux 不需要深入调度器所有分支。更合适的入口是 `perfrecord`、`perfreport` 和 `perfannotat` 的使用路径：采样如何关联到指令地址，用户态符号如何解析，内核事件如何显示 cycles、branches 和 cache misses。读工具源码或文档时，目标是理解指标从哪里来、有哪些权限限制、采样会有什么误差。

在 Termux 或权限受限环境中，某些 PMU 事件可能不可用。不可用不是实验失败，而是证据边界。可以用时间、输入分布、反汇编和 checksum 形成较弱但诚实的报告。教材要训练读者写“这

个结论当前不能由硬件计数器验证”，而不是把不可观测的猜测写成事实。

七、本章实验交付 本章的项目交付是一份 **HotKernelReport**。它包含四个解析器版本：朴素 switch、查表分类、冷错误路径拆分、块级扫描。每个版本使用同一输入集，输出同一 checksum 和错误摘要。报告解释每个版本改变了什么：分支形态、依赖链、前端体积、load 形状还是错误路径。

验收时不要只看谁最快。若查表版本在规则输入上变慢，在随机输入上变快，这正是好结果，因为它说明输入分布决定优化收益。若冷路径拆分只在错误稀少时有效，也要写清。微架构学习的目标是解释差异，而不是记住某个永远正确的优化招式。

2.9 补强案例：解析器为什么要拆正常路径和错误路径

很多热循环慢，不是因为正常逻辑复杂，而是因为错误处理、诊断构造、边界检查和日志格式化混在正常路径里。处理器每次取指和预测分支时，都要面对这些低频分支。高质量工程代码不是不处理错误，而是让错误路径完整但不污染热路径。

一、错误路径的成本有三层 第一层是指令体积。错误处理代码放在同一个函数里，会让热函数更大，影响指令缓存和前端供给。第二层是分支形状。错误分支虽然少发生，但如果条件复杂或和正常分支交织，会影响预测器学习。第三层是副作用。错误路径常分配字符串、写日志、构造异常对象，这些动作不应出现在每个字符扫描的内层循环里。

拆分错误路径不等于忽略错误。热循环可以只记录错误码、位置和少量上下文，循环结束或块结束后再构造完整错误对象。这样正常路径保持短，错误信息仍然完整。关键是错误摘要要足以重建诊断：record id、byte offset、错误类型、字段编号和少量上下文范围。

二、从状态机拆出快速路径 CSV 或日志解析器常有普通字段、引号字段、转义状态和错误状态。若每个字符都走一个大 switch，状态转移清楚但前端压力大。可以先识别最常见的普通字段路径：没有引号、没有转义、字符类别稳定。普通路径快速扫描到分隔符或换行；遇到引号、转义或异常字符时，跳到慢路径状态机。

这个设计要有严格测试。快路径不能改变语义，遇到复杂输入必须回落慢路径。测试要覆盖普通行、引号行、转义行、错误行和跨块边界。快路径的价值来自输入分布，若真实日志大多复杂，收益会下降。报告应写明普通路径命中率，而不是只写总时间。

三、冷函数和内联边界 编译器可以通过函数边界和属性把冷路径移出热代码。即使不用平台特定属性，手动把错误构造放进单独函数也能帮助阅读和优化。热函数只返回紧凑状态，调用者在外层判断是否进入冷函数。这样反汇编里热循环更短，采样也更容易定位。

但拆函数不能破坏语义。错误对象需要的上下文必须在热路径保存；不能为了避免拷贝而返回指向临时 buffer 的 view；不能在冷函数里读取已经被复用的输入块。微架构优化仍然受生命周期约束。

四、实验设计 准备三组输入：错误率接近零，错误率百分之一，错误率百分之十。比较大 switch 版本、快慢路径拆分版本、查表分类版本。记录循环体大小、branch misses、instructions、checksum、错误摘要是否一致。若错误率升高后拆分版本收益下降，这是合理结果；它说明冷路径不再冷。报告应把这种转折写出来。

2.10 补充案例：指令级并行和源码重构

乱序执行能利用独立指令，但源码必须给它独立性。很多性能重构不是为了少写几行，而是为了把依赖拆开，让硬件能同时推进更多工作。本节从几个常见源码重构解释 ILP。

一、多累加器是最小重构 单累加器把所有加法串成一条链。四累加器把链拆成四条，最后合并。它没有减少读取字节，也没有改变结果语义，却让后端有更多可发射操作。这个例子说明源码重构可以改变依赖图，而不仅是改变指令数。

同样思想可以用于校验和、计数和简单 hash。只要操作满足合并合同，就可以局部计算后合并。若操作有顺序依赖，例如加密链或状态机，就不能强行拆。ILP 优化仍然受语义约束。

二、分离地址计算和数据使用 某些循环先计算下一批地址，再使用当前批数据，可以增加内存级并行度。链表追踪很难做到，因为下一地址依赖当前节点；数组索引或哈希探测有时可以提前算多个候选地址。软件预取也是这个思路的一种，但预取距离需要测量。

源码上，这可能表现为分块处理：先收集一块记录的 key 或 offset，再批量访问对应表。这样增加了临时数组，却给硬件更多独立 load。是否值得，取决于随机访问延迟和临时数组成本。

三、减少不可预测控制依赖 控制依赖会限制前端供给。把多分支分类改成查表、把错误路径移出热循环、把小状态机拆成常见快路径和慢路径，都能减少不可预测控制依赖。不要把这些重构理解成“代码更绕”，它们是在把常见路径呈现给硬件。

四、重构验收 每次 ILP 重构都要证明三件事：结果和 reference 一致，反汇编中的依赖形状确实变化，性能收益出现在预期输入上。若只满足第一条，可能只是无效重构；若只满足第三条，没有语义证明，风险太高。三条都满足，才是可保留优化。

2.11 从热循环现象推回微架构假设

很多读者第一次看流水线、乱序执行和分支预测时，会把它们当成处理器内部的名词表：取指、译码、发射、执行、退休、预测、回滚。这样的记忆没有错，却很难指导优化。真正可用的学习方法是从一个热循环的异常现象往回推：为什么同样是遍历数组，有的循环接近内存带宽，有的循环连一个核心的整数吞吐都吃不满；为什么把一个条件分支改成查表，有时变快，有时变慢；为什么手写展开以后，二进制更大，速度却不一定更好。微架构不是凭空出现的概念，而是处理器为了让大量指令持续进入执行单元而建立的一套供给、调度和恢复机制。

一、先把源码变成指令流 热循环的第一层证据不是源码，而是反汇编和动态指令计数。源码中的一行可能展开成多条 load、store、整数运算、条件跳转和地址计算；一个看似简单的迭代器循环也可能包含边界检查、函数调用或不可内联的比较器。读者应先写出循环体中的关键指令：每次迭代读几个地址，写几个地址，是否有乘法、除法或分支，地址是否由前一次结果决定。只有把“这一行代码很简单”改成“每个元素需要三次 load、一次条件跳转和一次依赖加法”，后面的流水线分析才有入口。

二、再区分供给不足和执行不足 同样慢，原因可能完全不同。前端供给不足时，问题出在取指、解码、微操作缓存、指令缓存或分支回滚；后端执行不足时，问题出在执行端口、load/store 单元、访存延迟、除法单元或依赖链。一个常见错误是看到 IPC 低就说“CPU 没跑满”。更精确的问法是：前端每周期能送出多少微操作，后端有多少微操作因为等待数据而不能发射，退休端是否被长延迟操作堵住。性能计数器不能当成绝对真理，但可以帮助排除方向。若分支错误率高，先看输入分布和分支形状；若后端绑定明显，先看依赖链和内存层级；若指令缓存压力大，先看代码体积和内联边界。

三、依赖链解释为什么单累加器慢 求和循环是最适合教学的例子。单个累加器把每次加法串成一条长链，下一次加法必须等上一次结果。乱序执行可以把独立 load 提前，却不能改变同一个寄存器上的真依赖。多个累加器不是“玄学展开”，而是把一条长链拆成几条短链，让处理器在同一时刻安排多条独立加法。这里的优化从问题本身推导出来：瓶颈不是加法指令数量，而是关键路径长度。最后的合并又把多个部分结果收回一个值，所以读者也要看到代价：寄存器压力增加、尾部处理增加、浮点求和顺序改变。只有把收益和代价同时写清，展开才是工程选择而不是手法堆叠。

四、分支预测来自时间局部性假设 预测器的根本假设是过去能提示未来。循环分支通常稳定，错误率低；随机输入上的条件分支没有可学习模式，错误率高；按字段排序后再处理，分支可能突然变得友好。于是优化路径也从输入出发，而不是从“分支一定慢”出发。若条件非常便宜且可预测，保留分支往往最好；若条件随机且两边工作量接近，可以考虑掩码、查表或批量分组；若一边是少见错误路径，应把冷路径移出主循环，减少前端压力。每一种改法都改变了指令流、数据访问和代码布局，必须用同一组输入比较。

五、实验报告要能反驳自己 本章实验不能只写“优化后快了”。报告至少要包含三组输入：顺序友好的输入、随机输入和最坏输入；至少要包含两个二进制证据：编译选项和反汇编片段；至少要说明一个未验证边界：当前机器、当前编译器、当前数据规模。若多累加器版本变快，要说明它降低了关键路径还是只是改变了内存访问；若分支消除版本变快，要说明它是否增加了额外 load；若冷热路径分离变快，要说明是否牺牲了错误路径延迟。微架构学习的目标不是背会每个部件名称，而是能把热循环的现象还原成可验证的假设。

2.12 贯穿案例：解析器从快到慢的完整推导

把流水线讲清楚，最好不要从“现代处理器有多少级流水线”开始，而要从一个读者真的会遇到的问题开始。假设日志解析器在规则输入上很快：每行都有固定字段，分隔符稳定，错误行极少。后来线上接入用户自定义字段，行长变化大，字段缺失、引号、转义和非法字节都多了。同一份解析代码没有改，吞吐却下降，尾延迟也变高。这个现象比一张流水线图更适合做入口，因为它逼迫我们把源码、输入分布和硬件执行连起来。

第一步不要急着优化，而是把一个循环迭代拆成处理器看到的工作。每个字符进入循环，程序要读取字节，判断类别，更新状态，可能写入字段边界，可能进入错误路径。规则输入下，状态转移高度稳定，分支预测器能从历史中学到下一次更可能走哪条边；错误密集输入下，路径不再稳定，预测失败会清空部分流水线，前端已经取来和译码的工作被丢弃。吞吐下降不是因为“分支”这个词本身有罪，而是因为输入破坏了预测器赖以成立的时间局部性。

一、先保留朴素状态机作为语义锚点 朴素状态机通常最容易审查。它按字符推进状态，遇到分隔符结束字段，遇到引号进入 `quoted` 状态，遇到非法字节进入错误记录。优化前必须保留它作为 `reference`，因为后面的查表、批量扫描、冷热路径分离都可能悄悄改变边界行为。比如非法 UTF 字节是否计入字段长度，空字段是否保留，引号未闭合时是否整行丢弃，这些都不是性能问题，而是语义合同。没有 `reference`，任何微架构优化都可能用漏处理换速度。

二、把分支失败和状态语义分开 状态机慢，不等于状态机语义错。我们先构造四类输入：固定格式、随机分隔符、错误密集、超长字段。固定格式下，`switch` 分支可能非常稳定；随机分隔符下，分类分支更难预测；错误密集输入把冷路径拉进热循环；超长字段可能把写出和边界检查变成主成本。不同输入让不同硬件机制暴露。若只跑一种输入，报告会把“这批数据上的现象”误写成“处理器规律”。

三、查表优化从失败分支推导出来 当字符类别分支随机时，可以把多个条件判断改成类别表：每个字节映射到字母、数字、分隔符、引号、非法等类别。这样控制流减少，但多了一次 `table load`。若表常驻 L1，收益可能明显；若表访问引入额外地址计算，或类别组合后仍有大 `switch`，收益会变小。查表不是魔法，它只是把一部分控制依赖换成数据依赖。这个交换是否划算，要看输入随机性、表大小、寄存器压力和后续状态转移。

四、批量扫描从前端供给推导出来 若每个字符都走完整状态机，前端需要不断提供类似的小分支。对于普通未引用字段，可以先用批量扫描找到下一个特殊字符，再进入慢路径处理引号、转义和非法字节。这个结构的推导很自然：大多数普通字节不需要逐个复杂判断，只有特殊字节会改变解析状态。批量扫描让热路径更像连续内存搜索，慢路径保留完整语义。它也有代价：代码更复杂，短行上启动成本可能不划算，错误密集输入会频繁退回慢路径。

五、展开循环要从依赖图判断 循环展开常被误用。展开四次不自动快，因为它可能增大代码体积、增加寄存器压力、让错误路径复制多份。它在什么场景下有意义？当每个迭代之间有足够独立工作，且分支结构能被简化时，展开可以减少分支频率并提供更多独立 `load`；当下一步状态强依赖前一步状态时，展开对关键路径帮助有限。解析器有些阶段适合展开，例如扫描普通字节；有些阶段不适合，例如处理带转义的 `quoted` 状态。优化要按状态分，不要把整个解析器粗暴展开。

六、用反汇编确认源代码意图 源代码写成查表，不代表机器码就没有分支；源代码写成 `branchless`，不代表执行端口不拥挤；源代码写成 `inline`，不代表没有生成庞大代码。报告要检查热循环反汇编：是否仍有间接调用，类别表是否每次重复加载，错误路径是否真的离开主线，展开后是否发生寄存器溢出到栈。读汇编的目标不是背指令，而是确认“我以为改了依赖图”这件事在二进制里真的发生。

七、最终结论必须回到输入分布 这次案例最后不应写“查表优化有效”或“批量扫描最快”。更好的结论是：固定格式输入下，朴素状态机已经很接近预测器友好路径；随机分隔符输入下，类别表降低分支回滚；错误密集输入下，冷路径分离比展开更关键；短行输入下，批量扫描启动成本可能吞掉收益；超长字段输入下，内存扫描和边界记录成为主成本。这样读者学到的不是一个技巧，而是从输入分布推导微架构假设的方法。

2.13 案例深化：热循环优化的反向推导

流水线章节最容易被写成硬件名词表。为了避免这种写法，可以从一个反向问题开始：优化版本为什么没有变快。假设我们把解析循环里的 `if` 改成查表，把错误路径移出循环，还展开了四次，结果规则输入快了，随机输入没变，错误密集输入反而慢了。这个现象比单纯成功更有教学价值，因为它逼迫读者把源码、输入分布和机器执行对应起来。

一、先找旧版本为什么合理 旧的 `switch` 状态机并不是低级错误。它表达语义直接，错误处理清楚，对规则输入可能高度可预测。若日志格式固定，分隔符位置稳定，分支预测器很快学到路径，前端不会频繁回滚。教材要先承认旧方案在什么条件下成立，读者才会理解优化不是审美选择，而是条件变化后的成本重排。

二、再找它在哪个输入上失败 随机用户字段、转义、引号和错误字节改变了分支历史。预测器看到的路径不再稳定，错误路径也被带进热循环。此时 `cycles` 上升不一定来自指令数量，而可能来自预测失败后的流水线清空。报告应把输入分成规则、随机、错误密集和短行四类，分别记录 `branch misses`、`instructions` 和 `IPC`。只有这样，读者才能看出“分支成本”来自哪类数据，而不是泛泛说分支慢。

三、查表和 `branchless` 都要付账 字符类别查表减少了控制流，却增加了一次内存读取。若表在 `L1` 中，成本很小；若写法导致额外边界检查或更多寄存器压力，收益会变弱。`Branchless` 写法减少 `miss`，却可能执行本来可以跳过的计算。展开循环增加独立工作，也可能扩大代码体积，给 `I-cache` 压力。每一种优化都是交换，不是免费午餐。

四、用汇编确认源码猜想 读汇编不是背指令名，而是确认热循环事实。循环体是否有调用，条件跳转是否减少，错误路径是否离开主线，展开后是否出现寄存器溢出，向量化是否真的发生。若源码看起来短，机器码却多了间接调用和额外分支，说明抽象边界放错了。若机器码更长但性能更稳，可能是换来了更少回滚或更强并行度。报告要把这些判断写成证据链。

五、把优化结果写成适用条件 最终结论不应是“查表版最好”，而应是“规则输入下旧状态机足够好，随机字段下查表更稳，错误密集输入需要冷路径设计，短行输入受调用和分发成本影响”。这种结论看起来不如口号干脆，却更接近工程。微架构章节的目标是训练读者描述适用条件，避免把一次 `benchmark` 变成永久真理。

2.14 案例复盘：解析热循环为什么突然变慢

流水线和分支预测可以从一段解析循环讲清楚。规则日志里字段位置稳定，`switch` 状态机的分支高度可预测；换成用户文本、引号、转义和错误字节混杂的输入后，`branch miss` 上升，`IPC` 下降。源码没有变，硬件看到的控制流变了。这个案例让读者明白，微架构概念不是名词，而是输入分布在机器上的表现。

旧循环不是天然错误 直观状态机语义清楚，适合作 `reference`。它的问题出现在随机分支和冷错误路径混入热循环时。优化顺序应是先记录输入分布，再看热循环汇编，最后分别尝试查表、冷路径分离和块扫描。`branchless` 写法也要付账，它可能增加指令数和寄存器压力。

实验判据 同一批输入要分规则、随机、错误密集和短行四类。报告记录 `cycles`、`instructions`、`branch misses`、`IPC`、代码大小和 `checksum`。结论不写“某版本永远最快”，而写在什么输入形状下成本从分支迁移到内存、从调用迁移到代码大小。

2.15 本章习题

习题与作业

习题一：为 `sum_baseline`、`sum_four_accumulators` 和当前项目里的 `parallel_sum` 写同一组正确性测试。输入必须包含空数组、单元数组、长度不是四的倍数的数组、包含负数的数组和足够大的随机数组。报告每个测试验证的不是“跑得快”，而是哪一种语义边界。

习题二：构造两个 `count_greater` 输入。第一个输入使分支非常可预测，第二个输入使分支接近随机。用 `perfstat` 记录 `branches` 和 `branch-misses`。若当前设备权限不足，写出你本来要运行的命令、预期现象和无法运行的原因。

习题三：把一个链表求和改写成连续数组求和。要求说明两版在源码复杂度、内存布局、cache line、TLB、预取和对象生命周期上的差异。不要只写“数组更快”，要写清为什么更可能快，以及在哪些业务语义下不能直接替换。

习题四：阅读当前项目的 `books/cpu-volume-2/labs/compute_systems/src/parallel_reduce.cpp`。指出 `parallel_sum` 的 correctness oracle、任务切分方式、线程创建成本、partial 写入位置和可能的 false sharing 风险。给出一个改进计划，但不要直接删除 baseline。

第 3 章

缓存、TLB 与页级性能

先看一个反常结果：两个程序都处理一千万条记录，理论复杂度都是线性扫描，甚至第二个程序少做了几次比较，但它更慢。继续看代码才发现，第一版把记录放在连续数组中，热路径顺序读取少数字段；第二版为了“对象更清晰”，把每条记录拆成多个堆对象，字段通过指针互相跳转。算法复杂度没有变，地址序列变了。核心不再连续拿 cache line，TLB 要覆盖更多页，预取器也很难提前猜到下一次访问。

本章从这种地址序列的变化出发讲缓存和 TLB。缓存不是抽象的“快内存”，它按 cache line 搬运数据；TLB 不是背景细节，它决定虚拟地址到物理地址翻译是否能被快速命中；页面首次触碰、映射关系和写共享会把同一个源码循环变成完全不同的硬件行为。读者要学会先画出地址访问顺序，再谈缓存层级、page walk、首次触页和局部性优化。

3.1 从地址访问到缓存层级

缓存不是按单个字节移动数据，而是按 cache line 移动。常见 cache line 大小是 64 字节。顺序访问数组时，一次 miss 可以带来后续多个元素；随机访问链表时，每个节点可能引入一次新的 miss。这个差异解释了为什么连续数组常常比指针结构更快。

缓存有容量、相联度和替换策略。容量不足会产生容量 miss，相联度不足会产生冲突 miss，多核心共享写会产生一致性流量。一个算法理论复杂度更低，不代表真实更快；如果它把连续访问变成随机访问，可能输给复杂度略高但局部性更好的算法。

从第二册的角度看，cache line 还是多核协作的最小一致性单元。两个线程哪怕写的是两个不同字段，只要字段落在同一条 cache line 上，硬件就必须在核心之间移动同一个一致性单元。很多并发程序的扩展性问题，不是算法复杂度问题，而是写入布局问题。后面讲锁、原子和无锁队列时，都要回到这个事实。

```
one cache line, 64 bytes in a common x86 machine
```

```
| counter[0] | counter[1] | counter[2] | counter[3] | ...  
T0 T1 T2 T3
```

在源码里看似互不相关

```
shares one coherence unit in hardware
```

这类图要让读者形成一个直觉：源码里的变量边界，不等于硬件里的传输边界。硬件搬的是 cache line，TLB 记的是页，操作系统调度的是线程，分布式系统发送的是消息。边界错位，是系统性能问题最常见的来源之一。

缓存映射和冲突 缓存通常按 set associative 组织。一个地址会映射到某个 set，同一个 set 里只有有限个 way。如果多个热点地址恰好映射到同一个 set，而数量超过相联度，就会频繁互相驱逐，即使总工作集小于缓存容量也会慢。这类冲突 miss 很隐蔽，因为从容量角度看“不应该放不下”。

矩阵步长、数组对齐和结构体大小都可能触发冲突。若多个数组按相同大步长访问，并且基地址关系不佳，它们可能持续争用同一组 cache set。改变 padding、改变块大小或调整访问顺序，有时能显著改善性能。

Inclusive、Exclusive 和共享缓存 不同处理器的缓存层级策略不同。有的层级倾向 inclusive，也就是上层 cache 的内容也在下层保留；有的倾向 exclusive，尽量让不同层级存放不同内容；还有很多混合策略。软件通常不需要依赖某个具体策略，但要知道共享 LLC 不是无限资源。多个线程在同一 socket 上跑不同任务，可能互相驱逐 LLC。

这对并行任务调度有直接影响。把共享数据的任务放在共享缓存附近，可能复用 LLC；把互相

干扰的大流式任务放在同一 socket，可能争抢带宽和 LLC。操作系统调度器会做一般性决策，但高性能程序常需要自己记录拓扑并做亲和性实验。

3.2 共享、写入和一致性成本

读缓存可以共享，写缓存必须取得 cache line 的独占所有权。两个线程写不同变量，如果这些变量落在同一条 cache line 上，硬件仍然会把它们当成同一块一致性单元反复迁移，这就是 false sharing。它不是锁竞争，却会表现出类似的扩展性下降。

解决 false sharing 的方法包括按线程分配独立槽位、使用对齐和填充、批量合并结果、减少共享写频率。并行归约通常先让每个线程写自己的 partial sum，最后再合并，正是为了避免每次加法都写同一个共享变量。

Listing 3.1 用对齐槽位减少 false sharing

```
1 struct alignas(64) LocalCounter {
2     std::int64_t value = 0;
3 };
4
5 std::vector<LocalCounter> counters(worker_count);
6 for (std::int32_t worker = 0; worker < worker_count; ++worker) {
7     counters[static_cast<std::size_t>(worker)].value += 1;
8 }
```

这段代码不是说所有结构体都要强行 64 字节对齐，而是表达一个设计原则：如果多个线程会高频写不同槽位，这些槽位不要共享同一条 cache line。否则程序表面没有共享变量，硬件层面仍然在共享一致性单元。

false sharing 的危险在于它不像数据竞争那样直接破坏结果。程序可能完全正确，测试也全部通过，只是在核心数增加时吞吐突然停止增长。排查时要看写入频率、字段布局、线程到槽位的映射和 cache line 迁移事件。若没有 `perfctr` 或类似工具，也可以用对齐填充做对照实验：只改变布局，不改变算法。如果对齐版本显著更快，就说明共享写路径值得重点检查。

读共享和写共享 共享数据不总是坏事。只读共享表、常量配置、不可变索引和代码段可以被多个核心同时缓存，通常扩展性很好。真正昂贵的是写共享，尤其是多个核心高频写同一条 cache line。读多写少的数据可以使用读写锁、RCU、版本化快照或 copy-on-write，目的都是让读路径尽量不进入写共享热路径。

写共享也分层次。最差情况是每次操作都写同一个全局变量，例如所有线程对一个原子计数器做 `fetch_add`。较好的情况是每个线程写本地槽位，周期性合并。更好的情况是让写入天然按数据分片分散，例如按 key 分片的哈希表或按 NUMA 节点分组的内存池。布局、同步和算法在这里是同一个问题的三个面。

3.3 从虚拟地址到页表和 TLB

虚拟地址访问物理内存前需要地址翻译。TLB 缓存虚拟页到物理页的映射。工作集页数太多或访问模式太随机时，TLB miss 会增加，核心需要走页表。大页可以减少页数，提高 TLB 覆盖范围，但也会影响内存分配、NUMA 放置和碎片。

理解 TLB 对数据结构设计很重要。一个跨很多小对象的图遍历，不只会造成 cache miss，还可能造成 TLB miss。紧凑存储、池化分配和按块遍历，可以同时改善 cache 和 TLB 行为。

TLB 的特殊之处在于，它限制的是“能高效覆盖多少虚拟页”。一个程序的工作集如果只看字节数不大，但分散在大量页面上，仍然可能受到 TLB miss 影响。链表、树、哈希表桶、对象池和图邻接结构都可能出现这个问题。把对象压缩到连续数组里，不只是减少 cache miss，也是在减少活跃页数。

Page walk 的成本 TLB miss 后，硬件需要按页表层级查找映射。x86-64 常见四级或五级页表，每一级页表项本身也在内存里，虽然页表项可能被缓存，但最坏情况下一个地址翻译会触发多次内存访问。若程序随机访问大量页面，真正拖慢它的不只是数据 load，还有地址翻译。

大页把页大小从 4 KiB 扩大到 2 MiB 或更大，显著增加 TLB 覆盖范围。它适合大数组、数据库缓存和数值计算工作集，但不适合所有场景。大页可能增加内存碎片，NUMA 放置也要更谨慎。使用大页前应先使用计数器确认 TLB 确实是瓶颈。

Page walk 还会和缓存层级交织。页表项本身也要从内存层级读取，也可能命中或错过缓存。一个随机访问大数组的程序，表面上每次只读一个元素，实际上可能同时触发数据 cache miss 和页表访问。若工作集跨越大量页面，TLB miss 的成本会叠加在数据访问成本上。优化时不能只看“每个元素读几个字节”，还要看“每个时间窗口内活跃多少页”。

缺页和内存分配 分配内存不等于物理页已经准备好。操作系统通常按需分配页面，第一次触碰页面时可能发生 page fault。benchmark 如果把第一次触碰计入核心循环，会把页错误成本混入算法时间。严谨实验通常要预热、固定输入规模，并记录 page-faults。

缺页也分 minor fault 和 major fault。minor fault 不需要从磁盘读数据，可能只是建立页表映射；major fault 需要从存储设备读取，成本高得多。高性能计算通常要避免在热路径出现 major fault，也要避免把大量 minor fault 误判为算法本身慢。

第一册已经要求 benchmark 避免把初始化混进热路径。第二册要进一步要求初始化方式和计算方式一致。NUMA 机器上，如果主线程单独触碰全部页面，再让多个 socket 上的 worker 计算，页面可能集中在主线程所在节点，后续计算变成远端内存访问。正确实验通常让每个 worker 初始化自己之后要处理的分块，这样 page fault、物理页分配和计算拓扑保持一致。

3.4 文件、页缓存和持久化

mmap 可以把文件或匿名内存映射到进程地址空间。它让文件 I/O 看起来像内存访问，但不意味着没有 I/O 成本。第一次访问未驻留页面时仍然会缺页，顺序扫描大文件时内核预读可能有效，随机访问时可能产生大量 page fault。

数据库、搜索引擎和日志系统常利用 **mmap** 或 page cache，但必须理解内核何时回收页面、何时写回脏页、何时产生 I/O 抖动。系统工程里，虚拟内存既是便利抽象，也是性能变量。

Linux 源码阅读入口 第一册的 Linux 源码阅读只要求读者能从用户态现象找到入口。第二册要读出数据结构和性能路径。围绕本章，可以固定一个 Linux 版本，沿下面几条路径做窄范围阅读：页错误入口可从体系结构相关的 fault handler 进入，再走到通用内存管理逻辑；VMA 管理可关注 **mm/mmap.c** 和相关 maple tree 结构；页表和 fault 处理可关注 **mm/memory.c**；NUMA 策略可关注 **mm/mempolicy.c**；page cache 和文件映射可从 **mm/filemap.c** 开始。

源码阅读报告不要写成“Linux 使用某某机制”这么宽泛。合格报告要写清内核版本、阅读路径、关键结构、用户态实验和结论边界。例如研究首次触页，可以写一个大数组实验，记录 page-faults，再阅读 fault 路径中如何建立匿名页映射。若当前设备权限不足，仍然可以保留源码阅读，但必须承认它不是当前运行内核的直接证据。

3.5 内存实验与排查方法

缓存和 TLB 优化可以压缩成四条原则。第一，让热路径连续访问，减少随机指针跳转。第二，让共享写分散，避免多个线程争用同一条 cache line。第三，让工作集按块复用，避免超出缓存后仍反复访问。第四，让页面放置和线程执行位置一致，避免首次触页造成远端访问。这四条原则会贯穿后面的数据布局、并行归约、线程池、工作窃取和分布式 shuffle。

常见误区

不要把 **std::vector** 的连续性理解成所有成本都消失了。连续布局改善 cache line 和预取，但如果容量超过缓存，仍然会受内存带宽限制；如果首次访问大量新页，仍然会看到缺页成本。

从变量边界到硬件边界 本章最重要的模型，是把软件边界和硬件边界分开。C++ 源码里，**counter[0]** 和 **counter[1]** 是两个不同元素；硬件里，如果它们落在同一条 cache line 上，写入时就是同一个一致性单元。C++ 里，两个对象可能由两个 **new** 表达式产生；分配器里，它们可能挨在同一个页上，也可能散落到不同页。进程里，虚拟地址看起来连续；物理页可能分布在不同 NUMA 节点。性能问题经常不是因为某一层“错了”，而是因为层与层的边界没有对齐。

因此分析内存性能时，不要只画类图和对对象关系，还要画四种边界。第一是对象边界，说明字段和容器怎样组织。第二是 cache line 边界，说明哪些字段会一起被搬运和一致性维护。第三是页边界，说明工作集跨越多少页，TLB 是否能覆盖。第四是 NUMA 边界，说明页面在哪个节点，线程在哪个节点执行。只有对象图没有硬件边界，分析会停留在源码层；只有硬件术语没有对象图，优化又会脱离业务语义。

Compute Systems Engine 的 partial 数组就是一个很好的例子。源码里每个 worker 写一个 partial，逻辑上互不共享；如果每个 partial 只是一个 `std::int64_t`，八个 partial 可能正好落在同一条 64 字节 cache line 上。八个 worker 写不同元素，却让同一条 cache line 在核心之间迁移。若给每个 partial 做 `alignas(64)`，源码上的数组变胖了，内存占用增加了，但写共享路径被拆开。这里没有绝对答案，只有工作负载下的权衡：若 worker 很少或写入很少，padding 可能不值得；若每个 worker 高频写 partial，对齐常常值得。

局部性不是一个词 局部性至少有三种。时间局部性表示刚访问过的数据很可能很快再次访问；空间局部性表示访问一个地址后，附近地址很可能被访问；结构局部性表示程序的数据结构让相关数据天然挨在一起。很多教材只讲时间和空间局部性，但工程中结构局部性更容易被设计改变。

数组顺序扫描空间局部性很好，时间局部性可能不强，因为每个元素只用一次。矩阵分块同时制造时间局部性和空间局部性，因为一个块中的数据会被多次复用。哈希表查询如果 key 随机，空间局部性差；若桶数组连续但节点链表分散，第一跳和后续跳的局部性还不同。图算法更复杂，邻接表可能连续，但顶点状态数组访问可能随机。说“这个算法局部性不好”太粗，应说明是哪一类局部性不好，坏在数据结构的哪个位置。

局部性还要和读写性质一起看。只读随机访问通常只是慢，写随机共享访问可能同时慢且影响其他核心。只读大表可以被多个核心缓存；写热点计数器会触发一致性迁移；写不同字段如果落在同一 cache line 上，会形成 false sharing。后续讲无锁队列时，head 和 tail 的布局就是典型局部性问题：把它们放近可以减少对象体积，但若生产者写 tail、消费者写 head，放太近反而让两端互相干扰。

案例：三种计数器的内存路径 为了把缓存一致性讲清楚，可以比较三个计数器版本。第一个版本用 mutex 保护全局计数器，第二个版本用 atomic 全局计数器，第三个版本用分片计数器。三者都能给出正确总数，但内存路径完全不同。

Listing 3.2 baseline: mutex counter 表达最清楚的不变量

```
1 class MutexCounter {
2 public:
3     void add(std::int64_t delta) {
4         std::lock_guard<std::mutex> lock(this->mutex_);
5         this->value_ += delta;
6     }
7
8     std::int64_t value() const {
9         std::lock_guard<std::mutex> lock(this->mutex_);
10        return this->value_;
11    }
12
13 private:
14     mutable std::mutex mutex_;
15     std::int64_t value_ = 0;
16 };
```

mutex 版本的语义最直接：锁保护 `value_`。低竞争时，它可能只走用户态 fast path；竞争上升后，线程可能进入 futex 等待，调度器和唤醒成本会进入系统。它的瓶颈通常是临界区串行化和锁等待，不只是那一次整数加法。

Listing 3.3 改进但不一定扩展：atomic counter

```

1 class AtomicCounter {
2 public:
3     void add(std::int64_t delta) {
4         this->value_.fetch_add(delta, std::memory_order_relaxed);
5     }
6
7     std::int64_t value() const {
8         return this->value_.load(std::memory_order_relaxed);
9     }
10
11 private:
12     std::atomic<std::int64_t> value_{0};
13 };

```

atomic 版本消除了锁等待和内核睡眠路径，但所有线程仍写同一 cache line。低竞争时它可能比 mutex 快；高竞争时它可能因为 cache line 所有权迁移而扩展性很差。这里的 relaxed 只说明不需要用计数器同步其他数据，不说明这次写没有硬件成本。

Listing 3.4 扩展方向：按 worker 分片计数

```

1 struct alignas(64) CounterShard {
2     std::int64_t value = 0;
3 };
4
5 class ShardedCounter {
6 public:
7     explicit ShardedCounter(std::int32_t shard_count)
8         : shards_(static_cast<std::size_t>(shard_count)) {}
9
10    void add(std::int32_t shard, std::int64_t delta) {
11        this->shards_[static_cast<std::size_t>(shard)].value += delta;
12    }
13
14    std::int64_t value() const {
15        std::int64_t total = 0;
16        for (const CounterShard& shard : this->shards_) {
17            total += shard.value;
18        }
19        return total;
20    }
21
22 private:
23     std::vector<CounterShard> shards_;
24 };

```

分片版本把高频写入分散到多个 cache line。它牺牲了读取总数的成本和实时性：每次读取总数要扫描所有分片；如果读取和写入并发，还需要额外同步或定义弱一致语义。它适合写多读少的统计路径，不适合每次操作都需要立即得到全局精确值的协议。这个案例说明高性能设计常常不是“换一个更快原语”，而是改变读写比例和共享形状。

false sharing 的对照实验 false sharing 很适合用对照实验学习，因为它不改变算法结果，只改变布局。实验可以构造两个版本。版本一使用紧凑数组，每个线程反复写自己的槽位；版本二使

用 `alignas(64)` 的槽位。两个版本的循环次数、线程数和写入次数完全相同，只改变槽位是否共享 cache line。如果对齐版本在多线程下显著更快，而单线程差异不大，就说明写共享边界是关键变量。

实验报告要避免两个误区。第一，不要只跑一个线程数。false sharing 的危害随线程数和核心拓扑变化，至少应从 1 线程跑到物理核心数。第二，不要只看耗时。若工具允许，应观察 cache line 迁移或相关事件；若没有权限，也应写出对照设计为什么能隔离变量。第三，不要把 padding 当成默认答案。对齐会增加内存占用，可能降低缓存容量利用率。只有高频并发写的槽位才优先考虑 padding。

TLB 覆盖范围和数据结构 很多人把 TLB miss 当成“虚拟内存细节”，但它会直接改变数据结构选择。假设页面大小是 4 KiB，一个数据结构每个节点只有几十字节，但节点随机分布在几百万个页面上。每次访问节点不仅可能 cache miss，还可能 TLB miss。TLB 覆盖范围是有限的，活跃页数超过覆盖范围后，地址翻译本身就会变成瓶颈。

紧凑数组同时改善两件事：一条 cache line 带来多个相邻元素，一个 TLB 项覆盖同一页内多个元素。对象池也有类似作用：即使保留指针结构，至少让节点来自较少的连续页面。图算法中的 CSR 格式把邻接边压成连续数组，不只是为了少分配，也是为了提高 cache 和 TLB 友好性。哈希表开放寻址常常比链地址法更 cache 友好，也部分因为它减少指针跳转和页面分散。

大页是另一种提高 TLB 覆盖范围的方法。把 4 KiB 页换成 2 MiB 页，同样数量的 TLB 项能覆盖更大地址范围。它适合大数组和长时间驻留的大工作集，但要考虑碎片、权限、部署和 NUMA 放置。不要一看到 TLB miss 就无脑开大页。正确顺序是先证明活跃页数和 TLB miss 是瓶颈，再评估大页对分配、首次触页和系统配置的影响。

首次触页和 NUMA 的隐藏变量 NUMA 机器上，页面通常在首次写入时分配到触碰它的线程所在节点。若主线程分配并初始化整个数组，然后把计算分给其他 socket 上的线程，其他线程可能大量访问远端内存。程序源码里看不到这个问题，因为数组地址连续、线程切分也正确；硬件拓扑里却出现远端访问。

这就是为什么 benchmark 的初始化也属于实验设计。若计算阶段按 worker 分块，那么初始化阶段也应按同样分块让 worker 触碰自己的页面。这样 page fault、物理页分配和计算线程位置更一致。若为了方便只让主线程初始化，测到的可能不是算法性能，而是错误页面放置后的远端带宽。

Compute Systems Engine 后续要把输入生成拆成两种模式。第一种是 single-touch，由主线程初始化，用于展示错误实验设计可能带来的远端访问。第二种是 parallel first-touch，由每个 worker 初始化自己负责的块，用于展示正确放置。两者结果如果在单 socket 机器上接近，在多 socket 机器上可能差异明显。教材不能假设每个读者都有 NUMA 设备，但要让读者知道该怎样设计实验。

矩阵访问：从行列顺序到分块 矩阵是讲缓存最经典的案例。行优先存储的二维数组按行扫描，连续访问；按列扫描，步长等于一行长度，若矩阵很大，每次访问都可能落到新的 cache line 甚至新的页。这个例子不能只停在“行优先快”上，还要继续推进到分块。

矩阵乘法中，朴素三重循环会反复访问矩阵块。如果循环顺序不合适，某些数据刚被加载就被驱逐，下次又要从更慢层级取回。分块的思想是让一小块数据在缓存中被多次复用，提高算术强度。块太小，循环开销和复用不足；块太大，超过缓存后又失效。块大小还与 cache 容量、相联度、TLB、SIMD 宽度和数据类型有关。

这个案例的意义不是要求第二册变成矩阵计算专著，而是让读者学会一类推理：先估算工作集大小，再估算每个块的数据复用，再根据缓存层级选择实验矩阵。后面讲分布式 shuffle 时，思想相同：先本地聚合减少网络数据，再按分区批量发送，避免每条记录都独立跨网络移动。cache block 和 network batch 在不同尺度上解决同一个问题：让昂贵移动服务更多有用计算。

分配器和对象布局 C++ 程序常把性能问题归咎于容器，却忽略分配器。频繁小对象分配会让对象分散、增加元数据开销、破坏 cache 和 TLB 局部性，还可能引入锁竞争。多线程分配器通常有线程本地缓存、arena 或 tcache 机制，用于减少全局锁，但也会影响对象在哪个线程、哪个 NUMA 节点上实际分配。

对象池不是万能答案。它可以改善局部性和分配成本，但也会引入生命周期管理、碎片复用、调试困难和峰值内存占用问题。如果对象跨线程释放，池的所有权和回收路径又会变复杂。无锁数据结构尤其要小心：节点从结构中摘除，不等于可以立即释放；分配器复用同一地址还可能放大 ABA 问题。缓存、TLB、分配器和内存回收最终会在无锁章节汇合。

写回、脏页和 I/O 误判 并不是所有内存成本都来自 DRAM。文件映射、page cache 和脏页

写回会把内存访问和 I/O 混在一起。一个程序用 `mmap` 扫描文件，第一次访问可能触发缺页和磁盘读取；写入映射文件后，脏页可能稍后由内核写回，导致延迟在看似无关的时刻出现。若 benchmark 只测用户态循环，不记录 page faults 和 I/O，就可能把系统行为误判为算法波动。

日志系统和数据库特别容易遇到这个问题。用户写入内存缓冲并不等于数据已经安全落盘；调用 `fsync` 又会把持久化成本集中暴露出来。第二册虽然重点是计算系统，但分布式计算章节会讨论检查点和输出提交，那里必须区分“写入内存”“写入 page cache”“持久化到存储”和“对外宣布提交”。单机虚拟内存知识会直接影响分布式正确性。

Cache line 是共享和移动单位 CPU cache 不是按单个字节搬运，而是按 cache line 搬运。常见 cache line 大小是 64 字节，具体平台要查询。程序读取一个 `std::int32_t`，硬件通常会把它所在的整条 cache line 带入缓存。顺序扫描数组时，这很好，因为一条 cache line 包含多个相邻元素；随机访问时，可能每条 cache line 只用一个元素，大部分带宽被浪费。

Cache line 也是一致性协议的基本单位。两个线程写同一条 cache line 上的不同变量，也会互相争夺所有权，这就是 false sharing。软件层面看它们没有共享变量，硬件层面它们共享移动单位。理解这一点后，`alignas(64)`、结构体 padding、线程本地 partial 和 per-worker stats 的意义就很清楚：它们不是为了美观，而是为了让高频写不落在同一条 line 上。

但 padding 也有成本。一个 8 字节计数器对齐到 64 字节，内存占用变成 8 倍。若有百万个计数器，padding 会浪费大量 cache 容量和内存。只有高频跨线程写的槽位值得优先隔离。只读共享或低频写共享不一定需要 padding。布局优化始终是成本交换。

缓存相联度和冲突 miss Cache 容量不是唯一限制。多数 cache 是组相联结构，一个地址会映射到某个 set，set 中只有有限 way。如果多个热地址映射到同一个 set，超过相联度后会互相驱逐，即使总工作集小于 cache 容量，也可能 miss 很多。这叫 conflict miss。矩阵步长、哈希表大小和数组对齐都可能触发冲突。

这也是为什么实验要改变输入大小和步长。若某个规模突然变慢，可能不是算法复杂度变化，而是 cache set 冲突、TLB 覆盖范围或页边界效应。通过改变数组 padding、起始偏移、步长和容量，可以验证是否存在冲突。不要把所有拐点都简单归因于 L1、L2、L3 容量。

写分配和写回 写入内存也会触发 cache 行为。普通写 miss 常采用 write-allocate：先把目标 cache line 读入缓存，再修改，之后再写回内存。对将来不会再读取的大块输出，这可能浪费带宽，因为读入旧内容没有价值。某些架构提供 non-temporal store，用于流式写出，减少缓存污染和写分配。但它也有边界：如果后续马上读取这些数据，绕过缓存可能变慢。

并行算法写输出时要考虑写分配。Stream compaction 的输出可能是连续写，后续 reduce 也许马上读；此时普通写可能有益。文件 I/O 或网络发送缓冲如果只是写完交给设备，缓存污染可能更明显。教材不要求读者手写 non-temporal 指令，但要知道写也会产生内存流量。

硬件预取器能看懂什么 硬件预取器擅长顺序访问和某些固定步长访问。它不理解复杂指针结构，也不理解哈希表随机访问。若程序顺序扫描 `std::vector`，预取器能提前请求后续 cache line；若程序遍历链表，下一地址必须等当前节点加载出来，预取器很难帮忙。若程序按随机索引访问数组，即使数组本身连续，访问序列仍然不可预测。

这说明“连续存储”和“连续访问”是两件事。一个 vector 按随机索引访问，仍可能 miss 很多；一个链表如果由对象池按顺序分配，也许比完全随机链表好，但仍受下一指针依赖限制。优化访问模式比手写 prefetch 更可靠。软件预取只有在地址能提前算出、预取距离合适、不会污染缓存时才可能有效。

TLB 的多级结构 地址翻译也有缓存。TLB 保存虚拟页到物理页的映射，常有 L1 dTLB、iTLB 和更大的二级 TLB。TLB miss 时，硬件或内核需要遍历页表，成本可能很高。一个程序如果访问很多页面且每页只用很少数据，TLB miss 会成为瓶颈。链表、巨大哈希表和随机图遍历都容易遇到这个问题。

TLB 覆盖范围等于 TLB 项数乘以页面大小。使用大页可以扩大覆盖范围，但会带来分配、碎片、权限和 NUMA 放置问题。透明大页可能自动启用，也可能因为内存碎片或访问模式没有生效。性能报告中若讨论 TLB，应记录页面大小、是否启用透明大页、工作集跨越多少页，以及 dTLB 事件是否可用。

Page fault 的几类含义 Page fault 不一定表示错误。Minor fault 可能表示虚拟页第一次映射到物理页、按需分配零页、建立页表项；Major fault 通常涉及磁盘 I/O，例如文件页不在 page cache

中。还有写时复制 fault: fork 后父子共享页面, 某一方写入时内核复制页面。不同 fault 的成本差别很大。

Benchmark 如果没有预热和初始化, 第一次访问大数组会包含大量 minor fault; 第一次 mmap 文件会包含 page cache miss 和 major fault; fork 后写内存可能包含 COW 成本。若目标是热循环性能, 就应把这些成本分离; 若目标是冷启动或批处理端到端, 就应保留并单独报告。不要让 page fault 悄悄混进“算法时间”。

虚拟内存和安全边界 虚拟内存不仅是性能机制, 也是隔离机制。每个进程看到自己的虚拟地址空间, 页表控制哪些页可读、可写、可执行。缺页异常让内核有机会按需分配、加载文件、处理 COW 或拒绝非法访问。理解这些机制有助于解释为什么越界访问可能崩溃, 为什么 mmap 可以把文件映射为内存, 为什么进程间共享内存需要明确映射。

高性能系统有时会使用共享内存绕过网络或减少拷贝, 但共享内存会重新引入并发同步和生命周期问题。两个进程共享一块内存, 不代表它们自动有一致的协议。仍然需要 ring buffer、sequence、futex、eventfd 或其他同步机制。虚拟内存提供地址映射, 不提供业务语义。

Page cache 和持久化 Linux 的 page cache 会缓存文件数据。读取文件时, 数据可能来自内存中的 page cache, 而不是磁盘; 写文件时, 写入通常先进入 page cache, 稍后由内核写回。调用 write 成功, 不等于数据已经安全落盘。调用 fsync 或 fdatasync 才把持久化成本显式拉到当前路径, 但具体语义还受文件系统和设备影响。

这对检查点非常重要。一个分布式作业写 checkpoint 文件后, 如果没有正确 fsync, 进程崩溃或机器断电后文件可能不存在或内容不完整。写 manifest 时还要考虑目录项持久化, rename 后是否 fsync 目录。教学项目可以简化持久化, 但文字上必须说明“写文件”和“可恢复提交”不是同一件事。

mmap 的优点和陷阱 mmap 可以把文件映射到地址空间, 让程序像访问内存一样访问文件。它适合随机读取、由内核按需分页、避免显式 read 缓冲管理。缺点是 page fault 出现在访问指令处, 延迟可能分散; 错误处理也不同, 访问损坏或截断文件可能触发信号; 预取和释放需要 madvise 等机制辅助; 写映射文件还涉及脏页和持久化。

顺序扫描大文件时, 普通 read 加显式缓冲有时更容易控制 I/O 节奏和错误处理; mmap 更简洁, 但性能不一定总更好。系统教材应避免把 mmap 神化。选择 read 还是 mmap, 要看访问模式、错误处理、文件大小、内存压力和持久化要求。

内存分配和页粒度 用户态分配器向内核申请大块内存, 再切给小对象。小对象释放后, 内存可能回到分配器缓存, 不一定立即还给内核。进程 RSS、虚拟内存大小和业务对象大小之间可能差很多。并发分配器还可能为每个线程保留缓存, 线程退出或跨线程释放会影响内存回收和局部性。

这会影​​响性能实验。一个测试反复分配释放小对象, 第二轮可能比第一轮快, 因为分配器缓存已经热; 也可能 RSS 不下降, 让人误以为泄漏。对象池和 arena 可以改善可预测性, 但要报告峰值内存和释放策略。内存优化不是只看 malloc 次数, 还要看页、TLB、cache 和生命周期。

缓存一致性和虚拟内存的连接 Cache coherence 处理多个核心对物理内存中同一 cache line 的一致视图, 虚拟内存处理虚拟地址到物理页的映射。两个不同虚拟地址可能映射到同一物理页, 例如共享内存或文件映射; 两个进程通过不同地址写同一物理页, 仍会触发 cache line 一致性。理解这个连接, 有助于分析 mmap 共享文件、进程间队列和 page cache。

分布式系统中, 跨机器没有硬件 cache coherence, 必须用协议维护一致性。单机共享内存的 cache line 迁移, 在集群中变成 RPC、复制和共识。第二册先讲虚拟内存和 cache coherence, 再讲分布式复制, 就是为了让读者看到“一致视图”的成本如何随尺度上升。

内存实验的设计 本章实验应覆盖三类访问。第一, 顺序扫描, 观察带宽和预取。第二, 固定步长访问, 观察 cache line 利用率、set 冲突和 TLB 覆盖。第三, 随机访问或指针追踪, 观察预取失效和内存级并行度下降。每类实验都要保持逻辑工作量一致, 例如访问相同数量的元素, 返回相同类型的校验结果。

实验还要控制数据初始化。先分配再初始化, 记录初始化是否计时; 如果比较 NUMA, 初始化线程和计算线程要明确; 如果比较 page fault, 冷启动和热启动要分开; 如果比较 mmap, page cache 状态要说明。内存实验最容易被隐藏状态影响, 因此报告必须详细。

案例: 对象数组和指针数组 一个常见布局选择是保存对象数组, 还是保存指针数组。对象数组 `std::vector<Record>` 中对象连续, 遍历时 cache 友好; 指针数组 `std::vector<Record*>`

本身连续，但每个指针指向的对象可能分散。若热路径需要访问每个对象字段，指针数组会多一次间接访问，并可能导致随机 cache miss 和 TLB miss。

指针数组也有价值。对象很大且需要稳定地址，或者多个索引共享同一对象，或者对象由不同生命周期管理时，指针更灵活。高性能系统常用“稳定 ID 加对象池”折中：外部持有 ID，内部对象池尽量连续。这个案例提醒读者：局部性和稳定引用经常冲突，需要设计层面的取舍。

案例：内存映射日志文件 使用 mmap 扫描日志文件很方便。程序可以把文件映射成字节 span，然后查找换行和分隔符。但第一次访问页面可能触发 page fault，文件被截断时访问可能出错，映射区域的生命周期必须覆盖解析过程。若解析阶段把 `raw_offsets` 指向 mmap 区域，后续错误报告必须保证映射仍然存在。

普通 read 版本则显式管理缓冲。它可能多一次复制，但错误处理和背压更直接：读取一个 batch，解析完释放或复用缓冲。mmap 和 read 的选择取决于访问模式、错误处理和生命周期。项目可以同时保留两种输入后端作为实验，而不是假设 mmap 永远更快。

案例：Page fault 进入计时区 假设 benchmark 分配一个 1 GiB vector，然后直接开始计时求和，但初始化没有触碰所有页面。求和第一次读到页面时可能触发缺页，计时结果包含页表建立和物理页分配。另一个版本提前初始化数组，求和时没有这些 fault。两者时间差不一定来自求和算法。这个案例非常常见。

正确实验应明确分配、初始化、预热和计时。若要测端到端，就报告 page faults；若要测热循环，就在计时前触碰页面。没有这个区分，内存实验经常得出错误结论。

内存安全和性能 越界访问、use-after-free 和数据竞争不仅是正确性问题，也会破坏性能分析。未定义行为可能让编译器做出激进优化，导致 benchmark 结果没有意义。释放后访问可能偶尔命中旧内存，看似正常，压力下崩溃。性能代码更需要 sanitizer 和边界测试，因为它常使用手写布局、对象池和指针。

项目应在 debug/asan/tsan 构建中跑 correctness tests，在 release 构建中跑性能。不要用 sanitizer 构建得出性能结论，也不要用 release-only 测试证明内存安全。正确性工具和性能工具各有位置。

案例：Page cache 和 checkpoint Checkpoint 写入常被误测。程序把 checkpoint 写到文件后立即返回，测试显示很快；但真正崩溃恢复时，文件可能还在 page cache，没有持久化。若下一章分布式项目把这个时间当作提交点，就会得到错误可靠性。正确实验要区分 write 时间、fsync 时间和 rename 提交时间。

可以设计一个小实验：写固定大小 checkpoint，分别只 write、write 后 fsync、临时文件 fsync 后 rename 并 fsync 目录。比较时间，并说明每种语义。这个实验会让读者理解为什么可靠输出比普通文件写入复杂。性能和持久性是交换关系，不能只看最快路径。

案例：共享内存队列 两个进程通过 mmap 共享内存实现队列时，cache、TLB、虚拟内存和同步会同时出现。共享内存提供同一物理页映射，但 head/tail 的发布仍需要原子和内存序；进程退出时，队列状态可能停在中间；不同进程地址不同，不能在共享结构中保存普通指针，应保存 offset 或索引。

这个案例非常适合作为本章到无锁章节的桥梁。虚拟内存解决映射，cache coherence 解决同一机器上的可见性，C++ 原子解决语言级同步，状态机解决关闭和恢复。任何一层缺失，队列都不完整。

缓存章节总结 缓存、TLB 和虚拟内存共同决定“访问一个地址”的真实成本。连续访问利用 cache line 和预取，随机访问浪费带宽并增加 TLB 压力；page fault 和 page cache 会把内存访问与 I/O 混在一起；mmap 提供方便但不消除状态机；写入不等于持久化。理解内存层级后，数据布局、并行算法和分布式 checkpoint 才有坚实基础。

内存问题排查路径 遇到疑似内存瓶颈，可以按路径排查。第一，看工作集大小是否超过缓存层级，输入规模曲线是否有拐点。第二，看访问模式，是顺序、步长、随机还是指针追踪。第三，看每个元素实际使用多少字节，cache line 里有多少无用数据。第四，看页面数量和 TLB 覆盖范围，是否存在大量随机跨页访问。第五，看是否有 page fault、mmap 冷启动或 page cache 干扰。第六，看写入是否产生写分配、脏页和 fsync 成本。

这个路径可以应用到很多问题。哈希聚合慢，可能是随机访问和分配；图遍历慢，可能是邻接数据分散和 TLB；日志扫描慢，可能是 page cache 冷启动；checkpoint 慢，可能是 fsync 和写回。

把“内存慢”拆成具体层次，优化方向才清楚。

内存章节项目要求 项目中每个大数据结构都应写明三件事：内存布局、生命周期、访问模式。ParsedLogHotBatch 是列式热字段，生命周期为一个 batch，访问模式为顺序扫描；ShuffleBlock 是序列化后的中间数据，生命周期到 manifest 清理，访问模式为写一次读一次；TaskGraphStorage 是连续节点和边，生命周期为一个 stage，访问模式为随机 ready 更新和顺序 successor 遍历。这样写文档能让后续优化有依据。

缓存章的回归阅读要从地址序列开始，而不是从层级参数开始。选同一批记录，分别用连续结构数组、指针数组、列式热字段和 `mmap` 冷读四种方式处理。算法复杂度都可以是线性的，但硬件看到的地址完全不同：连续数组利用 cache line，指针数组暴露随机 load，列式热字段减少无用搬运，冷 `mmap` 可能把 page fault 混进热路径。

实验设计要把 cache 和 TLB 拆开。小工作集主要看 L1/L2 行为，中等工作集看 LLC 和预取，大工作集看内存带宽，跨越大量页面时再观察 TLB 和 page fault。步长扫描要覆盖 1、cache line 大小、页大小和非整齐步长；矩阵访问要比较行主序和列主序；链式访问要说明预取器为什么很难帮忙。只有这样，局部性才不是一句空话。

项目中的 **MemoryShape** 应记录字节数、元素大小、热字段、页数、对齐和访问顺序。它不需要预测所有性能，但要让报告有对象可指。若一个优化只写“缓存友好”，却不说明减少了哪些字段搬运、少跨了多少页、地址序列如何改变，就应退回重写。缓存章的核心能力，是把数据结构翻译成硬件会经历的地址流。

Linux 观察入口也要服务地址流。读者可以用 `/proc/self/maps` 确认映射范围，用 `/proc/self/smaps` 观察 resident、dirty 和 huge page 线索，用 `perfstat` 记录 cache 与 dTLB 相关事件，在权限不足时退回页错误计数、工作集阶梯和冷热运行对照。源码阅读可以从 `mm/memory.c` 的缺页路径、`mm/filemap.c` 的页缓存路径和 `mm/mmap.c` 的映射管理开始，但阅读目标不是背内核函数，而是理解一次地址访问怎样从用户态 load 走到页表、页缓存和回收策略。

C++ 容器也要放进这张图里。连续 `std::vector` 通常给预取器更清楚的地址序列，`std::deque` 分段连续，`std::list` 节点追踪会制造大量指针 load，`std::unordered_map` 可能在 bucket、node 和 key 上来回跳。容器选择不是只看复杂度表，复杂度相同的 $O(n)$ 遍历可能有完全不同的 cache 和 TLB 行为。后面数据结构与算法书也会复用这个原则。

页级行为还要和文件 I/O 联系。顺序扫描一个大文件时，页缓存和 readahead 可能让访问看起来像内存读取；随机访问同一文件时，页缓存命中、缺页和磁盘调度会交织在一起；使用 `mmap` 时，load 指令可能触发 page fault，错误路径不再显式出现在源码里。教材必须让读者知道，虚拟内存不是抽象背景，它会直接改变热路径的可见成本。

本章也适合讲“冷启动”和“热启动”的区别。第一次运行可能包含磁盘读取、页表建立和 cache 冷 miss，第二次运行可能大量命中页缓存。若报告没有区分冷读和热读，就很容易把操作系统缓存当成算法优化。项目中每个文件扫描实验都应记录是否清理缓存、是否预热、输入是否重新生成，以及 page fault 数量是否和结论一致。

3.6 从地址序列理解缓存、TLB 和虚拟内存

缓存和 TLB 不应从层级图开始背。先写一串地址：连续数组访问、固定步长访问、随机索引访问、链表追踪、二维矩阵按行访问、二维矩阵按列访问。硬件看到的不是变量名，而是地址序列。地址相邻，cache line 搬进来的一整块数据都可能被用上；地址跨页太多，TLB 覆盖不住，页表遍历进入成本；地址依赖上一次 load，预取器难以提前工作。把地址序列画出来，缓存章才不会变成“L1、L2、L3 多大”的参数表。

第一册讲过虚拟地址到物理地址的翻译，第二册要追问翻译成本怎样进入热路径。每次 load/store 都要经过地址翻译，TLB 命中时成本低，TLB miss 时可能触发多级页表遍历。若工作集跨越大量小页，TLB miss 会把看似顺序的扫描拖慢；若使用大页，TLB 覆盖范围变大，但分配、权限、碎片和系统配置也变复杂。教材要让读者看到大页不是开关，而是“用更大的页粒度换更少翻译项”的取舍。

Cache line 是搬运单位，也是争用单位

源码里你读取的是一个字段，硬件搬运的是一个 cache line。若结构体里热字段和冷字段混放，扫描热字段时会顺带搬动冷字段，浪费带宽。若多个线程更新不同变量，但这些变量落在同一条 cache

line, 硬件一致性协议仍会让整条 line 的所有权来回迁移, 形成 false sharing。读者必须同时记住两句话: cache line 是空间局部性的机会, 也是共享写的冲突边界。

教学实验可以从计数器数组开始。版本一让每个 worker 更新 `std::vector<std::uint64_t>` 中相邻槽位; 版本二给每个槽位按 cache line 对齐和填充; 版本三让每个 worker 写线程本地对象, 最后合并。若版本一随线程数增加反而变慢, 版本二或三改善, 就能把 false sharing 变成可观察事实。实验报告还要说明代价: 填充会增加内存占用, 线程本地合并会增加最终汇总阶段, 二者都不是免费。

Cache line 还影响写策略。普通写 miss 常常会先把旧 line 读入缓存再修改, 这叫 write allocate。对一次性大块输出, 读取旧内容可能没有价值; non-temporal store 可以减少缓存污染, 但若后续马上读取输出, 就可能变慢。高质量章节要写出这种边界, 避免把特殊指令讲成单向优化。

局部性不是容器属性, 而是访问属性

`std::vector` 连续存储, 但使用随机索引访问时仍然可能局部性很差; 链表节点逻辑相邻, 但物理地址可能散落各处; 结构体数组适合每次读取完整对象, 列式数组适合只扫描少数字段。局部性由布局和访问顺序共同决定, 不由容器名字决定。

日志项目可以用热冷字段说明。原始记录包含时间戳、用户 id、事件类型、原始文本、错误原因和调试标记。热路径只需要事件类型和少量数值字段, 如果所有字段放在一个大对象数组里, 扫描会搬动许多无用字节。列式 batch 把热字段拆出来, 可以让解析后的计算阶段只读必要数据。冷字段仍然保留, 用于错误报告和审计, 但不进入每条记录的热循环。

局部性还要考虑生命周期。一个对象若在队列中跨线程传递, 频繁分配释放会打散地址, 也会给分配器和 TLB 带来压力。Arena 或对象池能把同生命周期对象放在一起, 减少分配开销和提升局部性; 但它也会推迟释放, 可能增加峰值内存。章节要把“分配器优化”从“更快 malloc”提升到“生命周期和地址布局管理”。

TLB、页错误和 Linux 观察入口

TLB 是虚拟地址翻译的缓存。一个 4 KiB 页面能覆盖的连续数据很有限, 大数组随机访问会让 TLB 工作集膨胀。若每次访问跨到新页, 即使 cache line 只用少量字节, 地址翻译也可能成为主成本。大页、批量访问、排序索引和分块都可以减少 TLB 压力, 但它们影响不同层次: 大页改变页粒度, 分块改变访问顺序, 排序索引改变算法输出顺序或需要额外恢复顺序。

页错误要分清 major 和 minor。Minor fault 可能只是建立页表映射或分配匿名页, 不一定读磁盘; major fault 通常涉及 I/O。第一次触碰大数组时, 初始化成本、页面分配和 NUMA first-touch 都会混入测量。若 benchmark 把分配和初始化放进热循环, 总耗时就不能代表内核计算成本。测量章会系统讲报告格式, 但缓存章必须提醒: 内存相关实验要区分分配、触页、预热和核心循环。

Linux 观察可以从 `perfstat` 的 cache、TLB 和 page-fault 事件开始, 再结合 `/proc/self/smaps`、`numastat` 和 `perfmem`。在手机或受限环境没有权限时, 仍可通过输入规模阶梯、访问步长曲线和预热对照获取间接证据。重要的是把“没权限看计数器”写进报告, 而不是假装已经证明了硬件事件。

把地址序列优化落回项目

贯穿项目在本章应完成三个版本。第一个版本保留对象数组, 作为最直观 reference; 第二个版本拆出热字段 batch, 比较解析后计算阶段的字节移动和 cache 行为; 第三个版本为 worker partial、队列槽位和统计指标加入 cache line 对齐, 验证 false sharing 是否下降。每个版本都要保持输出一致, 不能为了局部性改变语义。

还要加入一个反例: 若输入很小, 小到完全落在 L1/L2 内, 列式布局和对齐可能收益不明显, 甚至因为代码复杂和额外索引而变慢。若热字段几乎总是和冷字段一起使用, 拆分也可能增加访存次数。教材要训练读者判断适用范围, 而不是把 SoA、padding 和大页当成固定答案。

本章最终判断力是: 看到一个数据结构, 能说出它会产生什么地址序列; 看到一个地址序列, 能推断可能的 cache、TLB、预取和一致性问题; 看到一个优化建议, 能设计对照实验证明它确实改变了地址序列或搬运成本。只会背缓存层级参数, 还没有进入系统工程。

案例追查: 同样是 vector, 为什么速度差很多

构造两个版本的用户统计表。版本一是 `std::vector<UserRecord>`, 每个 record 包含 id、name、debug string、last timestamp 和 counter。热循环只更新 counter。版本二把 counter 单独放成 `std::vector<std::uint32_t>`, 冷字段留在另一组结构里。两者都使用 vector, 但地址序

列和有效字节比例完全不同。

实验要分三步。第一步只读 counter，比较带宽和 cache miss；第二步随机访问用户 id，观察 TLB 和缓存行为变化；第三步多线程更新相邻 counter，加入对齐 partial 版本比较 false sharing。这样读者能看到 vector 不是局部性的保证，字段冷热、访问顺序和共享写才是关键。

报告还要写内存占用。列式拆分减少热路径字节，但增加多个数组和索引；对齐 partial 降低 false sharing，但增加空间。高性能数据结构不是只追求最快一组数据，而是在速度、空间、接口和可维护性之间做可解释取舍。

缓存实验的输入规模阶梯

缓存实验必须跨多个输入规模。只测一个大数组，会把 L1、L2、LLC、DRAM 和 TLB 混在一起。建议准备按容量阶梯增长的数据：小到 L1 能容纳，中等到 L2，接近 LLC，超过 LLC，最后到内存。每个规模下比较连续扫描、固定步长、随机索引、链表追踪。曲线比单点数字更能说明层级边界。

步长实验要注意解释。步长为 1 时空间局部性最好；步长等于 cache line 内元素数时，每条 line 只用一个元素；步长跨页时，TLB 压力上升；随机索引会破坏预取。报告应写出每次访问有效字节比例，而不是只写速度下降。这样读者能把“cache line 是搬运单位”落到数字上。

链表实验要避免分配器偶然性。可以先随机打乱数组索引，用索引模拟链表，保证节点在一个连续数组中但访问顺序随机；再比较真正分配节点的链表。前者隔离预取和地址依赖，后者加入分配碎片和 TLB。两者差异能帮助读者区分不规则访问的不同来源。

Linux 内存观察的降级路径

理想情况下，读者可以用 `perfstat` 看 cache 和 TLB 事件，用 `numactl` 控制绑定，用 `/proc/self/smaps` 看映射，用 `pmap` 看地址空间。但手机或容器环境可能权限不足。教材要给降级路径：没有 PMU 事件时，用输入规模曲线和步长曲线；没有 `numactl` 时，只做单节点 first-touch 说明；没有大页权限时，只解释设计和风险。

降级不是放弃严谨。报告要明确写“本实验未能直接读取 dTLB 事件，因此只能通过页数和耗时曲线间接支持假设”。这种表述比假装有硬件证据更专业。系统工程常在权限不足、生产环境不可扰动、硬件不可控的条件下工作，诚实写证据边界本身就是能力。

本章还建议读 [Documentation/admin-guide/mm](#) 下与 huge pages、transparent huge pages、NUMA balancing 相关的说明，再结合 `mm/` 目录中的 page fault 和 reclaim 入口做浅阅读。目标是知道内核中确实存在这些机制，而不是把它们当成用户态工具输出的黑盒。

综合练习：设计地址序列基准

本章综合题要求读者实现一个地址序列基准，而不是一个泛泛的内存 benchmark。基准提供四种访问器：连续访问、固定步长访问、随机索引访问、依赖式指针追踪。每种访问器都必须产生同样的 checksum，避免编译器删除循环。输入规模按 L1、L2、LLC、超过 LLC、远超内存缓存几个阶梯设置。报告不能只写“随机慢”，而要解释随机慢在哪里：空间局部性、预取、TLB、地址依赖分别扮演什么角色。

第二步把数据结构换成三个版本：对象数组、热字段列式数组、索引模拟链表。对象数组用于展示无效字段搬运，列式数组用于展示热路径变窄，索引链表用于分离地址依赖和分配碎片。若读者直接用真实链表，分配器碎片、节点大小和地址依赖混在一起，结论会不清楚。先用索引模拟，再加真实链表，才是逐步加因素。

第三步加入跨线程写。让多个线程更新相邻槽位，再比较紧凑槽位、cache line 填充槽位和线程本地槽位。这个实验把本章的 cache line 搬运和下一章的一致性流量连接起来。报告要同时写时间和内存占用，因为 padding 的收益来自空间换冲突减少。

这个综合题的评分标准不是谁跑得最快，而是谁能画出地址序列并解释曲线形状。若曲线出现异常，例如某个规模突然变快或变慢，要要求读者写候选原因：预取、页大小、CPU 频率、系统噪声、编译器优化、TLB 或分配器。不能解释异常的 benchmark 只是一组数字。

容易出错的边界：局部性优化的副作用

局部性优化最常见的副作用是空间换时间。Cache line 对齐可以减少 false sharing，但会扩大对象；列式布局可以减少热路径搬运，但会增加数组数量和索引；大页可以减少 TLB 压力，但可能增加内存碎片和配置复杂度；对象池可以减少分配开销，但可能推迟释放，抬高峰值内存。教材要把这些副作用和收益放在同一段里讲，否则读者会把所有布局技巧当成单向加速。

第二个副作用是接口复杂。AoS 对业务代码友好，SoA 对热循环友好。若把列式数组直接暴露给所有模块，业务代码会到处操作多个平行 vector，容易破坏不变量。更好的做法是提供批量视图和有限修改接口。这样热路径看见连续 span，控制面仍然通过清晰对象管理生命周期。布局优化不是让所有代码都变底层，而是让底层成本在合适边界内可见。

第三个副作用是调试难度。材料化中间对象多，内存大但容易检查；完全融合和列式化后，中间状态少，性能好但调试困难。贯穿项目可以保留 debug 模式：输出 batch 摘要、字段范围和错误索引。性能模式关闭这些检查，保留 checksum。这样读者会看到工程系统常常同时保留可诊断路径和高性能路径。

本章的源码索引也应包含 C++ 容器规则。`std::vector` 的 reallocation 失效，`std::deque` 的分段存储，`std::list` 的节点稳定和局部性差，`std::unordered_map` 的 rehash 和分配，这些都是数据布局的一部分。学习缓存不能只看硬件手册，也要看语言容器如何制造地址序列。

本章核心接口：MemoryShape

缓存章给项目留下 **MemoryShape** 描述。它记录对象大小、热字段字节数、冷字段字节数、连续数组数量、是否 cache line 对齐、是否可能 false sharing、估算工作集页数。每个核心数据结构都应能生成这个描述。描述不需要完美，但要迫使设计者写出数据如何占用内存。

后续布局优化和 NUMA 策略都依赖它。线程本地 partial 要声明是否按 cache line 隔离；batch 要声明热字段跨度；shuffle block 要声明字节数和页数。没有 MemoryShape，局部性讨论会退回感觉。

从缓存层级进入 NUMA 和一致性

本章解释了单个线程或单个核心附近的数据移动。下一章把多个核心放进同一张图：同一条 cache line 被谁读，谁写，页面归属哪个节点，远端访问经过哪条互连。很多 false sharing 和热点 atomic 问题，在本章看来只是 cache line 搬运，在下一章会变成一致性所有权迁移。很多大数组扫描，在本章看来是内存带宽，在下一章会变成内存控制器和 NUMA 放置。

项目推进时，先让数据结构具备可解释的 MemoryShape，再谈拓扑策略。没有对象大小、热字段字节、partial 布局和页数估算，NUMA 优化就无从下手。缓存章给的是局部地址地图，NUMA 章给的是多核心、多节点之间的交通规则。

3.7 项目落地

Compute Systems Engine 在本章至少要增加四类实验。第一，partial 对齐实验：比较紧凑 partial 和 `alignas(64)` partial，观察多线程写入扩展曲线。第二，访问模式实验：比较顺序数组、固定步长、随机索引和链表遍历。第三，页级实验：改变工作集跨越页数，记录 page faults 和可能的 dTLB 事件。第四，first-touch 实验：在支持 NUMA 的机器上比较主线程初始化和 worker 分块初始化。

这些实验都必须有 reference。访问模式版本必须返回同样的逻辑结果；partial 对齐版本必须只改变布局，不改变算法；first-touch 版本必须分离初始化时间和计算时间，同时也可以单独报告初始化成本。若实验一次改变多个变量，结论就不可靠。

cache line 处理器搬运的基本粒度通常不是一个字段，而是一条 cache line。读取一个 int 可能顺带搬入相邻字段；写一个小字段也可能让整条 line 进入独占状态。冷热字段混在一起，会让热循环被冷数据拖住。

空间局部性 连续数组之所以常快，不是因为名字叫数组，而是因为地址递增稳定，预取器能工作，一条 line 能服务多个元素。链表或随机指针会让每个元素都可能变成一次独立等待。

时间局部性 时间局部性要求数据在被驱逐前再次使用。若工作集超过缓存容量，重复扫描也可能没有收益。实验要逐步扩大输入规模，找到 L1、L2、L3 和内存之间的拐点。

TLB 覆盖 TLB 缓存虚拟页到物理页的转换。访问很多页、每页只用少量数据，会把周期花在地址翻译上。大页、紧凑布局和分块访问都可以改善 TLB 覆盖，但它们各自有分配、碎片和权限成本。

false sharing 两个线程写不同变量，如果变量落在同一条 cache line，也会互相使对方缓存失效。这个问题不靠锁数量判断，而靠地址和写入频率判断。按 cache line 分隔计数器，是为了隔离一致性流量。

页缓存 文件 I/O 经常先进入 page cache。第一次读取可能包含缺页和磁盘成本，第二次读取可能只是内存访问。benchmark 若不区分冷读和热读，就会把 I/O、缺页和解析混成一个数字。

mmap 边界 mmap 让文件像内存一样访问，但不是把 I/O 成本变没了。缺页发生在访问时，错误也可能在访问时暴露。教材应强调 mmap 的生命周期、页粒度和故障处理，而不是把它当作魔法优化。

C++ 表达 地址序列应进入接口。只读连续输入用 `std::span`，热字段拆成列式数组，跨线程计数器显式对齐，索引替代指针链。接口让访问模式可见，后续 SIMD 和并行章节才能复用。

案例：数组、指针数组、链表 三种结构处理同样数量元素，比较每元素耗时和 cache/TLB 现象。报告必须解释地址递增、随机跳转和节点分配对预取的影响。

案例：冷热字段拆分 结构体里放 `hot_value`、`status`、`debug_text`、`timestamp`。扫描 `hot_value` 时先用 AoS，再改成 SoA，记录搬运字节和收益边界。

案例：两线程计数器 让两个线程写相邻计数器，再加 padding 分隔。这个实验把 false sharing 从概念变成可观察现象。

源码阅读路线 Linux 源码阅读从 `mm/filemap.c`、`mm/memory.c`、`mm/mmap.c` 附近进入即可。阅读目标不是把整套子系统抄进书里，而是追一条和本章实验相关的路径：用户态动作怎样变成内核对象，内核对象怎样排队、等待、唤醒、计数、提交或回收。

阅读报告要写证据等级。能直接测到的数字放在第一层；从源码控制流推导出的解释放在第二层；当前设备无法验证、只能作为合理猜测的内容放在第三层。这样读者会知道地址序列报告中哪些结论可复现，哪些结论需要换机器继续确认。

源码阅读还要克制。看到一个复杂函数，不要把每个分支都展开；先回到本章的问题，问它能否解释一个等待点、一次状态转移或一条性能曲线。解释不了的内容可以留到索引，不要打断正文主线。

数组、指针数组、链表 三种结构处理同样数量元素，比较每元素耗时和 cache/TLB 现象。报告必须解释地址递增、随机跳转和节点分配对预取的影响。

冷热字段拆分 结构体里放 `hot_value`、`status`、`debug_text`、`timestamp`。扫描 `hot_value` 时先用 AoS，再改成 SoA，记录搬运字节和收益边界。

两线程计数器 让两个线程写相邻计数器，再加 padding 分隔。这个实验把 false sharing 从概念变成可观察现象。

cache line

硬件按 cache line 搬运数据，不按 C++ 字段搬运。读取一个热字段时，旁边冷字段也可能被带进缓存，浪费带宽。

实验设计：构造包含热计数、状态、调试文本和时间戳的结构体，对比 AoS 与 SoA 扫描。

工作集

小工作集停留在 cache 中，大工作集进入内存带宽和 TLB 压力。只说输入大小不够，要换算成访问字节和活跃页面数。

实验设计：从几 KiB 扫到数百 MiB，记录曲线拐点，并写出可能对应的层级。

步长访问

固定步长会改变 cache line 利用率和预取效果。步长越大，每次搬运的有效字节越少，TLB 覆盖也可能更早耗尽。

实验设计：对同一数组用步长一、八、六十四和页大小扫描，比较每元素时间。

TLB 覆盖

TLB 缓存虚拟页到物理页的翻译。页面数超过覆盖范围后，即使命中 cache 的数据不多，地址翻译也会成为等待来源。

实验设计：在相同字节量下改变页数量，观察小页、大页和随机页访问的差异。

页错误

minor fault、major fault 和写时复制 fault 语义不同。第一次触碰匿名页、读取文件页和 fork 后写共享页，成本不能混成一类。

实验设计：读取新映射文件、重复读取热文件和写入 fork 后页面，分别记录 fault 类型。

page cache

文件读取常先进入 page cache。第二次读取快，不一定是程序优化，可能是内核缓存。写入也可能先返回，稍后才真正落盘。

实验设计：比较冷读、热读、普通 read 和 mmap，报告明确是否测到存储设备。

mmap 生命周期

mmap 让文件内容像内存一样访问，但页面错误、SIGBUS、文件截断和映射生命周期都要处理。它不是免费数组。

实验设计：映射文件后模拟文件变化或输入缓冲释放，写出使用 mmap 的限制。

false sharing

两个线程写不同变量，如果变量落在同一 cache line，仍会争夺所有权。结构体字段不共享不等于硬件不共享。

实验设计：把每线程计数放在相邻元素和对齐槽位中，比较扩展曲线。

冷热分离

热字段进入连续数组，冷字段留在旁路结构，可以降低带宽和 cache 污染。代价是索引、生命周期和错误报告要重新设计。

实验设计：让解析阶段只输出热字段视图，错误报告通过稳定记录号回查冷字段。

源码观察

Linux 的页表和 page cache 路径很深，本章只追一条可解释实验的线：缺页怎样进入处理，文件页怎样查找，读写怎样触发缓存。

实验设计：阅读时只摘和实验现象相关的入口，不把整套内存管理搬进正文。

3.8 从地址序列而不是变量名理解内存系统

缓存和 TLB 章节不能只讲 L1、L2、L3 的层级表。读者真正需要学会的是把程序变成地址序列。源码说“遍历记录数组”，硬件看到的是一串 cache line；源码说“访问字段”，硬件搬运的是包含这个字段的一整段字节；源码说“读取文件”，内核可能先从 page cache 给出页，再由缺页路径建立映射。变量名和对象边界是给程序员看的，地址序列和页面边界才是硬件与内核看到的。

先看结构体扫描。日志记录里有事件类型、时间戳、用户 id、错误文本和调试字段。如果统计只需要事件类型，AoS 会把冷字段一起带进缓存；SoA 把热字段连成数组，扫描时每个 cache line 包含更多有效信息。这个优化不是简单地把结构体拆开，因为拆开后错误报告、生命周期和索引关系也要改变。若热字段数组里的第 1024 个元素出错，系统仍要能回到原始记录和冷字段。布局优化必须和语义账本一起写。

再看 TLB。很多教材会说 TLB 缓存页表项，但工程里要问的是：一次扫描触碰多少页面，访问顺序是否规律，页面大小和工作集如何影响覆盖。一个随机访问 64 MiB 数组的程序，可能不是数据本身太大，而是活跃页面数太多，地址翻译成为等待。实验可以固定总字节数，改变元素分布在页面上的方式：连续页面、每页一个元素、随机页。这样读者会发现“同样读 N 个元素”在机器上不是同一件事。

文件 I/O 也要放在同一套地址思维里。mmap 让文件像内存，但第一次访问可能触发缺页；文件被截断可能出现错误；映射区域的生命周期和文件描述符生命周期也要区分。普通 read 多一次缓冲管理，却让错误处理和背压更直接。不要告诉读者 mmap 永远更快，也不要告诉读者 read 永远安全。正确写法是按访问模式判断：顺序大读、随机小读、错误恢复、生命周期和 page cache 状态分别怎样影响选择。

page cache 是本章和 I/O 章的桥。第一次读文件可能慢在存储，第二次读同一文件可能快在内存；写入成功可能只是数据进入 page cache，真正持久化要看 fsync 和文件系统语义。性能报告如果不写冷缓存还是热缓存，读者无法判断优化是否有效。手机环境下可能没有权限清理 page cache，那就诚实写明限制，用不同文件或大于内存的数据集做替代，不要把热缓存结果当成磁盘性能。

本章实验可以做成四个小而清楚的系列。第一，AoS 与 SoA 扫描热字段，记录搬运字节估计和运行时间。第二，步长访问，从步长一到页大小，观察每元素成本。第三，TLB 覆盖，固定元素数但分散到更多页面。第四，mmap 与 read，对比首次读取、重复读取和错误处理代码复杂度。每个实验都应附一段地址序列解释，而不是只给图表。图表在 EPUB 里也许不方便，文字表和结论链更可靠。

Linux 源码阅读可从三个入口进入：缺页处理、filemap 查找页缓存、read/write 系统调用路径。读源码时不要求记住所有分支，而要回答实验里的问题：为什么第一次访问映射页会慢，为什么第二次读可能不碰磁盘，为什么写入返回和持久化不是一回事。这样缓存、TLB、虚拟内存和文件系

统才会在读者脑中连成一条线。

案例推导：冷热字段拆分为什么必须同时处理错误报告

旧日志记录结构体把事件类型、时间戳、用户 id、原始行偏移、错误文本和调试标签放在一起。统计事件类型时，每个 worker 顺序扫描结构体数组，只读取事件类型和用户 id。profile 显示内存带宽压力很高。最直接的优化是把热字段拆成列式数组，让扫描只读取事件类型和用户 id。这个优化看似纯性能，但很快会碰到语义问题：当某个聚合结果异常时，如何回到原始行和错误文本？

如果拆分后丢掉原始记录身份，错误报告就会退化。旧结构体里字段在一起，调试方便；新布局里热字段和冷字段分离，必须用稳定 record id 或 offset 连接。LayoutPlan 要记录热字段列、冷字段表、原始输入位置和生命周期。热循环不搬冷字段，但报告路径仍能按 id 找到冷信息。这样才能同时满足性能和可诊断性。

实验可以设计两组。第一组只测扫描吞吐：AoS、SoA、冷热分离三种版本，输入包含长错误文本和短正常文本。第二组测错误路径：故意让某些记录解析失败，要求最终报告能输出原始偏移、错误原因和对应热字段。若 SoA 版本吞吐提高但错误报告丢失，它不是合格优化。教材要明确这一点，因为工程系统不能用“快但不可诊断”作为最终答案。

TLB 也会参与这个案例。冷热分离后，热字段连续，活跃页面减少；冷字段表虽然仍占内存，但不在热循环中触碰。若冷字段被随机回查，回查路径可能慢，但频率低。报告应写出热路径和冷路径的访问频率，而不是把所有路径平均。这样读者能理解为什么某些布局让主路径快，同时保留慢但少见的诊断路径。

源码阅读可从 page cache 和缺页路径连接到输入读取。若热字段来自 mmap 文件，第一次扫描触发缺页；若来自 read 后的解析批次，成本转移到读取和复制阶段。两种方案都可以成立，但生命周期不同。mmap 版本需要处理文件截断和映射有效期；read 版本需要管理缓冲复用和背压。缓存章节的深度就在于：每个地址序列背后都有所有权和恢复边界。

实验：比较行优先访问矩阵和列优先访问矩阵。记录 cache-misses、dTLB 相关事件和时间。再把矩阵规模从 L1、L2、L3 可容纳范围逐步扩大，观察拐点。

地址序列比变量名更接近真相 缓存和 TLB 的学习要从地址序列开始，而不是从 cache line、set、way 的名词开始。贯穿项目里，读取日志文件、解析记录、更新哈希表、写出 partial block，都会产生不同地址序列。顺序扫描输入接近线性流，硬件预取容易工作；更新哈希表会根据 key 跳到不同桶，随机性更强；写输出 buffer 若按 partition 分散写，可能在多个区域之间来回切换；读取 mmap 文件会把缺页、页缓存和用户态指针连在一起。变量名叫 input 或 map 不重要，真正影响性能的是 CPU 看到的地址怎样变化。

第一类地址序列是流式访问。它的特点是步长稳定、局部性好、可预取。流式读取最容易把瓶颈推到内存带宽或 I/O，而不是指令执行。第二类是小工作集反复访问，例如固定大小的计数数组或字符分类表。它适合待在 L1 或 L2。第三类是大工作集随机访问，例如高基数哈希表。它会暴露 cache miss、TLB miss 和分配器碎片。第四类是冷热字段混合访问，例如每条记录的热字段只需要事件类型和数值，冷字段却包含长字符串。若结构体把冷热字段绑在一起，缓存会被不需要的数据占满。

页缓存和 mmap 要从失败语义讲清 很多教材把 mmap 写成“少一次拷贝”，这太粗。mmap 把文件页映射到进程地址空间，访问时可能触发缺页，由内核把页从页缓存或磁盘带入。它让代码像访问内存一样访问文件，但错误语义、生命周期和页级行为都变复杂。文件被截断后访问映射区域可能出错；随机访问大文件可能造成大量缺页；映射生命周期和文件描述符生命周期要区分；预读策略也会影响吞吐。

普通 read 路径更显式：系统调用把数据从内核缓冲拷到用户缓冲，用户代码处理这块 buffer。它有拷贝成本，但边界清楚，错误点清楚，buffer 生命周期由程序掌控。mmap 适合某些随机读取、共享映射或把文件当地址空间处理的场景；read 适合顺序流、明确 buffer 管理和错误控制。项目不应直接宣布哪种更快，而要按输入大小、访问模式、页缓存状态和错误处理实验。

TLB miss 为什么会突然出现 TLB 缓存虚拟页到物理页的翻译。若工作集跨越太多页面，或者访问模式在大范围随机跳页，TLB 容量会成为瓶颈。哈希表、图邻接表、对象池和多个大数组交错访问都可能增加 TLB 压力。把记录按块处理、压缩热字段、使用连续数组、减少指针对象数量，都可以降低页级随机性。

大页不是万能答案。它能减少页表项数量和 TLB 压力，但会影响内存分配、碎片、NUMA 放置和权限粒度。若工作集本来很小，大页没有意义；若访问只碰每页少量数据，大页可能浪费内存；若需要频繁映射和释放，大页管理成本也要考虑。本章应要求读者先用页级访问统计描述问题，再讨论大页。

实验：同一算法的三种内存形状 可以设计一个 key 计数实验。版本一使用 `std::unordered_map` 直接更新，观察随机桶访问和分配成本。版本二先把记录分块，每块使用局部 map，再合并，观察工作集是否变小。版本三若 key 范围可压缩，把 key 映射成紧凑整数，使用连续计数数组，观察 cache 和 TLB 行为。三个版本输出必须相同，但地址序列完全不同。

报告要写清 key 分布：低基数、高基数、Zipf 热点、随机字符串。低基数下连续数组可能极快；高基数字符串下哈希和分配可能主导；热点分布下局部 map 能减少共享写；随机字符串下字符串比较和内存分配可能超过哈希本身。这样读者会明白数据结构选择不是按名称决定，而是按地址序列、工作集和语义决定。

Linux 源码阅读连接点 读 Linux 页缓存时，重点看文件页如何被标识、查找、标记脏、回写和失效。页缓存不是一个魔法加速层，而是一套以页为单位的共享状态机。读虚拟内存相关代码时，关注 VMA、页表、缺页处理和权限。项目中的 manifest 和 block cache 可以借鉴这种思路：每个缓存对象必须有身份、状态、引用、失效规则和回写规则。

源码阅读还要注意快慢路径分离。缺页是慢路径，命中页缓存是快路径；写回是异步过程，fsync 是同步边界；内存回收可能在看似普通分配时触发。把这些现象带回项目，读者就会理解为什么一次 vector push、一次文件读取或一次哈希插入的延迟会有长尾。系统性能不是平均成本的线性相加，慢路径会决定尾延迟。

事故复盘：同样复杂度为什么跑出不同阶梯 两个版本都说自己是线性复杂度：一个把记录解析成节点对象后放进 unordered map，另一个把热字段压成连续数组并用索引关联冷字段。小输入时差异不明显，输入扩大以后，节点版时间突然出现阶梯，perf 显示 cache miss、TLB miss 和缺页都参与进来。复杂度没有说谎，只是它没有描述地址序列。CPU 看到的不是“大 O”，而是一串加载地址、页号和 cache line。

分析要把访问路径画出来。节点对象让 key、value、错误信息和 next 指针散在堆上，哈希桶跳转又引入随机访问；连续数组让扫描阶段按 cache line 前进，但可能需要额外索引回到原始文本；mmap 首次访问可能触发缺页，read 可能多一次拷贝却让预取和页缓存行为更稳定。大页能减少 TLB 压力，也可能浪费内存并扩大无用搬运。每个方案都不是口号，而是一组地址行为。

实验不应只给一组总时间。要逐步扩大工作集，记录冷运行和热运行，区分页缓存、缺页、TLB 覆盖和数据缓存容量的拐点。哈希表实验还要报告 key 分布、装载因子、冲突长度和节点分配策略。若一次优化让扫描更快却让错误定位丢失，就不是合格优化。缓存章节真正要建立的能力，是从程序对象推导地址序列，再从地址序列解释硬件和操作系统成本。

地址实验实验判读 缓存实验要看拐点，不只看最终时间。输入逐步扩大时，若时间在某个规模突然上升，要回到工作集、页数和访问步长；冷热运行差异大，说明页缓存或缺页参与；顺序和随机差异大，说明预取和 TLB 覆盖不同。报告中应写清每个版本的地址形状，而不是只写容器名。

这一段也服务于复习。读者合上书后，应该能凭本章的判读口径重新审查自己的代码：有没有最小正确版本，有没有能击穿旧方案的输入，有没有状态记录，有没有资源指标，有没有失败后的恢复动作。若五个问题缺两个以上，就说明实现还停在演示层，而不是工程层。

页缓存实验必须区分冷、热和污染 文件读取实验至少要跑三种状态。冷状态尽量减少页缓存影响，用来观察缺页、磁盘和预读；热状态重复读取同一文件，用来观察页缓存命中后的解析与内存路径；污染状态在两次运行之间读一个更大的无关文件，用来观察缓存被挤出后的行为。三种状态都不是完美真相，但合在一起能防止把页缓存命中误写成算法优化。

mmap 和 read 的比较也要写错误语义。mmap 让访问看起来像普通内存，但缺页发生在 load 指令附近，文件截断和映射生命周期会带来额外边界；read 把系统调用和用户缓冲区分开，拷贝成本更显式，也更容易控制 buffer 复用。若只比较一次运行时间，读者会错过最重要的生命周期差异。

地址实验的报告建议附一张文本表：访问形状、工作集大小、页面数量、是否顺序、是否随机、是否触碰冷字段、是否写回。这个表比容器名称更重要。unordered map、vector、mmap 都只是接口，真正决定硬件行为的是地址流。

实验课：同一个计数任务的四种地址形状 输入仍然是日志事件计数。第一版每条记录分配一

个节点对象，节点里保存字符串 key，然后更新 `std::unordered_map`。第二版先把事件名 intern 成整数 id，再更新连续计数数组。第三版按 split 建局部 map，最后合并。第四版把热字段列式化，聚合阶段只扫描 event id 列。四个版本输出必须和串行 reference 一致。

实验不要一上来跑最大输入。先按工作集阶梯扩大规模：能放入 L1 的小数据，能放入 L2 的中等数据，超过 LLC 的大数据，再到跨越大量页面的超大数据。每个规模记录时间、页错误、缓存相关事件或替代指标、内存峰值和输出 checksum。权限不足时，也要记录冷运行与热运行差异、输入规模拐点和访问步长曲线。

判读时重点看地址形状。节点对象版本可能在小输入上够快，但规模增大后随机指针和字符串比较会暴露；整数 id 数组版本可能极快，但要求有字典编码；局部 map 降低共享写，却增加合并成本；列式化减少热路径字节，却要保留冷字段回查。没有一个版本天然最好，选择取决于 key 分布、错误报告需求和内存预算。

再把文件读取加入实验。用 read 版本和 mmap 版本分别喂同样的解析器，跑冷、热、污染三种状态。报告写清哪部分时间来自读取、哪部分来自解析、哪部分来自页缓存状态。这样读者会看到：虚拟内存、页缓存和容器布局共同决定实际成本，复杂度记号只能提供很粗的上界。

地址序列练习验收重点 合格答案要把容器名翻译成地址序列。vector 是连续还是分段，unordered map 是否节点分配，mmap 是否会触发缺页，read 是否有用户缓冲复用。答案要区分冷运行和热运行，并写出如果没有硬件计数器时用什么替代证据。

答案还要写“为什么不采用更复杂方案”。例如没有证据证明队列是瓶颈，就不应把锁队列换成无锁；没有证明输出顺序可变，就不应随意重排；没有证明错误路径能恢复，就不应只提交正常路径。能解释不做什么，说明读者开始具备工程判断。

最后，答案必须包含可复查材料。至少有输入摘要、关键状态、期望输出、实际输出、运行命令和未验证边界。没有这些材料，老师或同伴无法判断结论是否成立。第二册的作业不是让读者写漂亮段落，而是训练读者把系统问题变成可验证证据。

项目实现：缓存、TLB 与页级性能 本章对应的贯穿项目实现不要求一次写成完整框架，但必须有最小可运行切片。这个切片包含三个部分：一个 reference，一个带本章机制的实现，一个能击穿 reference 之外隐含假设的测试。Reference 用来固定语义，机制实现用来展示本章知识，测试用来证明新增复杂度确实解决了问题。

代码组织上，不要把所有内容放进一个大文件。基础类型、输入生成、核心机制、报告输出和测试应分开。即使当前只是教学项目，也要让目录能表达责任边界。这样后续章节接入时，不会因为找不到状态归属而重新发明一套对象。

每个实现都要输出一段机器可读摘要。摘要至少包含输入规模、关键状态、错误数、输出校验、耗时和未验证边界。这里的机器可读不要求复杂格式，简单的 key value 文本也可以。关键是让脚本和读者都能复查，而不是只能看屏幕截图。

测试应包含正常路径、边界路径和失败路径。正常路径证明功能可用；边界路径证明长度、容量、尾部、空输入、关闭或超时没有遗漏；失败路径证明本章机制不是装饰。若失败路径写不出来，往往说明本章概念还没有落到工程对象上。

报告最后写一个取舍段。新增机制带来了什么好处，增加了什么成本，哪些场景不应该使用它，下一章会复用哪个字段或对象。这个取舍段非常重要，因为系统工程不是把所有高级机制都加进去，而是在证据和约束下选择合适的边界。

调试笔记：缓存问题先画访问步长 怀疑 cache 或 TLB 时，先画访问步长和工作集，不要先换容器。顺序步长、固定大步长、随机桶访问、指针追踪和冷热字段混读，对硬件完全不同。若工作集从小到大出现台阶，就把台阶和缓存层级或页数量联系起来；若冷运行慢、热运行快，就把页缓存和缺页写入报告；若随机访问慢，就检查是否能排序、分桶或压缩索引。

复查清单：缓存、TLB 与页级性能 复查本章时，先检查 reference 是否仍然存在。没有 reference，后面的并发、优化、异步或分布式改造都无法证明语义没有变。再检查压力输入是否尖锐：它应当攻击本章最核心的边界，而不是只把数据量放大。第三，检查状态是否可观察：关键对象的身份、状态、错误和提交结果应能在报告中找到。

第四，检查代码示例是否说明禁止行为。调用者不能做什么，往往比能做什么更能体现工程质量。第五，检查实验是否写出未验证边界；当前设备、权限或输入无法证明的结论，必须标注。第六，检查本章对象是否能被下一章复用，不能复用就说明边界还不稳。

完成这六项后，再看文字是否连贯。每个概念都应从问题推出，每个案例都应从朴素方案走到改进方案，每个实验都应能解释一个判断。若某段话放到其他章节也成立，就把它改成本章的具体对象、具体输入和具体失败。

3.9 工程案例：用地址序列解释缓存、TLB 和页缓存

缓存、TLB 和虚拟内存不是三块互不相干的知识。它们都在回答同一个问题：程序给机器的地址序列是什么形状，硬件和操作系统能否用有限的近处存储承接这个序列。若地址连续、重复利用高，缓存和预取器会帮忙；若地址随机、工作集跨越太多页，核心会频繁等待；若文件访问没有和页缓存语义对齐，I/O 和内存会互相污染。

一、从地址序列开始，而不是从缓存名词开始 同样是 $O(n)$ ，顺序扫描数组和追踪链表完全不同。数组扫描每次访问下一个元素，cache line 被充分利用，硬件预取器容易预测；链表追踪每次 load 下一个指针，下一步地址依赖当前 load 结果，预取困难，乱序窗口也难以产生足够独立 miss。算法复杂度没有错，但它没有描述地址序列。

因此本章实验要记录访问模式。连续、固定步长、随机索引、热点加长尾、二维行优先、二维列优先、分块访问，这些序列要用同一套输入规模比较。报告不要只写耗时，还要写元素大小、总字节数、访问页数、步长、是否重复使用、是否修改。这样缓存讨论才不会停留在“局部性好”这种口号。

二、cache line 是数据搬运单位 处理器通常按 cache line 搬运数据，而不是按单个字段。若热循环只用结构体中的两个小字段，却每次把包含字符串、调试信息和冷状态的大结构体一起搬进 cache，就会浪费带宽。数据布局章会继续讲 AoS 和 SoA，但本章先解释为什么浪费发生：硬件无法只搬你逻辑上关心的字段，它按连续字节块搬。

false sharing 也是同一件事的并发版本。两个线程写不同变量，如果变量落在同一 cache line 上，硬件仍要在核心之间迁移这条 line。源码看起来没有共享同一个字段，cache line 层面却共享了搬运和所有权。实验可以让多个线程分别写相邻计数器，再加入 padding 或按线程分块，观察扩展曲线变化。

三、TLB 把虚拟页转换成热点资源 虚拟地址访问需要转换到物理地址。TLB 缓存最近使用的页表翻译。若工作集跨越的页太多，TLB miss 会引发 page walk，核心即使 cache line 在内存中也要先解决地址翻译。随机访问大数组、稀疏矩阵、哈希表和大图遍历都容易遇到这个问题。它们慢的不只是 cache miss，还有页级覆盖不足。

实验可以固定访问字节数，改变元素步长和页数。比如每页访问一个 64 位整数，与连续扫描同样数量元素比较。前者浪费 cache line，也打爆 TLB 覆盖；后者充分利用每页和每条 line。若系统支持 huge page，可以比较普通页和大页，但报告要写明大页分配是否成功。不要把“开启大页”写成普遍建议，它会影响内存碎片、NUMA 放置和启动成本。

四、虚拟内存让首次触页成为实验变量 分配大数组不等于物理内存已经就位。很多系统采用按需分配，第一次写某页时才真正建立物理页映射。若 benchmark 把首次触页混在热路径里，测到的可能是 page fault 和清零成本，而不是算法本身。多线程程序还会受到 first-touch 策略影响：哪个线程首次写页，页就可能放到哪个 NUMA 节点。

因此实验要分清初始化、预热和测量。先分配，再按计划触页，再进入热循环。若研究冷启动，就明确把首次触页作为被测对象；若研究内核性能，就把首次触页排除。很多看似玄学的性能波动，来自没有控制页状态、缓存状态和 CPU 频率状态。

五、mmap 和 page cache 不是普通内存的简单别名 mmap 让文件内容映射到虚拟地址空间，读写看起来像内存访问，但背后仍有 page fault、page cache、脏页回写和文件系统错误。顺序读取大文件时，read 和 mmap 各有适用边界；随机小访问时，mmap 可能简化代码；写入持久数据时，何时 msync 或 fsync 又会进入语义。

日志统计项目可以用 mmap 做输入读取实验，但要写清：映射成功不代表所有页面已经在内存中；访问坏页或文件截断可能产生错误；page cache 命中和冷读差距很大。I/O 章节会继续讲可靠写入，本章只强调 page cache 是内存层级和文件系统之间的桥，不能当成普通数组完全忽略。

六、C++ 数据结构如何暴露地址形状 std::vector 暴露连续地址，适合扫描和 SIMD；std::deque 分段连续，适合两端操作但整体扫描局部性不如 vector；链表节点分散，插入删除稳定但遍历慢；哈希表平均查找快，却常有随机访问和分支。选择容器时，不能只背复杂度，还要看访问频率和地址序列。

项目中的 `parsed batch` 应优先使用连续列式数组保存热字段。原始字符串和错误文本可以放在冷存储，通过偏移关联。这样解析、统计和分区阶段看到的是连续数值列；诊断阶段需要时再追冷数据。这个设计会在第6章展开，本章先从 `cache line` 和 `TLB` 解释它的必要性。

七、实验交付：AccessTrace 本章可以给项目留下 `AccessTrace` 报告。它记录数据结构类型、元素大小、元素数量、访问顺序、总字节数、页数、是否写入、是否预热、是否使用 `huge page`、是否 `mmap`。每个实验输出 `checksum`，防止编译器优化掉访问。报告还要记录无法观测的指标，例如没有权限读取 `dTLB miss` 时，明确标记 `unavailable`。

验收时选择三组对照：`vector` 顺序扫描对链表遍历，`AoS` 热字段扫描对 `SoA` 热字段扫描，普通页随机访问对大页随机访问。每组都要求读者先预测地址序列，再运行实验。若结果和预测不同，回到编译器优化、缓存状态、页大小和硬件预取器解释差异。这样缓存章节就会变成可操作的分析方法。

3.10 源码阅读：从 Linux 页表和页缓存看内存成本

缓存和 `TLB` 的实验解释了现象，但还需要读者知道操作系统在背后维护哪些状态。源码阅读不要求把 `Linux` 内存子系统读完，而是选几条和项目直接相关的路径：缺页如何建立映射，页缓存如何承接文件读取，脏页如何回写，内存映射如何影响错误处理。这样读者会知道“内存访问慢”有时不是硬件 `cache alone`，而是硬件和内核状态共同作用。

一、缺页路径回答首次触页为什么贵 用户程序访问尚未映射的虚拟页时，会触发 `page fault`。内核要检查 `VMA`，判断权限，分配或找到物理页，建立页表项，可能清零页面，最后返回用户态重试指令。这个过程远比普通 `load` 昂贵。大数组第一次写入时，很多周期花在这里，而不是花在算法本身。

阅读时关注概念，不背函数名。`VMA` 描述一段虚拟地址的权限和来源；页表项描述虚拟页到物理页的映射；匿名页和文件页处理不同；写时复制会让读共享和写私有分开。项目报告中如果把首次触页计入热循环，就应该明确说明；如果排除，就要预触页。读者要能把 `page fault` 这个系统事件和 `benchmark` 曲线对应起来。

二、页缓存解释文件读为什么像内存又不像内存 文件读取常先进入 `page cache`。第二次读取同一文件可能很快，因为数据已经在内存；第一次读取可能受磁盘和 `readahead` 影响；内存压力下，页缓存会被回收。`mmap` 访问文件页时，第一次访问也可能触发缺页，把文件页带入 `page cache`。于是同一段代码在冷 `cache` 和热 `cache` 下差距巨大。

实验报告必须区分冷读和热读。若要测解析器，应尽量把输入预热或使用内存 `buffer`；若要测 `I/O pipeline`，就明确测冷读、热读和写回。不要把 `page cache` 命中后的速度当成磁盘吞吐，也不要把冷读瓶颈误判成解析器慢。页缓存让文件和内存连接起来，也让测量更需要边界。

三、mmap 的错误模型要进设计 `mmap` 简化了读取接口，但错误不一定在映射时出现。文件被截断、页读取失败、访问越界，都可能在后续内存访问中以信号或错误表现出来。普通 `C++` 异常无法自然捕获这些情况。若项目使用 `mmap` 读取输入，就要在设计中说明文件生命周期、映射范围、并发修改和错误处理策略。

对教学项目来说，使用普通 `read` 加 `buffer` 可能更容易处理错误；使用 `mmap` 可以作为高级实验，重点观察 `page fault`、`readahead` 和地址序列。不要为了显得高级强行 `mmap` 所有输入。接口选择应服务语义和证据，而不是服务风格。

四、脏页和持久化连接 I/O 章节 写入文件后，数据可能只在 `page cache` 中变脏，尚未到设备。内核会异步回写，也可以通过 `fsync` 等接口要求持久化。脏页太多会触发回写压力，写入线程可能被迫等待。这个机制解释了为什么写出阶段有时一开始很快，随后突然变慢：系统先吸收脏页，后来必须回写。

日志项目的 `shuffle block` 如果只写临时文件不 `fsync`，在进程崩溃和机器掉电下语义不同。教学中可以先处理进程崩溃，不承诺掉电持久化；如果要承诺掉电恢复，就要把 `fsync` 策略写清楚。内存章先埋下这个边界，`I/O` 章再详细处理。

五、内存压力会改变所有结论 当工作集接近或超过物理内存，系统开始回收页缓存、换出匿名页或触发更频繁的缺页。此时一个本来只受 `cache miss` 限制的程序，会变成 `I/O` 和内存回收问题。`Benchmark` 如果在不同内存压力下运行，结果可能完全不同。报告中至少要记录可用内存、输入大小、是否使用 `swap`、是否有其他进程占用内存。

Compute Systems Engine 的 block size、batch size 和队列容量都影响内存压力。队列太大，page cache 被挤出，后续读文件变慢；batch 太大，TLB 和 cache 局部性下降；保留太多旧配置或 retired node，内存峰值上升。内存不是无限背景资源，而是所有章节共享的预算。

六、源码阅读交付 本章的源码阅读交付不是函数调用图，而是一张“内存事件到系统影响”的表。page fault 对应首次触页和 mmap 冷读；TLB miss 对应页覆盖不足；dirty page 对应写回压力；page cache hit 对应热读；reclaim 对应内存压力；COW 对应 fork 或共享映射写入。每一项写一个项目中可能出现的位置和一个观测方法。

这张表会在后续章节复用。线程池队列积压可能导致内存压力，I/O 写出可能制造脏页，分布式模拟的 shuffle block 可能挤掉输入 page cache。读者若能把这些联系起来，就不会把内存问题局限在“cache 大小”层面。

3.11 补强案例：哈希表查询为什么经常慢在内存

哈希表平均复杂度是常数，但系统性能常被哈希表拖慢。原因是一次查询可能包含哈希计算、bucket 访问、控制字节读取、key 比较、节点指针追踪和 value 读取。每一步都可能触发 cache miss 或分支失败。复杂度告诉你元素数量增长趋势，却不告诉你一次查询要等多少个随机地址。

一、节点式哈希表的地址形状 节点式哈希表通常 bucket 数组连续，但节点分散分配。查找先访问 bucket，再追节点指针，再比较 key。若 key 是字符串，还要访问字符串内容。这个地址序列很难被预取器预测，也不容易被乱序执行隐藏，因为下一步地址依赖当前节点内容。工作集大时，LLC miss 和 TLB miss 会同时出现。

开放寻址哈希表把控制字节和元素放得更紧凑，减少指针追踪，但会受到负载因子、探测长度和删除标记影响。它不是永远更好，但在读多写少、key 可移动、需要高局部性的场景常有优势。教材不必实现完整高性能哈希表，但要让读者理解节点分配和局部性的关系。

二、字典编码改变问题形状 日志统计中，很多字符串 key 可以先字典编码成整数。编码阶段可能使用哈希表，但后续聚合变成数组或紧凑整数哈希，地址形状大幅改善。若同一批数据要多次查询或聚合，编码成本可以被摊销；若只处理一次且 key 很多，编码未必划算。选择取决于复用次数和 key 分布。

字典编码还带来语义问题。编码表的生命周期多长，是否跨 batch 复用，是否需要稳定 id，错误 key 如何处理，输出时如何反查字符串。性能优化一旦改变数据表示，就必须同时定义这些语义。否则后面分布式阶段会出现不同 worker 编码不一致的问题。

三、热点和长尾 若 key 分布高度倾斜，热点 key 会频繁命中同一 bucket 或同一计数器。它可能提高 cache 命中，也可能造成共享写热点。读多场景下热点有利，写多并发场景下热点危险。报告不能只写 key 数量，还要写分布，例如最大 key 占比、前十 key 占比、长尾长度。

项目中可以先对热点 key 做本地特殊计数，再把长尾交给哈希表。这样减少哈希探测和共享写。这个策略连接 histogram、partition 和分布式热点 shard。一个局部哈希表问题，最终会变成系统负载均衡问题。

四、实验 比较四种统计方式：节点式 unordered map、排序后线性合并、字典编码加数组桶、热点 key 特化加长尾哈希。输入覆盖均匀字符串、少量热点、大量长尾和批次复用。报告记录每条记录平均查询次数、分配次数、内存峰值、cache miss 替代指标和最终输出一致性。这样读者会看到复杂度、地址序列和业务分布怎样共同决定数据结构。

3.12 补充案例：工作集大小如何决定算法阶段

工作集是当前阶段频繁访问的数据集合。它可能是一个数组、一个哈希表、一个 block、一个 batch，也可能是任务队列和状态表。工作集能否放进 L1、L2、LLC 或内存，会直接改变算法选择。

一、分块的本质是控制工作集 矩阵乘分块的目的，是让一小块 A、B、C 在 cache 中重复使用。日志处理也能分块：每次处理一个 batch，把热字段、局部 histogram 和输出 buffer 控制在 cache 友好的大小。分块不是矩阵专用技巧，而是让工作集适合层级存储。

Batch 太小，调度和函数调用开销高；batch 太大，cache 和 TLB 压力上升，错误恢复范围变大。选择 batch size 要同时看缓存、I/O 和恢复。一个纯算法角度的最佳块大小，不一定是系统最佳块大小。

二、哈希表工作集随 key 基数变化 事件统计中，key 数少时局部表可以留在 cache；key 数大时，每次更新可能访问随机 bucket。若 key 基数随输入变化，性能曲线会出现拐点。报告要测不同 key 基数，而不是只测一个固定分布。

当局部表超过 cache，可以考虑分区处理：先按 key hash 把记录分成多个 partition，再分别聚合。这样增加一次数据重排，但每个 partition 的工作集变小。这个策略在单机和分布式 shuffle 中同源。

三、工作集和并发度互相影响 每个线程都有自己的局部工作集。线程数增加，总工作集可能超过 LLC；每线程 partial 增加，内存峰值也增加。一个单线程 cache 友好的算法，多线程后可能因为总工作集过大而变慢。报告要记录每线程工作集和总工作集。

四、实验 固定总输入，改变 batch size、key 基数和线程数。记录吞吐、cache miss 替代指标、TLB miss 可用性和内存峰值。让读者观察拐点，并解释它对应哪个缓存层级或内存预算。工作集分析能把“快慢变化”变成可解释现象。

3.13 从对象地址推导工作集模型

缓存、TLB 和虚拟内存最容易被讲成层次结构图：L1、L2、L3、内存、页表、磁盘。图有用，但还不够。工程中真正需要的是工作集模型：程序在一段时间内会反复访问哪些地址，这些地址落在多少 cache line、多少页面、多少 TLB 项里，访问顺序是否让硬件预取器看得懂，写入是否让多个核心争抢同一条 cache line。把对象名字换成地址序列以后，很多看似软件层面的选择都会变成硬件问题。

一、数组快不是因为数组高级，而是地址可预测 连续数组的优势来自两个事实：相邻元素通常位于相邻 cache line，下一次访问地址可以由当前地址加固定步长得到。硬件预取器擅长这种模式，TLB 也能用较少项覆盖较大的连续区域。链表慢也不是因为链表这个抽象低级，而是因为下一跳地址保存在当前节点里，处理器拿到当前节点前不知道下一个地址。乱序执行可以并行处理独立 miss，却很难提前追逐单条指针链。读者应把“数组和链表复杂度都是线性”继续细分为：数组的线性扫描是带宽问题，链表的线性扫描常常是延迟问题。

二、工作集大小决定局部性是否真实 一个算法声称有局部性，必须说清局部性窗口。若热数据只有几十 KiB，它可能长期留在 L1 或 L2；若热数据是几 MiB，就要看 L3 和内存带宽；若随机访问跨越数 GiB，TLB miss 和页表遍历会进入主路径。哈希表是很好的例子：平均复杂度看起来是 $O(1)$ ，但一次查询可能触发哈希计算、bucket 访问、节点访问和字符串比较，每一步都可能落在不同 cache line。若 key 很短、表负载低、bucket 连续，表现很好；若 key 很长、节点分散、rehash 频繁，常数会大到压过算法复杂度。

三、页不是透明的背景 虚拟内存让每个进程拥有连续地址空间，但硬件执行时仍要把虚拟地址翻译成物理地址。TLB 缓存翻译结果，页表保存完整映射。大数组顺序扫描时，TLB miss 相对可控；随机访问大表时，工作集可能超过 TLB 覆盖范围，导致翻译开销和数据 miss 叠加。大页能扩大 TLB 覆盖范围，但会影响内存分配、碎片、NUMA 放置和权限粒度。教学中不能把大页讲成“开了就快”，应把它放进地址序列和部署约束里分析。

四、布局优化要回答谁在同一条线 结构体布局的核心问题是哪些字段会一起读，哪些字段会一起写，哪些字段属于冷路径。若热循环只读 `id` 和 `score`，却把很长的错误消息、调试字段和状态字符串放在同一结构体里，每次扫描都会搬运不需要的数据。把热字段拆成列式数组可以减少带宽，但会改变接口、构造成本和缓存行为。反过来，过度列式化也会让本来一起使用的字段分散。布局优化没有固定答案，只有访问模式、对象生命周期和接口稳定性的共同约束。

五、用实验建立地址直觉 本章实验建议把同一份数据做成三种形态：连续数组、索引数组和指针节点。每种形态都跑顺序访问、固定步长访问和随机访问，记录吞吐、cache miss、TLB miss 和内存峰值。然后再加入冷热字段拆分、arena 分配和大页配置。若结果和预期不一致，不要急着改代码，先检查编译器是否优化掉循环、数据规模是否超过缓存、线程是否被调度到不同核心、输入是否真的随机。地址直觉不是靠背层级图得到的，而是靠一次次把对象还原成可测的地址序列得到的。

3.14 贯穿案例：同样是线性扫描，为什么一个像顺流，一个像追尾

考虑一个日志聚合作业。版本一把每行解析成 `LogRecord` 对象，对象里有用户字符串、事件字符串、时间戳、错误信息和若干指针，随后把对象指针放进哈希表。版本二先把热字段抽出来：时间戳放进连续数组，事件名经过字典编码变成整数，错误信息只在需要时通过 `RecordId` 回查。两版都处理每条记录一次，复杂度都可以被写成线性，但机器执行完全不同。第一版像在堆上追一串路标，第二版像沿连续道路顺序前进。

这个案例的目的不是说对象模型不好，而是让读者学会问：热循环真正碰了哪些字节，冷字段是否被无意搬进 cache，指针跳转跨了多少页，TLB 能否覆盖当前工作集，硬件预取器能否猜到下一地址。缓存和 TLB 不应该是章节标题里的名词，而应该是代码设计时能被追问的具体成本。

一、从对象生命周期看热字段和冷字段 解析阶段需要原始字符串和错误上下文，聚合阶段通常只需要事件 id、时间窗口和数值。若两阶段共用同一个胖对象，聚合时会反复把冷字段所在的 cache line 带进来。冷热拆分的动机就从这里出现：不是为了追求列式布局这个名字，而是因为不同阶段需要不同字段。拆分以后，热路径更紧凑，冷数据仍可通过 record id 找回。若错误报告要求保留完整原文，冷数据不能丢，只是不能让它占据热循环带宽。

二、从一次哈希查询拆出地址序列 一次 `unordered_map` 查询表面是 $O(1)$ ，地址序列却可能包含多步：读取 key 字节计算哈希，访问 bucket 数组，跳到节点，比较字符串长度和内容，必要时继续链表或探测。每一步都可能落在不同 cache line。若 key 很多、节点分配分散、字符串不短，常数成本会非常高。相反，字典编码后用整数 key 查询紧凑数组，可能把多次随机访问压成一次连续或近似连续访问。复杂度符号没有错，但它隐藏了地址形状。

三、TLB 从“页数”而不是“元素数”进入模型 一百万个小对象如果分散在堆页上，热循环可能触碰大量页面；一百万个整数连续存放，页面数量仍然大，但访问顺序稳定，TLB miss 和预取行为更可控。TLB 的问题不是数据总量大这么简单，而是单位时间内活跃页面数量超过覆盖能力。Arena 分配、冷热拆分和紧凑数组都能减少活跃页面。大页也能扩大覆盖，但它不能修复乱跳的指针图，最多推迟拐点。

四、把布局优化写成可逆设计 高质量布局改造要保留语义回路。热数组里只有 event id，不代表系统失去事件名；字典表必须能从 id 反查字符串；错误路径必须能从 record id 找到原始行和 offset；checkpoint 必须保存字典版本。若布局优化让调试、恢复或输出变得不可解释，它就是把成本转移给后续章节。经典系统设计舒服的地方在于每个抽象都能回到证据，本书也要保持这一点。

五、实验要覆盖工作集拐点 这个案例的实验不应只测一个数据规模。应固定记录形状，逐步增加记录数和 key 基数，观察吞吐在哪些点下降。第一个拐点可能对应 L2 工作集，第二个可能对应 LLC，第三个可能对应 TLB 或内存带宽。然后改变布局：胖对象、冷热拆分、字典编码、排序合并。若拐点移动，说明布局确实改变了工作集；若不动，说明瓶颈可能在 I/O、分支或写出。实验目标是找成本迁移，而不是证明某容器永远快。

六、把地址报告服务后续并发 单线程地址序列报告不是孤立材料。后面加线程时，热数组是否按线程分片、partial 是否 false sharing、页面是否 first touch、NUMA 节点是否远端，都依赖这里的布局。若第三章把地址地图写清，后续并发优化就不是重新猜。比如每线程写自己的连续 partial，合并时按 key 顺序扫；或者每线程哈希表分散在堆上，合并时随机访问很多页。两种并行策略在代码层面都能工作，但地址地图会提前告诉我们风险。

3.15 案例深化：地址序列报告如何写

缓存和 TLB 的学习要落到地址序列报告。假设同样是统计 key 出现次数，版本一为每条记录分配节点并插入 `unordered_map`，版本二先把 key id 压成连续数组，再做局部排序和线性合并。两者在复杂度上都能被说成接近线性，但硬件看到的路径完全不同。一个追指针、跳桶、分配节点；另一个顺序扫描、局部排序、批量合并。报告若只写大 O，就看不到差异。

一、把对象图翻译成地址图 对象图描述程序员看到的关系：记录、字段、节点、桶、错误信息。地址图描述 CPU 看到的访问：连续数组、随机桶、堆节点、页边界、cache line。优化前后先画这两张图，读者能立刻看到热字段和冷字段是否混在一起，指针跳转是否跨页，扫描是否能被预取。地址图不是插图任务，而是性能推理工具。

二、分清冷运行和热运行 第一次读取文件可能触发缺页和页缓存填充，第二次运行更多是在

内存里扫描。若不区分冷运行和热运行，mmap、read、预读和页缓存的影响会被混合。实验应至少记录第一次运行、重复运行、改变输入文件后的运行。若冷运行慢而热运行快，瓶颈可能在 I/O 和缺页；若热运行仍慢，才更可能是 cache、TLB 或算法访问模式。

三、TLB 问题经常由布局间接制造 TLB miss 不只来自数据大，也来自有效工作集跨越太多页面。节点对象分散在堆上，可能让一个热循环同时触碰很多页；冷热字段拆分后，热路径页数下降，TLB 覆盖就改善。大页可以扩大覆盖，但若程序仍然随机追指针，只是把问题推迟。先改善布局，再考虑大页，这个顺序通常更稳。

四、哈希表报告要写分布 哈希表性能不能只看平均查找。要报告 key 分布、装载因子、桶长度、rehash 次数和分配次数。热点 key 多时，局部聚合可能比直接全局 hash 更稳；key 空间小且可压缩时，数组或排序合并可能比 hash 更可预测；key 很稀疏时，hash 仍然合适。不同结构的选择来自数据形状，而不是来自“某容器快”。

五、把内存证据连接到后续章节 地址序列报告会被后面复用。SIMD 章节需要知道数据是否连续；线程章节需要知道多个 worker 是否写同一 cache line；NUMA 章节需要知道页面放在哪个节点；I/O 章节需要知道读取是顺序还是随机。第三章如果写清地址行为，后面的优化就有共同地图。

3.16 案例复盘：复杂度相同，地址序列不同

两个统计程序都可以声称是线性复杂度：一个把记录做成堆节点再进哈希表，一个把热字段压成连续数组后排序合并。小数据下差异不明显，数据一大，节点版出现 cache 和 TLB 拐点。原因不在大 O，而在 CPU 看到的地址序列：连续 cache line、随机桶、堆节点、页边界和缺页。

把对象图翻译成地址图 对象图说明记录、字段、桶和错误信息的关系；地址图说明它们落在哪些页和 cache line。节点对象把热字段和冷字段散在堆上，哈希桶引入随机跳转；连续数组让热路径容易预取，但需要 record id 回溯冷数据。优化不是“换容器”，而是改变硬件访问轨迹。

实验判据 报告要区分冷运行和热运行，记录 page fault、cache miss、TLB miss、分配次数和 key 分布。哈希表还要写装载因子和桶长度。若扫描变快但错误定位丢失，优化不合格，因为系统只是在性能和可恢复性之间做了未声明交换。

3.17 本章习题

习题与作业

习题一：在当前项目中为 partial 数组设计两个布局，一个紧凑，一个按 cache line 对齐。说明每个字段的大小、对齐、每个 cache line 可能容纳几个 partial，以及为什么这个设计能隔离 false sharing。

习题二：写三个遍历版本：连续数组遍历、随机索引数组遍历、链表遍历。要求三者处理相同数量的逻辑元素。报告中分别分析 cache line、TLB、预取器和内存级并行度，不允许只写“随机慢”。

习题三：设计一个首次触页实验。说明如何分离分配、初始化、预热和计算；如何记录 page-faults；在没有 NUMA 机器时，哪些结论不能验证，哪些结论仍然可以通过 page fault 观察。

习题四：阅读 `mm/memory.c`、`mm/mmap.c` 或对应内核版本中的 fault 路径入口。写一份不超过两页的源码阅读报告，要求把用户态实验现象和内核路径联系起来，不要罗列无关函数。

NUMA、互连与缓存一致性

先看一个并行归约事故：单线程求和很稳定，四线程以后吞吐几乎不涨，八线程还变慢。代码里没有复杂锁，只是每个线程对同一个全局 atomic 计数器做 `fetch_add`。从 C++ 看，它已经没有数据竞争；从硬件看，所有核心在反复争夺同一条 cache line 的修改权。再换一个版本，每个线程写自己的 `partial`，最后合并，吞吐突然恢复。这里逼出的不是某个协议名，而是一个事实：共享读容易复制，共享写必须协调所有权。

本章从这个共享写热点进入 NUMA、互连和缓存一致性。多核心 CPU 不是把多个核心简单放在一起。核心之间通过片上互连通信，共享某些缓存层级，连接内存控制器，并用一致性协议维护共享内存语义。多 socket 机器还会引入 NUMA：不同核心访问不同内存节点的成本不同。只有把“哪个 cache line 被谁写、哪个页面属于哪个节点、合并阶段穿过哪条互连”说清楚，并程序的扩展曲线才有解释。

4.1 从共享内存到一致性流量

在 C++ 程序里，多个线程似乎可以直接读写同一地址。但硬件上，写一个共享 cache line 需要让其他核心的副本失效或更新。一个原子自增不是普通加法，它需要获得 cache line 独占权，并按内存序约束发布结果。争用越激烈，cache line 在核心之间来回迁移越频繁。

这解释了为什么全局计数器扩展性差。四个线程分别做本地计数，最后合并，通常比四个线程同时 `fetch_add` 同一个原子变量快。前者减少共享写，后者让每次更新都进入一致性协议热路径。

MESI 的直觉 很多一致性协议都可以用 MESI 直觉理解：Modified 表示本核心有被修改且尚未写回的独占副本，Exclusive 表示本核心有未修改的独占副本，Shared 表示多个核心可以读，Invalid 表示副本无效。当一个核心要写 Shared 行时，需要让其他核心副本失效；当其他核心随后要读，又要重新获取数据。

软件看不到这些状态名，但能看到它们的后果。一个只读共享表可以被多个核心同时缓存，扩展性很好；一个频繁更新的共享 head 指针会让 cache line 在核心之间跳来跳去；一个包含多个线程本地字段的结构体如果没有 padding，也会因为 false sharing 触发同样的迁移。

真实处理器可能使用 MESIF、MOESI 或其他变体，但教学时先抓住一个核心事实：读共享容易扩展，写共享需要取得所有权。Modified 或 Owned 这类状态的具体名称不重要，重要的是写入会触发状态迁移，状态迁移会占用互连和缓存层级资源。高性能并发设计的第一反应，不应该是“换一条更快的原子指令”，而应该是“能不能减少共享写”。

```
read mostly table:
core0 Shared
core1 Shared
core2 Shared

hot atomic counter:
core0 Modified -> core1 Modified -> core2 Modified -> core0 Modified
```

上面的文本图表达两种完全不同的共享。前者多个核心一起读，硬件可以复制；后者多个核心轮流写，所有权不断迁移。很多扩展性问题就藏在这条差异里。

一致性和内存模型的分工 缓存一致性解决的是“同一内存位置的多个副本如何保持一致”，内存模型解决的是“不同内存操作之间能被哪些线程以什么顺序观察”。硬件一致性不等于程序自动无数据竞争。没有同步的普通读写仍然可能在 C++ 层面形成未定义行为。

一致性协议通常围绕状态机工作，例如 Modified、Exclusive、Shared、Invalid 这类状态。软件不需要手写这些状态，但必须理解状态迁移的成本：共享读便宜，共享写贵，频繁写同一行最贵。

C++ 内存模型和硬件一致性的边界也要分清。硬件一致性通常保证同一 cache line 或同一地址的副本最终按协议协调，但 C++ 程序如果对普通变量并发读写而没有同步，语言层面就是数据竞争。语言规则先决定程序是否有定义，硬件规则再决定有定义的同步怎样落地。不要用“我的 CPU 有缓存一致性”来为未同步普通变量辩护。

Store buffer 和可见性 核心执行 store 后，写入可能先进入 store buffer。对本线程来说，后续 load 可能通过 store forwarding 看到自己的写；对其他核心来说，这个写何时可见还受缓存一致性和内存序约束影响。锁释放、release store 和内存屏障会把这种可见性变成可依赖的同步关系。

这就是为什么“我在写 flag 前已经写了 data”在并发中不够。编译器和硬件都可能在允许范围内重排或延迟可见性。正确发布对象需要用 release/acquire、mutex 或其他同步机制建立 happens-before。

Store buffer 还会影响锁的性能。释放锁时，持锁线程在临界区内的写入必须以满足同步语义的方式对后续拿锁线程可见。获取锁时，线程可能需要等待锁所在 cache line 的最新状态。低竞争时这条路径很短；高竞争时，锁变量所在 cache line 会成为所有核心争夺的热点。锁的成本不是一个固定常数，而是竞争、拓扑和写入路径共同决定的结果。

4.2 从页面位置到拓扑感知

NUMA 机器上，内存离某些核心更近。线程在节点 A 上运行，却频繁访问节点 B 分配的内存，会产生远端访问。Linux 的 first-touch 策略让首次触碰页面的线程影响页面放置，因此并行初始化和线程绑定会影响后续计算性能。

NUMA 优化的基本原则是让计算靠近数据。大数据按块分给线程时，初始化也应按相同块并行完成。跨节点共享队列和全局分配器要谨慎，因为它们可能把远端访问和同步争用叠加在一起。

NUMA 不只出现在多 socket 服务器上。有些桌面和移动平台也有非均匀的缓存、内存或芯片 let 拓扑，只是操作系统暴露方式不同。教材里使用 NUMA 这个词，是为了让读者形成拓扑意识：核心不是一个无差别集合，内存也不是一个无差别池。线程、数据和 I/O 设备之间的位置关系会影响成本。

First-touch 和并行初始化 Linux 常见 first-touch 策略意味着页面通常分配到第一次写入它的线程所在 NUMA 节点。如果主线程单线程初始化一个大数组，然后工作线程分布在多个节点上处理这个数组，很多工作线程可能访问远端内存。更好的方式是让每个工作线程初始化自己后续要处理的分块。

这条规则在 benchmark 中尤其重要。若初始化方式和计算方式不一致，测到的可能是内存放置错误，而不是算法本身。实验报告必须记录初始化策略、线程绑定和 NUMA 策略。

互连不是无限带宽 核心、缓存、内存控制器和 I/O 设备之间的互连带宽有限。一个程序即使单线程局部性很好，多线程扩展后也可能撞上内存带宽或互连带宽上限。此时继续增加线程，只会让每个线程分到更少带宽，吞吐不再提高。

互连瓶颈常常表现为“每个线程单独跑都快，一起跑就都慢”。原因可能是多个核心争用同一内存控制器，多个 socket 之间远端访问过多，或者共享写导致 cache line 在互连上来回移动。排查时要把线程数、绑定位置、数据放置和访问模式作为实验矩阵，而不是只改算法代码。

4.3 从拓扑记录到分层资源管理

拓扑感知不是过早优化，而是避免把硬件当作平面资源池。线程池可以按 NUMA 节点分组，内存池可以按节点分配，队列可以按生产者或消费者分片，归约可以先在 socket 内合并再跨 socket 合并。这样的层次化设计和分布式系统中的本地聚合很像：先减少近处流量，再把少量结果发到远处。

Linux 观察入口 在 Linux 上，拓扑可以从 `lscpu`、`/sys/devices/system/cpu/`、`/sys/devices/system/node/` 和 `numactl--hardware` 开始观察。线程绑定可以用 `taskset`、`numactl--physcpubind` 或程序内亲和性 API。内存策略可以用 `numactl--membind`、`numactl--interleave` 和 first-touch 实验观察。若权限允许，`perf` 可以帮助识别 cache-to-cache 流量，`perfstat` 的 cache、LLC、context-switch 和迁核指标也能提供间接证据。

源码阅读可以从调度拓扑、NUMA 策略和内存分配路径切入。不要试图一次看完整调度器或完整内存管理。更好的问题是：线程迁移时哪些状态会变化，first-touch 影响页面放置的路径在哪里，NUMA 策略怎样被记录和查询，某个 benchmark 的现象能不能从这些路径得到合理解释。

层次化归约案例 并行归约是理解拓扑的好例子。最差设计是所有线程对同一个原子变量累加。改进设计是每个线程写自己的 partial，最后一个线程合并。进一步的拓扑感知设计是每个核心或每个 worker 先本地累加，每个 NUMA 节点内再合并，最后跨节点合并。这样做减少共享写，也减少远端通信。

这个结构和分布式计算中的树形聚合完全一致。不同只是成本单位变化：单机里远处是另一个 NUMA 节点，分布式里远处是另一台机器。理解这条相似性，可以帮助读者把第二册前后内容串起来。

一致性流量的三个等级 共享内存程序里的数据共享可以分成三个等级。第一个等级是只读共享，例如配置表、只读索引、常量权重、不可变输入数组。多个核心可以同时缓存这些数据，扩展性通常很好。第二个等级是低频写共享，例如阶段结束时写一次完成标志、每秒汇总一次指标、偶尔替换一份配置快照。它需要同步，但不一定成为瓶颈。第三个等级是高频写共享，例如所有请求更新同一个原子计数器、所有 worker 争用同一个队列 tail、多个线程反复写同一 cache line 上的不同字段。这个等级最危险，往往表现为核心数越多越慢。

优化时要先判断共享等级，而不是先决定使用 mutex、atomic 还是无锁结构。mutex 保护低频复杂状态可能完全合理；atomic 保护高频热点计数器仍然可能很慢；无锁队列如果所有生产者都争用同一个 tail，也逃不开一致性流量。把共享写变成分片写、批量写、线程本地写或按 NUMA 节点写，通常比替换同步原语更根本。

这也是第二册区分“正确性同步”和“扩展性设计”的原因。正确性同步回答的是数据竞争、可见性和不变量；扩展性设计回答的是共享写频率、cache line 迁移和拓扑距离。一个程序可以同步正确但扩展性很差，也可以局部很快但同步语义错误。高质量系统必须同时满足二者。

目录协议和窥探协议的直觉 一致性协议在小规模系统中可以用广播窥探思想理解：一个核心想写某条 cache line，就向其他核心广播或通过互连通知，让其他副本失效。核心数量增加后，纯广播会变贵，因此很多系统使用目录式信息，记录某条 cache line 可能在哪些核心或节点有副本，再有针对性地发送消息。具体实现非常复杂，但软件层面的直觉很简单：共享范围越大，维护一致性的通信越难便宜。

这条直觉能解释为什么“全局共享结构”在小机器上看不出问题，在大机器上突然变差。四核心机器上，一个全局队列的 cache line 迁移还勉强能承受；多 socket 服务器上，迁移可能跨互连和 NUMA 节点，延迟和带宽成本都更高。工程设计不能只在开发机上看正确性测试通过，还要在目标拓扑上看扩展曲线。

目录和窥探的差别不需要读者背协议细节，但要形成层次化思维。让数据只在一个核心私有，成本最低；让数据在一个 socket 内共享，成本较低；让数据跨 socket 高频写共享，成本高；让数据跨机器共享，成本更高。线程本地 partial、socket 内合并、跨 socket 合并、跨机器 reduce，就是把这个成本阶梯显式写进算法。

内存控制器和带宽饱和 NUMA 节点通常连接自己的内存控制器。一个节点上的线程大量访问本地内存，会消耗本地控制器带宽；远端访问则需要经过互连到另一个节点的内存控制器。若所有线程都访问一个节点上的内存，即使 CPU 核心分布在多个节点上，带宽也可能集中压在一个控制器上。此时线程数增加，吞吐未必提高，因为瓶颈不是核心数量，而是某个内存节点的带宽。

first-touch 实验的价值就在这里。主线程初始化全部页面，可能让所有页面落在一个 NUMA 节点；worker 分布到多个节点计算时，远端访问集中出现。按 worker 分块初始化，可以让页面分散到各自节点，使多个内存控制器共同工作。若实验显示 parallel first-touch 版本带宽更高，就说明优化来自页面放置和控制器并行，而不是加法本身改变。

带宽饱和也有反直觉现象。使用更多线程可能让总吞吐略增，但每线程吞吐下降；使用 SMT 可能让内存延迟隐藏更好，也可能只增加争用；把线程平均分到两个节点可能提高总带宽，也可能因为跨节点合并阶段变慢。结论必须来自实验矩阵，不应写成固定经验。

拓扑感知队列 队列是互连成本的集中点。一个全局 MPMC 队列很容易成为共享写热点：所有生产者争用 tail，所有消费者争用 head 或槽位状态。低线程数下它简单好用，高线程数下它可能把所有 worker 绑在一组 cache line 上。拓扑感知设计通常会拆成多级队列：每个 worker 或每个 NUMA 节点有本地队列，只有负载不均时才访问其他队列。

线程池的工作窃取就是这个思想。worker 优先使用本地 deque，空闲时才偷别人的任务。NUMA 感知线程池还可以先在同节点 worker 中窃取，再跨节点窃取。这样做不是为了复杂而复杂，而是为

了让高频路径局部化、低频路径承担远端成本。

有界队列的背压也要考虑拓扑。若所有生产者把任务推到单个队列，队列满时所有生产者都等待同一个条件变量，唤醒和竞争集中；若每个分片有自己的队列，热点分片仍会背压，但无关分片可以继续执行。分片设计会增加任务路由和负载均衡复杂度，因此要先测出全局队列确实是瓶颈，再拆分。

Linux NUMA 策略和迁移 Linux 提供多种 NUMA 相关策略：默认 first-touch、按节点绑定、交错分配、自动 NUMA balancing 等。自动机制可以在一般场景改善局部性，但它不是性能实验的替代品。自动迁移页面本身有成本，统计窗口和迁移决策也可能与 benchmark 生命周期不匹配。短时间 benchmark 尤其容易受到初始化策略影响。

用户态实验应记录三类信息。第一，线程位置：线程是否绑定，绑定到哪些 CPU，是否发生迁移。第二，页面位置：是否通过 first-touch、mbind 或 interleaved 控制，是否能观测各节点内存使用。第三，访问形状：线程是否主要访问本地分块，是否有跨节点共享写，合并阶段是否集中到一个节点。只记录 `numactl--hardware` 不够，必须把拓扑和程序行为联系起来。

源码阅读可以围绕 `mm/mempolicy.c`、NUMA balancing 相关路径和调度拓扑展开。不要试图一次读懂所有内存管理。合格阅读报告应选择一个问题，例如“为什么主线程初始化会影响页面节点”，然后追踪 VMA 策略、缺页处理和页面分配之间的关系。

C++ 接口如何表达拓扑 C++ 标准库不直接暴露 NUMA 拓扑，但工程代码可以在接口层表达位置意识。例如线程池可以接收 worker 拓扑描述；内存池可以按节点创建 arena；任务可以带有 preferred node；归约可以返回分层 partial；队列可以按 shard id 提交。接口不需要一开始绑定某个 Linux API，但要给后续拓扑策略留下位置。

下面是一个概念性的任务描述。

Listing 4.1 任务元数据中保留位置偏好

```
1 struct TaskPlacement {
2     std::int32_t preferred_worker = -1;
3     std::int32_t preferred_numa_node = -1;
4 };
5
6 struct ComputeTask {
7     std::size_t begin = 0;
8     std::size_t end = 0;
9     TaskPlacement placement;
10 };
```

这段代码不是要读者立刻实现完整 NUMA runtime，而是提醒接口设计不要把任务看成无位置的函数。对数组块任务，`begin/end` 暗含数据位置；对分布式 shuffle，partition id 暗含目标节点；对线程池，本地队列暗含 worker 位置。位置语义一旦在接口中完全丢失，后面很难再补回来。

拓扑不是核心数列表 很多程序只读取“有多少 CPU”，然后创建同样数量线程。这不足以描述现代机器。CPU 拓扑包含 socket、NUMA node、core、SMT sibling、cache 共享层级、内存控制器和 I/O 设备位置。两个逻辑 CPU 可能是同一个物理核心的 SMT sibling，也可能在同一个 socket 的不同核心，也可能跨 socket。它们之间共享资源和通信成本完全不同。

性能报告应至少记录 `lscpu` 中的核心数、线程数、socket 数、NUMA 节点和 cache 信息。在多 socket 机器上，还应记录每个 NUMA node 的 CPU 列表。线程池设计也要区分 worker id 和 CPU id。worker id 是运行时内部编号，CPU id 是操作系统调度对象，NUMA node 是内存位置。把它们混为一谈，会让绑定和 first-touch 实验失去意义。

互连成本如何出现 多核心之间通过片上网络、环、mesh 或 socket 间互连通信。软件不会直接调用“互连”，但互连成本会出现在 cache line 迁移、远端内存访问、跨 socket 锁竞争和共享 LLC 争用中。一个全局原子计数器在单 socket 小机器上看起来还能接受，在多 socket 上可能因为 cache line 在 socket 间来回迁移而急剧变慢。

互连成本也会出现在读多写少结构中。只读共享通常可扩展，因为多个核心可以缓存副本；写

入会让其他副本失效。若一个配置对象每秒更新一次，问题不大；若一个统计字段每次请求都写，问题很大。软件层面要区分只读共享、低频写共享和高频写共享。这比盲目选择 atomic 或 mutex 更重要。

目录协议的工程影响 目录协议维护某条 cache line 的副本位置或所有权信息。软件工程师不需要实现目录协议，但要理解它的结果：共享范围越大，一致性维护越贵；写者越分散，所有权迁移越频繁；数据越能保持本地私有，扩展性越好。分片、线程本地、per-NUMA partial 和分层合并，都是把共享范围缩小。

这也解释为什么“每个线程写自己的数组下标”不一定安全高效。如果这些下标落在同一 cache line，硬件仍然认为它们共享同一个一致性单位。软件的逻辑独立必须映射成硬件的物理独立，才真正减少一致性流量。

First-touch 的细节 First-touch 通常发生在页面第一次被写入或实际分配物理页时。只调用 `reserve` 或 `malloc` 可能只获得虚拟地址，不一定分配物理页。第一次写入触发缺页，内核选择物理页所在 NUMA 节点。若主线程初始化全部数组，页面可能集中在主线程所在节点；后续其他节点线程访问就变成远端访问。

因此 NUMA 实验要把分配、初始化和计算分开。分配阶段只申请地址；初始化阶段按目标线程分块写入，建立页面位置；计算阶段使用同样分块读取。若初始化也计入总时间，要单独报告，因为 parallel first-touch 可能把初始化变快或变慢。若只关心计算阶段，就要在计时前完成初始化并预热。

```
NUMA experiment phases:
allocate virtual memory
initialize pages by owner worker
optionally warm up
compute with the same worker-to-range mapping
merge partials by node then globally
```

线程绑定的收益和风险 线程绑定可以减少调度迁移，让 worker 更稳定地访问本地 cache 和本地 NUMA 内存。它也可能降低操作系统调度灵活性。如果绑定策略错误，把多个 heavy worker 绑到同一个核心或同一个 NUMA 节点，就会变慢。绑定还要考虑系统上其他进程、容器配额和可用 CPU 集合。

实验中，绑定的价值是减少变量。未绑定实验可能因为操作系统迁移而样本波动；绑定后更容易解释线程和数据位置。但生产系统是否绑定，要看服务类型和部署环境。批处理计算通常更适合显式绑定；通用服务可能需要给调度器更多空间。教材应要求读者记录绑定策略，而不是强制所有程序绑定。

分层归约 NUMA 友好的归约通常分层。每个 worker 计算本地 partial；同一 NUMA 节点内先合并成 node partial；最后跨节点合并。这样高频读写发生在本地，跨节点通信只在较小 partial 上发生。这个结构也对应分布式系统中的 local combine、node-level reduce 和 global reduce。

```
hierarchical reduce:
worker partials: no sharing during hot loop
node partials: merge within NUMA node
global result: merge node partials
```

分层合并的代价是代码复杂度和额外阶段。若数据很小或机器只有单 NUMA 节点，收益不明显。实验要比较平坦合并和分层合并，并记录每阶段耗时。不要把 NUMA 策略写成不可关闭的复杂路径。

跨节点队列的代价 全局队列在 NUMA 机器上常成为跨节点热点。生产者和消费者分布在不同节点，队列锁、head/tail 和槽位状态会在节点间迁移。分片队列或每节点队列可以减少这种迁移。任务优先进入本节点队列；本节点空闲时先偷同节点任务，再跨节点偷。这个策略和分布式调度中的数据本地性一致。

有界队列的容量也可以按节点分配。若每个 NUMA 节点有自己的输入队列，上游可以根据数

据位置提交任务；某个节点过载时，其他节点不必被同一个全局队列阻塞。缺点是负载均衡更复杂，任务路由需要知道位置。设计要看工作负载是否有明显数据本地性。

远端释放和内存池 NUMA 不只影响分配，也影响释放。一个对象在节点 A 分配，由节点 B 释放，可能导致分配器元数据和 cache line 在节点间移动。高性能分配器常使用线程本地或节点本地缓存，但跨线程释放仍然需要特殊路径。对象池设计应考虑 owner：对象最好由分配它的线程或节点回收，跨节点释放可以先放入 remote free 队列，再由 owner 批量回收。

无锁数据结构的回收协议也会遇到这个问题。Hazard pointer 或 epoch 延迟释放后，最终由哪个线程 delete 节点？如果所有删除都集中到一个线程，可能造成远端释放和内存热点。内存管理和拓扑不能完全分开。

I/O 设备和 NUMA 网卡、NVMe 和 GPU 等设备通常连接到某个 NUMA 节点附近。线程处理设备 I/O 时，如果运行在远端节点，可能增加 PCIe 和内存访问成本。高性能网络和存储系统常把中断、队列、缓冲区和处理线程放在同一 NUMA 节点。虽然本册第二册不深入设备驱动，但要让读者知道 I/O 也有拓扑。

分布式 shuffle 如果在真实服务器上运行，网络接收线程、解压线程和 reduce worker 的位置会影响吞吐。若网卡在节点 0，接收缓冲在节点 0，但 reduce worker 在节点 1 读取，远端内存流量会增加。拓扑意识不只属于计算内核，也属于 I/O 管线。

自动 NUMA balancing Linux 自动 NUMA balancing 会采样页面访问，并可能迁移页面到更常访问它的节点。它能改善一些长期运行程序，但也会引入额外 page fault、迁移成本和实验不确定性。短 benchmark 可能还没等自动机制收敛就结束；长服务可能在负载变化时发生页面迁移。

实验报告应记录是否启用自动 NUMA balancing，至少要知道它可能影响结果。若要做可控实验，可以使用 numactl、线程绑定和明确 first-touch。若要评估生产行为，则应在接近真实配置下测量，并记录自动迁移指标。不要让内核自动策略成为看不见的变量。

容器和 cgroup 的影响 现代部署常在容器里运行。容器可能限制 CPU 集合、内存节点、CPU 配额和内存上限。程序看到的 `std::thread::hardware_concurrency` 未必等于实际可用核心数；cgroup 配额可能让线程周期性被 throttled；cpuset 可能只允许访问部分 CPU。性能报告若忽略容器限制，就可能误判扩展曲线。

在 Termux、手机或云环境中，也要承认设备限制。手机通常没有多 socket NUMA，但有大小核、温度和频率限制；云主机可能共享物理资源；容器可能禁止读取某些性能事件。教材项目应记录环境，而不是假设每个读者都有理想服务器。

拓扑感知不等于硬编码 接口应表达拓扑信息，但不应把某台机器的拓扑硬编码进算法。正确做法是运行时发现或配置拓扑，然后把任务、内存池和队列映射到抽象 node。算法只知道 preferred node 或 shard id，不直接写死 CPU 编号。这样同一代码可以在单 socket、双 socket、容器和手机上运行，只是策略不同。

Compute Systems Engine 可以从简单配置开始：`worker_count`、是否绑定、每个 worker 的 node id。没有 NUMA 信息时，所有 worker 属于 node 0。后续在支持的 Linux 机器上再读取真实拓扑。这样项目不会因为缺少硬件而无法运行，也不会放弃拓扑扩展。

4.4 实验边界、降级和项目落地

没有 NUMA 机器时，不能声称验证了远端内存优化；但仍可以验证 false sharing、线程绑定、任务粒度和内存带宽曲线。单 socket 上 parallel first-touch 和主线程初始化可能差不多，这是正常结果，不是实验失败。报告要写清：本环境只能验证某些机制，不能验证跨 socket 页面放置。

有 NUMA 机器时，也不能只跑一次。需要比较绑定与未绑定、主线程初始化与 worker 初始化、平坦合并与分层合并、单节点内存与 interleave 内存。只有多组对照一致，才能说 NUMA 放置参与了性能结果。

案例：双 socket 归约 在双 socket 机器上，大数组归约可以展示 NUMA 的完整链条。版本一由主线程初始化数组，然后所有 worker 分块求和。版本二由每个 worker 初始化自己的块，再求和。版本三按 socket 分组：每个 socket 的 worker 求本地 partial，socket 内合并，再跨 socket 合并。若版本二和三明显更好，说明页面放置和分层合并参与了性能。

实验要记录线程绑定。若 worker 在计算阶段被迁移到其他 socket，first-touch 的页面位置和线程位置又不匹配。还要记录数组大小，太小可能全部被 cache 影响，太大才体现内存控制器。这个

案例把第3章的 page fault、第4章的 NUMA、第13章的分层 reduce 连在一起。

案例：跨 socket 队列 另一个案例是全局任务队列。所有 worker 从同一个队列取任务，队列锁和 head/tail 位于某个内存节点。跨 socket worker 频繁访问，会让队列状态在 socket 间迁移。本地队列加工作窃取可以减少高频跨 socket 访问：worker 优先本地 pop，空闲时才偷取。

实验可以构造大量短任务，让队列成本显著；再构造少量长任务，让队列成本被计算掩盖。这样读者能看到：同一个队列设计在不同任务粒度下表现不同。NUMA 优化不能脱离任务粒度。

分层资源管理 NUMA 机器适合分层资源管理。每个节点有本地 worker、本地内存池、本地队列、本地 partial。跨节点只交换压缩后的 partial 或被窃取的任务。这个结构和分布式系统的节点本地聚合非常像。若在单机上已经学会分层，扩展到集群时会更自然。

分层也会增加复杂度。每个层级都需要指标：本地队列长度、跨节点 steal 次数、节点 partial 大小、远端访问比例。没有指标，分层策略可能只是让代码复杂。项目可以先用单层实现，再在实验分支中加入分层，对比收益。

大小核和 NUMA 的区别 手机常见大小核，服务器常见 NUMA。二者都意味着执行资源不均匀，但原因不同。大小核是核心性能和能耗不同，同一内存访问拓扑未必像多 socket 那样分节点；NUMA 是内存位置和访问延迟不同，核心本身可能同构。调度策略也不同：大小核更关注把重计算放大核、后台任务放小核；NUMA 更关注线程和内存同节点。

在 Termux 环境中，读者更可能遇到大小核而不是多 socket NUMA。实验报告应避免把大小核现象误称为 NUMA。若线程数增加后性能下降，可能是小核加入、降频、温度或调度迁移，不一定是远端内存。记录环境是第一步。

拓扑抽象接口 项目可以定义一个抽象拓扑接口，而不是直接依赖某个 Linux API。接口返回 `worker_count`、`node_count`、`worker_to_node`、是否支持绑定。单 socket 或手机环境返回一个 node；双 socket 服务器返回多个 node。算法根据 `node_count` 决定是否启用分层 partial。这样代码能在不同设备上运行。

Listing 4.2 拓扑描述的接口形态

```
1 struct WorkerTopology {
2     std::int32_t worker_count = 1;
3     std::int32_t node_count = 1;
4     std::vector<std::int32_t> worker_to_node;
5 };
```

这个结构不直接包含 CPU id，是为了让教学项目先表达策略，再逐步接入真实 affinity。若绑定不可用，仍然可以使用 `worker_to_node` 做逻辑分层实验。接口应允许能力缺失，而不是让项目只能在服务器上运行。

拓扑指标 拓扑感知优化需要指标验证。每个 node 的任务数、处理字节、partial 大小、队列长度、跨节点 steal 次数、远端调度次数都应记录。若启用分层后跨节点 steal 很多，说明负载不均或任务划分不好；若某个 node 长期空闲，说明调度没有利用资源；若 node partial 合并很慢，说明最终合并成为瓶颈。

没有这些指标，拓扑策略容易变成不可解释的复杂代码。项目应先输出简单文本摘要，后续再画图。

拓扑和故障域 拓扑不只影响性能，也影响故障域。单机里，一个 NUMA 节点过载可能拖慢本地 worker；集群里，一个 rack、zone 或机房故障会影响一组节点。分布式副本放置通常要跨故障域，避免一处故障丢失多数副本。单机拓扑意识可以自然扩展到集群故障域意识：不要把所有关键副本、任务或队列集中在同一个风险位置。

Compute Systems Engine 的本机模拟可以先用 node id 表示逻辑故障域。后续多进程或多机版本中，node id 可以映射到主机、rack 或 zone。调度器在追求本地性的同时，也要避免把所有副本放同一域。性能和可靠性有时会冲突，系统设计要显式权衡。

拓扑章节总结 本章的核心不是让读者背 NUMA API，而是建立位置意识。线程有位置，内存有位置，队列有位置，I/O 设备有位置，副本也有位置。位置决定通信成本和故障相关性。没有位置意识的程序在小机器上可能正常，在大机器上会遇到扩展性和可靠性问题。

拓扑决策表 做拓扑相关设计时，先问机器是否真的有多多个内存节点，是否能设置亲和性，数据是否大到超过 cache，任务是否访问固定数据块，是否有跨节点共享写，是否需要跨故障域放置副本。若没有 NUMA 或数据很小，复杂拓扑策略可能无收益；若数据大且任务固定访问某块，first-touch 和绑定值得实验；若共享写频繁，先分片再谈绑定；若是分布式副本，优先考虑故障域而不是只考虑延迟。

拓扑策略的降级设计 拓扑感知代码必须能降级。很多读者会在手机、容器、云虚拟机或普通单 socket 机器上运行实验，程序不一定能读取完整 NUMA 信息，也不一定有权限设置 affinity。如果代码把真实 NUMA 当成必需条件，就会让教材项目难以运行。更好的策略是把拓扑能力分层：能读取真实 node 时使用真实 node；不能读取时退化成一个逻辑 node；能绑定线程时执行绑定；绑定失败时记录失败并继续运行；能控制内存策略时做 first-touch 实验；不能控制时只做普通分块。

降级不等于忽略拓扑。即使只有一个逻辑 node，代码仍然可以保留 worker 到 node 的映射、分层 partial 的接口和队列分片的抽象。这样程序在小机器上正确运行，在大机器上能启用更多策略。教材项目尤其适合这种设计：先让抽象存在，再逐步接入真实硬件能力。读者不会因为没双 socket 服务器就学不了拓扑意识。

拓扑策略还要写入输出。若报告只写“启用 NUMA 优化”，但实际上 affinity 设置失败，读者会得到错误结论。程序应输出能力检测结果、降级原因和最终策略。例如 `node_count` 是 1，`binding_enabled` 是 false，`binding_error` 是 permission denied，`memory_policy` 是 default。这样实验结果才诚实。高性能系统的第一步不是让所有优化都打开，而是知道哪些优化真的生效。

降级设计还有一个好处：它迫使接口表达意图，而不是表达某个平台的偶然细节。算法真正需要的是“这个任务最好靠近哪一组数据”和“这个 worker 属于哪个执行域”，不一定需要知道具体 CPU 编号。平台能力强时，执行域可以映射到真实 NUMA 节点；平台能力弱时，执行域只是逻辑分组。把意图和机制分开，代码才不会因为换设备而到处改。

这种抽象也能保护学习路径。读者先在普通设备上理解任务、数据和执行域的关系，再到服务器上验证真实远端访问成本。先学清楚问题，再打开平台能力，实验结论会稳得多。

NUMA 和一致性章的回归阅读应围绕一个并行归约推进。第一版所有线程更新一个全局计数器，暴露 cache line 所有权反复迁移；第二版每线程 partial，消除热写但仍可能出现 false sharing；第三版按 NUMA 节点分块初始化和合并，观察 first-touch 和远端内存；第四版把全局队列改成分层队列，减少跨节点争用。每一步都只改变一个因素，才能说明瓶颈从哪里迁移。

一致性协议在本章只提供硬件背景，不替代 C++ 内存模型。它解释为什么多个核心写同一条 cache line 会贵，为什么共享只读和共享写不是一回事，为什么 padding 有时有效。但程序正确性仍要靠锁、原子和明确的同步关系。把 MESI 当作语言级保证，是本章必须避免的误区。

本章验收要允许设备差异。手机或单 socket 机器可能没有可观察 NUMA，但仍能完成 false sharing、全局队列争用和线程本地 partial 实验；有多 socket 机器时，再加入 first-touch 和绑定策略。报告不能把某台机器上的绝对数字写成普遍规律，应写成“这种数据放置在这种拓扑下产生了这种成本”。

一致性成本要落到 cache line 所有权。两个线程更新不同变量，如果变量落在同一条 cache line 上，硬件仍可能把这条 line 在核心之间来回转移；这就是 false sharing 的根。把字段 padding 到不同 line 可能降低转移，但也会增加内存占用和 cache footprint。教材不能只说“加 padding”，还要让读者计算对象数量、额外字节和是否真的位于热写路径。

NUMA 策略要区分初始化、访问和合并。first-touch 只影响页面最初归属，后续线程迁移、重新分配和文件映射都可能改变实际成本；线程绑定只约束执行位置，不自动移动内存；本地 partial 能减少远端写，但最终合并仍然要跨节点读。项目报告应把这三步分开写：页面由谁触碰，热循环由谁访问，合并由谁执行。这样读者才能看出拓扑优化到底解决哪一段成本。

互连成本还会改变数据结构选择。一个全局无锁队列在单 socket 上可能还能接受，跨 socket 时每次 head 或 tail 更新都可能拉动 cache line 所有权；一个全局 hash table 的分片锁如果没有按 NUMA 节点分层，热点 bucket 会把远端访问集中到少数 line；一个线程本地 partial 如果最后由单线程合并，合并阶段又可能变成串行远端读。拓扑意识不是只给线程绑核，而是让共享状态也分层。

本章可以让读者写一个降级策略。若机器没有 NUMA，就运行 false sharing 和分层 partial；若机器有多 NUMA 节点，就增加 first-touch、线程绑定和远端访问对照；若没有 PMU 权限，就用吞吐曲线和内存带宽估算替代计数器。高质量教材应告诉读者在不同设备上如何诚实验证，而不是假设每个人都有同一台服务器。

4.5 把共享内存放回拓扑和一致性协议

多核机器给人的错觉是“所有核心访问同一块内存”。程序模型可以这么说，性能模型不能这么说。不同核心有自己的私有缓存，多个核心通过一致性协议维护共享数据的可见性；多 socket 或多 NUMA 节点机器上，内存控制器和物理页面也有位置。一次读写看似普通，实际可能涉及 cache line 所有权、远端内存、互连带宽和页面迁移。NUMA 章要从共享写和页面归属推出来，而不是从硬件拓扑图开始背。

最小例子是全局计数器。单线程里它只是一个变量；多线程里每个 worker 都更新它，cache line 所有权会在核心之间迁移，吞吐不会线性增长。把计数器改成 atomic 只修复数据竞争，不会消除共享写热点。把计数器改成线程本地 partial，再做分层合并，才真正改变共享模式。这里的推导非常关键：正确性原语和扩展性结构不是同一件事。

一致性协议提供可见性基础，不替代内存模型

MESI 之类状态模型适合建立直觉：一条 cache line 可以被多个核心共享读取，但写入需要获得独占或修改权限。若两个核心反复写同一条 line，即使写的是不同字段，也会发生所有权迁移。这个直觉足以解释 false sharing、热点 atomic 和全局队列锁为什么扩展性差。教材不需要让读者背每个协议变种的全部状态，但要让读者理解“写共享比读共享贵得多”。

硬件一致性和 C++ 内存模型分工不同。硬件负责在给定体系结构规则下维护缓存一致性，C++ 内存模型定义程序中哪些跨线程读写有数据竞争、哪些原子操作建立 happens-before。把普通变量交给硬件一致性并不能让数据竞争变成正确程序；把所有操作都改成 seq_cst 也不能消除热点 line 的性能成本。后续原子章会继续展开，本章只需要把边界立住：正确性要按语言模型证明，性能要按一致性流量和拓扑分析。

一致性流量也会影响锁。一个 mutex 不只是“进入临界区”这么简单，它内部会有原子状态、等待队列和可能的 futex 系统调用。低争用时锁成本很低，高争用时锁变量所在 line 会成为热点，等待线程可能睡眠和唤醒。锁章会讲不变量和条件变量，本章先让读者看到锁的硬件侧成本：临界区越短不一定越好，若进入次数极高，锁本身的 cache line 迁移就可能主导。

NUMA first-touch 和页面归属

NUMA 的第一条工程规则是 first-touch：页面通常归属第一次触碰它的节点。若主线程在 node 0 初始化一个大数组，随后 node 1 上的 worker 大量访问这批页面，worker 就会读远端内存。这个问题在小机器上可能看不出来，在双 socket 服务器上会让扩展曲线突然变差。多线程初始化不是形式主义，而是页面放置策略。

实验要设计成能触发差异。版本一由主线程初始化大数组，再让所有 worker 分块求和；版本二由每个 worker 初始化自己将要处理的块，再在同一绑定策略下计算；版本三故意交换 worker 和数据块，制造远端访问。报告要记录线程绑定、页面大小、输入规模、内存带宽和 NUMA 设备情况。若当前手机或笔记本没有 NUMA，就把实验写成设计和单节点替代，不把结论伪装成已验证。

NUMA 策略还要考虑迁移。Linux 的自动 NUMA balancing 可能把页面迁移到访问频繁的节点，但迁移本身有成本，也可能在 benchmark 中造成波动。生产程序不能只依赖“系统会帮我调好”。对于稳定大任务，显式 first-touch、线程绑定和数据分片更可控；对于动态任务，运行时要记录数据位置，让调度尽量先本地再远端。

互连带宽、设备拓扑和分层设计

互连不是无限总线。多个核心同时访问远端内存、争用共享 line 或从同一个内存控制器读数据时，互连和内存控制器会成为瓶颈。线程数增加后变慢，不一定是算法错了，也可能是远端访问、LLC 争用或内存带宽已经到顶。分层归约和本地队列的共同思想，是先在近处完成大部分工作，再把少量结果跨节点合并。

设备也有拓扑。网卡、NVMe、GPU 或其他设备连接在特定 PCIe root complex 下，离某些 CPU/NUMA 节点更近。虽然第二册第二册不进入 GPU AI 算子，但现代计算系统必须知道：I/O 线程、内存 buffer 和设备队列最好考虑拓扑。否则一个线程在远端节点准备 buffer，再交给近端设备 DMA，路径会绕远，cache 和 IOMMU 成本也可能增加。

分层设计不等于硬编码拓扑。项目可以定义一个 Topology 抽象：列出 worker 所在逻辑 CPU、NUMA node、缓存组和可用设备；调度器根据抽象做本地优先策略；没有 NUMA 的设备上退化成单节点策略。这样教学代码不会把某台机器的编号写死，也能让读者理解“拓扑感知”和“不可移植硬编码”的区别。

项目中的拓扑验收

贯穿项目在本章应完成 sharded counter、NUMA-aware block placement 和分层 reduce 三个检查点。Sharded counter 用来证明共享写热点；block placement 用来证明页面归属；分层 reduce 用来证明先本地合并再跨节点合并的结构。每个检查点都要保留不使用拓扑的 baseline，并写出何时 baseline 反而更好，例如小输入、单 socket、线程数少、合并成本大于远端成本。

源码阅读可以从 Linux 调度域、NUMA balancing 和内存策略入口开始，不要求一次读懂内核全部路径。读者只需要建立映射：用户态的线程绑定对应内核 task 的 CPU mask，页面触碰对应 VMA 和 page fault，NUMA 统计对应页面位置和访问事件。能把用户态实验现象映射到这些对象，本章就达到了目的。

本章最后要形成一句判断：凡是多个核心反复写同一份状态，先怀疑一致性流量；凡是多个节点访问大工作集，先确认页面归属；凡是线程数增加后不再加速，先画拓扑和共享写路径，再决定是改算法、改布局、改调度还是接受带宽上限。

案例走查：线程本地 partial 为什么还要分层合并

把全局 atomic counter 改成每线程 partial 后，共享写热点消失，但最终合并仍可能成为单点。若线程数少、partial 小，串行合并很好；若 partial 是大数组，且线程分布在多个 NUMA 节点，单线程跨节点读取所有 partial 会访问远端内存。分层合并先在每个节点本地合并，再跨节点合并少量结果，减少远端流量。

实验在单 NUMA 设备上也能做简化：把 partial 按 worker 分组，比较串行合并和树形合并的阶段耗时；若有 NUMA 设备，再加 first-touch 和绑定。指标包括每阶段合并字节、合并时间、线程空闲时间和最终 checksum。若没有 NUMA，报告写清只验证了算法结构，未验证远端内存收益。

这个案例说明拓扑优化不应写死。项目代码可以提供 `Topology::groups()`，单节点返回一个组，多节点返回多个组。算法按组做分层，而不是硬编码 socket 编号。这样同一份代码能在手机、笔记本和服务器的降级运行。

NUMA 章节的项目代码形状

拓扑感知不应散落在每个算法里。项目可以定义 `CpuTopology` 和 `PlacementHint`：前者描述 worker 属于哪个组，后者描述任务最好在哪个组运行。算法只产生 hint，例如 input block 属于 group 2；运行时负责尽力调度；无法满足时记录一次 remote execution。这样拓扑是可观策略，而不是硬编码条件。

`CpuTopology` 在没有 NUMA 的设备上返回一个组，所有任务本地执行。双 socket 机器上返回多个组。大小核设备可以按性能等级分组。这个抽象让同一章节能服务手机、笔记本和服务器的用户。现在在 Termux 上，也可以先实现单组版本，保留接口，后续换机器再验证多组策略。

代码还要避免让拓扑策略破坏正确性。任务无论在哪个 worker 执行，输出 checksum 必须相同；拓扑只影响性能和 locality，不影响语义。测试先随机打乱 placement，确认结果一致；再开启本地优先策略，测性能。这样读者知道调度策略和业务正确性是分开的。

一致性流量的观测替代

不是所有设备都能直接测 cache coherency traffic。可以用反例间接观察：全局 atomic 计数器、紧凑 partial 数组、填充 partial 数组、线程本地 vector 四个版本。若全局 atomic 随线程数增加很快饱和，而线程本地版本扩展更好，说明共享写路径参与成本。若填充 partial 改善紧凑 partial，说明 false sharing 参与成本。

报告要避免过度断言。没有专用 uncore 计数器时，不应写“互连流量为多少”，而应写“结果符合共享写热点假设”。若能在服务器上使用 uncore 或 numastat，再补更强证据。教材要教读者根据证据强度说话。

源码阅读可以从调度拓扑和内存策略文档开始，再看 `kernel/sched/topology.c`、`mm/mempolicy.c`、`mm/huge_memory.c` 的入口。只读入口，不追求深挖每个分支。能把 first-touch、线程绑定和 page migration 映射到内核概念，本章目的就达到了。

综合练习：拓扑感知归约的降级设计

本章综合题要求读者为归约写一个拓扑感知版本，并且必须支持没有 NUMA 的设备。接口先定义拓扑组，不直接暴露 socket 编号。单组设备上，所有 worker 属于组 0；多节点设备上，worker 按 NUMA node 分组；大小核设备上，可以按性能等级分组。算法只依赖组抽象：组内 partial 先合并，组间再合并。这样设计能在手机上运行，也能在服务器上验证远端成本。

实现时先写不感知拓扑的 baseline，再写分层归约。分层归约不允许改变最终语义，输出 checksum 必须一致。若浮点归约顺序改变，要写误差合同或固定合并树。数据初始化也要分版本：主线程初始化、worker 本地初始化、故意交叉访问。没有 NUMA 硬件时，只验证接口和合并树；有 NUMA 时，再验证 first-touch 和绑定收益。

报告中必须有降级说明。若系统无法读取 NUMA 拓扑，运行时如何退化；若线程绑定失败，是否继续运行；若拓扑组信息和实际性能不符，指标如何暴露。工程设计不能把“我的机器支持”当成前提。用户在 Termux 上也应能跑通功能，只是某些性能结论标记为未验证。

这个题还能训练边界意识。拓扑感知若写入算法核心，后续每个内核都要处理平台差异；若放在运行时和 placement hint 中，算法只描述数据位置偏好。好的设计不是到处写 if，而是把硬件差异收束到少数接口。

容易出错的边界：拓扑感知什么时候不值得

拓扑感知不是越复杂越好。若输入很小，所有数据都在 LLC 内，NUMA 策略可能没有收益；若任务极不均匀，强本地优先可能让某些 worker 空闲；若系统是手机或单 socket 机器，复杂 NUMA 分组只会增加代码负担；若数据很快被压缩成本地 partial，远端访问的时间占比可能很低。教材必须告诉读者：先测是否存在远端成本，再引入拓扑策略。

第二个反例是“绑定线程一定稳定”。绑定可能稳定实验，也可能破坏调度器对大小核、温控和后台任务的管理。移动设备上，固定大核长时间运行会触发降频，后半段反而慢。服务器上，绑定不当可能把线程集中到同一 LLC 组或同一 NUMA 节点，造成资源争用。绑定策略必须和拓扑、输入、持续时间一起报告。

第三个反例是“atomic 修复了共享写问题”。Atomic 修复数据竞争，不修复 cache line 所有权迁移。热点 atomic 仍然可能把互连打满。正确修复往往是减少共享写频率，例如线程本地 partial、分片、批量合并或改变算法。这个反例会在锁章和原子章再次出现，本章先从硬件成本角度立住。

源码阅读可以把 Linux 的自动 NUMA balancing 当作反例材料。系统可能迁移页面，但迁移不是免费，也不是每个 workload 都能判断正确。用户态设计若完全依赖自动迁移，就会把性能决定权交给难以观察的后台机制。更稳的方式是让项目记录 placement 和实际执行位置，至少知道策略是否被执行。

本章核心接口：PlacementHint

NUMA 章给项目留下 PlacementHint。任务可以声明 preferred group、data block id、expected bytes、can migrate 四个信息。运行时尽量按 hint 调度，但不保证一定满足；如果未满足，就在 trace 中记录 remote 或 fallback。这样语义和性能分开：任务在哪里跑都应正确，但 hint 可以帮助解释性能。

后续任务运行时和分布式调度会复用这个思想。本机是 NUMA group，集群里就是数据本地节点；本机 remote execution 只是慢，集群 remote execution 可能多一次网络读取。接口稳定后，概念能自然迁移。

从硬件模型进入测量模型

前四章给出硬件侧解释，但解释必须通过测量落地。知道 cache line、TLB、NUMA、互连和一致性，只能产生候选假设；要判断哪一个假设支配当前程序，还需要实验矩阵、计数器、profile 和 trace。下一部分从测量开始，是为了防止读者把硬件知识当成凭经验猜瓶颈的工具。

项目在这里应暂停一次，整理前四章产生的候选指标：热循环 cycles 和 instructions，地址序列和工作集，false sharing 对照，first-touch 和线程绑定。下一章把这些指标组织成报告格式。这样硬件知识不会停留在文字，而会变成后续每次优化都能复用的证据模板。

4.6 项目落地

Compute Systems Engine 在本章应加入三个设计点。第一，拓扑记录：benchmark 输出可见 CPU 数、用户指定 worker 数、是否启用绑定、是否能读取 NUMA 信息。第二，分层 partial：即使当前只实现每 worker partial，也在设计文档中说明未来可以扩展成每 NUMA 节点 partial。第三，队列分片路线：线程池初版可以用全局队列，但要记录全局队列可能成为共享热点，后续工作窃取章节再拆成本地队列。

本章的验收不是一定在手机或单 socket 机器上测出 NUMA 差异，而是实验设计正确。没有 NUMA 设备时，报告应写明限制：可以验证 false sharing、线程绑定和内存带宽曲线，但不能验证

远端内存放置。承认证据边界，比编造一个漂亮结论更专业。

一致性流量 多个核心读同一份数据通常容易扩展，多个核心写同一条 cache line 会制造所有权来回迁移。原子计数、共享队列尾指针、全局统计表都是典型热点。分析时先问“谁写，写在哪里，频率多少”。

NUMA 归属 页面通常在首次触碰时分配到某个 NUMA node。若主线程初始化全部数据，worker 在线程绑定后访问远端页面，性能会被隐藏地拉低。first-touch 不是小技巧，而是数据归属策略。

拓扑感知调度 线程绑定不是越死越好。绑定能稳定实验和保护本地性，也可能降低调度器处理负载不均的能力。工程报告要写清绑定策略、CPU 列表、容器 cpuset 和不可用时的降级。

分层合并 跨 node 共享写昂贵时，应先在 core 或 node 本地生成 partial，再分层合并。这个原则后面会出现在 reduce、histogram、shuffle 和分布式 combine 里。

远端访问预算 远端访问不一定绝对禁止，但要有预算。若本地 node 忙而远端空闲，远端执行也许更快；若数据巨大而计算很轻，移动线程比移动数据更合理。NUMA 策略要服务 workload。

自动迁移 Linux 可能尝试自动 NUMA balancing，但迁移页面有成本，也可能判断不准。用户态项目不能把性能正确性完全交给后台机制，至少要记录页面放置和线程位置。

容器环境 手机、虚拟机、容器和云实例可能隐藏部分拓扑信息。教材要教读者写“不可验证”的报告，而不是编造 NUMA 结论。可观察 CPU mask、调度迁移次数和缓存行为仍然有价值。

项目接口 PlacementPlan 应描述数据分片、建议 node、执行 worker、实际 CPU、partial 合并层级和远端访问估计。它不是调度器的全部实现，却让后续运行时有可讨论的对象。

案例：主线程初始化反例 先由主线程初始化大数组，再由绑定到另一个 node 的 worker 扫描；再改成 worker 本地 first-touch。比较两者，说明页面归属不是抽象名词。

案例：共享计数器热点 所有线程 `fetch_add` 同一计数器，再改成每 node partial。观察扩展曲线，从一致性流量解释为什么线程越多越慢。

案例：分层归约 core 本地 partial、node 本地 partial、全局合并三层实现同一 reduce。这个案例连接硬件拓扑和并行算法。

源码阅读路线 Linux 源码阅读从 `kernel/sched/topology.c`、`mm/mempolicy.c`、`mm/huge_memory.c` 附近进入即可。阅读目标不是把整套子系统抄进书里，而是追一条和本章实验相关的路径：用户态动作怎样变成内核对象，内核对象怎样排队、等待、唤醒、计数、提交或回收。

阅读报告要写证据等级。能直接测到的数字放在第一层；从源码控制流推导出的解释放在第二层；当前设备无法验证、只能作为合理猜测的内容放在第三层。这样读者会知道拓扑放置报告中哪些结论可复现，哪些结论需要换机器继续确认。

源码阅读还要克制。看到一个复杂函数，不要把每个分支都展开；先回到本章的问题，问它能否解释一个等待点、一次状态转移或一条性能曲线。解释不了的内容可以留到索引，不要打断正文主线。

主线程初始化反例 先由主线程初始化大数组，再由绑定到另一个 node 的 worker 扫描；再改成 worker 本地 first-touch。比较两者，说明页面归属不是抽象名词。

共享计数器热点 所有线程 `fetch_add` 同一计数器，再改成每 node partial。观察扩展曲线，从一致性流量解释为什么线程越多越慢。

分层归约 core 本地 partial、node 本地 partial、全局合并三层实现同一 reduce。这个案例连接硬件拓扑和并行算法。

一致性所有权

多个核心读同一 cache line 容易扩展，多个核心反复写同一 line 会让所有权在核心之间迁移。原子热点和 false sharing 都来自这个边界。

实验设计：让所有线程写同一计数器，再改成每线程槽位，比较扩展曲线。

NUMA 归属

页面通常由第一次触碰的线程决定物理节点。主线程初始化全部数组后让远端 worker 访问，可能把远端内存成本藏进计算阶段。

实验设计：分别用主线程初始化和 worker 本地初始化，记录带宽和线程绑定策略。

first touch

first touch 不是仪式，而是把数据归属和执行位置对齐。它要求初始化循环和计算切分采用同一拓扑视角。

实验设计：把分片初始化函数加入项目，要求每个分片记录建议 node 和实际 worker。

拓扑感知

CPU 列表、socket、core、SMT 和容器 cpuset 都会影响实验。绑定策略不能只写线程数，要写可见 CPU 和不可见限制。

实验设计：在 Termux、容器或普通 Linux 中记录可见 CPU mask，并说明哪些 NUMA 结论无法验证。

分层 partial

共享写昂贵时，先在线程本地或 node 本地生成 partial，再树形合并。这个原则后来会出现在 histogram、reduce 和 shuffle combine 中。

实验设计：为每个 worker 保存本地桶，再按 node 合并，最后全局合并。

自动迁移

操作系统可能迁移线程或页面。迁移有时修正局部性，有时破坏可重复实验。报告要区分默认调度和人为绑定。

实验设计：先跑默认策略，再跑固定亲和性，并记录迁移次数或替代指标。

互连瓶颈

当核心数增加后吞吐不再增长，原因可能不是算术，而是内存控制器、互连或一致性目录压力。证据要从共享写和远端访问入手。

实验设计：把任务从共享队列改成分片队列，观察是否减少所有权迁移。

队列槽位

多生产者队列的 head、tail 和状态字段容易形成共享热点。队列设计必须把控制字段与数据槽位分开看。

实验设计：给队列状态加对齐和分片指标，说明哪些字段被频繁写。

降级策略

没有 NUMA 设备时仍能学习原则：用线程绑定、false sharing、共享计数器和队列热点模拟拓扑成本。不能验证远端内存时要诚实说明。

实验设计：报告列出可验证项、替代实验和不可验证项。

项目接口

PlacementPlan 不只是记录 node 编号，还要连接数据分片、worker、partial、合并层级和证据来源。没有这个对象，拓扑优化会散落在命令行里。

实验设计：每个分片输出建议位置、实际位置和合并路径。

4.7 共享内存真正共享的是 cache line、页面和互连带宽

NUMA 和一致性章节要从一个反直觉现象开始：线程越多，程序越慢。初学者会怀疑线程池写错了，或者认为 CPU 核心不够；但如果所有线程都 `fetch_add` 同一个计数器，真正发生的是同一条 cache line 的所有权在核心之间来回迁移。共享内存模型让地址看起来统一，硬件却必须维护“谁能写这条线”的协议。读者理解这一点，才会知道为什么线程本地 partial 是原则，不是微优化。

一致性流量和 false sharing 可以用同一张图解释。多个核心读同一条线，硬件可以复制；多个核心写同一条线，写者需要独占；多个线程写相邻数组元素，如果这些元素落在同一条 cache line，也会互相干扰。源码里变量不同，不代表硬件里位置不同。实验不要一上来讲复杂 NUMA，先让每线程计数槽位相邻，再加 `alignas(64)` 或填充，对比扩展曲线。这个小实验能解释大量并发性能问题。

NUMA 把问题再放大一层。页面由哪个 node 提供，线程在哪个 CPU 上运行，决定了访问是本地还是远端。主线程初始化一个大数组，然后把任务分给其他 node 上的 worker，可能让所有 worker 都访问主线程所在 node 的内存。first touch 的意义就在这里：初始化和计算使用同样的切分，让页面归属靠近使用者。它不是一条命令，而是一种数据放置策略。

但是绑定也不是万能。线程绑得太死，负载不均时调度器无法帮你迁移；容器或手机环境可能隐藏拓扑；云实例的 NUMA 信息也可能受虚拟化影响。教材要教读者写限制：我能看到哪些 CPU，

能否看到 node, 能否读 `/proc/self/numa_maps`, 能否使用 `numactl`。不能验证远端访问时, 就退回 false sharing、共享计数器和线程迁移实验, 不要编造 NUMA 结论。

本章的工程设计落到分层 partial。日志统计可以先按 worker 聚合, 再按 node 合并, 最后全局合并。这样做的好处不只是减少锁竞争, 也是在拓扑上减少跨 node 写入。代价是合并阶段更复杂, 内存占用增加, 结果提交要等更多阶段完成。教材要把收益和代价一起讲: 若输入很小, 分层合并可能不值得; 若 key 高度倾斜, 分层只能缓解部分热点; 若任务调度频繁跨 node, 局部性仍会被破坏。

源码阅读可以从调度拓扑、内存策略和 numa maps 开始。用户态不需要照搬内核调度器, 但要学会它如何描述 CPU 层级、负载和亲和性。读者做报告时, 应把实际 CPU、建议 node、实际 worker、partial 所在位置和合并路径写在一起。这样后续任务运行时和分布式 combine 才能复用本章知识: 跨 node 的远端内存, 在集群里会变成跨机器网络; 线程本地 partial, 在分布式里会变成 map-side combine。

案例推导: 全局计数器如何一步步变成分层聚合

旧版本统计错误数量时, 所有 worker 对同一个 `std::atomic` 计数器执行 `fetch_add`。单线程很快, 四线程也还能接受, 线程再多后吞吐下降。表面看, atomic 已经避免数据竞争; 实际瓶颈是所有核心争夺同一条 cache line 的修改权。这个案例适合让读者区分“正确”和“可扩展”: atomic 让结果正确, 却没有让共享写变便宜。

第一步改成每线程槽位。每个 worker 写自己的计数, 最后合并。若槽位相邻, 仍可能 false sharing, 因此要让槽位按 cache line 分隔, 或把多个热计数组织成线程本地结构。第二步按 NUMA node 合并。每个 node 内部先合并成本地 partial, 再跨 node 合并, 减少远端写。第三步把这个原则推广到 histogram: 每线程本地桶、每 node 合并、最后全局合并。

这个优化也有代价。读取“当前全局计数”不再是一次 load, 而要合并多个槽位; 内存占用增加; 线程数量变化时槽位管理更复杂; 若 key 数量很大, 每线程桶可能爆内存。教材不能只说“分片更快”, 还要告诉读者什么时候不值得。低频计数、少线程、小输入, mutex 或 atomic 可能更简单; 高频共享写和多核扩展, 分层 partial 才值得。

实验要有扩展曲线。横轴是线程数, 纵轴是吞吐和每条记录成本; 版本包括全局 atomic、相邻槽位、对齐槽位、node partial。若设备没有 NUMA, 就至少完成前三个版本, 并在报告中写明 NUMA 不可验证。若能读取拓扑, 就记录 worker 绑定、页面 first touch 和合并层级。不要只写最终数字, 要写瓶颈从共享写迁移到内存带宽或合并成本的过程。

Linux 源码阅读可以看调度拓扑和内存策略, 但读者真正要带走的是一个工程问题: 数据在哪里被首次触碰, 之后由谁频繁写。贯穿项目中, PlacementPlan 应让每个分片知道建议位置 and 实际执行 worker。否则调度器、数据布局 and 合并算法会各自为政, NUMA 知识又会退回口号。

第一部分综合案例: 从一条记录的地址到一台机器的拓扑

第一部分四章合在一起, 要让读者形成一个完整判断: 一段程序慢, 不一定慢在源码看起来最复杂的地方, 而是慢在执行路径上最受限制的资源。我们用同一个日志解析任务贯穿。输入是一批文本行, 解析器找分隔符, 提取事件类型和用户 id, 本地聚合后写出 partial。先单线程运行, 再多线程运行, 再观察缓存和 NUMA 行为。这个任务不复杂, 正适合把硬件执行模型讲清。

第一步看热循环。解析器逐字节扫描, 如果每个字符都进入复杂状态机, 前端供给、分支预测和依赖链都会影响吞吐。优化前先看输入分布: 分隔符位置是否规律, 错误行比例是多少, 字段长度是否随机。若随机性高, 分支预测失败会增加; 若函数边界多, 前端和调用开销会增加; 若状态依赖强, 自动向量化可能失败。热循环报告要记录这些观察, 而不是只写“parser 慢”。

第二步看地址序列。解析后如果保存大结构体数组, 聚合只需要事件类型, 却会搬运原始文本指针、调试字段和错误信息。冷热分离把热字段放进连续数组, 减少 cache line 浪费, 也让 SIMD 和预取更容易。可是冷字段不能丢, 因为错误报告仍需要原始行和 offset。于是 AddressTrace 和 LayoutPlan 必须连接: 热路径看连续字段, 冷路径通过稳定 id 回查原始信息。硬件优化不能破坏可诊断性。

第三步看 TLB 和 page cache。输入来自文件时, 第一次读取和第二次读取成本可能不同; mmap 版本第一次访问页面时可能触发 minor fault; 普通 read 版本把读取和解析的生命周期显式化。报告要写清楚测的是冷文件、热 page cache, 还是解析后内存批次。若没有权限清理 page cache, 就用不同文件或大于内存的数据集替代, 并标注限制。这样的诚实比伪精确更有价值。

第四步看多线程后的共享写。单线程 parser 优化完成后, 多线程聚合可能不再慢在解析, 而慢在共享计数器。所有 worker 更新同一个 atomic 计数器, 会让同一条 cache line 的所有权来回迁移;

改成本地 partial 后，瓶颈可能迁移到合并阶段或内存带宽。这里要把第二章的热循环证据和第四章的一致性证据放在一张表里：优化前慢在哪里，优化后慢到哪里去了。

第五步看 NUMA 和拓扑。若主线程读取并初始化所有 batch，再把 batch 分给其他 node 上的 worker，远端访问可能隐藏在计算阶段。first touch 策略要求初始化和计算采用同样切分；线程绑定要求报告写清 CPU mask 和实际 worker。没有 NUMA 设备时，也可以用 false sharing 和线程迁移实验训练同一思维。不要因为环境限制就跳过拓扑，也不要无证据时写“NUMA 导致”。

这个综合案例的最终输出是一份硬件证据包。它至少包含：热循环反汇编摘要、输入分布、字段冷热表、地址序列说明、page cache 状态、共享写位置、partial 合并层级、线程绑定策略和不可验证边界。读者若能从这份报告解释“为什么某次优化有效、为什么下一次优化无效、瓶颈迁移到了哪里”，第一部分就达到了目标。硬件原理不是芯片图背诵，而是能把源码、地址、线程和拓扑放进同一条因果链。

实验：在支持 NUMA 的机器上，用 `numactl--hardware` 查看节点。写一个大数组初始化和求和程序，比较单线程初始化、多线程初始化、线程绑定和未绑定情况下的带宽。如果没有 NUMA 机器，也要用 `lscpu` 记录拓扑并解释限制。

从一个全局计数器推导一致性流量 多核程序最容易写出的错误性能模型，是认为多个核心同时加一个计数器就会线性变快。全局计数器在语义上只有一个变量，但在硬件上对应一个 cache line 的所有权反复迁移。核心 A 写完后，核心 B 要写就必须获得该 cache line 的独占状态，随后核心 C 又来抢。这个过程让互连上传输的不只是数据，还有所有权和失效消息。计数器越热，核心越多，迁移越频繁，程序越不像并行计算，越像所有核心排队修改同一小块内存。

因此本章要从全局计数器的事讲起。第一版每条记录直接更新全局 `std::atomic<std::uint64_t>`，正确但可能慢。第二版每个线程一个局部计数器，最后归并，减少共享写。第三版为局部计数器加 cache line padding，避免多个线程的计数器落在同一 cache line 上造成 false sharing。第四版按 NUMA node 做分层 partial，先节点内合并，再跨节点合并。每一步都不是技巧堆叠，而是把共享写从高频路径移到低频路径。

False sharing 要用内存布局解释 False sharing 的本质是两个逻辑无关的变量落在同一 cache line 上。线程 A 写变量 x，线程 B 写变量 y，源码上看没有共享；硬件上它们共享同一 cache line，写入仍会导致所有权争用。很多初学者只在“共享变量”上找问题，忽略了 cache line 是一致性的基本粒度。结构体数组、相邻计数器、队列头尾字段、worker 统计字段，都可能出现这种问题。

修复 false sharing 也不是到处 padding。Padding 会增加内存占用，可能降低缓存容量利用率。只有高频跨线程写字段才值得隔离。低频配置字段、只读字段或同一线程访问的字段不需要每个都占一行。项目里的 `WorkerStats` 应把每个 worker 高频写的 counters 放在独立 cache line，汇总线程低频读取；而共享配置和只读表可以紧凑存放。这样做体现的是访问模式驱动布局。

NUMA 的第一原则是内存有归属 NUMA 不是“内存慢一点”的抽象说法，而是页面有物理归属，核心访问本地节点和远端节点的成本不同。Linux 通常采用 first touch 策略，哪个 CPU 首次写入页面，页面就可能分配到对应节点。若主线程初始化全部大数组，然后 worker 分布在多个 NUMA node 上处理，很多 worker 会访问远端页面。正确做法可能是并行初始化，让每个 worker 触摸自己负责的页；也可能是显式设置绑定和内存策略；也可能是按 node 切分任务和数据。

贯穿项目中，输入 split、局部聚合表、shuffle buffer 和任务队列都应该有放置策略。线程本地 partial 若被分配在错误节点，局部性会丢失；work stealing 若随意跨节点偷任务，负载均衡可能换来远端内存访问；全局 merge 若集中到一个节点，会把所有 partial 拉过互连。NUMA 优化的目标不是永远不跨节点，而是把跨节点访问变成低频、批量、可解释的操作。

实验：从共享计数到分层归约 本章实验可以统计事件类型。输入分布固定，算法语义固定，逐步改变共享形态。版本一全局 atomic 计数；版本二每线程数组；版本三 padding 每线程数组；版本四按 NUMA node 分组；版本五把最终合并改成树形。每个版本记录吞吐、cache miss、远端访问指标、线程间方差和合并时间。若缺少硬件计数器，也至少记录不同线程数下的扩展曲线。

这个实验要特别关注反常情况。线程少时，全局 atomic 可能足够快，分层结构反而增加合并成本；事件类型很多时，每线程数组内存变大，缓存命中下降；热点极强时，局部聚合收益明显；NUMA node 很少或移动设备上拓扑简单时，复杂放置策略没有收益。教材要教读者解释这些差异，而不是给出固定答案。

Linux 源码阅读连接点 读 Linux NUMA 和调度相关代码时，关注任务迁移、CPU mask、内存策略和 page fault 后的放置。内核不会把“线程”当作孤立执行单元，它一直在处理任务、CPU、内存节点和负载之间的关系。读 RCU、per-cpu 变量和 slab 分配器时，也能看到把共享状态拆到 per-cpu 或局部缓存的思想。它们不是为了炫技，而是为了减少高频共享写和锁争用。

把这些思想带回 C++ 项目，读者应该能设计三层统计结构：worker-local、node-local、global。worker-local 追求无共享和 cache 热；node-local 负责同节点合并；global 只在阶段边界出现。这个结构会在后续并行归约、任务运行时和分布式 combine 中反复出现，是第二册的重要主线。

事故复盘：全局 atomic 为什么把拓扑拉进来 一个全局 atomic 计数器在四个线程下看起来还能接受，线程数继续增加后吞吐突然停住。代码里只有一次 fetch add，表面上比加锁短很多；硬件里却发生了 cache line 所有权在核心之间来回迁移。若线程跨 NUMA 节点，迁移还要经过互连。所谓共享内存并不是没有位置，cache line 的 owner、页的 home node、线程运行在哪个 CPU，都会进入成本模型。

这个事故能推出分层 partial。先让每个 worker 写自己的局部计数，避免热 cache line 在核心间抖动；再按 NUMA node 合并，减少跨节点流量；最后由一个线程汇总全局结果。这个结构改变了写路径，也改变了报告方式：不能只看最终计数，要记录每层 partial 的数量、合并时机、线程绑定和页面放置。Atomic 不是低级错误，它适合低频状态或线性化点，不适合高频共享写热点。

实验要把拓扑写进结论。线程数、CPU 列表、NUMA node、内存首次触碰位置、绑定策略都要可复现。若关闭绑定后结果波动很大，说明调度迁移已经影响 cache 热度；若本地 partial 快但最终合并长尾，说明瓶颈迁移到了归约阶段；若远端内存访问占比高，说明 first-touch 或分配策略有问题。NUMA 章不是让读者背拓扑名，而是让共享状态重新获得物理位置。

共享写实验实验判读 全局 atomic、线程本地 partial 和 NUMA 分层版本的结果要放在同一张扩展曲线里看。线程数少时全局 atomic 不一定差，线程数多后若吞吐停住，就检查 cache line 迁移和合并阶段。padding 后变快说明 false sharing 可能存在；padding 后不变，说明瓶颈可能不在同一行写。NUMA 结论必须写机器拓扑，不能从普通单节点设备外推。

这一段也服务于复习。读者合上书后，应该能凭本章的判读口径重新审查自己的代码：有没有最小正确版本，有没有能击穿旧方案的输入，有没有状态记录，有没有资源指标，有没有失败后的恢复动作。若五个问题缺两个以上，就说明实现还停在演示层，而不是工程层。

从 false sharing 反推结构体布局 false sharing 的教学案例可以从 worker 统计开始。每个 worker 只写自己的 completed、failed、stolen 计数，看起来没有共享；如果这些字段在数组里连续排列，多个 worker 可能写同一条 cache line。修复不是给所有结构体都 padding，而是先确认哪些字段被不同线程高频写，再只隔离这些字段。

报告里应画出逻辑字段和 cache line 的关系。字段 A 和字段 B 在源码上相邻，不代表它们应该共享缓存行；字段 C 和字段 D 虽然属于同一对象，但若一个只初始化时写、另一个每个任务都写，热度不同也应分开。布局不是按面向对象归属决定，而是按写者、频率和共享粒度决定。

NUMA 版本继续把这个思想放大。每个 node 内先合并局部 partial，跨 node 只在阶段边界交换汇总。若工作窃取跨 node 发生，要记录被偷任务访问的数据是否仍在远端。这样读者能看到单条 cache line 的所有权迁移和跨节点内存访问是一条连续问题，而不是两个独立概念。

实验课：从一个全局计数器走到分层 partial 第一版让所有 worker 对一个全局 atomic 计数器做加法。第二版改成每个 worker 一个计数器，最后串行合并。第三版给每个 worker 计数器加 cache line 隔离。第四版按 NUMA node 建 node local partial，再做全局合并。第五版把合并改成树形，避免单个线程长时间处理所有 partial。

每个版本都要在不同线程数下画扩展曲线，并记录每个 worker 的任务数、写入次数、合并时间和最终 checksum。若有工具支持，再记录 cache-to-cache 或 LLC 相关事件；没有工具时，也可以用 false sharing 对照实验：相邻计数器和隔离计数器在相同输入上的差异。实验的目标不是证明某个技巧永远快，而是观察共享写频率怎样改变系统形状。

判读时要把合并阶段单独列出。线程本地 partial 常常让热路径变快，但增加最后合并；padding 可能减少 false sharing，但增加内存占用；NUMA 分层可能减少远端访问，但如果任务负载不均，跨节点窃取仍会出现。报告要写成成本迁移：从共享写迁移到合并，从全局争用迁移到局部内存，从计算路径迁移到调度路径。

这个实验还要连接分布式思想。Worker local partial 对应线程本地聚合，node local partial 对

应节点内合并，最终 global merge 对应集群 reduce。读者如果理解这个层次，后面看到 map side combine、shuffle partition 和 reduce tree 时，就不会把它们当成新概念。

共享写练习验收重点 合格答案要画出共享 cache line 或共享页面。只说“用线程本地变量”不够，必须说明全局写如何变成局部写和阶段合并，padding 保护了什么字段，first touch 怎样影响页面位置。没有 NUMA 机器时，答案要诚实写只能验证 false sharing 和扩展曲线。

答案还要写“为什么不采用更复杂方案”。例如没有证据证明队列是瓶颈，就不应把锁队列换成无锁；没有证明输出顺序可变，就不应随意重排；没有证明错误路径能恢复，就不应只提交正常路径。能解释不做什么，说明读者开始具备工程判断。

最后，答案必须包含可复查材料。至少有输入摘要、关键状态、期望输出、实际输出、运行命令和未验证边界。没有这些材料，老师或同伴无法判断结论是否成立。第二册的作业不是让读者写漂亮段落，而是训练读者把系统问题变成可验证证据。

项目实施：NUMA、互连与缓存一致性 本章对应的贯穿项目实施不要求一次写成完整框架，但必须有最小可运行切片。这个切片包含三个部分：一个 reference，一个带本章机制的实现，一个能击穿 reference 之外隐含假设的测试。Reference 用来固定语义，机制实现用来展示本章知识，测试用来证明新增复杂度确实解决了问题。

代码组织上，不要把所有内容放进一个大文件。基础类型、输入生成、核心机制、报告输出和测试应分开。即使当前只是教学项目，也要让目录能表达责任边界。这样后续章节接入时，不会因为找不到状态归属而重新发明一套对象。

每个实现都要输出一段机器可读摘要。摘要至少包含输入规模、关键状态、错误数、输出校验、耗时和未验证边界。这里的机器可读不要求复杂格式，简单的 key value 文本也可以。关键是让脚本和读者都能复查，而不是只能看屏幕截图。

测试应包含正常路径、边界路径和失败路径。正常路径证明功能可用；边界路径证明长度、容量、尾部、空输入、关闭或超时没有遗漏；失败路径证明本章机制不是装饰。若失败路径写不出来，往往说明本章概念还没有落到工程对象上。

报告最后写一个取舍段。新增机制带来了什么好处，增加了什么成本，哪些场景不应该使用它，下一章会复用哪个字段或对象。这个取舍段非常重要，因为系统工程不是把所有高级机制都加进去，而是在证据和约束下选择合适的边界。

调试笔记：扩展曲线停住时分离热路径和合并路径 多线程扩展曲线停住后，先看热路径是否还在共享写，再看合并路径是否变成长尾。全局 atomic 停住，可能是一致性流量；线程本地 partial 停住，可能是内存带宽或合并成本；NUMA 版本停住，可能是跨节点偷任务或远端页面。每个解释都需要不同证据。不要把所有扩展失败都归因于线程太多。

复查清单：NUMA、互连与缓存一致性 复查本章时，先检查 reference 是否仍然存在。没有 reference，后面的并发、优化、异步或分布式改造都无法证明语义没有变。再检查压力输入是否尖锐：它应当攻击本章最核心的边界，而不是只把数据量放大。第三，检查状态是否可观察：关键对象的身份、状态、错误和提交结果应能在报告中找到。

第四，检查代码示例是否说明禁止行为。调用者不能做什么，往往比能做什么更能体现工程质量。第五，检查实验是否写出未验证边界；当前设备、权限或输入无法证明的结论，必须标注。第六，检查本章对象是否能被下一章复用，不能复用就说明边界还不稳。

完成这六项后，再看文字是否连贯。每个概念都应从问题推出，每个案例都应从朴素方案走到改进方案，每个实验都应能解释一个判断。若某段话放到其他章节也成立，就把它改成本章的具体对象、具体输入和具体失败。

4.8 工程案例：把多线程归约放回 NUMA 和一致性协议

多线程归约看起来很简单：把数组切成几段，每个线程算一段，最后合并。可是在线程数增加到跨 socket 或跨 NUMA 节点后，性能可能突然停住甚至下降。原因往往不在加法本身，而在页面放置、cache line 所有权、一致性流量、互连带宽和线程迁移。NUMA 章要把“共享内存”从抽象地址空间放回真实拓扑。

一、共享地址空间不等于同等距离 在 NUMA 机器上，所有线程看见同一虚拟地址空间，但物理页有位置。线程访问本地节点内存和远端节点内存，延迟和带宽不同。若主线程单线程初始化整个数组，页面可能集中在一个 NUMA 节点；随后多个节点上的 worker 并行读取时，远端访问会淹

没计算。这个问题在小机器上不明显，在多 socket 服务器上非常常见。

first-touch 策略说明页面位置常由首次写入决定。因此并程序不仅要并行计算，还要按最终访问计划并行初始化。若每个 worker 将来处理某个块，就让它先触碰这个块的页面。这样内存放置和执行位置一致。这个原则比背 NUMA API 更重要：数据在哪里，线程就尽量在哪里执行。

二、线程本地 partial 同时解决共享写和远端写 全局原子计数器在多核下有两个问题。第一，所有线程写同一 cache line，造成一致性迁移。第二，若该 line 常驻某个节点，其他节点写入还要跨互连。线程本地 partial 把写入分散到每个线程或每个 NUMA 节点的本地内存，热路径只写本地 line，最后再合并。它不是单纯算法技巧，而是拓扑感知的数据放置。

partial 的层次也要匹配机器。每线程 partial 可以避免线程之间写冲突，但合并时可能跨节点读很多小对象；每 NUMA 节点 partial 先在节点内合并，再跨节点做一次最终合并，往往更符合拓扑。项目中的事件计数可以按 worker、NUMA node、全局三层组织。报告要写清每一层的写入频率和合并频率。

三、一致性协议保护的是 cache line 所有权 当多个核心读同一份只读数据时，一致性成本很低；当多个核心写同一条 cache line 时，所有权必须不断迁移。锁、原子计数、队列 head/tail、引用计数都可能成为这种热点。性能曲线的典型形状是：少量线程很快，线程继续增加后吞吐不升，甚至因为互连流量下降。这个现象不能靠增加核心解决，因为瓶颈已经在协调。

减少一致性流量有几种方式。第一，减少共享写，例如 partial 和批量合并。第二，隔离热点 line，例如 padding 和 alignas。第三，改变协议，例如从全局队列改成本地队列加窃取。第四，降低实时一致要求，例如监控计数周期性汇总。每种方式都在用空间、延迟或复杂度换少量共享写。

四、拓扑感知队列不是越本地越好 任务运行时常把队列做成本地 deque，以保留 cache 和 NUMA 本地性。可是完全不窃取会让某些 worker 空闲，完全随机窃取会破坏本地性。较好的策略通常是先本地，再同 NUMA 节点，再跨节点。任务粒度也影响选择：大任务跨节点迁移成本可能可以接受，小任务迁移成本可能超过计算。

Compute Systems Engine 可以在任务 trace 里记录 worker id、NUMA node、输入块所在节点和是否窃取。这样当某个阶段不扩展时，读者可以看是本地队列不均、跨节点窃取过多，还是页面放置错误。没有拓扑 trace，NUMA 问题很容易被误判为算法不够快。

五、Linux 观察入口 Linux 提供 numactl、lscpu、/proc/self/numa_maps、perf c2c 等观察入口，具体可用性取决于环境和权限。学习时不必一次掌握所有工具，但要知道每个工具回答什么问题。lscpu 看拓扑，numactl --hardware 看节点和距离，numa_maps 看映射页分布，性能计数器看远端访问和 cache line 争用。

如果设备没有 NUMA，例如很多手机或单 socket 机器，也能做降级实验。用线程绑定和分片 partial 观察 false sharing，用首次触页和冷启动观察 page fault，用本地队列和全局队列观察锁竞争。报告必须写明“当前设备无法验证远端 NUMA 访问”，但仍然可以学习拓扑意识。

六、C++ 结构表达拓扑 拓扑信息不要散落在命令行字符串里。项目可以定义 TopologyPlan：worker 数量、每个 worker 所在逻辑 CPU、NUMA node、负责的输入块、partial 存放位置。算法不一定直接操作 NUMA API，但报告和运行时可以使用这个计划解释任务放置。这样从一开始就把硬件拓扑纳入工程对象。

Listing 4.3 拓扑计划记录 worker 和数据块的放置

```
1 struct WorkerPlacement {
2     std::uint32_t worker_id = 0;
3     std::uint32_t logical_cpu = 0;
4     std::uint32_t numa_node = 0;
5     std::uint64_t input_begin = 0;
6     std::uint64_t input_end = 0;
7 };
```

这个结构不承诺一定能绑定成功。绑定是操作系统能力，可能被权限、容器或移动设备限制。计划和实际观测要分开记录：计划希望 worker 在哪里，实际运行时是否迁移，输入页是否真的落在对应节点。高质量报告不会把愿望当事实。

七、实验交付：分层归约 本章实验从数组归约开始。版本一，单线程 reference。版本二，多个线程直接写全局原子。版本三，每线程 partial。版本四，每 NUMA 节点 partial 后全局合并。版本五，加入并行 first-touch。每个版本记录总耗时、合并耗时、线程数、绑定策略、页面放置证据和 checksum。

预期结果不是固定数字。小数组可能单线程最快；单 socket 机器上 NUMA 分层收益不明显；线程太多会被内存带宽限制；绑定失败时结果可能波动。教材要让读者学会解释这些情况，而不是追求一条漂亮的线性加速曲线。NUMA 学习的目标是知道什么时候拓扑是主因，什么时候不是。

4.9 源码阅读：从调度域和内存策略理解拓扑

NUMA 和一致性协议不只存在于硬件手册。操作系统也在用拓扑信息做调度和内存策略。源码阅读时不需要读完整调度器，而要抓住几个概念：CPU 拓扑如何组织，任务迁移为什么影响 cache，内存策略如何决定页面放置，自动 NUMA balancing 为什么可能改变实验结果。这样读者能把“线程跑在哪里”和“数据放在哪里”连起来。

一、调度域说明迁移不是免费动作 Linux 调度器会按拓扑组织 CPU，例如 SMT sibling、同核心、同 socket、不同 NUMA 节点。负载均衡会在这些层次之间迁移任务。迁移可以让空闲 CPU 得到工作，但也可能让任务失去 cache 热度，访问远端内存。调度器不是不知道局部性，而是在负载均衡和局部性之间取舍。

项目里的任务运行时也面对同样取舍。本地队列保留局部性，但可能负载不均；跨节点窃取改善利用率，但增加远端访问。读调度域的价值，是理解成熟系统也在做分层决策，而不是简单“永远绑定”或“永远迁移”。

二、内存策略影响页面归属 内存策略可以是默认 first-touch，也可以是绑定到某节点、交错分配或优先本地。不同策略适合不同工作负载。只读大数组按访问线程分块，first-touch 很自然；共享读多写少数据可能适合 interleave；强本地队列和 partial 适合绑定。错误策略会让线程和数据距离变远。

实验时要记录策略。若用了 `numactl--interleave`，就不要再把页面分散归因于 first-touch；若容器限制了 CPU 和内存节点，结果也会不同。系统书要让读者养成记录环境的习惯。

三、自动 NUMA balancing 是隐藏变量 某些 Linux 配置会周期性采样页面访问并迁移页面，让内存更接近访问线程。它可能改善长期运行程序，也可能让短 benchmark 波动。若实验中页面位置随时间变化，第一次运行和稳定运行结果不同，就要怀疑自动 NUMA balancing 或回收策略。报告可记录相关系统设置，至少说明没有验证。

这提醒读者：硬件拓扑不是静态背景，操作系统会主动调整。性能实验不能只看代码，也要看系统策略。

四、拓扑章节的复核问题 读完本章，面对多线程程序应能回答五个问题。线程是否固定或可能迁移；数据页由谁首次触碰；共享写是否集中到少数 cache line；任务队列是否跨节点窃取；合并阶段是否把远端数据集中到一个线程。每个问题都有证据入口，而不是凭感觉判断。

4.10 补强案例：一致性流量如何在锁和队列中放大

一致性协议不只影响原子计数器，也影响锁和队列。一个互斥锁变量本身是一条 cache line，等待线程反复读写它；队列的 head、tail、size 若放在同一 line，不同线程会争同一所有权；条件变量唤醒后，多个线程又会竞争同一锁。理解这些细节，才能解释为什么低竞争锁很好，高竞争锁突然很差。

一、锁变量也是共享写热点 获取锁通常要修改锁状态。多个核心竞争同一锁时，锁所在 cache line 在核心之间迁移。临界区越短，锁变量本身的迁移占比越高；临界区越长，等待和上下文切换占比越高。两种慢法不同，优化方向也不同。短临界区高竞争可以分片或减少共享；长临界区要缩短锁内工作。

这解释了为什么“锁次数”不是唯一指标。一个锁每秒被获取很多次但几乎无竞争，可能没问题；一个锁每秒次数不多但每次持有很久，尾延迟很差。报告要记录等待时间和持有时间。

二、队列元数据要分离读写者 生产者主要写 tail 和 buffer 槽位，消费者主要写 head。若 head、tail、size 放在同一 cache line，生产者和消费者会互相干扰。SPSC ring 常把 head 和 tail 分开 padding；MPMC 队列还会让每个槽位有序号，分散状态。布局不是装饰，而是减少一致性所有权迁移。

带锁队列也能受益于布局。锁、条件变量、容量、统计指标、热 head tail 和冷诊断字段不应随意堆在一起。监控计数器频繁更新时，也可能和锁变量 false sharing。队列结构体要按访问者和频率组织。

三、分片锁降低协调粒度 哈希表、任务表和计数器都可以用分片锁。每个 shard 有自己的锁和局部状态，线程只竞争相关 shard。这样减少一致性流量，但引入跨 shard 操作复杂性。例如需要全局快照、迁移 key 或关闭全部 shard 时，要按顺序获取多个锁或使用阶段协议。

分片锁不是免费午餐。Shard 太少，竞争仍高；shard 太多，内存和遍历成本上升；key 倾斜会让某个 shard 仍然热点。报告应记录每个 shard 的请求分布，而不是只看总吞吐。

四、拓扑实验 实现全局锁队列、分片队列和每 worker 本地队列三版。输入包含均匀任务和热点生产者。记录锁等待、cache line 争用替代指标、队列长度和任务完成时间。若没有 cache-to-cache 工具，也可以通过扩展曲线和等待时间推断。目标是让读者看到一致性流量如何从硬件概念变成锁和队列的工程问题。

4.11 补充案例：分层合并如何减少跨节点流量

多 NUMA 节点归约时，如果所有线程把 partial 发送到一个全局合并线程，跨节点流量集中，最后阶段成为瓶颈。分层合并先在每个节点内部合并，再跨节点合并少量结果。这个结构和树形 reduce、map-side combine、分布式聚合是一条线。

一、节点内合并 同一 NUMA 节点内的 worker 把 partial 合并到 node local partial。这个阶段尽量使用本地内存和本地队列。若 worker 被迁移或 partial 分配在远端，收益会下降。因此分层合并依赖线程放置和 first-touch。

二、跨节点合并 跨节点合并只处理每个节点的摘要，而不是每个线程的全部细节。这样减少互连字节和 cache line 迁移。代价是增加一层合并逻辑和内存。若节点内数据极不均衡，某个节点仍可能长尾。

三、和分布式聚合对应 节点内合并对应 worker local combine，跨节点合并对应 reduce。分布式系统中，map-side combine 减少网络字节；NUMA 系统中，node local combine 减少互连字节。尺度不同，原则相同：先在便宜通信范围内合并，再跨昂贵边界传摘要。

四、实验 模拟两层合并，即使在单 NUMA 机器上也可用线程组模拟。比较全局合并、每线程直接全局 atomic、分组 partial 后合并。记录合并时间和扩展曲线。若有 NUMA 机器，再加入节点绑定。实验目标是理解分层思想，不是依赖某台机器的具体数字。

4.12 从共享变量推导拓扑感知设计

多核程序常把共享内存看成一个均匀的大空间，仿佛任意核心读写任意地址的成本都一样。真实机器不是这样。每个核心有私有缓存，多个核心通过一致性协议维护同一地址的可见性；多个 socket 或多个 NUMA 节点之间还有不同距离的内存控制器和互连。于是“共享一个变量”这件小事，会把 cache line 所有权、页面归属、远端访问和合并策略全部牵出来。NUMA 章不应从名词开始，而应从一个扩展曲线突然停住的共享计数器开始。

一、共享写的瓶颈是所有权来回移动 多个线程同时递增同一个计数器，看起来每次只做一次加法。硬件上，这条 cache line 必须在核心之间转移独占所有权。线程越多，算术工作没有增加多少，一致性流量却快速增加。把计数器改成每线程本地 partial 后，热路径变成各自写本地 cache line，最后再合并。这个改法的本质不是“减少锁”，而是减少同一条 cache line 的所有权争夺。读者理解了这一点，后面看到 per-CPU 统计、分层归约、map-side combine 时，就能认出同一个结构。

二、false sharing 是布局层面的共享写 两个线程写不同变量，也可能争同一条 cache line。最容易出错的例子是数组中的相邻计数器：逻辑上每个线程有自己的槽位，物理上多个槽位可能在同一条 cache line 里。padding 可以解决，但 padding 不是免费午餐，它增加内存占用，可能降低 TLB 覆盖，也可能让扫描阶段搬运更多数据。工程上应先确认写热点确实落在同一条线，再决定是否对齐、填充或改成分层布局。不能看到并发就无脑 padding。

三、页面归属来自第一次触碰和后续迁移 NUMA 成本常被隐藏在初始化代码里。主线程分配并清零大数组，页面可能归属主线程所在节点；后续 worker 分布在其他节点上处理这些页面，就会产生远端访问。first touch 策略要求谁将来主要使用页面，谁就先触碰页面。若任务会迁移，页面和

线程的关系又会变化，自动 NUMA 平衡可能迁移页面，也可能在短作业中来不及发挥作用。报告要记录线程绑定、页面初始化方式和节点内存分布，否则 NUMA 结论没有可复现性。

四、分层合并把拓扑写进算法 拓扑感知设计的常见形态是本地处理、节点内合并、跨节点合并。比如统计日志时，每个线程先写本地计数；同一 NUMA 节点上的线程先合并成本节点结果；最后再跨节点合并。这样做的目的不是让代码看起来复杂，而是让高频写停留在近处，让低频合并承担远端成本。若输入极小，分层结构会得不偿失；若数据巨大且线程很多，分层结构可能决定扩展上限。算法是否需要拓扑感知，要由数据规模、共享写比例、远端访问成本和调度稳定性共同决定。

五、把扩展曲线分解成几段看 评估多核程序时，不要只画线程数到总耗时的曲线。还要把耗时拆成读取、计算、本地合并、跨节点合并和写出；把指标拆成本地内存访问、远端内存访问、cache line 争用和调度迁移。若一到八线程扩展很好，十六线程以后停住，可能是内存带宽饱和，也可能是共享写热点，也可能是跨 NUMA 访问，也可能是合并阶段串行化。只有把曲线拆段，才能知道下一步该改布局、改绑定、改合并树，还是承认机器资源已经到顶。

4.13 NUMA 实验如何写成可复现材料

NUMA 相关问题很容易因为机器差异而变得难以复现，所以实验设计比结论本身更重要。一个合格实验不能只说“绑定以后变快”，而要写明机器拓扑、线程绑定、内存初始化、输入大小、访问模式、合并策略和测量工具。即使读者的手机或开发机没有多 socket，也可以用同一套报告格式训练拓扑意识：哪些数据靠近哪个执行实体，哪些共享写会跨核心移动，哪些合并阶段会形成串行瓶颈。

一、拓扑先记录，不要事后猜 实验开始前先记录核心数、逻辑线程数、缓存层级、NUMA 节点和内存分布。Linux 上可以用 `lscpu`、`numactl--hardware`、`/sys/devices/system/node` 等信息；受限环境中至少记录可见 CPU 数和调度限制。没有拓扑材料，后面看到性能曲线变化时只能猜。经典系统教材读起来舒服，原因之一就是它会先把实验背景交代清楚，让读者知道结论站在哪块地上。

二、初始化方式必须和执行方式配对 大数组如果由主线程统一初始化，页面很可能集中在主线程所在节点；若后续多个 worker 跨节点访问，就会把远端访问带入主路径。实验应比较三种初始化：主线程初始化、worker 按分片 first touch、交错分配。每种初始化都要配同一份计算逻辑。这样读者能看到“同一算法、同一线程数”也会因页面归属不同而产生差异，NUMA 不是额外玄学，而是内存所有权的一部分。

三、共享写和远端读要分开测 一个扩展曲线停住，可能是共享计数器在争 cache line，也可能是远端读占用互连，也可能是最后合并阶段串行化。实验要设计三个版本：只读本地数据、写线程本地 partial、写共享全局计数器。只读版本观察带宽和放置，partial 版本观察本地写和合并，全局计数器版本观察一致性流量。三个版本的差异会比单个“优化前后”更能说明问题。

四、合并阶段要单独计时 分层归约常让主计算阶段变快，却把成本移动到合并阶段。报告应把 worker 本地处理、节点内合并、跨节点合并和最终写出拆开计时。若节点内合并很快，跨节点合并很慢，说明拓扑设计有效但最后聚合仍需优化；若本地处理时间差异巨大，说明负载切分比 NUMA 更紧急。没有分段计时，优化很容易把成本藏到结尾。

五、没有 NUMA 机器也能学习同一思想 单 socket 机器上也有缓存层级、核心迁移和共享 cache line。可以用线程绑定、padding、线程本地 partial 和合并树训练同一套思路。等迁移到多节点机器时，只是把“近”和“远”的距离拉大。教学不应把 NUMA 作为少数服务器才需要的知识，而应把它视为局部性、所有权和拓扑成本的自然延伸。

4.14 贯穿案例：全局计数器怎样一步步变成分层归约

本章最适合从一个很小的错误开始：日志统计程序用一个全局 atomic 记录已处理行数，每处理一行就执行一次 `fetch_add`。单线程正确，两线程也正确，线程数继续增加后吞吐几乎不涨，甚至下降。这个事故不需要先背 NUMA、互连、一致性协议这些词。先看事实：所有核心都在高频写同一条 cache line。每次写都要获得该 cache line 的独占权，其他核心手里的副本失效，下一次写又把所有权迁走。算术操作很小，所有权迁移很大。

从这个事故出发，分层设计自然长出来。第一层，每个线程写自己的局部计数器，让高频写停

在本核心附近。第二层，同一个 NUMA 节点内先合并，减少跨节点流量。第三层，再做全局合并，把远端成本降到低频阶段。这个过程不是套用“map reduce”模式，而是根据 cache line 所有权和拓扑距离推导出来的算法结构。

一、atomic 解决数据竞争，不保证扩展性 全局 atomic 的语义是正确的：没有数据竞争，最终值准确。但正确性和扩展性不是同一件事。Atomic 给出了线性化点，代价是所有线程争一个写位置。若这个线性化点发生在每条记录上，扩展性会被一致性协议吞掉；若它发生在每个任务完成时，频率低得多，代价可能可以接受。判断 atomic 是否合适，要先问它在热路径出现多少次，而不是看到 atomic 就删除。

二、本地 partial 需要处理读取时刻 把计数器拆成每线程 partial 后，热写变快了，但“什么时候读取全局值”变复杂了。阶段结束后合并很简单；运行中实时展示进度，则需要读多个 partial 并求和，结果可能只是一个近似快照。若业务要求严格单调进度，就要定义读取协议。这个细节说明优化不会只改变性能，也会改变可见状态的语义。教材必须把这种交换讲出来。

三、padding 要跟访问模式一起设计 每线程 partial 如果存在数组中，相邻元素可能落在同一 cache line，导致 false sharing。给每个 partial 对齐到 cache line 可以解决写写争用，但内存占用变大，合并扫描时也会跨更多 cache line。若 partial 很少且写频率高，padding 合理；若 partial 很多且写频率低，padding 可能浪费。设计不应是“所有结构都 padding”，而是先标出高频写字段，再对这些字段隔离。

四、first touch 把初始化也纳入算法 局部 partial 不只是谁写的问题，还包括页面归属。若主线程创建并清零所有 partial，页面可能集中在主线程所在节点；后续 worker 即使写自己的 partial，也可能写远端页面。更好的做法是让使用该 partial 的 worker 完成初始化，或者按 NUMA 节点批量初始化。初始化不再是无关准备工作，它决定页面第一次落在哪里。报告若不写初始化方式，NUMA 结论就不可复现。

五、合并树要和拓扑对应 最终合并可以由一个线程扫所有 partial，也可以做树形合并，也可以按 NUMA 节点先合并。本地合并减少远端读，树形合并减少单点串行时间，但会增加同步阶段。输入小的时候，复杂合并树不划算；输入大且线程多时，它能避免单线程尾部成为瓶颈。这个选择要通过分段计时判断：本地处理、节点内合并、跨节点合并分别占多少。

六、把拓扑信息放进 trace 当一次运行突然变慢，trace 里如果只有线程 id，很难判断原因。更好的记录包括 worker id、CPU id、NUMA node、partial 地址范围、初始化线程、合并阶段和迁移次数。这样报告能回答：慢线程是否跑到远端节点，页面是否集中在一个节点，某个 partial 是否和其他线程 false sharing。拓扑信息不是低级细节，而是解释多核性能的证据。

4.15 案例深化：共享计数器的分层改造

NUMA 和一致性协议最适合通过共享计数器讲清楚。朴素版本让所有线程执行 `fetch_add` 更新全局计数。它语义清楚，也能避免数据竞争；但线程数上来以后，cache line 所有权在核心之间反复迁移，互连流量上升，吞吐停住。这个事故能把 atomic、cache line、NUMA、线程绑定和局部归约串起来。

一、不要先否定 atomic Atomic 的问题不是“慢”，而是不适合高频共享写热点。低频状态、线性化点、任务完成计数都可能适合 atomic。教材要让读者先判断频率和共享范围：每条记录一次的计数是高频路径，适合局部 partial；每个 task 完成一次的状态更新频率低，可以用 atomic 或锁。把所有 atomic 都视为错误，会让工程判断变粗糙。

二、第一层拆到线程本地 线程本地 partial 让热写入落在各自 cache line 上。每个 worker 扫描自己的输入，更新自己的计数表，阶段结束后再合并。这个结构减少一致性流量，但增加合并阶段。若 key 很多，线程本地表会占用更多内存；若 task 很小，合并成本可能超过收益。报告要同时写扫描时间、合并时间和内存峰值。

三、第二层按 NUMA 节点合并 跨 socket 合并比同节点合并更贵。可以先在每个 NUMA node 内合并，再由一个节点汇总全局。页面首次触碰也要配合：哪个 worker 初始化 partial，页面就更可能放在哪个节点。若初始化线程和使用线程分离，可能出现远端访问。实验要记录绑定策略和 first touch 策略，否则数字很难复现。

四、避免 false sharing 即使每个线程有自己的计数，如果多个计数结构落在同一 cache line，

也会发生 false sharing。结构体数组尤其容易出现这个问题。解决方法可以是按 cache line 对齐、填充或改成每线程独立分配。报告里不要只写“用了 thread local”，还要说明 hot counter 是否可能共享 cache line。这个细节能把一致性协议从抽象概念落到 C++ 对象布局。

五、把拓扑写进接口 最终项目可以把 worker id、CPU id、NUMA node、local partial id 写进 trace。这样一次异常慢的合并能追到具体拓扑。接口不一定强制绑定，但要允许记录和控制。高性能计算不是假设机器均匀，而是在接口中保留拓扑信息，让调度器和报告有能力使用它。

4.16 案例复盘：一个 atomic 怎样拖出整机机器

所有线程对同一个 atomic 计数器执行 fetch add，代码很短，语义也正确。线程数增加后吞吐停住，是因为同一条 cache line 的所有权在核心之间迁移；跨 NUMA 节点时，还要经过互连。这个案例让读者看到，共享内存并不意味着没有位置，cache line、页和线程都在拓扑里。

分层 partial 的推导 第一步把高频写拆到线程本地，第二步按 NUMA 节点合并，第三步再全局汇总。这样热写入留在本地 cache line 上，跨节点流量只发生在合并阶段。Atomic 仍可用于低频状态和线性化点，但不适合每条记录一次的共享热点。

实验判据 报告写线程绑定、first touch、NUMA node、线程数扩展、合并时间和远端访问比例。若本地 partial 快而合并慢，瓶颈只是迁移到下一阶段；若绑定关闭后波动大，调度迁移已经进入成本模型。拓扑信息必须成为 trace 的一部分。

4.17 本章习题

习题与作业

习题一：把一个全局原子计数器、每线程计数器、每 NUMA 节点计数器画成三张数据共享图。分别标出哪些 cache line 被谁写，哪些阶段需要合并，读取全局值的成本在哪里。

习题二：设计一个 NUMA first-touch 实验。要求写清初始化线程、计算线程、绑定策略、输入规模、测量指标和预期现象。若当前设备没有 NUMA，写出哪些部分只能作为计划保留。

习题三：为线程池设计一个两级队列方案：worker 本地队列加全局溢出队列。说明高频路径、低频路径、锁保护对象、可能的窃取顺序和关闭语义。

习题四：阅读 Linux NUMA 策略或调度拓扑的一个窄入口。报告必须从一个用户态实验问题出发，不允许只复制函数名。

第零部分

单机高性能计算：从测量到数据流

可信测量、Roofline 与性能计数器

先看一个常见争论：同一个归约优化，有人测出三倍加速，有人测出几乎没提升。两边都没有撒谎，只是测的不是同一个对象。一个人把初始化、首次缺页和输出校验都放进计时区，另一个人只测热循环；一个人在小数组上测依赖链，另一个人在大数组上已经打满内存带宽；一个人固定线程，另一个人的线程在 NUMA 节点之间迁移。没有测量边界，所有性能结论都会变成口水仗。

本章从这种“数字互相矛盾”的场景出发。可信测量不是为了仪式感，而是为了知道一个改动到底作用在哪个瓶颈上。Roofline 帮我们判断当前内核是否已经被字节移动限制；性能计数器帮我们区分前端、后端、缓存、TLB、分支、同步和调度；对照实验帮我们排除错误解释。读者要学会先定义测量对象，再解释数字。

5.1 从问题到测量边界

测量前先定义对象。是测单次操作延迟，还是测批量吞吐；是包含内存分配，还是只测热循环；是冷缓存，还是热缓存；是单线程，还是多线程；是平均值，还是尾延迟；是关心总吞吐，还是关心每个 worker 是否均衡。不同对象需要不同方法，不能混用结论。

常见错误包括：把初始化时间算进内核，把编译器优化掉的循环当作结果，把首次缺页当作算法成本，把输出打印放进计时区，把不同 CPU 频率状态下的结果直接比较。

第二册还要特别避免一种错误：只测总时间，不看扩展曲线。多线程程序的核心问题不是“某一次运行快不快”，而是从 1 个线程到多个线程时，吞吐如何变化。一个程序在 2 个线程时加速，在 8 个线程时停滞，在 16 个线程时变慢，这条曲线本身就是证据。它提示瓶颈可能从单核执行转移到了内存带宽、锁竞争、调度、NUMA 或 I/O。

计时器和统计方法 计时器应使用单调时钟，避免系统时间调整影响。短任务要重复执行足够多次，并防止编译器消除结果。报告结果时不要只给一次运行时间，应至少给出多次运行的最小值、中位数和波动范围。最小值常代表干扰较少的情况，中位数更接近日常情况，尾部异常则提示系统噪声或资源竞争。

预热同样重要。第一次运行可能包含页错误、动态链接、分支预测冷启动、缓存冷启动和 CPU 频率爬升。若目标是冷启动延迟，就应该保留这些成本；若目标是热循环吞吐，就应该把它们排除。测量对象不同，预热策略不同。

多核 benchmark 还要记录线程绑定和运行顺序。若每轮测试先跑单线程再跑多线程，CPU 温度、频率和缓存状态可能系统性变化。更稳妥的做法是随机化版本顺序，保留原始样本，并记录每轮线程数、绑定策略和输入位置。若差异小于系统波动，就不要写强结论。

5.2 从字节数到 Roofline 和扩展曲线

Roofline 用算术强度连接计算峰值和内存带宽。算术强度是操作数与数据移动字节数的比值。低算术强度程序通常受内存带宽限制，高算术强度程序才可能接近计算峰值。这个模型不是精确预测器，而是帮助判断优化方向。

数组求和每读取一个整数只做一次加法，算术强度低；矩阵乘如果分块得当，每个元素可以被多次复用，算术强度高。若一个内核已经接近内存带宽上限，继续手写更复杂的算术指令可能没有意义；若内核远低于计算上限且数据复用充分，就要检查 SIMD、指令级并行和执行端口。

Roofline 的价值在于先给出“最多可能多快”的粗上限。假设一个内核每处理一个元素读取 8 字节、写入 8 字节，只做少量整数运算，那么它的算术强度很低。若平台可持续内存带宽是 50 GiB/s，那么不管循环写得多漂亮，只要每个元素必须从内存读写 16 字节，吞吐上限就被数据移动限制。反过来，若内核通过分块让同一批数据被重复使用，算术强度提高，才有资格讨论 FMA、SIMD 宽度和执行端口。

心智模型

Roofline 不是为了画漂亮图，而是为了阻止错误优化：内存带宽瓶颈不要只改算术指令，计算端口瓶颈不要只改数据结构，同步瓶颈不要只改循环展开。

从字节数推导瓶颈 Roofline 的关键是先估算字节数。以 `std::int32_t` 数组求和为例，每个元素读取 4 字节，做一次整数加法。如果数组远大于缓存，主要流量来自内存读取。若机器可持续内存带宽是 40 GiB/s，那么理想情况下每秒最多读取约 100 亿个整数。实际还要扣除循环开销、预取效率、TLB 和其他系统干扰。

这个估算能提前判断优化方向。如果实测已经接近带宽上限，就不该期待换一种加法写法获得数量级提升。若实测只有带宽上限的一小部分，就要检查访问是否连续、是否触发 TLB miss、是否有分支、是否被编译器向量化、是否被线程调度干扰。

字节数估算要区分“算法逻辑字节”和“实际流量”。逻辑上，一个原子计数器每次只更新 8 字节；硬件上，它可能让一整条 cache line 在核心之间来回迁移。逻辑上，一个对象只读取一个字段；硬件上，cache line 可能带入多个冷字段。逻辑上，`mmap` 文件像数组；系统上，第一次访问可能触发缺页和 page cache 读取。第二册的实验报告必须说明这些差异，否则字节数估算会过于乐观。

5.3 计数器、profile 和 trace

Linux 上常用 `perfstat` 查看 cycles、instructions、branches、branch-misses、cache-misses、page-faults、context-switches 等指标。更深入时可以用 `perfrecord` 和 `perfreport` 定位热点。不同 CPU 的事件名称不同，实验报告要记录命令和平台。

计数器要组合解释。IPC 低可能因为 cache miss，也可能因为分支失败或同步等待。cache-misses 高不一定说明程序差，流式扫描大数组本来就会不断 miss，但可能充分利用带宽。context-switches 多可能说明线程过多、阻塞频繁或系统有干扰。

进阶测量更重要的是“假设驱动”。如果怀疑前端瓶颈，就比较指令缓存、分支、uops 供给和代码布局；如果怀疑内存带宽，就比较工作集大小、线程数、带宽工具和 LLC miss；如果怀疑 TLB，就改变页大小或访问页数；如果怀疑锁竞争，就看吞吐曲线、等待时间、futex 路径和临界区长度；如果怀疑 NUMA，就改变 first-touch 和绑定策略。不要一次性收集一大堆事件再事后找故事。

Top-Down 思维 现代性能分析常把 CPU 周期粗分为前端受限、错误预测受限、后端受限和有效退休。不同平台工具名称不同，但思路相同：先判断核心为什么没有持续退休有用指令，再进入更细分类。前端受限时检查取指、译码、代码布局和间接跳转；错误预测受限时检查分支输入分布；后端受限时继续区分执行端口、内存延迟、内存带宽和同步等待；有效退休比例高时，说明核心大部分时间确实在做有用工作，优化要回到算法或工作量本身。

这套思维可以避免把 IPC 低简单等同于“CPU 不行”。同样是 IPC 低，链表随机访问、锁等待、page fault、系统调用阻塞和分支乱跳的解决方向完全不同。报告里应该写“当前证据更支持哪一类受限”，而不是只列一个 IPC 数字。

同步和调度的测量 多线程程序经常慢在不退休用户态指令的地方。线程可能睡在 futex 上，可能被调度器切走，可能自旋等待 cache line，可能阻塞在 I/O，也可能因为 cgroup 配额用完而被节流。只看 cycles 和 instructions，容易忽略这些等待。

测同步要记录至少三类事实。第一，吞吐随线程数怎样变化；第二，等待在哪里发生，是用户态自旋、内核 futex、条件变量、I/O 还是队列为空；第三，竞争对象是什么，是一把锁、一个原子变量、一条 cache line、一个队列还是一个分片。若能使用 `perfsched`、`perflock`、`perfc2c` 或 eBPF 工具，应记录工具版本和权限；若不能使用，也要用对照实验替代，例如拆分计数器、增加 padding、改变线程绑定和改变队列容量。

多核扩展曲线 一个高质量性能报告不只给“优化前后快了多少”，还要给扩展曲线。横轴是线程数或 worker 数，纵轴可以是吞吐、耗时、加速比、效率或尾延迟。理想线性加速很少出现，重要的是解释拐点。若 1 到 4 线程接近线性，8 线程开始变慢，可能是共享缓存、内存带宽、NUMA 或同步热点；若从 2 线程开始就没有提升，可能是任务粒度太小、串行部分太大或全局锁太重。

扩展曲线还要配合正确性校验。并行程序在高线程数下更快但偶尔错误，不是优化成功，而是暴露了竞争。每轮性能样本都应关联校验结果，校验失败的样本不能混入性能统计后继续分析速度。

5.4 实验设计、统计和报告

一份合格性能记录至少包含：机器型号、核心和 NUMA 拓扑、内存容量、编译器和编译选项、输入规模、数据类型、线程数、绑定策略、运行次数、计时范围、正确性校验、perf 命令和原始输出摘要。缺少这些信息，别人无法复现实验，你自己几周后也很难判断数字是否可靠。

性能结论要写成果因果假设，而不是口号。例如“并行版本在线程数超过 8 后不再加速，perf 显示 LLC miss 和内存带宽接近平台上限，因此瓶颈更像内存带宽，而不是线程创建成本”。这样的结论可以继续被验证或推翻。

本册建议把报告分为五段。第一段写被测问题和正确性 oracle。第二段写机器、系统、编译和绑定环境。第三段写实验矩阵，包括输入规模、线程数、版本、运行轮次和顺序。第四段写原始结果摘要，不只给平均值，也给中位数、最小值和异常。第五段写归因假设和下一步验证。若证据不足，要明确写“当前只能排除某些解释，不能确认最终瓶颈”。

从问题到实验矩阵 真正的性能实验不是随手跑一个命令，而是把问题拆成可证伪的假设。以 Compute Systems Engine 的归约为例，用户观察到“并行版本没有线性加速”。这句话本身不能直接指导修改，因为可能原因太多：线程创建成本太高，任务粒度太小，单线程 baseline 已经被编译器向量化，输入超过内存带宽上限，partial 发生 false sharing，worker 被调度到同一个物理核心的 SMT sibling，主线程首次触页导致远端内存，或者 benchmark 把输入生成混入了计时区。

实验矩阵要把这些解释逐步分开。第一组只测单线程：baseline、四累加器、可能的 SIMD 版本，输入规模从 L1 可容纳、L2 可容纳、LLC 可容纳一直扩到内存。若小规模下四累加器有效、大规模下无效，说明瓶颈可能从依赖链迁移到内存。第二组固定算法，改变线程数，从 1 到物理核心数，再到逻辑 CPU 数。若到物理核心数后增长变慢，再使用 SMT 变差，说明资源共享或带宽上限值得检查。第三组固定线程数，改变 partial 布局，比较紧凑和对齐。若对齐版本明显改善，说明 false sharing 是关键证据。第四组改变初始化方式，比较主线程初始化和 worker first-touch。若 NUMA 机器上差异显著，说明页面放置参与了结果。

这样的矩阵看起来比“优化前后跑一下”繁琐，但它能防止错误归因。性能工程最怕的是改了十个变量后得到一个数字，然后把提升归功于自己最喜欢的解释。严肃实验宁愿一次只改变一个变量，哪怕多跑几组，也不要吧线程数、布局、输入、编译选项和绑定策略同时改变。

正确性 oracle 和防优化 性能实验必须有正确性 oracle。归约的 oracle 可以是顺序 reference；过滤计数的 oracle 可以是简单版本；并发队列的 oracle 不能只看最终数量，还要验证 FIFO 或线性化语义；分布式 shuffle 的 oracle 需要验证每个 key 的最终聚合值和重复请求处理。没有 oracle 的 benchmark 很危险，因为最快的程序可能只是算错了，或者被编译器优化掉了。

防止优化器删除计算也要讲清。常见错误是循环计算了结果但从未使用，优化器发现结果对程序可观察行为没有影响，于是删除整个循环。为了保留计算，可以把结果参与校验、打印摘要、写入 **volatile** sink，或者通过 benchmark 框架的接口告知编译器。更好的方式是把性能样本和 correctness check 绑定：每轮运行都返回结果，结果不对则丢弃样本并报告错误，而不是继续统计耗时。

但防优化不能破坏测量对象。如果把每轮结果都打印到终端，测到的就不是计算时间，而是 I/O 时间。如果把结果写入同一个全局原子，可能引入新的共享写瓶颈。如果把校验放进热循环，校验成本会污染内核。正确做法通常是：热循环只做被测工作，循环外做轻量校验，输出只写汇总，不在计时区打印每个元素。

预热、冷启动和稳态 预热不是形式主义。第一次运行可能包含动态链接、页面分配、minor fault、缓存冷启动、分支预测器冷启动和 CPU 频率变化。若目标是服务冷启动延迟，这些成本应保留；若目标是稳定吞吐，就应先预热再测。实验报告必须说明测的是冷启动、热循环还是混合流程。

稳态也不是永远稳定。CPU 温度升高可能降频，后台任务可能抢占，容器 CPU 配额可能周期性限流，内核写回线程可能突然工作，透明大页可能在后台整理内存。一个严肃报告应保留多次样本，而不是只给最漂亮的一次。若最小值和中位数差距很大，说明系统噪声或尾部事件值得解释。若样本呈周期性波动，可能要检查 cgroup、频率、热限制或 I/O 写回。

本册建议在报告里至少列出三类值：最小值、中位数和最大值或尾部百分位。最小值常代表干扰较少的理想近似；中位数代表常规情况；尾部代表稳定性风险。高性能系统不只追求平均吞吐，还要理解波动来源。一个吞吐高但尾延迟不可控的队列，可能不适合作为服务请求路径。

计数器不是事实本身 性能计数器非常有用，但它们不是无条件真理。硬件事件可能受平台支

持、内核权限、计数器复用、采样误差和事件定义影响。同一个事件名在不同微架构上含义可能不同；某些高层事件由工具组合推导，不是硬件直接计数；同时请求太多事件时，内核可能 multiplex，导致估计值精度下降。

因此报告中应写出命令、事件列表、是否出现 multiplex、运行时间是否足够长、是否固定 CPU、是否只测当前进程。若工具输出里有权限限制或不支持事件，不能忽略。特别是在手机、容器或受限 Linux 环境中，很多硬件事件无法读取，这时可以退而使用时间、上下文切换、page faults、输入规模对照和源码结构分析，但要明确证据等级降低。

计数器解释也要组合。IPC 高不一定代表程序好，可能只是做了很多无用指令但都能退休；IPC 低不一定代表前端差，可能是内存等待、锁等待或分支错误。cache-misses 高也不一定坏，流式扫描大数组天然会 miss，但如果带宽接近上限，miss 可能是有效数据移动。branch-misses 高才需要结合输入分布、分支数量和耗时变化解释。单个数字不能代替因果链。

Roofline 的实际计算步骤 Roofline 很容易被画成漂亮图，却没有指导实验。实际使用时先做三步。第一步，估算每个元素的逻辑数据移动和算术操作。例如 `std::int32_t` 求和每个元素读取 4 字节，做一次整数加法，最终写一个总和。第二步，估算实际数据移动的额外成本：cache line 带入未使用数据，写分配带来额外读，原子写导致 cache line 迁移，TLB miss 导致页表访问。第三步，用平台可持续带宽和计算吞吐给出上限，再和实测比较。

例如一个简单拷贝内核读取 4 字节并写入 4 字节，逻辑上每元素 8 字节。但如果写入使用普通 write-allocate 路径，写 miss 可能先把目标 cache line 读入，再修改，实际内存流量高于 8 字节每元素。若使用非临时写入，可能减少写分配，但也改变缓存污染和后续复用。教材不能把字节数估算写成固定公式，要让读者知道实际流量取决于写策略、缓存命中和硬件行为。

Roofline 还要区分单线程和多线程。单线程可能受单核加载端口、预取能力或乱序窗口限制；多线程可能逐步接近内存控制器带宽。若单线程远低于平台总带宽，增加线程可能提升；若多线程已经接近可持续带宽，继续加线程只会增加争用。性能报告应把单线程曲线和多线程曲线放在同一个解释里。

Amdahl、Gustafson 和扩展曲线 多线程加速经常用 Amdahl 定律解释：如果程序中有不可并行的串行部分，那么线程数再多也会受串行部分限制。这个模型有用，但容易被误用。真实程序的串行部分不是固定不变的：任务切分、合并、内存分配、队列调度、锁竞争和 NUMA 远端访问都可能随线程数变化。换句话说，线程数增加后，程序结构本身也在变。

Gustafson 的视角强调问题规模随资源增加而增长。在很多计算任务中，固定输入规模时加速有限，但输入规模扩大后，并行资源可以处理更多工作。教材要同时讲这两种视角。面试刷题常把复杂度看成固定输入下的函数；系统工程还要看“给更多核心后，能否处理更大批量、更高吞吐或更高并发”。固定规模加速和弱扩展是两个不同实验。

扩展曲线至少有三种画法。强扩展固定总工作量，观察线程数增加后时间下降；弱扩展让每个线程工作量固定，观察线程数增加后总吞吐是否保持；尾延迟扩展关注并发请求增加后延迟分布怎样变化。Compute Systems Engine 的归约适合强扩展和弱扩展；有界队列和线程池更适合吞吐和尾延迟；分布式 shuffle 则要同时看吞吐、长尾任务和重试成本。

同步路径的测量陷阱 锁和原子很容易被微基准误导。一个锁在空循环里竞争，测到的是最坏的锁争用形态；在真实业务里，临界区可能保护较大的工作，锁成本占比反而不高。反过来，一个队列在单生产者单消费者测试里表现很好，不代表多生产者多消费者下仍能扩展。同步结构必须按目标竞争模式测。

测 mutex 时应区分无竞争、短竞争和阻塞路径。无竞争 fast path 通常很快；短竞争可能自旋；阻塞路径会进入 futex，涉及内核调度和唤醒。测 atomic 时应区分单线程原子成本和多线程 cache line 争用。测无锁队列时应记录 CAS 失败率或至少记录重试行为，否则只看吞吐无法解释尾延迟。测条件变量时应记录等待谓词、队列长度和唤醒策略，否则不知道线程是在等数据、等空间还是被错误唤醒。

案例：归约实验报告示范 下面给出一段报告式叙述，展示本册希望读者达到的分析密度。

深入理解

问题：比较顺序归约、四累加器归约、热原子并行归约和线程本地 partial 归约。输入为 256 MiB 的 `std::int32_t` 数组，结果用顺序 reference 校验。机器为某型号 8 物理

核心 16 逻辑 CPU，编译器使用固定版本和 **-O3**。每个版本预热一次，正式运行 10 次，报告中位数和最小值。线程数取 1、2、4、8、16。

预期：顺序 baseline 可能受依赖链和内存流量共同影响；四累加器在小输入下可能改善 IPC，但在大输入下可能接近带宽上限；热原子版本在多线程下会因为共享写扩展性差；线程本地 partial 应显著减少共享写，但 16 逻辑 CPU 可能因 SMT 和带宽争用收益下降。若观察结果与预期不同，下一步分别检查编译器是否已自动向量化、partial 是否 false sharing、初始化是否 first-touch、线程是否迁移。

这段报告没有给固定数字，因为数字属于具体机器；它给的是实验结构和解释路径。一本教材如果只给“某机器快了 3 倍”，读者换设备后很难迁移。更重要的是教读者怎样提出假设、怎样排除错误、怎样承认结论边界。

观测性进入项目设计 很多学生把观测性当成上线后再加的日志，这在系统工程里太晚。Compute Systems Engine 从第一天就应该记录版本、输入规模、线程数、运行时间、校验结果、队列长度、任务数和错误计数。后续加入线程池时，要记录每个 worker 执行任务数、空闲时间和偷取次数；加入有界队列时，要记录 push 等待次数、pop 空队列次数和最大队列长度；加入分布式模拟时，要记录消息数、重试数、重复请求数、提交延迟和检查点耗时。

观测性也要控制成本。热路径中每个元素都写日志会摧毁性能；每个任务都拿全局锁记录指标会引入新的瓶颈。常见做法是线程本地计数、批量汇总、采样、分级日志和只在实验模式下打开详细指标。观测系统本身也是计算系统，仍然受本册所有原则约束。

统计方法：不要只报平均值 性能数据有波动。平均值容易被少数异常样本拉偏，最小值容易过于乐观，最大值容易受偶然干扰影响。报告应至少给出中位数、最小值和尾部，必要时给出四分位或百分位。对于服务延迟，P95、P99 往往比平均值更重要；对于批处理吞吐，中位数和最小值能帮助区分稳定能力和偶发干扰。

样本数量也要足够。一次运行不能代表稳定性能。短任务应重复更多轮，长任务可以少一些但要记录环境。若样本方差很大，不应简单取平均，而要调查噪声来源：CPU 频率、后台任务、page fault、I/O 写回、GC、热限制、cgroup throttling、网络抖动。统计不是装饰，而是发现不稳定性的工具。

计时边界 计时边界决定实验测的是什么。一个归约实验如果把输入生成、随机数填充、线程创建、任务提交、计算和校验都放进同一个计时区，结果就不能解释计算内核。端到端时间有价值，但要单独命名。通常应同时报告两类时间：端到端时间和核心阶段时间。端到端时间回答用户体验，核心阶段时间回答优化方向。

线程池实验也类似。若每轮都创建和销毁线程，测到的是线程生命周期成本；若线程池预先创建，测到的是任务调度和执行成本。两者都可能有用，但不能混淆。分布式模拟中，是否把 checkpoint 写入、故障重试、输出提交放进计时，也要明确。

输入生成和随机性 随机输入要可复现。使用固定 seed，记录分布参数。不要使用运行时随机设备生成不可复现输入，否则性能异常无法重放。对于分支预测实验，要明确输入分布：全小于阈值、全大于阈值、排序后半段大于阈值、随机一半大于阈值。对于热点 key 实验，要明确 Zipf 参数或热点比例。输入分布是 workload 的一部分。

输入生成本身可能很慢，也可能污染 cache。若目标是测计算内核，生成后应在计时外完成，并决定是否预热 cache。若目标是端到端系统，输入生成可以进入计时，但报告要说明。性能实验的每个阶段都要有名字。

编译选项和二进制一致性 编译选项会大幅影响性能。**-O0** 适合调试，不适合性能结论；**-O2**、**-O3**、**-march=native**、LTO、PGO、fast-math 都会改变生成代码和语义边界。报告必须记录编译器版本和选项。若使用 **-march=native**，二进制可能只适合当前机器，换机器结果不可比。

浮点实验尤其要谨慎。fast-math 可能允许重排、忽略 NaN 或改变舍入语义。并行归约如果要求确定性，不能随意打开破坏语义的选项。性能优化不能偷偷改变正确性合同。每个编译选项都应有理由。

基准污染 Benchmark 可能被日志、断言、内存分配、锁、原子 sink、系统调用和调试构建污染。比如在热循环里输出日志，测到的是终端 I/O；在每个元素上调用虚函数，测到的是调度而不是算法；在每轮都分配大 vector，测到的是分配器和 page fault；在并行循环里更新全局统计，测到

的是原子热点。

这并不意味着这些成本不重要，而是要命名清楚。如果目标是测完整业务路径，日志和分配可能应保留；如果目标是测内核算子，它们应移出计时区。一个好 benchmark 应能拆分阶段，让读者看到每个成本分别是多少。

对照实验的单变量原则 优化前后只改变一个变量，结论才清楚。若同时把容器从 list 改成 vector、把算法从单线程改成多线程、把编译选项从 O2 改成 O3、把输入规模改大，就算变快也不知道原因。实际工程有时需要多改，但实验应尽量拆成多个对照：先只换布局，再只换线程，再只换编译选项。

单变量原则也适用于失败实验。如果某个优化没有效果，不要马上丢弃。先检查变量是否真的变化：编译器是否已自动做了同类优化，输入是否太小，瓶颈是否迁移，测量噪声是否太大。失败结果也是知识，只要记录清楚。

Benchmark harness 的数据结构 项目可以定义一个简单结果结构，统一记录实验。它不需要复杂框架，但要避免每个实验手写不同输出格式。

Listing 5.1 Benchmark 结果的基本字段

```
1 struct BenchmarkResult {
2     std::string name;
3     std::uint64_t input_items = 0;
4     std::int32_t worker_count = 1;
5     double milliseconds = 0.0;
6     bool correct = false;
7     std::uint64_t bytes_processed = 0;
8 };
```

生产代码应避免在热路径频繁构造字符串；这里的结构用于实验汇总。后续可以输出 CSV 或 JSON，方便画图。关键是每个样本都带 correct 字段，错误结果不能进入性能结论。

Linux perf 的使用层次 **perfstat** 适合整体计数，例如 cycles、instructions、branches、branch-misses、cache-misses、context-switches、page-faults。**perfrecord** 和 **perfreport** 适合采样热点函数。**perftop** 适合实时观察。调度问题可以看 **perfsched**，锁问题在支持环境中可以看 lock 相关工具，cache line 争用可以研究 c2c。不同发行版和权限支持不同，报告要写清不可用项。

使用 perf 时要避免测错进程。若程序启动子进程或线程，命令行要确认是否跟踪到目标。若运行时间很短，计数器误差会大。若事件 multiplex，工具会给出缩放估计，精度下降。若 CPU 迁移频繁，某些 per-CPU 事件解释更难。perf 是强工具，但不是免思考按钮。

Off-CPU 时间 很多程序慢在 off-CPU 时间，也就是线程没有在 CPU 上运行。原因可能是等待锁、等待 I/O、等待条件变量、被调度器抢占、cgroup 节流或睡眠。CPU profile 只看 on-CPU 热点，可能完全看不到等待原因。服务端延迟分析经常需要 off-CPU 视角。

简单替代方法是应用内部记录等待时间。例如 bounded queue 的 push 等待时间、pop 等待时间；线程池 worker 空闲时间；RPC in-flight 时间；I/O completion 等待时间。即使没有 eBPF 工具，也可以通过状态机指标构造 off-CPU 证据。第二册强调把观测性放进项目，就是为了在工具受限时仍能分析。

内存带宽测量 内存带宽不是硬件规格表上的峰值，而是当前工作负载下的可持续带宽。可以用顺序读、写、copy、triad 等简单内核估算平台上限，再和实际算法比较。若算法吞吐接近可持续带宽，继续减少几条算术指令通常收益小；若远低于带宽，可能受依赖、随机访问、TLB 或前端限制。

带宽也受线程数、NUMA、频率和写策略影响。单线程带宽远低于多线程总带宽很正常；跨 NUMA 访问可能降低带宽；写分配会增加实际流量。报告中写“内存带宽瓶颈”时，应说明依据：字节估算、线程扩展曲线、cache miss、平台带宽对照，而不是只凭感觉。

Profiling 和 Tracing Profiling 聚合“时间花在哪里”，tracing 保留“事件按什么顺序发生”。计算内核常用 profiling；运行时、队列、异步 I/O 和分布式系统更需要 tracing。一个线程池平均函

数耗时不高，但 trace 可能显示 worker 大量空闲或等待队列；一个分布式作业总耗时高，trace 可能显示最后一个 reduce partition 长尾。

Compute Systems Engine 应支持轻量 trace：任务开始、任务结束、队列满、消息发送、消息完成、重试、提交。Trace 要有开关，默认可以只记录摘要。详细 trace 会产生大量数据，适合调试而不是每次性能运行。

性能回归 项目变大后，需要防止性能回归。不是每次提交都跑完整 benchmark，但至少要有小规模 smoke benchmark 和关键路径指标。比如顺序归约、并行归约、有界队列吞吐、线程池任务调度和分布式模拟正确性。若某次修改让归约慢 30%，应能及时发现。

性能回归判断要允许噪声。可以设置相对阈值、重复运行取中位数，并保留历史环境信息。手机和共享云环境噪声大，阈值要更宽；专用机器可以更严格。回归测试的目标是发现明显退化，不是追求实验室级精确。

报告中的诚实边界 高质量报告会明确说哪些结论已经验证，哪些只是推测。比如“当前环境无法读取 dTLB 事件，因此 TLB 解释来自页数和访问模式对照，证据弱于直接计数”；“本机没有 NUMA，因此 first-touch 只作为设计保留”；“EPYC 服务器上结果不能直接外推到手机大小核”。承认证据边界不是减分，而是工程严谨。

相反，糟糕报告会是一次运行数字写成普遍结论，把某台机器的结果推广到所有 CPU，把不可复现随机输入当作事实，把没有正确性校验的最快版本当作成功。第二册要求读者避免这种写法。

案例：一次性能回归排查 假设某次提交后，parallel histogram 慢了 25%。第一步不是猜，而是确认正确性是否仍通过，输入和编译选项是否相同。第二步看阶段指标：本地统计慢了，还是合并慢了，还是队列等待增加。第三步看 diff：是否改变了桶布局、增加了全局指标、把本地 vector 改成 `unordered_map`、或在热循环中加入日志。第四步跑对照：关闭新指标、恢复旧布局、固定线程数。

这样的排查流程比“感觉是 cache 问题”可靠。性能回归常来自小改动：一个统计计数器从线程本地变成全局原子，一个调试日志进入热路径，一个结构体字段改变导致 padding 变大，一个 vector reserve 被删除导致频繁分配。性能工程需要把代码 diff、指标和实验联系起来。

案例：手机环境测量 在手机 Termux 上测性能，要特别谨慎。CPU 可能有大小核，温度和电量会影响频率，后台应用可能抢资源，某些 perf 事件不可用，存储 I/O 受系统策略影响。报告应记录设备型号、系统版本、是否充电、是否长时间运行、可见 CPU、是否能设置 affinity、哪些 perf 事件可用。不能把手机一次短跑结果当成服务器结论。

手机环境也有价值。它能训练受限环境下的证据意识。无法读取硬件计数器时，可以用输入规模曲线、线程数曲线、正确性校验、时间分布和源码分析。无法验证 NUMA 时，可以写明限制。工程严谨不要求工具完美，而要求诚实说明证据等级。

实验数据归档 性能实验应归档原始数据，而不是只在报告里写结论。至少保存命令、配置、输入摘要、运行时间、样本、git commit、机器信息和输出校验。CSV 或 JSON 都可以。图表可以从原始数据生成，报告引用图表。这样几周后发现问题，可以重新分析，而不是重新跑一切。

项目可以在 `results/` 下按日期或 commit 保存实验摘要。不要把巨大临时文件提交，但小的 CSV、报告和脚本可以保留。性能工程是可重复研究的一种形式。

Benchmark 伦理 不要只挑最好结果，不要删除失败样本后不说明，不要在不同机器结果之间做不公平比较，不要把 debug 构建和 release 构建混在一起，不要把不正确输出的时间当成成功，不要把一次偶然提升宣传成普遍优化。工程团队依赖性能报告做决策，报告必须可信。

这听起来像规范，但它直接影响技术质量。错误的 benchmark 会把团队带向错误优化，甚至牺牲正确性。第二册强调测量，就是为了建立这种专业习惯。

性能报告示例结构 本册建议每份性能报告采用固定结构。标题说明被测模块和 commit。环境说明机器、系统、编译器、编译选项、CPU 拓扑和限制。工作负载说明输入规模、分布、seed 和正确性 oracle。实验矩阵说明版本、线程数、batch 大小、队列容量和运行次数。结果部分给出样本统计、关键指标和图表。分析部分写瓶颈假设、支持证据、反证和下一步。限制部分说明哪些结论不能外推。

固定结构能提高比较效率。读者以后回看报告，可以直接找到环境和输入；别人审查报告，可以快速发现缺失信息。性能报告不是文学作品，稳定格式比花哨表达更有价值。

测量章节总结 测量的目标是建立证据链。计时告诉现象，计数器提供线索，trace 显示顺序，correctness oracle 保证没有优化错对象，对照实验排除错误解释。没有测量，优化是猜；没有语义，测量可能奖励错误程序。本章的方法会贯穿后面所有实验。

测量决策表 选择测量方法时，先问问题类型。单核循环慢，用计时、汇编、IPC、branch 和 cache 事件；多线程不扩展，用线程数曲线、context switches、锁等待、false sharing 对照；I/O 慢，用队列长度、等待时间、page faults、fsync 时间；分布式慢，用 trace、partition 指标、重试和队列。工具随问题变化，不要用一个 perf 命令解释所有系统。

测量章的回归阅读应审一份不合格性能报告。报告若只写“优化后快了 2 倍”，没有 reference、输入分布、预热、重复次数、方差、编译选项和原始命令，就不能进入后续章节。性能数字必须和可复查条件绑定，否则它只是一次机器状态的截图。

同一个内核至少要分三类测量。第一类是正确性测量，确认不同版本输出一致或在误差合同内；第二类是资源测量，估算字节移动、操作数、内存峰值和队列水位；第三类才是时间测量，包括 p50、p95、p99 或多次运行的置信区间。Roofline 不是为了画漂亮图，而是强迫读者问：这个内核可能受计算吞吐限制，还是受字节搬运限制。

计数器也要有边界意识。移动设备上可能没有 PMU 权限，云机器上可能有噪声，温度和频率会影响结果，容器可能限制事件。没有某个计数器时，可以退回 wall time、输入规模阶梯、工作集扫描和源码证据。高质量报告不是工具越多越好，而是每个结论都说明证据来源和证据限制。

Roofline 估算要从字节账本开始。对一个 map 内核，读入多少字节、写出多少字节、是否额外读写中间数组；对一个 histogram，读 key、写本地桶、合并桶分别是多少；对一个 join，构建表、探测表和输出放大各自占多少。很多报告直接给操作数，却没有写字节数，最后无法解释为什么算术优化没有收益。字节账本不一定精确到每个 cache miss，但必须足够说明算术强度的数量级。

测量脚本还应保存原始环境。至少记录编译器版本、优化选项、CPU 型号、核心数、频率策略、输入生成 seed、运行次数和失败的权限检查。若在 Termux 上跑，报告要说明哪些计数器不可用，哪些实验只做 wall time 和 checksum。可复查性不是学术洁癖，而是让下一次改代码时知道性能变化来自实现、输入、工具还是机器状态。

统计口径也要讲清。一个 benchmark 不能只报告最小值，因为最小值常常代表偶然没有干扰；也不能只报告平均值，因为长尾可能被平均掩盖。对于短内核，可以运行多轮并报告中位数、分位数和离群点处理；对于端到端作业，要报告吞吐、阶段耗时、p95 或 p99、内存峰值和失败重试。不同指标回答不同问题，不能混成一个“快了多少”。

测量章还应训练反证意识。若假设内核受内存带宽限制，可以通过减少计算、改变数据类型、改变工作集大小和比较带宽上限来反证；若假设受分支限制，可以改变输入分布；若假设受锁竞争限制，可以固定线程数和临界区工作量。一个好的性能结论往往来自排除几个错误解释，而不是找到一个看起来合理的解释就停。

5.5 把测量写成可复查的论证

性能测量不是给程序排名，而是建立论证。一个论证至少包含问题、假设、实验设计、原始数据、解释和边界。若只写“优化后快 30

同一个内核在不同阶段可能呈现不同瓶颈。小输入完全在 L1，依赖链和分支可能明显；中等输入落在 LLC，缓存容量和冲突进入成本；大输入超过 LLC，内存带宽主导；多线程后，NUMA、false sharing、调度和频率变化进入成本。若实验只测一个输入规模，就很容易把局部现象当成规律。Roofline 的价值不在图好看，而在逼迫你把算术量、字节移动和硬件上限放在同一张表里。

正确性 oracle 先于计时

没有正确性 oracle 的 benchmark 没有意义。优化版本可能少处理记录、溢出、丢弃错误路径、改变浮点顺序、重复提交或漏掉边界输入。测量前先写 reference，给每个输入生成 checksum 或字节级输出比较。对于浮点，写清允许误差和合并顺序；对于并行输出，写清是否要求稳定顺序；对于分布式提交，写清重复 attempt 如何去重。

Oracle 还要覆盖边界输入。数组长度为 0、1、非对齐长度、不是 SIMD 宽度倍数、记录跨 split 边界、key 高度倾斜、输入含错误行，这些都比随机大输入更能暴露语义问题。许多性能优化在 happy path 上正确，在边界上错。教材要让读者把边界测试视为测量准备，而不是功能测试的附属品。

计时范围也要先定义。分配、初始化、文件读取、预热、核心循环、提交和清理是否计入，取决

于问题本身。如果研究热内核，就应把分配初始化移出计时；如果研究端到端服务，就不能只测内核。报告必须明确计时边界，否则同一结果可以被解释成完全不同的瓶颈。

从字节和操作数建立 Roofline 估算

Roofline 先问两个数：做了多少计算，搬了多少字节。数组求和每个元素一次加载和一次加法，算术强度低，通常受内存带宽限制。矩阵乘如果分块得当，一个加载进缓存的数据块会参与许多乘加，算术强度高，可能受计算吞吐限制。Histogram 虽然每条记录计算不多，但随机写、冲突和分支会让简单 Roofline 不够，需要补充 cache miss 和同步成本。

估算不是为了精确预测每个周期，而是为了排除荒谬解释。若一个内核理论上只做少量算术，却声称通过优化加法指令获得 10 倍大输入加速，就要检查是否同时改变了字节移动或输入规模。若一个内核搬动的数据量已经接近机器内存带宽上限，再追求整数指令减少可能收益有限。Roofline 是把“我觉得”换成“至少要符合物理上限”。

项目里的报告可以固定三类内核：sum、saxpy、histogram。Sum 用来展示内存带宽，saxpy 展示流式读写，histogram 展示随机写和冲突。三者放在一起，读者会看到同样是单机内核，瓶颈模型并不相同。后续典型计算内核章再展开 map、reduce、scan、partition 时，就有测量基础。

计数器、profile 和 trace 分工

计数器回答“发生了多少事件”，profile 回答“时间花在哪里”，trace 回答“事件以什么顺序发生”。三者分工不同。一个锁竞争问题，perf 可能显示 futex 或自旋热点，profile 显示等待路径，trace 显示队列长度和唤醒延迟。一个缓存问题，计数器显示 cache miss，profile 显示热循环，trace 未必有帮助。一个分布式重试问题，硬件计数器几乎无关，trace 和协议指标更重要。

使用计数器要注意权限、复用和噪声。移动设备或容器可能无法访问某些 PMU 事件；多个事件同时测量可能复用，结果不稳定；CPU 频率、温度和后台任务会影响样本。报告中要写清事件名、工具版本、运行轮次、是否绑定 CPU、是否预热、是否丢弃异常值。没有这些信息，数字看起来精确，实际上无法复现。

Trace 的设计要低开销。每条记录都写详细日志会改变程序行为。更好的方式是对阶段边界、任务状态、队列水位、错误事件做结构化记录，必要时采样。trace id 要贯穿 driver、worker、队列和输出，否则三端日志无法拼起来。后续分布式章节会把这个原则用于 request id 和 attempt id。

统计和报告不是装饰

单次运行结果不可靠。至少要多轮运行，给出中位数、最小值、最大值或分位数，并记录异常。平均值容易被少数抖动拉偏；最小值可能代表理想环境；p95/p99 反映尾部。对于交互式或服务型系统，尾延迟比平均吞吐更关键；对于离线批处理，总吞吐和资源利用更关键。报告指标必须匹配目标。

实验矩阵要控制变量。一次只改变线程数、布局、batch size、队列容量或同步方式之一。若同时改变三项，即使变快也无法归因。矩阵不必无限大，但要覆盖能证伪假设的点。例如怀疑 false sharing，就比较紧凑 partial 和对齐 partial；怀疑 NUMA，就比较主线程初始化和 worker first-touch；怀疑任务太细，就扫描 chunk size；怀疑背压，就扫描队列容量和生产速度。

本章最终交付物是一份性能报告模板。模板包括：问题和 oracle，环境和限制，版本和变量，原始结果摘要，关键证据，瓶颈假设，反证或未验证边界，下一步实验。这个模板会贯穿全书。没有报告模板，后续每章的实验会变成零散截图和口头结论；有了模板，读者才能积累可比较的证据。

案例走查：一份不合格报告怎样改成合格报告

不合格报告常写成：“优化后从 120 ms 到 80 ms，提升 33

同样的 80 ms 可能有不同含义。若 instructions 大幅下降、cache miss 不变，优化可能减少了控制流；若 bytes/sec 接近内存带宽，后续应优化数据移动；若 p99 远高于中位数，可能有调度或 I/O 抖动；若 checksum 变了，所有性能数字作废。报告要把这些分支写出来，而不是只给百分比。

本章建议在仓库放一个 [reports/](#) 样例，哪怕只是文本。每次项目优化都按同一结构记录。这样几周后回看，读者能知道为什么当时做了某个选择，而不是只看到一堆快慢数字。

报告里的反证意识

好的性能报告不只支持一个假设，还要主动排除其他解释。若你认为瓶颈是内存带宽，就要说明为什么不是分支、锁、I/O 或调度。可以通过单线程热循环、预热输入、去掉输出、固定线程绑定、替换数据分布来排除。反证意识能防止报告变成“我喜欢哪个解释，就挑哪个数字”。

例如 parallel histogram 变慢，候选原因包括锁竞争、false sharing、key 倾斜、hash 分配、内存

带宽、线程过多。一次只改一个因素：把全局锁换成本地 partial，排除锁；给 partial padding，检查 false sharing；换均匀 key，检查倾斜；预分配 map，检查分配；固定线程数，检查过度订阅。报告把这些对照列出来，读者才能相信结论。

报告也要写负结果。某个优化没有收益，不代表失败，可能说明瓶颈不在此处。负结果如果有清楚实验，同样有教学价值。经典教材让人舒服的地方之一，就是不会把每个技术都写成神奇加速，而是展示什么时候不该用。

工具输出的保存和可复现

性能实验要保存命令、commit、编译选项和原始输出。不要只把手工整理后的表贴进书里。项目可以让每次 benchmark 生成一个目录，包含 `env.txt`、`commands.txt`、`results.csv`、`notes.md`。即使书中不展示全部文件，读者也知道工程上应如何保存证据。

可复现还要求 seed 固定。随机输入必须记录 seed，key 分布参数要写清，坏行比例要写清。若输入来自真实日志，至少记录摘要和脱敏方式。没有输入描述的性能数据无法比较。对于移动设备，还要记录温度、充电状态、CPU governor 或至少说明这些不可控因素可能影响结果。

本章作业可以要求学生互换报告复现。一个人只看另一个人的报告和命令，尝试跑出相近趋势。若复现失败，不一定是代码错，可能是报告缺环境信息。这种作业能让测量规范变成实际能力。

综合练习：审稿一份性能报告

本章综合题让读者当审稿人。给出一份故意不完整的报告：只写“多线程版本提升三倍”，不写输入、机器、正确性、运行轮次和指标。读者需要指出缺失项，并提出补实验计划。补实验至少包括：串行 reference checksum，输入规模阶梯，线程数扫描，预热和冷启动分离，partial 布局对照，至少三轮重复运行，异常值记录和结论边界。

审稿时要追问可证伪性。报告说“瓶颈是内存带宽”，那就要问是否估算了字节数，是否接近已知带宽上限，是否比较了小输入和大输入，是否排除了锁等待。报告说“锁太慢”，那就要问 `futex` 或等待指标在哪里，是否有无锁或线程本地版本对照，是否锁内做了 I/O。每一个瓶颈声明都必须有证据路径。

第二部分要求读者把不完整报告改写成合格报告。不能只补漂亮图表，必须补实验矩阵和原始命令。若某个工具不可用，要写替代证据和未验证风险。审稿训练能让读者以后写自己的报告时自动补齐这些项。

这个题也能防止书本变成“答案集合”。真实工程中，很多争论不是缺技术，而是缺证据。会审报告，才会做优化。测量章的最终能力不是会用 `perf`，而是能判断一个性能结论是否站得住。

容易出错的边界：测量数字为什么会骗人

第一个骗人数字是平均值。一次后台写回、温控降频或调度抖动可以把平均值拉偏。报告至少要给中位数、最小值、最大值或分位数，服务型路径还要看尾延迟。第二个骗人数字是总耗时。总耗时不拆阶段时，热循环、I/O、初始化、清理和提交都混在一起，任何解释都可能成立。第三个骗人数字是单次 speedup。没有正确性 oracle，优化可能只是少做了工作。

第四个骗人数字是硬件计数器本身。事件可能不可用、被复用、受内核版本影响或含义因架构不同而变化。计数器要和源码、反汇编、输入规模曲线互相印证。第五个骗人数字是“CPU 使用率”。高 CPU 可能是有用计算，也可能是自旋和重试；低 CPU 可能是 I/O 等待，也可能是背压保护系统。没有等待和队列指标，CPU 使用率无法单独解释性能。

第六个骗人数字是小输入结果。小输入适合暴露固定成本、分支和前端，大输入适合暴露内存、TLB 和 I/O。只测小输入得出的优化结论，可能在真实工作负载上消失。反过来，只测大输入也会掩盖小请求延迟问题。实验矩阵必须服务目标 workload。

本章可以要求每份报告都有“可能错误解释”一栏。比如当前结论认为瓶颈是内存带宽，但可能错误解释包括线程迁移、输入未预热、cache 事件不可用、编译器优化差异。把这些写出来不是削弱结论，而是让结论有边界。高质量测量的标志，是知道自己还不知道什么。

本章核心接口：BenchmarkCase

测量章给项目留下 `BenchmarkCase`。它定义输入生成、reference 校验、被测版本列表、运行轮次、预热策略、指标采集和报告路径。每个章节新增实验时，不再临时写一套 benchmark，而是注册一个 case。这样实验格式统一，结果可比较。

`BenchmarkCase` 还要支持跳过原因。某些 case 需要 NUMA、PMU、特定 SIMD 或文件系统能力，当前设备不满足时标记 `skipped with reason`，而不是静默不跑。这个机制对手机环境尤其重要：

能跑的认真跑，不能跑的明确说明边界。

从测量进入数据布局

测量章不直接告诉你改哪里，它告诉你慢在哪里、证据有多强、哪些解释被排除。若报告显示字节移动和 cache miss 是主成本，下一步自然进入数据布局；若报告显示分支和指令是主成本，回到热循环；若报告显示等待和队列是主成本，进入并发和 I/O。数据布局章之所以紧跟测量章，是因为很多高性能系统的第一大成本不是算得慢，而是搬得多、搬得散、搬得太远。

项目里，BenchmarkCase 的输出应直接喂给布局决策。比如报告显示统计阶段每条记录实际只需要 8 字节热字段，却从对象数组搬了 96 字节，就有充分理由做冷热分离。若报告显示热字段已经很窄，布局优化就不该成为下一步。测量给布局优化设置了进入条件。

5.6 项目落地

本章要求项目拥有一个最小但严肃的 benchmark harness。它至少要支持：选择输入规模，选择版本，选择线程数，重复运行多轮，校验输出，输出机器和编译信息，记录最小值和中位数。后续可以逐步加入 CPU 绑定、NUMA 初始化、perf 命令模板、JSON 输出和图表脚本。不要一开始就做复杂框架，但也不能继续依赖手工改代码和肉眼读终端。

项目里的每个优化提交都应附带实验说明：优化目标是什么，影响哪条路径，正确性如何验证，基线版本是什么，在哪些输入和线程数下有效，在哪些情况下无效。这样做看似增加工作量，但能避免“改完之后不知道为什么快，也不知道什么时候会慢”的维护风险。

测量问题 先写问题，再选指标。若问题是“热循环是否受内存带宽限制”，指标应包含字节数、时间、带宽估算和 cache/TLB 事件；若问题是“线程池是否过载”，指标应包含队列水位和等待时间。指标跟问题脱节，就会变成装饰。

基线 基线不是随便一个旧版本，而是能说明语义正确的最小实现。优化版必须和基线对同一输入、同一输出、同一错误条件比较。没有 reference，性能越好越危险。

Roofline Roofline 把计算强度和硬件上限放在同一张图里。它不能替代真实分析，却能帮助判断当前优化方向：若明显带宽受限，继续减少算术指令意义有限；若计算受限，改善数据布局可能收益小。

统计稳定性 单次运行容易被调度、温度、频率、后台任务影响。报告至少要有重复次数、中位数、分位数或误差范围。移动设备上还要记录电源状态和热 throttling 的可能性。

计数器限制 perf 权限、虚拟化、内核版本和硬件支持都会影响计数器可用性。不可用时写明原因，换用时间阶梯、输入缩放或应用 trace 作为替代证据，不能把缺失计数器的猜测写成事实。

输入设计 性能实验要有能击穿朴素直觉的输入：小工作集、大工作集、热点 key、随机访问、冷读、热读、单线程和多线程。只测一个默认输入，结论无法外推。

报告结构 一个好报告应从假设开始，以证据结束：假设、环境、命令、输入、正确性、指标、结果、解释、限制、下一步。这个结构后面每章都复用。

反证意识 测量要主动找反例。优化让一个输入变快，却让另一个输入变慢，并不一定失败；关键是知道适用范围。教材要鼓励读者写“此优化只适合顺序扫描大数组，不适合随机小对象”。

案例：同一算法三种输入 对同一个内核测小数组、L3 级数组、远超内存缓存数组。用曲线说明工作集变化，而不是只给一个平均时间。

案例：计数器不可用降级 在 perf 无权限时保留命令和错误输出，改用输入阶梯和反汇编解释。这个案例训练诚实报告。

案例：Roofline 初版 估算 bytes、operations 和运行时间，判断内核处于带宽区还是计算区。估算要写公式和限制，不要求一开始完美。

源码阅读路线 Linux 源码阅读从 `tools/perf` 的事件选择和输出路径进入，再对照 `kernel/events/core.c` 中 perf event 的创建、权限检查和计数更新。阅读目标是解释 BenchmarkReport 中的计数从哪里来，为什么某些事件在当前设备不可用，以及不可用时如何降级。

阅读报告要写证据等级。能直接测到的数字放在第一层；从源码控制流推导出的解释放在第二层；当前设备无法验证、只能作为合理猜测的内容放在第三层。这样读者会知道性能论证报告中哪些结论可复现，哪些结论需要换机器继续确认。

源码阅读还要克制。看到一个复杂函数，不要把每个分支都展开；先回到本章的问题，问它能否解释一个等待点、一次状态转移或一条性能曲线。解释不了的内容可以留到索引，不要打断正文主线。

同一算法三种输入 对同一个内核测小数组、L3 级数组、远超内存缓存数组。用曲线说明工作集变化，而不是只给一个平均时间。

计数器不可用降级 在 perf 无权限时保留命令和错误输出，改用输入阶梯和反汇编解释。这个案例训练诚实报告。

Roofline 初版 估算 bytes、operations 和运行时间，判断内核处于带宽区还是计算区。估算要写公式和限制，不要求一开始完美。

测量目标

先定义要比较什么，再决定测什么。吞吐、延迟、扩展曲线、内存峰值和恢复时间不是同一个目标。

实验设计：同一程序分别以总吞吐和尾延迟为目标，说明最佳参数可能不同。

reference

性能实验必须先证明结果正确。没有 reference 的加速可能只是少做了工作，尤其在并行和失败恢复中更危险。

实验设计：每个 benchmark 运行前后都校验 checksum、记录数和错误分类。

预热和冷态

JIT 不一定存在，但 page cache、CPU 频率、分配器缓存和分支预测都会造成前几轮异常。预热不是为了作弊，而是为了知道自己测的状态。

实验设计：保留每轮结果，不只保留最快值，观察前几轮是否稳定。

统计方法

一次运行没有说服力。最小值常接近理想路径，中位数反映常态，尾部暴露抖动。报告应说明选择哪个统计量。

实验设计：同一输入至少运行多轮，记录 min、median、p95 和异常轮原因。

Roofline

Roofline 不是彩色图装饰，而是把计算量、字节量、峰值算力和带宽放进同一张账本。它帮助判断程序受算力还是带宽限制。

实验设计：为数组求和、点积和矩阵分块分别估算字节与操作量。

计数器限制

perf 计数器可能受权限、复用、多路复用和硬件支持影响。数字出现小数或 warning 时，不能当作精确事实。

实验设计：把 perf 输出中的权限、事件支持和 multiplex 信息写进报告。

profile 与 trace

profile 告诉时间在哪里聚集，trace 告诉事件按什么顺序发生。锁等待、队列水位和 I/O 完成常常需要 trace 才能解释。

实验设计：为一个队列实验同时记录采样热点和每批次等待时间。

对照实验

好的对照一次只改一个因素。若同时改编译选项、输入规模、线程数和数据布局，结论无法归因。

实验设计：建立 baseline、单因素变化和回退版本三列结果。

环境记录

编译器、优化级别、CPU governor、线程数、输入文件和后台负载都会影响结果。报告缺环境就无法复现。

实验设计：自动打印编译命令、CPU 信息、可见核心和输入摘要。

负结果

没有提升也是结果。它可能说明瓶颈判断错误、优化被编译器已有策略覆盖，或成本迁移到别处。

实验设计：把失败优化写进报告，说明下一步为什么不继续。

5.7 性能报告不是跑分截图，而是工程论证

测量章节如果写得像工具说明，读者会学会几个命令，却不会学会判断。本章应该从一句模糊需求开始：“这个程序太慢了。”这句话没有工程意义，因为它没有说慢的是吞吐、平均延迟、尾延迟、启动时间、恢复时间，还是单位能耗。把目标拆清楚，才知道测什么。一个批处理作业追求总吞吐，一个交互服务关心尾延迟，一个 checkpoint 系统关心恢复时间；同一个优化可能改善其中一个目标，伤害另一个目标。

测量的第一原则是正确性先行。没有 reference 的性能数字不值得相信。并行版本少处理一部分记录，SIMD 版本忽略尾部，I/O 版本没有等待持久化，都可能得到漂亮速度。每个 benchmark 都应先写校验：记录数、错误数、checksum、输出顺序、浮点误差或 manifest 状态。性能报告里把校验放在环境之前也不过分，因为“算错但更快”不是优化。

第二原则是把实验状态写清。CPU 频率、温度、后台负载、page cache、输入文件、编译器、优化级别、线程数、亲和性和权限都会影响结果。手机 Termux 尤其要注意温度和调度限制：连续跑十轮，后几轮可能因为降频变慢。报告不要掩盖这种现象，应该把它当作系统行为记录下来。真实工程里，性能稳定性本来就是指标之一。

第三原则是统计。最快值能说明理想路径，中位数说明常态，尾部说明抖动。只给一次运行结果，没有复查价值。实验至少要多轮运行，保留每轮原始结果，再给出选择某个统计量的理由。若有异常轮，不要直接删掉；先判断是系统噪声、错误输入、page cache 状态、调度抖动还是程序 bug。异常轮经常比平均值更能暴露系统问题。

Roofline 的作用是把算术和数据移动放到同一张账本。数组求和每个元素只做很少计算，却要搬大量字节，通常受带宽限制；矩阵乘如果分块得当，同一数据被多次复用，可能转向计算瓶颈。读者不用一开始画很漂亮的图，但要会估算每条记录读多少字节、写多少字节、做多少操作。估算和测量差距大时，说明隐藏成本存在，例如字符串分配、cache miss、写放大或重试。

计数器要谨慎解读。perf 事件可能受权限限制、硬件支持、多路复用和采样误差影响。IPC 低不一定是前端问题，branch miss 高不一定就是总瓶颈，cache miss 数字也要结合工作集和输入分布。报告应把直接测量、工具限制和推断分开写。高质量的测量章不追求神秘命令，而追求可复查结论：为什么这个指标能支持当前假设，为什么它不能支持更强的结论。

本章可以要求读者实现一个最小 benchmark harness。它接收输入规模、版本名、线程数和重复次数；运行前构造输入，运行后校验输出；记录环境、原始结果、统计摘要和失败边界。后续每章都复用它。这样整本书的实验就不会变成散落的终端截图，而会形成同一种证据语言。

案例推导：一次“优化成功”的性能报告为什么可能不合格

某个并行归约版本比 baseline 快百分之三十，开发者准备提交。审稿时先问三个问题：输出是否和 reference 一致，输入和环境是否固定，瓶颈是否真的按预期移动。检查后发现，优化版本跳过了错误记录，baseline 把错误记录计入错误分类；两者做的工作不同。这个案例说明性能报告第一行不应是速度，而应是语义校验。

合格报告先列输入合同：记录数、字段范围、错误行比例、随机种子和预期输出摘要。再列环境：编译器、优化级别、CPU 可见核心、线程数、频率状态、输入是否热 cache。然后列版本差异：baseline、改进一、改进二分别改变了什么。最后才给结果表。这样的报告看起来繁琐，但它允许别人复查；没有这些信息，所谓提升只能算一次运行截图。

统计也要认真。若十轮结果分别是 10 ms、10.2 ms、10.1 ms、18 ms、10.0 ms，平均值会被异常轮拉高，最小值又掩盖抖动。报告可以同时给 min、median、p95，并解释异常轮。异常可能来自系统调度、page cache、温度，也可能来自程序内部偶发锁等待。性能工程不是把异常删掉，而是判断异常是否属于目标 workload。

Roofline 在这个案例中用于排除错误方向。若归约每个元素只做一次加法，却读取大量内存，优化算术指令可能没有明显收益；若矩阵分块后数据复用增加，算术端口可能成为瓶颈。报告要估算字节和操作量，再用测量校正。估算错误很正常，错误本身有价值：它提示隐藏拷贝、额外分配、cache miss 或写放大。

计数器使用要写限制。手机 Termux 或普通用户环境可能无法读取某些硬件事件，perf 也可能 multiplex。此时可以用阶段时间、输入规模、系统调用计数或 trace 替代，但要标注证据等级。不要因为拿不到理想计数器就放弃测量，也不要把替代指标说成硬件事实。测量章的目标是让读者诚实地建立证据，而不是崇拜工具。

最终，benchmark harness 应成为项目基础设施。每章新增优化都通过同一个 harness 运行：构

造输入、运行 reference、运行候选版本、校验输出、记录环境、保存原始数据。这样后续读者能看到性能结论如何积累，而不是每章重新发明一套测试脚本。

核心理想

测量不是为了得到一个数字，而是为了排除错误解释，缩小瓶颈范围，并验证优化是否真的改变了瓶颈。

实验：对 Compute Systems Labs 的顺序归约和并行归约运行 `perfstat`。记录时间、cycles、instructions、IPC、cache-misses 和 context-switches。解释并行版本是否真的更快，如果没有，瓶颈可能在哪里。

性能报告先回答一个具体问题 测量不是把程序跑十次取平均，而是先提出一个可回答的问题。比如“SIMD 版本是否更快”太粗；更好的问题是“在输入已经驻留内存、输出预分配、长度大于一百万、允许浮点重排的条件下，向量化归约是否提高热循环吞吐”。问题越清楚，测量边界越清楚，结论越不会被误用。端到端时间、热循环时间、系统调用时间和排队时间回答的是不同问题，不能混在一个数字里。

贯穿项目每个阶段都要有两套数字。第一套是端到端数字，包含真实用户会承受的全部成本：读取、解析、计算、写出、提交和恢复检查。第二套是隔离数字，只测某个内核或某个状态机，用来解释瓶颈。端到端数字告诉我们系统是否变快，隔离数字告诉我们为什么变快或为什么没变快。只有隔离数字没有端到端，容易优化错地方；只有端到端没有隔离数字，结论无法复查。

Roofline 要和实际字节数连接 Roofline 模型把内核分成算力受限和带宽受限，但它依赖两个输入：实际执行的操作量和实际搬运的字节数。很多报告只数源码里看得见的数组读写，漏掉临时对象、写分配、cache line 取整、哈希表桶访问、指针对象和输出材料化。若字节数估错，算术强度就错，Roofline 结论也错。项目报告应写出字节账本：每条记录读取多少字节，热字段加载多少，冷字段是否访问，partial map 写多少，最终输出写多少。

Roofline 也不是唯一答案。它适合解释规则数值内核，对分支密集、锁等待、I/O、任务调度和分布式重试不够直接。遇到这些场景，要配合 profile、trace、队列指标和系统调用统计。性能分析的工具链应按问题选择：热循环看 counters 和汇编，排队看 trace，I/O 看提交和完成，分布式看 request timeline。把所有问题都塞进 Roofline，只会让模型失真。

统计显著性来自实验纪律 性能数据有噪声：CPU 频率、温度、后台任务、页缓存状态、输入分布、内存分配地址、调度迁移都会影响结果。报告不能只给一次运行的时间。至少要给多次样本、median、p90 或 p99、最小值和最大值；还要说明是否预热、是否固定 CPU、是否清理或保留页缓存、是否禁用或记录 turbo 状态。若环境无法完全控制，就如实写出限制。

对系统项目来说，尾延迟往往比平均值重要。一个异步写入队列平均很快，但偶尔 fsync 卡住，会让上游堆积；一个任务调度器平均开销小，但少数任务长尾，会拖慢整个 stage；一个分布式请求 median 很低，但 retry 后 p99 很高，会影响用户感知。本章应让读者习惯同时看吞吐和分位数，而不是只看平均吞吐。

实验：审稿式复查一份坏报告 本章可以给一份故意不完整的性能报告：某优化声称速度提升三倍，但没有输入规模、没有编译选项、没有正确性校验、没有预热、没有原始样本、没有环境信息。读者要像审稿人一样提出问题：是否包含文件读取，是否复用了页缓存，是否改变数值语义，是否只测了一次，是否对比同一编译级别，是否有端到端收益。这个练习能训练工程判断。

随后让读者重写报告。固定输入生成器，保存 checksum，记录命令和版本，跑足够多样本，分离初始化和热循环，输出 CSV，画或用文字描述扩展曲线。报告最后必须有结论边界：本结果只说明某机器、某输入、某编译器下的行为；迁移到其他架构或输入分布需要重测。高质量性能教材必须反复强调这种边界。

Linux 工具链的定位 Linux 上的 `perfstat` 适合看整体事件计数，`perfrecord` 和 `perfreport` 适合定位热点，`ftrace` 或 `tracepoint` 适合看内核事件时间线，`strace` 适合确认系统调用形态，`iostat` 和 `pidstat` 适合观察 I/O 和进程行为。工具不是越多越好，而是每个工具回答一个问题。先写问题，再选工具。

项目中可以提供统一 benchmark 输出：配置、输入摘要、阶段时间、样本列表、checksum、错误数、队列水位和资源指标。这样后续章节每次优化都在同一格式下比较。报告格式一旦稳定，读

者会把注意力放在因果解释上，而不是每章重新猜测数字含义。

事故复盘：一份无法复现的三倍加速报告 性能报告里最危险的句子是“优化后三倍加速”，后面却没有输入、命令、预热、样本、正确性校验和环境说明。一次重跑发现加速消失，原因可能是页缓存热了、CPU 频率不同、编译选项改变、输入规模不一样，甚至优化版本少处理了错误行。测量不是按秒表，而是给一个性能结论建立可复查的证据链。

Roofline 的价值也在这里。先用字节账本估算必要数据搬运，再用算术操作数估算计算量，把点放到带宽和峰值算力之间。如果报告声称超过机器带宽，就要检查字节是否漏算；如果声称受计算限制，但 IPC 很低、cache miss 很高，就要回到 profile。计数器不是为了堆数字，而是和模型互相质疑：模型解释不了的地方，才值得继续挖。

合格报告要保留原始样本而不是只给均值。至少给出最小值、中位数、p95、标准差或多次运行列表，并说明是否固定频率、是否清理缓存、是否隔离后台负载。正确性校验放在性能数字之前：checksum、记录数、错误数、manifest 条目先一致，时间比较才有意义。后续每一章的实验都沿用这个规矩，否则“优化”可能只是测量噪声或语义损坏。

测量报告实验判读 判读性能报告时先找缺失项：有没有 reference，有没有输入摘要，有没有编译命令，有没有原始样本，有没有环境说明。再看数字之间是否互相支持：字节账本和带宽是否匹配，Roofline 判断和 profile 热点是否一致，trace 队列等待和端到端延迟是否一致。若数字相互矛盾，不要急着选喜欢的结论，应先修测量边界。

这一段也服务于复习。读者合上书后，应该能凭本章的判读口径重新审查自己的代码：有没有最小正确版本，有没有能击穿旧方案的输入，有没有状态记录，有没有资源指标，有没有失败后的恢复动作。若五个问题缺两个以上，就说明实现还停在演示层，而不是工程层。

给性能报告写审稿意见 一份性能报告可以像论文一样审稿。第一问是问题是否明确：它测的是端到端，还是热循环，还是某个系统调用路径。第二问是 reference 是否存在：优化后结果是否和基线一致。第三问是输入是否可复现：规模、分布、随机种子、冷热状态和错误比例是否写清。第四问是样本是否足够：一次运行不能说明稳定性。

审稿意见不要只写“补充数据”。应指出缺失数据会影响哪个结论。例如没有字节账本，就不能判断 Roofline；没有原始样本，就不能判断分位数；没有编译命令，就不能判断优化级别和 fast math；没有环境信息，就不能迁移到另一台机器；没有错误注入，就不能说明系统在失败下仍可靠。

这种审稿式训练能防止性能章节变成工具教程。工具只是证据来源，报告才是推理载体。读者学会审稿，就能反过来写出可复查的实验，也能识别网上大量“快了多少倍”但无法复现的结论。

实验课：重写一份不可复查的三倍加速报告 先给读者一份坏报告：某版本数组归约快了三倍，只写了两行时间，没有输入大小、编译命令、CPU 信息、线程数、预热策略、样本数量和 checksum。要求读者写审稿意见，指出每个缺失项会影响什么结论。比如没有 checksum，就不知道是否优化错程序；没有命令，就不知道是否改变优化级别；没有样本，就不知道是否只是偶然。

然后重写实验。固定输入生成器，保存随机种子和输入摘要；编译时记录编译器版本、优化级别和开关；运行时先预热，再收集多次样本；输出 raw CSV，不只输出平均值；同时记录端到端时间和热循环时间；最后验证 checksum。若使用 perf，再保存事件列表和权限限制。若没有 perf 权限，就写替代证据：输入阶梯、阶段时间和源码结构。

判读时要练习拒绝过度结论。若热循环快了三倍但端到端只快百分之十，说明瓶颈在其他阶段；若 median 快但 p99 变差，说明尾部路径出问题；若小输入变慢、大输入变快，说明固定成本和吞吐收益存在交叉点；若换输入分布后收益消失，说明优化依赖工作负载。报告要把这些边界写进结论，而不是只展示最好的一行。

最后把报告格式固化到项目。每次 benchmark 生成 env、commands、results、notes 和 checksum。书中可以只展示摘要，但工程上保留原始材料。性能优化是可复查过程，不是一次演示。

性能报告练习验收重点 合格答案要能被别人复跑。输入、命令、编译器、样本、checksum、原始输出、统计方法和结论边界都要有。Roofline 题必须写实际字节数，不允许只引用理论公式。若结论是“优化有效”，必须说明在哪个输入范围有效，在哪个范围不确定。

答案还要写“为什么不采用更复杂方案”。例如没有证据证明队列是瓶颈，就不应把锁队列换成无锁；没有证明输出顺序可变，就不应随意重排；没有证明错误路径能恢复，就不应只提交正常路径。能解释不做什么，说明读者开始具备工程判断。

最后，答案必须包含可复查材料。至少有输入摘要、关键状态、期望输出、实际输出、运行命令

和未验证边界。没有这些材料，老师或同伴无法判断结论是否成立。第二册的作业不是让读者写漂亮段落，而是训练读者把系统问题变成可验证证据。

项目实现：可信测量、Roofline 与性能计数器 本章对应的贯穿项目实现不要求一次写成完整框架，但必须有最小可运行切片。这个切片包含三个部分：一个 reference，一个带本章机制的实现，一个能击穿 reference 之外隐含假设的测试。Reference 用来固定语义，机制实现用来展示本章知识，测试用来证明新增复杂度确实解决了问题。

代码组织上，不要把所有内容放进一个大文件。基础类型、输入生成、核心机制、报告输出和测试应分开。即使当前只是教学项目，也要让目录能表达责任边界。这样后续章节接入时，不会因为找不到状态归属而重新发明一套对象。

每个实现都要输出一段机器可读摘要。摘要至少包含输入规模、关键状态、错误数、输出校验、耗时和未验证边界。这里的机器可读不要求复杂格式，简单的 key value 文本也可以。关键是让脚本和读者都能复查，而不是只能看屏幕截图。

测试应包含正常路径、边界路径和失败路径。正常路径证明功能可用；边界路径证明长度、容量、尾部、空输入、关闭或超时没有遗漏；失败路径证明本章机制不是装饰。若失败路径写不出来，往往说明本章概念还没有落到工程对象上。

报告最后写一个取舍段。新增机制带来了什么好处，增加了什么成本，哪些场景不应该使用它，下一章会复用哪个字段或对象。这个取舍段非常重要，因为系统工程不是把所有高级机制都加进去，而是在证据和约束下选择合适的边界。

调试笔记：数字矛盾时先怀疑边界 如果 perf 显示指令减少，但端到端时间没变，先检查测量是否包含 I/O、初始化或排队。如果 Roofline 判断是带宽受限，但 profile 热点在锁等待，说明模型用错了对象。如果 median 变快但 p99 变慢，说明慢路径被隐藏。性能报告的价值不是让所有数字一致，而是发现数字不一致时能回到边界重新拆分。

复查清单：可信测量、Roofline 与性能计数器 复查本章时，先检查 reference 是否仍然存在。没有 reference，后面的并发、优化、异步或分布式改造都无法证明语义没有变。再检查压力输入是否尖锐：它应当攻击本章最核心的边界，而不是只把数据量放大。第三，检查状态是否可观察：关键对象的身份、状态、错误和提交结果应能在报告中找到。

第四，检查代码示例是否说明禁止行为。调用者不能做什么，往往比能做什么更能体现工程质量。第五，检查实验是否写出未验证边界；当前设备、权限或输入无法证明的结论，必须标注。第六，检查本章对象是否能被下一章复用，不能复用就说明边界还不稳。

完成这六项后，再看文字是否连贯。每个概念都应从问题推出，每个案例都应从朴素方案走到改进方案，每个实验都应能解释一个判断。若某段话放到其他章节也成立，就把它改成本章的具体对象、具体输入和具体失败。

5.8 工程案例：把测量写成可复核的技术论证

性能测量最容易变成截图堆叠。一个耗时数字、一个 CPU 使用率、一个 perf 输出，看起来都很客观，但如果没有输入、边界、统计方法和反例，它们并不能支持工程结论。本章的核心不是教几个命令，而是训练读者把测量写成可复核的论证：假设是什么，变量是什么，证据是什么，哪些结论还不能说。

一、先定义被测问题和不变量 测量前先写问题。是比较两个解析器版本，还是评估线程数扩展，还是判断是否内存带宽到顶？每个问题对应不同边界。比较解析器时，输入、输出、错误处理和 checksum 必须一致；评估线程扩展时，线程创建、初始化和合并是否计入要写清；判断带宽时，要记录读写字节数和工作集大小。

不变量是防止测错程序的底线。优化版本必须输出同一 checksum 或满足同一误差合同；输入生成必须固定 seed；编译选项和目标架构必须记录；CPU 频率、温度、后台任务和权限限制要说明。没有这些信息，结果很难复现，也很难和别人的结果比较。

二、计时边界决定结论含义 同一个程序可以测端到端时间，也可以测热循环时间。端到端时间包含读取、初始化、调度、合并和写出，适合用户体验；热循环时间排除外围成本，适合分析内核。两者都有效，但不能混用。若你用热循环时间证明系统快，却端到端被 I/O 队列拖慢，结论就是错位的。

Compute Systems Engine 的报告可以把阶段拆成 input、parse、local aggregate、shuffle write、

reduce、manifest commit。每阶段记录开始结束时间、输入字节、输出字节和错误数量。这样当总时间变化时，能看到是热内核改善，还是成本迁移到写出或合并。测量的第一目的不是证明自己快，而是定位变化发生在哪里。

三、Roofline 从字节和算术强度推导 Roofline 模型把内核放到两个约束下：计算峰值和内存带宽。算术强度是每搬动一个字节做多少计算。数组求和算术强度低，容易受内存带宽限制；矩阵乘分块后重复使用数据，算术强度高，可能接近计算峰值。这个模型不要求读者背图形，而是先估算字节移动和操作数。

估算要诚实。读一个 `std::uint64_t` 数组求和，逻辑上每元素 8 字节，但如果写输出、读索引或访问对象指针，实际字节会更多；哈希表查询不仅读 key 和 value，还读 bucket、节点指针和控制字节；日志解析除了输入字节，还可能分配字符串和写错误对象。Roofline 的价值是逼迫你写出数据移动账本。

四、计数器是线索，不是判决书 `perfstat` 可以给 cycles、instructions、branches、branch misses、cache misses 等线索。但事件名、采样精度、权限、内核版本和 CPU 架构都会影响解释。IPC 低可能是内存等待，也可能是分支失败，也可能是前端供给不足；cache miss 高也要看 miss 的级别和访问频率。计数器必须和源码、输入、反汇编和阶段时间一起解释。

报告中最好写“该证据支持什么，不支持什么”。例如 branch misses 在随机输入上明显上升，支持“分支预测是变化因素”；但如果总时间没有变化，说明分支成本可能被其他瓶颈掩盖。LLC miss 下降但耗时不变，可能表示瓶颈已经迁移到写出或锁等待。成熟测量不是只找支持自己假设的数字，也要解释反例。

五、统计方法处理波动 单次运行时间不可靠。缓存状态、调度、温度、频率、后台任务、页错误都会造成波动。至少要多轮运行，报告中位数、最小值、最大值或分位数。若波动很大，不要急着取平均，而要先找波动来源：是否有冷启动，是否有 I/O，是否有垃圾输入生成混入，是否有 CPU 迁移。

小差异尤其要谨慎。一个 2

六、防止 benchmark 被编译器优化掉 编译器会删除无用计算。若求和结果没有被观察，循环可能被优化掉；若输入是编译期常量，计算可能被常量折叠；若函数太小，benchmark 框架开销可能主导。正确做法是使用运行时输入、输出 checksum、避免未定义行为，并把防优化方法写进报告。

C++ 示例中可以把结果写入 `volatile` sink 或返回给调用者统一校验，但不要滥用 `volatile` 去表达并发同步。`volatile` 不是原子，也不是内存序工具。性能测量里的防优化和并发语义要分开讲，避免读者把 benchmark 技巧带到生产并发代码里。

七、回归阈值和性能预算 一次测量解决不了长期维护。项目应为核心内核设置性能预算：输入规模、机器类型、编译选项、允许波动、失败阈值。CI 环境可能噪声大，不适合卡很小差异；本地基准机可以设更严格阈值。性能回归报告要保存历史曲线，而不是只看当前一次。

预算也要区分阶段。解析内核退化 5

八、实验交付：BenchmarkCase 本章给项目留下 `BenchmarkCase`。它记录输入描述、生成 seed、版本名、编译选项、线程数、绑定策略、预热轮数、测量轮数、checksum、阶段时间、可用计数器和不可用原因。每个后续章节的实验都复用这个结构。这样硬件、布局、同步、I/O 和分布式结果可以放在同一套报告里比较。

验收时让另一个人只看报告复现实验。如果他不知道输入怎么生成，说明报告不合格；如果他不知道计时边界，说明结论不合格；如果他无法判断输出是否正确，说明 benchmark 不可信。测量的质量标准是可复核，而不是命令看起来专业。

5.9 案例报告：一次优化为什么必须写反证

性能报告只写“优化后快了百分之多少”是不够的。高质量报告还要写反证：哪些解释被排除，哪些输入上优化无效，哪些指标和假设矛盾。反证不是否定工作，而是让结论更可信。没有反证的报告通常只是在讲故事。

一、先列候选瓶颈 假设日志解析版本 B 比版本 A 慢。候选瓶颈可能是分支失败、冷字段搬运、字符串分配、队列等待、page fault、写出背压、CPU 降频。报告第一步不是猜一个最熟悉的原因，而是列候选并安排证据。分支失败看 branch misses 和输入分布；冷字段搬运看对象大小和

cache miss；分配看 allocation count；队列等待看 trace；page fault 看缺页计数；写出背压看队列水位。

候选列表不需要无限长，但要覆盖最可能的层次。若只看 perf，不看队列，可能把等待误判成 CPU 内核慢；若只看应用日志，不看反汇编，可能把编译器生成差异漏掉。系统测量的难点是跨层归因，候选列表能防止过早收敛。

二、反证输入比理想输入更重要 一个优化在理想输入上有效，不说明它普遍有效。查表分类在随机字符上可能减少分支失败，在规则日志上可能增加 load；线程本地 partial 在高竞争计数上有效，在小输入上合并成本可能超过收益；大 batch 提高吞吐，却恶化尾延迟。报告要主动给出这些反例输入。

反证输入应针对机制弱点设计。优化分支，就提供可预测分支输入；优化 cache，就提供工作集小到全在 L1 的输入；优化多线程，就提供小输入和单热点输入；优化 I/O 批处理，就提供低延迟敏感输入。若优化在这些输入上无效，报告写清适用范围。知道边界的优化，比宣称全能的优化更可靠。

三、测量要区分效果大小和解释强度 效果大小是数字变化，解释强度是证据能否支持原因。一个版本快了 40

反过来，一个指标明显变化但总时间不变，也有价值。它说明优化确实改变了局部成本，但主瓶颈不在那里，或瓶颈迁移到其他阶段。这样的结论能避免继续在无效方向投入。没有提升也是结果，只要能解释。

四、统计显著性和工程显著性不同 多轮测量可能显示 1

这不是反优化，而是成本意识。教材要让读者敢于删除无效或收益不足的优化。性能工程不是只添加技巧，也是控制复杂度。

五、报告模板要服务推理而不是填空 项目报告可以固定字段：问题、输入、环境、版本、假设、指标、结果、反证、边界、下一步。但正文不能变成模板话。每个字段都要写本章具体内容。例如在 SIMD 章，反证是尾部和跨块状态；在锁章，反证是低竞争短临界区；在分布式章，反证是迟到响应和重复请求。格式统一，分析必须具体。

最终读者应能读报告复述因果链：为什么怀疑这个瓶颈，做了哪个最小改动，哪些指标支持，哪些输入限制结论，下一步瓶颈在哪里。能做到这一点，测量章才真正完成。

5.10 补强案例：阶段化报告如何防止局部优化误导

一个局部内核变快，不代表系统端到端变快。阶段化报告的意义就是防止局部优化误导。它把作业拆成读取、解析、聚合、分区、写出、提交和恢复几个阶段，每个阶段都有时间、字节、对象数量和错误统计。优化前后对比阶段表，才能看见成本迁移。

一、局部快可能暴露下游慢 解析器优化后，map 端更快地产生 shuffle block，writer 队列可能开始积压。端到端时间不变，不表示解析优化无效，而是主瓶颈迁移到写出。若没有阶段报告，团队可能继续优化解析，浪费时间。若有阶段报告，就会转向 I/O 背压或 block size。

这类迁移是性能工作的正常结果。优化像推瓶颈，不像消灭所有成本。报告要写“旧瓶颈是什么，新瓶颈是什么”，而不是只写“提升不明显”。

二、阶段边界要和语义边界一致 阶段不能随便按函数切。一个函数可能既解析又写队列，另一个函数可能只是调度任务。阶段应按数据状态划分：原始输入、解析后 batch、本地 partial、shuffle block、manifest。这样阶段时间能和对象生命周期对应。若阶段边界和语义边界不一致，恢复和性能报告会互相对不上。

三、报告表的最小字段 每阶段至少记录 input bytes、output bytes、records、tasks、elapsed time、queue wait、error count、retry count。硬件阶段可加 cycles 和 cache miss，I/O 阶段可加 fsync time，分布式阶段可加 timeout 和 late response。字段不必一次全满，但缺失要写明。

四、实验 先优化解析，再优化布局，再优化写出。每次只改一个阶段，阶段报告都保留。最终比较三次优化的端到端效果和瓶颈迁移。这个实验训练读者从系统角度看性能，而不是被单个 benchmark 牵着走。

5.11 补充案例：如何写一次性能回归说明

性能回归不可避免。重要的是写出能指导修复的说明，而不是只说“变慢了”。一份好的回归说明包含触发版本、输入、环境、变化幅度、阶段归因、最可能原因和下一步验证。

一、**先定位阶段** 回归报告先比较阶段表。如果总时间变慢 20

二、**再比较代码和数据** 性能回归可能来自代码，也可能来自输入分布。key 基数增加、错误率提高、行长变长、输出 block 变多，都可能让同一代码变慢。报告要记录输入摘要。如果输入变了，回归可能是容量问题，而不是代码退化。

三、**最小复现** 构造最小输入和命令，让别人能复现回归。不要只贴 CI 链接。最小复现应包含 seed、规模、线程数、环境和预期差异。若无法稳定复现，先写波动范围，不要过早下结论。

四、**修复后验证** 修复性能回归后，不只看总时间恢复，还要看原先被怀疑的阶段指标是否恢复，正确性 checksum 是否不变，反证输入是否没有恶化。性能修复也是代码变更，需要测试矩阵。

5.12 从一次跑分走向可信性能结论

性能测量最危险的地方在于它很容易给人确定感。程序跑了三次，取一个平均值，看起来就像有了结论。但系统性能不是考试算术题，任何一个数字都依赖输入、机器状态、编译器、温度、频率、后台任务、页缓存、调度和测量工具本身。可信测量的目标不是追求一个漂亮数字，而是建立一条证据链：我比较的是什么，我排除了哪些干扰，我的结论适用于哪些边界，我还不能证明什么。

一、**先定义问题，不要先开工具** 测量前要先写问题。是要判断算法是否被内存带宽限制，还是要判断一次优化是否减少了分支错误，还是要判断多线程扩展为何停住？不同问题需要不同输入和指标。若问题是内存带宽，输入要大到超过缓存，指标要包含字节数、时间和带宽上限；若问题是分支，输入要包含可预测和不可预测分布；若问题是尾延迟，平均值没有代表性，必须看分位数。没有问题定义，工具输出越多，报告越乱。

二、**基线不是慢版本，而是语义锚点** reference 或 baseline 的价值不只是对比速度。它定义正确输出、边界行为、异常处理和可接受误差。优化版本必须先和 baseline 对齐，再谈速度。若优化改变了浮点合并顺序，报告要写误差规则；若优化改变了输出顺序，报告要写顺序是否属于语义；若优化跳过了错误检查，速度比较就不公平。高质量性能报告会把“快了多少”放在“是否仍是同一个问题”之后。

三、**Roofline 是定位语言，不是装饰图** Roofline 的核心是算术强度和机器上限。若一个 kernel 每读写大量字节只做少量计算，它大概率受带宽限制；若每个字节对应很多计算，它可能受计算吞吐限制。图上的点不需要做得华丽，但要能解释下一步：若离带宽屋顶很远，先看访问模式、预取、对齐和写分配；若接近带宽屋顶，继续微调指令可能收益有限；若离计算屋顶很远，先看依赖链、向量化、端口和分支。Roofline 不是替代分析，而是把分析约束在机器资源上。

四、**计数器要互相校验** 性能计数器会受权限、虚拟化、采样、复用和微架构定义影响。一个指标不能单独定罪。比如 cache miss 高，可能是随机访问，也可能是预取策略不同；分支 miss 高，可能是输入随机，也可能是间接调用；IPC 低，可能是内存等待，也可能是前端供给不足。报告应把计数器、反汇编、输入构造和时间分段放在一起看。若计数器不可用，就要写明降级方案，比如用 wall time、字节数、可控输入和二进制证据构造较弱但诚实的结论。

五、**结论必须带边界** 一个优化在桌面 CPU 上有效，不代表在手机、云主机或另一个编译器上有效。报告结尾要写清机器型号、核心数、频率策略、编译器版本、编译选项、数据规模、输入分布和运行次数。还要写未验证边界：是否测试了冷缓存，是否测试了多进程干扰，是否测试了 NUMA，是否测试了错误输入。好的测量报告不是把结论说死，而是让别人知道怎样复现、怎样质疑、怎样继续验证。

5.13 案例：一次看似成功的优化怎样被重新判读

假设日志解析器优化后从每秒一百万行提升到一百三十万行。这个数字看起来很好，但还不能直接写进结论。首先要问输入是否相同：行长分布、错误行比例、字段数量、字符集、是否命中页缓存。其次要问输出是否相同：错误行是否仍被记录，字段顺序是否一致，统计值是否可复现。最后要问环境是否相同：CPU 频率、后台任务、编译选项、内存分配器和文件系统缓存。测量章要训练的就是这种“不急着相信好消息”的能力。

一、把提升拆成阶段 若总耗时下降，可能是解析变快，也可能是读取变快、写出减少、错误路径被跳过、内存分配减少或输出缓存变大。报告应分段计时：read、parse、filter、partition、write、commit。若 parse 没变而 write 变快，优化标题就不能写“解析器优化”；若错误行处理被跳过，优化可能是错误语义退化。分段计时能阻止结论偷换。

二、用输入族验证稳定性 同一个优化要跑短行、长行、固定字段、缺失字段、错误行密集和随机字符。若只在短行上变快，在长行上变慢，说明优化依赖输入分布。高质量报告会画成表：每类输入的行数、平均长度、错误率、输出大小和主要瓶颈。这样读者能知道优化适合什么场景，而不是记住一个单点速度。

三、计数器解释变化方向 如果优化减少了分支错误，分支相关计数器应有对应变化；如果优化改善了局部性，cache miss 或带宽利用应有证据；如果优化提高了 SIMD 使用，反汇编或向量化报告应能看到批量指令。计数器不一定精确，但方向应和叙述一致。若时间变快而所有证据都解释不了，先怀疑测量方法、输入差异或系统噪声，而不是编造原因。

四、记录失败的实验 报告里应保留失败尝试。比如手写预取没有收益、手写展开导致指令缓存压力、改用无锁队列让尾延迟变差。这些失败不是丢脸材料，而是边界知识。经典教材好看的原因之一，是它让读者知道方法为什么成立，也知道何时不成立。本书也要保留反证过程，让读者学会判断，而不是只背最终答案。

五、把结论写成可执行建议 最终结论不要写成“优化有效”，而应写成带条件的建议：在输入行长较短、错误率低、输出写出不是瓶颈时，冷热路径分离能降低分支回滚；在长行和错误密集输入上，应优先优化错误记录和内存分配。这样的结论能指导后续工程。性能报告的最高质量，不是数字最大，而是别人读完后知道下一步该做什么。

5.14 贯穿案例：三倍加速报告怎样被拆成证据链

假设团队提交了一份报告：新解析器比旧解析器快三倍。用户看到这个数字当然高兴，但教材不能在这里鼓掌结束。我们要像审稿人一样把它拆开。第一问：新旧版本是否处理同一份输入，错误行、空字段、超长字段、编码异常是否一致。第二问：输出是否一致，checksum、错误记录、字段顺序、manifest 条目是否匹配。第三问：时间范围是否一致，是否把读取、解析、聚合、写出和提交都算进去。第四问：环境是否一致，页缓存、CPU 频率、后台任务、编译器参数是否变化。三倍加速只有通过这些问题后，才可能成为工程结论。

这个案例要训练的不是怀疑一切，而是给数字建立出处。性能报告的每个数字都应该能追到输入、代码版本、测量范围和模型解释。经典教材读起来舒服，是因为它们不会把图表当作结论，而会让读者知道图表为什么能支持某个判断。本章也要保持这种纪律。

一、先把语义对齐写在第一页 优化版若少处理错误行、跳过校验、改变字段裁剪规则，它当然会快。报告第一页应放正确性表：输入行数、成功解析行数、错误行数、输出 key 数、输出 checksum、错误日志 checksum、manifest generation。若这些项不一致，性能比较暂停。性能优化不是另一份程序，它必须仍然解决同一个问题。

二、再把时间切成阶段 总时间下降可能来自解析本身，也可能来自写出减少、错误日志少写、内存分配变化、文件已经在页缓存中。报告要拆成 read、parse、aggregate、write、commit。若 parse 只快百分之十，总时间快三倍，就说明另一个阶段被改变了；若 write 时间消失，可能是输出语义变了；若 read 时间大幅下降，可能是热缓存而不是算法。分段计时让结论不能偷换。

三、用字节账本约束吞吐 每处理一行，程序读多少输入字节，写多少输出字节，访问多少临时数组，是否写错误日志。若报告声称吞吐远超机器内存带宽，可能是字节数漏算、缓存状态不同或只统计了热循环不统计 I/O。Roofline 和字节账本不是为了画漂亮图，而是用于质疑不合理数字。一个可信报告会主动说明自己离带宽上限多远，离计算上限多远，下一步瓶颈更可能在哪里。

四、计数器只能和反例一起使用 性能计数器不是法官。branch miss 降低支持“分支预测改善”这个解释，但还要看输入是否改变；cache miss 降低支持“局部性改善”，但还要看工作集是否相同；IPC 提高可能来自前端更顺，也可能来自少做了工作。报告应准备反例输入：规则行、随机行、错误密集行、长行。若新解析器只在规则行快三倍，在错误密集行变慢，结论就必须带条件。

五、保留失败尝试让结论更可信 高质量报告会写哪些优化无效。比如手写预取没有收益，说明硬件预取已经足够或访问模式不适合；展开循环让短行变慢，说明代码体积或启动成本增加；无

锁队列提高吞吐却恶化 p99，说明平均值掩盖尾延迟。失败尝试不是废话，它们告诉读者边界在哪里。没有失败记录的报告很容易像广告，而不是工程材料。

六、把结论写成下一步决策 审查结束后，报告不写“新版本更好”，而写成可执行决策：在规则日志和低错误率场景中默认启用；错误密集输入保留旧状态机或进入慢路径；长行输入继续优化内存分配和写出；移动设备上需要重新测频率和温度影响；没有 PMU 权限时用分段时间和输入族替代计数器。这样的结论能指导项目，而不是只留下一个漂亮倍数。

5.15 案例深化：性能报告审稿清单

可信测量章节要训练读者像审稿人一样看自己的报告。假设报告说“新解析器快了三倍”。第一反应不应是接受或否定，而是追问：输入是什么，版本是什么，编译命令是什么，机器状态是什么，结果是否一致，样本有多少，瓶颈解释是否和计数器一致。没有这些材料，三倍只是一个孤立数字。

一、先审正确性 性能报告第一项是 reference 对比。字段数、错误数、输出 checksum、manifest 条目、边界输入都要一致。若优化版跳过了错误行、改变了浮点合并顺序、少处理了尾部，它当然可能更快。正确性没有过关时，任何性能数字都不进入讨论。这个顺序要在全书保持一致。

二、再审样本和环境 一次运行不能代表性能。报告应说明运行次数、预热策略、是否固定 CPU 频率、是否隔离后台任务、输入是否在页缓存中、编译器和参数是什么。移动设备或 Termux 环境还可能受温度、调度和省电影响，更要保留多次样本。若无法控制环境，就诚实写出限制，并用趋势而不是绝对数做判断。

三、用模型质疑数字 Roofline 和字节账本是用来质疑结果的。若程序声称达到远超内存带宽的吞吐，可能是字节数漏算或缓存命中被误当成内存读；若程序声称受算力限制，但 IPC 很低且 cache miss 高，模型就不支持结论。计数器和模型相互校验，任何一方都不能单独成为真相。

四、Profile 要能定位到决策 火焰图显示某函数耗时高，只说明采样集中在那里，不自动说明怎么改。要继续问这个函数里是分支、内存、锁、I/O 还是系统调用。若 profile 显示 parser 热，可能是分支预测，也可能是错误路径，也可能是分配。报告要把 profile 结果连接到下一步实验，而不是把图当作结论。

五、最后写未验证边界 高质量报告会说明哪些东西没有测。没有 PMU 权限时，branch miss 只能推测；没有清理页缓存时，冷读结论不可靠；输入只覆盖规则日志时，随机字段结论未知；单机测试通过，不代表分布式重试正确。未验证边界不是扣分项，它告诉读者结论能用到哪里，不能用到哪里。

5.16 案例复盘：三倍加速为什么不能直接相信

一份报告说解析器三倍加速，却没有输入摘要、编译命令、运行次数、页缓存状态和 checksum。这样的数字不能指导工程。测量章节要让读者像审稿人一样追问：结果是否正确，样本是否稳定，模型是否支持，计数器是否解释得通。

先审正确性，再审速度 字段数、错误数、输出 checksum、manifest 条目先一致，时间比较才有意义。优化版如果少处理错误行或改变尾部规则，它当然可能更快。随后检查样本分布、CPU 频率、绑定策略、编译器版本和输入是否热缓存。

模型用来质疑数字 Roofline 和字节账本不是装饰。若吞吐超过可能带宽，说明字节漏算或缓存状态不同；若声称受算力限制但 IPC 很低、cache miss 很高，就要回到 profile。可信测量的价值在于让性能结论可复查，而不是给最终答案盖章。

5.17 本章习题

习题与作业

习题一：为顺序归约、多累加器归约和线程本地 partial 归约设计一份实验矩阵。要求列出输入规模、线程数、运行次数、预热策略、计时范围、正确性校验和需要记录的计数器。每个变量都要说明用于排除哪一种错误解释。

习题二：写一段报告，解释为什么热原子归约即使使用 relaxed 内存序也可能扩展性很差。报告必须同时使用 C++ 语义、cache line 所有权和测量指标三个层面的语言。

习题三：设计一个队列 benchmark。要求区分 SPSC、MPSC 和 MPMC 三种竞争模式，记录吞吐、队列长度和等待次数。说明为什么只测单生产者单消费者不能证明一个 MPMC 队列适合线程池全局队列。

习题四：阅读当前系统的 `perfstat` 可用事件。记录哪些事件可用，哪些事件因权限或平台不可用。对不可用事件，写出替代实验方案，而不是直接放弃分析。

第 6 章

数据布局、局部性与预取

先看一个结构体改造事故：团队给日志记录对象加了十几个诊断字段，热路径仍然只读取 timestamp、level 和 key，吞吐却下降。原因不是诊断字段参与了计算，而是每条记录变大后，同一条 cache line 能装下的热字段变少，扫描时带入了大量冷数据，TLB 覆盖也变差。后来把热字段拆成连续数组，冷诊断放到旁路表，性能恢复，诊断能力也没有丢。

本章从这种“字段组织方式改变硬件访问”的问题出发。数据布局决定程序怎样使用缓存、TLB、预取器和内存带宽。很多高性能优化不是改变算法复杂度，而是把热字段、冷字段、读字段、写字段和并发槽位摆成硬件更容易消费的形状。理解布局，是理解数组、结构体、队列、图和矩阵内核的共同基础。

6.1 从访问模式到数据布局

Array of Structures 把一个对象的字段放在一起，适合经常同时访问完整对象的场景。Structure of Arrays 把同类字段连续存放，适合批量处理某个字段的场景。例如粒子模拟中，如果每轮只更新所有粒子的 x 坐标，SoA 更容易形成连续访问和 SIMD；如果每次处理一个粒子的全部属性，AoS 更直观。

选择布局不能只看代码好不好写，要看热路径读写哪些字段、是否需要向量化、是否有 false sharing、是否跨页跳转。工程中常见折中是 AoSoA，把数据按小块组织，既保留局部对象关系，又适合 SIMD 和缓存块。

Listing 6.1 AoS 与 SoA 的访问差异

```
1 struct Particle {  
2     float x;  
3     float y;  
4     float z;  
5 };  
6  
7 struct ParticleSoA {  
8     std::vector<float> x;  
9     std::vector<float> y;  
10    std::vector<float> z;  
11 };
```

如果热路径只更新所有粒子的 x，SoA 版本会连续读取和写入 x 数组；AoS 版本每次还把 y 和 z 附近的数据带入 cache line。若热路径总是同时使用 x、y、z，AoS 的空间局部性反而更自然。

布局选择要从访问矩阵出发，而不是从结构体是否“面向对象”出发。可以把字段列成行，把热路径列成行，在每个格子里标出读、写、只读配置、冷路径调试。若一个热路径只读少数字段，SoA 或冷热分离更有优势；若多个字段总是一起使用，AoS 或 AoSoA 更清晰；若需要 SIMD，字段连续性和对齐会更重要；若需要并发写，线程到字段和槽位的映射必须避免 false sharing。

```
hot path: update position  
reads: x, y, z, vx, vy, vz  
writes: x, y, z
```

```
hot path: compute bounding box  
reads: x, y, z
```

```
writes: thread local min and max

cold path: debug print
reads: name, id, material, history
```

这张访问矩阵会自然推出布局：调试字段不应混在高频数值字段里；只在统计阶段使用的字段不应污染每帧更新；线程本地输出应分开存放，最后合并。

预取 硬件预取器擅长识别顺序访问和固定步长访问。连续数组扫描通常能被预取器提前加载，随机指针追踪则很难。软件预取可以在少数场景有效，但如果预取太早会挤出有用缓存，太晚则来不及隐藏延迟。

最好的预取优化通常不是手写预取指令，而是改变访问模式：把随机访问排序，按块处理，压缩节点，减少指针跳转，让硬件预取器能看懂程序。

软件预取只有在地址能提前算出、未来确实会访问、当前有足够独立工作、预取距离合适时才可能有效。指针追踪中，如果下一步地址必须等当前节点加载完成，预取很难发挥作用；批量处理多个游标时，才有机会提前为另一个游标发起预取。预取错误还可能污染缓存，把真正热的数据挤出去。

因此本书把预取看成最后一步，而不是第一步。先让数据连续，先分块，先减少对象分散，先测出内存延迟或带宽瓶颈，再考虑预取指令。很多工程代码的性能来自“让硬件预取器自然工作”，而不是手写大量 prefetch。

冷热数据分离 对象里不是所有字段都同样热。把热字段和冷字段混在一起，会让缓存带入很少使用的数据。冷热分离把热路径频繁访问的字段放在紧凑结构中，把调试信息、统计字段、名字、配置等冷数据放到旁边结构或间接存储。数据库执行器、游戏引擎和网络服务都常使用这种思路。

冷热分离也有代价：代码复杂度增加，对象关系不如原来直观。是否值得做，要看热路径字段访问比例和缓存压力。设计数据结构时，先写清哪个路径是热路径，哪个字段在热路径必读或必写。

6.2 生命周期、分配器和容器形状

小对象频繁分配会带来分配器开销、碎片、TLB 压力和缓存局部性问题。高性能系统常用 arena、对象池、批量分配或结构化生命周期来减少热路径分配。并发分配器还要避免全局锁和跨线程释放导致的远端缓存流量。

常见误区

不要把“使用堆”简单等同于慢。真正的问题是生命周期是否清晰、分配是否在热路径、对象是否分散、是否跨线程释放、是否破坏局部性。

对象池和 arena 的核心收益不是“分配函数少调用几次”这么简单。它们把生命周期相近的对象放在相近内存中，减少元数据开销和碎片，提高遍历局部性，也让释放可以批量发生。并发环境下，还可以按线程或 NUMA 节点维护本地池，减少全局分配器锁和远端释放。

代价同样要写清楚。Arena 适合批量释放，不适合单个对象精确析构；对象池可能保留大量空闲内存；跨线程归还对象可能重新引入同步；自定义分配器如果没有测试，可能比通用分配器更难维护。高质量教材不能只说“用内存池更快”，必须说明适用的生命周期形状。

6.3 并发写、索引和图布局

多线程程序的数据布局还要考虑写共享。线程本地数据应尽量分开，跨线程只读数据可以共享，跨线程写数据要减少同一 cache line 上的冲突。并行归约、线程池队列和日志缓冲区都常常需要为每个线程准备独立槽位。

结构体大小和 ABI 结构体字段顺序影响 padding 和总大小。较大的结构体会降低缓存密度，也会增加拷贝成本。公共 ABI 中随意调整字段顺序可能破坏二进制兼容，因此高性能库常把内部热结构和外部稳定接口分开。内部结构按性能布局，外部接口保持稳定语义。

6.4 迁移、测试和报告

数据布局优化应按流程进行。第一步，确认热路径和访问矩阵。第二步，估算每轮实际需要读写的字节数。第三步，检查对象是否连续、是否跨页、是否存在共享写。第四步，设计一个只改变布局、不改变算法语义的对照版本。第五步，用相同输入和相同绑定策略测量时间、cache、TLB 和扩展曲线。第六步，评估代码复杂度是否值得。

很多布局优化失败，是因为同时改了太多东西：换容器、改算法、改线程数、改内存分配、改输入规模。这样即使变快，也不知道原因。教材实验必须强调单变量对照，让读者学会建立因果证据。

AoSoA：折中不是妥协 AoS 和 SoA 常被讲成二选一，但真实系统经常使用 AoSoA。它把数据按小块分组，每个块内部采用 SoA，块与块之间保持对象批次关系。这样可以让一个块适配 SIMD 宽度或缓存块，同时又不会把整个系统改造成完全分离的字段数组。粒子系统、物理模拟和向量数据库里都能看到类似结构。

Listing 6.2 AoSoA 的基本形态

```
1 constexpr std::int32_t PARTICLE_BLOCK_WIDTH = 8;
2
3 struct ParticleBlock {
4     std::array<float, PARTICLE_BLOCK_WIDTH> x;
5     std::array<float, PARTICLE_BLOCK_WIDTH> y;
6     std::array<float, PARTICLE_BLOCK_WIDTH> z;
7     std::array<float, PARTICLE_BLOCK_WIDTH> vx;
8     std::array<float, PARTICLE_BLOCK_WIDTH> vy;
9     std::array<float, PARTICLE_BLOCK_WIDTH> vz;
10 };
11
12 struct ParticleBlocks {
13     std::vector<ParticleBlock> blocks;
14     std::int32_t size = 0;
15 };
```

这里的块宽度不是魔法数字。它可能来自 SIMD lane 数、cache line 大小、对齐要求、尾部处理成本和代码复杂度。块太小，循环层次变多但复用不足；块太大，尾部浪费和缓存压力上升。AoSoA 的价值是提供一个调节旋钮，让布局能同时服务向量化和对象组织。

AoSoA 也有成本。随机访问第 *i* 个对象需要计算块号和块内偏移；插入删除比 `std::vector<Particle>` 更复杂；调试打印不如 AoS 直观；接口要隐藏布局细节，否则业务代码会到处写块索引。工程中通常把 AoSoA 封装在专用容器或视图后面，让热路径拿到高效访问，普通路径仍保持清晰 API。

访问矩阵如何落到代码 访问矩阵不是文档装饰，它应该直接影响类型设计。假设日志统计系统的每条记录有时间戳、用户 ID、事件类型、原始文本、解析状态和调试信息。热路径只需要用户 ID 和事件类型做聚合，原始文本只在错误报告时使用。如果把所有字段放在一个大结构里，聚合时每次都把冷字段附近的数据带入缓存。更好的设计是把热字段压成连续数组，把冷字段放在旁边，通过索引关联。

Listing 6.3 冷热分离的记录批次

```
1 struct HotLogBatch {
2     std::vector<std::uint64_t> user_ids;
3     std::vector<std::int32_t> event_types;
4 };
5
6 struct ColdLogBatch {
7     std::vector<std::string> raw_lines;
```



```

8  std::vector<std::int32_t> parse_status;
9  };
10
11 struct LogBatch {
12     HotLogBatch hot;
13     ColdLogBatch cold;
14 };

```

这个设计牺牲了“一个对象包含所有字段”的直观性，换来热路径更少的数据移动。它还让并发更容易：聚合 worker 可以只读 **hot**，错误处理线程再查看 **cold**。若业务经常需要完整记录，冷热分离可能不值得；若绝大多数请求只走热路径，它就很自然。布局设计必须从访问频率和访问组合出发。

索引优于指针的场景 指针直接表达对象关系，但在大规模数据结构中，索引常常更适合性能和序列化。索引可以是 `std::uint32_t` 或 `std::uint64_t`，指向连续数组中的位置。它比指针更紧凑，容易持久化和跨进程传递，也让对象搬迁和压缩更容易。图的邻接表、编译器 IR、游戏 ECS 和列式存储都大量使用索引。

索引也有代价。每次访问需要多一步数组寻址；删除对象后要处理空洞、代际版本或稳定 ID；越界和悬空索引需要调试检查。若系统需要稳定引用，可以使用“索引加 generation”的句柄，避免旧索引误指向新对象。这个思想和无锁结构里的 ABA 类似：同一个位置再次被复用时，需要版本信息区分“看起来一样”和“语义上还是同一个对象”。

Listing 6.4 索引句柄表达稳定引用

```

1 struct EntityId {
2     std::uint32_t index = 0;
3     std::uint32_t generation = 0;
4 };
5
6 struct EntitySlot {
7     std::uint32_t generation = 0;
8     bool alive = false;
9 };

```

这里使用明确位宽类型，是为了让内存布局和序列化边界清楚。系统教材中不能随手使用平台相关宽度的整数类型。布局设计和数据类型选择是一件事：类型宽度会影响结构体大小、cache 密度、文件格式和网络协议。

压缩、编码和计算成本 有时减少内存流量比减少计算更重要。列式数据库和搜索系统常把整数压缩成变长编码、差分编码或位图。这样每个元素读取的字节数减少，但解码需要额外 CPU。是否划算取决于瓶颈：若系统受内存带宽限制，压缩可能提高吞吐；若系统受计算限制，压缩可能变慢。

这个权衡也出现在分布式 shuffle 中。压缩中间数据可以减少网络 and 磁盘 I/O，但会增加 CPU 时间和延迟。单机 cache 层面的“少搬字节，多做一点计算”和分布式层面的“少发网络，多做一点压缩”是同一个思想。教材应让读者从底层内存开始建立这种成本交换模型。

并发写布局 并发数据布局要先标出写者。字段是否共享，不看它是否在同一个结构体里，而看哪些线程写它、写入频率如何、是否落在同一 cache line 上。一个常见错误是把多个线程的统计字段放进同一个 **Stats** 数组，觉得每个线程写自己的下标就安全。结果每个字段很小，多个线程的字段落在同一条 cache line，形成 false sharing。

更好的设计是把写热字段按写者分开，并且让每个写者独占 cache line 或至少独占写热点。读者汇总时再扫描这些槽位。若读取频率很高，可以维护分层汇总；若读取只是指标展示，可以接受延迟。并发布局本质上是在写入吞吐、读取成本和内存占用之间取舍。

Listing 6.5 线程本地统计槽位

```

1 struct alignas(64) WorkerStats {
2     std::uint64_t tasks_executed = 0;
3     std::uint64_t steal_attempts = 0;
4     std::uint64_t empty_polls = 0;
5 };
6
7 std::vector<WorkerStats> stats(static_cast<std::size_t>(worker_count));

```

这类结构适合 worker 高频写自己的统计。它不适合每个请求都读取全局精确值。如果业务需要实时限流，统计槽位还要配合周期性汇总或原子发布。布局优化总是要回到语义。

迁移旧代码的策略 把成熟代码从 AoS 改成 SoA 或冷热分离，风险很高。直接大改会破坏接口、测试和调试习惯。稳妥策略是分阶段迁移。第一阶段写访问矩阵和性能证据，证明布局确实是瓶颈。第二阶段增加新布局的内部表示，但保留旧接口，通过 adapter 暴露兼容访问。第三阶段把热路径迁到新布局，保留 reference 对照。第四阶段逐步清理旧路径。

迁移时要防止“双写不一致”。如果旧结构和新结构同时存在，更新必须保证二者一致，或者明确一个是权威数据，另一个是缓存。缓存需要失效策略；双写需要事务边界；延迟同步需要版本号。布局重构不是单纯换容器，而是数据所有权迁移。

标准容器的内存形状 数据布局不只发生在自己定义的结构体里，也发生在每一次选择标准容器时。`std::vector` 的元素连续存放，适合顺序扫描、批量复制、SIMD 读取和按索引访问。它的扩容会搬迁元素，因此外部保存元素指针或引用时必须小心。`std::deque` 不是一个大连续数组，而是由多个固定大小块组成，头尾插入更稳定，但长距离顺序扫描的局部性通常不如 `std::vector`。`std::list` 每个节点独立分配，插入删除不移动其他节点，但遍历时会产生指针追踪、cache miss、TLB miss 和分配器开销。很多初学者只记住复杂度表，却没有把复杂度和硬件成本合起来看。

例如一个任务运行时维护 ready task。如果任务总数在构图后基本稳定，`std::vector` 加索引队列或环形缓冲通常更适合。如果任务需要频繁从中间删除，但遍历又很热，链表也未必正确，因为删除的理论优势可能被遍历成本吞掉。若真的需要稳定句柄，可以用索引加 generation，而不是直接把每个任务做成堆上节点。容器选择不是 API 风格问题，而是访问模式、生命周期和硬件局部性的共同结果。

Listing 6.6 索引队列比链表更容易保持局部性

```

1 struct ReadyTask {
2     std::uint32_t task_id = 0;
3     std::uint32_t generation = 0;
4 };
5
6 class ReadyRing {
7 public:
8     explicit ReadyRing(std::uint32_t capacity)
9         : buffer_(capacity), head_(0), tail_(0), size_(0) {}
10
11     bool push(ReadyTask task) {
12         if (this->size_ == this->buffer_.size()) {
13             return false;
14         }
15         this->buffer_[this->tail_] = task;
16         this->tail_ = (this->tail_ + 1) % this->buffer_.size();
17         ++this->size_;
18         return true;
19     }
20

```

```

21 std::optional<ReadyTask> pop() {
22     if (this->size_ == 0) {
23         return std::nullopt;
24     }
25     const ReadyTask task = this->buffer_[this->head_];
26     this->head_ = (this->head_ + 1) % this->buffer_.size();
27     --this->size_;
28     return task;
29 }
30
31 private:
32     std::vector<ReadyTask> buffer_;
33     std::size_t head_;
34     std::size_t tail_;
35     std::size_t size_;
36 };

```

这个例子还不是并发队列，因为它没有锁和内存序。它要说明的是另一个层次：在考虑同步之前，先让数据本身连续、容量有界、访问路径可预测。后续给它加互斥或 SPSC 语义时，至少不会再被链式节点和频繁分配拖住。很多高性能结构的第一步不是“无锁”，而是“固定容量、连续存储、清楚所有权”。

分配器不是只影响分配速度 分配器常被理解成“申请内存快不快”，但它对布局的影响更深。通用分配器需要支持不同大小、不同生命周期、不同线程的分配和释放，因此会维护元数据、空闲链表、线程本地缓存和大块映射。小对象如果被零散分配，遍历时访问的是分配器历史留下的空间分布，而不是业务逻辑希望的顺序。一个图算法使用链式邻接节点，即使每个节点只有十几个字节，也可能因为节点分散而消耗大量 cache 和 TLB 资源。

Arena 的优势来自生命周期整齐。解析一个输入文件时产生的临时节点，可以在解析结束后一次性释放；构建一轮任务图时产生的任务对象，可以在图执行结束后批量销毁。这样做减少了单个对象释放的成本，也让相近生命周期的数据更容易靠近。缺点是对象不能随意提前释放，析构顺序要清楚，长期存活对象不能混进短生命周期 arena。否则 arena 会保留大量已经无用但不能回收的空间。

对象池则适合固定大小、频繁复用的对象，例如网络请求上下文、日志批次缓冲、任务描述符。对象池可以把对象初始化和内存申请分开，减少热路径分配；也可以按线程或 NUMA 节点分池，减少跨线程竞争。对象池的风险是复用对象时忘记清理旧状态，或者跨线程归还导致远端 cache line 流量。高质量实现要把 reset、owner thread、debug generation 和峰值容量都写进设计。

Listing 6.7 固定大小对象池的基本接口形状

```

1 struct RequestContext {
2     std::uint64_t request_id = 0;
3     std::uint32_t attempt = 0;
4     std::uint32_t worker_id = 0;
5 };
6
7 class RequestPool {
8 public:
9     explicit RequestPool(std::size_t capacity) : storage_(capacity) {
10         for (std::uint32_t index = 0;
11             index < static_cast<std::uint32_t>(storage_.size());
12             ++index) {
13             this->free_.push_back(index);
14         }

```



```

15 }
16
17 std::optional<std::uint32_t> acquire() {
18     if (this->free_.empty()) {
19         return std::nullopt;
20     }
21     const std::uint32_t index = this->free_.back();
22     this->free_.pop_back();
23     this->storage_[index] = RequestContext{};
24     return index;
25 }
26
27 void release(std::uint32_t index) {
28     this->storage_[index] = RequestContext{};
29     this->free_.push_back(index);
30 }
31
32 private:
33     std::vector<RequestContext> storage_;
34     std::vector<std::uint32_t> free_;
35 };

```

这个池子仍然只是单线程教学版本。并发版本必须决定 free list 由谁保护，是否每个 worker 一个本地池，跨线程释放是否进入 remote free 队列，是否要用 generation 防止旧句柄复用。不要因为代码短就忽略这些语义。内存池的工程难点不在“写一个 vector”，而在生命周期、并发所有权和调试可见性。

面向数据的接口设计 布局优化失败的常见原因，是内部换了 SoA，外部接口仍然要求每次返回一个完整对象。这样热路径又不得不临时拼对象，或者频繁跨数组取字段，收益被接口吃掉。面向数据的接口应该让调用者表达批量意图。例如不是提供 `get_particle(i)` 返回完整粒子，而是提供 `x_span()`、`velocity_span()` 或批处理函数，让热路径直接操作连续字段。

接口也要避免泄漏实现细节。完全暴露 `std::vector<float>&` 会让外部随意 `resize`，破坏布局不变量；完全隐藏成单元素 `getter` 又丢掉批量优化机会。折中做法是暴露只读或可写 `std::span`，并用类型说明生命周期和访问权限。只读输入、可写输出、临时 `scratch`、线程本地 `partial` 应该在函数签名中区分开。

Listing 6.8 批量接口比单元素接口更能表达热路径

```

1 struct PositionView {
2     std::span<float> x;
3     std::span<float> y;
4     std::span<float> z;
5 };
6
7 void integrate(PositionView position,
8     PositionView velocity,
9     float delta_time) {
10     const std::size_t count = position.x.size();
11     for (std::size_t i = 0; i < count; ++i) {
12         position.x[i] += velocity.x[i] * delta_time;
13         position.y[i] += velocity.y[i] * delta_time;
14         position.z[i] += velocity.z[i] * delta_time;
15     }

```

16 }

这个函数把布局意图表达给编译器和读者：三个坐标数组连续，循环边界统一，写入位置清楚。实际生产代码还要检查 span 长度一致、别名关系和对齐条件。接口设计的目标不是把所有优化都写进函数，而是不要一开始就把优化可能性封死。

ECS 和列式布局 Entity Component System 常见于游戏引擎，但它背后的思想适用于任何大量对象、少数字段热访问的系统。实体只是 ID，组件按类型连续存放，系统按组件批量处理。渲染系统关心位置和网格，物理系统关心位置和速度，AI 系统关心状态和目标。每个系统只扫描自己需要的组件，避免把完整对象的冷字段一起带入缓存。

日志分析和数据库执行器也使用类似思想。列式存储把同一列连续存放，过滤某个字段时只读取相关列；投影阶段再把需要的列组合输出。若查询只需要用户 ID 和事件类型，就不应该读取完整 JSON 文本。若后续需要原始文本，再通过行号或 offset 访问冷数据。列式布局的核心不是“数据库专属格式”，而是按访问组合移动最少字节。

列式布局还有压缩优势。同一列的数据类型相同、分布相近，更容易做字典编码、差分编码、位图和运行长度编码。压缩后减少内存和 I/O 流量，但增加解码成本。若过滤条件能直接在压缩表示上执行，收益很大；若每次都要完整解码，收益可能下降。布局、编码和算法必须一起设计。

CSR：图结构的局部性案例 图算法最容易暴露指针结构的问题。一个朴素邻接表如果每条边都是独立节点，遍历邻居时会不断指针跳转。Compressed Sparse Row 把所有边目标压到一个连续数组里，再用 offsets 数组标出每个顶点的邻居范围。这样遍历顶点 v 的邻居时，只需要扫描 `edges[offsets[v]]` 到 `edges[offsets[v+1]]` 之间的连续区间。

Listing 6.9 CSR 图的核心布局

```
1 struct CsrGraph {
2     std::vector<std::uint32_t> offsets;
3     std::vector<std::uint32_t> edges;
4 };
5
6 std::uint64_t degree_sum(const CsrGraph& graph) {
7     std::uint64_t total = 0;
8     for (std::size_t vertex = 0; vertex + 1 < graph.offsets.size(); ++vertex) {
9         const std::uint32_t begin = graph.offsets[vertex];
10        const std::uint32_t end = graph.offsets[vertex + 1];
11        total += static_cast<std::uint64_t>(end - begin);
12    }
13    return total;
14 }
```

CSR 的代价也要讲清楚。它适合静态图或批量更新后的图，不适合高频单边插入删除。插入一条边可能需要移动后续大量边，或者维护增量结构再周期性重建。它还把顶点 ID 映射和边排序变成预处理问题。很多系统会使用双层结构：主图用 CSR 保持扫描性能，增量更新放在小的补丁区，定期合并。这个设计和 LSM-tree、列式数据库的 delta store 很像：热更新和冷扫描分层处理。

块大小的选择方法 分块是布局优化的核心技巧，但块大小不能拍脑袋。选择块大小时至少要考虑五个因素。第一，块内工作集是否能放进目标 cache 层级。第二，块内循环是否能形成足够 SIMD 和指令级并行。第三，块大小是否造成尾部处理过多。第四，块是否方便按线程或 NUMA 节点分配。第五，块元数据是否过多。

以矩阵乘法为例，一个块需要同时容纳 A 的子块、B 的子块和 C 的子块。若三者合起来超过 L1 或 L2，块内复用就会下降。以日志聚合为例，一个批次需要容纳 key、value、hash、输出缓冲和临时计数。批次太小，函数调用和调度成本高；批次太大，cache 复用差，背压粒度粗。块大小应通过候选集实验选择，例如从 1 KiB、4 KiB、16 KiB、64 KiB、256 KiB 开始，观察吞吐、cache miss、TLB miss 和尾延迟。

块大小还影响错误恢复。分布式计算中，任务块越大，调度开销越低，但失败重做成本越高；块越小，负载均衡更好，但调度和元数据成本更高。单机 cache block、线程池 task grain、分布式 partition chunk 其实是同一个问题在不同尺度上的版本。

可测量的布局报告 布局优化的报告至少应包含四类数据。第一，结构体大小和对齐：使用 `sizeof`、`alignof`，列出字段顺序和 padding。第二，访问字节数估算：每处理一个元素理论上读取和写入多少字节，哪些字节是冷字段。第三，硬件指标：时间、带宽、cache miss、TLB miss、分支和 IPC。第四，语义影响：接口是否改变，生命周期是否改变，是否影响稳定引用和并发写入。

只有耗时数字的报告不够。假设 SoA 比 AoS 快了 1.8 倍，你还要说明原因是否来自少读冷字段、SIMD 更好、预取更稳定、TLB 覆盖更高，还是因为新版本顺便少做了别的工作。若无法解释，就应该补实验。可以构造一个只访问全部字段的场景，如果 AoS 反而更快，说明 SoA 的优势确实来自字段子集访问，而不是神秘加速。

下面是一份布局报告的文字模板，但写报告时要填真实数据。

```
layout report:
workload: aggregate event type from N records
baseline: vector<Record>, hot fields mixed with raw line
candidate: HotLogBatch plus ColdLogBatch
semantic change: external record id remains stable
bytes per hot record: baseline estimated 80, candidate 12
metrics: time, cache misses, dTLB misses, bandwidth
result: candidate improves hot aggregation, cold error path unchanged
```

布局优化的反例 布局优化也会失败。第一类失败是工作集很小，本来就全部在 cache 里，换布局只增加代码复杂度。第二类失败是热路径需要完整对象，SoA 反而让字段分散在多个数组中，增加地址计算和 cache line 数。第三类失败是业务频繁插入删除，CSR 或紧凑数组重建成本太高。第四类失败是并发访问模式改变，新布局把原来分散的写入集中到同一 cache line。第五类失败是接口没有同步迁移，内部布局高效，外部仍然单元素调用。

因此本章不是要求读者把所有对象都改成 SoA，而是要求读者学会问问题：热路径读哪些字段，写哪些字段，数据能否批量处理，生命周期是否成批，是否有稳定 ID 需求，是否多线程写，是否需要序列化，是否有恢复和调试要求。布局是系统设计，不是局部技巧。

日志系统的布局案例 贯穿项目的日志统计可以用一个具体布局案例串起来。原始日志行是冷数据，解析后的 event type、user id、timestamp 和 value 是热数据，解析错误信息是低频冷数据。若每条记录都保存成一个包含字符串和多个字段的大结构，聚合 event type 时会把大量原始文本和调试字段带入 cache。更好的设计是先把输入按 batch 解析，热字段放入列式数组，原始行或错误上下文按 offset 另存。

这种设计还方便错误处理。正常路径只扫描热字段；遇到解析错误时，把错误行号和原始 offset 写入冷错误列表；需要输出错误报告时再回到原始文本。热路径和冷路径分开后，性能和调试都更清楚。很多系统性能差，不是因为算法复杂，而是每次处理热字段都拖着一堆冷调试信息走。

Listing 6.10 日志批次的热冷布局

```
1 struct ParsedLogHotBatch {
2     std::vector<std::uint64_t> timestamps;
3     std::vector<std::uint64_t> user_ids;
4     std::vector<std::uint32_t> event_types;
5     std::vector<std::int64_t> values;
6 };
7
8 struct ParsedLogColdBatch {
9     std::vector<std::uint64_t> raw_offsets;
10    std::vector<std::uint32_t> error_rows;
11 };
```


这个布局不要求原始输入立即复制。`raw_offsets` 可以指向 `mmap` 文件或输入缓冲中的位置，但生命周期要清楚：如果输入缓冲释放，`offset` 就不能再用于错误报告。布局 and 生命周期永远绑定在一起。

ECS 变体：稀疏集合 ECS 常用 `sparse set` 维护实体到组件位置的映射。`Dense` 数组保存实际组件，`sparse` 数组从 `entity id` 映射到 `dense index`。遍历组件时扫描 `dense`，局部性好；查询某个 `entity` 是否有组件时，用 `sparse` 快速定位。删除时可以用 `swap-remove`，把最后一个元素移到删除位置，再更新 `sparse`。这个结构牺牲稳定顺序，换取紧凑遍历。

如果业务需要稳定顺序，`swap-remove` 就不合适；如果 `entity id` 非常稀疏，`sparse` 数组可能很大；如果组件跨线程频繁增删，需要同步。再次说明，数据结构没有脱离语义的绝对优劣。`Sparse set` 适合大量实体、组件遍历热、增删可接受重排的场景。

Columnar batch 和 vectorized execution 数据库的 `vectorized execution` 通常以 `batch` 为单位处理列式数据。一个 `batch` 可能包含几千行，每个算子处理一个 `batch` 后交给下一个算子。`Batch` 太小，函数调用和调度成本高；`batch` 太大，`cache` 压力和延迟增加。`Batch` 模型是 `row-at-a-time` 和全量材料化之间的折中。

日志统计项目可以借鉴这个模型。输入解析成 `batch`，`filter` 处理 `batch`，`histogram` 处理 `batch`，`shuffle` 按 `batch` 发送。每个 `batch` 有热字段列、选择向量、错误列表和统计摘要。这样项目既能展示数据布局，也能展示 I/O 背压：下游慢时，上游 `batch` 队列满，读取暂停。

布局和序列化的差异 内存布局和磁盘/网络格式不必相同。内存中为了计算使用 `SoA`，网络上为了兼容和压缩使用长度前缀或 `protobuf` 类格式，磁盘上为了恢复使用稳定 `manifest`。直接把内存结构 `dump` 到磁盘，短期方便，长期会被 `ABI`、`padding`、字节序和版本拖垮。高质量系统会显式转换格式。

转换本身有成本，因此需要批处理和缓冲。一次转换一条记录，函数调用和分配成本高；一次转换一个 `batch`，可以顺序读取列、批量编码、批量校验。布局优化和 I/O 章节在这里相遇：好的内存布局也要服务序列化。

缓存友好的哈希表 哈希表是系统里常见的性能热点。链地址法每个 `bucket` 指向链表节点，容易指针追踪；开放寻址把元素放在连续数组中，探测时更 `cache` 友好，但删除、扩容和高负载因子要小心。`Robin Hood hashing`、`SwissTable` 类结构通过控制探测长度和元数据字节改善局部性。教材不要求实现这些成熟结构，但要让读者知道 `std::unordered_map` 不是唯一选择。

日志聚合如果使用 `std::unordered_map<std::string, std::int64_t>`，每个 `key` 可能分配字符串和节点，局部性较差。若先把 `event type` `intern` 成 `std::uint32_t`，再用数组或开放寻址表，热路径会更清楚。数据布局优化常常从“把复杂 `key` 转成紧凑 `id`”开始。

内存布局的调试工具 布局优化需要工具。可以用 `sizeof` 和 `alignof` 检查结构体大小；用编译器警告或工具查看 `padding`；用地址打印检查数组连续性；用 `perfstat` 观察 `cache` 和 `TLB`；用 `heap profiler` 看分配热点；用 `sanitizer` 检查越界和 `use-after-free`。布局优化如果没有调试工具，很容易变成猜测。

项目可以增加一个 `layout report` 命令，打印关键结构体大小、对齐、字段数量、`batch` 容量和估算字节数。这个命令不直接提升性能，但能让读者看到数据结构的物理形状。系统教材应让抽象类型落到字节。

布局迁移的测试策略 从 `AoS` 迁移到 `SoA` 时，测试要覆盖读写一致性。可以保留旧 `AoS reference`，随机生成记录，分别填充 `AoS` 和 `SoA`，执行同一聚合，比较结果。还要测试边界：空 `batch`、单元素、非对齐长度、错误行、超长字符串、冷热字段缺失。迁移期间如果保留双表示，要测试更新一个字段后另一个表示是否同步或明确无效。

性能测试则要比较至少三条路径：完整记录访问、只访问热字段、错误报告访问。`SoA` 可能在热字段聚合上快，但完整记录访问变慢。报告要承认这种取舍，而不是只展示最好场景。

对齐分配器和实际收益 有些内核希望输入按 32 字节或 64 字节对齐。可以使用对齐分配器或平台 API 分配内存，也可以让容器内部保证对齐。对齐能减少跨 `cache line` 访问和满足某些 `SIMD` 要求，但它不是万能优化。若访问本来连续且 CPU 对非对齐访问支持良好，收益可能很小。若切片从对齐数组中间开始，起始地址仍可能不对齐。

项目中可以把对齐作为实验参数。生成同样数据，分别使用普通 vector、对齐缓冲、故意偏移一个元素的视图。比较自动向量化和手写 chunked 版本。报告中写清对齐如何保证、如何验证、对尾部 and 偏移如何处理。

Scratch buffer 管理 高性能算法常需要 scratch buffer，例如 scan 的 block sums、partition 的 counts、top-k 的候选、解析的临时位置列表。每次函数调用都分配 scratch，会引入分配器成本和内存波动。更好的做法是由运行时或调用者提供 scratch，或者每个 worker 复用本地 scratch。

Scratch 复用要清楚生命周期和大小。一个 worker 处理过大 batch 后，scratch 可能扩容到很大，后续小任务继续占用内存。可以设置最大保留容量，超过后释放或归还池。Scratch buffer 是数据布局和资源管理的交叉点。

Layout API 的封装 内部使用 SoA 或 AoSoA，不代表外部调用者要知道所有数组。可以提供 batch view，让热路径拿 span，冷路径通过 record id 获取完整记录。封装的目标是同时保护布局不变量和提供批量访问能力。单元素 getter 方便但可能慢；完全暴露 vector 容易破坏不变量。View 是折中。

Listing 6.11 批量视图封装布局

```
1 struct EventColumnsView {
2     std::span<const std::uint32_t> event_types;
3     std::span<const std::int64_t> values;
4 };
5
6 EventColumnsView event_columns(const ParsedLogHotBatch& batch) {
7     return EventColumnsView{batch.event_types, batch.values};
8 }
```

这个接口让聚合内核只看到自己需要的列，不依赖完整 batch 结构。后续如果 batch 从 vector 改成 mmap 或压缩列，接口仍有机会保持稳定。

布局章节总结 本章的主线是让数据形状服务访问形状。顺序热访问需要连续，字段子集热访问需要列式或冷热分离，高频并发写需要分片和对齐，稳定引用需要索引和 generation，静态图需要 CSR，动态更新需要增量结构。布局优化不是把所有东西改成一种格式，而是让每条热路径少搬无用字节、少追随机指针、少共享写。

设计布局时，先写访问矩阵，再写生命周期，再写并发写者，再写序列化边界，最后才写代码。这个顺序能避免“为了性能换结构，最后语义乱了”的常见错误。

布局决策表 选择布局时，可以先填表。字段是否总是一起访问？若是，AoS 可能清晰；若只访问少数字段，SoA 或冷热分离更好。对象是否需要稳定地址？若需要，考虑索引句柄或对象池；若不需要，连续数组更简单。是否高频插入删除？若是，CSR 这类紧凑静态结构可能不合适。是否要 SIMD？字段连续和对齐更重要。是否要序列化？内存布局和 wire format 要分开。是否多线程写？写者要分片或对齐。

这张表能避免布局争论变成风格争论。面向对象、面向数据、列式、池化都只是工具。真正决定工具的是访问矩阵、生命周期和并发语义。

案例：任务图的布局 任务图也需要布局。最直观表示是每个节点一个对象，每个对象有 `std::vector` successors。小图这样写清楚，大图会有很多小 vector 分配，遍历后继时指针跳转多。CSR 式任务图把节点和边分别放入连续数组，ready count 在节点数组中，successor id 在边数组中。执行时，worker 完成节点后顺序扫描后继范围，局部性更好。

缺点是动态修改困难。如果任务图在执行中不断新增边，CSR 重建成本高。可以使用两层结构：静态主图用 CSR，动态新增任务用小的 pending edge 列表，stage 结束后合并。这个设计和静态图加增量边、列式主存加 delta store 是同一模式。布局思维可以迁移到运行时元数据。

案例：模型权重的预告 虽然 AI 在第三册展开，但模型权重布局可以作为预告。权重通常是大块只读数据，推理时被反复扫描；量化权重可能按 block 存放 scale、zero point 和 int8 数据；算子希望连续读取并适配 SIMD。这个场景和本章的列式、AoSoA、对齐、冷热分离完全相关。第二册学习日志 batch 和矩阵 block，不是只为日志系统，而是为后续权重和 tensor 布局打基础。

这也说明布局知识具有迁移性。字段从 event type 换成 tensor value，原则不变：热数据连续，元数据分离，block 大小匹配 cache 和 SIMD，序列化格式与内存计算格式分开。

布局章的回归阅读应围绕一次真实迁移，而不是只列 AoS、SoA 名词。起点是 **LogRecord** 对象数组：字段完整、接口直观，但热字段和冷字符串混在一起。第一步统计哪些字段进入热循环，哪些只用于错误报告；第二步把热字段迁移到列式 batch；第三步保留稳定 row id，让错误路径仍能回到原记录；第四步用测试保证迁移前后语义相同。

布局迁移最容易犯的错，是只看快路径而破坏生命周期。字符串视图是否指向仍然有效的 arena，列式数组扩容后 span 是否悬垂，冷热分离后错误报告还能否定位原始行，CSR 图结构的 offset 是否覆盖最后一个顶点，这些都属于布局语义。数据结构不是只为 cache 服务，也要为所有权、错误路径和测试服务。

本章验收应要求一份迁移说明。说明中写清旧结构、热字段、冷字段、新结构、兼容层、回滚方式和测量结果。若性能提升来自减少无用字节搬运，就估算减少了多少；若没有提升，也要说明瓶颈是否已转向分支、I/O 或同步。这样布局优化就不会变成凭感觉重排字段。

预取讨论也要保持克制。硬件预取器擅长连续和固定步长访问，不擅长依赖上一次 load 的链式结构；软件预取可能隐藏部分延迟，也可能污染 cache、增加指令压力或预取错误路径。讲预取时应先证明地址能提前知道，再证明被预取的数据会被使用，最后测量预取距离。没有这三步，预取只是把不确定性提前。

本章还应把索引和句柄讲清。列式 batch 常用整数索引连接热字段和原始记录，索引位宽要由最大 batch 大小决定，不能随意使用模糊整数。句柄比裸指针更适合跨阶段传递，因为它能在 arena 迁移、vector 扩容和错误回溯时保持稳定。布局优化的成熟标志，是热路径拿到连续数据，冷路径仍能可靠定位上下文。

布局迁移还要考虑写路径。读多写少的数据适合冷热分离和列式压缩；频繁写入的数据若被拆成太多列，可能增加写放大和一致性成本；多个线程写相邻字段时，还要考虑 false sharing。比如统计 partial 的桶数组适合按线程或按 NUMA 节点分开，而不是所有线程写同一个紧凑数组。数据布局不是只为读取服务，写入模式同样决定结构。

内存分配策略也要进入本章。大量小对象如果逐个 new，会带来分配器锁、元数据开销、碎片和随机地址；arena 可以把生命周期相同的对象放在一起，但会让单个对象释放变难；对象池可以复用内存，但要处理并发和悬垂引用。项目中解析阶段的临时字符串、错误记录和 batch 元数据都适合讨论这些取舍。

6.5 让数据结构服从访问模式

数据布局不是把 AoS、SoA、CSR、Arena 几个词背下来，而是先问访问模式。每条记录会读哪些字段，写哪些字段，字段是否一起出现，访问顺序是否连续，是否跨线程共享，生命周期是否一致，是否需要稳定引用。答案不同，布局就不同。若不先写访问矩阵，任何布局建议都像偏好。

访问矩阵可以很朴素：行是阶段，列是字段，格子写读、写、只在错误路径读或不访问。日志解析阶段需要原始文本、offset 和错误信息；统计阶段只需要 event type 和少量数值；输出阶段需要 key 和计数；调试阶段可能需要原始行。矩阵一画，冷热分离就不是术语，而是避免热阶段搬运冷字段的直接结果。

AoS、SoA 和 AoSoA 的选择

AoS 适合每次处理完整对象，代码直观，生命周期统一。SoA 适合批量处理少数字段，便于 SIMD、预取和列式扫描。AoSoA 在二者之间折中，把固定大小块内的字段列式化，既保留一部分对象局部性，又让热循环按列访问。选择不是凭风格，而是看一轮热循环到底读哪些字节。

例如日志 batch 若每次都要解析原始字符串并立刻生成输出，AoS 足够简单；若解析和统计分成两个阶段，统计阶段只看 event type、user bucket 和 timestamp bucket，SoA 更合适；若后续 SIMD 按 16 或 32 条记录处理，AoSoA 可以让一个块大小贴近向量宽度和 cache。章节要给出失败反例：若把所有内容都 SoA 化，但后续错误报告频繁需要回到原始记录，索引和跨列访问也会增加成本。

C++ 实现要把布局封装成批量视图，而不是让业务代码到处知道内部数组名字。接口可以提供 `std::span<const std::uint32_t>` 之类的热字段视图，避免拷贝，也让编译器看见连续范围。单元素 getter 在控制面可以接受，在热数据面会隐藏批量访问机会。布局优化最终会落到 API 设计，这一点比单纯换容器更重要。

索引、句柄和生命周期

指针表达“某个地址”，索引表达“某个容器中的位置或 id”。高性能系统常用索引句柄，因为对象可能存在 compact vector、arena 或 slab 中，索引更容易序列化、移动和检查代际。裸指针访问快，但生命周期一旦跨队列或异步边界，就容易悬垂。句柄可以带 generation，检测旧引用，代价是每次访问要多一次间接和检查。

对象池解决的是生命周期和分配形状，不是所有问题的答案。固定大小对象池适合大量同类型、同生命周期对象，例如 task node、queue slot、message header。若对象大小差异大、生命周期不规则，池可能造成内部碎片或延迟释放。Arena 适合整批释放，例如一个 stage 的临时节点；不适合需要单独回收的长期对象。教材要让读者把 allocator 看成生命周期设计的一部分。

索引还改善序列化和分布式边界。一个 task id、split id 或 batch id 可以写进 trace 和 manifest；一个进程内指针不能跨进程。第二册虽然主要讲单机计算，但后续分布式章节会把这些 id 继续使用。数据结构如果一开始就把稳定身份和内存地址分开，后面扩展到重试和持久化会顺很多。

预取、压缩和分块

预取器喜欢可预测地址。连续扫描和固定步长访问容易被硬件预取，随机哈希和指针链表难以预取。软件预取只在能提前知道未来地址、且当前有足够工作覆盖延迟时有意义；如果预取距离不合适，可能浪费带宽或污染缓存。讲预取时要从地址序列推导，不要把 `__builtin_prefetch` 当成固定优化。

压缩是用计算换带宽。小整数 key 可以用更窄类型保存，位图可以压缩布尔标记，字典编码可以把字符串变成整数 id。压缩后数据更小，cache 和 TLB 压力下降，但解码会增加指令和分支。若热路径频繁访问压缩字段，解码成本可能抵消收益；若压缩字段只用于 I/O 或跨网络传输，收益更明显。数据布局章要把压缩放进成本账本，而不是当作存储技巧。

分块连接缓存、SIMD 和任务粒度。块太小，调度和边界成本高；块太大，超过 cache 或造成负载不均。AoSoA 的块大小、任务 chunk size、I/O batch size 和 SIMD width 不是彼此无关的参数。贯穿项目可以先固定一个 batch size，再在测量章模板下扫描：记录 cache miss、吞吐、队列长度和尾延迟，说明为什么选择当前值。

布局迁移和测试策略

布局优化最容易破坏接口。把对象数组改成列式数组后，原先返回引用的 API 可能失效；原先指针稳定的容器可能因为 reallocation 失效；原先按插入顺序遍历的输出可能因为分区而改变顺序。迁移时先加 reference 和适配层，再逐步把热路径改到新布局。不要一次把语义、布局、并发和输出格式全改掉。

测试要覆盖内存安全和性能边界。功能测试比较输出；地址稳定性测试检查句柄 generation；压力测试覆盖大量插入删除；并发测试检查跨线程传递生命周期；性能测试比较字节移动和 cache 行为。若项目使用 sanitizers，要说明哪些路径能覆盖，哪些性能实验不能开 sanitizer 因为开销太大。

本章的项目交付物是 `ParsedLogBatch`、`TaskArena` 和 `StableHandle` 三类设计说明。每个说明都写访问模式、生命周期、所有权、失效规则和对照实验。这样后续 SIMD、任务图和分布式提交使用这些结构时，不需要重新猜它们为什么长这样。

案例走查：从对象数组迁移到列式 batch

迁移第一步不是改所有代码，而是增加一个适配层。保留原始 `LogRecord` reference，新增 `ParsedLogBatch`，由 parser 填充热字段数组和冷字段索引。统计内核只接收热字段 span，错误报告仍可通过索引回到原始文本。这样语义边界清楚，热路径也能变窄。

第二步写一致性测试。随机生成记录，分别走对象数组和列式 batch，比较最终计数、错误列表和稳定排序输出。再写边界测试：空 batch、单条记录、超长字段、错误行、重复 key。只有这些通过后，才做性能测量。否则布局迁移带来的错误会被误认为性能优化。

第三步测字节移动。报告不只写耗时，还写每条记录热字段字节数、冷字段字节数、batch 容量、内存峰值和 cache 行为。若列式版本在小输入上变慢，也要解释：可能是额外索引和初始化成本超过了缓存收益。这样的反例能帮助读者理解布局优化的边界。

布局 API 的设计作业

让读者为 `ParsedLogBatch` 设计 API：必须能追加记录、获得热字段 span、根据错误索引找到原始行、清空并复用内存、报告当前容量和字节数。禁止热路径返回可变内部 vector 引用，避免上层破坏不变量；允许控制面查询统计信息。这个作业能把布局和封装联系起来。

API 还要定义失效规则。追加记录是否会导致 span 失效？batch move 后句柄是否有效？clear 后旧索引是否可用？这些问题看似 C++ 容器细节，实际会影响异步任务和队列传递。若一个 span 被提交给 worker 后，上游继续追加导致 reallocation，worker 就可能读悬垂内存。所有权合同必须写清。

布局 API 的性能测试要检查是否发生意外拷贝。函数参数尽量用 `std::span` 和 `std::string_view`；大对象只读传 `const` 引用；返回热字段视图不复制。报告可以用分配计数或简单日志确认热循环没有每条记录分配。这样读者能把 C++ API 设计和缓存局部性连起来。

布局迁移的源码阅读

可以阅读成熟列式系统或数据库执行引擎的公开资料，但本书本地代码重点是自己实现小型 batch。Linux 源码侧则看内存分配和 page fault 的用户态表现，理解为什么大量小对象会造成分配器和页层面的压力。读 `/proc/self/status` 的 `VmRSS`、`/proc/self/smaps` 的匿名映射，也能帮助观察布局迁移后的内存峰值。

在 C++ 标准库层面，读 `std::vector` 的失效规则比读某个实现源码更重要。vector reallocation 会让指针、引用和迭代器失效；deque 分段存储，随机访问多一次间接；list 节点稳定但局部性差；unordered map rehash 会失效迭代器且分配多。数据布局章要把这些容器规则和硬件成本一起讲。

项目验收要求写“为什么不用链表”。不是因为链表低级，而是因为本项目热路径需要连续扫描、批量处理和稳定索引。能为不用某个结构给出理由，比只会说“vector 快”更接近工程判断。

综合练习：改造一份日志记录结构

给读者一个故意臃肿的 `LogRecord`：时间戳、用户字符串、事件字符串、原始行、解析错误、调试标签、计数字段全部放在一起。要求先画访问矩阵，而不是直接改结构。Parser 读原始行并写字段；统计阶段只读事件 id 和用户桶；错误报告读原始行和错误；输出阶段读聚合结果。根据矩阵提出热冷分离方案。

第二步实现迁移计划。先保留原结构作为 reference，再新增 `ParsedLogBatch`。Parser 同时填充两种结构，测试输出一致；然后统计内核改用 batch 热字段；最后删除热路径对旧结构的依赖。每一步都要能独立提交和回滚。若一次性删除旧结构，错误很难定位。

第三步写失效规则。Batch 追加是否使 span 失效，任务入队后 batch 是否可继续写，错误索引在 clear 后是否可用，跨线程传递是移动所有权还是共享只读。读者必须把这些写进接口注释或设计文档。布局优化如果不写生命周期，会变成悬垂引用来源。

验收包括功能、内存和性能三类。功能比较 reference，内存记录峰值和分配次数，性能记录热循环字节数和耗时。若性能提升不明显，也要解释是否输入太小、冷字段本来就少、编译器已经优化、或瓶颈不在内存。这个题训练的是完整迁移，而不是单点改结构。

容易出错的边界：数据布局不该破坏语义

第一个反例是为了列式布局丢失错误上下文。热路径只需要 event id，但错误报告需要原始行、列号和字段名。如果拆分后无法回到原始文本，系统可诊断性下降。正确设计是热字段和冷索引分离，冷字段不进热循环，但仍能按 id 找回。第二个反例是为了对象池复用 buffer，导致下游异步任务读到被覆盖数据。生命周期不清的布局优化，比慢代码更危险。

第三个反例是为了压缩字段引入溢出或错误解码。把 user id 从 `std::uint64_t` 压成 `std::uint32_t`，必须证明输入范围；把字符串字典化，必须处理未知 key 和版本；把布尔标记压成位图，必须处理并发写和位操作原子性。压缩是语义选择，不是单纯内存优化。

第四个反例是过度泛化布局 API。为了支持所有访问模式，接口提供任意字段组合和动态反射，结果热路径无法内联，数据访问重新变成间接。高性能系统常常保留窄而明确的热路径 API，同时在控制面提供通用查询。把这一区分写清楚，读者才能理解为什么经典系统代码经常有专用内核。

本章源码索引可以让读者阅读自己项目中的所有 `span`、`string_view` 和指针捕获位置，检查是否跨异步边界。每个跨边界的 view 都要回答上游对象是否仍然活着。这个检查比单纯看 cache miss 更能防止实际工程 bug。

本章核心接口：BatchView

布局章给项目留下 `BatchView`。它只暴露热字段的只读 span、记录数量、batch id 和错误索引查询，不暴露内部 vector 的可变引用。写入由 builder 完成，读取由 view 完成。这样上游可以构造 batch，下游只能按合同读取，避免热路径外部破坏布局。

BatchView 的生命周期写进类型文档：view 不拥有数据，不能跨过 owning batch 的生命周期；提交给异步任务时，要移动 owning batch 或使用共享只读所有权。这个接口会直接保护后续 SIMD、线程池和 I/O 章节的安全。

从数据布局进入 SIMD 和批量执行

数据布局把热字段整理成连续、可解释、生命周期清楚的批量视图。只有到了这一步，SIMD 和自动向量化才有稳定基础。若数据仍然散在对象、链表和字符串里，向量化只能面对 gather、分支和间接访问；若 BatchView 提供连续 span，编译器和手写内核才有机会把多个元素放进同一条批量语义。

项目应把 SIMD 章看作布局章的验收。一个布局如果不能支持批量扫描、尾部处理和错误索引，它就没有真正服务热路径。反过来，SIMD 失败也可能提示布局还不够清楚：字段不连续、别名不明确、输出位置不稳定或错误路径混入热循环。两章之间应形成反馈，而不是前后孤立。

6.6 项目落地

Compute Systems Engine 可以在本章加入两个实验模块。第一个是记录批处理模块：用 AoS、SoA 和冷热分离三种布局统计 key，比较热字段扫描、错误记录访问和内存占用。第二个是 worker stats 模块：紧凑统计槽位与 `alignas(64)` 槽位对照，观察多线程写入扩展性。这两个模块分别训练“读局部性”和“写共享”两种布局能力。

项目代码不必一口气实现复杂 AoSoA 容器，但要把布局实验和 benchmark harness 接起来。每个布局版本必须使用同一份输入、同一个 reference 结果和同一组线程策略。否则布局差异会和算法差异混在一起。

从业务记录推导列式批次 很多布局优化失败，是因为一开始就讨论“要不要 SoA”，却没有先讨论业务记录到底怎样被使用。正确推导应从一个具体记录开始。假设日志记录包含时间戳、用户编号、事件类型、数值、原始行、解析错误、调试标签和来源文件。解析阶段需要原始行、来源文件和错误字段；聚合阶段只需要用户编号、事件类型和数值；错误报告阶段需要原始行和错误字段；调试查询阶段才需要标签。把这些字段全部放在一个大对象里，聚合阶段每处理一条记录都会把大量冷字段一起带进缓存。聚合代码看起来只读三个字段，硬件实际搬动的字节却远多于三个字段。

推导列式批次时，可以先保留最直观的 AoS reference。它的价值是语义清晰，便于测试新布局。然后为热路径建立列式批次：用户编号是一列，事件类型是一列，数值是一列，解析状态可以是一列，原始行和调试信息放入冷批次。热聚合只接收热批次视图，错误报告才接收冷批次视图。这样做并不是为了追求某个抽象名称，而是为了让热路径少搬无用字节，让编译器更容易生成顺序扫描，让硬件预取器更容易工作。

迁移时要特别注意记录编号。列式批次里第 i 个用户编号、第 i 个事件类型、第 i 个数值和冷批次第 i 个原始行必须表示同一条逻辑记录。这个关系可以通过统一长度、统一追加、统一过滤掩码和稳定 record id 保持。如果某个阶段删除错误记录，不能只从一列删除而忘记其他列。更稳的做法是先生成 selection vector，表示哪些记录有效，后续聚合按 selection vector 读取；等到批次稳定后，再做 compact。这样能避免列之间长度和顺序不一致。

列式批次还要考虑异常和边界。解析一行失败时，热列是否填默认值？错误列如何记录原因？聚合阶段是否跳过失败记录？如果错误记录很多，selection vector 会不会成为新的热点？如果原始行很长，冷批次是否保存完整字符串，还是保存文件偏移和长度？这些问题都属于布局设计，而不是外围细节。真正的工程布局必须同时说明成功路径、失败路径、调试路径和序列化路径。

稀疏结构和图布局 图和稀疏矩阵是布局思想最集中的场景。最直观的图表示是每个节点一个对象，每个对象保存一个后继列表。这种写法适合教学和小图，但在大图上会产生大量小分配。遍历一个节点的邻居时，程序先读节点对象，再读 vector 元数据，再跳到邻居数组；如果每个邻居数组都来自不同分配位置，cache 和 TLB 压力会很高。CSR 表示把所有边放到一条连续边数组里，再用 offset 数组记录每个节点的边范围。遍历节点 v 的邻居时，只需要顺序扫描 edges 从 `offsets[v]` 到 `offsets[v + 1]` 的区间。

CSR 的核心收益是把指针追踪变成连续区间扫描。连续扫描让硬件预取器能提前加载，也让多线程分片更容易。若节点范围按编号切分，每个 worker 扫描一段节点和对应边，读路径相对稳定。代价是动态更新困难。插入一条边可能需要移动后续大量边，删除也可能留下空洞。因此 CSR 更适合静态图、阶段性重建图或批处理图。动态图可以使用主 CSR 加 delta 边列表：主图负责绝大多数

只读遍历，新增边暂存在小结构中，阶段边界再合并。

稀疏矩阵也类似。按行压缩适合行遍历和矩阵向量乘；按列压缩适合列遍历；块稀疏适合局部密集区域和 SIMD。选择格式时，不能只说“稀疏矩阵用 CSR”。要看内核按行访问还是按列访问，是否需要转置，是否需要并行写输出，非零分布是否倾斜，行长度差异是否很大。行长度极不均匀时，按行分配线程可能负载不均；可以按非零数量切分，也可以把长行拆成多个任务，但拆长行又会引入输出累加同步。布局、算法和并行调度在这里不能分开。

Compute Systems Engine 后续做 shuffle、join 和任务图时，都可以借用稀疏结构思维。任务图的 successors 数组就是边数组，任务的 successor range 就是 offset。分区后的 key 到记录位置，也可以看成倒排索引或稀疏结构。读者如果只把 CSR 当成图算法专用格式，就错过了更重要的思想：当大量对象之间的关系可以静态收集时，把关系压成连续区间，通常比每个对象挂一串指针更适合现代硬件。

布局实验报告怎样写 布局实验报告必须说明输入规模、记录大小、字段访问比例、线程策略、内存分配方式和测量指标。只报告“快了两倍”没有意义，因为读者不知道快在哪里。一个合格报告至少要写三组结果：完整对象访问、热字段访问、冷路径访问。SoA 可能让热字段聚合更快，但完整对象调试变慢；冷热分离可能减少缓存流量，但增加索引和封装复杂度。报告要把收益和代价都列出来。

指标上，时间只是起点。还应观察 cache miss、TLB miss、内存带宽、分配次数、峰值内存、输出是否一致。多线程布局实验还要看扩展曲线和 false sharing。若紧凑统计槽位在单线程下更快，对齐槽位在多线程下更快，这不是矛盾，而是写共享形状改变了瓶颈。单线程结论不能直接推到多线程，多线程结论也不能忽略内存占用。

报告还要防止把算法变化伪装成布局变化。AoS 版本如果使用普通循环，SoA 版本同时换成 SIMD 和分块，就无法判断收益来自哪里。更严谨的方式是逐步改变：先只改布局，算法保持一致；再加分块；再加 SIMD；再加线程。每一步都有 reference 结果和性能指标。这样得到的不是一组漂亮数字，而是一条可复查的因果链。

访问模式 布局设计从访问模式开始。若每次循环只读 key 和 value，就不应让 debug 字符串、错误码和时间戳陪着进入 cache。先列字段访问频率，再决定哪些字段同列、哪些字段拆出。

AoS 与 SoA AoS 适合整体对象一起处理，SoA 适合批量扫描同一字段。中间还有 AoSoA、分块列式、索引表等形式。教材要让读者根据访问和更新选择，而不是把 SoA 当成万能答案。

生命周期 布局也受生命周期影响。短生命周期对象适合 arena 或 batch buffer，长期对象需要稳定句柄，跨线程对象要明确所有权。只谈 cache，不谈释放和迁移，会把内存安全问题留到后面爆炸。

预取 硬件预取器喜欢稳定步长。手写 prefetch 只有在访问提前量、计算量和内存延迟匹配时才有意义。随机访问、分支复杂和小工作集下，prefetch 可能没有收益甚至污染 cache。

索引替代指针 指针链可读性强，但地址分散。索引能让数据保存在连续数组中，也便于序列化、移动和压缩。代价是需要稳定表和边界检查。教学项目优先用索引表达记录关系。

分配器 大量小对象会把时间花在分配和释放上，也会破坏局部性。arena、pool 和 monotonic buffer 可以减少分配成本，但会改变释放粒度。选择分配器必须和生命周期一起写。

迁移成本 把旧结构迁移到新布局本身要花费时间和内存。报告要记录转换成本，并说明它是否能被批处理摊销。如果每次查询都临时转 SoA，收益可能被转换抵消。

接口保护 LayoutPlan 应让调用者看到 span、列、索引和版本，而不是暴露内部 vector 让外部随意 push。局部性需要接口约束，否则一次方便调用就能破坏整个布局。

案例：记录表列式化 把 Recordid,key,value,debug 改成 key/value 列和 debug 辅助表。比较只扫描 key/value 的热路径，也记录构建列式 batch 的成本。

案例：arena 生命周期 为一次 batch 分配临时节点，用 arena 一次释放。报告说明为什么不适合长期持有的对象。

案例：索引图 用索引数组表示边，而不是每条边一个 new 节点。观察遍历、序列化和缓存行为的变化。

源码阅读路线 Linux 源码阅读从 mm/page_alloc.c、mm/slab.c、include/linux/mm_types.h 附近进入即可。阅读目标不是把整套子系统抄进书里，而是追一条和本章实验相关的路径：

用户态动作怎样变成内核对象，内核对象怎样排队、等待、唤醒、计数、提交或回收。

阅读报告要写证据等级。能直接测到的数字放在第一层；从源码控制流推导出的解释放在第二层；当前设备无法验证、只能作为合理猜测的内容放在第三层。这样读者会知道布局迁移报告中哪些结论可复现，哪些结论需要换机器继续确认。

源码阅读还要克制。看到一个复杂函数，不要把每个分支都展开；先回到本章的问题，问它能否解释一个等待点、一次状态转移或一条性能曲线。解释不了的内容可以留到索引，不要打断正文主线。

记录表列式化 把 Recordid,key,value,debug 改成 key/value 列和 debug 辅助表。比较只扫描 key/value 的热路径，也记录构建列式 batch 的成本。

arena 生命周期 为一次 batch 分配临时节点，用 arena 一次释放。报告说明为什么不适合长期持有的对象。

索引图 用索引数组表示边，而不是每条边一个 new 节点。观察遍历、序列化和缓存行为的变化。

访问模式

数据结构应从访问模式推出，而不是从习惯推出。顺序扫描、随机查询、按 key 聚合和图遍历需要不同布局。

实验设计：先写访问序列，再选择 vector、deque、哈希表、CSR 或列式批次。

AoS 与 SoA

AoS 适合整对象处理，SoA 适合批量处理单字段。冷热字段混在一起时，扫描热字段会搬运大量无用字节。

实验设计：把日志记录拆成热字段列和冷字段表，比较扫描字节。

容器语义

vector 连续但扩容会移动，deque 分块但长扫描局部性差些，list 插删稳定但遍历成本高。复杂度表必须和硬件成本合看。

实验设计：为同一任务分别用三种容器实现，说明为什么选择最终版本。

生命周期

视图不拥有数据，指针不说明寿命，索引句柄需要 generation 防止复用误判。布局迁移必须同时迁移所有权规则。

实验设计：构造上游释放缓冲、下游仍持有视图的错误案例，再用批次对象修正。

分配器

大量小对象分配会造成元数据成本、碎片和 TLB 压力。对象池能降低成本，但要处理归还、清理和线程归属。

实验设计：比较逐条 new、批量 arena 和固定对象池的分配次数。

预取

硬件预取喜欢规律访问，软件预取只在确实知道未来地址且有足够距离时才有意义。乱加预取可能增加带宽压力。

实验设计：对连续数组、固定步长和指针链分别尝试预取，记录收益边界。

图布局

图算法常受邻接访问和度数倾斜影响。CSR 把邻接表压成连续数组，可以减少节点分配和指针追踪。

实验设计：把邻接 vector 列表转换成 CSR，比较 BFS 的访问形状。

并发写布局

多个线程写相邻统计槽可能 false sharing。线程本地槽位、按 cache line 对齐和分层合并都是布局选择。

实验设计：让每线程写自己的 aligned slot，再与共享桶对照。

迁移计划

布局修改不能一次重写全部。先保留旧接口，新增批量视图和转换层，再逐步让热路径直接消费新布局。

实验设计：报告旧路径、新路径和兼容层成本，避免迁移中破坏语义。

项目对象

LayoutPlan 应说明字段冷热、存储容器、所有权、视图寿命、线程写入位置和迁移步骤。它是性能和接口设计的共同文档。

实验设计：每个布局优化都要更新 LayoutPlan，而不是只改结构体定义。

6.7 数据结构不是容器选择题，而是访问模式的结果

很多读者学数据结构时只记复杂度表：`std::vector` 随机访问 $O(1)$ ，`std::list` 插删 $O(1)$ ，哈希表平均 $O(1)$ 。第二册必须把这张表放回机器里。连续数组的 $O(1)$ 和链表节点的 $O(1)$ ，对 cache、TLB、分配器和预取器完全不同。布局章节的主线不是“哪个容器最好”，而是“访问模式要求数据怎样排列”。

以日志记录为例，原始输入是一行文本，解析后可能包含时间、用户、事件类型、错误文本和调试字段。统计事件类型时，热字段只有事件类型和计数；错误报告时，冷字段才重要。若直接保存一个大结构体数组，热循环会反复搬运冷字段。改成列式热字段后，扫描更快，但错误报告需要稳定 record id 回查原始位置。这个例子说明布局优化不能脱离语义：你减少了数据移动，也引入了索引和生命周期问题。

容器选择也要从访问序列推导。`std::vector` 适合顺序扫描、批量处理和 SIMD；扩容会移动元素，所以外部指针和引用要谨慎。`std::deque` 适合两端增长和较稳定引用，但分块布局让长扫描不如单块连续。`std::list` 插删不移动节点，但每个节点独立分配，遍历会变成指针追踪。教材要让读者看到：复杂度相同，常数和等待形状可能差几个数量级。

生命周期是布局迁移中最容易漏掉的部分。`std::span` 和 `std::string_view` 不拥有数据，只是视图。上游缓冲如果复用或释放，下游视图就悬垂。索引句柄也不等于对象身份：数组槽位复用后，旧 index 可能指向新对象，因此需要 generation 或稳定 id。布局章节若只讲 cache，不讲所有权，会把性能优化变成内存安全风险。

分配器成本同样具体。大量小对象分配会产生元数据、锁竞争、碎片和 TLB 压力。对象池或 arena 能降低成本，但要定义清理、复用和线程归属。一个线程分配、另一个线程释放，可能让分配器跨线程传递缓存；对象池如果没有 generation，旧句柄可能误用新对象。布局优化的代码实现必须包含这些边界，而不是只给一张结构体图。

预取要克制。硬件预取已经能处理许多规律访问；软件预取适合未来地址可知、计算距离足够、带宽仍有余量的场景。指针链上提前预取下一节点有时有用，但如果每次预取都错，或者工作集已经带宽受限，只会增加压力。实验要比较连续数组、固定步长和随机链表，不要把预取当作必胜技巧。

本章项目可以输出一个 LayoutPlan：字段冷热、容器选择、视图寿命、写入线程、对齐策略、分配策略和迁移步骤。迁移不能一次把全部代码改掉。先保留旧结构，新增批量视图和转换层；再让热路径直接消费新布局；最后删除旧路径。这样做更像工程教材，也更接近真实系统演化。

案例推导：把日志对象迁移成批量视图时，接口也要一起迁移

旧接口每次处理一个 `LogRecord` 对象。对象里有字符串、时间戳、事件类型和错误信息。热循环调用 `process(record)`，代码好读，但每次调用都带来对象访问、分支和潜在字符串处理。布局优化的第一反应是把 `std::vector<LogRecord>` 改成多个数组。若只改存储不改接口，热循环仍然每次构造临时对象，收益会被吃掉。真正要迁移的是接口：从单对象处理改成批量视图处理。

批量视图可以包含事件类型 span、用户 id span、时间戳 span 和 record id span。计算内核只读取热字段；错误报告和调试路径通过 record id 回到冷字段表。这个设计让编译器看见连续范围，也让所有权更清楚：视图不拥有数据，批次对象拥有底层存储。调用者不能把视图保存到批次生命周期之外。接口文档和类型要共同表达这个限制。

迁移时不要一次删除旧路径。第一阶段保留旧对象结构，新增从旧结构生成批量视图的转换层，用它验证新内核语义。第二阶段让解析器直接生成热字段列，减少中间对象。第三阶段清理旧对象路径。每一步都要测量转换成本和热循环收益，否则可能出现“内核快了，但总时间慢了”的情况。

对象池也要谨慎。批次反复分配大数组可能造成抖动，池化能降低分配成本；但池化带来复用风险。一个 batch 还在下游使用时，上游不能把同一块内存放回池里。可以用引用计数、队列所有权或显式状态保证。这里再次说明布局优化不是纯 cache 问题，它牵连生命周期和并发。

预取可以作为最后一层，而不是第一层。若热字段已经连续，硬件预取通常能处理；若访问用户表或字典编码表是随机的，可以尝试提前预取下一个 batch 的字典项。但预取距离要通过实验决定，

太近来不及，太远污染 cache。报告应写出输入分布，因为热点字典和随机字典的预取效果不同。

这个案例给读者一个重要判断：如果一个优化只改结构体定义，却没有改访问接口、生命周期和测试输入，它很可能是个不完整的布局优化。LayoutPlan 应把这些内容放在同一页上。

实验：实现 AoS 和 SoA 两个版本的字段求和。比较顺序访问和随机访问。再构造两个线程分别写相邻字段和分离字段的场景，观察 false sharing。

数据布局从访问模式推导，不从类型设计推导 很多 C++ 程序先设计对象，再让计算迁就对象。高性能系统往往反过来：先写出热路径访问哪些字段、以什么顺序访问、是否批量访问、是否跨线程写，再决定对象形状。贯穿项目的日志记录可能有时间戳、事件类型、用户 ID、数值、原始文本和错误信息。解析阶段需要原始文本和字段边界，聚合阶段只需要事件类型和数值，错误报告才需要原始文本。若所有字段绑在一个大结构体里，聚合阶段会把大量冷字段带进缓存。

AoS 适合按记录完整处理，SoA 适合按字段批量处理。AoS 让一条记录的所有字段相邻，代码直观；SoA 让同一字段连续，适合 SIMD、预取和列式扫描。项目可以使用两阶段形态：解析阶段产生紧凑的中间列，保留必要的记录身份；错误路径单独保存原始切片；聚合阶段只扫描热列。这样做不是为了追求某种数据布局名词，而是让每个阶段只搬运自己需要的数据。

生命周期决定容器和分配器 数据布局还要看生命周期。Batch 内临时字段、stage 内 partial、全局 manifest、错误报告，它们的生命周期完全不同。短生命周期对象适合 arena 或 monotonic buffer，统一释放；长期对象需要稳定所有权和明确析构；跨线程传递对象需要不可变或独占转移；跨异步 I/O 的 buffer 需要保证提交后直到完成前不能释放。若只看容器 API，不看生命周期，程序很容易在异步和并发章节出问题。

C++ 容器选择要写清成本。`std::vector` 提供连续存储，适合批量扫描；`std::deque` 分段存储，引用稳定性和连续性不同；`std::unordered_map` 查找平均快，但桶、节点、哈希和分配会制造随机访问；`std::map` 有排序语义，但指针追踪多；`std::pmr` 可以把分配策略显式传入。教材不应只说“哪个容器快”，而要说明它保护什么语义，产生什么地址序列。

从 LogRecord 迁移到 BatchView 一个好的案例是把 `LogRecord` 对象数组迁移成 `BatchView`。第一版每条记录是一个对象，包含多个 `std::string` 和字段。它易懂，但解析后聚合时会反复追字符串和对象指针。第二版把热字段提取为连续数组：event type、timestamp、value、status；原始文本保留为 input buffer 的 offset 和 length；错误信息单独放到 error vector。第三版为不同阶段提供只读 view，让聚合函数无法意外访问冷字段。

迁移要保持语义。字段 offset 必须在 input buffer 生命周期内有效；错误报告需要能回到原始行；排序或过滤后，记录身份不能丢；并发处理时，BatchView 的只读部分可以共享，输出部分必须按 partition 独占。这个案例能把数据布局、所有权、内存安全和性能连接起来。

预取要建立在规律访问上 硬件预取器擅长稳定步长和相邻 cache line，不擅长哈希表随机桶和链表。软件预取可以提前请求未来地址，但如果地址本身要等当前 load 才知道，或者预取距离不合适，收益有限甚至有害。很多情况下，重排数据让访问规律，比手写 prefetch 更可靠。比如先收集要访问的索引并排序，或者按 partition 分组后批量处理，能把随机访问转成更接近顺序访问。

项目里可以比较三种 top key 统计方式：直接哈希更新、先按 key hash 分桶再局部更新、对小整数 key 使用连续数组。观察指标不仅是时间，还包括 cache miss、分配次数和输出材料化大小。若数据布局改变导致后续 stage 更容易处理，即使单个阶段收益不大，也可能端到端更好。布局优化要看流水线，而不只看局部函数。

Linux 源码阅读连接点 Linux 内核大量使用 intrusive list、rbtree、xarray、per-cpu 变量和 slab cache。阅读时不要只记结构名称，而要看为什么选择这种结构：是否需要稳定节点地址，是否需要按 key 查找，是否需要范围查询，是否要减少分配开销，是否要 per-cpu 缓存。内核结构往往把生命周期和访问模式写进数据结构设计。

C++ 项目也应学习这种思路。任务对象若频繁创建销毁，可以使用对象池；ready 队列若只在 worker 本地访问，可以用本地 deque；manifest 若需要按 block id 查找和按提交顺序遍历，可能需要两种索引。一个数据结构不能只按“大 O”选择，还要按内存布局、生命周期、并发访问和恢复语义选择。

事故复盘：LogRecord 为什么拖慢缓存 朴素设计喜欢把一行日志装进一个 LogRecord：原始文本、解析字段、错误信息、时间戳、临时标志全在一起。聚合阶段只需要 key 和 value，却把整

块对象搬进 cache。输入一大，热循环不断加载冷字段，TLB 覆盖也被无关对象消耗。这里的问题不是对象风格本身，而是对象边界没有服从访问模式。

从事故推导出的结构是 BatchView。热字段放进连续数组，冷字段保留在原始 arena 或诊断表里，用稳定 record id 关联。扫描和聚合阶段只读 key、value、record id；错误报告阶段再按 id 回到原始行。这样既减少热路径搬运，又不牺牲诊断能力。Span 是视图，不拥有内存；arena 拥有批次生命周期；record id 是热冷数据之间的桥。

迁移布局必须带着测试走。先用 AoS reference 保证语义，再切出 SoA 热字段，检查空字段、超长字段、错误位置和原文回溯。报告不只写扫描变快，还要写内存峰值、分配次数、cache miss、TLB miss、arena 回收点和 span 生命周期。预取可以作为最后的局部优化，但如果访问模式本身混乱，预取只是掩盖设计问题。数据布局章节的核心是先问“谁在什么阶段读哪些字段”。

布局迁移实验判读 布局迁移不是只看扫描速度。热字段变快后，要检查错误报告是否还能回到原始行，record id 是否稳定，arena 是否在批次结束释放，span 是否悬垂。若 SoA 版本快但调试信息丢失，它不是合格迁移；若 AoS 在小批次更快，也要记录原因，可能是转换成本超过收益。

这一段也服务于复习。读者合上书后，应该能凭本章的判读口径重新审查自己的代码：有没有最小正确版本，有没有能击穿旧方案的输入，有没有状态记录，有没有资源指标，有没有失败后的恢复动作。若五个问题缺两个以上，就说明实现还停在演示层，而不是工程层。

从错误报告倒推冷热分离的约束 冷热分离不是把冷字段删掉。日志聚合热路径只需要事件类型、时间戳和数值，但错误报告需要原始行、字段偏移和解析原因。正确设计是热列保持连续，冷表通过 record id 或 offset 回查。这样热路径不搬冷数据，诊断路径仍然完整。

迁移时可以保留双写阶段。解析器同时生成旧 LogRecord 和新 BatchView，用同一组输入比较输出、错误报告和性能。双写阶段成本高，但它是迁移安全网。等 reference、错误路径和性能报告都稳定后，再删除旧结构。工程教材要讲这种过渡，因为真实系统很少允许一次性重写全部数据结构。

还要注意生命周期。BatchView 里的 span 不能比底层 buffer 活得更久；arena 适合批次内对象，不适合跨 stage 长期保存；string view 指向原始输入时，输入缓冲不能提前复用。很多布局 bug 不是访问模式错，而是生命周期没写进类型。

实验课：从 LogRecord 迁移到 BatchView 旧版本定义 LogRecord，包含事件名字符串、用户字符串、时间戳、数值、原始行、错误信息和调试标签。聚合阶段只需要事件 id 和数值，却必须跨过整个对象数组。新版本定义 BatchView，把 event id、value、timestamp 放进连续热列，原始行和错误信息留在冷表，通过 record id 回查。

迁移分三步。第一步双写：解析器同时产出 LogRecord 和 BatchView，用同一批输入比较结果。第二步切换热路径：聚合只读 BatchView，但错误报告仍从冷表恢复原始行。第三步删除旧路径前，运行错误密集输入，验证错误位置、原始偏移和错误原因没有丢。每一步都要保存内存峰值、分配次数、扫描时间和输出 checksum。

判读时要看两类收益。热路径收益来自连续访问、少搬冷字段、少分配；工程收益来自生命周期清楚，batch 结束后 arena 可以统一释放。代价也要写：转换阶段更复杂，record id 必须稳定，冷路径回查可能更慢。若错误报告不再完整，布局迁移就失败；若小输入转换成本超过收益，也要在报告里写阈值。

这节实验还要讲 prefetch 的边界。若访问已经连续，硬件预取可能足够；若访问随机，软件预取也常常太晚。比起手写 prefetch，更稳的改进通常是改变布局，让未来地址更早、更规则地出现。

布局迁移练习验收重点 合格答案要同时覆盖热路径和诊断路径。BatchView 快不够，还要能通过 record id 找回原始行和错误原因。答案要写对象生命周期，说明 span 指向谁、arena 何时释放、冷字段是否可能被异步使用。若这些说不清，代码再快也不合格。

答案还要写“为什么不采用更复杂方案”。例如没有证据证明队列是瓶颈，就不应把锁队列换成无锁；没有证明输出顺序可变，就不应随意重排；没有证明错误路径能恢复，就不应只提交正常路径。能解释不做什么，说明读者开始具备工程判断。

最后，答案必须包含可复查材料。至少有输入摘要、关键状态、期望输出、实际输出、运行命令和未验证边界。没有这些材料，老师或同伴无法判断结论是否成立。第二册的作业不是让读者写漂亮段落，而是训练读者把系统问题变成可验证证据。

项目实施：数据布局、局部性与预取 本章对应的贯穿项目实施不要求一次写成完整框架，但必须有最小可运行切片。这个切片包含三个部分：一个 reference，一个带本章机制的实现，一个能

击穿 reference 之外隐含假设的测试。Reference 用来固定语义，机制实现用来展示本章知识，测试用来证明新增复杂度确实解决了问题。

代码组织上，不要把所有内容放进一个大文件。基础类型、输入生成、核心机制、报告输出和测试应分开。即使当前只是教学项目，也要让目录能表达责任边界。这样后续章节接入时，不会因为找不到状态归属而重新发明一套对象。

每个实现都要输出一段机器可读摘要。摘要至少包含输入规模、关键状态、错误数、输出校验、耗时和未验证边界。这里的机器可读不要求复杂格式，简单的 key value 文本也可以。关键是让脚本和读者都能复查，而不是只能看屏幕截图。

测试应包含正常路径、边界路径和失败路径。正常路径证明功能可用；边界路径证明长度、容量、尾部、空输入、关闭或超时没有遗漏；失败路径证明本章机制不是装饰。若失败路径写不出来，往往说明本章概念还没有落到工程对象上。

报告最后写一个取舍段。新增机制带来了什么好处，增加了什么成本，哪些场景不应该使用它，下一章会复用哪个字段或对象。这个取舍段非常重要，因为系统工程不是把所有高级机制都加进去，而是在证据和约束下选择合适的边界。

调试笔记：布局变快后检查诊断路径 冷热分离或列式化后，必须专门跑错误输入。若错误报告找不到原始行，说明 record id 或 offset 丢失；若 span 指向已释放 buffer，说明生命周期合同错误；若异步写出还持有 batch 指针，说明 owner 过早复用。布局优化最容易在正常路径显得漂亮，在诊断和异步路径暴露问题。

复查清单：数据布局、局部性与预取 复查本章时，先检查 reference 是否仍然存在。没有 reference，后面的并发、优化、异步或分布式改造都无法证明语义没有变。再检查压力输入是否尖锐：它应当攻击本章最核心的边界，而不是只把数据量放大。第三，检查状态是否可观察：关键对象的身份、状态、错误和提交结果应能在报告中找到。

第四，检查代码示例是否说明禁止行为。调用者不能做什么，往往比能做什么更能体现工程质量。第五，检查实验是否写出未验证边界；当前设备、权限或输入无法证明的结论，必须标注。第六，检查本章对象是否能被下一章复用，不能复用就说明边界还不稳。

完成这六项后，再看文字是否连贯。每个概念都应从问题推出，每个案例都应从朴素方案走到改进方案，每个实验都应能解释一个判断。若某段话放到其他章节也成立，就把它改成本章的具体对象、具体输入和具体失败。

6.8 工程案例：把日志记录从对象改造成数据流

数据布局不是把结构体换成数组这么简单。它要从访问模式、生命周期、并发写和错误诊断一起推导。日志记录在程序员眼里像一个对象：原始文本、时间戳、用户、事件类型、错误状态、调试字符串都放在一起很自然。可是在热路径里，解析和统计可能只需要时间戳、事件类型和少量数值字段。把所有东西塞进一个对象，会让 cache line 搬运大量冷数据。

一、先写访问矩阵 访问矩阵记录每个阶段读写哪些字段。读取阶段需要原始字节和偏移；解析阶段写热字段和错误标志；统计阶段读事件类型和数值；shuffle 阶段读 key、partition 和 partial；诊断阶段才读原始文本和错误原因。若一个字段只在错误路径使用，就不应出现在每条记录的热结构体中。

访问矩阵能防止盲目 SoA。并不是所有对象都要列式化。若一组字段总是一起访问，AoS 可能更简单；若某个阶段只扫描一个字段，SoA 会减少无效搬运；若字段数量多但访问组合固定，AoSoA 可能折中。布局选择来自访问矩阵，而不是风格偏好。

二、热字段和冷字段分离 日志项目可以把解析结果分成 HotBatch 和 ColdStore。HotBatch 保存 event_type、timestamp_delta、user_bucket、record_id 等连续数组；ColdStore 保存原始文本切片、错误消息和调试上下文。统计阶段只读 HotBatch；出错或需要审计时，再通过 record id 查 ColdStore。

Listing 6.12 热字段批次只保存高频扫描字段

```
1 struct ParsedHotBatch {
2     std::span<const std::uint32_t> event_types;
3     std::span<const std::uint64_t> timestamp_ns;
```



```

4     std::span<const std::uint32_t> user_buckets;
5     std::span<const std::uint64_t> record_ids;
6 };

```

这个接口用 `span` 表达只读视图，不拥有内存。拥有者可以是 `batch storage` 或 `arena`。热路径函数不需要知道错误字符串放在哪里，这让编译器和读者都能看清访问范围。冷字段不是不重要，而是访问频率低，应从热循环拿出去。

三、生命周期决定能不能用视图 `std::string_view` 和 `std::span` 不拥有数据。它们适合表达只读窗口，但前提是底层存储活得足够久。异步 I/O、线程池和队列会拉长生命周期：生产者解析完 `batch` 后，消费者可能稍后才读取；如果生产者复用 `buffer`，视图就悬垂。布局设计必须和所有权设计一起做。

安全路线是让 `batch` 拥有自己的 `backing storage`，或者使用引用计数块，或者通过对象池保证直到下游完成才复用。不要为了减少拷贝随手把局部字符串暴露成 `view`。高性能 C++ 里，避免拷贝和保证生命周期同等重要；只做到前者会制造更隐蔽的 `bug`。

四、分配器成本不只在分配时出现 大量小对象分配会带来锁竞争、元数据、碎片、TLB 压力和缓存污染。日志解析若每个字段都创建 `std::string`，即使字符串很短，也可能把时间花在分配器和拷贝上。批量 `arena` 或块级 `buffer` 能把许多小分配变成少量大分配，释放时按块回收。

但 `arena` 也有代价。块太大导致内存峰值高，错误路径保留太多冷数据，异步消费者拖住 `arena` 释放，都会造成问题。报告要记录 `batch` 大小、`arena` 容量、峰值内存和下游持有时间。布局优化不能只看热循环快不快，还要看内存生命周期是否可控。

五、预取只适合可预测但来不及的访问 软件预取不是万能钥匙。顺序扫描通常硬件预取已经足够；随机访问预取地址如果太晚，来不及；如果太早，可能污染 `cache`；如果预取大量不会用的数据，会浪费带宽。适合预取的场景是地址可以提前算出，但硬件预取器难以识别，例如分块哈希探测、压缩索引间接访问或图遍历中的下一层邻接块。

实验要用对照证明预取价值。只改预取距离，其他不变；输入覆盖缓存内、内存级、随机和热点混合；记录耗时和 `cache miss` 变化。若结果不稳定，就保留无预取版本。教材要让读者明白：预取是针对特定地址序列的手段，不是性能代码的装饰。

六、并发写布局要避免同线争用 多线程写输出时，布局要考虑每个线程写哪些 `cache line`。线程本地 `partial` 数组若相邻存放，可能 `false sharing`；每个线程写不同列时，也可能在结构体数组中相互挤在同一 `line`。解决方法包括按线程分块、对热写字段 `padding`、把只读字段和写字段分离、最后再压缩布局。

这会带来一个常见取舍：运行时布局和持久化布局可能不同。运行时为了并发写和 `cache` 局部性使用分块列式；写出文件时为了压缩和读取方便使用另一种格式。不要强迫一个布局同时服务所有阶段。系统中常见的 `materialize`、`transpose`、`encode`，本质上都是在阶段边界重新组织数据。

七、迁移旧代码的步骤 旧代码如果到处传 `LogRecord&`，不要一次性全改。第一步写访问矩阵，找出热路径真正使用的字段。第二步加一个只读视图接口，让热函数接受 `span` 或 `batch view`。第三步保留旧对象作为 `reference`，批量生成新布局并比对输出。第四步再拆冷字段和调整所有权。每一步都要有 `benchmark` 和正确性测试。

迁移中最危险的是半新半旧。某些函数读新列式字段，某些函数偷偷修改旧对象，状态就会分裂。项目应明确一个阶段的权威数据源：解析前原始字节权威，解析后 `HotBatch` 权威，错误诊断通过 `record id` 查冷存储。权威源不清，布局优化会破坏语义。

八、实验交付：MemoryShape 本章给项目留下 `MemoryShape` 报告。它记录对象大小、字段冷热、数组长度、`stride`、页数、分配次数、峰值内存、`batch` 生命周期和并发写模式。布局优化提交必须更新这份报告。这样后续缓存、SIMD、线程和 I/O 章节都能复用同一份数据描述。

验收实验比较三种版本：完整对象数组、热冷分离的 `SoA`、按块 `AoSoA`。输入包括短日志、长文本字段、错误密集和 `key` 倾斜。报告不仅写速度，还写内存峰值、错误诊断是否仍可追踪、下游是否需要额外转换。数据布局的质量标准，是让常见访问快，同时让生命周期和诊断仍然清楚。

6.9 案例深化：容器选择要同时看复杂度和地址形状

C++ 容器常被按接口和复杂度学习：vector 随机访问快，list 插入删除快，unordered map 平均查找快。系统性能还要问地址形状、分配次数、元素大小、迭代频率和并发模式。一个算法复杂度看似更优的容器，可能因为随机访问和分配器成本变慢；一个看似需要移动元素的 vector，可能因为连续扫描和 cache 友好更快。

一、vector 的优势来自连续和简单所有权 `std::vector` 的元素连续，适合批量扫描、SIMD、预取和一次性写出。缺点是扩容会移动元素，插入中间成本高，指针和引用可能失效。若数据按批次生成、批次内只追加、之后只读扫描，vector 非常适合。日志 HotBatch、partition offset、block metadata 都可以用 vector 承载。

使用 vector 时要控制容量。频繁扩容会分配和拷贝；过度 reserve 会浪费内存。项目可以根据输入 split 估算记录数，预留合理容量；若估算错误，报告记录扩容次数。这样容器选择就和测量连接起来，而不是只凭经验。

二、list 的插入优势常被遍历成本抵消 `std::list` 节点分散，每个节点有指针，遍历需要追踪随机地址。它的稳定迭代器和 $O(1)$ splice 在某些场景有价值，但在热路径扫描中通常很差。很多初学者因为“中间插入删除快”选择 list，却忽略程序真正高频的是遍历和查找。

如果任务队列需要频繁从中间删除，可能可以用 deque、intrusive list 或索引数组加 freelist；如果只是在末尾追加并顺序处理，vector 或 deque 更好。容器选择要看操作频率，不是看某个操作的复杂度。写访问矩阵时，也应写容器操作矩阵：push、pop、erase、iterate、lookup 各占多少。

三、unordered map 的平均 $O(1)$ 隐藏了很多常数 哈希表查找涉及 hash 计算、bucket 访问、冲突处理、key 比较和可能的节点指针追踪。节点式 unordered map 对 cache 不友好；开放寻址哈希表通常局部性更好，但删除和负载因子处理不同。key 是整数、短字符串还是长字符串，差别也很大。平均 $O(1)$ 只说明渐进，不说明内存行为。

事件统计如果 key 范围小，用数组桶比哈希表好；key 范围大但可以字典编码，先编码再数组统计可能更快；key 是任意字符串时，局部哈希表加最后合并更稳。教材要让读者看到数据分布如何改变容器选择。

四、deque 和环形缓冲适合队列语义 `std::deque` 分段存储，支持两端高效 push pop，适合普通任务队列；固定容量环形缓冲更适合有界队列和 SPSC。Deque 的分段会让整体扫描不如 vector，但队列通常不做全量扫描。环形缓冲容量固定，能表达背压，但要处理空满和元素生命周期。

选择队列容器时，先问是否需要有限，是否需要阻塞等待，是否多生产者多消费者，是否需要关闭，是否需要稳定引用。容器只是状态存储，完整队列还需要同步合同。第 10 章会从不变量继续展开。

五、pmr 和 arena 要写清释放边界 C++ 的 polymorphic allocator 和 arena 能把许多小分配集中管理。它适合 batch 生命周期一致的对象，例如解析错误列表、临时 key 字符串、block metadata。它不适合生命周期交错复杂、对象可能被异步长期持有的场景。使用 arena 前要先说明释放边界：一个 batch 完成后释放，一个 job 完成后释放，还是进程结束释放。

错误的 arena 使用会造成悬垂 view 或内存峰值过高。若下游还持有 `std::string_view` 指向 arena，arena 不能释放；若 job 长时间运行而 arena 只增不减，内存会持续增长。布局优化必须带生命周期图。

六、容器实验 本章补充一个容器实验：同一组事件 key，用 vector 数组桶、节点式 unordered map、排序 vector 后合并、局部 map 加全局合并四种方式统计。输入覆盖小 key 范围、大 key 范围、热点 key、随机字符串 key。报告记录耗时、分配次数、内存峰值、cache miss 替代指标和代码复杂度。

这个实验会让读者理解：没有“最好的容器”，只有适合当前访问模式和生命周期的容器。它也为算法书和面试刷题知识补上工程层：复杂度是第一步，地址形状和所有权是系统中的第二步。

6.10 补强案例：压缩、编码和计算成本

数据布局还包括压缩和编码。压缩能减少内存和 I/O 字节，却增加 CPU；编码能把复杂 key 变成整数，却增加字典维护；位图能减少空间，却增加位操作和可能的分支。选择这些技术时，要把数据移动和计算成本放在同一张账本里。

一、字节减少不一定更快 如果程序受内存带宽或 I/O 限制，压缩可能更快；如果程序受 CPU 限制，压缩可能更慢。轻量编码如 delta、varint、dictionary、bit packing 各有适用范围。时间戳单调递增适合 delta，低基数字段适合 dictionary，布尔标志适合 bitset。随机高熵字符串不适合简单压缩。

二、编码会改变错误处理 字典编码失败时，系统要决定新增字典项、报告未知 key，还是使用 fallback。若字典跨 batch 共享，还要处理版本和并发更新。若分布式 worker 各自编码，最终 reduce 不能直接合并整数 id，必须共享字典或在输出前反查。编码不是纯性能技巧，它改变数据合同。

三、向量化和编码的关系 列式整数编码常帮助 SIMD，因为字段变窄、连续、类型统一。变长编码则可能阻碍 SIMD，因为每个元素边界不同。工程中常在存储格式和执行格式之间转换：磁盘上压缩，执行前解码成适合 SIMD 的块，处理后再编码写出。这是材料化点的一种。

四、实验 对事件类型使用原始字符串、字典编码整数、排序字典编码、位宽压缩四种表示。比较解析成本、聚合成本、内存峰值、输出大小和错误处理复杂度。报告要写何时选择哪种表示，而不是单纯追求最小字节数。

6.11 补充案例：热冷分离后的调试能力

热冷分离常被担心会降低调试能力，因为错误信息不再和热字段放在同一个对象里。这个问题必须正面处理。好的热冷分离不是丢掉调试信息，而是用稳定身份把热字段和冷诊断连接起来。

一、record id 是连接点 HotBatch 中每行保存 record id。ColdStore 用 record id 查原始文本、文件偏移、错误上下文。统计阶段只搬热字段；出错时通过 id 回查冷数据。这样既保持热路径紧凑，又保留诊断能力。

二、冷数据可以分层保存 不是所有冷数据都要一直保留。原始输入文件可作为最终冷源；批次内可以保存 offset 和长度；错误密集时再保存摘要；调试模式下保存更多上下文。生产模式和调试模式可以不同，但合同要一致：给定 record id，系统能在承诺范围内解释错误。

三、隐私和安全 冷数据可能包含用户文本或敏感字段。热冷分离还能帮助控制敏感数据流：热路径只处理编码后的 key 或 bucket，冷路径受权限控制。系统教材虽不深入安全，但要提醒读者数据布局也影响隐私边界。

四、实验 比较完整对象、热冷分离但无冷索引、热冷分离加 record id 索引。触发错误后，看能否定位原始记录。性能和可调试性都要验收。这样读者不会把布局优化理解成牺牲工程可维护性。

6.12 从访问模式推导数据结构形态

数据布局章节最容易退化成容器选择建议：数组快、链表慢、哈希表平均常数、预取可以加速。这样的说法既粗糙，也容易误导。工程中先出现的是访问模式：读多还是写多，顺序扫描还是随机查找，热字段是否集中，生命周期是否一致，是否需要稳定引用，是否要跨线程移动。数据结构只是这些约束的结果。把访问模式问清楚，布局选择才不会变成经验口号。

一、AoS 和 SoA 的选择来自字段共现 结构数组适合对象整体被一起使用的场景，列式数组适合只扫描少数字段的场景。比如日志记录包含时间戳、级别、模块、消息、线程号和解析状态。如果热循环只按级别和时间过滤，把所有字段绑在一个大结构体里，会把冷字段一起搬进缓存。改成列式布局后，过滤阶段只读少数列，带宽下降。可是后续若要输出完整记录，又要按索引回到原始对象，接口会复杂。教材要让读者看到迁移的全过程：先发现热字段，再拆分存储，再保留稳定 ID，最后处理回填和调试成本。

二、生命周期一致时才适合 arena arena 分配器的优势是批量分配、批量释放、减少碎片和锁开销。它的前提是对象生命周期接近一致。若一个请求内产生大量临时节点，请求结束统一释放，arena 很合适；若对象会被缓存、跨任务传递或单独删除，arena 会制造悬空引用和内存滞留。读者应把“分配快”继续追问成“谁拥有对象，什么时候释放，是否需要单独析构，是否允许指针长期存在”。生命周期不清楚时，任何分配器都会放大错误。

三、索引不是低级替代品，而是边界工具 用整数索引代替裸指针有两个好处：可以让对象存储更紧凑，也可以把引用的有效性检查集中到容器边界。比如图结构可以把节点放在连续数组里，边保存 `std::uint32_t` 节点编号。这样遍历时地址更集中，序列化更容易，跨进程传输也更清楚。代价是每次访问需要通过容器解析索引，删除和压缩要维护映射。若项目需要稳定句柄，可以把索

引和 generation 合成 handle，避免旧索引误指新对象。这里的重点不是“索引比指针快”，而是索引让所有权和布局从对象内部移到容器层。

四、预取只帮助可预测的未来 软件预取不是万能加速按钮。它要求未来地址足够早知道，预取距离合适，且预取的数据确实会被使用。顺序数组通常由硬件预取处理得很好，软件预取收益有限；指针追逐中，下一跳地址太晚才知道，预取也很难提前；批量处理多个独立链表时，预取可能有用，因为可以在处理当前节点时预取另一条链的下一节点。错误预取会占带宽、污染缓存、增加指令。报告中必须比较无预取、不同距离预取和不同输入规模，而不是只保留最快一次。

五、布局迁移要保护接口 数据结构改布局时，最容易破坏调用者假设。原来调用者可能保存对象指针，列式化后对象不再有稳定地址；原来字段可直接修改，压缩编码后修改需要解码和回写；原来遍历顺序自然稳定，分区后顺序可能改变。成熟的迁移会先引入访问接口和 ID，再内部改变存储，最后用测试验证语义不变。布局优化是系统工程，不是把一个结构体改成几个数组那么简单。

6.13 C++ 容器选择背后的内存模型

很多性能问题表面上是“该用哪个 STL 容器”，本质上是对象如何排列、引用是否稳定、迭代是否连续、插入删除是否频繁、分配是否集中。第二册不把容器当语法知识讲，而是把它放进内存层级和数据布局里讲。读者已经知道数组、cache line、TLB 和预取，再回头看 `std::vector`、`std::deque`、`std::list`、`std::unordered_map`，就能理解它们为什么在不同访问模式下差异巨大。

一、vector 的优势来自连续和简单 `std::vector` 把元素连续存储，顺序遍历对 cache 和预取器友好，size 和 capacity 也让内存占用容易估算。它的代价是扩容可能搬迁元素，插入中间元素需要移动后续元素，保存元素地址的调用者可能在扩容后失效。若系统需要稳定 ID，可以保存索引而不是指针；若系统知道元素数量，可以提前 reserve；若元素很大，可以把热字段拆出，vector 只保存紧凑结构。vector 不是因为名字高级而快，而是因为它让地址序列简单。

二、deque 解决两端增长但牺牲连续性 `std::deque` 通常由多个固定块组成，两端 push 和 pop 成本稳定，元素引用在某些操作下更稳定，但整体不如 vector 连续。它适合任务队列、滑动窗口和 work stealing 的基础结构，却不适合需要大规模线性扫描的数值数组。读者要避免把 deque 当成“更通用的 vector”。通用通常意味着额外间接层，额外间接层会改变缓存行为。

三、list 的问题不是复杂度，而是每个节点太贵 `std::list` 插入删除迭代器位置是常数复杂度，但每个节点独立分配，包含前后指针，遍历时地址跳跃，TLB 和 cache 都不友好。若业务真的需要频繁在中间 splice 且元素很大，list 有价值；若只是为了避免 vector 移动，常常可以用索引、稳定池、gap buffer 或分块 vector 替代。算法复杂度表没有把 cache miss 和分配器成本写进去，所以系统教材必须补上这一层。

四、unordered map 的常数不是一个数 `std::unordered_map` 平均查询是常数，但常数包含哈希计算、bucket 访问、节点访问、key 比较和可能的分配。短整数 key 和长字符串 key 完全不同；开放寻址表和节点式哈希表也完全不同。若热路径需要大量查找，可以考虑预哈希、字符串驻留、平坦哈希表、排序数组二分或分区索引。选择前先问：key 多大，查询多频繁，插入是否在线，是否需要稳定引用，是否能批量构建。这样容器选择才和数据布局同一条线。

五、容器接口要隐藏可替换布局 如果业务代码到处保存容器内部指针，后续布局优化会非常困难。更稳的做法是暴露 ID、span、迭代范围或只读视图，让底层可以从 vector 换成列式数组、从节点哈希换成平坦哈希、从裸指针换成索引。接口不是性能的敌人，糟糕接口才是。好的接口把语义稳定下来，把布局留给实现调整，这正是本章和全书项目的连接点。

6.14 贯穿案例：LogRecord 拆开以后为什么不能丢掉证据

数据布局优化最容易让人只看到速度。假设原始系统有一个 LogRecord：原文、字段边界、用户、事件、数值、错误状态、调试标签都在里面。聚合阶段只需要事件 id 和数值，却每次把整条记录相关的 cache line 搬进来。我们把它改成 SoA：event_id、value、record_id 三列连续存储，原文和错误上下文放进冷表。聚合速度上去了，但第一次错误回溯就失败：报告只能告诉用户某个 event id 异常，却找不到原始行、文件 offset 和解析状态。

这个事故说明布局优化不能只问“热路径少读了多少字节”，还要问“系统还能不能解释自己的

结果”。冷热分离的正确形态不是丢弃冷数据，而是让冷数据不进入热路径，同时保留稳定身份。于是 `RecordId` 出现了：热路径用它关联 `partial`，错误路径用它回到原文，`manifest` 用它说明输出来自哪批输入，恢复路径用它判断重复 `attempt` 是否处理过同一记录。身份字段是布局优化和系统可复盘之间的桥。

一、先画阶段访问矩阵 这个案例的第一步不是改结构体，而是列阶段访问矩阵。解析阶段需要原始字节、字段边界和错误状态；过滤阶段只看时间和级别；聚合阶段只看 `key id` 和 `value`；错误报告阶段要回到原文；写出阶段只看 `partial`、`checksum` 和 `record id` 范围。矩阵一出来，热字段和冷字段自然分开。没有矩阵，拆分容易凭感觉进行，最后不是热路径仍然带着冷字段，就是冷路径失去证据。

二、再定义身份而不是保存指针 如果热数组里保存指向原始对象的指针，SoA 很快又被对象生命周期拖回去。更稳的做法是保存固定宽度 `record_id`，由批次索引表把它映射到文件 `id`、`offset`、`length` 和冷表位置。索引比指针多一步查询，却带来三个好处：可以序列化，可以跨进程传递，可以在 `arena` 移动后保持稳定。代价是索引表必须版本化，删除和压缩要维护映射。这个代价比悬垂指针更可控。

三、生命周期要跟异步边界对齐 布局拆分后，很多数据变成 `view`。聚合线程可能持有 `std::span<const std::uint32_t>`，写出线程可能异步持有 `partial` buffer，错误报告可能稍后读取冷表。谁拥有内存，谁只能观察，什么时候释放，必须写进接口。若 `BatchArena` 在聚合结束就释放，而异步写出还持有 `span`，性能优化会变成 `use after free`。布局章节必须提前训练这种边界，因为后面的 I/O 和分布式章节会把生命周期拉得更长。

四、容器替换要经过接口层 原来调用者也许直接拿 `LogRecord*` 修改字段。SoA 后不再有一个完整对象地址。如果接口没有隔离，改布局会牵动全项目。迁移步骤应是：先引入只读视图和写入 `builder`；调用者通过 `RecordView` 读字段，通过 `RecordBuilder` 追加记录；内部仍用 `AoS`；测试稳定后再把内部换成 `SoA`；最后删除直接访问旧对象的路径。接口层不是官僚抽象，而是让布局可演进。

五、性能报告要同时写调试能力 布局改造报告除了吞吐、`cache miss`、分配次数，还要写错误回溯成功率、`record id` 覆盖率、冷表大小、`arena` 生命周期和调试模式开销。若热路径快了百分之四十，但错误报告丢失原文，这不是高质量优化。系统教材要把可调试性当成一等指标，因为后续分布式故障中，缺少证据会让所有性能收益失去意义。

六、从本章连接到分布式 `manifest` `RecordId` 和 `manifest` 是同一思想在不同尺度上的表现。`RecordId` 说明某个 `partial` 来自哪条输入记录；`manifest` 说明哪个 `block` 被最终接受。两者都把数据和提交证据分开。热路径可以紧凑高效，证据路径保持可回溯。理解这一点后，读者看到第三册模型权重、`tensor block` 或 `KV cache` 分片时，也会知道不能只搬字节，还要保留版本、身份和可见性。

6.15 案例深化：从 AoS 到 SoA 的迁移纪律

数据布局优化最容易伤到可维护性。一个 `LogRecord` 对象把字段都放在一起，调试舒服，但热路径搬运太多；改成多列数组以后，扫描快了，错误报告却找不到原文，生命周期也更难管。高质量迁移不是把 `AoS` 全部推翻，而是把访问模式、身份和生命周期一起改。

一、先列阶段访问表 解析阶段需要原始字节、字段边界和错误状态；聚合阶段需要 `key id`、`value` 和 `record id`；报告阶段需要错误位置和原文；写出阶段需要 `partial` 和 `checksum`。把阶段访问表写出来，就能看出哪些字段是热字段，哪些字段是冷字段，哪些字段只是诊断。没有这张表，布局调整容易变成凭感觉拆结构体。

二、热字段连续，冷字段可回溯 SoA 版本把 `key id`、`value`、`record id` 放进连续数组，聚合阶段顺序扫描。原始文本和错误详情留在 `arena` 或冷表中，通过 `record id` 回溯。这样热路径不搬运原文，诊断又不丢。若某个优化把 `record id` 删除，短期更快，长期会破坏错误报告和故障恢复。身份字段是布局优化的安全绳。

三、生命周期比指针更重要 `std::span` 只是视图，不拥有内存。它指向的 `arena` 必须活到所有使用者结束。若批次结束就释放 `arena`，而异步写出或错误报告还持有 `span`，就会悬垂。接口应明确 `owner` 和 `view`：`BatchArena` 拥有字节，`BatchView` 只观察热字段，错误报告在 `arena` 释放前完成或复制必要信息。布局章节必须把生命周期和性能放在一起讲。

四、预取只能优化已知路径 预取适合规律访问，不适合修复混乱布局。若程序随机追节点，预取距离难选，提前加载也可能污染 cache；若访问已经连续，硬件预取可能足够。手写预取前，先确认瓶颈来自可预测的未来访问。报告要比较无预取、布局重排、预取三种版本，避免把布局收益误归因给预取。

五、迁移要保留旧版本 **oracle** AoS reference 即使慢，也应保留到迁移稳定。每次布局变化都和 reference 比较字段数、错误数、checksum 和错误回溯。等 SoA 版本稳定后，reference 可以只在测试中运行。没有 oracle 的布局迁移，很容易把性能提升建立在丢字段、错生命周期或少报错误上。

6.16 案例复盘：LogRecord 拆分以后还能调试吗

一个 LogRecord 装下原始文本、字段、错误信息和临时状态，调试方便，却让聚合热路径搬运大量冷字段。改成列式布局后，扫描快了，但如果丢掉 record id，错误报告就找不到原文。数据布局优化的纪律，是热字段连续、冷字段可回溯、生命周期写清楚。

访问阶段决定布局 解析阶段需要原始字节和字段边界；聚合阶段只读 key、value 和 record id；错误报告阶段回到原文；写出阶段只处理 partial。阶段访问表写出来后，热字段和冷字段自然分开。Span 只是视图，arena 才是拥有者，批次结束前不能让异步使用者悬垂。

实验判据 AoS reference 保留到 SoA 稳定。每次迁移比较字段数、错误数、checksum、错误回溯成功率、分配次数、cache miss 和内存峰值。预取只在访问路径稳定后再试，不用它掩盖错误布局。

6.17 本章习题

习题与作业

习题一：为一个日志统计系统写访问矩阵。字段至少包括时间戳、用户 ID、事件类型、原始文本、解析错误、调试标签。热路径至少包括解析、按用户聚合、错误输出和调试查询。根据矩阵给出布局方案。

习题二：把一个指针链表设计改成索引数组设计。说明索引宽度、generation、删除策略、遍历局部性和序列化影响。不要只写“索引更快”，要写清生命周期如何保证。

习题三：设计一个冷热分离迁移计划。要求保留旧接口、建立 reference 测试、说明双写或缓存一致性策略，并给出何时删除旧布局的条件。

习题四：实现或设计 worker stats 的紧凑版和对齐版。报告每个版本的结构体大小、对齐、cache line 分布、写入线程和汇总成本。

SIMD、指令级并行与向量化

先看一个向量化失败的例子：把 ASCII 字段解析改成批量处理以后，规则输入变快了，错误输入却返回了错误位置。标量版本逐字节前进，遇到第一个非法字符就能给出精确 offset；向量版本一次处理多个 lane，只记录了“这一块有错误”，后续又继续提交部分结果。速度上去了，语义丢了。这个事故说明，SIMD 不是把循环换成更宽的指令，而是先证明多个元素可以在同一个语义合同下并行处理。

本章从标量 reference 出发讲 SIMD 和指令级并行。SIMD 让一条指令处理多个数据 lane，指令级并行让多个互不依赖的操作同时在不同执行资源上推进。二者都要求读者先回答：迭代之间是否独立，输出位置是否确定，尾部和错误如何处理，整数溢出和浮点误差是否允许重排。只有这些问题讲清，向量化才是优化，不是用更快的代码制造更隐蔽的错误。

7.1 从标量语义到向量执行

向量化不是把代码表面改成更复杂的形式，而是确认多个元素之间没有依赖，可以被同一条向量指令处理。数组逐元素加法容易向量化，因为每个元素独立；前缀和不容易直接向量化，因为后一个结果依赖前一个结果。

编译器自动向量化需要知道别名关系、对齐、循环边界和数据依赖。使用 `std::span` 表达连续区间有助于接口清晰，但编译器仍可能因为别名保守而放弃向量化。查看优化报告和汇编，比猜测更可靠。

判断循环能否向量化，要先写出每次迭代之间的关系。如果第 *i* 次迭代只读写第 *i* 个位置，通常容易向量化；如果第 *i* 次迭代读取第 *i-1* 次写出的结果，就存在循环携带依赖；如果每次迭代根据数据写入不同位置，就可能有冲突写；如果循环内部调用不可内联函数，编译器可能无法证明副作用边界。向量化首先是语义问题，其次才是指令问题。

别名和 restrict 思维 两个指针可能指向同一片内存时，编译器必须保守。例如 `a[i]=b[i]+c[i]` 中，如果 a、b、c 可能重叠，编译器要保证重叠情况下结果仍正确，这可能阻碍向量化。C++ 没有标准 `restrict` 关键字，但接口设计可以减少别名不确定性，例如使用不重叠容器、清晰文档和局部拷贝。

手写 SIMD 前，应该先让标量代码容易被编译器理解。清晰的循环边界、连续数据、少分支、少别名，是自动向量化的基本条件。

7.2 依赖、尾部和数值语义

即使不用 SIMD，多个独立累加器也能提高吞吐。单累加器形成长依赖链，每次加法依赖上一次结果；多个累加器把依赖链拆开，让乱序执行有更多可调度操作。SIMD 内核中也常见多个向量累加器，目的相同。

这就是并行归约在单线程内部也有优化空间的原因。先分成多个局部和，再合并，不仅适用于多线程，也适用于单核流水线。

Listing 7.1 多个累加器拆开依赖链

```
1 std::int64_t s0 = 0;
2 std::int64_t s1 = 0;
3 for (std::size_t i = 0; i + 1 < values.size(); i += 2) {
4     s0 += values[i];
5     s1 += values[i + 1];
6 }
7 const std::int64_t sum = s0 + s1;
```

这段代码的意义不是只用两个累加器，而是展示依赖链拆分。真实内核会根据执行端口、寄存器数量和向量宽度选择更多累加器。累加器太少隐藏不了延迟，太多会增加寄存器压力。

尾部处理和对齐 向量宽度通常不能整除数组长度，因此要处理尾部元素。常见方法是标量尾循环、掩码加载和补齐填充。对齐可以减少某些加载成本，但现代 CPU 对非对齐连续访问已经比较友好；真正要避免的是跨 cache line、跨页和随机访问。

尾部处理是性能代码里常见的正确性漏洞。主循环只处理完整向量宽度，剩余元素必须保证不丢、不重复、不越界。手写 intrinsics 时，尾部可以用标量循环、mask load/store 或提前 padding。选择哪种方式要看 ISA 支持、数据类型、越界读是否合法、输出是否允许覆盖 padding。教材示例宁愿保守清晰，也不能为了展示技巧隐藏边界条件。

常见误区

不要为了让 SIMD 主循环漂亮而忽略尾部。真实库里的很多错误，不发生在主循环，而发生在长度为 0、1、向量宽度减 1、刚好跨页或输入输出重叠的边界。

SIMD 与线程的关系 SIMD 和多线程不是替代关系。SIMD 提高单核吞吐，多线程使用多个核心。高性能程序通常先让单线程内核足够高效，再扩展到多线程。否则多线程只会复制低效内核，并更快撞上内存带宽。

数值语义 向量化可能改变浮点运算顺序，从而改变舍入误差。整数加法在不溢出的数学模型下更容易重排，浮点加法不严格满足结合律。编译器的 fast-math 选项会放宽数值语义，可能提高性能，也可能改变边界行为。工程中必须明确是否允许这种变化。

7.3 从自动向量化到手写 SIMD

优化顺序通常是先写清晰标量代码，再查看编译器是否自动向量化，再决定是否手写 intrinsics。自动向量化失败时，先看原因：别名不明确、循环边界复杂、函数调用副作用、数据依赖、类型转换、未对齐访问还是成本模型认为不值得。只有当标量语义已经清楚、数据布局稳定、热点足够重要、自动向量化不足时，手写 SIMD 才值得。

手写 SIMD 的维护成本很高。它引入 ISA 分发、不同宽度实现、尾部处理、测试矩阵和可移植性问题。生产库通常会把通用标量 reference、自动向量化版本和手写 ISA 版本分开，并用同一组 correctness tests 验证。第二册讲 SIMD 的目标不是让读者到处写 intrinsics，而是让读者知道什么时候应该追求向量吞吐，什么时候瓶颈根本不在这里。

向量化和内存带宽 SIMD 提高的是每条指令处理的数据量，但它不能凭空增加内存带宽。对于流式数组求和，如果单线程已经被内存带宽限制，更宽的 SIMD 可能只减少指令数，不显著提高总时间。对于算术强度高、数据能复用的内核，SIMD 才可能接近计算峰值。判断前必须回到 Roofline：这个内核到底缺算力，还是缺数据。

向量化前先定义语义 SIMD 教材最常见的错误，是上来就讲寄存器宽度和指令名，却没有先定义循环语义。向量化的前提不是“循环长得像数组”，而是多个迭代之间可以被重排、合并或并行执行。一个循环能否向量化，首先由数据依赖、异常语义、别名关系、写入冲突和数值模型决定。

以数组加法为例，每个输出位置只依赖同一位置的两个输入，且不同位置互不影响。只要输入输出不发生破坏性重叠，这个循环天然适合向量化。前缀和则不同，位置 *i* 的输出依赖位置 *i-1* 的结果，不能直接把所有元素当成独立 lane。直方图更复杂，每个元素会写入由数据决定的桶，不同 lane 可能写同一个桶，形成冲突写。图遍历则同时有随机读、分支和不均匀工作量。把这些循环都叫“处理数组”，对性能分析没有帮助。

因此本章的学习顺序是：先写出串行语义，再判断迭代独立性，再考虑编译器自动向量化，再考虑手写 SIMD。若语义不允许重排，强行使用 SIMD 只会写出错误程序。若语义允许重排但编译器没能向量化，应先找出失败原因，而不是直接责怪编译器。

baseline：标量数组变换 看一个最简单的变换：把两个数组相加写到输出。这个例子很朴素，但它包含自动向量化需要的几乎所有条件：连续内存、线性索引、无循环携带依赖、固定类型和简单表达式。

```

1 void add_arrays_scalar(std::span<const float> left,
2   std::span<const float> right,
3   std::span<float> output) {
4   const std::size_t count =
5     std::min(left.size(), std::min(right.size(), output.size()));
6   for (std::size_t i = 0; i < count; ++i) {
7     output[i] = left[i] + right[i];
8   }
9 }

```

这段代码适合作为 reference，但它没有明确说明三个 span 是否重叠。若 `output` 和 `left` 指向同一数组，语义仍然可能正确，因为每个位置先读后写同一位置；若 `output` 是 `left` 向后偏移一个元素的重叠视图，写 `output[i]` 可能影响后续 `left[i+1]` 的读取。编译器如果不能排除这种情况，就可能生成运行时别名检查，或者放弃某些向量化策略。

工程接口要尽量表达不重叠语义。可以在文档中声明三个区间不得破坏性重叠，可以在 debug 版本检查地址范围，也可以把输入和输出放在不同所有者对象中。C++ 标准没有可移植的 `restrict` 关键字，因此高质量 API 设计比某个编译器扩展更重要。

自动向量化报告 自动向量化不应该靠猜。Clang 和 GCC 都能输出优化报告，告诉你某个循环是否向量化、为什么失败、使用了多宽向量、是否做了运行时别名检查。不同版本选项略有差异，报告中应记录编译器版本和命令。一个报告可以包含如下问题：循环是否被 vectorized；vector width 是多少；interleave count 是多少；失败原因是依赖、调用、副作用、成本模型还是不支持的数据类型。

如果报告说循环未向量化，下一步不是立刻手写 intrinsics，而是修改源码让语义更清楚。把复杂函数调用移出循环，把热字段变成连续数组，把输出位置改成唯一写入，把分支拆成内部区域和边界区域，把可能别名的输入输出拆开。很多时候，自动向量化失败不是编译器弱，而是源码没有给足证明条件。

优化报告还要和汇编对照。报告说向量化，不代表生成了你预期的高效代码；报告说未向量化，也可能编译器通过其他方式优化。查看汇编时要关注是否出现向量 load/store、向量 add/mul/fma、循环展开、尾部处理和运行时别名检查。读汇编的目标不是背指令名，而是验证执行形态。

循环携带依赖和重写 循环携带依赖是向量化的主要障碍。最典型的是单累加器求和：每次迭代都依赖上一次 sum。编译器可以使用向量归约，把多个 lane 局部累加后水平合并，但这要求它确认重排语义允许。整数在有符号溢出语义、浮点舍入语义和 fast-math 选项下会有不同结果。

很多循环可以通过改写暴露独立性。例如计算差分数组：

Listing 7.3 可向量化的相邻差分

```

1 void adjacent_difference_kernel(std::span<const std::int32_t> input,
2   std::span<std::int32_t> output) {
3   if (input.size() < 2 || output.empty()) {
4     return;
5   }
6   const std::size_t count = std::min(input.size() - 1, output.size());
7   for (std::size_t i = 0; i < count; ++i) {
8     output[i] = input[i + 1] - input[i];
9   }
10 }

```

这个循环读相邻元素，但每个输出位置独立，通常可以向量化。相反，前缀和每个输出依赖前一个输出，不能直接按同样方式处理。判断时不要只看“是否读相邻元素”，要看“是否读本轮之前写出的结果”。

尾部处理的三种策略 尾部处理有三种常见策略。第一是标量尾循环：主循环处理完整向量宽

度，剩余元素用普通循环。它清晰、可移植、适合教学。第二是 mask load/store：使用掩码只处理有效 lane，适合 AVX-512、SVE 等支持较好的 ISA。第三是 padding：让数组末尾多出安全空间，主循环可以越过逻辑长度但不越过分配边界。这适合内部数据结构，不适合任意用户传入 span。

尾部策略还影响异常和内存安全。对输入做越界读即使结果被掩掉，在 C++ 语义上也可能是未定义行为；跨页越界读还可能触发 page fault。生产库不能随便假设尾部后面有安全字节，除非容器明确保证 padding。教材示例应优先使用标量尾循环，等语义清楚后再讨论 mask。

Listing 7.4 标量尾循环的结构

```
1 constexpr std::size_t SIMD_WIDTH = 8;
2
3 void add_arrays_chunked(std::span<const float> left,
4   std::span<const float> right,
5   std::span<float> output) {
6   const std::size_t count =
7     std::min(left.size(), std::min(right.size(), output.size()));
8   std::size_t i = 0;
9   const std::size_t vector_limit = count / SIMD_WIDTH * SIMD_WIDTH;
10  for (; i < vector_limit; i += SIMD_WIDTH) {
11    for (std::size_t lane = 0; lane < SIMD_WIDTH; ++lane) {
12      output[i + lane] = left[i + lane] + right[i + lane];
13    }
14  }
15  for (; i < count; ++i) {
16    output[i] = left[i] + right[i];
17  }
18 }
```

这段代码仍是标量 C++，不是 intrinsics，但它展示了向量主循环和尾循环的结构。编译器可能把内部 lane 循环向量化，也可能完全展开。教学中先写这种结构，有助于读者理解手写 SIMD 版本会怎样组织。

多版本派发和可移植性 手写 SIMD 不能只为一台机器写。x86 上有 SSE、AVX、AVX2、AVX-512；ARM 上有 NEON 和 SVE；不同机器支持的指令集不同。生产库通常需要多版本派发：保留标量 reference，再提供一个或多个 ISA 优化版本，运行时根据 CPU 能力选择。编译期只为当前机器生成一种版本，部署到其他机器可能无法运行。

多版本派发也有成本。每个版本都要测试，尾部处理要一致，数值结果要在允许误差内，构建系统要管理不同编译选项，调试时要知道实际跑的是哪个版本。对于不够热的循环，手写多版本不值得。自动向量化和标准库算法可能已经足够。

Compute Systems Engine 的第一阶段不需要手写所有 ISA 版本，但应保留接口层次：reference 标量版本、自动向量化友好版本、未来可能的 ISA 版本。这样项目不会因为一个手写内核绑死在某台 CPU 上。

Gather、scatter 和压缩写 SIMD 最喜欢连续 load/store。gather 从多个不连续地址加载，scatter 向多个不连续地址写入，通常比连续访问贵得多。它们能表达图遍历、哈希表查询和稀疏计算的一部分并行性，但不能把随机访问变成顺序访问。很多时候，重排数据、排序索引、分块处理比直接使用 gather 更有效。

压缩写常见于过滤：只把满足条件的元素写到输出。朴素分支版本会对每个元素判断并 push；并行版本需要先统计每个块保留多少，再扫描偏移，再写输出。某些 SIMD ISA 支持 compress store，但依然需要处理输出位置和尾部。过滤类问题把 SIMD、scan 和分区连接起来，因此第 13 章会专门展开。

分支、掩码和数据分布 向量化常把分支变成掩码。对于简单条件选择，例如 `output[i] = value > threshold ? a : b`，掩码很自然。对于复杂分支，如果两个分支都需要大量计算，把它们都执行再用掩码选择可能更慢。优化要看数据分布：如果分支高度可预测，标量分支可能很快；如

果分支随机且分支体短，掩码更可能有收益。

高性能内核常把数据预先分组，让同一批数据走同一条路径。日志解析可以按记录类型分桶，图算法可以按顶点度数分层，数据库执行器可以把 selection vector 传给后续算子。这样做的目的不是为了形式上“向量化”，而是让控制流和数据流更规则。

数值稳定性和重排 浮点 SIMD 需要更严肃的数值语义。浮点加法不满足数学结合律，改变合并顺序会改变舍入。向量化归约通常会改变加法顺序，因此结果可能与串行逐项求和不同。对于科学计算，这可能需要误差界、Kahan 求和、pairwise sum 或固定合并树；对于图形和机器学习某些场景，误差容忍度可能更高。教材不能只说“结果有一点不同”，要让读者决定业务是否允许。

整数也不是完全无忧。C++ 有符号整数溢出是未定义行为，编译器可以基于“不会溢出”做优化。若业务需要模 32 位或模 64 位行为，应使用无符号类型或明确的溢出处理。系统代码中的位宽类型和溢出语义要写清楚，否则优化和正确性会互相踩踏。

SIMD 章的回归阅读要先审语义合法性。数组变换、阈值过滤和 ASCII 字节分类看起来都能批量处理，但它们的输出语义不同：数组变换是一对一写回，过滤需要保存命中位置，字节分类需要报告第一个错误或所有错误。若输出位置和错误语义没有定义，直接写 intrinsics 只会把 bug 写得更隐蔽。

每个向量候选循环都要过六个检查：迭代之间是否独立，输入输出是否非法重叠，尾部如何处理，mask 如何转成语义结果，类型转换是否会溢出或改变舍入，平台不支持时是否有 scalar fallback。固定宽度整数类型必须写清楚，尤其是解析、量化和累加场景。窄 lane 中间结果一旦溢出，最后转成宽类型也无法修复。

本章的实验不要求读者马上手写 AVX 或 NEON。更稳的顺序是 scalar reference、自动向量化友好源码、优化报告、反汇编确认、强制 backend 对比、最后才是平台特化。这样第三册进入 AI 算子时，读者已经知道“先证明批量语义，再选择指令”，不会把 SIMD 当作神秘加速开关。

解析类 SIMD 更要强调结构拆分。完整 CSV 或日志解析包含引号、转义、变长字段、错误恢复和行号报告，很难一次性向量化；但查找换行、分隔符、ASCII 数字范围和非法字节可以拆成规则子内核。子内核输出 bit mask、位置列表或 selection vector，后续标量状态机消费这些结构信息。这样 SIMD 只承担适合它的字节分类，业务语义仍然保持清楚。

数值类 SIMD 则要写误差合同。整数 lane 要说明是否使用饱和、回绕、符号扩展和宽累加；浮点 lane 要说明是否允许重排、FMA 和不同合并顺序；混合精度要说明 narrow 和 widen 的位置。第三册量化推理会大量使用这些概念，第二册在这里先把语义边界讲清，避免以后把“向量指令更快”误写成“结果天然等价”。

SIMD 代码的接口也要避免污染业务层。公开函数应接收 `std::span`、配置和输出对象，不应暴露 AVX、NEON 或某个寄存器类型；平台特化藏在 dispatch 层，测试可以强制选择 scalar、auto-vectorized 和 native backend。这样读者会把 SIMD 看成可替换实现，而不是把业务函数绑死在一套指令集上。

尾部处理要单独测试。长度为 0、1、向量宽度减一、正好向量宽度、向量宽度加一、很大长度、非对齐起点和跨页边界，都可能触发不同路径。许多 SIMD bug 不在主循环，而在尾部、mask 到索引的转换、符号扩展和错误位置报告。教材中的每个批量内核都应列出这些边界，而不是只给理想长度。

7.4 先证明批量语义，再选择 SIMD 形式

SIMD 的核心不是“用了向量指令”，而是把多个独立元素放进同一条指令语义里执行。若元素之间有循环携带依赖，若输出位置依赖前面元素，若错误处理要求立即停止，若浮点顺序必须完全固定，那么批量执行就需要额外证明或重写。很多向量化失败不是编译器不够聪明，而是源码没有给出可批量执行的语义边界。

标量 reference 是 SIMD 的地基。先写清一个元素怎样处理，再写清一组元素能否独立处理，最后处理尾部和错误。若一开始就写 intrinsics，读者会把正确性和性能混在一起。高质量 SIMD 章节必须保留 scalar reference、自动向量化版本和手写版本，让读者看到每一步改变了什么。

自动向量化需要什么条件

编译器自动向量化通常需要连续内存、清晰循环边界、无未证明别名、无不可重排依赖和可接受的数值语义。`std::span`、restrict 思维、输入输出分离、批量接口都能帮助编译器看清这些条

件。相反，虚调用、函数指针、复杂迭代器、可能重叠的指针、循环中早退和异常路径都会让优化更保守。

优化报告是学习自动向量化的入口。GCC 和 Clang 都能输出为什么某个循环向量化或未向量化。报告不是性能证明，但能告诉你编译器卡在哪里。若报告说存在可能别名，就重写接口表达只读输入和独占输出；若报告说循环有依赖，就回到算法语义判断能否拆分；若报告说成本模型认为不值得，就比较不同输入规模和编译选项，不要盲目强制。

自动向量化成功后也要看反汇编和测量。编译器可能生成很宽的向量，也可能只做部分展开；可能为了尾部生成复杂 mask；可能因为对齐未知生成额外路径。报告要记录向量宽度、尾部策略、是否多版本派发，以及在小输入和大输入上的收益差异。

尾部、对齐和 mask 是正确性问题

数组长度通常不是向量宽度倍数，尾部处理不能靠“应该差不多”跳过。常见策略有标量尾循环、masked load/store、padding 输入。标量尾循环简单清楚，masked 版本减少分支但更复杂，padding 需要保证额外元素不改变语义。选择哪种策略，要看输入长度分布、平台指令集和错误处理要求。

对齐也不是单向答案。现代 CPU 能处理非对齐 load/store，但跨 cache line 或页面边界可能更贵。强制对齐需要分配器、容器和 API 一起保证；若只是写一个假设，代码就有未定义行为或隐蔽性能风险。教学项目可以先用不要求对齐的版本，测量后再决定是否加入 aligned allocator，并写清 fallback。

Mask 把控制流数据化。过滤、分隔符扫描、比较阈值都可以先生成 mask，再用 popcount、compress 或位置列表推进后续阶段。这里的难点是输出位置。若每个 lane 都可能输出，必须把 mask 转成紧凑位置，常见方式是局部计数、prefix 或查表。这个问题会连接到 scan/filter 章节，说明 SIMD 和并行算法不是割裂的。

数值、溢出和类型宽度

整数 SIMD 要写清溢出语义。std::uint8_t 加法是模 256，某些 saturating 指令是饱和到最大值，二者语义不同。解析 ASCII 数字时，临时乘加可能需要更宽类型；使用窄类型能提高并行度，但不能牺牲正确范围。代码示例应使用 std::uint32_t、std::uint64_t、std::int32_t 等位宽明确类型，不用含义随平台习惯变化的模糊长整型写法。

浮点 SIMD 更要说明重排。向量化归约会改变加法顺序，结果可能有微小差异；FMA 改变舍入路径；fast-math 允许更多变换但也放松 NaN、Inf、符号零等语义。教材不能只展示速度，要写出误差合同：允许绝对误差还是相对误差，是否要求确定性，是否保存固定归约树，是否禁用某些 fast-math 选项。

混合精度也要谨慎。用 float 替代 double 可以提高吞吐和减少带宽，但误差、溢出和稳定性要由 workload 决定。第三册 AI 算子会大量讨论量化和混合精度，本册先建立原则：类型变窄是语义改变，不只是性能优化。

手写 SIMD 的工程边界

手写 intrinsics 的成本很高：平台差异、代码可读性、测试矩阵、尾部路径、多版本派发、编译选项和维护成本。它适合热路径、稳定语义和明确收益，不适合随手替换所有循环。工程上常先写 scalar reference，再尝试自动向量化，最后只对关键内核写平台特化。

多版本派发要可测试。可以根据编译目标或运行时 CPU feature 选择 SSE、AVX2、AVX-512 或 NEON 版本，但每个版本都要跑同一套 oracle。若当前设备是 ARM 手机，x86 intrinsics 只能作为阅读材料，项目实现应保留 portable scalar 或标准库实验版本。教材必须尊重读者的 Termux/Linux 环境，不能假设每个人都有同一台服务器。

本章项目交付物可以是三个内核：ASCII 分隔符扫描、数字字段解析、向量化归一化或阈值过滤。每个内核都有 scalar reference、自动向量化尝试、手写或伪代码版本、尾部测试、错误输入测试和性能报告。这样读者学到的不是某条指令，而是 SIMD 化一个真实内核的完整流程。

案例走查：ASCII 数字解析的 SIMD 前置证明

ASCII 数字解析适合讲 SIMD，但必须先证明语义。输入字段长度可能不同，可能有非法字符，可能溢出目标类型，字段可能跨 batch 边界。标量 reference 要先定义这些行为：非法字符如何报告，溢出是否拒绝，空字段是否合法。没有这个合同，向量化版本即使快，也不知道对不对。

批量版本可以先不写 intrinsics，只把多个字符的分类和累加拆成数组操作，观察哪些步骤独立。字符是否在'0'到'9'之间可以批量判断；把位置权重乘上数字需要知道字段长度；错误 mask 需要能

回到具体行和列。这个分析会自然推出 mask、位置列表、尾部处理和旁路错误缓冲。

性能实验比较标量逐字节、批量标量、自动向量化、平台特化四个层次。每层都跑同一 oracle。若自动向量化已经足够，手写 SIMD 可以留作阅读；若手写版本收益明显，报告要写平台限制和 fallback。这样 SIMD 不再是炫技，而是从语义证明走向工程选择。

跨平台 SIMD 的教学策略

读者环境可能是 x86 服务器，也可能是 ARM 手机。教材不能把某一套 intrinsics 当作唯一主线。主线应是语义和数据形状：连续字节、固定宽度 lane、mask、尾部、水平归约、fallback。具体实现可以分成 portable scalar、编译器自动向量化、平台特化三层。平台特化作为深入案例，而不是唯一答案。

在 Termux/ARM 上，可以观察 NEON 自动向量化，也可以只做 scalar batch 版本。若书中展示 AVX2 代码，要同时写出它的语义等价伪代码和 fallback。这样手机读者不会因为硬件不同就无法学习。高质量教材要区分“原理必学”和“某平台指令可选”。

多版本派发的测试也要跨平台。每个版本都注册到同一个测试表，输入包括非对齐地址、短长度、长度不是 lane 倍数、非法字符和溢出。运行时 feature detection 选择某版本时，也要能强制指定 scalar 版本用于对照。否则平台特化出错很难定位。

向量化失败后的重构路线

当编译器报告无法向量化时，不要立刻写 intrinsics。先看失败原因。若是别名，改接口；若是循环中调用未知函数，把函数内联或改成批量回调；若是输出位置不确定，引入 scan；若是早停错误处理，把错误放到旁路 mask；若是数据不连续，回到布局章。这个路线能把 SIMD 和前面章节连接起来。

重构后仍可能不值得向量化。例如内核已经内存带宽受限，SIMD 减少算术指令也无法提升；或者 gather/scatter 太多，向量 lane 大量等待；或者尾部和错误路径占比高。报告要写出“为什么保留标量版本”。能选择不优化，是成熟工程能力。

本章作业可以给一个无法向量化的循环，让读者先读优化报告，再提出三种重构方案，并说明每种方案改变了哪些语义或接口。答案不要求都写代码，但必须说明正确性边界。

综合练习：SIMD 化前的语义审查

给出一个标量字段解析循环，要求读者先做语义审查。字段长度是否固定，是否允许空字段，非法字符如何报告，溢出如何处理，输出是否要求和标量完全相同，错误行是否继续解析后续字段。只有审查完成，才能进入批量化。若这些问题答不出来，写 intrinsics 只会把错误藏进更难读的代码。

第二步让读者写批量标量版本。它一次处理一个 batch，但仍然使用普通 C++ 循环。目标是暴露批量边界：哪些数组连续，哪些分支可以变成 mask，尾部如何处理，错误如何旁路。批量标量通常已经能帮助编译器自动向量化，也让后续手写 SIMD 有 reference。

第三步才比较平台实现。若是 ARM 设备，可以观察 NEON 或只保留自动向量化；若是 x86，可以写 AVX2 版本。每个平台版本必须跑同一套测试：长度小于向量宽度、等于向量宽度、不是倍数、非法字符、溢出、非对齐地址。报告写明当前平台，不能把某平台数字当成通用结论。

验收时要求读者解释三个“不优化”的情况：输入很小，错误路径占比高，内存带宽已经主导。能说出什么时候不写 SIMD，说明已经理解 SIMD 是工程选择，不是固定炫技动作。

容易出错的边界：向量化不是语义豁免

第一个反例是尾部丢元素。向量主循环处理宽度倍数，忘记处理剩余元素，小测试刚好长度对齐时不会暴露。第二个反例是错误 mask 丢失行号。批量检测非法字符后，只知道某个 lane 错，却无法报告输入位置。第三个反例是溢出。窄 lane 中间结果溢出，最后转回宽类型也救不回来。第四个反例是浮点顺序变化，导致测试在某些输入下不稳定。

第五个反例是平台假设。AVX2 版本在开发机快，手机 ARM 上不能运行；AVX-512 可能触发降频；非对齐 load 在某平台成本低，在跨页边界仍可能出问题。跨平台项目必须有 scalar fallback 和自动测试。书中展示平台代码时，也要写清它不是唯一实现。

第六个反例是 gather/scatter 滥用。看起来用了宽向量，实际每个 lane 都在随机访问，内存系统无法提供足够并行，性能不升反降。遇到 gather/scatter 频繁的内核，先回到数据布局和 partition，而不是继续堆指令。

本章源码索引建议读者给每个 SIMD 候选循环写“合法性清单”：无循环携带依赖，输入输出不非法重叠，尾部有定义，错误有位置，类型范围足够，浮点误差可接受，fallback 存在。清单过不

了，就不要进入 intrinsics。这个习惯能避免 SIMD 代码成为不可维护黑盒。

本章核心接口：KernelVariant

SIMD 章给项目留下 **KernelVariant**。每个热内核注册 scalar、auto vectorized、platform specific 等变体，统一签名、统一 oracle、统一 feature name。运行时可以选择最佳变体，测试也可以强制运行所有可用变体。这样平台差异不会散落在业务代码中。

KernelVariant 还记录语义标签：是否允许浮点重排，是否要求对齐，是否处理尾部，是否支持错误 mask。若某变体要求输入长度为特定倍数，它就只能作为内部 fast path，并必须有 fallback。接口把 SIMD 的前置条件写成代码对象。

从 SIMD 进入计算内核组合

SIMD 章关注一个内核内部怎样批量执行，下一章关注多个内核怎样组合成数据流。一个向量化扫描可能只是 pipeline 中的 filter 前置；一个批量解析内核的输出会进入 histogram；一个 mask 会变成 scan 的输入；一个局部 partial 会变成 reduce 的输入。单内核再快，也要放回 pipeline 才能判断端到端价值。

项目在这里要避免“为 SIMD 而 SIMD”。如果某个 SIMD 内核只占端到端 3

7.5 项目落地

Compute Systems Engine 在本章应加入三个版本的数组内核。第一，reference 标量版本，语义最清楚，用于校验。第二，自动向量化友好版本，数据连续、边界清晰、少别名、少分支，并输出编译器优化报告。第三，实验性 chunked 版本，显式展示主循环和尾部处理。项目不必马上手写 AVX 或 NEON，但要把 correctness tests 和 benchmark harness 准备好。

实验报告要记录：编译器版本、优化选项、目标 CPU、是否启用 fast-math、循环是否向量化、输入规模、对齐方式、尾部长度、结果校验和性能指标。若向量化版本没有更快，要分析是内存带宽、尾部、别名检查、前端压力还是数据太小。不要只写“SIMD 没用”。

Linux 和工具入口 本章更适合读编译器报告和反汇编，而不是直接读 Linux 内核源码。但 Linux 工具仍有用：**perfstat** 可以观察 instructions、cycles、IPC、cache miss；**perfrecord** 可以确认热点是否在目标循环；**lscpu** 可以记录 CPU flags；**objdump** 或编译器汇编输出可以确认指令形态。报告中应把工具输出和源码结构对应起来。

如果要读源码，可以从成熟库开始，例如 C++ 标准库实现中的算法派发、数学库中的多版本内核、数据库或向量搜索库的 SIMD 路径。源码阅读重点不是复制 intrinsics，而是看它如何组织 reference、ISA dispatch、尾部、对齐、测试和 fallback。

7.6 掩码、解析和批处理

数据库和批处理系统常使用 selection vector 表示哪些行通过过滤。它不是立刻把数据压缩到新数组，而是保存通过过滤的行号或位图，后续算子根据 selection 处理。这样可以避免每个 filter 都材料化输出，也能让多个过滤条件组合。Selection vector 连接了 SIMD 掩码、filter、compaction 和数据库执行。

若选择率很高，位图或掩码可能更紧凑；若选择率很低，索引数组只保存命中位置更高效。若后续算子需要随机访问原数组，索引数组会引入 gather；若后续先 compaction，再连续处理，可能更适合 SIMD。选择哪种表示，要看后续管线，而不只是当前 filter。

Listing 7.5 选择向量的接口形态

```
1 struct SelectionVector {
2     std::vector<std::uint32_t> selected_rows;
3 };
4
5 SelectionVector select_positive(std::span<const std::int32_t> values) {
6     SelectionVector selection;
7     selection.selected_rows.reserve(values.size());
8     for (std::uint32_t i = 0; i < static_cast<std::uint32_t>(values.size()); ++i) ←
    ↪ {
```

```

9  if (values[i] > 0) {
10 selection.selected_rows.push_back(i);
11 }
12 }
13 return selection;
14 }

```

这个版本是标量 reference。优化版可以用 SIMD mask 找出命中 lane，再把命中位置追加到 selection。无论实现如何，语义是稳定的：selection 中的行号按输入顺序排列。这个稳定性会影响后续 join、聚合和测试。

SIMD 和解析 文本解析看起来不适合 SIMD，因为字段变长、状态复杂、错误路径多。但解析中有一些子任务很适合向量化：查找换行符、逗号、冒号、引号；判断字节是否在 ASCII 数字范围；批量寻找非法字符；扫描固定格式字段。成熟 JSON、CSV 和日志解析器常先用 SIMD 找结构字符，再用标量状态机处理语义。

这种分层很重要。不要试图把完整解析器一次性 SIMD 化。先找出规则、重复、无依赖的子内核，例如扫描一块 64 字节里的分隔符位置。输出可以是 bit mask 或 position list，后续标量代码消费这些结构位置。这样 SIMD 处理的是字节分类，业务语义仍由清晰状态机负责。

Saturating、rounding 和溢出语义 某些 SIMD 指令提供饱和加法、舍入转换、最小最大、绝对值和打包。这些语义和普通 C++ 运算不一定相同。例如图像处理常需要 8 位饱和加法，超过 255 时保持 255；普通无符号 8 位加法则按模 256 回绕。若手写 SIMD 时不明确溢出和饱和语义，结果会和 reference 不一致。

量化和 AI 推理中，这个问题更重要。虽然 AI 放在第三册，本册仍要先建立概念：向量指令不仅影响速度，也可能携带特定数值语义。任何使用特殊指令的内核，都必须有 reference 和边界测试，覆盖最大值、最小值、负数、舍入边界和溢出边界。

混合精度和类型转换 SIMD 内核经常要在不同类型之间转换，例如 `std::int8_t` 到 `std::int32_t` 累加，`float` 到 `std::int32_t` 量化，`std::uint16_t` 到 `float` 计算。类型转换可能成为瓶颈，也可能改变精度和舍入。内核设计要把 load、extend、compute、narrow、store 的步骤写清楚。

混合精度优化必须有误差或溢出说明。若把 32 位累加改成 16 位累加，可能溢出；若把 double 改成 float，可能误差扩大；若把 float 转 int，舍入模式要定义。高性能不是牺牲语义的借口。

内核融合和寄存器压力 把多个循环融合成一个循环，可以减少内存读写。例如先 map 再 filter，若分开执行，需要写中间数组；融合后可以直接处理并写输出。但融合会增加循环体复杂度、分支、寄存器占用和代码大小。寄存器压力过高时，编译器会 spill 到栈，反而变慢。前端压力也可能增加。

因此内核融合要测量。适合融合的阶段通常是简单、连续、没有复杂依赖的 map/filter。Reduce、sort、join、shuffle 这类 pipeline breaker 不容易完全融合。数据库执行器中的 vectorized batch 模型就是折中：每个 batch 内尽量局部融合，但在需要重排或聚合的地方材料化。

SIMD 与线程并行的顺序 优化时常问：先做 SIMD 还是先多线程？稳妥顺序通常是先写标量 reference，再让单线程内核数据布局和自动向量化友好，然后再多线程。原因是多线程会引入调度、NUMA、同步和带宽变量，容易掩盖单核问题。单线程内核若已经受内存带宽限制，多线程能提高总带宽直到平台上限；单线程内核若受依赖链限制，多线程也许掩盖问题但不解决单核效率。

当然，某些场景先多线程更实用，例如业务任务天然独立且单个任务不热。工程选择取决于热点和成本。教材建议读者至少保留单线程优化报告，因为它是理解多线程扩展曲线的基础。

本章新增验收 本章在 Compute Systems Engine 中的验收应包括：每个内核有 scalar reference；自动向量化报告保存到实验记录；强制标量和自动向量化版本输出一致；尾部长度覆盖从 0 到向量宽度减一；对齐和非对齐输入都测试；浮点内核写明误差策略；小输入和大输入分别 benchmark；默认路径可回退到 scalar。

这些验收比手写一段漂亮 intrinsics 更重要。没有 reference 和回退，SIMD 内核很难维护；没有尾部测试，边界 bug 很常见；没有误差策略，浮点结果争议无法解决。

案例：ASCII 数字扫描 日志解析里常见子问题是判断一段字节是否全是 ASCII 数字。标量版本逐字节检查，遇到非数字退出。SIMD 版本可以一次加载多个字节，分别比较是否大于等于字

符 0 且小于等于字符 9，再把结果形成 mask。若 mask 表示所有 lane 都满足，就整块通过；否则找到第一个失败位置。

这个案例适合讲 SIMD，因为它规则、数据连续、每个字节独立，输出可以是一个位置或布尔值。它也展示了边界：输入长度不是向量宽度倍数时要处理尾部；跨页加载要小心；错误位置需要从 mask 转成索引；若字符串很短，标量可能更快；若输入大多第一字节就失败，标量早退出可能更好。SIMD 优化要看数据分布。

案例：分隔符位置列表 CSV 或日志解析还需要找逗号、空格、制表符或换行。SIMD 可以批量比较一块字节是否等于分隔符，生成 bit mask，再把 set bit 转成位置列表。位置列表就是一种 selection vector。后续标量解析器根据位置切分字段。这种结构把“找结构”与“解释语义”分开，常见于高性能解析器。

位置列表也有容量问题。若一块中分隔符很多，输出位置多；若分隔符很少，输出少。可以先用固定小数组保存一个 block 的位置，再追加到 batch-level vector。追加时要避免每个位置都抢全局锁，通常每个 worker 有本地位置缓冲，最后合并。

案例：向量化归一化 另一个适合 SIMD 的内核是数组归一化：`output[i]=(input[i]-mean)*inv_std`。它是纯 map，输入输出连续，迭代独立。若 mean 和 inv_std 已经计算好，循环很适合自动向量化。若 mean 也要从 input 求出，则整个算法分成 reduce 求和、计算 mean、map 归一化三阶段。这里可以看到 map 和 reduce 的组合。

归一化还展示浮点语义。求 mean 的归约顺序会影响结果，map 阶段的 fused multiply-add 也可能改变舍入。若业务要求严格复现，需要固定归约顺序和编译选项；若允许误差，需要写明 tolerance。一个看似简单的 SIMD map，也可能被前置 reduce 的数值语义影响。

SIMD 性能报告模板 SIMD 实验报告可以按固定模板写。第一，内核语义和 reference。第二，数据布局和对齐。第三，循环依赖和向量化理由。第四，编译器报告或汇编证据。第五，输入规模和分布。第六，正确性和误差。第七，性能结果和瓶颈归因。第八，反例和回退策略。

例如 ASCII 扫描报告应写：输入是随机 ASCII、全数字、首字节错误、末字节错误四种；输出是第一个非数字位置；reference 是标量逐字节；SIMD 版本对尾部使用标量；小于某个长度走标量；结果完全一致。这样报告才像工程文档，而不是只贴一段 intrinsics。

批大小和 SIMD SIMD 喜欢批量数据。若每次函数只处理一条记录，向量化很难摊薄固定成本；若把几千条记录放入 batch，很多字段可以列式处理，SIMD 才有空间。Batch 大小影响 SIMD、cache 和延迟。太小，向量利用率低；太大，cache 压力和尾延迟高。第 6 章的 columnar batch 是第 7 章 SIMD 的前提。

日志解析可以先按字节块扫描分隔符，再按记录 batch 解析字段，再按列做聚合。每一层 batch 大小可能不同。字节扫描适合较大连续块，字段解析适合记录 batch，聚合适合 event type 列。不要强求所有阶段同一个 batch 大小。

SIMD 和错误处理 错误处理会影响 SIMD 内核设计。若 SIMD 扫描发现某个 lane 非法，是立即返回错误，还是记录位置继续扫描？立即返回适合验证函数；记录位置适合批处理解析，因为一个 batch 里可能有多条错误记录，但仍希望处理其他记录。选择不同，输出结构也不同。

批处理系统通常把错误记录到 side buffer，不让少数错误打断整个向量内核。这样热路径保持规则，错误路径后处理。但如果错误表示输入损坏到无法确定边界，例如长度字段非法，必须停止当前消息。错误语义决定 SIMD 控制流。

跨平台 SIMD 的教学策略 本书不把某个 ISA 作为唯一答案，是因为读者设备可能是 x86、ARM 手机或云服务器。教学策略是先讲循环依赖和数据布局，再讲自动向量化，再讲抽象的 lane、mask、gather、horizontal reduce，最后才读具体 ISA。这样读者即使不写 AVX，也能理解为什么某个循环适合或不适合向量化。

第三册 AI 推理会更深入具体指令，但第二册先建立可迁移模型。现代系统工程师不应只会背某个指令名，而要能判断数据流是否值得向量化，如何设计 reference 和 fallback。

7.7 总结、决策表和反例

SIMD 的核心是把同一种操作作用到多个 lane 上。它要求数据规则、依赖清楚、输出位置明确、数值语义可接受。它经常和数据布局、batch、selection vector、scan、partition 一起出现，而不是

孤立出现。能自动向量化就先让编译器做；需要手写时，先设计 reference、dispatch、tail、对齐和测试。SIMD 是系统优化的一层，不是替代测量和算法设计的捷径。

SIMD 决策表 判断一个内核是否值得 SIMD 化，可以问：输入是否连续，迭代是否独立，输出位置是否固定，分支是否简单，数据量是否足够大，数值重排是否允许，目标平台是否稳定，是否已有自动向量化，是否有 scalar fallback。如果多个答案是否定，先改布局或算法。若答案大多肯定，再考虑编译器报告和手写优化。

这个决策表会在第三册 AI 算子中继续使用。矩阵乘、卷积、量化反量化适合 SIMD；复杂控制流和随机图遍历不一定适合。先判断形状，再选指令。

案例：向量化失败后的重构 假设编译器报告一个循环因为函数调用无法向量化。循环每轮调用 `classify(record)`，函数内部读取多个字段并返回类别。重构可以分两步：先把 `classify` 拆成只依赖热字段的纯函数，并确保可内联；再把记录字段从 AoS 改成列式，让 `classify` 处理连续数组。若仍有复杂分支，可以先按类型分桶。这个过程展示了 SIMD 优化不是最后一行指令，而是接口、布局和控制流共同重构。

重构后仍要保留原 reference。若 `classify` 涉及错误处理、字符串或时间解析，不应为了向量化改变语义。可以只向量化其中的简单子条件，把复杂记录留给标量慢路径。高质量 SIMD 优化常常是混合路径，而不是全量替换。

从依赖图看向量化 判断一个循环能否向量化，最直接的方法是画出一轮迭代依赖下一轮迭代的地方。若第 i 次迭代只读 `input[i]` 并写 `output[i]`，各次迭代相互独立，向量化自然。若第 i 次迭代要读上一轮写出的 `output[i-1]`，循环就有跨迭代依赖，普通 SIMD 很难直接处理。若依赖只是归约，例如所有元素累加到一个总和，可以把单一依赖链改成多累加器或树形归约。若依赖是状态机，例如字符串转义、括号嵌套或变长编码，通常要把能规则化的子任务拆出来。

依赖图还能解释为什么有些循环看起来简单却不能向量化。指针别名会让编译器不确定输入和输出是否重叠；函数调用会让编译器不知道它是否有副作用；异常和边界检查会让控制流变复杂；写入位置由数据决定时，多个 lane 可能写到同一位置。解决方法不是盲目加编译选项，而是改变接口：使用 `span` 表达连续范围，拆出纯函数，明确输入输出不重叠，把数据相关写入改成 `count`、`scan`、`scatter` 三阶段。

这也说明 SIMD 和并行算法是同一思想在不同粒度上的表现。SIMD lane 需要独立，线程任务也需要独立；SIMD 需要 mask 处理条件，线程并行需要 selection 和 partition 处理分支；SIMD 的 horizontal reduce 和线程归约都要面对结合性、顺序和误差。学 SIMD 时不要只盯着指令，应把它看成最细粒度的数据并行模型。

Gather 和 scatter 的真实代价 SIMD 并不要求所有数据都连续。有些 ISA 支持 gather 从多个地址收集 lane，也支持 scatter 把 lane 写到多个地址。但 gather 和 scatter 通常比连续 load 和 store 更贵，而且更容易受 cache miss、TLB miss 和地址生成限制影响。它们解决了表达能力，不等于解决了性能问题。一个随机索引数组上的 gather，可能只是把多个标量 miss 捆在一起，不能让内存系统变成连续访问。

因此，遇到随机访问时要先问能不能重排数据。若可以按 key 排序、按 bucket partition、把热点字段压成连续数组，通常比直接 gather 更可靠。若随机访问不可避免，可以批量处理多个请求，增加内存级并行，或者把索引按页和 cache line 分组，降低完全随机程度。只有在数据形状确实无法改变、且 gather 版本经过测量有收益时，才应把 gather 当成核心优化。

Scatter 更要小心写冲突。多个 lane 可能写到同一输出位置，语义是最后一个赢、累加、取最大还是报错？若是累加，就需要冲突检测、原子或分阶段归约；若是写位置唯一，最好用 scan 或 partition 先分配不重叠区间。数据相关写入是 SIMD 和线程并行共同的难点，不能因为用了向量指令就忽略输出语义。

选择标量回退的工程判断 高质量 SIMD 代码一定要有标量回退。回退不是失败，而是工程边界。短输入、未对齐尾部、罕见错误路径、平台不支持目标指令、调试模式和语义复杂路径，都可能走标量。标量 reference 让测试清楚，也让平台覆盖更稳。没有回退的 SIMD 内核容易把一个优化点变成全系统移植风险。

回退策略要写进接口。可以在运行时根据 CPU 能力选择 AVX、SSE、NEON 或 scalar；也可以在编译期生成不同目标；还可以让小输入直接走 scalar，避免向量准备成本。选择逻辑必须可测试，不能只在开发机器上跑最快路径。报告应包含 scalar、自动向量化、手写 SIMD 三组结果，而

不是只展示最好版本。

回退还帮助定位 bug。若 SIMD 输出和 scalar 不一致，先缩小输入，固定随机 seed，记录 lane mask、尾部长度和第一个不同位置。很多 SIMD bug 出在尾部、符号扩展、饱和和舍入。标量 reference 是这些 bug 的锚点。性能工程不能脱离正确性锚点，否则优化越多，系统越难信任。

实验：用编译器优化报告观察一个数组加法循环是否向量化。再写单累加器和四累加器求和，比较性能。记录编译选项和 CPU 指令集。

标量 reference 每个向量版本都要有清晰标量版本。标量版本定义边界条件、溢出处理、浮点误差、错误输入和输出顺序。向量版本只是在这个合同内改变执行方式。

lane 语义 向量寄存器里的 lane 不是独立线程。它们共享一条指令，分支需要 mask，异常和尾部也要统一处理。读者要学会把 if 转成 mask，而不是把控制流幻想成自动并行。

依赖与别名 循环携带依赖会阻止向量化，指针别名会让编译器保守。C++ 接口可以用 span、const、restrict 风格约束和不重叠断言帮助编译器，也帮助人理解。

尾部处理 长度不一定是向量宽度整数倍。尾部可以用标量收尾、mask load/store 或 padding。不同选择影响代码复杂度、越界安全和性能。

对齐 现代硬件对非对齐访问容忍更高，但跨 cache line 或页边界仍可能更贵。对齐策略要和分配器、数据布局和尾部策略一起设计。

数值误差 浮点 reduce 改变结合顺序会改变结果。性能报告不能只比较速度，还要比较误差界限。整数溢出也要明确，是按位宽回绕、饱和还是报错。

自动向量化 优先让编译器完成简单循环，再查看报告和反汇编。手写 intrinsic 应限于热点、语义已固定、编译器确实做不到的路径。

可移植边界 x86 AVX、ARM NEON、SVE 的宽度和能力不同。项目接口应把向量实现藏在策略层，外部仍看到同一标量语义。

案例：阈值过滤 标量扫描大数组生成 mask，再写向量版本。要求输出位置和稳定性与 reference 一致，尾部输入必须覆盖。

案例：浮点求和 比较串行、分块、向量 reduce 的误差和速度。结论必须说明误差来源，不允许只写 SIMD 更快。

案例：别名反例 让输入和输出可能重叠，观察错误或编译器无法向量化。再用接口禁止重叠，说明语义约束如何换来优化空间。

源码阅读路线 Linux 源码阅读从 [tools/perf](#)、[arch/x86/lib](#)、[arch/arm64/lib](#) 附近进入即可。阅读目标不是把整套子系统抄进书里，而是追一条和本章实验相关的路径：用户态动作怎样变成内核对象，内核对象怎样排队、等待、唤醒、计数、提交或回收。

阅读报告要写证据等级。能直接测到的数字放在第一层；从源码控制流推导出的解释放在第二层；当前设备无法验证、只能作为合理猜测的内容放在第三层。这样读者会知道向量化语义报告中哪些结论可复现，哪些结论需要换机器继续确认。

源码阅读还要克制。看到一个复杂函数，不要把每个分支都展开；先回到本章的问题，问它能否解释一个等待点、一次状态转移或一条性能曲线。解释不了的内容可以留到索引，不要打断正文主线。

阈值过滤 标量扫描大数组生成 mask，再写向量版本。要求输出位置和稳定性与 reference 一致，尾部输入必须覆盖。

浮点求和 比较串行、分块、向量 reduce 的误差和速度。结论必须说明误差来源，不允许只写 SIMD 更快。

别名反例 让输入和输出可能重叠，观察错误或编译器无法向量化。再用接口禁止重叠，说明语义约束如何换来优化空间。

标量语义

SIMD 之前必须有标量 reference。它定义溢出、错误输入、尾部元素、浮点误差和输出顺序。

实验设计：先写最普通循环，再用同一输入校验向量版本。

数据并行

SIMD 适合多个元素执行同一操作。若每个元素走不同控制流，掩码和压缩成本可能抵消收益。
实验设计：比较均匀输入和分支高度随机输入上的向量化效果。

尾部处理

输入长度不一定是 lane 数倍。尾部可以用标量循环、掩码加载或补齐缓冲，选择取决于语义和成本。

实验设计：对长度从零到多倍 lane 加一的输入做边界测试。

对齐

现代指令支持非对齐加载，但跨 cache line 或页边界仍可能有成本。对齐更多是稳定性和简化边界，不是绝对规则。

实验设计：比较对齐起点和偏移起点的吞吐，并记录是否出现异常。

掩码

掩码让向量代码表达条件，但掩码不是免费。它适合把控制流变成数据流，也可能增加寄存器和混洗压力。

实验设计：实现字节分类计数，比较分支、查表和掩码版本。

归约

向量归约需要横向合并，浮点结果还受合并顺序影响。整数求和、饱和加法和浮点累加不能混为一谈。

实验设计：对浮点数组固定合并树，记录误差范围。

自动向量化

编译器需要看到连续范围、无别名、无循环依赖和明确边界。写 intrinsics 前，先让自动向量化报告解释失败原因。

实验设计：保存编译器报告中未向量化的理由，并逐条消除。

手写 intrinsics

手写 SIMD 适合稳定热点和明确 ISA。它会增加维护、派发、fallback 和测试成本，不能作为第一反应。

实验设计：把内核分成 reference、portable fast path、ISA-specific path 和 dispatch。

数值语义

快速数学选项可能改变 NaN、舍入、结合律和异常语义。性能报告必须写清是否允许这种改变。

实验设计：同一输入比较严格模式和 fast math 结果差异。

项目接口

VectorizationPlan 记录输入布局、lane 宽度、尾部策略、误差规则、fallback 和测试矩阵。没有它，SIMD 代码很难维护。

实验设计：每个向量内核都保留标量 reference 和随机测试。

7.8 SIMD 之前先证明批量语义

SIMD 很容易被写成指令大全：加载、比较、掩码、混洗、归约。这样的章节看起来专业，却容易让读者误以为性能来自某条神秘指令。真正的顺序应相反：先证明标量语义，再证明多个元素能执行同一操作，再决定让编译器自动向量化还是手写 intrinsics。没有这条顺序，SIMD 代码只会变成难维护的特殊版本。

标量 reference 是第一步。它定义输入长度为零怎么办，尾部元素怎么办，整数溢出是否允许，浮点误差如何比较，遇到非法字符是跳过、计错还是终止。向量代码必须继承这些语义。比如字节扫描中，标量版本遇到非 ASCII 字节如何处理，SIMD 版本也要一致；浮点归约中，标量顺序和向量合并树可能产生不同舍入，报告必须说明误差范围。

数据并行是第二步。SIMD 适合多个元素做同一件事。若每个元素都走完全不同的控制流，向量化可能需要大量掩码和压缩，收益未必存在。掩码是把控制流变成数据流的工具，不是免费午餐。它会占寄存器，会增加逻辑操作，也可能让后续压缩或存储更复杂。因此本章不能只展示成功案例，也要展示随机分支、稀疏保留和复杂错误路径下 SIMD 的边界。

尾部处理是第三步。输入长度很少刚好等于 lane 数倍。标量尾循环最简单，掩码加载能减少分支但要小心越界，补齐缓冲会改变内存访问和错误处理。教材应要求读者对长度 0、1、lane-1、lane、lane+1 做测试。很多 SIMD bug 不在大输入上暴露，而在尾部和边界上暴露。

自动向量化是第四步。编译器需要看到无循环依赖、无别名、连续范围和明确边界。优化报告里常见失败原因包括可能别名、循环携带依赖、调用不可内联、控制流复杂。读者应该先读失败原因，而不是直接手写 intrinsics。手写 SIMD 适合稳定热点和明确 ISA，但会带来 dispatch、fallback、测试矩阵和跨平台维护成本。

本章可以围绕三个内核做实验。第一个是数组加法，用它讲连续范围、对齐和尾部。第二个是字节分类计数，用它讲掩码、查表和分支分布。第三个是浮点归约，用它讲合并树和误差。每个实验都要保留标量 reference、自动向量化版本和手写版本，报告中写清哪个版本在什么输入上成立。这样 SIMD 就不再是“会写指令”，而是“会证明批量执行没有改变语义”。

案例推导：字节分类内核怎样从标量版本走到 SIMD 版本

日志解析常要判断字符是数字、分隔符、空白还是其他。标量版本逐字节判断，很容易写对，也方便处理错误。向量化的第一步不是写 intrinsics，而是确认分类规则能批量表达：每个字节的分类只依赖它自己，还是依赖前后状态？如果要识别引号内分隔符，状态会跨字节传播；如果只是统计分隔符数量，批量比较就很适合。

先写 reference：输入为空、只有一个字符、包含非 ASCII、末尾无换行、连续分隔符、超长字段，都要定义输出。然后写一个自动向量化友好的版本：连续数组、简单循环、无函数调用、无别名。编译器如果仍不向量化，读优化报告，找原因是循环依赖、条件复杂还是数据类型不合适。很多时候，改好接口和循环结构，比手写 SIMD 更有价值。

手写 SIMD 版本可以先做最窄目标：一次加载多个字节，与分隔符向量比较，生成掩码，再统计掩码位数。尾部用标量处理，保证边界清楚。下一步再讨论多个分隔符、查表分类或压缩输出。不要一口气写完整解析器的 SIMD 版本，否则错误来源太多，读者无法归因。

掩码版本要和输入分布一起测。如果大多数行格式稳定，SIMD 字节扫描可能显著减少比较和分支；如果错误处理很多，向量路径频繁回退，收益会下降。若每次找到分隔符后还要立刻调用复杂字段解析，前端调用和状态机可能重新成为瓶颈。报告要写瓶颈迁移，而不是只写 SIMD 速度。

数值内核也有类似问题。浮点向量归约改变合并顺序，可能改变最后几位。若业务需要 bitwise 一致，就要固定树形合并或继续使用标量顺序；若业务允许误差，就写明误差界。编译选项如 fast math 会改变 NaN 和结合律语义，不能偷偷打开。SIMD 章节要把性能和语义绑在一起，才不会培养出只追指令的习惯。

最终 VectorizationPlan 应列出 scalar reference、批量语义、尾部策略、对齐假设、ISA dispatch、fallback、误差规则和测试矩阵。手写 SIMD 代码只有在这张计划表成立后才值得进入项目。

手写 SIMD 的接口边界 若最终确实需要手写 SIMD，第一步不是写 intrinsics，而是设计接口边界。一个好的 SIMD 内核通常有四层。第一层是公开 API，接收 span、配置和语义选项。第二层是 portable reference，用标准 C++ 实现，作为 correctness oracle。第三层是 dispatch，根据 CPU 特性选择实现。第四层是 ISA-specific kernel，只处理已经检查过的输入和尾部策略。

这种分层能把复杂性关在局部。公开 API 不暴露 AVX 或 NEON 类型；reference 不依赖平台；dispatch 只处理选择；具体内核只处理性能关键路径。若把 intrinsics 直接散落在业务代码里，后续测试、移植、调试和回退都会变得困难。

Listing 7.6 SIMD 内核的分层接口草图

```
1 enum class KernelBackend : std::int32_t {
2     Scalar,
3     AutoVectorized,
4     NativeSimd,
5 };
6
7 struct KernelOptions {
8     KernelBackend backend = KernelBackend::AutoVectorized;
9     bool allow_fast_math = false;
10 };
11
12 void transform_batch(std::span<const float> input,
13     std::span<float> output,
```

```
14 const KernelOptions& options);
```

这里没有把某个指令集写进公开接口，是为了让调用者表达语义需求，而不是绑定实现。运行时可以根据平台选择 AVX2、AVX-512、NEON 或标量 fallback。测试可以强制选择 Scalar 和 AutoVectorized，比较结果。若某个 ISA 版本出问题，可以临时关闭，而不影响业务 API。

运行时派发的测试矩阵 多版本派发的测试不只是“默认路径跑通”。必须强制每个 backend 跑同一组 correctness tests。输入要覆盖长度为 0、1、向量宽度减一、正好向量宽度、向量宽度加一、大数组、非对齐起点、输出和输入不同数组、允许的重叠或禁止重叠场景。浮点内核还要覆盖 NaN、Inf、负零、极大值和极小值，除非 API 明确不支持这些值。

性能测试则要区分小输入和大输入。小输入时 dispatch 成本、函数调用和尾部处理可能占主导；大输入时内存带宽或计算吞吐主导。若一个 SIMD 版本只在超大输入上快，API 可以设置阈值，小输入走标量路径。很多成熟库都有这种 size threshold，因为 SIMD 启动成本不是零。

对齐策略的工程现实 早期 SIMD 教材常强调对齐加载。现代 CPU 对连续非对齐访问已经更友好，但对齐仍然有价值：减少跨 cache line 或跨页访问，便于某些 ISA，简化内部块处理。工程上常见做法是让容器或分配器提供对齐保证，但公开 API 仍要能处理普通 span。内部可以先处理到对齐边界，再进入对齐主循环，最后处理尾部。

不过，对齐优化要防止过度复杂化。若内核主要受内存带宽限制，复杂的对齐前缀可能收益很小；若输入来自用户任意切片，对齐主循环比例可能不高；若需要跨平台，某些 ISA 的对齐要求不同。测量应覆盖对齐和非对齐输入，而不是只在理想对齐数组上报告性能。

水平归约和 lane 间通信 SIMD lane 之间并非完全免费通信。逐元素 map 每个 lane 独立，最适合 SIMD；归约需要把多个 lane 的 partial 合并成一个标量，涉及 horizontal add、shuffle 或树形 lane 合并。不同 ISA 对水平操作支持不同，成本也不同。因此向量归约通常会使用多个向量累加器，先在向量寄存器里积累很多元素，最后再做一次水平合并。

这个模式与线程本地 partial 类似。lane 内先局部累加，最后水平合并；线程内先本地 partial，最后线程间合并；节点内先 map-side combine，最后跨节点 reduce。层次化归约不是某一层技巧，而是贯穿硬件 lane、线程、NUMA 和集群的通用结构。

Vector-friendly 数据结构 不是所有数据结构都适合 SIMD。链表、树节点和虚函数对象通常难以向量化，因为地址不连续、分支多、类型不统一。若热路径需要 SIMD，应把数据组织成 vector-friendly 形状：连续数组、SoA、AoSoA、紧凑索引、固定宽度字段和分离冷数据。第 6 章的数据布局不是 SIMD 之前的附属内容，而是 SIMD 能否发挥作用的前提。

以日志解析为例，原始文本是变长字符串，很难直接 SIMD 化完整业务逻辑；但某些子问题可以向量化，例如扫描分隔符、统计字符类别、批量检查 ASCII 范围。工程中常把复杂解析拆成多个阶段：先用 SIMD 找结构位置，再用标量状态机处理复杂字段。不要要求 SIMD 解决整个业务函数，而要找出规则、重复、数据并行的子内核。

本章的反例清单 判断 SIMD 是否值得时，可以先看反例。第一，输入很小，dispatch 和尾部成本超过收益。第二，内核受内存带宽限制，SIMD 只减少指令数不减少字节。第三，数据结构是随机指针追踪，gather 成本高且预取困难。第四，分支复杂且各分支工作量大，掩码执行会做大量无用工作。第五，数值语义要求逐位一致，重排空间受限。第六，代码需要跨很多平台维护，但热点不够热。

这些反例不是让读者不用 SIMD，而是让优化更有目标。真正应该优先 SIMD 化的，是热、规则、数据连续、迭代独立、算术强度足够或函数调用成本可摊薄的内核。Compute Systems Engine 的 map、normalize、简单解析扫描、局部归约都可以成为候选；全局队列、锁等待和分布式重试不是 SIMD 能解决的问题。

自动向量化的前置条件 自动向量化不是编译器魔法，它需要源码表达出足够清楚的循环形状。典型前置条件包括：循环边界可分析，迭代之间没有无法证明的依赖，内存访问尽量连续或固定步长，分支能转成掩码或被分离，函数调用能内联或被证明无副作用，输入输出别名关系清楚。任何一个条件不满足，编译器可能保守放弃。

许多向量化失败不是因为编译器弱，而是接口太含糊。一个循环写 `output[i]`，同时读 `input[i+1]`，如果 input 和 output 可能重叠，编译器必须考虑写入影响后续读取。一个循环内部调用虚函数，编译器很难知道函数是否访问全局状态或抛异常。一个循环每轮可能 push 到 vector，

输出位置不是简单函数，也不容易向量化。软件工程师要做的是把数据依赖变成编译器能看懂的结构。

向量化报告要成为开发流程的一部分。GCC、Clang 和 MSVC 都有不同形式的优化报告。读报告时要关注“not vectorized”的原因，而不是只看成功。常见原因如 unsafe dependent memory operations、cannot identify array bounds、call in loop、control flow not understood。把这些原因翻译回源码，就能指导接口和布局改造。

ASCII 字节分类从 reference 开始 SIMD 章节最好的入口不是指令名，而是一个可完整定义的字节分类任务。输入是一段日志文本，输出是每个字节是否为数字、分隔符、换行或非法字节。标量 reference 逐字节判断，遇到非法字节记录位置，遇到换行更新行号。这个版本可能不快，但它定义了空输入、尾部、错误位置、非 ASCII 字节和连续分隔符的语义。没有 reference，SIMD 版本就没有校验对象。

接下来判断哪些部分能批量化。单字节类别判断只依赖当前字节，适合 SIMD；行号维护依赖前面换行数量，需要把 mask popcount 变成累计值；引号内分隔符识别依赖状态，不能简单把每个分隔符都当字段边界。这样拆开后，SIMD 只处理它擅长的子问题：批量比较、生成 mask、统计或输出候选位置；复杂语义仍由状态机消费候选结果。这个分层比“用 AVX 扫字符串”更准确。

尾部和错误位置要优先测试 字节分类的尾部很容易出错。长度为 0、1、lane 宽度减一、正好 lane 宽度、lane 宽度加一，都要覆盖。若用掩码加载，必须保证不会越界读；若用标量尾循环，必须保证主循环和尾循环没有重复或漏掉；若用 padding，必须保证 padding 字节不会被报告成真实错误或分隔符。错误位置也要精确：SIMD 一次发现多个非法字节时，是报告第一个、全部还是只计数，必须和 reference 一致。

很多性能 bug 来自“测试只用了长且规整的输入”。真实日志可能很短，可能没有换行，可能最后一行不完整，可能混入二进制字节。SIMD 内核要把这些情况当作常规输入，不是边缘装饰。项目的测试矩阵应该强制每个 backend 跑同一组输入，包含 scalar、auto-vectorized 和 native backend。

水平归约和多累加器 逐元素 map 很适合 SIMD，但归约需要 lane 间合并。比如统计数字字符数量，向量比较得到 mask 后，可以 popcount mask；统计多个类别时，可以分别累计多个 mask。若把每次 popcount 结果都加到同一个标量计数器，会形成依赖链；更好的方式是按块局部累计，或者使用多个独立累加器，最后合并。这个思路和多线程局部 partial 完全一致。

浮点归约还要处理数值语义。向量合并顺序和串行顺序不同，结果可能不逐位一致。若项目里的计算是整数计数，使用固定宽度无符号类型并检查溢出边界；若是浮点指标，报告要写清是否允许重排、FMA 和 fast math。SIMD 优化不能偷偷改变业务合同。

自动向量化失败时的排查顺序 如果编译器没有向量化，不要先写 intrinsics。先看优化报告。若失败原因是可能别名，检查输入输出是否可能重叠，是否能调整 API 表达只读输入和独占输出。若原因是调用不可内联，把分类函数改成 constexpr 表或内联逻辑。若原因是循环控制流复杂，把错误路径拆出热循环。若原因是成本模型不值得，比较不同输入长度，不要强迫小输入走重路径。

反汇编用于验证执行形态。报告说向量化，不代表生成了理想代码；可能有运行时别名检查，可能尾部代码很重，可能使用了窄向量。报告说没有向量化，也不代表函数慢，可能瓶颈在内存带宽或调用层。学习 SIMD 的关键是把源码语义、编译器证据和硬件行为连起来。

项目中的 VectorizationPlan 每个候选内核都要写 VectorizationPlan。它包含 reference 名称、批量语义、输入输出别名规则、尾部策略、对齐假设、数值规则、平台 backend、fallback 和测试矩阵。比如 ASCII 分类内核的计划会写：输入只读，输出 mask buffer 独占；尾部使用标量；不要求对齐；非法字节报告第一个位置和总数；默认 backend 为自动向量化，可强制 scalar；native SIMD 只在测试通过后启用。

这份计划让 SIMD 不再污染业务层。业务代码调用的是分类接口，不知道 AVX、NEON 或 SVE。dispatch 层根据机器能力选择实现，测试层可以强制每个实现。若某个平台出现 bug，可以关闭对应 backend，reference 仍可运行。这样的工程边界会在第三册 AI 算子中继续使用。

事故复盘：SIMD 主循环正确但尾部错了 手写 SIMD 最常见的事故不是编译不过，而是主循环很快，尾部、错误位置或边界长度悄悄错。输入长度刚好等于 lane 数时通过，长度为 lane 减一或 lane 加一时失败；标量版报错在第 17 字节，向量版只知道某个 32 字节块有错；字符串里出现转义和引号以后，批量分类和语法状态脱节。向量化如果没有标量语义做锚点，只会把 bug 批量放大。

推导顺序应先拆语义。哪些判断是逐字节独立的，例如分隔符候选、数字字符、空白字符；哪些

判断依赖前文状态，例如引号内外、转义、跨块字符串。独立部分适合 SIMD 批量产生 mask，状态相关部分可能需要标量确认或分块 carry。Mask、tail policy、alignment、fallback、dispatch 都来自这些语义边界，不是看到 for 循环就写 intrinsics。

实验要覆盖 0、1、lane 减一、lane、lane 加一、非对齐地址、错误密集输入和随机输入。小输入变慢不一定说明 SIMD 错，可能是 dispatch 成本超过收益；大输入变快也不代表语义完整，必须比较错误位置、字段数和 checksum。自动向量化报告、反汇编和 perf 只能回答机器是否生成了批量指令，不能替代 reference。SIMD 章节要训练的是“先证明可批量，再选择指令”。

向量化实验判读 SIMD 实验先看边界长度：0、1、lane 减一、lane、lane 加一是否全部通过。再看错误位置和标量 reference 是否一致。性能上，小输入慢不代表 SIMD 错，可能是 dispatch 和尾部成本；大输入不快也不代表指令没用，可能已经受带宽限制。浮点结果不同必须写误差合同，不能只写“略有误差”。

这一段也服务于复习。读者合上书后，应该能凭本章的判读口径重新审查自己的代码：有没有最小正确版本，有没有能击穿旧方案的输入，有没有状态记录，有没有资源指标，有没有失败后的恢复动作。若五个问题缺两个以上，就说明实现还停在演示层，而不是工程层。

字节分类的尾部测试矩阵 字节分类内核的测试矩阵应单独列出长度：0、1、2、lane 减一、lane、lane 加一、两倍 lane 减一、很大长度。每个长度再组合非对齐起点、非法字节、第一个字节非法、最后一个字节非法、连续分隔符和无换行结尾。主循环再漂亮，也必须通过这些输入。

mask 的语义也要写清。一个 mask 可以表示哪些字节是分隔符，也可以表示哪些字节非法，还可以表示候选换行位置。后续标量状态机消费 mask 时，必须知道 mask 对应原始偏移。若 SIMD 版本只统计数量，却业务需要第一个错误位置，它就不是等价实现。

这个案例会直接服务第三册算子开发。AI 算子里也有批量语义、尾部、对齐、误差和 backend dispatch。第二册在字节分类中练清这些边界，第三册进入量化矩阵乘时就不会把向量指令当作神秘开关。

实验课：ASCII 分类从标量 reference 到 backend dispatch 标量 reference 逐字节判断数字、分隔符、换行和非法字符，并返回第一个错误位置、错误总数和分隔符位置列表。这个版本定义所有边界：空输入、无换行结尾、非 ASCII、连续分隔符、尾部不足一个 lane。然后写一个自动向量化友好的版本，不使用平台 intrinsics，只让循环、数据和别名关系更清楚。

第三步才写 native backend。它一次处理多个字节，生成 mask，尾部用标量收尾。公开 API 只接受输入 span 和输出对象，backend 通过配置选择；业务层不知道 AVX、NEON 或其他 ISA。测试强制运行 scalar、auto 和 native 三个 backend，所有 backend 使用同一组输入。若 native backend 不支持当前平台，就明确走 fallback。

判读时先看正确性矩阵。长度 0、1、lane 减一、lane、lane 加一；非法字节在首尾；非对齐输入；输出缓冲正好大小；这些都通过后才看性能。性能报告区分小输入和大输入，小输入可能被 dispatch 成本淹没，大输入可能受内存带宽限制。若只在理想长度上测试，SIMD 代码不可靠。

最后补浮点归约对照。整数字节分类一般可以逐位一致，浮点归约不一定。报告要说明是否允许重排，是否使用 fast math，误差如何比较。这个习惯会直接迁移到第三册量化和推理算子。

SIMD 练习验收重点 合格答案先写 scalar reference 和批量语义，再讨论指令。必须列出尾部策略、错误位置语义、对齐假设、fallback 和测试矩阵。浮点题要写误差合同；整数题要写溢出合同。只给 intrinsics 代码，没有 reference 和尾部测试，不算完成。

答案还要写“为什么不采用更复杂方案”。例如没有证据证明队列是瓶颈，就不应把锁队列换成无锁；没有证明输出顺序可变，就不应随意重排；没有证明错误路径能恢复，就不应只提交正常路径。能解释不做什么，说明读者开始具备工程判断。

最后，答案必须包含可复查材料。至少有输入摘要、关键状态、期望输出、实际输出、运行命令和未验证边界。没有这些材料，老师或同伴无法判断结论是否成立。第二册的作业不是让读者写漂亮段落，而是训练读者把系统问题变成可验证证据。

项目实现：SIMD、指令级并行与向量化 本章对应的贯穿项目实现不要求一次写成完整框架，但必须有最小可运行切片。这个切片包含三个部分：一个 reference，一个带本章机制的实现，一个能击穿 reference 之外隐含假设的测试。Reference 用来固定语义，机制实现用来展示本章知识，测试用来证明新增复杂度确实解决了问题。

代码组织上，不要把所有内容放进一个大文件。基础类型、输入生成、核心机制、报告输出和测

试应分开。即使当前只是教学项目，也要让目录能表达责任边界。这样后续章节接入时，不会因为找不到状态归属而重新发明一套对象。

每个实现都要输出一段机器可读摘要。摘要至少包含输入规模、关键状态、错误数、输出校验、耗时和未验证边界。这里的机器可读不要求复杂格式，简单的 key value 文本也可以。关键是让脚本和读者都能复查，而不是只能看屏幕截图。

测试应包含正常路径、边界路径和失败路径。正常路径证明功能可用；边界路径证明长度、容量、尾部、空输入、关闭或超时没有遗漏；失败路径证明本章机制不是装饰。若失败路径写不出来，往往说明本章概念还没有落到工程对象上。

报告最后写一个取舍段。新增机制带来了什么好处，增加了什么成本，哪些场景不应该使用它，下一章会复用哪个字段或对象。这个取舍段非常重要，因为系统工程不是把所有高级机制都加进去，而是在证据和约束下选择合适的边界。

调试笔记：SIMD 失败多半在主循环之外 SIMD bug 常发生在尾部、对齐、错误位置和 fallback。排查时先强制 scalar backend，确认 reference；再强制 auto backend；最后强制 native backend。若只有 native 失败，缩小到长度矩阵和尾部策略。若所有 backend 都失败，说明批量语义或 reference 定义错了。不要一开始盯着向量指令，先把等价性边界查清。

复查清单：SIMD、指令级并行与向量化 复查本章时，先检查 reference 是否仍然存在。没有 reference，后面的并发、优化、异步或分布式改造都无法证明语义没有变。再检查压力输入是否尖锐：它应当攻击本章最核心的边界，而不是只把数据量放大。第三，检查状态是否可观察：关键对象的身份、状态、错误和提交结果应能在报告中找到。

第四，检查代码示例是否说明禁止行为。调用者不能做什么，往往比能做什么更能体现工程质量。第五，检查实验是否写出未验证边界；当前设备、权限或输入无法证明的结论，必须标注。第六，检查本章对象是否能被下一章复用，不能复用就说明边界还不稳。

完成这六项后，再看文字是否连贯。每个概念都应从问题推出，每个案例都应从朴素方案走到改进方案，每个实验都应能解释一个判断。若某段话放到其他章节也成立，就把它改成本章的具体对象、具体输入和具体失败。

7.9 工程案例：从标量解析语义推导 SIMD 版本

SIMD 最容易被误讲成“把 8 个或 16 个元素一起算”。真实工程中，难点通常不在指令本身，而在标量语义是否能批量化：是否有跨元素状态，尾部如何处理，错误如何报告，输出位置如何分配，数值结果是否允许改变。只有这些问题先清楚，向量化才是正确优化，而不是把 bug 做宽。

一、标量 reference 必须足够严格 以日志分隔符扫描为例，标量 reference 要定义逗号、换行、引号、转义和非法字符的语义。若引号内逗号不算分隔符，SIMD 版本不能简单比较所有字节是否等于逗号；若反斜杠转义影响下一个字符，状态会跨字节传播；若错误报告要求第一处错误位置，批量扫描必须保留顺序信息。reference 越清楚，SIMD 改写越安全。

很多 SIMD 教学例子选择无状态数组变换，这是必要入门，但不足以覆盖系统项目。日志解析、JSON 扫描、CSV 解析、UTF-8 校验都涉及状态传播。工程上常先用 SIMD 找候选位置，例如引号、反斜杠、逗号、换行，再用标量或位运算处理状态。这样不是完全替代标量，而是把批量能力用在最适合的子问题上。

二、掩码是批量控制流 向量比较产生掩码，掩码再用于选择、压缩、计数或查找第一个匹配位置。理解掩码，比背具体指令更重要。一个 32 字节块中哪些位置是逗号，哪些位置是换行，可以用位图表示；块内分隔符数量可以用 popcount；第一个分隔符可以用 countr zero；输出位置需要 scan 或 prefix popcount。

掩码让控制流从“每个字符一个分支”变成“每个块一个位集合”。这能减少分支失败，但也带来新问题：跨块状态怎么传，块尾不足如何处理，错误位置如何从掩码恢复到全局偏移。教材要把掩码和输出位置讲清楚，否则 SIMD 代码只会像魔法。

三、尾部处理是语义的一部分 数组长度不一定是向量宽度的倍数。尾部可以用标量处理，也可以用 masked load，也可以在输入末尾填充哨兵字节。每种方式都有边界。标量尾部简单可靠，但有额外分支；masked load 依赖指令集；填充哨兵要求输入 buffer 后面有安全空间，不能越过映射页或文件末尾。

尾部错误也要测试。空输入、长度 1、长度等于向量宽度减一、等于向量宽度、等于向量宽度加

一、最后一个字节是转义、最后一行无换行，这些都应进入测试集。很多 SIMD bug 不出现在大输入，而出现在尾部和页边界。

四、自动向量化需要源码配合 编译器自动向量化依赖别名、对齐、循环边界、数据依赖和异常路径。若输入输出可能重叠，编译器会保守；若循环内部有函数调用或复杂分支，向量化可能失败；若数据结构不是连续数组，编译器没有可向量化形状。源码要用清楚的 `span`、`restrict` 等价信息、简单循环和独立输出帮助编译器。

优化报告是必读材料。看到“loop not vectorized”不等于失败，而是线索。报告会告诉你是否存在依赖、无法证明别名、循环控制复杂，还是成本模型认为不值得。读者应先尝试让自动向量化成功，再决定是否手写 `intrinsics`。手写 SIMD 维护成本高，只有在自动向量化无法表达关键模式或收益足够明确时才值得。

五、手写 SIMD 要多版本派发 不同机器支持不同指令集。x86 有 SSE、AVX2、AVX-512，ARM 有 NEON 和 SVE。项目不能只在开发机器上写一个版本就结束。较稳妥的结构是：标量 `reference` 永远存在；`portable fast path` 作为默认；平台特化版本通过编译期或运行时派发选择；所有版本共享同一测试矩阵。

多版本派发还要考虑二进制体积和启动成本。若内核很小，派发开销可能不可忽略；若输入很短，标量可能更好；若平台特化很难维护，收益不稳定就不应进入主线。SIMD 不是信仰，而是一个带维护成本的实现选择。

六、数值语义不能被向量化偷偷改变 整数数组变换通常容易保持语义，浮点归约则不同。向量化和并行化会改变加法顺序，导致舍入差异。若业务要求 `bitwise identical`，就要固定顺序或使用补偿算法；若允许误差，就要在合同中写明误差范围和测试方法。不要让编译器的 `fast math` 选项悄悄改变 NaN、无穷、舍入和有符号零语义。

这个问题会在 AI 算子开发中继续出现。矩阵乘、归约、归一化都涉及数值误差和确定性。第二册先在普通高性能计算里讲清楚，第三册进入 AI 时才不会把“快”放在“数值合同”前面。

七、SIMD 和线程的组合顺序 同一个内核可以先向量化再多线程，也可以先多线程再向量化。工程上通常先固定单线程标量语义，再优化单线程批量执行，最后扩展到多线程。这样每一步的收益和 bug 都容易归因。如果同时改 SIMD、线程和布局，结果变快也不知道原因，结果变错更难定位。

线程会改变 SIMD 收益。单线程向量化后，内核可能从计算受限变成内存带宽受限；再加线程时，带宽更快到顶。报告要写瓶颈是否迁移。若 SIMD 版本已经打满内存带宽，多线程收益有限不是失败，而是系统达到另一层边界。

八、实验交付：VectorKernelMatrix 本章实验包括四类输入：普通数组变换、字节分类、稳定 `filter` 前的 `mask` 生成、浮点归约。每类至少有标量 `reference`、自动向量化版本、手写或平台特化版本。报告记录指令集、编译选项、向量化报告、`checksum`、尾部测试和不同长度下的性能。

验收要特别看失败输入。字节分类要测引号和转义跨块；`filter mask` 要测全保留、全丢弃和稀疏保留；浮点归约要测极大极小混合、NaN 和不同合并顺序。SIMD 章节的质量，不在于代码看起来底层，而在于批量语义和边界被证明清楚。

7.10 源码阅读：从编译器向量化报告反推代码边界

SIMD 学习不能只读 `intrinsics` 文档。很多时候，编译器已经能生成不错的向量代码；另一些时候，它拒绝向量化，并给出非常有价值的理由。读向量化报告，是理解代码边界的低成本入口。它会告诉你依赖、别名、控制流、成本模型和目标指令集怎样共同决定是否向量化。

一、别名信息决定编译器能否放心重排 若函数接受两个裸指针，编译器可能无法证明输入输出不重叠。它必须保守地按标量顺序生成代码，避免写输出影响后续读输入。使用 `std::span` 不能自动消除别名，但清晰接口、`const` 输入、独占输出和局部创建输出容器能帮助人和编译器理解边界。某些场景可以用编译器扩展表达 `restrict`，但要谨慎封装。

报告中出现 `alias` 相关信息时，不要立刻手写 SIMD。先看接口是否过宽：输入输出是否本来就不应重叠，是否可以拆成只读输入和独占输出，是否可以先复制小的控制表，是否可以把状态写入局部 `partial`。很多向量化失败其实是 API 语义没有表达清楚。

二、循环携带依赖要从算法上拆 编译器不能向量化真正的循环携带依赖。例如前缀和每个元素依赖前一个输出，普通方式不能直接变成独立 `lane`。解决不是强迫编译器，而是改算法：分块局

部 scan, 再 scan block sums, 再回填。这个思想和第 13 章一致。向量化报告指出依赖时, 读者要判断它是真依赖还是可以通过算法改写消除的依赖。

日志解析中的引号状态是真依赖的一部分, 但字符分类不是。可以先 SIMD 找出候选特殊字符, 再用标量状态机处理候选区间。这样把可批量部分和顺序状态部分分开。高质量 SIMD 不是所有东西都向量化, 而是把向量用在合适的子问题上。

三、成本模型拒绝向量化时要看输入规模 编译器有时认为向量化不值得, 因为循环太短、额外准备成本高、可能需要复杂尾部处理。对于小输入, 这可能是正确决定; 对于项目中的大 batch, 成本模型可能保守。报告要结合输入规模判断。若循环在真实 workload 中很长, 可以尝试手动改写; 若大部分输入很短, 标量路径保留更重要。

多版本策略可以按长度选择。短输入走标量, 长输入走向量。阈值由 benchmark 决定, 而不是由向量宽度拍脑袋决定。报告记录阈值、输入长度分布和平台差异。这样 SIMD 优化才不会伤害短请求尾延迟。

四、反汇编验证 lane 宽和尾部 优化报告说 vectorized 后, 还要看反汇编确认使用了什么指令、lane 宽多少、是否有 peel loop、是否有 scalar epilogue。某些情况下编译器会生成很宽的向量但降频明显, 或者生成复杂 gather 导致收益不佳。反汇编和运行结果结合, 才能判断向量化质量。

尾部路径也要看。若主体向量很快, 但尾部标量包含复杂分支, 小输入可能主要走尾部。若 masked tail 访问越界风险没有处理, 代码可能依赖未定义行为。反汇编不是只看主体循环, 也要看入口、对齐处理和尾部。

五、向量化报告交付 本章要求每个向量优化提交保存三份材料: 编译器报告、关键反汇编、输入长度分布。报告解释为什么编译器接受或拒绝向量化; 反汇编确认实际指令; 输入分布说明该优化覆盖多少真实数据。缺少任一材料, 结论都不完整。

读者做完这个练习后, 会更少把 SIMD 当成神秘技巧。它本质上是把批量语义、编译器证明和硬件执行对齐。能对齐时收益很大; 不能对齐时, 保留清晰标量版本才是正确选择。

7.11 补强案例: SIMD filter 和 scan 的连接

SIMD 常先用于生成 mask, 而真正写输出还需要 scan。以 filter 为例, 向量比较能一次判断多个元素是否保留, 得到一个 mask; 但每个保留元素写到哪里, 仍然取决于它之前有多少元素被保留。这就把 SIMD 章节和并行 scan 章节连接起来。

一、块内 mask 到块内 rank 一个 32 lane mask 可以用 popcount 得到保留数量, 用 prefix popcount 得到每个 lane 的局部 rank。支持压缩存储的指令集可以直接 compress store, 不支持时可以查表或标量写保留元素。无论实现方式如何, 语义都是把 mask 转成输出位置。

二、块间 offset 来自 scan 块内 rank 只给出块内位置, 块间起点需要对每个块的保留数量做 scan。这样 SIMD filter 的完整结构是: 向量生成 mask, 计算每块 count, scan 块 count, 按块 offset 写输出。它不是单独 SIMD 技巧, 而是批量执行和并行算法的组合。

三、稀疏和密集命中策略不同 命中率低时, 输出很少, 压缩写收益明显; 命中率高时, 几乎所有元素都保留, 直接复制或简单 map 可能更快; 命中率中等时, mask 和 scan 成本需要测量。输入分布决定策略。报告必须覆盖全丢弃、全保留和随机命中。

四、实验 实现标量 stable filter、SIMD mask 加标量写、SIMD mask 加块 count 和 scan。比较不同命中率、不同长度和不同尾部。输出必须和 reference 完全一致。这个实验会让读者看到 SIMD、scan、输出位置和稳定性怎样组合。

7.12 补充案例: 平台差异和降级路径

SIMD 代码必须面对平台差异。一本面向计算系统的教材不能只在 x86 AVX2 上写通, 也要说明 ARM NEON、不同向量宽度、不同编译器和无 SIMD fallback 的路径。

一、标量路径永远保留 标量 reference 不只是测试 oracle, 也是运行时 fallback。未知平台、短输入、禁用特化、调试模式都可以走标量。手写 SIMD 版本必须和标量共享语义测试。删除标量路径会让移植和排错困难。

二、运行时派发要可测试 运行时检测 CPU feature 后选择实现。测试不能只测当前机器支持的最快路径, 也要强制选择标量和较低级路径。可以通过环境变量或构建选项覆盖派发。否则某个

fallback 可能长期坏掉。

三、向量宽度不应进入业务语义 业务代码不应假设一次处理 16 或 32 个元素。向量宽度属于实现细节。接口只表达输入 span 和输出合同。内部可以按平台选择块宽，尾部处理保证所有长度正确。

四、实验 在同一机器上强制标量、自动向量化、手写特化三种路径。若有 ARM 设备，再比较 NEON。报告写出平台、指令集和 fallback 是否通过。这样 SIMD 章节为用户的 Termux 环境也留下合理路径。

7.13 从批量语义推导向量化

SIMD 常被误解成“用更宽的寄存器一次算多个数”。这句话只描述了表面。真正的前提是程序具有批量语义：多个元素之间没有必须串行等待的依赖，内存访问能按规则组织，尾部元素能被定义清楚，异常和数值误差有可接受边界。若这些语义没有先写清，向量代码就会变成难以审查的特殊版本。向量化章节要先讲为什么可以批量处理，再讲如何让编译器或手写 intrinsic 表达这种批量。

一、先证明 lane 之间独立 过滤、加法、乘法、阈值比较这些操作通常可以按 lane 并行，因为第 i 个元素的结果不依赖第 $i-1$ 个元素。前缀和、递归滤波、状态机解析则不同，后一个结果可能依赖前一个状态。读者看到循环时，第一步不是问“能不能用 AVX”，而是画出每次迭代之间的数据依赖。若依赖只在最后归约，可以用多个 partial；若依赖是前缀结构，可以用 scan；若依赖来自少数状态，可以考虑分块和边界状态。不同依赖形状对应不同算法，不是统一套一个向量模板。

二、内存形状决定向量质量 连续加载最适合 SIMD，步长访问次之，gather/scatter 成本更高，随机指针追逐最难。若数据布局仍是大结构体，而热循环只处理其中一个字段，向量加载会浪费带宽；若改成列式数组，lane 可以自然对应连续元素。对齐也要具体分析：现代处理器能处理未对齐加载，但跨 cache line 或页面仍可能增加成本。尾部处理同样是语义问题：用标量收尾、掩码收尾还是补齐输入，取决于输出是否允许额外元素、是否需要异常一致、是否追求固定路径。

三、自动向量化报告要逐条读 编译器没有向量化，通常会给出原因：可能是别名不明、循环边界不清、调用不可内联、存在循环携带依赖、内存访问非连续或浮点语义太严格。读者应把这些原因当成设计反馈。若是别名问题，可以调整接口为 `std::span<const float>` 和 `std::span<float>`，并保证输入输出不重叠；若是调用问题，可以把小函数内联或改成查表；若是浮点问题，要决定是否允许重排。不要一看到编译器没向量化就直接手写 intrinsic，先理解它为什么拒绝。

四、数值语义不能被速度掩盖 整数加法在不溢出的前提下容易重排，浮点加法不同。向量化和多累加器会改变合并顺序，结果最后几位可能变化。某些业务只要求误差范围，某些业务要求 bitwise reproducible。两者都合理，但必须写进合同。若要求完全可复现，可以固定归约树或使用更高精度中间值；若允许误差，要给出误差度量和测试输入。性能优化不能偷偷改变数值承诺。

五、可移植性是向量代码的组成部分 手写 AVX2、AVX-512 或 NEON 代码时，要准备标量 fallback 和特性检测。否则代码只能在作者机器上运行。接口层应把语义和实现分开：同一个 `filter_scores` 可以有标量版本、自动向量化版本和平台 intrinsic 版本；测试先比较语义，再比较性能。代码审查要检查尾部、对齐、别名、异常、数值误差和平台分派，而不是只看是否用了宽寄存器。向量化的最终目标是可维护的批量执行，不是展示指令名称。

7.14 尾部、掩码和异常路径的工程细节

向量化示例经常只展示能被向量宽度整除的输入，这会让读者误以为 SIMD 的主要难点是写出一条宽指令。真实工程里，尾部元素、条件掩码、异常路径和别名边界才是审查重点。一个处理 N 个元素的函数，必须在 N 小于向量宽度、 N 不是向量宽度倍数、输入输出重叠、出现非法值时仍然定义清楚。性能代码越底层，边界越不能模糊。

一、尾部处理有三种常见策略 第一种是主循环处理完整向量，剩余元素交给标量循环。这种方式最清楚，适合教学和多数业务。第二种是用掩码加载和掩码存储处理尾部，可以减少分支，但要求平台支持，且要确认 masked-off lane 不会触发非法访问。第三种是输入补齐，让数据长度向上对齐，尾部填充中性元素。补齐适合批处理和张量计算，但会改变内存占用和边界检查。选择哪一种，取决于平台、输入大小分布和错误处理要求。

二、掩码是数据化的分支 阈值过滤、条件赋值、取最大值都可以把分支变成 mask。mask 不

是魔法，它只是把“哪些 lane 有效”变成向量寄存器中的数据。若条件随机，mask 往往比分支更稳定；若条件高度可预测，分支可能更便宜；若两边工作量差异很大，mask 可能执行了大量本来不会执行的计算。报告要用输入分布解释 mask 的收益，而不是把“无分支”当成必然更快。

三、压缩输出需要 scan 或专用指令 向量过滤最难的不是比较，而是把保留元素紧凑写出。简单 mask store 会在原位置留下空洞，若要 compact，就需要根据 mask 计算每个有效 lane 的输出偏移。某些平台提供 compress store，某些平台需要查表或局部 prefix。这里和并行 scan 章节相连：输出位置来自前面保留了多少元素。把这个连接讲清楚，读者就能理解为什么 SIMD filter 不是几条比较指令就结束。

四、异常和非法值要保持语义一致 如果标量版本遇到非法输入会记录错误，向量版本也必须处理。常见做法是主向量路径先快速检测是否存在异常 lane，若没有异常就走快路径；若存在异常，再退回标量或慢路径逐个处理。这样正常输入保持高吞吐，异常输入仍有正确语义。不能为了向量化直接跳过错误检查，否则优化改变了业务合同。

五、向量代码审查表 审查一个向量函数时，至少检查七点：输入长度为零是否正确，长度小于向量宽度是否正确，尾部是否越界，输入输出是否允许重叠，浮点重排是否被允许，异常路径是否和标量一致，平台 fallback 是否存在。只有这些问题都有答案，向量代码才算工程代码。否则它只是某个输入上的演示。

7.15 贯穿案例：一个 SIMD 过滤器为什么把错误位置弄丢了

向量化最容易被讲成“把八个元素放进一个寄存器”。这句话没有错，但离工程还很远。考虑一个日志过滤器：输入是解析后的事件记录，规则是保留时间范围内、级别不低于阈值、事件名合法的记录。标量 reference 会在遇到非法事件名时记录精确 record id 和错误字节位置。手写 SIMD 版本先按字节批量比较，生成 mask，再把通过的 lane 写入输出。它在规则输入上很快，却在错误输入上只报告“某个向量块有错误”，甚至把块尾部的非法字节归到下一条记录。速度提升是真的，语义退化也是真的。

这个事故说明，向量化不是先选 AVX2、NEON 或 AVX-512 指令，而是先拆语义。哪些判断是逐元素独立的，哪些判断需要跨元素状态，哪些错误必须精确定位，哪些输出必须保持顺序，尾部元素如何处理，平台不支持时如何 fallback。只有这些问题答清，宽寄存器才是实现方式。

一、先把标量 reference 写成合同 Reference 不只是慢版本。它定义非法事件名如何报告、空输入如何返回、阈值边界是否包含、输出是否保持输入顺序、重复记录是否保留。向量版本必须先满足这份合同。若向量化后只返回块级错误，等于改变错误语义；若压缩输出改变稳定顺序，也要确认业务是否允许。性能优化没有资格偷偷修改合同。

二、局部判断批量化，跨记录状态保留标量确认 事件级别、时间戳范围、整数 key 是否在区间内，通常是逐记录独立判断，适合 SIMD 生成 mask。事件名合法性可能需要查字典、处理长度和字符编码，未必完全适合主向量路径。稳妥方案是：向量路径快速筛出候选，通过 mask 记录哪些 lane 需要慢路径确认；慢路径再用标量 reference 精确报告错误。这样普通输入快，异常输入仍保持语义。

三、压缩输出要借助 scan 思想 比较得到 mask 后，输出位置不是自然存在的。若直接按原位写，输出有空洞；若要紧凑写，需要知道当前 lane 之前有多少有效 lane。某些平台提供 compress store，其他平台要用查表或局部 prefix。无论实现如何，语义都和 scan 一样：根据前缀有效数量计算输出位置。第 7 章和第 13 章在这里连接起来，SIMD filter 不是几条比较指令，而是一套输出位置协议。

四、尾部处理优先选择清晰方案 输入长度小于向量宽度、等于向量宽度减一、刚好等于向量宽度、超过一位，这些都要测。教学实现优先使用标量尾部，哪怕慢一点，因为语义最清楚。Mask load 和 padding 都可以优化尾部，但各有前提：mask load 要平台支持，padding 要保证额外可读字节，且不能把 padding 字节当真实输入报告错误。尾部不是小角落，它经常是向量代码最容易越界的地方。

五、平台分派不能污染内核语义 同一个过滤接口可以有 scalar、auto-vectorized、AVX2、NEON 实现。分派应在外层完成，热循环只处理数据。测试先比较所有实现与 reference 的输出和错误报告，再比较性能。若某平台没有对应指令，fallback 仍必须正确。工程中的 SIMD 不是写一份作者机器能跑的代码，而是维护一个语义相同、实现可替换的内核族。

六、向量化报告要写“没有向量化”的理由 有些循环不适合直接向量化：前缀和有跨元素依赖，哈希表更新有随机写和冲突，状态机解析有跨字节状态，浮点 reduce 改变合并顺序。报告应写出为什么没有直接 SIMD，以及是否通过算法重写把它变成可批量部分。能解释“不做”的理由，和能写出 intrinsics 一样重要。系统工程不是追求每个循环都向量化，而是把可批量语义和不可批量语义分清。

7.16 案例深化：向量化前的语义分层

SIMD 章节要避免给读者一种错觉：只要看到循环就应该写 intrinsics。更稳的入口是一个失败案例。标量解析器能准确报告错误字节位置，手写 SIMD 版本只返回“某个向量块有错误”；标量版本能处理跨块引号状态，SIMD 版本把块内字符独立分类后误判。这个案例说明向量化不是先选指令，而是先把语义分成可批量部分和需要状态传播的部分。

一、先找纯局部判断 字符是否为逗号、换行、数字、空白，通常可以逐字节独立判断。这部分适合 SIMD 生成 mask。Mask 的语义要写清：一位对应一个输入字节，1 表示候选分隔符或候选错误。只要 mask 定义清楚，后续可以用不同指令集实现同一语义。这样 portable reference、自动向量化和手写 SIMD 之间有共同合同。

二、再处理跨字节状态 CSV 引号、转义、UTF-8 多字节边界、行内状态都不是纯局部判断。可以先用 SIMD 找候选位置，再用标量状态机确认；也可以为每个块计算 carry in 和 carry out，让状态跨块传播。无论哪种方案，都要说明块边界如何处理。很多 SIMD bug 都出在尾部和跨块状态，而不是主循环。

三、尾部策略是语义的一部分 输入长度不是 lane 倍数时，尾部可以走标量 fallback，也可以用 mask load，也可以读到安全 padding。三种策略性能不同，安全条件也不同。Padding 需要分配器保证额外可读字节；mask load 需要目标平台支持；标量 fallback 最清楚但有分支。教材示例应先用标量尾部保证正确，再说明何时值得优化。

四、dispatch 不应污染热路径 多平台代码常根据 CPU 特性选择 AVX2、NEON 或标量版本。Dispatch 应在外层完成，不要每个小块都判断一次。函数指针、模板实例或策略对象都可以使用，但要让热循环只处理数据。报告要分开测 dispatch 成本和内核成本，小输入场景下 dispatch 可能主导时间。

五、验证要覆盖语义边界 测试集合要覆盖空输入、短输入、刚好一 lane、跨 lane 的引号、错误字节在尾部、非对齐地址和随机数据。性能集合则要分短行、长行、规则字段、随机字段。若只拿大块规则输入，SIMD 很容易显得完美。高质量向量化教材应让读者先害怕边界，再相信速度。

7.17 案例复盘：向量化先证明批量语义

手写 SIMD 主循环很快，但尾部错、错误位置错、跨块引号状态错，这类事故比编译错误更危险。向量化前要把语义分层：哪些判断逐字节独立，哪些判断需要前文状态，哪些边界必须回到标量确认。

mask 合同先于指令 逗号、换行、数字、空白可以批量生成 mask；引号内外、转义和 UTF-8 边界需要 carry 或标量确认。Mask 的每一位对应哪个输入字节，错误位置如何还原，尾部如何处理，都要成为合同。指令集只是这个合同的实现。

实验判据 测试覆盖空输入、短输入、lane 减一、lane、lane 加一、非对齐、错误在尾部和跨块状态。性能报告分 dispatch 成本和内核成本。小输入慢不说明 SIMD 错，大输入快也不证明语义完整。

7.18 本章习题

习题与作业

习题一：选择三个循环：数组加法、前缀和、直方图。分别写出它们的迭代依赖、写入位置和向量化难点。说明哪个适合直接向量化，哪个需要算法重写，哪个需要分阶段处理。
习题二：为 `add_arrays_scalar` 设计 debug 版别名检查。说明哪些重叠是安全的，哪些重叠会破坏语义。不要依赖非标准 `restrict` 作为唯一方案。

习题三：运行或设计一次自动向量化报告实验。记录编译器命令、报告内容、是否发生 vectorization、失败原因和汇编证据。若当前环境不支持某个报告选项，写出替代观察方式。

习题四：为浮点归约写误差说明。比较串行求和、四累加器求和、pairwise 求和和 fast-math 下的可能差异。说明你的业务场景是否允许改变合并顺序。

典型计算内核模式

先看一个组合错误：团队把日志按 chunk 做局部 top-k，每个 chunk 保留前一百个 key，最后再合并所有候选。小数据能跑，线上却漏掉了一个总量很高但分散在很多 chunk 里的 key。单个局部 top-k 没错，组合后错了，因为局部候选集合没有保留足够信息。这个事故比列出 map、reduce、scan、filter 更重要：计算内核不是名字表，而是输入形状、输出位置、合并合同和材料化边界的组合。

本章从这种内核组合问题出发。Map 回答逐元素变换能否独立执行；reduce 回答多个 partial 如何合并；scan 回答输出位置如何被并行分配；partition 回答冲突写如何变成确定区间；stencil、join、BFS 和排序回答数据访问形状如何改变成本。掌握这些模式，不是记住某个库函数，而是能从一个具体任务推导出它需要哪种数据流结构。

8.1 从基本内核到组合流水线

Map 对每个元素独立变换，通常容易并行和向量化。Reduce 把多个元素合成一个结果，存在合并顺序问题。整数求和通常结合律明确，浮点求和则会因为舍入误差导致不同合并顺序产生不同结果。

并行 reduce 的基本形态是每个线程处理一段输入，写入线程本地 partial，然后合并。这个结构减少共享写，也让每个线程顺序扫描自己的数据块。Compute Systems Labs 的 `parallel_sum` 就是这个最小模型。

Reduce 的关键不是“把加法拆给多个线程”这么简单，而是确定合并操作是否满足结合律、是否允许重排、partial 放在哪里、合并树长什么样。整数求和如果可能溢出，重排也可能改变 C++ 语义；浮点求和即使不溢出，也会改变舍入；字符串拼接满足语义结合时，内存分配可能成为主要成本。因此每个 reduce 都要同时写正确性模型和成本模型。

Map 的融合 多个 map 连在一起时，如果每一步都写出中间数组，会增加内存流量。融合把多个逐元素变换合在一次循环里，减少中间读写。融合不是总是好事：融合后循环体变大，寄存器压力可能增加，分支也可能更复杂。是否融合，要看减少的内存流量是否超过新增的执行压力。

Stencil 与邻域计算 Stencil 计算每个输出元素时读取邻域元素，例如图像卷积、有限差分和网格模拟。它的关键是复用邻域数据，避免重复从内存加载。分块、滑动窗口和边界处理是主要问题。

Stencil 的边界处理会影响代码形状。可以把内部区域和边界区域分开，让内部热循环没有复杂分支；也可以用 padding 或 ghost cell 让边界访问统一。前者代码路径多但热循环干净，后者内存多一点但逻辑规律。并行 stencil 还要处理块之间的 halo 数据，单机上表现为相邻块共享边界，分布式上表现为节点之间交换 halo 消息。

矩阵和块算法 矩阵计算展示了数据复用的重要性。朴素矩阵乘的数学公式很简单，但访问顺序会决定是否复用缓存。分块算法把矩阵切成能放入缓存的小块，让一个块内的数据被多次使用。这个思想不仅用于矩阵，也用于排序、join、图处理和文件扫描。

块大小不是越大越好。太小会增加循环和调度开销，太大又放不进缓存。实际工程中常用 benchmark 或自动调优选择块大小，并针对不同 CPU 做参数化。

8.2 冲突、排序、Join 和图遍历

直方图看似简单：读一个元素，更新一个桶。但多个线程可能更新同一个桶，导致锁竞争或原子争用。常见优化是线程本地直方图，最后合并。这个模式体现了一个原则：把高频共享写变成低频合并。

直方图还会遇到数据倾斜。如果 key 分布均匀，线程本地桶很好；如果大量输入集中到少数桶，全局原子会退化，分片后合并也可能在少数桶上集中。工程实现常按线程、NUMA 节点或缓存块建

立局部桶，并在合并阶段使用连续扫描。若桶数量很大，本地直方图会占用大量内存，这时要在内存占用和冲突减少之间折中。

图遍历 图遍历通常局部性差、分支多、负载不均。压缩邻接表、按层处理、前沿队列和方向优化，都是为了改善访问模式和任务分配。图算法提醒我们，不是所有问题都能靠 SIMD 和连续数组轻松解决。

图遍历的性能问题往往同时包含三件事：邻接表访问随机，顶点度数不均，前沿大小变化剧烈。BFS 在某些层只有少量顶点，线程不够忙；在某些层前沿巨大，访问又非常分散。方向优化 BFS 会在 top-down 和 bottom-up 之间切换，目的就是根据前沿大小改变访问模式。这个例子说明，高性能算法不是固定套一个并行模板，而是随数据形状选择执行策略。

排序和分区 排序是数据系统的核心内核。快速排序、归并排序、基数排序和样本排序在不同场景下表现不同。并行排序的难点不只是比较次数，还包括分区均衡、临时空间、cache 行为和跨线程合并。分布式排序还会引入 shuffle 和热点分区。

内核选择清单 遇到一个新计算内核，可以按清单分析。第一，输入输出是否连续，是否能按块切分。第二，每个元素工作量是否接近，是否存在数据倾斜。第三，是否有跨元素依赖，依赖能不能分层处理。第四，写入位置是否唯一，是否会冲突。第五，是否能把全局共享写改成本地写加合并。第六，算术强度大概是多少，瓶颈更像计算还是内存。第七，单机算法扩展到分布式时，哪一步会变成 shuffle 或远程通信。

这份清单会在后面的并行算法和分布式计算里反复出现。它让读者从“我知道几个算法名字”转向“我能根据数据和依赖形状选择执行方式”。

核心思想

分析计算内核时先问四个问题：每个元素读写多少字节，是否存在跨元素依赖，写入是否冲突，工作量是否均衡。

内核模式不是算法名字 本章讲“内核模式”，不是为了让读者背 map、reduce、scan 这些词，而是训练一种分析方式。一个计算任务进入系统后，先不要急着选择线程池、SIMD 或分布式框架，而要先判断它的数据形状：输入是否连续，输出是否连续，读写比例如何，是否有跨元素依赖，是否存在冲突写，工作量是否均匀，能否按块处理，能否先局部处理再合并。

同一个业务问题可能拆成多个内核。日志统计可能先是 map：解析每行日志；再是 filter：丢掉无效记录；再是 partition：按 key 或时间分桶；再是 reduce：对每个 key 计数；最后是 sort：输出 top key。数据库执行器、搜索引擎、图计算和机器学习数据预处理都类似。理解内核模式，可以把一个看似复杂的系统拆成可优化、可测量、可验证的阶段。

Compute Systems Engine 后续的分布式 shuffle 也会复用这些模式。map 端解析和本地聚合，shuffle 前按 key 分区，reduce 端合并，输出前排序或 top-k。单机内核和分布式执行不是两本书的内容，而是同一组模式在不同尺度上的实现。

Map：最容易并行，也最容易被内存带宽限制 Map 的特征是每个输入元素独立产生输出。它通常最容易并行、向量化和分块，因为迭代之间没有依赖。朴素实现是一层循环。

Listing 8.1 baseline：逐元素 map

```
1 void scale_and_shift(std::span<const float> input,
2   std::span<float> output,
3   float scale,
4   float shift) {
5   const std::size_t count = std::min(input.size(), output.size());
6   for (std::size_t i = 0; i < count; ++i) {
7     output[i] = input[i] * scale + shift;
8   }
9 }
```

这个内核的正确性很简单：每个输出位置只依赖同一位置输入。性能上却要看字节数。如果每

个元素读 4 字节写 4 字节，只做一次乘法和一次加法，算术强度很低，输入很大时往往受内存带宽限制。此时 SIMD 可以减少指令数，但未必带来数量级提升。若 map 表达式很重，例如包含多项式、三角函数或复杂解析，计算可能成为瓶颈，向量化和函数近似才更重要。

Map 的常见优化是融合。若先对数组做 scale，再做 clamp，再做转换，每一步都写出中间数组，就会产生多轮内存读写。融合成一次循环可以减少内存流量。

Listing 8.2 融合多个 map，减少中间数组

```
1 void normalize_clamp(std::span<const float> input,
2   std::span<float> output,
3   float mean,
4   float inv_stddev) {
5   const std::size_t count = std::min(input.size(), output.size());
6   for (std::size_t i = 0; i < count; ++i) {
7     const float normalized = (input[i] - mean) * inv_stddev;
8     output[i] = std::clamp(normalized, -1.0F, 1.0F);
9   }
10 }
```

融合也有代价。循环体变大后，寄存器压力可能增加；若把多个不相关路径硬塞进一个循环，前端和分支预测可能变差；若中间结果可复用，融合反而丢掉复用机会。原则是：融合主要用于减少一次性中间读写，不是为了让代码看起来“更优化”。

Reduce：合并语义决定一切 Reduce 的重点是合并语义。一个 reduce 操作至少要说明单位元、结合律、交换律和重排是否允许。整数最大值很简单，单位元可以是最小可表示值，合并顺序不影响结果。浮点求和不严格满足结合律，合并树会影响舍入。字符串拼接满足结合律时仍可能被分配成本主导。哈希表聚合还要处理 key 冲突和内存分配。

朴素并行 reduce 的错误写法，是所有线程直接更新一个共享结果。

Listing 8.3 反例：reduce 中的共享写热点

```
1 void count_positive_bad(std::span<const std::int32_t> values,
2   std::atomic<std::int64_t>& total) {
3   for (const std::int32_t value : values) {
4     if (value > 0) {
5       total.fetch_add(1, std::memory_order_relaxed);
6     }
7   }
8 }
```

这个版本的结果可以正确，但高频共享写会让 cache line 所有权不断迁移。更好的结构是每个 worker 本地计数，最后合并。

Listing 8.4 线程本地 reduce，最后合并

```
1 struct alignas(64) CountPartial {
2   std::int64_t positive = 0;
3 };
4
5 void count_positive_range(std::span<const std::int32_t> values,
6   CountPartial& partial) {
7   std::int64_t local = 0;
8   for (const std::int32_t value : values) {
9     if (value > 0) {
```



```

10 ++local;
11 }
12 }
13 partial.positive = local;
14 }

```

这段代码还不是完整线程池实现，但表达了核心原则：热路径写寄存器和本地 `partial`，不写全局共享对象。它把问题从“每个元素一次同步”变成“每个 worker 一次合并”。当输入很大时，这个差异通常是数量级的。

8.3 Scan、Filter 和 Partition

Scan 看起来比 reduce 更串行，因为每个输出都是前缀结果。但 scan 可以拆成块内扫描、块摘要扫描、回填三阶段。以整数前缀和为例，块内扫描可以并行；每个块产生一个 block sum；对 block sum 做小规模 scan；最后每个块把前面所有块的和加回自己的输出。

Listing 8.5 单块局部 scan

```

1 std::int64_t scan_block(std::span<const std::int32_t> input,
2 std::span<std::int64_t> output) {
3     const std::size_t count = std::min(input.size(), output.size());
4     std::int64_t prefix = 0;
5     for (std::size_t i = 0; i < count; ++i) {
6         prefix += input[i];
7         output[i] = prefix;
8     }
9     return prefix;
10 }

```

这段局部 scan 本身是串行的，但多个块之间可以并行。并行 scan 的关键是把“所有元素依赖前面所有元素”改写成“块内局部依赖 + 块间摘要要依赖”。这个思想非常重要：很多系统问题并不是完全无依赖，而是依赖可以摘要化。分布式计算中的 map-side combine、数据库中的局部聚合、图计算中的 frontier 层处理，都有类似结构。

Scan 的测试要比 reduce 更细。reduce 只需要最终结果，scan 每个位置都要正确。测试要覆盖空输入、单元素、刚好一个块、多个块、最后一块不满、负数和可能溢出。若使用无符号类型，还要说明模运算语义；若使用有符号类型，要避免未定义溢出。

Filter 和 stream compaction Filter 选择满足谓词的元素。如果串行写，可以直接 `push_back`；并行写就不能让所有线程争用一个 vector。标准做法是三阶段：每个块统计保留数量，对数量做 scan 得到输出偏移，最后每个块写入自己的连续输出区间。

Listing 8.6 过滤前先统计每个块输出数量

```

1 std::int64_t count_selected(std::span<const std::int32_t> values,
2 std::int32_t threshold) {
3     std::int64_t count = 0;
4     for (const std::int32_t value : values) {
5         if (value >= threshold) {
6             ++count;
7         }
8     }
9     return count;
10 }

```

第一阶段只统计，不写输出，因此没有冲突写。第二阶段扫描每个块的 count，得到每个块输出起点。第三阶段每个块再次扫描输入，把满足条件的元素写到自己负责的输出区间。它多读了一遍输入，却消除了共享 push 的锁或原子热点。是否划算取决于选择率、元素大小、谓词成本和内存带宽。

如果选择率很高，输出接近输入大小，两遍读的成本较高；如果选择率很低，输出少，三阶段方法仍然可能更好，因为并发写冲突少且输出紧凑。GPU、数据库和列式执行器都大量使用 selection vector 或 prefix sum 来组织过滤输出。

Stencil: 边界和内部区域分离 Stencil 的核心是邻域复用。以一维三点 stencil 为例，输出位置 i 读取 $i-1$ 、 i 和 $i+1$ 。内部区域很规则，边界位置需要特殊处理。若在每次迭代里判断边界，热循环会有分支；若把边界单独处理，内部循环更干净。

Listing 8.7 内部区域和边界分离的一维 stencil

```
1 void smooth_three_point(std::span<const float> input,
2   std::span<float> output) {
3   const std::size_t count = std::min(input.size(), output.size());
4   if (count == 0) {
5     return;
6   }
7   output[0] = input[0];
8   if (count == 1) {
9     return;
10  }
11  for (std::size_t i = 1; i + 1 < count; ++i) {
12    output[i] = (input[i - 1] + input[i] + input[i + 1]) / 3.0F;
13  }
14  output[count - 1] = input[count - 1];
15 }
```

二维 stencil 更强调分块。一个像素或网格点会被相邻输出复用，按行扫描能利用一部分局部性；按 tile 处理可以把一块输入和 halo 留在缓存中。分布式 stencil 则要交换 halo 数据。单机的 cache block 和分布式的 halo exchange 是同一思想在不同层级上的表现。

直方图：桶数量决定策略 直方图的策略取决于桶数量、输入分布和线程数。桶很少时，全局原子会产生严重热点；每线程本地桶可以减少冲突，但合并成本低。桶很多时，每线程完整桶数组可能占用大量内存，TLB 和 cache 压力上升。输入分布倾斜时，少数桶成为热点；输入均匀时，分片效果更稳定。

一个合理实现通常从三版开始：全局锁或原子版本作为 baseline；每线程本地直方图版本作为优化；按 NUMA 节点或分块汇总版本作为扩展。每版都要说明内存占用。例如桶数是一百万、线程数是 64，每线程完整 `std::uint64_t` 桶数组需要 512 MiB，仅 partial 就可能过大。这时可以按块处理、稀疏本地 map、分桶分片或两级聚合。

排序：比较、移动和分区 排序不是只有比较次数。比较排序的成本包括比较函数、数据移动、分支预测、cache locality 和临时空间。若元素很大，排序索引可能比移动对象更好；若 key 是固定宽度整数，基数排序可能比较排序更适合；若数据已经部分有序，某些算法能利用已有顺序；若需要稳定性，算法选择又不同。

并行排序通常先分区，再局部排序，再合并。分区是否均衡决定负载；局部排序是否 cache 友好决定单核效率；合并是否形成长尾决定扩展性。分布式排序进一步把分区变成 shuffle：样本选择不准会造成某些 reducer 数据过多，形成长尾。排序是连接单机算法和分布式系统的典型内核。

图遍历：不规则内核的代表 图遍历是规则数组内核的反面。邻接表访问可能随机，顶点度数差异巨大，前沿大小每层变化，写 visited 状态可能有竞争。BFS 的 top-down 策略从当前前沿扩展邻居；bottom-up 策略从未访问顶点检查是否有邻居在前沿。当前沿很小时 top-down 更好，当前沿很大时 bottom-up 可能减少边访问。方向优化的本质，是根据数据形状切换内核。

图算法还暴露负载均衡问题。若一个顶点有百万邻居，而其他顶点只有几个邻居，按顶点静态分配会造成长尾。可以按边切分、按度数分层、把高阶顶点拆成多个任务，或者使用工作窃取。每种方案都改变局部性和同步成本。图遍历提醒我们：高性能计算不只是 SIMD 和矩阵，真实系统有大量不规则数据。

Top-k 和重 hitter 很多系统不需要完整排序，只需要 top-k 或 heavy hitters。朴素做法是统计所有 key 后全量排序；更好的做法是每个分片维护局部 top-k 或候选集合，最后合并。若 key 空间很大且数据流很长，可以使用近似算法，例如 count-min sketch 或 space-saving，牺牲精确性换取内存和吞吐。

近似算法必须写清误差语义。系统教材不能只说“近似更快”，要说明误差上界、误报/漏报、是否能用于计费或安全策略。对于日志趋势和监控，近似可能足够；对于金融结算，近似不可接受。算法选择受业务语义约束。

内核组合：日志统计流水线 把本章模式组合起来，可以得到一个日志统计流水线。第一阶段 map：解析每行日志，提取 timestamp、user、event。第二阶段 filter：丢弃无效记录。第三阶段 partition：按 event 或 user 分桶。第四阶段 local reduce：每个 worker 对本地桶计数。第五阶段 merge：合并 partial。第六阶段 top-k：输出最热门事件。

这个流水线可以作为 Compute Systems Engine 的中型案例。每个阶段都有 reference，每个阶段都有可测成本，每个阶段都能映射到后续分布式版本。单机时 partition 是内存写入偏移；分布式时 partition 是 shuffle 目标。单机时 backpressure 是有界队列；分布式时 backpressure 是网络 and 远端 reducer 的限流。这样一本书的前后内容就能真正贯通。

计算内核章的回归阅读要从输出位置和状态冲突分类。Map 的输出位置天然等于输入位置，filter 需要选择向量或压缩，scan 把数量变成位置，partition 重排数据，reduce 合并状态，join 可能输出放大，top-k 缩小候选，sketch 用误差换空间。名字相同不代表问题相同，必须写清输入分布、输出规模、稳定性和是否有共享写。

以日志聚合管线为例，parse map 生成字段列，valid filter 选择可用行，event histogram 本地计数，partition 按 reduce 分区，merge reduce 合并，top-k 输出热门事件。这个链路里每个阶段都有不同的验收：parse 要保留错误位置，filter 要覆盖选择率，histogram 要覆盖热点 key，partition 要检查每个 key 去向，top-k 要定义 tie-break。把这些阶段写清，比给一个大函数更像教材。

本章还要训练材料化判断。Map 和简单 filter 可以流水线，sort、join、shuffle、全局 reduce 往往是 pipeline breaker。材料化增加内存或 I/O，却换来调试、复用和恢复；融合减少中间写入，却可能增加寄存器压力和恢复复杂度。内核选择不是算法题答案，而是数据流、资源和失败语义的共同决策。

数据结构要和内核模式一起讲。Top-k 通常需要小堆、固定容量有序数组或分段候选；hash aggregate 需要开放寻址、分片表或排序聚合；graph frontier 需要在 vector、bitmap 和稀疏集合之间切换；join 需要哈希表、排序 run 或索引。每种结构都要说明内存布局、写冲突、迭代顺序和合并成本。只给算法名字，不讲承载它的数据结构，读者无法把理论放进系统。

项目中的 kernel suite 不应追求数量，而应覆盖形状。至少要有一对一 map、输出稀疏的 filter、需要 scan 的 compaction、写冲突明显的 histogram、会输出放大的 join 或替代案例、会缩小结果的 top-k。每个内核都保留 reference、输入生成器和验收报告。这样读者面对新业务时，不是背“某题怎么做”，而是识别它属于哪种计算形状。

内核组合还要写 pipeline breaker。Sort、global reduce、join build、shuffle 和 checkpoint 都会让上游不能简单继续推数据，因为它们需要重排、聚合、持久化或等待全局信息。识别 breaker 后，运行时才能知道哪里形成 stage 边界，I/O 层才能知道哪里值得写 block，分布式章节才能知道哪里必须处理重试和幂等。单机内核章因此是分布式工程的前置课。

本章还可以安排一个“同名不同题”的练习。让读者比较三个 histogram：小范围 event type、巨大用户 ID、倾斜热点 key。第一个适合数组桶，第二个适合哈希或排序聚合，第三个要考虑本地 partial、热点拆分或二级 reduce。三个都叫 histogram，但数据结构、并行策略和瓶颈完全不同。这个练习能纠正只背模式名的习惯。

哈希聚合还要拆开讲实现细节。开放寻址表访问更连续，容易减少节点分配，但删除、扩容和高负载因子会让探测变长；链式哈希表插入直观，却把节点、key 和 value 分散到堆上，cache 行为通常更差；排序聚合先付出 $O(n \log n)$ 或分块排序成本，再换来顺序扫描和确定输出。教材不能只告诉读者“哈希平均 $O(1)$ ”，还要让他知道这个平均值背后可能有随机访问、分配器、rehash 和倾斜

key。项目中如果实现稀疏聚合，至少应记录表容量、负载因子、冲突次数、rehash 次数和内存峰值。

Join 的案例也值得更具体。小表 hash join 的构建表必须有生命周期，probe 阶段不能让输出 vector 在每次命中时无界扩容；sort-merge join 要说明排序 key、稳定性和重复 key 的输出乘积；分布式 join 还要决定按哪个 key shuffle，是否先过滤和投影，是否能广播小表。若输出可能一对多爆炸，背压和临时文件就比查找本身更重要。把 join 写成“建表再查询”太轻了，真正的系统问题在输出规模、内存预算和失败恢复。

Graph frontier 可以作为不规则内核的代表。向量 frontier 适合活跃顶点少时顺序遍历，bitmap frontier 适合活跃顶点多时快速判断，稀疏集合适合需要去重且仍要遍历的位置。BFS 前期、中期、后期的 frontier 形状不同，数据结构也可能切换。这个例子让读者看到“同一个算法内部也可能换结构”，比把数据结构当作静态选择更接近真实工程。

8.4 典型内核要从数据流和冲突形式分类

Map、reduce、scan、filter、partition、histogram、sort、join、graph traversal 不是并列名词，而是不同数据流和冲突形式。Map 每个元素独立，最容易并行；reduce 多个元素合成一个结果，关键是结合律和局部合并；scan 把每个位置前面的信息变成当前位置输出；filter 需要为保留元素分配紧凑位置；partition 把记录按目标桶重排；histogram 多个输入竞争同一个桶；join 把两类数据按 key 关联；图遍历则前沿动态变化、访问不规则。

从数据流分类的好处是能直接推导实现。若每个元素独立，优先考虑批量 map、SIMD 和线程切块；若多个元素写同一结果，先考虑线程本地 partial；若输出位置不确定，先计数再 scan；若 key 倾斜，先考虑热点隔离或分层聚合；若访问图邻接，先考虑 CSR、frontier 表示和负载均衡。这样章节不会变成“每个内核给一个定义”。

Map 和 reduce：独立执行与合并语义

Map 的朴素版本是逐元素转换。优化点通常不是算法复杂度，而是中间数组、内存带宽、函数调用边界和融合。两个连续 map 若中间结果只被下一步使用，可以融合成一个循环，减少写出和读回。但融合也会增加寄存器压力和代码复杂度，若中间结果被多个下游复用，过度融合可能反而浪费计算。教材要给出取舍，而不是只说“融合更快”。

Reduce 的第一问题是语义。整数求和可结合但可能溢出，浮点求和不严格结合，字符串拼接顺序敏感，错误列表合并可能要求稳定顺序。确认合并语义后，才谈局部 partial、树形归约、SIMD 水平合并和多线程。全局 atomic 累加是很好的反例：正确性可能成立，扩展性却很差，因为所有线程争用同一条 line。

项目中可以把日志 event type 计数作为 reduce/histogram 混合案例。若 event type 范围小，用固定数组 partial；若 key 是字符串，先做字典编码；若 key 高度倾斜，线程本地 map 后合并；若输出需要稳定顺序，最后排序 key。每一步都从语义和冲突形式推出来。

Scan、filter 和 partition：先求位置再写出

Filter 的直觉写法是多个线程对共享 vector `push_back`。它简单，但输出顺序不稳定，锁竞争明显。更稳的并行写法是两阶段：每个块先统计命中数，scan 得到块的全局偏移，再让每个块把保留元素写到自己的区间。这里 scan 的意义非常具体：把数量变成位置，消除输出争用。

Partition 类似。若每条记录直接发到目标桶，会产生随机写、小消息和锁竞争。工程版本通常先统计每个桶数量，计算偏移，再批量写出；多线程版本还会有每线程每桶计数矩阵，然后做二维前缀和。这个结构也是分布式 shuffle 的前置：本机 partition 写出的 block 将来可以变成跨节点传输单位。

Scan 本身也有多种形式。Inclusive、exclusive、按块 scan、分层 scan、并行 prefix 都要先用小例子说明输出含义。错误最常出现在空块、全命中、最后一块长度不足和偏移溢出。代码中索引使用 `std::uint64_t` 或 `std::size_t` 要写清语义，避免大输入下计数溢出。

Histogram、Top-k、join 和排序

Histogram 的核心是冲突更新。桶数量少且 key 均匀时，线程本地桶数组很有效；桶数量大时，partial 内存可能太大；key 高度倾斜时，热点桶合并会成为长尾；key 是字符串时，哈希、分配和比较进入成本。一个成熟章节要给多种方案：固定小桶数组、线程本地 hash map、分片锁、局部排序后归并、热点 key 单独处理。每种方案都有适用范围。

Top-k 不等于完整排序。若只要前 k 个元素，可以维护小堆、分块 top-k 后合并、quickselect 或

近似 heavy hitter。选择取决于 k 的大小、是否需要稳定顺序、是否流式输入、是否允许近似。若 k 接近 n，排序可能更简单；若输入持续到达，流式结构更合适。教材要让读者从问题要求推导算法，而不是看到 top-k 就固定写堆。

Join 的成本在数据布局和基数。小表 join 大表时，可以把小表构建成紧凑哈希表，热路径只扫描大表；两张大表 join 时，要考虑分区、排序归并或外部存储；key 倾斜会让某些桶特别大。Join 还涉及缺失 key、重复 key、多值输出和输出爆炸。把 join 当作“查 hash map”会严重低估工程复杂度。

图遍历和不规则内核

图遍历代表不规则计算。Frontier 大小时，按边并行可能有效；frontier 很小时，线程很多会空转；度分布倾斜时，高度顶点会造成长尾；邻接表随机访问会让 cache 和 TLB 压力上升。CSR 布局能让邻接边连续，但顶点访问顺序仍然可能不规则。图内核让读者看到：并行不是把循环切成等长块这么简单。

项目里不一定要实现完整图计算框架，但可以用任务依赖图或日志用户关系图做小实验。用 frontier 表示 ready tasks，比较 bitmap frontier、vector frontier 和 queue frontier；记录 frontier 大小、边访问数、worker 空闲时间和 cache 行为。这个实验会连接任务运行时章节：任务图调度本质上也在处理动态 frontier。

本章最终要形成的判断力是：看到一个计算问题，先判断它的输入输出关系、写冲突、输出位置、key 分布和访问规则性。只有这些说清楚，才选择 map、reduce、scan、partition、sort、join 或图遍历的具体实现。

案例走查：热点 histogram 的多种解法

输入 key 若均匀，线程本地固定桶数组再合并很简单。若少数 key 占 90

每个版本解决不同问题。线程本地 map 消除共享写，但增加合并成本；热点隔离降低长尾，但需要采样或在线检测；固定数组很快，但要求 key 范围小；排序输出稳定，但增加最终成本。章节要让读者看到多解法不是堆答案，而是根据 key 分布和语义选择。

实验输入至少三类：均匀 key、Zipf 分布、单热点 key。指标包括吞吐、每线程 partial 大小、合并时间、最大桶负载和输出稳定性。这个案例之后，读者看到 histogram、top-k、join、shuffle 热点时，就能用同一套问题分析。

内核组合后的瓶颈迁移

单独看 map、filter、histogram 都能讲清，但真实项目是它们组合。Parser 产生 batch，filter 去掉错误，map 提取 key，histogram 本地聚合，partition 写 shuffle block。前一个内核优化后，下一个内核可能成为瓶颈。若 parser 加速两倍，histogram 热点可能显现；若 histogram 本地 combine 成功，I/O 写出可能成为瓶颈。

因此实验要测 pipeline，不只测单内核。每个 stage 记录输入条数、输出条数、耗时、字节数和错误数。优化某个 stage 后，重新看全链路比例。若某个 stage 已经只占 3

内核融合也要看全链路。融合 filter 和 map 减少中间数组，但可能让错误处理和统计耦合；融合 map 和 histogram 减少输出写入，但可能降低复用；保留材料化中间结果增加内存，却方便 debug 和重试。工程选择取决于是否需要检查点、复用和故障恢复。

内核章节的作业设计

给读者一个日志聚合需求：统计每个 event type 的次数，输出错误行样本，保留每个用户最近一次事件。要求先拆成内核：filter 错误、map 提取字段、reduce 计数、group-by 更新最近事件、partition 输出。然后让读者说明每个内核的冲突形式和输出位置。

第二步给 key 倾斜输入，让读者选择策略：线程本地 partial、热点隔离、排序归并或分片锁。答案要说明为什么选择它，放弃了什么。第三步要求写实验矩阵，覆盖均匀 key、热点 key、小输入、大输入和错误行。这样题目不是直接给答案，而是训练从问题推导内核组合。

这一章的验收重点是“分类能力”。读者拿到一个新问题，能说出它更像 map、reduce、scan、partition、join 还是图遍历，并指出写冲突和输出位置。做到这一点，后续并行算法和分布式 shuffle 才有基础。

综合练习：为同一需求选择不同内核组合

需求是“统计错误日志中每个用户最近一次失败事件，并输出前 100 个高频错误码”。这个需求至少包含 filter、group-by reduce、top-k 和稳定输出。读者先写串行 reference，再拆内核。Filter 保

留错误行；group-by 更新用户最近事件，要求比较 timestamp；错误码计数用于 top-k；最终输出要按次数和错误码稳定排序。

接着给三种输入分布。第一种错误均匀，用户均匀；第二种少数用户极热；第三种错误码高度倾斜。三种输入下的最佳内核组合可能不同。均匀时线程本地 partial 合并即可；用户极热时需要热点隔离或锁分片；错误码倾斜时 top-k 可以先局部 heavy hitter。读者要说明选择原因，而不是直接套一个答案。

第三步要求写内核边界。Filter 是否材料化输出，group-by 是否和 filter 融合，top-k 是否用完整排序，partition 是否提前发生。每个边界都影响调试、内存和恢复。若需求将来进入分布式 shuffle，材料化 block 和 manifest 可能比极致融合更重要。

这个题的评分看推导链：从需求到 reference，从数据分布到冲突形式，从冲突形式到内核组合，从组合到实验矩阵。只给最终代码不得满分，因为本章训练的是分类和组合能力。

容易出错的边界：同一个名字下可能是不同问题

“Reduce”可能是整数求和、浮点求和、错误列表合并、字符串拼接或 top-k 合并，它们的结合律、顺序要求和输出大小都不同。“Filter”可能要求稳定顺序，也可能不要求；可能只输出计数，也可能输出完整记录。“Join”可能是小表查大表，也可能是两张大表外部 join；可能一对一，也可能一对多导致输出爆炸。内核名字不能替代问题定义。

因此章节里的每个案例都要先写需求细节。Histogram 的 key 范围是多少，是否倾斜，是否需要稳定输出；Top-k 的 k 多大，是否允许近似，输入是否流式；Graph traversal 的 frontier 大小如何变化，图是否有高度顶点。没有这些条件，任何“最佳实现”都不成立。

另一个反例是过早组合内核。把 filter、map、histogram 全部融合成一个大函数，短期减少中间数组，但错误处理、调试和复用都变差。把每一步都材料化，又可能浪费内存和带宽。系统设计要在融合和材料化之间选择，而选择依据是热路径、检查点、复用和恢复，不是单一审美。

本章源码索引可以要求读者把项目中每个内核标注为 streaming、blocking、stateful、conflict-heavy 或 irregular。Streaming 内核适合流水线，blocking 内核需要等待全部输入，stateful 内核要写不变量，conflict-heavy 内核要处理共享写，irregular 内核要处理负载均衡。这样的分类比只列函数名更有用。

本章核心接口：KernelPlan

内核模式章给项目留下 KernelPlan。它描述一段 pipeline 由哪些内核组成，每个内核的输入输出、是否材料化、是否可融合、是否有写冲突、是否需要稳定顺序。优化器可以根据 plan 做简单选择，报告也可以按 plan 输出阶段耗时。

KernelPlan 不需要复杂框架。一个结构化表就足够。它的价值是让读者在写代码前先说清数据流。后续任务图可以从 plan 生成依赖，I/O 章节可以从材料化边界生成 block，分布式章节可以从 partition 边界生成 shuffle。

从单机内核进入并发同步

典型内核一旦多线程执行，就会遇到共享状态和等待。Reduce 需要合并 partial，histogram 需要处理桶冲突，partition 需要写出 block，join 可能访问共享维表，图遍历可能更新 visited。下一部分进入锁、原子和无锁结构，是因为算法形态已经暴露了共享边界。同步不是外加安全补丁，而是算法并行化后的自然问题。

项目里，KernelPlan 应把写冲突标出来。标出后，后续同步章节才能决定用线程本地 partial、mutex、atomic、分片锁还是队列。若内核计划没有写冲突说明，并发代码就只能靠临场加锁。内核章的输出，是并发章的输入。

8.5 项目落地

Compute Systems Engine 在本章应增加一个 kernel suite。至少包含：map 变换、count positive reduce、parallel count-if、stream compaction、thread-local histogram、one-dimensional stencil 和 top-k baseline。每个内核都保留串行 reference 和一个并行版本，benchmark harness 记录输入规模、线程数、输出校验和核心指标。

项目不要一开始追求所有内核极致优化。更重要的是让每个内核展示一种系统问题：map 展示内存带宽；reduce 展示共享写消除；scan 展示依赖分层；filter 展示输出位置分配；histogram 展示冲突更新；stencil 展示邻域复用；graph 或 top-k 展示不规则负载和数据倾斜。

8.6 流水线、材料化和验收

多个内核连接时，要决定中间结果是否材料化。材料化把中间结果写成数组或文件，便于调试、复用和容错，但增加内存或 I/O。流水线把上游输出直接送给下游，减少中间写入，但背压、错误恢复和调试更复杂。数据库执行器中的 volcano、vectorized execution、pipeline breaker 都围绕这个取舍。

Reduce、sort、join、shuffle 常常是 pipeline breaker，因为它们需要看到一批数据或按 key 重排才能继续。Map/filter 通常容易流水线。理解哪些阶段必须材料化，哪些阶段可以融合，是计算系统设计的核心能力。

Map 内核的细节 Map 是最简单的计算内核：每个输入元素独立产生一个输出元素。正因为简单，它适合作为性能分析起点。Map 的主要问题通常是内存带宽、函数调用、分支和类型转换。若每个元素只做一次加法，瓶颈很可能是读写字节；若每个元素调用复杂函数，瓶颈可能转向计算或前端；若每个元素有随机分支，分支预测会进入成本。

Map 内核的输出所有权很清晰：第 i 个输入写第 i 个输出。并行切分时，每个 worker 写自己的连续输出区间，不需要锁。这种结构应成为读者心中的“理想并行形状”。后续 filter、partition、join 的复杂性，很多都来自输出位置不再天然等于输入位置。

Map 还有一个常见优化：就地更新。若输入不再需要，可以直接写回原数组，减少输出内存。但就地更新改变了别名和生命周期。若后续还需要原始输入，就不能就地；若多线程中其他阶段仍在读取输入，就地会造成数据竞争或语义错误。内存节省不能脱离数据流图。

Reduce 内核的细节 Reduce 把多个输入合成一个或少数几个输出。它的难点是依赖和合并顺序。单线程 reduce 有一条累加依赖链；多累加器和分块 reduce 可以拆链；多线程 reduce 需要 partial；分布式 reduce 需要 map-side combine 和 shuffle。一个 reduce 的形态会随着尺度变化，但核心问题始终是：局部怎样合并，全局怎样合并，合并顺序是否影响语义。

不同 reduce 的单位元和结合性不同。计数求和简单，最小值最大值也简单；浮点求和有误差；集合合并可能需要去重；字符串拼接内存成本高；哈希表合并受 key 分布影响。教材不能把 reduce 简化成一个加法例子。每个 reduce 都要写清操作、单位元、是否结合、是否交换、是否确定性。

Scan 内核的角色 Scan 的价值在于把局部数量变成全局位置。Stream compaction、partition、基数排序、稀疏矩阵构建、GPU kernel 输出分配都需要 scan。它不是只用于前缀和题目，而是并行写输出的基础设施。

Scan 通常是 pipeline breaker，因为后续写位置依赖前面所有块的数量。优化 scan 时，常把它分层：块内 scan，块总和 scan，块内回填。这个结构在单机多线程和 GPU 中都常见。分布式系统里，对每个 partition 的输出大小做前缀求和，分配文件偏移，也是一种 scan 思想。

Filter 的选择率 Filter 的性能高度依赖选择率。选择率为 0 时，输出很小，但仍要读全部输入；选择率为 100% 时，filter 接近 copy；选择率中等且随机时，分支和输出位置都复杂。选择率还影响数据结构选择：低选择率适合 selected index list，高选择率适合 bitmap 或直接保留原数组加 mask，中等选择率可能需要 compaction。

测试 filter 时必须覆盖不同选择率。只测 50% 随机选择，无法代表真实业务；只测全通过，也看不到输出位置分配成本。日志过滤、权限筛选、数据库谓词和图 frontier 都会有不同选择率分布。算法实现要能处理这些变化。

Partition 的两种目标 Partition 有两类目标。第一类是为了并行写输出，例如把元素按 bucket 放到连续区间，便于后续每个 bucket 独立处理。第二类是为了改善局部性或控制流，例如按记录类型分桶，让后续同类记录走同一分支。前者关注无冲突写和输出布局，后者关注后续计算效率。很多系统同时使用二者。

Partition 的成本包括两遍扫描、counts 矩阵、输出写入和可能的稳定性维护。如果后续收益不大，partition 可能不划算。一个小输入上为了减少分支而先排序分桶，可能比直接处理更慢。判断 partition 是否值得，要估算后续阶段减少了多少分支、随机访问或网络数据。

Histogram 的冲突维度 Histogram 的冲突来自多个输入更新同一 bucket。冲突维度有三种：线程之间冲突、cache line 冲突、数据倾斜冲突。线程之间冲突可以用本地桶解决；cache line 冲突要注意相邻 bucket 是否被不同线程频繁写；数据倾斜冲突会让某些 bucket 合并阶段很重。桶数量越少，线程本地数组越小，但热点越强；桶数量越大，本地数组越大，合并成本也高。

稠密 histogram 的合并可以按 bucket 并行，也可以按 worker 并行。按 bucket 并行适合 bucket 很多，每个 bucket 合并所有 worker；按 worker 并行适合 worker 多但 bucket 少，先局部汇总再串行写。合并阶段也可能成为瓶颈，不能只优化本地统计。

Stencil 的边界处理 Stencil 使用邻域数据计算输出，例如一维平滑、二维卷积、网格 PDE。它的局部性比随机访问好，但边界处理和块间 halo 会带来复杂性。单线程中，边界可以单独处理；多线程中，每个块需要读取邻近块的边界数据。若只读输入写输出，halo 读取没有数据竞争；若就地更新，邻域依赖会破坏并行语义。

Stencil 优化常用 blocking，让一块数据在 cache 中被多次使用。二维 stencil 还要考虑行主序、行跨度、边界分支和向量化。GPU 和图像处理里 stencil 很常见，但 CPU 上同样重要。它训练的是“邻域复用”和“边界语义”。

Sort 的系统角色 排序不仅是算法题，也是系统工具。排序可以把随机 key 聚成连续区间，便于聚合、join、压缩和写文件。外部排序能处理超过内存的数据：先生成有序 run，再多路归并。分布式排序则先采样选择分区边界，再 shuffle 到 reducer。排序是典型的 pipeline breaker，但它换来后续顺序访问和确定性输出。

排序的稳定性也要声明。稳定排序保持相等 key 的原顺序，适合需要保留时间顺序或二级语义的场景；非稳定排序更快或内存更少。很多系统用复合 key 代替稳定排序，例如先按主 key，再按原始位置。稳定性和确定性是测试和恢复的重要边界。

Join 的输出放大 Join 最容易低估的是输出放大。两个输入各一百万行，如果某个 key 在左表出现一千次，在右表出现一千次，该 key 的 join 输出就是一百万行。即使输入不大，输出也可能爆炸。系统必须估算输出基数，并设置内存和背压策略。否则 join 不是慢，而是直接 OOM。

Hash join 构建阶段也要选择 build side。通常把较小表放入哈希表，较大表做 probe。但如果小表 key 极端倾斜，某些 bucket 很长；如果大表过滤后很小，先过滤再 join 更好；如果输出需要按 key 排序，sort-merge join 可能减少后续成本。Join 是优化器思维的入口：单个内核选择受上下游影响。

Sketch 和近似计算 在大规模系统中，有时无法保存所有 key 的精确状态。Sketch 类算法用固定内存近似频率、基数或分位数。Count-min sketch 估计频率，HyperLogLog 估计基数，t-digest 或 KLL 估计分位数。它们适合监控、推荐特征、流量分析和预估，不适合必须精确的结算。

近似算法的教材写法必须包含误差语义、内存大小、合并方式和适用边界。分布式系统喜欢可合并 sketch，因为每个 worker 可以本地构建，最后合并。它们和 reduce 结构天然匹配。若 sketch 不能合并，分布式使用会困难。

Kernel suite 的代码组织 Compute Systems Engine 的 kernel suite 应把每个内核按相同结构组织：reference、parallel、benchmark、tests。Reference 定义语义，parallel 做优化，benchmark 记录性能，tests 覆盖边界。不要把所有内核写进一个文件。每个内核都要有输入生成器和校验器，便于构造均匀、倾斜、小规模、大规模和边界输入。

接口应使用 span 和明确输出对象。Map 接收 input 和 output span；reduce 返回标量或 partial；histogram 接收 key span 和 bucket span；top-k 返回候选列表；join 返回输出范围或写入 sink。明确输出所有权，可以减少并发写冲突和隐藏分配。

阶段融合的验收 若要融合两个内核，例如 filter 后 map，应先有未融合 reference。融合版本必须输出完全一致或在声明误差内一致。性能报告要比较三项：未融合耗时、融合耗时、中间数据大小。若融合变快，要解释是少写中间数组、减少分配、改善 cache，还是减少函数调用。若融合变慢，要看寄存器压力、分支复杂度和代码大小。

融合还会影响调试和恢复。分布式作业中，materialized 中间结果可以作为 checkpoint；融合流水线失败后可能要从更早阶段重做。高性能和容错有时冲突。批处理系统常在 stage 边界材料化，就是为了恢复和重试。单机内核融合到分布式系统时，要重新考虑失败语义。

本章验收清单 本章完成后，读者应能为一个新计算任务做模式识别。它是 map、reduce、scan、filter、partition、sort、join、stencil、graph frontier、top-k 还是 sketch？它的输入输出是什么？输出位置是否天然确定？是否有共享写？是否需要稳定顺序？是否能局部聚合？是否有热点 key？是否有 pipeline breaker？是否能用 reference 校验？这些问题比背诵某个优化技巧更重要。

项目验收则要求至少三个内核串成一个流水线，并保留每阶段指标。比如日志记录先 filter 有效行，再 map 到 event type，再 histogram 聚合，再 top-k 输出。每阶段都要能单独关掉或替换，以

便做对照实验。

案例：日志统计的完整内核链 一个完整日志统计链可以按 batch 处理。第一步，parse map 把原始文本变成热字段列。第二步，valid filter 产生选择向量或错误列表。第三步，event histogram 对有效记录按 event type 本地计数。第四步，partition 把本地计数按 reduce partition 分组。第五步，reduce merge 合并所有 worker 的 partial。第六步，top-k 输出热门事件。

每一步都有独立 reference。Parse reference 可以用简单字符串处理；filter reference 检查每行有效性；histogram reference 用单线程 map；partition reference 检查每个 key 去向；reduce reference 合并所有 partial；top-k reference 全量排序。链路正确性来自每个内核正确加上接口语义正确。这个案例可以作为第 18 章项目的单机子集。

案例：近似 heavy hitter 如果 event type 很多，精确统计所有 key 成本高，可以使用 heavy hitter 近似。每个 worker 维护固定大小候选集合，记录可能热门 key；最后合并候选，再对候选做精确二次统计或输出近似。这个方法减少内存，但可能漏掉接近阈值的 key。它适合监控和预警，不适合结算。

近似案例的重点是误差合同。报告必须写：最多保留多少候选，什么情况下可能漏报，是否做二次精确验证，输出是否标记近似。系统工程中，近似不是偷懒，而是用明确误差换资源。

案例：Pipeline breaker 的选择 假设 parse、filter、map 都可以流水线，但 histogram 需要聚合，top-k 需要看到完整结果。Histogram 和 top-k 就是 pipeline breaker。系统可以在 histogram 前保持 batch 流动，在 histogram 后材料化 partial，再进入 top-k。若为了极致低延迟把所有阶段都强行流水线，恢复和调试会复杂；若每一步都材料化，I/O 和内存成本高。

选择 pipeline breaker 时，要问三个问题：后续是否需要全局重排或聚合，失败后是否需要从这里恢复，中间数据是否值得保存。这个判断会贯穿单机执行器和分布式 shuffle。

内核选择的业务语义 同一个输入可以有多种内核选择。要统计所有 key 的精确计数，用 histogram 或 hash aggregate；只要热门 key，用 top-k 或 heavy hitter；要按 key 连接两个数据集，用 join；要按时间窗口输出趋势，用 partition 加窗口 reduce；要快速估计基数，用 sketch。选择不是只看速度，而是看业务语义：是否精确，是否确定，是否可恢复，是否需要排序，是否允许旧数据。

日志系统里，计费日志必须精确，监控趋势可以近似；审计输出需要稳定顺序，调试指标可以无序；最终报表需要持久化，临时 dashboard 可以丢弃。内核模式和业务语义相连，系统设计不能把所有数据都当成同一种计算。

计算内核章节总结 计算内核是数据流里的可复用形状。Map 保持位置，filter 选择元素，scan 分配位置，partition 重排数据，reduce 合并状态，sort 建立顺序，join 组合关系，top-k 缩小结果，sketch 用误差换空间。识别这些形状，就能把复杂业务拆成可测试、可测量、可替换的阶段。

内核模式决策表 选择内核模式时，先问输出和输入的关系。一对一是 map，一对零或一是 filter，一对多可能是 flat-map 或 join，多个输入到一个输出是 reduce，需要位置就是 scan，需要按 key 分组就是 partition 或 sort，需要最热少数就是 top-k，需要固定内存估计就是 sketch。模式识别后，再讨论并行、SIMD 和 I/O。不要一开始就从实现细节出发。

内核验收报告 本章每个内核都应该有一份验收报告。报告至少包含：串行 reference、输入分布、输出校验、数据结构、内存流量估算、并行切分、共享写分析、负载均衡、异常边界和性能指标。没有这些内容的“优化”只是代码片段。

以直方图为例，验收报告要写：桶数量、key 分布、线程数、本地桶内存占用、合并策略、是否对齐、是否有 false sharing、输入是否随机、输出是否和 reference 一致、在均匀和倾斜输入下差异如何。这样读者才会把算法、数据和系统放在一起看。

本章扩展实验

习题与作业

实验一：实现稠密 key 直方图和稀疏 key 哈希聚合两个版本。构造小 key 空间、大 key 空间、均匀分布和倾斜分布四组输入，比较内存占用和吞吐。

实验二：实现本地 top-k 加全局合并。要求定义 tie-break 规则，并验证多线程和单线程输出顺序一致。

实验三：设计一个 hash join 和 sort-merge join 的成本比较，不要求完整实现大系统，但

要估算构建、排序、探测、输出和内存占用。

实验四：为 BFS frontier 选择 vector 和 bitmap 两种表示。说明当前沿从小变大时，为什么表示应该切换。

实验：在 Compute Systems Labs 中扩展一个线程本地直方图。先写全局 mutex 版本，再写线程本地版本，比较扩展性，并解释差异来自锁竞争还是 cache line 迁移。

map map 的输出位置与输入位置一一对应，最容易并行，也最容易被内存带宽限制。优化重点常是融合相邻 map、减少中间数组、保证连续访问。

filter filter 的输出数量未知，核心问题是输出位置分配。串行版本 **push_back** 很自然，并行版本需要局部计数、scan 或分块输出。

reduce reduce 的关键是结合律、单位元和数值稳定性。并行 reduce 不是把共享结果加锁，而是先本地 partial，再分层合并。

scan scan 把前缀状态暴露给每个位置，常用于输出位置、偏移和分区。它比 reduce 更复杂，因为每个元素都要得到自己的前缀结果。

histogram histogram 是共享写冲突的典型例子。key 范围小会产生热点，key 范围大又会增加内存。局部桶、分片桶和排序后统计各有成本。

join join 的成本取决于数据规模、key 分布、是否已排序和内存容量。hash join、sort-merge join、nested loop 不是固定优劣，而是输入条件不同。

stencil stencil 读取邻域，重复利用窗口。优化时要考虑边界、tile、缓存复用和写入缓冲。它连接矩阵、图像和数值计算。

图遍历 图算法常由不规则访问、frontier、重复访问和负载不均组成。它提醒读者：并不是所有计算都能靠连续数组轻松解决，数据结构和调度必须一起设计。

案例：稳定 filter 先串行 **push_back**，再并行局部计数加 scan 分配位置。报告说明为什么 scan 是为了确定输出位置，而不是为了炫算法。

案例：局部 histogram 比较全局原子桶和线程本地桶。解释热点 key 如何把共享写变成瓶颈。

案例：map-filter 融合 把两次中间数组写入改成单次流水线，记录内存带宽变化和可读性成本。

源码阅读路线 Linux 源码阅读从 `include/linux/rhashtable.h`、`lib/sort.c`、`lib/btree.c` 附近进入即可。阅读目标不是把整套子系统抄进书里，而是追一条和本章实验相关的路径：用户态动作怎样变成内核对象，内核对象怎样排队、等待、唤醒、计数、提交或回收。

阅读报告要写证据等级。能直接测到的数字放在第一层；从源码控制流推导出的解释放在第二层；当前设备无法验证、只能作为合理猜测的内容放在第三层。这样读者会知道内核组合报告中哪些结论可复现，哪些结论需要换机器继续确认。

源码阅读还要克制。看到一个复杂函数，不要把每个分支都展开；先回到本章的问题，问它能否解释一个等待点、一次状态转移或一条性能曲线。解释不了的内容可以留到索引，不要打断正文主线。

稳定 filter 先串行 **push_back**，再并行局部计数加 scan 分配位置。报告说明为什么 scan 是为了确定输出位置，而不是为了炫算法。

局部 histogram 比较全局原子桶和线程本地桶。解释热点 key 如何把共享写变成瓶颈。

map-filter 融合 把两次中间数组写入改成单次流水线，记录内存带宽变化和可读性成本。

map

map 的每个输出位置由输入位置直接决定，最适合向量化和并行切分。它的风险在于隐藏的分配、函数调用和冷字段读取。

实验设计：把逐条转换改成批量转换，观察中间数组和融合成本。

reduce

reduce 的核心是结合规则和合并树。共享写版本容易形成热点，线程本地 partial 才是可扩展起点。

实验设计：比较全局 atomic、mutex 桶和线程本地桶。

filter

filter 输出数量未知，难点是位置分配。串行 push 自然，并行版本需要局部计数、scan 或分块输出。

实验设计：先统计每块保留数，再 scan 得到写入偏移。

scan

scan 把前缀信息变成每个元素的输出位置，是很多并行算法的连接件。它不是炫技，而是消除共享 push。

实验设计：用 scan 实现稳定过滤，并记录每块 offset。

histogram

稠密 key 适合数组桶，稀疏 key 需要哈希或排序。热点 key 会让共享桶扩展失败。

实验设计：构造均匀 key 和单热点 key，比较局部桶合并策略。

join

join 不是单一算法。小表可以 build hash，大表可能排序合并，倾斜数据需要分区或热点拆分。

实验设计：同一数据用 hash join 和 sort merge，说明内存与局部性差异。

stencil

stencil 依赖邻域，内部区域和边界区域处理方式不同。分块要考虑 halo、cache 复用和边界语义。

实验设计：把一维 stencil 拆成内部批量循环和边界处理。

图遍历

BFS 受前沿大小、度数倾斜和随机邻接访问影响。top-down 和 bottom-up 切换是根据数据形状改变访问方式。

实验设计：记录每层前沿大小和边访问数，解释切换阈值。

融合与材料化

融合减少中间数据移动，但可能增加寄存器压力、代码体积和调试难度。材料化增加写读，但让阶段边界清楚。

实验设计：为两段 map 和一次 filter 选择融合或落地，并写出理由。

项目接口

KernelPipelinePlan 要记录每个内核的输入布局、输出位置规则、冲突形式、局部状态和合并策略。内核不是孤立代码片段。

实验设计：把每次内核组合写成数据流图和成本账本。

8.7 内核模式要按数据流和冲突形式组织

典型计算内核不能写成一堆模板代码。map、reduce、scan、filter、partition、join、stencil、BFS 看起来都是算法名，本质上是在回答三个问题：输出位置如何确定，是否有共享写冲突，数据访问是否规律。按这三个问题组织，读者才能从一个内核迁移到另一个内核，而不是背八套无关代码。

map 最简单，因为第 i 个输入通常产生第 i 个输出。它适合顺序扫描、SIMD 和线程切分。map 的陷阱不在并行语义，而在隐藏成本：每个元素调用虚函数、分配字符串、访问冷字段，都会让看似简单的映射变慢。融合多个 map 可以减少中间数组，但可能增加寄存器压力和代码体积。因此融合要有证据，不是把所有循环揉成一个。

reduce 的核心是冲突。多个输入贡献到一个结果，若所有线程直接写全局结果，就会形成锁、原子或 cache line 热点。线程本地 partial 再合并，是把冲突推迟到更低频的位置。这里要讲清结合律和单元：整数求和、浮点求和、top-k、字符串拼接和错误列表合并不是同一个 reduce。合并树既影响性能，也影响确定性。

filter 的核心是输出位置未知。串行 `push_back` 很自然，因为前面保留了多少元素，当前位置就写到哪里；并行版本如果每个线程都 push 到共享 vector，就会锁竞争或顺序混乱。scan 的作用在这里出现：先统计每块输出数量，对块计数做前缀和，再让每块写到确定区间。scan 不是为了炫算法，而是把共享输出位置分配变成确定计算。

partition 和 shuffle 是 filter 的放大版本。按谓词分两边是 partition，按 key 分多个桶就是分区；单机里输出是数组区间，分布式里输出是 shuffle block。稳定 partition 保持相对顺序，成本更高；不稳定 partition 可以少搬数据，但语义不同。教材要把这个选择写清，不能直接给最终代码。

join 和图遍历说明数据形状的重要性。join 的小表 build、大表 probe 是一种形状；两个已排序输入用 merge 是另一种形状；热点 key 会让单个桶巨大。BFS 的前沿大小在每层变化，top-down 和 bottom-up 的切换就是根据数据形状改变访问模式。高性能内核不是固定套并行模板，而是根据冲突、局部性和输出位置选择执行策略。

本章实验可以用同一个日志数据集派生。事件类型转换是 map，统计事件数是 reduce，过滤错误日志是 filter，按用户分桶是 partition，关联用户表是 join，按时间窗口扩散可以类比 stencil 或图遍历。每个实验都要求先写朴素版本，再指出冲突点或输出位置问题，再引入 scan、partial、排序或分区。这样章节会连贯，而不是每节换一个新世界。

案例推导：稳定 filter 为什么自然引出 scan

给定一批日志记录，保留所有错误记录并保持原顺序。串行写法很简单：遍历输入，遇到错误就 **push_back** 到输出。这个版本同时解决了两个问题：判断是否保留，计算输出位置。并行后，这两个问题必须拆开。每个线程都知道自己块内保留了多少，却不知道前面块保留了多少；如果所有线程共享 push，就会锁竞争并破坏稳定顺序。

因此，稳定 filter 的优化不是直接上线程，而是先局部计数。每个块扫描自己的输入，得到保留数量；对块计数做 scan，得到每个块的输出起点；第二遍扫描时，块内按局部顺序写到全局确定位置。scan 在这里不是抽象算法，而是输出位置分配器。读者只要理解这一点，就能把同样方法迁移到 stream compaction、partition 和 shuffle offset。

这个案例还说明材料化和融合的取舍。两遍扫描看起来多读一次输入，但避免了共享 push 和动态扩容。若保留比例很低，第一遍计数成本可能很小；若输入非常大，第二遍读取会增加带宽。也可以在块内先材料化局部输出，再一次性复制到全局位置。不同方案取决于保留比例、记录大小、内存预算和稳定性要求。

错误路径也要考虑。如果某条记录格式坏了，是过滤掉、计入错误输出，还是终止整个作业？串行 **push_back** 版本可能把错误处理夹在循环里；并行版本需要让每个块输出错误计数和错误位置。否则稳定 filter 只在正常输入上正确，遇到错误时无法解释。

这个案例可以扩展到 partition。按错误和正常两类分区，是二路 filter；按事件类型分多个桶，是多路 partition；按目标 reduce 分区，就是 shuffle。每一步都在问输出位置和冲突形式。KernelPipelinePlan 应把这些信息写出来：每个内核的输入、输出位置规则、是否稳定、冲突在哪里、局部状态如何合并。

实验不要只给最终并行版本。保留串行 push、共享锁 push、两遍 scan、块内材料化四种版本。均匀保留、全保留、极低保留和随机错误输入都要测。这样读者能看到算法选择如何随数据形状改变，而不是以为 scan 永远是答案。

第二部分综合案例：从测量到布局、SIMD 和内核流水线

第二部分要回答一个工程问题：如果已经知道程序慢，怎样从证据走到高性能数据流，而不是靠灵感乱改。我们继续使用日志统计任务。输入阶段读取文本，解析阶段生成热字段 batch，计算阶段做 map/filter/histogram/top-k，输出阶段交给后续 I/O。这里先不讨论多机，也不急着讲复杂运行时，只把单机数据路径打磨清楚。

第一步是测量闭环。Benchmark harness 接收输入规模、错误行比例、key 分布、版本名和重复次数；运行前生成输入，运行后校验 reference。报告保留原始结果、min、median、尾部和异常轮说明。若某次优化只在热 page cache 下有效，报告写热 cache；若某次优化在手机连续运行后变慢，报告写温度或频率限制。测量章给出的不是工具命令，而是全书后续优化的证据格式。

第二步是布局迁移。解析器最初输出 **LogRecord** 对象数组，计算阶段逐对象读取。迁移后输出 **ParsedLogHotBatch**：事件类型列、用户 id 列、时间戳列、record id 列。冷字段留在原始行表中。这个变化让 map、filter 和 histogram 都能顺序扫描热字段，但要求错误报告通过 record id 回查。布局报告要说明字段冷热、视图寿命、批次所有权和迁移步骤。若只改结构体，不改接口，热循环仍可能被对象构造和字符串访问拖住。

第三步是 SIMD。字节扫描、数字解析和分类计数都可能向量化，但每个内核都要先有标量 reference。字节分类可以先处理简单分隔符，再处理引号、转义和非法字节；整数解析要定义溢出和错误；浮点或计数归约要定义误差或顺序。SIMD 版本不应直接替代 reference，而应作为 fast path。

输入不满足 fast path 条件时回退，回退次数也要记录，否则性能和正确性都无法解释。

第四步是内核组合。map 把字段转换成内部编码；filter 保留错误或特定事件；histogram 统计事件类型；top-k 找高频用户；join 关联用户表；partition 为后续 shuffle 生成区间。每个内核都可以单独讲，但工程中难点在组合：是否融合 map 和 filter，是否材料化中间结果，是否先局部聚合再全局合并，是否按 key 分区减少随机写。KernelPipelinePlan 要写出每个阶段的输入输出、冲突形式和输出位置规则。

第五步是数据形状。稠密事件类型适合数组桶，稀疏用户 id 适合哈希或排序聚合；热点 key 会让一个桶巨大；图式前沿会让工作量按层波动。单机高性能不是把所有数据都塞进同一种容器，而是根据形状选择执行策略。实验应同时包含均匀 key、热点 key、低选择率 filter、高选择率 filter 和长错误文本。只有这样，读者才会看到优化的适用范围。

第六步是端到端管线。局部内核快，不代表整体快。若布局转换成本大于热循环收益，总时间会变慢；若 SIMD 减少 CPU 时间但输出写入成为瓶颈，后续 I/O 才是下一步；若 top-k 局部候选太多，合并阶段会成为新瓶颈。第二部分的最终报告应显示每个阶段的服务时间、数据字节、分配次数、中间结果大小和校验结果。它训练的是从证据到设计的路径，而不是孤立技巧清单。

稠密 key 和稀疏 key 聚合和直方图经常取决于 key 空间形状。若 key 是 0 到 1023 的事件类型，数组桶非常合适：连续、无哈希、cache 友好、合并简单。若 key 是 64 位用户 ID，实际出现的 key 很稀疏，直接分配巨大数组不现实，哈希表或排序聚合更合适。若 key 分布高度倾斜，普通哈希表也可能在少数 key 上形成热点或长链。

稠密 key 的本地直方图可以写成数组。

Listing 8.8 稠密 key 的本地直方图

```
1 constexpr std::int32_t EVENT_BUCKET_COUNT = 1024;
2
3 struct LocalHistogram {
4     std::array<std::int64_t, EVENT_BUCKET_COUNT> buckets{};
5 };
6
7 void histogram_dense(std::span<const std::int32_t> events,
8     LocalHistogram& histogram) {
9     for (const std::int32_t event : events) {
10         if (0 <= event && event < EVENT_BUCKET_COUNT) {
11             ++histogram.buckets[static_cast<std::size_t>(event)];
12         }
13     }
14 }
```

这个版本速度可能很高，但它把 key 范围写进了结构。若 event id 不是小整数，或者需要动态 schema，就不能直接使用。稀疏 key 可能使用 `std::unordered_map`，但本地哈希表会引入分配和随机访问。工程中常见折中是先对输入按块排序或使用 robin-hood/open-addressing 哈希表，减少分配和指针跳转。

排序聚合和哈希聚合 Group-by 聚合通常有两条路线。哈希聚合边读边更新哈希表，平均复杂度好，适合无序输入和点更新；排序聚合先按 key 排序，再线性扫描相同 key，适合需要排序输出、key 可比较、内存局部性重要或哈希表冲突严重的场景。排序聚合多了排序成本，却可能减少随机写和分配。

选择时要看 key 数量、输入大小、输出是否需要有序、内存预算和倾斜程度。若 key 很少，哈希聚合或数组桶好；若 key 很多且输出要排序，排序聚合可能更合适；若输入已经按 key 近似有序，排序聚合收益更大。数据库执行器通常同时实现多种聚合，并根据统计信息选择。

Top-k 的两阶段设计 Top-k 不必全量排序。每个 worker 可以维护一个大小为 k 的小堆或有序数组，得到本地 top-k；最后把所有本地 top-k 合并，再取全局 top-k。若 worker 数是 p，最终合并只处理 p 乘 k 个候选，而不是全部元素。这个结构和 map-side combine 一样，先局部减少数据

量，再全局合并。

Listing 8.9 本地 top-k 候选的接口形态

```
1 struct TopCandidate {
2     std::int32_t key = 0;
3     std::int64_t score = 0;
4 };
5
6 using TopList = std::vector<TopCandidate>;
7
8 TopList merge_top_candidates(std::span<const TopCandidate> candidates,
9     std::int32_t limit);
```

Top-k 的难点是 tie-breaking 和确定性。若两个 key 分数相同，输出顺序是否按 key 排序，还是任意？分布式系统中，不确定顺序会导致测试和缓存难以复现。接口应定义 tie-break 规则，例如先按 score 降序，再按 key 升序。

Join 是组合内核 Join 是数据库和数据处理系统中的核心操作。Hash join 先构建较小表的哈希表，再扫描较大表查找匹配；sort-merge join 先按 join key 排序，再线性合并；nested-loop join 朴素但只适合极小输入或有索引的情况。Join 的成本包括构建、探测、输出放大、内存占用和数据倾斜。

Join 也是多个内核的组合：构建哈希表是聚合/索引构造，探测是 map 加随机访问，输出是 compaction 或 append，分布式 join 还需要 partition/shuffle。若某个 key 一对多匹配很多行，输出可能爆炸，成为真正瓶颈。优化 join 不能只看输入大小，还要估算输出基数。

Graph frontier 的数据结构 图遍历常用 frontier 表示当前层活跃顶点。frontier 可以是向量、队列、bitmap 或稀疏集合。向量适合前沿较小，遍历活跃顶点；bitmap 适合前沿很大，快速判断某个顶点是否在前沿；稀疏集合适合需要快速去重和遍历。方向优化 BFS 的核心就是根据 frontier 大小和边数量切换数据结构和遍历方向。

这说明数据结构不是静态选择。一个算法的不同阶段可能需要不同表示。高质量系统会测量前沿大小、边访问量和重复访问，再切换策略。单一结构在所有阶段都“还可以”，往往不如在关键阶段选择合适表示。

内核模式要按数据流和冲突分类 计算内核不是一组孤立技巧。Map、filter、histogram、join、top K、scan、partition 和 graph frontier 的差别，来自三个问题：每个输入产生多少输出，输出位置是否唯一，多个输入是否会更更新同一状态。Map 是一对一，输出位置天然唯一；filter 是零或一输出，需要决定紧凑输出位置；histogram 是多输入写少量桶，会产生冲突；join 可能一对多，输出规模依赖数据；top K 需要维护有序候选；graph frontier 既有随机邻接访问，又有去重和负载倾斜。

按这个分类，优化方向就清楚。Map 关注连续访问、SIMD 和融合；filter 关注 mask、计数和 scan；histogram 关注局部聚合和合并；join 关注构建侧、探测侧、key 分布和输出爆炸；top K 关注堆、选择算法和分块候选；graph frontier 关注邻接布局、方向切换和去重。若只是按“常见算法列表”讲，读者会觉得每节都是新知识；按数据流讲，读者能看到它们都在处理输出位置和冲突。

从热点 histogram 推导局部聚合 日志事件计数是最好的 histogram 案例。朴素版本让所有线程更新一个全局 `std::unordered_map`。它直观，因为全局答案就在一个表里；它慢，因为哈希、锁、分配和热点 key 冲突全在热路径。第一步优化是每个线程或每个块维护局部 map，阶段结束后合并。这样把高频共享写变成低频合并，代价是增加内存和合并时间。

局部聚合也要看 key 分布。若 key 数很少，可以用连续数组；若 key 很多但每块只出现少量 key，局部 map 很合适；若 key 是长字符串，intern 或字典编码可能先于聚合；若热点极强，map side combine 收益大；若 key 几乎全唯一，combine 收益小，排序或分区可能更好。教材要让读者学会先观察分布，再选内核。

Filter 不是 push back 并行化 串行 filter 常写成判断后 `push_back`。并行时，多个线程同时 push 会争用；即使用锁保护，也会把输出位置分配变成瓶颈。正确推导是两阶段：先在每个块内统计命中数量，再对块计数做 prefix scan，得到每个块的输出起点，最后每个块写入独占输出范围。

这里 scan 不是附加知识，而是 filter 并行化所需的输出位置分配机制。

这个模式也解释 SIMD filter。向量比较得到 mask，mask 的 popcount 是本向量命中数；若要压缩写，必须知道这批命中写到哪里。小范围可以用局部缓冲，批量阶段用 scan 分配全局位置。读者理解 filter 后，再看 partition、shuffle 和 compaction，就不会觉得它们无关。

Join 和输出爆炸 Join 的难点不是只选择 hash join 或 sort-merge join，而是输出规模可能远大于输入。若一个 key 在左右两边都重复很多次，输出是笛卡尔积。优化前必须估计 key 分布和输出上界，否则任何局部性能优化都可能被结果材料化压垮。Hash join 适合构建较小侧并探测较大侧，sort-merge join 适合已排序或需要顺序输出的场景，nested loop 只适合很小输入或有强过滤条件。

项目里可以让 enrich 阶段把事件和用户表连接。用户表小且稳定时，构建只读 hash table；用户表大且事件按 user id 排序时，sort-merge 可能更适合；若用户表更新频繁，需要版本或快照语义。Join 不是纯算法题，它和内存布局、并发读取、版本管理和输出材料化都有关。

Graph frontier 的负载倾斜 图算法在系统书里很有价值，因为它同时展示随机访问和负载倾斜。Frontier 中每个顶点的度数可能差异巨大，一个高出度顶点就能让某个任务变成长尾。按顶点度数均分任务不够，应该按边数或估计工作量分块。邻接表若使用 CSR，邻接数组连续，但访问的顶点属性可能随机；若需要去重 visited，原子或 bitmap 会形成共享热点。

方向切换是图遍历的重要思想。Frontier 小时，从 frontier 向外 push；frontier 大时，从未访问顶点检查是否有已访问邻居，可能更划算。这个选择展示了算法模式不是固定模板，而是根据数据规模和访问成本切换。项目不一定实现完整图引擎，但应该通过小实验让读者看到负载估计和访问布局如何影响并行算法。

内核组合的验收方式 单个内核正确不代表流水线正确。Filter 输出的顺序会影响后续 join；histogram 的局部合并顺序会影响确定性；partition 的倾斜会影响任务调度；top K 的局部候选若 K 选错会丢答案。验收要同时检查局部不变量和组合不变量。比如日志聚合管线中，filter 后记录数、partition 每桶数量、局部 histogram 总和、全局合并总和和最终 checksum 都要一致。

实验报告应给一条数据从输入到输出的路径。它在哪个 filter 通过，进入哪个 partition，更新哪个局部桶，在哪次 merge 中合并，最终写到哪个结果。这样的路径比单独列出内核名更能说明系统行为，也能帮助读者调试后续分布式 shuffle。

事故复盘：单个内核都快，组合后为什么变慢 Filter、histogram、join、partition、top K 分开看都很快，串成流水线后却变慢，常见原因是中间结果膨胀和材料化。一个 filter 选择率比预期高，join 输出突然扩大，partition 出现热点桶，top K 为了保险保留太多候选，最后内存带宽和分配器被中间数据吃光。典型内核不能只按算法名分类，更要按输出位置、冲突形式和数据流边界分类。

从这个事故可以推出选择向量、局部直方图和分区 manifest。Filter 不一定立刻复制数据，可以先产生 selection vector；histogram 先做线程局部桶，再合并；join 要估计输出上界并监控膨胀；partition 不只是取模，还要记录每个桶的范围和大小，供后续阶段读取。每个内核都要说明它消费什么、产生什么、是否保持顺序、是否可能扩容、是否引入冲突。

实验报告要把每一步输出规模写出来。只看单内核耗时，会错过端到端瓶颈。若 filter 快但产生的 selection vector 让后续随机访问变多，收益可能被抵消；若 partition 快但热点桶过大，下一阶段会长尾；若 join 局部最快但分配次数暴涨，系统整体退化。内核模式章节的目标是让读者把函数调用链改写成数据流图，再按图检查材料化、冲突和局部性。

内核组合实验判读 内核组合要看每一步输出规模。filter 后剩多少，histogram 局部 map 有多大，join 是否输出膨胀，partition 是否倾斜，Top K 是否保留足够候选。单个内核时间下降但材料化字节上升，端到端可能更差。判读时沿一条记录追踪它通过哪些内核，比只看函数耗时更可靠。

这一段也服务于复习。读者合上书后，应该能凭本章的判读口径重新审查自己的代码：有没有最小正确版本，有没有能击穿旧方案的输入，有没有状态记录，有没有资源指标，有没有失败后的恢复动作。若五个问题缺两个以上，就说明实现还停在演示层，而不是工程层。

Filter、histogram 和 join 组合后的验收 单独测试 filter 时，只要保留元素正确就行；接到 histogram 后，还要保证过滤后的 record id 没有丢；再接 join 后，还要保证输出膨胀可控；最后接 Top K 后，还要保证局部候选足够覆盖全局答案。组合管线的验收比单内核复杂，因为每一步都改变了后续输入形状。

可以让一条记录贯穿全链路：它为什么通过 filter，进入哪个 partition，更新哪个本地桶，在 join 中匹配几条维表记录，最后是否进入 Top K 候选。这个追踪能发现许多局部测试看不到的问题，例

如 filter 后顺序改变导致 join 假设失效, 或者 map side combine 后丢失了错误报告所需的记录身份。

材料化也要有预算。每个 stage 输出多少字节、保留多久、是否可重算、是否需要 checksum, 都影响端到端。高性能并不总是少材料化; 可靠恢复有时需要在边界保存中间结果。关键是材料化必须有理由, 而不是每个函数都随手输出一个大 vector。

实验课: 用日志管线串起 filter、histogram、join 和 Top K 输入日志先经过 filter, 只保留目标事件; 然后 histogram 统计用户或事件; 再 join 一个小维表补充用户分组; 最后输出 Top K。朴素版本每一步都材料化 vector, histogram 用全局 map, join 不估计输出规模, Top K 直接全排序。它容易写, 但很快暴露内存峰值和热点 key 问题。

改进路线不要一次到位。先把 filter 改成 count 加 scan, 确保输出区间唯一; 再把 histogram 改成线程本地 local map, 最后合并; 然后为 join 选择构建侧和探测侧, 并记录输出规模; 最后把 Top K 改成局部候选加全局合并。每一步都记录输入条数、输出条数、材料化字节、最长 partition 和最终 checksum。

判读时要看组合不变量。filter 后记录数加上被过滤数应等于输入数; histogram 所有桶计数总和应等于 filter 输出数; join 输出规模要和 key 重复度解释一致; Top K 候选不能丢失全局前 K。若某一步局部正确但组合不变量失败, 就说明接口边界没有写清。

这个实验让读者理解内核模式不是算法清单, 而是数据流转换。每个模式都回答一个问题: 输出有多少, 输出位置在哪里, 冲突如何消解, 材料化是否值得。掌握这四个问题, 比背更多内核名更重要。

内核组合练习验收重点 合格答案要按数据流说明每个内核改变了什么: filter 改变记录集合, histogram 合并状态, join 可能放大输出, Top K 丢弃非候选。答案要给每一步输出规模和组合不变量。若只分别介绍算法复杂度, 没有说明组合后的材料化和冲突, 就不合格。

答案还要写“为什么不采用更复杂方案”。例如没有证据证明队列是瓶颈, 就不应把锁队列换成无锁; 没有证明输出顺序可变, 就不应随意重排; 没有证明错误路径能恢复, 就不应只提交正常路径。能解释不做什么, 说明读者开始具备工程判断。

最后, 答案必须包含可复查材料。至少有输入摘要、关键状态、期望输出、实际输出、运行命令和未验证边界。没有这些材料, 老师或同伴无法判断结论是否成立。第二册的作业不是让读者写漂亮段落, 而是训练读者把系统问题变成可验证证据。

项目实现: 典型计算内核模式 本章对应的贯穿项目实现不要求一次写成完整框架, 但必须有最小可运行切片。这个切片包含三个部分: 一个 reference, 一个带本章机制的实现, 一个能击穿 reference 之外隐含假设的测试。Reference 用来固定语义, 机制实现用来展示本章知识, 测试用来证明新增复杂度确实解决了问题。

代码组织上, 不要把所有内容放进一个大文件。基础类型、输入生成、核心机制、报告输出和测试应分开。即使当前只是教学项目, 也要让目录能表达责任边界。这样后续章节接入时, 不会因为找不到状态归属而重新发明一套对象。

每个实现都要输出一段机器可读摘要。摘要至少包含输入规模、关键状态、错误数、输出校验、耗时和未验证边界。这里的机器可读不要求复杂格式, 简单的 key value 文本也可以。关键是让脚本和读者都能复查, 而不是只能看屏幕截图。

测试应包含正常路径、边界路径和失败路径。正常路径证明功能可用; 边界路径证明长度、容量、尾部、空输入、关闭或超时没有遗漏; 失败路径证明本章机制不是装饰。若失败路径写不出来, 往往说明本章概念还没有落到工程对象上。

报告最后写一个取舍段。新增机制带来了什么好处, 增加了什么成本, 哪些场景不应该使用它, 下一章会复用哪个字段或对象。这个取舍段非常重要, 因为系统工程不是把所有高级机制都加进去, 而是在证据和约束下选择合适的边界。

调试笔记: 管线结果异常时沿记录追踪 内核组合出错时, 选一条记录从输入追到输出。它是否通过 filter, 分到哪个 partition, 更新哪个局部桶, join 产生几条记录, Top K 阶段是否保留。若单步都正确但最终错, 通常是顺序、身份或材料化边界出问题。沿单条记录追踪, 比只看每个函数的单元测试更容易定位组合错误。

复查清单: 典型计算内核模式 复查本章时, 先检查 reference 是否仍然存在。没有 reference, 后面的并发、优化、异步或分布式改造都无法证明语义没有变。再检查压力输入是否尖锐: 它应当攻击本章最核心的边界, 而不是只把数据量放大。第三, 检查状态是否可观察: 关键对象的身份、状

态、错误和提交结果应能在报告中找到。

第四，检查代码示例是否说明禁止行为。调用者不能做什么，往往比能做什么更能体现工程质量。第五，检查实验是否写出未验证边界；当前设备、权限或输入无法证明的结论，必须标注。第六，检查本章对象是否能被下一章复用，不能复用就说明边界还不稳。

完成这六项后，再看文字是否连贯。每个概念都应从问题推出，每个案例都应从朴素方案走到改进方案，每个实验都应能解释一个判断。若某段话放到其他章节也成立，就把它改成本章的具体对象、具体输入和具体失败。

8.8 工程案例：用数据流和冲突形状组织计算内核

计算内核不应按流行词分类，而应按数据流、输出形状和冲突形状分类。Map、reduce、histogram、top-k、join、sort、graph traversal 看起来差别很大，底层问题却反复出现：输入是否独立，输出位置是否已知，多个元素是否写同一状态，是否需要排序或分区，工作量是否倾斜。按这些问题组织，读者才能把一个内核的经验迁移到另一个内核。

一、Map 内核最简单，也最容易被带宽限制 Map 的语义是每个输入元素独立产生输出元素。它没有跨元素依赖，输出位置通常已知，因此容易并行和向量化。可是正因为计算简单，它很快会受内存带宽限制。把每个元素多做几条算术可能无所谓，多读一个冷字段却可能很贵。Map 内核优化首先看数据布局、连续性和写分配，而不是盲目加线程。

日志项目中的字段归一化就是 map。每条记录把 timestamp 转成 delta，把 event type 编码成整数，把 user id 映射到 bucket。若输入是列式热字段，map 可以顺序扫描；若输入是对象数组，map 会搬动冷字段；若输出和输入重叠，编译器可能保守。Map 看似简单，却能检验前面布局章是否真正落地。

二、Histogram 的核心是写冲突 Histogram 读每个元素的 bucket，然后增加对应计数。输入读取通常简单，难点是多个元素写同一 bucket。若 bucket 很少，全局数组会成为热点；若 bucket 很多且随机，cache miss 增多；若 key 极端倾斜，一个 bucket 会支配成本。优化方式包括线程本地 histogram、分片 bucket、按 key 预分区、热点 key 特殊处理和 SIMD 辅助分类。

选择策略取决于 bucket 数和分布。256 个桶的字节 histogram 可以每线程一份小数组，最后合并；百万 key 的事件统计可能需要哈希表和局部 map；热点 key 可以先单独计数，减少哈希探测。报告要记录 key 基数、倾斜度、线程数、局部表大小和合并成本。Histogram 是共享写问题的典型训练场。

三、Top-k 是减少数据移动的案例 Top-k 不需要完整排序。若只要最大的 k 个元素，可以维护局部小堆，最后合并；若数据分布稳定，可以先采样估计阈值，再过滤候选；若需要稳定 tie-break，就要把 key 和原始位置一起比较。Top-k 的核心是避免把大量无关元素带入全局排序。

工程细节也很多。k 很小时，小数组插入排序可能比堆快；k 很大时，partial sort 或 nth element 更合适；并行 top-k 需要每个线程局部候选，最后合并。若元素包含大对象，比较时应移动索引而不是移动对象本体。这个案例能把算法复杂度、数据移动和布局连接起来。

四、Join 和分区把随机访问变成阶段化访问 Join 的朴素形式是对每个左表元素查右表哈希表。若右表很大，随机访问和 cache miss 会主导。常见改进是先按 hash 分区，把大 join 拆成多个较小 join，让每个分区更容易进入 cache；或者对小表建立紧凑哈希表；或者使用 sort merge join，把随机访问换成顺序扫描和排序成本。

Compute Systems Engine 不一定实现完整数据库 join，但 shuffle 和 reduce 已经有类似结构：按 key 分区，本地聚合，目标端合并。Join 案例能提醒读者，分区不是分布式专属技术，它也是单机减少随机访问和控制工作集的工具。

五、Graph traversal 的困难是前沿和邻接分布 图遍历常被复杂度 $O(V + E)$ 概括，但实际性能受顶点度分布、前沿大小、邻接布局和访问随机性影响。低度均匀图和社交网络幂律图完全不同。前沿很小时，线程并行开销可能大；前沿很大时，写 visited 会产生原子或位图冲突；高度顶点会造成负载倾斜。

优化图遍历要从数据结构开始。CSR 连续保存邻接表，适合顺序扫描；顶点重编号可以改善局部性；按前沿大小选择 top-down 或 bottom-up BFS 可以减少边检查。并发写 visited 时，可以用位图、分块局部 visited 或按顶点范围分配所有权。图内核把随机访问、冲突写和负载倾斜集中在一起，是系统性能的综合练习。

六、材料化和流水线的取舍 多个内核串起来时，可以每阶段材料化完整中间结果，也可以流水线逐块传递。材料化让调试、重试和阶段测量更容易，但会增加内存和 I/O；流水线减少中间数据，却让背压、生命周期和错误传播复杂。没有绝对答案，要看中间数据大小、失败恢复要求、下游速度和观测需求。

日志项目中，解析后的 HotBatch 可以材料化到内存，便于多次统计；shuffle block 需要材料化到磁盘或文件，因为后续 reduce 和恢复要读取；某些临时 mask 可以不材料化，只在块内用于写出。每个材料化点都应有理由：为了复用、为了恢复、为了分区、为了异步，还是为了测量。

七、组合内核要保护阶段语义 单个内核正确，不代表组合后正确。Map 输出的 key 编码要和 histogram bucket 一致；partition 的分区函数要和 reduce 路由一致；top-k 的 tie-break 要和最终输出排序一致；graph traversal 的 visited 更新要和并发策略一致。阶段之间需要合同，不能靠函数名暗示。

项目可以为每个内核写 **KernelContract**：输入字段、输出字段、是否保持顺序、是否可重试、是否有外部副作用、是否需要稳定 seed、是否允许近似。这个合同会被任务运行时、I/O 和分布式章节使用。计算内核不是孤岛，它是整条系统链路中的一段。

八、实验交付：KernelPatternSuite 本章实验套件至少包含 map、histogram、top-k、partition、简单 BFS 五类。每类都有朴素版本、局部优化版本和压力输入。压力输入不是只放大规模，而是改变分布：热点 key、长尾文本、稀疏命中、高度顶点、小输入、大输入。报告写清每个内核主要受什么限制：带宽、随机访问、共享写、排序成本、负载倾斜还是输出材料化。

验收时要求读者做迁移解释。线程本地 histogram 的思想如何迁移到 map-side combine；partition 的输出区间如何迁移到 shuffle block；top-k 的局部候选如何迁移到分布式两阶段 top-k；BFS 的前沿如何迁移到任务图 ready set。能迁移，说明学的是模式；不能迁移，说明还停留在题目答案。

8.9 贯穿案例：一个 top-k 看起来正确，为什么合到流水线就错了

计算内核章节不能只讲 map、reduce、histogram、top-k、partition 的单点写法。真实项目里，内核会组合。考虑一个日志分析需求：先解析记录，再按用户聚合计数，再取 top-k 用户。单机测试中，每个线程各自维护本地 top-k，最后把所有候选合并，输出看起来正确。上线后发现结果偶尔缺少真正的第 k 名。原因不是 top-k 代码完全错，而是组合合同不完整：本地 top-k 只保留每个分片内前 k，若某个用户在每个分片都排第 k+1，但总和很高，它会在局部阶段被丢掉，最终阶段永远看不到。

这个事故逼出“内核组合合同”。局部 top-k 只有在聚合粒度和候选保留策略满足条件时才安全。若 key 的贡献分散在多个 chunk，必须先按 key 做完整 combine，或者本地保留更大的候选集并给出误差界。单个内核正确，不代表组合后正确。第 8 章的重点应是从语义、数据分布和材料化点推导内核形态，而不是把内核名字列成清单。

一、先问中间结果是否保留足够信息 Map 后的记录包含完整 key 和 value；局部 histogram 保留每个 key 的局部计数；局部 top-k 只保留少数候选，丢掉长尾。信息一旦丢掉，后续阶段无法恢复。若最终语义需要全局精确 top-k，局部阶段不能丢掉可能在全局进入 top-k 的 key，除非有数学保证或误差界。材料化多少中间信息，是性能和正确性的共同选择。

二、histogram 和 top-k 的顺序不能随便换 如果 key 基数不大，可以先完整 histogram，再从全局计数里选 top-k。若 key 基数巨大，完整 histogram 成本高，可以先做局部 heavy hitter 算法，但要明确它是近似还是精确。若业务要求精确，就不能把近似算法包装成优化。算法模式不是积木随便拼，每次交换顺序都要证明输出语义保持。

三、partition 影响下游工作量 把记录按 key partition 后，每个 reduce 只处理一部分 key，top-k 可以先在各 partition 求候选，再全局合并。这个设计正确的前提是同一个 key 的所有贡献被送到同一 partition，或者有明确二级合并规则。若 partition 函数版本不一致，同一个 key 被拆到多个 reducer，top-k 就会漏或重。计算内核和分布式路由在这里连成一条线。

四、材料化点决定恢复和调试 如果 parse、combine、top-k 全部流水线，不保存中间 partial，内存少、速度快，但结果错时很难定位。若保存局部 histogram 和 partition manifest，故障后可以比较每个阶段。项目可以提供两种模式：fast 模式尽量流水线，audit 模式材料化关键中间结果。教材要说明这是合同选择，不是调试临时开关。

五、压力输入要攻击内核组合 普通均匀输入很容易让局部 top-k 看起来正确。压力输入应构造“分散强 key”：某个 key 在每个 chunk 中都不是局部 top-k，但全局总和很高；还要构造热点 key、长尾 key 和 tie-break 相同的 key。只有这些输入通过，top-k 组合才可信。压力输入不是把数据量放大，而是攻击算法假设。

六、把 KernelContract 写进代码边界 每个内核应声明：是否保持顺序，是否丢弃信息，是否精确，是否可重试，是否有外部副作用，是否依赖随机 seed，输出是否可合并。下游只能在合同允许的条件下使用它。这个合同能阻止把近似局部 top-k 当成精确全局 top-k，也能帮助第 18 章自动生成 run report。计算内核的工程质量来自可组合合同。

8.10 案例深化：把内核模式组合成流水线

单个内核讲清以后，还要讲组合。真实系统很少只做一次 map 或一次 reduce，而是读取、解析、过滤、聚合、分区、写出、合并连续发生。组合后，单个内核的最优选择可能不再最优，因为中间数据大小、材料化点、背压和恢复边界都会改变成本。

一、解析后是否立即聚合 日志解析后可以先材料化所有记录，再统一聚合；也可以边解析边局部聚合。前者便于调试和多次复用，后者减少中间数据。若后续有多个分析任务，材料化 HotBatch 可能值得；若只做一次计数，直接 combine 可以减少内存。选择取决于复用次数、错误诊断和恢复需求。

这个问题不能只用性能判断。直接聚合可能让错误定位更难，因为原始记录和 partial 的关联变弱；材料化可能让失败后从中间状态恢复更容易。系统设计常在“少搬数据”和“保留可解释状态”之间取舍。

二、排序和分区的顺序会改变成本 如果先全局排序再分区，排序成本高但后续顺序好；如果先按 partition 分桶，再在桶内排序，可以减少工作集；如果只需要按 key 聚合，不需要完整顺序，先 hash combine 再排序输出可能更好。算法组合要按最终语义推导，不要看到排序就直接调用 sort。

项目中的输出若只要求按 key 排序，完全可以先本地 combine，再合并 key，再最终排序。中间 block 不必全局有序。这样减少 shuffle 字节和排序规模。这个例子说明，明确最终输出语义能避免不必要的中间强语义。

三、流水线需要背压合同 把多个内核流水线连接后，上游可能比下游快。解析比写出快，filter 比 reduce 快，map 比 shuffle 快。若中间队列无界，系统用内存掩盖背压；若队列有界但错误处理不清，关闭时可能丢数据。每个阶段连接处都要有 QueueContract：容量、关闭、取消、错误传播和所有权。

内核模式因此和同步章节相连。一个 map 内核本身没有锁，但它前后的队列有不变量；一个 reduce 内核本身纯计算，但它的输入 block 来自 manifest。组合流水线的正确性来自阶段合同，而不只是内核代码。

四、组合实验 本章补充一个端到端内核流水线实验：parse、filter、local combine、partition、write manifest。每次只改变一个阶段，例如把 parse 从对象输出改成 HotBatch，或把 combine 从全局 map 改成本地 map。报告记录每阶段字节数和时间。若总时间改善小于局部改善，要解释成本迁移到哪里。

8.11 补强案例：图遍历如何暴露所有系统问题

图遍历是计算内核的综合题。它有随机访问、负载倾斜、并发写、前沿变化、内存布局 and 任务调度。即使项目主线是日志统计，图遍历也值得讲，因为它迫使读者把前面学过的概念同时使用。

一、CSR 是数据布局选择 邻接表如果由 vector of vector 保存，每个顶点邻接数组分散，遍历会频繁追指针。CSR 把所有边放进一个连续数组，顶点只保存 offset。扫描某个顶点邻接时是连续访问，遍历前沿时可以按 offset 找边。CSR 牺牲插入灵活性，换取遍历局部性。

二、前沿决定并行形态 BFS 每层有一个 frontier。前沿小，开很多线程不划算；前沿大，并行扫描边更有效；高度顶点会让某个任务很重。任务切分可以按顶点，也可以按边范围。按顶点简单但负载不均，按边范围复杂但更均衡。报告要记录每层 frontier 大小和边数。

三、visited 是共享写热点 多个线程可能同时发现同一个顶点，需要原子测试或分区所有权。全局 atomic bitset 简单，但热点顶点会竞争；按顶点范围分配 owner 可以减少冲突，但需要跨线程

发送发现结果；每线程本地 next frontier 后合并，也是一种局部化策略。图遍历把原子、partition 和队列连接起来。

四、实验 准备规则网格图、随机图和幂律图。比较 vector of vector 和 CSR，比较按顶点切分和按边切分，比较全局 visited 和分区 visited。报告解释哪种图触发哪种瓶颈。这个案例能帮助读者把内核模式迁移到陌生问题。

8.12 补充案例：近似算法的系统合同

不是所有计算都需要精确结果。近似 distinct count、采样 top-k、概率过滤器、sketch 都能用更小内存换误差。但近似算法必须有系统合同：误差范围、合并方式、随机种子、可复现性和适用输入。

一、近似不是随便不准 例如 HyperLogLog 估计基数，它有明确误差模型和可合并结构。Bloom filter 有假阳性率，不应有假阴性。采样 top-k 有概率保证。近似算法要写数学合同和工程合同，不能只说“差不多”。

二、合并和分布式 很多 sketch 支持合并，适合分布式。每个 worker 构建局部 sketch，reduce 合并。合并是否结合、是否依赖随机 seed、是否版本兼容，都要写清。否则不同 worker 的 sketch 不能安全合并。

三、何时使用近似 监控、预估、优化器统计可以接受近似；账务、审计、最终计数通常不能。项目主线的最终日志计数应保持精确，但可以用近似结构估计 key 基数，帮助选择数据结构和 partition 数。近似用于控制决策时，要考虑误差导致的错误选择。

四、实验 实现精确 distinct count 和一个简单 bitmap 或 sketch 估计。比较内存、误差和合并成本。报告写明哪些结果只能用于估计，不能用于最终输出。这样计算内核章节会更接近真实系统。

8.13 从具体 kernel 提炼算法骨架

计算 kernel 不是一些孤立技巧的集合。一个 kernel 之所以值得单独研究，是因为它把数据访问、计算密度、边界条件和输出语义压缩在一个高频小区域里。矩阵乘、直方图、图遍历、字符串扫描、top-k、压缩编码看起来很不同，但都可以从几个问题开始：输入如何被切块，热数据能否复用，输出是否有共享写，错误路径是否进入热循环，是否能把不可预测工作移到批处理阶段。只要把这些问题问清，kernel 优化就能从经验转成方法。

一、矩阵乘从数据复用开始 朴素矩阵乘慢，不是因为三重循环这个形式天然糟糕，而是因为它没有让缓存中的数据被充分复用。分块算法把矩阵拆成小块，使一个块进入缓存后被多次使用。继续往下，寄存器分块让一组累加值停留在寄存器中，SIMD 让多个乘加并行，线程切分让不同核心处理不同输出区域。每一步都来自同一个问题：读进来的数据能否在被赶出缓存前多做几次有用计算。若输入矩阵很小，复杂分块反而不划算；若矩阵很大，块大小又必须受缓存、TLB 和线程调度约束。

二、直方图从共享写冲突开始 直方图的计算本身很简单：读一个值，找到 bucket，计数加一。难点在于多线程时大量元素可能写同一个 bucket。全局原子计数器会制造热点；每线程私有直方图可以把冲突推迟到合并阶段；按输入分区或按 bucket 分区又会改变内存访问模式。若 bucket 很少，私有表合并便宜；若 bucket 很多，私有表内存巨大；若 key 分布极度倾斜，还需要单独处理热点。直方图把“算法复杂度低但系统成本高”这件事讲得很清楚。

三、图遍历从不规则性开始 图算法的难点通常不是单个边的处理，而是顶点度数差异、访问随机、前沿规模变化和重复访问。BFS 在小前沿时适合按顶点展开，在大前沿时可能改成 bitmap 或方向优化；PageRank 看起来是线性迭代，却会被稀疏矩阵布局和缓存行为限制；最短路算法还要处理优先队列和松弛顺序。图 kernel 教给读者的是：当输入结构不规则时，负载均衡、去重、局部性和近似策略往往比单条指令优化更重要。

四、字符串和解析 kernel 从分支密度开始 文本解析常被低估。每个字符都可能触发分支，错误路径和正常路径混在一起，变长字段让输出位置难以提前知道。优化时可以先把扫描和语义动作分开：第一阶段批量找分隔符、换行和特殊字符，第二阶段再解析字段；可以把少见错误路径移出热循环；可以为常见 ASCII 输入建立快路径，同时保留完整 Unicode 或错误处理的慢路径。这里的原则和流水线章相连：把可预测的主路径变短，把不可预测的复杂性移到低频位置。

五、kernel 报告要写输入族 单个输入无法代表 kernel。矩阵乘要测试不同形状和大小，直方图要测试均匀分布和倾斜分布，图算法要测试低度图和幂律图，字符串解析要测试短行、长行、错误行和多字节字符。若只挑最友好的输入，优化会欺骗自己。高质量 kernel 章节要让读者学会设计输入族，并把每个输入族对应的瓶颈写出来。这样后续做推理引擎算子、分布式 shuffle 或日志处理时，才不会把一个样例上的速度误当成普遍结论。

8.14 Kernel 选择和拆解的三层方法

当一个系统里出现热 kernel 时，最先做的不是改代码，而是判断它属于哪一类瓶颈。第一层看数据移动：每处理一个元素要读写多少字节，是否有随机访问，是否会重复使用同一块数据。第二层看计算形状：是否是简单算术、分支密集、查表、归约、排序还是图遍历。第三层看输出结构：是否写共享结果，是否需要稳定顺序，是否需要 compact。三层合在一起，才决定是改布局、改算法、改并行方式还是改 I/O。

一、带宽型 kernel 优先减少字节 如果每个元素只做很少计算，却读写大量字段，优化方向通常是减少数据移动。可以拆冷热字段、压缩编码、只读必要列、批量处理、避免重复写回。此时手写复杂算术指令意义不大，因为瓶颈不在计算单元。报告要用字节数和带宽上限说明原因：每秒处理多少元素意味着每秒移动多少字节，和机器可提供带宽相差多少。

二、计算型 kernel 优先提高复用和并行 如果数据进入缓存后能做大量计算，比如矩阵块、卷积、哈希批处理，就要看寄存器复用、SIMD、展开、分块和线程切分。计算型 kernel 的危险是过度展开导致代码体积和寄存器压力上升。优化不应只追求单个循环体漂亮，还要看整体输入大小、缓存块大小和调度成本。计算密度越高，微架构细节越重要；计算密度越低，数据移动越重要。

三、分支型 kernel 优先改变输入形态 分支密集的解析、分类和状态机处理，常常不是靠删除某个 if 解决。更有效的方式是把输入预分类、把冷路径移出热循环、把常见格式设为快路径、把错误记录延后批处理。若输入本身随机，分支预测器很难学习；若先按类型分桶，后续每个桶里的分支就更稳定。这里的优化是改变工作组织方式，而不是盲目追求 branchless。

四、共享写型 kernel 优先拆输出 直方图、top-k、错误列表、日志聚合都可能写共享输出。优化方向通常是线程本地 partial、分区输出、两阶段合并或 scan 分配位置。若直接把共享写改成 atomic，低线程数可能可接受，高线程数会迅速暴露一致性流量。共享写型 kernel 要同时看正确性和写入热点，不能只看计算复杂度。

五、把 kernel 放回系统路径 一个 kernel 单独快，不代表系统快。若 kernel 输出格式让下游难以写出，或者为了局部速度制造大量临时内存，整体可能变慢。最终报告要把 kernel 放回读取、调度、写出和恢复路径中。优秀的 kernel 优化应让局部速度、系统资源和语义边界同时变清楚，而不是只让某个基准数字变好看。

8.15 案例深化：内核组合比单内核更重要

典型计算内核不能只按 filter、map、reduce、join 背名字。真实系统慢下来，往往不是某个内核单独慢，而是内核之间的中间结果失控。一个 filter 选择率从百分之一变成百分之八十，后面的 join 输出暴涨；一个 partition 把热点 key 打到同一桶，后面的 reduce 长尾；一个 top K 为了准确保留太多候选，内存带宽被材料化吞掉。内核组合要从数据流图分析。

一、每个内核写清输入和输出规模 Filter 的输出规模由选择率决定，histogram 的输出规模由 key 数和桶数决定，join 的输出规模由匹配度决定，partition 的输出规模虽然等于输入，但改变了顺序和局部性。报告中每一步都写 records in、records out、bytes out、temporary bytes 和最大桶大小。没有规模账本，单内核时间没有解释力。

二、选择向量不是临时技巧 Filter 后直接复制记录会增加带宽；只保存 selection vector 可以延迟材料化，让后续阶段决定是否真的需要原记录。代价是后续访问可能变成间接读取。选择向量适合过滤后仍需要回原数据的场景，不适合后续完全只读少数字段的场景。这个判断需要结合数据布局章节，而不是孤立决定。

三、局部 histogram 解决写冲突 全局 histogram 用锁或 atomic 会形成共享写热点。局部 histogram 让每个线程写自己的桶，最后合并。桶很多时，局部表内存大；桶少时，false sharing 要小心；key 分布倾斜时，某些桶合并仍会长尾。内核模式不是固定模板，而是围绕冲突、容量和合并

成本做选择。

四、Join 要先估计爆炸风险 Join 的输出可能远大于输入。若两个表某个 key 都很热，匹配时会爆炸。系统可以先采样估计 key 分布，对热点 key 单独处理，或者选择 semi-join、bloom filter、分区 join。教材不需要实现完整数据库优化器，但要让读者知道 join 不是一个普通双循环，而是可能改变整个流水线规模的内核。

五、把内核结果交给运行时 内核输出的 manifest、桶范围、选择向量和局部 partial，都应成为运行时可见对象。这样任务运行时可以按桶调度，I/O 层可以按 block 写出，分布式层可以按 partition shuffle。若每个内核只返回一个 vector，后续系统无法利用结构信息。内核章节要为后面的并行和分布式留下接口，而不是停在单函数优化。

8.16 案例复盘：内核组合后的中间结果账本

Filter、join、partition、top K 单独都快，组合后可能因为材料化和倾斜变慢。Filter 选择率变化、join 输出膨胀、partition 热点桶都会改变端到端成本。内核模式要按数据流和输出规模讲，而不是按函数名背。

每一步都写规模 报告记录 records in、records out、temporary bytes、最大桶、join 放大倍数和最终 checksum。Selection vector 可以推迟复制，但会引入间接访问；局部 histogram 减少共享写，但增加合并；partition 改变顺序和局部性。这些都是成本交换。

接口留给后续系统 内核输出不应只是 vector。选择向量、桶范围、局部 partial、partition manifest 都应对运行时可见。后续任务调度、I/O 写出和分布式 shuffle 需要这些结构信息。单内核优化如果抹掉边界，会让系统层更难做。

8.17 本章习题

习题与作业

习题一：把一个日志统计任务拆成 map、filter、partition、reduce、top-k 五个阶段。每个阶段写出输入、输出、是否可并行、是否可能向量化、是否有共享写和主要瓶颈。

习题二：实现或设计 parallel count-if。要求先写全局原子反例，再写线程本地 partial 版本。报告中解释为什么 relaxed 原子不能解决共享写扩展性问题。

习题三：设计 stream compaction。写出统计、扫描偏移、写输出三阶段，并说明每阶段的正确性测试。要求覆盖选择率为 0、选择率为 100

习题四：为直方图选择三种策略：全局原子、本地桶、稀疏本地 map。分别说明适用的桶数量、输入分布、线程数和内存占用。

第零部分

多线程、同步与内存模型

第 9 章

线程、调度与亲和性

先看一个“线程越多越慢”的实验：日志解析任务从 1 个线程加到 4 个线程有提升，加到 16 个线程后吞吐下降，`perfstat` 里 `context-switches` 和 `migrations` 都上升。代码里每个线程做的工作没有变，但操作系统调度、SMT 资源共享、线程迁移、锁等待和 I/O 阻塞一起改变了执行图。线程不是魔法倍增器，它只是让更多执行实体进入同一台机器竞争资源。

本章从这种扩展曲线异常出发讲线程、调度和亲和性。线程是操作系统调度的执行实体；多线程程序能利用多核心，也会引入上下文切换、调度噪声、缓存热度丢失、同步等待和资源竞争。理解线程，必须同时看用户态任务图和内核调度现象。

9.1 从任务到线程

创建八个线程不等于拥有八个物理核心。硬件可能有 SMT，一个物理核心暴露两个逻辑 CPU；系统里还运行着其他进程；线程可能被迁移到不同核心。线程迁移会破坏缓存热度，过多线程会增加上下文切换。

CPU-bound 程序通常不应远多于核心数，I/O-bound 程序可以有更多并发等待。判断依据不是经验数字，而是运行时是否在计算、等待锁、等待 I/O 或被调度器抢占。

用户线程和内核线程 C++ 的 `std::thread` 通常映射到操作系统线程。操作系统调度的是内核可见的执行实体，它拥有寄存器上下文、栈、调度状态和优先级。用户态协程则不同，协程切换通常不进入内核，它把等待点显式暴露给运行时。协程适合组织大量 I/O 等待，不会自动让 CPU 密集代码并行。

这一区分很重要。把 CPU 密集任务放进单线程协程事件循环，不会使用多个核心；把大量 I/O 等待都变成内核线程，可能造成过多栈内存、调度成本和上下文切换。运行时设计要把计算并行和等待并发分开。

9.2 调度、阻塞和上下文切换

上下文切换需要保存和恢复执行状态，可能破坏 cache 和 TLB 局部性。频繁阻塞、线程过多、锁竞争和系统调用都可能增加切换。性能计数器里的 `context-switches` 和 `migrations` 可以帮助定位这类问题。

Linux CFS 调度器大体追求公平，它会根据虚拟运行时间选择任务。实时调度策略和普通调度策略不同，容器和 cgroup 也会改变 CPU 配额。benchmark 时若机器上有其他负载，或者进程受 cgroup 限制，结果会明显波动。严谨实验应记录负载、CPU governor、容器限制和是否独占机器。

调度器不是只在“线程很多”时才重要。即使线程数等于核心数，线程也可能因为缺页、锁等待、I/O、信号、中断、抢占和 cgroup 配额离开 CPU。对于低延迟系统，一次迁核可能让热缓存失效；对于吞吐系统，频繁上下文切换会减少有效计算时间。性能报告如果只写线程数，不写线程是否真正持续运行在 CPU 上，就缺少关键证据。

Linux 调度源码阅读不建议一开始追完整策略。更好的入口是从一个用户态现象出发：为什么 `perfstat` 里 `context-switches` 增加，为什么线程会 migration，为什么绑定 CPU 后波动变小，为什么 cgroup 限额下吞吐呈周期性抖动。围绕这些问题，再去看调度实体、runqueue、唤醒路径和上下文切换路径，读出的内容才有边界。

阻塞、睡眠和自旋 线程等待锁时，可以自旋，也可以睡眠。自旋避免进入内核，适合等待时间极短的场景；睡眠释放 CPU，适合等待时间较长的场景。很多锁实现会先短暂自旋，再进入 `futex` 等内核等待路径。这样低竞争时快，高竞争时不至于烧满 CPU。

选择自旋还是阻塞，本质是在 CPU 时间和唤醒延迟之间取舍。如果临界区很短且核心充足，自旋可能好；如果锁持有者被抢占或等待 I/O，自旋会浪费大量周期。不要在没有测量的情况下把自旋当成高性能。

`futex` 是理解 Linux 用户态锁的重要入口。常见 `mutex` 在无竞争时尽量只走用户态原子操作；

竞争严重时，线程通过 `futex` 进入内核睡眠，锁释放时再唤醒等待者。这样做把低竞争 `fast path` 和高竞争 `blocking path` 分开。读锁性能时，应区分用户态自旋成本、`futex` 睡眠成本、唤醒成本和被唤醒后重新竞争锁的成本。

9.3 亲和性、拓扑和移动设备

线程亲和性让线程倾向或固定在某些 CPU 上运行。它可以改善缓存局部性和 NUMA 放置，也可能降低调度器自由度。使用亲和性前要理解拓扑：物理核心、SMT 线程、NUMA 节点和共享缓存层级。

亲和性实验要有对照。只绑定线程不绑定内存，可能仍然访问远端页面；只绑定内存不绑定线程，线程迁移后也可能变成远端访问。合理实验至少比较未绑定、只绑线程、只绑内存、线程和内存都按节点绑定几种情况。若平台没有 NUMA，也应记录物理核心和 SMT 拓扑，避免把两个重负载线程绑到同一个物理核心的两个逻辑 CPU 上后误判算法慢。

线程生命周期 频繁创建销毁线程成本高，也让任务调度不可控。线程池把线程生命周期和任务生命周期分离，适合大量短任务。但线程池不是万能的，如果任务太小，调度成本会超过计算成本；如果任务阻塞，线程池可能被耗尽。

线程局部存储 线程局部存储让每个线程拥有独立对象，适合缓存临时缓冲、统计计数和避免共享写。它可以减少锁竞争，但也可能增加内存占用，并让跨线程汇总更复杂。线程池里使用线程局部对象时，还要注意任务迁移：同一个逻辑请求的不同阶段可能被不同 worker 执行，不能把请求状态误放到线程局部。

9.4 线程池语义和配置

线程池把固定 worker 暴露成任务执行资源，但它不会自动解决调度语义。CPU 密集任务应该限制并发度，避免超过核心数太多；阻塞 I/O 任务如果占住 worker，可能让计算任务饥饿；任务内部再提交子任务并等待，可能造成线程池死锁。设计线程池时，要明确任务是否允许阻塞、是否允许嵌套提交、是否支持 work helping、是否需要独立 I/O 池。

线程池还会改变线程局部数据的含义。线程局部缓冲属于 worker，不属于请求。请求如果跨多个任务执行，就不能假设后续阶段还能访问同一个线程局部对象。运行时应把“worker 本地缓存”和“请求上下文”分成两个概念。

从任务到线程的映射 并发程序里最容易混淆的是任务和线程。任务是逻辑工作单元，例如处理一段数组、解析一个文件块、执行一个 `reduce` 分区、发送一个网络请求。线程是操作系统调度实体，它拥有栈和寄存器上下文，会被放入 `runqueue`，由调度器分配 CPU 时间。一个任务可以在线程上执行，一个线程可以执行很多任务；二者不是同一概念。

Compute Systems Engine 最初的 `parallel_sum` 每次调用都创建一组线程，任务和线程几乎一一对应。这种写法适合教学，因为边界清楚；但它不适合大量短任务，因为线程创建、栈分配、调度注册和 `join` 都有成本。线程池把线程生命周期延长，让任务通过队列进入 worker。这样任务变轻了，但又引入队列、等待、关闭和异常传播问题。每前进一步，性能和语义都会同时变化。

设计运行时，要先回答三个问题。第一，任务是否 CPU 密集。如果是，worker 数通常不应远超物理核心，除非需要隐藏内存或 I/O 等待。第二，任务是否可能阻塞。如果任务会等待磁盘、网络、锁或 `future`，把它放进 CPU 池可能耗尽 worker。第三，任务之间是否有依赖。如果一个任务提交子任务后等待，而线程池没有 work helping 或足够 worker，可能死锁。线程池不是“并发万能容器”，它只是执行资源的组织方式。

线程创建成本和任务粒度 任务粒度是并行运行时最基本的参数。粒度太大，负载不均衡，一个慢任务拖住整体；粒度太小，队列操作、调度、同步和计数器开销超过有效计算。归约任务如果每个 worker 处理上千万元素，线程创建成本相对很小；如果每个任务只处理几十个元素，线程池也可能被调度开销吞掉。

选择粒度不能只靠固定常数。不同机器核心数不同，缓存大小不同，内存带宽不同，任务内核成本也不同。一个合理策略是先用 `coarse chunk` 保证每个任务有足够计算量，再根据负载不均衡逐步拆小。对均匀数组求和，按连续大块切分足够；对图遍历或 key 倾斜聚合，任务成本不均匀，可能需要动态调度、工作窃取或分片负载估计。

实验上要画任务粒度曲线。固定总输入和线程数，改变 `chunk size`，记录吞吐、任务数、队列等

待、worker 空闲时间和尾部完成时间。曲线通常会出现一个中间区域：粒度太小，调度成本高；粒度太大，负载不均衡；中间区域才是合理范围。这个实验比背“任务数等于线程数几倍”更可靠。

阻塞任务和线程池耗尽 考虑一个线程池有四个 worker。每个任务启动后提交一个子任务，然后等待子任务完成。如果四个 worker 都在执行父任务并等待子任务，而子任务还在队列里没有 worker 执行，线程池就死锁了。这个问题不是 mutex 死锁，却同样来自等待关系形成环。

解决方向有几种。第一，禁止 worker 内同步等待同一线程池的子任务，把依赖交给任务图调度。第二，支持 work helping：worker 等待时不睡眠，而是继续执行队列里的其他任务。第三，分离 CPU 池和阻塞池，让阻塞 I/O 不占满计算 worker。第四，增加结构化并发语义，让任务的生命周期和父子关系明确。选择哪种方式取决于运行时目标，但必须在接口文档中说明，不能让调用者猜。

本章先建立问题，后续任务运行时章节会实现更完整的任务图。这里要记住：线程池接口不只是 `submit`，还包括等待、取消、关闭、异常和嵌套提交语义。一个只会把函数塞进队列的线程池，很容易在真实系统中出现停不下来、等不到、吞异常或死锁的问题。

亲和性和拓扑记录 亲和性优化的前提是拓扑记录。报告里至少要写清逻辑 CPU 数、物理核心数、SMT 关系、NUMA 节点、共享 L1/L2/L3 的范围和内存节点。Linux 上可以用 `lscpu`、`numactl--hardware`、`/sys/devices/system/cpu/` 下的拓扑文件和工具输出记录。没有这些信息，绑定 CPU 的结论很难解释。

绑定策略要和工作负载匹配。CPU 密集向量计算通常先使用每个物理核心一个线程，再测试 SMT；内存延迟敏感但单线程经常等待的任务可能从 SMT 获益；共享数据强的任务可能受益于放在共享 LLC 内；NUMA 分片任务应让线程和内存位于同一节点；跨分片通信强的任务要考虑互连成本。亲和性不是为了把线程“钉死”就结束，而是为了把执行位置、数据位置和共享层级对齐。

亲和性也可能伤害性能。过度绑定会减少调度器处理系统噪声的自由度；把两个重负载线程绑到同一物理核心的 SMT sibling 会互抢执行资源；把线程绑到一个 NUMA 节点但数据在另一个节点，会稳定地产生远端访问；在手机或容器环境里，系统可能限制可用 CPU，强行绑定到不可用 CPU 会失败。实验报告必须记录绑定是否成功。

上下文切换的真实代价 上下文切换不是一个固定纳秒数。直接成本包括保存和恢复寄存器状态、切换内核调度结构和更新运行队列；间接成本包括 cache 热度下降、TLB 上下文影响、分支预测状态变化和被迁移到不同核心后的数据重新加载。若切换发生在锁持有者身上，其他线程可能自旋或睡眠等待；若切换发生在低延迟请求路径上，尾延迟可能被放大。

测上下文切换要区分自愿和非自愿。自愿切换通常来自线程主动阻塞，例如等待条件变量、I/O 或 `futex`；非自愿切换通常来自时间片耗尽、抢占或更高优先级任务。两者解决方向不同。自愿切换多，可能要减少阻塞、批处理或改变等待策略；非自愿切换多，可能线程数过多、CPU 竞争严重或系统负载干扰。

在 `perfstat` 或系统工具中看到 context-switches 增加时，不要立刻下结论。一个 I/O 服务有上下文切换可能是正常的；一个 CPU-bound 热循环频繁切换就值得怀疑。要结合 CPU 利用率、线程数、锁等待、`runqueue` 长度和任务类型分析。

Linux 调度源码阅读路线 Linux 调度器很复杂，不适合一开始从所有策略全局阅读。本册建议按现象驱动。若你想理解普通线程为什么被调度，可以从核心调度入口、调度类、运行队列和上下文切换路径读起；若想理解唤醒延迟，可以沿 `futex` 或条件变量唤醒路径看到任务如何重新进入 `runqueue`；若想理解迁移，可以阅读负载均衡和 CPU affinity 相关路径；若想理解 `cgroup` 限额，可以阅读 CPU controller 对可运行时间的限制。

报告要写具体问题。例如“在 16 个线程测试中，migrations 明显增加，绑定 CPU 后波动下降”，然后阅读调度迁移和亲和性相关代码。不要写“Linux 使用 CFS 调度器，所以性能如何”这种大而空的句子。源码阅读的价值在于把用户态现象和内核数据结构连接起来：task、`runqueue`、调度类、唤醒、抢占、迁移和上下文切换。

线程局部缓存和内存生命周期 线程局部缓存可以减少分配和共享写，但它改变了对对象生命周期。线程池中的 worker 会长期存在，线程局部对象也会长期存在。如果线程局部缓冲按最大请求扩容，处理一次大请求后可能长期占用大量内存。若任务之间复用线程局部状态，还要保证不会泄漏上一个请求的数据。高性能系统里，缓存和生命周期总是绑在一起。

线程局部统计同样有读取语义问题。每个 worker 写自己的计数很快，但读取总数需要遍历所有 worker 状态。若 worker 正在写，读者是否需要精确值？如果只用于指标展示，可以接受近似或延迟

汇总；如果用于协议决策，例如限流额度，就需要更强同步。把所有统计都改成线程局部之前，必须说明读取方的语义需求。

案例：从每次创建线程到固定线程池 下面用归约说明线程生命周期的演化。baseline 是每次调用 `parallel_sum` 都创建 `worker_count` 个线程。它适合少量大任务，但不适合频繁短任务。改进方向是固定线程池：程序启动时创建 worker，归约请求被拆成多个 chunk，提交到队列，worker 执行后写 partial，主线程等待所有 chunk 完成。

这个改进引入了新问题。队列需要同步，提交任务需要分配或复用任务对象，等待所有 chunk 完成需要计数器或 latch，异常需要传回调用者，线程池关闭时不能丢任务。如果允许多个归约请求同时进入线程池，partial 的所有权和生命周期要隔离。一个高质量线程池章节不能只给一段 `while(true)` worker 循环，还要讲清这些语义。

本章只要求读者理解演化方向。后续运行时章节会把线程池拆成接口、队列、worker、任务状态、关闭协议和指标。这里先建立判断：当任务足够大且调用次数少，每次创建线程可以接受；当任务短且频繁，线程池值得引入；当任务之间有依赖，普通 FIFO 线程池不够，需要任务图或 work stealing。

线程章的回归阅读应从“每个任务开一个线程”的朴素设计开始。小输入时它看起来直观：提交一个 split，就启动一个线程处理。输入增大后，线程创建、上下文切换、栈内存、调度迁移和阻塞等待都会进入成本。再把阻塞 I/O 混进计算线程池，worker 可能都在等下游写出，CPU 空闲却没有可运行计算任务。

线程数不是越多越好。报告要区分 runnable 和 running，记录线程池大小、任务粒度、队列长度、阻塞时间和上下文切换。移动设备还要说明大小核、频率和温度；服务器要说明亲和性、NUMA 和同机噪声。若扩展曲线在某个线程数后停止增长，不能只说“线程开多了”，要判断是内存带宽、锁、I/O、调度还是任务粒度。

项目中的 `WorkerSnapshot` 应记录 worker id、当前任务、是否阻塞、运行时间、所在 CPU 和队列状态。它不是为了做监控界面，而是为了让线程章能解释事故：哪个 worker 在等什么，哪个队列在积压，是否发生了同池等待。没有这些记录，线程池问题会变成猜测。

Linux 观察可以从调度事件入手。使用 `perf sched`、`trace-cmd` 或可用的 `ftrace` 入口观察 `sched_switch`，能看到线程何时运行、何时被抢占、何时睡眠。源码阅读可以从 `kernel/sched/fair.c` 理解 CFS 的基本选择逻辑，从 `kernel/sched/core.c` 看调度入口。读源码的目标不是复述调度器实现，而是理解用户态线程池的 runnable 数量如何变成内核调度压力。

亲和性实验要同时记录收益和副作用。固定 worker 到 CPU 可以减少迁移和 cache 冷启动，也可能让某些核心过载、妨碍系统线程、在大小核设备上把重任务绑到慢核。合理报告应比较不绑定、粗粒度绑定和按 NUMA 分组绑定，并说明每种策略的适用条件。线程章不是教读者盲目绑核，而是教他把调度假设写成可验证配置。

线程池配置要从任务类型推导。CPU 密集任务需要接近核心数的 worker，阻塞 I/O 任务需要隔离或异步化，短任务需要批量或 work stealing 降低调度开销，长任务需要可观测和取消。把所有任务扔进同一个固定池，短期最简单，长期最容易出现饥饿和尾延迟。项目中至少应区分 compute pool 和 blocking path，哪怕实现很小，也要把边界写出来。

移动设备还要考虑能耗和热。线程数增加可能让短时间吞吐变高，但温度上升后频率下降，长时间运行反而变慢。报告可以增加持续运行曲线，记录前几秒和一分钟后的吞吐差异。这样读者会明白，高性能计算不是只看峰值，尤其在手机 Termux 环境中，更要看稳定吞吐和资源占用。

9.5 线程不是越多越好，调度要从生命周期看

线程是执行资源，不是任务本身。一个任务可能排队、运行、阻塞、被取消、完成；一个线程可能运行用户代码、等待锁、睡在 futex、阻塞 I/O、被内核抢占或迁移到另一个 CPU。若把任务和线程混为一谈，就会误以为“CPU 不满就加线程”。实际上 CPU 不满可能是 I/O 等待、锁等待、背压、任务粒度过大或调度抖动。

本章从任务生命周期出发。提交任务时，它只是工作描述；进入队列后，它等待 worker；worker 取出后，它占用线程；若内部等待 I/O 或 future，它占着线程但不推进计算；完成后，它释放未完成计数并可能唤醒等待者。每个状态都对应不同指标：队列长度、dequeue 延迟、运行时间、阻塞时间、上下文切换、worker 空闲时间。调度分析就是把这些状态分开。

Runnable 不等于 Running

Linux 中 runnable 任务表示可以运行，但不代表正在 CPU 上运行。若 runnable 数量大于可用 CPU，任务会在 run queue 中等待。过度订阅会增加上下文切换、cache 失效和尾延迟。对于 CPU 密集型任务，线程数通常不应远超物理核心；对于 I/O 密集型任务，可以用更多并发隐藏等待，但也要有背压和队列容量控制。

上下文切换不是绝对坏事，操作系统必须在多任务之间切换；问题在频率和原因。若因为任务太细导致线程频繁唤醒睡眠，调度成本会吞掉计算；若因为锁竞争导致大量线程阻塞，增加线程只会增加等待；若因为后台任务干扰导致迁移，cache 热状态会丢失。实验要记录 voluntary/involuntary context switches、run queue 延迟或至少记录任务排队时间。

用户态线程池要尊重内核调度。绑定线程可以减少迁移、稳定测量，但过度绑定可能破坏系统公平，影响其他进程，也可能在大小核设备上把重任务绑到慢核。Termux/手机环境尤其要注意温控、大小核和后台调度，不能把服务器经验直接照搬。

亲和性、大小核和拓扑

亲和性不是“固定绑核就快”。它解决的是迁移和拓扑不确定性，也可能降低调度器灵活性。对于热缓存状态明显的任务，保持同一 worker 处理相近数据有收益；对于负载不均的任务，过强亲和性可能让某些核心空闲。正确做法是把亲和性当作实验变量，而不是默认开关。

大小核设备让线程配置更复杂。大核适合重计算，小核适合后台和轻任务；系统可能根据功耗和温度迁移线程。若 benchmark 在手机上跑，必须记录设备温度、是否充电、CPU governor、核心频率和后台负载。没有这些信息，前后两轮结果差异可能只是温控。

拓扑感知线程池可以把 worker 分组：同一 NUMA node、同一 LLC 组、同一性能等级。任务带有 placement hint，例如数据所在节点、是否 CPU 密集、是否可能阻塞。调度器先本地，后远端；先计算池，后阻塞池。这样线程池不只是固定数量 worker，而是表达资源差异的运行对象。

阻塞隔离和线程池配置

线程池里最危险的情况，是计算任务和阻塞任务混在一起。少数任务执行同步 I/O、等待 future 或等待网络响应，会占住 worker，让其他纯计算任务饥饿。解决方式包括分离阻塞池、使用异步 I/O、限制阻塞任务数量、在等待时 work helping，或者把依赖交给任务图运行时而不是 worker 线程阻塞。

线程池大小要从 workload 推导。CPU 密集型：接近物理核心数，考虑 SMT 和 NUMA；I/O 密集型：按延迟、吞吐和 in-flight 预算估算并发；混合型：分池或明确阻塞点。队列容量也要配置，不能无限增长。无限队列会把过载藏进内存和延迟，最终让系统不可控。

项目可以实现一个线程池配置实验。固定总记录数，扫描 worker 数、chunk size、是否绑定、是否引入阻塞任务。记录吞吐、p95 任务耗时、worker 空闲比例、队列最大长度和上下文切换。报告应能解释为什么某个线程数之后不再加速，而不是只给“最佳线程数”。

Linux 源码和工具入口

源码阅读从 `kernel/sched/fair.c`、`kernel/sched/core.c` 和工作队列相关路径开始。目标不是读完调度器，而是理解用户态看到的 runnable、sleeping、wakeup 和 migration 在内核中有对应状态。工具可以用 `top`、`pidstat-w`、`perfsched`、`trace-cmd` 或简化的用户态 trace。权限不足时，至少在项目中记录任务状态转换时间。

本章项目交付物是 worker 指标结构。每个 worker 记录执行任务数、累计运行时间、空闲时间、阻塞任务数、最后 CPU、队列 pop 次数。指标不要所有线程写同一个全局原子，应使用每 worker 槽位，最后汇总。这个设计会连接前面的 false sharing 和后面的运行时章节。

最终判断力是：看到线程配置问题，先分清任务、线程和 CPU；看到 CPU 利用率问题，先找等待点；看到线程数增加不加速，先画 run queue、队列和共享状态，而不是马上换算法。

案例走查：线程池里混入阻塞任务

构造一个 4 worker 线程池，提交 100 个 CPU 任务和 4 个模拟慢 I/O 任务。若任务都进同一个队列，慢 I/O 任务可能占住 worker，让 CPU 任务排队。CPU 利用率可能不满，但增加 worker 只是掩盖问题。更好的做法是把阻塞任务放入单独池，或让 I/O 异步化，或限制阻塞任务并发。

实验记录每个任务从 submit 到 start 的延迟、运行时间、worker 空闲时间和队列长度。对照版本包括单池、分池、单池加更多 worker、异步模拟。若分池降低 CPU 任务尾延迟，就说明问题在阻塞隔离；若加 worker 也能改善但上下文切换上升，说明它是较粗糙补丁。

这个案例能纠正“CPU 不满就加线程”的直觉。线程池配置要看任务类型和等待点，而不是只

看核心数。

调度 trace 的最小实现

即使没有内核 trace 权限，项目也可以实现用户态调度 trace。每个任务记录 submit、enqueue、dequeue、start、finish 时间；每个 worker 记录取任务前后的状态；阻塞任务显式记录 block begin 和 block end。用这些数据可以计算排队延迟、运行时间、worker 空闲比例和任务长尾。

Trace 不能太重。可以只在实验模式开启，或者按任务采样。记录用结构体数组而不是每次打印文本，最后统一输出 CSV。高频日志本身会改变调度行为。教材要让读者知道观测也有成本，观测设计也要工程化。

用户态 trace 还能和内核工具对照。若用户态显示 worker 长时间空闲，内核 CPU 使用率也低，问题可能在上游供给或背压；若用户态显示任务 ready 但迟迟不 start，可能是线程数不足或调度竞争；若用户态显示 running 时间很长，但 perf 显示 futex 等待，说明任务内部阻塞。多层证据能互相校验。

亲和性实验的风险说明

线程绑定实验要提醒读者不要在共享手机或生产机器上长时间满载。固定高性能核可能触发温控；提高优先级可能影响系统交互；过度绑定可能让系统任务受影响。教材中的实验应默认低风险配置，并建议短时间运行、记录温度和恢复默认策略。

如果系统不允许设置 affinity，就写清权限限制，用用户态 worker id 代替 CPU id 做部分实验。不要因为工具不可用就跳过调度思想。调度章的核心是任务生命周期和等待点，亲和性只是其中一个变量。

本章作业可以要求读者比较三种配置：不绑定、绑定固定 worker、按数据块本地优先。即使没有 NUMA，也能观察迁移和稳定性差异。报告要写“不绑定不一定错，绑定也不一定对”，避免形成机械经验。

综合练习：线程池配置事故复盘

给出一个事故：上线后吞吐下降，CPU 利用率只有 45

接着要求收集最小指标：submit 到 start 延迟、running 时间、阻塞时间、worker 空闲比例、队列最大长度、上下文切换和错误数。若发现大量 worker 阻塞在写文件，增加计算线程可能无效；若发现 ready 任务很多但 worker 少，才考虑线程数；若发现任务运行时间长尾，可能需要拆分或 work stealing；若发现队列满，可能需要背压而不是更多线程。

复盘报告还要写“错误修复”和“正确修复”的区别。错误修复可能是把线程数从 8 改到 64，短期吞吐上升但上下文切换和内存增长；正确修复可能是分离 I/O 池、限制 in-flight、调整 chunk size 或改任务图。这个题训练读者从现象到等待点，而不是从现象到经验开关。

最后要求给出回归测试。构造一个混合 CPU 和阻塞任务的输入，验证修复后 CPU 任务 p95 排队延迟下降，内存峰值受控，输出一致。没有回归测试的事故复盘，只是故事。

容易出错的边界：线程问题常被误归因

第一个误归因是把锁等待看成 CPU 不够。线程很多但都睡在同一把锁上，增加线程只会增加竞争。第二个误归因是把 I/O 阻塞看成算法慢。Worker 占着线程等待写出，算法再快也会被下游拖住。第三个误归因是把任务长尾看成调度器不公平。若某些输入块本身更难，调度器只能暴露倾斜，不能凭空消除工作量。

第四个误归因是把大小核差异看成代码波动。手机上同一线程跑在不同核心，性能可能明显不同；温控后频率变化也会改变结果。第五个误归因是把队列积压看成队列实现慢。队列可能只是压力表，真正慢的是消费者。第六个误归因是把 runnable 当 running，忽视 run queue 等待。

本章源码索引要求读者在项目中标出所有可能阻塞点：锁等待、条件变量等待、future 等待、I/O 调用、sleep、join、队列满。每个阻塞点都要说明是否允许发生在 worker 线程里，是否有 deadline，是否会影响 shutdown。这个列表能提前发现线程池饥饿和关闭死锁。

调度实验的报告要把线程数、任务数、队列长度、worker 状态和系统 CPU 拓扑放在同一页。只给“8 线程最快”没有意义，因为读者不知道为什么。能解释为什么 6 线程比 8 线程稳定，才说明理解了调度。

本章核心接口：WorkerSnapshot

线程章给项目留下 **WorkerSnapshot**。每个 worker 定期或在实验结束时报告 worker id、last cpu、executed tasks、idle time、blocked time、steal count、queue pops 和 current state。它不是

调度策略本身，而是观察调度策略的窗口。

后续 work stealing、I/O 背压和分布式 worker 模拟都复用这个快照。没有 worker 级指标，系统慢时只能看总耗时；有了快照，能看到某些 worker 是否空闲、是否被阻塞、是否承担过多长尾任务。

从线程生命周期进入锁和等待谓词

线程章告诉读者任务会排队、运行、阻塞和完成；下一章回答阻塞通常发生在哪里、如何用不变量控制等待。线程池里的队列、任务完成计数、shutdown 和 worker 唤醒，都需要 mutex 和 condition variable。若没有线程生命周期图，锁章就只剩 API；若没有锁章的不变量，线程池又会变成竞态集合。

项目推进时，先记录 WorkerSnapshot，看到任务在哪里等待；再进入 QueueContract，定义等待谓词和关闭语义。这样线程调度和同步原语之间有因果关系：不是为了学习条件变量而写队列，而是因为任务生命周期需要可靠等待和唤醒。

9.6 项目落地

Compute Systems Engine 在本章要增加运行环境记录和线程策略实验。第一，程序启动时记录硬件拓扑和可用线程数，至少输出逻辑 CPU 数和用户指定 worker 数。第二，归约 benchmark 要支持从 1 到多倍逻辑 CPU 的线程扫描。第三，实验报告要记录 context-switches 和 migrations。第四，设计线程池前先保留每次创建线程版本作为对照，避免引入线程池后不知道收益来自哪里。

后续项目可以逐步加入亲和性选项，例如不绑定、按物理核心绑定、按 NUMA 节点绑定。每种绑定都必须能失败并报告原因，不能在不支持的平台上默默假装成功。手机或 Termux 环境尤其要谨慎，因为权限、CPU 拓扑和系统调度策略可能和服务端不同。

线程状态和等待图 线程在系统里并不只有“运行”和“结束”。它可能 runnable 但还没拿到 CPU，可能 running，可能 blocked 在 futex，可能 sleeping 等 I/O，可能 stopped，可能被 cgroup throttled。分析并发程序时，要把线程状态画进等待图。一个任务慢，可能不是它计算慢，而是它一直没被调度；一个线程池空闲，可能是上游没有提交，也可能是所有任务阻塞在下游队列。

等待图比线程列表更有用。节点可以是线程、队列、锁、I/O 设备、future、远端服务；边表示等待关系。若线程 A 等锁 L，锁 L 被线程 B 持有，线程 B 等队列 Q，队列 Q 等消费者 A，就形成等待环。很多死锁和线程池耗尽都能用等待图解释。无论是 mutex 死锁、future 死锁，还是分布式请求互相等待，本质都是等待关系没有退出路径。

Runnable 不等于 Running 一个线程处于 runnable 状态，说明它可以运行，但不代表它正在运行。如果 runnable 线程数长期大于可用 CPU 数，调度器必须轮流运行它们，context switch 增加，每个线程获得的 CPU 时间下降。CPU-bound 程序开远多于核心数的线程，通常不会更快，反而增加调度开销和 cache 污染。

I/O-bound 程序可以有更多线程，因为很多线程在等待 I/O，不同时占用 CPU。但这也有限度：线程栈、调度结构、唤醒风暴和内存占用都会增加。异步 I/O 和事件循环的价值，就是减少“每个等待一个线程”的成本。选择线程数要看任务是 CPU-bound、memory-bound、I/O-bound 还是混合。

大小核和移动设备 手机和某些笔记本使用大小核架构，不同核心性能和能效不同。线程绑定到大核和小核，性能可能差很多；系统还会根据温度、电源和前台状态迁移线程。Termux 环境下，用户不一定能完全控制调度。报告中如果在手机上测性能，应记录设备型号、电源状态、温度大致情况、可见 CPU 和是否允许 affinity。

大小核让“线程数等于核心数”更复杂。八个核心不代表八个等价执行资源。CPU 密集任务可能更适合使用大核数量作为主并发度；后台任务可以放小核；长时间满载可能降频，短时间 benchmark 不代表持续性能。第二册的实验要尊重设备现实，而不是假设所有机器都是服务器。

Oversubscription 的后果 Oversubscription 指可运行线程数超过可用硬件执行资源。适度 oversubscription 可以隐藏 I/O 等待；过度 oversubscription 会带来上下文切换、cache 失效、锁持有者被抢占、尾延迟增加和内存压力。线程池默认大小不应盲目等于逻辑 CPU 数乘很多倍，除非任务大部分时间阻塞。

更糟的是嵌套并行。上层已经用 8 个线程处理请求，每个请求内部又调用一个默认开 8 线程的库，瞬间产生 64 个 runnable 线程。矩阵库、压缩库、并行 STL 和业务线程池都可能叠加。工程系

统应限制嵌套并行，或者让库共享同一个运行时。性能报告要记录是否调用了内部多线程库。

调度公平和吞吐 公平调度让每个可运行任务获得合理 CPU 时间，但高吞吐系统有时希望减少迁移和切换。公平、局部性和优先级之间有张力。一个后台压缩线程如果和前台请求线程公平竞争 CPU，可能增加用户请求延迟；完全饿死后台线程，又可能导致 checkpoint 或日志堆积。调度策略要反映业务优先级。

用户态线程池也有类似问题。FIFO 队列公平但可能让长任务阻塞短任务；优先级队列改善前台延迟但可能饿死后台；工作窃取提高负载均衡但可能牺牲数据本地性。操作系统调度和用户态调度叠加，最终行为取决于两者共同作用。

Run queue 和唤醒延迟 当一个阻塞线程被唤醒，它不是立即运行，而是进入某个 CPU 的 run queue，等待调度器选择。若系统负载高，唤醒延迟可能明显。条件变量 `notify_one` 只是让等待线程有机会运行，不保证它马上持有锁。被唤醒线程还要和其他线程竞争 mutex，可能再次睡眠。

这解释了为什么高竞争条件变量和锁会产生尾延迟。一个生产者通知消费者后，消费者可能晚些才运行；消费者醒来后发现队列又被其他消费者取空；再睡回去。正确性由谓词循环保证，性能则需要减少惊群、控制队列容量、降低竞争和让任务粒度合适。

优先级反转的系统版 优先级反转不只存在实时系统。一个低优先级后台任务持有全局锁，高优先级请求线程等待这把锁，而中优先级 CPU 任务不断运行，使后台任务迟迟无法释放锁，这就是普通服务中的优先级反转形态。解决方法包括缩短临界区、避免后台任务持有前台锁、资源隔离、优先级继承或使用无锁/分片结构。

在分布式系统中也有类似现象：低优先级 checkpoint 占满 I/O，导致高优先级提交延迟；后台 compaction 占满 CPU，导致 RPC 超时。线程调度的公平性问题会在资源调度层面反复出现。第二册把这些主题放在同一册，是为了让读者看到共性。

线程栈和内存占用 每个线程都有栈。线程数很多时，栈虚拟地址和实际提交内存都可能成为问题。递归、大对象局部变量和深调用链会增加栈压力。C++ 并发代码中，任务对象如果捕获大 vector 按值复制到线程栈或闭包里，也会造成内存和复制成本。线程池减少线程数量，也能减少栈开销。

用户态 fiber 或 coroutine 可以用更轻量的栈或无栈状态机，但它们引入自己的调度和生命周期语义。第二册不展开 coroutine 细节，但要让读者知道：用更多并发单位隐藏等待时，线程不是唯一选择。线程适合 CPU 执行资源，异步任务适合等待很多 I/O。

信号和异步打断 Linux 线程还可能收到信号。信号处理函数会在异步位置执行，能做的事情非常有限。高性能和并发代码通常避免在信号处理函数里做复杂逻辑，而是写入 pipe、eventfd 或设置原子标志，让主循环在安全位置处理。信号也可能打断系统调用，导致返回 EINTR。I/O 代码必须考虑这种情况。

这部分不是本册重点，但它提醒读者：真实系统中的控制流不只来自函数调用和线程调度，还可能来自信号、取消、超时和内核回调。状态机要能处理被打断和重试。

ThreadSanitizer 和调度测试 ThreadSanitizer 能发现许多数据竞争，但它会改变性能，不能用于性能测量。调度压力测试可以通过增加线程数、插入 yield、随机 sleep、缩小队列容量和重复运行来暴露时序 bug。不要在正式性能报告中开启 TSan，也不要因为 TSan 通过就认为没有并发逻辑错误。

线程调度 bug 常常低概率出现。测试应固定随机 seed，记录失败场景，并尽量构造小模型复现。例如有界队列容量设为 1，比容量很大更容易暴露等待和唤醒错误；线程池 worker 数设为 1 或 2，比 16 更容易暴露嵌套等待死锁。小规模极端测试是并发正确性的朋友。

线程池大小的配置 线程池大小应该是配置，不应写死。默认值可以来自硬件并发度，但要允许用户覆盖。CPU-bound 池、I/O 池和后台池应有不同默认策略。若运行在容器中，还要尊重 cgroup 和 CPU 配额。读取 `std::thread::hardware_concurrency` 失败或返回不准时，应有保守 fallback。

配置还要进入实验记录。一个 benchmark 只写“默认线程池”，读者不知道默认是多少，也不知道是否与机器有关。项目输出应显示 `worker_count`、`hardware_concurrency`、绑定策略和是否使用 SMT。这样结果才可解释。

线程设计的反例 常见反例包括：每个请求创建一个线程导致高并发时内存爆炸；CPU-bound 任务开数倍于核心的线程导致调度开销增加；线程池 worker 内阻塞等待同池子任务导致死锁；把

I/O 阻塞任务放入计算池导致计算饥饿；在线程局部缓存里保存请求状态导致任务迁移后读不到；全局队列在多 socket 下成为共享热点；忽略关闭语义导致析构时悬挂。

这些反例不是孤立 bug，而是线程、任务、资源和状态边界没有分清。学习线程调度的目标，是在设计阶段就能问出这些问题。

案例：日志解析线程数 日志解析看起来 CPU 密集，但它可能同时受 I/O、分支和内存带宽影响。若输入来自 SSD，单线程解析可能跟不上读取；增加线程能提高解析吞吐。若输入来自慢存储，增加解析线程只会让线程等待 I/O。若解析产生大量错误日志，输出 I/O 又可能成为瓶颈。线程数选择必须和全链路一起看。

实验可以设置三种输入源：内存中已加载字符串、page cache 热文件、冷文件或模拟慢读取。对每种输入扫描 worker 数。内存输入主要测解析和内存带宽；热文件混合 page cache；慢读取则测 I/O。若三组结果差异大，说明线程数不能脱离输入源讨论。这个案例能训练读者避免“一律开满核心”的经验主义。

案例：后台任务干扰 假设 Compute Systems Engine 在 reduce 阶段同时做 checkpoint。Checkpoint 线程写大文件并 fsync，可能占用 I/O；如果它还做压缩，可能占用 CPU；如果它持有 driver 元数据锁，可能阻塞 commit。前台 reduce 变慢时，不能只看 reduce 代码，还要看后台任务是否干扰。

解决方法包括资源隔离、限速、独立线程池、优先级和调度窗口。Checkpoint 可以在 stage 边界执行，也可以后台增量写；压缩可以限制线程数；元数据锁应只保护状态切换，不覆盖大文件写入。线程调度和 I/O 背压在这里交汇。

线程池配置实验 项目应提供线程池配置实验。参数包括 compute worker 数、I/O worker 数、是否允许阻塞任务进入 compute 池、队列容量和任务粒度。实验输入包括 CPU-bound map、I/O-bound read、混合 pipeline。指标包括吞吐、队列长度、context switches、worker idle time、任务延迟和超时。

通过实验，读者可以看到：CPU-bound worker 过多会增加调度成本；I/O-bound 任务放 compute 池会让计算饥饿；队列容量太小会频繁背压，太大会增加延迟；任务粒度太细会增加调度开销。线程池配置不是一次性常量，而是 workload 参数。

用户态调度和内核调度的边界 用户态线程池决定哪个任务交给哪个 worker，内核调度器决定哪个线程在哪个 CPU 上运行。用户态运行时看不到所有系统负载，内核调度器不知道用户任务的业务优先级。两者之间存在信息差。绑定 CPU、设置线程优先级、分离线程池、使用 cgroup，都在试图协调这两个调度层。

项目文档应说明哪些由用户态控制，哪些由内核控制。比如 worker id 到 CPU id 的映射是亲和性策略；task 到 worker 的映射是运行时策略；worker 是否被抢占由内核调度；页面在哪个 NUMA 节点是内存策略。把边界说清，排查问题时才知道该看哪一层。

线程池和请求上下文 请求上下文不应该隐式绑定到线程。在线程池中，一个请求的 parse、compute、write 可能由不同 worker 执行；即使当前实现碰巧同一线程，后续 work stealing 或异步 I/O 也可能改变。上下文应作为显式对象传递，或者由 task id 在状态表中查找。线程局部只适合 worker 缓冲、统计和临时 scratch。

这个原则能避免很多隐蔽 bug。把 request id 放在线程局部，日志可能串到别的请求；把取消标志放线程局部，后续任务看不到；把输出缓冲放线程局部并在异步写完成前复用，会产生数据损坏。线程局部是性能工具，不是业务上下文容器。

调度章节验收清单 本章项目验收包括：能记录硬件并发度和用户 worker 数；能扫描线程数并输出扩展曲线；能记录 context switches 和 migrations；能区分 CPU-bound 和 I/O-bound 任务；线程池配置可调；任务粒度可调；请求上下文显式传递；阻塞任务不会默认进入计算池；报告说明当前环境的 CPU 拓扑限制。满足这些条件，后续线程池和任务图才有可靠基础。

调度章节总结 线程是执行资源，任务是逻辑工作，二者不能混淆。调度优化要同时考虑任务粒度、阻塞、亲和性、上下文切换、线程局部缓存和内核调度。线程池能减少线程创建成本，但引入队列、等待、关闭和嵌套提交语义。正确的线程设计不是“开几个线程”，而是让执行资源、数据位置、等待关系和业务上下文都有清楚边界。

调度决策表 配置线程和任务时，先问任务是否 CPU-bound、是否阻塞、是否有依赖、是否需要优先级、是否访问大块本地数据、是否会嵌套提交。CPU-bound 任务按核心控制并发；阻塞任务

隔离到 I/O 池；有依赖任务进入任务图；有优先级任务需要多队列或配额；访问大块数据要考虑亲和性；嵌套提交要禁止阻塞等待或支持 work helping。这个表能防止线程池被当成万能容器。

案例：任务粒度自适应的观察 自适应任务粒度不一定要一开始自动实现，但可以先观察。记录每个任务耗时、队列等待、worker 空闲时间和任务数。如果任务耗时远小于队列操作成本，说明粒度太细；如果 worker 空闲多且少数任务很长，说明粒度太粗或负载不均。运行时可以先把这些指标输出给报告，让人手动调整 cutoff。自动调节只是把这个反馈回路程序化。

这个案例提醒读者：很多“智能调度”可以从简单观测开始。没有观测，自动策略只是猜测；有了观测，即使手动调参，也比固定常数可靠。

案例：线程池里的阻塞隔离 假设 compute pool 有 8 个 worker，其中 8 个任务都在等待磁盘写完成，那么新的计算任务无法执行，CPU 可能空闲但队列积压。解决方法不是简单增加 compute worker，而是把阻塞写入交给 I/O pool 或异步写出。Compute worker 只生成输出 block，然后返回执行资源。I/O pool 完成后通过 completion 通知 driver。

这个案例说明线程池大小不是唯一问题，任务类型也必须分类。CPU-bound、I/O-bound、control-plane、background maintenance 应有不同资源策略。否则一个慢资源会拖住所有执行资源。

运行队列视角下的线程池 用户态线程池的队列和内核 run queue 是两层不同队列。任务先排在用户态 ready queue 里，worker 线程取到任务后进入执行；worker 自己又作为内核任务排在某个 CPU 的 run queue 里。用户态队列空，不代表系统没有可运行线程；内核 run queue 繁忙，也不代表线程池有任务。分析吞吐和延迟时，必须把这两层分开。

例如一个服务的用户态任务队列持续增长，但 CPU 利用率很低。可能原因不是 worker 数不足，而是所有 worker 阻塞在 I/O 或锁上，内核看来它们不可运行。再例如 CPU 利用率很高，任务队列也增长。可能是 worker 正在运行但每个任务太慢，也可能是系统里其他进程占用 CPU，让 worker runnable 但很少 running。只看一个队列会误判。正确报告应同时记录用户态队列长度、worker 状态、CPU 利用率、context switch、run queue 压力和阻塞原因。

Linux 的调度器不会理解用户态任务优先级。若线程池只有一个 FIFO 队列，短任务和长任务混在一起，内核只看到 worker 都在运行，不知道某个 worker 正在执行低优先级长任务。若业务需要前台请求优先，就要在用户态运行时设计优先级队列、时间片、任务拆分或隔离线程池。操作系统提供 CPU 时间，业务调度要由程序自己表达。

这个视角也解释了 work stealing 的边界。工作窃取能缓解用户态任务分配不均，让空闲 worker 从其他队列拿任务；但如果某些 worker 在内核层面长期得不到 CPU，或者被 cgroup 限额节流，窃取策略无法凭空创造执行资源。运行时优化要先确认瓶颈在用户态队列、内核调度、I/O 还是硬件资源。

尾延迟和调度抖动 吞吐系统关注总任务数，低延迟系统还关注尾部任务。一次迁核、一次锁持有者被抢占、一次缺页、一次 cgroup throttling，都可能只影响少数请求，却显著拉高 p99 或 p999。平均耗时变好，不代表用户体验变好。线程调度章节必须把尾延迟作为一等指标，而不是只看总时间。

尾延迟分析要按路径拆开。请求进入队列前等待了多久，排队多久，被哪个 worker 取走，执行中是否阻塞，是否迁移，是否等待锁，是否等待下游 I/O，是否被后台任务干扰。每一段都应有时间戳或事件记录。只有总耗时一个数，就无法知道尾部来自排队、执行、同步还是 I/O。Compute Systems Engine 的任务记录可以逐步加入 submit time、start time、finish time、worker id 和重试次数，这些字段比复杂优化更早产生价值。

调度抖动还有环境因素。手机设备会降频，笔记本会进入省电模式，云服务器可能有邻居噪声，容器可能被 CPU quota 周期性限制。实验报告若不记录环境，尾延迟结论很难复现。对于教学项目，可以先要求读者写出“本次实验不保证独占机器”这样的限制，再逐步学习如何隔离变量。

亲和性不是固定答案 亲和性常被误解成“绑定一定更快”。实际上，绑定只是减少调度自由度，让执行位置更可控。若任务的数据也在相同 NUMA 节点，绑定可能提高局部性；若数据在远端节点，绑定会稳定地产生远端访问；若系统有其他干扰，适度不绑定反而让调度器能避开忙 CPU。亲和性是一种实验变量，不是默认优化。

一个合理实验应至少比较四组。第一组完全不绑定，作为系统默认行为。第二组只绑定线程，看缓存热度和迁移是否改善。第三组只控制内存放置，观察线程迁移后远端访问。第四组同时绑定线程和内存，测试最佳局部性。若平台不支持 NUMA，也可以比较不绑定、绑定到物理核心、绑定到

SMT sibling、绑定到一组核心。每组都要记录绑定是否成功，因为容器和手机环境可能拒绝或修改 affinity。

亲和性还要和任务分片配合。若 worker 0 固定在 CPU 0，却不断处理来自所有分片的数据，绑定价值有限。更好的策略是 worker、数据分片和队列分片对齐：某个 NUMA 节点的 worker 优先处理本节点数据，跨节点窃取作为兜底。这样能把亲和性从“钉线程”提升为“数据和执行位置共同设计”。后续分布式章节也会看到同样思想：计算应尽量靠近数据。

线程池指标的最小闭环 线程池想要可调，必须先可观测。最小指标包括提交任务数、完成任务数、队列长度、worker 空闲时间、任务排队时间、任务执行时间、阻塞任务数、拒绝任务数、关闭状态和异常任务数。再进一步，可以记录每个 worker 的任务数、最大连续忙碌时间、窃取次数、空轮询次数和最后一次任务类型。没有这些指标，线程池参数只能凭感觉调整。

指标本身不能破坏性能。高频写指标应使用 worker 本地槽位或 relaxed 原子，周期性汇总；低频控制状态可以用锁保护。不要为了监控每个任务而在热路径写全局锁日志。更好的方式是采样、分片、批量写入和可配置开关。监控代码也是并发系统的一部分，它会改变被监控对象。

指标解释也要谨慎。队列长度高可能是任务太多，也可能是 worker 阻塞；worker 空闲高可能是上游供给不足，也可能是任务粒度太粗导致尾部不均；context switch 高可能是 I/O 服务正常现象，也可能是 CPU-bound 程序过度线程化。指标不是结论，而是提出下一步问题的证据。

任务粒度 任务太小，调度和队列成本超过计算；任务太大，负载不均和取消延迟变差。粒度要从每任务工作量、队列成本、缓存复用和错误恢复一起决定。

等待点 线程运行不等于工作在推进。它可能在锁、条件变量、I/O、page fault 或下游队列上等待。报告中要区分 running、runnable、blocked 和 sleeping。

线程池 线程池复用线程，减少创建销毁成本，但它也引入队列、关闭、异常传播和背压。线程池接口必须定义提交失败、shutdown、drain 和任务异常。

亲和性 亲和性可以稳定 cache 和 NUMA 放置，也可能限制调度器平衡负载。实验应先记录默认调度，再尝试绑定，最后说明收益和限制。

超额订阅 线程数超过核心数不一定错，但必须有理由。I/O 等待多的任务可以超额，CPU 密集任务通常不应盲目超额。

移动设备 手机环境有大小核、频率变化和温控。线程调优要记录设备状态，不要把一次热机结果当成稳定结论。

取消与关闭 任务提交后可能还在队列、正在运行或已经完成。取消语义要按状态定义：未开始可删除，运行中只能协作取消，完成后只能忽略。

项目接口 ThreadPoolPlan 应包含 worker 数、队列容量、亲和性策略、任务粒度、关闭方式和观测字段。这样后续锁、任务图和 I/O 章节能继续使用。

案例：粒度阶梯 把同一批工作切成不同大小任务，观察吞吐和队列水位。说明最佳粒度来自成本平衡，而不是固定数字。

案例：CPU 任务超额订阅 纯计算任务从核心数一半到两倍核心数扫描，解释为什么超过某点后收益消失甚至下降。

案例：阻塞任务线程池 任务中加入模拟 I/O 等待，比较固定线程数和稍微超额的线程数。这个案例说明等待比例会改变线程策略。

源码阅读路线 Linux 源码阅读从 [kernel/sched/fair.c](#)、[kernel/sched/core.c](#)、[kernel/sched/topology.c](#) 附近进入即可。阅读目标不是把整套子系统抄进书里，而是追一条和本章实验相关的路径：用户态动作怎样变成内核对象，内核对象怎样排队、等待、唤醒、计数、提交或回收。

阅读报告要写证据等级。能直接测到的数字放在第一层；从源码控制流推导出的解释放在第二层；当前设备无法验证、只能作为合理猜测的内容放在第三层。这样读者会知道线程调度报告中哪些结论可复现，哪些结论需要换机器继续确认。

源码阅读还要克制。看到一个复杂函数，不要把每个分支都展开；先回到本章的问题，问它能否解释一个等待点、一次状态转移或一条性能曲线。解释不了的内容可以留到索引，不要打断正文主线。

粒度阶梯 把同一批工作切成不同大小任务，观察吞吐和队列水位。说明最佳粒度来自成本平衡，而不是固定数字。

CPU 任务超额订阅 纯计算任务从核心数一半到两倍核心数扫描，解释为什么超过某点后收益消失甚至下降。

阻塞任务线程池 任务中加入模拟 I/O 等待，比较固定线程数和稍微超额的线程数。这个案例说明等待比例会改变线程策略。

线程生命周期

线程不是任务本身。创建、可运行、运行、阻塞、唤醒和退出都是状态。线程池复用线程，任务才是可调度工作。

实验设计：为每个任务记录提交、入队、开始、结束和异常状态。

任务粒度

任务太小会被调度和队列成本吞掉，任务太大又造成负载不均和取消延迟。粒度来自计算量、局部性和恢复范围。

实验设计：扫不同 chunk size，记录吞吐、队列等待和最长任务。

上下文切换

线程阻塞、抢占或主动让出都会切换。切换成本不只在 CPU 指令，还包括 cache 热度、调度延迟和锁竞争。

实验设计：比较忙等、条件变量等待和 I/O 阻塞三种路径。

runnable 与 running

CPU 使用率低不一定线程少，可能是线程都在等待。runnable 队列长和 sleeping 多代表不同问题。

实验设计：用系统工具或替代 trace 记录线程状态。

亲和性

绑定线程能稳定局部性，也可能损失负载均衡。实验要先跑默认策略，再说明绑定带来的收益和限制。

实验设计：给 worker 绑定 CPU 后比较迁移次数和吞吐。

线程池接口

submit、shutdown、drain、wait 和异常传播必须定义。没有关闭语义的线程池很容易析构时丢任务或悬挂。

实验设计：构造提交后立即关闭、任务抛异常和等待所有任务三类案例。

嵌套并行

任务内部再提交任务并等待，可能造成线程池饥饿。等待应转成依赖或 continuation，而不是占住 worker。

实验设计：固定两个 worker，构造互相等待的嵌套任务。

阻塞隔离

计算任务和 I/O 阻塞任务混在一个池里，阻塞任务可能占满 worker。需要专门池、异步 I/O 或阻塞标记。

实验设计：让写文件任务占住计算池，再拆出写出池对照。

移动设备限制

手机和笔记本会受功耗、频率和温度影响。线程数越多不一定越快，报告要记录设备状态和降频可能。

实验设计：连续运行多轮，观察吞吐是否随时间下降。

项目接口

ThreadPoolPlan 应记录 worker 数、队列容量、任务粒度、亲和性、阻塞策略、关闭方式和观测字段。

实验设计：后续锁、任务图和 I/O 章节都复用这份接口。

9.7 线程不是越多越好，任务生命周期才是主线

线程章最常见的错误写法是把线程数当成旋钮：慢了就加线程，CPU 不满就继续加。这会误导读者。线程只是执行资源，任务才是工作单元；调度、阻塞、队列、亲和性和关闭语义都围绕任务生命周期展开。一个系统中，任务从提交到入队，从可运行到运行，从阻塞到唤醒，从完成到报告结果，每一步都可能成为瓶颈或 bug。

任务粒度决定线程池能否有效工作。粒度太小，队列 push、pop、唤醒和调度成本超过计算；粒度太大，负载不均，最后一个大任务拖住全局完成。若任务还需要失败恢复，粒度又决定重做范围。日志统计中，一个任务处理多少行、多少字节或多少 batch，不是随便定的。它要结合 cache 局部性、I/O 批量、错误定位和 checkpoint 粒度。

调度状态要区分 running、runnable、blocked 和 sleeping。CPU 使用率低不一定说明线程少；可能所有线程都在等待 I/O、锁或下游队列。runnable 队列长说明 CPU 竞争，blocked 多说明等待资源。报告里如果只写“开了 8 个线程”，没有写线程在哪里等待，无法解释性能。手机或容器环境里还要记录可见 CPU、频率变化和调度限制。

线程池接口必须定义关闭。submit 在关闭后返回什么？已经入队但未运行的任务怎么办？正在运行的任务能否取消？任务抛异常是否传播到调用者？shutdown 是立即丢弃、等待 drain，还是带超时？这些问题如果不写，析构时就会出现丢任务、悬挂或访问已释放对象。线程池不是把线程放进 vector，而是一套任务状态机。

亲和性要讲收益和代价。绑定线程可以稳定 cache 和 NUMA 放置，让实验更可重复；但负载不均时，绑定也可能阻止调度器帮忙迁移。正确实验顺序是：先记录默认调度，再绑定观察是否改善，再解释限制。若设备没有 NUMA 信息，也可以用 CPU mask、迁移次数和吞吐稳定性作为替代证据。

嵌套并行和阻塞隔离是工程重点。worker 执行任务时又提交子任务并等待，可能让线程池饥饿；计算任务和阻塞 I/O 任务混在一个池里，I/O 会占住 worker。解决方法不是盲目加线程，而是把等待改成 continuation，把阻塞任务放进专门池，或让运行时知道某任务会阻塞。后续任务图和 I/O 章都会复用这个思想。

本章实验可以要求实现一个小线程池，并配套状态 trace。每个任务记录 submit、enqueue、start、finish、worker、异常和等待原因。然后构造三类输入：小任务风暴、大任务长尾、阻塞任务混入。读者会看到线程池问题不是“线程个数”四个字能解决，而是任务形状、等待位置和生命周期共同决定。

案例推导：线程池为什么需要关闭、异常和阻塞语义

一个最小线程池通常只有 submit 和 worker 循环。任务队列里有函数，worker 取出来执行。这个版本跑 demo 很快，但工程里马上遇到问题：调用者提交后对象析构怎么办，任务抛异常怎么办，队列满怎么办，线程池关闭时还没运行的任务怎么办，worker 阻塞在 I/O 上是否影响计算任务？这些问题不解决，线程池只是一个隐藏 bug 的容器。

先看关闭。立即关闭意味着不再接受新任务，队列中未开始任务可能被取消；drain 关闭意味着拒绝新任务，但等待已接受任务完成；带超时 drain 则在预算内等待，超时后报告未完成任务。三种语义都合理，但必须写清。若析构函数里直接设置 flag，然后 join，等待中的 worker 要被唤醒；若没有 closed 谓词，线程可能永远睡眠。

异常传播也必须定义。worker 如果捕获异常后只打印日志，调用者会以为任务成功。更好的做法是让任务状态记录 success 或 failure，必要时通过 future、completion 或报告返回错误。依赖任务不应继续消费失败任务的输出。这一点会在任务图章节放大，但线程池章节就应埋下接口：任务不是裸函数，而是有身份和结果状态的工作项。

阻塞隔离是性能边界。若写文件、网络等待和长计算都在同一个固定线程池，几个慢 I/O 任务可能占满 worker，让短计算任务排队。解决方案可以是独立 I/O 池、异步 I/O、任务标记或动态补偿线程，但每种都有代价。教材应让读者先复现问题：提交 N 个长睡眠任务，再提交短任务，观察短任务等待时间。这个实验比抽象讨论更有说服力。

任务粒度和亲和性也要一起看。小任务太多，队列锁和唤醒成本显著；大任务太少，负载不均。绑定 worker 到 CPU 后，局部性可能更稳定，但若任务大小不均，绑定可能让某个核心忙、其他核心空。线程调度报告要记录队列等待、运行时间、worker id、CPU id 和迁移。只写线程数没有意义。

一个合格 ThreadPoolPlan 至少包含：队列容量、submit 失败行为、关闭模式、异常传播、阻塞任务策略、worker 数、亲和性和 trace 字段。后续锁章的条件变量、任务图章的 ready count、I/O

章的 completion 都会复用这些语义。

核心理念

线程设计的核心不是“开几个线程”，而是决定哪些状态线程本地，哪些状态共享，哪些共享状态需要同步，哪些等待应该阻塞或异步化。

实验：把并行归约的 worker 数从 1 增加到逻辑 CPU 数的两倍。记录时间、context-switches 和 migrations。解释从哪个点开始扩展性变差。

线程不是任务，线程是执行预算 很多并发程序从“我要开多少线程”开始，这是顺序错了。应该先问任务是什么、会阻塞还是计算、生命周期多长、能否取消、是否需要固定数据局部性。线程只是执行预算，是操作系统调度的实体。把每条记录、每个请求或每个小函数都直接映射成线程，会把调度、栈内存和上下文切换成本推到无法控制的程度。任务运行时的价值正是把工作单位和线程分离。

贯穿项目中，ParseBatch 是 CPU 任务，WriteBlock 可能阻塞 I/O，CommitManifest 需要可靠提交，DriverLoop 负责调度和状态。它们不应该都用同一类线程处理。CPU worker 应尽量避免阻塞；I/O completion 或写出线程可以等待内核；driver 线程应保持响应。若 CPU worker 被慢 I/O 占住，计算吞吐下降；若 I/O 线程执行重计算，完成事件会延迟。线程池设计必须从任务类型推导。

上下文切换的成本不只是一条指令 上下文切换会保存和恢复执行状态，调度器要选择下一个任务，CPU cache 和 TLB 的热度也可能变化。若线程频繁阻塞和唤醒，程序会在用户态工作和内核调度之间来回跳。阻塞本身不一定错，等待磁盘或网络时让出 CPU 是合理的；问题是把本可异步或批量处理的短等待放进大量 worker 中，导致可运行任务和等待任务混杂。

实验可以比较三种设计：每个小任务一个线程、固定线程池加全局队列、任务批处理加线程池。记录任务吞吐、上下文切换次数、队列等待、worker 空闲和尾延迟。输入任务很粗时，三者差异可能不大；任务很细时，直接线程模型会崩；任务会阻塞时，固定计算线程池会被占住。读者要从数据中理解调度成本。

亲和性是约束，不是性能按钮 线程亲和性可以减少迁移，让线程更可能复用 cache 和本地内存。但绑死线程也可能破坏负载均衡，让某些核心空闲、某些核心拥塞。移动设备、容器环境和云主机还可能不对称核心、CPU quota 或动态频率，亲和性策略更要谨慎。教材应把亲和性当作实验变量，而不是默认优化。

项目可以先不绑定，记录 worker 在哪些 CPU 上运行；再尝试绑定 CPU worker 到一组核心，观察吞吐和迁移；最后在 NUMA 机器上按 node 绑定输入 split 和 worker。报告要写清绑定策略、核心拓扑和输入规模。若收益只在特定机器上出现，就写成特定结论，不要泛化。

线程池的最小语义 线程池不是简单的 `submit(function)`。它至少要定义：关闭后是否接受新任务，队列中任务是否继续执行，正在运行任务是否能取消，任务抛异常怎样处理，析构是否等待，`wait` 是否只等已提交任务，任务内提交任务是否允许。没有这些语义，线程池在演示里能跑，在项目里会制造死锁和悬垂引用。

第一版线程池可以用互斥锁、条件变量和队列实现，重点是语义清楚。提交增加 unfinished 计数，任务结束在异常路径也减少计数；关闭唤醒所有 worker；worker 在队列空且状态关闭时退出；析构调用 shutdown 和 join。这个版本未必最快，但它是后续任务图和工作窃取的 reference。

实验：阻塞任务污染计算线程池 构造一个线程池，提交两类任务：短计算任务和慢 I/O 模拟任务。若所有任务共享同一队列和同一组 worker，慢任务会占住线程，短任务排队。改进方案一是分离计算池和 I/O 池；方案二是慢任务使用异步提交，完成后再把 continuation 放回计算池；方案三是设置队列容量和优先级。每个方案都要记录短任务 p50、p99 和慢任务完成时间。

这个实验直接连接第 15 章异步 I/O。线程模型不是孤立章节，它决定等待发生在哪里。若等待占用计算线程，后续再优化 SIMD 和缓存也救不了端到端延迟。

Linux 源码阅读连接点 读 Linux 调度时，关注 task state、runqueue、wakeup、sleep 和 migration。用户态看到的是线程，内核看到的是可调度实体和状态转换。阻塞系统调用会让任务离开运行队列，唤醒会重新入队，迁移会影响局部性。项目线程池的状态机可以借鉴这种思路：任务状态要显式，等待和唤醒要围绕谓词，而不是围绕“通知应该到了”。

读 `/proc` 和调度统计时，可以把 worker 的运行 CPU、上下文切换、自愿和非自愿切换、CPU 时间与项目 trace 对照。这样读者会把操作系统调度和自己的运行时联系起来，而不是把线程当成黑盒。

事故复盘：线程池为什么会被慢写出占住 一个固定线程池同时处理解析、聚合和写出。正常输入下吞吐不错；磁盘变慢后，几个 worker 阻塞在 write，短计算任务排在队列里无人执行。增加线程数后上下文切换增多，缓存热度下降，吞吐仍然不稳。事故说明线程不是任务，线程是昂贵执行资源；任务要有类型、生命周期和阻塞点。

调度分析先看 runnable 数和队列等待。若 CPU 空闲但任务排队，可能 worker 被阻塞调用占住；若 runnable 远大于核心数，可能过度并发导致切换；若绑定后更快，说明 cache 或 NUMA locality 重要；若绑定后更慢，可能负载不均。亲和性不是固定答案，它要服务任务类型：计算任务需要本地 cache，I/O completion 需要及时唤醒，长阻塞任务不应占住计算 worker。

项目实施应把计算池、I/O 池和完成回调分开，至少在指标上分开。每类任务记录提交时间、开始时间、完成时间、阻塞点和 worker id。关闭时要明确 drain、cancel 和 reject 的策略。线程章节真正要讲清的是生命周期：任务何时 ready，何时运行，何时等待，何时完成，线程只是承载这些状态的资源。

线程运行曲线实验判读 线程实验不能只看吞吐。要同时看 runnable 数、上下文切换、worker 空闲、队列等待和阻塞点。线程数增加后吞吐不升，可能是任务太小、I/O 阻塞、锁竞争或内存带宽。亲和性实验必须写拓扑和绑定策略；若绑定后尾延迟变差，可能是负载不均而不是绑定无效。

这一段也服务于复习。读者合上书后，应该能凭本章的判读口径重新审查自己的代码：有没有最小正确版本，有没有能击穿旧方案的输入，有没有状态记录，有没有资源指标，有没有失败后的恢复动作。若五个问题缺两个以上，就说明实现还停在演示层，而不是工程层。

线程池配置不是一个固定数字 线程数要和任务类型一起决定。纯计算任务可以接近核心数，阻塞 I/O 任务可能需要单独池或异步接口，driver 控制线程需要保持响应，回调线程不能被长计算占满。一个全局线程数配置无法表达这些差异。

实验时不要只扫描线程数，还要扫描任务粒度。任务太小，队列和唤醒成本占主导；任务太大，长尾拖慢 stage；任务访问远端数据，增加线程可能只会制造更多内存争用。报告应同时写每个 worker 执行任务数、最长任务、平均等待和空闲时间。这样线程池配置才有依据。

关闭语义也要纳入配置。系统退出时是 drain 完成所有任务，还是立即取消未开始任务，还是带超时等待？不同业务不同答案。没有关闭语义的线程池不是简化，而是把最危险的状态留给调用者猜。

实验课：计算任务和阻塞任务混用的后果 构造两类任务：短计算任务处理一个 batch，慢写出任务 sleep 或 fsync 模拟 I/O。第一版把两类任务放进同一个线程池。输入一开始吞吐正常，写出变慢后，计算任务排队，worker 被阻塞任务占住。报告记录每类任务的队列等待、执行时间和 worker 占用。

第二版分离 CPU pool 和 I/O pool，计算任务只提交 OutputBlock 到写出队列。第三版给写出队列加容量，满时让 reduce 降速。第四版尝试绑定 CPU worker，观察迁移、吞吐和尾延迟。每一步都要说明新增语义：分离 pool 增加线程管理，容量队列引入背压，亲和性引入拓扑约束。

判读时若 CPU 使用率低，不要立刻加线程。先看 worker 是 runnable 还是 sleeping，队列是计算积压还是 I/O 积压，任务平均时间是否被少数长任务拉高。若加线程后吞吐短期上升但上下文切换和内存峰值也上升，要写出这个代价。线程调优不是找到一个神奇数字，而是让任务类型和执行资源匹配。

关闭测试必须保留。作业取消时，CPU pool 是否停止接收新任务，I/O pool 是否 drain 已提交写入，driver 是否等待 manifest 稳定。这些语义决定系统能否可靠退出。

线程调度练习验收重点 合格答案必须区分任务类型。计算任务、阻塞 I/O、driver 控制和回调不能混成一个队列解释。答案要给线程池状态、队列等待、上下文切换或替代指标，并说明关闭语义。亲和性题必须写拓扑和绑定策略，否则结论不可复查。

答案还要写“为什么不采用更复杂方案”。例如没有证据证明队列是瓶颈，就不应把锁队列换成无锁；没有证明输出顺序可变，就不应随意重排；没有证明错误路径能恢复，就不应只提交正常路径。能解释不做什么，说明读者开始具备工程判断。

最后，答案必须包含可复查材料。至少有输入摘要、关键状态、期望输出、实际输出、运行命令

和未验证边界。没有这些材料，老师或同伴无法判断结论是否成立。第二册的作业不是让读者写漂亮段落，而是训练读者把系统问题变成可验证证据。

项目实现：线程、调度与亲和性 本章对应的贯穿项目实现不要求一次写成完整框架，但必须有最小可运行切片。这个切片包含三个部分：一个 reference，一个带本章机制的实现，一个能击穿 reference 之外隐含假设的测试。Reference 用来固定语义，机制实现用来展示本章知识，测试用来证明新增复杂度确实解决了问题。

代码组织上，不要把所有内容放进一个大文件。基础类型、输入生成、核心机制、报告输出和测试应分开。即使当前只是教学项目，也要让目录能表达责任边界。这样后续章节接入时，不会因为找不到状态归属而重新发明一套对象。

每个实现都要输出一段机器可读摘要。摘要至少包含输入规模、关键状态、错误数、输出校验、耗时和未验证边界。这里的机器可读不要求复杂格式，简单的 key value 文本也可以。关键是让脚本和读者都能复查，而不是只能看屏幕截图。

测试应包含正常路径、边界路径和失败路径。正常路径证明功能可用；边界路径证明长度、容量、尾部、空输入、关闭或超时没有遗漏；失败路径证明本章机制不是装饰。若失败路径写不出来，往往说明本章概念还没有落到工程对象上。

报告最后写一个取舍段。新增机制带来了什么好处，增加了什么成本，哪些场景不应该使用它，下一章会复用哪个字段或对象。这个取舍段非常重要，因为系统工程不是把所有高级机制都加进去，而是在证据和约束下选择合适的边界。

调试笔记：CPU 使用率低不等于线程不够 CPU 使用率低时，先看线程是 sleeping、blocked 还是 runnable。若 worker 在 I/O 中睡眠，加计算线程不能解决写出慢；若任务太小，更多线程会增加调度成本；若队列为空，瓶颈可能在上游；若队列满，瓶颈可能在下游。线程章节的排查顺序是任务状态先于线程数量。

复查清单：线程、调度与亲和性 复查本章时，先检查 reference 是否仍然存在。没有 reference，后面的并发、优化、异步或分布式改造都无法证明语义没有变。再检查压力输入是否尖锐：它应当攻击本章最核心的边界，而不是只把数据量放大。第三，检查状态是否可观察：关键对象的身份、状态、错误和提交结果应能在报告中找到。

第四，检查代码示例是否说明禁止行为。调用者不能做什么，往往比能做什么更能体现工程质量。第五，检查实验是否写出未验证边界；当前设备、权限或输入无法证明的结论，必须标注。第六，检查本章对象是否能被下一章复用，不能复用就说明边界还不稳。

完成这六项后，再看文字是否连贯。每个概念都应从问题推出，每个案例都应从朴素方案走到改进方案，每个实验都应能解释一个判断。若某段话放到其他章节也成立，就把它改成本章的具体对象、具体输入和具体失败。

9.8 工程案例：从请求生命周期推导线程、调度和亲和性

线程不是越多越好，也不是系统自动帮你把并发变成吞吐。线程只是执行实体，真正要分析的是请求生命周期：什么时候进入队列，什么时候获得线程，什么时候运行，什么时候阻塞，什么时候完成，什么时候释放资源。若生命周期没有画清，线程数、亲和性和调度策略都会变成拍脑袋参数。

一、线程创建成本不是唯一问题 每次请求创建一个线程，最直观，也最容易解释。但线程创建需要内核资源、栈空间和调度开销；大量线程还会增加上下文切换、缓存污染和内存占用。固定线程池减少创建成本，却引入队列、关闭、背压和任务粒度问题。教材不能只说“线程池更快”，要讲清线程池把成本从创建迁移到排队和调度。

Compute Systems Engine 可以先比较三种模式：每任务一个线程、固定线程池、按阶段专用线程池。每任务线程适合作为反例；固定线程池适合 CPU 计算；专用线程池适合隔离阻塞 I/O。报告记录线程数量、任务耗时、排队时间、上下文切换和内存峰值。这样读者会看到线程模型影响的不只是平均吞吐。

二、阻塞会吞掉并行度 线程池里运行 CPU 任务时，worker 应尽量保持可运行。如果任务在 worker 内等待磁盘、网络、future 或条件变量，线程就被占住，ready 任务可能无人执行。CPU 使用率低并不一定说明线程不够，可能说明线程都在等待下游。增加线程有时能缓解，有时会让队列和内存更糟。

解决方式取决于等待类型。短等待可以自旋或让出；长 I/O 等待应交给异步 I/O 或专用阻塞

池；任务依赖等待应交给任务图 continuation；背压等待应反馈给上游，而不是无限堆积线程。线程章要把等待分类讲清楚，否则读者会把所有低 CPU 使用率都解释成“加线程”。

三、线程亲和性服务于局部性而不是仪式感 绑定线程到 CPU 可以减少迁移，保留 cache 热度，也能让 NUMA 放置更稳定。但绑定不当会造成负载不均，影响系统其他进程，或者在移动设备上被调度器策略覆盖。亲和性是工具，不是默认美德。使用前要知道你要保护什么局部性：输入块、线程本地 partial、本地队列，还是某个设备队列。

项目中，亲和性计划应和输入分片绑定。worker 0 处理 split 0，并在同一 CPU 或 NUMA node 上初始化对应 partial；若 worker 被迁移，trace 记录实际 CPU。这样可以比较“计划放置”和“实际执行”。没有实际观测的绑定策略，只是配置文件里的愿望。

四、线程局部存储有生命周期边界 线程局部缓存可以减少分配和共享写，例如每个 worker 持有解析 buffer、局部 histogram、临时 vector。问题是线程局部对象的生命周期通常长于任务。若其中保存上一个任务的状态，必须在任务开始或结束时清理；若线程池关闭，析构顺序要明确；若任务被取消，局部缓存不能泄漏未提交结果。

线程局部存储还会扩大内存峰值。每个 worker 一个 8 MiB buffer，128 个 worker 就是一大块常驻内存。高性能程序常用线程局部缓存，但要记录容量、复用策略和清理点。不要把 thread local 当成免费优化。

五、调度器看的是可运行实体 操作系统调度的是线程，不理解你的任务语义。一个线程在用户态忙等，在调度器眼里仍然可运行；一个线程阻塞在 I/O，调度器只知道它不可运行，不知道上游是否应该减速；一个线程池内部的 ready queue，内核也看不到。用户态运行时必须把任务语义转成线程行为，否则调度器无法替你做出正确决定。

这解释了为什么任务运行时重要。若任务图能避免 worker 阻塞，操作系统看到的是持续可运行的少量 worker；若任务都在 worker 内等待 future，操作系统只看到线程睡眠或忙等，却不知道 ready 任务被困在队列里。线程章和任务运行时章的连接点就在这里。

六、移动设备和 Termux 的特殊边界 用户当前环境可能是 Termux。移动设备常有 big.LITTLE 核心、温控降频、后台限制、权限不足和不同调度策略。性能实验必须记录设备型号、核心拓扑、是否充电、温度状态、CPU governor、可用 perf 权限。不能把手机上的数字直接推广到服务器，也不能因为某些计数器不可用就停止学习。

在 Termux 上仍可做很多实验：固定输入，多轮计时，比较线程数，观察 /proc 信息，记录可用 CPU 列表，测试线程创建和队列开销。报告写清“该设备无法验证 NUMA”或“PMU 事件不可用”，这比伪造完整证据更专业。

七、实验交付：ThreadTimeline 本章给项目留下 ThreadTimeline。它记录 task id、worker id、提交时间、出队时间、开始时间、结束时间、阻塞时间、实际 CPU 和状态。线程池实验至少比较每任务线程、固定线程池和阻塞隔离池。输入包含 CPU 密集任务、短任务、大量小任务和混合 I/O 等待任务。

验收时看三件事。第一，线程数增加后吞吐和尾延迟如何变化。第二，队列等待是否成为主成本。第三，worker 是否被阻塞任务耗尽。能解释这三件事，读者才真正理解线程和调度，而不是只会调用 `std::thread`。

9.9 案例深化：线程池配置从工作负载推导

线程池大小不应是固定口诀。CPU 密集、I/O 密集、混合任务、长短任务、阻塞任务和 NUMA 拓扑都会改变合适配置。高质量教材要教读者推导，而不是给“核心数乘以二”这样的死规则。

一、CPU 密集任务从核心数开始 纯 CPU 热循环通常从物理核心数附近开始试验。若开启 SMT，线程数略高可能有收益，也可能争执行资源。若内存带宽先到顶，继续加线程没有意义。报告要看扩展曲线，而不是只测一个线程数。

任务粒度也影响线程数。任务太小，调度开销主导；任务太大，负载不均和尾部任务拖慢。配置线程池时要同时调任务粒度和线程数。只改线程数，无法解决长尾。

二、I/O 阻塞任务需要隔离 如果同一个线程池既跑 CPU 任务又跑阻塞 I/O，I/O 任务可能占住 worker，CPU ready 任务排队。解决方式是专用 I/O 池、异步 I/O、或者运行时识别 blocking section 后临时补线程。不要简单把线程数调得很大，因为这会增加内存、上下文切换和调度噪声。

Compute Systems Engine 可以把 map compute 和 shuffle write 分成两个执行资源。Compute 池按核心数配置，I/O 池按设备并发和队列深度配置，两者之间用有界队列背压。这样配置来自资源模型，而不是来自经验常数。

三、混合任务需要测等待比例 若任务一半时间计算、一半时间等待，合适线程数可能高于核心数。但等待比例不是固定属性，它会随输入、设备和下游压力变化。运行时 trace 应记录 running time 和 blocked time。线程池配置可以依据测到的等待比例调整，而不是静态猜测。

动态调整也要谨慎。频繁创建销毁线程有成本，过度响应短期波动会造成抖动。教学项目可以先做静态配置和报告，不急实现自适应。理解指标比实现复杂控制器更重要。

四、配置报告 每次改变线程池配置，都写明任务类型、线程数、队列容量、任务粒度、阻塞比例、吞吐和尾延迟。若最佳线程数不是核心数，报告解释原因。若线程数增加后吞吐不变，说明瓶颈不在可运行线程数量。这样的配置报告能让线程章节落到工程实践。

9.10 补强案例：线程生命周期和资源泄漏

并发程序不只怕数据竞争，也怕线程生命周期泄漏。线程启动后谁负责 join，取消时怎样退出，线程局部资源何时释放，阻塞系统调用如何唤醒，异常是否会杀死进程，这些都是线程章必须讲清的工程细节。

一、join 和 detach 的语义差异 join 表示调用者等待线程结束并回收资源；detach 表示线程独立运行，调用者不再能同步它的结束。教学代码常随手 detach，生产代码应非常谨慎。Detached 线程如果还访问对象，而对象已销毁，就会悬垂；如果线程抛异常，错误难以传播；如果进程退出，清理顺序不清。

线程池通常避免暴露 detach，而是在 shutdown 中统一 join worker。这样生命周期集中管理。若确实需要后台线程，也要有停止标志、唤醒机制和析构顺序。

二、取消需要唤醒阻塞点 设置 cancel flag 不足以让线程退出。如果线程阻塞在条件变量、队列 pop、I/O 或 sleep，必须唤醒它，让它重新检查取消状态。关闭队列时 notify all，就是这个原则的例子。取消设计要列出所有阻塞点，逐个说明如何唤醒。

三、异常传播 线程函数抛出未捕获异常通常会导致终止。线程池应捕获任务异常，把它存入 future shared state 或 task record，再通知等待者。直接在 worker 打日志然后继续，会让调用者以为任务成功。异常是结果的一种，不能脱离状态机。

四、实验 构造三个错误版本：忘记 join，取消不唤醒，worker 未捕获异常。观察悬挂、资源泄漏或进程终止。再改成 RAII thread group、QueueContract close 和 future 异常传播。这个实验让线程生命周期从概念变成可见故障。

9.11 补充案例：线程池监控指标

线程池没有监控，就无法调优。指标不能只包括完成任务数，还要反映排队、运行、阻塞、失败和取消。监控本身也要低开销，不能成为新瓶颈。

一、基本指标 至少记录 submitted、started、completed、failed、cancelled、queue depth、running workers、blocked workers、steal count、average queue wait、p99 task latency。计数类指标可以分片 relaxed，状态类指标需要更强合同。

二、指标语义 queue depth 是近似监控还是调度决策？如果只是展示，可以 relaxed；如果用于关闭或限流，必须进入锁或状态机。指标的语义决定同步方式。不要让监控变量偷偷变成协议变量。

三、低开销采集 每任务都写全量日志会很重。可以采样、分线程缓冲、批量 flush。关键状态转换仍可追踪。调试模式和生产模式可以不同，但核心计数要一致。

四、实验 给线程池加指标，运行短任务、高阻塞任务和长尾任务。只看吞吐时很难解释的问题，用 queue wait 和 blocked workers 解释。这个实验帮助读者把调度问题变成数据。

9.12 补充专题：线程池里的任务粒度

任务粒度决定调度开销和负载均衡。一个任务太小，入队、出队、状态记录和唤醒成本超过计算；一个任务太大，长尾任务拖住阶段，其他 worker 空闲。线程池调优离不开任务粒度。

一、**粒度来自固定成本和可变成本** 每个任务都有固定成本：创建 task record、入队、出队、trace、可能的 future 状态。任务本身有可变成本：处理多少字节、多少记录、多少 key。合适粒度应让可变成本明显大于固定成本，同时保留足够任务数量做负载均衡。这个判断要测量，不应凭感觉。

二、**粒度和 cache** 任务太大可能让工作集超过 cache，任务太小又无法复用局部性。解析日志时，一个 batch 过小会让函数调用和队列成本高；过大则错误恢复范围大，HotBatch 占用 cache 多。粒度是硬件和运行时的交界点。

三、**粒度和取消** 任务越大，取消响应越慢；任务越小，取消更及时但调度成本高。长任务应主动检查 cancel token 或拆成子任务。线程池不能强行杀死 C++ 线程，因此协作式取消需要任务粒度配合。

四、**实验** 固定总输入，改变 batch 记录数：一百、一千、一万、十万。记录吞吐、队列等待、worker 空闲、cache miss 替代指标和取消延迟。报告选择默认粒度，并说明在哪些机器上需要重新测。

9.13 补充专题：线程池与操作系统调度的边界

用户态线程池负责决定哪些任务放入哪些 worker 队列；操作系统调度器负责决定哪些线程在哪些 CPU 上运行。二者不是同一个层次。线程池以为 worker 正在运行，实际可能被抢占；线程池把任务放到本地队列，线程却可能迁移到另一个 CPU。理解这个边界能减少很多误判。

一、**用户态只能表达意图** 线程绑定、优先级、nice 值、CPU set 都是向操作系统表达意图。系统可能因为权限、负载、节能策略或容器限制改变实际执行。报告应记录实际 CPU，而不是只记录计划。若计划和实际不同，性能分析要以实际为准。

二、**抢占会影响延迟** 一个 worker 执行长任务时可能被操作系统抢占。任务耗时变长不一定是任务本身慢，也可能是调度延迟。Trace 中记录 wall time 和 CPU time 可以区分。移动设备上温控和后台策略更明显。

三、**线程池不要假设独占机器** 普通用户程序通常和系统其他进程共享 CPU。线程数设置为全部核心，可能影响系统响应，也可能被其他进程干扰。教学实验应尽量减少干扰，但工程报告要承认共享环境。服务端部署中，还要考虑容器配额和 CPU throttling。

四、**实验** 运行同一线程池实验，一次不绑定，一次绑定，一次人为制造后台负载。记录实际 CPU、任务时间分布和上下文切换。读者会看到调度环境是性能结论的一部分。

9.14 从一个线程到可解释的执行资源

线程章节不能只讲创建、join、detach 和调度策略。更重要的问题是：为什么程序需要多个执行实体，它们竞争什么资源，等待发生在哪里，操作系统凭什么移动它们，移动以后缓存和 NUMA 成本如何变化。线程不是“让程序快”的按钮，而是把一个计算任务拆到多个执行资源上的合同。合同写得好，线程数、亲和性、任务粒度和队列长度都能解释；合同写得差，线程越多，系统越不可预测。

一、**线程数来自并行工作量和等待比例** 如果任务完全受 CPU 计算限制，线程数通常不应远超可用核心；如果任务大量等待 I/O 或远端响应，适度超额线程可能隐藏等待。这里的关键是先把工作拆成运行时间和等待时间。一个解析任务若 90% 时间都在 CPU 上，线程数超过核心数会主要增加上下文切换；一个网络抓取任务若大部分时间等待响应，多线程可以提高 in-flight 数。不要用固定公式回答“开多少线程”，要用工作负载、核心数、等待比例和内存预算共同推导。

二、**任务粒度决定调度成本是否可承受** 任务太大，负载不均会导致尾部线程拖住整体；任务太小，调度、队列和同步成本会压过计算。合理粒度通常来自测量：每个任务至少要有足够计算量覆盖入队、出队、状态更新和缓存冷启动成本，同时又不能大到无法均衡。日志处理可以按块切分，图遍历可能按前沿动态切分，矩阵计算可以按 tile 切分。任务粒度不是线程池参数，而是算法结构和运行时成本的交点。

三、**亲和性不是把线程钉死就完事** 线程亲和性有两个目标：减少迁移造成的缓存冷启动，维持线程和数据的 NUMA 关系。可是强行绑定也可能伤害负载均衡。若一个线程绑定在繁忙核心上，另一个核心空闲，系统仍不能迁移；若任务的数据会变化，固定绑定可能让线程远程访问更多内存。工程上应先测迁移次数、运行队列长度、NUMA 远端访问和尾延迟，再决定绑定策略。亲和性是证据驱动的约束，不是性能仪式。

四、等待点要从线程栈移到状态表 线程阻塞在锁、条件变量、future、I/O 或远端调用上时，它占用的是执行资源和调试注意力。复杂系统不能只看调用栈，要有状态表记录每个任务为什么等待、等待谁、能否取消、超时后如何处理。这样线程调试才不会变成“哪个线程卡住了”的猜谜。后续运行时章节会把等待进一步移动到任务图和 continuation，这里的基础就是先把线程等待显式化。

五、线程报告要包含操作系统视角 用户态日志只能说明程序自己看到的状态，不能解释所有调度现象。报告应结合 `top`、`perfsched`、`taskset`、`numactl` 或 `/proc` 信息，至少记录线程数量、核心分布、上下文切换、运行队列和 CPU 利用率。若在 Termux 或受限环境中工具不可用，也要写清降级证据：线程日志、阶段耗时和队列长度。线程章节的目标是让读者能解释执行资源的使用，而不是只会创建更多线程。

9.15 线程池状态表和排查路线

线程池是多线程程序最常见的基础设施，也最容易被写成黑盒。为了让它可解释，至少要维护一张状态表：每个 worker 当前是 Idle、Running、Blocked、Stopping 还是 Joined；当前任务 id 是什么；开始时间是什么；上一次从队列取任务是什么时候；是否持有本地 buffer；是否绑定核心。这样当系统慢下来时，读者能先看状态，而不是直接猜测锁、调度器或 CPU。

一、Idle 多说明任务供给不足或等待过多 如果大部分 worker 是 Idle，而输入队列也空，说明上游供给不足或系统已经完成；如果 worker Idle 但全局还有未完成任务，可能是依赖没有释放、ready 队列没有被唤醒、任务被错误取消或队列关闭过早。Idle 本身不是坏事，关键是它是否符合任务图状态。报告要把 worker 状态和任务状态放在一起看。

二、Running 多但吞吐低要看资源瓶颈 所有 worker 都 Running，不代表系统有效工作。它们可能在争同一把锁、打同一个原子、访问远端内存、等待 page fault 或执行低效分支。此时要看阶段耗时、锁等待、cache miss、内存带宽和队列长度。线程池只能让任务并发执行，不能消除共享资源瓶颈。若共享写热点存在，增加 worker 只会放大争用。

三、Blocked 多要追等待对象 worker 阻塞时，应记录等待类型：mutex、condition variable、future、I/O、semaphore、RPC 或 sleep。每种等待对应不同修复路线。等待 mutex 要看临界区和锁顺序；等待 condition variable 要看谓词和通知；等待 future 要看任务图是否饥饿；等待 I/O 要看背压；等待 RPC 要看重试和 deadline。没有等待对象，Blocked 只是一个模糊标签。

四、Stopping 和 Joined 检验关闭协议 关闭时，worker 应从 Running 或 Idle 进入 Stopping，完成资源释放后进入 Joined。若关闭后仍有 Running，说明任务不响应取消或关闭顺序错误；若仍有 Blocked，说明等待者没有被唤醒；若线程已经 Joined 但任务仍未完成，说明状态转移丢失。线程池的关闭测试应成为常规测试，而不是只在程序退出时顺便跑到。

五、用小输入复现调度问题 调度问题不一定需要大输入。两个 worker、三个任务、一个阻塞 future 就能复现饥饿；容量为一的队列就能复现背压；两个锁反向获取就能复现死锁。小输入更容易画出时间线。线程章节要鼓励读者先构造最小失败，再扩大规模确认，而不是直接在大系统里搜索偶发日志。

9.16 贯穿案例：写出任务占满线程池以后，解析为什么也停了

线程章节如果只讲 `std::thread`、`join`、`detach` 和 `affinity`，读者很难知道这些知识什么时候用。考虑一个日志处理程序，只有一个固定线程池。读取任务、解析任务、压缩任务、写出任务、completion 回调全都丢进同一个 FIFO 队列。正常 SSD 上写出很快，系统表现不错。某次磁盘变慢，写出任务长时间阻塞，线程池里的 worker 都在 write 或 fsync 上等待；新的解析任务已经 ready，却没有 worker 执行；completion 回调也排在队列里，导致已经完成的 I/O 不能及时提交 manifest。最后上游读取队列满，下游提交也慢，整个系统像死了一样。

这个事故能推出线程池设计的主线。线程不是越多越好，线程池也不是所有任务的垃圾桶。任务要按资源和阻塞性质分类：CPU 计算任务需要核心和 cache，阻塞 I/O 需要设备队列和 in-flight 限制，completion 回调需要短小及时，maintenance 可以低优先级。若所有任务共享同一组 worker，一个慢资源会绑架所有执行资源。

一、先把任务状态和资源状态分开 解析任务 ready 不代表线程会执行它；写出任务 Running 可能其实在内核里等待；completion ready 可能排在队列后面。报告要同时记录任务状态和 worker

状态。若 worker 全部 Blocked on I/O，而 compute queue 非空，问题不是 CPU 不够，而是资源隔离错误。线程池可解释性的第一步，是让每个 worker 说清自己在运行、阻塞、空闲还是停止。

二、计算池和 I/O 池来自阻塞点 一种修复是把 compute 和 blocking I/O 分成不同执行资源。Compute 池接近核心数，避免被阻塞任务占住；I/O 池或异步 I/O 层受设备并发和 buffer 预算限制；completion 可以在短回调队列中执行，不能被长任务挡住。这个设计不是为了复杂，而是让每类资源的队列高水位有独立含义。Compute 队列高说明 CPU 或任务粒度问题，I/O 队列高说明设备或写出背压问题。

三、线程数调大只是掩盖症状 把线程数从 8 加到 64，短期可能让解析任务抢到一些 worker，但内存峰值、上下文切换和 cache 失效率都会上升。阻塞写出仍然可能占满更多线程。正确路线是给写出设置 in-flight 上限，让上游背压，而不是让线程池无限吸收等待。线程数调参必须建立在任务分类之后，否则只是把等待从一个地方搬到另一个地方。

四、completion 回调要短且可抢先 Completion 的任务通常是更新请求状态、释放 buffer、触发 manifest 提交或唤醒后继任务。它不应执行大块解析、压缩或用户回调。若 completion 被长任务挡住，I/O 已经完成却不能进入提交阶段，buffer 也不能归还，背压会被放大。最终项目可以给 completion 单独队列，或者让 worker 在处理长任务前优先 drain 少量 completion。无论哪种策略，报告要记录 completion 延迟。

五、关闭协议按资源顺序执行 系统关闭时，先停止读取新输入，再让 compute 队列 drain 或 cancel，再停止提交新 I/O，再等待 completion 归还 buffer，最后提交或回滚 manifest。若顺序反过来，可能出现 compute 仍在产生 block，而 I/O 池已关闭；或者 I/O completion 迟到，manifest 对象已销毁。线程生命周期不是析构函数最后的清理，而是贯穿所有资源池的状态机。

六、这个案例如何验证 构造两个 worker 的 compute 池和两个 worker 的 I/O 池。先用单池版本，让写出任务 sleep 模拟慢磁盘，观察解析任务排队和 completion 延迟。再用双池加有界队列，观察 compute worker 不被 I/O 阻塞占住，上游在 I/O 队列满时背压。报告记录 worker blocked time、queue wait、completion delay、buffer pool high watermark。这样线程池设计从事故走到证据，而不是靠口诀。

9.17 案例深化：线程池从任务生命周期推导

线程章节不应只给“线程数等于核心数”这种经验。考虑一个日志处理程序：解析任务 CPU 密集，写出任务 I/O 密集，压缩任务既吃 CPU 又吃内存带宽，完成回调很短但必须及时执行。若它们都丢进同一个 FIFO 线程池，慢写出会占住 worker，短回调排队，解析吞吐抖动。线程池设计必须从任务生命周期和阻塞点推导。

一、先分类任务 任务至少分为 compute、blocking I/O、completion、maintenance。Compute 任务适合绑定核心并保持 cache 热度；blocking I/O 任务不应占满计算 worker；completion 要短小及时；maintenance 可以低优先级运行。分类不是为了复杂，而是防止一种任务把所有执行资源占住。

二、线程数来自资源而不是愿望 CPU 密集池通常接近核心数，I/O 池可能更多但要受文件描述符和内存限制，completion 池要保证不会被长任务阻塞。若任务会等待下游队列，增加线程只会增加等待者。线程数要和队列容量、任务粒度、阻塞比例一起调。报告中应记录 runnable 数、上下文切换、队列等待和 worker idle，而不是只写线程数。

三、亲和性服务局部性 绑定线程可以减少迁移，让 cache 和 NUMA 放置更稳定；也可能在负载不均时降低利用率。适合绑定的是长时间处理同类数据的 worker，例如按 split 或 partition 工作的计算线程。不适合死绑的是短任务或负载强烈波动的任务。亲和性策略要配合任务偷取或重平衡，否则会把局部性变成饥饿。

四、关闭路径是生命周期的一部分 线程池关闭时，要决定已提交任务 drain 还是 cancel，新任务 reject 还是阻塞，正在运行任务如何响应取消。若关闭只是设置一个 bool，等待任务可能永远不醒，completion 可能访问已销毁对象。线程池的生命周期状态至少包含 running、closing、draining、stopped。这个状态机会被 I/O 和分布式章节继续使用。

五、Trace 要能解释空闲和拥塞 运行时 trace 应记录 task submit、start、block、resume、finish、worker sleep、worker wake。看到吞吐下降时，先问 worker 是否空闲，队列是否有 ready task，任

务是否阻塞，是否发生迁移。没有这些事件，线程调优只能猜。线程章节的目标是让执行资源可见，而不是教几个 API 名称。

9.18 案例复盘：线程池按任务生命周期设计

解析、聚合、压缩、写出和 completion 放进同一个 FIFO 线程池，磁盘一慢，worker 被 write 占住，短计算任务排队。增加线程只带来更多切换和更差 cache 热度。线程池设计应从任务类型、阻塞点和关闭语义推导。

执行资源要分类 Compute 任务需要核心和 cache，blocking I/O 不应占满计算 worker，completion 要短而及时，maintenance 可以低优先级。每类任务记录提交、开始、阻塞、恢复和完成时间。线程数来自资源预算，不来自“越多越快”。

实验判据 报告写 runnable 数、上下文切换、worker idle、队列等待、阻塞点和绑定策略。关闭测试要覆盖 drain、cancel、reject。看见吞吐下降时，先判断 worker 是空闲、阻塞还是任务太小，而不是立刻改线程数。

9.19 本章习题

习题与作业

习题一：给当前 `parallel_sum` 写一份线程生命周期分析。指出每次调用创建多少线程，线程做多少实际工作，join 等待在哪里发生，哪些成本会在输入很小时占主导。

习题二：设计一个任务粒度实验。固定总输入大小和 worker 数，改变 chunk size。报告要包含任务数、总时间、每个 worker 任务数和尾部完成时间。解释粒度太小和太大分别会出现什么现象。

习题三：构造线程池死锁案例：worker 任务提交子任务并等待。画出等待图，说明为什么增加队列容量不能解决这个死锁。再提出两种运行时语义上的修复方案。

习题四：记录本机或手机上的 CPU 拓扑。写清逻辑 CPU、物理核心或可见核心、是否能看到 SMT、是否有 NUMA 信息、是否允许设置 affinity。不要把无法获取的信息编造成确定结论。

第 10 章

锁、条件变量与信号量

先看一个有界队列 bug：生产者关闭队列后，某个消费者仍然睡在条件变量上，测试偶尔卡死。代码里有锁，也有 `notify_one`，看起来同步原语都用上了；真正的问题是等待谓词没有包含 `closed` 状态，通知和状态转移没有形成一个完整协议。另一个版本把 `size` 在锁外读取，为了缩短临界区，反而让 `head`、`tail` 和 `size` 的不变量被拆开。

本章从这种队列关闭事故出发讲锁、条件变量和信号量。锁的作用不是给代码贴一个“线程安全”标签，而是保护一组共享状态不变量；条件变量不是让线程睡醒的工具，而是让状态变化和等待谓词配对；信号量不是万能限流器，而是把容量写成可等待的资源计数。正确使用这些原语，必须先明确保护对象、锁顺序、临界区长度和等待条件。

10.1 从共享状态到互斥锁

每把锁都应该有清晰的保护范围。一个队列锁保护 `head`、`tail`、`size` 和 `buffer` 状态；一个计数器锁保护 `value`；一个对象锁保护对象不变量。锁如果没有对应的不变量，就容易出现“看起来加锁了但仍然错”的情况。

临界区应尽量短，但不能为了短而破坏不变量。把状态检查放在锁外，可能引入竞态；把耗时 I/O 放在锁内，会扩大阻塞范围。工程上要在正确性和粒度之间找到边界。

Listing 10.1 锁保护队列不变量

```
1 class QueueState {
2 public:
3     void push(std::int32_t value) {
4         std::lock_guard<std::mutex> lock(this->mutex_);
5         this->values_.push_back(value);
6     }
7
8 private:
9     std::mutex mutex_;
10    std::vector<std::int32_t> values_;
11 };
```

这段代码的关键不是 `push_back` 这一行，而是锁和不变量的关系：`values_` 的内部结构不能被多个线程同时修改。如果未来再加入 `size_`、`head_` 或统计字段，它们也必须在同一把锁保护下保持一致。

写锁代码时，应先写“锁保护的不变量”，再写代码。比如有界队列的不变量可以是：`0<=size<=capacity`，`head` 和 `tail` 总是落在环形数组范围内，`size==0` 表示不能 `pop`，`size==capacity` 表示不能 `push`。互斥锁保护的是这些关系，不只是保护某一个变量。若 `size` 在锁内改，`head` 在锁外改，这个不变量就被拆碎了。

锁粒度 粗粒度锁容易证明正确，但并发度低。细粒度锁提高并发度，却增加锁顺序、死锁和状态拆分复杂度。分片锁是一种常见折中：按 `key`、队列分段或线程组拆成多把锁，让无关操作并行，热点操作仍受保护。

锁粒度应从访问模式出发。如果所有请求都访问同一个热点 `key`，分片没有帮助；如果请求天然分散，分片锁可以显著减少竞争。锁优化前要先测量等待时间和竞争比例。

10.2 从等待谓词到条件变量

条件变量用于等待某个状态变真。它必须和互斥锁配合，并在循环里检查谓词。原因是线程可能被虚假唤醒，也可能在被唤醒后发现条件已经被其他线程消费。正确模式是“持锁检查条件，不满足则等待，醒来继续检查”。

Listing 10.2 条件变量等待谓词

```
1 std::unique_lock<std::mutex> lock(this->mutex_);
2 this->not_empty_.wait(lock, [this]() {
3     return !this->values_.empty();
4 });
5 const std::int32_t value = this->values_.front();
```

这里谓词属于受锁保护的状态。通知本身不携带数据，数据在共享状态里。若不用循环检查谓词，就可能在虚假唤醒或竞争消费后读取空队列。

条件变量最重要的句子是：等待的是谓词，不是通知。通知可能在等待前发生，也可能唤醒多个线程，也可能出现虚假唤醒。只要谓词在锁保护下检查，程序就正确；只依赖“我收到过一次 notify”，程序就脆弱。生产者消费者队列里，`not_empty` 的谓词是队列非空，`not_full` 的谓词是队列未滿。两个谓词对应同一组受锁保护状态。

信号量和资源计数 信号量表示可用资源数量。它适合限制并发度、实现 bounded buffer 或控制连接池。信号量和互斥锁不同：互斥锁保护状态的一致性，信号量表达资源额度。混用时要小心锁顺序，避免死锁。

10.3 锁顺序、异常和性能路径

死锁通常需要互斥、持有并等待、不可抢占和循环等待四个条件。破坏循环等待的常见方法是规定全局锁顺序。复杂系统里，锁顺序文档和锁级别检查很重要，因为死锁往往在低概率路径中出现。

优先级反转和 convoy 优先级反转发生在低优先级线程持有锁，高优先级线程等待它，而中等优先级线程持续占用 CPU，导致高优先级线程无法前进。锁 convoy 则是多个线程排队等待同一锁，锁释放后唤醒和抢占成本让系统吞吐下降。这些问题说明锁不仅是用户态对象，也和调度器行为相互作用。

工程系统要避免在锁内做不可控工作，例如磁盘 I/O、网络调用、复杂回调和用户插件。锁内执行时间越不可预测，越容易出现尾延迟问题。

锁的性能路径 mutex 通常有三条路径。第一条是无竞争 fast path，用户态原子操作成功，成本较低。第二条是短暂竞争路径，线程可能自旋几轮，等待锁持有者很快释放。第三条是阻塞路径，线程进入内核等待，涉及 futex、调度和唤醒。性能分析要判断当前主要走哪条路径。一个低竞争 mutex 没必要改成复杂无锁结构；一个长期进入阻塞路径的热点锁，则需要拆分状态、缩短临界区或改变算法。

临界区长度不只由代码行数决定。锁内分配内存、写日志、调用用户回调、访问远端 NUMA 内存、发生 page fault、触发异常路径，都会让持锁时间变长。锁优化报告应列出锁内可能的慢操作，而不只是说“这个锁很粗”。

读写锁和 RCU 直觉 读多写少场景中，读写锁允许多个读者并发，但写者需要独占。它适合读临界区不太长、写入不太频繁的结构。若读者很多且读路径必须极轻，RCU 的思想更进一步：读者在不阻塞写者的情况下读取旧版本，写者发布新版本后，等所有旧读者离开再回收旧对象。

RCU 的难点不在“读很快”这个口号，而在对象生命周期。旧对象什么时候没人再读，什么时候可以释放，如何处理写者之间的同步，都是必须证明的问题。后面讲无锁内存回收时，会看到相同主题。

10.4 有界队列、关闭和背压

Compute Systems Engine 的第一个同步结构应是有界队列，因为它把互斥、条件变量、背压和关闭语义放在同一个小系统里。有界队列不是一个简单容器，它表达了生产者和消费者之间的容量契约：队列未滿时生产者可以放入，队列非空时消费者可以取出，队列满时生产者必须等待或失败，队列关闭后等待者必须能醒来并退出。

先写不变量，再写代码。一个环形有界队列至少有这些不变量：容量大于零；`head` 和 `tail` 总在 $[0, \text{capacity})$ 范围内；`size` 总在 $[0, \text{capacity}]$ 范围内；`size==0` 时没有元素可取；`size==capacity` 时没有空槽可写；`head` 指向下一个可读槽；`tail` 指向下一个可写槽。互斥锁保护的是这组关系，而不是某一个整数。

只要写代码时先写不变量，很多错误会提前暴露。例如 `push` 修改 `tail` 但忘了增加 `size`，`try_pop` 读取元素后忘了通知 `not_full`，`size` 用无符号类型导致下溢，析构时仍有线程阻塞，关闭后生产者继续写入，这些都不是语法问题，而是不变量和状态转换没有定义清楚。

从 try 到 blocking 的接口分层 队列接口不应只有一种操作。常见接口至少分成三类：非阻塞尝试、阻塞等待和关闭。非阻塞 `try_push` 或 `try_pop` 立即返回，适合事件循环或自定义调度；阻塞 `push` 或 `pop` 等待谓词变真，适合普通生产者消费者；关闭 `close` 唤醒等待者，告诉它们系统不再接受新任务或不再产生新元素。

如果队列没有关闭语义，线程池很难优雅退出。worker 可能永远阻塞在空队列上，主线程析构线程池时只能强行终止进程或依赖外部哨兵值。哨兵值在泛型队列中也不可靠，因为每种元素类型未必都有安全哨兵。正确做法是把关闭作为队列状态的一部分，受同一把锁保护，并在关闭时通知所有等待者。

Listing 10.3 有关闭语义的有界队列接口草图

```

1 class BoundedQueue {
2 public:
3     explicit BoundedQueue(std::int32_t capacity);
4
5     bool try_push(std::int32_t value);
6     bool push(std::int32_t value);
7     std::optional<std::int32_t> pop();
8     void close();
9
10 private:
11     std::vector<std::int32_t> buffer_;
12     std::int32_t head_ = 0;
13     std::int32_t tail_ = 0;
14     std::int32_t size_ = 0;
15     bool closed_ = false;
16     std::mutex mutex_;
17     std::condition_variable not_empty_;
18     std::condition_variable not_full_;
19 };

```

这里 `push` 返回 `bool`，是为了表达关闭后不能再写入；`pop` 返回 `optional`，是为了表达关闭且队列为空时消费者应退出。这个接口比单纯抛异常更适合作为线程池内部队列，因为关闭不是异常，而是正常生命周期事件。当然，业务层接口也可以选择抛异常，但内部状态机仍要明确。

等待谓词的完整写法 生产者等待的谓词不是“收到 `not full` 通知”，而是“队列未滿或已经关闭”。消费者等待的谓词不是“收到 `not empty` 通知”，而是“队列非空或已经关闭”。关闭必须出现在谓词里，否则关闭时即使 `notify_all`，线程醒来后发现队列仍满或仍空，又会继续睡回去，线程池无法退出。

Listing 10.4 `push` 的状态转换


```

1 bool BoundedQueue::push(std::int32_t value) {
2     std::unique_lock<std::mutex> lock(this->mutex_);
3     this->not_full_.wait(lock, [this]() {
4         return this->closed_ ||
5             this->size_ < static_cast<std::int32_t>(this->buffer_.size());
6     });
7     if (this->closed_) {
8         return false;
9     }
10
11     this->buffer_[static_cast<std::size_t>(this->tail_)] = value;
12     this->tail_ = (this->tail_ + 1) %
13         static_cast<std::int32_t>(this->buffer_.size());
14     ++this->size_;
15     this->not_empty_.notify_one();
16     return true;
17 }

```

Listing 10.5 pop 的状态转换

```

1 std::optional<std::int32_t> BoundedQueue::pop() {
2     std::unique_lock<std::mutex> lock(this->mutex_);
3     this->not_empty_.wait(lock, [this]() {
4         return this->closed_ || this->size_ > 0;
5     });
6     if (this->size_ == 0) {
7         return std::nullopt;
8     }
9
10     const std::int32_t value =
11         this->buffer_[static_cast<std::size_t>(this->head_)];
12     this->head_ = (this->head_ + 1) %
13         static_cast<std::int32_t>(this->buffer_.size());
14     --this->size_;
15     this->not_full_.notify_one();
16     return value;
17 }

```

这两个片段体现了条件变量的关键原则：状态改变和谓词检查都在同一把锁保护下；通知只用于让等待者重新检查谓词；关闭是状态机的一部分。注意通知可以在持锁时调用，也可以在释放锁后调用，不同写法有不同唤醒和竞争细节。教材阶段更重要的是先保证谓词正确，再讨论微小性能差异。

锁内代码的边界 有界队列的锁内代码应只做状态转换，不应执行用户回调、复杂分配、日志格式化或长时间 I/O。若 **push** 在锁内调用用户提供的处理函数，队列锁就保护了不可控代码，任何慢操作都会阻塞所有生产者和消费者。若 **pop** 在锁内执行任务，线程池全局队列会退化成串行执行器。

这条原则可以推广到所有锁。锁内只维护不变量，锁外做耗时工作。若必须在锁内移动复杂对象，应考虑预分配、移动语义、异常安全和对象析构成本。C++ 中对象析构也可能执行代码，如果最后一个引用在锁内释放，析构函数可能做不可控工作。高质量并发代码会尽量让重析构发生在锁外。

异常安全和状态回滚 C++ 队列如果存储复杂对象，`push` 可能在移动或拷贝元素时抛异常。若异常发生在 `tail` 和 `size` 已经修改之后，不变量就会损坏。教学示例用 `std::int32_t` 可以避免这个问题，但正式教材必须说明边界：泛型队列需要把可能抛异常的操作和状态提交点分开，或者要求元素移动不抛异常，或者使用预构造槽位和 `placement new carefully` 管理生命周期。

状态机里要有提交点。对于 `push`，元素成功放入槽位并且 `tail/size` 更新后，操作才对消费者可见；对于 `pop`，元素被取出并且 `head/size` 更新后，槽位才重新可用。若中间步骤可能失败，就要保证失败前队列仍保持旧状态，或失败后能回滚。锁只能提供互斥，不能自动提供异常安全。

信号量实现 bounded buffer 的视角 有界队列也可以用两个信号量表达：一个信号量记录空槽数量，一个信号量记录已有元素数量，再用互斥保护环形索引。生产者先获取空槽，再写入；消费者先获取元素，再读取。这个模型把“资源数量”和“状态互斥”分开，适合帮助读者理解信号量。

但信号量版本同样需要关闭语义。若消费者阻塞在元素信号量上，关闭时如何唤醒？若生产者阻塞在空槽信号量上，关闭时如何退出？标准信号量本身不携带“关闭”状态，需要额外状态和唤醒策略。条件变量版本把谓词写在代码里，更适合教学；信号量版本更接近资源额度控制。两者没有绝对优劣，关键是状态机是否完整。

锁顺序和组合操作 单个队列容易证明，多个锁组合就会出现锁顺序问题。假设线程池有全局队列锁、worker 本地队列锁和指标锁。如果 worker 持有本地队列锁时尝试拿全局队列锁，而另一个线程持有全局队列锁时尝试拿本地队列锁，就可能死锁。工程系统应定义锁级别：例如先全局配置锁，再队列锁，再指标锁；或者避免同时持有多把锁，用消息和局部复制拆开组合操作。

组合操作还会诱导“为了省事加一把大锁”。大锁让正确性简单，却可能把整个运行时串行化。合理做法是先用粗锁实现 reference，写清不变量和测试；再根据测量拆分锁，而不是一开始就写复杂细锁。拆锁时，每拆一把都要重新写不变量：哪些状态仍由旧锁保护，哪些状态由新锁保护，跨锁关系如何保持。

futex 慢路径和用户态 fast path Linux 上常见 mutex 和 condition variable 都会尽量把无竞争路径留在用户态。无竞争加锁可能只是一次原子状态改变；竞争时才进入 futex 让线程睡眠。条件变量等待也会释放 mutex 并进入等待，唤醒后重新竞争 mutex。这个设计说明锁不是一上来就系统调用，系统调用通常是竞争或等待的慢路径。

阅读 futex 相关内容时，要抓住三个点。第一，用户态变量保存同步状态，内核只在需要睡眠和唤醒时参与。第二，等待必须和用户态状态检查配合，避免丢失唤醒。第三，唤醒不等于立即执行，被唤醒线程还要进入调度器、被安排到 CPU，并重新竞争锁。因此高竞争条件变量的成本不只是 `notify_one`，还包括调度和再次竞争。

互斥锁保护的是不变量 很多人把 mutex 理解成“保护变量”，这句话不够精确。mutex 保护的是一组变量之间的不变量。以环形队列为例，`head`、`tail`、`size` 和 `buffer` 不是四个孤立变量，而是共同满足容量边界、满空状态、元素位置和可用槽位这些关系。只把 `size` 改成原子并不能让队列线程安全，因为 `size` 和 `head/tail` 的关系仍然会被并发修改破坏。

写并发代码时，第一步应写不变量表。哪些字段属于同一个状态机，哪些字段只用于指标，哪些字段可以近似，哪些字段必须精确。锁的粒度也由不变量决定。如果两个字段永远一起变化，拆成两把锁可能反而制造跨锁一致性问题；如果两个字段从语义上独立，一把大锁可能只是偷懒。锁设计不是“粗锁慢、细锁快”的简单比较，而是不变量边界和竞争形状的平衡。

```
bounded queue invariant:
0 <= size <= capacity
head and tail are valid positions
size == 0 means no readable slot
size == capacity means no writable slot
readable slots form one circular interval from head
writable slots are the complement interval
```

有了这张表，代码审查就更具体。每个会修改状态的函数，都必须在持锁状态下从一个合法状态转换到另一个合法状态。每个等待谓词，都必须对应一个状态条件，而不是对应一次通知。每个关闭路径，都必须说明关闭后哪些操作仍允许，哪些操作返回失败，哪些等待者必须被唤醒。

条件变量不是事件 条件变量最常见的误解，是把它当成事件通知系统。正确理解是：条件变量让线程在某个谓词不满足时睡眠，并在可能满足时醒来重新检查。通知本身不保存状态；如果通知发生时没有等待者，通知不会留在条件变量里等后来者。状态必须保存在受 mutex 保护的变量中。

这条原则可以解释为什么等待必须写成循环或谓词版本。线程可能被虚假唤醒；也可能被唤醒后别的线程先抢到锁并消费了资源；也可能关闭状态改变但队列仍空。醒来不代表条件成立，只代表“应该重新检查”。把 `if` 写成 `while` 不是形式主义，而是条件变量语义的一部分。

Listing 10.6 条件变量等待的是谓词，不是通知次数

```
1 std::optional<std::int32_t> pop_wait() {
2     std::unique_lock<std::mutex> lock(this->mutex_);
3     this->not_empty_.wait(lock, [this]() {
4         return this->closed_ || this->size_ > 0;
5     });
6
7     if (this->size_ == 0) {
8         return std::nullopt;
9     }
10
11     const std::int32_t value =
12         this->buffer_[static_cast<std::size_t>(this->head_)];
13     this->head_ = (this->head_ + 1) %
14         static_cast<std::int32_t>(this->buffer_.size());
15     --this->size_;
16     this->not_full_.notify_one();
17     return value;
18 }
```

这段代码里，`closed_` 和 `size_` 都在同一把锁下读写。若关闭函数只设置 `closed_` 却不 `notify_all`，等待线程可能永远睡眠；若关闭函数通知但不把 `closed_` 放进谓词，线程醒来后又睡回去。状态和通知必须配套。

关闭语义是并发容器的一等接口 很多教学队列只写 `push` 和 `pop`，但线程池真正需要的是关闭语义。关闭不是析构的附属动作，而是容器状态机的一部分。一个队列关闭后，生产者是否还能写入？消费者是否能继续取出已经存在的元素？空队列关闭后消费者返回什么？满队列关闭后生产者如何退出？这些问题决定 worker 能否优雅停止。

常见策略是：关闭后拒绝新 `push`；已经在队列中的元素仍允许被 `pop` 取出；当队列为空且关闭时，`pop` 返回空值；关闭函数唤醒所有生产者和消费者。这种策略适合线程池 `drain`：不再接收新任务，但已提交任务可以执行完。另一种策略是 `cancel`：关闭后直接丢弃队列中剩余任务。它适合进程退出或错误恢复，但必须明确通知调用者哪些任务未执行。没有声明的关闭策略，会让上层业务在退出、异常和超时时出现悬挂。

```
queue close policy:
open + not full: push succeeds
open + full: producer waits
closed: push fails
closed + nonempty: pop drains existing item
closed + empty: pop returns null
```

线程池的 `shutdown`、`shutdown_now`、`wait` 和析构函数应围绕这套语义设计。析构函数里悄悄阻塞等待所有任务完成，可能让用户在持锁析构时死锁；析构函数直接丢任务，又可能破坏业务语义。教材应鼓励读者把关闭写成显式 API，而不是把复杂语义藏在析构里。

锁粒度的演化 一个复杂系统最稳妥的路线，通常是先写粗锁 reference。粗锁版本把不变量集

中在一处，容易测试，容易与串行 reference 对比。等到测量证明锁确实是瓶颈，再拆成细锁、分片锁、读写锁或无锁结构。直接从细锁开始，很容易得到一个既不快也不正确的系统。

拆锁时要写迁移计划。第一步，确认锁保护的全部状态和不变量。第二步，把状态按独立性分组，设计新锁的保护范围。第三步，找出跨组操作，例如移动任务、关闭队列、汇总指标。第四步，为跨组操作定义锁顺序或避免同时持锁。第五步，保留粗锁版本作为 reference 或测试 oracle。拆锁不是把一个 mutex 机械替换成多个 mutex，而是重新设计状态机边界。

读写锁也不是默认更快。读多写少且读临界区较长时，它可能有收益；读临界区很短时，读写锁本身的元数据和公平策略可能比普通 mutex 更重；写者频繁时，读写锁还可能造成写者饥饿或读者抖动。若读路径只需要不可变快照，RCU 或 copy-on-write 可能比读写锁更适合。同步原语必须和访问模式匹配。

锁竞争的观测 锁竞争不能只靠感觉。应用指标可以记录等待次数、等待总时间、最长等待、持锁时间、队列长度和唤醒次数。系统工具可以观察 futex 调用、调度切换、CPU 利用率和 off-CPU 时间。若一个程序 CPU 使用率很低但延迟很高，可能大量时间在等待锁或 I/O；若 CPU 使用率很高但吞吐不升，可能在自旋或 CAS 失败。

持锁时间和等待时间要分开。持锁时间长，说明临界区里工作太多；等待时间长，可能因为竞争高，也可能因为持锁线程被调度走。若临界区很短但等待时间长，要看线程数是否远超核心、是否有优先级反转、是否发生 NUMA 迁移。锁问题是执行图问题，不是单行代码问题。

在 Linux 上，可以结合 `perf`、`strace-efutex`、调度 trace 和应用内部统计。教材不要求每个读者都能开启所有权限，但要让读者知道证据链：用户态等待指标说明哪里堵，futex 说明是否进入内核睡眠，调度指标说明线程是否被抢占，CPU 拓扑说明 cache line 迁移是否昂贵。

优先级反转和公平性 锁还涉及调度语义。优先级反转指高优先级线程等待低优先级线程持有的锁，而低优先级线程又迟迟得不到 CPU。实时系统中，这会造成严重延迟。某些系统通过优先级继承缓解：持锁的低优先级线程临时继承等待者的高优先级，尽快释放锁。普通服务端虽然不一定使用实时优先级，也会遇到类似现象：关键请求等待后台任务持有的锁，尾延迟被放大。

公平锁按等待顺序唤醒线程，可以减少饥饿，但可能降低吞吐，因为它限制了“刚好在 CPU 上的线程再次获得锁”的机会。非公平锁吞吐可能更高，但某些线程可能长期等待。条件变量唤醒一个还是唤醒全部，也有公平和惊群权衡。高性能系统要根据业务目标选择：批处理更关心总吞吐，交互服务更关心尾延迟和饥饿。

RAII 和异常边界 C++ 中锁必须用 RAII 管理。手写 `lock` 和 `unlock` 容易在异常、早返回和多分支中漏解锁。`std::lock_guard` 适合简单临界区，`std::unique_lock` 适合条件变量、延迟加锁和手动解锁。教材示例应让读者形成习惯：锁对象的作用域就是临界区，作用域越小越好，锁外做耗时工作。

RAII 也不能替你设计不变量。一个函数里拿了锁，但把引用返回给外部，让外部在锁释放后修改受保护状态，仍然错误。一个类内部用 mutex，但复制构造没有定义好，也可能把锁和状态复制出奇怪语义。并发类通常应禁用复制，或者显式定义移动和共享语义。锁只是同步工具，不是封装边界的替代品。

生产者消费者的完整测试 生产者消费者测试不能只检查几个值是否按顺序出来。完整测试至少包含容量为一、容量为多、单生产者单消费者、多生产者多消费者、生产者快消费者慢、消费者快生产者慢、关闭空队列、关闭非空队列、满队列关闭、重复关闭、析构前关闭、压力运行和 Thread-Sanitizer。每个测试对应一个状态机边界。

还要测试没有丢失和没有重复。可以让每个生产者生成带 producer id 和 sequence 的任务，消费者记录所有任务，最终检查总数、唯一性和每个生产者内部顺序。如果队列声明全局 FIFO，就检查全局顺序；如果只声明每生产者顺序或无序，就不要测试不存在的保证。接口保证必须和测试一致。

```
queue stress test data:
producer id: identifies writer
sequence: monotonically increasing per producer
payload: optional checksum
consumer log: records every consumed item
final check: no loss, no duplicate, declared order holds
```

性能测试也要有边界。线程创建成本不应混进队列操作；输入生成不应混进消费；日志输出不应放在热循环；关闭时间和正常处理时间要分开。否则队列性能报告很容易测到无关成本。

读写锁的适用边界 读写锁允许多个读者并发进入，写者独占进入。它看起来很适合“读多写少”，但真实收益取决于读临界区长度、写入频率、实现公平性和缓存行为。若读临界区很短，读写锁内部维护读者计数和状态的成本可能超过普通 mutex；若写者频繁，读者并发优势会被写者阻塞抵消；若实现偏向读者，写者可能饥饿；若实现偏向写者，读者尾延迟可能变差。

读写锁特别适合读操作较长、读者之间不修改共享状态、写操作低频且能接受等待的结构，例如较大的配置表、路由快照或只读索引。它不适合每次请求都要读写的小状态，也不适合读路径本身会更新统计字段的场景。很多“读操作”其实包含隐藏写，例如更新 LRU、增加引用计数、记录访问时间，这会破坏读者并发。

若读多写少结构可以使用不可变快照，RCU 或 copy-on-write 可能比读写锁更好。读者读取一个稳定指针，不加锁；写者复制一份新结构，修改后发布；旧结构延迟回收。这样读路径更轻，但写路径复制成本和回收复杂度更高。读写锁、RCU、copy-on-write 都是同一问题的不同取舍：读者成本、写者成本、一致性和回收。

锁分解案例：统计服务 假设一个统计服务维护三类状态：全局配置、每个 worker 的计数、最近错误列表。朴素实现可以用一把大锁保护所有状态，正确性最简单，但每次更新计数都和读取配置、写错误列表互相阻塞。拆锁前，应先写状态关系。配置很少更新，读多；计数高频写，读少；错误列表低频写，偶尔查询。它们的访问模式不同，适合不同同步策略。

一种改进是配置使用快照或读写锁，计数使用每 worker 分片，错误列表使用单独 mutex。这样高频计数不再抢配置锁，错误查询也不阻塞计数热路径。拆锁后要检查跨状态操作。例如“更新配置并清空统计”是否需要同时触碰配置和计数？如果需要，就要定义锁顺序或改成两阶段操作。拆锁的难点不在把 mutex 数量变多，而在找出跨状态不变量。

Listing 10.7 按访问模式拆分状态的结构形态

```

1 struct alignas(64) WorkerCounter {
2     std::uint64_t accepted = 0;
3     std::uint64_t rejected = 0;
4 };
5
6 class RuntimeStats {
7 public:
8     explicit RuntimeStats(std::int32_t worker_count)
9         : counters_(static_cast<std::size_t>(worker_count)) {}
10
11     void record_accept(std::int32_t worker_id) {
12         this->counters_[static_cast<std::size_t>(worker_id)].accepted += 1;
13     }
14
15 private:
16     std::vector<WorkerCounter> counters_;
17     mutable std::mutex error_mutex_;
18     std::vector<std::string> recent_errors_;
19 };

```

这段代码仍然省略了并发读取总数的同步语义。若只有 worker 写、实验结束后汇总，可以不加锁；若运行中监控线程读取，就要定义近似还是精确。如果要求精确实时快照，需要额外同步，分片计数的写入优势会被读取语义部分抵消。同步设计总是从读写语义出发。

锁顺序表 多个锁共存后，项目需要锁顺序表。锁顺序表不是形式主义，它让代码审查能发现潜在死锁。例如规定：配置锁先于任务图锁，任务图锁先于队列锁，队列锁先于指标锁。任何代码如

果反向获取，就必须重构或证明不会并发。没有锁顺序表，死锁常常在很久后由一次看似无关的功能触发。

锁顺序表还要说明哪些锁不能在持有时调用用户回调、不能做 I/O、不能等待 future。最危险的不是持有两把锁，而是持锁进入未知代码。用户回调可能再进入运行时，拿另一把锁，或者阻塞等待当前锁保护的状态。高质量库通常在锁内只修改内部状态，复制出需要处理的数据，然后锁外调用回调。

条件变量和多谓词 真实队列常有多个等待条件：不空、不满、关闭、取消、优先级任务到达、队列水位下降。把所有条件混成一个布尔变量会让状态机混乱。更好的方式是用明确状态字段和谓词函数。例如生产者等待“有空间或关闭”，消费者等待“有任务或关闭”，drain 等待“未完成计数为零”。每个谓词都应能用当前状态直接判断。

多个条件变量可以减少无关唤醒，但也增加复杂度。一个 bounded queue 常用 `not_empty` 和 `not_full` 两个条件变量；线程池可能还有 `idle_cv` 用于 wait；关闭时通常要 `notify_all` 所有相关条件变量。通知遗漏会导致悬挂，通知过多会导致惊群。先保证状态机正确，再用指标判断是否需要优化唤醒策略。

Semaphore 和资源额度 信号量适合表达资源额度。数据库连接池、并发请求上限、I/O in-flight 限额、令牌桶都可以用信号量思想理解。它和 mutex 的区别是：mutex 保护临界区不变量，semaphore 控制有多少个许可。把二者混淆会导致设计不清。一个线程拿到许可后，仍可能需要 mutex 修改共享队列。

信号量还要处理取消和关闭。线程等待许可时，如果上层请求超时，等待能否被取消？系统关闭时，等待者如何醒来？许可已经拿到但任务失败，是否归还？这些都是资源状态机的一部分。信号量不是“比条件变量简单”的替代品，它只是把数量资源表达得更直接。

锁保护对象的文档格式 并发类应在代码附近写清每个锁保护哪些字段，以及哪些字段是原子或线程本地。注释不需要冗长，但要能帮助维护者判断修改是否安全。比如：“`mutex_` protects head, tail, size, buffer, closed”，“`stats_` is worker-owned and read after join”，“`closed_` is only read while holding mutex”。这种注释比泛泛写“thread-safe”有用得多。

接口文档也要写清线程安全级别。某个对象是否允许多个线程同时调用 `push`？是否允许一个线程析构时其他线程仍在调用？`close` 是否幂等？`wait` 是否可以和 `submit` 并发？这些问题如果不写，调用者会按自己的理解使用，最终把内部不变量打破。

案例：关闭线程池 关闭线程池是锁和条件变量的综合案例。线程池有任务队列、未完成计数、worker 列表和状态。调用 `shutdown` 后，`submit` 应拒绝新任务，worker 应继续取出已接受任务，队列为空且状态为 shutting down 时退出。`wait_idle` 应等待未完成计数为零，但不一定关闭线程池。析构应确保 worker 已 join。

错误实现常见于三个地方。第一，只设置 `closed` 但没有 `notify_all`，睡眠 worker 不醒。第二，队列空就让 worker 退出，但 `submit` 仍可能并发加入任务。第三，任务抛异常后未完成计数没有减少，`wait` 永久阻塞。正确实现需要一张状态机，而不是几处布尔判断。

案例：有界队列和内存预算 有界队列的容量可以按任务数，也可以按字节。线程池任务通常按数量限制；I/O batch 队列更需要按字节限制。若每个任务大小差别很大，只限制数量可能导致内存爆炸。锁保护的状态也会更复杂：不仅有 `size`，还有 `bytes_in_queue`。生产者等待谓词变成“任务数未满足且字节数未超，或关闭”。

这个案例说明锁保护的是不变量集合。任务数、字节数、head/tail、closed 共同决定队列状态。拆开任何一个，都可能破坏背压语义。

锁章验收清单 本章完成后，项目里的每个锁都应能回答：保护哪些字段，临界区是否调用用户代码，是否可能同时持有多把锁，锁顺序是什么，等待谓词是什么，关闭时谁被唤醒，异常路径是否保持不变量，测试是否覆盖容量一、关闭、并发压力和重复关闭。若回答不出来，这把锁就是维护风险。

锁章节总结 锁的价值是让复杂不变量变得可证明，条件变量的价值是让线程在谓词不满足时睡眠，信号量的价值是表达资源额度。它们不是低级替代品，而是并发系统的基础工具。先用锁写正确 reference，再用测量决定是否拆锁、分片、读写锁、RCU 或无锁。把锁视为失败，只会诱导更难证明的错误代码。

同步原语决策表 选择同步原语时，先看状态形状。一个复杂不变量，用 `mutex`；等待某个谓词，用 `condition variable`；控制资源额度，用 `semaphore`；读多写少且读临界区较长，用读写锁或快锁；只需要统计，用 `relaxed atomic`；单生产者单消费者队列，可以考虑 `SPSC ring`；多生产者多消费者且关闭语义复杂，先用 `mutex bounded queue`。选择同步原语不是按“高级程度”，而是按状态语义。

案例：锁内析构 C++ 中最后一个 `std::shared_ptr` 在锁内离开作用域，可能触发对象析构。析构函数如果写日志、释放大内存、关闭文件或调用回调，就把不可控工作放进锁内。高质量代码会在锁内把对象移动到局部变量，释放锁后再让局部变量析构。这个细节常常影响尾延迟。

锁内析构说明临界区不只包含显式语句，也包含对象生命周期副作用。审查锁内代码时，要看构造、析构、分配、回调和异常，不只看表面几行。

案例：等待谓词丢失 一个常见 bug 是消费者等待 `not_empty`，但关闭时只设置 `closed` 并通知 `not_full`。结果消费者如果正睡在 `not_empty`，就永远不醒。另一个 bug 是等待谓词只写 `size` 大于零，不包含 `closed`；关闭后消费者醒来，发现 `size` 仍为零，又睡回去。正确代码必须让每个等待者的谓词覆盖所有能让它退出等待的状态。

这个案例看似简单，却是线程池停不下来的常见根源。并发代码的退出路径和成功路径一样重要。

锁章的回归阅读应围绕有界队列的关闭语义。一个队列不只是 `push` 和 `pop`，还要定义容量满时谁等待、空队列时谁等待、关闭后 `push` 是否失败、关闭后剩余元素能否继续取、异常时是否唤醒所有等待者。若这些谓词没有写清，条件变量代码就会出现丢唤醒、永久等待或关闭后仍写入。

互斥锁保护的是不变量，不是几行代码。队列的 `buffer`、`head`、`tail`、`size` 和 `closed` 必须在同一把锁下形成一致状态；分片 `map` 的每个 `shard` 锁只保护对应桶；双锁转移要定义锁顺序，防止死锁。临界区越小越好只是经验，真正的问题是最小临界区是否仍能保护完整不变量。

本章验收应要求写出队列状态表。状态包括 `Open`、`Closing`、`Closed`，动作包括 `push`、`pop`、`close`、`wait`、`notify`。每个动作都标出前置条件、状态变化和唤醒对象。通过这个表再写代码，读者会发现条件变量等待的是谓词，而不是通知本身。

Linux 源码阅读可以把 `mutex` 和 `futex` 联系起来。用户态互斥锁的快路径通常先尝试在用户态修改状态，竞争严重时才进入内核等待；`futex` 的价值就在于让“用户态可判定的等待条件”进入内核睡眠。读 `kernel/futex/core.c` 时不必追完整实现，重点看等待队列、唤醒和用户地址之间的关系。这样读者会明白：锁不是只有一条成本，低竞争、短临界区和高竞争睡眠路径完全不同。

本章还要讲异常和 `RAII`。C++ 中锁必须用 `std::lock_guard`、`std::unique_lock` 或更明确的封装管理，避免异常路径忘记释放。条件变量需要 `std::unique_lock`，因为等待时要原子地释放锁并睡眠，唤醒后再重新持有锁检查谓词。若示例代码为了短而手写 `lock/unlock`，读者会学到错误习惯。教材里的同步代码要比普通示例更严格。

信号量适合表达资源计数，但不适合替代所有队列语义。它可以限制 `in-flight` 请求数量、可用 `buffer` 数量或同时写文件数量；但元素所有权、关闭、错误传播和取消仍需要额外状态。若只用 `semaphore` 控制数量，却没有记录哪些请求已提交、哪些请求已取消，就会在关闭路径丢失语义。同步原语要服务不变量，而不是让代码看起来短。

锁性能分析也要包含临界区外的工作。一个队列慢，可能是锁等待，也可能是消费者处理慢导致队列满；一个 `map` 锁竞争高，可能是 `key` 分布倾斜，而不是 `mutex` 实现差；一个条件变量唤醒多，可能是批大小太小。优化锁之前，先画出保护对象、等待谓词和生产消费速率，否则很容易把业务背压误诊为锁问题。

10.5 同步原语要从不变量推出来

锁不是保护代码块，而是保护不变量。一个队列的不变量可能是：`head`、`tail` 和 `size` 一致；`size` 不超过容量；关闭后不再接受新元素；等待者在状态变化时能被唤醒。若不先写不变量，`mutex`、`condition variable`、`semaphore` 都会变成“觉得这里要加锁”的局部补丁。

最小有界队列是本章主线。生产者 `push`，消费者 `pop`，队列有容量，有关闭语义，有等待谓词。这个对象足够小，却包含真实并发系统的核心问题：互斥、等待、唤醒、虚假唤醒、异常安全、关闭、背压和析构。后续线程池、I/O 和分布式 `MessageBus` 都会复用这个模型。

Mutex 的边界：互斥、RAII 和临界区

Mutex 建立互斥，让同一时刻只有一个线程修改受保护状态。C++ 中应使用 RAII 锁对象，保证异常路径释放锁。临界区要尽量只包含状态检查和必要修改，不要在锁内执行长时间 I/O、用户回调或复杂计算。否则锁会把系统串行化，还可能引入死锁和优先级反转。

但“临界区越短越好”也不是绝对。若为了缩短锁，把一个不变量拆到多个锁外步骤，可能让状态中间暴露，或者需要额外原子和补偿。正确做法是先写不变量，再决定哪些操作必须在同一把锁下原子完成。比如 bounded queue 的 push：检查 closed、检查容量、写入元素、更新 size、通知消费者，这些步骤的边界要清楚。

多个锁时，要写锁顺序。双队列转移、分片 map rehash、任务从一个 deque 移到另一个 deque，都可能需要两把锁。若按调用者传入顺序加锁，会出现 ABBA 死锁。可以使用全局顺序、`std::scoped_lock` 同时锁多把 mutex，或重新设计避免同时持有。教材要给出死锁反例和修复方式。

条件变量等待的是谓词

条件变量不是消息队列，通知也不是状态本身。等待者必须在锁内检查谓词，通常写成 `cv.wait(lock, predicate)`。谓词可能是“队列非空或已关闭”，而不是“收到一次通知”。虚假唤醒、通知丢失、先通知后等待都要求谓词写清。只记 `notify_one` 和 `wait` 的 API，写不出可靠队列。

关闭语义是条件变量最好的教材案例。若消费者在空队列等待，生产者关闭队列时必须唤醒消费者，让它看到 closed 并退出。若只设置 closed 不通知，消费者可能永久睡眠。若 push 在关闭后仍然成功，析构时可能有任务泄漏。若 pop 只等待非空，不考虑 closed，关闭空队列会死锁。每个错误都能用测试复现。

Semaphore 适合表达资源计数，例如空槽数量、可用连接数、in-flight 请求预算。它不直接保护复杂不变量，通常还需要 mutex 保护队列本身。把 semaphore 当作 mutex 替代品会混乱。章节要比较：mutex 保护状态一致性，condition variable 等待状态变化，semaphore 控制数量资源。

背压和关闭不是异常路径

有界队列把容量变成系统契约。队列满时，push 可以阻塞、返回 false、超时失败或丢弃低优先级项。选择取决于上层语义。必须执行的任务不能静默丢弃；可丢指标可以采用 drop；服务请求可能需要 deadline。背压不是系统失败，而是把下游压力传给上游，防止内存无限增长。

关闭要有状态机。Open 接受 push 和 pop；Closing 拒绝新 push，但允许消费者 drain；Closed 表示队列空且不再有元素。不同项目可以简化，但必须写清。线程池 shutdown、I/O drain、分布式 driver 停止发送任务都依赖类似状态。如果关闭只是一个 bool，很多边界会变成猜测。

异常安全也要覆盖。若元素移动构造抛异常，队列 size 是否已经改变？若任务执行抛异常，未完成计数是否递减？若 shutdown 在等待者阻塞时调用，是否唤醒所有人？教学代码可以选择要求任务类型 noexcept move，但要求必须写进接口合同。

实验、源码和项目落地

本章实验围绕 bounded queue。正常输入验证生产消费；边界输入测试容量 1、空队列关闭、满队列关闭；压力输入测试多生产者多消费者；错误输入测试任务抛异常和超时 push。指标包括 push 等待次数、pop 等待次数、最大队列长度、关闭耗时和丢弃/失败数量。

Linux 源码阅读可以从 futex 概念入手。用户态 mutex/condition variable 在低争用时可能不进内核，高争用或睡眠时才进入 futex 等待队列。读者不必实现 futex，但要理解“锁慢”可能慢在用户态 cache line 自旋，也可能慢在内核睡眠唤醒。工具上可以用 `strace-f-efutex`、`perf` 或用用户态指标观察等待路径。

项目交付物是可复用的 `BoundedQueue<T>` 设计说明和测试。说明要列出不变量、状态机、等待谓词、关闭语义、背压策略和异常策略。后续线程池、I/O 和 MessageBus 都使用它，这样全书同步语义保持一致。

案例走查：有界队列 close 的四个错误版本

错误版本一：只设置 closed，不通知等待者。消费者可能永远睡眠。错误版本二：通知了等待者，但 pop 谓词只检查非空，不检查 closed，仍可能睡眠。错误版本三：关闭后 push 仍然成功，shutdown 后任务泄漏。错误版本四：析构时直接销毁队列，仍有 producer 或 consumer 访问，造成悬垂。

正确版本必须让等待谓词包含状态：push 等待“未满且 open”或发现 closed 失败；pop 等待“非空或 closed”；close 改状态后 notify all；析构要先 close，再等待外部线程结束或由更高层保证生命周期。每个错误版本都写成测试，比直接给正确代码更能说明条件变量为什么等待谓词。

这个案例还连接背压。容量为 1 的队列能暴露大多数等待交错，容量很大反而掩盖问题。教学测试要故意使用小容量和随机调度，让不变量问题尽早出现。

锁实验的公平性和尾延迟

锁实验不能只看总吞吐。一个锁方案可能平均吞吐高，但让某些线程长期饥饿；另一个方案吞吐略低，但尾延迟稳定。Bounded queue 实验应记录每个 producer 的成功次数、等待时间分布、消费者空闲时间和关闭耗时。公平性是系统可用性的一部分。

不同锁实现和调度器会影响结果。自旋锁在短临界区和低线程数下可能快，在过度订阅时会浪费 CPU；mutex 会睡眠和唤醒，尾延迟可能受调度影响；读写锁在读多写少时有效，但写者饥饿或升级问题要考虑。第二册不需要实现所有锁，但要让读者知道没有一个同步原语固定最优。

实验还要加入异常路径。生产者在持锁前抛异常、持锁后移动元素抛异常、消费者处理任务抛异常、关闭同时发生。RAII 和状态机能把这些路径收束起来。若实验只跑正常 push/pop，队列看起来正确，实际不可靠。

从有界队列迁移到线程池

线程池复用 bounded queue，但新增未完成计数和 worker 生命周期。这里要提醒读者：队列为空不等于线程池 idle，因为 worker 可能正在执行任务；closed 队列不等于所有任务完成，因为队列中任务可能还要 drain。未完成计数把 queued 和 running 统一起来，等待谓词应是 count 为零。

这个迁移能训练抽象复用。Bounded queue 不应该知道线程池的任务计数；线程池在 submit 成功后增加计数，在任务完成后减少。若 task 抛异常，也必须减少。这样两个模块边界清晰。后续任务运行时会继续扩展，而不是把所有逻辑塞回队列。

本章验收可以要求读者解释每个 notify 的必要性：push 后通知消费者，pop 后通知生产者，close 后通知所有等待者，task 完成后通知 wait_idle。能解释 notify，才说明谓词和状态真的理解了，也说明等待测试不是摆设。

综合练习：写出队列不变量审查表

本章综合题要求读者为 BoundedQueue<T> 写不变量审查表。容量不变量：size 在 0 和 capacity 之间。位置不变量：head、tail 和 size 对应同一组元素。状态不变量：Open 接受 push，Closing 拒绝 push 但允许 pop，Closed 表示不可再变化。等待不变量：生产者等待空间或关闭，消费者等待元素或关闭。生命周期不变量：队列析构时没有外部线程继续访问。

审查表写完后，再要求读者审三段错误代码。第一段 push 在检查 closed 后释放锁，再写入元素，状态可能变化；第二段 pop 等待非空，没有处理 close；第三段 close 只 notify one，多个等待者仍睡眠。读者要指出违反哪个不变量，而不是只说“这里有 bug”。这种训练能把并发 bug 从感觉变成可定位问题。

最后实现压力测试。多个 producer 和 consumer 随机 sleep，容量取 1、2、7，随机在运行中 close。每个元素有唯一 id，最终检查不丢不重；所有线程能退出；等待者不会永久阻塞。测试还要跑异常任务，确认未完成计数不泄漏。这个题完成后，线程池和 I/O 队列才有可靠基础。

评分标准包括文档和测试。只有代码没有不变量说明，不合格；只有说明没有压力测试，也不合格。同步原语的教学重点就是把不变量、代码和测试绑在一起。

容易出错的边界：同步代码最怕半语义

半语义的第一个表现是只有锁，没有不变量。代码到处 lock_guard，却没人知道锁保护哪几个字段。第二个表现是只有通知，没有谓词。看到 notify_one 就以为事件不会丢，忽视先通知后等待和虚假唤醒。第三个表现是只有 close flag，没有状态机。关闭时生产者、消费者和析构者各自猜行为，最终出现死锁或丢任务。

第四个表现是异常路径没有计数回收。任务抛异常后未完成计数不减，wait 永久阻塞。第五个表现是锁内调用用户回调。用户回调可能再次进入队列、等待 I/O 或抛异常，让锁语义失控。第六个表现是多个锁无顺序。小测试不死锁，压力下出现 ABBA。

本章源码索引要求读者为每把 mutex 写注释：保护哪些字段，禁止在持锁时做什么，和其他锁的顺序是什么。为每个 condition variable 写等待谓词和通知来源。为每个 semaphore 写资源含义和最大值。若写不出来，就说明同步设计还没完成。

工程上可以接受少量重复注释，因为并发边界需要显式。不要写“保护数据”这种空话，要写“保护 queue、closed、unfinished 三个字段的一致性”。这种注释能让后续审查真正发现 bug。

本章核心接口：QueueContract

锁章给项目留下 `QueueContract`。它不是代码类，而是每个队列必须填写的合同：容量，push 满时行为，pop 空时行为，close 后行为，是否 drain，是否支持 timeout，元素所有权如何转移，等待谓词是什么。不同队列可以有不同策略，但必须显式。

`BoundedQueue`、`I/O queue`、`MessageBus queue` 都应写这个合同。这样队列满、关闭和取消不会在不同模块各自解释。合同统一后，背压才能沿 pipeline 传播，而不是在某个层次被吞掉。

从锁保护不变量进入原子发布协议

锁章给出最清楚的同步模型：在临界区内维护不变量，条件变量等待谓词。下一章进入原子，是为了处理一些更窄的同步场景：发布只读配置、统计计数、槽位状态、一次性提交。原子不是替代锁，而是当状态足够小、协议足够清楚时，用更细粒度的方式表达可见性和线性化点。

项目中每个原子优化都应有锁版 reference。`BoundedQueue` 先用锁实现，`SPSC` 槽位再用原子发布；commit 状态先用 `mutex` 表达，再用 `atomic enum` 表达单进程状态转换。这个顺序能让读者看到原子协议从哪里来，而不是凭空写内存序。

10.6 项目落地

`Compute Systems Engine` 当前的 `BoundedMpmcQueue` 已经有容量、head、tail、size、mutex 和 `not_full`，但它还不是完整线程池队列，因为它只有阻塞 push 和非阻塞 try pop，没有 `not_empty` 等待，也没有 close。作为教学阶段的最小模块可以接受，但后续运行时章节必须补全关闭语义和消费者等待，否则 worker 无法优雅阻塞和退出。

本章要求项目至少增加设计文档或测试计划：列出队列不变量，列出 push、try pop、未来 blocking pop 和 close 的状态转换，列出需要的测试。测试不能只 push 两个值再 pop 两个值，还要覆盖容量为一、队列满时生产者等待、队列空时消费者等待、close 唤醒等待者、多生产者多消费者压力、异常容量和析构时没有等待线程。

状态机比代码片段更重要 同步代码最怕只看局部片段。一个 push 函数看起来正确，一个 pop 函数看起来正确，合在一起仍可能在关闭、异常、等待和析构时出错。因此并发容器应先写状态机。以有界队列为例，状态至少包括 open empty、open partial、open full、closing nonempty、closed empty。每个状态下，push、try_push、pop、try_pop、close、wait_empty 的行为都要定义。

状态机能暴露隐藏问题。open full 状态下，阻塞 push 等待什么谓词？如果此时调用 close，生产者应返回失败还是继续等待？closing nonempty 状态下，消费者能否继续取出剩余任务？closed empty 状态下，pop 返回空值还是抛异常？这些问题不回答，代码只能靠偶然行为。一个高质量并发库的接口文档，应像协议一样说明状态转换。

状态机还要说明线性化点。对于 push，元素写入槽位并更新 tail 和 size 后，操作对消费者可见；对于 pop，元素被取出并更新 head 和 size 后，槽位重新可用；对于 close，closed_ 从 false 变为 true 的那一刻，后续生产者必须观察到关闭。线性化点不清楚，就无法判断并发调用的先后语义。

背压不是错误路径 有界队列满了，生产者等待，这不是程序变差，而是背压在工作。背压的目的，是让下游资源不足时，上游不要无限制地产生任务和占用内存。没有背压的系统，短时间 benchmark 可能看起来更快，因为它把工作全部塞进内存；一旦输入持续增长，就会出现内存膨胀、延迟暴涨和最终崩溃。有界队列用等待或拒绝把压力反馈给上游。

背压策略要和业务语义匹配。日志系统可以选择丢弃低优先级调试日志；任务运行时通常不能丢计算任务；网络服务可以对新请求返回繁忙；分布式 shuffle 可以暂停上游分片读取。队列只提供机制，业务决定策略。教材中如果只写“队列满则阻塞”，还不够完整；还要讨论阻塞会不会占住 compute worker，是否需要异步等待，是否需要超时，是否需要取消。

背压指标也要写入报告。队列满的次数、生产者等待总时长、最大队列长度、丢弃任务数、拒绝请求数，这些指标能说明系统是在健康限流，还是下游长期跟不上。没有指标，开发者可能把背压误认为锁慢，然后盲目增加容量。容量变大只会把压力藏得更深，尾延迟和内存风险更高。

锁和异常传播 线程池中的任务可能抛异常。异常传播如果处理不好，会破坏锁保护的不变量。最基本原则是：任务执行不应发生在队列锁内，任务异常不应让未完成计数漏减，异常对象的发布要有同步关系。队列只负责交付任务，worker 从队列取出任务后释放锁，再执行任务。执行结束后，不管成功还是异常，都要更新任务状态和未完成计数，并通知等待者。

C++ 的 RAII 能帮助释放锁，但不能自动维护业务状态。若一个任务执行前增加了 in-flight 计数，异常路径必须减少；若 worker 把错误写入 shared state，等待者读取错误前要有同步；若异常对象包含动态分配资源，生命周期要覆盖读取者。并发异常不是“catch 一下”这么简单，它和状态机、通知和对象生命周期同时相关。

一个实用模式是把任务结果分成状态和载荷。状态可以是 Pending、Running、Succeeded、Failed、Cancelled；载荷是返回值或错误信息。worker 先写载荷，再在锁内或 release 语义下发布状态，然后通知等待者。等待者醒来后检查状态，再读取载荷。这个模式和原子章节的对象发布一致，只是这里多了一层任务语义。

测试等待路径要制造极端条件 同步代码的测试不能只跑正常路径。容量为一的队列比容量很大的队列更容易暴露满和空之间的状态转换；一个 producer 一个 consumer 能测基本交替；多个 producer 一个慢 consumer 能测 `not_full`；一个 producer 多个 consumer 能测 `not_empty` 和惊群；关闭时有线程阻塞在 push 和 pop，能测 `notify_all` 是否完整。小容量、慢消费者、随机 yield、重复关闭和异常任务，是并发测试的基本工具。

测试还应覆盖析构边界。线程池析构时是否自动 shutdown？如果还有外部线程提交任务，行为是什么？析构能否在 worker 正在执行任务时等待？如果析构发生在某个任务持有业务锁时，会不会死锁？这些问题不一定都由队列解决，但队列和线程池接口必须定义。未定义行为可以存在，但必须明确禁止，而不是让用户猜。

压力测试不能替代不变量证明，但能发现实现错误。可以把队列操作与串行 reference 对比：多个线程随机 push 和 pop，同时记录成功操作日志，最终验证元素没有丢失、没有重复、容量不越界、关闭后没有新元素进入。对于不可确定顺序的并发队列，验证重点不是全局顺序完全一致，而是每个成功 push 最多被 pop 一次，关闭语义符合文档。

不变量 队列不变量包括容量范围、head/tail 关系、size 与槽位状态、closed 后禁止新 push、drain 后没有剩余元素。锁保护的不仅是代码块，而是这些关系。

RAII 锁 C++ 中锁必须和作用域绑定。异常路径、早返回和条件分支都不能绕过 unlock。RAII 不是风格问题，而是维护不变量的基本手段。

谓词等待 `condition_variable` 的 wait 必须围绕谓词写。虚假唤醒、notify 丢失和状态竞争都要求醒来后重新检查条件。谓词应包含 closed 状态，否则关闭时线程可能永远睡眠。

notify 顺序 通常先在锁内修改状态，再 notify。通知不是状态本身，只是提醒等待者重新检查谓词。把 notify 当成消息，会导致醒来时条件并不成立。

容量令牌 有界容量是背压机制。生产者等待 `not_full`，不是性能退化，而是系统告诉上游下游处理不过来。容量设计要结合内存预算和延迟目标。

close 和 drain close 表示不再接受新输入，drain 表示处理完已接受输入。二者不能混成一个布尔值。push、pop、close、drain 在每个状态下的返回都要写清。

锁顺序 多个锁需要全局顺序。反向拿锁的死锁不是概率 bug，而是设计缺陷。能用一个更高层状态机避免双锁，就不要让调用者临时组合。

futex 直觉 用户态 mutex 快路径通常不进内核，竞争严重时才通过 futex 睡眠。理解这点能解释为什么低竞争锁很便宜，高竞争锁会出现调度和唤醒成本。

案例：容量为一的队列 容量为一最容易暴露 empty/full 转换。一个 producer、一个 consumer、随机 close，能检查谓词和 notify 是否完整。

案例：关闭时阻塞线程 让 producer 阻塞在满队列，consumer 阻塞在空队列，然后调用 close。所有线程都应有定义明确的返回路径。

案例：反向锁顺序 构造两个队列相互转移元素，故意反向拿锁，再用审查表修复。这个案例说明死锁来自状态组合，而不是线程运气。

源码阅读路线 Linux 源码阅读从 `kernel/futex/core.c`、`kernel/locking/mutex.c`、`include/linux/wait.h` 附近进入即可。阅读目标不是把整套子系统抄进书里，而是追一条和本章实验相关的路径：用户态动作怎样变成内核对象，内核对象怎样排队、等待、唤醒、计数、提交或回收。

阅读报告要写证据等级。能直接测到的数字放在第一层；从源码控制流推导出的解释放在第二层；当前设备无法验证、只能作为合理猜测的内容放在第三层。这样读者会知道同步状态机报告中哪些结论可复现，哪些结论需要换机器继续确认。

源码阅读还要克制。看到一个复杂函数，不要把每个分支都展开；先回到本章的问题，问它能否解释一个等待点、一次状态转移或一条性能曲线。解释不了的内容可以留到索引，不要打断正文主线。

容量为一的队列 容量为一最容易暴露 empty/full 转换。一个 producer、一个 consumer、随机 close，能检查谓词和 notify 是否完整。

关闭时阻塞线程 让 producer 阻塞在满队列，consumer 阻塞在空队列，然后调用 close。所有线程都应有定义明确的返回路径。

反向锁顺序 构造两个队列相互转移元素，故意反向拿锁，再用审查表修复。这个案例说明死锁来自状态组合，而不是线程运气。

不变量

锁保护的不仅是代码块，而是一组状态关系。队列的 head、tail、size、closed 和 buffer 必须一起满足约束。

实验设计：先写不变量表，再写 push 和 pop 的状态转换。

临界区

临界区越大，等待越多；临界区越小，越容易把不变量拆碎。边界应由状态关系决定，而不是只追求短。

实验设计：把队列操作拆得过细，构造 size 与 buffer 不一致的错误。

谓词等待

条件变量等待的是谓词，不是通知次数。虚假唤醒和丢通知都要求醒来后重新检查状态。

实验设计：把 wait 写成 if，再用 spurious wake 或关闭路径展示问题。

notify 顺序

通知只是提醒等待者检查状态。通常先改状态，再通知。若通知和状态更新脱离，醒来的线程可能看不到条件成立。

实验设计：记录生产者修改 size、释放锁和 notify 的顺序。

关闭语义

队列关闭后，push 应失败或返回状态，pop 应能取完已有元素后结束。没有关闭语义，消费者可能永远睡眠。

实验设计：构造 producer 提前退出，consumer 仍在等待的案例。

容量令牌

有界容量不是性能退化，而是背压。容量把内存预算和上游速度控制在可解释范围内。

实验设计：让生产者快于消费者，比较无界队列和有界队列的内存峰值。

信号量

信号量适合表达资源数量，但它不自动保护复杂不变量。把它和 mutex 混用时，要明确谁保护什么。

实验设计：用信号量限制 in-flight 请求，再用锁保护队列结构。

死锁

多个锁必须有顺序。反向拿锁不是概率问题，而是状态图有环。设计上能合并状态机就不要让调用者临时组合。

实验设计：画出两个锁的等待图，说明环在哪里出现。

futex 直觉

用户态锁低竞争时通常不进内核，竞争或等待时才通过 futex 睡眠。锁成本随竞争形态变化。

实验设计：比较低竞争锁、高竞争锁和条件等待的系统调用迹象。

项目接口

QueueContract 要写清容量、关闭、等待谓词、异常、push 失败和 pop 结束条件。后续线程池和 I/O 背压都依赖它。

实验设计：为每个状态转换写测试，不只测 happy path。

10.7 同步原语从不变量和等待谓词推导出来

锁章节如果只讲 API，用不了多久读者就会写出看似能跑但偶尔死锁、偶尔丢任务的代码。正确顺序是先写不变量，再选择原语。以有界队列为例，不变量包括：元素只存在于有效槽位，size 等于可取元素数量，head 和 tail 在容量范围内，closed 后不再接受新元素，等待者醒来后必须能重新判断状态。mutex 保护的是这些关系，不是某几行代码。

条件变量等待的是谓词。很多错误来自把 notify 当成消息：生产者 notify 了，消费者就以为一定有数据。实际中可能虚假唤醒，可能多个消费者争抢同一个元素，也可能 notify 发生在等待之前。正确写法是醒来后重新检查谓词，谓词包含数据可用、队列关闭、容量可用等状态。若关闭状态不在谓词里，消费者可能在生产者退出后永远睡眠。

notify 的顺序也要从状态推导。通常先在锁内修改状态，再 notify；通知只负责让等待者重新检查，不承载状态本身。若先通知再修改，或者修改的状态不受同一把锁保护，等待者醒来时可能看不到条件成立。这里要让读者明白：条件变量和 mutex 是一套协议，拆开理解会误用。

容量是背压，不是麻烦。有界队列让上游知道下游处理不过来，避免内存无限增长。容量太小会降低吞吐，容量太大会隐藏过载并增加尾延迟。队列容量应来自内存预算、batch 大小和延迟目标。比如每个 batch 4 MiB，容量 128 就是 512 MiB，仅这一条队列就可能超过手机环境预算。同步章节要把容量和资源预算连起来。

信号量适合表达资源数量，例如允许同时 in-flight 的请求数。但信号量不保护复杂结构，队列的 head、tail 和 buffer 仍需要不变量保护。把 semaphore、mutex 和 condition variable 混在一起时，必须说明每个原语负责什么。否则容易出现令牌归还了但队列状态没更新，或者队列关闭后令牌仍让 producer 进入的错误。

死锁要用等待图讲。两个锁 A 和 B，如果路径一先拿 A 再拿 B，路径二先拿 B 再拿 A，就形成环。这个 bug 不靠概率解释，而是设计缺陷。能用更高层状态机避免双锁，就不要让调用者自由组合；必须多锁时，就定义全局顺序，并在代码审查里检查。

Linux futex 可以作为直觉补充：低竞争 mutex 通常在用户态完成，竞争时才进入内核睡眠和唤醒。于是锁成本不是固定的，取决于竞争、临界区长度和唤醒形态。实验可以比较无竞争锁、高竞争锁和条件变量等待，记录吞吐、上下文切换或替代指标。读者会看到：锁不是绝对慢，错误的共享状态才慢。

案例推导：有界队列的正确性来自状态表，不来自锁的数量

有界队列是同步原语最好的综合案例。它有数据结构不变量：容量固定，head 指向下一个可取元素，tail 指向下一个可写位置，size 在零和容量之间。它有等待谓词：非空才能 pop，非满才能 push，关闭后等待者要退出。它还有背压语义：满不是异常，而是系统告诉上游下游处理不过来。用一把 mutex 保护这些关系是实现选择，不变量才是设计核心。

先写错误版本：**pop** 里用 if 判断空，然后 wait；醒来后直接取元素。这个版本在没有虚假唤醒、只有一个消费者时可能跑。加入两个消费者或虚假唤醒后，一个消费者可能在条件不成立时继续执行。修复方式是 while 或带谓词的 wait。教材要强调：wait 返回只表示“应该重新检查”，不表示条件已经成立。

再看关闭。生产者结束后，如果队列为空，消费者应返回结束状态；如果队列还有元素，消费者应取完后再结束；关闭后 push 应失败。这个语义要写成状态表：open 且 not full 可以 push，closed 不可 push；not empty 可以 pop，empty 且 closed 返回结束，empty 且 open 等待。没有这张表，代码很容易在某条路径漏 notify 或漏检查 closed。

容量设计也要进入报告。容量一暴露交替等待问题；容量很大能隐藏背压并增加内存峰值；容量按 batch 字节数设置更贴近 I/O 场景。实验可以让 producer 速度是 consumer 两倍，比较无界队列、有界阻塞队列和 try push 返回失败。观察内容包括吞吐、内存、平均等待和尾延迟。这样读者会明白有界队列不是为了“慢一点安全”，而是为了让系统可控。

信号量版本可以作为对照。一个 semaphore 表示可用槽位，一个 semaphore 表示可用元素，mutex 保护环形数组。这个设计能让资源计数更显式，但状态更多，关闭语义也更复杂。若关闭时只释放元素信号，不释放槽位等待者，producer 可能卡住；若随意释放信号，又可能破坏计数。对照能说明原语没有银弹，状态设计才是关键。

Linux futex 阅读可以帮助理解为什么低竞争时便宜、高竞争时昂贵。用户态先尝试原子修改，失败或需要等待时进入内核睡眠。条件变量也会围绕 futex 或类似机制等待唤醒。读源码不为背 API，而是解释实验现象：低竞争锁几乎不进内核，高竞争等待出现调度和唤醒成本。

常见误区

不要把锁竞争问题简单理解成“换成无锁”。如果临界区保护的是复杂不变量，无锁实现可能更难证明正确，也可能在高竞争下 CAS 失败更多。

实验：实现一个 bounded queue。先用 mutex 和 condition variable 写阻塞 push/pop，再记录高生产者数量下的等待时间。解释锁保护的状态和等待谓词。

锁保护的是不变量，不是代码块 讲锁不能从“进入临界区”开始，而要从共享对象的不变量开始。有界队列的不变量包括：head、tail、size 与 buffer 内容一致；size 不超过 capacity；队列关闭后不再接受新元素；等待者醒来后看到的状态必须能解释为什么醒来。互斥锁的作用是让检查和修改这些状态成为一个不可被并发打断的动作。若不变量没写清，锁范围就只能靠感觉。

以有界队列为例，push 必须在同一把锁下检查未关闭、检查未滿、写入元素、更新 tail 和 size。pop 必须检查非空或关闭，读取元素，更新 head 和 size。notify 不是凭空发送消息，而是在状态变化后唤醒可能等待该状态的线程。条件变量等待的也是谓词：not full、not empty 或 closed。把这些谓词写清，丢唤醒和虚假唤醒就不再神秘。

条件变量的正确模式从时间线推导 丢唤醒常发生在把通知当事件，而不是把状态当事实。生产者先修改队列状态再 notify；消费者等待时在锁下检查谓词，不满足才 sleep；醒来后重新检查谓词。即使 notify 发生在消费者真正睡眠之前，只要状态已经改变，消费者重新检查时就会继续。若代码只等待一次通知，不检查状态，就会把时间线偶然性当语义。

虚假唤醒也因此容易处理。等待循环不是为了防某个实现缺陷，而是因为醒来不等于谓词成立。多个消费者竞争同一个元素时，一个线程醒来后可能发现元素已被别人取走；关闭时所有等待者都醒来，但有的看到队列空且 closed，有的先取完剩余元素。while predicate 是条件变量的核心写法。

信号量表达资源数量 信号量适合表达可用资源数量，例如缓冲槽、连接许可、并发上传数。它不直接保护复杂不变量，通常和其他状态结构配合。比如有界队列可以用两个计数信号量分别表示空槽和已用槽，再用互斥保护 ring buffer 索引。这样 push 先获取空槽，再写 buffer，再释放已用槽；pop 反过来。它把容量约束拆成资源计数，但关闭、异常和取消仍需要额外状态。

因此不要把信号量当作“更快的条件变量”。它们表达模型不同。条件变量等待任意谓词，信号量等待资源计数。若谓词是复杂组合，如队列非空或关闭或超时，条件变量更直接；若只是限制同时执行的任务数，信号量更自然。教材要让读者按不变量选择原语。

死锁从资源顺序解释 死锁的四个条件可以通过项目案例具体化。线程 A 持有队列锁等待 manifest 锁，线程 B 持有 manifest 锁等待队列锁；两者都不释放，资源不可抢占，形成环。修复方式不是“少用锁”这么空泛，而是定义全局锁顺序、缩短锁持有时间、避免持锁调用外部回调、把等待移到锁外、使用 try lock 加回退。每种方式都要说明改变了哪个条件。

锁顺序表是工程文档的一部分。比如 RuntimeState 锁先于 WorkerQueue 锁，WorkerQueue 锁不能在持有 Manifest 锁时获取，用户回调永远在无锁状态调用。代码审查时检查这些规则，比看到死锁后再排查容易得多。

实验：写一个可关闭有界队列 本章实验应要求完整实现有界队列，而不是只写 push pop。功能包括阻塞 push、非阻塞 try push、阻塞 pop、关闭、等待 drain、超时版本和异常安全。测试要覆盖：空队列 pop 后关闭，满队列 push 后关闭，多生产者多消费者，任务函数抛异常，析构时仍有等待者，重复 close。每个测试都要验证没有线程永久等待。

性能实验比较容量大小、生产消费速度差异和通知策略。容量太小会增加等待，容量太大会隐藏背压并增加内存；notify one 通常够用，但关闭时需要 notify all；持锁时间过长会降低并发。这个队列会被第 14、15 章复用，所以语义必须比性能技巧更先稳定。

Linux futex 阅读连接点 条件变量和互斥锁在 Linux 上常会借助 futex。Futex 的思想是用用户态先用原子操作处理无竞争快路径，只有需要睡眠或唤醒时进入内核。源码阅读时关注用户态原子值和内核等待队列如何对应：等待前必须确认值仍然是预期状态，唤醒时唤醒等待该地址的线程。这个机制再次说明等待的是状态，不是单纯通知。

把 futex 思想带回项目，可以理解为什么锁的无竞争路径很重要，也能理解为什么把大量短任务压到一个共享锁上会放大内核等待成本。同步原语不是黑盒，它们在用户态状态和内核调度之间架桥。

事故复盘：关闭队列为什会卡死 有界队列的正常 push pop 很容易写对，关闭路径却经常卡死。Producer 在队列满时等待，主线程设置 closed 后只唤醒 consumer；consumer 醒来发现队列空但 closed 判断写错，又继续等待；或者 notify 被当成“消息”，醒来后不重新检查谓词。此时锁不是保护一段代码，而是保护队列不变量：buffer 内容、容量、关闭状态、等待条件必须同时一致。

条件变量等待的是谓词，不是通知本身。Producer 的谓词是“队列未滿或已关闭”，consumer 的谓词是“队列非空或已关闭”。任何 wait 返回后都要在锁下重新检查谓词，因为可能有虚假唤醒，也可能被其他线程抢先改变状态。关闭状态进入状态机后，push 应返回拒绝，pop 应能 drain 剩余元素并最终返回结束。Notify 只是提醒对方重新看状态，不携带业务语义。

实验要专门打关闭和异常路径。满队列关闭、空队列关闭、producer 正在等待时关闭、consumer 正在等待时关闭、多个等待者同时关闭、回调抛异常、持锁调用用户函数，这些都比正常路径更能暴露设计。资源顺序也要画出来，避免一个线程拿队列锁再拿日志锁，另一个线程反过来拿导致死锁。同步章节的核心是用不变量和谓词解释代码，而不是用 mutex 包住一切。

同步测试实验判读 有界队列测试最重要的是关闭和异常路径。若正常 push pop 通过，不代表队列正确。判读时检查等待者能否在 close 后全部醒来，满队列的 producer 是否能退出，空队列的 consumer 是否能退出，异常是否泄漏 unfinished 计数。性能结果也要和容量、水位线和持锁时间一起看。

这一段也服务于复习。读者合上书后，应该能凭本章的判读口径重新审查自己的代码：有没有最小正确版本，有没有能击穿旧方案的输入，有没有状态记录，有没有资源指标，有没有失败后的恢复动作。若五个问题缺两个以上，就说明实现还停在演示层，而不是工程层。

关闭路径比正常路径更能检验同步设计 有界队列正常 push 和 pop 很容易写通，真正检验设计的是关闭。队列为空时 consumer 等待，close 后应醒来并返回 closed；队列满时 producer 等待，close 后也应醒来并失败；队列中还有元素时，drain close 可以允许 consumer 取完，immediate close 则要定义剩余元素如何处理。

条件变量代码的核心是谓词。等待 not empty、not full、closed 这些状态，而不是等待某个 notify。notify 可以丢，虚假唤醒可以发生，多个线程醒来也会竞争同一个状态。只要谓词写对，这些都不会破坏正确性；谓词写错，再多 notify 也只是碰运气。

信号量版本可以作为对照。它把空槽和已用槽表达成资源数量，但关闭语义仍然需要额外状态。这个对照能让读者理解：同步原语不是越高级越好，而是表达模型是否贴合不变量。

实验课：可关闭有界队列的状态表 先画状态表再写代码。队列有 open、closing、closed 三种状态；buffer 有 empty、partial、full 三种容量状态；等待者分 producer 和 consumer。push 等待 not full 或 closed；pop 等待 not empty 或 closed；close 修改状态并唤醒所有等待者。状态表写清后，代码只是把表翻译成 mutex、condition variable 和谓词循环。

测试分六组。空队列上 consumer 等待后 close；满队列上 producer 等待后 close；多个 producer 和 consumer 同时运行；任务处理抛异常；drain close 允许取完剩余元素；immediate close 直接拒绝后续操作。每组都要有超时保护，防止测试本身永久挂住。若任何等待者没有退出，说明谓词或通知顺序有问题。

性能实验再看容量。容量太小，生产者和消费者频繁等待；容量太大，背压被隐藏，内存峰值上升。报告不要只写吞吐，还要写最大水位、平均等待、notify 次数或替代指标。这样队列不只是同步练习，也是 I/O 和任务运行时背压的基础。

信号量对照可以放在最后。用 empty slots 和 used slots 两个信号量实现队列，读者会看到资源计数更直接，但关闭语义仍然需要额外状态。这个对照能加深“原语服务模型”的理解。

同步原语练习验收重点 合格答案从不变量开始。队列 head、tail、size、buffer 和 closed 状态之间的关系要写清。条件变量题必须写等待谓词，并说明虚假唤醒和丢通知为何不破坏正确性。信号量题要说明它表达资源数量，而不是替代所有状态。

答案还要写“为什么不采用更复杂方案”。例如没有证据证明队列是瓶颈，就不应把锁队列换成无锁；没有证明输出顺序可变，就不应随意重排；没有证明错误路径能恢复，就不应只提交正常路径。能解释不做什么，说明读者开始具备工程判断。

最后，答案必须包含可复查材料。至少有输入摘要、关键状态、期望输出、实际输出、运行命令和未验证边界。没有这些材料，老师或同伴无法判断结论是否成立。第二册的作业不是让读者写漂亮段落，而是训练读者把系统问题变成可验证证据。

项目实施：锁、条件变量与信号量 本章对应的贯穿项目实施不要求一次写成完整框架，但必须有最小可运行切片。这个切片包含三个部分：一个 reference，一个带本章机制的实现，一个能击穿 reference 之外隐含假设的测试。Reference 用来固定语义，机制实现用来展示本章知识，测试用来证明新增复杂度确实解决了问题。

代码组织上，不要把所有内容放进一个大文件。基础类型、输入生成、核心机制、报告输出和测试应分开。即使当前只是教学项目，也要让目录能表达责任边界。这样后续章节接入时，不会因为找不到状态归属而重新发明一套对象。

每个实现都要输出一段机器可读摘要。摘要至少包含输入规模、关键状态、错误数、输出校验、耗时和未验证边界。这里的机器可读不要求复杂格式，简单的 key value 文本也可以。关键是让脚本和读者都能复查，而不是只能看屏幕截图。

测试应包含正常路径、边界路径和失败路径。正常路径证明功能可用；边界路径证明长度、容量、尾部、空输入、关闭或超时没有遗漏；失败路径证明本章机制不是装饰。若失败路径写不出来，往往说明本章概念还没有落到工程对象上。

报告最后写一个取舍段。新增机制带来了什么好处，增加了什么成本，哪些场景不应该使用它，下一章会复用哪个字段或对象。这个取舍段非常重要，因为系统工程不是把所有高级机制都加进去，而是在证据和约束下选择合适的边界。

调试笔记：死锁和卡住先写等待图 程序卡住时，先写每个线程持有哪些锁、等待哪个谓词、队列状态是什么、关闭状态是什么。若两个线程互等资源，是锁顺序问题；若所有消费者等待 not empty 但队列已 closed，是谓词没有包含关闭状态；若 producer 等 not full 但关闭后没醒，是通知路径缺失。等待图比猜测哪把锁慢更有效。

复查清单：锁、条件变量与信号量 复查本章时，先检查 reference 是否仍然存在。没有 reference，后面的并发、优化、异步或分布式改造都无法证明语义没有变。再检查压力输入是否尖锐：它应当攻击本章最核心的边界，而不是只把数据量放大。第三，检查状态是否可观察：关键对象的身份、状态、错误和提交结果应能在报告中找到。

第四，检查代码示例是否说明禁止行为。调用者不能做什么，往往比能做什么更能体现工程质量。第五，检查实验是否写出未验证边界；当前设备、权限或输入无法证明的结论，必须标注。第六，检查本章对象是否能被下一章复用，不能复用就说明边界还不稳。

完成这六项后，再看文字是否连贯。每个概念都应从问题推出，每个案例都应从朴素方案走到改进方案，每个实验都应能解释一个判断。若某段话放到其他章节也成立，就把它改成本章的具体对象、具体输入和具体失败。

10.8 工程案例：从有界队列推导锁、条件变量和背压

锁和条件变量最好的教学对象是有界队列。它同时包含共享状态、不变量、等待谓词、关闭语义、背压和异常安全。若这个队列讲不清，后面线程池、I/O pipeline 和分布式背压都很难讲清。不要把 mutex 当作“保护代码块”的工具，它保护的是状态不变量。

一、先写队列不变量 有界队列至少有这些状态：buffer、head、tail、size、capacity、closed。它的不变量包括：size 不超过 capacity；head 和 tail 始终落在环形范围；size 为 0 时不能 pop；size 等于 capacity 时不能 push；closed 后不再接纳新元素；closed 且 size 为 0 时消费者应退出。这些不变量必须由同一把锁保护，不能有的字段锁内改，有的字段锁外原子读写。

写锁代码前先写不变量，能防止很多错误。比如在锁外读 size 判断是否 pop，看起来减少锁开销，实际上破坏了 head、tail、size 的一致关系。size 是协议状态，不是近似指标。若只是监控队列长度，可以维护另一个 relaxed 指标；但实际 pop 决策必须在锁保护下完成。

二、条件变量等待的是谓词，不是通知 条件变量的通知可能早到、晚到、合并或虚假唤醒。等待者必须在循环里检查谓词。生产者 push 后通知 not empty；消费者 pop 后通知 not full；关闭时同时通知所有等待者。若等待者只等通知，不检查 closed 和 size，就会出现丢唤醒或关闭悬挂。

Listing 10.8 有界队列的等待谓词要包含关闭状态

```
1 bool can_pop(std::size_t size, bool closed) {
2     return size > 0 || closed;
3 }
```

```

4
5 bool can_push(std::size_t size, std::size_t capacity, bool closed) {
6     return size < capacity || closed;
7 }

```

这两个函数看似简单，却表达了核心语义。pop 等到有元素或关闭；push 等到有空间或关闭。等待返回后还要根据 closed 和 size 决定成功、失败或退出。谓词写清，条件变量代码才可审查。

三、关闭语义要区分 drain 和 cancel 关闭队列有两种常见语义。Drain 表示不接收新元素，但已有元素允许消费者取完；Cancel 表示队列中未处理元素被丢弃或标记取消。两者对生产者、消费者和等待者的返回值不同。很多 bug 来自只设置 `closed=true`，却不说明剩余元素怎么办。

Compute Systems Engine 的 parse-to-compute 队列通常适合 drain：上游不再产生新 batch，下游处理已有 batch 后退出。错误恢复或作业取消可能需要 cancel：未开始的 batch 被标记取消，等待者收到取消状态。队列接口应返回枚举结果，而不是只返回 bool，让调用者区分 success、closed、cancelled。

四、锁内代码越短越好，但不能拆散不变量 临界区应只做状态检查、移动元素和更新索引，不应在锁内解析大对象、写文件、调用用户回调或分配大量内存。锁内工作过长会造成 convoy，其他线程排队等待。但为了缩短锁而把不变量拆到锁外，也会出错。正确做法是先在锁外准备好元素，再进入锁内提交；pop 时锁内取出元素所有权，锁外处理。

异常安全也要考虑。push 时如果元素移动或构造可能抛异常，不能先增加 size 再构造；pop 时如果移动到输出对象抛异常，队列状态不能丢元素。实际工程中可以要求元素移动不抛，或使用 optional 存储，或先在槽位构造成功后再发布 size。教材应让读者知道异常路径不是 C++ 并发代码的边角料。

五、Semaphore 表达资源计数，但不保护复杂不变量 信号量适合表达“有多少资源可用”，例如空槽数量、满槽数量、并发请求额度。它不直接保护 head、tail 和 buffer 的一致性。一个环形队列可以用两个 semaphore 管理空槽和满槽，再用互斥保护索引；也可以在 SPSC 场景下用原子索引避免锁。选择取决于生产者消费者数量和不变量复杂度。

不要用 semaphore 替代所有 mutex。若状态是多个字段组合，仍然需要互斥或明确无锁协议。Semaphore 的优势是资源计数和等待路径清晰，尤其适合限流和并发度控制。I/O 章节会把它扩展成 in-flight 请求预算。

六、锁性能要看等待时间而不是锁次数 锁调用次数多不一定慢，低竞争短临界区可能很便宜；锁次数少但每次持有很久，尾延迟可能很差。测量锁性能要记录等待时间、持有时间、队列长度、上下文切换和 futex 慢路径。只看总吞吐会漏掉 convoy 和长尾。

Linux futex 的直觉是：无竞争时用户态 fast path 修改原子状态即可；竞争时进入内核睡眠和唤醒。理解这一点后，读者会知道为什么短临界区低竞争锁可以接受，为什么高竞争热点锁会突然变慢。工具上可以用 perf、trace 或应用内计时记录等待区间。

七、实验交付：QueueContract 本章给项目留下 **QueueContract**。它写明容量、push 返回值、pop 返回值、关闭模式、取消模式、是否保持 FIFO、元素所有权、等待谓词和异常保证。任何线程池、I/O pipeline 或分布式消息队列都要引用这个合同。

实验比较三种队列：无界 mutex 队列、有界条件变量队列、信号量控制并发的队列。输入包含上游快下游慢、下游快上游慢、关闭时仍有元素、取消时有等待者、异常注入。验收不只看吞吐，还看内存峰值、等待者是否悬挂、关闭后是否丢失已提交元素。这样锁章就真正服务后续系统，而不是停留在 API 介绍。

10.9 案例深化：锁顺序和组合操作

单个锁容易讲，多个锁组合才容易出事故。系统中常见组合操作包括从一个队列移动到另一个队列、同时更新任务表和队列、提交 manifest 时更新状态和文件索引。若锁顺序不清，死锁和不变量破坏会出现。

一、锁保护的是对象边界 每把锁都应有保护对象。队列锁保护 head、tail、size、closed 和 buffer；任务表锁保护 task state map；manifest 锁保护提交表。若一个函数同时需要队列和任务表，必须按

固定顺序获取锁，或把操作拆成两个阶段，中间使用稳定 id 连接。不能在不同代码路径里用不同顺序。

锁顺序应写进设计文档或类型注释。比如先 task table，再 ready queue，再 manifest。若某个路径必须反过来，说明边界设计可能有问题。锁顺序不是死板规定，而是消除循环等待的工具。

二、组合操作要找提交点 从 pending 队列取一个 task 并标记 running，是一个组合操作。若先从队列移除，后标记状态，中间抛异常或取消，task 可能丢失；若先标记 running，后出队失败，状态和队列不一致。正确做法是在同一临界区更新相关不变量，或设计可恢复的中间状态。

提交点也影响用户语义。submit 返回成功的那一刻，任务是否一定会运行？shutdown 开始后，并发 submit 是成功还是失败？close 队列时，已入队元素是否保证被处理？这些问题都要靠组合操作的提交点回答。

三、避免锁内调用未知代码 持锁调用用户回调、日志格式化、分配器、I/O 或另一个模块的复杂函数，会放大死锁和长临界区风险。锁内应只做状态变更和简单移动。需要回调时，把要通知的对象收集出来，释放锁后再调用。条件变量通知通常可以在释放锁后做，但要确保状态已经更新。

这个原则在任务运行时尤其重要。任务完成时要更新 shared state，然后唤醒等待者或调度后继；不要在持 shared state 锁时执行后继任务。否则一个任务完成可能递归进入任意用户代码，锁边界失控。

四、组合锁实验 构造两个队列和一个任务表，实现 move task。故意用相反锁顺序制造死锁，再改成固定顺序；故意在两步之间抛异常，观察状态丢失，再改成单临界区或中间状态。这个实验能让锁章从 API 使用升级到不变量设计。

10.10 贯穿案例：队列 close 后为什么还有消费者永远睡着

锁和条件变量如果从 API 开始讲，读者会记住 `wait`、`notify_one`、`notify_all`，却未必知道等待的到底是什么。考虑一个有界队列：生产者 push 解析后的 batch，消费者 pop 后计算。正常路径都能跑。作业取消时，主线程设置 `closed=true`，但只通知了 `not_full`，忘记通知 `not_empty`。此时某个消费者正等队列非空，它永远睡着；另一个生产者被唤醒后发现 `closed`，返回失败；主线程 join 消费者时卡住。没有数据竞争，程序仍然死锁。

这个事故能讲清条件变量的本质：等待的不是通知，而是谓词。消费者等待的是“队列有元素或队列关闭”；生产者等待的是“队列有空间或队列关闭”。关闭改变了两个谓词，所以必须唤醒两个方向的等待者。通知只是让线程有机会重新检查谓词，不能替代谓词本身。这个推导比直接说“防虚假唤醒要 while”更容易让人记住。

一、先写队列合同而不是先写锁 QueueContract 应说明容量、FIFO 是否保证、push 成功后元素是否一定可见、pop 在关闭后如何返回、drain 和 cancel 的差异、等待者如何唤醒、元素所有权何时转移。合同写清后，锁只是实现不变量的手段。若合同没有区分 ClosedAndDrained、Cancelled 和 Success，调用者只能用 bool 猜测，关闭路径必然混乱。

二、关闭状态要参与所有等待谓词 消费者谓词是 `size>0||closed||cancelled`；生产者谓词是 `size<capacity||closed||cancelled`。等待返回后，消费者还要区分有元素、关闭且无元素、取消；生产者要区分有空间、关闭、取消。这样关闭不会依赖某次通知刚好到达。条件变量代码的正确性来自谓词完整，而不是通知次数。

三、drain 和 cancel 要分开测试 Drain 表示不再接收新元素，但已有元素必须被消费者取完；Cancel 表示未处理元素可以丢弃或标记取消。两者唤醒方式类似，返回语义不同。日志处理正常结束适合 drain，用户取消或 fatal 错误可能需要 cancel。若一个接口只叫 close，却不说明是哪一种，后续线程池和 I/O 管线都会出现歧义。测试要分别覆盖两种模式。

四、锁内只更新不变量，锁外处理元素 Push 前可以在锁外构造元素，进锁后只检查谓词、移动元素、更新 head/tail/size，再通知；pop 在锁内取出元素所有权，锁外解析、写文件或执行回调。这样临界区短，不会因为用户代码阻塞其他线程。可是为了缩短锁，不能把 size、head、tail 的一致关系拆到锁外。短临界区和完整不变量要同时满足。

五、异常路径要保持队列不变量 如果元素移动可能抛异常，push 不能先增加 size 再构造；pop 也不能先减少 size 再把元素交给调用者却在移动时失败。教学实现可以要求元素 no-throw move，或者使用内部 optional 槽位和两阶段发布。关键是让读者知道异常不是 C++ 并发代码的边角料。异常路径破坏不变量，后续等待谓词也会失效。

六、从有界队列走向背压 有界队列不只是内存优化，它是背压边界。上游 push 阻塞或返回 Full，说明下游跟不上；队列容量决定最多积压多少 batch；关闭状态决定系统如何收束。I/O in-flight、MessageBus inbox、线程池 ready queue 都是同一类合同。第 10 章如果把队列 close 讲清，后面章节的背压和关闭就有共同语言。

七、最小复现实验 容量为一的队列，启动一个消费者等待 pop，不放入元素，主线程关闭队列。错误版本只通知 not full，消费者卡住；修复版本 notify all 后消费者返回 Closed。再加入一个生产者等待空间，关闭后它也返回 Closed。最后测试 drain：队列里已有一个元素，关闭后消费者先取元素，再下一次 pop 返回 Closed。这个小实验足以证明谓词和关闭合同，比大规模压力测试更能解释问题。

10.11 补强案例：读写锁和 RCU 的适用边界

读多写少的共享状态常让人想到读写锁或 RCU。二者都能提高读并发，但语义和成本不同。读写锁仍然让读者进入锁协议，写者需要等待读者；RCU 让读者更轻，但更新者延迟回收旧版本。选择前要看读写比例、读侧耗时、写侧要求和对象生命周期。

一、读写锁适合短读侧临界区 若读操作短、不会阻塞、写操作也不频繁，读写锁可以让多个读者并发。问题是读者太多时写者可能饥饿，或实现为了避免饥饿让新读者等待写者。读写锁不适合读侧长时间阻塞 I/O，也不适合读者在锁内调用未知代码。

二、RCU 适合不可变版本切换 RCU 的读者不阻塞写者发布新版本，但旧版本回收要等待读者离开。它适合配置表、路由表、读多写少索引。对象应尽量不可变，更新创建新版本。若写很多，旧版本堆积；若读者长时间不退出，回收延迟。RCU 的成本转移到更新和回收。

三、配置表案例 配置表读频率高，写频率低。Mutex 简单但所有读串行；读写锁允许多读；atomic shared pointer 或 RCU 让读者拿 snapshot。三者都正确，但成本不同。实验比较读延迟、写延迟、旧版本释放延迟和代码复杂度。不要只看吞吐，写侧尾延迟和内存峰值也重要。

四、和原子章连接 读写锁通过 lock unlock 建立 happens-before；RCU 或 atomic snapshot 需要发布和生命周期协议。若对象会被释放，必须说明谁等待读者。同步原语选择不是 API 喜好，而是围绕不变量、等待和回收边界的设计。

10.12 补充案例：条件变量和关闭竞态

关闭竞态是条件变量代码最常见的问题之一。生产者正在 push，消费者正在 wait，另一个线程调用 close。若状态和通知顺序不清，可能丢元素、悬挂等待者或让关闭后的 push 成功。

一、close 必须持锁修改状态 关闭状态和队列 size 属于同一不变量，应在同一把锁下修改。close 设置 closed 后 notify all。等待者醒来后检查 size 和 closed，决定取剩余元素或退出。若 close 不持锁，等待者可能错过状态变化。

二、并发 push 和 close push 进入临界区后，如果队列未关闭并成功放入元素，它应返回成功；close 之后进入的 push 应失败。push 和 close 的先后由锁内顺序决定，而不是调用开始时间。这个语义要写在 QueueContract 里。

三、drain 语义的消费者退出 Drain close 下，消费者在 size 大于零时继续 pop，即使 closed 已经 true；只有 size 为零且 closed true 时退出。很多错误版本看到 closed 就立即退出，导致已入队元素丢失。

四、实验 让多个生产者和消费者并发 push、pop、close。检查成功 push 的元素都被 pop，close 后 push 失败，所有等待者退出。用小容量队列放大竞态。这个实验比普通生产消费测试更能证明关闭语义。

10.13 补充专题：Semaphore 和令牌桶限流

信号量不仅能实现队列资源计数，也能表达并发额度。分布式和 I/O 章节里的 in-flight 限制，可以在单机用 semaphore 或令牌桶先理解。

一、并发额度 假设最多允许 64 个写请求同时在 I/O 子系统中飞行。每次提交前 acquire 一个令牌，completion 后 release。这样即使上游很快，in-flight 也不会无限增长。令牌数量就是资源预算。

二、令牌和队列容量不同 队列容量限制等待中的请求数，令牌限制已经提交但未完成的请求数。二者可以同时存在。队列满说明上游生产过快；令牌耗尽说明下游并发资源用满。指标解释不同，调参也不同。

三、超时和令牌归还 请求超时后是否归还令牌，要看底层请求是否仍可能完成。若超时不等于取消，过早归还会让 in-flight 实际超过预算。必须等 completion 或明确取消成功后归还。这个原则会在分布式请求窗口中重复出现。

四、实验 用 semaphore 限制同时写出的 block 数，模拟慢 completion。比较无限制、只限制队列、限制 in-flight 三种策略。记录内存、尾延迟和完成顺序。读者会看到资源计数比简单队列更精确。

10.14 补充专题：锁设计中的所有权转移

锁保护状态，但数据所有权常常要在锁内转移、锁外处理。这个模式在队列、任务表和 I/O completion 中反复出现。掌握它能同时减少临界区时间和生命周期错误。

一、锁内移动，锁外处理 消费者 pop 时，在锁内检查队列非空，把元素移动到局部变量，更新 head 和 size，然后释放锁。真正处理元素在锁外进行。这样队列不变量在锁内完成，耗时业务逻辑不阻塞其他线程。若元素移动可能抛异常，要设计异常保证。

二、所有权必须唯一 元素从队列槽位移动到消费者局部变量后，队列不再拥有它。若同时保留引用或指针，就会出现双重所有权。C++ 中使用 move-only 类型能帮助表达这一点。任务对象、I/O buffer 和 message payload 都适合用移动语义表达所有权转移。

三、通知不等于所有权 通知等待者只是告诉状态可能变化，不转移数据所有权。等待者醒来后仍要持锁检查，并在锁内取得元素所有权。把通知当作“我已经拥有数据”会导致竞态。条件变量教学必须强调这一点。

四、实验 实现一个队列传递 move-only buffer。验证 push 后生产者不能再使用，pop 后消费者独占。故意在锁外读取队列内部引用，观察关闭和复用时的悬垂风险。这个实验能把锁和 C++ 所有权结合起来。

10.15 从临界区推导同步语义

锁、条件变量和信号量不是三个孤立 API。它们解决的是不同形态的等待和互斥：锁保护共享不变量，条件变量等待状态变化，信号量管理可用令牌。若只记函数名，代码很容易写出偶发死锁、丢通知、虚假唤醒错误和容量失控。同步章节要从具体不变量开始：哪些数据必须一起变化，哪个状态让等待者继续，容量代表什么资源，关闭时谁负责唤醒。

一、锁保护不变量，不保护代码块 很多错误来自把锁理解成“进入这段代码的门”。更准确地说，锁保护一组共享状态的不变量。比如队列的 `buffer`、`head`、`tail`、`size` 和 `closed` 必须在同一把锁下检查和修改。若 push 修改了 buffer 但 size 更新在锁外，消费者就可能看到不一致状态。临界区大小也要围绕不变量决定：太小会破坏原子状态变化，太大会扩大等待。先写不变量，再决定锁的边界。

二、条件变量等待谓词，不等待通知 条件变量的正确写法是循环检查谓词。通知只是提醒“可能有变化”，不是保证条件已经成立。虚假唤醒、多个消费者竞争同一元素、通知发生在等待之前，都要求等待者重新检查状态。以有界队列为例，消费者等待的是 `!empty || closed`，生产者等待的是 `!full || closed`。关闭状态必须进入谓词，否则关闭时等待者可能永远睡眠。读者要把这条规则理解为状态机，而不是背诵模板。

三、信号量适合表达可用资源数量 信号量的值代表令牌数量，比如可用连接数、可用 buffer 数、并发请求配额。它不适合表达复杂不变量，因为拿到令牌不代表某个容器状态已经一致。用信号量限制 in-flight 请求时，还要考虑异常路径和取消路径是否归还令牌；若令牌泄漏，系统会逐渐停住；若重复释放，容量约束会失效。信号量让容量显式化，也要求生命周期显式化。

四、关闭协议是同步结构的验收题 队列、线程池、buffer pool 最容易在关闭时暴露设计问题。关闭不是简单设置一个 bool，而是声明不再接受新任务、唤醒等待者、让已提交任务完成或取消、回收资源并禁止迟到回调访问已析构对象。每个同步结构都应有关闭状态表：Open、Closing、Drained、

Closed。若没有状态表，析构函数常会和后台线程、条件变量或 future 发生竞态。关闭协议写清楚，平时运行的正确性也会更清楚。

五、同步实验要故意制造交错 并发 bug 不会稳定出现，所以实验要主动制造交错：小容量队列、多生产者多消费者、随机 sleep、异常退出、取消、关闭时仍有等待者。测试不只看最终输出，还要看是否死锁、是否丢元素、是否重复消费、是否泄漏令牌、是否能在 deadline 内退出。同步章节的代码示例应鼓励读者把调度不确定性暴露出来，而不是只跑一次 happy path。

10.16 有界阻塞队列的完整语义

有界阻塞队列是同步章节最值得完整讲透的数据结构。它同时包含互斥、条件变量、容量、关闭、异常安全和背压。只给出 push 和 pop 代码是不够的，必须先定义语义：队列容量是多少，push 在满时等待还是失败，pop 在空时等待还是失败，close 后是否还能取出剩余元素，等待者如何醒来，析构时是否允许仍有线程访问。

一、状态变量要少而完整 一个基本实现需要 buffer、head、tail、size、capacity、closed。buffer 保存元素，head 指向下一个可读位置，tail 指向下一个可写位置，size 区分空和满，closed 表示不再接受新元素。所有变量都由同一把 mutex 保护。不要把 size 改成 atomic 试图减少加锁，因为它和 head、tail、closed 共同构成不变量，单独原子化会制造不一致观察。

二、push 的谓词是有空间且未关闭 生产者进入 push 后，在锁内等待 `size < capacity | !closed`。被唤醒后若 closed，返回失败或抛出明确定义的错误；否则构造元素，推进 tail，增加 size，再通知消费者。元素构造如果抛异常，队列状态不能改变。这里可以用先在局部准备对象，再在临界区移动构造的方式缩短锁时间，但状态更新必须仍在锁内完成。

三、pop 的谓词是有元素或已关闭 消费者等待 `size > 0 | !closed`。若 size 大于零，取出元素、推进 head、减少 size、通知生产者。若 size 为零且 closed，返回结束信号。这样 close 后消费者仍能 drain 剩余元素。若业务希望 close 后立即丢弃剩余元素，也必须写成另一种明确语义，不能让调用者猜。

四、close 要广播所有等待者 close 在锁内设置 closed，然后同时通知 `not_empty` 和 `not_full`。等待 push 的生产者需要醒来发现队列关闭，等待 pop 的消费者需要醒来 drain 或退出。close 应该幂等，多次调用不破坏状态。析构函数通常应要求外部先停止线程并调用 close；若析构内部调用 close，也仍要保证没有线程在对象销毁后访问条件变量。

五、测试覆盖队列语义而非只覆盖结果 测试要覆盖满队列 push 等待、空队列 pop 等待、close 唤醒生产者、close 唤醒消费者、close 后 drain、异常构造、多个生产者、多个消费者和容量为一。每个测试都应检查状态转移和元素数量。这个队列讲清楚后，读者再看线程池 ready 队列、I/O buffer pool 和 MessageBus inbox，就能识别同一套同步语义。

10.17 案例深化：有界队列的完整合同

有界队列是锁、条件变量和背压的交汇点。它看似简单：push 等队列不满，pop 等队列不空。真正的合同还包括关闭、异常、多个生产者消费者、公平性和容量语义。若这份合同没有写清，后续线程池、I/O 背压和 MessageBus 都会继承错误。

一、队列不变量 队列不变量包括：元素数不超过容量，head 和 tail 指向合法位置，closed 后不接受新元素，drain 模式下已有元素仍可取出，cancel 模式下已有元素被丢弃或交给清理器。锁保护这些不变量，而不是保护某几行代码。任何函数返回前，不变量都必须成立。

二、等待谓词 Producer 等待“队列未滿或 closed”，consumer 等待“队列非空或 closed”。谓词必须在锁下检查，wait 返回后重新检查。Notify 不是消息，丢了也没关系，因为状态才是真相。这个原则一旦掌握，虚假唤醒、提前 notify、多个等待者竞争都不再神秘。

三、容量是背压合同 容量不是随便取一个数。它表达上游最多可以超前下游多少工作。容量太小会降低吞吐，太大又会隐藏过载并增加内存峰值。报告要说明每个元素平均大小、允许排队时间和内存预算。I/O 章节的 in-flight bytes、分布式章节的 worker window 都是同一个思想。

四、不要持锁调用用户代码 持锁回调会把未知代码放进临界区，可能重入队列、拿其他锁、阻塞 I/O 或抛异常。正确做法是在锁下取出元素和状态，在锁外执行用户回调。若必须在锁内调用，就要写清不会阻塞和不会重入的合同。很多死锁不是锁本身复杂，而是锁内做了不可控工作。

五、测试要覆盖关闭矩阵 关闭矩阵至少包括空队列关闭、满队列关闭、生产者等待时关闭、消费者等待时关闭、多个等待者、drain 和 cancel 两种策略。每个测试都检查没有线程泄漏、没有重复元素、没有丢失未声明丢弃的元素。一个能通过关闭矩阵的队列，才适合作为后面运行时和 MessageBus 的基础。

10.18 案例复盘：有界队列的关闭合同

有界队列正常 push pop 通过，不代表同步正确。关闭时 producer 可能等队列不满，consumer 可能等队列非空，若 closed 没进入谓词，线程会卡死。锁保护的是不变量，条件变量等待的是谓词，notify 只是提醒重新检查状态。

不变量和谓词 队列不变量包括容量、head、tail、closed、drain 或 cancel 策略。Producer 谓词是队列未滿或 closed，consumer 谓词是队列非空或 closed。Wait 返回后必须在锁下重查。持锁回调、锁顺序反转和关闭遗漏都要作为测试场景。

实验判据 关闭矩阵覆盖空队列、满队列、等待 producer、等待 consumer、多等待者、drain 和 cancel。指标包括等待线程数、重复元素、丢弃元素、关闭耗时和锁持有时间。这个队列合同会被线程池、I/O 和 MessageBus 复用。

10.19 本章习题

习题与作业

习题一：为当前项目的 `BoundedMpmcQueue` 写不变量清单。逐行解释 `push` 和 `try_pop` 怎样保持这些不变量。指出当前接口缺少哪些线程池需要的语义。

习题二：把 `bounded queue` 设计成支持 `close`。不要求立刻写完整代码，但必须画出状态转换：open nonempty、open empty、open full、closed nonempty、closed empty。说明每个状态下 `push` 和 `pop` 应返回什么。

习题三：设计一个队列等待实验。多个生产者写入，消费者故意变慢，让队列经常满。记录生产者等待次数、最大队列长度和总吞吐。解释这是背压，不是单纯性能退化。

习题四：阅读 `glibc`、`musl` 或 `Linux futex` 文档和源码入口，写清 `mutex` 低竞争 fast path 和竞争 slow path 的差异。报告中必须说明哪些部分发生在用户态，哪些部分进入内核。

第 11 章

原子操作与内存序

先看一个看似微小的发布 bug：生产者先填充对象字段，再把 `ready` 标志设为 `true`；消费者看到 `ready` 后读取字段。测试在开发机上一直通过，换平台或压力运行后偶尔读到旧字段。问题不在于 `ready` 是否原子，它确实没有数据竞争；问题在于这个原子标志有没有把普通字段的写入发布给另一个线程。只讲“原子读写不可分割”，解释不了这个 bug。

本章从对象发布问题进入原子操作和内存序。原子操作提供不可分割的读写或读改写，并参与线程间同步；内存序定义普通写如何被另一个线程看见、哪些操作不能被重排、哪个状态转移是线性化点。理解原子操作，必须区分原子性、可见性、顺序性和对象生命周期。

11.1 从数据竞争到原子操作

`std::atomic` 的 `load`、`store`、`fetch_add`、`compare_exchange` 等操作会生成特殊指令或特殊约束。它们可以避免数据竞争，但不自动保证算法正确。一个原子计数器容易写，一个无锁队列需要处理节点发布、竞争失败、ABA 和回收。

低竞争下，原子操作可能比 `mutex` 轻；高竞争下，多个线程反复修改同一 `cache line`，原子也会扩展性很差。判断依据仍然是共享写频率和一致性流量。

11.2 从发布数据到内存序

`memory_order_relaxed` 只保证原子性，不建立跨变量同步。计数器统计常用 `relaxed`，因为只关心最终数值。`memory_order_release` 发布写入，`memory_order_acquire` 获取发布的结果，二者配合可建立 `happens-before`。`memory_order_seq_cst` 提供更强的全局顺序，但成本和约束也更强。

选择内存序的原则是从语义出发，而不是从性能愿望出发。若一个线程写好对象后通过原子标志发布，写标志应 `release`，读标志应 `acquire`。若只是统计事件数量，`relaxed` 足够。

Listing 11.1 `release/acquire` 发布数据

```
1 struct Shared {
2     std::int32_t data = 0;
3     std::atomic<bool> ready = false;
4 };
5
6 void producer(Shared& shared) {
7     shared.data = 42;
8     shared.ready.store(true, std::memory_order_release);
9 }
10
11 std::int32_t consumer(Shared& shared) {
12     while (!shared.ready.load(std::memory_order_acquire)) {
13     }
14     return shared.data;
15 }
```

`release store` 之前的普通写，在 `acquire load` 看到该 `store` 后，对 `consumer` 可见。这里不能把 `ready` 简单改成普通 `bool`，也不能把 `acquire/release` 无脑改成 `relaxed`；否则程序缺少跨线程发布关系。

理解 `acquire/release` 时，不要把它想成“刷新所有缓存”这样粗糙的动作。更准确的教材模型

是：release 把之前的写放在发布点之前，acquire 在看到这个发布后，让之后的读写不能越过获取点，并建立 happens-before。具体硬件怎样实现，取决于架构强弱和编译器约束。C++ 程序只应依赖语言层面的同步关系。

Relaxed 的正确用途 Relaxed 常用于统计计数、采样标志、引用计数的某些阶段和不参与对象发布的指标。它保证原子变量自身不会数据竞争，但不保证其他变量的可见性。用 relaxed 写高性能代码时，必须能说清“我只需要这个原子变量自己的数值，不需要借它同步别的数据”。

例如请求计数器可以用 relaxed，因为最终统计只关心总次数。对象指针发布不能只用 relaxed，因为读到指针的线程还需要看到对象内部字段已经初始化。

11.3 从 CAS 到无锁协议

Compare-and-swap 读取旧值，若仍等于期望值就写入新值。无锁算法常用 CAS 循环处理竞争。CAS 失败不表示错误，而是说明其他线程已经修改状态，需要重新读取并尝试。

CAS 循环必须考虑进度。低竞争时很快，高竞争时可能大量失败。复杂结构还要考虑 ABA：一个位置从 A 变 B 又变回 A，CAS 看到 A 可能误以为没有变化。解决方法包括版本号、hazard pointer、epoch 回收等。

CAS 循环还必须考虑循环体里是否做了不可重复副作用。失败重试意味着循环体可能执行多次，如果每次尝试都分配内存、写日志、修改外部状态或增加不可回滚计数，就会产生额外语义问题。常见写法是先准备候选新状态，在 CAS 成功后才执行对外可见副作用；失败时只更新局部期望值并重试。

CAS 的 weak 和 strong 版本 `compare_exchange_weak` 允许伪失败，即使当前值等于期望值也可能失败，因此通常放在循环里。`compare_exchange_strong` 不允许伪失败，更适合不在循环里的单次尝试。很多平台上二者生成代码可能接近，但语义差异必须理解。

CAS 还有一个容易忽视的行为：失败时会把当前实际值写回 `expected` 参数。这让循环可以基于新值继续尝试，但也容易让初学者误用 `expected`。写无锁代码时，要把成功内存序和失败内存序都写清楚。

内存屏障和硬件 不同架构的内存模型不同。x86-64 相对强，ARM 和 POWER 更弱。C++ 内存序提供跨平台抽象，编译器和硬件会根据目标架构生成必要约束。不要因为某段代码在 x86 上“看起来没问题”，就认为它是正确的并发程序。

原子与缓存一致性 原子读改写通常会让目标 cache line 进入独占修改路径。多个核心同时对一个原子变量做 `fetch_add`，实际上是在争夺同一条 cache line 的所有权。内存序越强，编译器和硬件重排空间越小；竞争越强，一致性流量越大。

这就是为什么高性能计数常用分片计数器。每个线程或每个核心写自己的本地计数，读取总数时再合并。它牺牲实时精确读取成本，换取写入热路径扩展性。

内存序选择流程 选择内存序可以按问题反推。第一，是否只需要原子变量自身不撕裂、不数据竞争。如果是统计计数，通常 relaxed 足够。第二，是否通过某个原子变量发布了其他普通数据。如果是，需要 release/acquire 或更强同步。第三，是否需要多个线程观察到一个全局一致顺序。如果确实需要，才考虑 `memory_order_seq_cst`。第四，是否可以用 mutex 表达更清晰的不变量。如果共享状态复杂，mutex 往往比一组难懂的原子更可靠。

内存序不是性能调参旋钮。把 `memory_order_seq_cst` 改成 relaxed 之前，必须写出被保留的同步关系和被放弃的顺序关系。若写不出来，说明程序设计还没清楚到可以安全降级。

x86 和 ARM 的差异 x86-64 的内存模型相对强，很多 acquire/release 在硬件指令上看起来很轻；ARM 等架构更弱，可能需要更显式的屏障。可移植 C++ 代码不能因为在 x86 上测试通过，就省略必要内存序。教材示例必须按 C++ 语义写正确，而不是按某台机器的偶然行为写正确。

三件事必须分开 学习原子操作时，最重要的是把三件事分开：原子性、可见性和顺序性。原子性回答“这个变量本身的读写会不会撕裂，会不会产生数据竞争”。可见性回答“一个线程写入的其他数据，另一个线程什么时候能看到”。顺序性回答“多个操作之间能否被编译器或硬件重排，其他线程能以什么顺序观察到”。很多错误来自把原子性误当成全部同步。

例如 `std::atomic<bool>ready` 可以原子地表示标志，但如果生产者用 relaxed 存储 `ready=true`，消费者用 relaxed 读取到 true 后再读普通字段，普通字段的初始化不一定通过这

个标志建立可见性关系。标志本身没有数据竞争，不代表对象发布正确。反过来，一个 relaxed 计数器用于统计请求次数完全可以，因为读取计数不需要借它观察其他普通数据。

教材里的判断标准应是：这个原子变量是否只是自己的值有意义，还是它承担了发布其他状态的责任？只看自己的值，通常 relaxed 可以讨论；发布其他状态，就需要 release/acquire、mutex 或更强同步。

发布对象的完整故事 对象发布是内存序最常见的真实案例。生产者分配或准备一个对象，写入多个普通字段，然后把指针或 ready 标志交给消费者。消费者看到标志后读取对象字段。如果没有同步，消费者可能看到未初始化或部分初始化状态；即使在某台 x86 机器上看不出来，C++ 语义也不允许依赖这种偶然。

Listing 11.2 用原子指针发布只读对象

```

1 struct Config {
2     std::int32_t shard_count = 0;
3     std::int32_t queue_capacity = 0;
4 };
5
6 std::atomic<Config*> published_config{nullptr};
7
8 void publish(Config* config) {
9     config->shard_count = 16;
10    config->queue_capacity = 4096;
11    published_config.store(config, std::memory_order_release);
12 }
13
14 Config* acquire_config() {
15     Config* config = nullptr;
16     while (config == nullptr) {
17         config = published_config.load(std::memory_order_acquire);
18     }
19     return config;
20 }

```

release store 之前对 config 字段的写入，在 acquire load 读到同一个指针后，对消费者可见。这个例子还隐藏了生命周期问题：谁拥有 Config，何时释放，是否允许热更新？release/acquire 只解决发布时的可见性，不自动解决对象回收。无锁章节会继续处理这个问题。

消息传递和 happens-before 内存序最终要落到 happens-before 关系。若线程 A 的某些写入 happens-before 线程 B 的某些读取，B 就能按 C++ 规则观察到 A 发布的结果。mutex 的 unlock 和后续 lock 可以建立这种关系；release store 和读到它的 acquire load 也可以建立这种关系；线程创建和 join 也有同步语义。

理解 happens-before 时，不要把它想成“时间上先发生”。两个线程在物理时间上先后执行，不一定建立 C++ 同步关系。生产者可能确实先写了 data，再写 ready；消费者也可能确实后读 ready，再读 data。但如果 ready 使用 relaxed，语言层面没有建立 data 的发布关系，程序就缺少可移植保证。并发正确性看的是同步关系，不是肉眼看到的执行顺序。

Release sequence 和读改写 C++ 内存模型里，release sequence 处理了一类常见场景：一个 release 操作后面接着对同一个原子变量的一系列读改写操作，acquire 读到这个序列中的值时，仍可能与最初 release 建立同步。这个规则让一些引用计数、队列状态和发布协议能正确表达，但也容易让初学者迷糊。

本书不要求在入门阶段手写复杂 release sequence 算法，但要知道两个原则。第一，同步关系通常围绕同一个原子对象上的读写建立，不能随便把一个原子变量的 release 借给另一个原子变量。第二，读改写操作既读旧值又写新值，它的成功和失败内存序都需要理解。CAS 成功时可能发布新状态，失败时只是观察当前状态；失败内存序不能比成功内存序更强。

CAS 循环的副作用边界 CAS 循环最容易漏讲的是副作用边界。因为 CAS 可能失败，循环体可能执行很多次。若循环体里做了不可回滚副作用，例如写日志、发送网络请求、增加外部计数、释放对象或修改另一个结构，那么失败重试会让语义混乱。正确写法通常把“准备候选值”和“提交副作用”分开，只有 CAS 成功后才执行对外可见副作用。

Listing 11.3 CAS 循环只在成功后提交外部副作用

```
1 bool try_set_max(std::atomic<std::int64_t>& target,
2   std::int64_t candidate) {
3   std::int64_t observed = target.load(std::memory_order_relaxed);
4   while (observed < candidate) {
5     if (target.compare_exchange_weak(
6       observed,
7       candidate,
8       std::memory_order_release,
9       std::memory_order_relaxed)) {
10    return true;
11  }
12 }
13 return false;
14 }
```

这个函数在失败时，**observed** 会被更新成当前实际值，下一轮基于新值继续判断。函数没有在循环内部写外部日志或修改其他共享结构，因此失败重试不会产生额外语义。若调用者需要记录“最大值被更新”，应在返回 true 后记录。

原子引用计数的双重语义 引用计数常用原子实现，但它不是简单计数器。增加引用通常只需要保证计数本身原子，减少引用并发现自己是最后一个引用时，需要在销毁对象前看到其他线程对对象的修改。也就是说，引用计数同时涉及原子性和对象生命周期。很多成熟库在 increment、decrement 和最后释放之间使用不同内存序，就是因为不同阶段承担不同语义。

教材不应让读者以为“引用计数用 relaxed 就行”或“引用计数都用 **memory_order_seq_cst** 最安全”。更好的说法是：引用计数需要一套完整协议。获取引用时必须确保对象仍然活着；释放引用时如果不是最后一个，只是减少计数；如果是最后一个，必须和其他线程对对象的访问建立足够同步，然后才能析构。协议比单条原子指令重要。

Seq-cst 的价值和成本 **memory_order_seq_cst** 提供一个所有 seq-cst 操作参与的单一全局顺序。它让推理更接近直觉，适合初始实现、教学示例和确实需要全局顺序的算法。但它不是“让并发程序自动正确”的开关。若对象生命周期没处理，seq-cst 也不能防止 use-after-free；若不变量跨多个变量没有协议，seq-cst 原子也不能自动保持不变量。

性能上，seq-cst 可能限制编译器和硬件重排，在某些架构上需要更重屏障。x86 上某些 seq-cst load/store 也许看起来不重，但读改写和跨平台成本仍要考虑。工程建议是：先用清晰同步写正确，再在热点路径中根据证明降低到 acquire/release 或 relaxed。不要为了性能从一开始就写最弱内存序，也不要为了省脑把所有原子永远 seq-cst 后忽略结构设计。

11.4 从局部技巧到完整状态机

原子和 mutex 可以组合使用，但必须明确各自保护什么。常见模式是用原子做 fast path 标志，用 mutex 保护复杂慢路径状态。例如一个队列可以用原子 size 做近似指标，但真实 head、tail、buffer 仍由 mutex 保护。错误模式是同一份状态有时用锁保护，有时绕过锁用原子修改，导致不变量被拆碎。

如果一个变量参与锁保护的不变量，就不要随意在锁外用 relaxed 读它来做决策，除非这个决策允许过期和近似。例如队列长度指标可以近似展示；判断是否可以安全 pop 不能靠锁外近似 size。区分“指标”和“协议状态”非常重要。

内存序测试为什么困难 内存序 bug 很难通过普通测试证明不存在。它可能只在特定架构、编

译器优化、核心数和时序下出现。压力测试能增加信心，但不能替代语义证明。写并发代码时，应先用 happens-before 关系证明，再用 ThreadSanitizer、压力测试和架构测试辅助验证。

手机或 x86 设备上跑过不代表 ARM 服务器、POWER 或未来编译器也正确。可移植 C++ 并发必须按语言模型写。教材示例因此宁愿稍保守，也不能依赖某个硬件“实际上不会乱序”的经验。

原子性的边界 原子操作首先保证的是单个原子对象上的读写不会产生数据竞争，并且读改写不会被撕裂。它不自动保证跨多个变量的不变量，也不自动发布普通变量，更不自动管理对象生命周期。把一个字段改成 `std::atomic`，只解决了这个字段本身的并发访问问题；如果这个字段只是一个更大状态机的一部分，状态机仍然可能错误。

例如队列的 `size` 使用原子，并不能让 `head`、`tail` 和 `buffer` 的关系安全。两个消费者可能都看到 `size` 大于零，然后读取同一个槽位。再例如一个指针使用原子发布，并不能说明指针指向对象何时释放。另一个线程 `acquire` 读到指针后，如果发布者随后释放对象，读者仍然可能悬垂。内存序解决可见性和顺序问题，不解决所有权问题。

因此，选择 `atomic` 之前要先回答三件事。第一，这个原子对象的线性化含义是什么，是计数、标志、指针、状态枚举还是队列索引。第二，它是否同步其他普通数据。第三，读取者拿到值后，对值关联的资源有什么生命周期保证。回答不清楚时，用 `mutex` 保护完整不变量通常更稳妥。

Relaxed 的正确用法 `memory_order_relaxed` 不是“随便乱序也无所谓”的开关，而是只需要原子性、不需要跨线程发布其他数据时的选择。请求总数、统计计数、采样次数、CAS 失败计数这类指标，通常可以 `relaxed`，因为读取者只关心数值最终大致准确或精确累加，不依赖它看到其他普通变量写入。

一个 `relaxed` 计数器可以保证每次 `fetch_add` 不丢，但不能保证“看到计数增加就一定看到对应请求对象已经初始化”。如果业务要用计数器作为队列元素可见的信号，就需要 `release/acquire` 或锁。很多并发 bug 来自把统计变量升级成协议变量，却没有同步语义。

Listing 11.4 Relaxed 适合不发布其他数据的统计计数

```

1 class RequestStats {
2 public:
3     void record_success() {
4         this->success_count_.fetch_add(1, std::memory_order_relaxed);
5     }
6
7     std::uint64_t success_count() const {
8         return this->success_count_.load(std::memory_order_relaxed);
9     }
10
11 private:
12     std::atomic<std::uint64_t> success_count_{0};
13 };

```

这个类的接口没有承诺读取计数时能看到某个请求的响应对象，也没有承诺计数和其他指标之间有一致快照。若监控页面同时读取 `success`、`failure`、`timeout` 三个 `relaxed` 计数，它们可能来自不同时间点。作为指标，这可以接受；作为协议判断，这不够。

发布对象的完整协议 `release/acquire` 常用来发布对象，但完整协议包含初始化、发布、获取、使用和回收五个阶段。初始化阶段由生产者独占对象，普通写入即可。发布阶段用 `release store` 把指针或状态写给消费者。获取阶段消费者用 `acquire load` 读到 `release` 写入的值。使用阶段消费者可以读取对象字段。回收阶段必须保证没有消费者仍在使用对象，这一步不是 `release/acquire` 自动完成的。

如果对象只发布一次并且活到程序结束，协议很简单。如果对象会热更新，就需要版本、引用计数、RCU、shared pointer、epoch 或锁来管理旧对象。否则消费者可能刚 `acquire` 到旧指针，发布者就释放旧对象。不要把“可见”误认为“活着”。可见性和生命周期是两个不同问题。


```
publish protocol:
  producer initializes object fields
  producer release-stores pointer
  consumer acquire-loads pointer
  consumer reads fields
  reclamation waits until no consumer can hold old pointer
```

配置发布是学习这个边界的好例子。若配置对象数量很少，可以选择永不释放旧配置，用内存换简单性。若配置频繁更新，就需要引用计数或 RCU。若配置很大，又不能长时间保留旧版本，就要让读者理解回收成本会进入设计。

Acquire 和 Release 的方向 release 和 acquire 有方向。release 用在发布者把之前的普通写入推出去；acquire 用在消费者读到发布值后，把之后的普通读取拉进来。把消费者的 store 写成 release，不能让它看到生产者数据；把生产者的 load 写成 acquire，也不能发布数据。内存序要放在建立同步关系的那次读写上。

同步还要求 acquire 读到对应 release 或其 release sequence 中的值。如果消费者 acquire load 读到的是旧值，没有读到发布者的 release store，就没有通过这次操作建立同步。很多协议因此需要循环等待某个状态值，而不是只 load 一次。状态枚举比布尔值更清楚，因为它能表达 Empty、Writing、Ready、Closed 等阶段。

状态枚举比多个布尔值更稳 并发状态机如果用多个布尔值表达，很容易出现组合状态没有定义。例如 `ready=true`、`closed=true`、`failed=false` 到底代表什么？如果这些布尔值由不同线程更新，读者还可能看到中间组合。使用一个原子枚举表示状态，能让状态转换更集中，CAS 也更容易表达。

Listing 11.5 原子状态枚举表达提交阶段

```
1 enum class SlotState : std::int32_t {
2     Empty,
3     Writing,
4     Ready,
5     Closed,
6 };
7
8 struct SlotHeader {
9     std::atomic<SlotState> state{SlotState::Empty};
10 };
```

状态枚举不是万能的。若状态改变还伴随多个数据字段更新，仍然需要 release/acquire 或锁来发布这些字段。若状态之间有复杂不变量，可能要用 mutex。它的价值是减少非法组合，让线性化点更容易定位。

CAS 成功和失败的内存序 `compare_exchange_weak` 和 `compare_exchange_strong` 都有成功内存序和失败内存序。成功时，操作既读旧值又写新值；失败时，只读取当前值并把它写回 expected 参数。失败内存序不能是 release，因为失败没有发布新值；失败内存序也不能比成功更强。常见写法是成功使用 acquire/release 或 `acq_rel`，失败使用 relaxed 或 acquire，具体取决于失败路径是否要基于读取到的值观察其他数据。

CAS 循环还要处理 weak 可能伪失败。弱 CAS 允许即使值相等也失败，因此必须放在循环里。强 CAS 更适合不想处理伪失败的单次尝试，但在循环里弱 CAS 通常也可以。教材不应让读者机械背 API，而要理解：expected 是输入也是输出，失败后它变成实际观察值，下一轮判断要基于这个新值。

Listing 11.6 CAS 失败后 expected 被更新

```
1 bool mark_ready(std::atomic<SlotState>& state) {
```

```

2 SlotState expected = SlotState::Writing;
3 return state.compare_exchange_strong(
4     expected,
5     SlotState::Ready,
6     std::memory_order_release,
7     std::memory_order_relaxed);
8 }

```

如果返回 `false`，`expected` 不再一定是 `Writing`，而是当前实际状态。调用者可以用它决定是已经 `Ready`、已经 `Closed`，还是出现非法状态。很多 CAS 代码忽略失败后的 `expected`，错过了诊断信息。

Fetch-add 和热点 `fetch_add` 常被认为比 `mutex` 快，但在热点计数器上，它仍然会让同一 cache line 在多个核心之间迁移。原子读改写需要独占该 cache line；线程越多，迁移越频繁。低线程数下 atomic counter 很快，高线程数下可能成为扩展性瓶颈。这就是为什么 sharded counter 和 per-thread partial 如此重要。

计数器有三种典型语义。第一，精确实时全局值，每次操作后都要读到最新总数，这最难扩展。第二，精确最终值，运行过程中不频繁读取，适合分片后最后合并。第三，近似指标，允许短时间误差，适合线程本地缓存、周期性汇总或采样。不要在需求只要第二或第三种时，实现第一种最昂贵语义。

Fence 的谨慎使用 C++ 还提供 atomic fence，但初学者应谨慎使用。很多 release/acquire 协议可以直接写在原子读写上，更容易审查。Fence 把顺序约束和具体原子对象分离，表达能力强，但也更容易写错。读代码的人必须追踪 fence 前后的普通读写和相关原子操作，证明它们组成同步关系。

工程建议是：除非你在实现底层库、移植成熟算法或有明确性能证据，否则优先使用带内存序的原子操作、mutex 或成熟并发容器。Fence 不是高级优化标志，而是更底层的工具。教材可以介绍它存在，但不应把它作为普通业务代码首选。

内存序和硬件屏障的关系 C++ 内存序是语言层面的契约，编译器再把它映射到目标架构的指令和屏障。x86 的内存模型较强，某些 acquire/release load/store 可能不需要额外硬件屏障；ARM 和 POWER 更弱，可能需要显式屏障或带 acquire/release 语义的指令。代码不能因为在 x86 上看起来正确，就省略 C++ 同步关系。

编译器也会根据内存序限制优化。普通变量在没有同步关系时，编译器可以重排、缓存或消除某些访问。原子操作同时约束编译器和硬件。学习内存序时，不能只画 CPU 执行顺序，也要记住编译器参与了程序变换。可移植并发的基础是语言模型，而不是某台机器的偶然表现。

ThreadSanitizer 能做什么 ThreadSanitizer 擅长发现数据竞争：一个变量被多个线程访问，至少一个写，并且没有 happens-before 关系。它能帮助发现忘记加锁、普通变量错误共享、发布协议缺失等问题。但它不能证明算法线性化正确，不能证明内存序最弱但足够，也不能发现所有 ABA 或生命周期协议错误。没有数据竞争的程序仍然可能逻辑错误。

因此测试组合应分层。TSan 用于发现明显数据竞争；压力测试用于增加时序覆盖；模型化小测试用于验证状态机；代码审查用于检查 happens-before 和生命周期；性能测试用于判断同步结构是否扩展。并发程序不能只靠一种测试工具。

把内存序写进注释 热点原子代码应该把内存序理由写在附近。注释不应重复“使用 release store”，而应说明“发布 buffer 槽位内容给 consumer”或“relaxed 因为该计数器不发布其他数据”。这样的注释能让后续维护者知道哪些语义不能随便改。

如果你无法写出一句清楚的理由，说明当前内存序选择还没有被理解。此时宁愿使用更保守的锁或 seq-cst，也不要写一个看似高性能但没人能证明的 acquire/release 组合。高性能并发代码的质量标准是可证明，不是看起来很底层。

发布订阅的完整示例 配置发布是 release/acquire 的经典案例。生产者构造新配置，填好普通字段，然后用 release store 发布指针。消费者用 acquire load 读到指针后，可以读取字段。这个例子看似简单，但完整实现还要处理空指针、版本、旧配置回收和多次更新。

Listing 11.7 配置发布的最小形态

```

1 struct RuntimeConfig {
2     std::int32_t worker_count = 0;
3     std::int32_t queue_capacity = 0;
4 };
5
6 class ConfigPublisher {
7 public:
8     void publish(RuntimeConfig* config) {
9         this->current_.store(config, std::memory_order_release);
10    }
11
12    RuntimeConfig* current() const {
13        return this->current_.load(std::memory_order_acquire);
14    }
15
16 private:
17     std::atomic<RuntimeConfig*> current_{nullptr};
18 };

```

这个类只负责可见性，不负责所有权。若发布的配置由外部保证活到程序结束，代码可以成立。若配置会替换和释放，就必须加回收协议。把所有权藏在调用者约定里可以作为教学简化，但生产代码要把它写进类型或文档。

版本化发布 发布配置时，指针之外常需要版本号。消费者读到配置后，可以记录 version；如果执行过程中需要确认配置没有变化，可以再次读取 version。版本号也方便日志和调试：看到某次请求使用配置 42，而不是只看到一个地址。版本号可以放在配置对象内部，也可以和指针一起发布。

如果指针和版本分成两个原子，就要小心一致性。消费者可能读到新指针旧版本或旧指针新版本。更稳的设计是把 version 放在不可变配置对象中，发布指针即发布完整对象；或者使用锁保护多字段快照；或者用原子 shared pointer 等更高层机制。多个原子变量不能自动组成一致快照。

引用计数案例 原子引用计数常见，但容易误讲。增加引用需要确保对象还活着；减少引用发现自己是最后一个引用时，需要在析构前看到其他线程对对象的修改。一个简单 shared ownership 协议通常把 control block 和对象生命周期绑定。引用计数的 `fetch_add`/`fetch_sub` 只是协议中的一部分。

若线程已经持有一个有效引用，复制引用时的计数增加可以使用较弱内存序，因为对象可见性来自已有引用的同步。释放引用时，如果不是最后一个，也只是减少计数；如果是最后一个，就要执行 `acquire` 语义或等价屏障，确保析构看到其他线程在释放前的写入。成熟 `std::shared_ptr` 实现非常细致，不建议读者随手自研。

教材要强调一点：引用计数解决的是生命周期，不自动解决对象内部并发访问。多个线程通过 shared pointer 同时修改对象字段，仍然需要 mutex、原子或不可变设计。很多人以为 shared pointer 让对象“线程安全”，这是错误的。

状态发布和数据发布分离 有些协议同时有数据和状态。生产者写 buffer，再把 state 改成 Ready；消费者看到 Ready 后读 buffer。这里 state 的 `release`/`acquire` 同步发布 buffer 内容。若生产者先把 state 设 Ready，再写 buffer，消费者可能读到未初始化数据。顺序必须写清。

Listing 11.8 槽位状态发布数据

```

1 enum class BufferState : std::int32_t {
2     Empty,
3     Ready,
4 };
5

```



```

6 struct BufferSlot {
7     std::array<std::int32_t, 16> values{};
8     std::atomic<BufferState> state{BufferState::Empty};
9 };
10
11 void publish_slot(BufferSlot& slot, std::span<const std::int32_t> input) {
12     for (std::size_t i = 0; i < input.size() && i < slot.values.size(); ++i) {
13         slot.values[i] = input[i];
14     }
15     slot.state.store(BufferState::Ready, std::memory_order_release);
16 }

```

消费者必须 acquire 读取 state，并且只有看到 Ready 后才能读 values。若消费者 relaxed 读取 state，就没有可移植同步关系。若 values 写入可能抛异常或失败，不能提前发布 Ready。状态发布点就是线性化点。

Relaxed 指标的快照问题 多个 relaxed 指标组成的快照不一致是常见现象。假设一个系统有 accepted、completed、failed 三个计数器。监控线程分别读取它们时，accepted 可能来自较新时间，completed 来自较旧时间，因此 completed 加 failed 可能短暂大于 accepted 或小于 accepted。这不一定是 bug，而是近似指标语义。

如果监控页面不能接受这种不一致，就需要加锁快照、sequence lock、双缓冲或周期性汇总。不要在每个指标上简单使用 seq-cst 后以为解决了跨变量一致性。seq-cst 给单个原子操作提供全局顺序，但多个变量之间的业务不变量仍需要协议。

Sequence lock 的思想 Seqlock 常用于读多写少、读者可重试的场景。写者先把序号变成奇数，修改数据，再把序号变成下一个偶数。读者先读序号，复制数据，再读序号；如果两次序号相同且为偶数，说明读到一致快照；否则重试。它让读者无锁，但读者可能在写频繁时反复重试，且读取的数据复制必须能安全进行。

Seqlock 不适合包含指针和复杂生命周期的数据，除非另有回收保护。读者在没有锁的情况下复制数据，如果写者释放了指针对象，读者可能悬垂。它适合简单 POD 快照，例如时间、统计摘要、路由版本小结构。这个例子再次说明：原子协议和对象生命周期必须一起设计。

原子等待和通知 C++20 提供 `atomic::wait` 和 `notify_one/notify_all`，可以在原子值变化时等待，某些场景下比条件变量更直接。它适合等待单个原子状态变化，例如状态从 Pending 到 Ready。它仍然需要谓词思维：wait 返回后要重新检查值，通知不是业务状态本身。复杂不变量仍然更适合 mutex 和 condition variable。

原子等待也有关闭和取消问题。若线程等待状态变成 Ready，但系统关闭时状态应变成 Closed 并 notify；若只通知 Ready，不通知 Closed，等待者可能悬挂。工具变了，状态机原则不变。

内存序选择流程 选择内存序可以按流程。第一，是否只是线程本地？若是，不需要原子。第二，是否保护多个字段不变量？若是，优先 mutex 或明确协议。第三，是否只是统计计数，不发布其他数据？可以考虑 relaxed。第四，是否发布普通数据给消费者？需要 release/acquire 或更高层同步。第五，是否需要所有线程看到单一全局顺序？才考虑 seq-cst。第六，是否涉及指针生命周期？必须设计回收，内存序本身不够。

这个流程能减少随意性。内存序不是越弱越专业，也不是越强越正确。正确选择来自语义需求。

架构差异的测试策略 如果项目要跨 x86 和 ARM，不能只在 x86 上测。x86 较强内存模型可能掩盖某些缺少 acquire/release 的 bug；ARM 更弱，错误更容易暴露。若没有多架构机器，可以使用 ThreadSanitizer、模型检查工具或至少按照 C++ 内存模型审查。手机 Termux 可能正好是 ARM 环境，适合跑并发压力测试，但工具权限可能有限。

报告中应写明架构。一个在 x86 上“跑了很多次没出错”的 relaxed 发布示例，不是正确证据。C++ 并发正确性以语言模型为准。

案例：SPSC 发布协议 SPSC ring buffer 中，生产者先写 buffer 槽位，再 release 更新 tail；消费者 acquire 读取 tail 后，才能读到槽位内容。Head 由消费者写，tail 由生产者写，单写者让协议简化。若把 tail store 改成 relaxed，消费者可能在语言模型上没有看到 buffer 写入。若先更新 tail

再写 buffer，消费者可能读到未完成数据。

这个案例适合训练内存序注释。Tail release 的理由是发布槽位内容；tail acquire 的理由是获取槽位内容；head release 的理由是发布槽位可复用；head acquire 的理由是生产者看到可用空间。每个内存序都能对应到状态转换，而不是凭感觉选择。

案例：近似指标发布 运行时指标常使用 relaxed 分片计数。Worker 本地统计任务数、空转次数、窃取次数；监控线程周期性读取并汇总。这里不要求读者看到精确同时时刻快照，只要求趋势。因此 relaxed 或无锁读取加最终汇总可以接受。但如果指标用于触发限流或关闭，就必须重新定义同步。

同一个变量在不同用途下可能需要不同语义。作为监控，近似可以；作为协议，必须精确。设计时要防止“先当指标写，后来被业务拿去做决策”的语义漂移。

原子章的回归阅读要把变量分类，而不是先背内存序枚举。请求计数器只关心自身数值，可以使用 relaxed；配置发布 flag 要让读者看到普通字段初始化，需要 release/acquire；任务状态 CAS 要定义线性化点；原子指针还必须说明对象什么时候回收。四类变量如果混成“都是 atomic”，代码审查就会失效。

每个 CAS 循环都要检查副作用。失败会重试，所以循环体里不能提交文件、释放节点、写外部日志或发送消息，除非这些动作本身幂等并且和成功点绑定。成功内存序、失败内存序、expected 更新和退出条件都要写清。CAS 成功那一刻可能是线性化点，但它不自动解决 ABA 和生命周期。

项目中的 `AtomicProtocolNote` 应成为强制材料：变量名、写者、读者、是否发布普通数据、内存序、线性化点、禁止副作用和回收边界。若一个原子变量写不出这些内容，要么改回锁，要么重构状态机。原子代码贵在少而准，不在数量多。

C++ 内存模型还要求读者区分原子性、可见性和顺序。原子对象避免数据竞争，但并不自动让旁边的普通字段可见；release/acquire 能建立同步关系，但只在读到对应写入或其释放序列时成立；多个 relaxed 计数器各自正确，却不能组成一致快照。把这三件事分开，很多“我已经用了 atomic 为什么还错”的问题就能解释。

内存序教学还要避免过早追求最弱。项目里的第一版可以使用锁或 `seq_cst` 写出清楚语义，第二版再把纯计数降为 relaxed，把发布协议改成 release/acquire，把复杂多字段状态保留在锁下。每次降低顺序都要说明为什么仍满足协议。这样读者学到的是从强到弱的证明过程，而不是死记一张枚举表。

原子状态机应优先使用单个枚举表达主状态，减少多个 bool 组合出的非法状态。例如任务可以是 Pending、Running、Completed、Failed、Cancelled，而不是同时维护 done、failed、cancelled 三个标志。CAS 转换时要列出允许边，谁成功谁获得责任，谁失败谁读取新状态。这样后面的任务运行时和分布式提交都会复用同一种状态机思维。

等待策略也要和原子分开。原子状态可以快速检查，但长时间等待需要 `atomic::wait`、条件变量、futex 或运行时让出 worker。忙等适合极短等待，超过阈值就可能浪费核心甚至阻碍生产者运行。项目中应记录自旋次数、睡眠次数和唤醒原因，避免把“用了 atomic”误认为“等待成本消失”。

11.5 工程审查与项目落地

项目中每个原子变量都应写明：它表示什么状态，谁写谁读，是否发布其他数据，是否参与对象生命周期，选择的内存序理由是什么，失败路径如何处理，是否需要与其他变量组成快照。若只是统计，写明 relaxed 的理由；若是发布，写明 release/acquire 对应的数据；若涉及指针，写明回收协议。这个清单能显著降低并发维护风险。

原子章节总结 原子操作解决的是特定同步问题，不是并发正确性的万能答案。Relaxed 适合不发布数据的统计，release/acquire 适合发布和获取，seq-cst 适合需要简单全局顺序的场景。多个原子变量不会自动形成一致快照，原子指针不会自动管理生命周期。写原子代码时，必须能画出 happens-before、线性化点和回收边界。

内存序决策表 内存序选择可以压缩成一句话：先证明语义，再选择最弱足够顺序。只是计数，不发布数据，用 relaxed；发布对象字段，用 release/acquire；读改写既读又写，分别考虑成功和失败内存序；需要简单推理或教学，先用 seq-cst；涉及多个字段不变量，用锁；涉及指针生命周期，加回收协议。任何不能解释的内存序，都应该回到更简单同步。

案例：发布错误原因 如果一个任务失败，worker 写入错误对象，然后把状态设为 Failed。等

待者看到 Failed 后要读取错误对象。这和发布配置完全一样：错误对象字段必须在 release store 状态之前写好，等待者必须 acquire load 状态后读错误。若错误对象是动态分配的，还要保证生命周期覆盖等待者读取。错误传播也是发布协议，不是简单设置一个布尔值。

这类案例在任务图和分布式 driver 中很常见。状态枚举和结果对象应绑定设计，避免状态已完成但结果不可见或已释放。

本章落到贯穿项目 Compute Systems Engine 在本章应增加三类原子实验。第一，统计计数器：mutex counter、atomic relaxed counter、sharded counter，用来区分原子性和扩展性。第二，发布实验：一个 producer 初始化配置后 release 发布，consumer acquire 获取，用来说明对象可见性。第三，CAS 实验：实现 `try_set_max` 或无锁最大值更新，记录 CAS 失败次数，说明竞争下重试成本。

这些实验都要写清语义。计数器 relaxed 的理由是“不发布其他数据”；配置发布 acquire/release 的理由是“读取者需要看到普通字段初始化”；CAS 的理由是“成功那一刻是线性化点”。如果写不出理由，就不应该选择内存序。

从锁版本迁移到原子版本 把锁版本改成原子版本，不能只把字段类型换成 `std::atomic`。锁版本通常保护一组不变量，原子版本必须重新定义线性化点和状态转移。一个 mutex 队列的 `head`、`tail`、`size` 和 `closed` 在同一把锁下更新；如果只把 `size` 改成原子，其他字段仍会被并发破坏。迁移第一步不是写 CAS，而是写出哪些状态可以独立，哪些状态必须一起变化。

较稳妥的迁移路线是分层。先保留锁版本作为 reference，写足正确性测试。然后引入只用于指标的 relaxed 原子，例如任务完成数和 CAS 失败数，这一步不改变协议。再实现一个边界明确的 SPSC ring，因为单生产者单消费者让 head 和 tail 各自只有一个写者，协议简单。最后才考虑 MPSC、MPMC 或无锁内存回收。这个路线看起来慢，但它让每一步都有可证明的语义和对照测试。

迁移还要考虑性能目标。低竞争 mutex 可能已经很快，原子版本反而因为 cache line 争夺、CAS 重试和复杂回收变慢。原子优化最适合共享状态小、线性化点清楚、生命周期可控、竞争形状明确的场景。若状态复杂、关闭语义复杂、需要等待和取消，mutex 加条件变量往往更清楚。高水平工程判断不是“无锁更高级”，而是知道什么时候不该无锁。

内存序代码审查 审查原子代码时，应逐个原子变量建立表格。第一列写变量含义，例如计数、状态、指针、索引、版本。第二列写写者和读者。第三列写它是否发布其他普通数据。第四列写内存序。第五列写失败路径和生命周期。若某个原子变量没有清晰含义，只是为了避免数据竞争而存在，通常说明设计还没完成。

看到 relaxed，要问它是否只关心自身数值。若 relaxed 之后读取普通对象字段，就要怀疑发布关系缺失。看到 release，要问哪个普通数据在它之前被发布。看到 acquire，要问它是否确实读到了对应 release 写入的值。看到 CAS，要问成功和失败内存序是否不同，失败后 expected 如何更新，循环体是否有不可回滚副作用。看到原子指针，要问对象何时释放，读者如何保证使用期间对象活着。

审查还要防止“强内存序掩盖设计问题”。把所有操作都写成 seq-cst，可能让小例子更容易运行，但它不会自动解决多变量不变量、丢失唤醒、对象回收和关闭协议。强顺序只能约束操作可见顺序，不能替你定义状态机。若设计不清楚，应该回到 mutex 或更高层同步，而不是用更强内存序硬撑。

线性化点和用户可见语义 并发对象对外应该像每个操作在某个瞬间生效，这个瞬间就是线性化点。mutex 版本通常把线性化点藏在临界区内部，因为临界区串行执行；原子版本必须显式找出来。无锁栈的 push 线性化点可能是 CAS 成功把新节点挂到 head；pop 的线性化点可能是 CAS 成功把 head 移到 next；SPSC ring 的 push 线性化点可能是 release 更新 tail。

线性化点不只是理论概念，它决定用户看到的行为。若 `try_pop` 返回空，表示在它的线性化点上队列为空，而不是整个调用期间都为空；若 `close` 成功，后续 `push` 应失败，但和 `close` 并发的 `push` 是否成功，要看线性化先后。文档写清这点，调用者才能理解边界。很多并发争议来自用户以为“调用先开始就先发生”，而实现按线性化点定义先后。

线性化点还影响测试。并发队列的输出顺序可能不唯一，不能用单一串行顺序硬比。测试应验证存在某个合法线性化顺序，或者验证更具体的不变量：元素不丢不重，关闭后没有新元素，容量不越界，已成功返回的操作符合文档。对于复杂对象，可以使用模型检查或小状态穷举，把线程交错压到可分析范围内。

等待和原子的边界 原子操作适合表达状态变化，但等待是另一个问题。忙等会消耗 CPU；条件变量可以睡眠但需要 mutex；`atomic::wait` 可以围绕单个原子值睡眠；futex 是更底层的机制。选择等待方式时，要看等待时间、唤醒频率、状态复杂度和关闭语义。短等待可以自旋，长等待应睡眠，复杂谓词通常用条件变量更清楚。

自旋等待必须有退出策略。若等待对象的生产者可能被抢占，消费者一直自旋会浪费 CPU，甚至阻碍生产者被调度。常见策略是先短暂自旋，再 yield，再睡眠；或者让运行时知道这是阻塞等待，把 worker 释放给其他任务。等待策略和调度章节直接相连。原子状态本身很快，不代表围绕它等待的系统就快。

原子等待也要处理通知丢失的误解。和条件变量一样，通知不是业务状态。等待者应先读取状态，如果不满足再 wait；被唤醒后重新读取。关闭、取消、失败都应是状态枚举的一部分，并且对应通知。只通知成功状态，不通知关闭状态，会让等待者永久睡眠。工具从 condition variable 换成 atomic wait，谓词思维没有改变。

11.6 原子操作从发布关系开始理解

原子不是“更快的锁”。它首先解决的是数据竞争和跨线程可见性问题，其次才可能用于构建低开销同步结构。最容易理解的场景是发布配置：一个线程构造只读对象，写完所有字段后，用一个原子指针或状态位发布；另一个线程看到发布标志后读取对象。这里真正需要的是“数据先写好，标志后可见；读者先看到标志，再读数据”。这就是 release/acquire 的直觉。

若只把标志做成 relaxed 原子，读者可能看到标志更新，却没有建立读取对象字段的同步关系。若把所有原子都做成 `seq_cst`，语义通常更强，但性能和可读性未必最佳，也掩盖了真正需要的发布关系。内存序章节要让读者用协议思维写代码：哪个写发布了哪些数据，哪个读获取了哪些数据，哪些原子只是统计计数。

Relaxed、release/acquire 和 seq-cst 的角色

Relaxed 适合只关心原子变量自身的场景，例如统计请求数、失败次数、采样计数。它保证单个原子对象不会数据竞争，但不发布其他非原子数据。若一个 relaxed counter 增加只用于最终汇总，没问题；若一个 relaxed flag 用于告诉读者“配置已准备好”，就缺少同步关系。

Release/acquire 建立发布和获取。生产者先写普通数据，再 release store 标志；消费者 acquire load 标志为真后，能看到生产者在 release 前的写入。这个模型适合配置发布、槽位状态、SPSC 队列中的元素可见性。讲解时要画时间线，而不是只列枚举值。

Seq-cst 提供一个所有 seq-cst 操作参与的全局顺序，方便推理，但不是性能万能钥匙。它适合教学和少数需要全局顺序的协议；复杂高性能结构常用更精确的 release/acquire、relaxed 和 fence 组合。教材要保持克制：先让读者写对，再解释如何降低强度；不要一开始就追求最弱内存序。

CAS 循环和线性化点

Compare-exchange 常用于无锁更新。CAS 成功的那个瞬间通常是线性化点：从外部看，操作在这一点生效。失败时，`expected` 会被更新成当前值，循环需要重新计算。CAS 循环里不能随便放外部副作用，例如写文件、释放资源、提交消息；因为 CAS 可能失败重试，副作用会重复发生。副作用必须放在成功后，或者设计成幂等。

ABA 问题也从 CAS 推出来。指针从 A 变到 B 又变回 A，CAS 只比较地址可能误以为没有变化。解决方式包括标签指针、hazard pointer、epoch reclamation 或避免节点地址复用。无锁结构章会深入，这里先建立原则：CAS 比较的是你给它的位模式，不理解对象生命周期。

CAS 还会造成热点。多个线程反复 CAS 同一个 atomic，失败重试会消耗 CPU，并造成 cache line 迁移。若更新频率极高，sharded counter 或锁可能更合适。不要把“无锁”误解成“无等待、无成本、一定更快”。

原子状态机比一堆 flag 更可靠

并发对象常需要状态：Open、Closing、Closed、Committed、Failed。若用多个 atomic bool 表达，容易出现非法组合。用一个 atomic enum 表达主状态，再通过 CAS 转换状态，更容易审查。比如提交协议可以从 Pending CAS 到 Committing，再到 Committed；失败路径从 Pending 到 Failed；已经 Committed 的对象拒绝旧响应。

状态转换要写允许边。CAS 成功意味着当前线程获得某个责任，例如执行关闭、发布结果、唤醒等待者或写 manifest。CAS 失败意味着责任已经被别人拿走，应读取新状态再决定。这个模式会

贯穿线程池 shutdown、无锁队列、分布式 commit。

原子状态机仍然需要普通锁或条件变量配合。原子可以快速判断状态，但等待某个复杂条件时，条件变量更合适；保护复杂容器时，mutex 更清楚。高质量章节要训练读者选择组合，而不是在 mutex 和 atomic 之间做宗教选择。

测试内存序很难，所以设计要保守

内存序 bug 可能只在特定架构、优化级别、压力和时序下出现。x86 较强内存模型可能掩盖一些错误，ARM 更容易暴露。Termux/ARM 环境反而适合提醒读者：不要只在一台机器上验证就认为协议正确。可以用 ThreadSanitizer 检查数据竞争，但它不证明内存序协议完全正确；可以写压力测试和随机调度，但仍然不能穷尽所有交错。

因此项目中原子代码要少而集中。每个原子对象写注释说明它发布什么，使用什么内存序，配套测试覆盖哪些状态。简单统计计数用 relaxed，发布数据用 release/acquire，复杂结构优先用锁实现 reference，再决定是否需要原子优化。这样读者学到的是工程保守性，而不是炫技。

本章项目交付物是三个小协议：配置发布、SPSC 槽位状态、atomic state commit。每个协议都写 happens-before 说明、错误反例和压力测试。后续无锁结构章再把这些协议组合成数据结构。

案例走查：配置发布为何不能只用 relaxed flag

生产者构造 RuntimeConfig，写入多个字段，然后把 ready 设为 true。消费者轮询 ready，看到 true 后读取配置。若 ready 是 relaxed，程序没有建立发布关系；在某些平台和优化下，消费者可能看见 ready，却不能按 C++ 语义保证看到完整配置。把 ready 改成 release store 和 acquire load，才表达“字段写入先于发布，读取发布后可见字段”。

测试这类 bug 很难，因此代码审查和协议说明比压力测试更重要。项目中每个发布协议旁边写一句：release 发布哪些普通写，acquire 后允许读哪些数据。统计 counter 则写明 relaxed，因为不发布其他数据。这样内存序选择从枚举值变成协议文档。

反例也要写：若配置对象发布后又被修改，release/acquire 不能让并发写变安全。发布只适合不可变或有额外同步保护的数据。这个边界能防止读者把原子 flag 当作万能锁。

原子实验的可移植性说明

很多内存序错误在某些平台上很难复现，尤其在强内存模型机器上。教学实验不能把“跑不出错”当作正确证明。更好的方式是用小型 litmus test 解释允许行为，再用 C++ 标准术语说明 happens-before。压力测试作为辅助，而不是证明。

可以设计两个发布配置版本：一个错误 relaxed flag，一个正确 release/acquire。错误版本可能在本机也不失败，但书中要从标准模型说明它没有同步关系。正确版本再用 ThreadSanitizer 和压力测试检查没有数据竞争。这样读者学会区分标准保证和经验现象。

原子代码还要做代码审查。每个 atomic 变量旁边写用途：统计、状态、发布、引用计数、队列索引。不同用途对应不同内存序。若看到一个 atomic 没有说明发布什么或保护什么，就要求补文档。对内存序来说，文档不是装饰，是正确性证明的一部分。

从原子状态到分布式提交

原子状态机和分布式提交有相似结构。单机中 CAS 从 Pending 到 Committed，只有一个线程成功；分布式中 commit table 从未提交到已提交，只有一个 attempt 成功。前者用原子操作保证单进程并发，后者用持久状态和协议保证跨节点并发。这个对应关系能帮助读者理解为什么本册把原子章放在分布式章之前。

项目可以实现一个 CommitState，单机版本用 atomic enum，分布式版本用 driver table。两者都暴露 try_commit，返回 accepted、already committed 或 stale attempt。这样代码结构展示了概念迁移：线性化点在单机是 CAS 成功，在分布式是 manifest commit 成功。

作业要求读者画两张状态机，并指出它们的相同点和不同点。相同点是只允许一次提交，不同点是分布式版本还要处理崩溃、持久化和旧消息。这个比较能让内存模型不再孤立。

综合练习：给每个原子变量写使用合同

项目中每个 atomic 都要有使用合同。合同包含四项：变量表示什么状态，谁写谁读，使用什么内存序，是否发布其他数据。统计计数器可以写“多线程递增，最终汇总，只保证自身原子性，使用 relaxed”。发布 flag 必须写“生产者在 release 前写入配置，消费者 acquire 看到 true 后读取配置字段”。队列槽位状态要写“生产者写元素后 release 更新状态，消费者 acquire 状态后读取元素”。

综合题给读者一段含多个 atomic 的代码，让他补合同并找错误。若某个 relaxed flag 实际发布

指针，就指出缺少 release/acquire；若某个 seq-cst counter 只是统计，就指出过强但不一定错；若 CAS 循环里有日志写入或释放节点，就指出失败重试可能重复副作用。审查重点是语义，不是枚举值背诵。

第二步要求把合同变成测试和注释。测试覆盖发布前后、重复 CAS、状态已提交、取消路径。注释不写长篇，只写发布关系和线性化点。这样未来读者看到代码，知道为什么这里不是默认 seq-cst，也知道为什么不能随便改成 relaxed。

这个题的最终目标，是让原子代码少而可信。项目中能用 mutex 清楚表达的地方，不必强行改 atomic；必须用 atomic 的地方，就把协议写明。会克制使用原子，比到处使用原子更高级。

容易出错的边界：原子变量不会自动建立协议

第一个反例是 atomic flag 发布普通对象但使用 relaxed。变量本身原子，不代表对象字段可见。第二个反例是多个 atomic bool 表达状态，出现 `done=true` 且 `failed=true` 这类非法组合。第三个反例是 CAS 循环中做不可重复副作用。CAS 失败重试后，日志、释放、提交可能发生多次。

第四个反例是用 seq-cst 掩盖设计不清。强内存序可能让小程序看起来正确，但读者仍然不知道哪个写发布哪个读。第五个反例是用 atomic 替代锁保护复杂容器。容器内部多个字段需要一致修改，单个 atomic 不能保护结构不变量。第六个反例是只在 x86 测试，就认为 ARM 上也安全。

本章源码索引要求列出项目中所有 atomic，并分类为计数、状态、发布、索引或引用管理。计数多用 relaxed；状态需要写允许转换；发布需要 release/acquire；索引要说明生产者消费者关系；引用管理要说明回收协议。分类后，很多不必要 atomic 会暴露出来。

读者还要写一个“降级为锁”的版本作为 reference。若 atomic 优化出问题，锁版可以帮助判断语义。无 reference 的原子代码很难调试。系统工程里，先有清楚锁版，再有必要的原子优化，是更稳的路线。

本章核心接口：AtomicProtocolNote

原子章给项目留下 AtomicProtocolNote。每段原子协议旁边都有简短说明：原子变量名，发布者，获取者，内存序，线性化点，禁止的副作用。这个说明可以写成注释，也可以写进设计文档。重点是让审查者知道代码依据什么协议成立。

后续无锁结构和 commit 状态都会使用这种 note。没有 note 的原子代码默认需要补文档或改回锁。这个规则能显著降低维护风险。

从原子协议进入无锁数据结构

原子章只讲小协议：发布、计数、CAS 状态。无锁数据结构把这些小协议组合起来，并立刻引入更难的问题：多个操作并发改变结构、线性化点如何选择、节点什么时候安全回收。若没有原子章的发布关系和 CAS 语义，无锁章会变成看不懂的代码；若学完原子就直接写复杂队列，又会忽视回收和进展保证。

项目里，AtomicProtocolNote 会成为无锁结构审查的基本材料。SPSC ring 的 head/tail 发布、Treiber 栈的 CAS、hazard pointer 的保护发布，都要写成 note。无锁章不是新开一套规则，而是把原子协议放进数据结构生命周期里检查。

11.7 综合案例：配置热更新怎样推导出内存序

原子操作最容易被讲成枚举表：relaxed、acquire、release、acq rel、seq cst。这样的讲法看似完整，实际上会让读者把内存序当作性能旋钮。更好的入口是一个真实需求：系统运行中要热更新配置，worker 不能每次请求都抢全局锁，更新后新请求应看到新配置，旧请求如果已经拿到旧配置也不能悬垂。这个需求会自然推出原子发布、生命周期、版本和审查表。

一、先写业务合同，再选原子操作 配置热更新至少有四条合同。第一，配置对象发布后不可变，worker 只读。第二，worker 获取到某个版本后，在本次请求处理期间该版本必须活着。第三，发布新版本不能阻塞所有正在运行的 worker。第四，监控和日志要能说清某次请求使用了哪个版本。注意这里还没有出现 `memory_order`。如果这些合同不清楚，直接写 atomic 指针就是把生命周期风险藏到代码里。

最朴素的错误版本通常是一个全局裸指针和一个 ready 标志。生产者分配配置，填字段，写指针，再把 ready 设为 true。消费者看到 ready 后读指针。这个版本在 x86 上可能长期正常，因为硬件模型较强，测试时也不一定触发问题；但 C++ 语义上，如果 ready 使用 relaxed，它只保证 ready 自己原子，不保证配置字段通过 ready 发布。更严重的是，发布者如果释放旧配置，消费者可能刚

读到旧指针就发生悬垂。

因此，原子章的第一原则是：内存序只能解决可见性和顺序，不能替你定义所有权。发布一个对象时，要同时回答“读者怎样看到字段”和“读者使用期间对象怎样保持存活”。这两个问题必须分开。

二、只发布一次的对象最适合学习 release 和 acquire 先看最简单情形：配置只发布一次，之后活到进程结束。生产者独占构造对象，写完所有普通字段后，用 `release store` 发布指针。消费者用 `acquire load` 读到非空指针后，再读字段。这个协议的线性关系很短：字段写入在 `release` 之前，`acquire` 读到这个 `release` 写入后，后续字段读取能看到初始化结果。

Listing 11.9 一次性发布不可变配置

```

1 struct RuntimeConfig {
2     std::uint32_t worker_count = 0;
3     std::uint32_t queue_capacity = 0;
4     std::uint64_t version = 0;
5 };
6
7 class OneShotConfig {
8 public:
9     void publish(RuntimeConfig* config) {
10         this->current_.store(config, std::memory_order_release);
11     }
12
13     const RuntimeConfig* wait_current() const {
14         const RuntimeConfig* config = nullptr;
15         while (config == nullptr) {
16             config = this->current_.load(std::memory_order_acquire);
17         }
18         return config;
19     }
20
21 private:
22     std::atomic<const RuntimeConfig*> current_{nullptr};
23 };

```

这个例子刻意不处理释放，因为它的教学目标只有发布关系。教材要把边界写清楚：如果对象会替换或释放，这段代码不够；如果配置发布后还会被修改，这段代码也不够。`release` 和 `acquire` 建立的是“发布前写入对获取者可见”，不是“对象以后怎么改都安全”。

三、热更新需要生命周期协议 配置会热更新时，旧版本不能马上释放。一个实用入门方案是使用不可变对象加共享所有权。发布者构造新的 `std::shared_ptr<const RuntimeConfig>`，再用原子 `shared pointer` 发布；读者加载后持有一份 `shared pointer`，本次请求期间对象保持存活。这样生命周期由引用计数承担，发布可见性由原子 `shared pointer` 的 `acquire/release` 语义承担。

Listing 11.10 热更新配置使用不可变对象和共享所有权

```

1 class ConfigRegistry {
2 public:
3     void publish(std::shared_ptr<const RuntimeConfig> config) {
4         this->current_.store(std::move(config), std::memory_order_release);
5     }
6
7     std::shared_ptr<const RuntimeConfig> snapshot() const {
8         return this->current_.load(std::memory_order_acquire);
9     }
10 };

```

```

9     }
10
11 private:
12     std::atomic<std::shared_ptr<const RuntimeConfig>> current_;
13 };

```

这不是最底层、最快的方案，但教学上很稳。它让读者看到两层责任：原子发布让新对象字段可见，shared pointer 让读者使用期间对象活着。若后续热点路径证明引用计数成本太高，可以再讨论 RCU、epoch 或读者登记表。优化顺序应是先把协议写正确，再根据证据降低成本。

这个方案还有一个重要限制：配置对象必须不可变。如果某个线程拿到 shared pointer 后仍修改字段，那么 shared pointer 只保证生命周期，不保证字段并发访问安全。要么配置字段本身也有同步，要么使用 copy on write 发布新对象。把“共享所有权”等同于“对象线程安全”，是并发代码里很常见的错误。

四、版本号解决审计和迟到观察 热更新系统没有版本号，日志里只能看到地址，排查很困难。版本号可以放进不可变配置对象，随指针一起发布。消费者拿到 snapshot 后记录 **version**，后续所有日志、trace 和错误报告都能说明自己使用哪一版。版本号不是同步工具，而是观测工具；但良好的观测会让同步协议更容易审查。

不要把指针和版本分成两个独立原子随便读。消费者可能读到新指针旧版本，或旧指针新版本。若确实要发布多字段快照，应把多字段放进同一个不可变对象，通过一个原子指针发布；或者用锁保护快照；或者使用 seqlock 这类专门协议并说明适用边界。多个原子变量不会自动组成一致快照，这是本章必须反复强调的原则。

五、relaxed 计数器不能升级成协议变量 配置发布旁边常会有命中计数、更新次数、失败次数。它们可以使用 relaxed，因为读者只关心计数器自身的数值，不借它观察配置字段。可是工程代码会慢慢演化：某个维护者看到 update count 增加后，想当然地认为自己也能看到新配置，于是把统计变量当成协议变量。这个语义漂移很危险。

审查时要把原子变量分成五类：统计计数、发布标志、状态枚举、指针或索引、引用管理。统计计数通常可以 relaxed；发布标志需要说明发布了哪些普通数据；状态枚举需要说明允许转移和线性化点；原子指针需要说明生命周期；引用管理需要说明最后释放时的同步。分类以后，很多含糊写法会自动暴露。

六、CAS 状态机要把成功点和副作用分开 热更新不只发布配置，还可能涉及状态转换：Idle、Building、Published、Failed。若多个线程可能发起更新，就需要用 CAS 抢占更新责任。CAS 成功的线程负责构造和发布；失败的线程读取当前状态后决定等待、返回已有版本或报告冲突。这里 CAS 成功是状态转换的线性化点，但构造对象和写日志不能随便放进 CAS 循环。

Listing 11.11 更新责任使用原子状态表达

```

1 enum class ConfigUpdateState : std::int32_t {
2     Idle,
3     Building,
4     Failed,
5 };
6
7 bool try_start_update(std::atomic<ConfigUpdateState>& state) {
8     ConfigUpdateState expected = ConfigUpdateState::Idle;
9     return state.compare_exchange_strong(
10         expected,
11         ConfigUpdateState::Building,
12         std::memory_order_acq_rel,
13         std::memory_order_acquire);
14 }

```

失败内存序使用 `acquire`，是因为调用者可能想基于观察到的状态读取已经发布的失败信息或当前构建者信息；若失败路径只统计次数，`relaxed` 也可能足够。教材不能只给固定答案，而要讲清失败路径是否读取其他数据。CAS 的两个内存序要从成功路径和失败路径分别推导。

七、测试矩阵以语义证明为中心 原子程序不能靠“跑了一万次没出错”证明正确。测试矩阵应服务语义证明。配置发布至少要测：读者在发布前等待，发布后字段一致；多次热更新后旧 snapshot 仍可读；多个读者并发 snapshot 不出现悬垂；更新失败时状态回到 `Idle` 或 `Failed`；统计计数不被用作配置发布依据。压力测试可以增加信心，但报告必须写清 `happens-before` 关系。

工具也要分层。`ThreadSanitizer` 能发现很多数据竞争，但它不证明线性化点正确，不证明对象回收正确，也不证明最弱内存序足够。模型化的小状态机测试能覆盖状态转移，压力测试能暴露部分时序问题，代码审查则检查每个原子变量的使用合同。并发章节的质量标准，是读者能写出这个使用合同。

八、和 Linux RCU 的连接 Linux RCU 给配置热更新提供了一个成熟方向：读者进入读侧临界区，读取指针并使用对象；更新者发布新指针后等待宽限期，再释放旧对象。它不是简单的 C++ `release/acquire` 教学例子，而是一整套生命周期协议。读内核文档时要分清：指针发布的可见性是一层，读者临界区和宽限期是另一层，回收是第三层。

本书在这里不要求读者实现完整 RCU，但要建立对应关系。`shared pointer` 方案用引用计数解决读者存活；RCU 用读侧临界区和宽限期解决存活；`epoch reclamation` 用全局时代和线程登记解决存活。不同方案成本不同，但都在回答同一件事：读者拿到指针后，谁保证对象不会被释放。只要这个问题没回答，原子指针就没有资格进入生产路径。

11.8 深度案例：从引用计数到 RCU 的生命周期谱系

第 11 章前面已经说明 `release/acquire` 负责发布可见性，但很多并发错误真正死在生命周期。原子指针让另一个线程看到地址，不代表对象在使用期间不会被释放。这里把几种常见生命周期方案放在同一条谱系里：永不释放、`shared pointer`、引用计数、`hazard pointer`、`epoch` 和 RCU。它们不是孤立技巧，而是在不同成本下回答同一个问题。

一、永不释放是合法但有限的工程选择 某些配置对象数量很少，更新次数有限，进程生命周期不长，可以选择发布后永不释放旧版本。这个方案内存浪费，但语义简单：`release/acquire` 保证字段可见，生命周期由进程结束兜底。教学和小工具里，这可能是合理选择。它的优点是没有读者登记、没有引用计数、没有回收停顿。

限制也要写清。若配置频繁更新、对象很大、进程长期运行，永不释放会变成内存泄漏。若对象包含文件描述符、映射内存或外部资源，也不能无限保留。工程判断不是永远追求复杂方案，而是能说明简单方案的适用边界。

二、shared pointer 把生命周期交给引用计数 `std::shared_ptr` 让读者获取 snapshot 后持有引用，最后一个引用释放时销毁对象。它适合读者数量不太夸张、更新频率不高、代码清晰优先的场景。成本是原子引用计数和控制块访问。热点请求路径中，每次 snapshot 都增加减少引用计数，可能形成 `cache line` 争用。

使用 `shared pointer` 时仍要注意对象内部不可变。`shared pointer` 保护的是对象活着，不保护字段并发修改。配置对象应设计成 `const`，更新时创建新对象并发布。若对象内部有可变缓存，就需要单独同步，或者把缓存移出共享配置。

三、手写引用计数最容易漏最后释放同步 手写引用计数看起来只是 `fetch_add` 和 `fetch_sub`。真正困难是获取引用时对象必须还活着，释放最后引用时析构必须看到其他线程释放前的写入。若一个线程从原子裸指针读到对象，再去增加对象内部引用计数，中间对象可能已经被释放。这就是为什么安全获取引用需要额外协议。

成熟 `shared pointer` 控制块把指针和计数协议封装起来。教材不鼓励读者随手自研 `shared pointer`，但要让他们理解引用计数的双重语义：计数更新是原子问题，最后释放是可见性和生命周期问题。只会写 `relaxed` 计数是不够的。

四、Hazard pointer 用公开保护换精确回收 `Hazard pointer` 让读者在解引用前公开声明“我可能使用这个节点”。删除者延迟释放 `retired` 节点，直到扫描确认没有 `hazard slot` 指向它。它避免每次读取都修改对象内部引用计数，适合某些无锁结构。成本是读者写 `hazard slot`、删除者扫描和 `retired list` 内存。

它和 release/acquire 的关系也要讲清。发布 hazard pointer 本身需要让删除者看见；读者发布后要重新确认共享指针没有变化；删除者扫描时要按协议读取 hazard slots。这里的内存序服务生命周期协议，不是孤立选择。若流程错了，内存序再强也救不了。

五、Epoch 用批量回收换低读侧成本 Epoch 方案让读者进入临界区时登记自己处于某个时代，退出时清除。删除者把节点按 epoch 退休，等所有读者都离开旧 epoch 后批量释放。读侧成本通常比 hazard pointer 更低，但如果某个线程长时间不退出，所有旧节点都不能释放，内存峰值会上升。

这使 epoch 和调度联系很紧。线程池 worker 如果在持有 epoch 时阻塞 I/O，会拖住全局回收；协程或任务迁移如果没有正确登记，也会破坏协议。生命周期方案不能脱离运行时。一本系统书要把这种跨章联系讲出来。

六、RCU 把读侧临界区变成系统级合同 RCU 的读者进入读侧临界区后可以快速读取指针；更新者发布新版本后等待宽限期，再释放旧版本。它适合读多写少、读侧必须极快的场景。Linux 内核大量使用 RCU，因为它的运行时和调度器能配合识别 quiescent state。用户态实现也存在，但必须处理线程登记、阻塞和退出。

RCU 让读者看到一个重要原则：高性能读路径的成本不是消失了，而是转移到更新者、回收延迟和系统级协议。若更新很频繁或读者长时间不退出，RCU 也会遇到压力。不要把 RCU 当成“比锁高级”的口号，它是特定读写比例和生命周期模型下的选择。

七、生命周期选择表 选择方案时先问四个问题。对象多大，更新频率多高，读路径能接受多少开销，旧对象能延迟多久释放。对象小且更新少，可以永不释放；读者数量适中且清晰优先，用 shared pointer；无锁结构节点频繁删除且需要精确回收，考虑 hazard pointer；读多写少且能控制线程临界区，考虑 epoch 或 RCU。

这张表不是替代证明。每个方案仍要写使用合同：读者如何进入，如何获取指针，使用期间谁保证对象存活，更新者何时退休旧对象，何时真正释放，线程退出如何清理。生命周期合同写不出来，就不应该优化到原子指针。

八、内存序审查的逐行方法 审查原子代码时，逐行标注每个原子操作的角色。load 是获取发布数据、读取统计值、读取状态，还是尝试保护指针？store 是发布对象、更新指标、释放槽位，还是退出临界区？CAS 成功是否线性化操作，失败是否需要观察其他数据？每一行都要有一句理由。

若发现某个 acquire 后没有读取被发布的数据，可能过强；若发现 relaxed 后读取普通字段，可能缺同步；若发现 release store 前没有要发布的写，可能概念混乱；若发现原子指针没有回收合同，必须停下来补生命周期。审查不靠背枚举，而靠变量在协议里的角色。

九、项目实验：配置注册表三种实现 项目可以实现三种 ConfigRegistry。第一种永不释放旧版本，最简单。第二种 atomic shared pointer，平衡清晰和成本。第三种简化 epoch，读者进入 read section，更新者延迟释放。三种实现使用同一测试：多读者热更新、旧 snapshot 继续使用、更新失败、读者长时间持有、版本日志校验。

报告比较的不只是吞吐。还要记录读取成本、更新成本、内存峰值、旧版本释放延迟、代码复杂度和适用边界。这样原子章节就会从“内存序枚举”升级为“并发生命周期设计”。

11.9 补强案例：内存序错误怎样在代码审查中被发现

内存序 bug 不一定能在测试中复现，代码审查就必须有方法。审查不是问“这里是不是应该 acquire”，而是逐个变量追踪它承担的协议角色。每个原子变量都要能回答：它表示什么，谁写谁读，是否发布普通数据，是否参与生命周期，失败路径是否读取数据。

一、看到 relaxed 后问两个问题 第一个问题：读取者是否只需要这个原子对象自己的值。若是请求计数、CAS 失败次数、采样指标，relaxed 可能成立。第二个问题：读取者看到这个值后，是否读取其他普通对象并认为它们已经初始化。若是，relaxed 很可能错误。这个问题能抓住很多“ready flag 用 relaxed”的 bug。

多个 relaxed 指标也不能组成一致快照。监控页面读取 success、failure、timeout，可能来自不同时间点。作为趋势可以，作为协议判断不行。审查时要防止指标变量被业务逻辑拿去做决定。

二、看到 release 后问发布了什么 Release store 之前应该有需要发布的写入，例如对象字段、buffer 槽位、错误结果。若 release 前没有要发布的数据，它可能只是过强；若有数据但消费者没有

acquire 读同一个同步变量，发布关系仍然不成立。Release 不是空气中的屏障，它需要和对应 acquire 组成关系。

发布还要求顺序正确。先 store ready 再写 payload，即使 ready 是 release，也发布不了后面的写。审查要看代码实际顺序，不只看内存序枚举。

三、看到 acquire 后问读到了谁 Acquire load 只有在读到对应 release 写入或释放序列中的值时，才建立同步。消费者循环等待状态变 Ready，就是为了确保读到发布者写入的 Ready。若 acquire load 只是读到旧值，不能假装已经同步。状态机要让读者知道哪些值携带发布语义。

Acquire 后还要看后续读取了什么。如果没有读取被发布数据，可能只是过强；如果读取了指针对象，要继续问生命周期。Acquire 能看到字段，不保证对象不会被释放。

四、看到 CAS 后问成功和失败 CAS 成功可能是线性化点，失败只是观察当前值。成功内存序和失败内存序常常不同。失败内存序不能 release，因为失败没有写新值；失败后 expected 被更新，调用者是否使用它也要审查。循环体内不能做不可回滚副作用，除非副作用绑定在成功后。

五、审查输出 每段原子代码旁边写简短审查表：变量角色、发布数据、获取数据、线性化点、生命周期方案、失败路径和测试覆盖。表格不需要长，但必须具体。若某项写不出来，代码还没准备好进入高性能路径。

11.10 系统化审查：把每一种原子变量归类

原子章现在必须从零散规则进入系统化审查。一个并发程序里看到 `std::atomic`，不能先问它用了哪个内存序，而要先问它属于哪一类变量。分类决定证明方式，也决定测试方式。把所有 atomic 都当成“线程安全变量”，是内存模型学习中最危险的误区。

一、统计型原子 统计型原子只表达自身数值，例如请求数、失败数、CAS 重试次数、队列近似长度、采样命中次数。它不发布其他普通数据，也不参与对象生命周期。它通常可以使用 relaxed，因为读取者只关心这个计数本身。统计型原子的接口要写明它不是一致快照，多个指标同时读取时可能来自不同时间点。

统计型原子最常见的事是被后续代码升级成协议变量。例如监控线程看到 completed 增加，就认为某个结果对象已经可读；调度器看到 queue size 近似为零，就决定关闭队列；限流器看到失败计数变化，就读取另一个普通字段判断原因。只要读取者把统计值当成观察其他状态的信号，relaxed 合同就被破坏。审查时要搜索统计值的所有调用点，确认它没有被用于发布、关闭、提交和生命周期判断。

统计型原子的性能问题也要讲。所有线程写同一个 relaxed 计数器，仍然会争同一条 cache line。Relaxed 降低的是顺序约束，不消除一致性流量。高频统计应使用分片计数或线程本地计数，周期性汇总。这个结论把原子章和 NUMA、测量、并行 reduce 连起来。

二、发布型原子 发布型原子用于让一个线程看到另一个线程准备好的普通数据。典型例子是 ready 标志、状态枚举、发布指针、队列 tail、槽位 state。它的核心问题是：release 之前写了哪些普通数据，acquire 之后允许读取哪些普通数据。发布型原子的审查必须写出这两个集合。

发布型原子还要求读者读到对应值。消费者 acquire load 看到 Ready，才能读取 payload；如果看到 Empty，就没有同步关系。状态枚举比 bool 更清楚，因为它能表达 Writing、Ready、Closed、Failed 等阶段。多个 bool 容易产生非法组合，也难以说明哪个 bool 携带发布语义。

发布型原子不解决回收。原子指针 release 发布后，消费者 acquire 读到指针，可以看到对象字段；但对象是否活着，仍然由其他协议保证。发布型原子和生命周期型协议常常一起出现，不能混为一谈。

三、线性化型原子 线性化型原子用于定义并发对象某个操作生效的瞬间。CAS 成功通常是线性化点，例如无锁栈 push 把新节点挂到 head，状态机从 Pending 变成 Running，提交表从 Empty 变成 Committed。审查线性化型原子时，要写成功路径、失败路径、失败后 expected 如何使用，以及循环体里是否有不可回滚副作用。

线性化点决定用户可见语义。并发 `try_pop` 返回空，只能说明在线性化点队列为空，不说明调用期间一直为空。并发 close 和 push 的先后也由线性化点决定，而不是由函数开始时间决定。文档若不写线性化语义，用户会把现实时间误当成对象顺序。

线性化型原子常常不需要所有操作都是 seq cst。关键是建立对象语义需要的顺序。状态 CAS 成功可能需要 acq rel，失败可能只需要 acquire 或 relaxed。选择要从成功后发布什么、失败后读取什

么推导。写不出理由时，先用锁版本表达语义，再考虑原子优化。

四、生命周期型原子 生命周期型原子保护对象何时释放。引用计数、hazard pointer slot、epoch 活跃标记、RCU 读侧状态，都属于这一类。它们的错误往往不是结果算错，而是偶发 use after free。测试很难稳定复现，因此协议审查更重要。

引用计数要区分已有引用复制和从共享结构获取新引用。已有引用复制时，对象已经活着，计数增加通常不负责发布对象字段；从共享结构中获取引用时，必须保证对象在增加计数之前没有被释放。最后一个引用释放时，析构需要看到其他线程释放前的写入。成熟 shared pointer 把这些复杂性封装起来，手写引用计数必须非常谨慎。

Hazard pointer 和 epoch 不是替代内存序，而是生命周期协议。Hazard pointer 要先发布保护指针，再确认共享指针仍然相同；epoch 要保证读者临界区期间旧节点不释放。每一步都涉及原子读写，但内存序只是服务整体协议。若协议流程错了，所有操作都写 seq cst 也不能保证安全。

五、等待型原子 C++20 的 `atomic::wait` 让线程等待某个原子值变化。它适合状态简单、等待条件就是单个原子值的场景，例如任务状态从 Pending 变为 Completed。等待型原子仍然需要谓词思维：wait 返回后重新检查状态；关闭、取消、失败都要 notify；状态转换要有 release/acquire 或等价同步。

不要用 atomic wait 隐藏复杂不变量。若等待条件涉及队列非空、队列关闭、错误状态、容量和多个字段，条件变量加 mutex 更清楚。等待型原子是工具，不是条件变量的全面替代。它减少某些场景的锁开销，但要求状态机足够集中。

六、审查表如何落到代码 项目中的每个原子变量都写一条 `AtomicProtocolNote`。字段包括 name、category、writers、readers、published data、memory order、linearization point、lifetime owner、failure behavior、tests。示例如下：`ready` 是发布型原子，producer release store 之前写 payload，consumer acquire load 看到 Ready 后读 payload；`cas_failures` 是统计型原子，不发布任何数据，relaxed；`head` 是线性化型原子，CAS 成功是 pop 线性化点；`active_epoch` 是生命周期型原子，保护 retired nodes。

这张表不需要很长，但必须具体。若 published data 写成“全部数据”，说明没有想清；若 lifetime owner 写成“调用者保证”，但接口没有表达，说明风险被转移到文档之外；若 failure behavior 为空，CAS 失败和关闭路径很可能有 bug。高质量原子代码的特征不是内存序很花，而是协议很短、变量很少、理由很清楚。

七、原子代码的测试组合 测试分四层。第一，单线程状态机测试，覆盖所有状态转换。第二，模型化并发测试，用小容量、小对象和受控交错暴露线性化错误。第三，压力测试，在多线程、多核心、不同输入下跑长时间。第四，工具测试，用 ThreadSanitizer 查数据竞争，用 ASan 查回收错误。四层都不能替代协议证明，但能发现不同类型问题。

原子发布测试要有错误反例。把 release/acquire 改成 relaxed，测试可能仍然通过；书中要解释为什么这不是正确性证据。生命周期测试要强制延迟读者和快速回收旧对象，放大 use after free。CAS 测试要统计失败次数和重试时间，证明高竞争下成本没有被隐藏。这样读者会知道并发测试不是跑很多次这么简单。

八、从原子章进入无锁章 无锁数据结构只是把这些原子类别组合起来。SPSC ring 有发布型 head/tail；Treiber 栈有线性化型 head CAS 和生命周期型回收；MPMC 队列有槽位状态发布和索引线性化；hazard pointer 有生命周期型 hazard slots。若原子章分类不清，无锁章会变成难懂代码。若分类清楚，无锁结构就能被拆成几个可审查协议。

11.11 补充代码审查：SPSC ring 的内存序逐行解释

SPSC ring 是连接原子章和无锁章的最好代码审查对象。它足够简单，因为 head 只有消费者写，tail 只有生产者写；它又足够真实，因为元素发布、槽位复用、空满判断和关闭都需要清楚内存序。

一、生产者发布元素 生产者先检查空间，再在 tail 对应槽位构造元素，最后 release store 新 tail。Release 发布的是槽位内容。消费者 acquire load tail 后，如果看到 tail 已前进，就能读取槽位元素。若先更新 tail 再写元素，消费者可能读到未构造内容。若 tail store relaxed，语言层面缺少发布关系。

二、消费者释放槽位 消费者 acquire load tail 判断是否有元素，读取并移动元素，析构槽位，然后 release store 新 head。Release 发布的是槽位已可复用。生产者 acquire load head 后，看到空

间可用，才能重新构造该槽位。这里 head 的方向和 tail 对称：tail 发布数据可读，head 发布空间可写。

三、为什么单写者简化协议 Tail 只有生产者写，所以生产者本地 tail 可以缓存；head 只有消费者写，所以消费者本地 head 可以缓存。另一方读取对方发布的索引时需要 acquire。若变成 MPSC，多个生产者竞争 tail，就需要 CAS 或 fetch add；若变成 MPMC，槽位状态也要更复杂。SPSC 的简单来自角色限制，不是来自 atomic 魔法。

四、关闭和等待 纯 SPSC ring 只解决数据传递，不解决长时间等待。若队列空，消费者可以忙等、sleep、atomic wait 或配合条件变量。关闭状态应单独表达，生产者 close 后通知消费者，消费者在空且 closed 时退出。不要把 tail 等于 head 同时解释为空和关闭。状态语义要分开。

五、审查结论 SPSC ring 的每个内存序都能写成一句话：tail release 发布元素，tail acquire 获取元素；head release 发布空间，head acquire 获取空间。若某个原子操作写不出这种句子，就说明代码可能过强、过弱或语义不清。这个练习应成为读者审查所有原子协议的模板。

11.12 完整案例：无锁最大值、配置发布和任务完成标志的对比

同样是 atomic，三个小案例的语义完全不同。无锁最大值只关心一个数值的线性化更新；配置发布要让读者看到普通字段；任务完成标志要发布结果、异常和唤醒等待者。把它们放在一起比较，读者才能真正理解“按语义选择内存序”。

一、无锁最大值只维护一个原子数值 无锁最大值的状态只有一个 `std::atomic<std::int64_t>`。调用者提交 candidate，若 candidate 不大于当前值，什么都不做；若更大，就 CAS 更新。它不发布其他普通数据，成功点就是最大值改变的线性化点。这里 relaxed load 常可作为初始观察，CAS 成功内存序甚至可以 relaxed，前提是没有其他数据要发布。若成功后调用者要基于“最大值更新”发布外部事件，副作用应放在 CAS 成功之后，而不是循环内部。

这个例子训练读者区分“原子值自身正确”和“借原子同步其他状态”。最大值只要求自身单调，和对象字段、生命周期无关。把它写成 seq cst 也正确但可能过强；把它作为配置 ready flag 就完全错误，因为 ready flag 承担发布 payload 的责任。

二、配置发布同步普通字段 配置发布有两个对象：普通配置字段和发布指针或状态。生产者构造配置，写字段，再 release 发布指针。消费者 acquire 读到指针后读取字段。这里原子变量的值只是入口，真正需要同步的是配置对象里的普通字段。若改成 relaxed，指针本身仍然原子，但字段可见性没有合同。

配置案例还要求生命周期。一次性发布可以让对象活到进程结束；热更新可以用 shared pointer、epoch 或 RCU。内存序解决“看见初始化”，生命周期协议解决“使用期间活着”。任何把这两件事混在一起的解释，都容易误导读者。

三、任务完成标志发布结果和异常 任务完成状态比配置发布更复杂。Worker 写入 result 或 exception，然后把 state 设为 Completed 或 Failed，并通知等待者。等待者看到 Completed 后读取 result，看到 Failed 后读取 exception。这里 state 的 release/acquire 发布的是结果对象或异常对象。若等待者只用 relaxed 读 state，再读 result，就是典型发布错误。

任务完成还涉及等待。若使用条件变量，锁和 unlock/lock 建立同步；若使用 atomic wait，store state 后要 notify，等待者醒来后 acquire 读取状态并检查结果。无论工具是什么，语义都是“结果先写好，状态再发布，等待者看到状态后读结果”。

四、三者对比 无锁最大值：原子变量自身就是全部状态；配置发布：原子指针发布另一个对象；任务完成：原子状态发布结果并唤醒等待者。第一类可以讨论 relaxed；第二类需要 release/acquire 和生命周期；第三类还需要等待协议和异常传播。把三者都写成 atomic，不代表它们属于同一种同步问题。

审查时可以用一句话分类。若句子是“这个变量只统计次数”，它是统计型。若句子是“看到这个状态后可以读某个 payload”，它是发布型。若句子是“CAS 成功表示操作生效”，它是线性化型。若句子是“读者声明自己正在使用节点”，它是生命周期型。分类越早，内存序选择越不容易错。

五、三组测试 最大值测试关注单调性和 CAS 失败次数；配置发布测试关注字段一致和旧版本存活；任务完成测试关注结果可见、异常传播、等待唤醒和取消。不要用一套“多线程压力测试”笼统覆盖所有 atomic。不同语义需要不同测试 oracle。

例如任务完成测试要故意让等待者先 wait，再让 worker 写 result 和 state；也要让 worker 先完成，再让等待者后来读取；还要测试 Failed 和 Cancelled。配置测试要让读者持有旧 snapshot，同时发布新 snapshot 并释放候选旧对象。最大值测试则要验证最终值等于所有 candidate 的最大值，且循环没有外部副作用重复。

六、工程经验 业务代码中，能用 mutex 清楚表达多字段不变量，就不要为了“高性能”拆成多个 atomic。Atomic 最适合状态小、协议短、线性化点清楚的地方。复杂对象用锁，不是低级；无法证明的弱内存序，才是危险。原子章最终要培养的是克制和证明能力。

11.13 原子专题项目：实现一个可审查的任务完成对象

为了把本章所有概念落到一个完整对象，本节设计一个任务完成对象。它不是完整 future 库，却覆盖原子状态、结果发布、异常发布、等待、取消、生命周期和审查。这个对象比单个 ready flag 复杂，但比无锁队列简单，适合作为内存序综合训练。

一、对象合同 任务完成对象有五种状态：Pending、Running、Completed、Failed、Cancelled。创建后为 Pending；worker 成功接管后变 Running；写入结果后变 Completed；写入异常后变 Failed；尚未运行或协作取消时变 Cancelled。等待者可以等待任意终态，看到 Completed 后读取结果，看到 Failed 后读取异常摘要，看到 Cancelled 后知道没有结果。

这个合同先于内存序。若合同不写，代码会在 bool done、bool failed、optional result、exception pointer 之间摇摆。多个 bool 容易出现 done 和 failed 同时为真、cancelled 后又 completed 等非法组合。单个状态枚举让状态空间集中，CAS 转换也更容易审查。

二、为什么状态发布结果 Worker 写 result 后，必须让等待者看到 result。最自然的发布点是把 state 从 Running 写成 Completed。这个 store 应带 release 语义。等待者用 acquire 读取到 Completed 后，才读取 result。同理，写 exception 后 release 写 Failed，等待者 acquire 看到 Failed 后读取 exception。状态不是普通指标，它发布 payload。

如果 state 使用 relaxed，等待者可能看到 Completed，却没有语言层面保证看到 result 写入。这个错误在强内存模型机器上可能不复现，但 C++ 语义不成立。任务完成对象是 ready flag 的工程版本，能让读者看到发布关系在真实接口中的位置。

三、Running 转换的线性化点 多个 worker 不应同时执行同一个任务。谁把 Pending CAS 成 Running，谁获得执行责任。CAS 成功是接管任务的线性化点；失败者读取当前状态，决定返回 already running、already completed 或 cancelled。这里 CAS 成功不发布结果，它只是转移责任，常用 acq rel 或 acquire/release 组合取决于是否读取前序取消信息。

如果 cancel 与 start 并发，也由 CAS 顺序决定谁赢。Cancel 从 Pending CAS 到 Cancelled 成功，worker start 就失败；worker 从 Pending CAS 到 Running 成功，cancel 只能请求协作取消，而不能直接把 Running 改成 Cancelled。这个语义必须写清，否则用户会以为 cancel 一定能停止正在运行的任务。

四、等待不能只忙等 等待者可以短暂自旋，但长期等待应使用条件变量、semaphore 或 `atomic::wait`。若使用 atomic wait，等待者先读取 state，若不是终态，就 wait 当前值；worker 写终态后 notify all。Wait 返回后必须重新读取状态，因为可能虚假唤醒，也可能状态变化不是自己等待的终态。

等待协议还要处理取消和销毁。若对象析构时仍有等待者，行为必须定义。较简单的合同是：任务完成对象由 shared state 管理，最后一个 future 和 runtime 引用释放后销毁；析构前所有等待者已经离开。不要让等待者拿着裸指针等待一个可能被释放的对象。

五、结果存储和生命周期 结果对象可能很大，也可能构造失败。Worker 在发布 Completed 前，应先在 shared state 内构造结果。构造成功后 release store state。若构造抛异常，应写入异常摘要并发布 Failed。等待者 acquire 看到终态后读取对应存储。结果存储的生命周期由 shared state 覆盖，直到等待者读取完成。

若结果是指向外部 buffer 的 view，还要保证 buffer 生命周期。更稳的教学版本让 shared state 拥有结果值或引用计数 block。这样任务完成对象的发布语义和生命周期语义都能在同一对象中审查。

六、取消分两种 Pending 状态的取消可以直接 CAS 到 Cancelled，并通知等待者。Running 状态的取消不能强制释放 worker 正在使用的资源，只能设置 cancel request，让任务函数协作检查。

若任务函数响应取消，写入 Cancelled 或 Failed；若任务函数忽略取消并成功完成，合同要定义是否允许 Completed。许多运行时选择“运行后取消只是请求，最终状态由任务自己提交”。

取消状态也发布信息。等待者看到 Cancelled 后，需要知道是用户取消、shutdown cancel、前驱失败取消，还是超时取消。可以把 cancel reason 写入 shared state，然后 release 发布 Cancelled。等待者 acquire 后读取 reason。这样取消不是一个丢失上下文的 bool。

七、异常传播 异常对象本身可能复杂，跨线程传播通常用 `std::exception_ptr` 或结构化错误摘要。Worker 捕获异常，写入 shared state，再 release 发布 Failed。等待者 acquire 后读取并重新抛出或返回错误。不要在 worker 里只打印日志然后把任务标记 Completed，也不要让未捕获异常逃出 worker 线程导致进程终止。

异常传播和结果发布是同一个模式：先写 payload，再写状态。区别只在 payload 类型。把它们放在同一状态机中，可以避免 done、has exception、cancelled 三个标志交错。

八、状态转换表 任务完成对象的允许转换可以写成表。Pending 到 Running，由 worker start CAS 完成。Pending 到 Cancelled，由取消者 CAS 完成。Running 到 Completed，由 worker 写结果后 store 完成。Running 到 Failed，由 worker 写异常后 store 完成。Running 到 Cancelled，由 worker 接受取消后 store 完成。任何终态到其他状态都禁止。

这张表服务测试。每个允许转换写一个测试，每个禁止转换写一个反例测试。比如 Completed 后 cancel 应返回 already finished；Cancelled 后 worker start 应失败；Failed 后等待者读取异常；Running 时 cancel request 不应释放 shared state。状态表比自然语言更不容易漏边界。

九、内存序选择 Pending 到 Running 的 CAS 可以使用 acquire 或 acq_rel，取决于 Running 线程是否需要看到取消者或提交者在 Pending 前写入的内容。若 Pending 没有 payload，成功 acquire 足够；若 CAS 同时发布某些责任信息，可能需要 release。Running 到 Completed 的 store 必须 release，因为发布 result。等待者读取终态必须 acquire。读取 Pending 做轮询可以 acquire 或 relaxed 加后续 acquire，但教学版本建议直接 acquire，减少混淆。

状态查询接口如果只是监控展示，可以 relaxed 读取 state，但它不能随后读取 result。为了避免误用，接口可以分成 `is_ready_relaxed` 和 `wait_and_get`。命名让语义可见，比隐藏在实现里更安全。

十、代码草图

Listing 11.12 任务完成状态的最小草图

```

1 enum class CompletionState : std::int32_t {
2     Pending,
3     Running,
4     Completed,
5     Failed,
6     Cancelled,
7 };
8
9 struct CompletionSharedState {
10     std::atomic<CompletionState> state{CompletionState::Pending};
11     std::int64_t result = 0;
12     std::uint32_t error_code = 0;
13     std::uint32_t cancel_reason = 0;
14 };
15
16 bool try_start(CompletionSharedState& shared) {
17     CompletionState expected = CompletionState::Pending;
18     return shared.state.compare_exchange_strong(
19         expected,
20         CompletionState::Running,
21         std::memory_order_acquire,
22         std::memory_order_acquire);

```



```

23 }
24
25 void publish_result(CompletionSharedState& shared, std::int64_t value) {
26     shared.result = value;
27     shared.state.store(CompletionState::Completed, std::memory_order_release);
28     shared.state.notify_all();
29 }
30
31 std::int64_t wait_result(CompletionSharedState& shared) {
32     CompletionState observed = shared.state.load(std::memory_order_acquire);
33     while (observed != CompletionState::Completed &&
34            observed != CompletionState::Failed &&
35            observed != CompletionState::Cancelled) {
36         shared.state.wait(observed, std::memory_order_acquire);
37         observed = shared.state.load(std::memory_order_acquire);
38     }
39     if (observed == CompletionState::Completed) {
40         return shared.result;
41     }
42     return -1;
43 }

```

这段代码仍是草图，真实实现要处理结果类型、异常对象、shared state 生命周期和取消请求。它的价值在于每个内存序都有理由：start 获取执行责任，publish release 发布 result，wait acquire 获取 result，notify 唤醒等待者。代码中没有 relaxed，是为了教学清晰。优化版本可以把监控读取消降为 relaxed，但不能改变发布路径。

十一、测试矩阵 测试一，worker start 成功，写 result，等待者读到 result。测试二，等待者先 wait，worker 后 publish，等待者被唤醒。测试三，publish 先发生，等待者后调用 wait，不能挂起。测试四，两个 worker 同时 start，只有一个成功。测试五，Pending 取消成功，worker start 失败。测试六，Running 取消只设置请求，最终由 worker 发布 Cancelled。测试七，异常路径发布 Failed，等待者读 error code。

压力测试增加多个等待者、多个取消者和重复查询。工具测试使用 ThreadSanitizer 确认没有普通数据竞争。审查测试故意把 `publish_result` 的 release 改成 relaxed，说明测试可能仍过但语义不成立。这个反例要写进教材，提醒读者测试和语言模型是两种证据。

十二、和线程池、任务图连接 线程池 future 的 shared state 就是这个对象的扩展版。任务图中的节点完成后，发布 result 或 error，并唤醒 continuation。分布式 driver 中 attempt 状态也类似：Running 到 Committed 发布 block metadata，Failed 发布错误原因，Cancelled 发布取消来源。原子状态机的思想跨越单机和分布式。

学完这个案例后，读者应能审查任何“done flag + result”的代码。只要看到 done 是 relaxed，或者 result 生命周期不清，或者 cancel 直接销毁 running 任务，就能指出具体风险。原子章的目标不是背 API，而是培养这种审查能力。

11.14 原子专题项目：指标、协议和生命周期三账本

本章最后再给项目留下三张账本。第一张是指标账本，记录所有 relaxed 统计变量和它们的非协议用途。第二张是协议账本，记录所有发布型和线性化型原子。第三张是生命周期账本，记录所有原子指针、共享对象、retired node 和回收方案。三张账本分开，能防止“一个 atomic 管全部”的混乱。

一、指标账本 指标账本的字段包括变量名、递增位置、读取位置、是否近似、是否允许丢失实时一致性、是否用于决策。最后一列非常关键：若某个指标被用于调度、关闭、提交或读取其他对象，它就不再是普通指标。此时要把它移到协议账本，重新设计内存序。

例如 `completed_count` 如果只显示完成数量，可以 relaxed；如果 driver 用它判断所有任务

完成，就必须和任务状态表或锁保护的不变量结合。一个数字是否等于任务完成数，不等于所有任务结果已经发布并可读。账本让这种差异暴露出来。

二、协议账本 协议账本记录状态变量和线性化点。每条记录写：状态枚举，允许转换，成功线程获得什么责任，失败线程如何处理，release 发布哪些数据，acquire 后读取哪些数据。队列槽位、任务状态、配置指针、commit 状态都属于这里。

协议账本帮助做代码 review。看到一个状态转换没有失败路径，说明 CAS 失败会被忽略；看到一个 release 发布的数据集合为空，可能过强或设计不清；看到 acquire 后读取了未列出的普通字段，说明账本漏了或代码错了。账本和代码互相校验。

三、生命周期账本 生命周期账本记录对象从创建、发布、获取、使用、退休到释放的过程。每类对象写清所有者和读者：配置对象由 registry 发布，读者持 shared pointer；无锁节点由栈发布，pop 后退休，hazard 或 epoch 保护；任务 shared state 由 future 和 runtime 共同持有，终态后最后引用释放。

生命周期账本最能发现隐藏风险。若某个对象只有发布，没有退休；或只有退休，没有等待读者离开；或读者拿到裸指针，没有任何保护；这些都是 use-after-free 候选。内存序无法替代这张账本。

四、账本进入 CI 项目可以把账本写成文档，也可以写成注释和测试名。每新增一个 atomic，要求更新账本；每修改内存序，要求说明账本哪一项改变；每删除锁改 atomic，要求先补协议账本。CI 不一定自动验证所有语义，但 review 可以检查账本是否完整。

五、最终验收 原子章的验收不是写出最弱内存序，而是能解释每个 atomic 为什么存在。若一个 atomic 只是因为“多线程要安全”，说明设计还没完成。若能说“这是统计型，只看自身值，所以 relaxed；这是发布型，release 发布 payload，acquire 获取 payload；这是生命周期型，用 epoch 保护 retired node”，才算合格。

11.15 常见错误与修复路线

原子操作的错误有一个特点：代码通常很短，审查却很难。短代码会给人错觉，仿佛只要把普通变量换成 atomic，程序就自然线程安全。实际上，atomic 只把某个内存位置的并发访问变成有定义，真正的业务不变量、对象生命周期和等待语义仍然要由设计承担。本节列出项目中最常见的错误，并给出修复路线。每个错误都从具体现象开始，而不是从枚举名开始。

一、把 ready flag 当成锁 错误写法是生产者写多个字段，然后把 ready 设为 true；消费者看到 ready 后读字段。开发者认为 ready 是 atomic，所以字段也安全了。这里的问题有两层。第一，若 ready 使用 relaxed，消费者没有通过 ready 获取字段写入。第二，即使 ready 使用 release/acquire，字段发布后也不允许生产者继续无同步修改字段。Ready flag 只适合一次性发布不可变数据，不能替代锁保护可变对象。

修复路线先看对象是否会变。如果对象发布后不再修改，使用 release store 和 acquire load，并把对象类型设计成只读。若对象会修改，用 mutex 保护字段，或者每次修改创建新版本再发布。若对象会释放，补生命周期协议。不要在一个可变对象上加 ready flag 后继续并发修改字段。这个修复让读者看到：发布、修改、回收是三件事。

二、把多个 atomic bool 拼成状态机 错误写法是 done、failed、cancelled 三个 atomic bool 同时存在。某些交错下，等待者可能看到 done 和 failed 都为真，或 cancelled 后又看到 done。多个 bool 的组合状态没有被定义，测试只覆盖常见路径时很难发现。

修复路线是使用一个 atomic enum 表达主状态，并列允许转换。Pending、Running、Completed、Failed、Cancelled 互斥。CAS 或 store 改变状态时，代码明确当前线程获得什么责任。状态枚举不能表达所有数据，但它能消除非法组合。若状态变化伴随 result 或 error payload，使用 release/acquire 发布 payload。若状态复杂到一个枚举也不够，就回到 mutex 保护完整状态。

三、把 relaxed 指标用于关闭判断 错误写法是 worker 完成任务后 relaxed 增加 completed count，driver 看到 completed count 等于 task count 就关闭系统。这个计数器本来只是指标，现在被用于协议决策。它不能保证所有任务结果已经发布，不能保证错误状态已经写入，也不能保证队列里没有迟到 completion。

修复路线是把任务完成判断放到任务状态表或受锁保护的不变量中。每个 task 有明确终态，driver 在状态机里统计终态数量。监控用 completed count 可以保留，但只能展示趋势。协议状态和监控指标分开，能避免 relaxed 变量语义漂移。若确实要用原子计数判断完成，也要证明每个任务在

release 发布结果后才增加计数, driver acquire 读取计数后还要有办法读取对应结果; 通常这比状态表更绕。

四、CAS 循环里做外部副作用 错误写法是在 CAS 循环里写日志、提交文件、释放节点或发送消息。CAS 失败会重试, 循环体可能执行多次。低竞争测试中 CAS 几乎总成功, 副作用只发生一次; 高竞争下副作用重复, 系统出现重复日志、重复提交或 use after free。

修复路线是把循环分成准备、尝试和提交三段。准备阶段只计算候选值和局部对象; CAS 成功后, 才执行外部副作用; CAS 失败只更新 expected 并重试。若副作用必须在循环中发生, 它本身必须幂等, 并且有独立的 request id 或 commit table 去重。原子线性化点和外部提交点要对齐, 不能让副作用游离在线性化点之外。

五、原子指针没有回收协议 错误写法是 atomic 指针指向当前对象, 更新者发布新对象后立即 delete 旧对象。读者可能刚 acquire 读到旧指针, 更新者就释放它。这个错误在测试中不稳定, 因为内存可能还没被复用; 一旦复用, 就会出现难以定位的崩溃。

修复路线按成本选择。更新很少、对象小, 可以暂时不释放旧对象。读者需要清晰生命周期, 可以用 atomic shared pointer。读路径极热、更新少, 可以考虑 RCU 或 epoch。节点级无锁结构可以用 hazard pointer。无论选择哪种, 都要写清读者如何进入保护、更新者如何退休旧对象、何时真正释放。只写 acquire load 指针不够。

六、seq cst 掩盖设计问题 错误写法是所有原子都使用 seq cst, 然后认为程序一定正确。Seq cst 提供更强全局顺序, 但不保护多个普通字段的不变量, 不管理生命周期, 不定义关闭语义, 也不让不可回滚副作用自动幂等。它能减少某些重排推理, 却不能替代协议设计。

修复路线不是把 seq cst 全部改弱, 而是先写协议。哪些变量是统计, 哪些是发布, 哪些是状态机, 哪些涉及回收。对复杂不变量, 使用锁; 对发布关系, 使用 release/acquire; 对纯指标, 使用 relaxed; 对教学或非热点路径, 可以保留 seq cst 简化推理。强顺序应是有意识的选择, 而不是设计不清的遮羞布。

七、用 atomic 修复容器并发访问 错误写法是把容器 size 或 head 改成 atomic, 就认为 `std::vector`、`std::queue` 或自写 ring 多线程安全。标准容器内部有多个字段和迭代器不变量, 单个 atomic 不能保护整体结构。两个线程同时 push vector, 哪怕 size 是 atomic, 也会破坏容量、元素构造和重分配。

修复路线是用 mutex 保护容器操作, 或者设计专门并发数据结构。SPSC ring 依赖单生产者单消费者限制; MPMC 队列需要槽位状态和索引协议; 无锁结构需要回收协议。不要把通用容器局部原子化。容器并发安全是结构设计, 不是字段类型替换。

八、忽略失败内存序 错误写法是 CAS 只写成功内存序, 或者失败内存序随便写 seq cst。失败路径也是语义的一部分。CAS 失败时没有写新值, 但会读取当前值并更新 expected。调用者如果根据失败读到的状态读取其他数据, 失败内存序可能需要 acquire; 如果失败只用于重试数值, relaxed 可能足够。失败内存序不能是 release, 因为失败没有发布。

修复路线是为 CAS 写两段注释: 成功后发布或获取什么, 失败后观察什么。若失败路径只重新计算 candidate, 写 relaxed; 若失败后看到 Completed 并读取 result, 写 acquire; 若成功要同时读取旧状态并发布新状态, 写 acq rel。不要让失败路径成为未审查角落。

九、把测试通过当作内存模型证明 错误写法是在一台机器上压力测试很多轮, 然后断言 relaxed 发布正确。并发测试能增加信心, 不能证明语言模型保证。强内存模型、编译器当前优化、时序偶然都可能掩盖问题。尤其是发布和生命周期错误, 常常只在换平台、换优化级别或内存复用后出现。

修复路线是先做语义证明, 再做测试。证明写 happens before、线性化点和生命周期。测试覆盖状态转换、压力交错和工具检测。报告中明确区分“标准保证”“压力测试未复现”“当前平台观察到”。这种表述比“跑了很多次没问题”更专业。

十、把原子和等待混为一谈 错误写法是使用 atomic 状态后长期忙等, 认为无锁就应该一直自旋。Busy wait 会浪费核心, 甚至阻碍生产者运行。在移动设备上, 还会增加功耗和温控降频。原子状态只说明状态如何读写, 不说明等待策略。

修复路线是按等待时间分层。极短等待可以自旋; 超过阈值让出或使用 `atomic::wait`; 复杂条件使用条件变量; I/O 等待交给异步完成; 任务依赖等待交给 continuation。等待策略要有指标: 自旋次数、睡眠次数、唤醒原因、等待时间。这样原子代码既正确, 也不会把 CPU 烧在空转上。

十一、忽略对象构造和析构 错误写法是在无锁槽位中直接覆盖对象内存，未处理构造和析构。对于平凡整数可能没事，对于包含 string、vector、文件句柄或引用计数对象的类型，会泄漏或双重析构。原子发布槽位只解决可见性，不解决 C++ 对象生命周期。

修复路线是限制元素类型为 trivially copyable，或使用 placement new 和显式析构，或让队列存储智能指针或移动对象。SPSC ring 教学时可以先限制类型，生产实现必须写清构造、移动、销毁和异常保证。C++ 对象模型和内存模型同时存在，不能只看原子。

十二、审查清单 最终审查时，对每个 atomic 问十二个问题。变量类别是什么；谁写谁读；是否发布普通数据；是否读取被发布数据；是否有线性化点；失败路径是什么；是否有外部副作用；对象生命周期谁保证；是否需要等待和通知；是否会形成 cache line 热点；测试如何覆盖；能否用锁更清楚地表达。若这些问题回答完整，内存序选择通常自然浮现。若回答不完整，继续调枚举值没有意义。

这张清单是本章留给项目的最终工具。读者以后写线程池、无锁队列、配置热更新、分布式提交状态，都可以先用它审一遍。能审清楚，再谈性能；审不清楚，先回到锁和状态机。

11.16 原子专题复盘：从语言模型到工程模型

本章最后需要把 C++ 语言模型和工程模型明确分开。语言模型告诉我们哪些行为有定义，哪些同步关系成立；工程模型告诉我们对象属于谁，状态何时提交，内存何时释放，性能瓶颈在哪里。只懂语言模型，容易写出语义正确但生命周期缺失的代码；只懂工程直觉，容易写出在当前机器上能跑但不可移植的代码。

一、语言模型给出最低正确性 C++ 规定数据竞争导致未定义行为，规定 release 和 acquire 怎样建立同步，规定 relaxed 只保证原子对象自身的原子性，规定 CAS 成功和失败的语义。语言模型是跨平台底线。代码在 x86 上跑通、在某个优化级别下没出错、在压力测试中没复现，都不能替代语言模型保证。

因此教材示例必须按语言模型写正确。Ready flag 发布 payload，就写 release/acquire；统计计数不发布数据，才讨论 relaxed；多个普通字段组成不变量，就用锁或明确状态机。不要把“这个硬件实际上不会乱序”写成程序依据。硬件经验可以解释成本，不能替代 C++ 语义。

二、工程模型补上所有权和恢复 语言模型不会告诉你对象何时释放，任务失败后如何恢复，重复消息是否幂等，队列关闭后剩余元素怎么办。这些属于工程模型。一个原子指针 release/acquire 发布正确，仍可能在旧对象释放后被读者使用。一个 CAS 状态机线性化正确，仍可能在成功前写了不可回滚日志。一个 relaxed 计数器语言上正确，仍可能被业务误用成协议信号。

工程模型要写成对象合同。配置注册表说明旧版本回收；任务完成对象说明 shared state 生命周期；无锁栈说明 retired node 回收；分布式 commit 说明 attempt 去重。内存序只是这些合同里的一个字段，不是全部合同。

三、性能模型解释为什么不总用最强顺序 如果所有原子都写 seq cst，很多小程序更容易推理，但性能和可扩展性可能受限，也可能掩盖设计问题。强顺序不能让多个原子组成业务不变量，不能防止对象释放，不能解决队列关闭竞态。选择更弱顺序不是炫技，而是在语义已证明后减少不必要约束。

性能模型还要考虑 cache line。Relaxed 计数器在高频写下仍会争用；acquire/release 在某些平台很轻，在另一些平台需要更明确屏障；CAS 循环在高竞争下会反复失败。报告应记录 CAS 失败、cache line 热点和扩展曲线。内存序选择既是语义问题，也是成本问题。

四、从锁版到原子版的迁移纪律 迁移路线应保守。第一，保留锁版 reference。第二，写出锁保护的不变量。第三，判断哪些状态可以独立出来，哪些必须仍由锁保护。第四，只把统计指标或边界明确的状态改成原子。第五，为每个原子写协议说明和测试。第六，比较性能和复杂度，收益不足就保留锁版。

错误迁移通常是把字段逐个改成 atomic。队列的 size 原子化，head 和 tail 仍无保护；配置指针原子化，旧对象释放无协议；任务 done 原子化，result 发布无 acquire。这样的迁移只消除了编译器能看到的数据竞争，没保住业务不变量。

五、原子变量命名应表达语义 命名可以减少误用。统计变量使用 success_count_relaxed 不一定优雅，但至少提醒它只是指标；发布状态使用 published_state 比 flag 清楚；生命周期

对象使用 `epoch_guard` 或 `hazard_slot`，让读者知道它不是普通计数。命名不是证明，但能降低维护风险。

接口也要表达语义。提供 `snapshot()` 返回 shared pointer，而不是返回裸指针；提供 `try_commit()` 返回 accepted、already committed、stale，而不是 bool；提供 `wait_result()`，而不是让调用者自己轮询 state 后读 result。好接口把内存序细节封装在协议内，坏接口把证明责任推给每个调用者。

六、文档应靠近代码 原子协议文档不应只存在章节文字里。真实项目中，每段原子代码旁边要有短注释或设计文档链接，说明发布数据、获取数据、线性化点和回收边界。注释不要重复代码，而要写代码看不出来的语义。例如“release 发布 slot 中已构造元素给 consumer”，比“使用 memory order release”更有价值。

维护者修改内存序时，也要修改注释和测试。如果把 acquire 改成 relaxed，却没有更新协议说明，review 应阻止。内存序是设计的一部分，不是局部微优化。

七、教学中的反例必须保留 很多内存序错误不容易复现，反例仍然要写。Relaxed ready flag 可能在当前机器上一直正确运行，但语言上缺同步；原子指针可能一直没崩，但旧对象释放协议缺失；CAS 循环里写日志可能在低竞争下只写一次，高竞争下重复。反例的价值是让读者看到隐藏假设。

教学不应为了让实验一定失败而编造夸张输入，而应诚实说明：有些错误是标准层面的，不一定能用普通压力测试复现。读者要学会接受“测试没失败但代码仍不正确”这个事实。这是并发学习的关键门槛。

八、原子章和整本书的连接 原子章向前连接硬件一致性和 cache line，向后连接无锁结构、任务运行时和分布式提交。Release/acquire 的发布思想会在 SPSC ring 中出现；CAS 线性化会在无锁栈和 commit table 中出现；生命周期协议会在 hazard pointer、配置热更新和模型版本切换中出现；指标和协议分离会在线程池监控和分布式限流中出现。

如果读者只记住内存序枚举，本章失败；如果读者能在新代码中先分类原子变量，再写发布关系、线性化点和生命周期，本章成功。

11.17 内存模型小实验：用 litmus test 训练直觉

内存序最难的地方，是错误行为不一定在普通测试中出现。Litmus test 是很小的并发程序，用来讨论某个结果是否被内存模型允许。它不是生产测试，也不是性能 benchmark，而是训练读者把“看起来会怎样”转换成“语言模型允许怎样”的工具。

一、消息传递实验 最经典的实验是消息传递。线程一写普通数据，再写 ready；线程二读 ready，若 ready 为真就读普通数据。若 ready 是 release store，线程二用 acquire load 读到 true，那么读普通数据有保证。若 ready 是 relaxed，线程二读到 true 后仍没有通过 ready 建立同步。这个实验的重点不是一定在机器上读到旧数据，而是理解语言模型没有给出保证。

教学时可以让读者先预测三种版本：普通 bool、atomic relaxed、atomic release acquire。普通 bool 有数据竞争，程序未定义；relaxed 没有 ready 的数据竞争，但不发布普通数据；release acquire 才表达消息传递。三者的区别要写在报告里，而不是只跑程序看输出。

二、Store buffering 实验 另一个小实验是两个线程各自写一个变量，再读对方变量。若使用 relaxed，两个线程都读到旧值在弱内存模型中可能被允许。Seq cst 会给所有 seq cst 操作一个全局顺序，从而排除某些直觉上奇怪的结果。这个实验帮助读者理解：原子性不等于全局顺序，relaxed 允许更多重排和观察结果。

这里要避免把硬件细节讲错。C++ 程序讨论的是语言层面允许行为；具体硬件和编译器如何实现，是映射问题。x86、ARM、POWER 的差异可以作为背景，但教材结论应以 C++ 内存模型为准。

三、发布序列实验 Release sequence 对初学者很抽象，可以用引用计数或状态推进解释。线程一 release store 状态为 Ready；线程二对同一原子做读改写推进状态；线程三 acquire 读到这个序列中的值。C++ 对同一原子上的释放序列有专门规则，让某些同步关系可以穿过读改写操作。这个规则不是鼓励初学者写复杂协议，而是解释成熟库为什么能组合 release 和 RMW。

实验报告只需要说明两点：同步关系围绕同一个原子对象建立；读改写既读又写，因此成功和失败内存序要分开看。不要把一个原子上的 release 随便借给另一个原子，也不要以为所有 RMW 都

自动发布所有状态。

四、IRIW 和全局观察顺序 不同线程观察多个变量写入的顺序，也可能出现反直觉结果。Seq cst 的全局顺序能让推理更简单，而 acquire release 只围绕具体同步关系建立顺序。这个差异解释了为什么教学和调试中常先用 seq cst，等协议清楚后再降级。不是 seq cst 高级，而是它更接近初学者直觉。

工程上，不要为了追求最弱顺序，一开始就把所有操作写成 relaxed 和 acquire release 的复杂组合。先写清状态机，必要时用锁或 seq cst，等热点和协议都明确后再降低顺序。内存模型学习不是比赛谁写得最弱。

五、Litmus test 的局限 Litmus test 很小，能解释模型允许行为，但不能证明完整数据结构正确。无锁队列还涉及容量、生命周期、ABA、关闭、等待和异常；任务完成对象还涉及 shared state 和等待者；配置发布还涉及旧版本回收。Litmus test 是显微镜，不是整车测试。教材应把它用于训练概念，不应把它当作生产验证的全部。

六、项目交付 项目可以保留一组小实验：message passing、store buffering、relaxed counter、release acquire publish、CAS retry。每个实验有一页说明：程序是什么，哪些结果被允许，当前机器是否观察到，为什么观察不到也不代表语义更强。这个材料能帮助读者在换到 ARM 手机、x86 服务器或不同编译器时正确解释差异。

11.18 从锁版协议迁移到原子协议

学习原子操作时，不要从内存序枚举开始，而要从一个已经正确的锁版协议开始迁移。锁版代码通常把互斥、发布、等待和生命周期揉在一起：持锁写结果，持锁修改状态，条件变量唤醒等待者，析构时持锁关闭。改成原子版本时，这四件事必须拆开。若只把互斥锁替换成几个 atomic flag，代码很可能通过简单测试，却在对象发布、取消或析构时破坏语义。

一、先列出锁提供了哪些保证 锁至少提供三类保证：同一时刻只有一个线程修改受保护状态；解锁前的写入对随后加锁者可见；临界区还常常隐含对象仍然存活。迁移时要逐项替换。互斥可以用 CAS 状态机替代，可见性需要 release 和 acquire 建立，生命周期需要引用计数、epoch、hazard pointer 或外层所有权保护。若没有这张保证清单，迁移就会漏掉某个隐含条件。

二、把普通字段和原子状态分开 常见发布模式是先写普通字段，再用 release store 发布状态；读取者用 acquire load 看到状态后，才能读取普通字段。这里原子变量不是保存全部数据，而是建立同步边。若把普通字段也零散改成 atomic，反而容易失去结构：读者不知道哪些字段属于同一次发布，哪些只是统计指标。好的原子协议会让普通数据保持普通，把同步点集中在少数状态变量上。

三、状态机替代多个布尔值 多个 `atomic_bool` 很难表达互斥状态。一个任务不能同时是 Running、Succeeded、Failed、Cancelled；用多个布尔值会产生中间组合。改成 `std::atomic<TaskState>` 后，CAS 从 Running 转到 Succeeded 或 Failed，线性化点清楚，审查也清楚。状态枚举不是为了好看，而是为了把合法状态空间收窄。内存序也随状态转移而来：发布结果的转移使用 release，观察完成的读取使用 acquire，纯统计读取才可能 relaxed。

四、等待和原子要配合而不是混用 忙等能让示例短，但工程里会浪费 CPU，并增加缓存一致性流量。C++20 的 atomic wait 和 notify 可以把状态变化和睡眠结合起来，但仍然要遵守谓词思维：等待前读取状态，状态不满足才 wait，唤醒后重新读取。若还有条件变量或外部事件循环，必须规定谁负责唤醒、关闭时如何唤醒、迟到通知是否安全。等待不是内存序能解决的问题，它是状态机和调度的共同问题。

五、迁移完成后写协议注释 原子代码最怕“看起来能跑”。每个原子变量旁边应写它的角色：统计、发布、线性化、生命周期还是等待；每个 release 写明发布了哪些普通字段；每个 acquire 写明观察到什么状态后才能读哪些数据；每个 relaxed 写明为什么不参与协议。这样的注释不是繁琐，而是让未来维护者不会把 relaxed 变量升级成关闭条件，也不会把发布状态改成普通写。原子协议的可维护性来自可审查。

11.19 贯穿案例：ready 标志为什么不能只是一个布尔值

内存序最容易被讲成枚举表，但真实 bug 往往从一个很小的布尔值开始。生产者线程分配 buffer，写入 `payload_size`、`checksum` 和数据指针，然后把 `ready` 置为 true。消费者线程看到 `ready`

为 true，就读取这些普通字段并提交写出。测试机上跑了很多次都没错，于是有人把 `ready` 写成 `relaxed`，理由是“只有一个生产者一个消费者，而且 `bool` 写入是原子的”。这就是事故入口：原子性只保证 `ready` 这个对象不被撕裂，不保证它之前的普通字段在另一个线程可见。

从这个事故推出内存序，比直接背 `release/acquire` 更牢。我们真正需要表达的关系是：生产者写普通字段发生在发布 `ready` 之前；消费者观察到 `ready` 之后，才能读取这些字段。`Release store` 和 `acquire load` 正是为这条关系服务。若读者只记住“发布用 `release`，读取用 `acquire`”，仍然不够；他还必须能说出 `release` 发布了哪些字段，`acquire` 后允许读哪些字段，以及哪些字段不在这次发布合同里。

一、锁版先说明语义 第一版用 `mutex` 写最清楚。生产者持锁写 `buffer` 元数据和状态，解锁；消费者加锁检查状态，再读字段。锁提供互斥，也提供解锁到加锁的同步关系。迁移到原子前，先把锁提供的保证写出来：字段写入不和读取并发，消费者看到 `ready` 时字段完整，对象在读取期间仍然存活。后两项经常被初学者漏掉，尤其是生命周期。内存序只解决可见性，不自动解决对象释放。

二、`ready` 是发布点，不是数据容器 普通字段仍然是普通字段。它们不需要全部变成 `atomic`，因为我们不希望消费者看到一组彼此不一致的字段。同步点集中在 `ready` 或状态枚举上。生产者按普通写填好字段，最后 `release` 发布状态；消费者 `acquire` 观察状态，再读取字段。若把每个字段都改成 `relaxed atomic`，表面上没有数据竞争，实际却失去了“这些字段属于同一次发布”的结构。原子变量越多，协议不一定越安全。

三、状态枚举比多个标志更可审查 一个 `buffer` 可能处于 `Empty`、`Filling`、`Ready`、`Writing`、`Done`、`Failed`。若用多个 `atomic bool` 表示 `ready`、`failed`、`cancelled`，很容易出现 `ready` 和 `failed` 同时为真、`cancelled` 后又 `ready` 等非法组合。用一个 `std::atomic<BufferState>` 表示状态，CAS 转移就能收窄合法空间。内存序跟着转移走：`Filling` 到 `Ready` 发布普通字段，`Ready` 到 `Writing` 争取所有权，`Writing` 到 `Done` 发布完成结果，统计计数才 `relaxed`。

四、`relaxed` 要写明“不发布任何东西” 比如 `bytes_written_counter` 只用于监控，不决定释放，不决定提交，不唤醒等待者，它可以 `relaxed`。若某天有人用它判断“所有字节写完，可以释放 `buffer`”，它就参与了生命周期协议，原来的 `relaxed` 理由失效。代码注释应写角色，而不是写“为了性能使用 `relaxed`”。好的注释会说：此计数只用于近似指标，不同步任何普通字段，不作为关闭条件。

五、测试要覆盖错误角色变化 内存序 bug 可能不在当前机器上出现，所以测试不能假装证明最弱顺序正确。但测试可以防止角色被误用。比如编写单元测试或静态审查清单：任何决定提交、释放、关闭、唤醒的变量都不能被标成纯统计；任何 `release` 发布点都要列出普通字段；任何 `acquire` 读取后才能访问的字段都要在文档里出现。这样维护者改代码时，会先碰到协议边界。

六、从发布对象连接到任务完成 任务运行时的 `TaskState::Succeeded`、I/O 的 `completion` 状态、分布式消息的 `committed` 状态，都重复同一模式：先写结果字段，再发布状态，观察状态后再读取结果。内存序不是孤立知识，它是系统各层状态机的可见性语言。只要跨线程发布对象，就要回答这三个问题：谁写普通字段，哪个原子发布，谁 `acquire` 后读取。答不出来，就用锁保持清晰。

11.20 案例深化：从发布对象到内存序选择

内存序最容易被讲成枚举表：`relaxed`、`acquire`、`release`、`acq_rel`、`seq_cst`。这种讲法很快，但读者不知道为什么需要它们。更好的入口是发布一个对象。生产者填好普通字段，然后把指针或状态发布给消费者；消费者看到状态后读取普通字段。若发布和读取之间没有同步关系，消费者可能看到状态已就绪，却读到旧字段。内存序不是为了背单词，而是为了说明哪些普通写在另一个线程中可见。

一、先写强语义版本 第一版可以用 `mutex` 或 `seq_cst`，让语义清楚。生产者在锁下写字段并改状态，消费者在同一锁下读状态和字段。这个版本也许不快，但它给出 `reference`。然后再问：锁保护的是什么关系？答案是“字段写入先于状态可见，消费者看到状态后才能读字段”。只有把关系说清，才能安全地降到 `release/acquire`。

二、`release` 发布了哪些字段 `Release store` 不是给某个变量加光环，而是把它之前的普通写发布出去。代码旁边应写明：发布状态前写了 `payload_size`、`checksum`、`bufferpointer`。消费者用 `acquire load` 看到 `ready` 后，才允许读取这些字段。若后来维护者把某个字段写到 `release` 之后，就破坏协议。注释要写关系，不写长篇术语解释。

三、**relaxed 只适合不参与协议的计数** 统计次数、采样计数、性能指标常可用 relaxed，因为它们不发布对象、不决定生命周期、不作为关闭条件。若一个 relaxed 变量被用来判断“是否可以释放 buffer”，它就不再是普通统计。内存序选择要看变量角色，而不是看想不想快。报告中应把原子变量分类：统计、发布、线性化、生命周期、等待。

四、**CAS 的失败路径也要讲** CAS 成功通常是线性化点，失败也有语义。失败说明别人先修改了状态，当前线程要重新读取并判断。弱 CAS 可能虚假失败，循环要能处理。高竞争下 CAS 失败次数会影响性能，不能只看成功操作数。无锁结构章节会继续使用这个原则：线性化点和失败重试都要可观测。

五、**测试不能证明最弱内存序正确** 内存序 bug 很可能在测试机上不出现。测试可以帮助发现错误，但正确性仍需要推理。每次把 seq cst 降到 release/acquire 或 relaxed，都要写出 happens-before 关系和不参与协议的理由。若写不出来，就保持更强顺序。高质量工程不是追求最弱，而是追求可审查。

11.21 案例复盘：发布对象而不是背内存序表

生产者写 payload 字段，再把 ready 置为 true；消费者看到 ready 后读取 payload。若 ready 用 relaxed，消费者可能看到状态却没看到字段。内存序从这个事故中出现：release 发布之前的普通写，acquire 观察发布后才能读这些字段。

先强后弱 第一版用锁或 seq cst 保证语义，写清“字段写入先于 ready 可见”。第二版把 ready 改成 release/acquire，同时在注释中列出发布了哪些字段。Relaxed 只用于不参与发布、生命周期和关闭协议的统计变量。

实验判据 测试能帮助发现错误，但不能证明最弱顺序正确。报告要列每个原子变量角色：统计、发布、线性化、生命周期或等待。每次降低顺序都写 happens-before 关系。写不出关系，就保持更强顺序。

11.22 本章习题

习题与作业

习题一：给三个场景选择内存序并解释理由：请求计数器、配置对象发布、无锁最大值更新。每个场景都要说明是否同步其他普通数据，是否需要全局顺序，失败时会发生什么。

习题二：把一个错误的 relaxed 发布例子改成 release/acquire。要求画出 happens-before 关系，并说明对象生命周期仍然需要额外处理。

习题三：写一个 CAS 循环，要求循环体内没有不可回滚副作用。然后列出三种不应该放进 CAS 重试循环的副作用，并解释原因。

习题四：阅读项目中的 `AtomicCounter`。解释为什么当前 `fetch_add` 可以使用 relaxed，也说明它为什么不适合作为对象发布机制或队列同步机制。

第 12 章

无锁数据结构

先看一个 Treiber stack 事故：线程 A 读取 head 指向节点 A，还没来得及 CAS；线程 B 弹出 A，释放它，又分配了一个新节点，地址恰好还是 A；线程 A 的 CAS 看见 head 仍然等于 A，于是成功，把一个生命周期已经变化的节点当成旧节点使用。这里没有 mutex 阻塞，也没有普通数据竞争，但程序仍然错了。CAS 只比较值，不理解对象生命史。

本章从这种 ABA 和回收问题进入无锁数据结构。无锁的目标不是“没有同步”，而是在不使用互斥锁阻塞线程的情况下维护共享状态。它通常依赖原子操作、CAS、内存序和严格的对象生命周期管理。学习无锁结构，要先学习进度保证和失败模式，因为真正难点往往不在 CAS 写法，而在竞争失败、旧节点迟到、对象何时能释放。

12.1 从阻塞到无锁进展保证

Blocking 算法可能因为一个线程持锁停顿而阻塞其他线程。Lock-free 算法保证系统整体持续前进，但单个线程可能反复失败。Wait-free 算法保证每个线程在有限步内完成，通常更难实现。Obstruction-free 只在无竞争时保证进展。

这些术语不是装饰。一个队列在压力下 CAS 反复失败，可能仍然 lock-free，但尾延迟很差。工程上需要结合竞争程度、临界区复杂度和可维护性选择方案。

进度保证要和系统目标匹配。低延迟交易路径可能愿意为 wait-free 付出巨大复杂度；普通后台队列通常 mutex 就足够；高吞吐日志管线可能只需要 SPSC ring buffer；线程池全局队列可能更适合分片队列加工作窃取。不要把 lock-free 当成荣誉标签，它只是进度性质的一种选择。

无锁栈和 ABA Treiber stack 是经典无锁栈。push 读取 head，把新节点 next 指向旧 head，再 CAS head。pop 读取 head 和 next，再 CAS head 到 next。问题在于 ABA：head 可能从 A 变成 B 又变回 A，CAS 看见 A 成功，但 A 的生命周期可能已经变化。

解决 ABA 需要额外机制。版本标签把指针和计数器一起比较；hazard pointer 让线程声明自己正在访问的节点；epoch reclamation 延迟释放节点直到所有线程离开旧 epoch。无锁算法难点往往不在 CAS 本身，而在内存回收。

```
Treiber push shape:
load old head
new node next = old head
CAS head from old head to new node
```

```
Treiber pop shape:
load old head
load next from old head
CAS head from old head to next
```

这张形态图比直接贴完整代码更重要。读者必须先看出线性化点在哪里：push 和 pop 都在 CAS 成功那一刻生效。CAS 之前读到的状态只是一次尝试，失败后不能假装操作已经发生。

12.2 ABA、内存回收和生命周期

带锁结构中，线程持有锁时可以知道没有其他线程正在修改结构。无锁结构没有这个简单边界。一个线程从 head 读到节点指针后，可能还没来得及读取 next，另一个线程已经 pop 并释放了该节点。第一个线程随后访问这个指针，就会出现 use-after-free。

垃圾回收语言可以把节点释放推迟到不可达之后，C++ 程序必须自己设计回收协议。Hazard pointer 的思路是线程公开声明“我可能访问这个节点”，释放者扫描这些声明，确认没有线程持有

后再释放。Epoch 的思路是把释放延迟到所有线程都离开旧时代之后。二者都有开销，也都有使用约束。

Hazard pointer 更精确，但每个线程需要发布自己正在保护的指针，释放者需要扫描 hazard 集合。Epoch 回收批量化更强，读路径常常更轻，但如果某个线程长时间停留在旧 epoch，回收会延迟，内存占用可能增长。RCU 也是类似思想：读者进入读侧临界区，写者等待 grace period 后回收旧版本。不同机制的共同点是：释放不能只看“节点已经从结构中摘掉”，还要看“是否还有线程可能持有旧指针”。

ABA 的具体例子 假设栈顶是 A，线程 T1 读取 head=A，准备把 head 改成 A->next。此时 T2 pop A，又 pop B，然后把 A push 回栈顶。T1 再 CAS head，从 A 改成旧的 A->next。CAS 看到 head 仍然是 A，于是成功，但这个 A 已经不是 T1 当初理解的结构关系。版本标签通过让 head 变成“指针加版本号”，使 A 回来时版本不同，从而让 CAS 失败。

12.3 从 SPSC 到 MPMC 队列

有界环形队列适合固定容量、低分配、缓存友好的场景。单生产者单消费者队列可以用较简单的 head/tail 原子实现；多生产者多消费者队列需要为槽位维护序号或状态，避免多个线程同时写同一槽位。

环形队列的性能来自固定内存、连续布局和减少分配。但如果所有生产者争用同一个 tail，仍然会有原子热点。分片队列、每线程队列和工作窃取可以缓解热点。

SPSC、MPSC 和 MPMC 队列难度取决于生产者和消费者数量。SPSC 只有一个生产者和一个消费者，head 只被消费者写，tail 只被生产者写，可以用较轻的同步。MPSC 有多个生产者争用 tail，消费者独占 head。MPMC 两端都有竞争，通常需要更复杂的槽位序号或链表算法。

设计队列时先问清使用场景，不要直接上最复杂的 MPMC。日志线程、音频缓冲和管线阶段之间常常只需要 SPSC；线程池全局队列可能需要 MPMC；工作窃取运行时则常用每 worker deque 加窃取操作。

SPSC ring buffer 是学习无锁队列的最佳起点。生产者独占 tail，消费者独占 head，因此每个索引只有一个写者；双方通过 acquire/release 观察对方进度。MPSC 和 MPMC 复杂得多，因为多个线程会争用同一个端点，需要 CAS、槽位状态或序号。教材顺序应从 SPSC 到 MPSC 再到 MPMC，而不是一开始贴一个难以证明的万能队列。

12.4 线性化点、测试和工程边界

并发数据结构的线性化点，是操作在逻辑上生效的那一瞬间。带锁结构的线性化点通常在锁内某次状态修改；无锁结构的线性化点通常是成功 CAS。若无法指出线性化点，就很难证明结构正确。

例如无锁栈 push 的线性化点是 CAS head 成功。CAS 失败的尝试没有生效，必须重试。测试无锁结构时，不能只看最终数量，还要考虑每个操作是否能排列成某个合法串行顺序。

线性化点还帮助写测试。对于栈，可以记录每个 push 的值和每个 pop 的返回，检查是否存在某个合法串行顺序；对于队列，要检查 FIFO 语义是否在并发交错下仍成立；对于计数器，要检查每次成功自增是否唯一。压力测试只能增加信心，不能替代线性化论证。无锁结构没有明确线性化点，通常就不该进入生产代码。

何时不用无锁 如果共享状态复杂、竞争不高、临界区很短，mutex 可能更清晰也足够快。如果需要条件等待，锁加条件变量往往比忙等 CAS 更节能。无锁结构适合明确的高频热路径，且必须有严格测试、压力测试和代码审查。

工程验收标准 无锁结构的验收标准应比普通容器更高。第一，要有清晰的不变量、线性化点和进度保证说明。第二，要说明内存回收策略，不能留下 use-after-free 风险。第三，要有单线程 reference 行为测试、并发压力测试、边界容量测试和关闭语义测试。第四，要有性能对照，证明它在目标竞争模式下确实优于 mutex 或分片锁。第五，要说明不适用场景，例如高竞争 CAS 风暴、需要阻塞等待或对象生命周期复杂。

如果这些条件做不到，就应该优先使用更简单的同步结构。复杂无锁代码的维护成本非常高，错误通常低概率且难复现。严肃工程里，简单正确经常比炫技更有价值。

SPSC ring buffer 完整推导 学习无锁队列应从 SPSC 开始，因为它的所有权最清楚：只有一个生产者写 tail，只有一个消费者写 head。生产者会读 head 判断是否有空间，消费者会读 tail 判

断是否有元素。每个索引只有一个写者，这大幅降低了同步难度。SPSC 的关键不是“没有锁”，而是用所有权设计减少共享写。

环形队列保留一个空槽可以区分满和空。若 `next_tail==head`，队列满；若 `head==tail`，队列空。生产者写入元素后 `release` 发布新的 `tail`；消费者 `acquire` 读取 `tail` 后，能看到生产者写入的元素。消费者取出元素后 `release` 发布新的 `head`；生产者 `acquire` 读取 `head` 后，知道槽位已经可复用。对于简单整数元素，这个模型很直观；对于复杂对象，还要处理构造、析构和异常安全。

Listing 12.1 SPSC ring buffer 的核心实现

```

1 class SpscIntQueue {
2 public:
3     explicit SpscIntQueue(std::int32_t capacity)
4     : buffer_(static_cast<std::size_t>(capacity + 1), 0) {}
5
6     bool try_push(std::int32_t value) {
7         const std::uint64_t tail =
8         this->tail_.load(std::memory_order_relaxed);
9         const std::uint64_t next_tail = this->next(tail);
10        const std::uint64_t head =
11        this->head_.load(std::memory_order_acquire);
12        if (next_tail == head) {
13            return false;
14        }
15        this->buffer_[static_cast<std::size_t>(tail)] = value;
16        this->tail_.store(next_tail, std::memory_order_release);
17        return true;
18    }
19
20    std::optional<std::int32_t> try_pop() {
21        const std::uint64_t head =
22        this->head_.load(std::memory_order_relaxed);
23        const std::uint64_t tail =
24        this->tail_.load(std::memory_order_acquire);
25        if (head == tail) {
26            return std::nullopt;
27        }
28        const std::int32_t value =
29        this->buffer_[static_cast<std::size_t>(head)];
30        this->head_.store(this->next(head), std::memory_order_release);
31        return value;
32    }
33
34 private:
35     std::uint64_t next(std::uint64_t index) const {
36         const std::uint64_t size =
37         static_cast<std::uint64_t>(this->buffer_.size());
38         return (index + 1) % size;
39     }
40
41     std::vector<std::int32_t> buffer_;
42     alignas(64) std::atomic<std::uint64_t> head_{0};
43     alignas(64) std::atomic<std::uint64_t> tail_{0};
44 };

```

这段代码故意只支持 SPSC。若两个生产者同时调用 `try_push`，它们都会写 `tail` 和同一槽位，算法不再正确；若两个消费者同时 `try_pop`，它们都会写 `head`，也不正确。接口文档必须写清这个约束。无锁结构最忌讳把适用场景藏起来，让调用者误用。

`head` 和 `tail` 分开放置，是为了减少生产者和消费者频繁写同一 `cache line`。这里使用 `std::uint64_t` 是为了让索引范围清楚；容量参数仍需检查，生产代码还要处理容量溢出。教学代码展示核心同步，工程代码还要补足参数验证、移动禁用、析构约束和测试。

SPSC 的线性化点 SPSC `try_push` 的线性化点是 `release` 写 `tail`。写入 `buffer` 只是准备数据；`tail` 更新后，消费者 `acquire` 看到新 `tail`，元素才对消费者可见。`try_pop` 的线性化点可以看作读取元素并 `release` 更新 `head` 的组合：在返回元素前，它已经把槽位归还给生产者。更严格地说，对于整数槽位，消费者读取元素后操作结果已经确定，`head` 更新发布空间可用。

线性化点帮助分析失败路径。若 `try_push` 发现队列满返回 `false`，它没有写入 `buffer`，也没有更新 `tail`，操作没有生效。若 `try_pop` 发现空返回 `nullopt`，它没有更新 `head`。测试要覆盖这些失败路径，因为失败路径在有界队列中不是异常，而是正常背压。

从 SPSC 到 MPSC MPSC 增加多个生产者。此时 `tail` 不再只有一个写者，生产者之间必须争用写入位置。常见做法是使用 `fetch_add` 或 `CAS` 为每个生产者预留槽位，然后再处理槽位可见性。问题立刻变复杂：生产者 A 预留了槽位但还没写完，生产者 B 写完后能否让消费者看到后面的槽位？消费者如何知道某个槽位的数据已经准备好？队列满时已经预留的位置如何处理？

这就是为什么 MPSC/MPMC 队列常为每个槽位维护序号或状态。`tail` 只表示预留进度不够，还要知道槽位是否已提交。队列算法难度不是来自环形数组本身，而是来自多写者之间的提交顺序、可见性和容量回收。教材应明确告诉读者：不要把 SPSC 代码简单加 `CAS` 就当 MPSC。

MPMC 队列的槽位序号直觉 一种经典 MPMC 有界队列会让每个槽位带一个 `sequence`。生产者根据 `tail` 找到槽位，检查 `sequence` 是否表示槽位空闲；成功 `CAS tail` 后写入数据，再更新 `sequence` 表示可读。消费者根据 `head` 找到槽位，检查 `sequence` 是否表示槽位可读；成功 `CAS head` 后读取数据，再更新 `sequence` 表示空闲。`sequence` 用于区分同一个数组槽位在不同轮次中的状态，避免只靠 `head/tail` 造成混淆。

这个模型比 SPSC 难很多。每个端点都有多个写者，`CAS` 失败是正常路径；槽位 `sequence` 也要选择正确内存序；容量必须是固定的或 `carefully` 管理；队列关闭和阻塞等待又是额外层次。生产系统通常会优先使用成熟库，或者先用 `mutex bounded queue`，只有在明确瓶颈和使用场景后才自研 MPMC。

ABA 和内存回收的关系 ABA 不只是“值变回去了”的小问题，它经常和内存回收连在一起。节点 A 被 `pop` 后释放，分配器可能把同一地址重新分配给新节点。另一个线程手里仍有旧指针，`CAS` 看到地址相同，以为结构没变，实际上对象生命周期已经换了。版本标签能区分同一地址的不同代；`hazard pointer` 和 `epoch` 能防止节点过早释放；垃圾回收语言则把释放延后到不可达之后。

C++ 无锁结构必须把“从数据结构中摘除”和“释放内存”分开。摘除只说明新操作不会再从结构入口到达该节点，不说明旧操作没有持有指针。回收协议就是证明旧操作不再持有指针的机制。没有回收协议的无锁链表或栈，在轻测试下可能正常，在压力下可能 `use-after-free`。

Hazard pointer 的操作流程 `Hazard pointer` 的基本流程是：线程在读取共享指针后，把该指针发布到自己的 `hazard` 槽位；然后重新读取共享指针确认它没有变化；确认后才能安全解引用。删除节点的线程不能立即释放，而是把节点放入 `retired` 列表；当 `retired` 列表足够大时，扫描所有 `hazard` 槽位，把没有被任何线程保护的节点释放。

这套机制的成本很明确：读路径需要发布 `hazard`，释放路径需要扫描，线程数越多扫描成本越高。它的优点是精确，不需要等待所有线程离开全局 `epoch`。它适合指针访问较复杂、需要较及时回收的结构。教学时不一定要完整实现，但必须让读者知道无锁结构的真正难点在这里。

Epoch 回收的操作流程 `Epoch` 回收把时间分成阶段。线程进入无锁操作时宣布自己处于当前 `epoch`，退出时宣布离开。删除节点时不立即释放，而是记录删除发生的 `epoch`。只有当所有线程都已经离开旧 `epoch`，旧 `epoch` 中删除的节点才可以释放。它把单个节点的精确保护变成批量延迟回收。

`Epoch` 的读路径通常比 `hazard pointer` 更轻，但如果某个线程长时间不退出临界区，回收会被阻塞，内存占用可能增长。它适合操作短、线程行为可控的场景，不适合线程可能长期停顿在临界

区的场景。系统设计必须考虑线程被抢占、阻塞、崩溃或取消时对回收的影响。

无锁不等于低尾延迟 lock-free 保证系统整体前进，不保证每个线程快速完成。高竞争 CAS 循环可能让某些线程反复失败，尾延迟很差。wait-free 才提供单线程有限步完成保证，但实现通常更复杂。对于服务系统，尾延迟可能比平均吞吐更重要，因此 lock-free 结构不一定比有锁结构更适合请求路径。

mutex 在高竞争下可能进入内核睡眠，平均吞吐下降但节省 CPU；lock-free CAS 在高竞争下可能烧满 CPU，不断失败重试。哪种更好取决于等待时间、核心预算、能耗、尾延迟和业务语义。教材不能把 lock-free 简化成“更高级所以更快”。

进展保证的层次 无锁数据结构讨论中，必须区分 blocking、obstruction-free、lock-free 和 wait-free。Blocking 结构可能因为某个线程持锁后停顿而阻塞其他线程。Obstruction-free 只保证一个线程在没有竞争时能完成。Lock-free 保证系统整体总有某个线程能前进，但单个线程可能一直失败。Wait-free 保证每个线程在有限步骤内完成。进展保证越强，实现通常越复杂，常数成本也可能更高。

很多工程代码把“没有 mutex”就叫 lock-free，这是不严谨的。如果算法里某个线程可以让所有线程永久等待，例如等待某个槽位状态从 Writing 变成 Ready，而写该槽位的线程崩溃后无人修复，那么它可能并不满足想象中的进展保证。进展保证要写清楚依赖条件：线程是否可能被取消，是否允许阻塞内存分配，是否依赖操作系统调度，是否在队列满时返回失败。

线性化点是正确性的核心 无锁结构必须能为每个成功操作指出线性化点，也就是这个操作在并发历史中看起来瞬间生效的位置。SPSC push 的线性化点可以是发布 tail；Treiber stack push 的线性化点通常是 CAS 成功更新 head；MPMC 队列生产者的线性化点可能是槽位 sequence 从写入中变成可读。没有线性化点，就无法说明并发操作等价于某个串行顺序。

失败操作也要有解释。队列满返回 false 的线性化点可以是观察到满状态的那次读取；队列空返回 null 的线性化点可以是观察到 head 等于 tail 的那次读取。但如果观察过程中状态变化了，就要证明返回仍然合法。许多无锁 bug 就藏在失败路径：成功路径看起来能工作，空、满、关闭、竞争失败和回收路径没有证明。

Treiber 栈作为最小反例 Treiber stack 是学习 CAS 的经典结构，但它也能展示无锁结构的陷阱。push 分配新节点，把新节点的 next 指向当前 head，然后 CAS head 为新节点。pop 读取 head 和 next，然后 CAS head 为 next。看起来很短，但一旦考虑 ABA 和内存回收，复杂度立刻上升。

Listing 12.2 Treiber 栈的教学骨架，不包含回收协议

```

1 struct StackNode {
2     std::int32_t value = 0;
3     StackNode* next = nullptr;
4 };
5
6 class LockFreeStack {
7 public:
8     void push(StackNode* node) {
9         StackNode* observed = this->head_.load(std::memory_order_relaxed);
10        do {
11            node->next = observed;
12        } while (!this->head_.compare_exchange_weak(
13            observed,
14            node,
15            std::memory_order_release,
16            std::memory_order_relaxed));
17        }
18
19 private:
20     std::atomic<StackNode*> head_{nullptr};
21 };

```

这段代码不能直接作为生产栈，因为 pop 需要解决节点何时释放。即使 push 只有十几行，完整数据结构也必须定义节点所有权。节点由调用者拥有、栈拥有、对象池拥有，还是回收系统拥有？pop 返回节点后，其他线程是否仍可能持有旧指针？这些问题比 CAS 本身更重要。

ABA 时间线 ABA 可以用时间线理解。线程 T1 读取 head 等于地址 A，并读取 A 的 next 等于 B。此时 T1 暂停。线程 T2 pop 掉 A，又 pop 掉 B，释放 A，分配器恰好把地址 A 分给新节点 C，再 push 到 head。T1 恢复后看到 head 仍然是地址 A，于是 CAS 成功，把 head 改成旧的 B。此时栈被破坏，因为 T1 以为 A 还是原来的节点，实际地址 A 的对象已经换了生命周期。

解决 ABA 的方法不是单一的。带版本标签的指针把地址和计数一起 CAS，同一地址再次出现时版本不同。Hazard pointer 防止 T2 在 T1 仍可能持有 A 时释放 A。Epoch 回收延迟释放，直到所有可能持有旧指针的线程离开旧 epoch。垃圾回收语言则把这个问题转移给运行时。C++ 需要显式选择一种策略。

带标签指针的直觉 带标签指针把指针和版本号组成一个逻辑值。每次 head 改变，版本号递增。这样即使指针地址从 A 变成 B 又回到 A，版本号也不同，CAS 不会误以为状态没变。实际实现要考虑平台是否支持双字 CAS、指针对齐是否能偷低位、版本溢出和 ABI 可移植性。教材阶段理解思想即可，不应鼓励读者随手写不可移植指针位操作。

Listing 12.3 标签指针的语义形状

```
1 struct TaggedIndex {
2     std::uint32_t index = 0;
3     std::uint32_t generation = 0;
4 };
```

用索引加 generation 比直接偷指针位更适合教学，也更接近很多对象池和 ECS 系统。数组槽位复用时代 generation 增加，旧句柄即使 index 相同，也会因为 generation 不匹配而失效。这个思想贯穿本书：位置和对象生命周期不是同一件事，必须用版本把它们区分开。

Hazard pointer 的实现边界 Hazard pointer 要求每个线程拥有一个或多个 hazard 槽位。读取共享指针后，线程把指针写入自己的槽位，再重新读取共享指针确认没有变化。这个二次确认很重要：如果读取指针后还没发布 hazard，删除线程可能已经把节点放入 retired 列表并释放。只有发布并确认后，读者才能安全解引用。

释放线程不能每删除一个节点就扫描所有 hazard，因为扫描成本高。通常把删除节点放入 retired 列表，达到阈值后批量扫描。扫描时收集所有 hazard 指针，把没有出现在集合中的 retired 节点释放。阈值选择影响内存峰值和释放成本：阈值小，扫描频繁；阈值大，内存保留多。

Hazard pointer 还需要处理线程退出。如果一个线程退出前没有清空 hazard 槽位，其他线程可能永远不释放某些节点。生产实现要有线程注册、注销、槽位回收和异常安全。教学时可以用固定 worker 数简化，但必须在文字中说明简化假设。

Epoch 的暂停问题 Epoch 回收的优势是读路径轻，通常只需要进入和退出临界区时标记 epoch。但它害怕暂停线程。若一个线程进入 epoch 后被操作系统长时间抢占、阻塞在 I/O 或执行用户回调，所有在该 epoch 后退休的节点都可能无法释放。内存占用不断增长，系统可能因为一个暂停线程而退化。

因此 epoch 临界区必须短，不能在里面执行阻塞操作、分配大量内存、调用未知回调或等待锁。无锁结构常说“不要在临界区阻塞”，这里不仅是为了进展保证，也是为了回收。一个系统如果 worker 可能被长期暂停，hazard pointer 或引用计数可能更适合；如果操作非常短且线程数固定，epoch 更简单。

RCU 的读多写少模型 Read-Copy-Update 可以看成一种读多写少的发布和回收模型。读者进入 RCU read-side 临界区后，可以无锁读取当前指针；写者复制一份新结构，修改后发布新指针，然后等待所有旧读者离开 grace period，再释放旧结构。它适合路由表、配置表、只读索引、分片元数据等读频率高、更新频率低的场景。

RCU 的代价是写路径复制和延迟回收。若结构很大，复制成本高；若读者临界区很长，旧版本保留时间长；若更新频繁，多个旧版本会堆积。它不是通用替代 mutex 的工具。对本书项目而言，分

布式路由表和配置快照可以用 RCU 思想解释：worker 读取稳定快照，driver 发布新版本，旧版本在没有读者后释放。

SPSC、MPSC、MPMC 的接口差异 队列名称必须包含生产者和消费者模型。SPSC 允许一个生产者和一个消费者，因此 head 只有消费者写，tail 只有生产者写，内存序和 cache line 布局都相对简单。MPSC 多个生产者争用 tail，消费者独占 head。SPMC 生产者独占 tail，多个消费者争用 head。MPMC 两端都有多个写者，最复杂。

接口也要体现差异。SPSC 可以在构造时绑定 producer 和 consumer，或者在文档中明确不可跨线程多端调用。MPSC 的 `push` 可能需要返回失败或阻塞，必须处理多个生产者之间的公平性。MPMC 的 `try_push` 和 `try_pop` 在高竞争下 CAS 失败很正常，调用者要能接受重试成本。把一个 SPSC 类型命名成 `FastQueue` 是危险的，因为调用者不知道约束。

有界和无界的取舍 有界无锁队列用固定数组，容量清楚，内存布局稳定，适合实时和高性能场景。队列满时返回失败或背压。无界队列通常需要动态分配节点，避免固定容量限制，但引入分配器、回收和内存峰值问题。很多系统宁愿选择有界队列，因为过载时应该背压，而不是无限分配直到 OOM。

有界队列的容量也不是小细节。容量太小，生产者频繁遇到满，吞吐下降；容量太大，缓存局部性变差，排队延迟增大，过载被隐藏更久。容量应来自吞吐、延迟和突发流量模型。异步 I/O 章和分布式 shuffle 章会继续使用这个思想：有界缓冲让过载显性化。

无锁结构里的内存分配 在 lock-free 操作内部调用通用分配器，可能破坏进展保证。分配器内部可能加锁，可能触发系统调用，可能因为内存压力阻塞。若算法声称 lock-free，但每次 push 都要进入可能阻塞的 allocator，那么整体语义要重新说明。成熟无锁队列常使用预分配节点池、有界数组或专门的非阻塞分配策略。

即使分配本身很快，释放也不能随意发生。pop 成功后节点从结构中摘除，但其他线程可能仍持有旧指针用于 CAS 验证。直接 delete 会制造 use-after-free。无锁结构的性能报告如果没有说明分配和回收，就只讲了一半。

测试无锁结构 无锁结构测试要覆盖三类性质。第一，功能正确：没有丢失、重复、越界、错误顺序。第二，并发安全：TSan 无数据竞争，压力测试在多线程、多核心、不同容量下稳定。第三，进展和资源：高竞争下不会无限自旋，失败重试次数可观测，retired 节点不会无限增长，关闭和析构不会悬挂。

测试数据应带唯一 ID。生产者生成连续 ID，消费者记录到位图或哈希集合，最终检查每个 ID 恰好出现一次。若队列只保证每生产者 FIFO，就为每个生产者单独检查 sequence；若队列不保证顺序，就不要要求全局 FIFO。内存回收测试应故意让消费者慢、生产者快、线程暂停和节点复用频繁，以增加 ABA 暴露机会。

```
lock-free queue test matrix:
capacity: 1, 2, 3, power of two, non power of two
producers: 1, 2, many
consumers: 1, 2, many
workload: burst, steady, producer fast, consumer fast
checks: no loss, no duplicate, declared order, no stuck shutdown
```

何时不要自研无锁结构 如果问题的主要成本是 I/O、序列化、哈希计算、远端网络或磁盘，队列是否无锁可能不重要。如果临界区保护复杂不变量，使用 mutex 可能更清楚。如果团队没有能力维护内存回收协议，自研无锁结构风险很高。如果已有成熟库满足需求，优先使用成熟库并写清接口保证。无锁结构适合明确的热点、清晰的生产者消费者模型、固定容量或明确回收策略，以及足够的测试预算。

本书仍然详细讲无锁，是为了让读者理解现代系统里的边界：原子不是魔法，线性化点要证明，ABA 要处理，回收要设计，进展保证要声明。理解这些以后，即使最终选择 mutex，也会是更成熟的选择。

MPMC 有界队列的序号模型 MPMC 有界队列常用每槽位 sequence 模型。数组容量固定，每个槽位有一个 sequence。生产者读取 tail，找到槽位，比较槽位 sequence 是否等于 tail，表示空

闲；成功 CAS tail 后写数据，再把槽位 sequence 设置为 tail + 1，表示可读。消费者读取 head，找到槽位，比较 sequence 是否等于 head + 1，表示可读；成功 CAS head 后读数据，再把 sequence 设置为 head + capacity，表示下一轮空闲。

这个模型的难点在于同一个槽位会被循环复用。只用 Empty/Ready 两个状态不足以区分第几轮，sequence 把轮次编码进状态，避免旧状态被误认。它也展示了无锁结构的典型复杂性：tail/head 只是预留进度，槽位 sequence 才说明数据是否真正发布。生产者预留成功但尚未写完时，消费者不能读这个槽位。

教学中不必完整实现工业级 MPMC，但要让读者能画出状态。某个槽位从空闲到写入中到可读到读取后空闲，每次状态变化的发布点在哪里，哪个线程写 sequence，哪个线程读 sequence，失败时是否重试，队列满如何判断，都是正确性核心。

Work stealing deque 的边界竞争 工作窃取 deque 比普通队列更特殊。Owner 从 bottom push/pop，stealer 从 top steal。大多数时候 owner 独占 bottom，操作很快；只有当队列接近空，owner pop 和 stealer steal 可能争夺最后一个元素。这个边界需要 CAS 决定谁赢。若处理不当，可能同一个任务被执行两次或丢失。

Chase-Lev deque 的论文和实现值得阅读，但不要在没测试和理解的情况下照抄。它涉及 top、bottom、环形数组扩容、内存序和最后元素竞争。教学项目可以先用 mutex deque 验证工作窃取策略，再考虑无锁 deque。策略正确和数据结构无锁是两件事，分阶段实现更稳。

无锁和可阻塞边界 很多系统需要同时支持无锁 fast path 和阻塞等待。SPSC ring buffer 可以在 try_push 满时返回 false；上层若需要阻塞，就用 condition variable、semaphore 或事件机制等待空间。把阻塞语义直接塞进无锁结构，会增加复杂度。常见做法是底层提供非阻塞 try 操作，上层用有界队列协议处理等待、关闭和取消。

这种分层很重要。无锁结构负责快速状态转换，运行时负责背压和生命周期。若一个无锁队列没有关闭语义，上层必须定义关闭时如何唤醒等待者、如何拒绝新写、如何 drain 旧数据。不要以为无锁结构自动解决线程池退出问题。

内存回收的测试 无锁回收测试要专门设计。普通功能测试很难触发 ABA 和 use-after-free。可以使用小对象池，故意快速复用地址；使用少量节点，增加循环复用频率；插入 yield 或短 sleep，让线程在读取指针后暂停；开启 AddressSanitizer 或 ThreadSanitizer；统计 retired 列表长度，确保不会无限增长。

测试还要覆盖线程退出。一个线程持有 hazard pointer 后退出，系统是否清理槽位？一个线程进入 epoch 后长时间暂停，内存是否持续增长？对象池销毁时，是否还有未释放 retired 节点？这些问题在 happy path 中看不到，但生产中非常重要。

无锁结构和异常 无锁结构通常要求热路径不抛异常。若 push 过程中构造元素可能抛异常，槽位状态如何回滚？若 CAS 成功预留了位置，元素构造失败，消费者如何知道该槽位不可读且队列仍然一致？为了避免这类问题，有界队列常要求元素可无异常移动，或先在外部分构造好对象，再把指针或索引放入队列；也可以使用预构造槽位和明确状态机。

C++ 泛型无锁容器因此很难写。支持任意 T、任意构造抛异常、任意析构副作用，会让协议复杂很多。教学示例使用 `std::int32_t` 或指针，不是偷懒，而是先隔离同步机制。生产容器要在模板约束和文档中写明要求。

无锁指标 无锁结构需要自己的指标。吞吐之外，应记录 CAS 尝试次数、CAS 失败次数、队列满次数、队列空次数、最大重试次数、retired 节点数量、回收扫描次数和平均扫描成本。没有这些指标，高竞争下的性能问题很难解释。一个 lock-free 队列吞吐高但 CAS 失败极多，可能浪费 CPU；一个 epoch 回收读路径快但 retired 节点堆积，可能隐藏内存风险。

指标也要避免成为共享热点。每个 worker 可以维护本地失败计数，最后汇总。若每次 CAS 失败都更新全局原子指标，指标本身会改变性能。观测性仍要遵守数据布局和同步原则。

成熟库评估清单 使用成熟无锁库也要评估。它支持 SPSC、MPSC 还是 MPMC？有界还是无界？内存回收策略是什么？元素类型要求是什么？是否支持阻塞等待、关闭、取消？是否保证 FIFO？是否有 ABA 保护？是否支持目标平台和编译器？许可证是否合适？测试和维护状态如何？

不要只因为库名里有 lockfree 就引入。若项目只需要单生产者单消费者，使用简单 SPSC ring buffer 可能更清楚；若需要复杂 MPMC 阻塞队列，成熟库可能比自研安全；若需要关闭和 drain，库的语义必须匹配。库选择同样是工程设计。

无锁学习路线 本章建议学习路线是：先用 mutex 队列理解不变量；再写 SPSC ring buffer 理解 acquire/release 和单写者；再学习 MPSC 预留和提交分离；再学习 MPMC sequence 槽位；再学习 ABA 和回收；最后阅读工作窃取 deque。不要跳过前面直接写 MPMC。每一步都要有测试和线性化点说明。

Compute Systems Engine 也应按这个路线演进。第一版使用 mutex bounded queue；第二版在单向管线中使用 SPSC；第三版只在明确瓶颈处评估 MPMC；工作窃取先用锁实现策略，再考虑无锁 deque。这样项目可以持续正确，而不是为了“高级”牺牲可维护性。

案例：日志管线中的 SPSC 日志管线可以用 SPSC 连接一个读取线程和一个解析线程。读取线程是唯一 producer，解析线程是唯一 consumer，固定 batch 槽位在 ring buffer 中循环使用。这个场景非常适合 SPSC，因为生产者和消费者模型清楚，容量固定，背压明确。若后续增加多个解析线程，SPSC 就不再适用，需要每个解析线程一个队列，或者改用 MPSC/MPMC。

这个案例强调接口约束。类型名和文档应写清 SingleProducerSingleConsumer。运行时如果需要动态扩展消费者，不能偷偷复用 SPSC。无锁结构的安全来自约束被遵守，而不是来自代码本身能抵抗所有误用。

案例：对象池中的 ABA 对象池会复用 slot，因此特别容易出现 ABA 形态。线程 A 持有 slot 5 的旧句柄；线程 B 释放 slot 5，又分配给新对象；线程 A 只看 index 仍然是 5，以为对象没变。Generation 可以解决这个问题。每次 slot 复用 generation 加一，句柄包含 index 和 generation，访问时检查两者匹配。

这个结构不一定是 lock-free，但它体现了 ABA 思想。ABA 不只属于无锁栈，也属于任何“位置复用但旧引用可能存在”的系统。分布式里的 epoch、防旧 leader，也是同一思想在更大尺度上的版本。

无锁章验收清单 项目若加入无锁结构，必须提供：适用生产者消费者模型，容量和满空语义，线性化点，内存序理由，关闭和取消如何处理，内存回收策略，ABA 防护，元素类型要求，压力测试矩阵，指标和 fallback。缺少其中任何一项，都不应把它作为默认基础设施。无锁结构进入系统的门槛应该高于 mutex 结构。

无锁章节总结 无锁结构的难点不在少写一个 mutex，而在进展保证、线性化点、内存序、ABA、回收、异常和误用边界。SPSC、MPSC、MPMC 是不同问题，不能互相冒充。工程上应优先把生产者消费者模型说清楚，把锁版本作为 reference，再在明确热点上引入无锁。无锁代码必须可证明、可测试、可回退。

无锁决策表 考虑无锁结构前，先问：生产者和消费者数量是否固定，容量是否可以有界，元素是否可无异常移动，是否需要阻塞等待，是否需要关闭和取消，节点是否动态分配，是否有回收协议，CAS 失败是否可接受，尾延迟是否重要。如果答案不清楚，优先使用锁结构。无锁是高要求工程工具，不是默认优化按钮。

案例：无锁队列的关闭外置 一个 SPSC ring buffer 可以只提供 `try_push` 和 `try_pop`，关闭状态由外层管线管理。外层有一个原子 closed 标志和条件变量或 eventfd 唤醒等待者。这样 ring buffer 保持简单，上层负责生命周期。若把关闭、阻塞等待、取消和超时全部塞入 ring buffer，结构会迅速复杂。

分层设计让每个组件职责明确。底层 ring 负责无锁数据交换，上层 channel 负责阻塞和关闭，运行时负责任务生命周期。系统工程常常不是写一个万能类，而是把几个小语义组合起来。

无锁结构章的回归阅读要先问“为什么锁版不够”。低竞争队列、复杂关闭语义、需要阻塞等待和对象生命周期复杂的场景，mutex 加条件变量往往更清楚。适合进入无锁讨论的，是 SPSC ring 这类边界窄、单生产者单消费者、head 和 tail 写者唯一、容量固定、元素生命周期可证明的结构。

SPSC ring 的重点不是环形下标，而是发布顺序。生产者先写槽位，再 release 更新 tail；消费者 acquire 看到 tail 后读取槽位，再更新 head。容量要留一个空槽或显式区分满空，索引类型要位宽明确，wrap around 要测试。这个结构讲清楚后，再看 MPSC、MPMC、Treiber 栈和 hazard pointer，读者才不会把所有无锁代码都当成 CAS 模板。

本章还必须把内存回收放到中心。ABA、hazard pointer、epoch reclamation 和 RCU 讨论的都是“读者还可能访问节点时，谁能释放它”。如果书中只给无锁 push/pop 而不讲回收，就会误导读者。项目实现可以保留锁版 reference，并把无锁版限定在清楚的生产者消费者模型内。

Treiber 栈适合作为反面教学案例。push 和 pop 的 CAS 看起来短，但 pop 读到 head 后，另一

个线程可能弹出并释放该节点，再把同一地址重新用于新节点，ABA 就出现了。给指针加版本标签可以发现部分 ABA，hazard pointer 可以让读者声明“我正在使用这个节点”，epoch 回收则批量延迟释放。每种方案都增加内存、扫描或延迟成本，不能只把它们写成术语。

无锁进展保证也要精确。Lock-free 表示系统整体总有线程前进，不表示每个线程都不会饥饿；wait-free 才给每个操作有界完成保证，但实现成本通常更高；obstruction-free 还要弱。项目文档应写明使用哪种保证，以及为什么足够。若一个结构在高竞争下 CAS 失败率很高，理论上 lock-free，工程上也可能不如锁版稳定。

SPSC ring 的完整实现要列出不变量：容量固定，head 只由消费者写，tail 只由生产者写，元素在发布前完成构造，消费后槽位可以复用，满和空可区分。测试要覆盖 wrap around、容量一半、容量减一、生产者提前关闭、消费者读完后关闭。这个小结构讲清楚，比直接给复杂 MPMC 队列更有教学价值。

无锁章节还要提醒读者使用成熟库。生产系统中，复杂 MPMC 队列、RCU 和 hazard pointer 最好优先选择被充分测试的实现，自己实现必须有模型、压力测试和回收证明。本书给出实现草图是为了理解原理，不是鼓励把教学代码直接放进关键路径。这个边界讲清楚，才是负责任的工程教材。

Hazard pointer 的直觉可以用“先声明再解引用”解释。线程准备读取某个节点时，先把节点地址写入自己的 hazard slot，再重新检查共享指针是否仍然指向这个节点；若仍然相同，才能安全读取节点内容。删除者不能立刻释放弹出的节点，而是放入 retire list，定期扫描所有 hazard slot，确认没有线程声明正在使用它，再真正释放。这个流程增加了扫描和延迟，但换来明确的安全边界。若省掉重新检查，就可能声明了旧节点，却读到已经被替换的结构。

Epoch reclamation 的直觉则是“分批过期”。所有线程进入读侧临界区时记录当前 epoch，删除者把节点放入当前 epoch 的回收列表；只有当所有线程都离开旧 epoch，旧列表里的节点才可释放。它比 hazard pointer 扫描单个指针更批量，适合读多写少，但一个长期不退出临界区的线程会阻塞回收，造成内存积压。教材要把这些成本说清楚，否则读者会以为回收只是给无锁代码加一个库函数。

RCU 还要强调适用范围。它适合读多写少、读路径极短、更新可以复制或延迟发布的结构，例如路由表、配置快照和读多的索引；它不适合频繁写、需要强实时释放或读侧可能阻塞很久的路径。第二册只需建立模型：读者看到无锁指针结构时，第一反应不是“CAS 怎么写”，而是“读者使用期间节点怎样保持存活”。

12.5 无锁结构先讲适用范围和回收边界

无锁数据结构不是“把锁拿掉”。它要求操作在某种进展保证下完成，例如 lock-free 表示系统整体总有线程能前进，wait-free 表示每个线程在有限步内前进。进展保证只说明阻塞性质，不自动说明延迟更低、吞吐更高或代码更可靠。无锁结构常把锁竞争换成 CAS 重试、内存序推理、节点生命周期和调试困难。

学习顺序要从 SPSC ring buffer 开始。单生产者单消费者有天然所有权：生产者独写 tail 和数据槽位，消费者独写 head，双方通过 acquire/release 交换可见性。这个结构能清楚展示发布、容量、环绕、满/空判断和 false sharing。直接跳到 MPMC 队列，会把所有难点叠在一起，读者看不到每个机制为什么需要。

SPSC ring buffer 的不变量

SPSC ring 的核心不变量是容量、head、tail 和槽位状态。常用设计让容量为固定值，保留一个空槽区分满和空，或者额外维护 size。生产者写入元素后 release 更新 tail；消费者 acquire 读取 tail 后才能读取元素。head 和 tail 应避免落在同一 cache line，减少 false sharing。

环形缓冲区最常见错误是 off-by-one、覆盖未消费元素、读取未发布元素和关闭语义缺失。测试要覆盖容量 1 或 2、刚好满、刚好空、环绕多轮、生产者提前结束、消费者关闭等待。若元素类型有非平凡构造析构，还要处理对象生命周期：槽位内存什么时候构造，什么时候析构，异常时如何恢复。

SPSC 适合对一流水线，例如解析线程把 batch 交给计算线程。若有多个生产者或多个消费者，不能直接复用 SPSC 代码。可以按生产者分配多个 SPSC 队列，再由消费者轮询或 steal；也可以进入 MPSC/MPMC 结构，但复杂度上升。章节要强调“适用范围”是设计的一部分。

Treiber 栈、ABA 和内存回收

Treiber 栈是 CAS 学习经典例子：push 构造新节点，让 new node 指向旧 head，再 CAS head；pop 读取 head 和 next，再 CAS head 到 next。线性化点清楚，但生命周期不清楚。pop 线程读到 head 后，另一个线程可能 pop 并释放该节点，当前线程再读 next 就悬垂。仅仅 CAS head 不能解决节点安全访问。

ABA 是同一问题的另一个表现。节点地址 A 被 pop、释放、重新分配，又回到 head，CAS 看到还是 A，误以为结构未变。标签指针给指针加版本，能降低 ABA 风险；hazard pointer 让线程声明自己正在访问哪些节点，释放者延迟回收；epoch reclamation 让节点在所有线程离开旧 epoch 后再释放；RCU 适合读多写少场景。每种方案都有成本和适用范围。

教材不能只给 Treiber 栈代码然后说“完整实现省略”。至少要明确：没有回收协议的 Treiber 栈只能作为教学骨架，不能进入项目热路径。若为了教学实现，可以选择不释放节点或在测试结束统一释放，但必须写清这不是生产方案。

MPMC 队列和工程替代

MPMC 队列比 SPSC 难得多，因为多个生产者竞争 tail，多个消费者竞争 head，槽位状态、序号、内存序和回收都复杂。高性能库通常经过多年验证，不建议项目随手手写生产级 MPMC。第二册可以讲原理和阅读成熟实现，但贯穿项目优先使用 mutex queue、多个 SPSC 或经过验证的库思想。

工程替代常常更好。若每个 worker 有本地队列，跨线程 steal 低频发生，可以用每队列 mutex 先验证策略；若生产者消费者关系固定，用 SPSC 组合；若热点在队列外，复杂无锁队列没有收益。无锁是否值得，要用端到端 profile 判断。队列操作只占 2

线性化点是测试和审查核心。每个操作要说明从外部看在哪一瞬间生效；失败操作是否改变状态；取消或关闭如何与正在进行的操作交错。没有线性化点，压力测试偶尔通过也不能证明正确。

并发测试和源码阅读

无锁测试要包含 reference、压力、随机交错和模型化思维。SPSC 可以用单调序号验证不丢不重；栈可以用多线程 push/pop 后集合比较；队列可以记录每个元素唯一 id。测试还要跑 ThreadSanitizer，但对自定义原子内存序代码可能需要理解工具限制。随机 sleep 和 yield 能增加交错，但不是证明。

Linux 源码阅读可以看 RCU、lock-free list 思想和内核 ring buffer 的部分设计。目标不是照抄，而是观察成熟系统如何把读侧、写侧、回收、barrier 和调试接口分开。用户态 C++ 项目要遵守标准内存模型，不能直接搬内核宏。

本章项目交付物：一个 SPSC ring buffer 的完整实现和测试，一个 Treiber 栈的教学骨架加“为什么不能生产使用”说明，一个无锁决策表。决策表写：是否有固定单生产者/单消费者，是否需要多生产者，是否能接受锁，队列是否真是瓶颈，回收协议是否已有验证。这样读者不会把无锁当成性能捷径。

案例走查：SPSC ring 从 mutex queue 演进而来

先写 mutex queue reference，保证一个解析线程生产 batch，一个计算线程消费 batch，输出不丢不重。然后观察热点：若队列操作在 profile 中占比高，且确实只有单生产者单消费者，才引入 SPSC ring。若生产者可能多个，或者队列不是瓶颈，就不该提前复杂化。

SPSC ring 的测试用单调序号最直观。生产者写 0 到 N-1，消费者要求严格递增；再加入不同容量、环绕、多轮、生产者提前关闭和消费者慢速。指标包括吞吐、缓存行对齐影响和 CPU 占用。若容量太小，生产者频繁等待；容量太大，缓存局部性下降。容量也是系统参数，不是随便写一个大数。

这个案例让无锁结构落地：先有锁版本的语义，再有瓶颈证据，再有更窄适用范围的无锁版本，再有比锁版更严格的压力测试。顺序反过来就会变成危险代码。

内存回收的教学底线

任何弹出节点并释放内存的无锁结构，都必须讲回收。哪怕本书不完整实现 hazard pointer，也要说明没有回收协议的代码不能用于生产。教学可以使用三种降级：测试中不释放节点，epoch 结束统一释放，或用锁保护回收路径。每种降级都要写清限制。

Hazard pointer 的直觉是“我要读这个节点，先公开声明它危险，释放者看到有人声明就暂缓释放”。Epoch 的直觉是“线程进入读侧临界区，退出后推进时代，旧时代的节点等所有线程离开后释放”。RCU 的直觉是“读侧极轻，写侧复制更新并延迟回收”。三者不是背名词，而是不同读写比

例和延迟要求下的回收合同。

项目里如果只需要 SPSC ring，就不必引入节点回收。固定数组槽位没有 pop/delete 节点问题，生命周期由队列对象管理。这也是为什么学习从 SPSC 开始：它展示无锁发布和消费，却避开最难的通用回收。

无锁结构的源码阅读方法

阅读成熟无锁队列时，不要从第一行代码啃到最后。先找数据成员：head、tail、sequence、buffer、hazard records。再找线性化点：哪一个 CAS 或 store 让操作生效。再找内存序：哪条 release 发布数据，哪条 acquire 读取数据。最后找回收：节点什么时候安全释放。

读不懂全部细节时也要能写阅读笔记。比如“这个队列用每槽 sequence 区分空和满，生产者 CAS tail 取得槽位，写数据后 release 更新 sequence；消费者 acquire 读取 sequence，CAS head 取得槽位”。能写出这种摘要，说明你已经抓住主线。不要把无锁源码当成神秘技巧堆。

本章作业可以给一段缺少回收的 Treiber 栈，让读者标出线性化点、ABA 风险和悬垂读取位置，再提出三种修复方向。答案不要求写完整 hazard pointer，但必须说明每种方向的成本。

综合练习：判断一个无锁替换是否值得

给出一个系统，profile 显示队列操作占总耗时 4

接着修改输入，让任务极小且队列操作占 35

第三步审查回收。若方案包含动态节点，就必须说明 hazard pointer、epoch 或不释放节点的教学限制。若方案使用固定环形缓冲，就说明容量、满空判断和关闭语义。没有回收说明的无锁方案不得通过。

这个题让读者明白“无锁是否值得”是端到端问题。数据结构再漂亮，如果不在主成本路径上，就是不必要复杂性；反过来，若队列确实成为瓶颈，也要选择最窄适用范围的结构。

容易出错的边界：无锁代码的隐藏成本

第一个隐藏成本是重试。CAS 失败循环在低争用下很好，高争用下会消耗大量 CPU。第二个隐藏成本是内存回收。锁保护结构中，删除节点后没人再访问通常容易保证；无锁结构中，其他线程可能已经读到节点指针。第三个隐藏成本是测试。很多错误只在罕见交错下出现，普通单元测试很难覆盖。

第四个隐藏成本是可维护性。一个团队里能正确修改无锁 MPMC 队列的人很少，错误修改代价高。第五个隐藏成本是可观测性。锁等待可以用 futex 或队列指标看，无锁自旋和重试若不记录，问题会隐藏成 CPU 高。第六个隐藏成本是平台差异。内存序错误可能在某些架构上更容易出现。

本章源码索引要求读者阅读无锁结构时先找四个点：线性化点、所有权转移点、内存序发布点、回收点。找不到其中任何一个，就不能声称理解了结构。对于 SPSC ring，线性化点可能是 tail 发布或 head 发布；所有权在槽位状态改变时转移；回收由固定数组生命周期保证。对于 Treiber 栈，回收点正是难点。

项目中应把无锁结构放在小范围内。SPSC ring 可以服务固定一对生产消费；通用队列先用锁版；复杂无锁只作为阅读和可选扩展。这样的克制不是保守，而是对正确性成本的尊重。

本章核心接口：ProgressAndReclaim

无锁章给项目留下 ProgressAndReclaim 审查项。每个无锁候选结构必须说明进展保证、适用生产者消费者数量、线性化点、内存序、回收策略和 fallback。SPSC ring 的回收策略是固定槽位生命周期；Treiber 栈教学版的回收策略是“不生产使用”。这两者都要明说。

后续如果项目想加入 MPMC 队列，必须先填这张审查项。填不出来，就不能替换锁版。接口的意义在于让复杂性先过审，而不是让代码先进入仓库再补解释。

从同步结构进入并行算法运行时

无锁章结束后，读者应明白同步结构有成本和适用范围。下一部分进入并行算法和运行时，不是为了继续追求更复杂同步，而是把同步选择放进算法和任务图里。很多时候，正确的算法切分能避免共享写，比换无锁结构更有效；任务本地 partial、scan 分配位置、工作窃取本地队列，都在减少高频共享。

项目推进时，先问能否改变数据流减少冲突，再问需要什么同步。若 filter 可以通过 scan 分配输出位置，就不需要多个线程 push 共享 vector；若 histogram 可以线程本地 partial，就不需要每条记录 CAS 全局桶；若任务图能 continuation，就不需要 worker 阻塞等待。同步章给的是工具，算法章决定是否需要这些工具。

12.6 项目落地

Compute Systems Engine 的无锁路线应分三步。第一步，实现并测试 SPSC ring buffer，只用于单生产者单消费者管线，例如一个 I/O 模拟线程向一个计算线程发送批次。第二步，比较 SPSC 和 mutex bounded queue 在目标场景下的吞吐和等待行为，确认收益来自减少锁和固定内存，而不是改变语义。第三步，再研究 MPMC 或工作窃取 deque，并在实现前写出不变量、线性化点、内存序和回收策略。

项目不应急着把所有队列替换成无锁队列。线程池全局队列最开始用 mutex 更容易证明关闭语义；SPSC 用于清晰管线；MPMC 用于明确的多生产者多消费者热点；work stealing deque 用于本地队列和窃取。不同结构服务不同场景。

进展保证 wait-free、lock-free、obstruction-free 不是形容词，而是失败和竞争下的承诺。教学项目至少要说明：某个线程暂停时，其他线程是否仍能推进。

线性化点 每个成功操作都要有一个瞬间使它看起来原子发生。队列入队可能在线程写槽位时线性化，也可能在发布序号时线性化。没有线性化点，正确性无法证明。

ABA 指针从 A 变 B 再变回 A，CAS 只看值会误以为没变。版本号、tagged pointer、hazard pointer 和 epoch 都是围绕这个问题展开的不同策略。

内存回收 删除节点后不能立即 free，因为别的线程可能还拿着旧指针。回收是无锁结构最困难的部分之一，必须和算法一起设计。

SPSC 单生产者单消费者 ring buffer 可以用较弱同步表达，是学习发布和消费边界的好起点。它不代表 MPMC 简单，只是限制了竞争形态。

MPMC 多生产者多消费者需要为每个槽位记录序号，区分空、可写、可读和已消费。全局 head/tail 不够表达所有状态。

性能反例 高竞争 CAS 会反复失败，消耗 cache 一致性流量。若临界区很短、竞争很高，锁加睡眠可能比忙等 CAS 更好。

测试方法 无锁测试要结合线性化检查、压力、随机 yield、TSAN 能力边界和长期运行。普通单线程测试只能证明接口大致能用。

案例：SPSC ring 先写固定容量 SPSC 队列，明确 producer 写数据后 release 发布序号，consumer acquire 读取序号。

案例：ABA 小剧场 用复用节点池构造 A-B-A，让读者看到值相同不代表历史相同。

案例：epoch 延迟回收 线程进入读侧临界区登记 epoch，删除节点进入 retire 列表，等待所有读者越过 epoch 后再释放。

源码阅读路线 Linux 源码阅读从 `kernel/rcu`、`include/linux/rcupdate.h`、`kernel/trace/ring_buffer.c` 附近进入即可。阅读目标不是把整套子系统抄进书里，而是追一条和本章实验相关的路径：用户态动作怎样变成内核对象，内核对象怎样排队、等待、唤醒、计数、提交或回收。

阅读报告要写证据等级。能直接测到的数字放在第一层；从源码控制流推导出的解释放在第二层；当前设备无法验证、只能作为合理猜测的内容放在第三层。这样读者会知道无锁生命周期报告中哪些结论可复现，哪些结论需要换机器继续确认。

源码阅读还要克制。看到一个复杂函数，不要把每个分支都展开；先回到本章的问题，问它能否解释一个等待点、一次状态转移或一条性能曲线。解释不了的内容可以留到索引，不要打断正文主线。

SPSC ring 先写固定容量 SPSC 队列，明确 producer 写数据后 release 发布序号，consumer acquire 读取序号。

ABA 小剧场 用复用节点池构造 A-B-A，让读者看到值相同不代表历史相同。

epoch 延迟回收 线程进入读侧临界区登记 epoch，删除节点进入 retire 列表，等待所有读者越过 epoch 后再释放。

进展保证

lock-free 保证系统整体前进，不保证每个线程都快。wait-free 更强但更复杂。没有 mutex 不等于真正 lock-free。

实验设计：在高竞争 CAS 下记录失败次数和尾延迟。

SPSC 所有权

SPSC 简单是因为 head 只有消费者写，tail 只有生产者写。无锁不是靠运气，而是靠写者数量约束。

实验设计：故意让两个生产者使用 SPSC，说明算法假设被破坏。

线性化点

每个成功操作必须能指出瞬间生效的位置。push 可能在线程发布 tail，pop 可能在读取元素并归还槽位。

实验设计：为 try push、try pop、满和空四种路径标出线性化解释。

ABA

指针从 A 到 B 再回 A，CAS 只看地址会误判。版本标签、hazard pointer 和 epoch 都围绕生命周期变化展开。

实验设计：画出 Treiber 栈 ABA 时间线。

hazard pointer

读者先公布自己要解引用的指针，再二次确认共享指针未变。删除者扫描 hazard 后才能释放。

实验设计：模拟 retired 列表和 hazard 槽位，说明释放条件。

epoch

epoch 把回收延后到所有旧读者离开阶段。读路径轻，但暂停线程会阻塞回收。

实验设计：让一个 worker 长时间停在 epoch 内，观察 retired 节点增长。

RCU

RCU 适合读多写少：写者复制并发布新版本，等待旧读者离开后释放旧版本。它不是通用无锁替代品。

实验设计：用路由表快照解释 RCU 思想。

有界队列

固定数组让容量和内存可控，无界节点队列会引入分配和回收。过载时背压通常比无限分配更可靠。

实验设计：比较有界返回失败和无界持续分配的内存行为。

测试矩阵

无锁测试要覆盖容量一、非二次幂、多生产者、多消费者、随机 yield 和长时间压力。单线程测试没有意义。

实验设计：用唯一 ID 检查丢失、重复和顺序承诺。

项目接口

LockFreeContract 记录生产者消费者模型、容量、线性化点、进展保证、回收策略、失败路径和测试矩阵。

实验设计：没有写清这些字段的无锁结构不进入项目主路径。

12.7 无锁结构的难点在对象生命周期，不在少写一把锁

无锁数据结构很容易被神化。短短几行 CAS 循环看起来优雅，真正难的是线性化点、进展保证、ABA 和内存回收。教材应明确告诉读者：没有 mutex 不等于 lock-free；lock-free 不等于每个线程都快；通过小测试不等于回收协议正确。无锁章节要降低炫技感，提高证明密度。

SPSC ring buffer 是最好的起点，因为它的所有权清楚。生产者独写 tail，消费者独写 head；生产者只读取 head 判断空间，消费者只读取 tail 判断数据。单写者约束让同步简单很多。若两个生产者同时写 tail，这个算法立即失效。队列类型名和文档必须写清 SPSC，而不是命名成模糊的 FastQueue。接口语义本身就是正确性的一部分。

线性化点是并发对象的核心。成功 push 何时对消费者可见？通常是 release 发布 tail 或槽位状态的那一刻。成功 pop 何时从队列移除元素？可能是读取元素并发布 head 的组合。失败路径也要解释：队列满返回 false 的瞬间是什么，队列空返回空值的瞬间是什么。没有线性化点，所谓线程安全只是愿望。

ABA 要和回收一起讲。Treiber 栈中，线程 T1 读到 head 指向 A 后暂停；T2 弹出 A 和 B，释放 A，又把同一地址分配给新节点 C；T1 恢复看到地址还是 A，CAS 成功，却把结构改坏。问题不是字母游戏，而是地址相同不代表对象生命周期相同。版本标签、hazard pointer、epoch 和 RCU 都是在解决“旧指针是否还能安全解引用”。

Hazard pointer 的流程要讲细：读共享指针，发布到本线程 hazard 槽位，重新读取共享指针确认未变，然后才能解引用。删除者不能立即释放节点，而是放入 retired 列表，扫描所有 hazard 后释放无人保护的节点。它的成本是读路径发布和释放路径扫描。epoch 的成本不同：读路径轻，但暂停线程会阻塞整个旧 epoch 回收。两者没有绝对优劣，取决于线程行为和回收延迟。

无锁结构也可能性能很差。高竞争 CAS 循环会不断失败，烧 CPU，制造 cache 一致性流量；mutex 在竞争严重时让线程睡眠，反而可能降低资源浪费。服务系统关心尾延迟和核心预算，不只关心平均吞吐。实验要记录 CAS 失败次数、重试分布、CPU 占用和尾延迟，而不是只喊“无锁更快”。

测试矩阵必须严肃。容量一、容量二、非二次幂容量，多生产者、多消费者，随机 yield，长时间压力，TSan 能力边界，节点快速复用，线程暂停，关闭和析构，都要覆盖。生产者可以生成唯一 id，消费者检查是否丢失或重复。回收测试要故意让旧指针更容易被复用。只有这样，无锁章节才像教材，而不是博客技巧。

案例推导：Treiber 栈为什么不能只展示 push 和 pop

Treiber 栈的 push 很短：读 head，把新节点 next 指向旧 head，用 CAS 把 head 改成新节点。pop 也很短：读 head 和 next，用 CAS 把 head 改成 next。许多资料到这里就停了，给读者一种无锁结构很简单的错觉。真正的问题从 pop 成功后开始：被弹出的节点什么时候可以释放？还有没有别的线程拿着旧 head 指针？地址是否可能被分配器复用？

ABA 时间线必须讲清。线程 T1 读到 head 为 A，准备把 head 改成 A 的 next。T1 暂停。T2 弹出 A，又弹出 B，释放 A；分配器随后把地址 A 给了新节点 C，并把 C push 回栈。T1 恢复后看到 head 地址仍是 A，CAS 成功，把 head 改成旧的 B。此时栈被破坏，因为地址相同但对象已经不是原来的 A。这个故事能让读者理解为什么“值没变”不等于“状态没变”。

带标签指针是一种思路：把地址或索引和 generation 一起比较。地址回到 A，generation 也会变化，CAS 不会误判。实际实现要考虑双字 CAS、对齐和版本溢出。教学中可以用数组索引加 generation，更容易跨平台，也能连接对象池和句柄系统。这样读者不会误学偷指针低位这种不可移植技巧。

hazard pointer 解决另一个层面：防止节点过早释放。读者读取 head 后，把它写入自己的 hazard 槽位，再重新确认 head 未变，之后才能解引用。删除者把节点放入 retired 列表，扫描所有 hazard 槽位后释放无人保护的节点。这个流程比 CAS 长得多，却是 C++ 无锁结构绕不开的内容。没有回收协议的 Treiber 栈只能算教学片段，不能算完整数据结构。

epoch 回收则把节点释放批量延后。线程进入无锁操作时标记当前 epoch，退出时离开；删除节点记录退休 epoch；所有线程离开旧 epoch 后才能释放旧节点。读路径轻，但一个线程暂停会阻塞回收。实验可以让一个 worker 故意停在 epoch 内，观察 retired 列表增长。这个实验能说明进展保证和资源回收是两条不同曲线。

最终，LockFreeContract 应要求每个无锁结构写明生产者消费者模型、线性化点、ABA 策略、回收策略、容量、失败路径和测试矩阵。若这些写不出来，用 mutex 版本更诚实。无锁不是教材里的等级徽章，而是非常具体的工程选择。

第三部分综合案例：从线程池到队列、原子和无锁生命周期

第三部分要把并发从“能多线程运行”提升到“能说明共享状态怎样变化”。贯穿项目中，reader 生产 batch，parser 消费 batch 并生产热字段，worker 做聚合，writer 接收输出。最朴素的实现是多个线程共享几个队列和计数器。它很快会遇到四类问题：任务关闭说不清，队列等待说不清，原子发布说不清，节点回收说不清。

先从线程池开始。submit 之后任务可能还在队列、正在运行、已经成功、失败或被取消。线程池关闭时，未开始任务如何处理，运行中任务是否协作取消，异常如何返回调用者，必须写成状态。若没有这些状态，后面队列关闭和 I/O drain 都无从谈起。线程池报告记录任务粒度、等待位置、worker id 和异常传播，这是并发部分的入口。

然后看有界队列。队列不是一个容器，而是一组不变量：head、tail、size、buffer 和 closed 的关系。mutex 保护这些关系，condition variable 围绕谓词等待。队列满时 producer 等待或失败，这

是背压；队列空且未关闭时 consumer 等待；队列空且已关闭时 consumer 结束。只要这张状态表写清，代码就可以审查；没有状态表，notify 顺序和虚假唤醒会变成偶发 bug。

接着看原子。队列长度的近似统计可以 relaxed，因为它只是指标；ready 标志发布 batch 不能 relaxed，因为消费者还要看 batch 字段；配置热更新不能只用 seq-cst 指针，因为旧对象何时释放仍未解决。原子同步合同要求每个原子对象说明责任：它只保护自己，还是发布其他数据，是否参与生命周期。写不出责任，就优先使用锁。

最后看无锁结构。SPSC ring buffer 适合 reader 和单个 parser 之间的固定通道，因为 head 和 tail 单写者清楚；MPSC 或 MPMC 队列复杂得多，需要槽位状态、CAS 失败路径和内存回收。Treiber 栈示例提醒读者：push/pop 代码短，完整结构难在 ABA 和回收。hazard pointer、epoch 和 RCU 都是生命周期协议，不是语法技巧。

这四层应放进同一个并发状态报告。一个 batch 的生命周期可以写成：reader 独占填充，release 发布到队列；parser acquire 获取，生成热字段；worker 读取热字段并写本地 partial；writer 接收 OutputBlock；关闭时队列 drain，未完成任务记录取消。每个状态都有所有者、等待条件和回收动作。若某个阶段“大家都可以访问”，那就是未来 bug 的位置。

实验要覆盖失败路径。线程池提交后立即关闭，队列容量为一，消费者被虚假唤醒，producer 提前退出，配置频繁热更新，SPSC 被两个 producer 误用，epoch 中一个线程暂停。这些输入规模都很小，但比大吞吐测试更能暴露语义。第三部分的目标不是让读者迷恋无锁，而是让读者能判断何时 mutex 更清晰，何时 atomic 足够，何时无锁值得承担复杂度。

实验：阅读 Compute Systems Labs 的 bounded queue，写出 mutex 版本保护的不变量。再设计一个 SPSC ring buffer 的接口，说明哪些状态可以用 relaxed，哪些位置需要 acquire/release。

无锁结构先问是否需要 无锁数据结构不是同步章节的终点，也不是性能的默认答案。它解决的是特定问题：在某些线程暂停时，其他线程仍能推进；在高争用锁上减少阻塞；在中断、信号或实时场景避免不可控等待。代价是状态机复杂、内存回收困难、测试困难、调试困难、平台语义敏感。很多普通业务队列用互斥锁更清楚、更快、更可靠。本章必须先讲适用范围。

进展保证要分清。Lock-free 表示系统整体总有线程能推进，不表示每个线程都不会饥饿；wait-free 才给每个操作有限步保证，通常更难；obstruction-free 条件更弱。若教材只说“无锁不会阻塞”，读者会误解。CAS 循环在高争用下可以反复失败，虽然没有线程睡眠，但延迟仍可能很高。

SPSC 环形队列是最好的起点 单生产者单消费者队列比 MPMC 简单，因为 head 只由消费者写，tail 只由生产者写。生产者读取 head 判断是否满，写入 buffer 后发布 tail；消费者读取 tail 判断是否空，读取元素后发布 head。这个结构能清楚展示所有权拆分、缓存行隔离、release/acquire 和容量取模。它也适合项目中某些一对一流水线，如解析线程到写出线程。

即便是 SPSC，也要处理细节：容量通常留一个空槽区分满和空；head、tail 要避免 false sharing；元素构造和析构要正确；关闭语义需要额外标志；批量 push/pop 可以减少原子读写；队列容量影响延迟和背压。把 SPSC 写完整，读者就会明白无锁不是少写代码。

Treiber 栈和 ABA Treiber 栈是 CAS 教学经典。push 创建节点并 CAS head；pop 读取 head 和 next，再 CAS head 到 next。问题在于 ABA：线程 A 读到 head 为节点 X，暂停；线程 B pop X，释放或复用 X，又 push 一个地址仍为 X 的节点；线程 A CAS 看到 head 还是 X，于是成功，但栈语义已经变了。地址相同不代表对象身份相同。

解决 ABA 有多种方向：带版本标签的指针、hazard pointer、epoch reclamation、引用计数节点、避免地址快速复用。每种都有成本。Tagged pointer 受地址位数和平台限制；hazard pointer 每个线程公布自己正在访问的节点，回收时扫描；epoch 把回收推迟到所有旧 epoch 读者离开；引用计数增加原子开销。教材不能只给一个“标准答案”，要说明它解决什么、牺牲什么。

内存回收是无锁结构的核心 很多无锁示例只写 push pop，不写回收，这在教学上是不完整的。pop 成功后节点不能立即 delete，因为其他线程可能已经读到指针但尚未完成 CAS。锁保护的数据结构可以在持锁状态判断无人访问；无锁结构缺少这种天然排他区，必须有独立回收协议。没有回收协议的无锁结构只能泄漏内存或冒 use-after-free 风险。

项目中若实现 MPSC 或 MPMC 队列，应先选择回收策略。教学版本可以限制节点来自固定池，程序结束统一释放；这能降低复杂度，但必须明说限制。进一步版本再引入 hazard pointer 或 epoch。

读者要学到：无锁算法的正确性包含线性化点和生命周期两部分，缺一不可。

测试线性化点 无锁结构的测试不能只跑功能样例。要进行多线程压力、随机操作序列、与带锁 reference 对比、线程 sanitizer、边界容量、重复关闭和内存回收检查。线性化测试可以记录每个操作的开始时间、结束时间和结果，尝试判断是否存在某个串行顺序解释所有结果。完整线性化验证很难，但小规模历史检查能发现许多错误。

项目可以实现一个带锁队列作为 reference，再实现 SPSC 或 MPSC 无锁队列。所有随机操作先在 reference 上生成期望，再在并发版本中检查不变量和最终集合。性能报告必须包含争用情况和失败重试次数。若无锁版本在低争用下更慢，也要如实解释：它买到的是进展保证或特定路径延迟，不一定买到所有场景吞吐。

Linux 源码阅读连接点 Linux 的 RCU、lockless list、per-cpu 数据和环形缓冲区都能作为阅读对象。重点看读者如何声明临界区，更新者如何发布新版本，旧对象何时释放，内存屏障在哪里，调试配置如何检查误用。内核无锁代码通常有大量注释，因为没有注释就很难维护。项目中的无锁结构也应该写明每个原子字段的读写者和内存序。

读这些源码时不要急于照搬。内核的约束和用户态 C++ 不同，可能有特定屏障原语、禁止睡眠上下文、抢占控制和体系结构封装。我们借鉴的是状态机和回收思想，不是复制宏。

事故复盘：无锁栈为什么不能只写 CAS Treiber 栈的核心 CAS 很短，事故却发生在 pop 之后。线程 A 读到旧 head 指针后暂停，线程 B pop 这个节点、释放它，又 push 一个复用到同一地址的新节点；线程 A 恢复后 CAS 看到地址相同，以为状态没变。ABA 和 use after free 说明无锁结构的难点不只是原子修改，还包括节点生命周期、线性化点和回收策略。

因此无锁结构要先讲适用范围。SPSC ring buffer 可以用每端单写的序号避免复杂回收；MPMC 队列需要每个 slot 的 sequence 区分轮次；Treiber 栈需要 hazard pointer、epoch reclamation 或其他安全回收。每个结构都要指出线性化点：哪一次 store 或 CAS 让操作对外生效。没有线性化点，测试失败时无法判断两个操作谁先发生。

性能也不能用“无锁一定快”概括。高竞争下 CAS 失败会烧 CPU，慢线程会拖住 epoch 回收，retire list 可能增长，内存峰值上升。实验要报告成功操作数、CAS 失败次数、尾延迟、回收队列长度和峰值内存。无锁章节应让读者看到：去掉互斥锁只是第一步，真正的工程边界是进展保证、生命周期和可审查的线性化证明。

无锁结构实验判读 无锁实验要同时报告成功次数、CAS 失败次数、尾延迟、retire 列表长度和内存峰值。SPSC 误用两个生产者应失败或被接口禁止；Treiber 栈若没有回收策略，只能作为教学骨架；epoch 若被慢线程拖住，内存增长要进入报告。无锁是否值得，要和锁版 reference 对照，而不是单独看吞吐。

这一段也服务于复习。读者合上书后，应该能凭本章的判读口径重新审查自己的代码：有没有最小正确版本，有没有能击穿旧方案的输入，有没有状态记录，有没有资源指标，有没有失败后的恢复动作。若五个问题缺两个以上，就说明实现还停在演示层，而不是工程层。

Treiber 栈为什么不能只给二十行代码 Treiber 栈的 push 和 pop 看起来很短，但短代码隐藏了三个问题。第一，CAS 成功那一刻才是线性化点；失败后要重新读取 head。第二，ABA 会让同一个地址代表不同历史。第三，pop 成功后节点不能立即释放，因为其他线程可能已经读到这个节点。

教学可以先给泄漏节点的版本，只为了观察线性化点；然后给复用节点池构造 ABA；最后引入 hazard pointer 或 epoch 的思想。每一步只解决一个问题，读者才能知道复杂度从哪里来。若一开始给完整生产级实现，细节会淹没原理；若只给短实现，又会误导读者把不完整代码当成可用结构。

项目中应明确写：Treiber 栈是阅读案例，不进入主路径；SPSC ring 可用于固定一对生产消费；通用队列先用锁版。这个选择不是技术保守，而是把正确性成本和收益放在同一张表里比较。

实验课：SPSC ring 和 Treiber 栈的边界对照 SPSC ring 的正确性来自限制：只有一个 producer 写 tail，只有一个 consumer 写 head。实验先实现固定容量队列，保留一个空槽区分满和空，producer 写元素后 release 发布 tail，consumer acquire 看到 tail 后读取元素。再故意用两个 producer 调用同一个 SPSC，说明算法假设被破坏，错误不是实现细节，而是模型不适用。

Treiber 栈实验只作为教学。第一版泄漏节点，避免回收干扰，观察 push 和 pop 的线性化点。第二版引入节点池复用，构造 ABA 时间线。第三版讲 hazard pointer：线程读取 head 前发布 hazard，回收者扫描 hazard 后再释放 retired 节点。每一步都只解决一个问题，让读者看到复杂度来源。

判读无锁性能时要记录 CAS 失败次数。高争用下，所有线程都在忙等失败，CPU 使用率很高

但吞吐不一定好。锁版队列可能在高争用时睡眠，尾延迟反而更稳定。报告要比较锁版和无锁版在低争用、高争用、单生产消费、多生产消费下的行为。

项目结论要克制。SPSC 可以进入固定管线，Treiber 栈保留为阅读案例，MPMC 无锁队列需要完整回收和线性化测试后才能考虑。学会不用无锁，是掌握无锁的重要部分。

无锁结构练习验收重点 合格答案至少写进展保证、线性化点、ABA 风险和回收策略。SPSC 题要说明单生产者单消费者假设；Treiber 栈题要说明为什么短代码不生产可用；epoch 或 hazard pointer 题要说明慢线程对回收的影响。没有回收策略的答案只能算教学草图。

答案还要写“为什么不采用更复杂方案”。例如没有证据证明队列是瓶颈，就不应把锁队列换成无锁；没有证明输出顺序可变，就不应随意重排；没有证明错误路径能恢复，就不应只提交正常路径。能解释不做什么，说明读者开始具备工程判断。

最后，答案必须包含可复查材料。至少有输入摘要、关键状态、期望输出、实际输出、运行命令和未验证边界。没有这些材料，老师或同伴无法判断结论是否成立。第二册的作业不是让读者写漂亮段落，而是训练读者把系统问题变成可验证证据。

项目实现：无锁数据结构 本章对应的贯穿项目实现不要求一次写成完整框架，但必须有最小可运行切片。这个切片包含三个部分：一个 reference，一个带本章机制的实现，一个能击穿 reference 之外隐含假设的测试。Reference 用来固定语义，机制实现用来展示本章知识，测试用来证明新增复杂度确实解决了问题。

代码组织上，不要把所有内容放进一个大文件。基础类型、输入生成、核心机制、报告输出和测试应分开。即使当前只是教学项目，也要让目录能表达责任边界。这样后续章节接入时，不会因为找不到状态归属而重新发明一套对象。

每个实现都要输出一段机器可读摘要。摘要至少包含输入规模、关键状态、错误数、输出校验、耗时和未验证边界。这里的机器可读不要求复杂格式，简单的 key value 文本也可以。关键是让脚本和读者都能复查，而不是只能看屏幕截图。

测试应包含正常路径、边界路径和失败路径。正常路径证明功能可用；边界路径证明长度、容量、尾部、空输入、关闭或超时没有遗漏；失败路径证明本章机制不是装饰。若失败路径写不出来，往往说明本章概念还没有落到工程对象上。

报告最后写一个取舍段。新增机制带来了什么好处，增加了什么成本，哪些场景不应该使用它，下一章会复用哪个字段或对象。这个取舍段非常重要，因为系统工程不是把所有高级机制都加进去，而是在证据和约束下选择合适的边界。

调试笔记：CAS 循环异常时记录失败历史 无锁结构吞吐异常时，要记录 CAS 尝试次数、失败次数和失败时观察到的状态。失败次数高说明竞争或算法切分问题；retire 列表增长说明回收被慢读者阻塞；ABA 复现说明地址身份不足；尾延迟高说明 lock free 不等于 wait free。没有这些历史，CPU 高只会被误读成程序很忙。

复查清单：无锁数据结构 复查本章时，先检查 reference 是否仍然存在。没有 reference，后面的并发、优化、异步或分布式改造都无法证明语义没有变。再检查压力输入是否尖锐：它应当攻击本章最核心的边界，而不是只把数据量放大。第三，检查状态是否可观察：关键对象的身份、状态、错误和提交结果应能在报告中找到。

第四，检查代码示例是否说明禁止行为。调用者不能做什么，往往比能做什么更能体现工程质量。第五，检查实验是否写出未验证边界；当前设备、权限或输入无法证明的结论，必须标注。第六，检查本章对象是否能被下一章复用，不能复用就说明边界还不稳。

完成这六项后，再看文字是否连贯。每个概念都应从问题推出，每个案例都应从朴素方案走到改进方案，每个实验都应能解释一个判断。若某段话放到其他章节也成立，就把它改成本章的具体对象、具体输入和具体失败。

12.8 工程案例：无锁队列为什么先讲回收再讲速度

无锁数据结构很容易被讲成 CAS 技巧。真实难点通常不是把 head 从旧值改成新值，而是节点什么时候还活着，失败重试有没有副作用，ABA 怎样出现，线性化点如何定义，压力下是否真的比锁快。若这些问题没有讲清，无锁代码越短越危险。

一、SPSC ring 是最合适的第一步 单生产者单消费者环形队列适合作为无锁入门，因为 head 只有消费者写，tail 只有生产者写。单写者让很多竞争消失，协议集中在发布槽位内容和发布槽位

可复用。生产者写元素后 release 更新 tail；消费者 acquire 看到 tail 后读元素；消费者释放槽位后 release 更新 head；生产者 acquire 看到 head 后知道空间可用。

这个结构仍要处理容量、空满判断、尾部环绕、元素构造析构和关闭语义。不要把 SPSC 简化成两个整数。若元素类型有复杂生命周期，要使用原地构造和析构；若队列关闭，生产者和消费者都要能退出；若容量不是 2 的幂，取模成本和边界要处理。SPSC 简单，但不是玩具。

二、MPSC 和 MPMC 引入真正竞争 多生产者队列中，多个线程竞争 tail 或槽位；多消费者队列中，多个线程竞争 head；MPMC 两边都竞争。CAS 成功的线程获得某个槽位或节点的责任，失败的线程重试。这里就会出现 ABA、伪失败、缓存行热点和公平性问题。每个操作都要能指出线性化点：push 何时对消费者可见，pop 何时拥有元素。

不要直接从 SPSC 跳到复杂 MPMC 生产代码。项目可以先实现带锁 MPMC 作为 reference，再实现 SPSC ring，再阅读成熟 MPMC 队列的槽位序号思想。槽位序号让每个 slot 携带阶段信息，生产者和消费者通过序号判断空、满和是否轮到自己。它把共享 head/tail 的一部分压力分散到槽位状态上，但实现和内存序更复杂。

三、Treiber 栈的 ABA 从生命周期里出现 Treiber 栈 push 和 pop 都围绕 head CAS。ABA 问题的典型过程是：线程 T1 读到 head=A 和 next=B；T2 pop A，释放 A，又分配出同一地址 A 并 push 回来；T1 CAS 看到 head 仍等于 A，于是把 head 改成旧 next B，破坏结构。CAS 比较的是地址位模式，不知道对象是不是同一个生命周期。

这个例子说明，ABA 不是抽象名词，而是内存回收和地址复用造成的。解决方法包括 tagged pointer、hazard pointer、epoch reclamation、引用计数节点或避免地址快速复用。每种方法都有成本。Tagged pointer 需要可用位宽或双宽 CAS；hazard pointer 需要每个线程发布正在访问的指针；epoch 需要线程登记和延迟释放。

四、Hazard pointer 的操作流程 Hazard pointer 的核心流程是：读者先读取候选指针，把它写入自己的 hazard slot，再重新读取共享指针确认没有变化；确认后才解引用。删除者不立即释放节点，而是放入 retired list，定期扫描所有 hazard slot，只有没有任何线程保护的节点才能释放。这个流程用额外的全局可见保护表换安全回收。

它的成本包括 hazard slot 写入、扫描开销、retired list 内存峰值和线程退出清理。优点是回收较精确，长期停顿线程不会阻止所有 epoch 推进。项目不一定要完整实现，但要让读者画出“保护、确认、使用、退休、扫描、释放”的状态线。无锁结构没有这条线，就没有资格释放节点。

五、Epoch 回收的操作流程 Epoch reclamation 让线程进入临界区时登记当前 epoch，退出时清除活动标记。删除者把节点放入当前 epoch 的 retired list，等所有线程都不再处于较旧 epoch 后释放。它比 hazard pointer 扫描单个指针少一些，但如果某个线程长时间停留在临界区，回收会停滞，内存峰值上升。

Epoch 适合短读侧临界区和线程数量可控的系统。若线程可能阻塞、被取消或长时间挂起，epoch 要有更复杂的管理。读者要看到，回收协议和调度模型有关：任务运行时若随意挂起持有 epoch 的 worker，会影响全局释放。章节之间要连起来看。

六、无锁不等于 wait-free Lock-free 通常保证系统整体有线程能前进，不保证每个线程有限步完成；wait-free 才强调每个操作有限步完成。很多 CAS 循环在高竞争下某些线程可能长期失败。它们没有互斥锁，但仍然有活性问题、cache line 争用和尾延迟。把“无锁”等同于“无等待”是严重误解。

性能上，低竞争 mutex 可能比复杂无锁结构更快、更省电、更易维护。无锁适合临界区极短、阻塞不可接受、状态边界清楚、生命周期协议成熟的场景。若队列需要复杂关闭、取消、优先级、批处理和阻塞等待，带锁设计可能更可靠。

七、测试无锁结构要包含模型化小输入 压力测试能发现一些 bug，但无锁错误往往依赖特定交错。模型化测试可以用很小容量、少量线程和受控调度枚举更多状态。例如 SPSC ring 容量 2，生产者消费者交错 push/pop/close；Treiber 栈两个节点，多线程 pop/push 并延迟释放；MPMC 队列容量 1 或 2，观察序号变化。小模型比大压力更容易定位线性化错误。

测试还要覆盖内存回收。故意让一个线程读取指针后暂停，另一个线程 pop 并退休节点，检查回收协议是否阻止释放。若测试只比较最终元素集合，可能完全漏掉 use-after-free。ThreadSanitizer、ASan、UBSan 和压力测试可以组合使用，但语义证明仍然必需。

八、实验交付：LockFreeLifecycleReport 本章给项目留下 LockFreeLifecycleReport。

每个无锁结构都要记录操作、线性化点、原子变量含义、成功和失败内存序、节点所有权、回收协议、ABA 处理、进展保证和不适用场景。没有这份报告的无锁代码，不允许替换带锁 reference。

实验路线是：带锁队列 reference，SPSC ring，Treiber 栈加故意延迟释放反例，hazard pointer 或 epoch 的简化演示。验收时既看吞吐，也看内存峰值、CAS 失败次数、尾延迟和回收延迟。无锁章节的目标不是鼓励读者到处无锁，而是让他们知道无锁代码要满足怎样的证明标准。

12.9 案例深化：什么时候保留锁版本

无锁章节必须强调 reference 的价值。带锁版本不是低级版本，而是语义锚点、测试 oracle 和回退方案。很多无锁优化失败时，唯一能帮助定位问题的就是清楚的锁版本。

一、锁版本定义线性化语义 Mutex 队列天然把临界区内操作串行化。它可以清楚定义 push、pop、close 的先后关系。无锁版本要模拟同样用户语义，就必须找出对应线性化点。若锁版本语义都没写清，无锁版本只能靠猜。

保留锁版本还能帮助测试。随机生成一串并发操作，记录可能的结果集合，和锁版本或模型比较。无锁版本如果结果不同，先判断是允许的并发顺序差异，还是违反语义。没有 reference，测试会变成只看是否崩溃。

二、回退路径是工程可靠性 某些平台原子能力不同，某些 workload 下无锁版本不划算，某些 bug 暂时难以修复。保留锁版本作为可配置回退，可以降低风险。高性能库常有多实现：简单锁版本、SPSC 特化、MPMC 高性能版本。选择由编译选项、运行时配置或类型参数决定。

回退不是承认失败，而是工程成熟。它让项目可以先上线正确版本，再逐步验证无锁优化。教材应鼓励这种迭代，而不是制造“必须一次写出最强无锁结构”的压力。

三、无锁性能报告要包含失败率 CAS 循环的失败次数、重试次数和尾延迟必须记录。平均吞吐高但 p99 很差，可能不适合请求路径；低竞争下无锁快，高竞争下 CAS 风暴严重，也要写清。报告还要包含回收延迟和 retired list 峰值。无锁结构的成本不只在成功路径。

四、锁与无锁对照实验 同一队列语义实现三版：mutex 条件变量、SPSC ring、MPMC 无锁或半无锁。输入包括单生产单消费、多生产多消费、低竞争、高竞争、关闭和取消。报告写清哪一版适合哪类场景。这样读者学到的是选择能力，而不是崇拜某个实现。

12.10 补强案例：无锁结构和内存分配器的关系

无锁结构常被讲成节点和 CAS，但节点来自内存分配器，释放后又回到分配器。分配器是否复用地址、是否线程安全、是否有 per-thread cache、是否会在分配时加锁，都会影响无锁结构的正确性和性能。

一、地址复用制造 ABA 条件 ABA 需要地址复用。普通分配器为了性能会快速复用刚释放的内存，这会增加 ABA 触发机会。测试中如果分配器不复用地址，ABA 可能很难出现；换成另一种分配器或压力模式后，bug 才暴露。无锁测试应主动使用容易复用地址的对象池来放大问题。

二、分配器锁会掩盖无锁收益 Treiber 栈 push 每次分配节点，pop 每次释放节点。如果分配器内部有锁，整个结构虽然 head 无锁，性能仍可能受分配器锁影响。高性能无锁结构常使用对象池、批量分配或延迟回收。报告要把分配成本单独列出，否则会误判数据结构本身。

三、延迟回收改变内存峰值 Hazard pointer 和 epoch 都会延迟释放节点。吞吐提升可能伴随 retired 节点堆积。内存峰值是无锁结构的重要指标。若系统内存紧张，延迟回收可能引发 page fault 或回收压力，抵消收益。测试必须记录 retired list 长度和实际释放延迟。

四、实验 同一个无锁栈使用系统分配器、简单对象池、延迟回收池三种方式。压力测试 push pop，记录吞吐、CAS 失败、分配次数、锁等待替代指标、内存峰值和 ABA 检测。这个实验能让读者看到，无锁正确性不能脱离分配器和回收策略。

12.11 补充案例：无锁队列的关闭语义

很多无锁队列示例只实现 push 和 pop，不实现 close。真实系统必须关闭队列。关闭语义会立刻暴露无锁结构的复杂性：并发 push 和 close 谁赢，已经入队的元素是否 drain，等待者如何醒来，关闭状态如何和 head tail 一致。

一、**关闭不是一个附加 bool** 若 close 只是设置 atomic bool, push 可能已经通过检查并正在写槽位。到底这个 push 是否成功? 需要线性化点定义。可以规定 push 的线性化点在槽位发布; close 的线性化点在状态 CAS; 二者按线性化先后决定。没有定义, 调用者无法理解边界。

二、**Drain 和 cancel 同样适用** 无锁队列也要区分 drain 和 cancel。Drain 下, close 后不接收新元素, 但已有元素可 pop; cancel 下, 未消费元素可能被丢弃或标记取消。无锁实现中取消更难, 因为消费者可能正持有槽位。简单项目应先实现 drain。

三、**等待者唤醒** 无锁结构本身不等于无等待。消费者等待空队列时, 需要 atomic wait、semaphore 或条件变量辅助。close 必须唤醒等待者。否则队列已经关闭, 消费者仍可能睡在旧值上。等待协议要和数据结构协议一起设计。

四、**实验** 对 SPSC ring 增加 drain close。并发执行 producer push、consumer pop、close。检查 close 前成功发布的元素都被消费, close 后 push 失败, consumer 最终退出。这个实验让读者看到无锁结构也需要完整系统语义。

12.12 补充专题：无锁结构中的可观测性

无锁结构如果没有指标, 性能和活性问题很难解释。它没有 mutex 等待时间, 但有 CAS 失败、自旋次数、回收延迟、队列深度和尾延迟。可观测性要从设计时加入。

一、**CAS 失败计数** CAS 失败不是错误, 但失败次数高说明竞争强。每个操作可以记录本地失败次数, 周期性汇总。不要让所有线程写同一个失败计数器, 避免新热点。失败分布比总数更有价值, 某些线程长期失败说明公平性问题。

二、**自旋和退避** 无锁算法常自旋重试。自旋次数过高会浪费 CPU, 退避可以减少争用但增加延迟。指标要记录自旋次数和退避次数。没有这些数据, 很难判断高 CPU 使用率是在做有效工作还是空转。

三、**回收指标** Hazard pointer 和 epoch 要记录 retired 节点数量、扫描次数、释放数量和最长滞留时间。内存峰值上升时, 这些指标能说明是否有线程长期阻止回收。

四、**实验** 在无锁栈或队列中加入分片指标。比较低竞争和高竞争输入, 观察 CAS 失败、吞吐和尾延迟。指标本身要低开销, 不能把无锁结构重新变成共享写热点。

12.13 从少一把锁到完整生命周期协议

无锁数据结构最容易被神化, 好像只要不用 mutex 就更高级。实际难点恰好相反: 没有锁以后, 互斥提供的对象生命周期、临界区边界和等待关系都要由协议自己承担。一个节点从分配、发布、被读取、被摘除到最终释放, 中间可能同时被多个线程观察。若只盯着 CAS 成功与否, 看不到生命周期, 代码很可能在压力测试中出现 ABA、use-after-free 或不可恢复的忙等。

一、**进展保证要具体到操作** lock-free、wait-free、obstruction-free 这些词不能只背定义。要问某个操作在什么条件下能完成。SPSC ring 中, 单生产者和单消费者让 head、tail 分别只有一个写者, 进展分析相对清楚; MPMC 队列中, 多个生产者和消费者争同一位置, CAS 失败可能反复发生; 若内存分配器内部有锁, 所谓无锁队列也可能在分配路径阻塞。报告要写清 push、pop、close、reclaim 各自的进展保证, 而不是给整个类贴一个标签。

二、**线性化点定义用户看到的瞬间** 无锁结构必须说明操作在哪一刻对外生效。队列 push 的线性化点可能是 slot 状态从 Empty 变成 Full, 栈 push 的线性化点可能是 head CAS 成功, 引用计数释放的线性化点可能是计数从一降到零。线性化点不是论文装饰, 它决定测试 oracle。若 pop 返回失败, 是因为当时队列真的空, 还是因为竞争中暂时看不到元素? 若 close 与 push 并发, push 是否可能成功? 这些语义都要围绕线性化点写清楚。

三、**ABA 来自地址身份不够用** CAS 比较的是某个值是否仍等于旧值。如果一个指针从 A 变成 B 又变回 A, CAS 可能误以为没有变化。ABA 的根本问题是地址本身不能表达对象历史。解决路径包括带版本的指针、hazard pointer、epoch 回收、引用计数或避免复用节点。每种方案都有成本: 版本需要位宽和溢出策略, hazard pointer 增加读侧发布, epoch 延迟释放, 引用计数增加原子写。教材要把 ABA 放在生命周期里讲, 而不是作为一个独立谜题。

四、**内存回收是无锁结构的核心设计** 节点被逻辑删除后, 不能马上释放, 因为其他线程可能刚读到指针但还没完成访问。锁版本可以靠临界区保证没人持有, 非阻塞版本必须选择回收协议。

Hazard pointer 让读者公开“我正在看这个节点”，释放者扫描保护表；epoch 让线程声明自己处于哪个时代，释放者等所有旧时代读者离开；RCU 把读侧临界区和宽限期变成系统合同。选择回收协议时，要看读多写少、线程数、释放延迟、内存峰值和实现复杂度。

五、无锁不等于无等待成本 CAS 循环失败会消耗 CPU，忙等会增加功耗，缓存行争用会让多个核心互相干扰。低争用下无锁结构可能很快，高争用下锁加退避反而更稳定。实验要覆盖单线程、低争用、高争用、关闭、异常和内存压力。评价指标不能只有吞吐，还要有尾延迟、失败重试次数、CPU 使用率和内存滞留。无锁结构的工程目标是明确进展和降低阻塞，不是盲目替换所有锁。

12.14 案例：SPSC Ring 的完整审查路线

SPSC ring 是学习无锁结构的好入口，因为它有一个关键简化：生产者独写 tail，消费者独写 head。这个简化让很多复杂 CAS 消失，但并不意味着可以随便写。ring 的正确性仍然依赖槽位生命周期、发布顺序、容量判断、关闭语义和对对象析构。把这个案例讲透，比直接跳到 MPMC 队列更有教学价值。

一、先定义槽位状态 每个槽位在逻辑上经历 Empty、Constructing、Full、Consuming、Empty。实际实现未必保存完整枚举，但审查时要用这张图。生产者只有在槽位 Empty 时构造元素，构造完成后发布 tail；消费者看到 tail 后才能读取元素，读取并析构后发布 head。若发布 tail 早于元素构造完成，消费者可能读到未构造对象；若消费者更新 head 早于析构完成，生产者可能覆盖仍在使用的对象。

二、容量判断要留一个空位或保存计数 环形队列常用 head 等于 tail 表示空。若还想用同一条件表示满，就会歧义。常见方案是容量为 N 时最多存 N 减一个元素，或者额外保存计数。SPSC 下用索引距离判断最简单，但要注意无符号回绕和容量必须是固定值。索引类型应使用明确位宽，例如 `std::uint64_t`，并让容量远小于回绕周期。这里不要使用含糊的数据类型表达长期运行的队列位置。

三、发布顺序围绕 head 和 tail 生产者写元素后，用 release store 更新 tail；消费者用 acquire load 读取 tail，确认有元素后读取槽位。消费者析构元素后，用 release store 更新 head；生产者用 acquire load 读取 head，确认有空间后写槽位。若只有单向数据发布，某些读取可以放宽，但教材应先给出保守清晰版本，再说明哪些地方能优化。新手最需要的是正确协议，而不是最弱内存序挑战。

四、关闭语义不能靠析构猜测 ring 如果被析构时还有生产者或消费者访问，任何内存序都救不了。关闭应是外部生命周期协议：先通知生产者停止提交，再让消费者 drain，最后确认没有线程访问对象。若 ring 自身提供 close，应定义 close 后 push 返回什么、pop 是否继续取剩余元素、等待者如何醒来。无锁结构越底层，越不能把生命周期交给调用者口头保证。

五、测试要覆盖边界容量和回绕 SPSC 测试不能只 push 三个元素再 pop 三个。要测试容量为二的极小 ring、反复回绕、生产者快消费者慢、消费者快生产者慢、关闭时有剩余元素、对象构造抛异常、元素带析构副作用。每个测试都要和串行模型比较：队列顺序是否保持，元素是否丢失，析构次数是否正确。这样读者能看到无锁结构的审查从不变量开始，而不是从神秘的 CAS 循环开始。

12.15 Hazard Pointer 和 Epoch 的选择边界

无锁结构讲到内存回收时，不能只列两个名词。Hazard pointer 和 epoch 的差异来自读者和释放者的成本分配。Hazard pointer 要求读者在访问节点前把指针发布到保护槽位，释放者扫描所有槽位，确认没人保护后释放。Epoch 要求线程声明自己处于某个时代，释放者把删除节点放进退休列表，等所有旧时代读者离开后批量释放。两者都能避免过早释放，但适用场景不同。

一、Hazard pointer 适合精确保护 当读者数量不大、每次访问节点数量有限、释放延迟需要较可控时，hazard pointer 很合适。它的优点是保护对象明确，释放者能精确判断某个节点是否仍被观察。缺点是读侧每次保护都要写共享可见槽位，可能增加缓存流量；释放者扫描槽位也有成本。若数据结构每次遍历会同时保护多个节点，还要管理多个 hazard slot，接口会变复杂。

二、Epoch 适合读侧低成本 Epoch 的读侧通常只需要进入和退出读临界区，成本比逐节点保护低。它适合读多写少、读路径短、允许释放延迟的结构。缺点是只要某个线程长时间停留在旧 epoch，所有相关退休节点都不能释放，内存峰值可能上升。移动端或内存紧张环境中，这个缺点不能忽略。报告要记录退休列表长度和最大释放延迟。

三、线程生命周期影响回收协议 线程退出、线程池复用、任务取消都会影响回收。Hazard slot 必须在线程退出时清空，epoch 参与者必须在阻塞或退出前离开读临界区。若一个 worker 在 epoch 内等待 I/O，释放会被长期阻塞。无锁结构不能只看数据结构代码，还要看运行时如何管理线程和任务。回收协议和线程生命周期是绑定的。

四、调试要暴露退休队列 内存回收 bug 很难复现，所以调试材料要更丰富。记录每个线程的保护槽位、当前 epoch、退休节点数量、释放批次和最长滞留时间。压力测试中故意让一个线程暂停在读临界区，观察内存是否按预期暂时增长并在退出后释放。这样读者能看到“延迟释放”是设计行为，而不是泄漏或随机卡顿。

五、选择前先问三个问题 第一，读路径是否允许写共享保护槽位；第二，释放延迟是否可接受；第三，线程是否可能长时间阻塞在读临界区。若读路径极热且短，epoch 可能更合适；若释放延迟必须受控，hazard pointer 可能更清楚；若线程模型复杂，先用锁或引用计数实现正确语义，再迁移到更复杂回收协议。无锁结构的成熟做法是先选可审查方案，再优化。

12.16 贯穿案例：一个 Treiber 栈为什么把内存回收拉进主线

无锁数据结构如果从 CAS 指令开始讲，很快会变成技巧展示。更好的入口是一段失败时间线。线程 T1 读取栈顶指针，看到节点 A，准备 CAS 把 head 从 A 改成 A 的 next。此时 T1 暂停。线程 T2 pop 出 A，释放 A；随后分配器把同一地址重新用于新节点 C，并且 C 又被 push 到栈顶。T1 恢复，发现 head 地址仍然等于自己当初读到的 A 地址，于是 CAS 成功。它以为自己操作的是旧 A，实际地址已经代表新 C。这个事故同时暴露 ABA 和过早释放。

从这里讲无锁结构，概念就不会散。CAS 只比较地址，不理解对象身份；分配器可能复用地址；其他线程可能仍持有旧指针；节点释放需要等待所有可能观察者离开；版本标签、hazard pointer、epoch 都是在补这个信息缺口。无锁并不是“没有锁所以简单”，而是把互斥锁隐含保护的生命周期事实显式写出来。

一、先指出锁版栈保护了什么 锁版栈里，线程持锁期间读 head、改 next、释放节点，其他线程不能同时观察中间状态。锁不仅保护结构修改，也间接保护节点生命周期。改成无锁以后，结构修改靠 CAS 线性化，但生命周期保护消失了。这个对比能让读者明白为什么 Treiber 栈代码短却不能单独上线：push 和 pop 的线性化点只是结构正确性，释放节点还需要另一套协议。

二、版本标签解决“地址又回来了”的一部分 把 head 从裸指针扩展成指针加版本号，可以让 A 地址被复用时版本不同，T1 的 CAS 失败。这缓解 ABA，但不自动解决 use after free。T1 在 CAS 前可能已经读了 A 的 next，如果 A 已释放并复用，这个读取本身就不安全。版本标签保护比较结果，内存回收保护读取过程，两者不能互相替代。教材必须把这条边界讲清，否则读者会以为给指针加计数就万事大吉。

三、Hazard pointer 从“我要读这个节点”推出 T1 读取 head 后，先把自己正在观察的指针发布到 hazard slot，再重新检查 head 是否仍然等于该指针。若一致，说明释放者能看到 T1 正保护这个节点，暂时不能释放它。释放者 pop 节点后不是立即 delete，而是放进 retire list，扫描所有 hazard slot，确认无人保护后才释放。这个协议的成本也清楚：读侧要写共享保护槽位，释放侧要扫描，线程退出要清空槽位。

四、Epoch 从“读临界区很短”推出 若读路径很多、每次保护节点都写 hazard slot 成本太高，可以让线程进入一个 epoch 读临界区。删除的节点进入退休列表，等所有线程都离开旧 epoch 后批量释放。它的优点是读侧成本低，缺点是某个线程长时间停在 epoch 内会阻止释放，内存峰值上升。这个选择来自运行时事实：读临界区是否短，线程是否可能阻塞，释放延迟是否能接受。不要把 epoch 当成更高级的 hazard pointer，它只是不同成本分配。

五、SPSC 和 MPMC 为什么不能混讲 SPSC ring buffer 没有多个生产者争同一个 tail，也没有多个消费者争同一个 head，所以它的难点主要是发布顺序、槽位生命周期和关闭。MPMC queue 要处理多方竞争和环绕，通常需要 slot sequence。Treiber 栈要处理节点回收。三者都叫无锁，但问题完全不同。教材若把它们混成“CAS 数据结构”，读者就无法判断什么时候该用简单结构，什么时候必须付出回收成本。

六、无锁报告要把内存峰值当性能 如果 epoch 版本吞吐很高，但一个 worker 偶尔在读临界区里阻塞 I/O，退休列表积压到几 GB，这不是成功。无锁结构报告要记录吞吐、CAS 失败、spin、

退避、退休节点数量、最大释放延迟、hazard 扫描成本和内存峰值。锁版可能平均慢一些，却有更可控的内存生命周期。工程选择必须把这些都写进账本。

12.17 案例深化：SPSC、MPMC 和栈的不同难点

无锁数据结构不应该一上来就讲最难的 MPMC。SPSC ring buffer、MPMC bounded queue、Treiber stack 解决的问题不同，难点也不同。把它们放在一条线上看，读者才能明白为什么有些结构很适合工程使用，有些结构必须带回收方案才能出现在线上。

一、SPSC 的简洁来自角色限制 单生产者单消费者队列中，head 只由消费者写，tail 只由生产者写。这个角色限制让它不需要 CAS，只需要发布顺序和环形缓冲。它不是“低级版本”，而是通过限制使用场景换来简单和高性能。接口必须阻止多个生产者误用，否则结构假设失效。教学中要把这种限制写在类型和测试里。

二、MPMC 需要 slot sequence 多生产者多消费者同时竞争同一个环形缓冲，单纯 head tail 原子计数不够。每个 slot 需要 sequence 表示它属于哪一轮，生产者和消费者通过 sequence 判断 slot 是否可写或可读。这个设计解决环绕后旧数据和新数据混淆的问题。读者如果只看 CAS，会错过 sequence 这个真正的状态变量。

三、Treiber 栈突出回收问题 Treiber 栈的 push pop 很短，但节点回收困难。Pop 返回节点后，其他线程可能仍持有旧指针。Hazard pointer 让线程声明自己正在观察哪些节点，epoch reclamation 让系统等所有旧 epoch 线程离开后再释放。两种方案都增加运行时状态和内存峰值。无锁不是少代码，而是把阻塞换成更复杂的生命周期协议。

四、线性化点要能画出来 每个操作要指出哪一步对外生效。SPSC push 的线性化点可能是 tail 发布，MPMC enqueue 是 slot sequence 发布，Treiber push 是 head CAS 成功。若测试出现两个消费者拿到同一元素，线性化点能帮助定位是哪条发布规则坏了。没有线性化点，代码再短也不可审查。

五、性能报告要包含失败和回收 无锁结构的性能报告不能只写吞吐。要写 CAS 失败次数、spin 次数、退避策略、retire list 长度、hazard 扫描成本、epoch 卡住时间和内存峰值。若一个慢线程长时间不退出 epoch，所有 retired 节点都不能释放，吞吐再高也可能内存爆。工程使用无锁结构前，必须把这些副成本纳入账本。

12.18 案例复盘：无锁结构的生命周期账本

Treiber 栈的 CAS 很短，但线程读到旧 head 后暂停，其他线程 pop 并释放节点，再把同一地址复用，旧线程恢复后 CAS 可能错误成功。ABA 和 use after free 说明无锁难点不在“没有锁”，而在节点生命周期和线性化点。

不同结构不同边界 SPSC ring buffer 靠单写角色限制避免复杂 CAS；MPMC queue 需要 slot sequence 区分轮次；Treiber stack 需要 hazard pointer 或 epoch 回收。每个结构都要写线性化点和释放条件。接口也要防止把 SPSC 当 MPMC 用。

实验判据 报告写 CAS 成功失败、spin 次数、retire list 长度、epoch 滞留、内存峰值和尾延迟。慢线程长期不退出 epoch 时，内存可能增长。无锁结构不是默认更快，而是用更复杂的生命周期协议换取某些阻塞边界。

12.19 本章习题

习题与作业

习题一：为本章 SPSC ring buffer 写出不变量、满空判断、push 线性化点和 pop 线性化点。说明为什么它不能被多个生产者同时调用。

习题二：把 SPSC ring buffer 扩展到支持容量检查和关闭语义。只要求设计状态机，不要求完整代码。说明关闭后 producer 和 consumer 分别应看到什么返回值。

习题三：写一份 ABA 场景时间线，至少包含两个线程、两个节点、一次 pop、一次释放和一次地址复用。然后分别说明版本标签、hazard pointer、epoch 如何缓解。

习题四：选择一个成熟无锁队列实现或论文，阅读其不变量和内存回收策略。报告必须

说明它适用 SPSC、MPSC 还是 MPMC，不能只说“这是无锁队列”。

第零部分

并行算法与运行时

归约、扫描与分区

先看一个并行 filter 的错误实现：多个线程扫描输入，把满足条件的元素直接 `push_back` 到同一个输出 vector。加锁后结果正确但很慢；换成原子递增下标后快一些，却把输出顺序改乱；为了保持顺序又回到复杂同步。真正的问题不是缺少某个容器技巧，而是并行程序还没有为每个保留元素分配唯一输出位置。于是 scan 出现了：先统计每块保留数量，再做前缀和得到每块写入区间，最后无冲突写出。

本章从这种“输出位置未知”的问题进入归约、扫描和分区。并行算法的核心是处理依赖：归约把多份 partial 合成一个结果，扫描把前缀依赖拆成块内和块间两层，分区把共享 push 改成确定写入区间。数据库聚合、日志统计、图计算、排序、渲染和科学计算都会反复遇到这些形状。

13.1 从串行语义到并行归约

归约把多个输入合并成一个输出。并行归约的基本策略是局部归约加全局合并。它的性能取决于输入划分、局部性、合并成本和结果类型。整数加法简单，浮点加法要考虑精度和顺序，字符串拼接则可能有大量分配。

归约优化通常先避免共享写。每个线程写自己的 partial，最后串行或树形合并。树形合并可以减少合并深度，也更适合分布式系统。

归约的第一步是写出代数性质。操作是否有单位元，是否满足结合律，是否满足交换律，是否允许改变顺序。整数最小值、最大值、按位或通常容易并行；浮点求和需要说明误差模型；哈希合并需要说明冲突处理；对象聚合需要说明生命周期和分配。没有这些语义前提，谈并行归约是不完整的。

归约的第二步是写出内存布局。partial 结果应避免 false sharing，通常每个 worker 一个对齐槽位。若 partial 很大，可以按 NUMA 节点或缓存块组织，避免最后合并阶段变成单线程长尾。归约不是只有“线程本地加一下”这一种形式，它可以是线性合并、二叉树合并、分层合并或流水线合并。

13.2 Scan 和输出位置分配

扫描也叫 prefix sum，输出每个位置之前元素的累计结果。它比归约更难，因为每个输出都依赖前缀。并行扫描通常分两阶段：先对每个块做局部扫描，得到块总和；再扫描块总和；最后把块前缀加回块内结果。

扫描是很多算法的 building block，例如 stream compaction、并行分配输出位置、基数排序和图前沿构造。理解扫描，可以帮助读者把“有顺序依赖”的问题拆成局部和全局两层。

扫描比归约更能体现并行算法设计。表面上，每个输出都依赖前面所有输入，似乎完全串行；但把输入分块后，每个块内部可以先做局部扫描，块总和再做一次小规模扫描，最后把块前缀加回去。这个套路说明，很多“看起来串行”的依赖，可以拆成局部依赖和块间摘要。分布式系统里的很多计算也是这个形状：节点内先处理，节点间交换摘要，再回填结果。

13.3 Partition、Filter 和 Shuffle 前置

分区把元素按谓词或 key 分到不同区域。并行分区要先统计每个线程每个桶的数量，再计算全局偏移，最后写入输出。这与扫描密切相关。直接让多个线程 push 到共享 vector，会引入锁或原子热点。

分区的关键是先分配写入位置，再并行写入。以 `copy_if` 为例，朴素并行写法可能让所有线程对输出 vector 做 push，需要锁或原子。更好的写法是每个线程先统计自己保留多少元素，对这些计数做扫描得到每个线程的输出起点，然后每个线程写入自己的连续区间。这样写入没有冲突，输出也保持可解释顺序。

13.4 负载、确定性和容错

静态划分适合每个元素成本相近的场景。若元素成本差异大，静态划分会让某些线程早早结束，其他线程仍在工作。动态调度、任务队列和工作窃取可以改善负载均衡，但会增加调度开销。

负载均衡不是越动态越好。静态划分没有调度开销，局部性好，适合规则数组和均匀工作。动态队列能处理不均匀工作，但会增加同步和队列争用。工作窃取减少全局争用，但实现更复杂。设计时要估算任务成本方差：如果每个块成本接近，静态块划分更好；如果成本长尾明显，动态调度才值得。

并行算法的正确性模板 本册要求每个并行算法都写出四个正确性问题。第一，串行语义是什么，reference 怎样实现。第二，并行切分后，每个 worker 负责哪些输入和输出。第三，局部结果怎样合并，合并顺序是否影响结果。第四，是否存在共享写、重复写、漏写或越界写。性能优化只能在这些问题回答之后进行。

这套模板能防止“并行版本跑得快但其实错”。很多并行 bug 不会在小输入暴露，因为线程交错少、边界简单、缓存状态稳定。测试必须覆盖空输入、极小输入、刚好一个块、多个块、不能整除块大小、重复 key、倾斜 key 和高线程数。

从串行 reference 开始 并行算法一定要从串行 reference 开始。reference 不一定最快，但它必须简单、可读、边界清楚。所有优化版本先和 reference 比结果，再谈速度。没有 reference 的并行优化很危险，因为你很难判断一个高吞吐结果是否只是漏写、重复写或改变了语义。

以 `count_if` 为例，串行 reference 很短。

Listing 13.1 串行 `count-if` reference

```
1 std::int64_t count_at_least(std::span<const std::int32_t> values,
2   std::int32_t threshold) {
3   std::int64_t count = 0;
4   for (const std::int32_t value : values) {
5     if (value >= threshold) {
6       ++count;
7     }
8   }
9   return count;
10 }
```

它的语义是：遍历每个输入元素，满足谓词则计数加一。并行版本不能改变谓词，不能漏掉元素，不能重复计数，不能因为线程数不同得到不同整数结果。这个 reference 也能用于测试空输入、负数、重复值、全部满足、全部不满足和随机输入。

静态切分的边界 静态切分把输入区间接 worker 数分成若干连续块。连续块有两个优点：每个 worker 顺序扫描自己的数据，缓存和预取友好；输出 partial 数组按 worker 索引存放，容易合并。切分时要处理不能整除的情况，不能简单地让每个 worker 处理 `n/worker_count` 个元素后丢掉尾部。

Listing 13.2 连续区间切分公式

```
1 std::size_t block_begin(std::size_t size,
2   std::int32_t worker,
3   std::int32_t worker_count) {
4   return size * static_cast<std::size_t>(worker) /
5     static_cast<std::size_t>(worker_count);
6 }
7
8 std::size_t block_end(std::size_t size,
9   std::int32_t worker,
10  std::int32_t worker_count) {
```



```

11 return size * static_cast<std::size_t>(worker + 1) /
12 static_cast<std::size_t>(worker_count);
13 }

```

这个公式保证所有块覆盖 `[0, size)`，相邻块首尾相接，不重叠也不漏元素。若 `worker_count > size`，某些块可能为空，因此实际 worker 数可以取 `min(worker_count, size)`，或者允许空块但测试要覆盖。工程里还要检查 `worker_count` 必须为正。

并行 count-if: 从错误版本到正确版本 错误版本让所有线程更新一个全局原子计数器。它简单但共享写很重。正确的第一步是每个 worker 写自己的 partial。

Listing 13.3 并行 count-if 的 partial 版本

```

1 struct alignas(64) CountIfPartial {
2     std::int64_t count = 0;
3 };
4
5 void count_if_worker(std::span<const std::int32_t> values,
6     std::int32_t threshold,
7     CountIfPartial& partial) {
8     std::int64_t local = 0;
9     for (const std::int32_t value : values) {
10         if (value >= threshold) {
11             ++local;
12         }
13     }
14     partial.count = local;
15 }

```

这个 worker 函数只写自己的 partial，不触碰共享计数器。主线程或合并任务最后扫描 partial 得到总数。这个结构的正确性来自区间划分：每个输入元素属于且只属于一个 worker；每个 worker 的 local 是该区间内满足谓词的数量；合并是整数加法。性能收益来自减少共享写和保持连续扫描。

但 partial 版本也有边界。若输入很小，线程创建和合并成本可能超过收益；若谓词很复杂，每个元素成本高，动态调度可能比静态切分更适合；若阈值和输入分布导致分支不可预测，分支优化可能比并行更重要。并行不是第一反应，正确的流程是先写 reference，再测瓶颈，再选择并行结构。

归约树和关键路径 线性合并 partial 很简单：一个线程从头到尾加所有 partial。worker 数很少时足够；worker 数很多或 partial 很大时，线性合并会成为串行尾巴。树形合并把 partial 两两合并，关键路径从 $O(p)$ 变成 $O(\log p)$ ，其中 p 是 partial 数。

树形合并也有代价。它需要额外任务或多轮同步；若每个 partial 只是一个整数，线性合并可能已经足够快；若 partial 是大型直方图，树形合并更有价值。不要把树形合并当成默认更好，要看合并成本占比。

```

linear merge:
    (((p0 + p1) + p2) + p3) + p4 ...

tree merge:
    round 1: p0+p1, p2+p3, p4+p5, p6+p7
    round 2: s0+s1, s2+s3
    round 3: t0+t1

```

树形合并还为分布式 reduce 做准备。单机里 round 是线程池任务，分布式里 round 是不同节点之间的聚合阶段。理解关键路径后，读者才能解释为什么某些系统使用分层聚合，而不是把所有 partial 发到一个中心节点。

浮点归约的语义选择 浮点求和不能简单说“加法满足结合律”。串行从左到右、四累加器、树形合并、按 NUMA 节点合并都会产生不同舍入。工程系统必须选择语义：是否要求逐位一致，是否允许误差范围，是否使用确定性合并树，是否使用补偿求和，是否开启 fast-math。

逐位一致通常成本更高，因为合并顺序要固定，某些向量化和并行优化会受限。误差容忍则需要测试误差界，而不是只比较完全相等。科学计算、金融、渲染、机器学习统计的容忍度不同，不能用一种答案覆盖。

扫描的三阶段实现 扫描的三阶段可以写得很清楚。第一阶段，每个 worker 对自己的块做局部 scan，并记录块总和。第二阶段，对块总和做 scan，得到每个块之前的总偏移。第三阶段，每个 worker 将该偏移加到自己块内所有输出上。这个算法把一个长依赖链拆成多个短依赖链和一个小规模依赖链。

Listing 13.4 局部 scan 和回填偏移

```
1 std::int64_t local_inclusive_scan(std::span<const std::int32_t> input,
2   std::span<std::int64_t> output) {
3   const std::size_t count = std::min(input.size(), output.size());
4   std::int64_t prefix = 0;
5   for (std::size_t i = 0; i < count; ++i) {
6     prefix += input[i];
7     output[i] = prefix;
8   }
9   return prefix;
10 }
11
12 void add_block_offset(std::span<std::int64_t> output,
13   std::int64_t offset) {
14   for (std::int64_t& value : output) {
15     value += offset;
16   }
17 }
```

如果使用 inclusive scan，输出位置 *i* 包含 input[0] 到 input[i]。如果使用 exclusive scan，输出位置 *i* 包含 input[0] 到 input[i-1]。两者都常用，但语义不同。教材和 API 必须在名称中写清 inclusive 或 exclusive，不能只叫 scan 后让调用者猜。

Scan 的工作量和空间 并行 scan 通常比串行 scan 做更多工作。它需要写输出、写 block sums、扫描 block sums、再回填。若输入很小，串行 scan 更快；若输入很大且多核可用，并行 scan 才有机会获益。scan 还需要额外存储 block sums，通常大小是 worker 数或 block 数，远小于输入，但仍要管理生命周期。

scan 也会受内存带宽限制。对每个元素，局部 scan 读输入写输出，回填又读写输出一次。若元素类型大或输出巨大，内存流量很可观。优化方向可能是把 scan 和后续写输出融合，例如 stream compaction 中，统计和写出阶段可以根据选择率和缓存做调整。算法分阶段清晰，但实现可以在不破坏语义的前提下融合。

Partition 的两级偏移 按 key 分区通常比 copy_if 更复杂，因为有多个桶。每个 worker 先统计自己块中每个桶的数量，形成一个二维计数矩阵：worker 乘 bucket。然后对每个 bucket 的 worker counts 做 scan，得到每个 worker 在该 bucket 的写入偏移。最后每个 worker 再次遍历自己的输入，把元素写到对应 bucket 的对应区间。

这个过程避免了多个 worker 对同一 bucket 的共享 push。每个 worker 写入的输出区间是唯一的，不需要锁。代价是计数矩阵和两遍输入扫描。桶数很多时，计数矩阵可能很大；输入很大时，两遍扫描消耗带宽；key 分布倾斜时，某些 bucket 输出很大，后续处理仍会倾斜。分区解决写入冲突，不自动解决下游负载均衡。

```
parallel partition shape:
pass 1: worker local counts per bucket
scan: compute global bucket offsets
pass 2: worker writes to unique output ranges
```

这套结构是分布式 shuffle 的单机版本。pass 1 是 map 端计数，scan 是计算每个分区的输出位置，pass 2 是写分区文件或缓冲。分布式系统把 output range 换成远端节点或磁盘分区，核心思想不变。

稳定分区和非稳定分区 分区还要说明是否稳定。稳定分区保持相同桶内元素的原始相对顺序；非稳定分区不保证。稳定分区更容易推理和测试，但可能限制优化；非稳定分区可以使用更自由的交换和块内重排，适合只关心集合不关心顺序的场景。

并行稳定分区通常依赖每个 worker 处理连续块，并按 worker 顺序给每个 bucket 分配输出区间。这样同一 worker 内保持顺序，worker 间按原始区间顺序排列。若使用动态调度，稳定性就更难保证，因为任务完成顺序和输入顺序不同。稳定性不是免费属性，必须在接口里声明。

负载均衡和任务切分 归约和 scan 对均匀数组很友好，静态切分通常足够。分区和过滤则可能不均匀：某些块满足谓词的元素很多，写输出更重；某些 key 特别热，后续 bucket 处理更重。图算法和字符串解析更不均匀，元素成本可能差几个数量级。任务切分要考虑工作量方差。

一种策略是过分割：把输入切成比 worker 多得多的块，由线程池动态调度。这样负载更均衡，但任务调度成本和 partial 数增加。另一种策略是先采样估计工作量，再按估计切分。数据库和分布式系统常用采样来处理倾斜。没有一种策略永远最好，关键是通过指标观察 worker 空闲时间和长尾任务。

内存分配和输出所有权 并行算法最怕在热路径里动态扩容共享容器。多个线程对同一个 `std::vector push_back` 必须加锁，扩容还会移动数据。正确做法通常是先计算输出大小或每个块输出大小，再一次性分配输出，最后每个 worker 写自己的区间。scan 的重要性就在这里：它把“我要写到哪里”变成一个无冲突的确定偏移。

如果输出大小无法提前准确知道，可以使用 per-worker buffer，最后合并；也可以使用分块 allocator；或者使用有界队列把输出流式送给下游。每种方案都有代价：per-worker buffer 需要额外内存和合并；allocator 需要生命周期设计；队列引入背压和调度。输出所有权是并行算法设计的一部分，不是实现细节。

测试矩阵 归约、扫描和分区的测试要覆盖结构性边界。归约测试包括空输入、单位元、一个元素、多块、负数、溢出边界和浮点误差。扫描测试包括 inclusive/exclusive、块边界、最后不满块和输出长度不足。分区测试包括零个元素、全部同桶、每个桶均匀、极端倾斜、稳定性验证和输出容量。并发测试还要在不同线程数下重复运行，捕捉隐藏竞态。

性能测试则要分离三个成本：读取输入、写输出、合并 partial。对于 count-if，读取输入和分支是主成本；对于 stream compaction，写输出和 scan 偏移也重要；对于 histogram，partial 内存占用和合并成本重要。报告不应只给总时间，而要解释哪一阶段主导。

从串行 reference 开始 并行算法的第一步不是开线程，而是写清串行 reference。Reference 的任务是定义语义：空输入返回什么，溢出如何处理，浮点误差如何比较，输出顺序是否稳定，异常输入是否允许。没有 reference，并行版本即使跑得快，也不知道是否正确。

以 count-if 为例，串行 reference 可以非常朴素。它不追求速度，不使用 SIMD，不提前分块，只表达每个元素被检查一次且满足谓词时计数加一。并行版本的正确性证明要回到它：输入被分成互不重叠的块，每个块使用同一谓词统计，partial 求和等于全局计数。这个证明比代码更重要，因为它让后续优化有基准。

Listing 13.5 串行 reference 定义 count-if 语义

```
1 std::int64_t count_positive_reference(std::span<const std::int32_t> values) {
2     std::int64_t count = 0;
3     for (const std::int32_t value : values) {
4         if (value > 0) {
5             ++count;
6         }
7     }
8     return count;
9 }
```



```

6 }
7 }
8 return count;
9 }

```

Reference 还可以故意保持简单类型和清晰边界。若输入长度可能超过 `std::int32_t`，计数使用 `std::int64_t`。若数据来自文件，reference 不负责 I/O，只处理已经解析好的数组。把语义层和 I/O 层分开，后续测试更稳定。

范围切分的数学边界 并行数组算法通常先把区间 $[0, n)$ 切成若干块。切分必须保证覆盖完整、互不重叠、不会越界、空输入正确、线程数大于元素数时仍正确。一个常见写法是根据 block index 计算 begin 和 end，最后一块自然处理余数。不要用容易溢出的 `i*block_size` 盲目计算，也不要假设 `n` 一定能整除 worker 数。

Listing 13.6 按块编号计算半开区间

```

1 struct Range {
2     std::size_t begin = 0;
3     std::size_t end = 0;
4 };
5
6 Range block_range(std::size_t total,
7     std::size_t block_index,
8     std::size_t block_count) {
9     const std::size_t begin = (total * block_index) / block_count;
10    const std::size_t end = (total * (block_index + 1)) / block_count;
11    return Range{begin, end};
12 }

```

这个写法让每个块大小相差不超过一，且不需要单独处理余数。但如果 `total*block_index` 可能超过 `std::size_t`，超大输入下仍要使用更稳的计算方式。教材阶段至少要让读者知道半开区间和覆盖证明：第 `k` 块的 end 等于第 `k+1` 块的 begin，第一块 begin 为零，最后一块 end 为 total。

单位元和结合性 归约要求一个二元操作和单位元。整数求和的单位元是零，乘法单位元是一，最小值的单位元可以是类型最大值，最大值的单位元可以是类型最小值。若没有正确单位元，空块就难处理。并行切分后，有些 worker 可能拿到空范围，因此算法不能只在“每块至少一个元素”的假设下成立。

结合性决定能否任意重排合并。整数加法在不溢出的数学整数上结合，但固定宽度整数溢出有语言和业务边界；浮点加法因为舍入不严格结合；字符串拼接结合但内存成本高；哈希合并可能依赖顺序。并行归约的 API 应说明操作是否必须结合，是否要求交换，是否保证确定性顺序。

确定性归约 并行归约常遇到一个现实需求：每次运行结果是否完全一样。整数和某些集合运算容易保持确定；浮点、哈希表迭代、动态调度和非稳定合并会造成结果波动。对机器学习训练日志、科学计算和金融报表，波动可能不能接受；对监控指标和近似统计，误差范围可能可以接受。

确定性归约可以使用固定切分和固定合并树。无论任务完成顺序如何，最终合并按 block id 顺序进行。这样牺牲一部分调度自由，换来可复现。另一个策略是使用补偿求和或更高精度 partial，降低误差但不一定逐位一致。教材应要求读者在报告中写明选择：追求吞吐、误差范围还是逐位复现。

```

deterministic reduce:
    split input into fixed block ids
    compute partial[block_id]
    merge partials by a fixed binary tree
    compare floating output by declared tolerance or exact rule

```

Histogram 的两级设计 Histogram 是 reduce、partition 和缓存布局的交汇点。朴素并行写法让所有线程对全局 bucket 做原子加法。若 bucket 少且输入大，热点非常明显。更好的写法是每个 worker 维护本地 histogram，最后合并。这样热路径写线程本地数组，避免全局原子争用。

Listing 13.7 线程本地 histogram 的核心形态

```
1 std::vector<std::uint64_t> histogram_local(
2     std::span<const std::uint32_t> keys,
3     std::uint32_t bucket_count) {
4     std::vector<std::uint64_t> buckets(bucket_count, 0);
5     for (const std::uint32_t key : keys) {
6         ++buckets[key % bucket_count];
7     }
8     return buckets;
9 }
```

这个版本只展示本地统计。完整并行版本要为每个 worker 准备一个 buckets 数组，再按 bucket 或按 worker 合并。`bucket_count` 很小时，本地数组很小，合并成本低；`bucket_count` 很大时，每个 worker 一份数组可能占用大量内存，而且合并会扫描很多零。稀疏 key 场景可能改用局部哈希表；密集小整数 key 场景用数组更快。选择数据结构要看 key 空间和实际非零数量。

Filter 和 Compaction 的推导 过滤输出满足谓词的元素，看起来像对 vector 做并发 `push_back`。朴素做法给输出 vector 加锁，每个命中元素都 push。这个做法正确但慢，因为锁进入每个命中元素的热路径。优化思路是把“我要写哪里”提前算出来。

第一步，每个 worker 统计自己块里命中的数量。第二步，对这些数量做 exclusive scan，得到每个 worker 的输出起始偏移。第三步，每个 worker 再扫描自己的输入，把命中元素写到自己的输出区间。这样每个元素最多读两遍，但写输出无锁且连续。若命中率很低，两遍读取可能仍划算；若谓词计算很重，可以在第一遍保存局部选择位图，第二遍避免重复计算。

```
stream compaction:
  count pass: local selected counts
  scan pass: worker output offsets
  write pass: each worker writes selected elements to owned range
```

这个推导要让读者看到优化从哪里来：不是凭空想到 scan，而是因为共享 push 的冲突需要被“所有权区间”替代。Scan 的作用就是把局部数量转换成全局唯一偏移。后续分布式 shuffle 也是同理：先知道每个 partition 有多少数据，再为每个 writer 分配输出位置。

Partition 的内存布局 多桶 partition 可以把输出组织成连续 bucket 区间，也可以每个 bucket 一个 vector。连续区间适合后续顺序扫描和一次性写文件，bucket vector 适合增量构建但容易产生多次分配。高性能实现常先统计 counts，计算 bucket offsets，分配一个大输出数组，再让 worker 写唯一范围。

如果需要稳定分区，worker 处理的输入区间顺序必须和输出区间顺序一致。对每个 bucket，worker 0 的元素排在 worker 1 前面，worker 内部保持原顺序。若不需要稳定性，可以在块内使用更快交换或局部重排。接口里声明稳定性很重要，因为稳定性会影响测试、性能和后续算法。

排序与分区的关系 分区可以看成粗粒度排序：只把元素按 bucket 聚到一起，不要求 bucket 内有序。排序则要求全局顺序。很多系统先 partition 再局部 sort，例如数据库按 hash 分区后每个 partition 内排序，或者分布式计算按 key 分区后 reduce 端排序。这样做能把全局大问题拆成多个小问题。

排序也能帮助 reduce。若输入按 key 排好，合并相同 key 只需要顺序扫描；若输入无序，需要哈希表聚合。排序成本是 $O(n \log n)$ ，哈希平均接近线性但内存随机访问多。小 key 空间可以用数组 histogram，稀疏大 key 用哈希，范围查询或外部合并用排序。并行算法教材应展示这种选择，而不是把所有聚合都写成一个 `unordered_map`。

并行算法和内存带宽 归约、scan、filter 和 partition 都可能受内存带宽限制。多线程能提高吞吐，直到内存控制器、LLC 或 NUMA 互连饱和。线程数继续增加，时间可能不降反升。报告中应画线程数扩展曲线，而不是只比较 1 线程和最大线程数。

算法阶段也要估算字节流量。Count-if 每个元素读一次，写 partial 很少；scan 读输入写输出，回填又读写输出；compaction 至少读输入两次或读输入加读位图；partition 读输入、写 counts、读写输出。若某个优化减少了计算却增加了内存流量，在带宽瓶颈下可能变慢。理解字节移动，是并行算法的基本功。

任务粒度和调度开销 并行算法要把工作拆成任务，但任务太小会被调度开销吞掉。每个任务提交、入队、唤醒、执行和记录统计都有成本。若一个任务只处理几十个整数，调度成本可能比计算成本大。任务太大又会负载不均，尤其是过滤、图遍历和字符串解析这类元素成本不均匀的工作。

工程上常用最小粒度阈值。例如每个任务至少处理数千或数万个元素，或者让每个任务目标运行几十微秒以上。这个阈值要测量，不同机器和任务不同。运行时可以自适应：开始使用较大块，发现长尾后切分剩余范围；或者过分割到 worker 数的若干倍，而不是无限细分。

错误传播和取消 并行算法不只处理成功路径。若某个 worker 在解析输入时发现错误，其他 worker 是否继续？已经写入的输出如何处理？partial 是否有效？线程池如何传播异常？取消后是否要等待正在运行的任务安全退出？这些问题决定算法能否进入真实系统。

一种简单策略是失败快速传播：共享一个原子错误标志，worker 在块边界检查；发现错误后停止提交新任务，等待已运行任务结束，然后丢弃输出。若输出写到外部文件，就必须写临时文件并在成功提交后重命名；失败时删除临时文件。分布式章节中的 attempt 提交协议，正是这个思想的跨机器版本。

源码阅读：并行 STL 和内核思想 C++ 标准库的并行算法接口展示了一个重要边界：用户提供操作，库决定执行策略。但并行 STL 的具体实现依赖标准库和后端，不能假设每个环境都真正多线程执行。阅读实现时，应关注执行策略如何切分范围、如何处理异常、如何限制任务粒度，以及哪些操作要求可结合或无副作用。

Linux 内核里也有类似思想，例如 per-cpu 计数、RCU 遍历和批量处理。它们不一定叫 reduce 或 scan，但思路一致：把高频写放到本地，低频合并；把读路径做轻，把更新延迟；把全局锁变成分层结构。源码阅读的目标不是照搬内核 API，而是识别这些并行结构在真实系统中的形态。

Segmented scan 普通 scan 在一个连续序列上计算前缀。Segmented scan 则把输入分成多个段，每段内部独立 scan。它适合按 key 分组后的前缀、稀疏矩阵行偏移、批量变长记录处理和图邻接范围处理。CSR 的 offsets 数组，本质上也和段边界有关。Segmented scan 让 scan 从“数组题”扩展到“结构化批数据”的位置分配工具。

实现 segmented scan 时，要处理段边界。若每个段很短，直接串行处理每段可能更快；若段很长，可以在段内并行；若段长度高度不均，长段会造成负载不均。可以先按段长度分类：短段批处理，长段拆任务。这个策略和图算法按顶点度分层类似。

Prefix sum 和内存分配器 内存分配器也会用到 scan 思想。假设每个 worker 需要输出不同数量的记录，先统计每个 worker 需要多少字节，对这些字节数做 prefix sum，就能一次性分配大缓冲，并给每个 worker 一个不重叠区间。这样比每个 worker 动态追加到共享 vector 更高效，也更容易恢复。

这种“先计数、再分配、再写入”的模式在本书会反复出现。Stream compaction 是元素级，partition 是 bucket 级，shuffle 文件是 partition 文件级，checkpoint manifest 是 block 级。理解 scan，就理解了许多无锁写输出的基础。

Radix partition 和 radix sort Radix partition 按 key 的若干 bit 分桶。第一轮按低几位分桶，第二轮按更高几位分桶，最终可以得到 radix sort。它常用于整数 key、哈希分区和数据库 join。与比较排序不同，radix 方法依赖 key 表示和桶计数，核心步骤是 histogram、scan offsets、scatter。也就是说，它把本章的三个核心模式组合在一起。

Radix partition 的性能取决于每轮处理多少 bit。每轮 bit 多，桶数多，counts 矩阵大，cache 压力大；每轮 bit 少，轮数多，读写次数增加。选择 radix bits 是 cache、TLB、内存带宽和任务粒度的综合问题。教材不需要给固定答案，但要让读者知道这是可测参数。

并行 stable sort 的代价 排序经常被当作库函数，但并行排序有复杂成本。它需要划分、局部排序、合并或采样分区。稳定排序还要保持相等 key 的原始顺序，可能需要额外内存或更保守合

并。若后续只需要按 bucket 分组，不一定需要完整排序；若后续需要 top-k，也不一定需要全排序。算法目标要匹配业务需求。

在 Compute Systems Engine 中，如果日志统计只需要每个 event 的计数，排序全部记录通常是过度；如果输出要按 event id 排序，最终对聚合结果排序即可，不必排序所有输入。这个例子提醒读者：先问输出语义，再选内核。

并行哈希表的困难 并行哈希聚合看似简单：多个线程往哈希表里更新 key。困难在于冲突、扩容、内存分配和 cache line 争用。全局哈希表加锁简单但竞争大；每个 bucket 一把锁降低冲突但锁数量多；线程本地哈希表最后合并避免热路径共享，但内存占用高；无锁哈希表实现复杂，删除和扩容尤其难。

因此很多系统先做线程本地聚合，再合并。合并阶段可以按 key 分区，把不同 key 交给不同 worker，减少冲突。若 key 空间倾斜，热点 key 仍会造成长尾，需要特殊处理。并行哈希表不是一行 `unordered_map` 加 mutex 能解决的。

负载估计和采样 Partition 和 reduce 的负载均衡可以通过采样改善。先抽样估计 key 分布或记录成本，再决定分区边界或热点 key 列表。数据库查询优化、分布式排序和 shuffle 都常用采样。采样有成本，也可能不准，尤其是输入分布随时间变化或热点很稀有时。

采样结果应进入报告。若系统根据样本决定把 key 42 视为热点，就要记录样本大小、估计频率和实际频率。否则热点处理失败时无法判断是采样不准、实现错误还是输入变化。性能系统里的“智能策略”都需要可观测证据。

并行算法的可组合性 单个内核正确，不代表组合后正确。Filter 后的输出顺序会影响后续 stable partition；partition 后的 bucket 顺序会影响 deterministic reduce；top-k 的 tie-break 会影响最终输出；scan 的 exclusive/inclusive 选择会影响写偏移。组合系统要把每个内核的语义写成合同。

Compute Systems Engine 的 kernel suite 应为每个内核记录：是否保持输入顺序，是否确定性，是否允许误差，是否可能丢弃，是否改变 key 表示，是否需要 materialization。这样流水线组合时，语义冲突能早发现。

案例：错误行压缩 日志解析中，错误行通常占少数。我们希望把错误行的行号和 offset 写到错误列表，用于最后报告。朴素并行做法是每个 worker 发现错误就加锁 push 到全局 vector。更好的做法是 stream compaction。第一遍每个 worker 统计错误行数；scan 得到每个 worker 的错误输出偏移；第二遍写入错误行元数据。这样错误列表稳定、无锁、可复现。

如果错误率很低，两遍扫描似乎浪费。另一种折中是每个 worker 使用本地 vector 收集错误，最后按 worker 顺序合并。本地 vector 会动态分配，但错误少时成本可接受。选择两遍 compaction 还是本地 vector，取决于错误率、稳定顺序要求和内存分配成本。教材案例要展示这种权衡，而不是只有一种答案。

案例：输出 offset 分配 Shuffle 文件写入也需要 offset 分配。每个 map worker 对每个 reduce partition 产生不同大小的 block。若希望把同一 partition 的 blocks 写入一个连续文件，可以先统计每个 block 大小，对 sizes 做 prefix sum，得到每个 block 在文件中的 offset。然后每个 worker 可以写自己的区间。这个过程和数组 compaction 完全同构，只是单位从元素变成字节。

这种设计避免了多个 worker 共享文件 append offset 的锁竞争，也让 manifest 可以记录每个 block 的位置和大小。代价是需要先知道大小，可能要先编码到临时缓冲或先估算。若大小无法提前知道，可以先写每 worker 临时文件，最后合并。算法选择受 I/O 和内存约束影响。

案例：并行去重 去重可以用 sort 或 hash。Sort-based 去重先排序，再扫描相邻相等元素；hash-based 去重边读边插入集合。并行去重时，sort 路线可以分区排序再合并，hash 路线可以按 key partition 到不同 worker，每个 worker 独占一部分 key 空间。若直接让所有线程更新一个全局 set，锁竞争和分配会很重。

去重语义也要明确。保留第一个出现，还是任意一个？输出是否保持原顺序？若保留第一个出现，并行 partition 会破坏顺序，需要额外位置比较。若只关心集合，算法可以更自由。很多算法题默认语义简单，系统工程必须把这些边界写清。

并行算法的内存上限 并行算法常用额外数组：partial、counts、offsets、selection、temporary output。线程数越多，partial 越多；bucket 越多，counts 矩阵越大；输入越大，selection 可能接近输入大小。内存上限必须进入设计。一个算法吞吐高但需要 10 倍输入内存，可能无法处理大数据。

报告中应列出额外空间复杂度和实际字节。例如 `partition counts` 是 `worker_count` 乘 `bucket_count` 乘 8 字节；`selection vector` 最坏是 `input_count` 乘 4 字节；per-worker hash map 取决于本地 unique key 数。写出这些数字，读者才能判断算法是否适合目标机器。

并行算法章节总结 Reduce、scan 和 partition 是并行数据处理的三根支柱。Reduce 消除共享写，scan 分配唯一输出位置，partition 把数据按后续处理需求重排。它们组合出 histogram、compaction、radix partition、shuffle 和分布式 reduce。掌握这些模式，比记住某个线程库 API 更重要。每个模式都要同时说明语义、内存上限、确定性和负载均衡。

并行算法决策表 设计并行算法时，可以先填决策表。输出大小是否已知？若已知，可以预分配并按区间写；若未知，先 count/scan 或使用 per-worker buffer。输出顺序是否稳定？若稳定，切分和合并必须保序；若不稳定，可以更自由重排。操作是否结合？若结合，可以树形合并；若不结合，需要固定顺序。key 是否倾斜？若倾斜，要考虑热点拆分。内存预算是否允许 per-worker state？若不允许，可能要分批处理。

这张表能把算法选择从“凭经验”变成“按约束推导”。同样是 filter，稳定输出和非稳定输出不同；同样是 histogram，稠密 key 和稀疏 key 不同；同样是 reduce，整数精确和浮点误差不同。并行算法的难点在约束，而不只是线程 API。

案例：分批 partition 当输入太大，counts 矩阵或输出缓冲无法一次容纳时，可以分批 partition。每批独立统计、scan、写临时 block，最后再合并 block。这样降低峰值内存，但增加多批调度和最终合并。分批处理还更容易 checkpoint，因为每个 batch 可以作为恢复单位。代价是全局稳定顺序更难，需要记录 batch 顺序和 bucket 内顺序。

分批 partition 是从单机算法走向分布式 shuffle 的中间形态。分布式系统本质上就是把超大输入拆成可调度、可恢复的批次。

案例：并行算法和容错 单机并行算法通常默认进程不崩溃，但大规模计算必须考虑容错。Reduce partial 可以重算，scan offsets 可以重算，partition block 可以根据 input split 重写。若每个阶段的输入输出边界清楚，失败后就能从最近边界恢复；若多个阶段完全融合，失败后只能从更早位置重做。算法分阶段不仅是性能问题，也是恢复问题。

这解释了为什么分布式系统喜欢 stage 边界和 materialization。它牺牲一些 I/O，换来可恢复性。单机高性能和分布式容错之间常有张力，系统设计要明确哪条路径更重要。

案例：确定性输出 并行算法常因为调度顺序不同产生不同输出顺序。若业务只关心集合，这可以接受；若输出文件要用于缓存、测试或审计，确定性很重要。确定性输出通常要求固定 partition 顺序、固定 worker block 顺序、固定 tie-break 和固定合并树。它可能牺牲一些性能，但能显著提高可测试性和可恢复性。

日志统计最终输出可以按 event type 升序，这样无论 map task 如何调度，输出文件都稳定。Top-k 可以按 score 降序、key 升序打破平局。Manifest 可以按 task id 和 partition 排序。确定性是大型系统调试的朋友。

并行算法章的回归阅读要从串行语义写起。Reduce 需要单位元、结合律、交换性和确定性说明；scan 需要说明前缀是 inclusive 还是 exclusive；filter 需要说明是否稳定；partition 需要说明输出桶顺序和元素顺序。没有这些语义，线程数增加后得到的只是某种结果，不一定是同一个问题的结果。

并行 filter 是本章最好的贯穿案例。朴素方案让每个线程 `push_back` 到共享 vector，立刻遇到锁竞争和输出顺序问题；改进方案先按块统计命中数量，再 scan 出每块写入偏移，最后并行写输出。优化点不是“用了并行”，而是把数据相关输出位置转成可计算位置。

项目中的 `ParallelSemanticContract` 应记录操作、单位元、顺序要求、误差容忍、块边界和重做边界。这样后续 shuffle 和分布式 reduce 才能复用同一语义。若本机 reduce 已经没有确定性，扩展到多节点只会把问题放大。

Scan 的工程价值要反复强调。它不是只用于算法题里的前缀和，而是并行分配输出位置的通用工具。每个块先统计数量，块数量再做 scan，最后每个块按自己的偏移写出，这个三段结构可以用于 filter、partition、稀疏矩阵构建和分布式文件偏移分配。读者掌握 scan，就掌握了从“数据决定输出位置”到“并行无冲突写”的桥。

本章还要讲确定性和浮点误差。整数计数通常结合且确定，浮点求和会随合并顺序改变低位，top-k 合并需要 tie-break，错误列表合并需要定义顺序。并行算法不是只追求快，很多生产系统宁可接受稍慢的固定合并顺序，也要得到可复现结果。报告里应写明哪个输出要求逐位一致，哪个只要

求误差范围，哪个允许无序。

Partition 要讲两类目标。第一类是为了并行写出，把元素按 bucket 放到连续区间，后续每个 bucket 独立处理；第二类是为了改善后续局部性或分支，把同类记录聚到一起。前者关注输出偏移和无冲突写，后者关注后续阶段收益。若后续阶段很轻，partition 的两遍扫描可能不划算；若后续 join、histogram 或分布式 shuffle 很重，partition 可能是关键。

重做边界也属于并行算法语义。一个 block 处理失败后，能否只重做这个 block，取决于它是否有独立输入、确定输出和无外部副作用。Scan 和 partition 如果写出了全局共享文件，就会让重做复杂；如果每个 block 写自己的临时输出，再由 manifest 提交，就更容易恢复。这个思想会直接进入 I/O 和分布式章节。

13.5 并行算法从串行语义推导切分方式

Reduce、scan、partition 的共同点，是把一个串行循环拆成多个可并行阶段。但不是所有循环都能随便切。先写串行 reference，说明输入、输出、错误顺序、溢出和确定性；再寻找可切分边界；最后才安排线程和任务。若跳过串行语义，结果可能快但不等价。

并行 count-if 是好入口。串行 reference 从左到右数满足条件的元素。并行版本可以把输入分块，每块本地计数，最后求和。这个改写成立，因为计数加法可结合，且输出只有一个总数。若问题变成“输出所有满足条件的元素并保持相对顺序”，就不能只本地 push，还要 scan 分配位置。问题微小变化会改变算法结构。

Reduce: identity、结合律和确定性

Reduce 需要 identity 和合并函数。整数加法 identity 是 0，最大值 identity 可能是最小可表示值，集合并 identity 是空集合。若 identity 选错，空输入和空块就会错。结合律决定能否改变合并顺序；交换律决定能否重排元素；确定性决定输出顺序和浮点误差是否必须固定。教材要把这些性质写成工程合同。

树形归约减少共享写，也缩短关键路径。线程本地 partial 先在本地累加，再按固定树合并。树太深会增加阶段，树太扁会形成热点，合并顺序影响浮点。项目可以实现线性合并、二叉树合并和按 NUMA 分层合并，比较关键路径、cache 行为和确定性。

错误处理也属于 reduce 语义。若某个块解析失败，是立即停止、收集所有错误、跳过坏行还是返回部分结果？不同选择会影响并行结构。只讲成功路径的 reduce 不够真实。

Scan: 从数量到位置

Scan 把局部数量变成全局偏移，是并行 filter、compaction、partition 的基础。按块 filter 时，第一轮每块统计命中数，第二轮对块计数做 exclusive scan，第三轮每块把命中元素写到自己的输出区间。这样没有共享 vector push，也能保持块间顺序。

Scan 的边界输入很多：空输入、单元素、所有块命中数为零、最后一块不足、计数溢出。实现时要明确索引类型和最大输出规模。对于很小输入，三轮 scan 的固定成本可能大于串行 push；对于大输入或多线程，消除争用的收益明显。实验要扫描选择率：0

分层 scan 连接任务图。每个块本地 scan，块总数再 scan，然后回填偏移。这个结构自然形成任务依赖：回填必须等块总数 scan 完成。后续运行时章节会把这些依赖交给任务图调度，而不是让 worker 阻塞等待。

Partition: 为 shuffle 做准备

Partition 按目标桶重排记录。朴素做法是每条记录直接 append 到目标桶，产生锁竞争和随机写。工程做法通常是每线程每桶计数，scan 得到每个线程在每个桶的输出区间，再批量写出。这样 partition 结果可以直接变成 shuffle block，也能记录每个 block 的字节数和 checksum。

热点 key 会造成分区倾斜。哈希分区不能保证每个 reduce 负载相等，尤其在 key 分布高度倾斜时。解决方式包括热点 key 单独分区、局部 combine、动态拆分大 partition、两级 hash 或范围分区。每种方式都会影响确定性和恢复。章节要把热点作为常规情况，不是异常边角。

Partition 函数一旦改变，输出兼容性也改变。分布式系统中，旧 worker 和新 worker 如果使用不同 partition 规则，会把同一个 key 发送到不同 reduce。项目里应给 partition 规则一个版本，manifest 记录版本，driver 拒绝混用。这样并行算法和分布式协议连接起来。

容错切块和重做边界

并行算法的块也是容错边界。一个 map block 失败，最好只重做这个 block，而不是整个作业。

块太小，调度和元数据成本高；块太大，失败重做成本高，长尾明显。块大小要同时考虑 cache、线程调度、I/O 和恢复。

项目实验可以构造三类输入：均匀记录、变长记录、热点 key。比较静态等量切分、按字节切分、动态任务队列和热点隔离。指标包括每块耗时、输出大小、worker 空闲时间、重做成本和最终确定性。报告要说明切分策略如何服务语义，而不只是吞吐。

本章最终判断力是：看到一个串行循环，先问合并语义、输出位置、写冲突和重做边界。能回答这些问题，再谈并行化；不能回答，就先回到 reference。

案例走查：并行 filter 为什么需要 scan

目标是保留所有错误日志行，并保持原始顺序。朴素并行版本让多个线程 push 到共享 vector，即使用锁保护，也会按线程调度顺序输出，破坏稳定顺序。若每线程本地 vector 后再按块顺序拼接，能恢复块级顺序，但还需要知道每块输出放到最终数组哪里。Scan 正是把每块命中数变成偏移。

实验输入包括无错误、全错误、稀疏错误、错误集中在某一块。比较串行 reference、共享 vector、每块 vector 拼接、scan 写出。指标包括输出正确性、稳定顺序、锁等待、内存分配次数和总耗时。共享 vector 可能在小输入上看起来够快，但顺序语义已经错；这能让读者理解正确性先于性能。

这个案例还能连接分布式 partition。Filter 的输出位置是本地数组偏移，partition 的输出位置是目标 block 内偏移，shuffle 的输出位置是 manifest entry。都是“先计数，再分配位置，再写出”的同一思想。

并行算法实验的确定性检查

并行算法最容易出现“结果看起来差不多”的松懈。整数计数必须字节级相等；浮点可以按误差合同比较；输出列表若要求稳定顺序，就要逐元素比较；若不要求顺序，也要排序或用多重集合比较。确定性检查要写在测试里，不要靠人工看样例。

多次运行同一输入也很重要。并发调度不同可能暴露非确定输出或数据竞争。对 filter、partition、histogram，连续运行 100 次比较 checksum 和输出顺序。若 checksum 稳定但字节顺序不稳定，要判断业务是否允许。分布式恢复也依赖稳定输出，否则重试后结果难以比较。

实验还要记录 partial 和 block 的中间摘要。最终结果错时，能定位是本地计数错、scan 偏移错、partition 写错还是合并错。只比较最终输出，调试成本太高。高质量项目会让每个阶段都有可检查摘要。

从本机 partition 到分布式 shuffle

本机 partition 写出多个连续 block，分布式 shuffle 只是把这些 block 交给对应 reduce 节点。若本机阶段已经记录 partition id、offset、bytes、checksum，分布式阶段就能直接复用。若本机只把数据塞进内存 vector，不记录边界，后续跨节点就要重构。

因此本章代码应提前采用 shuffle-friendly metadata。每个 block 有 job、stage、map task、attempt、partition、record count、bytes、checksum。即使当前只在本机运行，这些字段也能帮助测试和 trace。不要等到分布式章才发现前面的数据结构没有身份。

作业可以要求把 parallel partition 的输出写成本地 manifest，再由 reduce 读取 manifest 合并。这样读者会看到并行算法和 I/O 提交之间的连接。

综合练习：把 filter 结果变成 shuffle block

本题把本章三个算法连起来。输入是一批日志记录，先 filter 出错误记录，再按错误码 partition，最后为每个 partition 生成一个 block。要求保持错误记录在同一 partition 内的原始相对顺序，并给每个 block 生成 record count、bytes 和 checksum。

解法不能从并行版本开始。先写串行 reference：逐条扫描，错误记录进入对应 partition vector，最后输出稳定排序的 manifest。然后写并行版本：每块本地统计每个 partition 的错误数，二维 scan 计算每块每 partition 的写入偏移，再并行写出。这个过程同时使用 filter count、scan offset 和 partition layout。

测试包括空输入、无错误、全错误、单 partition 热点、多 partition 均匀、最后一块不足。每个 block 的 checksum 与 reference 比较。性能报告记录计数阶段、scan 阶段、写出阶段耗时。若 scan 阶段在小输入上占比很高，说明小输入应保留串行路径或提高 cutoff。

这个题直接服务分布式 shuffle。完成后，block metadata 可以交给 I/O 和 driver；若这里没有稳定 block 边界，后续分布式提交会很难写。

容易出错的边界：并行化不能改变问题定义

第一个反例是浮点求和。串行顺序固定，并行树形合并改变顺序，结果略有差异。若业务要求字节级一致，就必须固定合并树或使用更高精度；若允许误差，就写误差合同。第二个反例是稳定 filter。多个线程 push 共享输出即使元素都在，顺序也可能错。第三个反例是错误处理顺序。串行程序遇到第一条错误就停止，并行程序可能已经处理后面块。

第四个反例是 partition 版本变化。分区函数改动后，旧 block 和新 block 不能混合。第五个反例是负载按记录数平均。记录长度、key 倾斜和解析复杂度不同，记录数相等不代表成本相等。第六个反例是小输入并行化。任务创建、scan 和合并成本可能超过收益。

本章源码索引要求为每个并行算法写语义合同：是否要求顺序，是否允许重复处理，是否可重试，合并函数是否结合，identity 是什么，输出是否确定。合同写不清，不允许进入线程实现。这个要求会让读者在动手前先思考问题本身。

实验也要有 cutoff。项目不应对所有输入都强行并行。小输入走串行，大输入走并行，阈值由测量决定。报告写出 cutoff 如何选择，哪些设备需要重新测。并行算法成熟的标志，是知道什么时候不用并行。

本章核心接口：ParallelSemanticContract

并行算法章给项目留下 **ParallelSemanticContract**。它记录 identity、merge function、是否结合、是否交换、是否稳定顺序、是否允许浮点误差、输出位置如何分配、失败后能否重试。每个并行内核都要填写。这个合同决定能否 reduce、能否 scan、能否 partition 和能否重做。

后续任务图根据合同安排依赖，分布式根据合同决定 block 是否可重试。若一个操作不满足结合律，就不能被普通树形归约处理；若输出顺序稳定，就不能随便让线程并发 push。合同让算法语义进入工程对象。

从并行算法进入任务运行时

Reduce、scan、partition 给出了依赖结构：本地块先完成，块总数再 scan，偏移完成后才能写出，所有 map block 到齐后 reduce 才能合并。若用普通线程池，这些依赖要么靠任务内部等待，要么靠阶段性 barrier。任务运行时的价值，就是把依赖显式化，让等待不占 worker，让 ready 队列只保存真正可运行的节点。

项目中，ParallelSemanticContract 应能生成 TaskStateRecord 的依赖。比如 filter 的计数任务完成后，scan 任务 ready；scan 完成后，写出任务 ready。这样算法语义直接驱动运行时，而不是运行时盲目调度函数。这个衔接能让第四部分成为整体。

13.6 项目落地

Compute Systems Engine 在本章应增加三个模块。第一，parallel count-if：作为 reduce 的简单案例，展示全局原子反例和 partial 版本。第二，parallel scan：实现局部 scan、block sums scan、回填，并提供 inclusive/exclusive 两个接口或至少明确选择一个。第三，stream compaction：用 count、scan、write 三阶段把满足谓词的元素写到输出。

这三个模块会成为后续分布式 shuffle 的前置训练。count-if 训练局部统计，scan 训练偏移分配，compaction 训练无冲突写输出。分布式章节会把“写输出区间”升级成“写目标分区”，把“block sums”升级成“map 端摘要”。

从串行语义推导并行形态 并行算法不应从“开几个线程”开始，而应从串行语义开始。以 filter 为例，串行语义是按输入顺序检查每个元素，保留满足谓词的元素，并保持相对顺序。这个语义里有三个信息：谓词独立，输出数量未知，输出顺序稳定。谓词独立说明检查可以并行；输出数量未知说明不能让所有线程直接写同一个当前位置；顺序稳定说明每个元素的输出位置必须等于它之前满足谓词的元素数量。这样自然推出 count、scan、write 三阶段。

如果业务不要求稳定顺序，算法形态会改变。每个 worker 可以写本地 buffer，最后任意合并；甚至可以按 bucket 输出，不保留原顺序。这样减少 scan 成本，但输出语义变弱。若业务只需要集合，非稳定输出可以接受；若输出要用于测试、审计或和原记录对齐，稳定输出更重要。并行优化的第一步永远是明确哪些串行语义必须保留，哪些可以放松。

Reduce 也一样。串行求和有固定顺序；并行求和改变合并树。整数加法在不溢出的数学模型下结合，结果一致；浮点加法不严格结合，结果可能变化；字符串拼接结合但顺序重要；带副作用的操作根本不能随意并行 reduce。教材讲 reduce 时不能只写公式，还要让读者检查操作是否结合、是否交换、是否有单位元、是否要求确定性顺序。

并行写输出的三种模式 并行算法最大的工程问题之一，是多个 worker 怎样写输出。第一种模式是预先切分固定区间，例如 map 每个输入元素写一个输出元素，worker 按输入区间写输出区间，没有冲突。第二种模式是 count/scan/write，例如 filter、partition、错误行压缩，先计算每个 worker 或每个 bucket 的输出数量，再分配不重叠区间。第三种模式是本地 buffer 后合并，适合输出数量未知但稳定性要求较弱，或者输出很稀疏的场景。

全局锁 append 是最直观但通常最差的模式。它把所有输出写入串行化，还会让输出顺序依赖调度。全局原子 `fetch_add` 分配位置比锁轻，但仍然有共享热点，而且稳定顺序不一定满足。若只需要任意顺序，它可能可用；若需要输入顺序，它不够。工程报告要说明为什么选择某种写出模式，而不是只说“避免锁”。

写输出还要考虑内存分配。预先分配需要知道输出上界或精确数量；本地 buffer 需要处理扩容；临时文件需要处理 I/O 和失败清理；mmap 输出需要处理页错误和持久化。单机算法讲清这些细节，后续分布式 shuffle 就更容易理解，因为 shuffle 只是把输出区间从内存数组扩展到磁盘文件和网络分区。

可复现性和测试 oracle 并行算法的测试 oracle 可以是串行 reference，但要先定义输出是否要求完全一致。稳定 filter、确定性 sort、按 key 排序的聚合，适合和串行结果逐元素比较。非稳定 partition、任意顺序去重、近似 top-k，可能只能比较集合、计数或误差范围。测试必须匹配语义，不能用过强 oracle 错杀合理实现，也不能用过弱 oracle 放过错误。

可复现性在大型系统里很有价值。输出顺序固定，失败重跑更容易比较；stage manifest 固定，缓存命中更稳定；日志行号稳定，排查更容易。为了可复现性，系统可能选择固定合并顺序、固定 worker 分区、固定随机 seed 和固定 tie-break。这些选择会牺牲一点调度自由，但换来测试和运维上的确定性。教材应让读者知道，性能不是唯一目标。

当可复现性和吞吐冲突时，要按业务分类。离线训练数据生成、账务报表、审计日志通常更重视确定性；在线推荐候选召回可能更重视吞吐并接受顺序变化；科学计算可能重视数值误差可控。并行算法的“正确”不是抽象的，它来自业务语义和可验证性要求。

串行 reference 并行版本必须先有串行 reference。reference 定义顺序、误差、重复元素、异常和输出格式。没有 reference，线程越多越难判断正确。

reduce reduce 需要单位元和结合规则。整数加法和浮点加法的语义不同，字符串拼接和 top-k 更不同。合并树会影响结果顺序和误差。

scan scan 给每个位置分配前缀，常用于并行 filter 和 partition 的输出 offset。它把“我该写到哪里”从共享 push 转成确定位置。

partition partition 把数据按谓词或 key 拆分。稳定 partition 需要保持相对顺序，不稳定 partition 可以换更少数据移动。语义决定算法。

负载均衡 均匀切块在数据分布均匀时有效，热点或变长记录会造成尾部任务拖慢整体。切分策略要看输入分布。

确定性 并行执行可能改变顺序。某些任务允许不确定顺序，某些任务必须稳定输出。报告要明确是否要求 deterministic。

重做范围 失败恢复时，最小重做单位是什么？一个 chunk、一个 partition、一个 stage 还是整个作业？重做范围决定 checkpoint 和 manifest 的设计。

per-CPU 思想 Linux percpu counter 展示了本地计数再汇总的思想。用户态项目也可以先写线程本地 partial，再做可控合并。

案例：并行 filter 每个块先计数，通过 scan 得到全局 offset，再写输出。解释为什么这比共享 push 更容易证明。

案例：浮点 reduce 比较不同合并树对结果的影响，给出误差范围。性能报告必须包含正确性策略。

案例：redo chunk 让某个 chunk 故意失败，只重做该 chunk。manifest 记录 chunk 输入、输出和提交状态。

源码阅读路线 Linux 源码阅读从 `lib/percpu_counter.c`、`kernel/sched/core.c`、`lib/sbitmap.c` 附近进入即可。阅读目标不是把整套子系统抄进书里，而是追一条和本章实验相关的路径：用户态动作怎样变成内核对象，内核对象怎样排队、等待、唤醒、计数、提交或回收。

阅读报告要写证据等级。能直接测到的数字放在第一层；从源码控制流推导出的解释放在第二层；当前设备无法验证、只能作为合理猜测的内容放在第三层。这样读者会知道并行语义报告中哪些结论可复现，哪些结论需要换机器继续确认。

源码阅读还要克制。看到一个复杂函数，不要把每个分支都展开；先回到本章的问题，问它能否解释一个等待点、一次状态转移或一条性能曲线。解释不了的内容可以留到索引，不要打断正文主线。

并行 filter 每个块先计数，通过 scan 得到全局 offset，再写输出。解释为什么这比共享 push 更容易证明。

浮点 reduce 比较不同合并树对结果的影响，给出误差范围。性能报告必须包含正确性策略。

redo chunk 让某个 chunk 故意失败，只重做该 chunk。manifest 记录 chunk 输入、输出和提交状态。

串行 reference

并行算法先要有串行语义。reference 定义重复元素、顺序、误差、异常和输出格式。

实验设计：所有并行版本先与串行输出逐项比较。

结合律

reduce 需要结合规则。整数加法、浮点加法、top-k 和字符串拼接的合并语义不同。

实验设计：浮点归约固定合并树，记录误差范围。

partial

线程本地 partial 避免共享写热点，但增加合并阶段。合并树决定确定性、局部性和尾部成本。

实验设计：比较全局 atomic、每线程 partial 和树形合并。

scan

scan 给每个元素或块分配前缀位置，让并行 filter 不再抢共享 push。

实验设计：先局部计数，再全局 scan，再回填输出。

partition

稳定 partition 保持相对顺序，不稳定 partition 可以少搬数据。语义要求决定算法选择。

实验设计：给同一输入写稳定和不稳定版本，说明哪个适合 shuffle。

负载均衡

均匀切分元素数不一定均匀切分工作量。图、字符串和倾斜 key 需要按成本或二级分区。

实验设计：构造少数重元素，观察最长任务。

确定性

并行执行可能改变顺序和浮点误差。若结果需要可复现，就要固定合并树或输出排序。

实验设计：重复运行同一输入，检查输出是否完全一致。

重做范围

失败恢复不能总是重跑全作业。任务边界、partial 文件和 manifest 决定最小重做范围。

实验设计：让一个 partition 失败，只重做相关 chunk。

shuffle 前置

本章的 partition 和 scan 是分布式 shuffle 的前置知识。输出 offset 在单机上是数组位置，在集群里是 block 位置。

实验设计：把 scan 得到的 offset 映射到 shuffle block manifest。

项目接口

ParallelSemanticContract 记录串行语义、切分方式、合并规则、确定性、失败重做和测试输入。

实验设计：每个并行优化都要说明它是否改变输出顺序。

13.7 并行算法从串行语义和输出位置推导出来

并行算法章节不要一上来给线程版本。读者先要知道串行语义是什么：输入中哪些元素参与计算，输出顺序是否重要，浮点误差如何处理，异常和非法输入如何进入结果。没有串行 reference，线程越多越难判断正确。并行不是把循环切开那么简单，而是把串行语义拆成能独立执行、能合并、能恢复的片段。

reduce 的第一个问题是结合规则。整数求和通常容易，浮点求和会因合并顺序改变结果，top-k 需要保留候选集合，错误列表合并可能要求稳定顺序。合并树不是实现细节，它定义性能和确定性。若项目要求可复现输出，就要固定合并顺序；若只要求误差范围，就要在报告中写清容忍标准。

partial 是 reduce 的工程形式。全局 atomic 计数在多线程下形成共享写热点；每线程 partial 把冲突转成本地写，最后合并。这个思想从 NUMA 章延续而来，也会进入分布式 map-side combine。代价是内存增加、合并阶段增加、结果可见时间后移。教材要把这三个代价写出来，否则读者会把 partial 当成万能技巧。

scan 的出现来自 filter 的输出位置问题。串行 `push_back` 隐含了“前面保留了多少元素”；并行时每个块不知道全局位置。先局部计数，再对块计数做前缀和，每个块就得到自己的输出区间。这个推导比直接给 scan 代码重要。读者理解了输出位置分配，就能把 scan 用到 stream compaction、partition、shuffle offset 和内存分配中。

partition 要讲稳定性。稳定 partition 保持相对顺序，适合需要复现或后续依赖顺序的场景；不稳定 partition 可以交换元素，减少数据移动。按 key 分多个 partition 时，还要处理热点 key 和分区大小不均。单机 partition 的输出区间，到分布式阶段会变成 shuffle block 和 manifest 条目。两者共享同一个“输出归属”问题。

负载均衡不能只按元素数量。字符串长度、图顶点度数、key 倾斜、错误路径和 I/O 等待都会让相同元素数产生不同成本。并行算法报告要记录最长任务、平均任务、任务数量和尾部任务原因。若最后一个任务拖住全局完成，增加线程数不会自动解决，需要重新切分或引入动态调度。

容错重做范围是本章和分布式计算的桥。一个块失败后，是重做整个输入、一个 partition，还是一个 chunk？这取决于任务边界、partial 是否可识别、输出是否通过 manifest 提交。并行算法的切分方式如果从一开始就记录输入范围和输出范围，后面做 checkpoint 和 retry 会自然很多。

案例推导：并行 count-if 如何逐步长出 partial、scan 和重做边界

从最小问题开始：统计数组中满足谓词的元素个数。串行版本一行循环就够。并行化时，最坏写法是所有线程对一个 atomic 计数器 `fetch_add`。结果正确，但共享写热点严重。第一步改成每线程 partial，每个线程统计自己的区间，最后合并。这个版本已经展示了并行 reduce 的基本形状：切分、局部计算、合并。

如果问题从 count-if 变成输出所有满足谓词的元素，partial 还不够。每个线程知道自己保留多少元素，却不知道写到全局输出的哪里。于是 scan 出现了：先对每个块保留数量做前缀和，得到块的输出起点，再让每块写到确定区间。这里 scan 是由输出位置需求推导出来的，不是凭空引入的算法。读者理解这条推导，就能自然理解 stream compaction 和稳定 filter。

再进一步，按 key 分 partition。每个块不仅要知道总保留数，还要知道每个 partition 的数量；对每个 partition 的块计数做前缀，得到每个块在每个 partition 中的输出区间。这个结构就是单机 shuffle 的雏形。分布式时，输出区间变成文件偏移或网络 block，manifest 记录每个 block 的身份和范围。

重做边界也在这里出现。若一个块处理失败，只要输入区间、输出区间和 partial 身份清楚，就可以重做这个块，而不必重跑全作业。若输出是共享 push 出来的，没有稳定位置和身份，恢复就困难得多。并行算法的切分不仅服务性能，也服务容错。这个观点能把算法章和分布式章连起来。

实验可以按四个版本推进。版本一，串行 reference。版本二，全局 atomic count。版本三，每线程 partial count。版本四，filter 使用局部计数加 scan 写输出。版本五，按 key partition 输出。每个版本都保留相同输入和校验。报告中写明：共享写在哪里消失，输出位置如何确定，失败后能重做哪一段，哪些语义改变了。这样并行算法章节就不再像刷题，而像系统工程的算法基础。

实验：在 Compute Systems Labs 中扩展 parallel `count_if`。先让每个线程本地计数，再合并。随后实现输出压缩：先统计、扫描偏移，再写输出数组。

并行归约从串行语义开始 Reduce 的第一问题不是怎么分线程，而是合并函数是否有 identity，是否满足结合律，是否要求交换律，是否要求确定顺序。整数计数的加法适合并行归约；字符串拼接可以结合但输出顺序敏感；浮点求和数学上近似结合但机器舍入下不严格；带外部副作用的合并函数通常不能随意重排。若没有先证明语义，直接分块就是把 bug 并行化。

项目中的事件计数适合作为 reduce 入门。每个 block 产生局部 `std::unordered_map<EventId, std::uint64_t>`，合并时同 key 计数相加。Identity 是空 map，合并满足结合律；若输出需要按 key 排序，排序是后处理语义；若计数可能溢出，必须使用固定宽度类型并定义

溢出处理。这样读者能看到“可并行”不是感觉，而是由代数性质和业务合同决定。

树形归约减少关键路径和共享写 朴素并行归约常让所有线程把 partial 合并到一个全局 map，即使用锁也会形成热点。树形归约把 partial 两两合并，逐层减少数量。这样每层有多个独立合并任务，关键路径从线性合并变成对数层数，且每个任务独占自己的输出。代价是需要额外临时结构和任务调度。

树形归约还改善 cache 行为。小 partial 可以在 cache 中合并，避免所有线程争一个大表。若 key 分布倾斜，可以先对热点 key 单独处理，或者按 key range 分区后合并。归约策略要结合 key 数量、partial 大小、内存峰值和确定性要求。固定树形顺序还能让结果更可复现，尤其是浮点或需要稳定输出顺序时。

Scan 解决输出位置分配 Prefix scan 的价值在 filter 和 partition 中最直观。每个块先统计自己会输出多少元素，scan 把块计数转成全局起始偏移。之后每个块写入自己的连续区间，不需要全局 push 和锁。这就是从串行 **push_back** 推导并行 compaction 的过程。Scan 不是独立算法题，它是并行输出位置分配的基础设施。

Scan 也有 inclusive 和 exclusive 之分。输出起点通常用 exclusive scan；若要每个元素知道自己在保留元素中的排名，可以对 mask 做局部 scan。SIMD filter、GPU compaction、分布式 shuffle partition 都会用到相同思想。读者掌握 scan 后，很多“如何并行写输出”的问题会变得清楚。

Partition 是本地性和后续阶段的合同 Partition 把记录按目标分组。它的目的不仅是分发，还包括让后续阶段顺序读取同一类数据。一个好的 partition 方案要写清分区函数、分区数量、倾斜处理、输出格式、checksum 和恢复边界。若分区函数不稳定，重试后数据会落到不同位置；若倾斜严重，某个 reduce 会成为长尾；若输出没有 manifest，失败后无法判断哪些 block 有效。

项目中的 shuffle 可以先在单机模拟。Map 任务为每个 reduce partition 写一个 block，block 完成后写 manifest 条目。Reduce 只读取 manifest 中校验通过的 block。这个案例把 partition、可靠写入和分布式恢复连接起来，也为第 18 章毕业项目铺路。

实验：从全局锁计数到 scan filter 本章安排两组实验。第一组是事件计数：全局锁 map、线程本地 map、树形合并、按 key range 分区合并。比较锁等待、合并时间、内存峰值和确定性。第二组是 filter：串行 push、并行锁保护 push、两阶段 count 加 scan、块内 SIMD mask 加 scan。比较输出正确性、顺序稳定性和不同命中率下的性能。

报告要包含反例。若 filter 命中率极低，输出写很少，count 阶段可能占比高；若命中率极高，输出接近输入大小，scan 仍有成本；若输出顺序不重要，可以使用每线程局部输出后拼接，少做全局 scan。算法选择应从语义和输入分布推导，而不是只记一种套路。

Linux 和库源码阅读连接点 并行库中的 reduce、scan 和 partition 往往把任务切分、局部结果、合并树和异常处理封装起来。阅读时关注 chunk 如何确定、临时结果由谁拥有、异常如何取消后续任务、输出位置如何分配。Linux 内核里的 per-cpu counter 和分层统计也体现同样思想：高频本地更新，低频全局汇总。

这些源码能帮助读者理解一个更大的结论：并行算法不是把 for 循环拆开，而是把状态更新重新组织成局部、分层、可合并的形式。这个结论会贯穿任务运行时间和分布式计算。

事故复盘：并行输出位置从哪里来 把串行 for 循环切成多个块后，最容易出错的是输出位置。多个线程同时 push 到一个 vector，顺序不确定；锁住全局 map，结果正确但吞吐被锁拖住；浮点 reduce 换了合并顺序，低位误差变化；filter 每个线程通过数量不同，输出区间无法提前知道。并行算法首先要从串行语义推出切分方式，而不是先开线程。

Reduce 要先证明 identity 和 associativity，至少说明在本业务误差规则下是否允许重排。Filter 通常先 count，再 exclusive scan 得到每块输出区间，最后 scatter；partition 先做局部 histogram，再 scan 桶偏移，再写入目标范围。这样每个线程都有确定输出位置，避免共享 push。热点 key 和倾斜桶则需要额外策略，例如局部聚合、二级 reduce 或动态拆分。

实验要同时检查结果、顺序和重做范围。整数 reduce 可以严格比较，浮点 reduce 要写误差界；filter 要验证输出稳定性；partition 要报告每桶数量和最大倾斜；shuffle 前置要记录 partition manifest，供失败后只重做受影响范围。并行算法章节不是把算法名背熟，而是把“谁拥有哪段输入、哪段输出、哪次合并”讲清楚。

并行算法实验判读 reduce 结果正确后，还要检查合并顺序、输出顺序和重做范围。filter 的 scan 版本要验证每个块的输出区间不重叠；partition 要记录每桶数量和最大倾斜；浮点 reduce 要

比较误差范围；失败重试要只重做对应 chunk。若只给最终 checksum，无法证明并行语义完整。

这一段也服务于复习。读者合上书后，应该能凭本章的判读口径重新审查自己的代码：有没有最小正确版本，有没有能击穿旧方案的输入，有没有状态记录，有没有资源指标，有没有失败后的恢复动作。若五个问题缺两个以上，就说明实现还停在演示层，而不是工程层。

并行 filter 的三步推导 串行 filter 的直觉是看到一个元素满足条件就 push。并行版本不能让多个线程抢同一个 vector。第一步，每个块只数自己会输出多少元素；第二步，对块计数做 exclusive scan，得到每个块的输出起点；第三步，每个块写自己的连续区间。这样输出位置从共享副作用变成可计算结果。

这个推导比直接给 scan 算法更重要。读者会看到 scan 为什么存在：它不是抽象题，而是解决“并行输出位置唯一”这个工程问题。命中率低、命中率高、输出顺序要求稳定和不稳定，都可以在这三步基础上讨论取舍。

同样思路迁移到 shuffle。每个 map task 先统计 partition 大小，再分配 block，再写 manifest。若没有计数和偏移，多个 writer 会争输出；若没有 manifest，失败后 reduce 不知道哪些 block 有效。并行算法和可靠提交在这里连起来。

实验课：从全局 push 到 scan 分配输出 Filter 的朴素并行版本常用一个全局 vector 和锁。每个线程命中后 push，正确但争用严重，输出顺序也依赖调度。改进版先按块统计命中数，得到 count 数组；对 count 做 exclusive scan，得到每块输出起点；最后每块把命中元素写入自己的区间。这个过程把共享 push 变成确定偏移。

实验要覆盖命中率。命中率极低时，count 阶段成本可能明显；命中率极高时，输出接近输入大小，scan 仍然要做；命中率随机时，分支和写出都更复杂。若业务不要求全局稳定顺序，可以用 per worker buffer 后拼接，报告要说明语义差异。若要求稳定顺序，scan 方案更合适。

Reduce 实验同样从语义开始。整数计数有 identity 和结合律；浮点求和要说明误差；字符串拼接若要求顺序，就不能任意重排。Partition 实验要记录每个桶的数量，热点桶会决定 reduce 长尾。最终报告把 reduce、scan、partition 放在一条 shuffle 前置链路里，而不是三个孤立算法。

这个实验会让读者明白：并行算法的关键不是“开线程”，而是重新设计输出位置、合并顺序和重做范围。

并行算法练习验收重点 合格答案从串行语义开始。reduce 要写 identity 和结合律，scan 要写 exclusive 还是 inclusive，filter 要写输出位置，partition 要写倾斜和稳定性。答案必须给 reference 和至少一个反例，例如浮点顺序变化、热点 key 或输出顺序要求。

答案还要写“为什么不采用更复杂方案”。例如没有证据证明队列是瓶颈，就不应把锁队列换成无锁；没有证明输出顺序可变，就不应随意重排；没有证明错误路径能恢复，就不应只提交正常路径。能解释不做什么，说明读者开始具备工程判断。

最后，答案必须包含可复查材料。至少有输入摘要、关键状态、期望输出、实际输出、运行命令和未验证边界。没有这些材料，老师或同伴无法判断结论是否成立。第二册的作业不是让读者写漂亮段落，而是训练读者把系统问题变成可验证证据。

项目实现：归约、扫描与分区 本章对应的贯穿项目实现不要求一次写成完整框架，但必须有最小可运行切片。这个切片包含三个部分：一个 reference，一个带本章机制的实现，一个能击穿 reference 之外隐含假设的测试。Reference 用来固定语义，机制实现用来展示本章知识，测试用来证明新增复杂度确实解决了问题。

代码组织上，不要把所有内容放进一个大文件。基础类型、输入生成、核心机制、报告输出和测试应分开。即使当前只是教学项目，也要让目录能表达责任边界。这样后续章节接入时，不会因为找不到状态归属而重新发明一套对象。

每个实现都要输出一段机器可读摘要。摘要至少包含输入规模、关键状态、错误数、输出校验、耗时和未验证边界。这里的机器可读不要求复杂格式，简单的 key value 文本也可以。关键是让脚本和读者都能复查，而不是只能看屏幕截图。

测试应包含正常路径、边界路径和失败路径。正常路径证明功能可用；边界路径证明长度、容量、尾部、空输入、关闭或超时没有遗漏；失败路径证明本章机制不是装饰。若失败路径写不出来，往往说明本章概念还没有落到工程对象上。

报告最后写一个取舍段。新增机制带来了什么好处，增加了什么成本，哪些场景不应该使用它，下一章会复用哪个字段或对象。这个取舍段非常重要，因为系统工程不是把所有高级机制都加进去，

而是在证据和约束下选择合适的边界。

调试笔记：并行结果不同先查合并顺序 并行结果和串行不同，先查合并函数是否满足条件。整数计数通常可以重排，浮点求和可能因舍入不同，字符串拼接可能因顺序不同。再查输出位置是否唯一，filter 是否重复写，partition 是否漏桶。并行算法 bug 很多不是线程库问题，而是串行语义没有允许这种重排。

复查清单：归约、扫描与分区 复查本章时，先检查 reference 是否仍然存在。没有 reference，后面的并发、优化、异步或分布式改造都无法证明语义没有变。再检查压力输入是否尖锐：它应当攻击本章最核心的边界，而不是只把数据量放大。第三，检查状态是否可观察：关键对象的身份、状态、错误和提交结果应能在报告中找到。

第四，检查代码示例是否说明禁止行为。调用者不能做什么，往往比能做什么更能体现工程质量。第五，检查实验是否写出未验证边界；当前设备、权限或输入无法证明的结论，必须标注。第六，检查本章对象是否能被下一章复用，不能复用就说明边界还不稳。

完成这六项后，再看文字是否连贯。每个概念都应从问题推出，每个案例都应从朴素方案走到改进方案，每个实验都应能解释一个判断。若某段话放到其他章节也成立，就把它改成本章的具体对象、具体输入和具体失败。

13.8 综合案例：从串行输出推导 reduce、scan 和 partition

并行算法最常见的误讲，是先给线程版本，再解释它为什么快。对系统工程来说，顺序应该反过来：先写串行语义，再找出哪些状态可以局部计算，哪些输出位置需要全局协调，哪些合并顺序会改变结果。Reduce、scan 和 partition 不是三个孤立算法题，而是并行输出、局部聚合和分布式 shuffle 的共同骨架。

一、从 count-if 开始看共享写热点 问题先从最小形式开始：统计数组中满足谓词的元素数量。串行版本有一个计数器。直觉并行版本让所有线程对一个全局 atomic 计数器执行 `fetch_add`。这个版本结果正确，却把所有 worker 放到同一条 cache line 上竞争。线程越多，一致性流量越重，吞吐可能不升反降。

改进不是“换一个更高级原语”，而是改变状态形状。每个线程写自己的局部计数，最后合并。局部计数把高频共享写变成高频本地写，把低频合并留到阶段末尾。这就是 reduce 的工程本质：先局部，后合并。它的代价也要写清楚：需要 partial 存储，读取全局结果有延迟，最后合并阶段可能成为新瓶颈。

二、合并函数决定能否 reduce 并不是所有操作都能 reduce。合并函数需要 identity，通常还需要结合律；若要自由调整顺序，还要考虑交换律和确定性。整数计数的加法适合；浮点求和会因舍入导致顺序差异；字符串拼接虽然可以结合，但输出顺序敏感；带外部副作用的函数不能随便重排。讲 reduce 必须把这些语义条件放在代码之前。

项目中的事件计数可以定义为 `EventId` 到 `std::uint64_t` 的映射。identity 是空映射，合并规则是同 key 计数相加。如果要求输出可复现，就在合并后按 key 排序；如果 key 极端倾斜，就要考虑分区合并或热点 key 特殊处理。这个例子让读者看到，算法性质和工程输入分布要一起决定实现。

三、树形归约不是形式，而是关键路径控制 若有一千个 partial，单线程按顺序合并，关键路径就是一千次合并。树形归约把 partial 两两合并，多个合并任务可并行执行，关键路径接近层数。树形结构还让每个合并任务独占输出，减少锁和共享写。代价是临时对象更多，调度更复杂，内存峰值可能上升。

树形归约的形状也影响可复现性。固定树形顺序可以让浮点误差稳定，便于测试；动态窃取合并可能吞吐更高，但结果顺序不稳定。系统教材不能只写“树形更快”，还要写“树形定义了合并顺序、关键路径和临时结果所有权”。这三件事会在任务图和分布式 reduce 里继续出现。

四、scan 从稳定 filter 的输出位置里长出来 Filter 的串行语义是：按输入顺序检查元素，保留满足谓词的元素，并保持相对顺序。并行时，谓词检查可以分块独立执行，但输出位置不能靠全局 `push_back`。一个元素的输出位置等于它之前满足谓词的元素数量。这个数量本质上就是前缀和。于是稳定 filter 自然推导出三阶段：每块计数，对块计数做 exclusive scan，每块写到自己的输出区间。

Listing 13.8 稳定 filter 的三阶段计划对象

```

1 struct FilterBlockPlan {
2     std::size_t input_begin = 0;
3     std::size_t input_end = 0;
4     std::size_t local_count = 0;
5     std::size_t output_begin = 0;
6 };

```

这个结构比代码技巧更重要。它把输入范围、局部输出数量和全局输出起点写成可测试对象。若某个块失败，可以根据 input range 重做；若输出校验错误，可以定位到对应 block；若负载不均，可以看每个 block 的 count 和耗时。Scan 在这里不是抽象算法，而是稳定输出位置分配器。

五、partition 是多目标版本的 scan Partition 把每条记录送到某个目标分区。它比 filter 多了一个维度：每个块不再只有一个输出数量，而是每个 partition 都有数量。先统计每块每分区数量，再对每个分区沿 block 维度做前缀和，就能得到每个块在每个分区里的输出区间。这个结构就是单机 shuffle 的雏形。

分布式阶段把这些输出区间写成 shuffle block。每个 block 需要 partition id、attempt id、记录数、字节范围和 checksum。Reduce 端只读取 manifest 中提交成功的 block。这样第 13 章的 partition 不只是算法题，它直接生成第 18 章计算引擎的中间数据合同。

六、负载均衡不能只看元素个数 把数组平均切成 N 段很容易，但真实成本不一定按元素个数平均。日志行长度不同，字符串解析成本不同；图顶点度数不同，邻接扫描成本不同；key 倾斜会让某个 partition 特别大；错误密集输入会让冷路径变热。并行算法报告必须记录每个 chunk 的输入数量、输出数量、耗时和错误数量，不能只看总耗时。

当长尾出现时，有三类处理方式。第一，改变切分粒度，让任务更小，运行时可以动态调度。第二，按估算成本切分，例如按字节数、顶点度数或历史耗时。第三，对热点 key 或超大记录使用特殊路径。每种方法都带来额外元数据和调度成本。高质量教材应让读者知道：负载均衡不是“多开线程”，而是让工作量估算、任务粒度和调度策略匹配。

七、C++ 接口要限制副作用 并行 reduce 的合并函数应尽量是纯函数或局部可控副作用。它不能在每次合并时写外部文件、提交网络请求或修改全局状态，否则合并顺序一变，外部效果就会变化。Filter 谓词也应避免外部副作用，因为并行执行顺序和调用次数可能和串行不同。若业务确实需要记录错误，应把错误写到块级局部缓冲，阶段结束后按合同合并。

代码接口可以通过类型表达这一点。输入使用只读 span，输出区间由计划对象分配，局部错误写入属于当前 block 的 error buffer。全局提交只发生在阶段边界。这样做不是为了繁琐，而是为了让并行算法保留可重试能力。一个 block 失败后，如果外部效果还没提交，就可以安全重做。

八、实验矩阵要覆盖命中率和倾斜 本章至少安排两组实验。第一组是 count-if：全局 atomic、线程本地 partial、树形合并。输入覆盖低命中率、高命中率和随机命中。第二组是 stable filter 和 partition：全局锁 push、每线程本地 buffer 后拼接、count scan write、按 partition 的二维 scan。输入覆盖均匀 key、单热点 key、多热点 key 和极短输入。

报告要解释什么时候不用 scan。若输出顺序不重要，每线程本地 buffer 后拼接可能更简单；若命中率极低，count 和 scan 的固定成本可能占主导；若输入很小，串行路径更好。并行算法成熟的标志，是知道在哪些输入上关闭并行优化，而不是对所有输入强行上同一套结构。

九、和 Linux per-cpu 统计的关系 Linux 中大量 per-cpu 统计体现了同一思想：高频更新留在本 CPU，本地写避免全局 cache line 争用；读取全局值时再汇总。这和线程本地 partial 没有本质区别。区别在于内核还要处理 CPU 热插拔、抢占、读取一致性和近似值语义。读这些实现时，不要只看 API 名称，要看它们如何在“写入便宜”和“读取一致”之间取舍。

把这个思想迁移回项目，事件计数、任务完成数、窃取次数、I/O 完成数都可以先做局部统计，再周期性汇总。若某个计数用于协议决策，例如判断所有任务是否完成，就不能随便使用近似统计。统计和协议再次需要分开。

13.9 深度案例：把并行算法写成可恢复阶段

Reduce、scan 和 partition 在单机里是算法，在系统里也是恢复边界。每个阶段如果记录输入范围、局部结果、输出位置和提交状态，就能在失败后局部重做；如果所有线程直接写共享结果，失败后只能重跑全作业或接受不确定状态。本节把并行算法和容错联系起来。

一、ChunkPlan 是算法和恢复的共同对象 ChunkPlan 记录输入起止、预计成本、所属 partition、attempt id 和输出范围。算法用它安排并行任务；恢复用它判断失败后重做哪一段；测量用它记录耗时和输出数量。不要把 chunk 只当循环变量，它是系统对象。

一个好的 ChunkPlan 可以让 driver 重新调度失败块，让 reduce 识别重复 block，让 trace 解释长尾。若 chunk 只是线程内部的 begin/end 局部变量，失败后这些信息就丢失了。并行算法要进入系统，第一步就是把切分结果材料化成可记录对象。

二、局部结果要可校验 每个 partial 不仅有值，还应有输入范围、记录数、checksum 或摘要。这样合并前可以发现损坏或重复。对于简单计数，partial 记录 count 和 processed records；对于 map-side combine，partial 记录 key 数、总计数和 checksum；对于 partition block，记录每个 partition 的条数和字节数。

校验不是为了不信任自己，而是为了恢复和调试。并程序中，一个块的错误如果只在最终结果体现，很难定位。局部结果有身份和摘要，错误可以停在块边界。这个思想会直接迁移到分布式 shuffle block。

三、Scan 的中间数组是可审计材料 许多实现把 block counts 和 prefix offsets 当临时数组，用完就丢。教学项目中应把它们保留到报告里。Counts 解释每个块输出多少，offsets 解释每个块写到哪里。若输出错位，首先检查 counts 和 offsets；若负载倾斜，counts 也是证据。

保留中间数组还有助于恢复。若 count 阶段完成，write 阶段失败，可以不必重新 count；若输入没变，offsets 可以复用。真实系统是否这么做取决于成本，但教材要让读者知道这些中间结果的语义价值。算法里的“临时变量”在系统里可能是恢复状态。

四、确定性输出降低恢复成本 稳定 filter 和固定合并树让输出可复现。可复现输出有成本，可能限制调度自由或增加 scan；但它让测试、缓存和恢复更简单。若同一输入每次产生不同 block 顺序，manifest 和 checksum 管理会更难；若浮点归约顺序不固定，恢复后结果可能无法和前一次 bitwise 比较。

项目应按阶段选择确定性。事件计数输出可以按 key 排序，代价很小；浮点统计可以写误差合同；临时任务调度可以不确定，但提交 manifest 顺序应稳定。确定性不是绝对要求，而是工程工具。高质量教材要让读者学会选择在哪里购买确定性。

五、失败后重做要避免外部副作用 并行算法任务如果在局部计算中写外部文件、更新全局状态或发送消息，失败重做会产生重复效果。更稳的设计是局部计算只产生 partial，阶段边界统一提交。Map task 可以写临时 block，但只有 manifest commit 后有效；count task 只写局部 count，scan task 只写 offsets，write task 只写分配给自己的输出区间。

这条规则和 CAS 副作用边界类似：可重试区域内不要做不可回滚副作用。并行算法、原子协议和分布式提交在这里共享同一个原则。读者看到这种复用，才会觉得全书是连贯系统，而不是专题集合。

六、实验：故障注入的 parallel filter 实现 stable filter 后，故意在某个 chunk 的 write 阶段失败。系统应保留 count 和 scan 结果，清理或标记该 chunk 的输出区间，然后只重做该 chunk 的 write。若输出数组无法局部重写，说明输出位置合同不够清楚；若无法判断哪些元素属于失败 chunk，说明 ChunkPlan 不够完整。

第二个故障是重复执行同一 chunk。稳定输出区间让重复写同样内容是幂等的，前提是输入和谓词不变；若任务会追加到全局 vector，重复执行就会重复输出。这个实验能直观看到 scan 分配输出区间的恢复价值。

七、从单机恢复到分布式恢复 单机 ChunkPlan 到分布式就是 TaskAttempt；局部 partial 到分布式就是 shuffle block；offsets 到分布式就是 manifest 中的 block 范围；固定合并树到分布式就是 reduce 读取顺序。概念名字变了，问题没有变：输入是谁，输出在哪里，重复执行是否幂等，提交点在哪里。

因此第 13 章不只是算法章节。它为第 18 章毕业项目提供恢复骨架。读者如果在单机 filter 中已经理解 count、scan、write 和重做边界，后面理解 MapReduce 会容易很多。

13.10 补强案例：并行算法的数值合同

整数计数让并行 reduce 看起来简单，但许多工程计算涉及浮点、近似统计和概率结构。并行改变合并顺序后，结果可能不再 bitwise 相同。系统教材要把数值合同写清楚：允许误差多少，是否要求确定性，是否要求跨平台一致。

一、浮点加法不严格结合 串行浮点求和按输入顺序舍入，树形归约按另一种顺序舍入。结果通常只差很小，但在极大极小混合、正负抵消或 NaN 处理时差异明显。若业务是科学计算或财务，不同容忍度完全不同。并行前先写误差合同。

二、固定树形换确定性 固定 chunk、固定合并树和固定线程数可以让结果可复现。动态调度可能更快，但合并顺序变化。若需要 reproducible build 或审计，固定顺序值得。若只关心统计趋势，可以接受误差和非确定顺序。性能和确定性是显式取舍。

三、补偿算法和更高精度 Kahan summation、pairwise summation、更高精度累加都能降低误差，但增加计算成本和实现复杂度。并行中可以每块使用补偿求和，再树形合并 partial。报告要比较误差和时间，而不是只比较速度。

四、实验 构造浮点数组：全正均匀、极大极小混合、正负抵消、包含 NaN。比较串行、动态并行、固定树形、补偿求和。输出误差、确定性和耗时。这个实验能让读者知道，并行算法的正确性不总是逐字节相等。

13.11 补充案例：Cutoff 决定何时回到串行

并行算法不是输入越小越要并行。每个并行版本都有固定成本：任务创建、队列调度、partial 存储、scan、合并和同步。小输入上，串行可能更快。Cutoff 是工程算法的一部分。

一、Cutoff 来自测量不是猜测 对不同输入规模运行串行、并行 partial、并行 scan 三种版本，找到并行开始稳定胜出的规模。这个规模和机器、编译器、数据类型、命中率、线程池状态有关。不能把某台机器上的 cutoff 写成永恒常数，但可以写成默认值并允许配置。

二、Cutoff 也保护尾延迟 在线系统中，小请求走并行路径可能增加排队和调度延迟。离线系统中，小块太多会让任务图膨胀。Cutoff 让小任务在本地完成，大任务才进入并行运行时。它是性能和调度稳定性的共同工具。

三、分层 cutoff 可以有多个 cutoff：小输入串行，中等输入单线程 SIMD，大输入多线程，超大输入分布式。每层增加一种固定成本，也获得一种并行能力。报告要说明每层切换依据。

四、实验 对 count-if、filter 和 partition 测不同规模和命中率，画出串行与并行交叉点。把 cutoff 写入 ParallelSemanticContract，并在后续任务运行时使用。这样并行算法就不会对所有输入套同一方案。

13.12 补充专题：稳定性和审计需求

稳定输出不只是算法洁癖。审计、测试、增量构建和故障恢复都喜欢稳定输出。稳定 filter、稳定 partition 和固定 reduce 顺序能让同一输入产生同一输出文件，便于比较和缓存。

一、稳定性的成本 稳定 filter 需要 scan 分配位置；稳定 partition 需要保留相对顺序；固定 reduce 树可能牺牲动态调度自由。成本是真实的，因此稳定性要按业务购买。账务、审计日志、训练数据生成通常值得；临时监控可能不需要。

二、稳定性降低调试成本 输出不稳定时，diff 很难看，失败重跑难比较，缓存命中降低。稳定输出能让小差异暴露出来。项目中的最终结果应按 key 排序，manifest 条目按 partition 和 task id 排序。这样恢复后结果更容易验证。

三、稳定和幂等 稳定输出位置让重复执行更容易幂等。同一个 chunk 写同一个区间，同一个 attempt 写同一个临时 block 名，同一个 manifest entry 只提交一次。稳定性和幂等在系统里经常一起出现。

四、实验 实现稳定和非稳定 partition。比较速度、输出 diff、重复执行后的文件一致性和恢复复杂度。报告说明为什么最终项目选择哪些地方稳定，哪些地方允许不稳定。

13.13 补充专题：并行算法里的内存预算

并行算法常用更多内存换并发：每线程 partial、block counts、offset arrays、本地 buffers、临时 partition。内存预算不写清，算法可能在大输入上因内存峰值失败。

一、**Partial 的空间** 每线程 histogram 如果有一百万个桶，64 个线程就会非常大。可以按 shard 分配、稀疏 map、两级聚合或动态压缩。Partial 不是越多越好，局部化写入和内存峰值要平衡。

二、**Scan 的空间** Scan 需要 block counts 和 offsets。Block 越小，数组越大；block 越大，负载均衡越差。空间和调度粒度相关。报告要记录 block 数和 offset 内存。

三、**本地 buffer 的空间** Filter 或 partition 的本地输出 buffer 可以减少锁，但命中率高时会接近输入大小。若每线程都预留最大容量，内存浪费严重。可以按估算容量增长，或先 count 再分配精确空间。

四、**实验** 对 histogram 和 filter 记录内存峰值。改变线程数、bucket 数和 block size。报告解释某个版本虽然更快但内存不可接受的情况。系统算法必须同时满足时间和空间预算。

13.14 从输出位置推导并行算法族

归约、扫描和分区应该放在一起讲，因为它们解决的是同一个根问题：串行循环里隐含的“到目前为止”状态，在并行环境中必须变成可合并、可定位、可恢复的结构。串行 `push_back` 隐含前面已经输出了多少元素；串行求和隐含前面已经累加到什么值；串行分区隐含前面每个类别已经放了多少元素。并行算法不是把循环平均切开，而是把这些隐含状态显式化。

一、**reduce 把全局状态拆成 partial** 若操作满足合并规则，就可以让每个块产生 partial，再合并 partial。这里最重要的是合并函数，而不是线程 API。整数计数、最大值、最小值容易合并；top-k 要合并候选集合；错误列表可能要求稳定顺序；浮点求和要写误差规则。若合并函数不满足结合性或业务要求固定顺序，就不能简单并行 reduce。教学中要让读者先写串行 reference 和合并合同，再写并行版本。

二、**scan 解决每个元素写到哪里** 并行 filter 的困难不是判断元素是否保留，而是保留元素的输出位置。每个块先统计本块保留数量，对块计数做前缀和，就得到每个块的输出基址；块内再做局部前缀，就得到每个元素的位置。这一套推导比直接给 prefix sum 公式更重要。读者理解了它，就能把 scan 用到内存分配、压缩、稀疏矩阵构建、radix sort 和分布式 shuffle 的 block manifest 中。

三、**partition 是多目的的 scan** 二路 partition 可以看成 true 和 false 两个输出区间，多路 partition 则为每个 key range 或 bucket 分配区间。第一阶段统计每个块的每个 bucket 数量，第二阶段对每个 bucket 做前缀，第三阶段回填。稳定 partition 要保留块内和块间顺序，代价是额外扫描和更严格写入位置；不稳定 partition 可以交换元素，减少移动。选择稳定还是不稳定，取决于后续阶段是否依赖顺序和调试是否需要可复现。

四、**恢复要求阶段边界可记录** 并行算法一旦进入工程项目，就要处理失败和重做。一个 chunk 已经写了部分输出，另一个 chunk 失败，系统应重做什么？若输出位置由 scan 决定，chunk 的输入范围、输出范围和 checksum 都可以记录进 manifest。失败后只重做相关 chunk 或 partition，而不是全作业重跑。于是 scan 的中间结果不仅是性能结构，也是恢复材料。把算法阶段和恢复阶段统一起来，是本册区别于普通刷题讲法的地方。

五、**负载均衡要看工作量而非元素数** 相同元素数可能对应完全不同成本。字符串长度不同、图顶点度数不同、key 倾斜不同、错误路径比例不同，都会造成尾部任务拖慢整体。静态切分适合均匀输入，动态调度适合不规则输入，但动态调度会增加队列和同步成本。报告应记录最长 chunk、平均 chunk、任务数、每类输入分布和重做次数。并行算法不是“开线程”，而是把语义、位置、负载和恢复共同设计出来。

13.15 多路 partition 的实现细节

多路 partition 是连接单机并行算法和分布式 shuffle 的关键案例。它的目标不是简单把元素分成几类，而是为每个目标分区生成连续输出区间，并让每个 worker 知道自己应该写到哪里。这个过程天然分成统计、前缀、回填和提交四个阶段，每个阶段都有清楚的输入输出。

一、**局部直方图先统计形状** 每个 worker 扫描自己的输入 chunk，计算本 chunk 中每个 bucket 的数量。结果是一个二维表：chunk 乘 bucket。这个表的大小等于 chunk 数乘 bucket 数，bucket 很

多时会占用可观内存。若 bucket 数太大，可以分批处理 bucket，或者先采样估计热点。局部直方图不是附属结构，它决定后续输出布局。

二、对每个 bucket 做前缀 有了二维计数表后，对每个 bucket 在 chunk 维度上做 prefix sum，就得到每个 chunk 在该 bucket 输出区间中的起始位置。同时对所有 bucket 总量做 prefix sum，得到 bucket 在全局输出中的基址。两层前缀相加，才是元素最终写入位置。这个推导能帮助读者理解为什么 partition 和 scan 不可分。

三、回填阶段要避免共享写位置 回填时，每个 worker 只写自己 chunk 对应的区间。它可以维护本地 bucket offset，每处理一个元素就写到对应 bucket 的下一个位置。不同 worker 写不同区间，不需要共享 push，也不需要全局锁。若输出元素很大，可以先写索引，再批量搬运；若要求稳定顺序，要保持 chunk 顺序和 chunk 内顺序。

四、manifest 记录每个 bucket 的输出块 分布式 shuffle 需要知道每个 reducer 对应哪些 block。单机多路 partition 可以提前训练这个模型：每个 bucket 输出一个或多个 block，manifest 记录 bucket id、offset、length、checksum 和生成 attempt。失败后，只要某个 chunk 的回填失败，就重做该 chunk 影响的 bucket 区间或整个阶段。算法结构和恢复结构在这里合并。

五、热点 bucket 需要特殊策略 若大多数元素落在同一个 bucket，多路 partition 仍会产生负载倾斜。可以对热点 bucket 做二级切分，或者在 shuffle 阶段把热点拆成多个子分区。选择策略时要看后续 reducer 是否能合并子分区。热点处理不能放到最后临时修补，因为它改变输出归属和 manifest 语义。

13.16 确定性、审计和可复现输出

并行算法常被默认允许“不确定顺序”，但工程项目不能这么粗糙。某些场景只关心集合内容，输出顺序无所谓；某些场景需要 bitwise reproducible，方便测试、缓存和审计；某些场景允许浮点误差但要求误差范围稳定。确定性不是装饰，而是语义合同的一部分。并行版本是否允许改变顺序，必须在算法设计开始时写清楚。

一、输出顺序决定测试 oracle 如果 filter 输出要求保持输入相对顺序，测试可以逐项比较；如果允许乱序，测试就要排序后比较或按 key 统计；如果输出包含浮点值，还要定义误差比较。很多并行 bug 不是计算错，而是测试 oracle 和语义不匹配。先写清输出顺序，才能选择 stable partition、non-stable partition、固定合并树或任意合并树。

二、固定合并树换来可复现 浮点 reduce、top-k 合并、错误列表合并都会受合并顺序影响。固定合并树让每次运行按同一结构合并，即使线程实际完成顺序不同，最终结果也可复现。代价是调度自由度下降，某些快完成任务必须等指定伙伴。是否值得，要看业务是否需要审计。科学计算、账务、测试基准常常愿意为可复现付出代价；实时统计可能只需要误差边界。

三、审计材料来自中间结果 并行算法若要可审计，不能只保存最终输出。局部 partial、chunk 计数、scan 中间数组、partition manifest、checksum 都是审计材料。出现结果差异时，可以逐层比较：先看输入 chunk 是否一致，再看局部计数，再看前缀和，再看回填区间。这样排错从全局结果缩小到具体阶段。中间结果不一定长期保存，但调试和故障注入时应能打开。

四、确定性和性能的折中要明说 稳定输出、固定合并树和完整审计都会增加成本：更多内存、更多同步、更少调度自由、更复杂 manifest。不能把这些成本藏起来。报告应比较 deterministic 模式和 fast 模式，说明默认模式适合什么场景。高质量系统常常提供两种模式：开发、测试和审计使用确定性模式，生产热路径在语义允许时使用更快模式。

五、把可复现性写进接口 并行算法接口可以包含选项：是否稳定输出，是否固定合并树，是否保存中间审计材料，是否允许近似。调用者选择后，算法根据合同执行。这样确定性不是实现细节，而是可见语义。读者把这件事理解透，后续写分布式 shuffle、推理引擎 batch 或多线程日志处理时，都会主动问“这个输出需要可复现吗”。

13.17 贯穿案例：共享 push 改成 scan 后，为什么恢复也变简单

并行算法最适合从一个共享输出事故讲起。多个线程 filter 记录，每个线程遇到保留元素就 `push_back` 到同一个 vector。为了避免数据竞争，程序在 push 外加锁。小输入正确，大输入时锁竞争明显；去掉锁后结果乱写；改成每线程本地 vector 后，最后合并顺序不稳定；某个线程中途

失败时，系统不知道全局 vector 中哪些元素来自失败线程，哪些可以保留。这个事故同时暴露性能、确定性和恢复三个问题。

Scan 的价值就在这里出现。第一阶段，每个 chunk 只统计自己会输出多少元素，不写全局 vector。第二阶段，对每个 chunk 的 count 做 exclusive scan，得到每个 chunk 的输出起点。第三阶段，每个 chunk 只写自己的区间。这样没有共享 push，输出顺序由 chunk 顺序定义，某个 chunk 失败时它负责的输出区间也清楚。Scan 不是算法题孤岛，而是把“不知道会输出多少”转成“每个并行任务有确定写入范围”的系统工具。

一、先从错误方案保留教训 共享 push 的问题不只是慢。它把输出位置变成临界区中的运行时事件，谁先拿到锁谁先写。若业务要求稳定顺序，这个顺序不一定正确；若线程失败，已写内容和未写内容混在一起；若要重试，重复 push 可能产生重复记录。错误方案的价值在于暴露需求：我们需要提前知道位置，需要区间归属，需要可重做边界。

二、count 阶段只收集形状 第一阶段不写输出，只计算每个 chunk 保留数量。这个阶段幂等、容易重做，也便于审计。若 count 和 write 使用同一 predicate，就必须保证 predicate 是纯函数或输入状态一致；否则 count 得到的位置和 write 实际数量会不匹配。工程实现中，predicate 的版本、配置和错误处理都应写入任务记录。否则 scan 只是数学上正确，项目里仍会错。

三、exclusive scan 给出提交前的地址合同 Scan 结果告诉每个 chunk 的写入区间。这个区间可以进入 manifest：chunk id、output offset、output count、predicate version、checksum。写阶段只在自己的区间内写，完成后校验写入数量是否等于 count。若不等，说明 predicate 不一致或输入被修改，不能提交。这样并行算法和提交协议连接起来。

四、partition 是多维版本的同一事故 多 bucket partition 只是把 count 从一维变成 chunk 乘 bucket 的二维表。每个 bucket 单独 scan，得到每个 chunk 在每个 bucket 的区间。热点 bucket 会让某个输出区间特别大，下游 reduce 长尾也能从这张表看出来。若 bucket version 变化，旧输出区间不能和新版本混用。第 17 章的分片 epoch，其实就是 partition 规则在分布式尺度上的版本化。

五、浮点 reduce 也需要位置和顺序合同 Reduce 看起来不需要输出位置，但需要合并顺序。整数计数通常无所谓，浮点求和、top-k tie-break、错误列表合并都可能受顺序影响。固定合并树给出可复现合同，动态合并给出更高调度自由。接口要让调用者选择 deterministic 或 fast。这个选择和 scan 的稳定输出一样，都是把并行不确定性收束成可见语义。

六、恢复测试要故意打断 write 阶段 构造五个 chunk，count 和 scan 完成后，让第三个 chunk 写到一半失败。恢复程序应知道第三个 chunk 的输出区间无效，其他 chunk 的区间可保留或按 manifest 判断。若输出由共享 push 产生，恢复程序很难定位边界。这个测试能让读者看到 scan 对恢复的价值，而不只是对性能的价值。

13.18 案例深化：Scan 为什么是输出位置的算法

很多读者把 scan 当成一个独立算法题，忽略它在系统中的作用。Filter、partition、compaction、并行写出都需要回答同一个问题：每个线程输出到哪里。若多个线程直接 push 到共享 vector，正确性和性能都不稳；若每个线程先数自己要输出多少，再用 exclusive scan 得到偏移，输出位置就变成确定合同。Scan 是把不确定输出数量转成确定位置的工具。

一、Filter 的三阶段 第一阶段每个线程只 count，不写全局输出；第二阶段对每个块的 count 做 exclusive scan，得到块输出起点；第三阶段每个线程把通过过滤的元素写到自己的区间。这个结构避免锁，也让输出顺序可定义。若业务要求稳定顺序，按输入块顺序 scan；若不要求顺序，可以选择更快但不稳定的写法。语义决定算法。

二、Partition 是多桶 scan Partition 先做局部 histogram，得到每个线程每个桶的数量；再对每个桶做 scan，得到每个线程在该桶的写入区间；最后 scatter。它比 filter 多一个桶维度，也更容易出现倾斜。报告要写每桶大小和最大桶，否则后续 reduce 长尾无法解释。这个结构直接连接分布式 shuffle。

三、Reduce 要先讲结合律 整数加法通常满足结合律，但浮点加法在有限精度下不严格满足。并行 reduce 改变合并顺序，会改变低位结果。教材不能只写“reduce 要满足结合律”，还要让读者决定业务误差规则。若需要可重复结果，可以固定树形合并顺序；若允许误差范围，可以报告误差界。正确性合同先于并行化。

四、容错要知道重做范围 块级 partial 和 partition manifest 让系统知道失败后重做哪一段。若输出只是一个全局 vector, 某个线程失败后很难知道哪些元素有效。并行算法的输出区间和 manifest 不是分布式以后才需要, 单机容错和调试也需要。第 13 章应把算法输出边界和第 18 章的 manifest 连接起来。

五、实验要保留串行 oracle Filter、scan、partition、reduce 都应先和串行 oracle 比较。然后再测线程数扩展、桶倾斜、浮点误差和失败重做。若并行版只在正常输入下快, 却在倾斜输入下长尾严重, 报告要写出原因。并行算法的质量不在“用了多少线程”, 而在语义、位置和负载是否都可解释。

13.19 案例复盘: Scan 给并行输出定位置

多个线程 filter 后直接 push 到共享 vector, 顺序不稳定, 锁竞争也高。三阶段方案更清楚: 每块先 count, 通过 exclusive scan 得到输出起点, 再把元素写到自己的区间。Scan 在这里不是算法题, 而是把不确定输出数量转成确定输出位置的系统工具。

从 filter 到 partition Filter 是一维 scan, partition 是按桶做多维 scan。每个线程每个桶先做局部 histogram, 再按桶 scan 得到写区间, 最后 scatter。Reduce 则先证明 identity 和 associativity, 浮点场景还要写误差规则。语义决定是否允许重排。

实验判据 报告写每块输出数、scan 偏移、桶倾斜、输出顺序、浮点误差和重做范围。Partition manifest 记录每桶范围, 后续 shuffle 和失败恢复会复用它。并行算法的核心是可解释输出, 而不是只提高线程数。

13.20 本章习题

习题与作业

习题一: 为 parallel count-if 写完整正确性证明。证明每个输入元素被统计一次且只统计一次, partial 合并结果等于串行 reference。

习题二: 设计 inclusive scan 和 exclusive scan 的 API。给出输入 `[3,1,4]` 的两个输出, 并说明哪个更适合分配输出偏移。

习题三: 为 stream compaction 写三阶段伪代码。要求说明每阶段读写哪些数组、是否并行、是否有共享写、需要哪些临时空间。

习题四: 设计一个倾斜 key 的 partition 测试。要求解释为什么写入区间没有冲突, 但下游 reduce 仍可能负载不均。

习题五: 比较线性合并和树形合并。给出 partial 数为 8 时的合并步骤, 并说明什么时候树形合并不值得。

任务运行时与工作窃取

先看一个线程池死锁：四个 worker 正在执行四个父任务，每个父任务提交一个子任务后立即 `future.get()`。子任务都在同一个队列里等待，但没有空闲 worker 去执行它们；四个父任务又都占着 worker 等子任务完成。线程池没有崩溃，队列也没有数据竞争，却停止前进。问题不是线程数太少这么简单，而是运行时没有理解任务依赖，把“等待”当成了普通函数调用。

本章从这个饥饿事故进入任务运行时。线程是执行资源，任务是工作单位。把线程和任务分离，是构建可扩展运行时的关键；但一旦有 future、依赖、关闭、异常、取消和工作窃取，运行时就必须知道哪些任务 ready、哪些任务 waiting、哪些 worker 不能被同步等待占住。线程池、任务队列、工作窃取和任务图都从这个问题里推导出来。

14.1 从线程池到任务运行时

线程池创建固定数量或可控数量的工作线程，任务提交到队列，由工作线程取出执行。这样避免频繁创建线程，也便于控制并发度。线程池接口看似简单，难点在关闭、异常传播、任务等待、队列阻塞和避免死锁。

如果线程池里的任务再等待同一个线程池中的子任务，而工作线程数量不足，就可能发生饥饿。运行时设计必须考虑任务依赖，而不只是一个全局队列。

线程池的最小语义包括三件事：提交任务是否可能失败，关闭后队列中任务是否继续执行，调用 `wait` 或析构时是否等待所有任务完成。没有这些语义，线程池只是一个会运行函数的对象，不是可靠运行时。教材实现应先把这些边界写清，再谈性能。

14.2 Future、依赖和任务图

Future 把“将来会完成的结果”变成对象。它适合表达异步任务，但如果 worker 线程在执行任务时同步等待另一个 future，就可能形成线程池内阻塞。更好的方式是依赖表达给运行时，让运行时在依赖满足后再调度后续任务，而不是占着 worker 等。

这也是任务图运行时的价值：等待不再消耗工作线程，依赖边决定任务何时 ready。构建系统和数据流引擎都大量依赖这种模型。

Future 的危险是把异步伪装成同步。用户看到 `future.get()` 很方便，但如果在 worker 线程里调用它，就可能把 worker 变成等待者。运行时可以通过 work helping 缓解：等待时主动执行其他 ready 任务；也可以要求任务不能阻塞等待，只能注册 continuation。两种设计都要明确写进接口语义。

任务粒度 任务太大，负载不均；任务太小，调度成本高。合理粒度通常需要根据输入规模、核心数、缓存块和调度开销选择。递归拆分算法常设 cutoff，小于某个规模就顺序执行。

任务粒度可以通过一个简单不等式判断：每个任务的计算时间应显著大于一次提交、入队、出队和唤醒的固定成本。若任务只做几十条指令，调度成本会吞掉收益；若任务运行几百毫秒，负载不均会造成长尾。实践中常把任务切到缓存友好的块大小，再用 benchmark 调整 cutoff。

14.3 工作窃取、本地性和 NUMA

工作窃取让每个线程拥有本地双端队列。线程优先从本地取任务，空闲时从其他线程窃取任务。这减少全局队列争用，并改善缓存局部性。常见策略是 owner 从底部 push/pop，stealer 从顶部 steal。

工作窃取的正确性和性能都依赖队列设计。Chase-Lev deque 是经典方案，但实现复杂，涉及原子、内存序和边界竞争。教学时先理解模型，再进入源码。

工作窃取还有一个重要性质：正常情况下 worker 访问自己的本地 deque，只有空闲时才访问别人的 deque。这把高频操作留在本地，低频窃取才跨线程同步。相比所有 worker 抢一个全局队列，它减少了共享热点。这个结构和 NUMA、分布式本地优先原则一致：先消费近处工作，再去远处拿工作。

本地性和窃取方向 工作窃取通常让 owner 按 LIFO 处理本地任务，让 stealer 从另一端 FIFO 窃取较老任务。LIFO 有利于缓存局部性，因为新产生的子任务往往访问相近数据；窃取较老任务则倾向拿到较大工作块，减少窃取频率。这个设计体现了运行时对局部性和负载均衡的折中。

任务图 很多计算不是简单队列，而是任务之间有依赖。任务图把节点表示任务，边表示依赖。运行时在依赖满足后调度任务。构建系统、数据处理引擎和图计算框架都大量使用任务图思想。

任务图需要维护 ready 计数。每个任务记录还有多少前驱未完成；当前驱完成时，后继计数减一；计数变为零，任务进入 ready 队列。这个模型避免 worker 阻塞等待依赖。它的难点在于任务失败、取消、异常传播和图生命周期：一个任务失败后，后继任务是跳过、取消还是继续运行，必须在运行时语义里说明。

14.4 取消、关闭和异常传播

运行时必须定义关闭语义。提交中的任务是否接受，队列里的任务是否执行完，正在运行的任务是否可取消，异常如何传播，这些都要明确。没有关闭语义的线程池很容易在程序退出时丢任务、死锁或访问已销毁对象。

运行时观测指标 任务运行时不能只测总时间。还要观察队列长度、每个 worker 执行任务数、空闲时间、窃取次数、任务平均耗时、最长任务、等待依赖数量和关闭耗时。没有这些指标，负载不均和调度开销很难定位。一个任务图慢，可能是关键路径太长，不是 worker 不够；一个线程池慢，可能是任务太小，不是锁太慢。

这些指标也会指导分布式计算。单机里 worker 空闲，对应集群里 executor 空闲；单机里队列积压，对应集群里 stage backlog；单机里任务倾斜，对应分布式里的热点 key。运行时思维是从多线程走向分布式框架的桥。

最小线程池的状态机 线程池最小实现通常包含任务队列、worker 列表、停止标志、未完成任务计数和条件变量。状态机比代码更重要。open 状态允许 submit；shutting down 状态拒绝新任务但继续执行队列中任务；stopped 状态表示 worker 已退出。若没有这些状态，析构、等待和异常路径会变得模糊。

Listing 14.1 线程池状态的概念模型

```
1 enum class RuntimeState : std::int32_t {
2     Open,
3     ShuttingDown,
4     Stopped,
5 };
6
7 struct RuntimeCounters {
8     std::uint64_t submitted = 0;
9     std::uint64_t completed = 0;
10    std::uint64_t failed = 0;
11 };
```

submit 在 Open 状态下把任务加入队列并通知 worker；在 ShuttingDown 或 Stopped 状态下应失败或抛出明确异常。**shutdown** 把状态从 Open 改成 ShuttingDown，并唤醒所有 worker；worker 在队列为空且状态不是 Open 时退出；最后一个 worker 退出后，线程池进入 Stopped。这个流程必须能重复调用而不破坏状态，即 shutdown 应尽量幂等。

未完成任务计数用于 **wait**。提交任务时增加，任务完成或失败时减少，计数变为零时通知等待者。异常不能让计数泄漏，否则 **wait** 会永远阻塞。生产级线程池还需要保存异常并传给 future 或错误回调；教学实现至少要保证异常不会让 worker 线程静默终止后破坏计数。

任务对象和生命周期 任务不是单纯的 `std::function<void()>`。它可能携带输入范围、优先级、placement、依赖计数、取消标志、异常存储和指标字段。使用 `std::function` 便于教学，但可能分配内存、类型擦除成本较高，也不方便表达任务元数据。高性能运行时通常会使用自定义 task object、对象池或 intrusive queue。

任务生命周期至少有五个阶段：构造、提交、ready、running、completed。任务图还会有 waiting 状态，因为依赖未满足。取消和失败会引入 canceled、failed 等状态。每个状态允许哪些操作必须定义清楚。例如 running 任务是否能被取消？completed 任务的结果保存多久？shutdown 后 waiting 任务怎么处理？这些语义比 worker 循环更难。

全局队列线程池的 baseline 全局队列线程池是最好的 baseline。它容易实现，容易证明关闭语义，适合作为 correctness reference。它的缺点也清楚：所有 worker 竞争同一个队列锁；任务入队出队集中；任务局部性弱；负载高时队列可能成为瓶颈。正因为缺点明确，它才适合作为后续 work stealing 的对照。

教材实现应先写全局队列版本，并记录指标：每个 worker 执行任务数、队列最大长度、worker 空转次数、submit 等待时间。若这些指标显示全局队列不是瓶颈，就不应急着实现复杂 work stealing。若 worker 大量争用队列锁、队列长度波动大、任务很小且频繁，才有证据推进到本地队列。

工作窃取 deque 的所有权 工作窃取 deque 的核心是所有权不对称。owner 线程高频 push/pop 本地端，stealer 低频从另一端 steal。这个不对称让常见路径更轻，但边界竞争很复杂，尤其是 deque 里只剩一个任务时，owner pop 和 stealer steal 可能竞争同一个元素。Chase-Lev 算法之所以复杂，正是为了解决这些边界和内存序。

学习时可以先用 mutex 保护每个 worker deque，实现工作窃取策略，再用测量判断策略收益。等策略正确后，再研究无锁 deque。这样分层能避免把“调度策略是否有效”和“无锁 deque 是否正确”混在一起。运行时工程最怕同时引入新策略和新同步算法，出了问题很难定位。

关键路径和并行度 任务图的性能由两个量共同决定：总工作量和关键路径长度。总工作量是所有任务耗时之和，关键路径是依赖图中最长路径。线程数再多，也不能把关键路径压短到依赖允许之外。一个任务图如果关键路径很长，增加 worker 只会让更多线程空闲；如果总工作量大且依赖宽，worker 才有足够 ready 任务可执行。

因此任务图优化不只是让 worker 更多，还要改变依赖结构。把串行合并改成树形合并，可以缩短关键路径；把过大的任务拆分，可以增加并行度；把过小任务合并，可以降低调度开销；把热点 key 单独处理，可以减少长尾。任务图分析把算法结构和运行时调度连接起来。

Continuation 代替阻塞等待 阻塞等待会占住 worker，continuation 则把“前置任务完成后做什么”注册给运行时。前置任务完成时，运行时把后续任务加入 ready 队列。这样等待不占线程，依赖由计数器和队列表达。异步 I/O、构建系统和数据流引擎都大量使用 continuation 思想。

Continuation 的代价是控制流不如同步代码直观，异常传播和取消更复杂。结构化并发试图把异步任务组织成有作用域的树，让父任务能等待子任务完成并统一处理取消。第二册后续不需要完整实现现代 coroutine runtime，但要让读者理解：高性能运行时的很多复杂性来自“不要让线程闲等，但还要保持语义清楚”。

背压进入运行时 任务队列不能无限增长。无限队列把生产速度和消费速度的差异藏进内存，最终可能造成延迟爆炸或 OOM。有界队列把容量作为系统契约：生产者太快时必须等待、失败、丢弃、降级或转移。背压不是性能失败，而是系统保护机制。

线程池 submit 如果使用有界队列，应定义队列满时语义。阻塞 submit 会把背压传给调用者；try submit 返回 false 让调用者决定；带超时 submit 可以避免永久等待；丢弃策略适合可丢指标，不适合必须执行的任务。每种策略都影响上层业务。运行时不能只说“队列满了就等”，要说明谁等、等多久、能否取消。

运行时指标的低开销实现 指标本身不能成为瓶颈。每个任务完成时更新全局原子计数器，在高频小任务场景下可能形成共享写热点。更好的做法是每个 worker 写自己的 WorkerStats，周期性汇总。队列长度可以在队列锁内更新最大值，也可以采样。窃取次数由 stealer 本地记录，最后合并。指标精度和成本要匹配用途。

若指标用于调度决策，例如根据队列长度做负载均衡，就需要更实时、更一致；若指标只是实验报告，可以延迟汇总。不要把观测性和控制路径混为一谈。实验指标可以近似，协议状态不能近似。

任务运行时章的回归阅读应从全局队列 baseline 出发。所有任务进一个队列时，实现简单，但提交、窃取、本地性和长尾都混在一起。加入任务图后，每个 task 需要 ready count、依赖边、执行状态和完成回调；加入 work stealing 后，worker 本地 deque、窃取方向、任务粒度和 NUMA 本地性都会影响行为。

Future 等待是本章最典型的事故。worker A 在池内等待 future B，B 却排在同一个池的队列后面，

如果所有 worker 都这样等待，就会饥饿。修复不是随便加线程，而是区分阻塞等待和 continuation，或者让等待者帮助执行可运行任务。运行时必须知道哪些等待会占住 worker，哪些等待可以释放执行资源。

本章验收需要文本 trace。每条记录包含 task id、parent、ready count、worker id、start、finish、steal、cancel 和 exception。没有 trace，工作窃取只是“看起来更高级”；有了 trace，读者能看到关键路径、长尾任务、窃取成本和取消传播。任务运行时的价值是把并行控制流变成可诊断对象。

工作窃取队列还要说明所有权。常见设计让 owner 从一端 push/pop，thief 从另一端 steal，减少同一端竞争；但一旦涉及扩容、关闭和取消，状态机就变复杂。教材项目可以先实现简单固定容量版本或用锁保护的 deque 作为 reference，再讨论 Chase-Lev 这类无锁结构的边界。不要在没有测试和回收策略时直接把复杂 deque 当作教学起点。

任务拆分也要避免递归式失控。书中的 C++ 项目实现应优先使用显式 worklist、ready queue 和状态记录，而不是让运行时递归调用自己。显式状态更容易记录 trace、处理取消、限制任务数量和做故障恢复。算法解释可以画树，但工程实现要把树摊平成可观察的任务表。

取消语义必须早讲。一个 task 被取消，不表示已经运行的子任务自动消失，也不表示已经提交的 I/O 自动撤销。运行时要区分 not started、running、cancelling、finished 和 failed；取消只能阻止尚未开始的任务或让正在运行的任务在检查点退出。若代码把取消当作强制终止，资源释放、锁状态和部分输出都会变得不可信。

任务粒度要和 cache、调度和负载均衡一起选择。粒度太小，调度和队列开销吞掉收益；粒度太大，长尾任务拖慢 stage，工作窃取也难以平衡。一个实用策略是让每个 task 处理一个能放入 cache 的 block，同时保证任务数量明显多于 worker 数。项目报告应扫描 block 大小，而不是凭感觉选一个常数。

14.5 任务运行时把依赖、就绪和等待写成状态机

线程池只回答“有函数就找线程执行”，任务运行时还要回答依赖什么时候满足、任务何时 ready、等待是否占用 worker、失败如何传播、取消如何收束、长尾如何处理。工作窃取不是线程池的花样，而是在本地性和负载均衡之间做动态折中。若这些问题没有写成状态机，运行时只是一组难以调试的队列。

贯穿项目里的解析、map、shuffle、reduce 天然任务是任务图。Map task 生成 partition block，reduce task 必须等对应 partition block 到齐；checkpoint 必须等提交边界稳定；失败 task 可能重试，旧 attempt 的输出不能被下游读取。用队列顺序隐式表达这些关系，在动态重试和部分完成面前会崩溃。任务图把这些关系变成节点、边和 ready count。

从全局队列 baseline 到任务图

全局队列线程池是必要 baseline。它语义简单，容易测试 submit、wait、shutdown、异常计数和队列长度。它的缺点也容易观察：所有 worker 争用同一把队列锁，任务局部性弱，依赖只能靠任务内部阻塞等待表达。保留这个 baseline，后续 work stealing 或任务图才有对照。

任务图节点需要状态：Waiting、Ready、Running、Completed、Failed、Canceled。每个节点记录 remaining dependencies 和 successors。前驱完成时递减后继计数，减到零的线程把后继入 ready 队列。这样等待变成状态变化，不占 worker。失败时后继是取消、跳过、继续还是等待重试，由任务图语义决定，不能让每个任务函数自己猜。

任务图生命周期必须明确。图对象在所有节点完成或取消前不能销毁；任务捕获的数据必须由图或上层上下文拥有；异常要归集到 driver，而不是让 worker 线程静默退出。C++ 中提交 lambda 捕获局部引用是常见错误，本章要把它作为反例写透。

Work stealing 的本地性和窃取方向

工作窃取让每个 worker 优先执行本地 deque 中的任务，空闲时从其他 worker 偷取。Owner 高频 push/pop 一端，stealer 低频从另一端拿任务。这种不对称设计降低常见路径竞争，也保留刚产生任务的缓存局部性。窃取较老任务通常能拿到更大工作块，减少偷取频率。

但窃取不是越多越好。偷太小的任务，调度成本超过计算；偷访问大量本地数据的任务，会破坏 cache 或 NUMA 本地性；随机 victim 会在大机器上跨节点访问。项目可以先用 mutex deque 实现策略，不急着写无锁 Chase-Lev。先证明调度策略有效，再优化 deque 同步结构，这是工程上更可控的顺序。

指标决定判断。记录本地 pop 次数、steal 尝试、steal 成功、victim、被偷任务耗时、worker 空闲时间和任务队列长度。若 work stealing 后 steal 很多但吞吐不升，可能任务粒度太小；若 steal 很少但长尾仍在，可能 victim 选择或任务拆分有问题；若跨 NUMA steal 后变慢，说明本地性损失超过负载均衡收益。

Future、continuation 和阻塞等待

Future 表示未来结果，但 `future.get()` 如果在 worker 内调用，可能把 worker 变成等待者。若所有 worker 都在等待同一线程池里的子任务，系统会饥饿。Continuation 把“结果准备好后执行什么”注册给运行时，依赖满足后再入 ready 队列，避免线程空等。

外部 API 可以暴露 future，内部不必每个小任务都用重量级 future。运行时内部更适合 task node、ready count 和错误槽位；stage 级别再返回 future 给 driver。这样 API 易用，内部调度也不会被大量 future 状态拖慢。

结构化并发提供另一个方向：任务形成有作用域的树，父任务负责等待和取消子任务。第二册不要求实现完整 coroutine runtime，但要让读者理解异步控制流必须有收束点。没有收束点的异步任务，会在关闭、异常和资源释放时制造悬垂工作。

取消、异常和运行时可观测性

取消通常是协作式。强行杀线程会破坏 RAII、锁和共享状态。运行时设置 cancel token，任务在安全点检查；Waiting 或 Ready 任务可以直接标记取消，Running 任务只能请求停止，Completed 任务不能取消。取消传播沿任务图边发生，上游失败时下游是否取消取决于业务语义。

异常传播要收束。多个任务失败时，driver 需要首错、错误摘要和受影响任务列表。错误记录至少包含 task id、attempt、stage、输入摘要和异常分类。若异常只打印到 worker 日志，用户无法知道整个 stage 是否可靠完成。未完成计数必须在异常路径递减，否则 wait 会永久阻塞。

运行时 trace 是本章项目核心交付物。每个 task 记录 submit、ready、start、finish、cancel、fail 时间点；每个 worker 记录空闲、执行、steal；每个队列记录水位。trace 不要每条记录都打印文本，可以写结构化摘要。后续分布式 driver 的 request trace 会复用同样思想。

案例走查：Future 等待造成线程池饥饿

构造两个 worker 的线程池。任务 A 和 B 运行时各自提交一个子任务，然后立即 `get()` 等待。若两个 worker 都被 A、B 占住，子任务在队列里没有 worker 执行，系统死锁或饥饿。这个反例比直接讲 continuation 更有说服力：等待本身没有错，错在 worker 阻塞等待同一运行时的未执行任务。

修复一是禁止 worker 内同步等待，改用 continuation；修复二是在等待时 work helping，让 worker 执行其他 ready 任务；修复三是把阻塞等待交给外部线程，不占用计算 worker。每种修复都改变运行时语义。Continuation 清晰但控制流更复杂；work helping 更灵活但要防重入和栈增长；外部等待简单但可能降低并行度。

实验记录 ready 队列、running worker、blocked worker 和未完成任务数。报告不是只说“修复了死锁”，还要写出运行时新增的不变量：worker 不得在没有 helping 的情况下等待同池任务。这条规则会在项目代码审查中反复使用。

任务图 trace 的可视化文本格式

即使 EPUB 不适合图片，任务图也可以用文本格式表示。每行记录 task id、state、worker、start、finish、successors。再用缩进或箭头展示依赖：map tasks 指向 partition tasks，partition tasks 指向 reduce tasks。文本图在微信读书里不会变成图片，也便于复制到日志中。

例如 task 12 从 Waiting 到 Ready 用时 4 ms，从 Ready 到 Running 等待 2 ms，Running 10 ms，之后唤醒 task 30。若大量任务 Ready 后等待很久，说明 worker 不足或队列策略问题；若 Running 时间长尾明显，说明任务粒度或输入倾斜；若 Waiting 时间长，说明依赖关键路径长。trace 文本把运行时概念变成可诊断数据。

项目可以提供 `enginetracesummarize` 命令，输出 worker 利用率、最长任务、最长 ready 等待、steal 次数和关键路径估计。书中展示的是摘要，不是截图。这样也符合用户要求：EPUB 中不要依赖图片。

工作窃取的迭代实现路线

第一版只用全局队列。第二版每 worker 一个 mutex deque，owner 本地 push/pop，空闲时随机 steal。第三版加入 victim 选择策略和本地优先。第四版再考虑无锁 deque 或更精细内存序。每一版都有测试和指标，不跳级。

这种路线防止两类混淆。若一上来写无锁 deque，出错时不知道是调度策略错还是同步结构错。若一上来加入 NUMA、优先级和取消，指标也无法归因。运行时是复杂系统，必须小步推进。

作业可以要求读者解释为什么 owner LIFO、stealer FIFO 常见。答案应提到局部性和偷取较老较大任务，而不是只背结论。再要求设计一个反例：任务访问远端大数组时，随机偷取可能降低性能。这样 work stealing 不会被神化。

综合练习：估算任务图关键路径

给出一个任务图：8 个 map，4 个 partition，2 个 reduce，1 个 manifest commit。每个 map 耗时不同，partition 依赖对应 map，reduce 依赖多个 partition，commit 依赖 reduce。读者要计算总工作量和关键路径长度。线程数再多，也不能低于关键路径时间。这个题把运行时性能从“开几个线程”推进到依赖结构。

接着要求改图。把线性 reduce 合并改成树形合并，关键路径如何变化；把过大的 map 拆成两个 task，调度成本如何变化；把每条记录一个 task，ready 队列成本如何爆炸。读者要说明每次改动同时改变并行度和调度开销。没有这种分析，任务图优化容易走极端。

最后加入工作窃取。某个 worker 处理慢 map，其他 worker 空闲，steal 能否帮助取决于慢任务是否可拆。如果慢任务已经运行且不可拆，steal 没用；如果慢任务由多个子块组成，steal 可以拿走剩余子块。这个结论能防止读者把 work stealing 当成万能长尾药。

项目验收要求输出关键路径估计和实际 trace 对比。若实际耗时远高于关键路径，说明有调度、I/O 或同步开销；若接近关键路径，再增加 worker 收益有限。运行时成熟度来自这种解释能力。

容易出错的边界：运行时复杂度必须买到能力

第一个反例是为了工作窃取写复杂 deque，但任务负载本来均匀，全局队列不是瓶颈。第二个反例是任务图过细，每个节点只做几十条指令，调度成本超过计算。第三个反例是 continuation 滥用，让控制流分散，异常传播比同步版本更难懂。第四个反例是取消 token 存在但任务从不检查，取消只是装饰。

第五个反例是指标写成全局原子热点。运行时为了观测性能，反而制造新的共享写。第六个反例是 work stealing 破坏数据本地性，长尾减少了但 cache miss 和远端访问上升。运行时优化必须用 trace 证明买到了能力：更短关键路径、更高 worker 利用、更低队列争用、更稳定关闭或更清楚错误传播。

本章源码索引要求为每个运行时机制写“购买理由”。任务图购买依赖表达；本地队列购买局部性；窃取购买负载均衡；continuation 购买非阻塞等待；取消购买资源收束；trace 购买诊断能力。若某机制说不出购买理由，就不应进入最小项目。

项目可以保留运行时配置开关：global queue、local queue、stealing、trace、cancellation。实验按开关逐个打开，观察指标变化。这样读者能看到每个复杂度的收益，而不是面对一个一次性完成的黑箱运行时。

本章核心接口：TaskStateRecord

运行时章给项目留下 TaskStateRecord。它包含 task id、attempt、state、dependencies remaining、worker id、placement hint、submit time、ready time、start time、finish time、error code。每个任务状态变化都更新记录或 trace。运行时不再是黑盒，driver 可以根据记录判断长尾、取消和失败传播。

TaskStateRecord 也服务分布式章节。单机 task attempt 和远端 task attempt 共享字段，后续只增加 worker node、request id 和 epoch。这样本机运行时和分布式 driver 不会割裂。

从任务运行时进入 I/O 和背压

任务运行时处理 CPU 任务的依赖和调度，但真实 pipeline 中有些任务会等待 I/O。若 worker 直接阻塞写文件，运行时的 ready 队列再精巧也会被占住。下一章进入 I/O 和背压，是为了把等待设备完成这件事从 worker 执行中拆出来，让完成事件重新进入任务图或 driver 状态表。

项目中，TaskStateRecord 和 CompletionRecord 要衔接。写出任务提交 I/O 后，不应一直占用 worker；completion 到达后，后继 commit 或 reduce 任务才 ready。这样 I/O 等待不会伪装成 CPU 任务运行时间，背压也能通过队列水位反馈给上游。

深入补充：运行时的状态压缩和调试边界

任务运行时一旦进入项目，很快会遇到一个现实问题：状态太多。每个任务都有等待、就绪、运行、完成、失败、取消，每个 worker 有空闲、执行、窃取、阻塞，每个队列有空、非空、关闭、满。

若把这些状态都散落在日志字符串里，调试时无法聚合；若把它们都做成复杂类，又会让热路径负担过重。工程做法是状态压缩：运行时代码使用少量枚举和整数计数，trace 层再把它们解释成人可读文本。

例如 `TaskStateRecord` 中的 state 可以是 `std::uint8_t` 枚举，error code 也是稳定枚举，时间戳用单调时钟纳秒或微秒计数。热路径只写结构化记录，不格式化字符串。实验结束后，离线工具把记录转换成表格。这样既保留可观测性，又避免每个任务完成时都做昂贵日志格式化。教学代码可以简化，但要让读者知道生产系统为什么喜欢结构化事件。

调试边界还包括采样。所有任务都记录详细 trace，在小实验里很好，在百万任务场景下会制造大量数据。可以只记录慢任务、错误任务、每 N 个任务采样或每个阶段摘要。采样会丢信息，所以错误路径和状态转换异常不能被采样丢掉。这里的原则和性能计数器类似：观测要服务假设，不要把观测变成新的瓶颈。

深入补充：任务运行时和内存分配

任务运行时常常慢在看不见的分配。每个 task 用 `std::function` 捕获大对象，每个 continuation 分配共享状态，每个 future 分配控制块，任务图边用许多小 vector 动态增长，这些都会让分配器、cache 和 TLB 参与成本。优化运行时不能只看队列锁，还要看任务对象的内存形状。

教学项目可以先用简单实现，但要给出进阶路线。第一步，任务只捕获轻量 id，真正数据放在 batch 或 task context 中；第二步，任务节点用 vector 预分配，successor 边用连续数组或压缩邻接表；第三步，为短生命周期 task node 使用 arena，stage 结束统一释放；第四步，指标和 trace 使用每 worker buffer，减少共享分配。这些改动每一步都要有 profile 支持，不要一开始写复杂 allocator。

这个补充和数据布局章呼应。运行时对象也是数据结构，也有热字段和冷字段。Worker 取任务时需要函数指针或执行标签、状态和少量 id；调试信息、长错误消息和详细依赖列表不应放在热路径对象前几个 cache line。若运行时对象本身布局混乱，工作窃取再精巧也会受影响。

深入补充：优先级、饥饿和公平

任务运行时不只追求吞吐，还要处理优先级和公平。Manifest commit、取消传播、错误收束这类控制面任务可能很短但关键；大批普通 compute task 如果占满队列，控制面任务迟迟不执行，会让系统尾延迟变差。简单做法是给控制面任务单独队列或高优先级；复杂做法是多级队列和老化策略，防止低优先级长期饥饿。

优先级也会伤害局部性。高优先级任务频繁插队，可能破坏 worker 本地 deque 的 LIFO 局部性。公平也会伤害吞吐。严格轮转每个队列，可能让 cache 热任务被打断。运行时设计总是在吞吐、延迟、局部性和公平之间取舍。书中至少要让读者看到这些目标不是同一个方向。

项目可以给一个小实验：一批长 compute task 中插入短 control task，比较 FIFO、control priority 和双队列策略。指标包括 control task p95 延迟、compute 总吞吐、steal 次数和 cache 相关间接指标。这样优先级不是抽象调度理论，而是项目中的可测选择。

14.6 项目落地

Compute Systems Engine 的运行时路线应分四步。第一步，全局队列线程池：实现 submit、wait、shutdown，队列用 mutex 和条件变量，保证关闭不丢任务。第二步，任务指标：记录每个 worker 执行任务数、空转次数、队列最大长度和任务耗时摘要。第三步，任务图：实现 ready count，支持归约中的树形合并和并行扫描。第四步，工作窃取：先用 mutex deque 验证调度策略，再决定是否实现无锁 deque。

每一步都要保留前一步作为对照。全局队列版本是 work stealing 的 reference；顺序执行是任务图的 reference；固定任务粒度是动态调度的 reference。没有 reference，运行时一旦出错就很难判断是调度策略、同步结构还是任务语义的问题。

任务图 任务图把工作拆成节点和边。节点表示可执行单元，边表示依赖。运行时只应调度依赖计数为零的任务。

就绪队列 就绪队列不是普通容器，它决定局部性、公平性和窃取成本。每 worker 本地 deque 可以让新任务优先留在本地，减少共享队列竞争。

work stealing 窃取解决负载不均，但会破坏局部性并增加同步。常见策略是本地 LIFO、窃取 FIFO，让本地递归任务保持 cache 热度，空闲 worker 偷较老任务。

future 饥饿 worker 执行任务时阻塞等待另一个还没运行的任务，可能让线程池全部睡住。任务运行时应把等待表达成 continuation，而不是占住 worker。

取消传播 取消不是杀线程。取消信号要沿任务图传播，未开始任务可跳过，运行中任务协作检查，已完成任务只记录状态。

异常处理 任务异常要记录到任务状态并传播给依赖或调用者。吞掉异常会让图看起来完成，实际结果损坏。

NUMA 和本地性 任务窃取不能只看数量，也要看数据位置。远端窃取也许能减少空闲，但可能增加远端内存访问。

Linux workqueue Linux workqueue 展示了工作项、worker pool、调度和救援线程等工程问题。用户态不照抄，但可学习状态分离。

案例：future 饥饿复现 固定两个 worker，提交两个任务都等待第三个未运行任务。再改成 continuation，让依赖完成后触发后续。

案例：本地 deque 每个 worker 生成子任务放本地队列，空闲 worker 从别人尾部窃取。记录窃取次数和执行时间。

案例：取消传播 构造一个任务失败，依赖它的任务不应继续运行。报告说明每个任务状态。

源码阅读路线 Linux 源码阅读从 `kernel/workqueue.c`、`kernel/sched/fair.c`、`include/linux/workqueue.h` 附近进入即可。阅读目标不是把整套子系统抄进书里，而是追一条和本章实验相关的路径：用户态动作怎样变成内核对象，内核对象怎样排队、等待、唤醒、计数、提交或回收。

阅读报告要写证据等级。能直接测到的数字放在第一层；从源码控制流推导出的解释放在第二层；当前设备无法验证、只能作为合理猜测的内容放在第三层。这样读者会知道任务图运行报告中哪些结论可复现，哪些结论需要换机器继续确认。

源码阅读还要克制。看到一个复杂函数，不要把每个分支都展开；先回到本章的问题，问它能否解释一个等待点、一次状态转移或一条性能曲线。解释不了的内容可以留到索引，不要打断正文主线。

future 饥饿复现 固定两个 worker，提交两个任务都等待第三个未运行任务。再改成 continuation，让依赖完成后触发后续。

本地 deque 每个 worker 生成子任务放本地队列，空闲 worker 从别人尾部窃取。记录窃取次数和执行时间。

取消传播 构造一个任务失败，依赖它的任务不应继续运行。报告说明每个任务状态。

任务图

节点表示可执行工作，边表示依赖。运行时只能调度 ready count 为零的节点，否则会读到未完成输入。

实验设计：为 map、partition、reduce 画出任务图。

ready count

ready count 是依赖状态，不是调度建议。它必须原子或在锁内更新，避免任务重复进入队列。

实验设计：两个前驱同时完成，检查后继只入队一次。

本地队列

每 worker 本地 deque 减少全局队列竞争，也保护局部性。空闲 worker 才从别人尾部窃取。

实验设计：记录本地执行次数、窃取次数和全局队列访问次数。

work stealing

窃取改善负载均衡，但会破坏 cache 热度和 NUMA 局部性。策略要考虑任务大小和数据位置。

实验设计：构造不均匀分治任务，比较无窃取和窃取。

future 饥饿

worker 阻塞等待未运行任务会占住线程，导致池子全部睡死。等待应改成 continuation。

实验设计：固定两个 worker 复现互等，再改成依赖触发。

取消传播

任务失败后，依赖任务不应继续消耗资源。取消要区分未开始、运行中和已完成。

实验设计：让上游任务失败，检查下游状态。

异常

任务异常不能吞掉。它应进入任务状态、报告和依赖传播，否则图会显示成功但结果已坏。

实验设计：构造解析异常并检查 reduce 是否跳过。

NUMA 本地性

任务窃取不能只看队列长度，也要看数据在哪个 node。远端窃取可能比等待本地任务更贵。

实验设计：记录任务分片建议位置 and 实际 worker。

运行时 trace

任务运行时需要 trace：ready 时间、开始时间、结束时间、worker、窃取来源和等待原因。没有 trace，调度问题只能猜。

实验设计：输出每个 task 的状态行，再聚合关键路径。

项目接口

TaskRuntimePlan 写清任务字段、依赖存储、就绪队列、窃取策略、取消规则、异常传播和关闭协议。

实验设计：后续 I/O 和分布式模拟复用这套状态机。

14.7 任务运行时要把等待从线程上移到状态图上

任务运行时章节的核心不是“写一个更复杂线程池”，而是把依赖、就绪、等待、窃取和取消变成状态机。普通线程池只知道队列里有没有函数；任务运行时知道某个任务等待哪些前驱，完成后唤醒哪些后继，失败后取消哪些下游。这个差别决定它能否表达并行算法和 I/O 管线，而不是只能跑一堆互不相关的函数。

DAG 的 ready count 是关键状态。一个 reduce 任务依赖多个 map partial，只有所有前驱完成，它才可运行。两个前驱几乎同时完成时，ready count 更新必须保证后继只入队一次。这个问题看起来小，实际是并发状态机。用锁、原子还是单线程调度器都可以，但报告要说明线性化点和重复入队如何避免。

work stealing 解决负载不均，但不是免费。本地 deque 让子任务优先留在产生它的 worker 附近，保护 cache 热度；空闲 worker 从别人尾部偷较老任务，减少对本地递归路径的干扰。若任务携带 NUMA 数据位置，远端窃取也可能增加内存成本。调度策略不能只看队列长度，还要看任务大小、数据位置和关键路径。

future 饥饿是必须讲透的反例。worker 执行任务时阻塞等待另一个尚未运行的任务，可能让线程池所有 worker 都睡住。增加线程数只是推迟问题，不是语义修复。正确方向是把等待变成 continuation：当前任务登记后继，释放 worker；依赖完成时，后继进入 ready 队列。这样等待从线程栈上移到了任务状态图里。

取消和异常传播也要写进图。一个上游任务失败，下游任务不能继续用不存在的数据；一个任务抛异常，运行时不能吞掉然后标记成功。状态应区分 pending、ready、running、succeeded、failed、canceled。取消未开始任务可以直接跳过，运行中任务只能协作检查，已完成任务只能记录结果。没有这些状态，复杂项目会在失败路径里混乱。

Linux workqueue 可作为工程参照。内核里工作项、worker pool、救援线程和调度策略都围绕状态分离展开。用户态不需要照抄，但可以学习它如何避免把所有逻辑塞进一个全局队列。源码阅读的目标不是背函数名，而是回答：工作如何入队，worker 如何被唤醒，阻塞时如何避免系统停住。

本章实验可以构造三个场景。第一，两个 worker 的 future 饥饿复现，再用 continuation 修复。第二，不均匀分治任务，比较全局队列和 work stealing。第三，上游任务失败，验证取消传播和异常记录。报告必须输出每个 task 的 ready 时间、start 时间、finish 时间、worker、窃取来源和最终状态。这样读者会看到任务运行时是可观察系统，不是调度黑箱。

案例推导：从递归分治任务看 work stealing 的收益和代价

考虑一个分治计算：把大数组递归拆成两半，小到阈值后顺序处理。若每个子任务都进入全局队列，所有 worker 都竞争同一把队列锁，局部性也差。若当前 worker 生成子任务后把一半留在本地继续做，另一半放入本地 deque，空闲 worker 从尾部窃取，就能减少全局竞争，并让新近产生的数据保持在本地 cache 中。

work stealing 的策略有细节。本地端通常 LIFO, 保持递归深度和 cache 热度; 窃取端通常 FIFO, 偷较老、较大的任务, 提高负载均衡。若反过来, 可能偷到太小任务, 增加调度开销; 也可能破坏当前 worker 的局部路径。教材应通过任务时间线展示这些选择, 而不是只说“使用工作窃取”。

任务大小阈值也很关键。阈值太小, 任务数量爆炸, deque 操作、ready count 和窃取成本上升; 阈值太大, 负载不均, 最后几个大任务拖住完成。阈值还影响 NUMA: 大任务更可能保持数据本地, 小任务更容易被偷到远端。报告要记录任务数量、平均任务时间、最长任务、窃取次数和远端执行次数。

future 饥饿在分治中也可能出现。若一个任务 fork 两个子任务后阻塞等待它们, 而 worker 数量有限, 所有 worker 都可能在等待自己的子任务, 没人执行队列里的工作。正确方式是 continuation: 父任务登记未完成子任务数, 自己释放 worker; 子任务完成后递减计数, 最后一个完成者触发父 continuation。这样等待不占线程。

取消和异常要沿任务图传播。若左半部分发现输入损坏, 右半部分是否继续? 若业务允许部分结果, 可以继续并标记不完整; 若必须全局一致, 应取消下游 reduce。运行时不能替业务做决定, 但要提供状态表达。TaskRuntimePlan 应让任务携带 id、依赖计数、状态、异常摘要、取消原因和输出身份。

实验可以用不均匀数组: 左半部分计算轻, 右半部分计算重。比较全局队列、本地 deque 无窃取、本地 deque 加窃取。再加入一个失败任务, 观察取消传播。最后加入一个阻塞 future 版本, 复现饥饿。通过这些场景, 读者能看到任务运行时是算法、调度和失败语义的交汇点。

线程池接口的最小完整性 一个线程池要进入项目, 至少要有四类接口: 提交任务、等待完成、关闭运行时、查询指标。只有 submit 而没有 wait, 调用者不知道任务何时全部结束; 只有 shutdown 而没有明确拒绝新任务语义, 析构时可能丢任务; 没有指标, 性能问题无法定位。接口越小越好, 但不能缺少生命周期。

Listing 14.2 线程池接口的教学形态

```

1 class ThreadPool {
2 public:
3     explicit ThreadPool(std::int32_t worker_count);
4     ~ThreadPool();
5
6     bool submit(std::function<void()> task);
7     void wait_idle();
8     void shutdown();
9     [[nodiscard]] RuntimeCounters counters() const;
10
11 private:
12     void worker_loop(std::int32_t worker_id);
13 };

```

这只是接口草图, 不是最终性能实现。它暴露了几个必须讨论的问题: submit 返回 false 表示线程池关闭; wait_idle 等待已提交任务完成但不关闭线程池; shutdown 拒绝新任务并让 worker 退出; 析构必须安全调用 shutdown。若 task 抛异常, worker 不能直接终止而让未完成计数泄漏, 至少要捕获并记录 failed。

未完成计数和条件变量 线程池 wait_idle 常用未完成任务计数。提交成功时计数加一, 任务执行完毕后计数减一, 计数为零时通知等待者。这个计数必须和任务队列状态一起受锁保护, 或者用原子和条件变量 carefully 组合。若任务抛异常、被取消或线程池关闭时漏减, 等待者会永远阻塞。

等待谓词可以写成“未完成任务数为零”。这里要注意: 队列为空不等于任务完成, 因为 worker 可能已经取出任务正在执行; worker 空闲也不等于没有任务, 因为提交者可能正在入队。未完成计数把 queued 和 running 任务都算进去, 才适合 wait。

异常传播策略 线程池任务的异常不能悄悄消失。教学版可以记录失败次数, 并把异常转成日

志；future 版应把异常保存在 future 中，让调用者 `get` 时重新抛出；任务图版要决定一个节点失败后后继节点是取消、跳过还是继续。异常策略是运行时语义的一部分。

如果项目暂时不实现 future，可以要求提交的任务内部处理异常，线程池只捕获所有逃逸异常并记录 failed。这个策略简单，但调用者无法知道哪个任务失败。后续如果要支持可靠计算，必须引入 task id、结果对象或错误回调。

任务图 ready count 实现 任务图的核心是 ready count。每个节点记录还未完成的前驱数量。初始为零的节点进入 ready 队列；一个节点完成后，遍历后继，把它们 ready count 减一；某个后继减到零时，进入 ready 队列。这个模型把等待变成状态变化，而不是让 worker 阻塞。

Listing 14.3 任务图节点的核心字段

```
1 struct TaskNode {
2     std::uint32_t id = 0;
3     std::atomic<std::int32_t> remaining_dependencies{0};
4     std::vector<std::uint32_t> successors;
5 };
```

这里 successors 是 vector，适合教学。大型运行时会更关心内存布局，可能用连续边数组压缩图。ready count 必须能处理并发完成：多个前驱可能同时减少同一个后继计数，因此通常需要原子。减到零的那一次线程负责把任务入队。

任务图的生命周期 任务图不能在还有任务运行时被销毁。最简单方式是图对象由运行时持有，直到所有节点完成或取消。若任务捕获图中数据引用，生命周期更要清楚。C++ 中常见错误是提交 lambda 捕获局部引用，函数返回后 worker 才执行，导致悬垂引用。教学代码应优先捕获值或使用共享状态对象，并明确所有权。

任务图还要处理失败。如果一个前驱失败，后继是否仍然执行？对于构建系统，依赖失败通常取消后继；对于日志统计，某个 map task 失败可以重试，后继 reduce 等待最终成功 attempt；对于 best-effort 指标，失败可能被忽略。运行时不应替业务猜语义，但必须提供表达方式。

工作窃取和 NUMA 普通工作窃取从任意 victim 偷任务，NUMA 感知工作窃取会先在本节点内偷，再跨节点偷。这样可以优先保持 cache 和内存本地性。跨节点 steal 不是禁止，而是作为负载不均时的后备。这个策略和分布式数据本地调度一致：先本地，后远程。

实验可以构造两类任务。第一类任务只做 CPU 计算，数据很小，跨节点窃取影响小；第二类任务访问大数组块，数据按 NUMA 节点 first-touch 放置，跨节点窃取会访问远端内存。比较两类任务能说明“负载均衡”和“数据本地性”之间的张力。

任务粒度自动调节 固定 cutoff 很难适应所有机器。运行时可以记录任务平均耗时、队列长度和 worker 空闲时间，动态调整拆分粒度。若任务太短且队列争用高，就增大 chunk；若 worker 空闲多且长尾明显，就减小 chunk 或启用 work stealing。自动调节要谨慎，避免频繁震荡。

教学项目可以先不做自动调节，但 benchmark 应输出足够指标，让读者手动选择 cutoff。最终目标不是让运行时神秘地“自动变快”，而是让调整有证据。

本章项目验收清单 线程池进入 Compute Systems Engine 前，至少要通过这些验收：提交 N 个任务全部执行一次；任务抛异常不会杀死 worker；`wait_idle` 在任务完成后返回；shutdown 后 submit 失败；析构不会遗留 joinable thread；多线程提交不会破坏队列；指标能统计 submitted、completed、failed。任务图进入项目前，还要测试依赖顺序、并发前驱、失败取消和空图。

性能验收则包括：全局队列在小任务下的队列锁争用，任务粒度曲线，每个 worker 执行任务数分布，worker 空闲时间和空闲耗时。没有这些数字，工作窃取是否必要无从判断。

本章扩展习题

习题与作业

习题五：为线程池设计异常策略。比较“吞掉异常只计数”、“future 保存异常”、“错误回调”三种方案的语义和适用场景。

习题六：设计任务图内存布局。比较每个节点一个 vector successors 和统一边数组两种

方案的局部性、分配次数和修改难度。

习题七：构造一个 worker 捕获局部引用导致悬垂的例子，并改成安全的所有权模型。说明为什么并发代码里 lambda 捕获尤其危险。

习题八：设计 NUMA 感知 work stealing 策略。写出本地偷取、跨节点偷取、victim 选择和指标记录方案。

Future 等待的死锁风险 线程池里最危险的接口之一是 worker 内部等待 future。假设线程池只有四个 worker，四个任务都在运行，并且每个任务都等待同一个池中尚未开始的子任务。队列里有子任务，但没有空闲 worker 执行它们，系统就死锁。这个问题不是 future 本身的错，而是阻塞等待和固定 worker 数组合后的结果。

解决思路有几种。第一，禁止 worker 内部阻塞等待同池任务，改用 continuation。第二，允许 worker 等待时帮助执行队列里的其他任务，类似 work-first 或 help-first 调度。第三，把可能阻塞的任务放到独立线程池，避免占满计算 worker。第四，使用结构化并发，让父任务的等待由运行时管理，而不是任意阻塞线程。哪种策略合适，取决于运行时复杂度和业务语义。

```
pool deadlock shape:
worker 0 runs parent A and waits for child A1
worker 1 runs parent B and waits for child B1
worker 2 runs parent C and waits for child C1
worker 3 runs parent D and waits for child D1
child tasks are queued but no worker is free
```

教材要让读者形成一个原则：线程是执行资源，不要轻易拿执行资源去等待同一资源池里的未来工作。等待可以发生，但必须被运行时理解，或者发生在不会饿死被等待任务的地方。

Work-first 和 help-first 工作窃取运行时常讨论两种调度风格。Work-first 倾向于让当前 worker 继续执行新生成的子任务，把 continuation 留给别人偷；help-first 倾向于把子任务放入队列，当前 worker 继续执行 continuation。二者会影响 cache 局部性、栈深度、窃取频率和关键路径执行。经典 Cilk 风格偏 work-first，因为它让本地深度优先执行，减少调度开销，窃取者拿到较大的 continuation。

教学项目不必完整实现这些理论，但要理解任务提交顺序不是无关紧要。若每个任务递归拆分两个子任务，然后当前线程等待两个子任务，容易制造过多任务和等待。更好的方式是当前线程直接处理其中一部分，把另一部分提交给运行时，最后合并结果。这样能减少任务数，并保持本地性。

Listing 14.4 分治任务中保留一半本地执行

```
1 std::int64_t reduce_range(std::span<const std::int32_t> values,
2   ThreadPool& pool,
3   std::size_t cutoff) {
4   if (values.size() <= cutoff) {
5     return sequential_sum(values);
6   }
7
8   const std::size_t mid = values.size() / 2;
9   std::future<std::int64_t> right =
10  pool.submit_value([right_values = values.subspan(mid)]() {
11    return sequential_sum(right_values);
12  });
13  const std::int64_t left = reduce_range(values.first(mid), pool, cutoff);
14  return left + right.get();
15 }
```

这段代码展示思想，但也暴露等待风险：如果 submit_value 把任务放进同一个固定池，get

可能阻塞 worker。生产实现应让等待者帮助执行，或者使用任务图 continuation。教材写代码时要明确这是讨论调度形态，不是最终线程池 API。

任务优先级和饥饿 运行时常需要优先级：用户请求比后台压缩重要，控制面任务比数据面批处理重要，提交协议比指标刷新重要。优先级队列能让高优先级任务先执行，但也可能让低优先级任务长期饥饿。若低优先级任务负责释放资源或推进 checkpoint，饥饿会反过来影响高优先级任务。

一种常见策略是多队列加配额。高优先级队列可以获得更多调度机会，但运行时周期性执行低优先级任务。另一种策略是 aging：任务等待越久，优先级逐渐提高。还有一种策略是资源隔离：不同优先级使用不同线程池或不同队列容量，避免后台任务占满前台资源。优先级不是简单排序，而是调度公平性和业务目标的表达。

阻塞任务和计算任务分池 计算线程池适合 CPU-bound 任务，阻塞 I/O 任务适合异步 I/O 或独立阻塞池。把大量阻塞任务提交到计算池，会让 worker 被占住，CPU 计算任务无处执行。反过来，把 CPU 密集任务提交到 I/O completion 线程，也会延迟完成回调，破坏异步系统。运行时设计应区分任务类型。

Compute Systems Engine 可以定义两个执行域。Compute domain 运行归约、scan、partition、hash 聚合等 CPU 任务；I/O domain 负责文件读取、网络模拟和完成回调。两者通过有界队列连接，I/O 太快时对 compute 背压，compute 太慢时 I/O 暂停读取。第 15 章会继续展开这个管线。

本地队列的缓存行为 本地队列不仅减少锁竞争，也改善缓存行为。worker 刚生成的子任务通常访问相邻数据或共享上下文，放入本地队列后由同一 worker 继续执行，cache 命中更高。窃取者从另一端偷较老任务，通常粒度更大，减少窃取频率。这个设计把“常见路径本地执行，少见路径跨 worker 平衡”写进数据结构。

但本地队列也会让负载短时间不均。某个 worker 本地队列很长，其他 worker 空闲，需要窃取策略及时发现。victim 选择可以随机，可以轮询，也可以优先同 NUMA 节点。随机选择实现简单，避免所有窃取者同时冲向同一 victim；拓扑感知选择更复杂，但对大机器更好。实验要记录 steal attempts、successful steals 和 empty steals。

任务对象池和运行时局部性 任务对象频繁分配会成为隐藏成本。每次提交 `std::function` 可能分配闭包对象，任务图节点可能分散在堆上，worker 访问队列时不断追指针。运行时可以使用任务对象池或 arena，把同一批任务连续存放。这样既减少分配器开销，也改善遍历和调试。

对象池必须和任务生命周期一致。任务完成后，结果是否还被 future 持有？异常对象是否还要读取？指标是否还要汇总？如果任务对象一完成就回收到池里，而 future 还保存指向任务的指针，就会悬垂。高性能运行时常把任务执行状态和用户可见结果分离：任务对象可以复用，结果对象由 future 或 promise 管理。

运行时的内存屏障位置 线程池初版用 mutex 和 condition variable，内存可见性由锁提供。工作窃取和无锁 ready 队列会暴露内存序问题。任务生产者必须先写完任务对象，再把它发布到队列；worker 从队列取到任务后，必须看到任务字段。这个发布关系通常由锁、release/acquire 或队列内部协议提供。不要在队列外再用一个普通布尔标志表示任务 ready。

任务完成也需要发布。若 worker 写结果后把状态设为 completed，等待者读取 completed 后要看到结果字段。future/promise 已经封装这些同步；自研结果对象就必须自己建立 happens-before。运行时不是只调度函数，它还负责结果可见性。

取消传播 取消不是强杀线程。安全取消通常是协作式：运行时设置取消标志，任务在边界检查并尽快退出。边界可以是处理完一个块、完成一次 I/O、写完一个输出批次。取消标志可以是原子布尔，但如果它还发布错误对象或原因，就需要更强同步或锁。

任务图取消要传播给后继节点。若某个节点失败，依赖它的节点可能永远不 ready，因此运行时要把它们标记为 canceled，并减少全图未完成计数。取消还要处理已经在队列中但未执行的任务。最简单策略是 worker 取出任务后检查状态，若 canceled 就跳过并完成计数。更复杂策略是从队列中删除任务，但删除本身可能昂贵。

运行时和调试可视化 任务运行时适合可视化。一个时间线图可以显示每个 worker 何时执行任务、何时空闲、何时窃取、何时等待 I/O。一个任务图可以显示关键路径和长尾任务。一个队列图可以显示队列长度随时间变化。没有图，运行时问题很难凭日志理解。

即使不做完整 GUI，也可以输出 CSV 或 JSON trace。字段包括 timestamp、worker id、task id、event type、queue size、steal victim、duration。后续可以用脚本画图。教学项目中的 trace 不

需要高性能到生产级，但要让读者学会从执行历史看调度问题。

运行时源码阅读入口 源码阅读可以从三个成熟系统切入。第一，Cilk 或 oneTBB 的任务调度，关注工作窃取和任务粒度。第二，LLVM/MLIR 或编译器构建系统里的任务图，关注依赖和 worklist。第三，Linux scheduler 的 CFS 和 NUMA balancing，关注线程不是由用户态运行时独占控制，操作系统调度会影响 worker。阅读时不要试图一次读完全部源码，而是围绕一个问题：为什么 worker 空闲，为什么任务迁移，为什么等待时间高。

把用户态运行时和内核调度联系起来很重要。用户态线程池以为自己有 N 个 worker，但操作系统可能把它们迁移到不同 CPU，可能和其他进程共享核心，可能因为 I/O 睡眠唤醒改变亲和性。性能报告应记录线程绑定、CPU 利用和上下文切换，否则运行时结论可能不稳定。

结构化并发 结构化并发的核心是让任务生命周期有作用域。父作用域启动子任务，离开作用域前必须等待子任务完成或取消。这样资源不会在调用栈之外漂浮，取消和错误能沿任务树传播。相比随意提交 detached task，结构化并发更容易保证关闭和异常安全。

在线程池里，可以用 task group 表达这个思想。Task group 记录未完成子任务数、取消标志和错误状态。提交到 group 的任务完成后减少计数；某个任务失败后设置错误并请求取消；调用者等待 group 结束。这样 wait 的对象不再是整个线程池，而是一个有边界的任务集合。

结构化并发也有成本。任务必须属于某个作用域，不能随意长期存在；后台服务型任务需要单独生命周期管理；跨作用域依赖要谨慎。它不是所有场景的万能答案，但对批处理计算、并行算法和测试非常有帮助。

Work helping Work helping 用于避免 worker 阻塞等待时浪费执行资源。若 worker 等待某个 task group 完成，它可以继续从队列取任务执行，帮助推进系统，而不是睡眠。这样能缓解父任务等待子任务导致的线程池耗尽。Work helping 要小心重入和栈增长：等待路径执行了其他任务，其他任务可能又等待更多任务。

一种简单策略是只允许在等待同一 task group 时帮助执行该 group 的 ready 任务，避免任意重入。另一种策略是全局帮助，吞吐更好但控制流更复杂。教学项目可以先禁止 worker 内等待，再在运行时章节作为扩展设计 work helping。关键是接口必须写清：wait 在 worker 内调用时会做什么。

任务图内存布局 任务图节点如果每个都单独分配，执行时会产生大量指针追踪和分配器开销。更高效的布局是把节点数组和边数组连续存放。每个节点记录 successor 起始偏移和数量，successors 全部放在一个 `std::vector<std::uint32_t>` 中。这类似 CSR 图结构。任务图本身就是有向图，完全可以借鉴图布局。

Listing 14.5 任务图的 CSR 式布局

```
1 struct TaskGraphNode {
2     std::uint32_t first_successor = 0;
3     std::uint32_t successor_count = 0;
4     std::atomic<std::int32_t> remaining_dependencies{0};
5 };
6
7 struct TaskGraphStorage {
8     std::vector<TaskGraphNode> nodes;
9     std::vector<std::uint32_t> successors;
10 };
```

这个布局适合构图后执行多次或执行一次但节点很多的场景。若任务图动态变化频繁，连续边数组修改成本高，可能需要增量结构。再次说明：数据布局选择要看生命周期和修改模式。

任务窃取的粒度 窃取太频繁会增加跨 worker 同步和 cache 迁移；窃取太少会负载不均。一个常见策略是偷较大的任务或半个队列，而不是每次偷一个极小任务。分治算法中，窃取较老的 continuation 往往能拿到更大工作块；本地 worker 继续处理新生成的小任务，保持 cache 局部性。

实验要记录 empty steals。空窃取多，说明 victim 选择或粒度不合适；成功窃取少但 worker 空闲多，说明任务不足或本地队列不暴露；成功窃取多但性能差，可能窃取成本太高或破坏局部性。Work stealing 的好坏不能只看总时间。

任务取消的检查点 协作式取消需要任务在安全点检查取消标志。安全点通常是块边界、循环批次边界、I/O 完成边界或输出提交前。不能在任意指令强行取消 C++ 任务，否则可能破坏对象不变量、锁状态和资源释放。取消检查太频繁会增加开销，太少会延迟响应。

长任务应拆成可取消块。例如解析大文件时，每处理一批记录检查一次取消；归约大数组时，每处理一个 chunk 检查；shuffle 发送时，每个 block 边界检查。取消后要定义已经产生的 partial 是否丢弃、是否提交、是否需要清理。取消不是简单设置 bool。

运行时错误分类 任务运行时的错误至少分为用户任务异常、运行时内部错误、取消、资源不足和关闭拒绝。用户任务异常应归还给提交者或 task group；内部错误可能需要停止运行时；取消是预期控制流；资源不足可能触发背压；关闭拒绝说明提交太晚。把所有错误都记成 failed，会让上层无法做正确处理。

错误分类也影响指标。failed tasks、canceled tasks、rejected submissions、dropped low-priority tasks 应分开计数。否则看见失败数上升，不知道是业务 bug、取消策略还是过载保护。

运行时 trace 格式 Trace 事件应简单稳定。可以包含 event type、timestamp、worker id、task id、queue id、duration、extra value。事件类型包括 submit、start、finish、fail、cancel、steal_attempt、steal_success、wait_begin、wait_end、queue_full。输出 CSV 足够教学使用，后续可转 JSON。

Trace 采样也要考虑成本。每个任务都记录详细事件适合调试小规模；大规模时可以采样或只记录慢任务。运行时允许关闭 trace，只保留轻量 counters。观测性的开销要被测量，不能默认忽略。

运行时和分布式 driver 的对应 线程池运行时和分布式 driver 结构相似。线程池调度本地任务，driver 调度远端 task attempt；线程池 ready queue 对应分布式 pending task 队列；worker stats 对应节点指标；task group wait 对应 stage completion；异常传播对应 task failure；work stealing 对应任务重新分配。理解本地运行时后，分布式调度不再是全新概念，而是把执行实体从线程换成节点，把队列从内存结构换成 RPC 和持久化元数据。

这种对应关系能帮助项目设计。Compute Systems Engine 可以先在本机线程池里实现 task id、attempt、状态和指标，再把同样字段用于分布式模拟。字段稳定，尺度变化，系统更容易演进。

运行时章节总结 运行时把算法的并行性变成实际执行。它需要队列、worker、任务状态、依赖、等待、取消、异常、指标和关闭协议。工作窃取解决负载不均，但也带来本地性和同步问题；任务图解决依赖等待，但也带来生命周期和错误传播问题。一个好的运行时不是只会跑函数，而是能解释任务从提交到完成的每个状态。

运行时决策表 选择运行时结构时，可以先判断任务关系。任务独立且均匀，固定线程池加静态切分足够；任务独立但不均匀，需要动态队列或 work stealing；任务有依赖，需要任务图；任务会阻塞 I/O，需要分离 I/O 池或异步；任务需要优先级，需要多队列或调度策略；任务需要取消，需要 task group 和取消检查点。不要一开始就上最复杂运行时。

运行时还要和算法协同。算法切分太细，运行时开销大；算法切分太粗，work stealing 也救不了长尾；算法隐藏依赖，任务图无法调度；算法把大对象复制进 task，队列会变慢。运行时不是算法外部的黑盒，而是算法实现的一部分。

案例：慢任务隔离 如果某些任务可能非常慢，例如解析超长行、处理热点 key 或写慢磁盘，把它们和普通短任务放同一个 FIFO 队列，可能拖慢整体。运行时可以检测任务耗时，把慢任务标记出来，后续单独调度或拆分。也可以为不同任务类型设置队列，避免慢任务占满所有 worker。

慢任务隔离需要指标支持。没有 task duration 和类型标签，运行时无法知道谁慢。这个案例再次说明观测性是调度策略的前提。

案例：Speculation 的本地版本 分布式系统有 speculative execution，本地运行时也可以有类似思想。若某个任务长时间未完成，运行时可以把剩余可拆分范围重新切成更小任务，让其他 worker 帮忙。但这只适用于可拆分、幂等、输出可去重的任务。普通函数任务不能随便复制执行，否则副作用会重复。

这个案例连接运行时和分布式 attempt。Speculation 的前提永远是：重复执行不会产生重复外部效果，或者提交协议能去重。没有这个前提，speculation 是错误放大器。

实验：设计一个线程池接口，列出 submit、shutdown、wait 的语义。不要急着实现完整工作窃取，先实现固定线程数和全局队列，并写出关闭时不丢任务的测试。

任务运行时从等待事故开始 线程池最常见的事故是 worker 在池内等待同一池子的子任务。两个 worker 执行任务 A 和 B, A、B 各自提交子任务后立即等待 future; 子任务排在队列里, 但 worker 都被等待占住, 系统就饥饿。这个例子比直接讲任务图更有说服力: 问题不在 future 这个对象, 而在等待占用了执行资源, 运行时不知道依赖关系, 只看到两个正在运行但不推进的任务。

修复方向有三种。第一是禁止 worker 内同步等待, 改用 continuation, 把后续工作注册为依赖满足后的任务。第二是 work helping, worker 等待时主动执行其他 ready 任务, 但要防止重入、栈增长和异常传播复杂化。第三是把阻塞等待交给外部线程, 计算 worker 只执行 ready 任务。每种方向都改变运行时语义, 必须写进接口文档。

任务图把等待变成状态变化 任务图的节点记录状态、依赖计数、后继列表、输入摘要、输出位置和错误槽。前驱完成时, 后继的依赖计数减一; 计数为零, 后继进入 ready 队列。等待不再占用 worker, 而是体现在节点状态里。失败时, 后继是取消、跳过、等待重试还是继续, 也由图语义决定。这样运行时能解释整个计算, 而不是只执行函数。

贯穿项目天然任务是任务图。InputSplit 产生 ParseBatch; ParseBatch 产生 LocalAggregate; 多个 LocalAggregate 进入 MergeAggregate; CommitOutput 等所有 reduce 完成。Retry 会产生新的 attempt, 旧 attempt 的输出不能自动生效。用任务图表达后, driver 可以记录 trace、估计关键路径、取消下游任务和恢复未完成节点。

工作窃取从局部性推导 全局队列线程池的问题是所有 worker 抢同一队列, 而且新产生的子任务可能被任意 worker 拿走, 局部性差。工作窃取让每个 worker 有本地 deque, 优先执行本地任务, 空闲时才从其他 worker 偷取。Owner 通常 LIFO 取本地新任务, 保持 cache 热; stealer 从另一端拿较老任务, 往往拿到更粗的工作, 减少窃取频率。这个设计同时服务局部性和负载均衡。

不要一开始就写无锁 Chase-Lev deque。教学路线应先实现全局队列 reference, 再实现带锁本地 deque 的 work stealing, 记录 steal 次数、本地命中、worker 空闲和任务耗时。只有确认策略收益后, 再考虑无锁 deque。否则同步算法 bug 和调度策略问题会混在一起, 读者无法定位。

任务粒度决定运行时是否值得 任务太小, 提交、入队、出队、唤醒和 trace 成本会吃掉计算; 任务太大, 长尾无法平衡。合理粒度通常让每个任务有足够计算量, 且任务数量明显多于 worker 数, 同时保留 cache 友好的块大小。项目可以扫描 block size, 记录吞吐、队列等待、steal 次数和关键路径。不要凭感觉选择常数。

任务拆分还要可取消和可恢复。递归拆分看起来自然, 但工程实现应使用显式 worklist 和 task table, 避免深递归, 也便于 trace、取消和重试。树形任务可以在解释时画成树, 代码里则把节点、边和状态摊平。

Trace 是任务运行时的眼睛 没有 trace, 工作窃取只是“好像更高级”。每个 task 应记录 submit、ready、start、finish、fail、cancel、worker、attempt 和 parent。每个 worker 记录执行时间、空闲时间、steal 尝试和成功次数。用这些数据可以看到关键路径、长尾、队列积压、窃取是否有效和取消是否传播。Trace 也能用纯文本表示, 避免 EPUB 里图片解析问题。

例如一条记录可以写成: task 42, stage parse, parent split 3, ready 后等待 2 ms, 在 worker 5 运行 8 ms, 唤醒 aggregate 17。若许多任务 ready 后等待长, 说明 worker 不足或队列策略差; 若 running 长尾明显, 说明输入倾斜或任务太粗; 若 steal 成功很多但吞吐不升, 说明任务太小或局部性损失。运行时成熟度来自这种解释能力。

实验: 估算关键路径再对照实际 trace 给一个小任务图: 8 个 parse, 4 个 aggregate, 2 个 merge, 1 个 commit。每个节点有估计耗时和依赖。读者先计算总工作量和关键路径, 再用不同 worker 数运行模拟。线程数增加只能降低可并行部分, 不能突破关键路径。若实际耗时远高于关键路径, 说明有调度、同步、I/O 或负载倾斜成本。

随后改图: 把线性 merge 改成树形, 把过大 parse 拆分, 把过小任务合并, 比较关键路径和调度成本变化。这个实验能防止读者把性能问题简化成“多开线程”。任务运行时优化的是依赖结构、粒度、本地性和等待语义的组合。

事故复盘: future 等待为什么能饿死线程池 一个 worker 执行父任务时同步等待 future, 子任务却排在同一个线程池里。若所有 worker 都进入这种等待, 队列里有 ready 子任务也无人执行, 系统看起来没有死锁锁环, 却实际停住。这个事故说明任务运行时不能只是一条全局 FIFO 队列, 它必须把依赖、ready、running、waiting 和 completion 写成状态机。

任务图先记录依赖计数。依赖满足后任务进入 ready; worker 从本地 deque 取任务, 空闲时从

别人队尾 steal; 任务等待子任务时, 不应占住计算 worker 原地睡眠, 而应帮助执行其他 ready 任务或把 continuation 挂回运行时。Work stealing 的目标不是制造更多调度动作, 而是保持 worker 有活干, 同时尽量保留本地性。

Trace 要能回答四个问题: 哪个任务在等依赖, 哪个任务 ready 后排队多久, 哪个 worker 空闲, 哪次 steal 让任务跨 NUMA 或 cache 边界。Steal 次数很多但吞吐不升, 可能任务粒度太小; worker 空闲但 ready 队列不空, 可能调度结构有 bug; 等待链很长, 可能应改成 continuation。运行时章节的重点是让“等待”显式化, 避免它藏在线程调用栈里。

任务运行时实验判读 运行时 trace 要能回答四个问题: 哪些任务等待依赖, 哪些任务 ready 后排队, 哪些 worker 空闲, 哪些任务被偷。Work stealing 后 steal 很多但吞吐不升, 可能任务太小或破坏局部性; steal 很少但长尾明显, 可能任务不可拆。future 饥饿反例必须保留, 用来证明 continuation 或 helping 的必要性。

这一段也服务于复习。读者合上书后, 应该能凭本章的判读口径重新审查自己的代码: 有没有最小正确版本, 有没有能击穿旧方案的输入, 有没有状态记录, 有没有资源指标, 有没有失败后的恢复动作。若五个问题缺两个以上, 就说明实现还停在演示层, 而不是工程层。

关键路径报告比线程数更能解释加速上限 任务图运行后, 应计算两件事: 总工作量和关键路径。总工作量说明理论上需要多少 CPU 时间, 关键路径说明即使无限 worker 也不能低于多少时间。若实际时间接近关键路径, 再加线程收益有限; 若实际时间远高于关键路径, 就要看队列等待、任务粒度、窃取失败或同步开销。

工作窃取的判读也要放进关键路径。若长尾任务已经开始运行且不可拆, 其他 worker 再怎么偷也帮不上它; 若长尾来自一批未开始子任务, stealing 才能改善。这个区别能防止读者把 work stealing 当成万能负载均衡。

Trace 输出可以用文本, 不必画图片。每行 task id、parent、ready time、start time、finish time、worker、steal source、successors。这样的文本在 EPUB 中稳定, 也能被脚本分析。运行时章节要教读者看 trace, 而不是只看线程池接口。

实验课: future 饥饿到 continuation 的改造 构造两个 worker 的线程池。任务 A 和任务 B 都在运行时提交子任务, 然后同步等待 future。两个 worker 被 A 和 B 占住, 子任务在队列中没有机会执行。这个反例应作为运行时测试长期保留, 因为它精准说明“等待占住执行资源”会导致饥饿。

改造一是禁止 worker 内同步等待, 要求任务注册 continuation。前驱完成时, 后继 ready count 减一, 减到零才进入 ready 队列。改造二是 work helping, worker 等待时可以执行其他 ready 任务, 但要记录重入和异常传播。教学项目可以先实现 continuation, 因为它把依赖显式写入任务图。

工作窃取实验从带锁 deque 开始。Owner 本地 push pop, 空闲 worker 从别人队列偷旧任务。记录本地 pop、steal 尝试、steal 成功、被偷任务耗时和 worker 空闲。若 steal 很多但吞吐不升, 可能任务太小; 若 steal 很少但长尾明显, 可能任务不可拆或 victim 策略差。不要在策略没验证前写复杂无锁 deque。

最后计算关键路径。给每个 task 记录 ready、start、finish, 用文本 trace 估算最长依赖链。线程数再多也不能突破关键路径, 读者要用这个事实解释扩展曲线。

任务运行时练习验收重点 合格答案要给任务图状态, 而不是只给线程池代码。ready count、successor、worker id、steal record、cancel state 都要有解释。future 饥饿题必须说明为什么等待占住 worker, 并给 continuation 或 helping 的语义差异。

答案还要写“为什么不采用更复杂方案”。例如没有证据证明队列是瓶颈, 就不应把锁队列换成无锁; 没有证明输出顺序可变, 就不应随意重排; 没有证明错误路径能恢复, 就不应只提交正常路径。能解释不做什么, 说明读者开始具备工程判断。

最后, 答案必须包含可复查材料。至少有输入摘要、关键状态、期望输出、实际输出、运行命令和未验证边界。没有这些材料, 老师或同伴无法判断结论是否成立。第二册的作业不是让读者写漂亮段落, 而是训练读者把系统问题变成可验证证据。

项目实现: 任务运行时与工作窃取 本章对应的贯穿项目实现不要求一次写成完整框架, 但必须有最小可运行切片。这个切片包含三个部分: 一个 reference, 一个带本章机制的实现, 一个能击穿 reference 之外隐含假设的测试。Reference 用来固定语义, 机制实现用来展示本章知识, 测试用来证明新增复杂度确实解决了问题。

代码组织上，不要把所有内容放进一个大文件。基础类型、输入生成、核心机制、报告输出和测试应分开。即使当前只是教学项目，也要让目录能表达责任边界。这样后续章节接入时，不会因为找不到状态归属而重新发明一套对象。

每个实现都要输出一段机器可读摘要。摘要至少包含输入规模、关键状态、错误数、输出校验、耗时和未验证边界。这里的机器可读不要求复杂格式，简单的 key value 文本也可以。关键是让脚本和读者都能复查，而不是只能看屏幕截图。

测试应包含正常路径、边界路径和失败路径。正常路径证明功能可用；边界路径证明长度、容量、尾部、空输入、关闭或超时没有遗漏；失败路径证明本章机制不是装饰。若失败路径写不出来，往往说明本章概念还没有落到工程对象上。

报告最后写一个取舍段。新增机制带来了什么好处，增加了什么成本，哪些场景不应该使用它，下一章会复用哪个字段或对象。这个取舍段非常重要，因为系统工程不是把所有高级机制都加进去，而是在证据和约束下选择合适的边界。

调试笔记：运行时慢先看 ready 队列和关键路径 任务运行时慢时，先看有多少任务 ready 但未运行，有多少 worker 空闲，有多少任务 waiting 依赖。若 ready 很多而 worker 空闲，是调度或唤醒问题；若 ready 很少，是依赖关键路径问题；若 steal 频繁但执行时间短，是任务粒度过小；若某个 running 任务特别长，是不可拆长尾。

复查清单：任务运行时与工作窃取 复查本章时，先检查 reference 是否仍然存在。没有 reference，后面的并发、优化、异步或分布式改造都无法证明语义没有变。再检查压力输入是否尖锐：它应当攻击本章最核心的边界，而不是只把数据量放大。第三，检查状态是否可观察：关键对象的身份、状态、错误和提交结果应能在报告中找到。

第四，检查代码示例是否说明禁止行为。调用者不能做什么，往往比能做什么更能体现工程质量。第五，检查实验是否写出未验证边界；当前设备、权限或输入无法证明的结论，必须标注。第六，检查本章对象是否能被下一章复用，不能复用就说明边界还不稳。

完成这六项后，再看文字是否连贯。每个概念都应从问题推出，每个案例都应从朴素方案走到改进方案，每个实验都应能解释一个判断。若某段话放到其他章节也成立，就把它改成本章的具体对象、具体输入和具体失败。

14.8 综合案例：任务运行时从等待问题中长出来

线程池不是把函数丢给几个线程那么简单。真正的任务运行时要回答：任务什么时候 ready，谁拥有结果，worker 能不能等待另一个任务，取消如何传播，关闭时已经入队的任务怎么办，异常在哪里重新抛出。若这些问题不先定义，工作窃取、future 和任务图都会变成难以调试的隐式控制流。

一、从固定线程池的三个状态开始 最小线程池也至少有三个状态：Open、Draining、Stopped。Open 允许提交新任务；Draining 拒绝新提交但执行已经接纳的任务；Stopped 表示 worker 已退出，所有等待者都被唤醒。很多教学代码只有一个 bool closed，导致关闭和排空混在一起。生产者不知道 submit 失败是因为系统正在停止，还是队列满；等待者也不知道未完成任务会继续执行还是被取消。

状态机和接口一起写。submit 在 Open 下成功，返回 task id 或 future；在 Draining 和 Stopped 下失败。shutdown drain 把状态从 Open 转到 Draining，等待队列清空和运行中任务结束；shutdown cancel 把未开始任务标记 Cancelled，并唤醒等待者。状态不写清，条件变量谓词和异常传播都无从审查。

二、任务结果必须独立于 worker 栈 Future 的核心不是语法，而是 shared state。任务函数运行在 worker 栈上，调用者等待在另一个线程里，结果、异常、取消状态和完成通知必须放在一个共享状态对象中。这个对象要有明确生命周期：submit 成功后创建，任务完成或取消后进入 ready，最后一个 future 或 runtime 引用释放后销毁。

共享状态通常包含状态枚举、结果存储、异常指针、等待队列或条件变量。若结果直接放在任务 lambda 捕获里，worker 退出后调用者可能拿不到；若异常只打印日志，调用者无法知道任务失败；若取消只改一个标志但不唤醒等待者，系统会悬挂。任务运行时的第一个工程质量门槛，就是 future shared state 能完整表达 Completed、Failed、Cancelled。

三、worker 内等待 future 会制造饥饿 一个常见死锁是：线程池有两个 worker，任务 A 等待任务 B，任务 B 还在队列里；如果两个 worker 都在等待尚未运行的任务，ready 任务就没有线程

执行。更隐蔽的情况是任务图里大量任务在 worker 内阻塞 I/O 或等待其他 future，线程池看起来有很多线程，实际 ready 队列无人消费。

解决方向不是简单增加线程，而是改变等待模型。任务图运行时应把依赖显式化：当前驱完成后，后继任务才进入 ready 队列。Worker 不应该长期等待未 ready 的 future，而应该把 continuation 注册到 shared state，自己去执行其他 ready 任务。若确实需要阻塞等待，运行时要记录 blocking worker，并可能临时补充线程或把阻塞任务隔离到专用池。

四、任务图把算法依赖变成调度事实 第 13 章的 stable filter 有 count、scan、write 三阶段。朴素线程池可能让 count 任务内部等待所有 count 完成，再启动 scan；更好的运行时把依赖写成图：每个 count block 完成后减少 scan 的前驱计数；所有 count 完成后 scan ready；scan 完成后所有 write block ready。这样 worker 不需要占着线程等待，ready 队列只保存真正可运行的任务。

Listing 14.6 任务节点只记录调度必须知道的状态

```
1 enum class TaskState : std::int32_t {
2     Pending,
3     Ready,
4     Running,
5     Completed,
6     Failed,
7     Cancelled,
8 };
9
10 struct TaskNode {
11     std::uint64_t task_id = 0;
12     std::atomic<std::uint32_t> remaining_predecessors{0};
13     std::atomic<TaskState> state{TaskState::Pending};
14 };
```

这个结构不是完整实现，但它展示了核心：依赖计数决定 ready，状态决定生命周期。任务的业务数据、函数对象和诊断信息可以放在别处，调度热路径只需要少量字段。这样也能让 trace 清楚记录每个任务从 Pending 到 Ready、Running、Completed 的时间。

五、工作窃取解决的是局部性和长尾，不是所有调度问题 全局队列线程池简单，但所有 worker 竞争同一队列，任务提交和取任务都可能形成锁热点。工作窃取让每个 worker 有本地 deque，本地生成的子任务留在本地，其他 worker 空闲时从尾部窃取。这样既减少全局争用，也保留一定缓存局部性。它适合递归分治、任务图展开和粒度不均的计算。

工作窃取也有边界。任务如果频繁阻塞 I/O，本地 deque 并不能解决等待；任务如果共享大状态，窃取可能破坏 NUMA 本地性；任务如果太小，deque 操作和调度开销会超过计算；任务如果有严格优先级，随机窃取可能不合适。因此本章要把工作窃取放在运行时工具箱里，而不是把它讲成“高级线程池”的默认答案。

六、Chase-Lev deque 的直觉边界 经典工作窃取 deque 常让 owner 从一端 push 和 pop，stealer 从另一端 steal。单 owner 简化了本地操作，多 stealer 需要 CAS 竞争同一个 top。理解这个结构时，读者不必一开始掌握每个内存序细节，但要看懂所有权：本地 worker 是唯一频繁写 bottom 的线程，窃取者通过 top 抢占任务。最后一个元素是竞争点，需要特殊处理。

这正好连接第 11 和第 12 章。Deque 的索引是原子状态，任务对象的生命周期要保证被窃取后不会被 owner 同时销毁，状态转换要有线性化点。若读者还说不清发布关系和回收边界，就不应直接实现复杂无锁 deque。项目第一版可以用 mutex deque 加 trace，第二版再替换为工作窃取协议。

七、取消不是删除任务对象 取消要分阶段。Pending 任务可以从队列中跳过并标记 Cancelled；Ready 但尚未运行的任务可以在取出时发现取消；Running 任务通常只能协作式检查 cancel token；Completed 或 Failed 任务不能再取消，只能让等待者观察结果。若取消实现成“把任务从容器里 erase”，future 可能还持有 shared state，worker 可能已经拿到指针，生命周期会非常危险。

取消还要传播。若一个前驱失败，后继任务是否取消，取决于任务图语义。有些任务可以在部

分输入失败时继续处理，有些必须全部取消。运行时不应替业务随意决定，而应提供状态传播机制和错误聚合接口。Compute Systems Engine 中，map task 失败可能取消对应 reduce 输入，但不一定取消整个 job；这要由 driver 的策略决定。

八、运行时 trace 是调试工具也是设计约束 任务运行时没有 trace，很快会变成黑盒。最小 trace 应记录 task id、stage id、worker id、状态变化时间、入队时间、开始时间、结束时间、窃取次数、失败原因和取消来源。记录这些字段会带来开销，因此热路径可以使用环形缓冲、采样或分级开关；但完全没有 trace，长尾、死锁和饥饿几乎无法解释。

Trace 还会约束接口设计。若 task 没有稳定 id，无法关联日志；若状态没有枚举，无法统计 Pending 或 Running 时间；若 worker 没有编号，无法观察负载倾斜；若任务图依赖没有显式记录，无法知道谁在等待谁。观测不是最后加的调试输出，而是运行时语义的一部分。

九、实验从错误线程池开始 本章实验可以故意实现一个错误线程池：worker 允许在任务内部等待同池 future。构造一个只有两个 worker 的输入，让两个任务分别等待尚未运行的后继任务，观察系统悬挂。然后改成任务图 continuation 模型，前驱完成后后继才入队。这个实验比直接给正确代码更能说明为什么运行时需要显式依赖。

第二组实验比较全局队列和本地 deque。输入包含均匀小任务、倾斜任务和递归生成任务。报告记录吞吐、队列锁等待、窃取次数、最长任务、worker 空闲时间和 cache miss 替代指标。若没有 perf 权限，也要保留 trace。结论应写成“在哪种任务形状下哪种调度更合适”，而不是简单宣布工作窃取更快。

14.9 案例深化：任务图如何承接并行算法

第 13 章把 filter 拆成 count、scan、write。第 14 章要说明运行时如何承接这种依赖。如果运行时只提供 submit 一个函数，算法只能用 barrier 或在任务里等待；如果运行时支持任务图，就能把依赖写成 ready 条件，worker 不再被等待占住。

一、barrier 简单但会制造阶段停顿 阶段性 barrier 的写法是：提交所有 count，等待全部结束；提交 scan，等待结束；提交所有 write。它简单可靠，适合第一版。缺点是阶段之间没有重叠，某个慢 count 会让所有 write 都等待；worker 在 barrier 处可能空转或阻塞。

任务图把依赖更细粒度地表达出来。某些 partition 的 count 完成后，可以先做局部 scan 或准备 metadata；某些 write 只依赖对应 offset 就可以开始。是否值得这样细化，取决于任务粒度和依赖结构。运行时要提供能力，但算法选择仍要测量。

二、Continuation 避免 worker 阻塞 Future 的 then 或 continuation 思想是：前驱完成后，把后继放入 ready 队列，而不是让某个 worker 等待。这样等待变成状态关系，不占用执行线程。实现上，shared state 保存后继列表；完成时减少后继 remaining predecessors；计数归零则入队。

这个模型也便于错误传播。前驱失败时，后继可以取消、跳过或走补偿路径。策略由任务图定义，不由 worker 临时决定。Compute Systems Engine 的 map failure 可以取消对应 reduce 输入，或触发重试后再让 reduce ready。

三、工作窃取和任务图的结合 任务图决定哪些任务 ready，工作窃取决定 ready 任务由谁执行。本地生成的后继放入当前 worker deque，可以保留局部性；全局 ready 或跨 partition 任务可以被窃取。若任务绑定数据分片，窃取时要记录是否跨 NUMA 或跨数据 owner。调度不是只追求公平，还要考虑数据位置。

四、运行时实验 实现三种调度：阶段 barrier、任务图全局队列、任务图加本地 deque。使用 parallel filter 和树形 reduce 作为 workload。报告记录 worker 空闲时间、ready 队列长度、窃取次数、关键路径长度和端到端时间。若任务很粗，三者差别可能小；若任务不均，任务图收益更明显。实验结果要按任务形状解释。

14.10 补强案例：关键路径决定任务图上限

任务图的最大加速不只由任务数量决定，还由关键路径决定。关键路径是依赖图中必须串行经过的最长路径。即使有一万个任务，如果它们依赖链很长，可并行度也有限。运行时再聪明，也不能突破依赖本身。

一、工作量和跨度 并行算法常用 work 和 span 分析。Work 是总工作量，span 是无限处理器

下的最短时间。可并行度约等于 work 除以 span 。任务运行时的目标是接近这个上限，同时控制调度开销。若 span 很长，加线程无效；若任务太碎，调度开销吞掉收益。

二、从 trace 估计关键路径 任务图 trace 可以记录每个任务开始、结束和前驱。离线分析可估计关键路径长度。若端到端时间接近关键路径，说明调度已经不错，优化应减少依赖或加速关键任务；若端到端远大于关键路径，说明调度、等待或资源争用有问题。

三、关键路径优化 优化关键路径可能包括提前启动 scan、减少串行提交、把大任务拆分、对长尾 partition 特殊处理、让 manifest commit 批量化。优化非关键任务可能对端到端没有效果。报告要说明改动是否在关键路径上。

四、实验 构造三种任务图：完全并行、长链依赖、宽图加一个长尾任务。比较同一线程池下的吞吐和尾延迟。用 trace 标出关键路径。这个实验会让读者理解，任务运行时不是凭任务数自动加速，依赖结构才是上限。

14.11 运行时案例：从一个 future 死锁推导任务图接口

任务运行时最好的入门事故，是两个 worker 都在等待同一个线程池里的后继任务。程序没有数据竞争，也没有崩溃，却永远不前进。这个事故说明，线程池如果只提供 submit 和 future，很容易让等待占住执行资源。任务图接口正是从这个问题里长出来的。

一、死锁时间线 线程池有两个 worker。任务 A 运行后提交 A1，然后等待 A1 的 future。任务 B 运行后提交 B1，然后等待 B1。此时两个 worker 都阻塞，A1 和 B1 在 ready 队列中无人执行。增加队列容量没有用，future 本身没有错，问题是等待发生在 worker 内部，耗尽了执行实体。

这个事故在实际系统里会更隐蔽。不是所有 worker 都永久等待，而是大量 worker 等待 I/O、等待远端响应、等待子任务，导致 ready 任务延迟很高。CPU 使用率低，队列里有任务，开发者可能误以为线程数不够；实际上是运行时没有区分可运行任务和等待依赖。

二、任务图接口的最低能力 任务图接口至少需要三种操作：创建任务节点，声明前驱依赖，当前驱完成后让后继 ready。任务不应该在 worker 中阻塞等待前驱，而是把依赖交给运行时。Shared state 保存结果和后继列表；任务完成时更新结果，减少后继的 remaining predecessor；计数归零后后继入队。

这个接口还要表达失败传播。前驱失败时，后继是取消、继续使用部分结果，还是进入补偿任务？这不能由运行时猜。任务节点应有失败策略或由上层 driver 决定。Compute Systems Engine 中，map attempt 失败会触发重试，而不是让 reduce 立即失败；但超过重试预算后，reduce 应取消。

三、Continuation 和栈安全 任务完成后直接调用后继函数，看起来省队列开销，但可能造成深递归、锁内调用用户代码或破坏调度公平。更安全的方式是把后继变成 ready task，放回队列。若运行时做 inline continuation，也要有深度限制和锁边界。教材要让读者知道 continuation 是控制流，不是普通回调。

完成任务时通常持有 shared state 锁或原子状态。不要在这个锁内执行后继。应先收集需要唤醒或入队的任务，释放锁，再入队或通知。这个原则和锁章一致：持锁更新状态，锁外执行未知代码。

四、任务粒度和调度开销 任务图越细，依赖越清楚，负载均衡越好，但调度开销越高。每个 task 需要状态、队列操作、trace、可能的原子递减。若单个任务只做几十纳秒计算，运行时开销会主导。若任务太粗，长尾无法平衡。粒度要通过实验选择。

并行 filter 可以按块作为任务。块太小，count、scan、write 的任务数量巨大；块太大，某个热点块拖住阶段。报告应记录每个块耗时分布和调度开销占比。任务运行时不是替代算法切分，它要求算法提供合适粒度。

五、工作窃取的公平性和局部性 工作窃取让空闲 worker 从别人队列尾部拿任务。它改善负载均衡，但不保证严格公平。某些任务可能在本地 deque 中排很久；某些 worker 频繁被窃取导致局部性下降。任务优先级、截止时间和数据位置如果重要，就要在窃取策略里表达。

项目可以先实现无优先级窃取，再记录问题。若 reduce 长尾任务需要优先处理，或者 I/O completion 任务应尽快提交 manifest，单纯 LIFO/FIFO 策略可能不够。运行时设计总是在简单性、局部性、公平性和优先级之间取舍。

六、运行时状态和内存回收 任务对象什么时候销毁？Future 可能仍持有结果，trace 可能仍

引用 task id，后继列表可能还没处理。任务完成不等于 task node 可以立即释放。可以用引用计数 shared state，也可以由运行时在所有引用释放后回收。若使用无锁队列，还要考虑节点回收协议。

把这任务运行时和原子生命周期再次连接起来。一个任务图系统里，shared state、queue node、continuation list、trace record 都有不同生命周期。把它们混在一个对象里，容易要么泄漏，要么悬垂。工程实现应分清热调度节点和冷诊断记录。

七、实验：从 future 等待改成 continuation 实现同一个递归分治求和。版本一在 worker 内等待子任务 future；版本二使用任务图 continuation；版本三加入工作窃取。输入包括平衡任务和倾斜任务。报告记录死锁是否出现、worker 阻塞时间、ready 队列长度、任务数量、调度开销和端到端时间。读者会看到，运行时接口设计直接决定程序是否能前进。

14.12 补充案例：运行时关闭不是简单停止 worker

任务运行时关闭比线程池关闭更复杂，因为存在 ready 任务、running 任务、pending 依赖、future 等待者、continuation 和 trace。关闭语义必须定义清楚，否则测试结束、错误恢复和进程退出都会出问题。

一、Drain shutdown Drain 表示不接受新任务，已经 ready 或 running 的任务继续执行，pending 任务如果前驱完成也可以运行。所有任务进入终态后，worker 退出，future 得到结果。这个语义适合正常作业结束。它需要运行时知道还有多少未终态任务，而不只是队列是否为空。

二、Cancel shutdown Cancel 表示未开始任务取消，running 任务协作式检查 token，pending 后继根据策略取消。Future 等待者收到 Cancelled。Cancel 不应直接销毁 running task 的对象，因为 worker 可能还在使用。取消是状态转换，不是内存释放。

三、关闭期间 submit 关闭开始后 submit 应返回明确错误。若 submit 和 shutdown 并发，谁先获得状态转换谁决定结果。这个线性化点要写清。否则调用者会看到偶发任务丢失或关闭后仍执行。

四、实验 提交一批带依赖任务，同时调用 drain；再提交一批长任务，同时调用 cancel。检查所有 future 都进入终态，没有等待者悬挂，worker 全部 join。运行时关闭测试应成为项目必测项。

14.13 补充专题：Work stealing 的观测字段

工作窃取如果没有观测，很难判断是否有效。偷得多不一定好，可能说明负载均衡好，也可能说明本地性差；偷得少也不一定坏，可能任务分配本来均匀。需要结合空闲时间和任务耗时解释。

一、基础字段 每个 worker 记录 local pop 次数、local push 次数、steal attempts、successful steals、failed steals、idle time、executed tasks、stolen tasks、average task time。每个任务记录产生者 worker 和执行者 worker。这样可以判断任务是否跨 worker 迁移。

二、NUMA 字段 若有拓扑信息，记录 steal 是否跨 NUMA node。跨节点窃取可以解决空闲，但可能增加远端访问。报告要比较同节点窃取和跨节点窃取比例。没有 NUMA 设备时，字段可以标记 unavailable。

三、解释方式 高 failed steals 加高 idle time，说明任务不足或依赖限制；高 successful steals 加低 idle time，可能说明负载均衡有效；高跨节点 steal 加 cache miss 上升，可能说明局部性损失。指标要组合解释。

四、实验 构造均匀任务、单长尾任务和递归生成任务。观察三类指标。让读者写出调度解释，而不是只说工作窃取快或慢。

14.14 补充专题：任务运行时的错误聚合

任务图中可能多个任务失败。运行时不能只保留第一个错误，也不能让错误日志散在 worker 中。错误聚合决定调用者看到什么，也决定后续任务如何取消。

一、错误分类 任务错误可以分为输入错误、计算错误、I/O 错误、取消、超时和内部 bug。不同错误影响不同范围。输入某一行坏了，可能记录错误继续；I/O 写失败，可能重试 attempt；内部 bug 可能终止 job。

二、错误传播 前驱失败后，后继任务是否取消取决于策略。Reduce 依赖所有 map block，某个 map 永久失败后 reduce 应取消；监控任务失败可能不影响主计算。任务图要允许上层定义传播规则。

三、错误聚合对象 JobErrorSummary 记录错误类型、task id、attempt id、第一处错误、错误数量和是否可重试。等待 job 完成的调用者读取这个摘要，而不是扫描 worker 日志。

四、实验 让多个 map task 同时失败，检查 driver 聚合错误并取消依赖 reduce。再让一个可重试错误成功恢复，确认最终 job 不报告失败。错误聚合是运行时可靠性的核心部分。

14.15 运行时实现细节：从全局队列到本地队列

任务运行时可以分阶段实现。第一版全局队列，第二版本地队列，第三版工作窃取。每一版都要保留同一任务状态机和同一测试。这样读者能看到复杂度为什么增加，也能在出现 bug 时回退到简单版本。

一、全局队列第一版 全局队列只有一个 mutex 和条件变量，所有 worker 从同一队列取任务。它容易实现，关闭语义清楚，适合作为 reference。缺点是高并发下队列锁成为热点，任务本地性差，递归生成任务时所有 worker 竞争同一个结构。第一版不要急着优化，要先把 submit、shutdown、future、异常传播和 trace 写正确。

全局队列测试覆盖：多个生产者提交任务，worker 正常执行；shutdown drain 后不接收新任务但完成旧任务；shutdown cancel 后未运行任务取消；任务抛异常后 future 能观察；队列为空时 worker 能睡眠并被唤醒。这个测试矩阵后续所有版本都复用。

二、本地队列第二版 第二版给每个 worker 一个本地队列。外部提交可以进入全局注入队列，worker 生成的子任务进入自己的本地队列。Worker 优先本地取，空了再全局取。即使暂时不做窃取，本地队列也能减少部分争用，并保留递归任务的局部性。

本地队列引入新问题：外部线程如何选择目标 worker，关闭时如何收集所有队列，某个 worker 本地队列很多而其他 worker 空闲怎么办。没有窃取时，负载不均可能更严重。因此第二版报告应比较全局队列和本地队列在均匀任务、递归任务和倾斜任务下的表现。

三、工作窃取第三版 工作窃取让空闲 worker 从其他 worker 的队列拿任务。Owner 本地操作应尽量便宜，stealer 操作可以稍贵。先用带锁 deque 实现窃取语义，再考虑无锁 Chase Lev deque。这样可以把调度策略和无锁细节分开。很多项目失败，是因为一开始同时实现复杂调度和复杂无锁容器，调试范围过大。

窃取策略可以先简单随机，再加入拓扑意识。随机窃取容易实现；同 NUMA 节点优先能改善局部性；按队列长度选择能减少空偷，但需要读取更多共享状态。每种策略都要通过 trace 评价，而不是凭直觉。

四、Trace 贯穿三版 三版运行时都输出同一 trace 字段：task id、worker id、enqueue time、start time、end time、source queue、是否 stolen、前驱数量、状态。这样比较时，数据结构变化不会改变观测口径。若第三版变快，可以看成是锁等待减少、空闲时间减少，还是任务局部性改善。若变慢，也能看到窃取失败、跨节点迁移或调度开销。

五、实现顺序 推荐实现顺序是：全局队列 reference，任务 shared state，异常传播，drain/cancel shutdown，trace；然后本地队列；最后工作窃取。不要先写无锁 deque。一个稳定的运行时是逐步长出来的，不是从最复杂版本开始。

14.16 从阻塞线程推导任务运行时

任务运行时的概念不应从复杂框架开始，而应从一个线程池死锁开始：两个 worker 都在任务内部等待另一个还没运行的任务，线程被占住，队列里有工作却没人执行。这个例子说明，线程池只提供执行资源，不自动理解任务依赖。任务运行时的目标是把依赖、就绪、等待、取消和异常从线程调用栈提升到显式状态图上，让系统在等待时仍能调度其他 ready 工作。

一、任务图把执行顺序变成数据 任务节点表示可执行工作，边表示依赖。一个节点只有当前驱都完成后才能进入 ready 队列。这个结构让运行时能看到哪些任务可执行、哪些任务等待输入、哪些任务因失败应取消。若依赖藏在 future 的阻塞等待里，运行时只能看到线程睡住，看不到图的真

实状态。教学中应让读者先画 map、partition、reduce 的任务图，再写调度器；这样 ready count、continuation 和取消传播都会自然出现。

二、work stealing 解决不平衡但改变局部性 每个 worker 有本地 deque，本地生成的新任务优先在本地执行，空闲 worker 从其他队列窃取较老任务。这种策略同时服务两个目标：本地递归任务保持 cache 热度，空闲线程能找到工作。可是窃取不是免费，它需要同步，可能破坏 NUMA 局部性，也可能让数据跨节点移动。报告要记录窃取次数、远端窃取比例、队列长度和尾部任务，而不是只看总耗时。

三、等待应表达成 continuation 任务 A 需要任务 B 的结果时，朴素写法是在 A 中阻塞等待 B。更好的写法是把 A 的后半段注册成 B 完成后的 continuation，释放当前 worker 去执行其他任务。这样等待不再占用线程。这个改变会影响接口：任务结果要有状态对象，异常要能传播，取消要能标记依赖，调试日志要能串起原始任务和 continuation。continuation 看起来比直接 wait 复杂，但它把等待变成可调度状态。

四、取消和异常是运行时语义的一部分 一个任务失败后，依赖它的任务是否继续？已经 ready 但未执行的任务是否跳过？运行中的任务如何协作检查取消？异常是否只返回给提交者，还是传播到整个 stage？这些问题不能放到最后补丁式处理。任务状态表至少要区分 Pending、Ready、Running、Succeeded、Failed、Cancelled。状态转换必须有单一责任位置，否则会出现任务重复完成、异常丢失或取消后仍写输出。

五、运行时实验要暴露调度缺陷 测试任务运行时不能只跑均匀任务。要构造深依赖链、宽图、少量超长任务、任务内部错误、取消、future 饥饿和 NUMA 不均。指标包括 makespan、关键路径估计、worker 空闲时间、窃取次数、队列长度、取消传播时间和异常记录完整性。这样读者才能理解运行时的价值：它不是比手写线程更酷，而是在复杂依赖下保持执行资源可用、状态可解释、失败可恢复。

14.17 任务运行时的最小实现剖面

为了避免运行时章节停留在概念层，本章应给出一个最小实现剖面。它不需要复制工业框架，但要包含几个不可省的对象：Task、DependencyCounter、ReadyQueue、Worker、Scheduler、Completion 和 CancellationSource。每个对象都解决一个明确问题。Task 保存函数、状态和后继；DependencyCounter 决定何时 ready；ReadyQueue 保存可运行任务；Worker 消费任务；Scheduler 负责提交、唤醒和关闭；Completion 保存结果或异常；CancellationSource 把失败传播到相关任务。对象边界清楚，读者才知道复杂性从哪里来。

一、提交任务时先建立图再入队 提交一批有依赖的任务时，不能边建图边让 worker 运行依赖未完成的节点。稳妥做法是先创建所有节点和边，初始化每个节点的未完成前驱计数，再把计数为零的节点放入 ready 队列。若任务运行中动态生成子任务，也要保证父子关系和取消关系先写入状态，再发布子任务。任务图的构建顺序就是一种发布协议，和原子章节相连。

二、ready 队列要区分本地和全局 一个全局队列实现简单，但所有 worker 都争同一把锁或同一组原子位置。每 worker 本地队列能减少争用，并提高局部性；全局队列可以保存外部提交任务或公平性任务；work stealing 在本地队列空时发生。最小实现可以先用锁保护 deque，等语义稳定后再考虑无锁 deque。教材应明确：先正确实现任务状态和关闭，再优化队列结构。否则读者会把最难的无锁队列放在最不稳定的阶段。

三、完成任务时只做必要工作 任务执行结束后，需要保存结果或异常，状态转为 Succeeded 或 Failed，递减后继依赖计数，把变 ready 的后继入队，并处理取消传播。这里最容易出现重复完成和异常丢失。完成路径应有单一函数负责状态转移，不能让 worker、future 和取消逻辑各自修改任务状态。若后继很多，完成路径还可能成为热点，需要批量入队或分层唤醒。先写清语义，再谈优化。

四、关闭运行时要分阶段 关闭不是简单停止 worker。第一阶段停止接收新任务，第二阶段让已提交任务完成或取消，第三阶段唤醒所有 worker，第四阶段 join 线程，第五阶段检查未完成任务和异常。若用户请求立即停止，可以把未开始任务标记 Cancelled，运行中任务协作检查取消。若析构时仍有 future 等待，必须定义等待者看到什么错误。运行时关闭协议写不好，后面 I/O 和分布式章节都会出问题。

五、实现报告要包含状态转移图 最小运行时的交付物不是只有代码。报告要包含任务状态转

移图、队列结构、关闭流程、异常传播规则、取消规则和关键测试。测试包括单任务、链式依赖、菱形依赖、宽图、任务抛异常、取消、关闭、future 饥饿反例和压力运行。只要这套最小实现讲清楚，读者以后看到 TBB、OpenMP task、std::execution 或 Linux workqueue，也能抓住对象和状态，而不是被 API 名称淹没。

14.18 贯穿案例：两个 worker 为什么会把 ready 任务饿死

任务运行时最容易被讲成 API 列表：submit、future、steal、continuation。更好的入口是一个很小的死锁。线程池只有两个 worker。任务 A 在 worker 0 上运行，它提交子任务 A1 后同步等待 A1 的 future；任务 B 在 worker 1 上运行，它提交子任务 B1 后同步等待 B1 的 future。A1 和 B1 已经在 ready 队列里，但两个 worker 都被父任务的等待占住，没有线程执行 ready 任务。系统没有崩溃，CPU 可能还在忙等，但作业不前进。

这个事故逼出任务图。问题不在于 worker 太少，也不在于 future 这个词不好，而在于“等待”被藏在调用栈里，占用了执行资源。运行时必须把依赖写成状态：A 的后半段依赖 A1，B 的后半段依赖 B1；A1 完成后，A 的 continuation 才能 ready。这样 worker 不需要阻塞在父任务里，可以去执行 ready 子任务。Continuation 不是高级技巧，而是把等待从线程栈移到调度器状态表。

一、先把同步等待改写成依赖边 任务 A 不应在 worker 内调用阻塞 wait，而应创建两个节点：A prepare 和 A finish。A prepare 产生 A1，并把 A finish 注册为 A1 的后继。A1 完成时递减 A finish 的依赖计数，计数为零后入队。这个改写让调度器看到等待关系，而不是让等待隐藏在用户函数里。读者一旦看到 ready 队列有任务但 worker 被 wait 占住，就能理解为什么任务图是事实中心。

二、future 结果对象要保存状态而不占线程 Future 不是线程句柄，而是结果状态。它保存 pending、ready、failed、cancelled，保存值或异常，保存等待者或 continuation。等待者可以是外部线程，也可以是任务图中的后继节点。若实现把 future 等同于阻塞等待，运行时就回到死锁事故。结果对象和 worker 执行资源必须分离，这是任务运行时的核心边界。

三、work stealing 只能解决负载不均，不能解决隐藏等待 工作窃取能让空闲 worker 从别人队列里拿任务，但如果所有 worker 都阻塞等待，没人去 steal。先把等待表达成 continuation，再讨论 steal。Steal 的任务是平衡 ready 任务分布，不是修复状态机设计。这个顺序很重要：运行时语义不清时，上来做无锁 deque 和复杂 stealing，只会让 bug 更难复现。

四、异常和取消也要沿依赖边传播 A1 失败时，A finish 不能悄悄等待永远，也不能在缺少输入时继续写输出。状态机要规定：A1 的 fatal 异常让 A finish 进入 Cancelled 或 Failed；可恢复错误可以创建 retry 节点；用户取消则把未 ready 的后继标记为 Cancelled，运行中任务协作检查取消。异常传播不是 try catch 的局部问题，而是任务图上的状态传播。

五、trace 要能复盘 ready 任务为何没运行 事故报告应能列出：A start、A wait hidden、A1 ready、B start、B wait hidden、B1 ready、worker 0 blocked、worker 1 blocked。修复后 trace 应变成：A prepare finish、A1 ready、worker 0 执行 B prepare、worker 1 执行 A1、A finish ready。没有 trace，只看最终超时，很难分清是任务太少、锁等待、队列 bug 还是隐藏 wait。运行时章节要把 trace 作为设计对象，而不是调试附属品。

六、从任务图走向后续 I/O 和分布式 异步 I/O completion、RPC reply、分布式 task attempt 都和 continuation 是同一类问题：事件尚未到来时，不要占住执行资源；事件到来后，把后续动作重新放进状态机。第 14 章如果把等待讲透，第 15 章的 completion 和第 16 章的 late reply 就会自然衔接。全书的连贯性来自这些问题结构，而不是术语重复。

14.19 案例深化：任务图调度的可复盘设计

任务运行时的目标不是把函数丢给线程，而是让依赖、就绪、等待和完成都可复盘。考虑一个解析作业：读取任务产生多个解析任务，解析任务产生局部聚合任务，聚合完成后触发写出任务。若父任务同步等待子任务，而所有 worker 都在等待，ready 子任务无人执行。这个事故逼出任务图和 continuation，而不是简单增加线程数。

一、任务节点保存依赖计数 每个任务节点记录未完成依赖数、后继任务列表、状态和所属 worker。依赖数降到零时进入 ready 队列。这样任务何时可以运行由状态决定，不由某个父任务阻塞等待决定。状态包括 created、ready、running、waiting、completed、cancelled。运行时 trace 应

记录每次状态变化。

二、等待要变成 continuation 父任务需要子结果时，不应占住 worker sleep。它可以把后续逻辑拆成 continuation，等子任务完成后重新入队；或者在等待期间帮助执行其他 ready 任务。这样 worker 不会因为等待 future 而浪费。这个设计和异步 I/O 的 completion、分布式 RPC 的 reply event 是同一类问题：等待事件到来后继续，而不是占住执行资源。

三、Work stealing 的代价也要记录 偷取能提高利用率，但可能破坏 locality。被偷任务访问的数据可能在原 worker cache 中，跨 NUMA 偷取还可能远端访问。Trace 要记录 steal source、target、task size 和执行时间。若 steal 很多但吞吐不升，可能任务太小；若几乎不 steal 但 worker 空闲，可能队列分配不均。Work stealing 不是魔法，它是本地性和负载均衡的取舍。

四、取消和异常必须沿图传播 一个任务失败后，依赖它的后继任务应取消、跳过或进入降级路径。异常不能只停在 worker 线程里，否则 Driver 不知道作业状态。任务图需要失败传播规则：fatal 错误取消整个 stage，局部可恢复错误触发 retry，用户取消进入 cancelling。这个状态机会在分布式章节扩展成 task attempt 和 driver epoch。

五、运行时实验要看事件而不只看吞吐 实验输出应包含 ready 队列长度、worker 空闲时间、steal 次数、任务等待时间、最长依赖链和取消传播时间。吞吐下降时，读者能判断是任务太细、依赖链太长、偷取太多还是阻塞等待。任务运行时的教学重点是可解释调度，而不是实现一个最短线程池。

14.20 案例复盘：future 等待怎样变成 continuation

父任务在 worker 中同步等待子任务，子任务却排在同一池里；所有 worker 都这样等待时，ready 队列有任务也无人执行。任务运行时要把依赖、ready、running、waiting、completed 写成状态机，而不是让等待藏在调用栈里。

任务图是事实中心 任务节点保存依赖计数和后继列表。依赖归零进入 ready；需要等待时，把后续逻辑变成 continuation，等事件到来再入队。Worker 空闲时可以 steal，但要记录偷取来源和 NUMA 代价。Steal 多不一定好，可能说明任务太细或本地性被破坏。

实验判据 Trace 记录 task submit、ready、start、wait、resume、finish、steal。指标包括 ready 队列长度、worker idle、最长依赖链、steal 次数和等待时间。取消和异常要沿任务图传播，不能停在 worker 线程里。

14.21 本章习题

习题与作业

习题一：为一个最小线程池写状态机。状态至少包括 Open、ShuttingDown、Stopped。列出每个状态下 submit、wait、shutdown 和 worker loop 的行为。

习题二：设计线程池指标结构，要求避免所有 worker 高频写同一个 cache line。说明哪些指标需要精确，哪些可以近似或延迟汇总。

习题三：把并行归约的最后合并从线性合并改成树形合并。画出任务图，计算关键路径长度如何变化，说明任务数增加带来的调度成本。

习题四：设计一个工作窃取实验。先用 mutex deque 实现策略，不使用无锁 deque。比较全局队列和本地队列在任务粒度不同、负载不均不同情况下的 worker 空闲时间和窃取次数。

I/O、异步与背压

先看一个写出队列事故：解析和计算阶段很快，写磁盘偶尔变慢。无界输出队列一开始掩盖了问题，几分钟后内存持续上涨，延迟越来越大，最终进程被杀。把队列改成异步写以后，又出现新 bug：submit 返回成功后，业务线程复用了 buffer，但内核尚未完成写入，落盘内容偶尔损坏。这里逼出的概念不是“异步更高级”，而是等待、完成、所有权、背压和提交点必须分开。

本章从这个 I/O 事故进入异步和背压。计算系统不只做 CPU 运算。磁盘、网络、管道、日志和用户请求都会引入 I/O。I/O 的延迟和吞吐特征与 CPU 完全不同，因此运行时必须区分计算等待和 I/O 等待；也必须让上游知道下游已经处理不过来，而不是用无限队列把问题推迟到内存崩溃。

15.1 从阻塞 I/O 到异步流水线

同步 I/O 调用会让当前线程等待结果。少量线程处理少量 I/O 时简单可靠；大量并发连接或慢设备场景下，同步等待会浪费线程。异步 I/O 把提交和完成分离，让线程可以处理其他工作。

事件循环、回调、future、协程和 `io_uring` 都是组织异步 I/O 的方式。选择哪种方式，要看系统需要的并发数、延迟要求、代码复杂度和平台支持。

异步 I/O 的核心不是“换一种语法”，而是把等待从工作线程上移走。同步调用让当前线程停在系统调用或库调用里；异步模型把请求提交出去，完成时再处理结果。这样可以用较少线程管理大量等待。但异步也会引入状态机、取消、超时、错误传播和资源生命周期问题。若并发度很低，同步 I/O 可能更简单可靠。

15.2 背压、队列和批处理

背压是系统告诉上游“我处理不过来”的机制。没有背压，队列会无限增长，内存耗尽，延迟失控。背压可以通过 bounded queue、令牌桶、限流、拒绝请求、降级和批处理实现。

并发系统里 bounded queue 的意义不只是节省内存，更是把过载变成可观察、可控制的状态。阻塞、返回错误或丢弃，都比默默无限排队更可管理。

背压必须一路传到真正的生产者。只在中间队列限长，但上游仍然无限接收请求，过载会换一个地方堆积。一个服务端如果 worker 队列满了，可以选择拒绝新请求、降低读取速度、返回重试、丢弃低优先级任务或触发降级。选择哪种方式取决于业务语义，但不能没有选择。

批处理 批处理用一次调度或一次 I/O 处理多个元素，摊薄固定成本。网络发送、日志写入、数据库提交和向量计算都常用批处理。批太小浪费固定成本，批太大增加延迟和缓存压力。

批处理要控制两个阈值：数量阈值和时间阈值。只按数量凑批，在低流量时可能让请求等太久；只按时间刷批，在高流量时可能批太小。很多系统使用“达到 N 条或等待 T 时间就发送”的策略。批处理还要考虑失败语义：一批里部分成功时如何重试，是否会重复执行，是否需要幂等 ID。

CPU 与 I/O 的边界 I/O 密集任务不应占满计算线程池。常见设计是分离 I/O 线程、计算线程和后台维护线程，或者使用异步运行时统一调度但明确资源限制。否则慢 I/O 会阻塞计算任务，计算任务也会延迟 I/O 完成处理。

队列是系统的压力表 队列长度、入队速率、出队速率和等待时间，是理解系统压力的基本指标。队列持续增长说明下游处理能力不足或上游没有背压；队列经常为空说明下游可能在等上游；队列长度锯齿状变化可能是批处理或周期性调度造成的。只看 CPU 使用率可能误判，因为系统可能 CPU 很空但卡在 I/O，也可能 CPU 很满但队列仍然增长。

有界队列实验要记录三种结果：生产者是否阻塞，消费者是否空闲，丢弃或拒绝是否发生。这样才能判断背压策略是否真的控制住过载。

I/O 和 CPU 的成本模型不同 CPU 计算的基本问题是怎样让核心持续退休有用指令，I/O 的基本问题是怎样管理等待、排队和设备吞吐。一次磁盘读取、网络请求或管道写入可能比一条 CPU 指令慢很多数量级。若用 CPU-bound 的思维处理 I/O，很容易开太多线程、阻塞 worker、隐藏队

列积压，最后系统看起来 CPU 不满但请求延迟很高。

I/O 还有两个重要特征。第一，吞吐和延迟经常需要批处理折中。单条写入延迟低但吞吐差，大批写入吞吐高但单条等待更久。第二，错误和取消是常态。磁盘可能返回短读，网络可能超时，远端可能重置连接，写入可能部分成功。高质量运行时不能只写成功路径。

Compute Systems Engine 后续的分布式 shuffle 会大量使用 I/O 思维。map 端产生中间数据，缓冲、批量写入、按分区发送、等待 reduce 端接收、处理重试和背压。如果单机阶段没有建立 I/O 模型，分布式阶段就会写成“把数据发过去”这种空话。

同步阻塞的适用边界 同步阻塞 I/O 并不是低级。对于简单工具、并发度低的批处理程序、启动脚本或一次性文件读取，同步 I/O 清晰可靠。问题出在大量连接或大量慢设备等待时，每个等待都占用一个线程。线程有栈、调度状态、上下文切换成本和内存占用；阻塞线程太多，系统会被调度和内存压力拖垮。

因此选择同步还是异步，不是看语法潮流，而是看并发等待数量和生命周期复杂度。若程序同时等待几十个文件读取，同步线程池可能足够；若服务同时维持几十万连接，事件循环或异步 I/O 更合适。若任务既有 CPU 密集计算又有慢 I/O，至少要把计算线程池和 I/O 等待分开，避免慢 I/O 占满计算 worker。

事件循环的核心 事件循环把“等待多个事件”集中到少量线程中。它通常维护一组已注册的 fd、定时器和任务队列，等待内核通知哪些事件 ready，然后执行对应回调。Linux 上常见机制包括 `epoll` 和 `io_uring`。事件循环不等于更快的 CPU，它只是避免每个等待都占用一个线程。

事件循环的难点是回调不能长时间占用 loop 线程。若在事件循环线程里做 CPU 密集解析、压缩或排序，其他 I/O 完成事件会被延迟处理。常见架构是事件循环负责接收和发送，CPU 工作提交到计算线程池，完成后再把结果交回事件循环。这样系统同时需要 I/O 队列、计算队列和背压策略。

io_uring 的直觉 `io_uring` 的核心直觉是提交队列和完成队列。用户态把 I/O 请求放入提交队列，内核处理后把结果放入完成队列。它减少某些系统调用和数据结构转换成本，也支持更丰富的异步操作。但它不是魔法：设备仍有真实延迟和带宽，队列仍可能满，请求仍可能失败，完成处理仍需要 CPU。

学习 `io_uring` 时，不要一开始追求所有高级特性。先理解三件事：提交多少 in-flight 请求，完成事件如何被消费，失败和取消如何传播。过多 in-flight 请求会增加队列延迟和内存占用；完成处理太慢会让完成队列积压；取消语义不清会导致请求完成后访问已释放对象。

15.3 超时、取消和错误传播

异步系统必须处理超时和取消。一个请求提交出去后，上层可能已经超时返回，用户对象可能准备释放，但底层 I/O 仍可能稍后完成。若完成回调访问已销毁对象，就会出现 use-after-free。正确设计通常需要请求上下文对象、引用计数或所有权转移，确保完成路径和取消路径共享同一生命周期协议。

超时也不是简单地“过了时间就删掉”。如果底层操作不能立即取消，超时只表示上层不再等待，底层完成事件仍要被回收和丢弃。若请求已经发送到远端，远端可能仍执行，重试可能造成重复提交。因此超时、取消和幂等会在分布式章节重新出现。单机异步 I/O 是理解分布式失败语义的前置训练。

背压的几种策略 背压不是只有阻塞。常见策略包括阻塞等待、立即失败、带超时等待、丢弃低优先级任务、降级处理、批量合并、采样、令牌桶限流和动态降低读取速度。每种策略都有业务语义。日志采样可以丢一部分低优先级日志；支付请求不能随便丢；监控指标可以合并；用户请求可能返回 429 或重试建议。

有界队列是背压的最小机制。队列满时，生产者必须面对现实：等、失败、丢弃或降级。无限队列只是把这个选择推迟到 OOM 或尾延迟爆炸。第二册里的 bounded queue 不只是并发练习，它是流控思想的基础。

Listing 15.1 带超时的提交语义草图

```
1 enum class SubmitResult : std::int32_t {
2   Accepted,
3   TimedOut,
```

```

4  Closed,
5  };
6
7  struct SubmitPolicy {
8      std::int32_t timeout_milliseconds = 0;
9      bool allow_drop = false;
10 };

```

这个片段没有实现队列，而是表达接口语义。调用者需要知道任务是否被接受、是否超时、队列是否关闭。若接口只返回 `void`，调用者无法区分提交成功和任务丢失。系统工程里，返回值设计就是背压协议的一部分。

批处理的延迟预算 批处理必须有延迟预算。假设日志写入器每满 4096 条刷一次磁盘，如果流量很低，最后几条日志可能等很久。因此通常设置两个阈值：数量达到 `N` 或时间达到 `T` 就刷。`N` 控制吞吐，`T` 控制尾延迟。选择 `N` 和 `T` 要看业务：监控指标可以延迟几秒，交互请求响应不能等太久。

批处理还要考虑内存和缓存。批太大，可能超过缓存，处理时局部性下降；批太小，系统调用和队列成本占比高。网络批处理还要考虑 MTU、拥塞控制和远端处理能力。数据库提交还要考虑事务语义和 `fsync` 成本。没有一个“最佳批大小”，只有工作负载下的实验矩阵。

异步流水线 一个典型异步流水线可以分成读取、解析、计算、写出四段。读取是 I/O 密集，解析可能 CPU 密集，计算可能需要线程池，写出又是 I/O 密集。段与段之间用有界队列连接。每个队列的长度、入队速率和出队速率告诉我们瓶颈在哪。

```

read stage -> parse queue -> compute workers -> write queue -> output stage
I/O bounded CPU bounded I/O

```

若 `parse queue` 持续增长，说明解析或计算跟不上读取；若 `write queue` 持续增长，说明输出慢；若所有队列为空而 CPU 很空，说明上游读取慢；若 CPU 满且队列也增长，说明计算是瓶颈。队列指标让系统不再是黑箱。

优先级和公平性 真实系统通常有不同优先级任务。高优先级请求、后台压缩、指标上报、日志写入不应完全同等对待。单个 FIFO 队列可能让大量低优先级任务阻塞高优先级请求。可以使用多队列、优先级队列、配额、公平调度或令牌桶隔离。

优先级也会带来饥饿风险。若高优先级持续不断，低优先级任务可能永远不执行。公平性策略要说明最低保障、老化机制或后台窗口。系统设计经常是在延迟、吞吐、公平和实现复杂度之间取舍。

I/O 错误和重试 I/O 错误必须分类。临时错误可以重试，例如网络超时、EAGAIN、远端暂时不可用；永久错误不应盲目重试，例如权限错误、格式错误、目标不存在。重试要有预算、退避和幂等保障。无限立即重试会放大故障，形成重试风暴。

写入操作尤其要小心部分成功。一次批量写可能只写了一部分；一次远程请求可能已经被对方执行但响应丢失；一次文件写入可能进入 `page cache` 但还没持久化。系统必须定义提交点：什么时候认为任务完成，什么时候可以重试，重复执行是否安全。分布式章节会把这套语义扩展到幂等 ID 和检查点。

15.4 Linux 观察、可靠写入和项目落地

Linux I/O 观察可以从几个层次开始。系统调用层可用 `strace` 观察 `read/write/epoll/io_uring` 相关调用，但它会带来明显开销。性能层可以用 `perf` 看 CPU 热点和调度，使用 `iostat`、`pidstat` 或 `ss` 观察设备和网络状态。内核源码阅读可以从 VFS、`page cache`、`block layer`、`epoll` 或 `io_uring` 子系统选择一个窄问题进入。

不要写“Linux I/O 很复杂”这种泛泛结论。应围绕现象阅读：为什么 `mmap` 首次访问触发 `page fault`，为什么 `epoll` 能等待多个 `fd`，为什么写文件返回不等于持久化，为什么 `io_uring` 有提交队列和完成队列。读源码要和用户态实验绑定。

I/O 章的回归阅读要把提交和完成拆开。调用 `write` 或提交异步请求，只表示请求进入某个路径，不表示数据已经持久化或外部可见。buffer 在 completion 之前能否复用，短写如何继续，错误如何传播，rename 何时发生，manifest 何时提交，这些问题决定可靠性。

背压要沿管线传播。写出队列满时，parser 和 compute 不能继续无限制造 block；慢磁盘或慢网络不能被隐藏成内存增长；checkpoint 频率不能只看恢复方便，也要看写放大和尾延迟。报告要记录队列水位、in-flight bytes、完成延迟、短写次数和取消次数。没有这些指标，异步化只会把阻塞从线程栈搬到堆内存。

本章项目验收应使用崩溃点矩阵。临时文件写一半、fsync 前崩溃、rename 前崩溃、manifest 写一半、manifest 后重启，每个点都要说明恢复后哪些文件可见、哪些可清理、哪些需要重试。I/O 可靠性不是“最后有文件”，而是每个中间状态都能解释。

Linux I/O 源码阅读可以分层进行。普通文件路径可以从 `fs/read_write.c` 理解系统调用入口，从具体文件系统看写回和同步语义；页缓存相关内容回到 `mm/filemap.c`；异步接口可以阅读 `io_uring` 目录的请求提交和 completion queue。读这些代码时不要追求一次读完，而要带着问题看：请求何时拥有 buffer，完成事件何时出现，错误码从哪里返回，取消是否一定成功。

背压还要进入 API 设计。写出接口不能只返回 bool，而要区分 accepted、would block、closed、failed 和 cancelled；队列容量要按 bytes 和请求数同时限制；上游看到 would block 时要停止生产或降级，而不是忙等。这样 I/O 章节才能和线程池、任务图、分布式流控接起来。

可靠写入要区分临时结果和可见结果。worker 可以把 block 写到临时路径，完成 checksum 和 fsync 后再由 driver 把 manifest 指向它；外部读者只信 manifest，不扫描临时目录。这样崩溃后，孤儿临时文件可以清理，已提交文件可以保留，未提交文件不会被误读。这个模式会在分布式 shuffle 和第三册模型文件加载中复用。

异步完成还要求 buffer 生命周期清晰。提交写请求后，输入 buffer 不能被复用或释放，直到 completion 明确返回；如果使用引用计数或 arena，要保证完成回调持有生命周期；如果请求取消，也要等待取消完成或定义请求仍可能完成。许多 I/O bug 不是系统调用错误，而是上层过早复用内存。

Backpressure 的传播路径要写成闭环。writer 队列达到高水位后，partition 阶段停止生成新 block；partition 停止后，compute 阶段减少提交；compute 阶段停下来后，parser 不再继续读 split；parser 停下来后，输入读取速率下降。若某一层只记录队列满却继续接收上游数据，背压就在这里断裂，内存会继续增长。项目报告可以用四个水位记录这个闭环：parse batch 数、ready task 数、in-flight write bytes、manifest pending 数。水位一起下降，说明背压真正传回去了。

批处理大小也要和 I/O 一起选。block 太小，系统调用、metadata、checksum 和 manifest 项过多；block 太大，单次失败重做成本高，尾延迟变长，内存峰值上升。一个合理实验应扫描 block size，记录吞吐、p99 写入延迟、临时文件数量、重试成本和恢复时间。这样读者会看到 I/O 参数不是配置表里的魔法数字，而是吞吐、延迟和恢复粒度的折中。

错误分类应进入类型系统。短写、权限错误、空间不足、校验失败、取消、超时和关闭不是同一种失败；有的可以重试，有的必须停止，有的需要人工处理。若所有错误都变成字符串，driver 就无法正确决定重试和提交。I/O 章节的 CompletionRecord 应至少包含错误类别、可重试标志、已写字节、attempt 和资源路径，让上层能做结构化决策。

15.5 I/O 从完成事件和所有权边界讲起

阻塞 I/O 的语义直观：调用开始，线程等待，调用返回，缓冲区可用或错误明确。异步 I/O 改变的是控制流和所有权：提交只表示请求进入系统，不表示完成；缓冲区在完成前不能复用；错误可能稍后到达；取消不一定能阻止底层操作完成。若不先写完成事件，异步章节会变成 API 名词表。

日志项目里，reduce 端写 shuffle block 或 checkpoint manifest 就是 I/O 主线。第一版可以同步写文件，语义简单但会阻塞 worker；第二版把写请求提交给 I/O 线程或事件循环，计算线程继续工作；第三版加入有界队列和背压，防止写出速度跟不上计算速度时内存无限增长。每一步都改变等待位置和错误传播方式。

阻塞、非阻塞和异步的区别

阻塞 I/O 占住调用线程，适合简单程序和低并发路径。非阻塞 I/O 调用立即返回 `EAGAIN` 之类状态，需要事件机制告诉你何时可读可写。异步 I/O 提交请求，完成队列稍后返回结果。三者不是谁更高级，而是控制流和并发模型不同。

事件循环的核心是 ready 事件，不是完成所有工作。**epoll** 告诉你 fd 可能可读可写，真正读写仍要处理短读、短写、**EAGAIN** 和关闭。**io_uring** 更接近提交/完成队列模型，但仍要处理 buffer 生命周期、取消、错误码和内核版本差异。教材要避免把具体 API 神化，重点放在完成事件和状态机。

同步阻塞仍然有适用边界。配置文件读取、低频管理操作、简单命令行工具，用同步 I/O 更清楚。只有当 I/O 等待占据关键路径、并发度需要提高或线程资源有限时，异步化才值得复杂性。项目中先保留同步 reference，再引入异步写出，是为了让复杂性有证据支撑。

背压、水位线和批处理

背压回答一个问题：下游处理不过来时，上游怎么办。无界队列把问题藏起来，最终表现为内存膨胀、尾延迟上升或 OOM。有界队列把容量变成可调参数，队列满时上游阻塞、失败、丢弃低优先级数据或降级。对于必须提交的 shuffle block，静默丢弃不可接受；对于采样指标，可以丢弃或合并。

水位线比单一容量更细。低水位恢复生产，高水位限制提交，紧急水位触发降级或拒绝。网络和磁盘都有类似缓冲。队列长度不是越大越好：大队列提高吸收突发能力，也增加排队延迟和内存占用。实验要扫描容量和批大小，观察吞吐、p99 延迟、内存峰值和提交错误。

批处理把固定成本摊薄。每条记录一次系统调用很慢，把多条记录合成 block 可以减少 syscall、元数据和 fsync 成本。但批太大，会增加等待时间、cache 压力和失败重做成本。Group commit 用一次持久化提交多个结果，提升吞吐，但扩大了崩溃恢复需要检查的窗口。批大小必须由延迟预算和恢复边界共同决定。

可靠写入、checkpoint 和崩溃点

可靠写入不是 **write** 返回成功就结束。文件系统缓存、rename、fsync、目录 fsync、临时文件和 manifest 都会影响崩溃后一致性。常见模式是写临时文件，flush 数据，原子 rename，再 fsync 目录，最后发布 manifest。不同文件系统语义有差异，教材要说明这是工程模型，不是所有平台绝对保证。

Checkpoint 要设计崩溃点矩阵。崩溃可能发生在写临时文件前、写一半、fsync 前、rename 后、manifest 更新前、manifest 更新后。恢复程序必须能识别哪些文件可见、哪些是垃圾、哪些需要重做。若直接覆盖 checkpoint，崩溃可能留下半新半旧状态。这个问题和分布式提交高度相似：提交点必须明确，清理不是提交路径。

错误分类也很重要。短写、磁盘满、权限错误、临时 **EAGAIN**、取消、校验失败、超时，都需要不同处理。无限重试会放大故障；不重试会降低可用性；重试必须有预算和幂等提交。I/O 章为分布式 RPC 的 deadline、retry 和 idempotency 做铺垫。

项目中的 I/O 管线

项目交付物是一个有界异步写出管线。Map 或 reduce 产生 **OutputBlock**，提交到 I/O queue；I/O worker 写临时文件并返回 completion；driver 根据 completion 更新 manifest；队列满时向上游传递背压。每个 block 有 id、attempt、字节数和 checksum。

实验包括：同步写 baseline、无界队列反例、有界队列版本、不同 batch size、注入短写和磁盘满、模拟慢设备。指标包括提交延迟、完成延迟、队列水位、内存峰值、重试次数和成功提交数。若在手机环境无法安全制造磁盘满，使用模拟 I/O 层返回错误即可，重点是状态机和恢复。

Linux 阅读入口可以选择 **epoll** ready 语义、**io_uring** SQ/CQ 概念、page cache 和 writeback。不要试图一次讲完 I/O 子系统；从用户态现象映射到内核对象即可。本章最终判断力是：任何异步 I/O 设计，都先问请求完成前 buffer 归谁、完成后谁处理错误、队列满时谁等待、崩溃时哪个状态算提交。

案例走查：无界输出缓存怎样拖垮系统

Reduce 产生 block 的速度高于磁盘写出速度。第一版把 block 放进无界 vector，后台线程慢慢写。短时间 benchmark 可能很好，因为计算线程从不等待；长时间运行内存持续增长，最终 OOM 或触发系统回收，尾延迟暴涨。这个版本必须作为反例保留。

有界队列版本让 submit 在容量满时阻塞或返回 backpressure。吞吐可能略降，但内存峰值受控，系统能稳定运行。再加入 batch size 扫描，找到吞吐和延迟的折中点。实验要画队列水位随时间变化，而不只给总耗时。水位图能直观看到下游是否长期落后。

错误注入也要加入：模拟写失败、短写、慢写和取消。完成事件返回后，driver 才能提交 manifest；

失败事件必须让上游知道 block 未提交。异步 I/O 的核心就在这里：提交请求不是完成，完成才改变业务状态。

可靠写入的文本状态图

为了兼容 EPUB，本章不用图片也能画可靠写入状态图。可以用文本列出：Created temp、Write data、Flush data、Rename、Fsync dir、Publish manifest、Cleanup。每个状态旁边写崩溃后恢复动作。比如 Write data 前崩溃，临时文件不存在；Write data 后 Rename 前崩溃，临时文件可删除；Rename 后 Publish 前崩溃，恢复程序检查 checksum 后决定是否补 manifest 或删除孤儿文件。

这个文本状态图比流程图更适合严谨说明。每一行都是一个崩溃点，读者可以写测试模拟。测试通过方式不是看程序没崩，而是重启恢复后 manifest 和输出目录满足不变量：manifest 引用的文件都存在且 checksum 正确，未引用临时文件可清理，最终结果不重复。

手机环境做崩溃测试时，可以用模拟文件系统层，不一定真的 kill 进程。模拟层在每个状态点返回错误或中断，让恢复函数处理。重点是协议，而不是破坏设备文件系统。

异步 I/O 和线程池的边界

很多项目用“另一个线程做阻塞 I/O”模拟异步。这是合理教学版，但要写清边界：它消耗线程，完成事件由队列传回，取消只能协作，底层阻塞调用可能无法中断。真正的非阻塞或 `io_uring` 模型有不同系统调用语义，但状态机相似：submit、in-flight、complete、fail、cancel。

因此项目先实现 I/O worker 版本，重点练背压和 completion；后续可选阅读 `io_uring`。如果一开始就讲复杂 API，读者容易忽视真正重要的 buffer 生命周期和完成语义。工程教材要选择能承载原理的最小实现。

作业要求给 `WriteRequest` 定义字段：block id、attempt、path、bytes、checksum、deadline、callback 或 completion queue id。再说明 request 提交后 buffer 是否可复用。若答不出所有权，本章就没学透。

综合练习：写出 checkpoint 崩溃点测试

本题要求读者为 checkpoint 写崩溃点测试。状态步骤包括生成内容、写临时文件、flush、rename、fsync 目录、更新 manifest、清理旧文件。测试框架在每一步后模拟崩溃，重启恢复函数，检查 manifest 和文件集合是否满足不变量。不要只测试正常写入。

每个崩溃点都要写期望。写临时文件前崩溃，恢复后没有新 checkpoint；写一半崩溃，临时文件被删除；rename 后 manifest 前崩溃，文件存在但未发布，恢复可校验后补提交或清理，具体策略要写清；manifest 后崩溃，恢复应看到新 checkpoint 已提交。不同策略都可以，但必须一致。

再加入背压。若 checkpoint 写入慢，上游是否继续产生无限状态？有界队列满时，driver 是等待、跳过 checkpoint 还是降级？不同选择影响恢复时间和内存峰值。报告要写选择理由。

这个题训练读者把 I/O 可靠性变成测试，而不是把 fsync、rename 当成背诵点。真实系统里，崩溃点矩阵比漂亮流程图更有价值。

容易出错的边界：异步化不等于可靠化

第一个反例是提交异步写后立即复用 buffer，完成事件到来时数据已经被覆盖。第二个反例是只处理成功 completion，错误 completion 只打印日志，driver 仍然提交 manifest。第三个反例是取消请求后假设底层一定没有写出，实际完成可能迟到。第四个反例是无界队列让 benchmark 短期变快，长期内存爆炸。

第五个反例是 fsync 位置不清。数据写入 page cache 不代表崩溃后可恢复；rename 原子不代表目录项已经持久；manifest 发布前后的崩溃动作不同。第六个反例是重试不可幂等。写同一个输出路径时，重复 attempt 可能覆盖已提交结果。I/O 可靠性和分布式幂等提交是同一条线。

本章源码索引要求为每个 I/O request 写生命周期：buffer 创建、提交、in-flight、完成、提交业务状态、释放。每个状态允许哪些操作，错误如何传播，取消如何处理。若 request 只有 path 和 callback，没有 attempt、checksum、deadline 和状态，就不足以支撑可靠系统。

实验中要保留慢设备模拟器。真实磁盘状态不可控，模拟器能稳定触发短写、延迟、错误和队列满。高质量教材不依赖运气触发故障，而是把故障变成可复现输入。

本章核心接口：CompletionRecord

I/O 章给项目留下 `CompletionRecord`。它记录 request id、block id、attempt、status、bytes requested、bytes written、checksum、error code、start time、complete time。业务层只根据 completion 改变提交状态，不能在 submit 后提前认为成功。

CompletionRecord 会成为分布式 reply 的本地版本。I/O completion 可能迟到、失败或被取消；远端 reply 也可能迟到、失败或属于旧 attempt。用同一套思维处理二者，读者能看到异步和分布式之间的连续性。

从 I/O 完成进入分布式消息

I/O completion 和分布式 reply 有相同形状：请求已经提交，结果稍后回来，可能成功、失败、迟到或取消。I/O 章训练的是本机异步边界，分布式章把同样边界放到网络和远端执行里。若读者已经理解 CompletionRecord，就更容易理解 MessageEnvelope 和 MapTaskReply。

项目中，I/O 写出的 block 会成为分布式 shuffle 的数据面。只有 completion 成功并通过 checksum 后，driver 才把 block 写入 manifest；只有 manifest 可见后，reduce 才能读取。这个提交顺序正是分布式幂等提交的本地前奏。I/O 章不是并行部分的尾声，而是分布式部分的入口。

深入补充：背压传播不能断在中间层

背压最容易失败的地方，是某一层把压力吞掉。I/O 队列满了，writer 返回 false；但 runtime 把 false 当普通失败重试，driver 又立刻提交更多任务，压力没有回到输入端。或者 MessageBus 队列满了，send 阻塞在 worker 线程里，导致计算线程被拖住。真正的背压要沿调用链传播：I/O 慢，reduce 降速；reduce 慢，shuffle 拉取降速；shuffle 慢，map 输出降速；map 输出降速，parser 不再无限产生 batch。

传播方式可以有多种。阻塞传播最简单，但可能占住关键线程；try-submit 返回 false 让上层选择；带 deadline 的 submit 可以避免永久等待；令牌桶或窗口控制可以把压力变成数量预算。选择哪种方式要看任务是否可丢、是否可延迟、是否有外部用户等待。指标数据可以丢，shuffle block 不能丢；交互请求有 deadline，离线作业可以等待更久。

项目中应把背压作为状态，而不是异常。QueueContract 写明 full behavior；CompletionRecord 写明 pending bytes；driver trace 写明 throttled 时间。报告里若看到吞吐下降但内存稳定、错误不增加，可能是背压在保护系统；若吞吐高但内存持续增长，反而是危险信号。这个观念和很多初学者相反：不是所有等待都是坏事，失控的不等待更坏。

深入补充：I/O 和 page cache 的测量陷阱

文件 I/O benchmark 很容易被 page cache 误导。第一次读文件可能慢在磁盘，第二次读同一文件可能快在内存；写文件返回快，可能只是写进 page cache，真正落盘由后台 writeback 完成。若实验不说明是否 drop cache、是否 fsync、是否使用临时文件系统，结果就无法解释。手机环境上，文件系统和存储介质更复杂，温控和后台任务也会影响 I/O。

因此本章的 I/O 实验要区分三类目标。研究应用层背压时，可以使用模拟 I/O，稳定控制延迟和错误。研究 page cache 行为时，要明确预热、文件大小和系统限制。研究可靠提交时，必须关注 fsync、rename 和崩溃点，而不是吞吐最大化。三类目标不能混在同一张图里。

Page cache 还会影响内存账本。大量写出未落盘的数据会占用系统缓存，进程 RSS 可能不高，但系统内存压力上升。队列水位稳定不代表整体内存没有压力。报告可以记录进程内存、输出目录大小和系统可用内存的粗略变化。即使指标不精确，也比完全忽略 I/O 缓冲更诚实。

深入补充：从单机背压到分布式流控

单机背压和分布式流控是同一个概念在不同尺度上的表现。单机 I/O queue 用容量限制写出请求；分布式 driver 用 per-worker in-flight 限制远端任务；shuffle reader 用 in-flight bytes 限制拉取；客户端用 deadline 限制等待。容量、窗口、令牌、deadline 这些名字不同，本质都是让系统不要接收超过恢复能力的工作。

项目可以把 FlowControlWindow 抽象出来，单机 I/O 和 MessageBus 都使用它。窗口按 task 数和 bytes 计数，完成事件释放预算，超时或失败也释放但记录错误。这个抽象不需要很复杂，却能让读者看到本章和分布式章的连接。若两个模块各自写一套限流，后续行为很难统一。

这个补充也解释为什么背压要在第二册讲清楚。第三册推理引擎中，batch、KV cache、权重加载和请求队列都会遇到同样问题。没有背压，推理服务会在突发流量下内存膨胀；有背压，系统可以选择排队、拒绝、降级或缩小 batch。第二册的 I/O 背压是未来服务化推理的基础。

15.6 项目落地

Compute Systems Engine 在本章应实现一个本机异步模拟，不必立刻依赖真实网络。可以用一个生产者按固定速率生成批次，一个解析阶段消耗 CPU，一个写出阶段按可配置延迟模拟 I/O。阶

段之间使用 bounded queue，记录队列长度、阻塞次数、丢弃次数、批大小、端到端延迟和每阶段吞吐。

然后加入三种背压策略：阻塞生产者、try submit 失败、批量合并。再加入超时和取消：请求超过延迟预算后标记过期，但底层完成事件仍要被消费。这个实验会直接服务分布式 shuffle：网络发送、reduce 端接收、重试和检查点都可以从这里演化出来。

完成事件 异步接口必须告诉调用者何时完成、成功多少字节、错误是什么、请求上下文能否回收。没有 CompletionRecord，异步只是把控制流藏起来。

短读短写 read/write 可能只处理部分字节。可靠写入要循环处理 partial completion，并在错误时返回已处理范围。

持久化点 write 返回不等于数据落盘。fsync、rename、目录 fsync 和 manifest commit 构成不同持久化边界。检查点系统必须写清哪一步算提交。

背压 有界队列和 in-flight 限制是 I/O 系统的安全阀。下游慢时，上游应减速、拒绝或降级，而不是无限缓存。

批处理 批处理减少固定成本，也增加等待。batch 大小要受延迟预算约束。写出线程可按字节数、条目数或时间窗口 flush。

取消 取消通常是尽力而为。请求可能已经完成、正在内核中、还在队列里或已经返回达到完成。上下文生命周期要覆盖所有路径。

page cache page cache 会让热读看起来很快，也会让写入先变成内存压力。性能报告要区分冷读、热读、同步写和异步回写。

流水线 I/O 阶段应和计算阶段通过有界队列连接。队列水位、in-flight 数、完成延迟和失败原因共同描述系统状态。

案例：短写循环 模拟每次 write 只接受部分字节，要求可靠写出完整 buffer。报告说明偏移如何更新。

案例：checkpoint 崩溃点 在临时文件写入、fsync、rename、manifest 更新之间注入崩溃，重启后分类处理。

案例：背压传播 写出线程变慢时，计算队列逐渐满，解析线程最终减速。记录每级队列水位。

源码阅读路线 Linux 源码阅读从 `fs/read_write.c`、`fs/sync.c`、`io_uring`、`mm/filemap.c` 附近进入即可。阅读目标不是把整套子系统抄进书里，而是追一条和本章实验相关的路径：用户态动作怎样变成内核对象，内核对象怎样排队、等待、唤醒、计数、提交或回收。

阅读报告要写证据等级。能直接测到的数字放在第一层；从源码控制流推导出的解释放在第二层；当前设备无法验证、只能作为合理猜测的内容放在第三层。这样读者会知道 I/O 背压报告中哪些结论可复现，哪些结论需要换机器继续确认。

源码阅读还要克制。看到一个复杂函数，不要把每个分支都展开；先回到本章的问题，问它能否解释一个等待点、一次状态转移或一条性能曲线。解释不了的内容可以留到索引，不要打断正文主线。

短写循环 模拟每次 write 只接受部分字节，要求可靠写出完整 buffer。报告说明偏移如何更新。

checkpoint 崩溃点 在临时文件写入、fsync、rename、manifest 更新之间注入崩溃，重启后分类处理。

背压传播 写出线程变慢时，计算队列逐渐满，解析线程最终减速。记录每级队列水位。

阻塞 I/O

阻塞调用简单，但等待时占住线程。线程池里混入大量阻塞写，会让计算任务没有 worker。

实验设计：把写出放在计算池和独立写出池中比较。

完成事件

异步提交成功只表示请求被接受，真正结果要从 completion 得到。提交路径和完成路径必须拥有同一个请求身份。

实验设计：让请求超时后 completion 迟到，检查资源是否安全回收。

短读短写

read 或 write 返回部分字节不是罕见异常。协议必须能继续处理剩余字节，不能假设一次完成。
实验设计：模拟短写概率，验证输出拼接仍正确。

背压

有界队列让过载可见。队列满时可以阻塞、返回失败、丢弃低优先级或降速，不能无限堆内存。
实验设计：生产速度大于消费速度时比较无界与有界队列。

水位线

高低水位线避免满和不满之间抖动。高水位停止上游，低水位恢复，上游才不会频繁开关。
实验设计：记录队列长度随时间的锯齿形状。

deadline

timeout 是当前操作预算，deadline 是整个请求截止时间。流水线应传递 deadline，避免每阶段都重新获得完整时间。

实验设计：让同一请求经过多个阶段，比较固定 timeout 和全局 deadline。

可靠写入

write 成功不等于持久化。临时文件、fsync、rename、目录 fsync 和 manifest 定义真正提交点。
实验设计：在每个状态注入崩溃，写恢复表。

epoll 语义

epoll 通知 fd ready，不通知业务消息完整。应用仍要处理 framing、短读和 EAGAIN。
实验设计：把 socket 字节流拆成长度前缀消息，验证半包处理。

group commit

多个提交共享一次持久化能提高吞吐，但会增加等待。阈值和时间窗口决定延迟。
实验设计：比较每批 fsync、每 N 批 fsync 和只 write。

项目接口

CompletionRecord 记录请求身份、提交时间、deadline、完成状态、字节数、错误分类和清理动作。

实验设计：第 18 章的 shuffle 和 checkpoint 直接复用它。

15.7 I/O 的本质是等待、完成、背压和提交

I/O 章节不能只比较 blocking、epoll、io_uring 这些接口。接口会变，核心问题稳定：提交请求后谁拥有缓冲，完成事件什么时候到达，上游过快时如何背压，写出结果什么时候算提交，超时后底层完成如何清理。把这些问题讲清，读者换任何 API 都能分析；只讲 API，换平台就失效。

阻塞 I/O 简单，因为调用者在当前线程等待，生命周期跟栈接近。异步 I/O 提高并发度，但把生命周期拉长：请求提交后，缓冲和上下文必须活到 completion；上层超时后，底层完成仍可能到达；取消只是尽力，不等于设备停止。很多 use-after-free 来自把“我不等了”误认为“底层不会再回来”。CompletionRecord 的价值就在这里：记录请求身份、提交、超时、完成和清理。

短读短写必须进入教材。**read** 返回部分字节，**write** 写出一部分，socket 收到半个消息，都是正常情况。应用协议要有 framing，例如长度前缀；写出路径要循环处理剩余字节；错误路径要区分 EAGAIN、EOF、超时和不可恢复错误。若示例代码默认一次读写完整消息，读者会把错误模型学错。

背压是 I/O 系统的安全阀。无界队列让上游继续生产，直到内存耗尽；有界队列让过载显性化。队列满时可以阻塞、返回 busy、丢弃低优先级、降低读取速率或批量合并。选择取决于业务语义。日志调试消息可以丢，checkpoint 提交不能丢。背压必须沿管线传播到真正源头：write queue 满让 reduce 降速，reduce 降速让 shuffle 降速，最终 reader 停止读取。

可靠写入要区分“写到内核”和“崩溃后可恢复”。写临时文件、fsync 文件、rename、fsync 目录、发布 manifest，这些状态共同定义提交。对象存储、NFS 和本地文件系统语义还可能不同。本书项目可以简化，但必须让读者知道 write 返回不等于 durable。第 18 章的 checkpoint 和 manifest 会直接复用这里的提交模型。

deadline 比 timeout 更适合流水线。每个阶段固定 timeout，端到端时间可能无限膨胀；全局 deadline 让每个阶段知道剩余预算。请求进入队列时，如果剩余时间已经不足，可以提前拒绝或降级。分布式系统中 deadline 还要处理时钟差异；本机 I/O 至少要使用单调时钟，避免墙钟调整。

本章实验可以做一个 I/O 模拟器。参数包括生产速率、消费速率、队列容量、batch 字节数、短写概率、错误率、deadline 和重试次数。先让生产快于消费，观察无界队列内存增长；再加入高低水位线，观察背压传播；最后加入超时后迟到 completion，验证资源被回收且业务回调不重复。这个模拟器会成为分布式消息和 shuffle 背压的前置训练。

案例推导：写出阶段如何把背压传回读取阶段

日志统计项目中，writer 阶段最容易被低估。解析和聚合都很快时，输出文件和 manifest 写入可能成为瓶颈。旧实现让 worker 直接写文件；写慢时，worker 被 I/O 占住，计算能力下降。改进版本把输出变成 OutputBlock，提交给写出队列，由专门 writer 处理。这样计算和 I/O 分离，但也引入了背压问题：writer 慢时，write queue 会增长，reduce 不能无限生成块。

背压链应从 write queue 传回 reduce，再传回 shuffle，再传回 map，最后让 reader 减速。若中间任意一段使用无界队列，压力就会在那一段积累，内存升高，尾延迟变差。高低水位线可以避免频繁开关：超过高水位时，上游暂停或减小 batch；低于低水位时恢复。报告应记录队列长度和等待时间，因为长度相同，在不同处理速率下代表的延迟不同。

写出语义也要严肃。OutputBlock 先写临时文件，fsync，rename，fsync 目录，再由 manifest 发布。若上层 deadline 到了，底层写仍可能稍后成功。迟到成功不能直接覆盖最终输出，而应由 driver 判断 attempt 是否仍有效。若另一个 attempt 已经提交，迟到文件只作为孤儿清理。这个逻辑和分布式 task attempt 完全一致，只是先在本机 I/O 中出现。

短写和错误分类也要纳入。磁盘满、权限错误、短写、EAGAIN、校验失败、deadline 超时的处理不同。Retryable 错误可以重试，fatal 错误应失败作业，corrupt 错误需要重新生成或从副本恢复，backpressure 表示保护机制生效。错误分类进入指标，才能让系统在过载时可诊断。

实验可以用模拟 writer 控制消费速率。第一轮无界队列，观察内存增长。第二轮有界队列但无水位，观察 producer 频繁阻塞和唤醒。第三轮高低水位线，观察系统趋于稳定。第四轮注入写出超时后迟到完成，验证业务回调不会重复执行，底层资源仍被清理。最后加入 manifest 崩溃点，验证恢复只接受 committed 输出。

这个案例让 I/O 章自然连接第 18 章。分布式 shuffle 只是把本机 writer 换成网络和远端 reduce；背压、deadline、attempt、manifest、迟到完成和清理孤儿文件，语义完全相通。

第四部分综合案例：并行算法、任务运行时和 I/O 怎样形成一条可恢复流水线

第四部分把算法切分、任务运行时和 I/O 背压合成一条流水线。输入被切成 chunk，map 任务解析并聚合，partition 任务生成 shuffle block，reduce 任务合并，writer 任务提交输出。单看每个算法都不难，难的是它们共享同一批资源：worker、队列、内存、磁盘和 deadline。若没有运行时状态机，算法正确也可能在等待和失败中失控。

并行算法先定义可切分边界。chunk 要记录输入范围、输出范围、partial 身份和重做范围。reduce 需要合并规则，scan 需要输出位置，partition 需要稳定性或不稳定性的语义。算法切分若从一开始就记录这些信息，任务运行时才能调度，I/O 阶段才能提交，失败后才能重做局部范围。若算法只返回一个匿名 vector，后续恢复就没有抓手。

任务运行时负责把这些边界变成执行图。map 完成后触发 partition，partition 完成后触发 reduce，writer 完成后触发 manifest。ready count 防止任务提前运行，continuation 防止 worker 阻塞等待 future，work stealing 缓解负载不均。运行时 trace 记录每个任务 ready、start、finish、worker、窃取和状态。没有 trace，长尾任务和饥饿只能靠猜。

I/O 背压负责让流水线不过载。writer 慢时，write queue 达到高水位，reduce 暂停生成更多 OutputBlock；reduce 队列满后，partition 减速；partition 减速后，map 输出缓冲满；最终 reader 暂停读取。这个链条必须设计出来，不能依赖系统“自然变慢”。无界队列会切断背压，让内存堆积。按 item 和 byte 双重限流能处理少量大 block 与大量小 block 两种情况。

恢复语义贯穿三章。一个 map task 失败，重做对应 chunk；一个 partition block 写了一半，清理临时文件；一个 writer fsync 后响应丢失，manifest 判断是否已提交；一个 reduce 等待的上游失败，任务图取消下游。并行算法提供重做范围，任务运行时提供状态传播，I/O 提供提交边界。三者缺一，系统都只能处理 happy path。

综合实验可以这样安排。输入十个 chunk，其中一个很大、一个含错误行、一个写出时短写、一个 writer 延迟。运行时输出任务 trace，I/O 输出队列水位和 completion，算法输出每个 chunk 的 partial 和 partition 大小。最终报告要能回答：最长尾来自哪里，背压从哪里开始，哪些任务被取消，哪些输出进入 manifest，哪些临时文件被清理。这个实验让读者看到第四部分不是三组知识，而是

一条可恢复流水线。

完成队列和回调线程 异步系统通常有完成队列。I/O 提交后，完成事件进入 completion queue，某个线程负责取出事件并执行后续处理。这个线程不应该做重 CPU 工作，否则完成队列会积压，导致已经完成的 I/O 迟迟不被用户态处理。常见做法是完成线程只解析结果、更新状态，然后把 CPU 工作提交到计算池。

完成事件必须携带请求上下文 ID，而不是直接裸指针访问业务对象。若请求已经超时，上下文可能处于 canceled 状态；完成线程应安全地把结果丢弃或记录，而不是继续执行已过期业务逻辑。这里的状态机通常包括 submitted、completed、timed out、canceled、reaped。reaped 表示底层完成事件已经被消费，资源可以释放。

短读、短写和 partial completion I/O API 经常允许短读和短写。读取文件或 socket 时，返回的字节数可能小于请求字节数；写入也是如此。正确代码必须循环直到满足条件、遇到 EOF、出错或超时。若把一次 read/write 返回当成完整消息，就会在压力和边界情况下出错。

协议通常需要 framing：先读固定长度 header，再根据长度读 body；或者使用分隔符；或者使用固定大小消息。异步实现中，一个逻辑消息可能对应多次底层完成事件。请求状态机要记录已经读写多少字节、剩余多少字节、下一步做什么。I/O 编程的难点经常不在系统调用，而在状态机完整性。

限流器和令牌桶 背压可以由队列容量触发，也可以由限流器主动控制。令牌桶是一种常见模型：系统按固定速率产生 token，请求需要消耗 token 才能发送；桶容量允许短时间突发；长期速率受 token 生成速度限制。它适合控制出站 RPC、日志写入、后台压缩和重试。

令牌桶的参数要有业务含义。生成速率对应可持续吞吐，桶容量对应允许突发，等待策略决定超出速率时阻塞还是失败。若重试也共用同一个令牌桶，就能防止重试风暴；若重试绕过限流，就会在故障时放大流量。

队列容量的选择 队列容量不是越大越好。容量太小，短暂抖动就会阻塞上游，吞吐可能下降；容量太大，过载时延迟积压很深，失败反馈太晚，还可能耗尽内存。容量应根据服务时间、突发大小、延迟预算和内存预算选择。

一个实用方法是先估算：如果消费者每秒处理 10000 个任务，允许吸收 100 ms 突发，则队列容量至少约 1000；如果端到端延迟预算只有 50 ms，队列排队就不能无限增长。测量时要记录队列等待时间，而不只是队列长度。长度相同，处理速率不同，等待时间也不同。

优雅关闭和 drain I/O 系统关闭时要定义 drain 语义。立即关闭会丢弃队列中尚未处理的请求；drain 关闭会拒绝新请求，但继续处理已接受请求；带超时 drain 会在预算内尽力完成，超时后取消或丢弃。不同业务选择不同语义。日志系统可能允许丢低优先级日志，数据库提交不应随便丢。

线程池、bounded queue 和异步 I/O 都需要关闭协议。关闭不是析构时把标志设为 true 就结束，还要唤醒等待者、停止接受新请求、等待 running 请求、消费完成队列、释放资源并报告未完成请求。若完成队列没有 drain，底层稍后完成可能访问已经销毁的上下文。

本章项目验收 Compute Systems Engine 的 I/O 模拟模块应通过这些验收：生产速度大于消费速度时队列不会无限增长；阻塞策略能降低生产速度；try 策略能报告失败；批处理策略能降低固定成本但不超过延迟预算；超时请求的完成事件仍被安全回收；关闭时不丢已接受且必须完成的任务；指标能显示队列长度、等待时间、批大小、超时数和丢弃数。

这些验收看起来像工程细节，但它们决定系统能否在过载和退出时保持可控。很多线上事故不是算法算错，而是队列无界、关闭不完整、重试没有预算、完成事件访问已释放对象。

文件 I/O 的提交语义 文件写入有多层语义。应用把数据复制到用户态缓冲，只说明用户态内存有数据；调用 `write` 成功，通常说明数据进入内核 page cache 或设备队列，不一定持久化；调用 `fsync` 成功，才更接近“崩溃后可恢复”，但还要考虑文件系统、目录项和设备缓存。若用临时文件加 `rename` 提交，还要考虑 `rename` 后 `fsync` 目录，确保目录项持久化。

这对 checkpoint 和 manifest 非常关键。一个作业写完 manifest 后就宣布成功，但 manifest 没有 `fsync`，机器断电后可能丢失。或者输出文件已经 `rename`，但目录项没持久化，恢复时看不到文件。教学项目可以在手机上简化持久化实验，但正文必须说明真实可靠提交需要明确持久化边界。

```

durable output shape:
write temporary output

```

```
fsync temporary output
rename temporary output to final name
fsync parent directory
record manifest commit
```

不同文件系统和存储服务语义不同。对象存储可能没有 POSIX rename，NFS 语义也可能不同。本书不展开存储系统细节，但要求读者不要把“写了文件”误认为“已经安全提交”。

Page cache 对 benchmark 的影响 文件读取 benchmark 很容易测到 page cache。第一次读大文件可能来自磁盘，第二次读同一文件可能来自内存。若报告没有说明冷 cache 还是热 cache，结果无法解释。清 page cache 需要权限，也会影响系统其他进程；普通用户可以通过读取不同文件、增大数据集或明确报告热缓存来处理。

写入 benchmark 也类似。写入 page cache 很快，后台写回稍后发生。若只测 write 返回时间，可能低估真实持久化成本；若测 fsync，成本会集中暴露。日志系统通常用批量 fsync 或 group commit，在吞吐和延迟之间取舍。第 17 和 18 章的 checkpoint 提交，需要继承这里的语义。

epoll 的 ready 语义 epoll 通知的是 fd ready，而不是业务消息完成。Socket 可读表示读一次不会阻塞，但不保证完整应用消息已经到达；可写表示发送缓冲有空间，但不保证整个响应能一次写完。应用仍然要处理短读短写和消息 framing。事件循环只是等待机制，不替你实现协议状态机。

边沿触发和水平触发也有差异。水平触发下，只要状态仍 ready，事件会继续出现；边沿触发下，状态从 not ready 变 ready 时通知一次，应用必须读写到 EAGAIN，否则可能遗漏后续机会。边沿触发可以减少重复通知，但更容易写错。教学阶段建议先理解水平触发，再研究边沿触发。

网络缓冲和 backpressure 网络发送通常先进入用户态缓冲、内核 socket buffer、网卡队列，再到网络。send 成功不表示对端应用已经处理，只表示本端接受了数据。若对端处理慢，接收窗口缩小，本端发送缓冲会填满，最终 send 阻塞或返回 EAGAIN。这是 TCP 层面的背压。

应用层还需要自己的背压。TCP 只能说明字节流压力，不知道业务优先级、请求幂等、队列容量和服务端 CPU。应用可以根据 reduce partition 队列长度返回 busy，可以根据请求 deadline 拒绝，可以按优先级丢弃低价值消息。网络背压和应用背压要配合，而不是互相替代。

零拷贝的生命周期问题 零拷贝听起来总是好事，但它把缓冲区生命周期变长。普通发送可以把数据复制到内核缓冲后复用用户内存；零拷贝发送可能要求用户缓冲在发送完成前保持不变。若应用过早复用或释放缓冲，就会发送错误数据或崩溃。完成通知必须告诉应用何时可以回收缓冲。

这和异步 I/O 的请求上下文相同：提交出去的东西，在完成前不能随便销毁。对象池、引用计数、completion queue 和取消状态机都要配套。没有生命周期协议的零拷贝优化，会把一个性能问题变成正确性问题。

Group commit Group commit 把多个提交合并到一次持久化操作中。数据库日志、消息队列和检查点系统都常用这个思想。单个请求每次 fsync，延迟高且吞吐低；多个请求共享一次 fsync，可以大幅提高吞吐，但每个请求可能等待批次凑齐。数量阈值和时间阈值决定延迟和吞吐。

Group commit 还要处理失败。如果批次 fsync 失败，批内所有请求都失败还是部分失败？如果 fsync 成功但响应发送失败，客户端重试时如何返回旧结果？如果进程在 fsync 后、响应前崩溃，恢复时是否能根据日志判断请求已提交？这些问题直接连接分布式幂等表和提交点。

异步错误传播 同步函数可以通过返回值或异常报告错误，异步系统的错误可能在提交之后很久才出现。提交成功只表示请求被接受，不表示操作成功。调用者需要一个 completion、future、回调或事件来接收最终结果。若接口只返回 accepted，却没有完成路径，错误会丢失。

异步错误还可能发生在取消之后。上层已经超时，底层返回失败或成功，完成路径仍要消费事件并更新状态。状态机应区分 user-visible result 和 internal cleanup。对用户来说请求已超时；对系统来说底层资源还没 reaped。混淆这两者会导致内存泄漏或 use-after-free。

队列的水位线 除了满和空，队列还可以设置高水位线和低水位线。超过高水位线时上游减速或停止；降低到低水位线时恢复。水位线可以避免在满和不满之间频繁抖动。网络协议、日志系统和消息队列都常用类似机制。

水位线选择要结合处理速率和突发。高水位线太接近容量，反馈太晚；太低，吞吐可能受限。低水位线太接近高水位线，系统频繁开关；太低，恢复太慢。实验可以画队列长度随时间变化，观察是

否出现锯齿、振荡或持续增长。

优先级丢弃 有些数据可以丢，有些不能丢。监控采样、调试日志、推荐候选可以在过载时丢弃或降级；支付请求、提交协议和检查点不能随意丢。队列满时，如果只用 FIFO，低价值任务可能占住空间，高价值任务被拒绝。优先级丢弃或多队列隔离可以改善这种情况。

但丢弃必须可观测。系统要记录丢弃数量、原因、优先级和影响范围。静默丢弃会让上层误以为系统正常。对于可丢数据，也要定义最大丢弃比例或降级策略，避免过载时完全失明。

背压传播链 背压必须沿数据流传播到真正的源头。假设 write queue 满了，compute worker 被阻塞；compute worker 阻塞后 parse queue 满；parse queue 满后 read stage 停止读取；read stage 停止读取后 socket receive buffer 满；最终 TCP 窗口缩小，上游发送变慢。这是一条健康的传播链。若中间某处使用无限队列，背压就断了，内存会积压在那里。

分布式 shuffle 也一样。Reduce 队列满，应反馈给 map 端；map 端降低发送或合并更多；如果 map 端仍无限读取输入并缓存 shuffle，内存会爆。背压链条要在设计图里画出来，不能只在某个队列上写容量。

超时和 Deadline 的区别 Timeout 是从当前操作开始计算的一段时间，deadline 是整个请求的截止时间。异步流水线更适合传 deadline，因为请求经过多个阶段，每个阶段都应知道剩余预算。若每个阶段都设置固定 timeout，端到端延迟可能超过用户预算。Deadline 还能帮助队列做决策：已经没有足够时间完成的任务，可以提前拒绝或降级。

Deadline 也要进入重试策略。一个请求剩余 5 ms，却启动一个预期 50 ms 的重试，通常没有意义。重试库应检查剩余预算、幂等性和重试次数。分布式系统中，deadline 还要考虑时钟差异；本机流水线中，单调时钟即可。

I/O 模拟器的参数 本章项目的 I/O 模拟器应有可调参数：生产速率、消费速率、队列容量、批大小、批时间阈值、错误率、短写概率、超时预算、重试次数和是否允许丢弃。通过改变这些参数，读者可以观察队列从稳定到过载的过程。模拟器不是为了假装真实设备，而是为了训练状态机和背压思维。

每次实验要记录参数。否则“队列爆了”没有意义。记录后可以复现：在生产速率两倍于消费速率、队列容量 1024、批大小 64、错误率 1% 时，阻塞策略如何表现，try 策略丢多少，批处理如何影响延迟。

I/O 章节和分布式章节的连接 本章所有概念都会在分布式章节放大。短写对应消息分片，completion queue 对应 RPC 响应，deadline 对应跨服务预算，令牌桶对应重试限流，group commit 对应提交日志，manifest fsync 对应 checkpoint，背压链对应 shuffle 流控。学 I/O 不是为了记住某个系统调用，而是为了理解等待、完成、提交和过载这些通用结构。

端到端流水线案例 以日志统计为例，端到端流水线可以这样设计。Read stage 从文件读取 batch，Parse stage 把文本转成列式热字段，Compute stage 做本地聚合，Shuffle stage 按 partition 发送 partial，Write stage 写 manifest 或最终输出。每两个 stage 之间都有有界队列。每个 batch 都有 batch id、输入 offset、记录数、deadline 和错误计数。

这个流水线里，任何阶段慢都会反映到队列。Parse 慢，read-to-parse 队列增长；Compute 慢，parse-to-compute 队列增长；Shuffle 慢，compute 输出缓冲增长；Write 慢，提交延迟上升。通过这些队列，系统可以定位瓶颈，而不是只看到总时间变长。队列也是背压传播路径，防止 read stage 无限读取文件占用内存。

Listing 15.2 流水线 batch 元数据

```
1 struct PipelineBatch {
2     std::uint64_t batch_id = 0;
3     std::uint64_t begin_offset = 0;
4     std::uint32_t record_count = 0;
5     std::uint32_t error_count = 0;
6 };
```

Batch 元数据应和数据分开。热字段列可能很大，元数据用于调度、指标和恢复。后续分布式版本中，batch id 可以映射到 task id 或 shuffle block id。

可靠写入的状态机 写输出可以分成 preparing、writing、flushing、committing、committed、failed。Preparing 创建临时路径和缓冲；writing 写数据；flushing 执行必要持久化；committing 发布 manifest 或 rename；committed 对外可见；failed 清理临时状态或保留诊断。把这些状态写清，恢复时才知道半成品如何处理。

如果进程在 writing 中崩溃，临时文件可能不完整，应丢弃或重写；在 flushing 后、committing 前崩溃，数据可能持久但未对外宣布，恢复时应根据 manifest 决定是否采用；在 committing 后响应丢失，重试应返回已提交。这个状态机和分布式 task attempt 提交完全一致，只是资源从网络输出变成文件。

读放大和写放大 I/O 优化要关注读放大和写放大。读取一条记录可能需要读入整页、解压整个块或读取索引；写一条记录可能导致写一个 page、一个日志段或一个压缩块。若每条小记录单独写文件，元数据和 fsync 成本会非常高；批量写能降低写放大。若每次查询都读完整原始日志，只为了统计一个字段，读放大很高；列式存储和索引可以降低读放大。

读写放大和 cache line 放大是同一思想在存储层的版本。硬件按 cache line 搬，文件系统按页和块搬，网络按包和缓冲搬。系统设计要让每次搬动包含足够有用信息。

异步管线的内存预算 异步流水线中，每个队列容量乘以 batch 大小，就是内存预算的一部分。若 read queue 容量 128，每个 batch 4 MiB，只这一段就可能占 512 MiB。多个阶段叠加后，内存峰值很快上升。队列容量不能只按任务数量设置，还要按字节设置。

因此实际系统常同时限制 item count 和 byte count。一个 batch 很大时，即使 item count 不高，也可能触发背压。Shuffle 阶段尤其需要按字节限流，因为不同 partition 的 block 大小差别很大。第 18 章的 map in-flight shuffle bytes 就来自这个原则。

完成回调的幂等 异步完成回调可能被重复触发吗？理想 API 不应重复，但上层重试、取消和状态恢复可能让同一逻辑请求出现多个完成路径。回调内部应先检查请求状态，只有从 submitted 转到 completed 的那一次执行用户可见副作用。后到的完成、取消或超时只做清理或记录重复。

这可以用 CAS 状态转换表达。状态从 Submitted 到 Completed 成功的线程负责发布结果；若状态已经 TimedOut，完成线程只回收底层资源；若状态已经 Canceled，同样不执行业务回调。这个小状态机和分布式 exactly-once 提交同构。

I/O 失败的可恢复性 不是所有 I/O 失败都可恢复。临时 EAGAIN、超时、连接重置可以重试；磁盘满、权限错误、校验失败、格式错误可能需要停止或人工处理。系统应把错误分为 retryable、fatal、corrupt、backpressure。Corrupt 错误尤其重要：如果读取 block 校验失败，盲目重试同一损坏文件可能无用，需要重新生成或从副本恢复。

错误分类进入 metrics。Retryable 错误升高可能说明下游过载；fatal 错误说明配置或权限；corrupt 错误说明存储或写入协议；backpressure 说明容量保护生效。没有分类，错误数只是噪声。

本章附加实验设计 附加实验一：模拟 writer 每批写入后是否 fsync。比较只 write、每批 fsync、每 N 批 group fsync 的吞吐和延迟，并说明持久化语义差异。附加实验二：模拟队列按 item count 限制和按 byte count 限制，构造少量大 batch 和大量小 batch，观察哪种策略能控制内存。附加实验三：模拟 completion 在 timeout 后到达，验证资源能被 reaped 且业务回调不会重复执行。

这些实验会让 I/O 章和分布式章真正连接起来。可靠计算系统的难点不是一次成功写入，而是在超时、失败、重复和关闭时仍保持状态机正确。

案例：限速读取 当下游处理慢时，读取阶段不应继续把文件读入内存。限速读取可以根据 parse queue 或 compute queue 的水位线决定是否暂停。若队列超过高水位线，read stage 暂停或减小 batch；低于低水位线后恢复。这样背压从 compute 传回 read。限速读取比读完再排队更稳定。

这个案例在本机文件上也成立。即使文件读取很快，如果解析和聚合慢，无限制读取会占用内存。真实网络输入更明显：停止读取 socket 会让 TCP 接收窗口缩小，把背压传给发送端。系统应尽量让背压接近源头，而不是在中间堆积。

案例：写出线程和计算线程分离 Reduce 完成后要写输出。若 worker 在线程池里直接执行写文件和 fsync，它会占住计算 worker，其他 reduce 任务无法运行。更好的设计是计算 worker 生成 OutputBlock，提交给 write stage；write stage 有自己的并发度和队列。写慢时，write queue 背压 reduce；reduce 再背压 shuffle；最终 map 降速。

分离后也要处理提交结果。Write stage 成功后通知 driver commit，失败后通知 driver 重试或失败作业。Compute worker 不能假设提交立即成功。这个结构把 I/O 等待从计算池中剥离，但增

加了异步状态机。

案例：超时后的输出 一个写出请求可能在上层超时后仍然成功落盘。如果上层已经重试并提交了一个 attempt，迟到的成功不能覆盖最终输出。解决方法是写出路径只写 attempt 临时文件，最终 manifest 由 driver 决定。迟到 attempt 即使写成功，也不会进入 manifest；清理线程稍后删除它。I/O 超时和分布式 attempt 去重在这里完全重合。

这个案例说明为什么写出路径不能直接写最终文件。直接写最终文件没有给 driver 选择获胜 attempt 的机会，也无法安全处理迟到完成。临时输出和 manifest 提交是可靠计算的基本结构。

端到端延迟分解 流水线报告应把端到端延迟分解为队列等待和服务时间。一个 batch 的延迟可以拆成 read service、parse queue wait、parse service、compute queue wait、compute service、write queue wait、write service、commit wait。若只看总延迟，无法知道瓶颈在哪。若队列等待占比高，说明下游容量不足或背压；若 service 时间高，说明该阶段本身慢。

Trace 可以记录每个 batch 的阶段时间。聚合后可以看中位数和尾部。很多系统平均吞吐正常，但少数 batch 因为写出 fsync 或热点 key 尾部很长。端到端分解能帮助定位尾延迟。

I/O 背压验收清单 本章最终验收应包含：队列有 item 和 byte 两种容量；生产者在高水位线暂停；低水位线恢复；写出使用临时文件和提交点；completion 迟到不会重复业务回调；超时请求最终 reaped；错误分类进入指标；关闭支持 drain 和立即取消两种策略；trace 能显示每个 batch 的等待和服务时间。达到这些要求，I/O 模块才足以支撑第 18 章的分布式计算。

I/O 章节总结 I/O 系统的核心是等待和完成。同步阻塞简单但占线程，异步减少线程等待但增加状态机；批处理提高吞吐但增加延迟；有界队列暴露过载；deadline 和取消控制生命周期；临时文件和 manifest 定义提交。I/O 代码的质量不在于用了哪个 API，而在于是否能在短读、短写、超时、取消、关闭和失败时保持状态正确。

I/O 决策表 设计 I/O 路径时，先问并发等待数量是否大，是否需要异步；数据是否必须持久化，是否需要 fsync 和 rename；是否允许丢弃，队列满时阻塞还是失败；是否有 deadline，超时后底层完成如何 reaped；是否需要批处理，数量阈值和时间阈值是多少；是否按 item 和 byte 双重限流；错误是否分类；关闭是 drain 还是立即取消。把这些答案写清，I/O 状态机才不会在边界条件下崩溃。

案例：检查点写入调度 Checkpoint 写入可能和正常计算争用磁盘。若每个 stage 结束都立即写大 checkpoint，作业延迟会出现尖峰；若后台慢慢写，又要处理写未完成时进程崩溃。可以把 checkpoint 分成元数据和大数据：关键 manifest 小而同步提交，大块数据在 stage 输出时已经写好并校验。这样 checkpoint 提交只发布指针和摘要，减少关键路径 I/O。

这个案例展示了控制面和数据面分离在 I/O 层面的意义。真正必须同步持久化的是“哪些数据有效”的小元数据，大数据可以提前流式写入临时位置。第 18 章 manifest 正是这种结构。

案例：日志丢弃策略 系统过载时，调试日志可以丢，但错误报告和提交日志不能丢。可以为日志设置优先级队列：critical 日志阻塞或落盘，debug 日志在队列满时丢弃并增加 drop counter。这样过载时系统仍保留关键证据，不会被低价值日志拖垮。

丢弃策略必须透明。Drop counter 要进入指标，报告中要说明哪些日志可能丢。静默丢弃会让故障排查失去证据。背压策略本质上是业务价值排序。

案例：异步写入和 backpressure 闭环 假设 reduce worker 生成输出 block，write stage 写磁盘。如果 write queue 超过高水位，reduce worker 停止接收新的 shuffle block；reduce 队列满后 map worker 降低发送；map 输出缓冲满后 input reader 暂停读取。这个闭环把磁盘压力传到输入源。若任何一段使用无限队列，闭环断开，内存会在那段堆积。

设计背压时，要画出这条闭环，并标出每段的容量和策略。哪个阶段阻塞，哪个阶段返回 busy，哪个阶段丢弃，哪个阶段降级，都要明确。背压不是单个队列属性，而是整条数据流的协议。

案例：超时预算贯穿流水线 一个 batch 从读取到提交可能有总预算。Read stage 用掉一部分，parse queue 等待用掉一部分，compute 用掉一部分，write fsync 又用掉一部分。如果每段只看自己的 timeout，端到端可能超预算。更好的方式是 batch 携带 deadline，每个阶段根据剩余时间决定继续、降级还是失败。

Deadline 也帮助背压。队列里等待太久的 batch，即使后续执行也可能超过预算，可以提前丢弃或标记失败，释放资源给仍有机会成功的任务。这个策略不适合必须完成的批处理，但适合交互请求和有时效性的日志分析。时间语义是调度的一部分。

本章扩展习题

习题与作业

习题六：设计 completion queue 的请求状态机。状态至少包含 submitted、completed、timed out、canceled、reaped。说明每个状态允许哪些转移。

习题七：写一个短读短写处理流程。要求支持 header/body 消息，记录已处理字节数，并说明错误和 EOF 如何处理。

习题八：为重试请求设计共享令牌桶。说明正常请求和重试请求是否共用 token，为什么。

习题九：设计 shutdown drain 策略。比较立即关闭、drain 关闭、带超时 drain 三种语义，以及它们适合的业务。

实验：用 bounded queue 模拟生产者和消费者。让生产速度高于消费速度，观察队列大小和阻塞行为。解释这为什么是背压，而不是简单的性能问题。

I/O 的核心是提交和完成分离 阻塞 I/O 把提交和完成放在同一个调用里：调用返回时，要么成功，要么失败，要么超时。它简单，但会把等待带进当前线程。异步 I/O 把提交和完成拆开：提交只是把请求交给内核或后台线程，完成以后通过 completion 事件、回调或队列返回。拆开以后，所有权、生命周期、错误传播和背压都会变得更复杂。本章必须从这个分离讲起，而不是先列 epoll、io_uring 或回调名词。

贯穿项目写 shuffle block 时，buffer 提交给写出层后，直到完成事件到来前都不能释放或修改。若写失败，要知道这个 block 对应哪个 task attempt、哪个 partition、哪个临时文件、是否已经写入部分数据。若上游继续产生 block，而下游写盘变慢，内存会堆积。异步 I/O 的难点不是“非阻塞”，而是完成之前系统处于中间状态。

背压是容量合同 无界队列把下游慢的问题藏进内存，直到延迟爆炸或 OOM。有界队列让容量成为明确合同：队列满时，生产者等待、失败、丢弃、降级或转移。项目中的写出队列不能无限缓存 reduce 输出；它应该有容量、水位和策略。计算阶段看到写队列满，就要减速或停止提交更多写请求。这不是性能失败，而是系统保护自己。

背压要能沿流水线传播。写盘慢导致 write queue 满，reduce 不能继续材料化输出，map 可能暂停产生更多 partition，driver 看到 backlog 后减少新任务提交。若背压只停在局部，其他阶段仍然狂生产，就会把压力转移到别处。Trace 应记录每个队列的水位和等待时间，帮助读者看到压力链条。

可靠写入从临时文件开始 写输出不能直接覆盖最终文件。可靠写入常用流程是写临时文件、刷新文件内容、必要时刷新目录、原子 rename 到目标名、写 manifest。不同文件系统和挂载选项下细节会不同，教材不需要穷尽所有平台，但必须让读者知道“write 返回成功”不等于“断电后一定存在”。本章的目标是建立提交边界意识。

项目可以把每个 shuffle block 写成临时文件，完成后计算 checksum，fsync 文件，再 rename 为包含 stage、partition 和 attempt 的稳定名字，最后更新 manifest。恢复时只信 manifest 中校验通过的 block；孤立临时文件可以清理；重复 attempt 通过 attempt id 和提交表判断有效性。这样 I/O、幂等和分布式恢复连起来。

超时和重试必须有预算 I/O 失败不一定能简单重试。磁盘满、权限错误、路径不存在通常不是重试能解决；临时 EAGAIN、网络抖动或远端超时可能适合重试。重试要有预算，包括次数、总时间、退避策略和幂等保证。没有预算的重试会放大拥塞；没有幂等的重试会制造重复输出。

错误分类应成为代码结构。PermanentError 直接失败并报告；TransientError 进入带预算重试；Canceled 停止下游；DataCorruption 触发隔离和恢复；Timeout 记录上下文并按策略处理。把所有错误都 catch 后再试一次，是系统不可靠的来源。

实验：慢写出如何拖垮上游 构造一个 pipeline：parse 产生 block，reduce 压缩 block，writer 慢速写入。版本一使用无界内存队列，观察内存峰值和尾延迟；版本二使用有界队列，生产者等待，观察吞吐和内存稳定性；版本三加入批量写和 manifest，观察系统调用次数和恢复行为；版本四注入写失败，验证临时文件清理和重试预算。

报告必须解释取舍。有界队列可能降低峰值吞吐，但换来内存上界和延迟可控；批处理减少系

统调用，但增加单批延迟；fsync 提高可靠性，但增加提交成本；异步写减少 worker 阻塞，但增加状态管理。I/O 章节不能只追求快，要把可靠性和容量边界写清楚。

Linux 源码和工具连接点 Linux I/O 观察可以从 `strace` 看系统调用形态，从 `iostat` 看设备吞吐和队列，从 `perf` 或 `tracepoint` 看内核路径，从 `lsof` 看文件句柄。读内核源码时，关注 page cache、writeback、dirty page、fsync 和 completion。普通 write 可能只是把数据放进页缓存，真正回写稍后发生；fsync 才是用户请求持久化边界。

这些知识回到项目，就是不要把“函数返回”误当“系统已完成”。异步请求要有 completion，可靠提交要有 manifest，背压要有队列水位，错误要有分类。这样第二册的 I/O 章节才能服务后面的分布式计算，而不是孤立地介绍系统调用。

事故复盘：submit 成功为什么还会丢数据 异步 I/O 的 submit 只表示请求被接受，不表示数据已经写到目标位置。事故常见于无界输出队列：计算线程不断提交 block，磁盘变慢后队列膨胀；某个 completion 迟到，业务回调已经超时重试，又被旧 completion 触发一次；短写没有处理，程序却把 block 标成完成。异步化如果没有 completion 状态，就只是把阻塞变成延迟爆炸。

正确模型要区分 request、in-flight、completion 和 commit。Submit 后 buffer 不能立即复用，除非 I/O 层已经复制或取得所有权；短写要带偏移继续写；取消是尽力而为，completion 仍可能迟到；关闭要决定 drain 还是 cancel。背压来自有界队列、in-flight bytes 和磁盘服务时间估计。队列水位上升时，上游应降低生产速度，而不是继续把内存当缓冲池。

实验要把服务时间和队列等待分开。无界队列可能短时间吞吐高，但内存持续上涨；有界队列可能让计算线程等待，却把系统保持在可恢复范围。故障注入包括短写、EAGAIN、延迟 completion、取消后完成、关闭时仍有请求。最终验收不是“没有报错”，而是写入偏移、checksum、completion 次数、回调次数和临时文件状态都一致。

I/O 背压实验判读 I/O 实验要把服务时间和队列等待分开。无界队列吞吐高但内存涨，说明压力被隐藏；有界队列吞吐下降但内存稳定，说明背压生效。短写测试要检查已写偏移；迟到 completion 要检查业务回调是否重复；崩溃点测试要检查 manifest 是否只发布稳定输出。

这一段也服务于复习。读者合上书后，应该能凭本章的判读口径重新审查自己的代码：有没有最小正确版本，有没有能击穿旧方案的输入，有没有状态记录，有没有资源指标，有没有失败后的恢复动作。若五个问题缺两个以上，就说明实现还停在演示层，而不是工程层。

迟到 completion 的状态机 I/O 请求超时后，底层写入可能仍然完成。若上层已经重试并提交了新 attempt，旧 completion 不能再次触发业务成功。请求状态至少要区分 submitted、timed out、completed、committed、ignored、reaped。completion 到达时先查状态，再决定是提交、忽略还是告警。

这个状态机也解释为什么直接写最终文件危险。旧 attempt 即使迟到写成功，也只应该写自己的临时文件；最终 manifest 由 driver 决定哪个 attempt 生效。没有临时文件和 manifest，迟到 completion 就可能覆盖新结果。I/O 章节的可靠性核心就在这里。

背压要和这个状态机一起设计。队列满时，生产者等待或失败；超时和取消要释放 in-flight 预算；completion 到达后也要释放预算，即使业务结果被忽略。否则系统会在异常路径泄漏容量，最终所有请求都无法提交。

实验课：写出队列从无界到可恢复 第一版让 reduce worker 直接写文件并 fsync。写盘慢时，计算 worker 被阻塞。第二版把 OutputBlock 放入无界队列，I/O worker 写出；计算 worker 不再阻塞，但队列可能无限增长。第三版给队列设置 item 和 byte 双重容量，高水位暂停上游，低水位恢复。每一版都记录吞吐、内存峰值、队列等待和尾延迟。

可靠写入实验加入临时文件、checksum、fsync、rename 和 manifest。故意在写临时文件中途、fsync 前、rename 后、manifest 前崩溃，恢复程序要能识别文件状态。只有 manifest 中 committed 的 block 能被 reduce 读取。这个实验让读者看到：I/O 性能和恢复语义不能分开设计。

迟到 completion 单独测试。请求超时后，上层重试；旧请求后来完成。正确行为是旧 completion 释放资源预算，但不重复提交业务结果。如果它对应的 attempt 已被新 attempt 取代，就标记 ignored 或 reaped。报告中要展示请求状态转换，而不是只看最终输出。

背压闭环最后连接输入源。Writer 慢导致 write queue 满，reduce 降速，shuffle 缓冲减少，map 暂停，reader 停止继续读。若某一段用无界队列，闭环断开，内存就在那一段堆积。I/O 章节要让读者画出这条链。

I/O 背压练习验收重点 合格答案要把 submit、complete、commit 分开。WriteRequest 的 buffer 生命周期、短写进度、completion 状态、manifest 提交点都要写清。背压题要说明高水位和低水位如何传回上游；可靠写题要说明 fsync、rename 和崩溃点。

答案还要写“为什么不采用更复杂方案”。例如没有证据证明队列是瓶颈，就不应把锁队列换成无锁；没有证明输出顺序可变，就不应随意重排；没有证明错误路径能恢复，就不应只提交正常路径。能解释不做什么，说明读者开始具备工程判断。

最后，答案必须包含可复查材料。至少有输入摘要、关键状态、期望输出、实际输出、运行命令和未验证边界。没有这些材料，老师或同伴无法判断结论是否成立。第二册的作业不是让读者写漂亮段落，而是训练读者把系统问题变成可验证证据。

项目实现：I/O、异步与背压 本章对应的贯穿项目实现不要求一次写成完整框架，但必须有最小可运行切片。这个切片包含三个部分：一个 reference，一个带本章机制的实现，一个能击穿 reference 之外隐含假设的测试。Reference 用来固定语义，机制实现用来展示本章知识，测试用来证明新增复杂度确实解决了问题。

代码组织上，不要把所有内容放进一个大文件。基础类型、输入生成、核心机制、报告输出和测试应分开。即使当前只是教学项目，也要让目录能表达责任边界。这样后续章节接入时，不会因为找不到状态归属而重新发明一套对象。

每个实现都要输出一段机器可读摘要。摘要至少包含输入规模、关键状态、错误数、输出校验、耗时和未验证边界。这里的机器可读不要求复杂格式，简单的 key value 文本也可以。关键是让脚本和读者都能复查，而不是只能看屏幕截图。

测试应包含正常路径、边界路径和失败路径。正常路径证明功能可用；边界路径证明长度、容量、尾部、空输入、关闭或超时没有遗漏；失败路径证明本章机制不是装饰。若失败路径写不出来，往往说明本章概念还没有落到工程对象上。

报告最后写一个取舍段。新增机制带来了什么好处，增加了什么成本，哪些场景不应该使用它，下一章会复用哪个字段或对象。这个取舍段非常重要，因为系统工程不是把所有高级机制都加进去，而是在证据和约束下选择合适的边界。

调试笔记：内存上涨先查哪段队列无界 I/O 管线内存上涨时，沿 read、parse、compute、write、commit 队列逐个看水位。哪一段没有容量，压力就会在那里堆积。若 write queue 满而上游继续生产，背压链断了；若 timeout 后 in flight 不释放，容量泄漏了；若 completion 到来后重复回调，状态机没有区分 committed 和 ignored。

复查清单：I/O、异步与背压 复查本章时，先检查 reference 是否仍然存在。没有 reference，后面的并发、优化、异步或分布式改造都无法证明语义没有变。再检查压力输入是否尖锐：它应当攻击本章最核心的边界，而不是只把数据量放大。第三，检查状态是否可观察：关键对象的身份、状态、错误和提交结果应能在报告中找到。

第四，检查代码示例是否说明禁止行为。调用者不能做什么，往往比能做什么更能体现工程质量。第五，检查实验是否写出未验证边界；当前设备、权限或输入无法证明的结论，必须标注。第六，检查本章对象是否能被下一章复用，不能复用就说明边界还不稳。

完成这六项后，再看文字是否连贯。每个概念都应从问题推出，每个案例都应从朴素方案走到改进方案，每个实验都应能解释一个判断。若某段话放到其他章节也成立，就把它改成本章的具体对象、具体输入和具体失败。

15.8 工程案例：从写出队列推导异步 I/O、背压和可靠提交

I/O 章节不能只讲 API。真正的核心是完成事件和所有权：数据什么时候交给内核或设备，什么时候可以复用 buffer，写出失败如何传播，上游什么时候应该减速，结果什么时候算提交。一个 shuffle block 写出队列足以把这些问题串起来。

一、同步写最清楚，但会阻塞执行线程 最简单版本在 worker 里直接写文件。它的语义清楚：函数返回后要么成功，要么失败；buffer 生命周期容易管理。但 worker 会被磁盘或文件系统阻塞，CPU 任务并行度下降。若写出很慢，上游继续产生 block，就会堆积内存或拖慢整个阶段。

把写出交给专用 I/O 线程，可以隔离阻塞。但这引入队列和背压：队列多大，满了怎么办，block buffer 由谁拥有，写失败怎么通知产生者，关闭时队列里尚未写出的 block 怎么处理。同步到异步的代价不是代码复杂一点，而是语义边界从调用栈转移到完成事件。

二、异步提交后 buffer 不能立即复用 无论使用 I/O 线程、回调、`io_uring` 还是 AIO，只要写出尚未完成，buffer 就不能被上游复用或释放。完成事件到达之前，内核或 I/O 线程可能仍在读取 buffer 内容。很多异步 bug 都是生命周期 bug：上游为了减少分配复用块，结果写到磁盘的是后来的内容。

解决方式包括引用计数 block、对象池状态机、移动所有权到 I/O 队列、完成后归还 buffer。接口要表达所有权转移。比如 `submit_write(Block&&)` 表示调用者提交后不再使用 block；完成事件带回 block id 和状态；对象池只有在 completion 后才能复用内存。

三、有界队列是背压的本地形式 无界 I/O 队列会把下游慢的问题变成内存峰值。队列必须有界。满了以后，上游可以阻塞、返回 `would block`、降低批大小、丢弃低优先级数据或触发全局限速。选择取决于业务语义。离线计算通常阻塞或减速，在线服务可能快速失败或降级。

背压要向上游传播到足够远。若 writer queue 满了，map worker 阻塞；若 map worker 阻塞，线程池 ready 任务减少；若 driver 继续提交新 split，内存仍可能上涨。因此背压不是某个队列的局部状态，而是 pipeline 的流量控制。报告要记录每个队列水位和等待时间。

四、短写和 fsync 是可靠写人的核心 写文件不是一次 `write` 返回正数就结束。可能短写，可能被信号中断，可能写入 page cache 后尚未持久化，可能 fsync 失败，可能 rename 成功前崩溃。可靠提交通常需要临时文件、完整写入、校验、fsync 数据、rename、fsync 目录、manifest 提交。不同文件系统和设备语义会影响细节，但教材要让读者知道调用栈返回和持久化承诺不是一回事。

离线 shuffle 可以采用临时文件加 manifest。Worker 写临时 block，计算 checksum，完成必要同步后报告给 driver；driver 只有在校验和元数据完整后写 manifest。Reduce 只读 manifest，不扫描临时目录。这样写出失败、崩溃和重复 attempt 都能被控制面解释。

五、批处理在吞吐和尾延迟之间取舍 I/O 批处理能减少系统调用和 fsync 次数，提高吞吐；但批太大，单个 block 等待时间增加，尾延迟变差，失败后重做范围变大。批大小应由数据量、设备带宽、延迟目标和恢复成本共同决定。不要只用平均吞吐选 batch size。

实验可以固定输入总量，改变 block size 和 fsync group size。记录吞吐、p50/p99 写入延迟、内存峰值、重做字节数和队列水位。若大 batch 吞吐高但 p99 很差，报告要写清适用场景。离线作业可能接受，交互服务可能不能接受。

六、完成事件要统一错误传播 异步 I/O 的错误不会在 submit 时全部出现。Submit 成功只表示请求进入队列；完成事件可能报告短写、EIO、取消、超时或校验失败。运行时需要统一的 `IoCompletion` 对象，携带 block id、attempt id、字节数、状态和错误码。上游等待的是 completion，不是 submit 返回值。

完成事件还要和任务图连接。写出成功后，manifest commit 任务 ready；写出失败后，对应 attempt failed，可能触发重试；取消后，buffer 归还但结果不提交。若 completion 只是打印日志，driver 就无法做正确恢复。I/O 章节要把完成事件写成控制面事实。

七、Linux I/O 路径阅读入口 读 Linux I/O 不必一开始深入块层所有细节。可以从用户态系统调用语义、page cache、dirty page、writeback、fsync 和 `io_uring` completion queue 的文档入手。关注问题是：数据何时复制，何时排队，何时对设备可见，何时对崩溃恢复有保证，完成事件在哪里报告。

在权限有限环境中，可以用普通文件和故障注入模拟许多语义：短写包装器、延迟写线程、随机失败、响应丢失、目录扫描反例。没有真实 NVMe 或 `io_uring` 权限，也能学习所有权、背压和提交点。工具不是目标，语义才是目标。

八、实验交付：IoPipelineReport 本章给项目留下 `IoPipelineReport`。它记录 block id、buffer owner、submit time、completion time、bytes requested、bytes written、checksum、fsync policy、queue depth、retry count 和 commit state。每个 I/O 实验都要能回答：buffer 何时可复用，失败如何传播，结果何时对 reduce 可见。

实验比较同步写、I/O 线程写、有界队列写和批量 fsync。故障注入包括短写、写失败、completion 延迟、取消和崩溃后目录里有临时文件。验收时 reduce 只能读取 manifest，不能读取未提交临时文件。这样 I/O 章才能和分布式提交闭合。

15.9 案例深化：背压如何穿过整条流水线

背压不是队列满了这么简单。真正的问题是下游处理能力下降时，上游如何感知并减速，系统如何避免把压力转成内存峰值、重试风暴或尾延迟。日志计算流水线里，reader、parser、map、writer、driver 和 reduce 都可能成为下游。

一、队列水位是信号，不是目标 队列水位升高说明生产速度超过消费速度，但解决方式不一定是加消费者。消费者可能阻塞在磁盘，增加消费者只会增加写竞争；队列容量太小可能导致频繁阻塞，太大又增加内存和延迟。水位要和阶段耗时、I/O 完成、错误率一起看。

报告应记录水位时间序列，而不是只记录最大值。短暂尖峰和持续积压含义不同。持续积压说明系统进入过载，需要减速、扩容或改变批处理；短暂尖峰可能只是阶段性 burst。背压控制需要时间维度。

二、背压策略有语义差别 阻塞生产者保持数据完整，但可能占住线程；返回 would block 让上游选择稍后重试，但需要重试策略；丢弃低优先级数据适合监控，不适合准确计算；降级处理可能改变结果精度。不同业务选择不同策略。离线日志统计通常不能丢数据，所以应阻塞或减速，而不是丢弃。

策略要写进接口。push 失败是永久关闭、临时满、取消还是错误？上游根据不同结果采取不同动作。一个 bool 返回值不足以表达背压语义。

三、端到端背压实验 构造一个慢 writer，让 parser 和 map 比写出快。比较无界队列、有界阻塞队列、would block 加重试、动态降低 reader 速度四种方案。记录内存峰值、吞吐、p99 延迟、丢失或重复记录、线程阻塞时间。预期无界队列短期吞吐好看但内存上涨；有界队列稳定但上游等待增加。

四、和分布式限流连接 单机背压到分布式就是 in-flight 限制和 Busy 响应。Driver 对每个 worker 维护窗口，worker 忙时返回建议重试时间，driver 用退避避免风暴。I/O 队列水位也可以反馈到 map 调度，少给写慢的节点分配任务。背压是一种控制面协议，不只是本地容器参数。

15.10 补强案例：可靠写入和性能之间的显式取舍

可靠写入不是只有一种强度。进程崩溃恢复、机器掉电恢复、跨文件系统 rename、远端存储确认，对应不同成本。教学项目必须明确承诺哪一层可靠性，不能模糊地说“写入成功”。

一、进程崩溃和掉电不同 若只要求进程崩溃后恢复，临时文件和 manifest 在同一进程重启后可检查，很多 page cache 数据仍可能存在。若要求掉电后恢复，就要考虑 fsync 数据和目录。两者成本差很多。项目可以先承诺进程崩溃恢复，再把掉电持久化作为扩展实验。

二、rename 的语义边界 常见模式是写临时文件，fsync，rename 到最终名，再 fsync 目录。Rename 通常在同一文件系统内原子替换目录项，但跨文件系统不成立。若项目把临时目录和最终目录放在不同挂载点，语义会变化。报告要记录路径和文件系统。

三、批量 fsync 的代价 每个 block 都 fsync，可靠但慢；多个 block 批量 fsync，吞吐好但崩溃时可能丢失更多未提交工作。若 manifest 只提交已 fsync block，语义仍清楚，只是重做范围变大。可靠性和性能的取舍要用重做成本表达。

四、实验 实现三种策略：不 fsync，只保证进程内完成；每 block fsync；批量 fsync。用故障注入模拟 commit 前崩溃。报告写吞吐、提交延迟、可恢复 block 数和需要重做的字节数。这样读者会理解可靠写入不是 API 名称，而是承诺级别。

15.11 补充案例：I/O buffer 池和生命周期

异步 I/O 常使用 buffer 池减少分配。Buffer 池如果设计不好，会写出错误数据或造成内存泄漏。核心问题是 buffer 状态：free、filling、submitted、completed、recycling。每次状态转换都要有所有者。

一、提交后所有权转移 Producer 从池中取 free buffer，填充数据后 submit 给 writer。Submit 成功后，buffer 所有权转移给 I/O 子系统，producer 不能再写。Completion 返回后，buffer 才能进入 recycling 或 free。若 producer 提前复用，磁盘上可能出现后一个 block 的内容。

二、失败也要归还 buffer 写失败、取消、短写都要产生 completion。Completion 负责把 buffer

交给错误处理或归还池。若失败路径忘记归还，池会耗尽；若失败路径重复归还，会出现双重使用。状态机比裸指针安全。

三、池大小和背压 Buffer 池大小决定最多 in-flight 字节。池耗尽时，上游必须等待或减速。这是比队列长度更直接的内存背压。报告要记录 buffer 总数、in-flight 数和等待时间。

四、实验 实现小 buffer 池，注入写延迟和写失败。检查没有提前复用、没有泄漏、没有双重归还。再改变池大小观察吞吐和内存峰值。这个实验能让异步 I/O 的所有权问题变得可见。

15.12 补充专题：I/O 错误分类

I/O 错误不能统一处理。短写、权限错误、空间不足、设备错误、校验失败、取消、超时，每种错误的恢复策略不同。结构化错误分类比字符串日志更重要。

一、可重试错误 临时 EAGAIN、队列满、远端忙、短暂超时可能可重试。重试要受预算控制，并保留同一 block id 或 attempt id。重试前要确认旧请求是否仍可能完成，避免重复提交。

二、不可重试错误 权限错误、输入不存在、格式版本不支持通常不可重试。继续重试只会浪费资源。系统应快速失败，并把错误传播到 job 状态。错误分类能防止重试风暴。

三、数据完整性错误 Checksum 不匹配、短写后文件残缺、header 版本错误属于完整性错误。策略通常是丢弃该 block 并重算对应 attempt；若重复出现，可能说明硬件或代码 bug，应升级为 job failure。

四、实验 为 writer 注入每类错误，验证 driver 策略不同。报告记录 retry、fail、ignore、cleanup。这样 I/O 章和分布式重试策略能对齐。

15.13 补充专题：异步完成顺序和提交顺序

异步 I/O completion 顺序不一定等于 submit 顺序。系统不能因为 block 3 先完成，就假设 block 1 和 block 2 已完成。提交协议要按 block 身份和依赖判断，而不是按完成到达顺序。

一、乱序完成是正常现象 设备、线程池和操作系统调度都可能让后提交的请求先完成。Completion 必须携带 block id 和 attempt id。Writer 收到 completion 后更新对应状态。没有身份，乱序完成会让状态表错乱。

二、提交可以乱序也可以排序 Manifest 是否要求按 task id 排序，是语义选择。可以让完成一个提交一个，但最终读取时排序；也可以等一组 block 全部完成后按顺序提交。前者延迟低，后者输出稳定。项目应写清选择。

三、依赖完成 Reduce 某个 partition 只能在它需要的 map block 都提交后运行。不是任意 completion 都能触发 reduce。Driver 要按 partition 维护依赖计数。这和任务图 remaining predecessors 同源。

四、实验 提交多个 block，随机打乱 completion 顺序。检查 manifest 条目身份正确，reduce 等待完整依赖。这个实验能防止异步代码依赖偶然顺序。

15.14 I/O 实现细节：Completion loop 的职责

异步 I/O 的中心不是 submit，而是 completion loop。Submit 只是把请求交出去；completion loop 接收完成事件，更新状态，归还 buffer，触发后续任务，传播错误。若 completion loop 设计不清，异步代码会散成回调和共享变量。

一、Completion loop 的输入 输入是完成事件。事件至少包含 request id、block id、attempt id、status、bytes transferred、error code、buffer handle 和 completion time。不要只传一个 bool。短写、取消、超时、校验失败和成功需要不同处理。

Completion loop 收到事件后，先验证身份，再查找 in flight 表。若找不到，说明重复 completion、旧 completion 或程序 bug。不要默默忽略所有未知事件，至少记录诊断。若找到，按状态机处理，并确保同一个 request 只完成一次。

二、归还 buffer Buffer 只有在 completion 处理完后才能归还池。若成功，可能先计算或验证 checksum，再写 manifest；若失败，可能交给 retry 或清理。无论哪条路径，buffer 状态必须从 submitted 走到 completed，再走到 free 或 retired。双重归还和忘记归还都要通过状态检查发现。

三、触发后续任务 Completion 成功可能让 manifest commit task ready；某个 partition 所有 block 完成后，reduce task ready；失败可能让 retry task ready；取消可能让等待者收到 Cancelled。Completion loop 因此连接 I/O 和任务运行时。它不应该直接执行大量后续计算，而是更新状态并提交任务。

四、错误汇总 Completion loop 也是错误汇总入口。它把底层错误码转成项目错误类型，例如 ShortWrite、ChecksumMismatch、Cancelled、DeviceError、PermissionDenied。Driver 根据结构化错误决定重试或失败。若只把错误写成字符串日志，上层无法自动恢复。

五、实验 实现一个模拟 completion queue，随机打乱完成顺序，注入短写和失败。验证每个 request 只完成一次，buffer 最终全部归还或 retired，后续任务按依赖 ready，manifest 不包含失败 block。这个实验能把异步 I/O 的控制面训练清楚。

15.15 从等待 I/O 推导完成事件和背压

I/O 章节要从一个朴素循环开始：读一块、处理一块、写一块。这个程序容易理解，但磁盘、文件系统、页缓存和下游设备都可能让线程等待。异步 I/O 的目标不是把代码写得花，而是把“提交请求”和“请求完成”分开，让 CPU 在等待设备时继续处理其他工作。可是提交和完成分开后，生命周期、顺序、错误、短读短写、取消和背压都必须显式设计。

一、完成事件是状态转移，不是回调装饰 一次异步写提交后，buffer 不能马上复用，因为设备可能还没读完。完成事件回来时，程序才能知道写了多少字节、是否出错、是否需要继续写剩余部分。若只把 completion 当作回调函数，很容易丢失对象生命周期。更稳的做法是为每个请求建立 CompletionRecord，记录 buffer、offset、期望长度、已完成长度、状态和提交 epoch。这样迟到完成、重复完成和取消完成都能被状态机处理。

二、短读短写是正常路径 系统调用返回的字节数可能小于请求字节数。网络、管道、非阻塞文件描述符和某些设备都可能出现短读短写。正确代码要循环推进 offset，直到完成、出错或遇到可重试等待。把 write 返回值当作一次完成，是 I/O 程序常见错误。教学中应把可靠写循环作为基础案例，再讨论异步化；否则读者会把错误语义带进更复杂的完成队列。

三、背压来自有限 buffer 和有限下游能力 如果上游产生速度超过写出速度，内存会不断增长，队列会变长，尾延迟会恶化。背压不是性能优化，而是系统稳定性要求。有界队列、buffer pool、in-flight 限制和 deadline 都是背压形式。容量设置要来自内存预算和下游吞吐，而不是随意写一个大数字。比如每个 buffer 1 MiB，允许 512 个 in-flight，就已经占用 512 MiB；在手机环境中，这可能直接触发系统回收或杀进程。

四、持久化点决定恢复语义 写入文件不等于结果持久。数据可能在页缓存中，目录项可能还没落盘，rename 和 fsync 的顺序也影响崩溃恢复。贯穿项目要把输出写到临时文件，校验完成后原子提交到 manifest；崩溃恢复时只读取已提交条目。若只检查文件存在，可能读到半写文件；若只写 manifest 不同步数据，重启后 manifest 可能指向丢失内容。I/O 章节必须把性能和提交语义一起讲。

五、异步模型要有调试视角 异步程序出错时，调用栈常常已经离开提交点。调试需要 request id、buffer id、offset、attempt、提交时间、完成时间和错误码。报告要能回答：哪个请求还在飞行，哪个 buffer 被谁持有，哪个完成迟到，哪个阶段触发背压，关闭时是否还有未完成请求。没有这些日志，异步 I/O 会变成不可解释的偶发失败。完成事件模型的价值就在于把这些状态集中起来。

15.16 可靠写出路径的逐步推导

I/O 章节还应把“写出一个结果文件”这件事讲透，因为它连接短写、背压、异步完成、崩溃恢复和 manifest。朴素代码通常直接打开目标文件，写入数据，关闭文件，然后认为完成。这个过程在真实系统里有多漏洞：写入可能只完成一部分，进程可能在中途崩溃，页缓存可能还没落盘，目标文件可能被读者提前看到，manifest 可能指向半成品。可靠写出路径就是逐步补上这些漏洞。

一、先写临时文件而不是目标文件 输出应先写到临时路径，临时路径包含 task id、attempt id 和随机后缀，避免多个 attempt 写同一个文件。只有数据完整、校验通过并完成必要同步后，才用 rename 或 manifest 提交把它变成可见结果。这样读者能看到临时文件不是实现细节，而是隔离未提交状态的边界。若程序崩溃，恢复时扫描临时文件，可以按 attempt 和 manifest 决定删除或重做。

二、可靠写循环推进 offset 每次 write 或异步完成都只说明完成了某个字节数。代码必须维护 **offset** 和 **remaining**，直到全部字节完成或遇到不可恢复错误。短写后继续写剩余部分，可重试错误进入等待或重试，不可重试错误标记 attempt 失败。异步版本只是把这个循环拆成提交和完成事件，并没有改变语义。把同步可靠写写清楚，是理解异步可靠写的前提。

三、同步顺序决定崩溃恢复 如果要求崩溃后结果可见，通常要先同步文件内容，再提交 manifest 或 rename，再同步目录或 manifest 文件。具体系统调用语义受文件系统影响，教材不应承诺所有平台完全一致，但要讲清顺序的目的：不能让元数据指向未持久化内容，也不能让恢复逻辑误认半成品为完成品。在手机或受限环境中，即使无法严格验证掉电语义，也要用故障注入模拟进程崩溃点。

四、背压保护可靠写路径 可靠写通常比单纯写内存慢。如果上游无限产生 output block，临时文件、buffer 和完成队列都会膨胀。写出阶段应有 in-flight 上限和 buffer pool；当写队列超过阈值，上游 compute 阶段减速或停止提交新任务。背压不是单独的性能模块，而是保护提交语义的资源边界。没有背压，崩溃恢复时还会面对大量未提交临时文件。

五、恢复扫描要有确定规则 启动恢复时，系统读取 manifest，列出已提交 block；扫描临时目录，删除不在 manifest 且没有活跃 attempt 的临时文件；对 manifest 缺失但输入仍需要的任务重新排队。规则必须确定，不能依赖文件修改时间猜测正确性。报告要给出崩溃点矩阵：写前崩溃、写中崩溃、写完未提交崩溃、提交后崩溃、删除临时文件前崩溃。每个崩溃点都要有恢复结果。

15.17 Linux I/O 源码阅读的分层路线

用户不需要一口气读完整个 Linux I/O 栈，但要知道从哪里进入、每一层回答什么问题。源码阅读不是把函数名堆进正文，而是带着本章的四个问题进去：数据什么时候从用户态复制，什么时候进入页缓存，什么时候排队到设备，什么时候完成，什么时候对崩溃恢复有意义。只追这条线，源码才不会变成无边无际的森林。

一、从系统调用语义进入 第一层看 **read**、**write**、**pread**、**pwrite**、**fsync**、**rename** 的用户可见语义。读者要先理解返回值、错误码、短读短写和阻塞条件，再看内核实现。若用户态语义没理解，直接读内核函数只会迷路。阅读笔记应先写“调用者能观察到什么”，再写“内核内部可能怎样实现”。

二、再看 VFS 如何统一文件对象 VFS 把不同文件系统暴露成统一接口。这里重点不是背所有结构体，而是理解 file、inode、dentry、address space 的角色：file 表示打开实例，inode 表示文件对象，dentry 连接路径名和对象，address space 连接文件和页缓存。读者理解这些角色后，才能解释为什么 rename 是目录项变化，为什么页缓存按文件对象组织，为什么同一个文件多次打开共享底层数据。

三、页缓存解释写入为何不等于落盘 普通 write 常把数据复制进页缓存并标记 dirty，之后由 writeback 把 dirty page 写到设备。于是 write 返回并不等于设备已经持久化。fsync 的意义是推动相关 dirty 数据和元数据到持久化边界。不同文件系统的细节不同，但教学中要抓住主线：用户看到的是系统调用返回，崩溃恢复关心的是持久化顺序。可靠写出章节正是围绕这个差距建立临时文件和 manifest。

四、异步接口看提交队列和完成队列 io_uring 这类接口把提交和完成拆得更清楚：用户提交请求，内核在后台推进，完成队列返回结果。阅读时不要只看性能宣传，要看请求对象生命周期、buffer 注册、取消语义、完成顺序和错误返回。它和本章 CompletionRecord 的关系很直接：用户态也需要保存 request id、buffer、offset、状态和 deadline，才能正确处理迟到完成和部分完成。

五、源码阅读报告要回到实验 每次源码阅读都应回答一个实验问题。例如短写包装器对应系统调用返回值；可靠写出对应 page cache 和 fsync；背压对应 dirty page、writeback 和队列长度；异步完成对应 completion queue。读完源码后，要写“它解释了我哪个现象，哪些仍然只是推测”。这样源码阅读服务于工程判断，而不是变成炫耀阅读量。

15.18 贯穿案例：提交成功、完成成功和结果提交不是同一件事

异步 I/O 最容易被误解为“把 write 放到后台”。真正的事故是这样的：计算线程生成一个 64 MB shuffle block，把 buffer 交给 I/O 层，**submit_write** 返回成功。业务层看到成功，就复用

buffer，随后把 block 写进 manifest。几毫秒后 completion 返回短写，只有前 16 MB 写入；更糟糕的是，buffer 已经被下一批数据覆盖，重试也无法恢复原内容。最后 reduce 按 manifest 读取这个 block，checksum 不匹配，或者在没有 checksum 时悄悄得到错误结果。

这个事故把本章概念全部逼出来。Submit 成功只表示请求被接受，不是写完；Completion 成功也只是 I/O 完成，不是业务提交；短写要维护 offset；buffer 在 completion 前不能复用；manifest 才是外部可见提交点；写出队列必须背压，否则磁盘慢时内存会被 in-flight buffer 撑爆。异步 I/O 不是性能装饰，而是把生命周期、资源和提交边界拆开后重新管理。

一、同步可靠写先作为语义 reference 异步之前先写同步可靠写。循环调用 write，返回正数就推进 offset，返回 EAGAIN 就等待可写，返回可重试错误按策略重试，返回 fatal 错误标记失败。直到 offset 等于 total bytes，文件内容才算完整候选。这个 reference 不一定快，但它定义短写、错误和 offset 语义。异步版本只是把循环拆成事件，不应改变这些语义。

二、请求对象保存 buffer 所有权 异步写请求必须拥有或租借 buffer，并在 completion 前阻止复用。若使用 buffer pool，请求对象持有 pool handle；completion 后归还；取消后也要等内核不再使用或明确完成取消。裸指针和长度不足以表达所有权。接口应让调用者看见：提交后 buffer 不属于计算线程，直到 completion 或取消完成才释放。这个设计和第 16 章消息 payload 所有权完全一致。

三、completion handler 不能直接提交 manifest Completion 只是写出结果事件。Handler 应校验写入字节数、更新 offset、必要时提交剩余写；全部完成后计算或验证 checksum；然后进入 commit 阶段。Manifest commit 还要检查 task attempt、request id 和当前状态。若 handler 里直接写 manifest，短写、取消、旧 attempt 和重复 completion 都可能越过状态机。完成事件和提交事件分开，是可靠写出的核心纪律。

四、背压要按字节和阶段传播 队列里 100 个 4 KB 请求和 100 个 64 MB 请求不是同一压力。I/O 层应限制 in-flight bytes、buffer pool bytes、submitted requests 和 completion backlog。达到高水位时，上游计算不再生成新 block，或者减小 batch，或者暂停某些低优先级任务。背压要传播到任务运行时，否则 worker 会继续生产 buffer，把内存消耗转移到 I/O 层。资源上限是正确性的一部分，因为内存耗尽会破坏所有未提交状态。

五、崩溃点矩阵决定恢复合同 临时文件创建前崩溃，恢复时重新执行任务；写一半崩溃，临时文件 checksum 不匹配，删除或保留诊断；写完但 manifest 未提交崩溃，临时文件仍不能算有效；manifest 提交后崩溃，恢复应接受该 block；清理旧临时文件前崩溃，恢复扫描再清理。每个点都要写成测试。可靠 I/O 不靠“应该没事”，靠明确哪条记录是提交事实。

六、Linux 源码阅读回到这个事故 读 Linux I/O 路径时，目标不是背 VFS 函数名，而是解释事故中的三个问题：write 返回时数据在哪里，fsync 推动了什么持久化，异步 completion 到来时 buffer 是否还能被内核使用。带着这三个问题读 file、inode、address space、dirty page、writeback、io_uring completion，就能把源码连接到用户态状态机。读不到的细节也要诚实标注，不把推测当保证。

15.19 案例深化：异步写出管线的状态机

I/O 章节可以用写出 shuffle block 作为贯穿案例。计算线程生成 block，交给 I/O 层写临时文件，写完后计算或校验 checksum，再把 block 交给 manifest。若写出队列无界，磁盘变慢时内存上涨；若 completion 迟到，业务层可能重复提交；若短写没有状态，文件内容损坏还被当成成功。异步写出必须写成状态机。

一、请求状态比系统调用返回值更重要 一次写出请求可以经历 Created、Queued、Submitted、PartialWritten、Completed、Failed、Cancelled、Committed。系统调用返回成功只表示提交给内核或写了一部分，不等于业务完成。短写时更新 offset，EAGAIN 时等待可写，错误时分类可重试或 fatal。状态机让这些分支可测试，而不是散在回调里。

二、Buffer 所有权贯穿提交前后 计算线程生成 buffer 后，若 I/O 层没有复制，就不能在 completion 前复用。Completion 后，buffer 可以释放，但 block 文件还未必进入 manifest。Manifest commit 后，临时文件才变成外部可见输出。取消请求时，buffer、临时文件和 manifest entry 的处理不同。所有权图必须和状态机一起写，否则异步优化会制造悬垂和重复提交。

三、背压用字节而不是只用请求数 一个 1 KB block 和一个 64 MB block 都是一条请求，但内存和磁盘压力完全不同。I/O 队列应同时限制请求数和 in-flight bytes。上游看到队列高水位时，可以暂停计算、减小批量、切换到压缩或直接返回 Busy。只按请求数限流，容易被大 block 击穿内存预算。

四、可靠写入要有崩溃点矩阵 临时文件打开后崩溃、写一半崩溃、fsync 前崩溃、rename 前崩溃、manifest 写入后崩溃，每个位置恢复行为不同。教学项目可以简化同步强度，但必须写清承诺：哪些崩溃点保证恢复，哪些只保证检测失败。I/O 可靠性不是一句“写文件”，而是一组崩溃点合同。

五、I/O 指标要分队列等待和设备服务 总写出延迟由排队等待和设备服务组成。队列等待高说明上游过快或队列容量小，设备服务高说明磁盘或文件系统慢，completion 处理慢说明回调或提交阶段阻塞。实验应分别记录 queued 到 submitted、submitted 到 completed、completed 到 committed。这样背压调参才有依据。

15.20 案例复盘：异步写出必须分清完成和提交

计算线程生成 block 后提交异步写。Submit 成功只表示请求被接受，不表示数据已写完，更不表示 manifest 已提交。磁盘变慢时，无界队列会吃内存；取消后 completion 仍可能迟到；短写如果没处理，坏文件会被当成成功输出。

写请求状态机 请求经历 queued、submitted、partial written、completed、failed、cancelled、committed。Buffer 在 completion 前不能复用，临时文件在 manifest 接受前不是有效输出。短写更新 offset 继续写，EAGAIN 等待可写，fatal 错误进入恢复或失败路径。

实验判据 报告拆分 queued 到 submitted、submitted 到 completed、completed 到 committed。队列限制既按请求数也按 in-flight bytes。故障注入包括短写、延迟 completion、取消后完成、关闭时未完成请求和 fsync 前崩溃。

15.21 本章习题

习题与作业

习题一：设计一个三阶段流水线：读取、计算、写出。为每个阶段写出资源类型、队列容量、背压策略、指标和失败语义。

习题二：比较阻塞 submit、try submit 和 timed submit。分别说明调用者能感知哪些状态，适合哪些业务，不适合哪些业务。

习题三：设计批处理策略。给定数量阈值 N 和时间阈值 T，说明高流量和低流量下的行为，解释吞吐和尾延迟如何权衡。

习题四：写一份 `io_uring` 学习报告提纲。只要求解释提交队列、完成队列、in-flight 请求和取消语义，不要求完整实现。

习题五：构造一个重试风暴场景。说明没有退避和预算时系统怎样恶化，再设计带预算、退避和幂等 ID 的重试策略。

第零部分

分布式计算：把单机原则扩展到集群

第 16 章

分布式计算模型

分布式系统不应该从一串术语开始。把 RPC、CAP、租约、逻辑时钟、幂等、背压按字典顺序列出来，读者很快就会背会几个名字，却仍然不知道一段程序为什么会错。本章从一段会在小样例里通过、在故障注入里立刻出错的代码开始。Driver 把 split 7 发给 worker，worker 已经把结果写到临时 block，成功响应在返回途中丢失。Driver 等不到响应，按本地 timeout 重新派发同一个 task。几秒后，旧响应和新响应先后到达，两个物理执行都声称自己完成了 task 7。

先把坏代码摆出来，后面每个机制都从这段代码的具体漏洞里推出来。

Listing 16.1 反例：把远端调用当成本地函数

```
1 enum class RemoteStatus : std::uint16_t {
2     Ok,
3     Timeout,
4     Fatal,
5 };
6
7 struct RemoteReply {
8     RemoteStatus status = RemoteStatus::Fatal;
9     std::uint32_t task_id = 0;
10    std::uint64_t block_id = 0;
11    std::uint32_t checksum = 0;
12 };
13
14 bool run_task_once(std::uint32_t task_id) {
15     RemoteReply reply = run_remote(task_id);
16     if (reply.status == RemoteStatus::Timeout) {
17         mark_failed(task_id);
18         return false;
19     }
20     if (reply.status != RemoteStatus::Ok) {
21         mark_failed(task_id);
22         return false;
23     }
24     append_manifest(reply.task_id, reply.block_id, reply.checksum);
25     return true;
26 }
```

这段代码的错误不是“没有用某个高级框架”，而是把本地调用栈的保证带到了远端世界。第一处漏洞在 timeout 分支：Driver 只知道本地等待结束，不知道请求是否到达、worker 是否开始执行、输出是否已经落盘。第二处漏洞在 retry：重新发送同一个 task 不是简单再试一次，而是制造了新的物理副作用。第三处漏洞在响应处理：Ok 只说明某个远端执行声称成功，不说明它属于当前尝试、当前 Driver、当前 deadline，也不说明它有权进入 manifest。第四处漏洞在恢复：进程重启以后，内存里的“失败”或“运行中”都消失了，但旧 block、旧响应和旧 worker 仍然可能存在。

因此，本章不是先教一组名词，再给例子套名词；而是反过来读这段坏代码。先问 timeout 到底证明了什么，再问 retry 新增了什么副作用，再问成功响应怎样被验明身份，最后问崩溃恢复时谁是权威事实。沿着这条审查线，消息身份、物理 attempt、逻辑 deadline、提交表、epoch、trace、背压和恢复日志会自然出现。读者要掌握的不是“分布式系统很复杂”这句话，而是当调用跨过进程、

机器或网络边界时，原来由调用栈、共享内存和同步返回值提供的保证必须被状态表、消息字段和提交边界重新写出来。

16.1 从一次响应丢失开始

先按代码审查的方式跑一遍失败。正常路径里，Driver 取出 split 7，调用 `run_remote(split)`，worker 读取输入、统计 key、写出一个 block，再返回 block 的路径和 checksum。小样例能跑，日志也很干净：发送、执行、返回、提交。很多坏设计都在这种正常路径里显得合理，因为还没有任何事件打破本地调用直觉。

现在只改变一件事：worker 的业务代码全部成功，只有成功响应在返回途中丢失。Driver 看到 timeout，于是执行刚才反例里的 `mark_failed(task_id)`。这里第一处错误已经发生了。timeout 是本地 timer 事件，不是远端事实。请求可能根本没有到达；请求可能已经进入 worker 队列但尚未执行；worker 可能已经执行完成但响应丢失；worker 可能正在发送响应；worker 可能崩溃；网络可能只是慢。把这些情况都写成 Failed，就是把“我不知道”伪装成“它失败了”。

再让 Driver 重试。重试不是撤销第一次执行后再试一次，第一次执行可能已经写出了 block。若第二次执行也成功，临时目录会出现两个候选输出。此时程序如果让 reduce 扫描目录，就会把同一个 split 统计两次；如果让 `append_manifest` 无条件追加，也会制造重复提交。于是提交表不是凭空出现的概念，它是从“重复执行可以发生，但外部结果只能生效一次”这个缺口里长出来的。

接着让旧响应迟到。响应里只有 `task_id`、`block_id` 和 `checksum`，Driver 无法判断它属于第几次物理执行，无法判断它是否来自重启前的 Driver，无法判断它是否已经过了逻辑 deadline，也无法判断它是否是重复包。于是 `request_id`、`attempt`、`epoch`、`deadline` 和 `trace` 不是装饰字段，而是响应进入状态机前必须携带的证据。缺一个字段，就会有一个故障位置只能靠猜。

最后让 Driver 在 manifest 写入前后各崩溃一次。写入前崩溃，临时 block 存在但没有权威提交记录，恢复时不能自动接纳；写入后崩溃，内存状态可能丢失，但 manifest 已经是外部可见事实，恢复时必须承认它。这个对照把提交点讲清楚：worker 执行成功不是提交，响应到达不是提交，只有权威记录接受某个 task 的某个 attempt 才是提交。

这里有一个关键判断：Exactly once 不能解释成“远端函数只执行一次”。在真实故障下，执行可能发生零次、一次或多次。工程上更可实现的目标是外部可见提交只生效一次。Worker 可以重复算，临时文件可以重复写，响应可以重复到达，但最终 manifest 只能接受一个结果。这个目标听起来弱一些，却是系统能落地的正确性边界。

本地调用直觉为什么失效 本地调用的失败边界很窄。函数没被调用、函数抛异常、函数返回值错误，三者通常能在同一进程里用断点和栈跟踪观察。远端调用的失败边界被拆到多个位置：客户端序列化、发送队列、网络、服务端接收队列、业务线程、输出写入、响应队列、客户端完成回调。每一段都可能排队，也都可能在“对方已经做了事”以后才失败。

这也是为什么 RPC 库不能把远端调用变成本地函数。RPC 能隐藏编码、连接池和线程调度的一部分样板代码，但不能消除不确定性。调用者必须明确写出：请求身份是什么，超时以后进入什么状态，是否允许重试，重复请求如何处理，迟到响应如何处理，远端输出如何提交，取消是否影响远端提交。没有这些答案，程序只是把分布式错误藏在库调用后面。

超时表示未知状态 很多初学程序把 timeout 当作 failure。这个写法在本章故事里马上失败。响应丢失以后，worker 的输出可能已经存在，甚至已经满足所有业务条件；Driver 只是没等到响应。如果 Driver 直接把 task 标为 Failed，然后允许后续 attempt 无条件提交，旧 attempt 的输出就变成了无人管理的副作用。若旧响应迟到，程序还会面临“已经失败的任务为什么又成功了”的状态冲突。

更稳的状态机要把 TimedOut 当作 Unknown 的一种表现。TimedOut 后可以重试，可以查询，可以降级，也可以放弃，但不能凭本地计时推断远端事实。Worker 也要理解 deadline：收到过期请求时不应继续启动昂贵计算；正在执行的长任务应在安全点检查 deadline；已经写出但未提交的结果要等待 Driver 的提交决定。Deadline 和 timeout 的区别在这里出现：timeout 是某一层本地等待多久，deadline 是这次逻辑请求最晚还能被接受到什么时候。

消息必须携带自己的身份 本地返回值靠调用栈归属，远端响应靠字段归属。最小消息信封至少要带协议版本、消息类型、job、stage、task、attempt、request、epoch、deadline、trace、payload 长度和 checksum。字段看起来很多，但每个字段都对应一个失败问题。没有 task，响应不知道属于

哪个逻辑工作单元；没有 attempt，迟到响应和当前尝试无法区分；没有 request，重复消息无法去重；没有 epoch，Driver 重启或路由切换后的旧消息无法拒绝；没有 trace，跨组件日志无法串联。

Listing 16.2 消息信封的教学版字段

```

1 enum class MessageKind : std::uint16_t {
2     TaskRequest = 1,
3     TaskReply = 2,
4     Busy = 3,
5     Cancel = 4,
6 };
7
8 enum class RpcStatus : std::uint16_t {
9     Ok = 0,
10    Retryable = 1,
11    Busy = 2,
12    StaleEpoch = 3,
13    DeadlineExpired = 4,
14    InvalidRequest = 5,
15    Fatal = 6,
16    Unknown = 65535,
17 };
18
19 struct MessageEnvelope {
20     std::uint16_t protocol_version = 1;
21     MessageKind kind = MessageKind::TaskRequest;
22     RpcStatus status = RpcStatus::Ok;
23     std::uint32_t job_id = 0;
24     std::uint32_t stage_id = 0;
25     std::uint32_t task_id = 0;
26     std::uint32_t attempt = 0;
27     std::uint64_t request_id = 0;
28     std::uint64_t trace_id = 0;
29     std::uint32_t driver_epoch = 0;
30     std::uint64_t deadline_ns = 0;
31     std::uint32_t payload_bytes = 0;
32     std::uint32_t checksum = 0;
33 };

```

这个结构不是直接写到网络上的二进制格式。真实 wire format 要明确大小端、变长字段、对齐、版本兼容和校验规则。这里用 C++ 结构体只是为了把语义字段摆清楚。教学项目可以在本机进程内复制这个结构；一旦进入 socket、文件或跨版本存储，就应定义按字节解析的协议，而不是把结构体内存直接发送出去。

16.2 把调用栈改写成状态表

事故发生以后，调试最容易陷入日志碎片。Driver 日志写“task 7 timeout”，worker 日志写“task 7 done”，MessageBus 日志写“drop reply”，manifest 日志写“commit attempt 1”。如果没有统一状态表，四段日志互相矛盾。调用栈消失以后，状态表就是新的事实中心。

状态表的第一条规则是区分逻辑任务和物理尝试。Task 7 是逻辑工作单元，attempt 0 和 attempt 1 是两次执行尝试，request 1001 是一次消息交互，trace 9001 是跨组件观测链。它们不能混成一个 id。混在一起会产生两类错误：重试时换 request id 导致服务端无法去重；迟到响应只按 task id 匹配导致旧 attempt 覆盖新 attempt。

状态表的第二条规则是终态要明确。Committed 表示外部结果已经生效；Failed 表示系统决定不再接受本次逻辑任务的自动成功；Abandoned 表示上游已经放弃等待但远端事件仍可能迟到。终态以后，迟到响应不能重新打开状态，除非进入人工恢复流程且带新的 epoch。很多分布式 bug 都来自“失败以后又成功”“取消以后又回调”“重启以后旧 leader 又写入”这类终态不清。

状态表的第三条规则是响应处理必须先验证身份，再决定动作。收到响应后，Driver 不应直接写 manifest。它要先按 task 找到逻辑状态，再比较 request、attempt、epoch 和 deadline，然后检查输出 checksum，最后才尝试提交。任何一项不匹配，都只能进入 late、stale、duplicate 或 invalid 指标，不能改变最终结果。

状态枚举表达允许的转移 教学代码可以先用一个紧凑状态机表达。状态不需要覆盖工业系统的全部细节，但必须足以解释本章事故。Pending 表示尚未发送；Sent 表示消息已交给 bus；Running 表示 worker 已开始或 Driver 认为它可能正在执行；TimedOut 表示本地等待结束但远端未知；Retrying 表示新 attempt 已创建；Committed 表示结果已发布；Failed 表示自动恢复失败；Abandoned 表示上游不再等待。

Listing 16.3 Driver 侧的任务状态记录

```

1 enum class TaskState : std::uint8_t {
2     Pending,
3     Sent,
4     Running,
5     TimedOut,
6     Retrying,
7     Committed,
8     Failed,
9     Abandoned,
10 };
11
12 struct TaskRuntimeState {
13     std::uint32_t job_id = 0;
14     std::uint32_t stage_id = 0;
15     std::uint32_t task_id = 0;
16     std::uint32_t current_attempt = 0;
17     std::uint64_t active_request_id = 0;
18     std::uint32_t driver_epoch = 0;
19     std::uint64_t deadline_ns = 0;
20     TaskState state = TaskState::Pending;
21     std::uint32_t retry_count = 0;
22     std::uint32_t late_reply_count = 0;
23     std::uint32_t stale_reply_count = 0;
24 };

```

这段代码里使用固定宽度整数，是为了让日志、文件格式和跨进程消息的边界更清楚。任务编号、attempt、epoch、payload 大小、deadline 都不是“随便一个 int”。当这些字段写入 manifest 或传给另一个进程时，位宽就是协议的一部分。第二册所有涉及跨边界状态的示例都应保持这种习惯。

响应处理的顺序 响应处理函数的结构要体现推理顺序。先验证协议版本和 checksum，防止把损坏消息当业务结果。再验证 epoch，拒绝旧 Driver 或旧路由产生的消息。然后验证 task 和 request，确认它确实属于当前逻辑任务。接着比较 attempt，判断它是当前 attempt、重复响应还是迟到响应。最后才进入提交路径。

Listing 16.4 响应处理的最小判定流程

```

1 enum class ReplyDecision : std::uint8_t {

```

```

2   AcceptForCommit,
3   DuplicateOfCommitted,
4   LateAttempt,
5   StaleEpoch,
6   DeadlineExpired,
7   CorruptMessage,
8   UnknownRequest,
9 };
10
11 ReplyDecision classify_reply(const MessageEnvelope& reply,
12                             const TaskRuntimeState& task,
13                             std::uint64_t now_ns) {
14     if (reply.checksum == 0) {
15         return ReplyDecision::CorruptMessage;
16     }
17     if (reply.driver_epoch != task.driver_epoch) {
18         return ReplyDecision::StaleEpoch;
19     }
20     if (reply.task_id != task.task_id ||
21         reply.request_id != task.active_request_id) {
22         return ReplyDecision::UnknownRequest;
23     }
24     if (now_ns > task.deadline_ns) {
25         return ReplyDecision::DeadlineExpired;
26     }
27     if (reply.attempt != task.current_attempt) {
28         return ReplyDecision::LateAttempt;
29     }
30     if (task.state == TaskState::Committed) {
31         return ReplyDecision::DuplicateOfCommitted;
32     }
33     return ReplyDecision::AcceptForCommit;
34 }

```

真实代码还要查询 commit table，不能只看内存里的 `TaskRuntimeState`。但这个流程已经展示核心纪律：成功响应不是直接成功，响应只是一个事件。事件进入状态机以后，状态机根据当前事实决定它是否还有权改变结果。这个思路会在下一章变成分片 epoch、leader term 和 commit index。

Trace 替代跨进程调用栈 调试本地函数时，调用栈告诉你谁调用了谁。分布式系统中，Driver、MessageBus、worker、manifest writer 分散在不同线程、进程或机器里，栈不再连续。Trace 的任务就是把一次逻辑请求的事件串起来。每个事件至少写 trace、request、attempt、component、state、time 和 reason。响应丢失事故的 trace 应能看到 send、deliver、execute、write temp block、drop reply、timeout、retry、commit、late reply。

Trace 不是为了日志好看。它直接决定事故能否复盘。只有全局平均延迟，无法知道慢在网络、队列、执行还是提交。只有 worker 日志，无法知道 Driver 是否已经放弃。只有 Driver 日志，无法知道 worker 是否写过输出。Trace 把这些局部事实按身份连起来，变成可审查的事件序列。

16.3 幂等提交点：重复执行和唯一生效

响应丢失事故最危险的地方不是请求失败，而是重复提交。失败通常会暴露；重复提交可能让结果悄悄变错。日志统计作业中，同一个 map task 的两个 block 都进入 manifest，reduce 就会把同一份输入统计两次。输出仍然有格式，程序也可能返回成功，只是数值已经错了。

幂等性不能解释成“函数执行多次结果一样”。许多计算函数本身并不幂等：写文件、追加日志、更新计数、扣库存、创建订单都会产生副作用。工程上要把副作用推迟到提交边界，或者让提交边界能识别重复。Worker 可以把 attempt 输出写到临时位置，这个动作允许重复；Driver 把某一个 block 写入 manifest，这个动作必须唯一。唯一性发生在 commit，不发生在 worker 计算本身。

本章项目可以用 manifest 作为提交权威。Worker 输出路径包含 job、stage、task、attempt 和 checksum。Driver 收到响应以后先校验 block，再尝试向 manifest 写入一条 **ManifestEntry**。若 task 已经有 accepted attempt，新响应只能被记录为 late 或 duplicate。Reduce 只读取 manifest 中的 accepted block，而不扫描临时目录。这样临时目录里即使存在多个 attempt 输出，也不会进入最终结果。

幂等表保存结果而不只是保存见过的 ID 很多错误实现只保存一个 set：见过 **request_id** 就拒绝重复。这不够。若第一次请求已经完成但响应丢失，客户端重试时需要拿到同一语义结果；只知道“见过”不能告诉客户端上次结果是什么。幂等表应保存状态、accepted attempt、结果摘要、错误分类和过期策略。对于计算作业，结果摘要可以是 block id、checksum、record count 和 output bytes。

Listing 16.5 提交表记录外部可见结果

```
1 enum class CommitState : std::uint8_t {
2     Empty,
3     Preparing,
4     Committed,
5     Rejected,
6 };
7
8 struct CommitRecord {
9     std::uint32_t job_id = 0;
10    std::uint32_t stage_id = 0;
11    std::uint32_t task_id = 0;
12    std::uint32_t accepted_attempt = 0;
13    std::uint64_t request_id = 0;
14    std::uint64_t block_id = 0;
15    std::uint32_t checksum = 0;
16    std::uint32_t output_bytes = 0;
17    CommitState state = CommitState::Empty;
18 };
```

提交表的持久化策略决定恢复语义。若它只存在内存里，Driver 重启后就会忘记哪些 task 已提交，旧 worker 的响应可能再次进入 manifest。教学版可以把 manifest log 作为权威状态，恢复时从 log 重建 commit table；工业系统可能使用事务日志、复制状态机或数据库事务。无论实现多简化，都必须说清哪个文件或日志是提交事实。

提交必须和结果发布在同一边界 把 request id、业务状态和响应结果分开写，会产生崩溃窗口。若先把 block 写入 manifest，再更新 commit table，中途崩溃后恢复可能不知道这条 manifest 是否已被接受。若先更新 commit table，再写 manifest，恢复后可能以为已提交，但 reduce 看不到 block。一个稳妥的教学方案是让 manifest entry 本身携带 task、attempt、request、checksum 和状态，恢复时只相信完整、校验通过的 manifest entry。

文件系统实现可以采用临时文件加原子 rename 的模型。先把 attempt 输出写到临时路径，写完后计算 checksum。Driver 构造 manifest entry，追加到 manifest log，必要时按实验承诺调用同步；恢复时读取完整 entry，校验 block 是否存在且 checksum 一致。这样外部可见的结果由 manifest 决定，临时输出只是候选。这个设计也和单机 I/O 章节的崩溃点矩阵衔接起来。

重复请求和重复响应是不同问题 重复请求发生在 worker 侧。Worker 收到同一个 **request_id** 或同一个 task attempt 时，要么返回保存结果，要么拒绝过期 attempt，要么把请求转给当前 owner。重复响应发生在 Driver 侧。Driver 已经提交或放弃以后，旧响应又到达，它不能再次触发

提交。两个方向都需要身份字段，但状态表不同。把它们都叫“重复消息”会掩盖处理差异。

本章实验要分别覆盖三个单点。第一，请求丢失：worker 没执行，重试应正常完成。第二，响应丢失：worker 已执行，重试必须依赖幂等表或唯一 commit。第三，响应迟到：Driver 已经进入终态，旧响应只能进入指标和清理路径。单点问题单点分析，读者才能看到每个概念解决哪一个信息缺口。

16.4 Deadline、重试预算和背压

如果只有一个请求，重试看起来总是合理；如果有一万个请求同时遇到慢 worker，重试就可能成为攻击自己的流量。慢节点已经排队，Driver 等不到响应，于是把同样的任务发给更多 worker。更多 worker 也开始排队，响应更慢，Driver 继续重试。这个过程就是重试风暴。它不是网络层的小概率问题，而是分布式计算引擎必须设计的控制回路。

从事故推导控制字段也很自然。Deadline 让每一层知道这次逻辑请求还能花多久，避免每层都重新给自己完整 timeout。Retry budget 限制一次逻辑请求最多制造多少 attempt，避免多层调用把请求量乘起来。Backpressure 让慢或满的下游把压力传回上游，而不是让上游继续投递。Jitter 让大量客户端不要在同一时刻重试。Circuit breaker 让持续失败的下游获得恢复时间。

Deadline 不是局部 timeout 局部 timeout 只描述当前等待动作。Driver 等 worker 500 ms，worker 等磁盘 500 ms，manifest writer 又等 500 ms，三层相加已经超过用户预算。Deadline 是绝对截止点，随消息传递。每一层读取当前单调时间，计算剩余预算，再决定执行、排队、降级或拒绝。跨机器 deadline 需要考虑时钟偏差，但它仍然比每层独立 timeout 更适合表达整体请求预算。

本机模拟中，deadline 可以用同一进程的单调时间表示。进入真实多机以后，应避免用墙钟时间证明安全顺序。墙钟适合日志和观测，协议新旧更应依赖 epoch、term、version 和 log index。Deadline 主要控制资源消耗，不应被滥用成一致性证明。

重试预算按逻辑请求计算 预算要绑定逻辑 request，而不是每个库调用各自决定。一个上游请求调用十个下游，每个下游重试三次，看起来每层都很克制，整体尝试数却可能显著放大。若下游已经过载，这些额外尝试只会增加队列长度和尾延迟。预算应记录原始请求数、attempt 数、busy 数、timeout 数、最终成功和最终失败。没有这些指标，重试策略无法评价。

Listing 16.6 重试预算和退避状态

```
1 struct RetryBudget {
2     std::uint32_t max_attempts = 3;
3     std::uint32_t used_attempts = 0;
4     std::uint64_t deadline_ns = 0;
5     std::uint64_t next_retry_ns = 0;
6 };
7
8 bool can_retry(const RetryBudget& budget, std::uint64_t now_ns) {
9     if (budget.used_attempts >= budget.max_attempts) {
10         return false;
11     }
12     if (now_ns >= budget.deadline_ns) {
13         return false;
14     }
15     return now_ns >= budget.next_retry_ns;
16 }
```

退避和抖动不是为了让系统显得复杂。固定间隔重试会让大量请求在同一时间点再次打到慢节点；指数退避拉开连续失败后的间隔；抖动打散不同客户端的重试峰值。代价是恢复时间可能变长，所以预算、退避和业务 deadline 要一起调。

背压把资源限制写进协议 服务端队列满时，不能只让消息在内存里无限堆积。Inbox 容量、每 worker in-flight 窗口、每 job 字节窗口、连接池上限都是背压边界。队列满可以返回 Busy，可以延

迟低优先级消息，也可以拒绝新请求。无论选择哪一种，都要让上游知道这是过载信号，而不是普通 fatal 错误。把 Busy 当普通失败立即重试，会把背压信号反向放大。

流控要同时按任务数和字节数看。一个小 task 和一个大 shuffle block 都是一条消息，但内存和网络成本差别很大。教学项目可以先实现两个窗口：每个 worker 最多 N 个 in-flight task，每个 worker 最多 M 字节 in-flight payload。响应返回后释放窗口，Busy 响应降低窗口，连续成功可以慢慢恢复窗口。这个机制把单机队列章节的思想扩展到跨 worker 消息。

慢节点不是失败节点 故障检测常常只能给出怀疑。Worker 心跳超时，可能是进程崩溃，也可能是磁盘卡顿、CPU 被抢占、长 GC、网络拥塞或队列太长。把慢节点立即当死节点，会制造重复 attempt；把死节点长期当慢节点，会拖慢恢复。正确做法是把检测结果写成状态：Healthy、Busy、Suspect、Draining、Dead，并让调度和重试按状态选择不同动作。

Speculative execution 和 hedged request 也要有边界。它们能降低尾延迟，却会制造重复执行。只有当幂等提交和预算已经建立，才适合加入 speculation。否则每个慢分片都可能变成多份输出，最终结果靠运气不重复。

16.5 本机 MessageBus 的最小语义

本册不要求一开始就搭真实集群。用本机 MessageBus 可以训练分布式语义：Driver 和 worker 不共享业务对象，只通过消息通信；消息进入 bus 后可以延迟、丢弃、重复、乱序；每个 endpoint 有有界 inbox；Timer 产生 timeout 和 retry 事件；TraceLog 记录所有状态变化。这个模拟不能证明真实网络性能，却能把协议状态讲清楚。

MessageBus 最容易写错成函数调用包装。若 send 直接调用 receiver 的处理函数，调用栈又回来了，分布式不确定性被掩盖。正确的教学实现应让 send 只把消息复制或移动到网络队列，然后由事件循环按规则投递。Driver 不能知道 worker 何时处理，worker 也不能依赖 Driver 栈上的对象还活着。发送后 payload 的所有权必须明确，要么复制，要么转移到共享缓冲对象；不能保存指向调用者栈内存的裸指针。

故障规则是核心功能 没有故障注入的 MessageBus 只是多线程队列。本章至少需要 drop request、drop reply、delay、duplicate、reorder、busy 和 corrupt 几类规则。规则要可复现，不能靠“随机跑几次”。报告中应能写出：丢弃 request 1001 的第一次 reply，延迟 attempt 0 的响应 300 ms，重复投递 attempt 1 的请求一次。只有这样，失败测试才能重放。

Listing 16.7 可复现故障规则的教学形态

```
1 enum class FaultAction : std::uint8_t {
2     Drop,
3     Delay,
4     Duplicate,
5     Reorder,
6     CorruptChecksum,
7     ReturnBusy,
8 };
9
10 struct FaultRule {
11     std::uint64_t match_request_id = 0;
12     std::uint32_t match_attempt = 0;
13     MessageKind match_kind = MessageKind::TaskReply;
14     FaultAction action = FaultAction::Drop;
15     std::uint32_t trigger_limit = 1;
16     std::uint32_t triggered = 0;
17     std::uint64_t delay_ns = 0;
18 };
```

故障规则还要标明注入点。丢请求和丢响应不同；worker 执行前崩溃和写出后崩溃不同；Driver

commit 前崩溃和 commit 后崩溃不同。单点注入的价值在于把故障位置和恢复动作一一对应。等单点都清楚后，再做组合故障才有意义。

Timer 驱动状态而不是业务线程 sleep 超时不应写成业务线程 sleep 一段时间后随便改状态。Timer 应该根据 deadline 和 `retry_at` 产生事件。Timer 事件到来时，状态机检查 request 是否已经完成，是否还有预算，是否可以创建新 attempt。若响应先到，Timer 事件应发现状态已变成 Committed 或 Failed，然后忽略。这样 timeout 和 reply 成为两个竞争事件，正好模拟真实分布式系统的不确定性。

Event loop 的日志要记录事件来源。TimeoutEvent、ReplyEvent、BusyEvent、CancelEvent、ShutdownEvent 都是状态机输入。把它们统一输入状态表，会比在多个回调里写 if 更容易审查。后续进入真实 epoll、`io_uring` 或 RPC 框架时，底层事件不同，但状态机可以复用。

Inbox 有界才能看到背压 每个 endpoint 的 inbox 都应有容量。满了以后，bus 返回 Busy 或把消息放入延迟队列。无限队列会让测试永远看不到过载，只在内存耗尽时崩溃。容量不是随便的数字，报告要写选择依据：worker 数、平均 payload、可接受排队时间、内存预算。若队列水位长期高而吞吐不升，说明系统已经过载；若 Busy 数上升但 retry 不下降，说明客户端没有尊重背压。

本机实现可以把指标写成纯文本：sent、delivered、dropped、duplicated、busy、inflight、queue depth、late replies、stale replies。EPUB 中保留文本即可，不需要图。读者能从这些指标看出故障怎样传播，比看一张静态示意图更有用。

16.6 恢复、epoch 和持久化最小集

分布式系统最怕“进程重启以后什么都忘了”。Driver 重启会丢失内存中的 in-flight 表，但 worker 可能仍在运行，旧响应可能稍后回来，临时 block 可能已经写在磁盘上。如果没有持久状态，Driver 只能重新全量执行；如果没有 epoch，旧响应可能被新 Driver 当作当前响应；如果没有 manifest 校验，临时目录里的旧文件可能被误读成有效输出。

恢复要先定义最小持久集。对本章计算项目，至少包括 job id、driver epoch、stage id、task table、commit table、manifest、输入 split 描述、输出 block checksum 和协议版本。Task table 告诉 Driver 哪些任务未完成；commit table 防止重复提交；manifest 告诉 reduce 哪些 block 有效；epoch 拒绝旧 Driver 或旧路由产生的响应；checksum 防止损坏输出进入结果。

Driver epoch 区分新旧控制面 Driver 每次恢复或领导权变化，都应获得新的 epoch。消息里带 epoch，worker 响应也带回 epoch。新 Driver 收到旧 epoch 响应时，不能直接提交；它要根据恢复日志判断旧 attempt 是否仍然属于当前作业。简单教学版可以拒绝所有旧 epoch 响应，并重新派发未提交 task。更复杂系统可以查询旧 attempt 输出并纳入恢复，但必须有明确协议。

Listing 16.8 恢复时需要重建的持久任务记录

```

1 enum class PersistedTaskState : std::uint8_t {
2     Pending,
3     InFlight,
4     Committed,
5     Failed,
6 };
7
8 struct PersistedTaskRecord {
9     std::uint32_t job_id = 0;
10    std::uint32_t stage_id = 0;
11    std::uint32_t task_id = 0;
12    std::uint32_t last_attempt = 0;
13    std::uint32_t driver_epoch = 0;
14    std::uint64_t input_split_id = 0;
15    std::uint64_t committed_block_id = 0;
16    std::uint32_t committed_checksum = 0;
17    PersistedTaskState state = PersistedTaskState::Pending;

```

18 };

持久文件格式也要有版本和校验。教学版可以用一行一条记录的文本格式，但每行应包含 version、record kind、字段和 checksum。恢复时遇到半行、未知版本或 checksum 不匹配，不能悄悄忽略。恢复程序必须把“读到了什么、丢弃了什么、需要重试什么”写进日志。

临时目录要和 manifest 对账 恢复不只是读 task table。还要扫描 manifest 引用的 block，确认文件存在且 checksum 正确；扫描临时目录中未引用 block，判断它们属于旧 attempt、当前 attempt 还是损坏残留。未引用 block 可以延迟清理，但不能进入 reduce。这个对账过程把单机 I/O 崩溃恢复和分布式幂等提交连接起来。

如果 manifest 指向的 block 不存在，系统不能假装成功。它应把 task 标为需要恢复或 job 标为失败，取决于输入是否可重算。若临时目录有一个 checksum 正确但未在 manifest 中的 block，也不能自动接纳，因为它可能来自旧 attempt 或旧 epoch。分布式系统里，文件存在不是提交事实；manifest 才是提交事实。

恢复测试要故意切在提交边界 测试恢复不能只在正常结束后重启。要在几个位置故意崩溃：发送请求后但未记录 in-flight；worker 写出临时 block 后但响应丢失；Driver 校验 block 后但 manifest 未写；manifest 写入后但内存状态未更新；Driver 返回成功前崩溃。每个位置恢复后的行为都要写清。若某一格写不出来，说明提交边界仍然模糊。

这些崩溃点和下一章的共识日志是一条线。单机教学项目用 manifest log 做权威；分布式元数据服务可能用 Raft log 做权威。无论规模如何，问题都是：哪一条记录表示状态已经对外生效，恢复时谁说了算。

16.7 消息边界、序列化和所有权

响应丢失事故只讲了控制面身份，还没有讲清消息里的数据怎样跨边界移动。很多本机模拟写得不真实，问题就在这里：Driver 构造一个对象，把对象指针交给 worker；worker 直接读指针里的 vector；测试通过以后，作者以为自己实现了消息系统。实际上这还是共享内存程序，只是换了函数名。真正的消息一旦发送，接收方看到的应是发送时刻的独立字节序列或独立拥有权对象，而不是发送方栈上的一段可变内存。

这个边界会立即影响正确性。Driver 发送请求后，如果继续修改 payload，worker 不能看到修改后的内容；Driver 超时后释放请求上下文，worker 迟来处理时也不能访问悬垂引用；MessageBus 注入延迟时，消息必须能在队列里独立存活；故障重放时，历史消息应能按当时内容再次投递。若 MessageBus 只是保存指针，这些实验都被本机便利掩盖。到了真实 socket、共享内存 IPC 或多进程环境，错误才集中爆发。

发送边界：复制、移动和引用计数 教学项目可以从最保守的复制语义开始。Driver 调用 send 时，Envelope 和 payload 被复制到 bus 队列；send 返回以后，Driver 可以释放原对象。复制会增加成本，但语义最清楚，适合先把故障矩阵跑通。第二版可以改成移动语义：payload 放进拥有型 buffer，send 后发送方不再使用。第三版才考虑引用计数 buffer 或 scatter-gather，把大块数据避免重复复制。每前进一步，都要证明生命周期仍然闭合。

零拷贝不是“没有成本”，而是把成本从复制转成生命周期管理。Buffer 必须在完成事件之前保持不变，发送方不能提前复用；若发送失败或取消，buffer 由谁释放要清楚；若同一 payload 被多个 receiver 读取，是否允许共享也要写明。网络栈、文件系统和 DMA 都会遇到同一问题：硬件或内核尚未完成时，用户态不能把内存当成已经空闲。这个原则和异步 I/O 章节完全一致。

协议格式不能等同于 C++ 对象布局 教学代码用结构体表达字段，但持久化或跨机器时不能直接写结构体内存。C++ 对象布局可能含 padding，不同平台大小端不同，枚举底层类型若未固定会变化，编译器 ABI 也不应成为协议合同。稳定协议要按字段编码：先写 version，再写 kind，再写固定宽度整数，再写 payload 长度和 checksum。读取时按 version 判断字段是否存在，未知字段如何跳过也要定义。

这个要求不是形式主义。Driver 重启后要读取旧 manifest，新版本程序可能多了 trace 字段；worker 滚动升级时，旧 worker 可能收不到新字段；EPUB 或 PDF 里的代码只是教学示例，真实项目要把协议作为单独模块测试。协议测试至少包括旧版本消息、新版本消息、缺失字段、payload length 不匹配、checksum 错误和未知 kind。这样“版本”就不再是消息头里的装饰，而是升级和恢

复时的安全边界。

序列化成本要进性能账本 序列化会访问字段、分配 buffer、复制字节、计算 checksum，可能还要压缩。小消息太多时，固定头部和调度成本占主导；大消息太大时，内存峰值、重传成本和尾延迟上升。分布式系统的性能报告如果只写业务函数耗时，就会漏掉很大一块。一次 task request 的端到端时间至少要拆成 encode、client queue、bus delay、server queue、execute、write output、encode reply、reply queue、client handle。

批处理是常见优化，但也要从失败成本分析。把一百条小消息合成一批，可以减少头部和调度成本；若这批最后一条记录损坏，是否要重传整批；若批里一半 task 已过 deadline，是否还继续发送；若响应丢失，幂等表按整批去重还是按单个 task 去重。批处理不是简单把 vector 包起来，它改变了错误边界和重做范围。

控制消息和数据消息分开 控制消息要小、及时、可靠地表达状态；数据消息要高吞吐地搬运字节。心跳、Busy、Cancel、CommitAck、RouteRefresh 属于控制消息，shuffle block、input split、partial result 属于数据消息。若控制消息和大数据消息共用一个无优先级队列，大 block 会阻塞 cancel 和 busy，系统过载时更难恢复。若所有数据都拆成控制消息大小，又会浪费吞吐。

本机 MessageBus 可以用两个队列模拟这个区别：control inbox 容量小但优先投递，data inbox 按字节窗口限流。Driver 收到 Busy 控制消息以后降低 data in-flight；worker 写出大 block 时不阻塞 heartbeat；关闭时先发送 control close，再 drain data 或取消。这样读者能看到，分布式系统不是“消息越统一越好”，而是统一身份字段，区分资源路径。

16.8 故障矩阵：从单点位置推导处理规则

分布式章节最怕一句“网络会失败”带过所有问题。网络失败、进程失败、磁盘失败、超时和慢节点不是同一种事。即使都表现为 Driver 没收到响应，恢复动作也可能完全不同。高质量教材必须把故障按位置拆开：请求生成前、请求发送后、请求到达前、请求入队后、执行前、执行中、输出写入前、输出写入后、响应发送前、响应返回中、Driver commit 前、Driver commit 后。每个位置都改变已知事实。

请求未到达：重试为什么相对简单 请求在进入 worker 之前丢失，worker 没有任何副作用。Driver 超时后用同一 request 创建新 attempt，通常可以安全重试。这里仍然不能换逻辑 request id，因为 Driver 不能证明旧请求未到达；只是从故障注入规则里我们知道它未到达。真实系统里，客户端永远只能看到 timeout，不能看到“未到达”这个事实。因此代码路径仍按 Unknown 处理，而测试矩阵把它标成请求未到达，用于验证最简单恢复。

这个场景的指标应显示：worker 没有 output，manifest 没有 entry，retry count 加一，最终只有一个 attempt 提交。若临时目录出现旧 block，说明注入点并非请求未到达；若 manifest 有两条，说明提交表没有按 task 去重。故障矩阵的价值就在于它能反过来检查测试是否真的命中了目标位置。

请求到达但未执行：队列和 ownership 进入判断 请求进入 worker inbox 后，worker 可能因为队列长、进程崩溃或被关闭而没有执行业务。Driver 看到的仍然是 timeout，但处理多了一层状态归属。若 task 可以在任意 worker 执行，Driver 可以创建新 attempt；若 task 绑定某个 partition owner，必须先确认 owner 是否仍然合法；若旧 worker 恢复后仍持有旧 epoch 请求，它应拒绝或重新校验，而不是继续执行。

这个场景连接到下一章。单个 task 的 retry 看起来简单，一旦 task 修改某个分片状态，就不能随便换 worker。Ownership epoch 让系统知道“谁现在有权执行”。本章只要求 Driver epoch，下一章扩展成 route epoch 和 leader term。读者应看到字段如何从事故中自然长出来。

执行完成但响应丢失：幂等表必须回答旧结果 这是本章核心场景。Worker 已经写出输出并记录可返回结果，响应丢失。Driver 超时重试以后，服务端若再次执行，可能产生重复输出；若只回复“见过 request”，客户端又拿不到结果。正确做法是让幂等表保存结果摘要，重复请求到达时返回同一语义结果。若原 worker 崩溃，恢复程序也要能从 manifest 或 commit log 找回结果。

注意这里的“同一结果”不一定是同一个物理文件。第一次 attempt 写 block A，第二次 attempt 写 block B，只要 manifest 只接受一个，最终语义仍可正确。更节省的做法是第二次请求直接返回 block A 的摘要，不再执行。选择哪一种取决于结果是否容易查询、临时文件是否可靠、重复执行成本多高。教材要给出这个权衡，而不是只写“做幂等”。

响应迟到：终态不允许被旧事件重开 响应迟到时，Driver 可能已经提交新 attempt，可能已

经向用户返回失败，也可能进入降级模式。迟到响应不是垃圾，它是一个需要分类的事件。若它属于已提交 attempt，可以记为 duplicate ack；若属于旧 attempt，记为 late attempt 并清理临时输出；若属于旧 epoch，记为 stale epoch；若 checksum 不匹配，记为 corrupt reply。任何一种都不能直接写 manifest。

这条规则和本地异步 completion 相同。取消以后 completion 仍可能到达，关闭以后队列仍可能有旧事件，future 放弃以后任务可能仍运行。第二册反复强调这件事，是因为它跨越所有层次：完成事件不等于业务仍然需要它，旧事件必须经过状态表重新判定。

Driver commit 前后崩溃：权威记录决定恢复 Driver 校验 block 后、写 manifest 前崩溃，恢复时不能把临时 block 自动视为有效，因为它还没有权威提交记录。Driver 写 manifest 后、更新内存状态前崩溃，恢复时应从 manifest 重建 Committed，而不是重新执行。两者只差一个提交点，恢复动作完全不同。若程序把内存状态当权威，第二种场景会重复执行；若程序把目录扫描当权威，第一种场景会误提交。

因此恢复测试要故意切在这两个位置。只在正常结束后重启，永远证明不了提交协议。可以在代码中加入故障注入点：after validate、before manifest append、after manifest append、before ack user。每个点都跑一次，最终 checksum 和 manifest 条目都必须符合预期。这个实验比背 WAL 定义更能让读者理解提交边界。

16.9 部分失败、降级和业务语义

分布式系统很少只有整体成功和整体失败。一个查询访问十个 partition，九个成功一个超时；一个统计作业处理一百个 split，两个 split 输入损坏；一个监控任务需要近实时结果，但某个 worker 暂时不可用。系统要决定返回失败、返回 partial、使用旧 checkpoint、后台补齐，还是让用户选择严格模式。这个决定不是纯技术问题，而是业务语义的一部分。

严格结果和部分结果 计费、库存、权限和 manifest commit 通常需要严格结果。少一个 partition，答案就不能对外承诺。搜索、监控、推荐和探索式统计可能允许 partial，但必须标注缺失范围、数据时间、错误原因和补齐状态。若系统把 partial 当 complete 返回，就是把可用性提升建立在语义欺骗上。教材项目可以提供 strict 和 monitor 两种模式，让读者看到同一故障在不同业务合同下的处理差异。

Partial 也需要状态。ResultState 可以是 Complete、Partial、Stale、Failed、Recovering。Partial 要记录缺失 task、缺失字节、缺失 key range 和是否会后台重试。Stale 要记录使用的 checkpoint generation。Recovering 要说明结果尚不可用但系统正在补齐。这样上层调用者看到的不是“成功但不准”，而是一份带元数据的结果。

降级不是忽略错误 降级常被误写成 catch error 后返回空结果。正确降级要有合同。比如实时监控可以先返回已完成 partition 的计数，同时在元数据里写明 coverage 为百分之九十八；搜索可以少返回某个 shard 的结果，但页面应标记结果可能不完整；审计统计则不能降级，只能失败并保留恢复任务。降级选择来自用户能否接受不完整信息，不来自工程师想让系统看起来可用。

降级还要和重试预算配合。预算耗尽后进入 Partial，不应继续在后台无限重试压垮系统；如果后台补齐，补齐任务应使用新的 request 或新的 recovery epoch，避免和用户请求的 deadline 混在一起。这样系统既能给出及时响应，又不会把过期任务伪装成当前请求。

数据本地性和降级的关系 分布式计算常把任务移动到数据附近。若本地 worker 慢，调度器可以等待本地队列，也可以远程读取数据，也可以返回 partial。三者的成本不同。等待保留数据本地性但增加延迟；远程读取增加网络字节但可能更快；partial 降低完整性但满足时限。调度器如果没有业务语义，只能机械选择“最快”或“最近”，很容易做错。

这个判断和 NUMA 章节相似。单机里远端内存访问不一定绝对错误，如果本地队列太长，跨节点也可能更快；集群里远程 split 读取也不一定错误，如果本地 worker 已经过载，远程执行可能减少尾延迟。高质量报告要写明：当前选择优化的是吞吐、尾延迟、网络成本、完整性还是恢复成本。

16.10 观测性：把不确定性变成可复盘事件

分布式观测性不是最后加日志。没有统一身份字段，日志无法关联；没有状态转移记录，metrics 无法解释；没有 trace，迟到响应只能靠猜。观测性至少包含三层：metrics 看趋势，logs 看离散事件，traces 串起一次请求路径。三者必须共享 request、attempt、task、epoch、trace、worker 和 manifest

generation。

指标要按原因拆开 只记录 RPC latency 平均值没有太大价值。端到端延迟要拆成 client queue、bus delay、server queue、execute、write output、reply delay、commit。失败要拆成 timeout、busy、stale epoch、deadline expired、checksum mismatch、fatal。重试要记录原始请求数、attempt 数和放大倍数。队列要记录深度、in-flight task、in-flight bytes 和拒绝次数。只有拆开原因，系统变慢时才知道该降并发、扩 worker、修代码还是调整 deadline。

平均值还会掩盖长尾。分布式作业的完成时间常由最慢的少数 task 决定。报告至少要看 p50、p90、p99、max 和分 partition 分 worker 的分布。若全局平均正常但某个 partition p99 很高，可能是热点 key；若所有 worker 都慢，可能是上游发送过快或磁盘瓶颈；若只有重试请求慢，可能是退避策略过长。指标设计要服务诊断，而不是服务漂亮图表。

Trace 要表达因果而不只表达时间 两个机器的时间戳不一定可靠，即使在本机模拟中，不同线程的日志也可能交错。Trace 不能只按时间排序，还要记录 parent event。Retry attempt 的 parent 是哪次 timeout，commit 的 parent 是哪次 valid reply，late reply 的 parent 是哪次旧 send。这样即使时间戳接近或乱序，读者也能还原因果链。

TraceLog 可以用纯文本表达：event id、parent id、trace id、request id、attempt、component、state、reason、logical time。EPUB 里不需要图片，文本反而更容易检索。一次响应丢失实验的 trace 应能独立说明：旧 attempt 执行完成，reply 被 drop，timer 创建新 attempt，新 attempt commit，旧 reply late。若 trace 不能讲清故事，说明状态字段还不够。

Runbook 是观测性的最终形态 教材项目应为常见事故写 runbook。比如“作业长尾上升”先看 partition p99，再看 queue depth，再看 retry amplification，再看 worker CPU 和 I/O；“结果 checksum 不一致”先看 manifest 是否重复 task，再看 late reply 和 duplicate commit，再看 input split checksum；“恢复后重复执行”先看 driver epoch、manifest generation 和 task table 重建日志。Runbook 把知识从正文变成操作顺序。

这也回应为什么本章不从工具列表开始。工具只是实现观察的手段，真正重要的是问题顺序。先问语义是否正确，再问状态是否完整，再问资源是否到顶，再问内核或网络在哪里等待。没有这个顺序，ss、tcpdump、perf、strace 都会变成截图堆砌。

16.11 RPC 生命周期：把一个函数调用拆成事件

远程调用的外表像函数，内部却是一串事件。若教材直接说“RPC 不是函数调用”，读者仍然可能把它当口号。更好的办法是把一次 `run_remote(split)` 按时间拆开：构造请求对象，序列化，进入客户端发送队列，等待连接可写，进入内核或用户态网络栈，经过对端接收队列，交给 worker 线程，执行业务，写临时输出，生成响应，响应排队，返回客户端，进入 completion handler，最后由 Driver 尝试提交。每一步都可能排队、失败、重复或迟到。

本地函数调用的成本主要是调用约定、栈帧和被调函数执行。远程调用的成本包含固定头部、编码解码、队列等待、系统调用或用户态轮询、网络传输、对端调度、业务执行和回调处理。把一次 RPC 延迟写成一个数字，会把这些成本混在一起。性能优化也会因此走错方向：业务函数只占一小部分时，手写 SIMD 可能看不到端到端收益；客户端队列已经满时，增加 worker 也没用；响应处理持锁太久时，网络再快也会在完成队列排队。

客户端路径：提交不等于发出 Driver 调用 send 后，消息可能只是进入客户端队列。连接不可写、窗口满、限流、序列化线程忙，都会让消息尚未离开本进程。若此时 Driver 的 timeout 从 send 返回开始计算，它测到的是本地排队加远端执行的混合时间。更清楚的状态是 queued、encoded、sent、delivered、started、finished、replied、handled。每个状态有自己的时间戳，报告才能判断慢在哪里。

客户端还要决定排队策略。队列满时，是阻塞调用线程、返回 Busy、丢弃低优先级请求，还是把请求放入延迟队列？阻塞会把背压直接传回上游，返回 Busy 需要上游理解错误分类，丢弃低优先级请求要求业务允许。没有策略的无界队列最危险，因为它让系统在内存耗尽前都假装正常。

服务端路径：接收不等于执行 消息到达 worker 以后，也未必马上执行业务。它可能在 socket buffer、RPC 框架队列、worker inbox、线程池 ready queue 或磁盘 I/O 前等待。服务端如果只记录“收到请求”和“处理完成”，就无法区分排队慢和计算慢。一个健康的服务端 trace 至少记录 receive time、enqueue time、start time、finish compute、finish output、send reply。这样 Driver 看到 timeout 时，worker 侧能解释它到底卡在哪里。

服务端还要在开始前检查 epoch 和 deadline。若消息已经过期，直接返回 DeadlineExpired 比继续执行更好；若 epoch 旧，返回 StaleEpoch 比悄悄执行安全；若 inbox 已经高水位，返回 Busy 比把请求塞进队列更诚实。错误分类不是日志装饰，它决定客户端下一步是刷新路由、退避重试、停止重试还是报告 fatal。

完成路径：回调不是提交点 客户端收到响应后通常进入 completion handler。这个 handler 只说明响应到了，不说明可以提交。它仍要通过状态表验证 request、attempt、epoch、deadline、checksum 和 commit state。若 handler 里直接写 manifest，就把网络完成和业务提交混在一起。异步 I/O 章节讲过 completion 和 commit 的区别；分布式 RPC 只是把 completion 来源换成网络。

Completion handler 还要避免持锁执行复杂业务。若它在全局 Driver 锁下校验大文件、写 manifest、触发回调和打印日志，其他响应会排队，尾延迟会恶化。更好的结构是：短锁下查状态并记录决策，把耗时校验或写入交给明确的提交阶段，再短锁更新结果。这样状态一致性和耗时 I/O 不会互相拖累。

端到端报告应拆成事件表 本章建议把一次请求写成纯文本事件表，而不是画图。每行包含 event、component、request、attempt、time、queue depth、bytes、decision。响应丢失事故的事件表可以清楚显示：attempt 0 执行完成但 reply drop，attempt 1 创建并提交，attempt 0 late reply 被拒绝。这样的表在 PDF 和 EPUB 中都稳定，也方便读者复制到测试输出。

事件表还能成为性能数据。统计所有请求的 queued 到 sent 时间，可以发现客户端拥塞；统计 delivered 到 started 时间，可以发现 worker 队列；统计 finished 到 handled 时间，可以发现响应路径阻塞；统计 handled 到 committed 时间，可以发现 manifest 写入瓶颈。分布式性能优化从这里开始，而不是从“网络慢”这种模糊判断开始。

16.12 调度、数据本地性和慢节点

分布式计算不是把任务随机丢给 worker。每个 task 都有输入位置、数据大小、预期成本、deadline、重试历史和输出位置。每个 worker 都有队列长度、CPU 状态、磁盘状态、网络距离、已有缓存和健康状态。调度器要在这些信息不完整、会过期、还可能相互矛盾的情况下做选择。若教材只讲“把任务发给空闲 worker”，就会漏掉分布式计算的核心成本。

响应丢失事故也会影响调度。Driver 超时后要不要把 task 发给另一个 worker？如果输入 split 只在原 worker 本地，换 worker 需要远程读；如果原 worker 只是慢，重复执行会浪费资源；如果原 worker 已经写出临时 block，换 worker 可能产生两个候选输出；如果 deadline 快到，等待本地 worker 可能比远程重算更差。这些判断不能靠单个 retry 函数解决，需要调度器知道数据位置和状态。

数据本地性是 NUMA 思想的集群版本 单机 NUMA 中，线程访问本地内存更快，但本地队列太长时，跨节点也可能更合算。集群里同样如此：任务靠近输入 split 可以减少网络读取，但本地 worker 过载时，远程执行可能降低尾延迟。数据本地性不是绝对规则，而是成本函数的一项。成本函数至少考虑输入字节、网络带宽、worker 队列、预期执行时间、deadline 和重做代价。

日志统计项目可以给出简单规则：优先选择持有 split 的 worker；若本地 worker queue depth 超过阈值，并且 split 可远程读取，则选择远程空闲 worker；若 task 已经有一个正在运行的 attempt，只有在达到 speculation 条件时才启动第二个 attempt；若输出已经进入 commit 阶段，不再启动新 attempt。这个规则不完美，但每个分支都有可解释的成本。

慢节点要分类型处理 慢节点可能慢在 CPU、磁盘、网络、锁、队列或后台任务。CPU 忙时，继续给它计算任务会更慢；磁盘慢时，给它 CPU 密集任务也许仍可接受；网络慢时，本地输入任务可能还行，shuffle 输出会受影响；队列长时，短任务可能被长任务挡住。把慢节点统一标成 bad，会损失资源；完全不标记，又会拖慢尾延迟。

教学项目可以让 worker 周期性上报一个简化快照：running tasks、inbox depth、in-flight bytes、recent p95 execute time、recent p95 output time、busy count。Driver 不要把这些数字当绝对真理，因为它们有延迟；但可以用它们避免明显错误。比如某个 worker 连续返回 Busy，就降低它的窗口；某个 worker 输出时间长，就少派写出重的 task；某个 worker CPU 空闲但 inbox 高，说明调度或完成路径可能阻塞。

Speculation 要和幂等提交绑定 Speculative execution 的动机很直观：某个 attempt 太慢，启动另一个 attempt，谁先完成用谁。它能降低长尾，也能放大资源消耗。没有幂等提交时，它会产生

重复输出；没有预算时，它会在系统最慢时制造更多工作；没有 trace 时，它会让事故更难复盘。因此 speculation 必须满足三个条件：同一 task 的多个 attempt 可被区分，commit table 只接受一个结果，retry budget 允许额外 attempt。

触发 speculation 也要谨慎。不能 task 刚超过平均值就复制，否则长任务都会被重复执行。可以使用分位数阈值：当某 attempt 运行时间超过同类 task 的 p95，且 deadline 仍足够，且 worker 健康状态可疑，且系统有空闲资源，才启动 speculative attempt。这个策略要在报告中写成条件，不要藏在代码常量里。

热点 key 会把分布式调度拉回数据结构 有些慢不是 worker 慢，而是 key 热。所有 map worker 都把同一个热门 key 发到同一个 reduce partition，那个 partition 变成尾部瓶颈。调度器如果只看 worker 空闲，会不断给它更多输入，却无法解决 reduce 热点。此时需要回到数据结构：热 key 拆成多个子 key，本地先聚合，二级 reduce 再合并。这个方案改变数据流，也改变 manifest 和 checkpoint 结构。

因此分布式调度不是孤立主题。它依赖前面章节的数据布局、并行归约和背压。一个好的教材应让读者看到这些线索汇合：先在 map 端局部聚合减少 shuffle 字节，再按 key 分区保证 reduce 语义，再识别热点 key 做分层聚合，再用 manifest 保证重复 attempt 不重复计数。这样“分布式”不再是新名词，而是前面系统原则的集群尺度版本。

16.13 时间、顺序和租约边界

分布式系统里的时间有两种用法。第一种是资源控制：等待多久、何时重试、deadline 是否过期。第二种是正确性判断：谁先写、谁是 owner、租约是否有效。前一种可以用单调时间做工程控制；后一种不能轻易依赖墙钟。许多错误来自把日志时间戳当成协议顺序，把 timeout 当成失败证明，把 lease 当成永远安全的本地缓存。

本章响应丢失事故中，Driver 的 timer 只是本地事件。它不能证明 worker 没执行，也不能证明旧响应无效。旧响应是否有效，要看 request、attempt、epoch 和 commit state。时间只触发状态机检查，不直接决定业务事实。这个原则进入下一章后会变成：旧 leader 是否还能写，不由它本地时钟说了算，而由 term、quorum 和 commit index 决定。

单调时间适合测量等待 在一个进程内计算 elapsed time，应使用单调时钟，而不是 wall clock。Wall clock 可能被系统校时向前或向后调整；单调时钟适合 timeout、retry_at、queue wait 和 service time。教学项目中的 deadline_ns 可以使用单调时间基准，因为所有组件在同一进程或同一机器模拟。进入真实多机时，跨机器 deadline 仍可作为预算提示，但要留出时钟偏差和传输延迟余量。

报告中要分开写 log_time 和 monotonic_delta。log_time 方便人读，monotonic_delta 方便计算等待。若系统重启，单调时间基准会变化，持久化 deadline 不能简单保存本机 monotonic 值；可以保存相对预算或 wall clock 截止并在恢复时保守处理。这个细节看似琐碎，却会影响恢复测试。

逻辑时间表达协议顺序 Epoch、term、version、generation、log index 都是逻辑时间。它们不表示真实时刻，而表示协议里的新旧关系。Driver epoch 区分重启前后，route epoch 区分分片归属版本，leader term 区分领导权，manifest generation 区分提交日志版本。旧消息迟到时，系统先看逻辑时间，再决定是否接受。墙钟时间只能帮助排查，不能替代这些字段。

逻辑时间还让系统能拒绝未来消息。若 worker 收到比自己更高 route epoch 的请求，说明它状态过旧或消息来自未来配置，不能盲目执行；应返回刷新或拒绝。只处理“旧版本”不处理“未来版本”，滚动升级和恢复时也会出错。版本比较必须写成协议规则，而不是散落在 if 语句里。

租约需要暂停和漂移假设 租约常用于优化读路径或缓存所有权：某节点在一段时间内拥有某资源，可以少一次协调。租约的危险在于进程暂停。节点拿到十秒租约后暂停三十秒，恢复时如果只看内存状态，可能以为自己仍有权利。安全租约要求持有者在使用权利前检查当前时间和 term，授予者也要考虑时钟漂移和消息延迟。若无法保证这些假设，就应使用更保守的协议。

教学项目可以不实现租约，但要说明哪些优化类似租约。比如 worker 缓存 route table，以为一段时间内路由不变；Driver 缓存 worker 健康状态，以为短期可用；reduce 缓存 manifest，以为不会再变。每个缓存都要有 version 或 refresh 机制。这样读者将来遇到 leader lease、read lease、cache lease 时，不会把它们当简单过期时间。

16.14 连接管理、服务发现和负载均衡

本机 MessageBus 把网络抽象成队列，真实系统还要处理连接和服务发现。连接不是透明管道，它有建立成本、拥塞窗口、发送缓冲、接收缓冲、队头阻塞、重置和关闭状态。服务发现也不是一张永远正确的列表，实例会上下线，版本会变化，健康状态会滞后。若这些控制面没有进入模型，RPC 行为仍然会在故障时变得不可解释。

连接池也是资源池 每个请求新建连接通常太贵，系统会使用连接池。连接池满时，新请求等待还是失败？等待是否消耗 deadline？连接被重置时，in-flight 请求如何分类？同一连接上多路复用，一个大响应是否会阻塞小响应？这些问题和线程池、有界队列完全同构。连接池需要容量、排队策略、关闭语义和指标。

教学项目可以把连接池简化为 per-worker window。每个 worker 同时允许多少 in-flight task，多少 in-flight bytes，多少 pending control message。窗口满了，Driver 不再发送新 data message；收到 Busy 或连续 timeout，窗口下降；连续成功且队列低水位，窗口缓慢上升。这个简化不等于真实 TCP，但能训练资源池思维。

服务发现结果必须有版本 服务发现可以来自静态配置、DNS、注册中心或控制面。结果会缓存，缓存就会过期。Worker 下线后，客户端可能仍把请求发过去；新 worker 上线后，客户端可能暂时不知道；滚动升级时，新旧协议版本可能并存。发现结果要携带 generation 或 version，错误响应应能触发刷新。否则客户端会在旧列表里反复重试，把故障放大。

负载均衡策略也要从场景出发。轮询简单，但不知道请求轻重；随机能避免全局同步，但可能打到慢节点；最少连接适合长请求，但短请求场景可能抖动；按延迟加权容易被瞬时噪声影响；按 locality 选择能省网络，但可能形成热点。教材项目可以实现“优先本地，过载远程，Busy 降权”的简单策略，并要求报告说明它失败的反例。

熔断和隔离保护上游 下游持续失败时，上游若继续发送请求，会消耗线程、连接和队列。熔断器把下游临时标成 open，快速失败或降级；一段时间后进入 half-open，允许少量探测；探测成功再 closed。熔断不是隐藏错误，而是保护系统不被坏下游拖死。它必须和业务语义配合：某些请求可以降级，某些必须失败。

隔离则把不同下游、不同 job、不同优先级放进不同资源池。若所有请求共享一个连接池，低优先级大作业可能拖慢高优先级查询；若所有 worker 共享一个无界队列，一个热点 partition 会影响无关 partition。隔离降低故障传播，但可能降低资源利用率。工程设计要写清目标：吞吐、公平、尾延迟还是故障半径。

16.15 贯穿项目的实现路线

本章概念最终要落到 Compute Systems Engine 的一个可运行子系统，而不是停在文字。实现可以分五步推进，每一步都保留 reference 和故障测试。第一步，只实现串行 Driver 和 worker 函数，确认输入 split、输出 block、checksum 和 manifest 语义。第二步，引入 MessageEnvelope 和本机 MessageBus，但不注入故障，保证消息边界和直接调用结果一致。第三步，注入丢请求、丢响应、重复和延迟，建立状态表和幂等提交。第四步，加入窗口、Busy 和 retry budget，观察重试风暴被抑制。第五步，加入 driver epoch、持久 task table 和恢复测试。

第一版：把远程调用写成显式消息 第一版不要急着做网络。把原来的函数参数拆成 Envelope 和 payload，worker 不再访问 Driver 对象，只读取消息内容并返回 reply。这个版本应故意复制 payload，牺牲性能换清晰边界。测试比较直接函数调用和消息调用的输出，确保语义一致。若这一步都不通过，后续故障注入只会把问题变复杂。

消息版本从第一天就写入，即使只有 version 1。缺字段、错 kind、payload length 不匹配都要返回 InvalidRequest。这样协议校验成为正常代码路径，而不是将来补丁。日志也从第一天带 trace id，哪怕本机只有一个进程。后续一旦引入并发和故障，这些字段就会立刻派上用场。

第二版：状态表和提交表先于重试 很多人先写 retry，再补幂等，顺序反了。第二版应先写 TaskRuntimeState 和 CommitRecord，即使暂时没有故障。正常响应也通过状态表提交，重复响应测试也能手工构造。这样 retry 加进来时，只是产生新的事件，不会改变提交路径。状态表是系统骨架，retry 是骨架上的策略。

CommitRecord 要从 manifest log 重建。程序启动时读 manifest，构造已提交 task 集合；收到响应时先查集合；写 manifest 后再更新内存表。这个顺序让恢复逻辑和正常逻辑共享权威记录。若

正常路径和恢复路径使用两套规则，迟早会出现“重启后结果不同”的问题。

第三版：故障注入只开一个开关 故障测试按单点推进。先 drop request，确认无副作用重试；再 drop reply，确认重复执行不重复提交；再 duplicate request，确认幂等表返回旧结果；再 delay reply，确认迟到响应被分类；再 corrupt checksum，确认坏消息不进入业务；最后组合两个故障。每次只开一个新故障，报告写预期和实际差异。

不要用“随机故障率百分之一”作为第一批测试。随机故障适合压力阶段，不适合教学和调试。可复现规则能让读者看到状态机转移。等所有单点规则通过，再用固定 seed 做随机混合，验证系统在长时间运行中仍不破坏 manifest。

第四版：背压指标作为验收条件 加入窗口和 Busy 后，测试不能只看最终成功。要看 retry amplification 是否下降，worker inbox 是否稳定，deadline expired 是否合理，Busy 是否让上游降速。若加了 Busy 但 Driver 立即重试，队列仍会爆；若窗口太小，吞吐会被 RTT 限制；若窗口太大，尾延迟上升。背压是控制系统，需要指标闭环。

可以安排一个慢 worker 实验。无背压版本中，Driver 不断发送，timeout 和 retry 上升；有 per-worker window 后，慢 worker 的 in-flight 限制住；加入全局 retry budget 后，attempt 总数不再失控；加入 jitter 后，多 worker 同时恢复时不会同步冲击。这个实验把“重试风暴”从概念变成曲线和事件表。

第五版：恢复测试决定是否真正分布式 最后加入 Driver 重启。运行到指定故障点退出进程，再从持久状态恢复。恢复后，已提交 task 不重做，未提交 task 重新排队，旧 epoch 响应被拒绝，临时目录对账后清理未引用 block。若这一版通过，本机项目才真正具备分布式语义。它仍然不是真实集群，但已经把身份、时间、提交和恢复讲清楚。

恢复报告要列出读取的 manifest generation、task table 条目数、被丢弃的半写记录、重新派发的 task、清理的临时 block、拒绝的旧响应。读者看到这些材料，就能理解分布式系统为什么要把调用栈变成日志和表。没有恢复测试的 MessageBus，只是一个更复杂的函数调用包装。

16.16 C++ 接口设计：把协议约束写进类型

如果 C++ 接口仍然暴露 `run(task)` 这种同步函数，调用者很容易继续按本地函数推理。接口设计要逼迫使用者面对消息语义：发送得到的是 request handle，不是业务结果；响应进入状态机，不是直接返回值；取消返回的是取消请求是否被接受，不是远端是否已经停止；提交返回的是 manifest 是否接受，不是 worker 是否执行过。好的接口不只是方便调用，还会减少错误推理。

RequestHandle 不拥有业务结果 发送后，Driver 应得到一个轻量 handle，用于查询状态、取消、等待或关联 trace。Handle 可以保存 request id、task id、attempt、deadline 和 driver epoch，但不应直接持有 worker 输出。输出只有进入 commit 流程以后才成为业务结果。这样调用者不会把“我有一个 handle”误解成“任务已经在执行”或“最终一定会返回”。

Listing 16.9 请求句柄只表达控制身份

```
1 struct RequestHandle {
2     std::uint64_t request_id = 0;
3     std::uint32_t task_id = 0;
4     std::uint32_t attempt = 0;
5     std::uint32_t driver_epoch = 0;
6     std::uint64_t trace_id = 0;
7     std::uint64_t deadline_ns = 0;
8 };
```

这个类型看起来简单，却能改变调用方式。业务层不能直接从 handle 拿结果，只能把 handle 交给状态机或等待 API。若 timeout 发生，handle 仍然有效，用于标记旧 attempt、记录 late reply 或发 cancel。若 Driver 重启，持久化记录可以重建类似 handle 的身份字段。类型把“请求身份”和“业务结果”分开，是分布式接口设计的第一步。

Result 不应该只有成功和失败 远端调用结果至少要表达 Ok、Busy、Retryable、StaleEpoch、DeadlineExpired、InvalidRequest、Fatal、Duplicate、LateAttempt。若接口只返回 bool，调用者

只能把所有错误混成重试或失败。Busy 应触发降速，StaleEpoch 应触发刷新路由或拒绝旧响应，DeadlineExpired 应停止继续消耗资源，Duplicate 应返回已保存结果，Fatal 才是真正不可重试错误。

Listing 16.10 响应事件和提交决策分开

```

1 struct ReplyEvent {
2     MessageEnvelope envelope;
3     std::uint64_t received_ns = 0;
4     std::uint64_t output_block_id = 0;
5     std::uint32_t output_checksum = 0;
6 };
7
8 struct CommitDecision {
9     ReplyDecision decision = ReplyDecision::UnknownRequest;
10    std::uint64_t committed_block_id = 0;
11    std::uint32_t committed_checksum = 0;
12    RpcStatus status = RpcStatus::Unknown;
13 };

```

这里 `ReplyEvent` 是输入事件，`CommitDecision` 是状态机输出。二者分开后，测试可以直接构造旧响应、重复响应、checksum 错误响应，不需要真的跑网络。接口也能自然支持审计：每个响应事件都有一条决策记录，说明它为什么被接受或拒绝。分布式系统的可维护性，很大一部分来自这种可审查的中间对象。

状态机 API 要避免回调里写业务 异步框架喜欢让用户注册回调：响应到了就调用 lambda。若 lambda 里直接写 manifest、更新 UI、释放 buffer、启动新 attempt，状态很快散落。更稳的接口是让回调只投递 `ReplyEvent`，由 Driver 的状态机统一处理。状态机处理完返回 `CommitDecision` 和下一组动作，例如发送 retry、清理临时文件、降低窗口、触发用户完成。

这种设计看起来绕了一步，却让测试和恢复容易得多。恢复时也可以把持久化事件重新喂给状态机；故障注入也可以伪造事件；trace 可以记录状态机输入输出。若业务逻辑散在回调里，重放几乎不可能。分布式系统要把“事件”和“动作”分开，和任务运行时把 ready、running、waiting 分开是同一个思想。

接口要表达所有权 发送 payload 的 API 可以设计成 `send(MessageEnvelope, OwnedPayload)`，明确 bus 接管 payload；或者设计成 `send_copy(MessageEnvelope, std::span<const std::byte>)`，明确复制。不要设计成接收裸指针和长度，却不说明调用后谁拥有内存。Buffer 池也要有租借对象，发送完成前不能归还。若为了性能使用引用计数，引用计数对象应进入类型签名，而不是藏在注释里。

输出 block 也一样。Worker 写出的临时 block 由 worker 或存储层拥有；Driver commit 后，manifest 拥有外部可见引用；清理线程只能删除 manifest 未引用且不在当前 attempt 集合里的文件。所有权如果只靠目录命名约定，迟到响应和恢复扫描时很容易误删。类型和状态表要共同表达这些边界。

16.17 状态机测试：用反例驱动接口收敛

分布式状态机不能靠“跑一遍正常作业”证明。它要靠反例收敛。每个反例都应让旧接口暴露一个说不清的问题，然后引入一个字段、状态或提交规则解决它。这样读者看到的是概念生长过程，而不是概念罗列。下面几组测试可以作为本章最小测试集。

反例一：旧 attempt 比新 attempt 晚到 构造 task 7，attempt 0 执行完成但响应延迟，attempt 1 先完成并提交。随后 attempt 0 响应到达。期望：状态机把 attempt 0 判为 LateAttempt，manifest 不变，late reply 指标加一，若 attempt 0 的临时 block 未被引用则进入清理列表。若程序只按 task id 匹配，它会失败；若程序没有 accepted attempt，它也会失败。这个反例逼出 attempt 和 commit record。

测试要检查日志顺序，而不是只检查最终输出。最终输出可能碰巧一致，但 manifest 如果记录了两条，后续 reduce 或恢复会出错。事件表中应能看到 commit attempt 1 在前，late attempt 0 在后。这个检查让“迟到响应”从抽象故障变成可重复案例。

反例二：服务端执行后崩溃 Worker 写出临时 block 后崩溃，没有发送响应。Driver 超时重试。恢复程序启动后看到临时 block，但 manifest 没有 entry。期望：不能自动把临时 block 视为有效；可以清理，也可以作为诊断保留；新 attempt 成功后 manifest 只接受新 block。若系统扫描目录并把所有 block 合并，就会重复计数。这个反例逼出 manifest 权威性。

进一步变体是 manifest append 后崩溃。此时恢复程序应接受 manifest 中的 block，而不是重做。两个变体只差一个提交点，测试必须分别覆盖。读者通过这对反例会明白，崩溃恢复不是“重跑一遍”这么简单，而是围绕权威记录做判定。

反例三：Busy 被当成普通失败 让 worker inbox 容量很小，并让 worker 执行变慢。若 Driver 把 Busy 当 Retryable 立即重试，消息量会上升，Busy 更多，timeout 更多。期望：Busy 降低该 worker 的窗口，`retry_at` 加退避，deadline 不足时停止重试。这个反例逼出错误分类、窗口和 retry budget。它也说明正确性和性能纠缠在一起：错误分类错了，系统就会自我放大。

实验报告要写请求放大倍数。比如原始 task 为 100，attempt 为 280，说明重试策略让系统多做了 180 次尝试；加入预算后 attempt 降到 130，成功率和尾延迟如何变化。没有这些数字，背压只是一句口号。

反例四：Driver 重启后旧响应到达 Driver epoch 5 发送 attempt 0，进程崩溃，epoch 6 恢复并重新派发 attempt 1。旧 worker 迟到返回 epoch 5 的响应。期望：状态机识别 StaleEpoch，不直接提交；若 manifest 已有 epoch 5 的提交记录，则按恢复规则处理；若没有提交记录，则拒绝旧响应并清理临时输出。这个反例逼出 driver epoch 和恢复查询。

测试中要同时记录持久 task table 和内存 state。恢复后内存表是重建结果，不是旧事实本身。若程序依赖旧内存，就无法通过重启测试。这个案例会让读者理解为什么分布式系统总是把“当前状态”写成日志、表和 generation。

16.18 从本机模拟迁移到多进程

本机单进程 MessageBus 适合教学，但它仍然共享地址空间、共享时钟、共享文件系统缓存，也不会遇到真实进程崩溃后资源清理问题。下一步可以迁移到本机多进程：Driver 一个进程，worker 多个进程，通信先用 Unix domain socket 或管道，消息格式仍使用本章 Envelope。这样不需要真实集群，也能让共享内存捷径消失。

多进程会暴露隐藏共享状态 单进程版本里，某些全局变量可能被 Driver 和 worker 共同访问；多进程后它们自然断开。若程序之前依赖全局 config、全局 allocator、全局 metrics map，多进程会让这些依赖失败。正确做法是把配置版本放进消息，把 metrics 通过事件上报，把内存所有权限限制在进程内。这个迁移能检查本机模拟是否真的遵守消息边界。

多进程还会改变崩溃语义。Worker 进程退出后，Driver 要看到连接关闭或心跳丢失；临时文件可能留在磁盘；未发送完的响应会消失；操作系统会回收进程内存但不会清理业务文件。恢复测试因此更真实。单进程里抛异常不等于进程崩溃，多进程能把这个边界讲清楚。

Unix domain socket 仍然不是网络全貌 Unix domain socket 能训练 framing、序列化、连接关闭和进程隔离，但没有真实跨机延迟、丢包、路由变化和时钟漂移。教学报告要写清这个边界。它验证协议语义，不验证集群性能。进入 TCP 或真实机器后，窗口、拥塞、Nagle、MTU、网卡队列和交换机会引入新成本，之前的状态机仍然有用，性能参数必须重新测。

这也是本章反复强调“先语义后性能”的原因。若单进程模拟的语义不清，多进程只会更乱；若多进程语义清楚，真实网络主要增加观察和调参难度。系统工程需要分层验证，而不是一步跳到最复杂环境。

文件格式和协议格式开始统一 多进程迁移时，消息格式、manifest 格式和 checkpoint 格式最好共享同一套编码原则：固定 version、固定整数位宽、长度前缀、checksum、未知字段策略。这样读者不会在网络消息和持久文件之间来回切换风格。许多工业系统的 bug 来自“网络协议很严谨，checkpoint 却直接 dump 对象”，或反过来。第二册项目应从教学阶段就避免这种割裂。

统一格式还方便工具。可以写一个小的 dump 工具，把 manifest、task table、trace event 和消息文件都打印成可读文本。故障报告不需要读二进制，也不需要依赖进程内调试器。工具化是工程

质量的一部分，尤其对分布式系统这种事后复盘很重要的领域。

16.19 Linux 网络观察：从事件回到协议

真实 Linux 上的网络路径很长，教材不需要把内核网络栈完整展开，但必须告诉读者怎样把系统现象接回本章协议。一次 RPC 慢了，可能慢在用户态排队、socket 发送缓冲、内核协议栈、对端接收缓冲、服务端线程池、磁盘写出或响应完成。若只看应用层 timeout，就会把所有位置混成一个“网络慢”。本节的目标是给出一条可执行观察路线：先用应用 trace 定位阶段，再用 Linux 工具确认某一段是否有系统层证据。

先从应用事件表开始 不要一上来抓包。先看应用事件表：queued、encoded、sent、delivered、started、finished、reply sent、reply handled、committed。若 queued 到 sent 很长，问题在客户端队列或连接池；若 sent 到 delivered 很长，可能是网络或对端接收；若 delivered 到 started 很长，服务端队列或线程池有压力；若 finished 到 handled 很长，响应路径或客户端完成回调有问题。应用层阶段能把 Linux 观察范围缩小。

事件表还要记录字节数和队列水位。小消息大量排队与大消息占满发送缓冲是两种问题；控制消息被数据消息挡住与业务执行慢也不同。没有这些应用字段，系统工具看到的只是连接状态和包，而不是业务因果。

ss 看连接，不替代协议判断 ss 可以观察连接状态、发送队列、接收队列和 TCP 信息。发送队列长期非零，说明用户态或内核有数据尚未发送；接收队列堆积，说明应用读取不及时；连接反复重置，说明错误分类应进入 Retryable 或 Fatal。但 ss 不能告诉你请求是否幂等，也不能告诉你旧响应是否应提交。系统工具只解释传输状态，协议状态仍由 MessageEnvelope 和 TaskState 决定。

教学报告可以写两层结论：系统层事实和协议层决定。比如“发送队列高水位持续 800 ms”是系统事实；“Driver 降低该 worker 的 in-flight window，并把超时 attempt 标为 unknown”是协议决定。把两层混在一起，容易把网络现象直接当业务事实。

tcpdump 和消息边界 抓包能看到字节流、重传和延迟，但 TCP 不保留应用消息边界。一次 write 不等于一个包，一条消息可能被拆成多个段，多个小消息也可能合并。应用协议必须有 length prefix 或 framing，抓包时才能按字节重组消息。若没有 framing，调试时很难判断某条消息是否完整到达。

本书项目可以让 MessageEnvelope 前面有固定魔数、版本和 payload length。抓包或日志 dump 时，工具能扫描魔数并解析长度。这样协议本身支持调试。不要把可观测性留给外部工具猜测；协议设计时就要考虑如何被 dump、重放和校验。

tcnetem 和本机故障注入 在有限的 Linux 环境中，tcnetem 可以注入延迟、抖动、丢包和重排。它适合验证真实 socket 路径上的超时和重试。但它不如 MessageBus 精准：你很难指定“只丢 request 1001 的第一次 reply”。因此教学顺序应是先用 MessageBus 做确定性单点故障，再用 netem 做系统级扰动。前者验证状态机，后者验证参数和工具链。

如果在 Termux 或容器中没有权限使用 netem，可以用用户态 MessageBus 延迟队列替代。报告要写清这是用户态模拟，不代表真实 TCP 拥塞。诚实说明边界比强行给出不可靠结论更重要。

strace 只看系统调用边界 strace 能看到 read、write、poll、epoll_wait、fsync 等系统调用的时间和错误码。它适合确认应用是否卡在系统调用，是否短写，是否收到 ECONNRESET，是否频繁小写。但它看不到用户态队列和状态机。若应用在 epoll_wait 前已经把请求排了很久，strace 不会告诉你业务队列发生了什么。

因此 strace 输出要和应用 trace 对齐。Trace 说 request 1001 queued 了 300 ms，strace 说随后 write 只用了 1 ms，说明慢不在系统调用；Trace 说 write 后 reply 很久不到，strace 和 ss 再帮助判断网络或对端。工具之间要互相补证，不要互相替代。

16.20 协议兼容和滚动升级

教学项目可以假设同一版本运行，但真正工程系统很少永远同版本。Driver 升级后可能发送新字段，旧 worker 仍在运行；worker 升级后可能返回新错误码，旧 Driver 不认识；checkpoint 由旧版本写出，新版本要恢复。协议兼容不是大型公司才需要的功能，而是任何长期项目都会遇到的边界。若教材不讲，读者会把 C++ 结构体当成临时内存对象，直到第一次升级时全部失败。

新增字段要有默认语义 假设 version 1 的消息只有 request、attempt 和 payload，version 2 加入 deadline。旧消息没有 deadline 时，新 worker 应给出默认语义，例如使用 job 级 deadline 或拒绝缺字段请求。不能让缺失字段默认为零后立即过期，除非协议明确如此。新增字段如果没有默认语义，就不是兼容新增，而是破坏性升级。

默认语义要写进测试。构造 version 1 request 给 version 2 parser，检查它如何解释 deadline；构造 version 2 request 给 version 1 parser，检查旧 parser 是否能跳过未知字段或拒绝带清楚错误码。兼容测试是协议的一部分。

改变字段含义必须换版本 字段名不变但语义改变，是最危险的升级。比如 attempt 原来表示物理尝试次数，后来被拿来表示重试原因编号；epoch 原来表示 driver 重启版本，后来表示路由版本。旧日志和新代码都会变得不可解释。若语义改变，应增加字段或升级 major version，并让恢复程序明确拒绝不兼容版本。

教材反复强调命名，是因为名字承载协议含义。第三册引入模型推理时，不能把 epoch 拿去表示训练轮次，不能把 attempt 拿去表示 token index。新概念给新名字，和旧概念建立映射。这样多册项目才能演进，而不是名词复用导致混乱。

错误码也要兼容 旧 Driver 不认识新 worker 返回的错误码时，应该如何处理？保守策略是把未知错误归为不可重试或 unknown，并记录原始码；激进策略可能把未知错误当 retryable，导致风暴。错误码应分层：大类稳定，细节可扩展。客户端重试逻辑只依赖稳定大类，日志和诊断再读细节字段。

比如 RpcStatus 大类保留 Ok、Busy、Retryable、StaleEpoch、DeadlineExpired、InvalidRequest、Fatal。新版本可以在 detail 中写 DiskThrottled、ChecksumMismatch、UnsupportedCompression。旧客户端不认识 detail 也能按大类做安全动作。

Checkpoint 兼容比消息更严格 网络消息错了可以拒绝，checkpoint 错了可能导致作业无法恢复。Checkpoint 里要保存 schema version、job config version、manifest generation 和校验。新程序读取旧 checkpoint 时，要么完整迁移，要么明确拒绝并提示重新运行。不要半读半猜。恢复路径最怕“差不多能读”，因为错误会变成重复输出或漏输出。

教学项目可以只支持当前版本恢复，但要把拒绝逻辑写清楚。比如 checkpoint version 不等于当前版本时，程序输出 UnsupportedCheckpointVersion，并保留旧文件不修改。这比硬读旧格式安全得多。等项目成熟后，再增加迁移工具。

16.21 消息重放和确定性调试

分布式 bug 最难的地方是偶发。一次响应迟到、一次 timer 竞争、一次队列满，可能在下次运行中消失。消息重放的目标，是把一次失败运行中的关键事件保存下来，让状态机在相同事件序列下再次执行。重放不是完整模拟真实时间，而是把协议输入固定。只要状态机确定，bug 就能复现。

重放日志记录状态机输入 重放日志不需要保存所有 printf 文本，而要保存状态机输入：send request、deliver request、drop reply、timer fired、busy returned、worker crashed、driver restarted、manifest append result。每条事件有 event id、parent id、logical time、envelope、fault action 和必要 payload 摘要。状态机输出可以在重放时重新计算，并和原运行比较。

若重放输入一致，输出不一致，说明状态机依赖隐藏状态，例如真实时间、随机数、容器迭代顺序或全局变量。教学项目应尽量消除这些隐藏依赖。随机故障使用 seed，当前时间通过 TimerEvent 注入，容器输出排序后写报告。确定性不是为了学术美观，而是为了能调试。

重放和 trace 的区别 Trace 面向人，重放日志面向程序。Trace 可以省略 payload，只写摘要；重放需要足够信息让状态机再次处理。Trace 可以按可读性组织；重放必须按事件顺序和因果关系组织。两者可以共享 event id，但不要混成一个文件。这样报告可读，测试也可执行。

一个响应丢失事故的 trace 可能写“reply dropped”；重放日志要写具体哪条 reply、对应 request、attempt、payload checksum 和 drop action。重放时，MessageBus 不再随机决定，而是按日志执行 drop。失败复现后，读者才能逐步修改状态机并验证修复。

状态快照帮助缩短重放 长作业的完整事件日志可能很大。可以周期性保存状态快照：task table、commit table、window、retry budget、manifest generation。重放时从某个快照开始，应用后续事件。快照仍然要有版本和 checksum。若快照和事件日志不匹配，应拒绝重放。

这个设计和 checkpoint 很像，但目的不同。Checkpoint 用于生产恢复，重放快照用于调试。两者可以共享序列化格式，却应分开目录和语义。不要让调试快照被生产恢复误用。

重放测试进入回归套件 修复一个分布式 bug 后，把触发它的重放日志裁剪成最小回归。回归不一定需要大输入，只要保留关键事件：旧 attempt、timeout、retry、late reply、commit。以后每次改状态机都跑这些重放。这样项目质量会逐步提升，而不是同类 bug 反复出现。

16.22 安全边界和输入防御

第二册不是安全教材，但分布式系统必须讲基本输入防御。消息可能来自旧版本、损坏文件、buggy worker，甚至未来真实网络中的不可信对端。即使本机模拟，也不能假设“内部消息一定合法”。协议解析、长度检查、checksum、版本拒绝和权限边界，都是系统可靠性的一部分。许多故障看起来像安全问题，本质上也是边界没有写清。

长度字段必须先校验 收到消息后，先检查 header 长度、payload length、最大允许大小和 checksum，再分配内存。不要根据未验证 length 直接 resize vector。一个损坏 length 可以让程序分配巨大内存或读越界。教学项目可以设置最大 payload bytes，并在错误码中返回 InvalidRequest 或 PayloadTooLarge。

长度检查还服务性能。最大消息大小明确后，队列容量和 in-flight bytes 才能预算。若消息大小无上限，背压无法设计。安全边界和资源边界在这里是同一件事。

路径不能由远端随意指定 Worker 响应中如果直接带文件路径，Driver 不能无条件信任。路径可能包含上级目录、旧 job 目录或非法字符。更稳的做法是响应带 block id 和摘要，Driver 根据 job id、stage、task、attempt 计算合法路径，或者验证路径必须位于 job tmp 目录内。教学项目也应避免让消息 payload 任意决定写入位置。

这条规则和 manifest 恢复相关。恢复程序扫描目录时，也要避免跟随不应跟随的符号链接或读取目录外文件。即使在个人项目里，清晰路径边界也能防止误删和误读。

错误处理要关闭资源 解析失败、checksum 失败、deadline 过期、version 不兼容，都要释放 payload、减少 in-flight 计数、唤醒等待者或记录指标。错误路径如果只 return，队列窗口可能永远不释放，系统会慢性死锁。每个错误分类都应有资源清理动作。状态机测试要覆盖错误路径，不只覆盖 Ok。

认证可以先不实现，但要留边界 真实跨机器系统需要认证和授权，教学项目可以不实现，但接口要知道认证信息属于消息元数据还是连接元数据。比如 future 版本可以在 Envelope 里加入 auth context id 或 source worker id。当前版本即使忽略认证，也要拒绝未知 source 或非法 job id。这样将来扩展时，不需要推翻消息模型。

16.23 从消息模型走向分片和复制

本章只处理一个 task attempt 的消息语义。下一章会把状态扩展到很多 partition、很多副本和很多 owner。扩展以后，新问题自然出现：key 属于哪个分片，路由表何时变化，旧 owner 的请求如何拒绝，副本之间何时算提交，leader 失效后谁能继续写。它们看起来是新的概念，其实都在重复本章的核心：身份、版本、提交边界和恢复日志。

分片需要 route epoch，是因为旧路由请求会迟到；复制需要 commit index，是因为多个副本要知道哪条日志已经生效；leader 需要 term，是因为旧 leader 网络恢复以后不能继续接受写；quorum 需要交集，是因为读路径要和已提交写路径有共同事实。不要把这些词背成分布式名词表，要把它们看成对本章事故的推广。

CAP 也要放回具体事故 CAP 不是让读者背“一致性、可用性、分区容忍性任选两个”。网络分区发生时，旧 Driver 或旧 owner 可能仍能和一部分 worker 通信，新 Driver 或新 owner 也可能开始工作。此时系统必须决定：继续接受请求，可能产生互相冲突的提交；还是拒绝一部分请求，保持单一提交事实。这个选择才是 CAP 在工程中的具体形态。

正常情况下，系统仍然同时追求低延迟、高可用和足够一致。只有把故障场景写清楚，CAP 才有意义。比如统计监控可以返回 partial 并标注缺失分片；计费 and manifest commit 不能接受两个 owner 各自提交。业务语义不同，故障选择也不同。

控制面和数据面分开 本章 MessageEnvelope、task table、commit table、manifest 都属于控

制面；worker 读取输入、扫描日志、写 block、传输 shuffle 属于数据面。控制面维护少量关键状态，要求可恢复和一致；数据面处理大量字节，要求吞吐和背压。把所有数据都放进强一致日志会拖垮性能；把控制面状态放进临时内存会毁掉恢复。

第二册最终项目应保持这个边界。Driver 控制 task、attempt、manifest 和 checkpoint；worker 数据面处理输入和输出；MessageBus 负责消息语义和故障注入。第三册如果进入 AI 推理引擎，也会沿用同一思想：模型元数据和调度是控制面，张量计算和算子执行是数据面。

16.24 实验和源码阅读路线

本章实验不要追求大规模。三个 split、两个 worker、一个 Driver、一个 MessageBus 就足够。关键是故障矩阵必须完整。第一组只丢请求，验证重试能完成且没有临时输出残留。第二组只丢响应，验证 worker 已经写出结果时，Driver 重试仍然只提交一次。第三组只延迟旧响应，验证旧 attempt 不会覆盖新 attempt。第四组让 worker inbox 满，验证 Busy 会降低发送速率。第五组让 Driver 在 manifest 写入前后崩溃，验证恢复路径不同。

每个实验报告都按同一格式写。先写规则，例如“丢弃 request 1001 的第一次 TaskReply”。再写预期，例如“retry count 为 1，manifest 中 task 7 只有一条记录，late reply 为 1，最终 checksum 等于串行 reference”。然后写实际状态：task table、commit table、临时目录、trace 摘要和指标。最后写边界：本机模拟没有覆盖真实 TCP 拥塞、跨机时钟漂移、滚动升级和多进程崩溃后的资源泄漏。这样报告能独立阅读，不需要读者翻几千行日志猜结论。

Linux 连接点只服务本章问题 本章不是网络栈教材，但可以阅读和使用几个 Linux 连接点。用 `ss` 看连接状态，用 `tcpdump` 或应用 trace 看消息，用 `tcnetem` 注入延迟和丢包，用 `perf` 或日志时间戳区分 CPU 执行和等待。源码阅读可以从 socket buffer、epoll ready、timer 和调度唤醒的高层路径进入，但阅读目标始终是解释本章事件：消息如何排队，ready 如何唤醒，timer 如何触发，队列满如何反馈。

不要把 Linux 源码函数名堆进正文。读到一个复杂函数时，先问它能不能解释一个等待点、一次状态转移或一条指标曲线。解释不了的细节放到索引，不要打断主线。源码阅读的产物应该是“这个 timeout 由哪个 timer 事件触发”“这个 busy 来自哪个队列容量”“这个延迟可能在 socket buffer 还是 worker inbox”，而不是一页函数列表。

本章验收清单 读完本章，读者应能从响应丢失事故写出完整状态表。至少包含 request、attempt、deadline、epoch、task state、commit record 和 trace event。应能解释为什么 timeout 不是 failure，为什么 retry 必须有预算，为什么幂等要发生在提交边界，为什么 MessageBus 不能直接调用 receiver，为什么恢复要依赖 manifest 而不是扫描临时目录。若这些问题还只能用概念名回答，说明学习还没有进入工程层。

16.25 贯穿事故：像代码审查一样读完一次失败运行

本章前面把很多概念从响应丢失事故中拆了出来。为了避免读者把这些概念重新背成清单，本节把它们放回一段完整代码审查里。假设团队提交了一个最小分布式统计引擎，正常输入能算对，单元测试也通过；代码审查时我们不问“系统用了哪些分布式术语”，而问一个更具体的问题：当 worker 已经写出输出，成功响应被丢弃，Driver 超时重试，旧响应随后迟到时，这份代码每一行如何保证最终 manifest 只接受一个结果。

这个审查顺序很重要。先看正常路径，代码会显得简洁；再看一个失败点，才会暴露隐藏假设；然后根据假设缺口引入字段、状态和协议。这样学到的不是“request id、attempt、epoch、幂等、背压”这些名词，而是这些东西分别补上了哪一个事实空洞。若某个字段不能回答一个具体故障问题，它就不该进入协议；若某个故障问题没有字段或状态能回答，系统就还没有设计完。

一、先审最朴素的成功路径 朴素实现通常像一个远程函数调用。Driver 把 split 发送给 worker，等待 `TaskReply`，收到 `Ok` 就把 block 写入 manifest。这段逻辑在正常路径里没有明显问题，因为所有事件按作者希望的顺序发生：发送、执行、响应、提交。代码审查不能停在这里。我们要立刻问：`Ok` 来自哪个 attempt，属于哪个 driver epoch，是否仍在 deadline 之内，block 是否已经被别的 attempt 提交，响应是不是旧请求的重复包。

这些问题一出现，朴素接口就撑不住了。若响应结构里只有 `task_id` 和 `block_id`，Driver 无法区分当前 attempt 与旧 attempt；若没有 `request_id`，重复请求和重复响应无法关联；若没有

`driver_epoch`, 重启前的响应可能被重启后的 Driver 接受; 若没有 commit table, 两个成功响应会争抢 manifest; 若没有 trace, 事故复盘只能靠肉眼翻日志。也就是说, 字段不是凭经验加的, 而是从一次失败运行里被逼出来的。

二、再审第一处错误: timeout 分支直接失败 很多代码在 timeout 分支里写 `mark_failed(task)`。这行看似合理, 其实把“本地等待超时”误写成“远端没有完成”。审查时要追问: 这个判断的证据是什么? Driver 只知道 timer 触发, 不知道请求是否到达、worker 是否开始执行、输出是否已经写盘、响应是否正在路上。若 timeout 分支直接失败, 旧响应迟到时系统会出现自相矛盾的事实: 状态表说失败, worker 说成功, 临时目录里有输出, manifest 可能已经接受新 attempt。

正确的修改不是把 timeout 时间调长, 而是把状态改成 Unknown 或 TimedOut, 并让后续动作通过状态机决定。TimedOut 可以触发重试, 也可以触发查询或放弃, 但不能直接推断远端事实。这个修改就是分布式系统里最基础的思维转换: 计时器产生的是一个本地事件, 不是远端事实证明。Timer 只把状态机唤醒, 真正的业务结论必须由 request、attempt、epoch、commit record 和恢复日志共同决定。

三、再审第二处错误: retry 创建了新的未知副作用 朴素重试只是一行 `send(task)`, 但它实际创建了新的物理执行 attempt。若第一次 attempt 已经写出 block, 第二次 attempt 又写出 block, 系统必须说明哪个 block 对外可见。审查时要把输出目录、manifest 和 reduce 输入一起看。只要 reduce 通过扫描目录收集 block, 重复 attempt 就会变成重复计数; 只有 reduce 只读取 manifest 中 accepted 的 block, 临时目录里的重复输出才只是候选。

因此 retry 之前必须先有提交边界。Worker 写临时 block 不等于提交, Driver 收到 Ok 不等于提交, 只有 manifest 接受某个 task 的某个 attempt 才是提交。重试只是产生候选结果, 不能绕过 commit table。这个顺序非常关键: 不是先写重试再补幂等, 而是先定义唯一生效点, 再允许重复执行。没有唯一生效点, 任何优化重试策略都只是把错误变得更难复现。

四、再审第三处错误: 响应处理没有分层验证 收到响应后立即写 manifest, 是本章事故的核心 bug。审查时应把响应处理拆成固定顺序。第一层是消息完整性, 检查版本、长度和 checksum; 第二层是控制面新旧, 检查 driver epoch; 第三层是请求归属, 检查 request 与 task; 第四层是物理尝试, 检查 attempt; 第五层是资源预算, 检查 deadline 和取消状态; 第六层才是业务提交。前面任意一层失败, 响应都不能进入提交路径。

这个顺序也能解释为什么状态码不能只有成功和失败。旧 epoch 响应不是业务失败, 而是 stale; 旧 attempt 响应不是 fatal, 而是 late; 队列满不是业务错误, 而是 busy; deadline 过期不是 worker 计算错, 而是预算耗尽。错误分类直接决定动作: stale 触发拒绝或刷新, busy 触发降速, late 触发清理, fatal 才可能终止作业。分类越粗, 系统越容易把控制信号变成重试风暴。

五、把审查结果写成事件状态机 代码审查最后要落到一个可测试的状态机。状态机输入不是抽象概念, 而是具体事件: 发送请求、worker 开始、worker 写出、响应丢失、timer 触发、创建新 attempt、响应到达、manifest 接受、旧响应迟到、Driver 重启。每个事件进入状态机后只做一件事: 读取当前事实, 产生下一组动作。状态机不应该在回调里到处写业务状态, 否则重放和恢复都会失败。

Listing 16.11 把事故输入写成状态机事件

```
1 enum class DriverEventKind : std::uint8_t {
2     RequestSent,
3     WorkerStarted,
4     WorkerOutputWritten,
5     ReplyArrived,
6     TimerFired,
7     CommitAccepted,
8     CommitRejected,
9     DriverRecovered,
10 };
11
12 struct DriverEvent {
```

```

13 DriverEventKind kind = DriverEventKind::RequestSent;
14 std::uint64_t event_id = 0;
15 std::uint64_t parent_event_id = 0;
16 std::uint64_t trace_id = 0;
17 std::uint64_t request_id = 0;
18 std::uint32_t task_id = 0;
19 std::uint32_t attempt = 0;
20 std::uint32_t driver_epoch = 0;
21 std::uint64_t time_ns = 0;
22 RpcStatus status = RpcStatus::Ok;
23 };

```

这段事件结构本身并不复杂，价值在于它强迫程序承认“所有事情都是事件”。Timer 是事件，响应是事件，恢复也是事件。事件是否改变状态，要由状态机判断。旧响应到达时，它不是被忽略的异常情况，而是一条正常输入；状态机给出 LateAttempt 决策，并记录清理动作。这样事故不再靠人脑临时判断，而是进入可重复测试。

六、把一条失败运行写成可读的审查记录 审查报告不要只写“通过”。它要像下面这样讲清楚因果。事件一：Driver epoch 4 为 task 7 创建 attempt 0，request 1001，deadline 为当前时间后一秒。事件二：worker 2 收到请求并写出 block 9007，checksum 为某个固定值。事件三：MessageBus 按故障规则丢弃 attempt 0 的 reply。事件四：Driver timer 触发，状态从 Running 变成 TimedOut，retry budget 允许创建 attempt 1。事件五：worker 3 写出 block 9011，Driver 校验后 manifest 接受 attempt 1。事件六：attempt 0 的旧响应迟到，状态机按 task 7 查到已提交 attempt 1，于是把旧响应判为 LateAttempt，block 9007 进入延迟清理。

这段文字比一张术语表更有教学价值。它显示 timeout 为什么不能等于失败，attempt 为什么必须存在，manifest 为什么是权威，旧响应为什么仍要记录，retry budget 为什么要限制 attempt 数。读者若能独立写出这种审查记录，就已经掌握了分布式模型的骨架。若只能说“这里用了幂等和重试”，说明还没有真正理解。

七、从审查记录推出测试用例 测试应该逐条覆盖审查记录，而不是只验证最终输出。第一条测试断言 request 1001 的 reply 被丢弃后，attempt 0 的输出存在但未进入 manifest。第二条测试断言 timer 事件只创建一个新 attempt，并且 retry budget 的 used count 增加。第三条测试断言 attempt 1 提交后，manifest 对 task 7 只有一条 accepted entry。第四条测试构造旧响应迟到，断言状态机返回 LateAttempt，manifest generation 不变，late counter 增加。第五条测试重启 Driver 后重放同一 trace，断言恢复结果一致。

这些测试的共同点是都检查中间状态。只检查最终 word count 很容易漏掉重复提交，因为某些输入上重复可能刚好抵消或被覆盖。分布式系统的测试要检查事实链：哪个事件发生过，哪个事件被拒绝，哪个提交生效，哪个副作用被清理。测试越接近状态机，越能解释失败。

八、从测试失败反推概念是否讲清 如果旧响应测试失败，原因通常不是“没有讲 CAP”，而是 attempt 或 commit table 没讲清。如果 Driver 重启后重复提交，问题不是“恢复很复杂”，而是 manifest 权威性没讲清。如果 Busy 实验中 attempt 数暴涨，问题不是“网络不稳定”，而是错误分类和背压没讲清。如果 trace 无法串起事件，问题不是“日志太少”，而是身份字段没讲清。概念是否需要加深，应该由这些反例来决定。

这也是本书后续章节的写法标准。讲分片时，不从哈希函数开始，而从旧 owner 迟到写入开始；讲共识时，不从协议步骤开始，而从双 Driver 同时提交开始；讲无锁结构时，不从原子操作表开始，而从节点释放后被另一个线程读到开始；讲任务运行时，不从 API 名称开始，而从 worker 同步等待同池 future 造成停顿开始。概念必须从反例中长出，代码和底层事实必须能解释它。

九、审查 C++ 代码时额外检查的边界 C++ 代码还要检查对象生命周期。MessageBus 队列里不能保存指向 Driver 栈变量的指针；payload 发送后发送方不能继续修改；completion handler 不能在全局锁下执行大块 I/O；状态表中的 id 使用固定宽度整数；错误路径必须释放 in-flight bytes；清理临时 block 时必须确认 manifest 未引用。每一条都和本章事故相关，不是风格洁癖。旧响应迟到时若持有悬垂引用，程序会变成内存错误；Busy 路径若不释放窗口，系统会假死；清理路径若只按文件名删除，可能删掉已提交 block。

因此，本章代码示例不追求短，而追求边界清楚。短代码若隐藏生命周期，教材会误导读者。分布式系统的 C++ 实现应该把协议身份、资源所有权、状态转移和错误清理都写出来。等语义稳定后，再考虑减少复制、合并队列、批量发送和零拷贝。性能优化必须站在清晰语义之上，否则只是让错误跑得更快。

十、把第十六章压缩成一句工程判断 如果一个远端操作可以被执行多次、响应可以迟到、调用者可以重启，那么系统就不能把“执行成功”当作“外部生效”。它必须定义消息身份、物理 attempt、逻辑 deadline、提交权威、恢复日志和观测事件。第十六章讲的所有内容，最终都围绕这句话展开。读者以后看到新的 RPC 框架、服务发现系统或分布式队列时，也应先问这六件事，而不是先问它用什么名词。

16.26 贯穿实验：从单进程模拟迁移到多进程以后会暴露什么

第 16 章前面用 MessageBus 在本机模拟网络。模拟的价值是可控，但它也容易给人错觉：只要消息队列能丢包和延迟，就已经像分布式系统。真正的下一步，是把 Driver 和 worker 拆成多个进程，仍然跑在同一台 Linux 上。这个实验不需要真实集群，却会立刻暴露单进程模拟藏住的很多问题：全局变量不再共享，裸指针不能跨进程，结构体内存布局不能当协议，进程崩溃会留下临时文件，连接关闭和半写响应会成为真实事件。多进程迁移是本章最好的工程审查，因为它迫使所有“我以为它还在”的状态变成显式协议。

一、全局配置不再天然一致 单进程模拟里，Driver 和 worker 可能都读同一个全局配置对象：当前 job id、partition 数、错误策略、输出目录。拆成多进程后，worker 只能看到启动时传入的配置或消息中的配置版本。若 Driver 更新 route epoch，而 worker 仍用旧配置，旧请求就可能继续执行。解决办法不是在每个进程里随手读配置文件，而是给配置加 version，把 version 写进消息，把旧 version 请求拒绝或刷新。配置一致性从一个工程便利变成协议字段。

二、指针边界会让消息所有权错误立刻显形 单进程中，MessageBus 如果保存 payload 指针，也许测试能过，因为发送方和接收方共享地址空间。多进程后，指针值只在发送进程内有意义。于是 payload 必须被编码成字节，或者放进共享内存并带上明确 handle。这个变化非常有教育意义：它证明“消息”不是函数参数的别名，而是独立拥有对象。发送后，发送方是否还能修改，接收方何时释放，取消时谁清理，都要写进接口。

三、结构体直接写 socket 会遇到协议现实 C++ 结构体有 padding、对齐、枚举底层类型和大小端问题。单进程复制结构体没事，多进程跨版本或跨平台就不稳。教学项目迁移到 Unix domain socket 时，应定义一个简单 wire format：固定魔数、协议版本、消息类型、header 长度、payload 长度、固定宽度整数、checksum。读取时先收完整 header，再按 length 收 payload，任何长度不合法都返回 InvalidRequest。这个过程让协议格式从口头承诺变成可测试代码。

四、部分读写会替代“消息一次到达”的幻想 Socket 是字节流。一次 write 可能只写一部分，一次 read 也可能只读到半个 header。单进程队列投递一条消息就是一条消息，多进程 I/O 必须维护 offset 和 framing 状态。Driver 发送 TaskRequest 时，连接层可能先写出 12 字节，后面再写；worker 读取时也可能先拿到半个 payload。连接层因此需要自己的状态机：ReadingHeader、ReadingPayload、WritingHeader、WritingPayload、Closed、Error。远端调用又一次被拆成事件，而不是同步函数。

五、进程崩溃会留下业务副作用 单进程测试中，worker 抛异常可能被测试框架捕获；多进程中，worker 直接退出，Driver 只能看到连接关闭、心跳消失或进程状态。更重要的是，worker 退出前可能已经写出临时 block。恢复程序不能因为 worker 死了就删除所有文件，也不能因为文件存在就提交。它仍然要查 manifest、task table、attempt、checksum 和 epoch。进程边界让第 16 章的提交权威原则更真实。

六、连接关闭不是统一错误 连接关闭可能发生在请求未发送完、请求发送完但未执行、worker 执行中崩溃、响应发送一半、响应已发送但 Driver 未处理。应用层看到的都是读到 EOF 或写失败，但恢复动作不同。连接层只能报告 TransportClosed，并附带连接状态；业务层要结合 task state 判断是否重试、查询、拒绝旧响应或标记失败。把所有连接关闭都写成 Fatal，会导致过度失败；全部写成 Retryable，又会导致重复副作用。

七、心跳只能证明最近还有通信，不证明业务安全 多进程后很容易加心跳：worker 定期告诉 Driver 自己活着。心跳有用，但它不能证明某个 task 没有卡住，也不能证明 worker 输出已提交，更

不能证明旧 epoch 仍有效。心跳应该更新 worker health 和队列水位，不能替代 task attempt 状态。若 worker 心跳正常但某个 task 超时，Driver 仍要按 request 和 attempt 处理；若心跳丢失，Driver 也只能怀疑 worker，不是数学证明。

八、取消消息要接受迟到 completion Driver 超时后可以发送 Cancel，但 worker 可能已经写出结果，Cancel 也可能在执行完成后才到。Worker 收到 Cancel 时，应检查 request、attempt、deadline 和当前执行阶段：未开始可以取消，执行中在安全点取消，已写出则等待提交决定或返回 AlreadyCompleted。Driver 收到 CancelAck 也不能直接删除临时文件，因为旧 completion 可能已经进入连接队列。取消是协议事件，不是强制停止远端线程。

九、日志聚合要跨进程恢复调用栈 单进程调试可以靠栈和断点，多进程只能靠 trace id 和事件日志。每个进程写自己的事件：Driver send、connection write partial、worker read header、worker start task、worker write block、worker send reply、Driver handle reply。最终报告按 trace id 合并。若缺少 trace id，跨进程事故只能靠时间戳猜；若缺少 parent event，timer、retry 和 reply 的因果也会乱。Trace 字段是多进程调试的调用栈替代品。

十、权限和路径边界必须提前收紧 多进程 worker 不应能随意写 Driver 的任意路径。Driver 给 worker 的输出目录、job id 和 attempt id 应形成合法路径；worker 响应只返回 block id、checksum 和相对标识，Driver 自己构造 manifest 路径。这样即使 worker bug 返回奇怪路径，也不会让恢复扫描越界。安全边界在教学项目里也有意义，因为它减少误删和误读。

十一、多进程实验的最小矩阵 最小矩阵包含六个实验。第一，正常请求跨 socket 完成，输出与单进程 reference 一致。第二，请求 header 只发送一半后关闭，worker 拒绝半消息且不执行业务。第三，worker 写出 block 后崩溃，Driver 重试且 manifest 只接受一个 block。第四，响应发送一半后连接关闭，Driver 按 Unknown 处理。第五，旧 epoch worker 恢复后发送旧响应，Driver 拒绝。第六，Cancel 和 completion 交叉到达，状态机仍不重复提交。每个实验都应有 trace 和恢复报告。

十二、从多进程到真实集群还缺什么 本机多进程仍然共享文件系统、时钟更接近、网络延迟很低，也没有真实机架故障和跨机时钟漂移。它验证协议边界，不验证集群性能。下一步进入真实机器时，要重新测连接池、网络带宽、磁盘、时钟、服务发现和部署权限。但状态机、消息身份、manifest 和恢复日志不应重写。分层验证的价值就在这里：先用多进程消灭共享内存捷径，再用真实集群验证物理成本。

16.27 案例复盘：响应丢失怎样推出分布式模型

Worker 执行成功并写出结果，成功响应在返回途中丢失。Driver 只看到 timeout，于是重试。旧 attempt 和新 attempt 都可能产生输出，旧响应还可能迟到。这个事故足以推出消息身份、deadline、retry budget、幂等提交、trace、epoch 和恢复日志。

远端事实不能靠 timeout 推断 Timeout 只说明本地等待结束，不说明远端未执行。响应到达也不说明可以提交，因为它可能属于旧 attempt 或旧 epoch。Driver 处理响应时先查状态表，再校验 request、attempt、epoch、deadline 和 checksum，最后才写 manifest。成功响应只是事件，不是提交点。

实验判据 故障矩阵分别注入丢请求、丢响应、迟到响应、Busy、Driver 重启和 checksum 损坏。指标包括 attempt 数、late reply、stale epoch、retry amplification、queue depth、manifest 条目和最终 checksum。所有故障都要能重放，不能只靠随机概率。

16.28 本章习题

习题与作业

习题一：把“worker 执行成功但响应丢失”的事故写成时间线。要求列出 Driver、MessageBus、worker、manifest 四处状态，每一步说明哪些事实已知、哪些事实未知。

习题二：设计一个 **MessageEnvelope**。字段必须包含 request、attempt、deadline、epoch、trace、payload 长度和 checksum。说明每个字段解决哪个故障路径，不能只写字段名。

习题三：实现一个本机 MessageBus 的最小模型。要求 **send** 不直接调用 receiver，消息进入有界队列，支持丢响应、重复投递、延迟和 Busy。给出三个可复现故障规则。

习题四：设计幂等提交表。说明 request id、业务输出、manifest entry 和响应结果如何在同一提交边界内更新。分别讨论崩溃发生在 manifest 写入前、写入后、内存状态更新前的恢复行为。

习题五：构造一个重试风暴实验。让一个 worker 变慢，比较无预算立即重试、固定间隔重试、退避加抖动、退避加窗口四种策略。报告原始请求数、attempt 数、Busy 数、timeout 数、最终成功数和队列水位。

习题六：给 Driver 重启增加 epoch。注入一个旧 epoch 响应，让它在新 Driver 恢复后到达。说明为什么不能直接提交，以及如何通过 manifest 和 task table 决定恢复动作。

习题七：把本章模型推广到下一章分片场景。写出 route epoch、owner、leader term、commit index 与 request、attempt、driver epoch 的关系。说明哪些字段属于单个请求，哪些字段属于状态归属。

第 17 章

分片、复制与共识

第 16 章只跟踪一个 task attempt：请求可能丢、响应可能迟到、提交必须唯一。现在把问题扩大一点：日志统计系统按 key 做 reduce，某个 key 突然变成热点，partition 12 的队列持续上涨。运维同学把 partition 12 从 worker 2 迁到 worker 5，希望把压力移走。迁移过程中，旧客户端仍按旧路由把写发给 worker 2，新客户端已经按新路由把写发给 worker 5；worker 2 网络短暂隔离后恢复，还带着旧 owner 身份继续接受写；worker 5 只复制到一半历史数据就开始服务读。最后，两个 worker 都认为自己有权提交同一个 key range 的结果。

这个事故比定义更能说明本章。为什么需要分片？因为单个节点无法承担所有 key，也无法让所有 reduce 都落在一个地方。为什么需要路由版本？因为客户端和 worker 会在迁移期间看到不同的归属事实。为什么需要复制？因为只有一个副本时，worker 2 慢或死就让整段 key range 不可用。为什么需要共识？因为“partition 12 现在归谁、哪个日志位置已经提交”这类控制面事实不能靠两个节点各自宣布。为什么需要 fencing？因为旧 owner 即使还活着，也必须失去写入权。

本章沿着这个热点迁移事故展开。分片先解决数据放在哪里；复制再解决副本怎样共同保存状态；共识最后解决关键控制面事实由谁说了算。三者不是并列术语，而是同一个工程问题逐步升级后的三个回答：容量不够时拆开，单点不可靠时复制，多个副本同时可能行动时建立唯一提交顺序。

17.1 从数据布局到分片

先看最朴素的 reduce 设计：所有 worker 把同一个 key 的 partial 都发到同一个 reduce worker。它在均匀 key 上很好理解，输出位置唯一，也不需要跨节点合并。但一旦某个 key 的请求量远高于其他 key，这个 reduce worker 就会变成全局锁的集群版本。CPU 可能没满，别的 worker 可能空闲，整个 job 仍然被一个热点 key 拖住。分片就是从这里被逼出来的：不能让所有 key 都共享同一个执行位置，必须把 key 空间、输入 split、shuffle 分区和输出范围拆成多个可调度单元。

分片把数据按 key、范围或哈希分到不同节点。好的分片要负载均衡、支持扩容、减少跨分片事务。热点 key 会破坏均衡，需要拆分、缓存、限流或特殊路由。

分片后的查询可能需要 scatter-gather：向多个分片发请求，再合并结果。它类似并行归约，但网络延迟和节点故障让合并更复杂。

分片策略要匹配访问模式。哈希分片适合点查和均匀 key，范围分片适合范围扫描和排序，但容易出现热点区间。虚拟分片把逻辑分片数量做得比物理节点多，便于迁移和均衡。热点 key 不能只靠增加节点解决，因为所有请求仍然打到同一个逻辑位置；需要缓存、拆 key、限流、异步化或改变业务模型。

17.2 路由版本和再分片状态机

系统运行一段时间后，数据量和热点会变化，需要再分片。再分片要迁移数据、更新路由、处理迁移期间的读写，并保证失败后可恢复。简单哈希分片在节点数变化时可能移动大量 key，一致性哈希或虚拟分片可以减少迁移范围。

再分片最难的是迁移期间的一致性。客户端可能按旧路由写，控制面可能已经发布新路由，数据可能正在复制。常见做法是让分片有状态机：准备迁移、双写或转发、追平增量、切换路由、清理旧副本。每一步都要能失败重试。没有状态机的再分片脚本，往往在中途失败后留下难以解释的双写和漏写。

17.3 复制、Quorum 和一致性

复制保存多个副本，提高可用性和读吞吐。同步复制延迟高但数据更安全，异步复制延迟低但可能丢失最新写入。主从复制简单，multi-leader 和 leaderless 复制更复杂，需要处理冲突。

复制协议必须说明写入何时算成功。如果 leader 写本地日志就返回，延迟低但 leader 崩溃可能丢数据；如果多数副本确认后返回，延迟高但故障恢复更稳；如果所有副本确认后返回，可用性和

尾延迟都受最慢副本影响。工程系统通常让不同数据选择不同级别：元数据强一些，缓存和统计弱一些。

读写仲裁 Quorum 系统常用 W 个写副本和 R 个读副本满足 $R+W>N$ ，从而让读集合和写集合有交集。这个模型帮助理解一致性和可用性权衡。 W 大提高写安全性但增加写延迟， R 大提高读新鲜度但增加读延迟。实际系统还要处理慢副本、hinted handoff、read repair 和版本冲突。

一致性 强一致性让读者看到最新写入，但通常需要更多同步。最终一致性允许副本暂时不同，换取可用性和低延迟。选择一致性模型要看业务语义：计费、库存和元数据通常更偏强一致；缓存、推荐和日志统计可以接受延迟收敛。

17.4 Leader、日志和共识边界

Raft 和 Paxos 这类共识协议用于在故障中让多个节点就日志顺序达成一致。共识不是让所有操作都变快，而是让关键状态在故障中仍然可恢复。共识路径应尽量小而清晰，不应把所有大数据流量都压进共识日志。

共识适合维护小而关键的顺序状态，例如元数据、配置、leader 选举、分片归属和事务提交点。把大对象、热数据流和高频统计全部放进共识日志，通常会让系统吞吐受限。高性能分布式系统常把控制面和数据面分开：控制面用共识保证关键决策一致，数据面用分片、复制、批处理和校验保证吞吐。

Leader、任期和日志 以 Raft 的直觉看，系统在一个任期内选出 leader，客户端写入先进入 leader 日志，leader 复制给 follower，多数确认后提交。任期号防止旧 leader 在网络恢复后继续破坏新状态，日志索引和任期共同描述顺序。理解这些概念，能帮助读者读懂很多分布式存储的元数据路径。

读路径和租约 写路径需要顺序，读路径也要定义新鲜度。强一致读可能需要联系 leader 或多数副本，延迟较高；从 follower 读延迟低，但可能读到旧数据；租约读通过时间约束减少通信，但依赖时钟和租约安全边界。选择读路径时要问：业务是否允许旧读，旧到什么程度，故障切换时如何避免旧 leader 继续服务强一致读。

复制和备份 复制和备份经常被混为一谈。复制解决的是在线可用性：某个副本不可用时，系统还能从其他副本继续服务。备份解决的是历史恢复：数据被误删、程序 bug 写坏、勒索软件破坏或运维误操作后，系统能回到过去某个正确版本。复制通常追求低延迟和快速同步，备份通常追求隔离、保留周期和可验证恢复。一个系统可以有三个副本，仍然必须有备份。

常见误区

不要把复制等同于备份。复制会快速传播错误删除和错误写入；备份强调历史恢复。两者解决的问题不同。

分片是分布式版的数据布局 单机数据布局决定 cache line、TLB 和 NUMA 成本；分布式分片决定网络、负载和故障边界。把 key 放在哪个分片，不只是存储问题，也是计算调度问题。一个 group-by 如果按 key 分片，reduce 端可以把同一个 key 的 partial 聚到一起；一个范围查询如果按范围分片，可以少访问无关分片；一个热点 key 如果没有特殊处理，会压垮单个分片。

分片策略应从访问模式出发。哈希分片让点查和均匀聚合比较平衡，但范围扫描需要访问多个分片。范围分片支持顺序扫描和范围查询，但时间戳、递增 ID 这类 key 容易把新写入集中到尾部分片。一致性哈希适合节点增减时减少迁移，但范围查询不友好。没有通用最佳分片，只有和 workload 匹配的分片。

分片还定义故障半径。一个分片不可用，影响它负责的数据；一个节点承载多个分片，节点故障影响这些分片；一个热点分片过载，会拖慢访问它的请求。虚拟分片把逻辑分片数量做得比物理节点多，便于迁移、均衡和限流。它类似单机里把工作拆成比线程更多的任务块，让调度器有调整空间。

路由表和版本 客户端或 coordinator 需要知道 key 属于哪个分片、分片当前在哪些节点、谁是 leader、读写应该走哪个副本。这些信息组成路由表。路由表必须有版本，否则迁移和故障切换时，旧路由和新路由会混在一起。请求中通常携带路由版本或 epoch，服务端发现版本过旧时返回重定向或刷新提示。

路由表更新是控制面问题。控制面要保证“某个分片当前由谁负责”这个决定一致；数据面负责具体读写和数据传输。把控制面和数据面分开，是高性能分布式系统的重要设计。控制面通常用共识维护小而关键的元数据，数据面用分片、复制和批处理处理大量数据。

再分片状态机 再分片不能只靠一次复制命令。一个稳健再分片通常有状态机。准备阶段创建目标副本和迁移任务；复制阶段把历史数据搬到新位置；追增量阶段处理迁移期间的新写入；切换阶段发布新路由；清理阶段删除旧副本。每个阶段都要能重试，且重试不能重复破坏状态。

迁移期间读写策略有几种。可以让旧分片继续服务，目标追增量；可以让旧分片把新写转发到目标；可以短暫停写切换；也可以使用双写。每种策略都有代价。双写要处理一边成功一边失败；转发增加延迟；停写影响可用性；追增量需要日志或变更流。工程选择要看业务写入量、可用性要求和实现复杂度。

热点 key 的处理 热点 key 是分片系统的常见敌人。哈希分片只能把不同 key 分散，不能拆开同一个 key。若某个 celebrity user、热门商品或全局计数器承载大量请求，它仍会集中到一个分片。解决方法包括缓存、读写分离、key 拆分、局部聚合、限流、异步化和业务重构。

以日志统计为例，某个 event key 特别热。map 端可以先按 worker 做本地 partial，再把同一个热 key 拆成多个 subkey 发给多个 reduce 任务，最后再做二级合并。这和单机里把全局计数器改成 sharded counter 是同一个思想。热点 key 是分布式版的共享写热点。

复制的提交点 复制协议的核心是提交点。客户端写入何时算成功？如果 leader 写入内存就返回，崩溃后可能丢失；如果写入本地 WAL 后返回，单机持久性好一些但 leader 丢失时仍可能没有副本；如果多数副本持久化后返回，故障恢复更强但延迟更高。不同数据需要不同提交强度。

提交点还影响读路径。若写只有 leader 本地提交，follower 读可能很旧；若多数提交，强一致读可以通过 leader 或带任期的读协议实现；若异步复制，读扩展性好但新鲜度弱。教材要让读者看到：复制不是“多存几份”这么简单，写成功、读新鲜和故障恢复都由提交协议决定。

WAL 和状态机 很多复制系统使用 write-ahead log。先把操作写入日志，再应用到状态机。日志提供顺序和恢复依据：崩溃后可以重放已提交日志，未提交日志根据协议处理。状态机复制的思想是所有副本以同样顺序执行同样操作，就得到同样状态。

这个模型适合小而确定的操作。若操作包含读取本地时间、随机数、外部系统调用或不确定迭代顺序，就会破坏状态机一致性。因此共识日志通常记录已经决定好的命令和参数，执行必须确定。大数据流量一般不直接放进共识日志，而是把元数据、提交点和路由放进去。

Quorum 的边界 Quorum 模型用 R 、 W 、 N 解释读写集合交集，但真实系统比公式复杂。慢副本会拖高尾延迟，失败副本需要 hinted handoff 或补数据，读到多个版本需要冲突解决，时钟偏差会影响最后写入胜出的策略。 $R + W > N$ 只说明集合有交集，不自动处理冲突、删除、版本向量和故障恢复。

读修复是一种常见机制：读请求发现副本版本不一致，返回最新值的同时修复旧副本。它能逐步收敛，但把修复成本放到读路径。反熵任务则在后台比较和修复副本，减少读路径成本但收敛更慢。选择哪种方式，要看读延迟、数据新鲜度和后台资源。

一致性模型要面向业务 一致性模型不是越强越好。强一致简化上层推理，但需要更多同步和更高延迟；最终一致提高可用性和低延迟，但上层要处理旧读和冲突。业务语义决定选择。库存扣减、元数据路由、权限变更通常需要强一些；日志统计、推荐特征、缓存可以弱一些。

还要区分不同读保证。读已之写保证用户写完自己能读到；单调读保证同一客户端不会读到时间倒退的数据；前缀一致保证不会看到日志后面的操作却看不到前面的操作。很多系统不需要全局线性一致，但需要某些会话保证。精确说出需要哪种保证，比泛泛说“要一致”更有工程意义。

Raft 的工程直觉 Raft 可以从三个问题理解：谁能接受写入，日志顺序如何确定，故障后如何避免旧 leader 继续写。任期号回答领导权的新旧，日志索引回答操作顺序，多数派确认回答提交安全。leader 接收客户端命令，追加日志，复制给 follower，多数确认后提交，再应用到状态机。

Raft 的价值不是速度，而是在故障和网络分区中保持一条一致的日志。它的成本也很明确：写入通常要经过 leader 和多数副本，跨机 RTT 进入关键路径。因此工程上要把 Raft 用在小而关键的控制面，例如分片归属、配置、任务提交点和元数据，不要把所有大数据 shuffle 记录都塞进 Raft。

Leader 租约和读 强一致读如果每次都走多数派，延迟较高。leader 租约是一种优化：在租约有效期内，leader 相信自己仍是合法 leader，可以本地服务某些读。但租约依赖时间边界，必须考虑时钟漂移、网络延迟和暂停。若旧 leader 在租约误判下继续服务强一致读，就会返回过期数据。

因此租约读不是“读很快”的简单技巧，而是一套时间假设和安全边界。若系统无法可靠控制时钟和暂停，应该使用更保守的读协议。对教材来说，关键是让读者知道读路径也需要一致性设计，不是只有写路径才重要。

控制面和数据面 控制面维护少量关键决策：分片在哪里，谁是 leader，任务属于哪个 stage，哪些输出已经提交，哪些节点健康。数据面处理大量数据：读取文件、执行 map、shuffle、写中间块、reduce 输出。控制面需要强一致和可恢复，数据面需要吞吐和背压。把二者混在一起会导致系统又慢又难恢复。

Compute Systems Engine 的分布式模拟可以用一个 driver 作为简化控制面：它分配 partition，记录 task attempt，维护提交表。worker 数据面负责读输入、统计 key、发送 shuffle 消息。后续如果扩展，可以把 driver 状态写入日志或复制，但不必把每条日志记录都写入控制面共识。

备份、快照和恢复演练 复制不能替代备份，备份也不是只把文件复制到另一个目录。备份要有快照时间点、保留策略、隔离权限、校验和恢复演练。没有恢复演练的备份只是心理安慰。分布式系统中，备份还要保证多个分片之间的一致性时间点，或者记录足够日志让恢复后能达到一致状态。

计算系统也需要类似思想。长时间作业的 checkpoint 就是计算状态的备份。checkpoint 要能恢复，不只是写文件；恢复要验证输入 offset、算子状态、输出提交点和去重表是否一致。第 18 章会把它放进日志统计系统里。

分片是布局问题 分片可以看成数据布局在集群尺度上的版本。单机上，我们关心数组元素怎样落到 cache line 和 NUMA 节点；集群里，我们关心 key、文件块、任务和副本怎样落到机器。布局决定访问成本、负载均衡、故障域和扩容方式。一个错误分片策略会让后续复制、调度和缓存都很痛苦。

哈希分片把 key 均匀打散，适合点查和均匀 key，但不适合范围扫描，也不能自动处理热点 key。范围分片保持 key 顺序，适合范围查询和顺序扫描，但递增 key 容易打到最后一个分片，范围边界也可能因数据分布变化而倾斜。一致性哈希减少节点增减时迁移的数据量，适合缓存和某些存储系统，但虚拟节点数量、负载权重和热点仍要处理。没有一种分片策略能同时解决所有问题。

教学时要把访问模式列出来。点查、范围查、批量 scan、按时间写入、按用户聚合、按事件类型统计，它们需要的分片方式不同。日志统计系统如果按文件分片读取，map 阶段局部性好；shuffle 阶段按 key 分区，reduce 聚合好。一个作业里可能同时存在多种分片：输入分片、shuffle 分区、输出分区和检查点分区。每一种都要有理由。

路由缓存和失效 客户端通常会缓存路由表，否则每次请求都问控制面，控制面会成为瓶颈。缓存带来失效问题：分片迁移、leader 切换或节点故障后，客户端可能继续把请求发到旧位置。服务端收到旧路由请求时，不应静默执行，而应返回 stale route 或 redirect，让客户端刷新。

路由缓存要配版本。请求中带 route epoch，服务端比较自己的 epoch。若请求版本旧，返回新版本提示；若请求版本未来，说明客户端和服务端状态不一致，也不能盲目执行。版本检查是迁移期间避免双写和旧写的重要保护。

控制面还要考虑推送和拉取。推送能让客户端更快更新，但需要维护连接和处理丢通知；拉取简单，但更新延迟高。很多系统结合二者：错误时拉取刷新，后台周期性拉取，控制面也可以推送重大变更。教材项目可以先用错误触发刷新，保证语义清晰。

再分片期间的一致性 再分片最难的是迁移期间的新写入。历史数据复制只是第一步；复制期间旧分片仍可能收到写入。如果没有增量日志或双写，目标分片复制完历史数据后仍然落后。切换路由时，还要防止旧客户端继续写旧分片。一个完整再分片状态机必须同时处理历史复制、增量同步、路由切换和旧请求拒绝。

一种清晰策略是以日志位置为边界。准备阶段记录迁移开始位置；目标复制开始位置之前的历史数据；随后追赶从开始位置到切换位置的增量；切换时控制面发布新 epoch；旧分片拒绝旧 epoch 之后的写或转发到新分片；确认没有旧写后清理。这个策略需要每个分片有可重放日志或变更流。

另一种策略是短暂停写。它实现简单：停止旧分片写入，复制剩余数据，切换路由，再恢复写入。代价是可用性下降。对低写入量、维护窗口或教学系统，停写可能可以接受；对高可用服务，必须使用更复杂的在线迁移。工程选择要写清业务允许的停顿时间。

热点 key 的层次化处理 热点 key 不能只靠扩容解决，因为同一个 key 无论多少节点都可能落到一个分片。处理热点要分层。读热点可以用缓存、副本读、只读快照和 CDN；写热点可以用分片计数、局部聚合、异步批处理和业务限流；混合读写热点可能需要改变业务模型，例如把全局排

行榜做成分桶近似，再周期性合并。

日志统计中的热点 key 是理想例子。若所有 map worker 都把同一个 key 发到同一个 reduce partition，reduce 长尾会很明显。可以把热 key 拆成 **key, shard**，让多个 reduce worker 分别合并，再做二级 reduce。这个方案会增加最终合并阶段，但能平摊第一阶段压力。它和单机 sharded counter 完全同构。

```
hot key split:
normal key: partition = hash(key) % reduce_count
hot key: partition = hash(key, local_shard) % hot_reduce_count
final: merge all shards of the hot key
```

关键是热 key 检测和正确性。热 key 可以通过采样、历史统计或运行时指标发现。拆分后，最终输出必须把所有子 key 合回原 key，且重复 attempt 不能重复计数。热点处理不是单纯性能优化，它改变了数据流图。

复制拓扑 复制不一定是简单一主多从。常见拓扑包括 leader-follower、链式复制、quorum 复制、多主复制和无主复制。Leader-follower 写路径清楚，冲突少，但 leader 是写入瓶颈。链式复制按固定顺序传递写入，尾节点确认，吞吐和延迟有不同权衡。Quorum 复制用多数或读写集合交集提高容错，但冲突处理复杂。多主复制提高可用性，但冲突解决困难。

教材不需要把每种协议细节都展开成论文，但要让读者理解选择维度：写延迟、读延迟、故障恢复、冲突处理、运维复杂度和业务一致性。对控制面元数据，leader-follower 加共识通常更合适；对缓存或日志指标，最终一致复制可能足够；对银行账本，弱冲突解决不可接受。

日志复制和快照压缩 状态机复制如果只保留无限日志，存储会增长，恢复也会越来越慢。快照用于把某个日志索引之前的状态压缩成状态文件，之后恢复时先加载快照，再重放快照之后的日志。快照必须和日志索引绑定，不能只写一个状态文件却不知道它覆盖到哪里。

快照也有一致性边界。写快照时，状态可能仍在变化。可以暂停应用日志写快照，也可以使用 copy-on-write 或增量快照。快照写完后还要校验，确保恢复能读。很多系统的故障不是因为没写日志，而是从未演练过快照恢复，真正崩溃后发现快照损坏或版本不兼容。

Compute Systems Engine 的简化版本可以把 driver 元数据写成 checkpoint：包含路由 epoch、task 状态、已提交 attempt 和 shuffle manifest。每次 stage 完成后写 checkpoint，恢复时读 checkpoint 并校验文件存在和摘要匹配。这个训练比空谈“做 checkpoint”更实用。

Quorum 的读写路径 用 N 表示副本数， W 表示写入需要确认的副本数， R 表示读取需要查询的副本数。如果 $R + W > N$ ，读写集合至少有交集，读有机会看到最新写。但“有机会”不等于自动返回正确值。读路径需要版本比较，可能读到多个版本；写路径需要处理并发写；删除需要 tombstone；故障恢复需要补数据。

例如 $N=3, W=2, R=2$ 。写入成功到两个副本后返回。读取两个副本，如果其中一个有新版本，另一个旧版本，客户端或协调者要选择新版本，并可能发起读修复。若两个并发写写到不同副本集合，版本冲突需要解决。若用最后写入时间胜出，时钟偏差可能丢数据。Quorum 是框架，不是完整数据库。

一致性模型的例子 线性一致要求每个操作看起来在调用和返回之间某个瞬间生效，并且所有客户端看到同一个全局顺序。它最容易给上层推理，但成本高。顺序一致要求所有操作有某个全局顺序且每个客户端程序顺序保持，但不要求实时顺序。因果一致保留因果相关操作顺序，不强制无关操作顺序。最终一致保证没有新写时副本最终收敛。

业务常常需要的是特定保证。用户修改密码后，自己必须读到新密码，这是读己之写；用户连续刷新页面，不应看到版本倒退，这是单调读；日志统计可以最终一致，但审计记录不能丢。教材要训练读者说具体保证，而不是笼统说“强一致”或“弱一致”。

租约的时间假设 租约允许某个节点在一段时间内拥有权利，例如 leader 读、本地缓存写入或分片所有权。租约依赖时间，因此必须处理时钟漂移、进程暂停和网络延迟。安全租约通常要求保守的时间边界：授予方认为租约到期之前，持有方必须更早停止使用；持有方不能因为本地时钟慢而延长权利。

进程暂停是租约的敌人。一个节点拿到 10 秒租约后暂停 30 秒，恢复时如果只看本地状态，可

能以为自己仍有权利。正确系统要在使用租约前检查当前时间和任期，或者使用更保守的共识读。租约能优化读路径，但不能成为绕过一致性的借口。

控制面数据的规模 控制面适合维护少量关键元数据，不适合承载大数据流。把每条 shuffle 记录都写进共识日志，会让控制面被数据面拖垮。正确做法是控制面记录 stage、partition、attempt、manifest、提交点和路由，数据面传输大批量数据文件或消息。控制面决定“哪些数据有效”，数据面负责“搬运数据”。

这个边界和单机运行时类似。线程池控制面维护任务状态，任务函数处理大数组；队列控制面维护 head/tail，buffer 承载数据；分布式 driver 控制面维护 manifest，worker 数据面写中间块。系统设计的很多层次都在重复这个分离。

备份和复制的区别 复制用于可用性和读扩展，备份用于灾难恢复和人为错误恢复。若用户误删数据，复制会迅速把删除同步到所有副本；只有独立备份能恢复旧状态。若软件 bug 写坏数据，复制也会复制坏数据。备份必须隔离权限、保留历史版本、定期校验和演练恢复。

快照还要考虑一致点。多分片系统如果分别备份每个分片，恢复时可能得到不同时间点的状态。可以使用全局 checkpoint barrier，也可以记录每个分片的日志位置，恢复后重放到一致点。计算作业的 checkpoint 同样如此：输入 offset、算子状态和输出提交表必须对应同一个逻辑时间。

恢复演练流程 恢复演练应写成流程，而不是口头承诺。第一步，选择一个备份或 checkpoint。第二步，在隔离环境恢复元数据和数据文件。第三步，运行校验：行数、校验和、manifest、去重表、路由 epoch。第四步，执行一组只读查询或模拟任务，确认业务语义。第五步，记录恢复耗时和失败点。没有演练，就不知道 RTO 和 RPO 是否真实。

Compute Systems Engine 可以模拟 driver 重启：保存 checkpoint 后删除内存状态，重新加载 checkpoint，继续运行未完成任务。测试要覆盖 checkpoint 前崩溃、checkpoint 后响应丢失、重复 attempt 到达和 manifest 文件缺失。这样恢复不再是抽象概念。

成员变更 复制组不是永远固定。节点会扩容、缩容、维护、故障替换。成员变更比普通写入更危险，因为它改变 quorum 的定义。若旧配置和新配置没有安全交叠，可能出现两个不同多数派，各自认为自己能提交，造成脑裂。成熟共识协议会把成员变更作为日志中的配置变更，并使用联合共识或分阶段切换，确保新旧配置之间有交集。

教学阶段不必实现完整成员变更，但必须理解控制面不能随意改副本列表。一个 driver 如果把 partition 从 worker A 移到 worker B，要确保旧 worker 不再接受新 epoch 写入，新 worker 已经拥有必要状态，客户端路由刷新。成员变更是再分片、故障恢复和复制的一部分。

脑裂和 fencing 脑裂指两个或多个节点同时认为自己拥有同一资源的写权限。网络分区、旧 leader 暂停后恢复、租约误判都可能造成脑裂。解决脑裂需要 fencing：让旧持有者即使还活着，也无法继续对资源产生有效写入。常见 fencing token 是单调递增 epoch、term 或 lease version。下游资源只接受最新 token 的写入。

例如一个 worker 持有 partition epoch 5 的写权限。控制面故障切换后，把 partition 交给另一个 worker，epoch 变成 6。旧 worker 恢复后仍尝试提交 epoch 5 输出，driver 或存储层必须拒绝。没有 fencing，旧 worker 的迟到写会覆盖新结果。Epoch 不只是日志字段，它是防旧写的安全边界。

Leader 读和线性一致 Leader 处理写入并不自动意味着 leader 本地读一定线性一致。若 leader 已经失去领导权但还不知道，它可能返回过期读。解决方法包括读前确认自己仍拥有 quorum、使用 lease 且满足时间假设、或把读也走日志。不同方法在延迟和安全性之间取舍。

很多系统提供多种读：强一致读、leader 本地读、follower stale read、快照读。业务应选择所需语义。配置和权限通常需要强一些；监控和推荐可以接受旧读；日志统计作业读取 checkpoint manifest 时要确保 manifest 已提交。教材要训练读者把读语义说清，而不是只关注写复制。

冲突解决 无主或多主复制会遇到并发写冲突。冲突解决可以是最后写入胜出、版本向量、CRDT、业务合并或人工处理。最后写入胜出简单但可能丢数据，且依赖时钟；版本向量能检测并发但元数据更大；CRDT 适合某些可合并数据类型，例如集合、计数器和寄存器变体；业务合并需要理解语义。

日志统计中的计数器是可合并的，多个副本的增量可以求和；订单状态不是简单可合并，两个并发状态变更可能冲突。数据类型决定复制策略。不要把最终一致当成一个开关，它需要冲突语义。

删除和 Tombstone 复制系统中，删除不能简单地从某个副本移除数据。若一个副本离线时错过删除，恢复后可能把旧数据重新传播回来。Tombstone 用一个删除标记表示“这个 key 已被删

除”，让其他副本也能学习删除。Tombstone 需要保留一段时间，直到确认旧副本不会再带回旧值。

Tombstone 的代价是空间和查询成本。保留太短，旧数据复活；保留太长，存储膨胀。后台 compaction 会清理安全 tombstone，但必须知道所有副本或日志位置已经越过删除点。这个细节说明复制系统的“删除”比单机 map erase 复杂得多。

反熵和 Merkle tree 副本之间可能因为故障、丢包或离线而不一致。反熵任务在后台比较副本并修复差异。直接比较所有数据成本高，Merkle tree 可以把数据分块计算哈希，先比较上层哈希，不同再下钻到具体范围。它适合大规模 key-value 副本修复。

反熵是最终一致系统的重要补充。读修复只修复被读到的 key，冷数据可能长期不一致；反熵后台扫描能逐步收敛，但消耗 I/O 和网络。调度反熵时要避免影响前台请求。Compute Systems Engine 的简化恢复可以用 manifest checksum 类似思想：先比较摘要，再决定是否重读 block。

检查点和日志截断 复制日志不能无限增长。生成快照后，可以截断快照索引之前的日志，但前提是所有需要恢复的节点都已经拥有快照或不再需要旧日志。若 follower 落后太多，leader 可能发送快照而不是从很旧日志补起。日志截断是存储空间和恢复能力的平衡。

计算作业也有类似问题。Stage 完成并 checkpoint 后，是否可以删除中间 shuffle 文件？如果后续 stage 失败需要回滚，删除太早会导致无法恢复；保留太久占用磁盘。系统要定义保留策略：保留到下一个 checkpoint 成功，保留到作业完成，还是按时间清理。清理也是协议的一部分。

配置变更的审计 控制面配置变更应可审计。谁把 partition 从 A 迁到 B，路由 epoch 多少，何时生效，旧副本何时清理，是否完成校验，都应记录。没有审计，出现数据错乱时无法追踪。分布式系统的操作日志和业务日志同样重要。

本书项目可以把 driver 的每次状态变更写入简单事件日志：assign partition、start attempt、commit attempt、publish manifest、checkpoint、recover。这个日志用于调试和恢复演练，不需要一开始就高性能。先有可见性，再谈优化。

一致性选择表 为业务选择一致性时，可以列一个表：数据类型、读写比例、是否允许旧读、是否允许冲突、丢失风险、延迟要求、恢复要求。比如路由表：小、读多写少、不允许旧写、需要强一致；日志指标：大、写多读少、允许短暂旧读、可合并；输出 manifest：小、提交关键、不允许重复、需要持久化。表格化思考能避免一句“用强一致”覆盖所有数据。

Compute Systems Engine 的不同状态也应有不同保证。任务状态和 manifest 需要强提交；worker 运行指标可以最终一致或近似；日志统计中间 partial 可以重复生成但最终提交要去重；调试 trace 可以丢弃低优先级事件。把所有状态都放进同一个强一致路径，会让系统过慢；把所有状态都弱一致，会让结果不可信。

分区元数据的数据结构 分区元数据至少包含 partition id、key range 或 hash range、leader、followers、epoch、状态和迁移目标。状态可以是 Serving、MigratingOut、MigratingIn、ReadOnly、Offline。请求处理前先检查 partition 是否 serving 且 epoch 匹配。迁移期间旧 partition 可以进入 ReadOnly 或 Forwarding，避免继续接受旧写。

Listing 17.1 分区元数据的教学形态

```
1 enum class PartitionState : std::int32_t {
2     Serving,
3     MigratingOut,
4     MigratingIn,
5     ReadOnly,
6     Offline,
7 };
8
9 struct PartitionMetadata {
10     std::uint32_t partition_id = 0;
11     std::uint64_t epoch = 0;
12     std::uint32_t leader_worker = 0;
13     PartitionState state = PartitionState::Offline;
14 };
```


这段结构没有包含完整副本列表和 key range，但展示了关键字段。Epoch 和 state 是防旧请求的基础。若请求只带 partition id，不带 epoch，服务端很难区分旧路由请求和当前合法请求。

迁移校验 再分片或副本迁移完成后，需要校验。校验可以比较记录数、字节数、checksum、Merkle tree 或抽样查询。没有校验就切换路由，可能把不完整副本变成主副本。校验成本可以很高，因此系统常在后台增量校验，但切换关键状态前至少要有基本一致性检查。

Compute Systems Engine 的迁移简化为 shuffle manifest 校验。Map 输出 block 写完后记录 checksum，reduce 读取前校验；checkpoint 恢复时再次校验。这个小机制训练的是同一思想：状态切换前证明数据完整。

Read repair 的成本归属 读修复把副本修复放到读请求路径中。读者发现旧副本后，返回结果同时发修复写。这样冷数据可能长期不修复，热数据很快收敛。缺点是读路径延迟和写放大增加。若前台延迟敏感，读修复要异步或限流；若一致性要求高，可能需要同步修复或强读。

反熵把修复放后台，前台读更稳定，但收敛慢。两者可以同时使用。教材要让读者看到：一致性维护成本总要付，要么前台付，要么后台付，要么接受更长不一致窗口。

租约和缓存失效 客户端缓存路由表或配置时，可以用租约限制缓存有效期。租约过期后必须刷新。租约能减少控制面查询，但引入时间假设。另一种方式是版本检查：客户端可以长期缓存，但每个请求带版本，服务端发现旧版本就拒绝。版本检查不依赖客户端准时过期，更稳，但每次请求都要服务端判断。

很多系统结合二者：正常情况下按租约周期刷新，异常时根据 stale route 错误立即刷新。这样控制面负载低，迁移时也能快速纠错。Compute Systems Engine 的 route epoch 就是版本检查的简化形式。

副本滞后指标 复制系统应记录副本滞后。日志复制可以用 last log index 差距，异步复制可以用字节或时间，计算 checkpoint 可以用 stage 或 task 数。没有滞后指标，读旧副本是否安全无法判断，故障切换时也不知道候选副本差多少。副本滞后还影响备份和清理：落后副本可能需要旧日志，不能过早截断。

指标要按 partition 维度记录。全局平均滞后小，不代表每个分区都健康。一个热点 partition 的 follower 落后，故障时就会影响关键数据。分片系统的一切指标都应带 partition 维度。

共识的使用边界 共识很强，但不应滥用。适合放进共识的是少量关键决策：leader、路由、配置、任务提交、manifest 指针。不适合放进共识的是大批数据记录、每条日志原文、每个中间 partial。把所有东西都放进共识会让系统吞吐被最慢副本和日志复制限制。

正确模式是：共识决定哪份数据有效，数据本身走高吞吐路径。比如 driver 通过强一致状态记录某个 shuffle block 已提交，block 内容存储在文件或对象存储中；控制面记录 block 的 checksum 和路径，数据面负责传输和读取。这种控制面/数据面分离是大型系统的基本结构。

操作手册 分片复制系统需要操作手册。节点下线前如何迁移分片，节点故障后如何提升副本，路由错误如何回滚，checkpoint 损坏如何恢复，热点 key 如何拆分，旧副本何时清理。没有操作手册，系统只能在正常路径运行，一遇到故障就靠临场判断。教材项目可以写简化 runbook，训练工程思维。

Runbook 应包含前置条件、执行步骤、验证指标、回滚方案和风险。比如“迁移 partition 7”：确认目标 worker 健康，创建迁移任务，复制数据，校验 checksum，发布 epoch，观察 stale route 错误下降，清理旧副本。这个流程比单独讲算法更接近真实工程。

分片复制章节总结 分片是集群尺度的数据布局，复制是失败下保持服务和数据的机制，共识是少量关键决策的安全顺序。分片要考虑访问模式和热点，复制要定义提交点和读语义，共识要限制在控制面，恢复要靠快照、日志、校验和演练。一个系统能否可靠，不取决于是否“有副本”，而取决于旧写是否被挡住、提交点是否清楚、恢复是否演练过。

分片复制决策表 选择分片和复制策略时，先问：查询是点查还是范围，写入是否递增，是否有热点 key，是否允许旧读，写成功后允许丢失吗，故障切换能等待多久，副本是否跨故障域，删除如何传播，备份如何恢复。每个答案都会改变设计。哈希分片、范围分片、同步复制、异步复制、quorum、Raft、租约、读修复都不是孤立选项，而是这些约束的组合结果。

复制协议审查表 每个复制协议都应审查写路径、读路径、故障路径和恢复路径。写路径：写到哪里算成功，是否写 WAL，是否多数确认。读路径：读 leader 还是 follower，是否可能旧读，如何

保证读己之写。故障路径：旧 leader 如何被 fencing，慢副本如何处理，成员变更如何保证 quorum 交叠。恢复路径：快照和日志如何重放，tombstone 保留多久，备份如何演练。缺任何一项，复制都不完整。

分片复制章的回归阅读要从状态归属开始。一个 key 属于哪个 partition，路由表版本是多少，迁移期间旧 owner 是否还能提交，新 owner 何时接管，这些问题比“哈希一下”更重要。没有版本和所有权轮次，热点拆分、重分片和故障恢复都会出现旧请求覆盖新结果的风险。

复制不是免费提高可靠性。多数写需要定义 quorum、leader、日志位置和读写语义；异步副本可以提高吞吐，但会产生滞后；旧 leader 如果继续接受写入，系统会分裂。教材不必在第二册实现完整 Raft，但必须讲清共识解决的是“谁有权提交”，不是“让机器多几份副本”这么简单。

项目中的 **OwnershipEpoch** 应进入每个路由和提交请求。driver 看到旧 epoch 的响应要拒绝，worker 迁移后要停止旧所有权下的提交，manifest 要记录提交时的 epoch。这个接口能把分片、复制和最终项目连起来，也给第三册服务化推理的模型热更新留下基础。

重分片要写生命周期。准备阶段先生成新路由计划，复制或重算目标分片需要的数据；双写或冻结阶段要定义旧 owner 和新 owner 各自能处理哪些请求；切换阶段提高 epoch 并发布路由；清理阶段删除旧副本和临时状态。每一步都可能失败，所以每一步都要可重试或可回滚。若只写“迁移到新节点”，就跳过了分布式系统最危险的部分。

复制读写还要和业务语义绑定。监控指标可以读稍旧副本，计费 and 提交表不能接受旧读；热点 key 可以拆成多个子 key 再二级 reduce，但需要定义最终合并顺序；多数写可以提高故障容忍，却增加尾延迟和写放大。分片复制不是单纯的数据结构问题，它把延迟、可用性、一致性和业务正确性绑在一起。

热点 key 的处理不能只靠增加节点。若某个 key 占据大部分流量，普通 hash partition 会把它集中到一个 reducer；增加 reducer 数量只能让其他 key 更分散，热点仍然长尾。常见处理是 map-side combine、把热点 key 拆成子分片、分两级 reduce，或给热点单独策略。每种方法都要说明是否保持精确、是否改变输出顺序、是否增加最终合并成本。

共识边界要保持清楚。第二册不实现完整生产级共识，但要让读者知道何时需要它：多个节点都可能认为自己有提交权，且错误提交不可接受时，就需要共识或等价的强协调。若只是本地教学模拟，可以用单 driver 加持久 manifest 简化；若 driver 也要容错，就必须引入日志、任期和多数派。范围控制不是逃避，而是把问题边界说清楚。

路由表发布也要遵守版本协议。driver 生成新路由后，worker 不能凭本地缓存永久处理旧分片；请求必须携带 routing version 或 ownership epoch；worker 收到旧版本任务时可以拒绝或转发，但不能沉默执行。客户端同样要处理 NotOwner 或 StaleRoute 响应，再刷新路由。没有这些响应类型，重分片期间的错误会表现成随机超时和重复提交。

副本落后要有恢复路径。一个 follower 断开后重新加入，不能直接认为自己数据可用；它需要知道自己落后到哪个日志位置或 manifest 版本，从 leader 或其他副本补齐，再恢复服务。若数据量很大，全量复制可能太慢，需要 snapshot 加增量日志。第二册不展开数据库实现，但应让读者看到复制不是“多写几份文件”，而是一套版本、补齐和可见性协议。

热点拆分还会影响观测。把一个 hot key 拆成多个子 key 后，指标系统不能只看子 key，否则会以为热点消失；最终报告要能把子 key 重新聚合，说明拆分前后的总量、长尾和二级 reduce 成本。系统优化不能让业务观察失真。这个原则在第三册 batch 调度和模型服务限流中也会继续出现。

17.5 分区、复制和共识都从状态归属推导

分布式计算的第一步不是复制，而是决定数据归谁管。一个 key、一个 partition、一个 task、一个 manifest entry，在任意时刻必须有清楚的归属。归属决定请求路由、负载均衡、故障恢复和提交责任。若归属不清，复制会放大混乱：多个副本都以为自己能提交，旧路由继续接受写入，恢复时无法判断哪个状态最新。

分区是把大状态拆成可管理的责任单元。日志统计中，event key 可以按 hash 分到 reduce partition；任务可以按 input split 分到 map worker；输出可以按 partition 写成 block。每个分区要有 id、版本、owner、状态和迁移规则。分区不是只为并行，也为恢复和观测服务。

分区函数、热点和迁移

Hash 分区简单，但不保证负载均衡。若少数 key 占大多数记录，它们会形成热点 reduce。范围分区保留顺序，但需要采样和边界；一致性 hash 适合动态节点，但不自动解决热点；热点 key 单独

拆分能降低长尾，但会增加合并和确定性复杂度。分区策略要从 key 分布和查询/计算模式推导。

分区函数需要版本。作业进行中改变分区规则，会让相同 key 前后落到不同目标，结果错误。Manifest 和 shuffle block 应记录 partition version，reduce 端拒绝混用。滚动升级场景下，新旧 worker 可能同时存在，协议必须允许兼容或明确禁止混跑。教材要让读者看到版本不是文档字段，而是正确性边界。

迁移要有两阶段。先冻结或复制旧分区状态，再让新 owner 接管，再拒绝旧 epoch 请求。迁移期间可能有 in-flight 请求，必须通过 epoch 或 lease 区分。单机任务运行时里的“任务从一个队列移到另一个队列”是这个问题的最小版本，分布式里只是成本和失败模式更大。

复制解决可用性，也引入一致性问题

复制让状态在节点故障后仍可用，但多个副本之间必须决定谁能接受写、读到什么版本、冲突如何处理。主从复制简单：leader 接受写，follower 复制；leader 故障需要选主。多主复制提高可用性，但冲突解决复杂。Quorum 读写通过多数派交集保证一定一致性属性，但延迟和尾部更高。

复制日志要有顺序。若两个写入在不同副本上顺序不同，状态机结果可能不同。确定性状态机复制要求所有副本按同一日志顺序执行相同命令。这里连接到第一册和第二册前面的确定性：并行计算里固定合并顺序是为了可复现，复制协议里固定日志顺序是为了副本一致。

读路径也要定义。读 leader 最简单，可能成为瓶颈；读 follower 延迟低但可能读旧；带版本读可以让客户端选择一致性等级。教材不需要把所有数据库模型讲完，但要把“复制不是免费备份”讲清：它把单点故障变成一致性、延迟和运维问题。

共识解决的是谁有权提交

共识不是所有分布式系统的开头，而是在多个节点可能同时认为自己有权提交时需要的机制。它要解决在故障、消息延迟和重试下，系统仍然只选出一个值或一个日志顺序。Raft/Paxos 的细节很深，本册第二册只需建立工程直觉：term/epoch 防止旧 leader，log index 表示顺序，majority commit 表示足够副本确认，apply 表示状态机可见。

把共识用于贯穿项目时，不必实现完整 Raft。可以做简化：driver 有 epoch，worker 响应带 epoch；driver 重启后 epoch 增加，旧 epoch 响应被拒绝；manifest commit 采用单 leader 文件表。这样读者先理解 epoch 和提交权限。完整共识适合后续专门项目，不应在本册中抢走计算主线。

Lease 是另一种常见思想。某个节点在时间窗口内拥有权限，但依赖时钟和续约。若时钟漂移或网络分区处理不当，旧 lease 可能造成冲突。教学中可以比较 epoch 和 lease：epoch 不依赖物理时间但需要中心化或共识推进，lease 降低通信但引入时间假设。

备份、恢复和演练

复制不等于备份。副本会忠实复制错误删除、错误写入或程序 bug；备份提供历史恢复点。分布式系统要设计 checkpoint、snapshot、WAL、manifest 和校验。恢复不是上线后才想，而是协议的一部分。每个提交点都要能回答：从哪里重放，如何验证，如何跳过重复，如何清理垃圾。

项目实验可以模拟三个 reduce partition，每个 partition 有主副本和一个备份 manifest。注入 leader 崩溃、旧响应、重复 commit 和分区迁移。目标不是实现工业数据库，而是让读者亲自看到：分区归属、复制状态和提交权限必须一起设计。

本章最终判断力是：遇到分布式状态，先问归属；需要可用性，再问复制；多个节点可能争提交，再问共识；需要历史恢复，再问备份。顺序不能乱，否则概念会堆成清单。

案例走查：旧 epoch 请求为什么必须拒绝

Driver epoch 1 把 partition 3 分给 worker A。A 处理较慢。Driver 重启或重新配置后进入 epoch 2，把 partition 3 分给 worker B。若 A 的旧响应在 B 提交后到达，系统必须能识别它属于旧 epoch。没有 epoch，旧响应可能覆盖新结果；有 epoch，driver 拒绝旧响应并清理临时输出。

这个案例是共识直觉的最小版本。系统不一定实现完整 Raft，但必须有“谁现在有权提交”的版本概念。Epoch 增加后，旧 leader、旧 worker、旧 route 都失去提交权。消息、manifest 和状态表都要携带 epoch，否则版本只存在于内存想象里。

实验模拟 epoch 切换、旧响应迟到和重复 commit。期望新 epoch 输出可见，旧 epoch 只记录 stale response。这个实验能让读者理解分区和复制不是静态数据结构，而是带时间和权限的协议。

复制实验不要从完整共识开始

完整 Raft 或 Paxos 实现超出本册主线。教学实验可以从单 leader 加 epoch 开始：只有当前 epoch 的 driver 可以 commit，worker 响应必须带 epoch，旧 epoch 被拒绝。这个实验已经能解释

旧 leader、旧路由和迟到响应问题。等这个模型清楚后，再阅读共识协议会更容易。

第二步可以加入多数确认的简化模型。Manifest 写入三个副本，至少两个成功才认为提交。这里不实现 leader election，只研究 quorum commit 的可见性和失败路径。若一个副本慢，commit 延迟受尾部影响；若两个副本失败，系统不可提交；若成功后一个副本落后，恢复时要按版本补齐。读者能看到复制带来的可用性和延迟取舍。

第三步再讨论为什么真正系统需要 leader election、日志匹配和安全性证明。这样共识不是凭空出现，而是从“谁有权提交”和“多数确认怎样恢复”自然推进。

分区元数据的实现边界

项目可以定义 **PartitionAssignment**: partition id、epoch、owner worker、replicas、version、state。状态包括 Active、Migrating、Draining、Closed。迁移时先进入 Migrating，停止接受新写或只接受带新 epoch 写；旧 owner drain in-flight；新 owner 接管；最后关闭旧路由。这个状态机比一句“迁移分区”可靠得多。

分区元数据要持久化。若 driver 重启后忘了当前 epoch 和 owner，就无法判断旧响应。持久化不一定复杂，可以先写本地 JSON 或文本表，再加 checksum。重点是恢复路径可测试。重启后读 assignment，拒绝旧 epoch，重新调度未完成 partition。

作业可以要求读者手工演算一次迁移：partition 7 从 worker A 到 B，期间 A 的 attempt 3 返回，B 的 attempt 4 返回，driver 重启一次。写出哪个响应被接受，哪个被拒绝，manifest 最终是什么。这个演算能检验概念是否真正理解。

综合练习：分区迁移和副本落后的恢复

本题设置 partition 5 有主副本 A 和备副本 B。A 接受写入到版本 10，B 只复制到版本 8。此时 A 崩溃，系统要决定能否让 B 接管。若没有其他副本或日志，B 接管会丢失版本 9 和 10。读者需要说明哪些条件下可以接管，哪些条件下必须等待恢复或回滚。

第二步加入 quorum。三个副本 A、B、C，写入必须两个确认。若 A 崩溃，B 到 10，C 到 10，B 可以接管；若只有 A 到 10，B 和 C 到 8，版本 10 未提交。这个简化模型能让多数派交集变得具体。不要先背 quorum 定义，先看恢复时哪些版本可信。

第三步加入分区迁移。Partition 5 从 A 迁移到 D，迁移过程中旧 epoch 的写入到达。读者要写出 assignment 状态、epoch、允许写入集合和拒绝规则。副本落后和迁移叠加后，系统最容易出错。题目要求只处理一个 partition，不许泛泛说“用共识解决”。

这个题的目标，是把复制和共识从宏大概念压回状态归属。只要读者能判断哪个版本已提交、哪个副本可接管、哪个 epoch 有权写，就已经掌握本章核心。

容易出错的边界：复制常制造新的不确定性

第一个反例是把复制当备份。程序错误删除数据，副本也会删除；备份需要历史版本和恢复点。第二个反例是 follower 读。读 follower 可能读旧数据，是否允许要写进一致性合同。第三个反例是多主写入。多主提高可用性，但冲突解决可能非常复杂，不能靠“最后写 wins”糊弄所有业务。

第四个反例是分区迁移无版本。旧 owner 的响应迟到，新 owner 已提交，没有 epoch 就无法拒绝旧结果。第五个反例是 quorum 数字背诵。多数派有用是因为读写集合相交，恢复时能找到已提交版本；如果读者不能解释交集，就只是背公式。第六个反例是把共识用于所有数据面。大数据 block 不应全部进入共识日志，控制面提交元数据即可。

本章源码索引要求把控制面和数据面分开。Partition assignment、epoch、manifest 属于控制面，需要强一致；shuffle block 和大输出属于数据面，需要校验和可重做。把大数据塞进控制面会拖垮协议，把控制状态放进普通文件又会破坏恢复。这个边界对后续 AI 模型 artifact 管理也很重要。

实验中要让读者故意读旧副本，观察结果怎样偏离。再加入版本检查，让旧读被拒绝或标记 stale。这样的反例比直接讲一致性级别更直观。

本章核心接口：OwnershipEpoch

分区复制章给项目留下 **OwnershipEpoch**。任何可提交对象都带 owner、epoch 和 version。Owner 表示当前谁有权处理，epoch 表示权限轮次，version 表示状态内容版本。旧 epoch 的消息可以被记录，但不能提交；旧 version 的副本可以恢复，但不能覆盖新版本。

这个接口比抽象谈共识更贴近项目。Driver 重启、分区迁移、worker 迟到响应、manifest 更新都通过 OwnershipEpoch 判断权限。第三册管理模型 artifact 时，也会用同样思路判断哪个文件版本可见。

从状态归属进入完整计算引擎

分区、复制和共识解决的是控制面：谁有权处理和提交状态。最后一章把控制面和数据面合在一起，形成完整计算引擎。Partition assignment 决定 reduce block 去哪里，manifest commit 决定哪些输出可见，replica/version 决定恢复时信谁。数据面负责高吞吐处理 block，控制面负责少量但关键的状态变化。

项目中必须保持这个分工。不要把每条日志记录写进共识，也不要让大 block 决定自己的提交权限。大数据用 checksum 和可重做保护，控制状态用 epoch、version 和 manifest 保护。这个分工是从单机运行时到分布式系统都通用的工程原则。

17.6 项目落地

Compute Systems Engine 在本章应加入分片元数据和提交表。分片元数据描述 key 到 partition 的映射、partition 到 worker 的映射和版本号。提交表记录 task attempt、request id、输出文件或结果摘要。第一版可以由 driver 单进程维护，后续再讨论复制和持久化。

项目还要实现热点 key 处理实验。构造一个 key 分布高度倾斜的输入，比较普通 hash partition、map-side combine、热 key 拆分和二级合并。这个实验能把单机 sharded counter、分布式 partition、shuffle 和 reduce 长尾串起来。

分片归属 每个 key 或 partition 必须有归属。归属决定谁处理请求、谁保存状态、谁生成 checkpoint。没有归属，恢复和扩容都会变成全局扫描。

路由版本 客户端路由表会过期。每次路由变化都要有 epoch 或 version，旧版本请求要被拒绝、转发或按迁移规则处理。

再分片 再分片不是拷贝数据那么简单。迁移期间可能同时有旧 owner、新 owner 和 in-flight 请求。状态机要表达 copying、catching up、switching、committed。

复制 复制提供容错，但引入一致性选择。主从复制、quorum、链式复制和共识日志各有延迟、吞吐和故障语义。

quorum quorum 的关键是交集。读写集合有交集，才能让新读看到旧写或发现更新。不是副本数越多越安全，还要看提交规则。

leader leader 是某个任期内有提交权的副本。旧 leader 可能还活着，但任期过期后不能继续提交。term 解决的是身份和时间顺序。

日志 共识日志把操作排序，状态机按日志顺序执行。snapshot 和日志截断必须保证已提交状态能恢复，不丢必要历史。

CAP 边界 CAP 不是背口诀。分区发生时，要选择继续响应还是保持强一致。工程讨论必须具体到读、写、超时和恢复场景。

案例：路由过期 driver 用旧 route table 向旧 partition 写入。服务端根据 epoch 拒绝，并返回刷新指令。

案例：quorum 读写 三个副本，写两个算成功，读两个并选择最新版本。让一个副本落后，说明交集如何发现新值。

案例：snapshot 截断 提交若干日志后生成 snapshot，再截断旧日志。重启时用 snapshot 加后续日志恢复。

源码阅读路线 Linux 源码阅读从 [fs/jbd2](#)、[kernel/locking](#)、[net/core/dev.c](#) 附近进入即可。阅读目标不是把整套子系统抄进书里，而是追一条和本章实验相关的路径：用户态动作怎样变成内核对象，内核对象怎样排队、等待、唤醒、计数、提交或回收。

阅读报告要写证据等级。能直接测到的数字放在第一层；从源码控制流推导出的解释放在第二层；当前设备无法验证、只能作为合理猜测的内容放在第三层。这样读者会知道状态归属报告中哪些结论可复现，哪些结论需要换机器继续确认。

源码阅读还要克制。看到一个复杂函数，不要把每个分支都展开；先回到本章的问题，问它能否解释一个等待点、一次状态转移或一条性能曲线。解释不了的内容可以留到索引，不要打断正文主线。

路由过期 driver 用旧 route table 向旧 partition 写入。服务端根据 epoch 拒绝，并返回刷新指令。

quorum 写读 三个副本，写两个算成功，读两个并选择最新版本。让一个副本落后，说明交集如何发现新值。

snapshot 截断 提交若干日志后生成 snapshot，再截断旧日志。重启时用 snapshot 加后续日志恢复。

状态归属

分布式系统首先要问状态归谁。任意副本都能写看似可用，故障和分区时会立刻产生冲突。

实验设计：构造两个副本同时接受写入，观察脑裂结果。

分片

分片把 key 空间切成多个归属范围。按 hash 简单但范围查询弱，按 range 支持范围但容易热点。

实验设计：对均匀 key 和热点 key 比较两种分片。

路由版本

客户端缓存旧路由时，请求可能发到旧 owner。route epoch 让旧路由被明确拒绝，而不是悄悄写错位置。

实验设计：扩容迁移时让旧客户端继续写，检查 epoch 拒绝。

再分片

再分片不是复制完就切换。它包含准备、全量复制、追增量、切换路由和清理旧状态，每步都要可重试。

实验设计：在每个状态注入失败，写恢复表。

复制

复制提升容错，也引入一致性选择。主从、quorum、链式复制和共识日志的延迟与故障语义不同。

实验设计：比较写一个副本成功和多数派成功的读写结果。

quorum

多数派读写通过交集保证看到最新提交，但它仍需要版本、冲突处理和故障超时。

实验设计：构造三副本一慢一故障，计算读写是否能完成。

leader

leader 让提交顺序集中，但 leader 失效要用 term 或 epoch 防止旧 leader 继续写。

实验设计：旧 leader 恢复后尝试提交旧 term 写入，检查拒绝。

共识边界

共识解决日志顺序和提交，不解决业务幂等、存储格式、热点 key 和查询优化。不要把所有问题推给共识。

实验设计：把 request id 去重放在状态机层，而不是只依赖日志。

租约

租约用时间限制所有权，但依赖时钟假设和安全余量。强一致路径不能随意把租约当作永久证明。

实验设计：讨论时钟漂移和暂停进程对租约的影响。

项目接口

ReplicaGroupPlan 记录分片范围、epoch、副本列表、leader、quorum、commit index、迁移状态和恢复动作。

实验设计：第 18 章 shuffle 输出提交复用 route epoch 和 manifest。

17.7 分片、复制和共识都从状态归属问题长出来

分片复制章节不能一开始就讲 Raft 或 quorum。更自然的起点是状态归属：某个 key 当前由谁负责，哪个副本有权提交，客户端请求应发到哪里，旧 owner 何时失效。没有状态归属，复制只是多份数据，分片只是取模，故障一来就会出现脑裂、重复写和旧路由。

分片先回答数据归属。hash 分片让 key 均匀分散，适合点查和简单扩容，但范围查询差；range 分片保留顺序，适合范围扫描，却容易遇到热点。热点 key 是分片最好的反例：均匀 hash 也挡不住

一个 key 特别热，range 分片也挡不住单个范围被打爆。教材要把分片和 workload 绑定，而不是只给公式。

路由版本解决旧知识问题。客户端缓存路由表，driver 发布新路由，worker 正在迁移分片；这时旧请求可能发到旧 owner。route epoch 让旧 owner 明确拒绝旧路由，而不是悄悄接受写入。再分片状态机通常包含准备、全量复制、追增量、切换、清理。每个状态失败后都要能重试，否则扩容就是一次高风险手术。

复制回答容错，但它引入一致性选择。主从复制延迟低但主节点是提交中心；quorum 通过多数派交集保证读写相遇，但要处理慢副本和冲突版本；链式复制让顺序明确，却增加尾部延迟；共识日志提供强顺序，但成本更高。没有一种复制策略适合所有场景。教材应让读者按故障语义、延迟和吞吐选择，而不是把“复制三份”当答案。

leader 的价值是集中顺序，危险是旧 leader 复活。term 或 epoch 的作用是让旧 leader 即使还活着，也不能继续以旧身份提交。客户端看到两个 leader 响应时，不能只看谁先回来，要看任期和提交证明。这个思想和前面原子章节的版本、无锁章节的 generation、I/O 章节的 manifest 都是一条线：位置或身份相同，不代表状态仍然有效。

共识边界也要讲清。共识能让多个节点对日志顺序达成一致，但它不自动解决业务幂等、热点 key、数据布局、查询优化或磁盘损坏。请求去重表仍要进入状态机，输出文件仍要校验，重试预算仍要限制，分片热点仍要拆。把所有问题推给共识，是分布式教材常见的偷懒。

本章实验可以用三副本本机模拟。第一，任意副本都能写，构造网络分区看脑裂。第二，引入 leader 和 term，让旧 leader 请求被拒绝。第三，实现多数派提交，让一个副本慢、一个副本失败时仍可处理部分写。第四，模拟再分片，在每个状态注入崩溃并恢复。报告要记录 route epoch、leader term、commit index、request id 和恢复动作。这样读者才会真正理解分片、复制和共识是状态归属的不同层次。

案例推导：再分片为什么需要路由版本和可重试状态机

系统最初有两个分片，按 key hash 到 worker A 和 B。数据增长后要扩成四个分片。朴素做法是发布新路由，让客户端按新规则发送请求，同时后台复制旧数据。问题立刻出现：旧客户端还按旧路由写 A，新客户端按新路由写 C；复制过程中增量写入如何追上；切换时某个节点崩溃后，系统如何知道处于哪个阶段。再分片不是一次赋值，而是一台状态机。

准备阶段先冻结迁移计划，生成新的 epoch。复制阶段把旧分片的全量数据复制到新 owner，但旧 owner 仍可能接收写入。追增量阶段把复制期间的新写补到目标。切换阶段发布新路由，旧 owner 对旧 epoch 请求返回刷新错误。清理阶段删除旧副本或降级为备份。每个阶段都要可重试：崩溃后读取迁移状态，继续复制、继续追增量或确认已经切换。

route epoch 是防止旧知识写错位置的工具。客户端请求带上自己看到的 epoch；服务端发现 epoch 旧，就拒绝并返回新路由，而不是勉强处理。这个机制和无锁 generation、配置版本、manifest commit index 是同一思想：身份相同不代表状态仍然有效，必须带版本。

复制策略决定切换安全性。若只复制一份，新 owner 崩溃会丢数据；若三副本 quorum，切换要等多数派确认。quorum 写入成功后，读取也要带版本或多数派，不能从落后副本读旧值。这里不必完整实现 Raft，但要让读者知道 quorum 的交集为何能提供保证，以及慢副本如何影响尾延迟。

leader 和 term 解决旧 leader 问题。迁移过程中旧 owner 可能网络隔离后又恢复，如果它仍认为自己还是 leader，就可能接受旧写。term 让旧身份失效。请求必须携带 term 或通过当前 leader 路径提交。旧 leader 可以转发或拒绝，但不能悄悄提交。这个规则要写进状态归属报告，而不是留给“应该不会发生”。

实验可以在本机模拟四个 worker。先实现无 epoch 的再分片，构造旧客户端写入，观察数据分裂。再加入 epoch 拒绝旧写。然后在复制、追增量、切换三个阶段分别崩溃 driver，验证恢复能继续。最后让一个副本慢，比较单副本提交和 quorum 提交的延迟与可用性。这样分片、复制和共识边界会自然出现。

实验：设计一个三副本 key-value 存储的写入流程。分别画出同步多数写、异步主从写和读修复流程，并说明各自的延迟和失败行为。

分片从状态归属开始 分片不是把数据平均切几份这么简单，而是决定每个 key 的状态归谁管理。贯穿项目的统计结果可以按 key range 或 hash 分片。某个 key 的计数只能由一个分片负责，或者必须有明确合并规则。若分片函数改变，旧数据如何迁移，新请求如何路由，迁移期间重复写如

何处理，都需要状态机。分片首先是状态归属问题，其次才是负载均衡问题。

路由表要有版本。Driver 和 worker 看到的分片版本可能不同，消息必须带 route epoch。若 worker 按旧版本写到了旧分片，driver 需要知道这是允许的迁移窗口内写入，还是过期写入。没有版本，系统只能靠时间猜测。版本化路由把再分片从“瞬间切换”变成可恢复流程。

再分片是多阶段协议 再分片可以拆成几个阶段：准备新路由，冻结或双写受影响范围，复制旧分片数据，校验新分片，切换路由，清理旧副本。不同系统会选择不同细节，但核心是不能让请求在迁移中无归属，也不能让同一更新被两个分片独立接受而无法合并。项目可以实现简化版本：停止相关 key 的新写入，复制状态，校验 checksum，提升 route epoch，再恢复写入。

双写看似提高可用性，但会带来一致性问题。若写旧分片成功、新分片失败，如何恢复；若读请求看到旧或新，是否允许；若重试重复写，幂等如何保证。教材应把这些问题写出来，而不是直接给一个“平滑迁移”结论。

复制从读写合同推导 复制的目的可能是容错、读扩展、低延迟或维护。不同目的对应不同一致性合同。主从复制让 leader 接受写，follower 复制日志；quorum 读写通过多数派交集保证较强一致性；异步复制延迟低但可能丢最近写；同步复制可靠但写延迟高。读者需要先问业务能接受什么，再选择复制策略。

项目中的 manifest 和分区元数据适合用强一点的提交语义，因为它决定哪些输出有效；临时指标或缓存可以弱一些，因为丢失可重建。不是所有数据都需要同样一致性。高质量系统会按数据重要性分层，而不是全局套一个协议。

共识边界要讲清楚 共识算法解决的是多个节点在故障和消息延迟下对日志顺序或值达成一致。它不是所有分布式问题的万能解。若只有单机模拟项目，可以不实现完整 Raft，但必须讲清哪些地方需要 leader、term、log index、commit index 和多数派确认。比如 driver leader 决定哪个 task attempt 有效，manifest commit 决定哪些 block 被接受，这些都类似需要有权权威顺序。

教学中应避免把 Paxos 或 Raft 写成步骤背诵。先从两个 driver 同时提交不同结果的事故讲起：如果没有领导权和提交顺序，worker 可能接受两个互相冲突的 manifest。引入 leader 后，还要处理 leader 失效和旧 leader 的消息。Term 或 epoch 就是为了让新旧领导权可区分。这样共识概念从事故中长出来。

实验：路由版本和旧消息 构造一个分片路由表，key 通过 hash 到 partition。Route epoch 1 时 key A 属于 partition 0；再分片后 epoch 2 属于 partition 3。注入一条 epoch 1 的延迟写消息，让它在切换后到达。正确行为不是随便接受或丢弃，而是按迁移状态处理：若旧 epoch 已关闭写入，拒绝并返回 stale route；若处于双写窗口，转发或记录到迁移日志；若消息是重复 request，查询幂等表。

这个实验能让读者理解版本不是装饰。所有跨组件状态都需要版本：路由版本、配置版本、driver epoch、manifest generation。版本让系统在乱序消息中判断新旧。

Linux 和工程源码连接点 Linux 内核里的 per-cpu、RCU 和文件系统 journal 都体现“状态归属、版本、提交边界”的思想。阅读分布式系统源码时，可以关注元数据服务如何做 leader、epoch、lease、log 和 snapshot。项目不必实现完整工业系统，但要学习这些源码如何把状态变化写成日志和版本，而不是依赖内存里的临时对象。

把这些思想带回毕业项目，driver 的提交表、route table、manifest 和 checkpoint 都应该带 generation。恢复时先读最新稳定 generation，再决定哪些 running task 需要重试。这样第 17 章自然过渡到第 18 章的完整计算引擎。

事故复盘：旧路由写请求为什么危险 再分片事故通常不是数据搬不动，而是旧请求在切换后迟到。Route epoch 1 中 key A 属于 shard 0；控制面发布 epoch 2 后，key A 迁到 shard 3；一个 epoch 1 的写请求因为网络延迟，在切换后才到达旧 owner。若旧 owner 仍按旧 hash 接受写，新 owner 又接受新写，同一个 key 就出现两个权威。分片首先是状态归属，不是取模函数。

从这里推出 route table、ownership epoch、move state、replica group、leader term 和 commit index。Route table 告诉客户端发给谁；epoch 让服务端判断请求新旧；move state 区分 freezing、copying、switching、completed；replica group 定义副本集合；leader term 区分新旧权威；commit index 说明哪条日志已经对外生效。每个字段都解决旧消息、旧 owner 或旧 leader 的具体问题。

实验要让旧 epoch 请求真实迟到。冻结策略应返回 stale route；双写策略必须记录部分成功和补偿；完成迁移后旧 owner 只能拒绝或转发，不能悄悄提交。复制实验不必实现完整工业 Raft，也

要保留 term、log index 和多数派确认的语义。热点 key 实验则说明加节点不能拆开同一个 key，必须引入缓存、子 key、局部聚合或业务限流。分片、复制和共识都要从“谁有权改变状态”开始讲。

状态归属实验判读 再分片实验要看旧 epoch 请求如何处理。若旧消息在切换后仍被接受，必须说明是否处于迁移窗口；若被拒绝，要返回 stale route 而不是静默丢弃。复制实验要写读写 quorum、leader term 和 commit index。热点 key 拆分后，还要验证合并结果和路由版本一致。

这一段也服务于复习。读者合上书后，应该能凭本章的判读口径重新审查自己的代码：有没有最小正确版本，有没有能击穿旧方案的输入，有没有状态记录，有没有资源指标，有没有失败后的恢复动作。若五个问题缺两个以上，就说明实现还停在演示层，而不是工程层。

再分片期间的旧路由请求 再分片最容易错在切换瞬间。Route epoch 1 中 key 属于 shard A；epoch 2 中 key 属于 shard B。一个 epoch 1 的写请求因为网络延迟，在 epoch 2 生效后才到达。系统不能只按当前 hash 接受，也不能静默丢弃。它必须检查请求携带的 epoch 和迁移状态。

如果处于冻结阶段，旧写应被拒绝并返回 stale route；如果处于双写阶段，系统要记录双写结果和补偿规则；如果已经完成迁移，旧 epoch 请求只能作为重复或过期处理。每种选择都影响可用性和复杂度。教材要让读者看到再分片是协议，不是重新取模。

复制和共识也围绕同一问题：谁有权决定状态归属和提交顺序。Leader term、commit index、quorum 交集都不是名词装饰，而是为了让新旧权威在故障和延迟下可区分。项目简化实现也要保留这些字段的语义。

实验课：路由版本驱动的再分片状态机 先实现一个静态路由表：key hash 到 partition。然后增加 route epoch，每条请求携带自己看到的 epoch。再设计再分片状态：planning、freezing、copying、validating、switching、completed。实验中让 epoch 1 的请求在 switching 后到达，系统应返回 stale route 或按迁移窗口处理，而不是直接写当前 owner。

双写实验展示取舍。迁移期间同时写旧分片和新分片，提高可用性，但会产生部分成功问题。若旧写成功、新写失败，系统要有补偿或重试；若读请求到达，要说明读旧、读新还是读两边。教学版可以选择冻结写入，牺牲可用性换清晰语义，但必须解释为什么。

复制实验不追求完整共识实现，但要保留 leader term 和 commit index。两个 driver 不能同时提交不同 manifest；旧 leader 的消息必须被新 term 拒绝。Quorum 读写要说明多数派交集，否则读可能看不到已提交写。读者应理解这些字段解决的是“谁有权决定顺序”。

热点 key 实验把状态归属和负载联系起来。把热点 key 拆成子 key 可以平衡写入，但最终合并必须保持语义。路由表、子 key 规则和合并规则都要带版本，避免迁移期间结果漂移。

分片复制练习验收重点 合格答案从状态归属开始。route epoch、owner、迁移阶段、旧消息处理必须写清。复制题要说明读写合同和 quorum 交集；leader 题要说明 term 如何拒绝旧 leader。只写 hash 取模或多副本保存，不算理解分布式状态。

答案还要写“为什么不采用更复杂方案”。例如没有证据证明队列是瓶颈，就不应把锁队列换成无锁；没有证明输出顺序可变，就不应随意重排；没有证明错误路径能恢复，就不应只提交正常路径。能解释不做什么，说明读者开始具备工程判断。

最后，答案必须包含可复查材料。至少有输入摘要、关键状态、期望输出、实际输出、运行命令和未验证边界。没有这些材料，老师或同伴无法判断结论是否成立。第二册的作业不是让读者写漂亮段落，而是训练读者把系统问题变成可验证证据。

项目实现：分片、复制与共识 本章对应的贯穿项目实现不要求一次写成完整框架，但必须有最小可运行切片。这个切片包含三个部分：一个 reference，一个带本章机制的实现，一个能击穿 reference 之外隐含假设的测试。Reference 用来固定语义，机制实现用来展示本章知识，测试用来证明新增复杂度确实解决了问题。

代码组织上，不要把所有内容放进一个大文件。基础类型、输入生成、核心机制、报告输出和测试应分开。即使当前只是教学项目，也要让目录能表达责任边界。这样后续章节接入时，不会因为找不到状态归属而重新发明一套对象。

每个实现都要输出一段机器可读摘要。摘要至少包含输入规模、关键状态、错误数、输出校验、耗时和未验证边界。这里的机器可读不要求复杂格式，简单的 key value 文本也可以。关键是让脚本和读者都能复查，而不是只能看屏幕截图。

测试应包含正常路径、边界路径和失败路径。正常路径证明功能可用；边界路径证明长度、容量、尾部、空输入、关闭或超时没有遗漏；失败路径证明本章机制不是装饰。若失败路径写不出来，

往往说明本章概念还没有落到工程对象上。

报告最后写一个取舍段。新增机制带来了什么好处，增加了什么成本，哪些场景不应该使用它，下一章会复用哪个字段或对象。这个取舍段非常重要，因为系统工程不是把所有高级机制都加进去，而是在证据和约束下选择合适的边界。

调试笔记：迁移故障先查 epoch 再分片故障先查消息携带的 epoch 和当前 route epoch。旧 epoch 写入如果被当前 owner 接受，可能破坏状态归属；新 epoch 请求如果被旧 owner 接受，也可能重复写。迁移阶段必须写在状态机里，不能只靠时间窗口口头约定。复制和 leader 切换同理，先看 term，再看 commit index。

复查清单：分片、复制与共识 复查本章时，先检查 reference 是否仍然存在。没有 reference，后面的并发、优化、异步或分布式改造都无法证明语义没有变。再检查压力输入是否尖锐：它应当攻击本章最核心的边界，而不是只把数据量放大。第三，检查状态是否可观察：关键对象的身份、状态、错误和提交结果应能在报告中找到。

第四，检查代码示例是否说明禁止行为。调用者不能做什么，往往比能做什么更能体现工程质量。第五，检查实验是否写出未验证边界；当前设备、权限或输入无法证明的结论，必须标注。第六，检查本章对象是否能被下一章复用，不能复用就说明边界还不稳。

完成这六项后，再看文字是否连贯。每个概念都应从问题推出，每个案例都应从朴素方案走到改进方案，每个实验都应能解释一个判断。若某段话放到其他章节也成立，就把它改成本章的具体对象、具体输入和具体失败。

17.8 综合案例：从状态归属推导分片、复制和共识

分片、复制和共识经常被讲成三个并列主题。更连贯的讲法是从一个问题开始：某个 key 的状态归谁负责。如果状态只在一台机器上，吞吐和容量有限；如果状态按 key 分片，就需要路由和再分片；如果状态要容忍机器故障，就需要复制；如果多个副本都可能接受写入，就需要决定谁的顺序算数。这样分片、复制和共识会沿着同一条因果链出现。

一、分片先定义所有权 分片不是简单取模。它定义某段 key 空间由哪个 shard 负责，读写请求应送到哪里，迁移期间旧 owner 和新 owner 如何处理请求。没有所有权语义，分片函数只是哈希。Compute Systems Engine 的 shuffle partition 也有同样问题：partition id 决定 reduce 归属，manifest 决定哪些 block 属于该 partition，route epoch 决定旧路由是否还有效。

最小路由表应包含 epoch、分片范围、owner、状态和迁移目标。请求带上自己看到的 epoch；服务端若发现请求 epoch 过旧，可以拒绝并返回新路由。这样旧 driver、迟到消息和重试请求不会在迁移后悄悄写到错误位置。

二、再分片是状态机，不是后台拷贝 再分片常被误讲成“把数据从 A 拷到 B”。真正难的是迁移期间写入如何处理。一个清晰状态机可以包含 Serving、MigratingOut、MigratingIn、CatchingUp、ServingNew、Retired。旧 owner 在 MigratingOut 中可以继续服务读写并记录增量，或冻结写入；新 owner 在 CatchingUp 中加载快照和增量；切换点由 route epoch 发布。

不同策略有不同成本。冻结写入最简单，但可用性差；双写能降低停顿，但一致性和错误处理复杂；日志追赶需要 WAL 或变更流。教材要让读者看到这些策略为什么出现，而不是直接给“再平衡算法”。问题的核心是：迁移期间哪个版本的所有权对请求有效，迟到请求如何拒绝，失败后从哪个状态恢复。

三、复制先问读写语义 复制可以提高可用性和读吞吐，但也引入一致性选择。主从复制让 leader 处理写入，followers 复制日志；读可以走 leader，也可以在满足租约或已知进度的 follower 上读。Quorum 复制让写入等待多数副本，读也可能等待多数，以交集保证看到最新写。不同读路径的延迟、可用性和新鲜度不同。

不能只说“三副本更安全”。如果三个副本都在同一机架、同一电源、同一磁盘故障域，复制的容错意义很弱。副本放置要考虑故障域：机器、机架、可用区、磁盘类型和网络拓扑。对本书项目来说，本机模拟也可以给 worker 设置 failure domain，训练读者不要把副本数等同于可靠性。

四、WAL 是恢复边界 复制状态机需要一条可恢复的顺序记录。Write ahead log 的核心不是文件格式，而是“状态改变先进入持久日志，再对外承诺”。若 leader 告诉客户端写入成功，但日志没有持久化，崩溃后就可能丢失已承诺结果；若日志持久化但响应丢失，客户端重试时系统应能从日志或提交表识别重复请求。

WAL 也连接第 15 章 I/O 语义。写日志不是调用 write 就结束，还要考虑短写、fsync、文件系统错误和批处理。批量 fsync 可以提高吞吐，但会增加提交延迟。复制协议常把多个日志项批量发送和落盘，这又引入尾延迟和背压。分布式一致性最终仍然会落回 I/O 完成事件。

五、共识解决的是谁有权定义顺序 当 leader 可能崩溃，系统需要选出新 leader，并保证新 leader 不会丢掉已经提交的日志。Raft 这类协议把问题拆成任期、投票、日志匹配和提交规则。任期用于区分新旧领导权，投票限制同一任期只有一个 leader，日志匹配保证新 leader 拥有足够新的日志，提交规则决定何时可以对外承诺。

教材不应把共识讲成魔法。它解决的是“在部分失败和消息延迟下，哪些副本承认同一条顺序”。它不解决所有问题：业务操作仍需幂等，日志应用仍需确定性，快照仍需正确安装，读路径仍需处理线性一致或陈旧读的选择。共识给出控制面顺序，数据面和业务语义还要自己设计。

六、读路径比写路径更容易被低估 写路径通常会认真走 leader 和 quorum，读路径常被为了性能放松。Follower read 如果没有租约、读索引或已提交位置证明，可能读到旧数据；leader lease 如果依赖时钟，就要考虑时钟漂移和网络暂停；read index 可以避免落盘，但仍需和当前多数派确认 leader 身份。每种读优化都要写清能提供什么一致性。

对 Compute Systems Engine 来说，manifest 读取也有类似问题。Reduce 读取 manifest 时，是否要求看到所有已提交 map block？若 driver 正在提交新 block，reduce 读到旧 manifest 是否允许？若允许增量读取，就要处理重复和迟到；若要求固定快照，就要先冻结一个 manifest version。读路径语义不清，系统会出现偶发缺数据或重复读。

七、热点 key 不是分片函数能完全解决的 一致哈希或范围分片能均衡很多输入，但热点 key 会让某个 shard 过载。解决方式包括热点 key 拆分、读缓存、局部预聚合、动态迁移、按时间窗口分桶和请求限流。每种方法都会改变状态归属。把一个热点 key 拆成多个子 key 后，读路径需要合并；局部预聚合会引入延迟；缓存会引入失效和新鲜度问题。

这与第 13 章 partition 倾斜直接相连。单机 partition 中的热点桶，到分布式系统里变成热点 shard。若 map 端能先 combine，发送到热点 shard 的字节数会减少；若分区函数完全不考虑 key 分布，reduce 长尾会支配作业完成时间。算法和分布式不是两门孤立课程。

八、再分片实验要包含旧路由消息 本章实验不应只测试均匀分片。应构造一个 route epoch 从 7 切到 8 的场景：旧 driver 仍发送 epoch 7 请求，新 driver 发送 epoch 8 请求，旧 worker 迟到响应，迁移目标正在 catch up。系统要证明旧 epoch 写入不会污染新 owner，重复提交不会进入 manifest，失败后能从快照和增量恢复。

第二组实验测试复制。设置三副本，注入 leader 崩溃、follower 落后、响应丢失和磁盘写失败。报告记录哪些写被承诺，哪些未承诺，重启后状态机应用到哪里。不要只看最终 key 数量，还要检查日志 index、term、commit index 和 applied index。这样读者才会把共识看成可审查状态，而不是背术语。

九、工程边界：什么时候不需要共识 并不是所有分布式数据都需要强共识。临时 shuffle block 可以通过 manifest 提交保证最终有效，未提交临时文件可清理；监控指标可以近似；缓存可以失效重建；可重算的中间结果可以用任务重试恢复。强共识成本高，应该用于不可轻易重算、对外承诺强、需要唯一顺序的控制状态，例如 route epoch、job commit、元数据变更和持久任务状态。

会使用共识不等于到处使用共识。高质量工程判断是区分数据面和控制面：大块数据尽量可重算、可校验、可批量传输；小而关键的控制状态使用更强一致性。这个原则贯穿第二册，也为第三册 AI 推理引擎的模型版本、权重 manifest 和任务调度打基础。

17.9 案例深化：把元数据和大数据分开复制

分布式计算里，大数据和元数据不应使用同一复制策略。Shuffle block 可能很大，但可重算；manifest、route epoch、job commit 状态很小，却决定外部语义。高质量设计通常让大数据可校验、可重算、按需复制，让小而关键的控制面走更强一致性。

一、Shuffle block 可以重算 Map 输出 block 如果丢失，只要输入 split 还在、map 逻辑确定，就可以重算。它需要 checksum 和身份，不一定需要强共识复制。把所有 block 都写入强一致日志，成本很高。更好的做法是 block 存储负责完整写和校验，manifest 决定哪些 block 有效，失败时重做 attempt。

二、Manifest 需要强语义 Manifest 记录哪些 block 被提交。如果 manifest 丢失或分叉, reduce 就不知道读取哪些数据, 可能重复或遗漏。它比 block 小得多, 却更关键。单机项目可以用本地文件加 fsync; 多机系统可以把 manifest 放入复制状态机或数据库。这里强一致性用于控制面, 而不是所有数据面。

三、Route epoch 也是控制面 分片迁移时, route epoch 决定请求发往哪里。旧 epoch 请求应被拒绝或重定向。若不同 driver 看到不同 epoch, 写入可能分裂。Route epoch 小而关键, 也适合强一致控制。大块数据迁移可以异步进行, 但所有权切换点必须有清楚一致语义。

四、实验: 控制面损坏 构造两个 manifest 版本, 一个包含旧 attempt, 一个包含新 attempt。Reduce 如果扫描目录, 会重复计数; 如果只读单一提交 manifest, 结果正确。再构造 route epoch 过旧请求, 检查 worker 是否拒绝。这个实验能让读者理解为什么控制面强一致比盲目复制大数据更重要。

17.10 补强案例: 快照和日志压缩

复制状态机不能无限保留日志。随着操作增长, 日志需要压缩, 状态需要快照。快照不是简单把内存 dump 出来, 而是记录某个 log index 已经应用后的状态, 并保证恢复时不会重复或遗漏操作。

一、快照边界 快照要说明包含哪些状态: key value 数据、route epoch、manifest version、已提交 job、去重表的安全部分。不能包含正在执行但未提交的临时状态。快照对应一个最后应用的 log index; 恢复时从快照加载, 再重放之后的日志。

二、去重表和快照 去重表不能随便丢。若客户端重试旧请求, 而服务端快照后忘记它, 可能重复执行。系统通常保留某个窗口内的 request outcome, 或让客户端请求带 epoch, 超过窗口的请求明确失效。去重状态也是控制面的一部分。

三、安装快照 Follower 落后太多时, leader 可以发送快照而不是全部日志。安装快照要处理旧快照、部分传输、校验和当前状态替换。快照安装期间读写如何处理, 要由协议定义。教学项目可以简化为暂停 shard, 安装完整快照, 再切换 epoch。

四、实验 构造一百条元数据操作后生成快照, 删除前半日志, 再恢复。检查 commit index、applied index、manifest 和去重表。再发送旧请求, 确认不会重复提交。这个实验让复制章节从共识术语进入可恢复工程。

17.11 协议案例: 再分片期间的读写如何不丢不重

再分片是分布式系统里最容易被低估的操作。平时 key 属于 shard A, 迁移后属于 shard B。问题在于迁移期间请求不会停止: 旧路由还在发送, 客户端还会重试, 旧 owner 和新 owner 都可能看到同一个 key。若没有状态机, 数据很容易丢失或重复。

一、冻结写入最简单但影响可用性 最简单策略是迁移期间冻结该范围写入。旧 owner 拒绝新写, 新 owner 加载快照, 切换 route epoch 后恢复写入。优点是语义清楚, 缺点是迁移期间不可写。适合控制面小状态或维护窗口, 不适合高可用在线服务。

冻结策略也要处理读。读可以继续走旧 owner, 或在快照点后走新 owner。若读要求最新, 必须知道哪些写已经进入快照。若读允许短暂不可用, 可以统一拒绝。即使最简单策略, 也需要 route epoch 和错误返回。

二、日志追赶减少停顿 另一种策略是旧 owner 生成快照, 同时继续记录增量日志。新 owner 加载快照后追赶增量, 追到某个切换点, 再发布新 route epoch。停顿时间缩短, 但需要 WAL、增量传输、重复过滤和切换协议。若增量追赶速度低于写入速度, 迁移永远追不上。

这个策略把复制章节和 I/O 章节连接起来。增量日志要可靠写入, 传输要校验, 应用要幂等, 切换要有提交点。再分片不是后台拷贝, 而是一个小型复制协议。

三、双写降低停顿但增加一致性复杂度 双写让旧 owner 和新 owner 同时接收写入, 等待两边成功后返回。它可以减少切换停顿, 但处理失败很复杂: 一边成功一边失败怎么办, 重试是否重复, 读从哪边读, 什么时候停止双写。没有强协议的双写很危险。

教学中可以把双写作为反例。它看似提高可用性, 实际上把一致性责任放大。读者应先掌握冻结和日志追赶, 再讨论双写。高可用策略不能只看可用性, 也要看恢复和证明成本。

四、旧 epoch 请求的处理 无论哪种策略，请求都要带 route epoch。服务端发现 epoch 旧，返回 stale route，不执行写入；发现 epoch 未来，说明路由异常，也不能盲目接受。Epoch 是逻辑时间，保护所有权切换。没有 epoch，迟到请求会写到旧 owner，造成分裂。

Response 也要带 epoch。Driver 收到旧 epoch 成功响应时，不能直接提交。它要查当前 route 和提交表。旧响应可以记录为 late success，但不改变新 epoch 状态。这个规则和第 16 章重复提交一致。

五、实验矩阵 构造 key range 从 shard A 迁移到 shard B。注入迁移中写请求、旧 epoch 重试、快照传输失败、增量日志重复、切换后旧响应迟到。验收要求每个 key 最终值等于某个合法串行顺序，且没有请求既被旧 owner 又被新 owner 提交两次。报告写清使用的是冻结、追赶还是双写策略。

六、和 Compute Systems Engine 的关系 Shuffle partition 再分配也是再分片的简化版。Reduce partition 从 worker A 移到 worker B 时，旧 block、旧 route 和新 manifest 必须协调。虽然教学项目可以不做动态再分片，但理解这个协议能帮助读者设计第三册中的模型分片、KV cache 分片和请求路由。

17.12 补充案例：读一致性的三个等级

读路经常被忽略。写入走了复制协议，如果读取随便从任意副本返回，用户仍可能看到旧状态。读一致性至少可以分成三个等级：可能陈旧读、单调读和线性一致读。不同等级成本不同。

一、可能陈旧读 Follower 直接返回本地状态，延迟低，但可能落后 leader。适合缓存、监控和允许延迟的统计。不适合读取刚提交的 manifest 或 route epoch。接口要标明 stale read。

二、单调读 客户端带上自己见过的版本，服务端至少返回不低于该版本的数据。如果 follower 落后，就等待或转发 leader。这样用户不会看到时间倒退。需要版本号和客户端上下文。

三、线性一致读 读必须反映读开始前已提交的写。可以走 leader，或使用 read index 确认 leader 仍属于多数派。成本更高，但适合控制面。Manifest、route epoch 和提交表通常需要这种等级或等价保证。

四、实验 模拟 leader 写入后 follower 延迟复制。分别实现 stale read、带版本单调读和 leader read。让客户端先写后读，观察三种行为。报告写明哪个接口用于数据面，哪个用于控制面。

17.13 补充专题：副本放置和故障域

复制数量不等于可靠性。三个副本如果在同一台机器、同一机架或同一电源域，某个故障可能同时影响它们。副本放置必须考虑故障域。

一、故障域建模 故障域可以是磁盘、机器、机架、可用区、网络交换机。项目模拟中可以给 worker 标注 `failure_domain`。放置策略要求同一 shard 的副本尽量分布在不同 domain。即使本机模拟，也能训练这种思维。

二、放置和延迟 跨故障域放置提高容错，但增加延迟。跨可用区 quorum 写入比同机房慢。控制面强一致要付出这个成本；可重算数据可能选择较弱复制。放置策略再次体现控制面和数据面差异。

三、再平衡 节点加入或离开时，副本要迁移。迁移会消耗网络和 I/O，并可能影响前台请求。系统应限制同时迁移数量，并用背压保护前台。再平衡不是后台免费任务。

四、实验 模拟三个 failure domain，随机让一个 domain 失败。检查副本是否仍有 quorum。再模拟错误放置，让三个副本同域，观察同域失败导致不可用。这个实验让可靠性从副本数变成放置策略。

17.14 补充专题：共识之外的确定性状态机

共识协议只决定日志顺序，状态机本身还必须确定。相同日志在不同副本上应用，必须得到相同状态。若状态机读取本地时间、随机数、目录扫描顺序或非确定哈希遍历，就可能分叉。

一、确定性输入 日志项应包含状态机需要的全部输入，例如 request id、payload、timestamp 如果业务需要也应由 leader 写入日志，而不是 follower 各自读取本地时钟。随机选择也应使用日志中的 seed。

二、确定性遍历 C++ unordered map 遍历顺序不应成为外部输出顺序。若状态机输出 manifest 或快照，应按 key 排序或使用稳定顺序。否则不同副本或不同运行可能生成不同字节，影响校验和调试。

三、外部 I/O 状态机应用日志时不应直接执行不可重复外部 I/O。外部效果要通过提交表和幂等协议处理。否则日志重放会重复发送消息或重复写外部系统。

四、实验 构造两个副本应用同一日志，一个使用 unordered map 直接输出，一个按 key 排序输出。比较快照 checksum。这个实验说明共识之外仍需要确定性工程。

17.15 复制实现细节：从单 leader 日志开始

共识完整实现很复杂，但教学项目可以先实现单 leader 复制日志的简化模型。Leader 接收元数据操作，写本地日志，发送给 followers，达到多数确认后提交。这个模型能训练日志、任期、提交索引和恢复，不必一开始实现完整选举。

一、日志项内容 日志项包含 index、term、operation type、request id、payload checksum。Operation 可以是 commit manifest entry、update route epoch、mark job failed。日志项应确定性应用，不读取本地随机状态。Request id 支持去重。

二、提交规则 Leader 收到多数副本持久化确认后，推进 commit index。状态机只应用 commit index 以内的日志。Follower 也按 leader commit index 应用。未提交日志崩溃后可以丢弃或被新 leader 覆盖。读者要分清 append、persist、commit、apply 四个阶段。

三、恢复 节点重启后读取日志，重建 last index 和 applied state。若日志尾部有未提交项，不能对外承诺。快照可以减少重放长度，但快照也要记录 last included index。恢复实验是复制章节的重点。

四、和完整共识的差距 简化模型没有选举和 leader 变更，因此不能容忍 leader 永久失败。书中要明确边界。完整 Raft 还需要投票、日志匹配、任期比较和安全选举。教学模型是台阶，不是伪装成完整协议。

五、实验 三个副本，leader 提交 manifest entry。注入一个 follower 丢日志，一个 follower 延迟确认，观察多数提交。再让 leader 在未达到多数前崩溃，恢复后该项不能出现在 committed manifest。这个实验让复制语义具体可见。

17.16 从状态归属推导分区、复制和共识

分区、复制和共识常被分开讲，导致读者以为它们是三组独立技术。实际主线是状态归属：某一份数据当前由谁负责接收写入，谁拥有最新副本，客户端该把请求发给谁，故障后谁有资格继续推进状态。数据一大，单节点放不下，就需要分区；节点会故障，就需要复制；多个副本可能意见不同，就需要选主、日志和共识规则。概念的顺序应从归属问题自然长出来。

一、分区解决容量和并行，但引入路由版本 按 key 分区后，每个分区由某个 owner 负责。客户端必须知道 key 到 owner 的映射。问题在于映射会变化：扩容、缩容、热点迁移和故障转移都会产生新路由。若客户端持有旧路由继续写旧 owner，状态会分裂。于是路由表需要 version 或 epoch，服务端收到旧 epoch 请求要拒绝或转发。分区不仅是哈希函数，还包括归属版本、迁移状态和旧请求处理策略。

二、再分片的难点在双写窗口 迁移分区时，旧 owner 和新 owner 之间会有一段交接期。若旧 owner 继续接受写，新 owner 同时导入快照，就可能丢失迁移期间的更新；若新 owner 太早接收写，客户端可能读到不完整状态。常见做法是冻结、复制增量、切换 epoch、拒绝旧 epoch 写入，再清理旧副本。每一步都要有可恢复记录。教学中要用一个 key 的时间线讲清楚：它什么时候归旧 owner，什么时候只读，什么时候归新 owner，旧请求如何处理。

三、复制把可用性问题变成一致性问题 单副本故障会丢服务，多副本提高可用性，但也引入“哪个副本算最新”。异步复制吞吐好，但主节点崩溃时可能丢最后更新；同步复制更可靠，但增加延迟；quorum 读写用多数派交集保证读到足够新的值，但也需要版本、冲突规则和失败处理。复制不是简单多存几份，而是决定写入何时成功、读取允许多旧、故障后如何选出权威历史。

四、共识日志把决定顺序固定下来 共识解决的是多个节点在故障和消息乱序下对同一串决定达成一致。对于教材项目，不必把完整 Raft 或 Paxos 变成形式化证明，但要讲清 leader、term、log

index、commit index 和状态机应用的意义。leader 负责提出日志顺序，term 防止旧 leader 继续写，log index 定义操作位置，commit index 表示多数派确认后的安全前缀。客户端看到的是状态机结果，系统内部维护的是一条可恢复、可复制、可截断的日志。

五、CAP 要落到具体请求 CAP 不能只背“一致性、可用性、分区容忍”。网络分区发生时，某个请求到底返回成功、返回失败、阻塞等待还是返回旧值，才是工程问题。计费写入可能选择拒绝以保持一致；搜索结果可能返回部分旧数据；监控统计可能接受短暂不一致。教材要让读者为不同业务写选择，而不是把 CAP 当口号。只有把请求语义、用户后果和恢复成本写出来，分区策略才有意义。

17.17 复制日志的工程化讲法

复制日志容易被讲成算法论文摘要，但教学项目需要更工程化的剖面：一条客户端命令进入 leader，leader 追加日志，复制给 follower，多数派确认后提交，状态机按顺序应用，snapshot 截断旧日志，故障后新 leader 继续维护同一条历史。每一步都可以用具体状态解释，而不必一开始陷入形式化证明。

一、日志条目保存命令和任期 每条日志至少包含 index、term、command、checksum 和 client request id。index 表示位置，term 表示哪个 leader 产生，command 是状态机输入，checksum 帮助检测损坏，request id 用于客户端重试去重。若 leader 崩溃，新的 leader 通过 term 和 index 判断哪些日志能保留，哪些需要回退。日志不是普通数组，而是系统共同承认的历史草稿。

二、多数派提交建立安全前缀 leader 把日志发给 follower，收到多数派确认后，某个 index 及其之前的日志成为 committed。状态机只能应用 committed 前缀，不能因为本地追加了日志就立即对外返回成功。多数派的意义在于任意两个多数派有交集，新 leader 至少能看到已提交历史的一部分。教材不必把证明写得很重，但要让读者明白为什么“两个节点确认”在三副本里足够，而“一个节点写完”不够。

三、客户端重试要跨 leader 去重 客户端超时后可能向新 leader 重试同一 request。若状态机命令不是幂等的，比如账户加款或任务提交，重复应用会出错。复制组需要在状态机或日志层保存 client id 和 request id，已经应用过的请求返回旧结果。这个问题和第 16 章幂等表相连，只是提交边界从单节点 manifest 扩展到复制日志。

四、snapshot 是日志和状态的共同检查点 日志无限增长不可接受。snapshot 保存某个 index 的状态机快照，之后更早日志可以截断。难点是 snapshot 必须和 committed index 对齐，恢复时先加载 snapshot，再回放后续日志。若 snapshot 包含未提交状态，故障后会暴露错误；若截断了 follower 还需要的日志，就要发送 snapshot。这里的核心不是文件格式，而是状态和历史的一致边界。

五、本机项目可以模拟简化复制组 第二册不要求实现完整工业级共识，但可以实现三节点本机模拟：一个 leader，两个 follower，支持追加、复制、提交、leader 崩溃和新 leader 接管。网络层沿用 MessageBus 注入丢包和延迟。实验检查同一 command 是否只应用一次，旧 leader 恢复后是否拒绝旧 term 写入，snapshot 后恢复是否得到同一状态。这个简化项目足以让读者理解分区、复制和日志的关系。

17.18 贯穿事故：一个热 key 迁移为什么会逼出分片、复制和共识

第 16 章的事故是一条 task 响应丢失，第 17 章把事故换成一个更大的状态归属问题。日志统计作业中，`event=login` 突然成为热点 key，原本负责它的 reduce partition 7 在 worker A 上排队严重。Driver 决定把 partition 7 迁移到 worker B，并顺手把它拆成两个子分片，让后续请求分散到 B 和 C。迁移刚开始，旧路由表仍在一些客户端缓存里，worker A 还在处理旧 epoch 的请求，worker B 已经加载快照，worker C 正在追增量。此时网络分区发生，旧 Driver A 和新 Driver B 都认为自己可以发布 manifest。

这个事故足以推出本章所有核心概念。为什么需要分片，因为单个 owner 扛不住热点 key；为什么需要路由 epoch，因为旧路由请求会迟到；为什么需要迁移状态机，因为快照、增量、切换和清理不能混成一句“搬过去”；为什么需要复制，因为 owner 故障后还要继续服务；为什么需要 leader term 和 commit index，因为两个 Driver 不能同时定义 manifest 真相；为什么要区分控制面和数据面，因为热点数据很大、可重算，而 route epoch 和 manifest generation 小但必须唯一。

一、先从没有分片的失败开始 假设所有 `login` 记录都由一个 reduce worker 合并。正常输入上它能跑通，热点流量一来，队列开始积压，Driver 看到的是 reduce 阶段尾延迟上升。最朴素的反应是增加 worker 数量，但如果 partition 函数仍把同一个 key 发给同一个 worker，新增 worker 只会处理其他冷 key，热点仍然慢。这个失败说明分片不是“机器越多越快”，而是必须让责任单元能被拆开。

拆热点 key 的第一步也不能直接改 hash 函数。若上半作业按旧函数输出 block，下半作业按新函数读取，reduce 端会把同一个 key 的 partial 分到不同规则下，最终结果不可解释。分片函数必须有 version。Manifest 中记录每个 shuffle block 使用的 partition version，reduce 只能在同一个 version 下合并，或者执行一个明确的迁移转换。版本字段由这个反例推出，而不是为了协议好看。

二、再分片不是复制文件，而是改变写权限 把 partition 7 的旧数据从 A 拷到 B，只解决历史状态，不解决迁移期间的新写。A 拷贝开始后继续收到写入，B 加载的快照就会落后；B 如果太早接受写入，可能在历史状态未完整时产生结果；客户端如果用旧路由继续写 A，切换后这些写可能被漏掉。真正的问题不是文件搬运，而是谁在什么阶段有权接受写。

因此迁移要写成状态机。Prepare 阶段控制面记录迁移计划；Snapshot 阶段从旧 owner 取一个可校验快照；CatchUp 阶段追赶快照之后的增量；Switch 阶段发布新 route epoch；Drain 阶段旧 owner 拒绝或转发旧 epoch 写；Clean 阶段清理旧副本。每个阶段都必须能恢复。若 Driver 在 CatchUp 后崩溃，恢复程序要知道切换是否已经发布；若旧 owner 在 Drain 中收到写，它要按 epoch 拒绝，而不是凭本地缓存继续执行。

三、路由 epoch 是防旧写的 fence 旧请求迟到是再分片事故的核心。客户端在 epoch 41 时把 `login` 写到 A；控制面在 epoch 42 发布新路由，让 `login` 的两个子分片分别归 B 和 C；旧请求延迟后才到 A。若 A 只按 key 判断自己曾经拥有这个 partition，它会接受旧写；若 B 和 C 同时接受新写，系统就分裂了。Route epoch 的作用就是把“曾经拥有”和“现在有权”分开。

请求和响应都要带 epoch。服务端收到旧 epoch 写，返回 StaleRoute 或 Forwarding，而不是执行业务。Driver 收到旧 epoch 成功响应，也不能提交到新 manifest，只能按迟到事件记录。Epoch 不是缓存刷新提示，而是 fencing token。下游存储、manifest writer 和 worker 都要检查它，否则只在 Driver 内存里有 epoch 不够。

Listing 17.2 分片请求里的所有权字段

```
1 struct PartitionRequest {
2     std::uint64_t request_id = 0;
3     std::uint32_t partition_id = 0;
4     std::uint64_t route_epoch = 0;
5     std::uint32_t owner_worker = 0;
6     std::uint32_t attempt = 0;
7     std::uint64_t input_block_id = 0;
8     std::uint32_t input_checksum = 0;
9 };
```

这段结构刻意没有把 key 字符串放进来，因为本节讨论的是控制身份。真实请求还会有 payload，但防旧写的关键是 partition、epoch、owner 和 attempt。若读者看到这里还觉得 epoch 只是“版本号”，应回到旧 owner 迟到写入事故：没有它，系统无法区分旧权利和新权利。

四、复制从“owner 会死”这个事实中出现 只有一个 owner 时，A 崩溃就无法继续处理 partition 7。复制的动机很朴素：让 B 或 C 拥有足够状态，A 死后能接管。但复制不是把文件多写一份就结束。要问：A 返回写成功时，B 是否已经拿到这条写；B 落后两个版本时能否接管；C 收到日志但未应用时能否服务读；A 恢复后是否还能继续以旧 owner 身份写。

这会逼出写提交点。若写只在 A 内存完成就返回，A 崩溃会丢已承诺写；若写进入 A 的 WAL 后返回，A 磁盘还在时可恢复，但 A 永久丢失时 B 可能没有；若多数副本持久化后返回，故障恢复更稳，但延迟和尾部成本上升。复制的核心不是副本数，而是“客户端看到成功时，哪些副本已经保存了这个事实”。

五、Quorum 要通过恢复案例理解 三副本 A、B、C，写版本 10。若 W=2，A 和 B 确认后

返回成功，A 随后崩溃，B 和 C 中至少 B 知道版本 10。新 leader 从多数派中选出时，有机会看到已提交版本。若只有 A 写完就返回，A 崩溃后 B 和 C 都停在版本 9，系统没有证据证明版本 10 曾被承诺。多数派的意义不是公式漂亮，而是恢复时保留共同事实。

读也要按这个事故解释。若客户端刚写入版本 10，又从落后的 C 读到版本 9，它会以为写丢了。强读可以走 leader 或读 quorum；单调读要求客户端带上自己见过的版本；陈旧读可以读 follower，但必须标注 lag。不同读路径不是 API 偏好，而是用户能否接受旧历史的问题。

六、双 Driver 事故把共识边界推出来 迁移和复制都还没有解决“谁发布 route epoch 和 manifest generation”。网络分区时，旧 Driver A 还活着，它认为自己是 leader；新 Driver B 被另一侧选出，也认为自己是 leader。A 发布 epoch 42a，B 发布 epoch 42b，两个 manifest 都声称 partition 7 的结果有效。Worker 只要听到一个 Driver 的命令就执行，最终 reduce 会看到两个互斥真相。

这就是共识边界。系统需要让控制面写入进入一条唯一顺序：先发布哪个 route epoch，哪个 manifest generation 被提交，哪个 owner 有权处理写。Leader term 防止旧 leader 继续写，log index 给控制操作排序，commit index 说明哪些操作已被多数派接受。共识不是从论文术语开始，而是从“双控制面同时提交会毁掉系统”这个事故中被逼出来。

七、共识日志里放什么，不放什么 热点 key 的所有输入记录很大，把每条记录都写进共识日志会让控制面崩溃。但 route epoch、partition assignment、manifest generation、迁移状态、leader term 这些元数据很小，却决定数据是否有效。正确边界是：大数据块走数据面，带 checksum、可重算、可清理；决定哪些大数据有效的元数据走控制面，要求强一致和可恢复。

这个边界能解释很多工程选择。Shuffle block 丢了可以重算，manifest 分叉则会重复或遗漏；worker 指标丢了可以近似，route epoch 错了会写错 owner；trace 丢几条会影响诊断，commit log 丢一条会破坏语义。不是所有状态都同等重要，系统要把强路径留给不可随便重算、对外承诺强、需要唯一顺序的状态。

八、迁移期间的三种策略都要写失败路径 冻结写入最容易证明：旧 owner 停止写，复制完成后切 epoch，新 owner 服务。缺点是不可用窗口。日志追赶减少停顿：旧 owner 继续写 WAL，新 owner 加载快照后追增量，追到切换点再发布新 epoch。难点是增量追赶可能永远赶不上热点写入。双写看起来可用性最高：旧 owner 和新 owner 都写成功才返回。难点是一边成功一边失败时需要补偿，读路径还要知道谁是权威。

教材不能只说“可以双写”。双写必须回答：request id 如何去重，两个 owner 的响应谁提交，某边失败是否重试，切换后旧响应如何处理，恢复时以哪边日志为准。若这些问题答不出，双写就是把不一致藏到正常路径。对于本书项目，更推荐先实现冻结或日志追赶的教学版，把状态机和 epoch 讲清楚，再讨论双写的复杂性。

九、用一条时间线审查完整事故 时间线应这样写。t0，route epoch 41 中 partition 7 归 A。t1，热点出现，Driver 决定迁移并在控制日志中追加 Prepare。t2，A 生成快照到 log index 300，B 加载快照。t3，A 继续接收写并记录增量到 340。t4，B 追到 340，Driver 在 term 8、log index 350 提交 Switch，发布 epoch 42。t5，旧客户端按 epoch 41 给 A 发写，A 返回 StaleRoute。t6，新客户端按 epoch 42 给 B 或 C 发写。t7，旧 Driver A 从网络分区恢复，尝试提交 epoch 41 manifest，存储层因 term 过旧拒绝。t8，Clean 阶段删除旧副本。

这条时间线把分片、复制和共识连在一起。分片解释 partition 7 为什么能迁移；复制解释 B 如何获得状态；epoch 解释旧写为什么拒绝；term 解释旧 Driver 为什么无权提交；commit index 解释 Switch 何时对外生效；manifest generation 解释 reduce 读取哪个结果。若某个概念不能放进时间线，它就还没有被讲透。

十、测试应该攻击所有权，而不是只看输出 分片复制测试最容易写成“最终 key 数正确”。这不够。要检查旧 epoch 写是否被拒绝，旧 owner 是否停止提交，新 owner 是否只在 Switch committed 后服务写，落后副本是否不能接管，旧 Driver 是否被 fencing，manifest 是否只有一个 generation，热点拆分后的子 key 是否能精确合并。输出正确只是最后结果，所有权测试才说明系统为什么正确。

可以构造一个最小回归：三条 login 记录，partition 7 从 A 到 B，旧请求在切换后迟到。期望 A 返回 StaleRoute，B 接受新 epoch 写，manifest generation 42 只引用 B 的结果。再加入复制：A、B、C 三副本，版本 10 只在 A 上，A 崩溃后不能让 B 接管为版本 10；版本 10 在 A 和 B 上，A 崩溃后 B 可以接管。通过这两组小测试，Quorum 和 epoch 就不再是口号。

十一、C++ 实现要让非法提交无法轻易写出 接口设计要把 epoch 和 term 放进提交路径。不

要让 `commit(partition_id, block_id)` 这种函数存在，因为调用者可以忘记所有权。更好的教学接口要求传入 `PartitionRequest`、当前 `OwnershipRecord` 和 `ManifestGeneration`，提交函数内部先检查 `partition`、`epoch`、`owner`、`state`、`term` 和 `checksum`。只有检查通过，才追加 manifest entry。

Listing 17.3 提交接口显式要求所有权记录

```

1 enum class OwnershipState : std::uint8_t {
2     Serving,
3     MigratingOut,
4     MigratingIn,
5     Draining,
6     Retired,
7 };
8
9 struct OwnershipRecord {
10     std::uint32_t partition_id = 0;
11     std::uint64_t route_epoch = 0;
12     std::uint64_t leader_term = 0;
13     std::uint32_t owner_worker = 0;
14     OwnershipState state = OwnershipState::Serving;
15 };
16
17 bool can_commit_partition_output(const PartitionRequest& request,
18                                 const OwnershipRecord& owner,
19                                 std::uint64_t current_term) {
20     if (request.partition_id != owner.partition_id) {
21         return false;
22     }
23     if (request.route_epoch != owner.route_epoch) {
24         return false;
25     }
26     if (owner.leader_term != current_term) {
27         return false;
28     }
29     if (request.owner_worker != owner.owner_worker) {
30         return false;
31     }
32     return owner.state == OwnershipState::Serving;
33 }

```

这个示例仍然简化，但它体现了本章要求：提交不是把 block 写到文件，而是验证当前权利。若未来把控制面改成真实 Raft，`leader_term` 和 `route_epoch` 的来源会变，但提交前验证权利的接口不变。这样代码结构就和概念推导一致。

十二、把本章压缩成工程判断 分片解决“状态太大或太热”，复制解决“owner 会故障”，共识解决“多个控制面可能同时声称自己有权”。三者不是并列名词，而是从状态归属问题一步步长出来。设计时先问某份状态当前归谁，迁移时谁有权，故障后哪个版本可信，读取时允许看到多旧，提交时哪个日志或 manifest 说了算。答清这些问题，本章概念自然成立；答不清，术语再多也不能保证系统正确。

17.19 贯穿审查：只跟踪一个 key 的一生

分片、复制和共识如果讲得太大，很容易变成系统名词列表。一个更有效的审查方法，是只跟踪一个 key：`event=login`。它最初属于 partition 7，owner 是 worker A，route epoch 是 41。随着流量增加，它被识别为热点，系统决定把它拆成两个子 key，迁移到 worker B 和 worker C。期间 A 仍在处理旧请求，B 正在加载快照，C 正在追增量，Driver 又经历一次 leader 切换。只要把这个 key 的所有事件写清，本章的大部分概念都会自然出现。

一、出生：key 先进入分片函数 记录被解析后，系统根据 partition function 把 `login` 映射到 partition 7。这里至少有三个事实：使用的分片函数版本，key 的规范化规则，partition id。若两个 worker 对字符串大小写、编码或 hash seed 理解不同，同一个 key 会进入不同 partition。分片函数不是数学背景，而是协议的一部分。它的 version 应进入 manifest 和 trace。

二、成长：热点不是平均负载能解释的 全局吞吐正常时，partition 7 仍可能长尾，因为大部分 `login` 都打到它。报告不能只看平均每个 partition 多少记录，要看最大 partition、前十 key 占比、reduce p99 和队列水位。热点 key 让系统明白：负载均衡不是只按元素数平均，而是按真实工作和资源路径分析。这个事实逼出热 key 拆分和二级合并。

三、拆分：子 key 不能改变最终语义 把 `login` 拆成 `loginshard0` 和 `loginshard1` 可以分散 reduce 压力，但最终输出仍必须是 `login` 的总计数。子 key 是执行层身份，不是业务层 key。Manifest 要记录子 key 的来源和合并规则，最终报告要把子 key 聚回业务 key。若监控只看子 key，就会误以为热点消失；若输出忘记合并，业务语义就错了。

四、迁移：旧 owner 只拥有旧 epoch Epoch 41 时 A 有权处理 partition 7；Epoch 42 发布后，B 和 C 接管子分片。A 可以清理旧状态，可以转发，可以拒绝旧请求，但不能在 epoch 42 下提交新结果。这个规则要写进 A 的请求处理和 manifest 提交层。只在 Driver 内存里记住新 owner 不够，因为旧 A 可能恢复后继续发送结果。Fencing 必须出现在所有写入路径。

五、复制：哪个副本有最新的 login 状态 迁移前，A 的状态可能复制到 B 和 C。若 A 到版本 100，B 到 98，C 到 100，A 崩溃后 C 可以成为候选；若只有 A 到 100，B 和 C 都到 98，版本 100 是否已提交取决于写提交点。跟踪一个 key 的版本比背 quorum 更直观：客户端看到成功的版本，故障后必须能在足够副本中找回证据，否则不能承诺它仍然存在。

六、读取：用户到底能看到哪个 login 迁移后，监控看板可能读 follower，任务调度可能读 leader，manifest commit 必须读强一致状态。若看板读到 epoch 41 的 `login`，可以标注 stale；若调度器用旧读决定 owner，就会把请求发错；若 manifest 用旧读提交，结果可能分裂。读一致性要按用途选择。一个 key 的读路径审查能避免把所有读都简单归为“查询”。

七、删除：旧子分片不能立刻物理删除 迁移完成后，旧 owner A 的临时 block 和旧子分片状态不能马上物理删除。旧 attempt 响应可能迟到，恢复扫描可能还需要解释旧文件，落后 follower 可能还要补日志。清理要等 manifest generation、route epoch 和 checkpoint 都越过安全点。删除是协议动作，不是磁盘清理脚本。这个问题也解释 tombstone 和日志截断为什么复杂。

八、回滚：迁移失败时回到哪个版本 如果 B 加载快照成功但 C 追增量失败，系统是否回滚到 A，是否冻结 `login`，是否继续只迁移一半？答案取决于迁移状态机。Prepare、Snapshot、CatchUp、Switch、Clean 每个阶段都要能恢复或回滚。只说“迁移失败重试”不够，要写清旧 owner 是否仍有写权限，新 owner 已接收的写如何处理，客户端看到什么错误。一个 key 的时间线能逼出这些细节。

九、审计：最终报告要能重放 login 的历史 最终 run report 应能按 key 查询：`login` 在 epoch 41 属于 partition 7，热点检测在某个时间触发，迁移计划生成，快照 checksum，增量日志范围，switch log index，epoch 42 发布，旧请求拒绝数，二级合并 checksum。这样的审计材料不是企业级花活，而是出现错误时唯一能说明系统为什么可信的证据。

十、从一个 key 推广到全系统 系统有很多 key，但每个 key 都遵守同一套归属规则。跟踪一个 key 能讲清状态归属，跟踪一个 partition 能讲清负载和迁移，跟踪一个副本能讲清复制和恢复，跟踪一条控制日志能讲清共识。学习顺序应该从小对象开始，再扩大尺度。若一个 key 的故事讲不清，全系统的宏大图也只是概念堆叠。

17.20 读一致性的具体选择

复制系统不仅要讲写入，也要讲读取。读请求看起来没有副作用，但它决定用户看到哪个历史。读 leader 可以获得较强顺序，但会增加 leader 压力；读 follower 可以分散流量，但可能读到旧值；quorum read 可以提高新鲜度，但增加延迟；带版本的陈旧读可以服务搜索、统计等可容忍场景。读一致性必须按业务语义选择，而不是统一套一个答案。

一、强读适合决策型请求 如果读取结果会直接决定后续写入、扣费、权限或任务归属，通常需要强读或至少读到当前 leader 已确认的状态。否则客户端可能基于旧 owner、旧余额或旧权限做错误决定。强读的成本是延迟和 leader 负载，但它减少了业务补偿复杂度。教材应让读者把“读”也当成协议选择，而不是默认无风险操作。

二、陈旧读适合观测型请求 搜索结果、监控看板、统计预览常能接受短暂陈旧。此时可以读 follower，并在响应中标注版本或滞后程度。用户看到的是某个时间点的近似视图，而不是线性化最新状态。陈旧读必须诚实表达边界：最大滞后、是否可能缺少刚提交数据、是否可能读到已删除数据。没有边界说明，陈旧读会在业务层制造误解。

三、读修复和后台同步 读 follower 发现版本落后时，可以触发读修复或后台同步。读修复不能阻塞所有请求，也不能让客户端承担无限等待。工程上常把修复作为异步任务，并用指标记录落后副本数量、最大 lag 和修复时间。这样系统既能提供较低延迟读，又能逐步收敛。读路径和修复路径要分开，避免一次慢修复拖垮用户请求。

四、版本号让读结果可解释 响应中带上 epoch、term、log index 或 data version，能让客户端和调试工具知道结果来自哪个历史点。没有版本号，读到旧值时只能猜。版本号还可以帮助缓存失效、路由更新和幂等处理。它不是额外字段，而是分布式系统解释自身状态的语言。

五、把读策略写进接口 接口可以显式区分 strong read、bounded stale read 和 best effort read。调用者根据业务选择，系统根据选择路由到 leader、quorum 或 follower。这样一致性不再藏在实现里。读者实现项目时，即使只支持两种模式，也应把模式写进请求头和报告中，为后续扩展留下清楚边界。

17.21 案例深化：从双 Driver 事故推导共识边界

共识不应该从 Paxos 或 Raft 的步骤开始。先看一个更具体的事故：Driver A 认为自己仍是 leader，Driver B 在网络分区另一侧也被选为 leader。两个 Driver 都接收 task commit，一个把 attempt 3 写进 manifest，另一个把 attempt 4 写进另一个 manifest。Worker 都觉得自己听的是合法控制面，最终 reduce 不知道哪个 manifest 是真相。这个事故逼出 leader term、log index、commit index 和多数派确认。

一、Leader 解决谁能接受写 系统需要一个当前权威接受控制面写入，例如 route table 变化、manifest commit、task ownership 转移。Leader 不是为了显得集中，而是为了让写入口唯一。没有 leader 或等价机制，两个控制面可能同时做互斥决定。Leader 身份必须带 term，旧 leader 网络恢复后不能继续以旧 term 写入。

二、日志解决顺序 只知道谁是 leader 还不够，还要知道操作顺序。Manifest commit A 在 route change 前还是后，决定它是否有效；shard move 的 copy、switch、clean 必须按阶段发生。日志 index 给这些操作编号，commit index 表示哪些操作已经被多数派接受并可应用。没有日志，恢复时只能猜哪些操作完成。

三、多数派解决分区中的唯一事实 多数派确认的直觉是两个多数集合必有交集。若 term 5 的某条日志已经被多数派接受，term 6 的新 leader 也必须从多数派中获得信息，不能完全不知道旧提交。这个交集让系统在分区恢复后收敛到一条日志。Quorum 不是公式游戏，而是在故障中保留共同事实的办法。

四、共识边界要小 把每条数据记录都放进共识日志会拖垮系统。控制面需要共识：route epoch、leader term、manifest generation、shard ownership。数据面需要吞吐：shuffle block、输入字节、局部 partial。共识边界越小，系统越容易扩展；边界太小又会丢失关键提交。工程判断是把“决定哪些数据有效”的元数据放进强路径，把“大量数据搬运”放进数据路径。

五、教学项目可以模拟语义 本书不要求实现完整 Raft，但项目可以模拟 term 和 commit index。Driver 写 manifest 前必须持有当前 term；旧 term 消息被拒绝；manifest generation 单调递增；恢

复时读取最高 committed generation。这样读者先学会共识字段解决什么事故，再进入完整协议时不会只背选举流程。

17.22 案例复盘：旧 owner 迟到写入怎样推出共识边界

Route epoch 切换后，旧 owner 收到一条迟到写请求。如果旧 owner 按旧 hash 接受写，新 owner 又按新路由接受写，同一个 key 就出现两个权威。分片首先是状态归属，复制和共识解决的是故障中谁有权决定顺序。

从 route epoch 到 leader term Route epoch 判断请求新旧，move state 判断迁移阶段，leader term 判断控制面权威，commit index 判断哪些日志已经生效。冻结、转发、双写都不是绝对正确，必须写出部分成功、旧请求和恢复路径。Quorum 的意义是让两个多数集合保留共同事实。

实验判据 让 epoch 1 写请求在 epoch 2 后到达，分别测试冻结、转发和双写。再让旧 leader 网络恢复后继续发送 commit，验证新 term 拒绝它。指标包括 stale route、迁移耗时、quorum ack、热点 key 倾斜和恢复 generation。

17.23 本章习题

习题与作业

习题一：为一个 key-value 系统选择分片策略。分别讨论点查、范围查询、递增 key 写入和热点 key 下哈希分片与范围分片的表现。

习题二：设计再分片状态机。至少包含准备、复制、追增量、切换、清理五个状态。说明每个状态失败后如何重试。

习题三：比较三种写提交点：leader 内存返回、本地 WAL 返回、多数副本返回。分别说明延迟、数据丢失风险和读路径影响。

习题四：用 Raft 直觉解释任期号、日志索引和多数提交分别解决什么问题。不要写协议伪代码，重点写工程意义。

习题五：给日志统计 shuffle 设计热点 key 拆分方案。说明如何拆、如何二级合并、如何保持幂等提交。

分布式计算工程实践

先看最终项目要能复现的一次完整事故：输入包含三个 split，其中一个 split 有非法字段，一个 key 是热点；worker 1 写出 task 0 后 Driver 在 manifest 提交后崩溃；worker 2 的 task 1 第一次写出遇到短写；task 2 的成功响应被 MessageBus 丢弃，Driver 超时重试，旧响应随后迟到。正常路径里这只是一个日志统计作业，事故路径里它同时考验解析错误、I/O 可靠写、任务状态、重试、幂等提交、热点 key、trace 和恢复。

本章从这个端到端事故进入分布式计算工程实践。分布式计算工程把算法、运行时、存储、网络和观测性结合起来。真正的系统不仅要在正常路径跑得快，还要在机器慢、网络抖动、任务失败和数据倾斜时保持可诊断、可恢复。前面章节讲的每个局部机制，最后都要回到这个项目里接受同一次故障演练。

18.1 从 MapReduce 到 Shuffle

Shuffle 把数据按 key 重新分布，是很多分布式计算框架的核心成本。它包含序列化、压缩、网络传输、落盘、排序、合并和反序列化。一个 group-by 或 join 的成本，常常主要来自 shuffle，而不是用户函数。

优化 shuffle 的方向包括减少数据量、提前聚合、压缩、分区均衡、避免热点 key、使用本地磁盘顺序写和控制并发拉取。它是分布式版的数据布局问题。

Shuffle 的基本流程可以拆成 map 端分区、局部排序或缓冲、落盘、网络拉取、reduce 端合并。每一步都有独立瓶颈：序列化消耗 CPU，压缩节省网络但增加计算，落盘受磁盘带宽影响，拉取并发过高会压垮对端，key 倾斜会让少数 reduce 任务变成长尾。优化 shuffle 不能只调一个参数，要先确认慢在哪一步。

18.2 检查点、幂等和容错

长任务必须考虑失败。检查点保存中间状态，让失败后从最近点恢复，而不是从头开始。检查点太频繁会拖慢正常路径，太稀疏会增加恢复成本。流式计算还要处理 exactly-once 或 at-least-once 语义。

检查点设计要说明一致性边界。批处理任务可以在 stage 边界保存输出，失败后重跑某个分区；流式任务需要同时保存输入 offset、算子状态和输出提交位置。若只保存算子状态，不保存已经输出到哪里，恢复后可能重复输出；若只保存 offset，不保存状态，聚合结果会丢失。所谓 exactly-once 往往不是“消息真的只执行一次”，而是通过幂等提交、事务输出或去重让外部效果等价于一次。

观测性 没有观测性，分布式系统无法维护。日志、指标、trace、profile 和事件审计要能回答：请求经过哪些节点，在哪里等待，重试了几次，队列多长，哪个分片慢，哪个任务倾斜，哪次部署引入变化。

观测性要围绕问题设计。延迟问题需要端到端 trace 和每段耗时；吞吐问题需要输入速率、处理速率和队列长度；倾斜问题需要按 key、分片、worker 的分布；可靠性问题需要错误码、重试、超时和恢复路径；资源问题需要 CPU、内存、磁盘、网络 and GC 或分配器指标。只打印一行“任务失败”没有工程价值。

从单机原则到集群原则 第二册前面讲的原则在分布式系统中仍然成立。局部性对应数据本地调度；锁竞争对应热点 key；false sharing 对应多个任务争用同一分片；背压对应限流和队列控制；归约对应 map-side combine 和树形聚合；缓存一致性对应复制一致性和失效策略。系统规模变大，原则没有消失，只是成本单位从 cache line 变成网络消息和磁盘块。

一个贯穿工程案例 考虑一个日志统计系统：输入日志按文件分片放在多台机器上，每个 worker 读取本地分片，解析出 key，先做本地 hash 聚合，再把 partial 按 key 分区 shuffle 到 reduce worker，最后合并并写出结果。这个系统把第二册几乎所有原则串在一起。本地解析要考虑 CPU 和内存布

局，本地聚合要避免锁热点，shuffle 要处理网络和背压，reduce 要处理 key 倾斜，输出要保证幂等，失败后要能从检查点恢复。

如果某个 key 特别热，单机里它像一个热点锁，分布式里它像一个热点分片。解决方法也相似：先把热点拆成多个局部 partial，再在后续阶段合并。若网络拉取过多，像单机任务队列无背压一样，系统会排队爆炸。若 worker 失败后重跑，输出必须用任务 ID 和分区 ID 保证幂等提交。这样的案例比孤立讲术语更能训练系统思维。

核心思想

分布式计算不是单机计算的替代，而是单机计算原则在通信、故障和复制约束下的扩展。

18.3 贯穿项目架构

Compute Systems Engine 在第二册最后要变成一个本机可运行的分布式计算模拟。它不追求替代 Spark 或 Flink，而是把前面章节的核心原则压进一个可测、可失败、可恢复的小系统。输入是一批日志记录，map worker 读取本地分片并解析 key，map 端先做本地聚合，shuffle 把 partial 按 key 分发给 reduce partition，reduce worker 合并结果，driver 负责任务分配、attempt、检查点和最终提交。

这个项目的价值在于贯穿。顺序 reference 来自第一批归约；数据布局来自热字段和冷字段分离；SIMD 和 map 内核用于解析和过滤；scan 和 partition 用于分配 shuffle 输出；bounded queue 和背压用于阶段之间限流；线程池和任务图用于本机执行；幂等和 request id 用于重试；检查点用于恢复；观测性用于解释慢在哪里。它不是另起炉灶，而是把整本书串起来。

数据格式和 reference 任何分布式项目都要先有可验证 reference。日志记录可以简化成三列：timestamp、user id、event id。reference 单线程读取全部输入，按 event id 计数，输出排序后的计数结果。分布式模拟的最终输出必须和 reference 一致。只有 reference 清楚，后面的重试、分区、合并和检查点才有正确性锚点。

Listing 18.1 日志记录和聚合结果的简化类型

```
1 struct LogRecord {
2     std::uint64_t timestamp = 0;
3     std::uint64_t user_id = 0;
4     std::int32_t event_id = 0;
5 };
6
7 struct EventCount {
8     std::int32_t event_id = 0;
9     std::int64_t count = 0;
10 };
```

这里用明确位宽类型，是为了让文件格式、网络消息和内存布局都可解释。真实系统还会处理字符串、变长字段和 schema 版本，但教学项目先用固定字段，能把注意力放在计算和系统语义上。

Map 端本地聚合 朴素 shuffle 会把每条日志都发送到 reduce 端。若一亿条日志中只有一万个 event id，这样会浪费大量网络和序列化成本。map-side combine 先在每个 worker 本地对 event id 计数，再发送 partial。这样发送量从“记录数”下降到“本地出现过的 key 数”。这是 reduce 原则在网络尺度上的应用。

本地聚合可以用哈希表，也可以在 event id 范围小且密集时用数组。哈希表适合稀疏 key，但有随机访问、分配和 rehash 成本；数组适合密集小 key，连续且容易向量化，但 key 空间大时浪费内存。教材项目可以同时保留两种策略，让读者根据 key 分布选择。

Shuffle 文件和网络缓冲 真实 shuffle 常把 map 输出按 partition 写成多个缓冲或文件。每个 reduce partition 拉取属于自己的数据。即使本机模拟，也可以按 partition 组织缓冲：map worker 产生 `partition_id->partials`，message bus 按 partition 投递给 reduce worker。这样读者能看到 shuffle 的核心结构，而不用先搭真实网络。

Shuffle 的成本包括序列化、压缩、缓冲、落盘、发送、接收和合并。每一步都可以成为瓶颈。压缩减少网络字节但增加 CPU；落盘提高容错但增加 I/O；内存缓冲快但可能 OOM；并发拉取提高吞吐但可能压垮 map 端。优化前必须先记录每一步的字节数和耗时。

Reduce 端合并和热点 key Reduce 端收到多个 map worker 的 partial 后，按 key 合并。若 key 分布均匀，每个 reduce partition 工作量接近；若某个 key 极热，它所在 partition 会成为长尾。热点 key 的处理通常要二级聚合：先把热 key 拆成多个子分区，让多个 reduce worker 合并 partial，再把这些结果做最后合并。

这和单机 sharded counter 完全对应。全局 hot key 就像全局原子计数器；map-side combine 是线程本地 partial；热 key 拆分是 sharded counter；最终合并是读取总数。第二册前面的并发原则在这里直接变成分布式算法。

任务 attempt 和幂等输出 分布式任务可能失败重试，因此一个逻辑 task 可能有多个 attempt。输出提交必须按逻辑 task 去重，不能让两个 attempt 都提交成功。常见做法是每个 task 输出到 attempt 临时位置，完成后由 driver 或 commit protocol 原子地把某个 attempt 标记为成功。其他 attempt 即使晚完成，也不能覆盖已提交结果。

Listing 18.2 任务提交元数据

```
1 enum class TaskState : std::int32_t {
2     Pending,
3     Running,
4     Committed,
5     Failed,
6 };
7
8 struct TaskAttempt {
9     std::uint64_t task_id = 0;
10    std::uint32_t attempt = 0;
11    TaskState state = TaskState::Pending;
12 };
```

这个模型同样适用于本机模拟。worker 完成后发送 commit 请求，driver 检查 task id 是否已经 committed。若没有，则提交该 attempt；若已经提交，则丢弃重复结果并记录 duplicate attempt。这个机制训练 exactly-once 外部效果的基本思想。

检查点的内容 检查点不能只保存“程序运行到第几步”。对于日志统计系统，检查点至少应包含输入分片进度、已完成 map task、shuffle 输出位置、reduce partition 状态、已提交输出和请求去重表。缺少任何一项，恢复后都可能漏处理或重复输出。

检查点频率要权衡。每处理一条记录就 checkpoint，正常路径太慢；只在最后 checkpoint，失败恢复成本太高。批处理可以在 stage 边界 checkpoint，例如 map 输出全部完成后记录 shuffle manifest，reduce 输出提交后记录最终结果。流式系统则需要周期性 checkpoint，并把输入 offset、状态和输出提交绑定。

Exactly once 的工程含义 Exactly once 常被误解成“每条消息真的只执行一次”。在真实系统中，消息可能重复，任务可能重跑，函数可能执行多次。工程上追求的是外部效果等价于一次：重复执行被幂等提交、去重表或事务输出吸收。理解这点非常重要，否则读者会在网络不可靠时追求不可能的语义。

对于日志统计，若同一个 map attempt 输出被 reduce 合并两次，计数会翻倍。解决方法是 reduce 端按 `task_id, attempt` 或 logical map id 去重，或者 driver 只让一个 attempt 的输出进入 manifest。提交协议要定义清楚哪些输出属于最终结果，哪些只是临时文件。

调度和数据本地性 Driver 调度 map task 时，应优先把任务放到拥有输入分片的 worker；若本地 worker 忙或失败，可以远程调度。Reduce task 则按 partition 分配，尽量让同一 partition 的数据由同一个 worker 合并，减少跨节点状态移动。若某个 reduce partition 太大，可以拆分或引入二级 reduce。

调度还要考虑 speculative execution。若某个 task 明显慢，可以启动另一个 attempt。第一个完成并提交的 attempt 获胜，另一个 attempt 的结果丢弃。Speculation 能处理慢节点，但会增加资源消耗和重复输出风险。没有幂等提交，不应开启 speculation。

背压贯穿 shuffle Shuffle 最容易出现过载。Map 端产生数据太快，reduce 端处理不过来，网络缓冲和队列增长。正确做法是让 reduce 端容量反馈到 map 端：每个 reduce partition 有接收队列容量，队列满时 map 端暂停、降低发送速率或批量合并。若 map 端无限发送，最终只是把 OOM 从一个节点转移到另一个节点。

背压指标包括每个 partition 的队列长度、入站字节率、处理字节率、拒绝或 busy 次数、map 端等待时间和端到端延迟。没有这些指标，shuffle 慢时只能猜。第 15 章的有界队列实验在这里变成跨 worker 流控。

观测性设计 一个可维护的分布式计算系统至少要回答七个问题：哪个 stage 慢，哪个 task 慢，哪个 worker 慢，哪个 partition 数据最多，哪个 key 最热，重试发生在哪里，输出提交是否重复。指标要按 stage、worker、partition、task attempt 和 key 分布聚合。

日志要包含 task id、attempt、partition、request id 和错误码。Trace 要能从 driver 调度追到 worker 执行、shuffle 发送、reduce 合并和输出提交。Profile 要能分辨 CPU 解析、hash 聚合、序列化、压缩、网络等待和磁盘写入。观测性不是最后加的，它决定系统是否能被调试。

故障注入 没有故障注入的分布式系统测试是不完整的。项目应主动注入：随机丢消息、延迟消息、重复消息、让 worker 中途失败、让 reduce 队列变慢、让某个 key 极热、让磁盘写入失败、让 commit 响应丢失。每个故障都应有预期行为：重试、去重、背压、恢复或明确失败。

故障注入要可重复。使用固定随机种子，记录注入事件，失败时能重放。否则低概率 bug 很难定位。系统工程的测试不只看 happy path，更要让失败路径成为日常。

本机模拟实现路线 第一阶段，只实现串行 reference 和单进程多 worker 模拟。输入切成分片，worker 本地聚合，driver 合并所有结果。第二阶段，加入 shuffle partition，每个 map worker 把 partial 发到 reduce partition 队列。第三阶段，加入 request id 和 attempt，支持重复消息去重。第四阶段，加入 checkpoint manifest，失败后从 manifest 恢复。第五阶段，加入热点 key 拆分和背压。

每阶段都保留测试。第一阶段测试结果正确；第二阶段测试 partition 后结果仍一致；第三阶段测试重复消息不重复计数；第四阶段测试崩溃恢复；第五阶段测试热点和过载下系统不会无限排队。这样项目逐步长大，不会变成一次性堆代码。

端到端 reference 分布式计算项目必须先有端到端 reference。对日志统计来说，reference 可以是单线程读取所有记录，解析出 key，然后用一个 map 聚合计数，最后按 key 排序输出。它不追求性能，只定义语义：哪些行有效，解析失败如何处理，key 如何规范化，计数类型是什么，输出顺序如何确定。后续 map-side combine、shuffle、reduce、热点拆分和重试都要和它对比。

Reference 还要固定输入格式。不要一边开发分布式执行器，一边让输入格式隐式变化。可以定义每行包含 timestamp、user id、event type 和 value；解析失败的行进入错误计数；event type 作为聚合 key；value 使用 `std::int64_t` 累加。若后续加入更多字段，也要版本化。计算系统项目不是只练线程和网络，也要练数据合同。

Listing 18.3 日志记录的教学格式

```
1 struct LogRecord {
2     std::uint64_t timestamp = 0;
3     std::uint64_t user_id = 0;
4     std::uint32_t event_type = 0;
5     std::int64_t value = 0;
6 };
```

这段结构体用于内存中的解析结果，不等于文件格式。文件格式可以是文本或二进制，但解析后进入稳定内存结构。所有优化版本都以同样的 `LogRecord` 或等价列式布局作为输入，避免把解析差异误当成执行器差异。

输入分片和 offset 按文件分片时，不能随意从字节中间开始解析一行。若每个 worker 负责文件字节范围，分片边界要调整到行边界：非零 begin 先跳到下一行起点，end 可以读到当前行结束。

每个分片记录文件名、begin offset、end offset 和分片 ID。这样失败恢复时可以重新读取同一范围。

如果输入是二进制块，分片边界要按记录格式对齐。若记录变长，就需要索引或块目录。不要把“把文件切成 N 份”说得过于简单。分片正确性决定每条记录是否恰好处理一次。漏一行和重复一行都会让最终计数错误。

Listing 18.4 输入分片元数据

```
1 struct InputSplit {
2     std::uint64_t split_id = 0;
3     std::uint64_t file_id = 0;
4     std::uint64_t begin_offset = 0;
5     std::uint64_t end_offset = 0;
6 };
```

`split_id` 是逻辑任务 ID 的基础。即使 attempt 重试，`split_id` 不变。输出提交时按 `split_id` 去重，而不是按 attempt 去重。这个原则贯穿整个项目。

Map-side combine 的内存设计 Map worker 不应每读一条记录就发一条网络消息。它应在本地按 key 聚合，形成 partial map，再按 reduce partition 输出。这样减少网络消息数量和字节数，也把热点 key 的一部分压力留在本地。Map-side combine 是分布式版的线程本地 partial。

本地聚合的数据结构要根据 key 空间选择。event type 如果是小整数，可以使用 vector buckets；如果 key 是字符串或大整数，可能使用哈希表；如果输入已经按 key 排序，可以顺序合并。教材项目可以从小整数 key 开始，后续扩展到字符串 key。无论使用哪种结构，都要记录内存峰值和 key 数量，否则大输入可能让 map 端内存失控。

Listing 18.5 小整数 key 的本地聚合

```
1 std::vector<std::int64_t> combine_events(
2     std::span<const LogRecord> records,
3     std::uint32_t event_type_count) {
4     std::vector<std::int64_t> partial(event_type_count, 0);
5     for (const LogRecord& record : records) {
6         partial[record.event_type] += record.value;
7     }
8     return partial;
9 }
```

生产代码要检查 `event_type` 越界，并处理解析错误。教学片段突出 combine 思想。局部聚合后的数据再按 partition 打包，发送给 reduce。

Shuffle 文件和消息 Shuffle 可以走内存消息，也可以写本地文件再由 reduce 拉取。内存消息简单，适合本机模拟；文件 shuffle 更接近大规模批处理，因为中间数据可能超过内存，也需要失败恢复。无论哪种方式，都要有 manifest 描述哪些 map 输出存在、属于哪个 partition、来自哪个 attempt、大小和校验是什么。

Manifest 是控制面，shuffle 数据是数据面。Reduce 不应扫描目录猜哪些文件有效，而应读取 driver 提交的 manifest。这样重复 attempt 产生的临时文件不会被误读；失败 attempt 的残留文件也不会进入结果。控制面决定有效数据集合，是 exactly-once 外部效果的关键。

Listing 18.6 Shuffle manifest 条目

```
1 struct ShuffleBlock {
2     std::uint64_t map_task_id = 0;
3     std::uint32_t attempt = 0;
4     std::uint32_t reduce_partition = 0;
5     std::uint64_t byte_size = 0;
```

```

6  std::uint64_t checksum = 0;
7  };

```

真实系统还会记录文件路径、压缩格式、记录数、写入时间和副本位置。教学项目至少要有 map task、attempt、partition、大小和 checksum。恢复时，driver 读取 manifest 后校验每个 block 是否存在且摘要匹配。

Reduce 端去重 Reduce 端不能简单地把收到的每条 partial 都加进去。若 map attempt 因响应丢失而重试，reduce 可能收到两个内容相同的 partial。正确策略是按逻辑 map task 去重，或者只接受 manifest 中被 driver 选中的 attempt。若 reduce 直接接收推送消息，也要维护已处理 request id 或 map task id 集合。

去重的粒度要慎重。按 attempt 去重只能防止同一 attempt 重复发送，不能防止不同 attempt 的同一逻辑 task 重复合并。按 map task id 去重能保证一个逻辑输入分片只贡献一次，但要处理 attempt A 部分发送后失败、attempt B 重试成功的情况。更稳的做法是 attempt 输出先写临时 block，driver 只在完整成功后把某个 attempt 加入 manifest，reduce 根据 manifest 拉取。

输出提交协议 最终输出也需要提交协议。Reduce task 不应直接写最终文件名，因为失败重试可能留下半文件，多个 attempt 可能互相覆盖。常见做法是每个 attempt 写到临时路径，写完并校验后向 driver 提交。Driver 检查该 reduce partition 是否已经提交；若未提交，原子发布该 attempt 的输出；若已提交，拒绝重复 attempt 并清理临时文件。

在本地文件系统里，可以用 rename 模拟原子发布。跨对象存储或分布式文件系统时，rename 语义可能不同，需要专门 commit protocol。教材项目可以先用本地目录，但必须在文字中说明提交点：只有 driver manifest 记录的输出才是有效最终结果，临时文件不是。

Checkpoint 内容清单 本项目 checkpoint 至少包含六类内容。第一，输入 splits 和每个 split 的任务状态。第二，map attempt 和已提交 shuffle block manifest。第三，reduce partition 的状态和已提交输出。第四，路由或 partition 版本。第五，请求去重表或其可恢复摘要。第六，作业配置和输入数据摘要。缺少作业配置，恢复后可能用不同 reduce 数重新解释旧 shuffle；缺少去重表，重复消息可能再次生效。

Checkpoint 还要有版本。项目代码升级后，旧 checkpoint 如何读取？教学阶段可以要求版本不兼容时拒绝恢复，并提示重新运行。成熟系统需要迁移工具。不要把 checkpoint 当作临时 dump，它是持久化协议。

故障注入驱动开发 分布式项目应从一开始就有故障注入，而不是最后补测试。每个阶段都加入一个最小故障。Map 阶段加入任务失败重试；shuffle 阶段加入重复消息；manifest 阶段加入响应丢失；reduce 阶段加入慢 partition；checkpoint 阶段加入 driver 重启。这样协议随功能一起成长。

故障注入要可配置。可以在 MessageBus 中按 request id 决定是否丢弃，也可以在 worker 执行到某个 split 时故意失败。随机故障用于压力测试，确定性故障用于单元测试。单元测试应能精准复现“attempt A 写完但提交响应丢失，attempt B 后完成”这种场景。

热点 key 实验设计 热点 key 实验不能只说“构造倾斜输入”。要定义分布。例如 90% 的记录使用 event type 1，其余 10% 均匀分布到其他 key。运行普通 hash partition，记录每个 reduce partition 的输入字节、记录数、处理时间和队列等待。然后启用 map-side combine，再启用热 key 拆分和二级 reduce，对比长尾是否下降。

还要记录网络流量。Map-side combine 可能大幅减少消息数；热 key 拆分可能增加最终二级合并流量，但降低第一阶段长尾。报告要解释总吞吐、尾延迟和资源消耗之间的取舍。不要只写“热点拆分更快”，要说明在哪个阶段更快，代价是什么。

数据倾斜和任务倾斜 数据多不一定任务慢，任务慢也不一定数据多。某些 key 处理逻辑复杂，某些记录解析成本高，某些分区落在慢节点，某些节点 I/O 慢。观测性要区分数据倾斜和执行倾斜。每个 task 记录输入记录数、输入字节、输出字节、CPU 时间、排队时间、重试次数和所在 worker。只有这样，慢任务才能归因。

调度策略也不同。数据倾斜可以通过重新分区、热点拆分和二级聚合处理；执行倾斜可能需要 speculative execution、节点隔离或资源调整；I/O 倾斜可能需要数据本地性和预取；解析倾斜可能需要改变输入格式。系统工程不能把所有慢都归为“节点慢”。

资源隔离 分布式计算作业里，map、shuffle、reduce、checkpoint 和指标上报共享 CPU、内存、

磁盘和网络。若 shuffle 发送占满网络，checkpoint 可能超时；若 reduce 哈希表占满内存，map 输出队列可能 OOM；若指标日志太多，磁盘写入干扰输出提交。资源隔离要明确每个阶段的上限。

本机模拟可以用简单参数表达：每个 reduce partition 最大队列字节数，map 端最大 in-flight shuffle 字节数，每个 worker 最大本地聚合内存，checkpoint 写入频率和并发数。达到上限时触发背压，而不是继续申请内存。资源上限是系统稳定性的组成部分。

压缩和编码 Shuffle 数据常需要压缩。压缩减少网络和磁盘字节，增加 CPU。若 reduce 端是网络瓶颈，压缩有益；若 map 端 CPU 已满，压缩可能变慢。编码也类似：小整数 key 可以用定长数组，稀疏 key 可以用 varint 或字典，字符串 key 可以先 intern 成 ID。压缩格式要进入 manifest，否则恢复和跨版本读取会出错。

实验应比较无压缩、轻量压缩和更强压缩。指标包括 map CPU 时间、shuffle 字节、reduce 解压时间、端到端时间和内存峰值。不要把压缩当作固定开关，要让读者看到它是 CPU 与 I/O 的成本交换。

本机到多机的迁移边界 本机模拟验证语义，但不能证明真实多机性能。迁移到多机时，会新增真实网络延迟、连接管理、序列化成本、内核缓冲、TLS、时钟漂移、节点故障和部署问题。模拟中一个队列 push 可能微秒级，真实 RPC 可能毫秒级；模拟中进程共享时钟，真实机器时钟不一致；模拟中磁盘路径可靠，真实对象存储有不同一致性语义。

因此项目文档要把“已验证”和“未验证”分开。已验证：去重、manifest、提交协议、背压状态机、故障恢复逻辑。未验证：真实网络吞吐、跨机时钟、对象存储 commit、节点安全、部署滚动升级。成熟工程知道自己的证据边界。

代码组织建议 Compute Systems Engine 应按责任分模块。reference 模块只做串行正确性。input 模块处理分片和解析。map 模块做本地聚合。shuffle 模块处理 partition、消息、manifest 和背压。reduce 模块合并 partial。driver 模块维护任务状态、attempt 和 checkpoint。fault 模块提供故障注入。metrics 模块记录指标。不要把所有逻辑写进一个 main 函数，否则后续实验无法扩展。

接口要围绕数据合同，而不是围绕演示脚本。Map worker 接收 InputSplit，输出 ShuffleBlock；Reduce worker 接收 partition id 和 manifest blocks，输出 OutputBlock；Driver 只看 task id、attempt 和状态，不关心内部聚合代码。模块边界越清楚，故障注入和测试越容易。

验收测试清单 本项目最终至少要有这些测试。串行 reference 与分布式模拟结果一致。输入分片覆盖每条记录一次。Map attempt 重试不会重复进入 manifest。Reduce 重复接收同一 block 不会重复计数。Driver 重启后能从 checkpoint 恢复。旧 route epoch 的消息被拒绝。Reduce 队列满时 map 端背压。热点 key 拆分后最终结果仍合并成原 key。输出提交只接受一个 attempt。损坏 block 校验失败并触发重试或作业失败。

性能测试至少包括均匀输入、热点输入、大 key 空间、小 key 空间、慢 reduce、丢消息、重复消息和 checkpoint 频繁写入。每个测试都要有正确性 oracle。性能数字没有 correctness oracle，意义不大。

项目报告结构 综合实验报告建议按固定结构。第一，语义：输入格式、输出格式、错误行处理、计数类型。第二，系统结构：driver、worker、message bus、manifest、checkpoint。第三，正确性：reference 对比、去重、提交点、恢复。第四，性能：各 stage 时间、队列长度、shuffle 字节、热点 key、重试次数。第五，故障注入：列出注入事件和系统行为。第六，限制：哪些只在本机模拟验证，哪些需要真实 Linux 多进程或多机验证。

这样的报告训练的是系统工程表达能力。面试或工作中，真正有价值的不是说“我写了一个分布式计算框架”，而是能清楚说明语义、瓶颈、失败模式和证据边界。

项目目录结构 贯穿项目应该有清晰目录，而不是所有代码堆在一个文件中。建议结构包括 `include/` 放公共接口，`src/input/` 放分片和解析，`src/kernels/` 放 map、filter、histogram、top-k，`src/runtime/` 放线程池、队列和任务图，`src/shuffle/` 放 partition、manifest 和消息，`src/driver/` 放 task 状态和 checkpoint，`src/fault/` 放故障注入，`tests/` 放单元和集成测试，`benchmarks/` 放性能实验。

目录结构体现责任边界。Input 模块不应知道线程池内部锁；kernel 不应直接写 checkpoint；driver 不应解析每行日志；shuffle 不应决定业务 key 的含义。边界清楚，测试才能独立，后续第三册 AI 推理引擎也能复用 runtime、benchmark 和 I/O 模块。

核心类型清单 项目可以先定义少量稳定类型。JobId、StageId、TaskId、AttemptId、PartitionId

使用明确位宽整数，避免随手用平台相关的整数类型。InputSplit 描述输入范围，MapOutput 描述 map 产生的 shuffle block，ReduceOutput 描述最终输出，TaskState 描述任务状态，Checkpoint 描述可恢复状态。这些类型是系统语言。

Listing 18.7 项目 ID 类型的形态

```

1 struct JobId {
2     std::uint64_t value = 0;
3 };
4
5 struct TaskId {
6     std::uint64_t value = 0;
7 };
8
9 struct AttemptId {
10    std::uint32_t value = 0;
11 };
12
13 struct PartitionId {
14    std::uint32_t value = 0;
15 };

```

强类型 ID 可以防止把 partition id 误传成 worker id。C++ 中可以进一步定义比较和哈希。教学阶段即使只用简单 struct，也比裸整数更能表达语义。

18.4 Driver、Worker 和 MessageBus

Driver 是控制面。它维护 job 状态、stage 状态、task attempt、manifest 和 checkpoint。一个 task 可以处于 pending、running、committed、failed、canceled。Attempt 是 task 的一次执行尝试，可以成功、失败或超时。Task committed 表示某个 attempt 的输出被接受，其他 attempt 的输出无效。

Driver 的核心操作包括 schedule task、record attempt start、commit attempt、fail attempt、checkpoint、recover。每个操作都要幂等。比如 commit attempt 请求可能因为响应丢失被重复发送，driver 再次收到同一 attempt commit，应返回已经提交或重复，而不是再次添加 manifest。幂等不是只在 worker 端需要，控制面同样需要。

Worker 执行模型 Worker 从 driver 获取 task，读取 input split，执行 map-side combine，写 shuffle block，发送 commit。Worker 本身不决定最终输出是否有效，它只产生 attempt output。Worker 可以失败、超时或被取消。Worker 重启后，不应依赖内存中旧状态决定提交；有效性由 driver manifest 和 checkpoint 决定。

Worker 也要有本地资源限制：最大本地聚合内存、最大 in-flight shuffle 字节、最大输出临时文件数、最大并行 task 数。没有限制，单个 worker 可能因为热点 key 或慢 reduce OOM。资源限制触发背压或任务失败，driver 再调度。

MessageBus 接口 本机 MessageBus 可以先提供 send 和 poll completion。send 接受 LocalMessage，可能返回 accepted、busy、closed；poll 返回已完成消息或超时。故障注入在 MessageBus 内部进行：延迟、丢弃、重复、乱序。这样 worker 和 driver 不直接调用随机故障逻辑，测试可以统一控制。

MessageBus 还应记录指标：发送消息数、丢弃数、重复数、busy 次数、队列最大长度、平均延迟和超时数。分布式系统的网络层必须可观测，否则上层只会看到任务慢，不知道是网络、reduce 还是 driver。

Manifest 格式 Manifest 是恢复的核心。它应能回答：哪些 map task 已提交，哪个 attempt 获胜，产生了哪些 shuffle block，每个 block 属于哪个 reduce partition，大小和校验是多少，文件路径或消息范围在哪里。Reduce 阶段只读取 manifest 中的 block，不能读取临时目录中所有文件。

Manifest 还要版本化。第一版可以是简单文本或 JSON，但字段要稳定；后续若改格式，version 用于拒绝或迁移。Manifest 写入要使用临时文件加提交步骤，避免半写文件被当成有效。恢复时先校验 manifest，再校验 block。若 block 缺失或 checksum 不匹配，系统应重跑对应 map task 或报告不可恢复。

18.5 恢复、故障注入和验收

恢复算法可以写成明确步骤。第一，加载最新 checkpoint 文件并校验版本和 checksum。第二，恢复 driver 的 job、stage、task 状态。第三，读取 manifest，确认已提交 outputs 存在。第四，把 running 但未 committed 的 task 改回 pending，因为旧 attempt 是否还活着不可依赖。第五，重新调度 pending task。第六，重复收到旧 attempt commit 时，根据 epoch 和 task state 拒绝或去重。

这个算法说明恢复不是“读回内存结构”这么简单。运行中的 task 在崩溃后状态不确定，必须回到可重试状态；已经 committed 的 task 不能重复；临时文件需要清理但不能误删已提交文件。恢复算法应有单元测试。

故障注入表 项目可以维护一张故障注入表。按阶段列：input read failure、parse error、map worker crash、shuffle message drop、shuffle duplicate、reduce busy、commit response lost、checkpoint write fail、driver restart。每项写注入方式、预期行为、需要的测试和指标。表驱动能防止只测自己想到的 happy path。

例如 commit response lost：worker 写出 block 并向 driver commit，driver 已记录 manifest，但响应被丢。Worker 超时后重试 commit。预期：driver 返回 already committed，manifest 不重复，reduce 只读取一次。这个测试能验证幂等提交。

性能里程碑 项目性能里程碑应逐步建立。第一，串行 reference 正确。第二，单机多 worker 比串行在大输入上有加速。第三，map-side combine 减少 shuffle bytes。第四，热点 key 拆分降低 reduce 长尾。第五，背压下队列不无限增长。第六，故障注入下正确性保持。每个里程碑都有指标，不要求一开始达到生产性能。

里程碑顺序很重要。不要在 reference 还不稳定时优化 shuffle；不要在 manifest 去重没做好时开启 speculation；不要在指标缺失时调复杂调度。系统工程需要可验证台阶。

实验输入集 至少准备四类输入。第一，均匀 key，小规模，用于快速 correctness。第二，均匀 key，大规模，用于吞吐和带宽。第三，热点 key，用于倾斜和二级聚合。第四，含错误行和边界值，用于解析和错误处理。每类输入都要有固定 seed 和摘要。测试不能只靠随机生成后立即丢弃 seed。

输入还应覆盖空文件、单行文件、没有换行结尾、超长行、非法字段、event_type 越界、value 溢出边界。系统教材的质量体现在这些边界上。只处理理想 CSV，不是真正的数据处理系统。

结果校验 结果校验不只是比较输出文件文本。可以先把输出解析成 key 到 value 的 map，再和 reference 比较。若输出顺序声明确定，再比较顺序；若输出顺序不保证，就不要因为顺序不同判错。错误行计数、重复请求计数和丢弃低优先级日志也要进入校验。校验规则要和语义合同一致。

浮点或近似算法需要误差校验。日志计数通常是整数精确，适合完全相等。若未来 AI 推理引擎加入浮点算子，校验策略会不同。第二册先用整数聚合训练 exactly-once 和确定性，是有意降低数值复杂度，让系统语义更清楚。

从本机线程到多进程 本机多线程模拟共享地址空间，bug 和真实多进程不同。下一步可以把 worker 做成多进程，通过 pipe、Unix domain socket 或本地 TCP 通信。多进程能训练序列化、进程崩溃、文件持久化和真实内核队列。代价是实现复杂度增加。项目可以先把 MessageBus 接口抽象好，后续替换传输层。

多进程版本要处理进程启动、退出、信号、日志收集和临时目录清理。Worker 崩溃后，driver 要发现并重试任务。共享内存不再可直接传递对象指针，消息必须序列化。这是从“模拟分布式”走向“本机分布式”的关键一步。

安全和权限边界 即使是教学项目，也要意识到权限边界。Worker 不应随意写任意路径，输出目录应由 driver 分配；消息解析要检查长度和版本，避免越界；临时文件名要避免冲突；恢复时不要信任损坏 manifest。真实系统还需要认证、授权和加密，本册不展开，但接口应留出 metadata 和错误码空间。

安全不是第三册或生产阶段才考虑。很多数据损坏来自缺少边界检查和信任内部输入。分布式系统里，“内部消息”也可能因为 bug、旧版本或损坏而非法。

第三册的接口预留 虽然 AI 推理放在第三册，第二册项目可以预留计算内核接口。今天的 map/reduce kernel 处理日志记录，未来可以处理 tensor block；今天的 thread pool 执行 histogram，未来可以执行 GEMM tile；今天的 manifest 记录 shuffle block，未来可以记录模型分片或量化权重缓存。接口应围绕任务、数据块、运行时、指标和检查点，而不是写死日志业务。

这也是为什么第二册先把多核、高性能、并发、I/O 和分布式计算讲透。第三册的 CPU 推理引擎不是凭空出现，它会复用数据布局、SIMD、线程池、背压、checkpoint 和观测性。第二册越扎实，第三册越能深入算子和模型，而不是返工基础设施。

最小可运行版本 最小可运行版本应非常克制。一个命令读取固定输入文件，使用串行 reference 输出结果；另一个命令使用多 worker 模拟，输出同样结果。此时不需要真实网络、复杂 checkpoint 或热 key 拆分。目标是建立输入格式、输出格式、校验器和 benchmark harness。没有这个最小版本，后面所有功能都缺少落脚点。

最小版本也要有错误处理。输入文件不存在、格式错误、event type 越界、value 溢出、输出路径不可写，都要返回明确错误。教材项目不能只在理想输入上成功。系统质量从第一个版本就开始积累。

第二阶段：单机并行 第二阶段加入线程池或固定 worker。输入 splits 被分配给 worker，每个 worker 本地 combine，driver 合并 partial。这个阶段重点是任务切分、partial 布局、worker stats、正确性对比和扩展曲线。还不需要 shuffle，因为所有 partial 可以在进程内合并。

实验应比较串行、每次创建线程、固定线程池三种版本。输入小的时候，串行可能最快；输入大时，多 worker 有收益；任务太细时，线程池调度成本上升。这个阶段验证第二册前半部分：数据布局、线程、NUMA、测量和并行 reduce。

第三阶段：本机 shuffle 第三阶段加入 shuffle partition。Map worker 不再直接把 partial 交给 driver，而是按 reduce partition 写入队列或 block。Reduce worker 从 partition 队列读取并合并。此时出现背压、partition 队列长度、热点 key 和 reduce 长尾。所有通信仍在本机内存中，但数据流已经像分布式系统。

这个阶段要加入 manifest 概念，即使 block 还在内存中。Manifest 记录哪些 map task 的输出属于哪个 partition。Reduce 只读 manifest 中有效 block。这样第四阶段加入失败重试时，不需要重写架构。

第四阶段：故障和重试 第四阶段加入 attempt、request id 和故障注入。Map task 可能失败重试，shuffle message 可能重复，commit response 可能丢失。Driver 只接受一个 attempt 提交，manifest 去重。Reduce 端不能重复合并同一逻辑 map task。这个阶段验证 exactly-once 外部效果。

测试要使用确定性故障。比如 task 3 的第一次 attempt 在写完 block 后 commit response 丢失；第二次 attempt 重试；最终 manifest 只能有一个 task 3 输出。再比如 reduce 队列第一次返回 busy，map 端退避后重试。每个故障都有明确预期。

第五阶段：Checkpoint 第五阶段加入 checkpoint 和恢复。Driver 在 stage 边界写 checkpoint，包含 task 状态和 manifest。测试中强制 driver 在 checkpoint 后重启，恢复后继续运行。Running task 回到 pending，committed task 保持 committed，临时文件清理，重复 commit 被拒绝。

Checkpoint 阶段要非常保守。不要同时引入复杂压缩和真实网络。先让恢复语义正确，再优化写入成本。恢复测试应成为常规测试，不是偶尔手工演练。

第六阶段：热点和优化 只有在正确性、重试和恢复稳定后，才加入热点 key 拆分、压缩、拓扑调度、work stealing 和多进程传输。此时每个优化都有 reference 和故障测试保护。热点 key 拆分要保证最终合并回原 key；压缩要保证 checksum 覆盖压缩数据或解压后数据；多进程传输要保证消息 framing 和版本。

优化阶段要保留开关。能关闭热点拆分回到普通 hash partition，能关闭压缩，能关闭 speculation。开关让对照实验和故障定位更容易。生产系统也常需要 feature flag 控制风险。

接口示例：MapTask MapTask 输入是 InputSplit 和作业配置，输出是若干 ShuffleBlock。它不直接写最终结果。它可以返回 MapTaskResult，包含 attempt id、block 列表、记录数、错误数和指标。Driver 决定是否提交这个结果。

```

1 struct MapTaskResult {
2     TaskId task_id;
3     AttemptId attempt_id;
4     std::vector<ShuffleBlock> blocks;
5     std::uint64_t records_read = 0;
6     std::uint64_t records_rejected = 0;
7 };

```

这个接口把执行结果和提交分开。Worker 产生 result，不代表 result 生效。Driver commit 才改变控制面状态。这个分离是分布式计算正确性的核心。

接口示例：ReduceTask ReduceTask 输入是 partition id 和 manifest 中属于该 partition 的 block 列表，输出是临时 OutputBlock。Reduce 不应扫描所有 map 输出，也不应接受 manifest 之外的 block。这样 repeated attempt 和临时文件不会污染最终结果。

ReduceTask 还要处理输入 block 缺失或损坏。若 block 缺失，返回 retryable 或 corrupt，让 driver 重新调度对应 map task 或失败作业。若直接跳过，最终计数会少。错误处理必须进入输出语义。

指标示例 项目指标可以分五组。Input：读取字节、解析记录、错误行。Map：本地聚合 key 数、输出 block 数、map 耗时。Shuffle：发送字节、队列等待、busy 次数、重复消息。Reduce：输入 block 数、合并 key 数、热点 key 时间、输出字节。Control：attempt 数、重试数、commit 延迟、checkpoint 耗时、恢复耗时。

每组指标都要带 job、stage、task、partition 或 worker 维度。全局总数用于概览，维度指标用于定位。指标命名稳定后，实验报告和图表才能复用。

作业失败策略 不是所有错误都无限重试。每个 task 应有最大 attempt 数；每个 job 应有总重试预算；某些错误如格式不兼容、输出路径权限错误应立即失败；某些错误如 reduce busy 可退避重试；某些错误如 block checksum mismatch 可能重跑 map。失败策略要写成表，而不是散落在 if 语句里。

作业失败后要清理资源：停止提交新任务，取消 pending task，等待 running task 到安全点，保留或删除临时文件，写失败报告。失败报告应包含错误分类、失败 task、attempt 历史和可恢复建议。一个只返回 false 的作业系统很难调试。

实验报告中的图 本书不能在 EPUB 中依赖图片，但报告可以用文本图、CSV 和外部脚本生成图。关键图包括线程扩展曲线、队列长度时间线、每 partition 输入大小、每 worker 任务数、重试次数、热点 key 分布、checkpoint 耗时。即使最终书里用文字描述，项目输出应保留数据，方便读者自己画。

图的价值是发现形状。一个 partition 明显比其他大，说明倾斜；队列长度持续上升，说明背压不足；worker 任务数差异大，说明调度不均；checkpoint 耗时周期性尖峰，说明持久化影响。数字表和图要互相印证。

测试分层 项目测试应分四层。第一层是纯函数单元测试，例如 parse line、hash partition、top-k merge、checksum。第二层是并发结构测试，例如 bounded queue、thread pool、SPSC ring。第三层是 stage 集成测试，例如 map stage、shuffle stage、reduce stage。第四层是端到端故障测试，例如 driver restart、duplicate commit、hot key split。分层测试能快速定位问题。

每层测试的速度和覆盖不同。纯函数测试应该很多且快；并发结构测试需要重复和 sanitizer；stage 测试需要临时文件和较小输入；端到端故障测试较慢但覆盖协议。不要只写端到端测试，失败时很难定位；也不要只写单元测试，协议组合 bug 会漏掉。

属性测试 日志统计适合属性测试。随机生成一批记录，reference 和分布式模拟结果应相同；打乱输入顺序后，如果语义是按 key 求和，结果应相同；把输入分成不同 split，结果应相同；重复发送同一个 committed block，结果不应改变；删除一个未提交临时 block，结果不应改变；删除一个已提交 block，恢复应失败或重跑。属性测试比手写几个例子更能覆盖边界。

属性测试要记录 seed。失败时输出 seed、输入摘要和最小化后的反例。这样复杂系统 bug 才能复现。随机测试如果不能复现，只会增加噪声。

性能测试分层 性能测试也要分层。Kernel benchmark 测 map、histogram、top-k；runtime

benchmark 测队列和线程池；stage benchmark 测 map stage 和 reduce stage；end-to-end benchmark 测完整作业。每层回答不同问题。End-to-end 慢了，不知道具体瓶颈；kernel 快了，不代表系统快。分层性能数据才能指导优化。

每次优化要说明影响哪一层。比如 AoSoA 改动影响 kernel；work stealing 影响 runtime；map-side combine 影响 stage 和 end-to-end；manifest fsync 优化影响 commit 阶段。若优化目标不清，性能报告就会散。

文档和学习产物 贯穿项目还应产出文档：设计文档、状态机、故障矩阵、性能报告、源码阅读笔记和实验记录。代码只是学习的一部分。系统工程需要把设计讲清楚，让别人能审查和复现。每章结束时，项目文档应补充相应部分：数据布局章补 batch layout，锁章补队列不变量，I/O 章补背压链，分布式章补 manifest 和恢复。

这些文档不应长篇空话，而应包含具体字段、状态、返回值和测试。比如“BoundedQueue close 语义”比“队列是线程安全的”有用得多；“commit response lost 测试”比“支持故障恢复”有用得多。

最终验收标准 第二册阶段的最终项目验收可以定义为：在固定 seed 输入下，串行 reference、多线程本机模拟、带 shuffle 模拟输出一致；在注入重复消息、响应丢失和 worker 失败时输出仍一致；在热点 key 输入下能输出倾斜指标并可选启用热点拆分；在生产速度超过消费速度时队列不无限增长；checkpoint 后 driver 重启能恢复；所有结果文件或 manifest 有 checksum；性能报告包含线程扩展、队列、partition 和重试指标。

这个验收不要求商业级分布式系统，但要求语义完整、证据完整、失败路径可控。达到这个标准，读者已经具备进入第三册 CPU 推理引擎的系统基础。

学习成果清单 完成第二册项目后，读者应该能交付几类成果。第一，能写出一个正确的串行 reference，并用它校验优化版本。第二，能把数据按 batch 和列式布局组织，说明冷热字段和生命周期。第三，能实现有界队列和线程池，说明关闭和等待语义。第四，能使用 reduce、scan、partition 构造无共享写的并行内核。第五，能设计本机 shuffle、manifest 和幂等提交。第六，能写故障注入测试并解释恢复行为。第七，能用指标和 trace 分析瓶颈。

这些成果比某个单独优化技巧更重要。第三册的 AI 推理引擎会需要矩阵内核、量化数据布局、线程池调度、模型文件加载、缓存和错误处理。如果第二册的成果扎实，第三册就能把精力放在模型和算子本身。

本册收束 第二册没有直接进入 AI，是因为现代 CPU 推理离不开本册知识。一个量化模型的推理引擎需要理解 cache、SIMD、线程池、NUMA、I/O、任务图和观测性；一个可靠本地服务还需要背压、超时和恢复。先把计算系统讲清楚，再讲 AI 算子开发，学习路径会更稳。

到这里，第二册形成闭环：硬件执行模型解释单核成本，内存层级解释数据移动，多线程和同步解释共享状态，并行算法解释任务切分，I/O 和背压解释等待，分布式计算解释故障和提交，贯穿项目把它们组合成一个可运行系统。后续第三册只是在这个系统上换一种更复杂的计算负载。

项目章节总结 本章把全书原则落成一个工程：日志统计计算引擎。它从 reference 开始，用数据布局改善热路径，用线程池组织执行，用 reduce/scan/partition 构造并行内核，用 I/O 管线和背压控制等待，用 manifest 和 checkpoint 定义提交与恢复，用故障注入验证幂等。这个项目不是商业产品，却覆盖现代计算系统最重要的骨架。读者真正完成它，就不再只是知道概念，而是能把概念组合成系统。

项目决策表 推进项目时，每加一个功能都要填决策表。功能改变了哪个语义？影响哪个状态机？需要哪些新字段？失败时如何恢复？是否影响 reference？需要哪些新测试？需要哪些指标？性能瓶颈预期迁移到哪里？是否能开关回退？例如加入热点 key 拆分，就要新增 hot key 识别、subkey、二级 reduce、结果合并、重复 attempt 去重和倾斜指标。没有决策表，功能很容易堆成不可维护的复杂度。

面向第三册的过渡任务 第二册结束后，可以做一个过渡任务：把日志统计里的 batch、thread pool、benchmark、manifest 保留下来，把计算内核替换成一个简单 tensor elementwise 算子，例如 $y = a * x + b$ 。这个任务不涉及模型推理，却能验证第二册基础设施是否通用。如果基础设施只能处理日志而不能处理 tensor batch，说明抽象边界还不够好。第三册会在这个过渡上继续走向量化算子、矩阵乘、量化和本地推理引擎。

第二册项目风险清单 项目最大的风险不是性能不够，而是边界不清。输入格式边界不清，会导致解析和恢复不一致；任务 ID 边界不清，会导致去重失败；输出提交边界不清，会导致重复或丢失；队列容量边界不清，会导致 OOM；线程池关闭边界不清，会导致悬挂；checkpoint 边界不清，会导致恢复错误。每个风险都对应本册一个主题。

因此项目评审应先看风险清单，再看性能数字。一个快但无法恢复的系统，不是高性能系统；一个吞吐高但队列无界的系统，只是把故障推迟；一个无锁但没有回收协议的队列，是潜在崩溃点。质量要求高，就必须把这些边界写进设计。

第二册项目交付目录 交付时可以包含四类文件。源码目录包含模块化实现；测试目录包含单元、并发、集成和故障测试；结果目录包含 benchmark CSV 和报告；文档目录包含状态机、故障矩阵、布局报告和恢复演练。书中的文字只是指导，项目产物才是读者真正掌握的证据。

这也符合用户要求的多书仓库结构：每本书有自己的 source、labs、materials、reports 和 results。后续第三册可以复用第二册的 labs 基础设施，但输出到独立书目录，避免版本混乱。

项目审查表 项目审查可以按模块进行。Input 模块：split 是否覆盖每条记录一次，错误行如何处理。Kernel 模块：reference 是否存在，边界测试是否完整。Runtime 模块：队列是否有界，shutdown 是否唤醒所有等待者。Shuffle 模块：block 是否有 checksum，partition 是否可恢复。Driver 模块：attempt 是否幂等，manifest 是否只提交一次。Checkpoint 模块：恢复是否把 running task 变回 pending。Metrics 模块：每个指标是否有维度和成本控制。

审查表让项目保持高质量。没有审查，代码越写越多，语义却越来越不清楚。第二册的目标不是堆功能，而是让每个功能都有边界、测试和证据。

本册之后的复盘问题 完成第二册后，建议读者复盘十个问题：哪一次优化改变了瓶颈？哪一个 bug 来自生命周期不清？哪一个测试暴露了并发问题？哪一个指标最有助于定位？哪一个队列需要背压？哪一个数据结构布局收益最大？哪一个失败注入最有价值？哪一个模块最难恢复？哪一处抽象边界需要调整？哪些基础设施能复用到第三册？这些问题能把项目经验固化下来。

从实验到教材的写法 项目报告可以反过来成为教材内容。每个章节都应包含一个问题、一个朴素版本、一个瓶颈或失败、一个改进版本、一个验证方法和一个边界说明。比如队列章节：朴素无界队列会积压，改成有界队列后出现背压，验证队列长度和生产者等待，边界是业务必须选择阻塞或失败。这样的写法比直接给最终答案更适合教学。

第二册后续继续扩写时，也应保持这种写法：从问题出发，解释为什么朴素方案不够，推导改进，给出代码或状态机，最后说明测试和限制。高质量教材不是堆知识点，而是让读者跟着推理过程建立判断力。

第二册最终定位 第二册定位为现代计算系统和高性能并发的基础册。它不是 AI 册，也不是单纯操作系统册，而是把硬件、操作系统、运行时、算法、I/O 和分布式计算连接起来。它的目标读者是已经会写 C++ 和基础算法，但想把程序写成可靠、高性能、可测量系统的人。第三册 AI 推理引擎会在这个基础上继续，而不是替代它。

完整作业演练 一次完整作业演练可以这样写。准备输入：一份均匀日志、一份热点日志、一份包含错误行的日志。先运行串行 reference，保存结果摘要。再运行多 worker 本机版本，比较结果。随后开启 shuffle，记录每个 partition 的 block 数和字节数。再注入一次 commit response lost，验证 manifest 不重复。再让一个 reduce partition 慢下来，观察背压链。最后写 checkpoint，重启 driver，继续完成作业。

演练报告要把每一步的预期和实际写清。不是只说“测试通过”，而是说明哪个机制被验证：reference 验证语义，多 worker 验证并行正确性，shuffle 验证分区，commit 丢失验证幂等，慢 reduce 验证背压，driver 重启验证恢复。这样的演练才符合本书的质量要求。

项目扩展边界 第二册项目还可以扩展，但要控制边界。可以加入多进程 worker，可以加入真实 socket，可以加入压缩，可以加入更复杂 key 类型，可以加入简化 Raft 控制面。但每次扩展都要保持 reference、manifest、故障测试和性能报告。不要一次加入多项，否则问题难以定位。

扩展的判断标准是：它是否服务本书主线，是否能被测试，是否能被解释。如果只是为了功能看起来多，而没有语义和证据，就不应加入教材主体。高质量教材宁愿少而深，也不要多而散。

最终章的回归阅读应审一个毕业作业，而不是再列概念。输入 split 进入 parser，输出 `ParsedLogBatch`；map-side combine 生成 partial；partition 写出 `ShuffleBlock`；I/O 层完

成临时文件和 checksum; driver 提交 `ManifestEntry`; reduce 读取 manifest 并输出最终结果。任何一步缺少身份、所有权或校验, 系统都只能算 demo。

故障注入必须覆盖正常路径之外的状态: 坏行、慢 worker、短写、响应丢失、重复 attempt、旧 epoch、driver 重启和热点 key。每个故障都要有预期状态表, 而不是只看程序是否退出。最终报告要列出 reference checksum、阶段耗时、队列水位、重试次数、manifest 内容和未验证边界。这样第二册才真正把硬件、并发、I/O 和分布式组合成一个系统。

第三册的 CPU 推理引擎可以从这里接上, 但不能倒过来污染第二册。权重加载、tensor buffer、算子调度、KV cache 和流式输出都会复用本册接口; 本册只预留接口和边界, 不提前讲模型细节。这样的收束能保证第二册主题仍然是现代计算系统, 而第三册再专心进入 AI 模型和算子开发。

最终项目还应提供一份 runbook。第一段写如何生成固定输入和 reference; 第二段写如何运行单线程、列式、多线程、任务图、异步 I/O 和分布式模拟六个版本; 第三段写如何打开每个故障开关; 第四段写如何比较 checksum、manifest、trace 和性能报告; 第五段写发现失败时先看哪张状态表。runbook 的价值是让读者不靠记忆运行系统, 而是按步骤复现实验。

代码目录也要体现书的结构。可以把 reference 放在 `engine/reference`, 热内核放在 `engine/kernel`, 运行时放在 `engine/runtime`, I/O 放在 `engine/io`, 分布式模拟放在 `engine/distributed`, 报告和测试放在 `tests` 与 `labs`。目录不是装饰, 它让模块边界在文件系统里可见。若所有内容塞进一个大文件, 书里讲的边界就没有落地。

第二册结束时, 读者应能独立回答一个具体问题: 为什么一个本地日志统计系统需要硬件事件、数据布局、线程池、队列、I/O、消息身份和 manifest。答案不应是“因为书里讲了这些”, 而应是每个机制都处理一个真实失败: 热循环慢、地址流差、共享写冲突、worker 饥饿、写出无界、响应丢失、重复提交。能把机制和失败一一对应, 第二册才算真正完成。

最终评审还要看删减能力。若项目里某个抽象没有测试、没有 trace 字段、没有失败场景, 也没有被第三册预留接口使用, 就应删除或降级为 TODO。系统教材常见问题是越写越复杂, 最后每个名词都有类, 却没有足够语义支撑。第二册的项目要反过来: 每个接口都能说出它保护什么, 测量什么, 或者让哪个故障可恢复。

进入第三册前, 第二册还应冻结一组稳定文件格式。输入 split 描述、benchmark report、task trace、shuffle manifest、fault injection 配置和 final report 都要有简单文本或 JSON 形态。AI 推理引擎接入时, 可以新增 model artifact 和 tensor metadata, 但不要推翻已有身份、attempt、epoch 和 manifest 语义。这样三册之间不是重新开项目, 而是逐层扩展同一个计算系统。

最终项目的文档应包括“从零运行”和“事故复现”两条路径。从零运行面向第一次读者, 只求生成输入、运行 reference、运行并行版本、比较输出; 事故复现面向完成本册的读者, 要求打开坏行、慢 worker、短写、迟到响应和 driver 重启, 逐项解释状态变化。两条路径分开, 能避免新读者一上来被故障矩阵压住, 也能让进阶读者真正检查系统边界。

最终报告还要包含删减记录。经过本轮整理, 凡是只换名词的模板段落都应删除或改成具体对象; 后续维护也一样。删减记录写明删掉了什么、为什么删、用什么具体内容替代。这样书稿质量不会只靠主观感觉, 而能像代码重构一样留下理由。高质量教材不是永远增加, 而是持续把空泛内容换成可解释内容。

第三册接入时, CPU 推理引擎的大项目可以直接使用本册验收框架: reference 输出变成小模型推理结果, HotKernelReport 变成算子报告, MemoryShape 变成 tensor 和权重布局, TaskStateRecord 变成 layer 或 token 任务, CompletionRecord 变成模型加载和缓存写入, MessageEnvelope 变成请求协议。这样的延续能让读者感觉第二册不是前置负担, 而是第三册能写深的原因。

18.6 全书收束与扩展

到这里, 第二册的主线闭环了。核心流水线解释单线程为什么慢; 缓存、TLB 和 NUMA 解释数据移动; 测量章提供证据; 数据布局 and SIMD 提供单核内核; 锁、原子和无锁结构提供同步基础; 线程池和任务图组织单机并行; I/O 和背压处理等待; 分布式模型、分片复制和本章工程实践把这些原则扩展到集群。后续第三册 AI 推理引擎, 会在这个基础上讨论算子、模型、量化和 CPU 推理优化。

18.7 把全册机制组合成一个可运行计算引擎

最后一章不能只总结概念，而要把 Compute Systems Engine 组装成一个可运行、可测量、可故障注入的教学系统。目标不是追求工业框架规模，而是让读者看到一个日志统计作业如何从输入文件走到最终 manifest，并在每个边界上使用前面章节的知识：热循环分析、布局、SIMD、线程池、任务图、背压、I/O、消息身份、幂等提交和恢复。

系统的最小端到端路径包括：读取 input split，解析成 `ParsedLogBatch`，本地 combine，partition 成 shuffle block，异步写出 block，driver commit manifest，reduce 读取 manifest，合并输出，写最终结果。每一步都要有 reference 或可检查摘要。若端到端只有“跑完了”，读者无法知道哪个阶段错了，也无法定位性能瓶颈。

模块边界和数据合同

Parser 接收字节范围，输出 batch 和错误摘要。它不负责线程调度，也不提交结果。Compute kernel 接收热字段 span，输出本地 partial。Partitioner 接收 partial 或记录，输出按 partition 分组的 block。Runtime 负责任务状态和调度，不理解业务 key。I/O 层负责写临时文件和 completion，不决定业务提交。Driver 负责状态表、重试、commit 和 manifest。边界清楚，模块才能单独测试。

数据合同要写进类型。`InputSplit` 使用文件 id、begin/end offset；`TaskAttemptId` 区分逻辑 task 和物理 attempt；`ShuffleBlock` 带 partition id、attempt、bytes 和 checksum；`ManifestEntry` 表示外部可见提交。不要用裸字符串和散落的整数传递关键身份。C++ 示例中应使用位宽明确的 id 类型，避免模糊类型和魔法数字。

错误合同同样重要。Parser 可能遇到坏行；I/O 可能短写；worker 可能超时；driver 可能看到旧响应；reduce 可能发现 checksum 不匹配。每个错误要分类：可重试、不可重试、可跳过、需要人工介入。错误分类决定重试和提交，不是日志文本。

端到端 reference 和分阶段优化

第一版端到端 reference 可以完全串行：按顺序读文件、解析、计数、输出。它慢，但定义语义。第二版加入 batch 和列式热字段，只改变布局；第三版加入线程本地 partial，只改变并行 reduce；第四版加入任务图，只改变调度；第五版加入异步 I/O 和背压；第六版加入 MessageBus 故障注入和幂等提交。每一步都保留前一步对照。

这种分阶段推进防止“优化大杂烩”。若一次同时加入 SIMD、线程池、异步 I/O 和重试，结果变快或变慢都无法解释。工程教材要展示慢慢加机制的过程，因为真实系统维护也需要这种可回退路径。

每个阶段都有验收。布局阶段验收字节移动和输出一致；并行阶段验收扩展曲线和 false sharing；运行时阶段验收 task trace 和取消；I/O 阶段验收队列水位和崩溃点；分布式阶段验收重复请求、响应丢失和 driver 重启。验收项进入仓库，不能只写在书里。

性能总账和瓶颈迁移

端到端系统的瓶颈会迁移。串行 reference 慢在单线程解析；列式布局后可能慢在内存带宽；多线程后可能慢在 partition 热点；异步 I/O 后可能慢在 manifest fsync；分布式模拟后可能慢在重试和背压。最后一章要把这些迁移串成一张总账：每阶段主要成本是什么，改动把成本推到哪里，下一步证据是什么。

总账还要记录资源预算。CPU 核数、内存峰值、队列容量、in-flight bytes、临时文件数量、manifest 大小、checkpoint 频率、重试预算。一个系统不是某个函数最快，而是在资源预算内稳定完成工作。若吞吐提升伴随内存无限增长，不能算成功；若平均延迟下降但 p99 爆炸，也要解释。

项目报告可以给一个固定工作负载：若干日志文件、确定 key 分布、可注入坏行和慢 I/O。对每个版本输出同一张表：正确性 checksum、吞吐、p50/p95/p99、内存峰值、队列水位、重试数、提交数。读者用这张表能看到全册知识如何合并。

故障演练作为毕业测试

最后的毕业测试不是最大吞吐，而是故障演练。演练一：响应丢失，driver 超时重试，旧响应迟到，最终只提交一次。演练二：worker 写出 block 后崩溃，driver 重试，临时文件清理不影响 manifest。演练三：I/O 队列满，上游背压，内存峰值受控。演练四：热点 key 导致 reduce 长尾，任务运行时记录倾斜并用热点策略改善。演练五：driver 重启，从 checkpoint 恢复 task table 和 commit table。

每个演练都要有命令、输入、期望状态表、关键 trace 和恢复后输出。这样项目才像系统，不像 demo。读者完成这些演练，就已经把硬件、软件、算法、并发和分布式的主线打通。

第三册 AI 推理引擎可以建立在这个基础上。模型权重是大规模只读数据，算子是计算内核，batch 是数据布局，线程池和任务图调度算子，I/O 层加载权重，分布式思想处理分片和恢复。第二册不急上 AI，是为了把计算系统地地基打稳。最后一章要明确这条延伸线，但不把第三册内容提前混进本册。

案例走查：一次完整毕业演练

毕业演练输入 4 个日志 split，其中一个 split 含坏行，一个 worker 被设置为慢节点，MessageBus 丢弃一次响应，I/O 层让一次写出短写。系统应完成以下动作：parser 把坏行放入错误摘要但不影响其他行；driver 对慢 worker 触发 deadline 和重试；commit table 拒绝迟到响应；I/O completion 报告短写并让对应 attempt 失败；最终 manifest 只引用成功 block；reduce 输出和 reference 在定义语义下相等。

演练报告分五栏：事件时间线、状态表变化、输出文件变化、指标变化、最终判断。事件时间线让读者看到因果；状态表证明协议正确；输出文件变化证明提交边界；指标变化证明性能和背压可观察；最终判断说明哪些问题已验证，哪些只是模拟。没有这五栏，毕业项目就容易退化“运行成功”。

这个演练也是第二册和第三册的交界。第三册要做 CPU 推理引擎时，也会面对大权重加载、算子调度、batch、线程池、I/O、量化误差和端到端验收。第二册把这些系统能力训练好，第三册才能专心进入 AI 模型和算子开发。

最终项目的代码审查清单

最后一章应给出代码审查清单。数据层：每个跨阶段对象是否有稳定 id、bytes、checksum 和 owner；热字段是否避免无用搬运；span 和引用生命周期是否清楚。执行层：任务是否有状态机，ready/wait/running 是否可观测，worker 是否会阻塞等待同池任务。同步层：每个锁保护什么不变量，每个 atomic 发布什么，每个队列关闭语义是什么。

I/O 层：请求完成前 buffer 是否可复用，短写如何处理，manifest 何时可见，崩溃后如何恢复。分布式层：request、task、attempt、epoch 是否区分，超时后旧响应如何处理，commit 是否幂等，重试是否有预算，背压是否向上游传播。测量层：每个优化是否有 reference、实验矩阵、原始指标和未验证边界。

清单不是官僚流程，而是把全书知识压成可执行审查。读者以后写自己的系统，也可以拿这份清单走查。若某个模块无法回答清单问题，就说明边界还没设计清楚。

从第二册过渡到第三册的接口

第三册 CPU 推理引擎会复用第二册的许多接口。权重加载需要可靠 I/O 和内存布局；token 或 batch 调度需要任务运行时；矩阵乘、归一化、采样需要内核测量和 SIMD；量化需要明确数值语义；模型文件和缓存需要 manifest；端到端推理需要 trace 和性能报告。第二册最后要把这些接口留好，而不是把 AI 内容硬塞进本册。

可以在项目中保留 `TensorBuffer`、`KernelTask`、`ModelArtifact` 这类预留名，但不展开 AI 算子。只说明它们将来如何使用现有运行时和 I/O 层。这样读者知道第二册不是结束，而是第三册的工程地基。

最终验收是两份产物：一本书中的端到端解释，以及仓库中的可运行最小系统。书解释为什么这么设计，代码证明设计能跑，报告证明结果能测，故障演练证明边界能守住。四者缺一，第二册就不完整。

综合练习：第二册最终评审会议

最后的综合题模拟一次评审会议。每个模块负责人必须用五分钟说明自己的模块：输入输出合同、不变量、性能指标、失败路径、未验证边界。Parser 负责人说明 split 边界和错误行；Kernel 负责人说明热字段和统计语义；Runtime 负责人说明任务图和取消；I/O 负责人说明完成事件和 checkpoint；Driver 负责人说明 retry、epoch 和 commit；Measurement 负责人说明报告如何复现。

评审不接受“代码已经跑通”。每个模块都要展示至少一个反例测试。例如 Parser 展示跨 split 行，Runtime 展示 future 等待死锁反例，I/O 展示 rename 前崩溃，Driver 展示迟到响应。反例测试是系统成熟度的证据。没有反例，说明模块只覆盖 happy path。

会议最后做端到端演练。选择一个固定输入，打开响应丢失、慢 I/O、热点 key 和坏行四个开关，但一次只解释一个开关的影响。输出最终 manifest、阶段耗时、任务 trace、重试指标和 checksum。若解释不清，就回到对应章节补设计。最终章不是庆祝完成，而是用全书标准审查系统。

这个评审题也为第三册铺路。AI 推理引擎进入项目时,同样要过这些问题:权重归谁拥有, tensor buffer 何时可复用, 算子任务如何调度, 量化误差如何验收, 模型文件如何提交, 端到端延迟如何报告。第二册的最终评审清单将直接成为第三册的项目入口。

容易出错的边界: 最终系统不能只追求跑通

第一个反例是端到端跑通但没有 reference。输出看起来合理, 却可能重复计数、漏掉坏行或忽略迟到响应。第二个反例是单机跑通但没有故障演练。响应丢失、短写、driver 重启一来就崩。第三个反例是性能好但没有资源预算。内存、临时文件、队列和 in-flight 请求无限增长, 长任务会拖垮机器。

第四个反例是模块边界模糊。Parser 直接写 manifest, I/O 层决定业务提交, Runtime 理解业务 key, 这些都会让后续维护困难。第五个反例是指标不贯通。Parser 有自己的 id, driver 有另一套 id, I/O 文件名又没有 attempt, 事故时无法串联。第六个反例是第三册提前侵入。AI 推理引擎很重要, 但第二册还没把计算、并发、I/O 和分布式地基打稳时, 直接上模型会让两册都散。

最终源码索引要求读者从 main 函数追踪一条记录, 列出每个函数所在模块和状态变化。若路径中出现“顺手访问全局变量”“直接改别的模块内部状态”“没有错误返回”这类点, 就要重构边界。最终章的价值在于把全书审查标准集中应用一次。

毕业项目的最后报告应包含三份摘要: 正确性摘要列 reference 和故障演练; 性能摘要列阶段耗时和瓶颈迁移; 工程摘要列不变量、未验证边界和第三册接口。这样这本书不只是读完, 而是留下一个可继续演进的系统。

本章核心接口: 第二册能力清单

最终章把前面接口收成能力清单: 统一 ID、HotKernelReport、MemoryShape、PlacementHint、BenchmarkCase、BatchView、KernelVariant、KernelPlan、WorkerSnapshot、QueueContract、AtomicProtocolNote、ProgressAndReclaim、ParallelSemanticContract、TaskStateRecord、CompletionRecord、MessageEnvelope、OwnershipEpoch。每个接口都对应一个章节的核心能力, 也对应项目中的一个检查点。

这份清单不是为了让项目显得复杂, 而是为了防止知识散掉。若某个接口在代码里没有最小实现, 说明对应章节还停留在阅读, 没有落地。若所有接口都有最小实现和测试, 第二册就真正形成了计算系统基础, 可以承接第三册 AI 推理引擎。

第二册结束时应达到的能力

读完第二册, 读者应能做四件事。第一, 拿到一个热循环, 能从源码、编译器和硬件事件建立证据链。第二, 拿到一个多线程 pipeline, 能写出不变量、队列合同、任务状态和背压路径。第三, 拿到一个并行算法, 能从串行语义推导 reduce、scan、partition 和任务图。第四, 拿到一个本机分布式模拟, 能处理身份、超时、重试、幂等提交、epoch 和恢复。

这四项能力比记住某个 API 更重要。第三册进入 AI 推理引擎后, 矩阵乘、量化、KV cache、模型加载、batch 调度和端到端延迟都会复用它们。第二册的质量标准, 就是让读者面对第三册的大项目时, 不再被线程、缓存、I/O 和分布式边界绊住, 而能专心理解模型和算子。

把全书接口落成最小实现顺序

最终项目不应一次实现所有接口。最小顺序可以分八步。第一步, 实现串行 reference 和统一 ID。第二步, 实现 BatchView 和 MemoryShape, 让数据布局可见。第三步, 实现 HotKernelReport 和 BenchmarkCase, 让热内核能测。第四步, 实现 QueueContract 和 WorkerSnapshot, 让线程池可观察。第五步, 实现 ParallelSemanticContract 和 KernelPlan, 让并行算法有语义。第六步, 实现 TaskStateRecord, 让任务图可追踪。第七步, 实现 CompletionRecord 和有界 I/O, 让背压可见。第八步, 实现 MessageEnvelope 和 OwnershipEpoch, 让分布式模拟可恢复。

每一步都要能单独运行, 不能依赖后面步骤。这样项目即使中途暂停, 也有稳定成果。第一步只有串行程序也有价值, 因为它定义 oracle; 第三步没有线程也有价值, 因为它能解释热循环; 第五步没有分布式也有价值, 因为它能生成 shuffle-friendly block。高质量工程路线不是一口气搭大框架, 而是每个阶段都可验证。

最终章还要明确哪些内容没有放进第二册。没有完整数据库事务, 没有完整 Raft 实现, 没有 GPU, 没有 AI 算子, 没有生产级网络框架。这些不是遗漏, 而是范围控制。第二册的目标是把现代计算、多核、并发、高性能、I/O 和分布式计算地基讲清。第三册再在这个地基上实现 CPU 推理引擎。

书稿质量自查

这本书的质量不能只靠字数。自查第一项是连贯性：每章是否从具体问题推出概念，是否说明上一章结论如何进入本章。第二项是具体性：每章是否有项目对象、代码边界、实验和失败路径，而不是只列名词。第三项是可验证性：每个重要结论是否能落到测试、指标、trace 或报告。第四项是边界意识：是否说明适用范围和反例。第五项是排版可读性：章节层级是否少而清楚，topic 是否不会把正文切碎，EPUB 是否不依赖图片。

自查还包括重复句式。若多个章节出现同一套句子，只换名词，就说明内容没有真正进入本章对象。硬件章应讲指令和地址，锁章应讲不变量和等待，I/O 章应讲完成事件，分布式章应讲消息身份。共同方法可以相同，但语言和案例必须服务本章。这一点直接回应本书的写作标准：不做模板机器。

最后，所有代码示例都要遵守清晰类型和可维护性原则。C++ 示例使用位宽明确类型，热路径避免无意义拷贝，类方法示例中成员访问保持清楚，递归式算法在项目实现中优先改为显式 worklist。书稿不是生产代码库，但示例会塑造读者习惯，不能随便写。

端到端项目的验收脚本设计

最终项目需要一个统一验收脚本，而不是人工按章节逐个运行。脚本可以分成五组。第一组是 correctness：串行 reference、列式 batch、并行 partial、shuffle/reduce 输出都必须产生相同 checksum 或符合误差合同。第二组是 concurrency：bounded queue 关闭、线程池 wait、任务图取消、SPSC ring 单调序号都要通过压力测试。第三组是 performance smoke：小规模运行 HotKernelReport 和 BenchmarkCase，确认没有明显退化，不追求固定绝对数字。第四组是 I/O recovery：短写、rename 前崩溃、manifest 后恢复。第五组是 distributed fault：丢响应、重复请求、旧 epoch、driver 重启。

每组测试都要有清楚的输入和预期。性能 smoke 不应因为不同设备速度不同而失败，但可以检查 checksum、阶段是否输出、计数是否非零、队列水位是否受控。故障测试不应依赖随机，而使用固定规则文件。脚本输出一份摘要，列出通过、跳过和失败。跳过必须有原因，例如当前设备没有 PMU 权限或没有 NUMA。这样用户在手机 Termux 上也能得到诚实结果。

验收脚本还要保护书稿示例。书中给出的接口名、状态名和字段名应在项目里有最小对应，否则书和代码会逐渐分离。每次修改代码，都要检查书中核心概念是否仍然准确。反过来，书稿新增机制，也要在项目里留下最小测试或 TODO。教材和项目互相校验，质量才不会只停留在排版上。

为什么第二册不直接进入 AI

第二册没有直接讲 AI，不是因为 AI 不重要，而是因为 AI 推理引擎本身就是计算系统。一个 CPU 推理引擎要加载模型文件，管理权重内存，选择数据布局，执行矩阵乘和归一化，处理量化误差，调度 token 或 batch，控制线程池，复用 KV cache，记录端到端延迟，并在失败时给出可诊断状态。如果读者还没有掌握缓存、线程、队列、I/O 和测量，直接写推理引擎只会把所有问题混在一起。

第三册会把第二册接口重新使用。ModelArtifact 类似 manifest entry，TensorBuffer 需要 MemoryShape，算子实现需要 HotKernelReport 和 KernelVariant，推理 pipeline 需要 KernelPlan 和 TaskStateRecord，请求队列需要 QueueContract 和 FlowControlWindow，模型加载需要 CompletionRecord，服务化推理需要 MessageEnvelope 和 OwnershipEpoch 的思想。第二册的每个接口都不是临时教学对象，而是第三册的地基。

因此第二册结束时，读者应对“计算”有完整理解：数据如何移动，指令如何执行，线程如何等待，算法如何拆分，I/O 如何完成，远端消息如何提交。AI 只是下一组 workload，不是跳过系统基础的理由。这个定位能让三册形成连续路线：第一册讲程序如何在机器上运行，第二册讲计算如何扩展和可靠完成，第三册讲 AI 推理如何建立在这些能力之上。

书稿交付前的人工复查项

交付前应人工复查五类问题。第一，目录和层级：目录只收章，正文小节有编号但不过度切碎，topic 不再形成满屏小标题。第二，LaTeX 转义：正文不应出现未处理的花括号、下划线、命令名和环境名这类原始排版符号，代码里的符号只在代码块中出现。第三，代码风格：C++ 示例使用位宽明确类型，关键成员访问清楚，避免把递归作为项目实现主线。第四，EPUB：不依赖图片解释核心内容，图示尽量用文本图和表述，避免微信读书把图当成奇怪图片。第五，重复：扫描明显模板句，发现后改成本章自己的对象和案例。

人工复查不是和自动检查重复。自动检查能发现输入文件、构建错误、字数和部分符号；人工复查能发现逻辑散、案例空、衔接断、段落像模板。高质量教材需要两种检查都做。尤其这本书经过

多轮扩写，更要防止旧的机械段落残留。复查时如果看到一句话可以把章名换掉仍然成立，就要怀疑它太空，应该改成具体机制、对象、实验或失败路径。

最终交付物包括 PDF、EPUB、手机目录中的副本、正文计数、构建日志检查、EPUB 无图片检查、git commit 和 push。交付信息要简洁，但证据要真实。这样用户拿到的不只是一个文件，而是一套可继续迭代的书稿工程。

第三册前置项目：CPU 推理引擎的系统边界

第三册的贯穿项目会是 CPU 推理引擎，但第二册最后应先说明它将如何接入当前系统。模型文件不是普通输入文件，它包含权重、tokenizer、配置、量化参数和版本。加载模型时要用 I/O 章的可靠读取和 CompletionRecord；权重进入内存后要用 MemoryShape 描述字节规模、对齐、页数和冷热区域；权重发布给 worker 前要用原子章的发布协议或更高层只读对象生命周期；多个请求共享权重时，要避免重复加载和悬垂释放。

算子不是孤立函数，它是 KernelVariant 和 HotKernelReport 的自然延伸。矩阵乘、layer norm、softmax、采样、量化/反量化都需要 scalar reference、误差合同、批量接口和平台特化。第二册的 SIMD 章没有讲 AI，但它训练了“先证明批量语义，再选择指令”的方法；第三册只是在更复杂的张量形状上重复这条路。算子调度也不是新问题，它会复用 TaskStateRecord、WorkerSnapshot 和背压窗口，区别是任务粒度变成 token、batch、layer 或 tile。

KV cache 会把数据布局、生命周期和并发重新组合。它是跨 token 持续增长的状态，既要高效访问，又要按请求隔离；既要避免频繁分配，又要在请求结束后释放；如果支持多请求 batch，还要处理不同序列长度和 padding。第二册的 BatchView、MemoryShape、QueueContract 和 ProgressAndReclaim 都会成为设计材料。没有这些基础，KV cache 很容易写成巨大 vector 和一堆特殊 if。

推理服务还会用到分布式思想。即使只在本机跑，request id、deadline、取消、重试、背压和 trace 仍然重要。用户取消请求时，已经排队的 token task 如何取消；模型加载失败时，错误如何传播；batch 太大时，系统是排队、拒绝还是降级；输出 token 流式返回时，哪一步算提交。这些问题都不是 AI 特有，而是第二册已经训练过的系统问题。

因此第二册结束时，可以在项目目录中预留 `engine/inference`，但只放接口草案，不写第三册正文。接口草案包括 `ModelArtifact`、`TensorBuffer`、`KernelVariant`、`InferenceRequest`、`TokenTask` 和 `InferenceTrace`。每个接口旁边标注它复用第二册哪个能力。这样第三册开篇就能从系统地基出发，而不是重新搭工程骨架。

书稿维护路线：以后如何继续扩展而不散

这本书后续还会继续扩展，必须提前规定维护路线。新增章节或大节时，先写它在全书链路中的位置：它处理数据移动、执行调度、同步等待、I/O 完成、分布式提交还是 AI 算子语义。找不到位置，就不要直接加入正文。新增内容必须带一个项目对象、一个反例、一个实验或一个源码阅读入口，至少满足其中两项。这样可以防止书稿变成资料堆。

修改已有章节时，先检查是否重复第一册内容。第二册可以引用第一册的 ABI、虚拟地址、进程和汇编，但不能大段重讲；它要把这些知识用在多核、性能、并发和分布式场景。若某段文字只是在复述第一册，应删短或改成“在第二册中它怎样改变性能和可靠性”。这能保持两册边界清楚。

维护还要保护排版。目录只保留章级，正文用少量编号小节组织大块内容，topic 只作为段内提示。新增图示优先用文本图，除非 PDF 中确有必要；EPUB 中避免图片和复杂表格。代码块保持 Maple 字体、固定宽度类型和可读缩进。每次生成 PDF 和 EPUB 后，都要抽查目录、章节标题、代码换行和手机阅读效果。

最后，维护要保护字数统计口径。正文计数只看 chapters，代码不算，附录不算。若为了过目标把内容塞到附录或代码注释里，就是偏离质量要求。正文扩展必须增加原理、案例、实验、源码阅读或项目设计中的真实内容。当前第二册超过 60 万目标后，也不能因此放松复查；宁可删掉空话再补具体内容，也不要保留看似有字数但没有教学价值的段落。

读者完成本册后的自测项目

读者可以用一个自测项目检验第二册。项目输入是一组本地日志文件，输出是按 event type 和用户桶聚合的结果。要求先写串行 reference，再实现列式 batch，再实现线程本地 partial，再实现并行 filter 和 partition，再实现任务图调度，再实现有界异步写出，最后实现本机 MessageBus 和故障注入。每一步都必须保留上一版对照。

自测项目有明确通过标准。功能上，所有版本输出与 reference 一致；性能上，报告能解释主要

瓶颈和迁移；并发上，队列关闭、任务取消和异常收束有测试；I/O 上，短写和崩溃点能恢复；分布式上，响应丢失、重复请求和旧 epoch 不会重复提交。若某一步做不到，不要继续加功能，而是回到对应章节补不变量和测试。

自测项目还要求写一页复盘。复盘不写“学到了很多”，而写具体能力：哪个数据结构因为 MemoryShape 改了布局，哪个热循环因为 HotKernelReport 找到瓶颈，哪个队列因为 QueueContract 修复关闭，哪个 task 因为 TaskStateRecord 找到长尾，哪个故障因为 MessageEnvelope 和 OwnershipEpoch 被拒绝。能写出这种复盘，就说明第二册真正被消化。

这也是给第三册的入场考试。只有自测项目跑通并能解释，才适合进入 CPU 推理引擎。否则 AI 项目中的每个问题都会被缓存、线程、I/O 和协议基础拖住。第二册的目的不是延迟 AI，而是让 AI 项目站在可靠计算系统上。

术语、接口和章节边界的一致性

最后还要做一项经常被忽视的工作：术语一致性。一本系统教材如果前面叫 task，后面叫 job unit，再后面叫 work item，读者会以为它们三个概念。第二册规定：job 表示一次完整作业，stage 表示一组依赖边界相同的任务，task 表示逻辑工作单元，attempt 表示一次物理执行，request 表示一次消息交互，block 表示可校验的数据输出，manifest 表示已提交输出集合，epoch 表示所有权轮次。任何新增章节都要沿用这些词，除非明确解释为什么需要新词。

接口一致性同样重要。HotKernelReport 不应在某章变成 BenchmarkResult 的同义重复；QueueContract 不应在 I/O 章另起名字；CompletionRecord 和 MessageEnvelope 的共同字段应保持一致；OwnershipEpoch 不应被普通 version 字段替代。术语一致不是文字洁癖，而是让读者能把一个对象从硬件、运行时、I/O、分布式一路追踪到最终项目。如果名字频繁变化，trace 和报告也会断裂。

章节边界也要复查。第一册已经讲过进程、虚拟地址、ELF、调用约定和基础汇编，第二册引用这些知识时，应服务于多核、性能、并发或分布式问题。例如虚拟地址在第二册中主要进入 TLB、page fault、NUMA first-touch 和 mmap I/O；汇编在第二册中主要进入热循环、依赖和向量化；进程在第二册中主要进入隔离、消息和故障恢复。若某段内容只是重复第一册定义，就应该删短。

第二册内部也要避免混讲。数组局部性放在缓存和布局章，原子可见性放在内存序章，数据结构回收放在无锁章，重试和幂等放在分布式模型章。章节之间可以衔接，但不要在每章都重新讲一遍全部概念。读者需要的是一条推进线，而不是每章都从头开始。

最终书稿的人工复查可按接口追踪法进行。选择 TaskStateRecord，检查线程章、锁章、运行时章、I/O 章、分布式章是否使用同一含义；选择 ManifestEntry，检查 I/O、分布式和最终项目是否同一提交语义；选择 BatchView，检查布局、SIMD 和内核章节是否同一生命周期语义。接口追踪比随机翻页更能发现断裂。

排版上也要保持一致。章节主标题承担大结构，小节承担聚合，topic 只作为段内提示。代码块不应被过窄字体压扁，Maple 字体和语法颜色服务阅读而不是装饰。正文以黑色为主，色彩用于盒子、标题和代码强调。EPUB 中不依赖图片和复杂表格，文本图必须能独立说明。这样 PDF 和手机阅读都能保持稳定。

这项一致性工作看似不像“新增知识”，实际决定书的品质。经典教材读起来舒服，往往不是因为每页都有新术语，而是因为术语稳定、结构克制、案例递进、反例明确。第二册要达到这个标准，就必须把接口、章节和排版一起维护。

18.8 最终交付清单和后续迭代规则

第二册最终交付时，仓库里不应只留下 PDF 和 EPUB，还应留下能解释书稿质量的工程痕迹。第一类痕迹是构建痕迹：LaTeX 能生成 PDF，EPUB 能生成书本格式，手机目录中有对应副本。第二类痕迹是计数痕迹：正文计数只统计 chapters，代码、附录和 backmatter 不计入目标。第三类痕迹是格式痕迹：目录只列章，正文小节编号，topic 不进入目录也不打断正文。第四类痕迹是内容痕迹：每章都有原理、案例、实验、源码或工具入口、项目接口和反例。第五类痕迹是版本痕迹：git commit 记录本次改动，push 到远端，后续可以追溯。

后续迭代规则也要明确。若继续扩第二册，不要再用一次性脚本生成大段机械文本。每次新增内容先选定章节，写清要修补的具体缺口，再补对应案例、实验或源码阅读。新增后运行正文计数、模板句扫描、LaTeX 构建和 EPUB 检查。若为了增加字数牺牲连贯性，就应删除重写。书稿越大，越要靠规则维护，而不是靠临时热情。

第三册开始前，应冻结第二册的基础接口。冻结不是永远不改，而是任何改名、改语义、改字段

都要检查全书引用和项目代码。比如 `attempt` 不能在第三册突然表示模型推理轮次，`epoch` 不能同时表示训练轮次和所有权轮次，`manifest` 不能一会儿表示文件列表，一会儿表示内存缓存。若第三册需要新概念，就给新名字，并说明和第二册概念的关系。

读者使用本书时，也可以把交付清单变成自己的学习清单。读完一章，不是问“我看完了吗”，而是问“我能不能把本章接口写进项目，能不能跑一个最小实验，能不能解释一个反例，能不能写出未验证边界”。如果答案是否定的，就回到该章，而不是继续往后赶。系统知识不是靠阅读速度积累，而是靠可运行、可测量、可解释的对象积累。

第二册最终要达到的气质，是让读者觉得每个概念都从具体问题里长出来，每个实验都能检验一个判断，每个接口都能服务后续项目，每个反例都能防止误用。做到这一点，60 万正文计数才有意义；否则字数再多也只是堆砌。本次书稿的后续复查也应围绕这个标准进行。

下一轮人工审稿流程

下一轮审稿不要从头到尾泛读，而要按问题切片。第一轮只看主线：第一册到第二册的边界是否清楚，第二册是否从硬件执行推进到单机高性能、并发同步、并行运行时、I/O 背压和分布式计算，第三册 AI 推理引擎是否只是被预告而没有抢占第二册主题。若某段内容无法说明它在这条主线中的位置，就标记为散点内容，后续要么删除，要么改写成项目对象、失败路径或实验。

第二轮只看每章内部推导。审稿人从每章开头挑一个核心概念，追问它是不是从具体问题推出来的。例如缓存章不能只说 `cache line`，而要从地址序列和字段访问推出来；锁章不能只说 `mutex`，而要从队列不变量和等待谓词推出来；I/O 章不能只说异步，而要从提交和完成事件分离推出来；分布式章不能只说 `RPC`，而要从本地调用栈失效推出来。若概念先出现，问题后补，就要调整顺序。教材的高级感来自“问题逼出机制”，不是“机制排列成表”。

第三轮只看案例质量。每个案例都要有朴素版本、失败信号、改进版本和验收指标。朴素版本不能省，因为读者需要看到直觉方案为什么诱人；失败信号不能空，因为它决定机制是否必要；改进版本不能只给答案，因为它要解释成本迁移；验收指标不能只写耗时，因为正确性、资源和恢复也要验证。若某个案例只有最终方案，就要求补推导；若某个案例只有概念解释，没有输入和指标，就要求改成实验。

第四轮只看代码示例。C++ 示例要使用位宽明确的整数类型，避免模糊的长整型；热路径接口应优先使用 `span`、`string view` 和只读引用，避免无意义拷贝；类方法示例中成员访问要清楚，状态枚举要表达非法组合不可出现；项目实现路线中避免把递归作为主要运行时策略，任务图和遍历更适合显式 `worklist`。代码块还要检查换行、缩进和 `Maple` 字体下的可读性。示例代码不是生产库，但会塑造读者习惯，不能因为是书稿就随意。

第五轮只看可验证性。每章至少要能落到一种验证材料：`benchmark`、`trace`、状态表、故障矩阵、源码阅读笔记、恢复演练或接口审查表。硬件章节偏 `benchmark` 和反汇编，并发章节偏状态表和压力测试，I/O 章节偏崩溃点矩阵，分布式章节偏故障注入和 `commit table`，最终项目偏端到端 `runbook`。若某章没有可验证材料，就说明内容仍停在讲述层，没有进入工程层。

第六轮只看排版和手机阅读。目录只收章，避免几十个小节挤满目录；正文小节编号用于组织，不要把每个小点都升成大标题；`topic` 是段内提示，不再像小标题列表；文本图优先于图片，复杂表格能拆成列表就拆成列表；代码块宽度、字号和颜色要让手机阅读可接受。PDF 可以更丰富，EPUB 要更稳。若某个表达只能靠图片才能懂，就要补文字说明，避免微信读书解析后信息丢失。

第七轮只看重复和模板痕迹。审稿人可以随机抽取十个小节，把章名和术语遮住，判断这段话是否仍能放到其他章节。如果能无损替换，就说明它太通用，应改成本章对象。例如“机制要有边界”这种话可以作为原则出现一次，但不能每章靠它凑内容；缓存章要写 `cache line`、`TLB`、页和地址序列；任务运行时章要写 `ready count`、`worker`、`deque` 和 `continuation`；分布式章要写 `request`、`attempt`、`epoch` 和 `manifest`。具体对象越多，模板味越少。

第八轮只看删减空间。高质量书稿不是越长越好，超过目标后仍要敢删。删掉重复定义，合并相邻小节，压缩没有案例的泛泛段落，把真正有价值的内容留下。若删掉一段后章节逻辑不受影响，就说明它本来不承担结构功能。字数目标是下限，不是保留空话的理由。下一轮如果继续扩写，应先删后补，用更具体的案例和实验替换空泛说明。

最后形成一份审稿记录。每条记录写章节、问题类型、建议动作和验证方式。问题类型可以是主线断裂、概念先行、案例不足、代码风格、验证缺失、排版风险、模板重复或可删减。建议动作要具体，例如“把第十五章背压段落改成 `OutputBlock` 队列水位案例”，而不是“写详细一点”。验证方式也要具体，例如“重新运行 `text-target`、PDF 构建、EPUB 无图片检查、模板句扫描”。这样

书稿迭代就会像工程项目一样可追踪。

审稿修改的合并规则

审稿之后不能把所有建议一次性混进书稿。合并顺序应按风险排序。第一类是正确性问题，例如术语含义冲突、代码示例错误、内存序说明不成立、分布式提交语义不完整，这类必须先修。第二类是结构问题，例如章节顺序不连贯、小节过碎、目录层级不合适、案例和原理脱节，这类会影响整章阅读，也应优先处理。第三类是深度问题，例如某个原理缺少推导、某个实验缺少变量、某个源码入口没有说明，这类按章节逐个补。第四类才是文字润色和局部排版。

每次合并只处理一组同类问题。比如修分布式章的超时和幂等，就不要同时改 SIMD 代码排版；整理目录层级，就不要同时扩写 Linux 源码阅读。这样做不是为了慢，而是为了让 diff 可审查。一本大书如果每次提交同时改几十类问题，后面发现错误时很难知道是哪次改动引入的。书稿工程和代码工程一样，需要小步、可回退、可解释。

合并后要做三项局部复查。第一，读改动前后各三段，确认衔接没有断。第二，运行正文计数和模板句扫描，确认没有因为删改掉到目标线下，也没有引入机械句式。第三，如果改动包含 LaTeX 命令、代码块、下划线或反斜杠，立即构建 PDF，并抽取文本检查是否有原始符号泄漏。不要等全书改完再构建，LaTeX 错误越早发现越便宜。

若两个审稿意见冲突，要按本书目标裁决。比如一个意见希望增加更多小节标题，另一个意见希望减少标题碎片，应优先保持教材阅读流畅：正文可以有编号小节，但不要把每个知识点都升成标题。再比如一个意见希望加图片，另一个意见担心 EPUB 解析，应优先保证手机阅读稳定；能用文本图和清楚叙述解决的，就不引入图片。质量标准不是让所有建议都进入书稿，而是让书稿更连贯、更准确、更可读。

最后，任何新增内容都要经过“本章对象”检查。硬件章节新增内容必须能落到指令、地址、cache line、TLB、NUMA 或计数器；并发章节新增内容必须能落到不变量、等待、状态、内存序或回收；I/O 章节新增内容必须能落到请求、完成、buffer、队列或崩溃点；分布式章节新增内容必须能落到 message、attempt、epoch、commit 或 recovery。若一段文字找不到本章对象，它很可能只是泛泛原则，应删掉或改写。

审稿后的回归阅读

审稿修改合并后，还需要做一次回归阅读。回归阅读不是重新润色全文，而是沿三条路径抽查。第一条路径是数据路径：从 InputSplit 到 ParsedLogBatch，到 partial，到 ShuffleBlock，到 ManifestEntry，到最终输出，检查每个状态是否仍然有稳定身份、所有者和提交语义。第二条路径是执行路径：从 worker 线程到 task graph，到 I/O completion，到 driver retry，检查等待点是否清楚、取消是否收束、背压是否传播。第三条路径是证据路径：从 reference 到 benchmark，到 trace，到 fault injection，到 final report，检查每个结论是否能被验证材料支撑。

这三条路径覆盖的是书的骨架。随机读一页可能发现错别字，却不一定发现系统逻辑断裂；沿路径读，能发现前后术语是否一致、接口是否变形、某章新增内容是否破坏后文。比如 BatchView 生命周期如果在布局章改了，就要检查 SIMD、任务运行时和 I/O 章节有没有继续假设旧生命周期；MessageEnvelope 如果新增 epoch，就要检查分区复制章和最终项目是否使用同一含义。回归阅读的目标是保持整本书像一个系统，而不是一组文章。

回归阅读还要保留记录。每次发现问题，写明路径、位置、影响和修复方式。若问题只影响文字，可以局部改；若影响接口，就要检查全书引用；若影响项目代码，就要补测试。这样书稿和项目能同步演进。第二册的体量已经足够大，后续维护必须依靠这种路径化复查，否则很容易在局部修补中引入新的断裂。

逐章验收索引

最终审稿可以按逐章索引执行。第一章检查系统账本：record、task、attempt、block、manifest 是否贯通，数据面和控制面是否分开。第二章检查热循环证据：源码、反汇编、输入分布、分支和依赖是否能互相解释。第三章检查地址序列：连续、步长、随机、mmap 冷读和容器遍历是否有对照。第四章检查拓扑：false sharing、线程本地 partial、first-touch 和分层合并是否各自有实验。

第五章检查测量：reference、字节账本、计数器、统计口径和环境记录是否齐全。第六章检查布局：热字段、冷字段、arena、句柄、索引位宽和写路径是否讲清。第七章检查 SIMD：标量语义、尾部、mask、溢出、误差和 fallback 是否存在。第八章检查内核模式：map、filter、scan、partition、histogram、join、top-k 和 frontier 是否各有数据结构和失败点。若这八章只剩概念介绍，没有实验和对象，就说明基础部分还没写实。

第九章检查线程：worker snapshot、阻塞路径、亲和性、大小核和持续运行曲线是否可解释。第十章检查同步：队列谓词、关闭状态、RAII、futex 和信号量资源计数是否分清。第十一章检查原子：计数、发布、状态机、CAS 副作用和等待策略是否分类。第十二章检查无锁：SPSC ring、ABA、hazard pointer、epoch 和 RCU 是否围绕生命周期讲，而不是只展示 CAS。并发部分的验收标准，是每个共享状态都能回答谁写、谁读、什么时候可见、什么时候释放。

第十三章检查并行语义：identity、结合律、scan 输出位置、partition 稳定性、浮点误差和重做边界是否明确。第十四章检查运行时：ready count、worker deque、future 饥饿、取消、trace 和任务粒度是否形成状态机。第十五章检查 I/O：completion、buffer 生命周期、短写、崩溃点矩阵、背压闭环和错误分类是否进入类型。并行运行时部分的验收标准，是一个失败 task 或失败 write 能被追踪到明确状态，而不是只在日志里出现一行错误。

第十六章检查消息：request、attempt、deadline、版本、幂等表、重试预算和观察字段是否完整。第十七章检查状态归属：partition、routing version、OwnershipEpoch、热点拆分、副本落后和共识边界是否分清。第十八章检查毕业系统：runbook、fault injection、manifest、trace、final report 和第三册接口是否闭环。分布式部分的验收标准，是超时、迟到响应、旧 epoch 和 driver 重启都不会改变已经提交的语义。

这个索引还有一个作用：防止后续扩写重新退回模板化。新增段落如果找不到对应章节对象，就先不要写；能找到对象，也要补一个输入、一个状态、一个实验或一个失败点。比如给缓存章新增内容，就必须落到地址、页、cache line 或 TLB；给 I/O 章新增内容，就必须落到 request、buffer、completion、manifest 或崩溃点。用这种索引审稿，书稿会越来越具体，而不是越来越厚但越来越空。

交付后还应做抽样阅读。第一种抽样是顺序抽样：每一部分选一章，从章首读到章尾，检查概念是否从问题推出，案例是否从朴素方案走到改进方案，实验是否能验证判断。第二种抽样是对象抽样：选择 `BatchView`、`QueueContract`、`CompletionRecord`、`MessageEnvelope` 等接口，沿全书查它是否保持同一含义。第三种抽样是失败抽样：选择短写、迟到响应、线程池饥饿、热点 key、TLB miss 这些失败，看书中是否有现象、原因、修复和验收。三种抽样比从头泛读更容易发现结构问题。

抽样阅读时要允许删改。若一个段落读起来像任何章节都能用，就改成当前章节的具体对象；若一个案例只有最终答案，就补朴素版本和失败信号；若一个实验只有耗时，就补正确性、资源和边界；若一个接口只在书里出现，项目中没有最小对应，就补测试或降低承诺。书稿达到字数线之后，质量控制反而更重要，因为多余段落会稀释真正有教学价值的内容。

最终章还要给读者一个判断标准：学完第二册后，面对一个新的计算任务，应先画数据身份和所有权，再写串行 reference，然后建立测量基线，再决定布局、并行、同步、I/O 和分布式边界。若读者一上来就想开线程、写无锁队列或拆分节点，说明还在用工具驱动设计；若能先问输入是什么、输出如何提交、失败如何恢复、瓶颈如何证明，说明已经形成系统思维。

这套标准也约束第三册。CPU 推理引擎不是一个孤立 AI 程序，而是一组计算系统问题的组合：模型文件是大输入，tensor 是布局对象，算子是热内核，batch 是调度单位，KV cache 是生命周期和并发问题，流式 token 是提交语义，量化误差是正确性合同。第三册可以引入模型和算子细节，但不能放弃第二册建立的账本、证据、状态和恢复习惯。

因此，第二册最终交付不是把所有话说完，而是把一条可靠学习路线固定下来。遇到硬件问题，先找地址、指令和计数器；遇到并发问题，先找不变量、等待和生命周期；遇到并行算法问题，先找串行语义、输出位置和合并顺序；遇到 I/O 问题，先找完成事件、buffer 和提交点；遇到分布式问题，先找消息身份、epoch 和幂等边界。读者能按这条路线独立分析新问题，本册才真正完成了它的任务。

读者能力验收还可以换成口头答辩。第一题给一段变慢的循环，要求读者先问输入规模、地址序列、分支分布、编译选项和计数器，而不是直接建议改写代码。第二题给一个偶发卡死的线程池，要求读者画出任务状态、等待关系、队列容量和关闭路径，而不是只说加线程。第三题给一个写出后崩溃的作业，要求读者说明临时文件、校验、提交记录和恢复扫描怎样配合。第四题给一个响应丢失的远端任务，要求读者区分超时、重试、迟到响应和重复提交。若这些问题能回答清楚，说明第二册知识已经能迁移到新场景。

答辩时还要追问取舍。为什么不用更复杂的无锁队列，为什么某个阶段要材料化，为什么某个数据结构牺牲了内存换局部性，为什么某个重试预算宁可早失败，为什么某个指标只做趋势而不做绝对门槛。能回答取舍，说明读者不只是记住方案，还理解了约束。系统工程没有永远正确的技巧，

只有在给定工作负载、硬件、失败模型和维护成本下更合适的选择。

最后一项验收是让读者删掉一个自己写过的抽象。删之前要说明它原本想解决什么问题，实际有没有被测试、有没有被项目调用、有没有让状态更清楚；删之后要说明哪些代码变简单，哪些能力没有丢。这个练习看似和性能无关，却能训练工程判断。很多系统变差不是因为缺少知识，而是因为每次学习一个新概念就把它加进项目，却没有证明它真的承担责任。

本册的高质量标准最终落在这三件事上：概念从问题中长出来，代码从不变量中长出来，优化从证据中长出来。只要后续修改仍守住这三点，书稿可以继续扩展；如果某次扩写只是增加名词、增加标题、增加看似专业的段落，就应回到这一节重新审查。第二册已经足够长，后续最重要的不是继续堆字，而是保持每一段都能服务读者理解真实计算系统。

最后还要强调学习顺序。读者不要把本册当作速查手册，从某个名词跳到另一个名词；更好的方式是把每章都放回同一个作业里观察。先看一条数据怎样进入系统，再看它怎样被核心执行，再看它怎样跨缓存和页移动，再看多个线程怎样分享或避免分享状态，再看算法怎样把输出位置算出来，再看运行时怎样安排任务，再看写出怎样变成可恢复结果，最后看远端消息怎样保持同一次提交。沿着这条路线读，硬件、软件、算法和分布式不会互相割裂，而会变成同一个计算过程的不同侧面。这样的学习方式慢一些，但每一步都能落到对象、状态和实验，比背概念更可靠。

如果后续审稿还要继续扩写，优先补那些读者会真正卡住的地方：一是从朴素方案到改进方案之间缺少失败信号，二是代码示例没有说明生命周期和错误路径，三是实验只给结果没有解释变量，四是章节之间缺少承接，五是术语在不同章节含义漂移。每次只修一个具体缺口，并在修完后重新构建和抽样阅读。这样第二册会越来越像一本能带人做项目的教材，而不是越来越像资料合集。

最终读者应带走一种工作习惯：先把问题放到一个可观察的场景里，再写出最小正确版本，再设计能击穿它的压力条件，然后才引入更复杂的机制。每个机制都要回答它解决了什么失败，增加了什么成本，怎样被测试，怎样在失败后恢复。只要坚持这个顺序，面对新的硬件、新的运行时、新的数据规模或新的分布式边界，读者也能继续推导，而不是只能复述书里的例子。

这也是本册和普通资料汇编的区别。资料汇编回答“有什么”，本册要回答“为什么需要、怎样长出来、哪里会失败、怎样证明”。后续所有修改都应守住这个差别。只要一段内容不能帮助读者形成判断，就算写得再多也应重写。

判断力比名词更重要。

观测性

Compute Systems Engine 从输入 split 到最终 manifest 的毕业项目中，观测性首先表现为对象边界不清：InputSplit、ParsedLogBatch、ShuffleBlock、ManifestEntry、TaskStateRecord、CompletionRecord 和 MessageEnvelope 里至少有一个对象的身份、所有者或状态没有写明。

Map 端本地聚合

Compute Systems Engine 从输入 split 到最终 manifest 的毕业项目里，Map 端本地聚合的出发点是串行语义：输入怎样变成输出，输出位置由谁决定，合并顺序是否影响结果。

检查点的内容

Compute Systems Engine 从输入 split 到最终 manifest 的毕业项目中，检查点的内容首先表现为对象边界不清：InputSplit、ParsedLogBatch、ShuffleBlock、ManifestEntry、TaskStateRecord、CompletionRecord 和 MessageEnvelope 里至少有一个对象的身份、所有者或状态没有写明。

事故复盘：最终引擎为什么先做 reference 最终项目最容易变成“正常路径能跑”的拼装程序：读取输入、并行 map、shuffle、reduce、输出 word count。真正的事发生在故障路径。丢一个 shuffle block、Driver 在 commit 后重启、旧 attempt 晚到、worker 临时文件残留，程序仍然返回成功，却无法解释结果是否等于串行语义。没有 reference，所有性能和容错讨论都没有地基。

Reference 不是慢版本凑数，而是语义法庭。它定义输入解析、错误行处理、key 规范化、计数溢出、输出排序和 checksum。并行版本、异步版本、分布式模拟版本都要向它对齐。Manifest、checkpoint、trace、temporary block、retry budget 和 message envelope 都围绕这个 reference 服务。每次加入优化，都先说明它改变了执行顺序、数据布局还是故障恢复，不能改变什么语义。

毕业引擎的验收按阶段打开复杂度。先跑串行 reference，再跑无故障并行，再开背压，再开响应丢失，再开 Driver 重启，再开旧 attempt 迟到。每一步检查最终输出、manifest 条目、checkpoint generation、trace 事件和临时目录清理。性能报告最后写，先证明系统知道自己做了什么。这样第二册收束成一个可复查的计算系统，而不是一堆 demo。

毕业项目实验判读 最终引擎不能只展示正常运行。判读顺序是先跑串行 reference，再跑无故障并行版本，再逐个打开故障注入。每次故障后检查最终输出、manifest、checkpoint、trace 和临时文件清理。性能报告要说明瓶颈从 parser、shuffle、writer、driver 哪一段迁移。若 runbook 无法让另一个人复现，项目还没有交付完成。

这一段也服务于复习。读者合上书后，应该能凭本章的判读口径重新审查自己的代码：有没有最小正确版本，有没有能击穿旧方案的输入，有没有状态记录，有没有资源指标，有没有失败后的恢复动作。若五个问题缺两个以上，就说明实现还停在演示层，而不是工程层。

最终 runbook 的内容标准 毕业项目 runbook 要让另一个人能复现同一结果。它应包含构建命令、输入生成、串行 reference、无故障运行、每个故障注入开关、预期输出、检查 manifest 的命令、清理临时文件的方式和性能报告位置。没有 runbook，项目只是作者机器上的演示。

Runbook 还要写失败时怎么判断。比如 shuffle block 缺失时，应该看到哪个 task 回到 pending；checkpoint 损坏时，driver 应从哪个 generation 恢复；旧 attempt 晚到时，manifest 是否拒绝；慢 writer 时，哪个队列水位上升。每个判断都要对应 trace 字段，而不是靠人工读日志猜。

给第三册留下接口时，也要写在 runbook 里。第二册只预留 tensor batch、kernel backend、inference request 和 trace 位置，不展开模型原理。这样第三册可以接入 CPU 推理引擎，同时复用第二册的调度、测量、I/O 和恢复框架。第二册的收束才不会和第三册抢内容。

实验课：最终引擎的端到端故障矩阵 毕业项目先跑串行 reference，再跑无故障并行版本，最后逐个打开故障注入。故障矩阵至少包含：跨 split 行、parse error、map worker 崩溃、shuffle block 截断、写出短写、manifest 写失败、响应丢失、旧 attempt 晚到、checkpoint 损坏、driver 重启。每个故障都要写预期状态，而不是只写应该成功。

端到端验收不只看最终计数。还要看 manifest 中有效 block 数、临时文件清理、task table 状态、checkpoint generation、trace 中的 retry 和 ignored response。若最终输出正确但临时文件无限增长，系统仍不成熟；若故障后需要人工猜测恢复点，runbook 也不合格。

性能报告和故障报告要同源。无故障时记录 parser、map、shuffle、reduce、writer、commit 各阶段时间；故障时记录额外重试、等待和恢复时间。这样读者能看到可靠性成本在哪里，而不是把恢复当作免费能力。某些场景下更频繁 checkpoint 会让恢复更快但正常路径更慢，报告要明确这种取舍。

第三册接口只做预留。把 TensorBuffer、KernelVariant、InferenceRequest、InferenceTrace 写成草案，说明它们复用第二册的 batch、backend dispatch、任务图和 trace。不要在第二册末尾突然讲模型结构；第二册的任务是交付计算系统地基，让第三册可以自然接上。

毕业项目练习验收重点 合格答案是一份可执行 runbook，而不是项目介绍。它要包含 reference、输入、构建命令、故障矩阵、manifest 检查、checkpoint 恢复、trace 摘要和性能报告。每个模块至少有一个反例测试。第三册接口只能预留，不应把 AI 正文提前塞进第二册。

答案还要写“为什么不采用更复杂方案”。例如没有证据证明队列是瓶颈，就不应把锁队列换成无锁；没有证明输出顺序可变，就不应随意重排；没有证明错误路径能恢复，就不应只提交正常路径。能解释不做什么，说明读者开始具备工程判断。

最后，答案必须包含可复查材料。至少有输入摘要、关键状态、期望输出、实际输出、运行命令和未验证边界。没有这些材料，老师或同伴无法判断结论是否成立。第二册的作业不是让读者写漂亮段落，而是训练读者把系统问题变成可验证证据。

项目实现：分布式计算工程实践 本章对应的贯穿项目实现不要求一次写成完整框架，但必须有最小可运行切片。这个切片包含三个部分：一个 reference，一个带本章机制的实现，一个能击穿 reference 之外隐含假设的测试。Reference 用来固定语义，机制实现用来展示本章知识，测试用来证明新增复杂度确实解决了问题。

代码组织上，不要把所有内容放进一个大文件。基础类型、输入生成、核心机制、报告输出和测试应分开。即使当前只是教学项目，也要让目录能表达责任边界。这样后续章节接入时，不会因为找不到状态归属而重新发明一套对象。

每个实现都要输出一段机器可读摘要。摘要至少包含输入规模、关键状态、错误数、输出校验、耗时和未验证边界。这里的机器可读不要求复杂格式，简单的 key value 文本也可以。关键是让脚本和读者都能复查，而不是只能看屏幕截图。

测试应包含正常路径、边界路径和失败路径。正常路径证明功能可用；边界路径证明长度、容

量、尾部、空输入、关闭或超时没有遗漏；失败路径证明本章机制不是装饰。若失败路径写不出来，往往说明本章概念还没有落到工程对象上。

报告最后写一个取舍段。新增机制带来了什么好处，增加了什么成本，哪些场景不应该使用它，下一章会复用哪个字段或对象。这个取舍段非常重要，因为系统工程不是把所有高级机制都加进去，而是在证据和约束下选择合适的边界。

调试笔记：毕业项目失败先查控制面 最终引擎失败时，不要先改热内核。先查控制面：input manifest 是否完整，task table 是否有重复有效 attempt，shuffle manifest 是否只引用校验通过的 block，checkpoint generation 是否单调，driver 重启后 running 任务如何处理。控制面解释不了，数据面再快也无法保证结果可信。

复查清单：分布式计算工程实践 复查本章时，先检查 reference 是否仍然存在。没有 reference，后面的并发、优化、异步或分布式改造都无法证明语义没有变。再检查压力输入是否尖锐：它应当攻击本章最核心的边界，而不是只把数据量放大。第三，检查状态是否可观察：关键对象的身份、状态、错误和提交结果应能在报告中找到。

第四，检查代码示例是否说明禁止行为。调用者不能做什么，往往比能做什么更能体现工程质量。第五，检查实验是否写出未验证边界；当前设备、权限或输入无法证明的结论，必须标注。第六，检查本章对象是否能被下一章复用，不能复用就说明边界还不稳。

完成这六项后，再看文字是否连贯。每个概念都应从问题推出，每个案例都应从朴素方案走到改进方案，每个实验都应能解释一个判断。若某段话放到其他章节也成立，就把它改成本章的具体对象、具体输入和具体失败。

18.9 毕业项目：实现一个可恢复的本机分布式计算引擎

第二册的最终项目不是做一个庞大框架，而是把全书概念收束成一个可运行、可测量、可恢复的小系统。它在一台 Linux 机器上模拟 driver、worker、message bus、shuffle storage 和 manifest，用本机进程或线程传消息，但语义按分布式系统写：消息有身份，attempt 可重复，输出通过 manifest 提交，故障可以注入，恢复可以演练。

一、项目目标和非目标 目标是处理一批日志，按 key 聚合计数，输出确定结果。系统要支持输入切分、map 本地聚合、shuffle block 写出、reduce 合并、manifest 提交、失败重试和 checkpoint。非目标是高吞吐商业框架、完整网络协议、复杂 SQL、自动资源管理。范围小不是降低质量，而是让每个语义都能被读者看见和测试。

项目必须保留串行 reference。任何并行或分布式输出都先和 reference 比较。若 key 顺序不同，就按合同排序后比较；若错误处理不同，就先统一一错误合同。没有 reference，毕业项目会变成“看起来能跑”的 demo，而不是教材级工程。

二、模块边界 Driver 负责 job 状态、task 切分、attempt 管理、重试预算和 manifest commit。Worker 负责执行 map 或 reduce，不直接决定全局提交。MessageBus 负责传递消息、注入延迟、重复、丢失和乱序。ShuffleStore 负责写临时 block、校验、列出已提交 block。Manifest 负责定义哪些输出对 reduce 可见。

这些边界不能混。Worker 可以写临时文件，但不能把自己写出的文件直接变成全局有效；MessageBus 可以丢消息，但不能修改 payload 语义；ShuffleStore 可以报告写失败，但不能替 driver 决定重试。每个模块只承担自己的责任，故障时才能定位。

三、数据格式从热路径和恢复共同决定 输入 split 保存文件名、offset、length 和 split id。Map 输出 block 保存 partition id、attempt id、record count、checksum 和 payload。Reduce 输出保存 key、count 和 deterministic order。格式要既适合顺序读写，也适合恢复校验。不要把调试文本塞进热 payload；调试信息通过 manifest 和 trace 关联。

Block 文件可以先用简单二进制或文本格式，但必须有 header。Header 至少包含 magic、version、partition id、attempt id、payload size 和 checksum。版本字段让后续格式变化可检测；checksum 让 reduce 不只相信文件存在；attempt id 让重复输出可识别。格式小，也要有工程边界。

四、Map 阶段先本地 combine Map worker 读取 split，解析记录，生成本地 `unordered_map` 或排序数组，再按 partition 写 block。不要每条记录都发消息给 reduce。Map-side combine 减少 shuffle 字节，也降低热点 key 压力。这里复用了第 13 章 reduce 和 partition 的思想：局部聚合，分区输出，manifest 提交。

本地 combine 的数据结构要按 key 基数选择。key 少时数组桶更快；key 多时哈希表更通用；key 极端倾斜时热点 key 可以单独计数。项目第一版可以用哈希表，但报告要记录 key 数量、最大 bucket、输出 block 大小和 combine ratio。这样后续优化有依据。

五、Reduce 阶段只读 manifest Reduce worker 不扫描临时目录，只读取 manifest 中属于自己 partition 的 block。它校验 header 和 checksum，合并 key，输出最终结果。若某个 block 文件存在但未提交，reduce 必须忽略；若 manifest 条目指向的 block 校验失败，reduce 报告输入损坏，driver 决定重试或失败。

这个规则看似严格，却是恢复语义的核心。目录扫描会把未提交、旧 attempt、部分写入文件混进结果；manifest 把“写过”与“有效”分开。第 15 章 I/O、第 16 章幂等、第 17 章控制状态都在这里汇合。

六、Checkpoint 记录控制面，不复制所有数据 Checkpoint 至少记录 job id、stage 状态、已完成 task、已提交 manifest、route epoch、重试预算和未完成 attempt。大块 shuffle 数据不必全部写进 checkpoint，只要它们有稳定文件、checksum 和 manifest 引用。这样 checkpoint 小，恢复快，也符合控制面和数据面分离。

恢复时，driver 读取 checkpoint 和 manifest，重建 task 状态。已提交 block 不重复计算；未提交或校验失败的 attempt 重新调度；迟到响应根据 attempt id 和 manifest 状态忽略。恢复流程要写成测试，而不是只写文档。杀进程、删除临时文件、重复投递响应，都应在故障矩阵里出现。

七、故障矩阵 毕业项目至少覆盖八类故障。第一，map 请求丢失。第二，map 响应丢失但 block 已写出。第三，worker 写出短写。第四，driver 在 manifest commit 前崩溃。第五，driver 在 commit 后响应前崩溃。第六，旧 attempt 迟到。第七，reduce 读取 block 时 checksum 失败。第八，热点 partition 导致长尾。每类故障都要写期望状态和验收指标。

故障矩阵让项目不像玩具。很多系统 bug 不是 happy path 不会跑，而是某个边界状态没人定义。矩阵中的每一行都迫使读者回答：哪个对象拥有状态，哪个结果已经提交，哪个动作可以重试，哪个临时产物要清理。

八、性能报告和恢复报告分开 毕业项目要有两份报告。性能报告记录输入规模、key 分布、线程数、block size、map 时间、shuffle 写时间、reduce 时间、manifest 时间、队列水位和硬件证据。恢复报告记录故障注入、重试次数、重复响应、忽略的旧 attempt、恢复耗时和最终 reference 比较。两份报告都重要，不能用吞吐掩盖恢复漏洞，也不能用可靠性忽略明显瓶颈。

报告中还要写未验证边界。例如没有真实多机网络，没有磁盘断电测试，没有跨 NUMA 设备，没有真实权限读取某些 PMU。诚实边界不会降低项目质量，反而说明读者知道系统结论依赖环境。

九、代码组织建议 项目可以按模块组织：`engine/driver`、`engine/worker`、`engine/message`、`engine/shuffle`、`engine/runtime`、`engine/tests`。公共类型放在 `engine/model`，例如 `JobId`、`TaskId`、`AttemptId`、`PartitionId`、`MessageEnvelope`、`ManifestEntry`。不要让所有东西放在一个大文件里，也不要让 worker 直接 include driver 内部状态。

C++ 接口要表达所有权。消息 payload 使用值或引用计数块；I/O block 提交后移动所有权；manifest entry 是不可变记录；driver 状态由单线程事件循环或明确锁保护；worker 不持有 driver 指针，只通过 MessageBus 通信。这样的组织能把第二册的并发和分布式原则落到代码结构。

十、最终验收 最终验收分五步。第一，串行 reference 通过全部输入。第二，无故障分布式模拟输出与 reference 一致。第三，故障矩阵全部通过，且没有读取未提交 block。第四，性能报告能解释主瓶颈和一次优化前后成本迁移。第五，代码审查能列出每个共享状态、每个原子、每个队列和每个持久提交点的合同。

完成这五步，第二册的目标就达到了：读者不是背了一堆硬件、并发和分布式概念，而是能把它们组合成一个可以运行、可以测量、可以恢复的计算系统。第三册进入 AI 推理引擎时，这个项目会变成基础设施：模型权重是输入数据，算子是计算内核，线程池是运行时，KV cache 和中间张量是数据布局，模型版本和输出提交仍然需要 manifest 和恢复语义。

18.10 最终复盘：第二册怎样为第三册 AI 推理引擎铺路

第二册没有直接进入 AI，是有意安排。AI 推理引擎不是只会调用模型文件和矩阵乘。它需要模型权重加载、内存布局、算子调度、线程池、量化数据格式、缓存、I/O、版本管理和恢复。第二册的 Compute Systems Engine 正是这些能力的通用底座。

一、权重文件对应输入 split 和 manifest 模型权重不是普通数组。它有文件格式、tensor 名称、shape、dtype、量化参数、校验和版本。加载时需要 mmap 或 read，需要 page cache，需要校验，需要 manifest 说明哪些 tensor 属于同一模型版本。这和日志输入 split、shuffle block 和 manifest 是同一类问题：大数据可校验，控制面定义有效版本。

二、算子对应计算内核 矩阵乘、卷积、归一化、激活、采样都可以用第二册的内核分析方法看：数据流是什么，热字段是什么，是否可向量化，是否受内存带宽限制，输出位置是否固定，线程如何切分。量化算子还要处理整数位宽、溢出、缩放和误差合同。第二册的 SIMD、布局和测量章节会直接复用。

三、推理请求对应任务图 一个推理请求不是单个函数，而是一组算子依赖。某些算子顺序严格，某些预处理和后处理可以并行，batching 会改变延迟和吞吐，KV cache 会引入生命周期和并发访问。任务运行时，背压和线程池配置都要再次使用。若第二册没有讲清任务状态机，第三册的推理调度会很散。

四、模型版本对应配置热更新 在线替换模型和第 11 章配置热更新相似。新模型加载完成后发布，正在使用旧模型的请求必须继续安全完成，旧权重何时释放需要生命周期协议。shared pointer、epoch 或 RCU 的选择会再次出现。模型版本号还要进入日志和输出，便于复现。

五、第三册项目接口 第三册 CPU 推理引擎可以复用第二册的对象：BenchmarkCase、MemoryShape、HotKernelReport、QueueContract、TaskStateRecord、MessageEnvelope、ManifestEntry。不是复制代码，而是复用思维框架。这样 AI 算子开发不会变成孤立专题，而是现代计算系统的延伸。

18.11 最终补强：项目评分标准

毕业项目需要清楚评分标准，否则读者容易只追求能跑。评分不应只看代码量或吞吐，而应看语义、证据、恢复和可维护性。下面给出一套面向本书的验收口径。

一、语义完整性 输入合同、错误合同、输出排序、重复执行、提交点和恢复边界都写清。Reference 覆盖正常和边界输入。任何优化版本都能解释自己保留了哪些语义，放松了哪些语义。若某个模块只能说“通常不会出错”，语义不合格。

二、工程结构 Driver、worker、message bus、runtime、shuffle store、manifest 分工清楚。共享状态有锁或原子合同，队列有关闭语义，I/O buffer 有所有权，任务有状态机。没有万能 manager，没有所有逻辑堆在一个文件里。结构清楚不是为了好看，而是为了失败时能定位责任。

三、测试和故障注入 测试覆盖 reference、并行输出、重复 attempt、迟到响应、短写、崩溃恢复、热点 partition 和取消。故障注入不是可选加分项，而是系统项目的核心。没有故障注入，只能证明 happy path。

四、性能证据 性能报告有阶段时间、输入规模、线程数、block size、队列水位、checksum 和环境信息。优化说明瓶颈假设、改动、结果和反证。报告承认不可验证边界。只给一张最终耗时表，不合格。

五、代码质量 C++ 类型使用位宽明确整数，所有权清楚，热路径避免不必要拷贝，状态使用枚举，错误使用结构化返回或异常合同。代码不靠全局变量串状态，不在锁内调用未知复杂逻辑，不把 relaxed 变量当协议信号。代码质量服务系统语义。

六、面向第三册的延展 项目最后写一页说明：若把日志统计换成 CPU 推理引擎，哪些模块保留，哪些模块替换。保留 driver、runtime、I/O、manifest、benchmark；替换 parser 为 model loader，替换 map reduce 为 tensor kernels，替换 shuffle 为中间张量和 KV cache 管理。能写出这页，说明第二册知识已经可以迁移。

18.12 项目扩展：从单机模拟到两进程部署

毕业项目第一版可以在一个进程里模拟 driver、worker 和 message bus。为了更接近真实系统，下一步可以拆成两个进程：driver 进程和 worker 进程，通过本机 socket 或管道传消息。这个扩展不追求网络性能，而是强迫系统放弃共享指针和调用栈，真正使用消息身份和序列化边界。

一、序列化让所有权更清楚 单进程模拟很容易偷懒，把 payload 指针直接放进消息。两进程后，消息必须序列化或通过共享文件传递。序列化会暴露类型版本、字节序、长度、校验和错误处

理。它也迫使 driver 和 worker 不共享内存对象，所有状态都通过 MessageEnvelope 和 payload 表达。

这一步能发现很多隐藏耦合。若 worker 需要访问 driver 内部表，拆进程就会失败；若消息没有 attempt id，响应无法归属；若 payload 没有长度和 checksum，worker 无法验证输入。两进程部署是结构审查工具。

二、进程崩溃更接近真实故障 单线程故障注入可以模拟响应丢失，但进程崩溃会测试更多边界：worker 写了一半 block 后退出，driver socket 断开，临时文件残留，进程重启后 request id 是否复用，旧响应不再可能到达但旧文件仍在。系统必须通过 manifest 和 checkpoint 恢复，而不是依赖内存状态。

实验可以让 worker 在写出 block 的三个点崩溃：写 header 前，写 payload 中，写完但未报告。Driver 重试后，reduce 仍只能看到一个已提交 block。这个实验直接验证 I/O 和分布式语义。

三、部署脚本也要记录环境 两进程实验需要记录端口或 socket 路径、工作目录、临时目录、日志目录、输入路径和清理策略。路径错误、旧临时文件和权限问题都会影响结果。不要把部署脚本当成附属物；它是实验可复现的一部分。

四、扩展后的验收 两进程版本通过同一 reference，同一故障矩阵，输出同一 manifest。性能可以慢一些，因为目标是语义边界。报告比较单进程和两进程：多了哪些序列化成本，少了哪些共享状态假设，哪些 bug 被暴露。这样的扩展为真正多机部署打基础，也为第三册模型服务化打基础。

18.13 补充案例：毕业项目的最小代码骨架

为了让读者能真正开始写项目，本章应给出最小代码骨架的职责说明。不是贴完整代码，而是说明每个模块最先实现哪些接口，哪些接口暂时不要做复杂。

一、model 层 model 层只放值类型：JobId、TaskId、AttemptId、PartitionId、RecordId、MessageEnvelope、ManifestEntry、BlockHeader。它不依赖 driver 或 worker。值类型使用位宽明确整数，支持比较、打印和简单校验。这个层稳定后，其他模块都围绕它通信。

二、reference 层 reference 层实现串行日志统计。它不使用线程，不使用消息，不写 shuffle，只定义语义。测试先覆盖 reference。任何后续输出都和 reference 比较。这个层慢可以接受，但必须清楚。

三、runtime 层 runtime 层先实现有界队列和固定线程池，不急着手工作窃取。接口包含 submit、shutdown drain、shutdown cancel、wait。等项目语义稳定后，再替换任务图或工作窃取。不要一开始把运行时做复杂。

四、shuffle 层 shuffle 层实现 write temp block、verify block、commit manifest、list committed blocks。Reduce 只能通过 manifest 读取。这个层是恢复语义的核心，优先写测试。

五、driver 和 worker Driver 切分 task，发送 MessageEnvelope，维护 attempt 状态和重试预算。Worker 接收 task，运行 map 或 reduce，写临时 block，返回响应。即使单进程模拟，也要通过消息接口通信，避免共享内部状态。

六、测试顺序 测试顺序是 reference，block 写读，manifest commit，单 worker map reduce，无故障多 worker，响应丢失，重复 attempt，崩溃恢复，热点 partition。按这个顺序推进，比一上来写完整系统更稳。

18.14 补充专题：最终项目文档怎么写

项目完成后，文档不是 README 几行命令。它应让别人能理解语义、运行实验、复现故障和继续扩展。文档质量也是工程质量。

一、运行文档 写清构建命令、输入生成、运行命令、输出路径、手机目录复制方式和清理命令。每个命令说明会产生哪些文件。不要让读者猜 main.pdf 或 main.epub 在哪里；项目产物同样要有 manifest 思维。

二、语义文档 写输入格式、错误策略、输出排序、重试策略、提交点、恢复边界。语义文档比接口注释更重要，因为它定义系统承诺。后续优化不能破坏这些承诺。

三、实验文档 列出 benchmark case、故障注入 case、预期结果和不可验证边界。性能数字要带环境。故障注入要说明如何触发、如何判断通过。这样别人才能复查。

四、扩展文档 说明如何把单进程模拟扩展到两进程，如何替换消息总线，如何加入真实网络，如何在第三册替换成模型推理任务。扩展文档让项目成为后续工作的地基，而不是一次性作业。

18.15 补充专题：毕业项目的演示脚本

最终项目应有一套演示脚本，让读者和审查者快速看到系统能力。演示不是替代测试，而是把核心路径串起来：生成输入、运行 reference、运行分布式模拟、注入故障、恢复、比较输出、生成报告。

一、正常路径演示 脚本生成固定 seed 的日志输入，运行 reference，运行 Compute Systems Engine，比较输出，打印阶段时间和 manifest 摘要。输出目录包含 reference result、engine result、manifest、trace 和 benchmark report。

二、故障路径演示 脚本启用响应丢失、短写或 driver 崩溃注入。运行后恢复，再比较 reference。报告列出重试次数、忽略的迟到响应、清理的临时 block 和最终 manifest。这个演示能直观看到系统不是 happy path demo。

三、性能路径演示 脚本改变 block size、线程数和 key 分布，生成三份报告。读者可以看到瓶颈如何变化。演示脚本不要隐藏命令，应该打印完整参数，方便复制和修改。

四、交付目录 最终产物目录应包含 PDF、EPUB、源码、测试报告和运行报告。手机阅读目录只放书籍产物，仓库内保留可复现材料。这样书和项目都能被继续使用。

18.16 项目实施细节：测试先于优化

毕业项目很容易被性能目标带偏。正确顺序是先测试语义，再优化热路径。没有测试矩阵的高性能版本，只是在制造更快的未知行为。

一、第一组测试只测 reference Reference 测试覆盖输入解析、错误策略、输出排序和溢出处理。它不依赖线程和消息。只要 reference 不稳，后面所有比较都没有意义。Reference 慢可以接受，但必须易读、确定、覆盖边界。

二、第二组测试测存储和 manifest 在没有 worker 和 driver 前，单独测试 block 写入、checksum、manifest commit、未提交临时文件忽略、重复 entry 拒绝。这样 I/O 和提交语义先稳定。不要等整个系统跑起来后才发现 manifest 设计不清。

三、第三组测试测消息和 attempt 使用模拟 MessageBus，发送 map request，注入响应丢失、重复和迟到。Driver 状态表应只提交一次。此时 compute 可以是很小的假任务，重点测协议。协议稳定后再接真实 map 计算。

四、第四组测试测端到端 最后才运行完整日志统计。先无故障，再有故障，再性能输入。每次都和 reference 比较。端到端失败时，根据前面分层测试定位，而不是在巨大系统里盲查。

五、优化门槛 只有当测试覆盖某个模块后，才允许优化该模块。优化必须附带 benchmark 和反证输入。比如替换哈希表、改线程池、加入 SIMD、改变 block size，都要证明 reference 不变，故障测试仍过，性能报告解释收益。这个门槛保证项目越快越可信，而不是越快越脆。

18.17 把第二册收束成一个可交付系统

第二册最后的项目不是把每章代码拼在一起，而是把硬件理解、性能测量、并发控制、并行算法、运行时、I/O 和分布式语义压成同一个可交付系统。一个 Compute Systems Engine 至少应能读取输入、切分任务、并行处理、写出结果、记录 manifest、处理失败、生成报告，并在本机模拟分布式执行。项目越复杂，越不能靠口头解释正确性，必须把状态、提交点、指标和恢复材料写进系统。

一、先交付串行 reference 毕业项目的第一步不是线程池，也不是分布式模拟，而是串行 reference。它定义输入格式、输出格式、错误处理、排序规则、数值误差和边界行为。所有优化版本都必须能和 reference 对比。没有 reference，后续每个并发 bug 都会变成争论：到底是优化错了，还是原始语义没定义。reference 可以慢，但要清楚、确定、覆盖边界。

二、再交付阶段化流水线 系统应拆成读取、解析、partition、compute、merge、write、commit 几个阶段。每个阶段写明输入、输出、所有权、失败后是否可重做、指标如何记录。阶段边界不是为了画图，而是为了让恢复和测量有位置。若 compute 成功但 write 失败，可以重写输出；若 commit

失败，要检查 manifest；若 parse 失败，要记录错误输入而不是让后续阶段猜测。阶段清楚，系统才有可维护性。

三、把性能优化绑定到证据 项目中每个优化都要有前后对照。数据布局优化应有地址或带宽证据，SIMD 优化应有反汇编或向量化报告，线程优化应有队列和空闲时间，I/O 优化应有 in-flight、buffer 和完成延迟，分布式优化应有尝试数和故障矩阵。若一个优化没有证据，只是让代码更复杂，就不应进入主线。第二册项目要训练的是工程判断，而不是堆砌机制。

四、把失败注入纳入常规测试 系统必须能处理短写、崩溃点、重复消息、响应丢失、worker 慢、任务取消、队列关闭、配置热更新和重复提交。失败注入不是附加题，而是验证状态机是否完整的核心方法。每个失败实验都要写期望状态：哪些文件可以存在，哪些 manifest 条目有效，哪些 request id 可重试，哪些 buffer 必须释放，哪些任务应取消。失败路径讲清楚，正常路径通常也会更稳。

五、为第三册保留推理引擎接口 第三册会进入 AI 和 CPU 推理引擎，但第二册已经准备了基础：数据布局、SIMD、线程池、任务图、异步 I/O、分布式切分和性能报告。最终项目不需要提前实现模型推理，却要保留可复用接口：连续 tensor buffer、算子调度、batch 输入、权重文件读取、量化数据类型、kernel 报告和本机 worker 模拟。这样第三册不是另起炉灶，而是在第二册的计算系统基础上继续推进。

18.18 最终验收：让项目像系统而不是作业

第二册项目的最终验收要比“能跑通样例”严格得多。一个系统至少要证明五件事：结果正确，资源有界，性能可解释，失败可恢复，文档能让别人接手。每一件事都要有材料，而不是口头承诺。这样收束整本书，读者才会明白前面反复讲 reference、状态表、指标、manifest、故障注入和报告的原因。

一、正确性验收 正确性材料包括串行 reference、输入生成器、边界输入、随机输入、错误输入和优化版本对比。并行版本必须说明输出顺序是否要求稳定，浮点结果是否允许误差，重复记录如何处理。若项目有分布式模拟，单机和分布式输出必须能按 manifest 对齐。正确性不是一条 assert，而是一组可复现输入和明确 oracle。

二、资源验收 资源材料包括内存峰值、队列容量、buffer pool 大小、线程数、文件描述符、临时文件数量和 in-flight 请求数。每个容量都要能解释来源。若 buffer pool 容量扩大，吞吐提升多少，内存增加多少；若队列容量缩小，背压提前多少，尾延迟如何变化。资源有界是系统能长期运行的基础。

三、性能验收 性能材料包括阶段耗时、吞吐、延迟分位数、主要瓶颈、一次优化前后对照和未验证边界。至少选择一个优化做完整论证：例如列式布局减少带宽、线程本地 partial 降低共享写、异步写出隐藏等待、重试预算降低故障放大。性能验收的重点是解释，不是堆数字。

四、恢复验收 恢复材料包括崩溃点矩阵、重复消息矩阵、关闭测试和重启扫描结果。系统应证明不会读取未提交输出，不会重复应用同一 request，不会泄漏临时文件，不会在 close 后留下后台线程。恢复验收最能区分演示程序和工程系统，因为 happy path 无法覆盖这些状态。

五、交接文档验收 最后要有一份交接文档，说明目录结构、核心对象、状态机、运行命令、测试命令、已知限制和第三册预留接口。文档不需要长篇宣传，但必须能让另一个读者拉下仓库后运行项目、复现实验、理解关键设计。系统知识的最终形态不是作者脑中的解释，而是代码、测试、报告和文档共同构成的可交接对象。

18.19 项目目录、测试矩阵和报告结构

最终项目要落到仓库结构里，否则整本书会停在文字层面。一个推荐结构可以分为 `engine/core`、`engine/runtime`、`engine/io`、`engine/distributed`、`engine/tools`、`tests` 和 `reports`。`core` 放串行语义、数据模型和 reference；`runtime` 放线程池、任务图和并行算法；`io` 放可靠写出、manifest 和恢复扫描；`distributed` 放 MessageBus、Envelope、worker 模拟和故障规则；`tools` 放基准和报告生成；`tests` 放正确性、压力和故障注入；`reports` 保存每次实验的可复盘材料。目录不是形式，它把责任边界写进文件系统。

一、核心对象要少而稳定 最终项目不需要一开始设计很多抽象。核心对象保持少而稳定更重

要：Record 表示输入记录，Chunk 表示输入切分，Task 表示可执行工作，Block 表示输出块，ManifestEntry 表示已提交结果，Envelope 表示跨 worker 消息，TraceEvent 表示可观测事件。每个对象都要有明确所有权和状态。对象少，读者容易追踪；状态清楚，后续优化不会把系统撕裂。

二、测试矩阵按风险排列 测试不要只按模块排列，也要按风险排列。第一组是 reference 正确性：小输入、空输入、错误输入、随机输入。第二组是并行等价：不同线程数、不同 chunk 大小、不同输出顺序要求。第三组是资源边界：小队列、小 buffer、低内存预算。第四组是故障注入：短写、崩溃点、重复消息、响应丢失、worker 慢、关闭。第五组是恢复：重启后扫描 manifest、删除临时文件、重做未提交任务。这样的矩阵能直接对应本书每章主题。

三、报告文件要机器可读也要人可读 实验报告最好同时输出结构化 JSON 和简洁文本。JSON 方便后续比较和脚本检查，文本方便读者阅读。报告字段包括 git commit、机器信息、编译选项、输入摘要、阶段耗时、资源峰值、错误计数、故障规则、输出 checksum 和未验证边界。若没有这些字段，半年后再看一次实验，很难知道当时的结论是否仍然可信。

四、命令行接口服务于复现 项目应提供稳定命令：生成输入、运行 reference、运行并行版本、运行分布式模拟、注入故障、恢复扫描、生成报告。每条命令都应能指定随机种子、线程数、chunk 大小、buffer 数和故障规则。这样教材中的案例就不是一次性文字，而是可以反复运行的实验。命令行不是附属工具，它是教学内容的执行入口。

五、代码审查按状态机进行 最终审查不按文件大小排序，而按状态机排序。先审查 Task 状态，确认没有重复完成；再审查 Manifest 状态，确认未提交结果不可见；再审查 Queue 状态，确认 close 唤醒所有等待者；再审查 Envelope 状态，确认 request id、attempt 和 deadline 贯穿全程；最后审查恢复状态，确认重启后能确定每个输出块的命运。这种审查方式能让读者把全书概念聚合到一个系统里。

六、性能验收要有退路 如果某个优化在用户机器上没有变快，项目仍然要能给出解释。比如 SIMD 不明显，可能输入太小或编译器已自动向量化；线程池不明显，可能 I/O 才是瓶颈；分布式模拟不明显，可能本机队列开销大于并行收益。报告要把“没有收益”也纳入结论，而不是强行宣称成功。高质量教材要教读者面对反证，而不是只展示顺利路径。

七、为第三册保存算子接口 第三册的 CPU 推理引擎会需要 tensor、operator、kernel、scheduler、weight loader 和 quantized buffer。第二册最终项目可以先定义轻量接口：连续 buffer、shape、element type、block reader、kernel report 和 task graph。即使暂时只处理日志或数值数组，也要让数据路径能承载未来的张量块。这样第三册进入 AI 时，读者不会重新学习并发、I/O 和测量，而是把已有系统能力用于算子开发。

八、最终交付以可复现为准 交付时，应能从干净目录执行一组命令，生成输入，运行 reference，运行并行版本，运行故障模拟，恢复扫描，生成 PDF 或文本报告，并比较 checksum。若某一步依赖手工解释，系统就还没有交付完成。可复现不是形式主义，而是让知识从“我懂了”变成“别人也能验证”。第二册的学习目标也在这里闭环：从硬件原理一路走到一个可测、可恢复、可解释的计算系统。

18.20 端到端案例：一次作业从输入到恢复

为了让第二册真正收束，最后用一次完整作业走通所有层次。输入是一批日志或数值记录，目标是过滤、分区、统计并输出结果。这个目标看起来普通，却足以覆盖硬件、布局、线程、同步、并行算法、I/O、消息、恢复和报告。教材不应把每章机制留在各自章节里，而要让读者看到它们在一个作业生命周期里如何互相约束。

一、输入进入系统时先建立身份 每个输入文件、每个 chunk、每条输出 block 都要有稳定身份。文件身份可以来自路径、大小、修改时间和 checksum，chunk 身份来自文件身份加偏移范围，输出 block 身份来自阶段、partition、attempt 和 checksum。身份建立以后，后续线程调度、分布式消息、manifest 提交和恢复扫描才有共同语言。没有身份，系统只能说“有个任务失败了”；有了身份，系统能说“文件 A 的第七个 chunk 在第二次 attempt 写出 block 3 时短写失败”。这就是第一章系统账本在最终项目里的具体形态。

二、读取阶段体现 I/O 和背压 读取线程不能无限把数据塞进内存。它从输入文件读 chunk，放入有界队列；队列满时，读取阶段被背压，说明下游处理速度跟不上。此时正确反应不是再开更

多读取线程，而是观察瓶颈：解析慢、partition 慢、写出慢，还是 manifest 提交慢。读取阶段还要处理短读、错误输入和取消。若作业被取消，读取线程应停止产生新 chunk，唤醒等待者，并把未处理 chunk 状态写入报告。

三、解析阶段体现布局 and 热路径 解析器先用串行 reference 定义字段、错误行、编码和输出语义。优化版本再把热字段拆到连续数组，把冷字段留在原始记录或错误表中。若常见输入格式稳定，可以设置快路径；若发现异常字符或缺失字段，再进入慢路径记录错误。性能报告要能说明热路径每条记录读多少字节、写多少字节、分支是否稳定、错误行是否影响主循环。这样数据布局和流水线章节不会停留在抽象讨论。

四、分区阶段体现 scan 和 manifest 解析后的记录按 key 或谓词进入多个 partition。每个 worker 先统计本 chunk 的 bucket 数量，做前缀和后得到输出区间，再回填结果。回填完成后生成局部 manifest 条目，记录 bucket、offset、length 和 checksum。若某个 worker 崩溃或被取消，只需要重做对应 chunk 或 partition，而不是全作业重跑。这里的重点是：scan 的中间结果同时服务性能和恢复，partition 的输出区间同时服务并行写和分布式 shuffle。

五、计算阶段体现线程和运行时 计算阶段由任务图驱动。读取完成某个 chunk 后，解析任务 ready；解析完成后，partition 任务 ready；所有相关 partition 完成后，merge 任务 ready；写出任务依赖 merge。worker 不能在任务内部阻塞等待另一个未运行任务，而应通过依赖计数和 continuation 让运行时调度后续任务。若某个任务失败，异常进入任务状态，依赖任务被取消或转入降级路径。最终报告要列出关键路径、worker 空闲时间、队列长度和取消传播时间。

六、写出阶段体现提交边界 输出先写临时文件，可靠写循环处理短写，完成后计算 checksum，再提交 manifest。manifest 提交前，结果对外不可见；manifest 提交后，恢复扫描必须能找到它。若写中崩溃，临时文件在恢复时删除或重做；若写完未提交崩溃，仍不能被当作有效输出；若提交后崩溃，输出必须可被读取。这个阶段把 I/O、生命周期和分布式幂等连接起来，因为重复 attempt 也只能提交同一个逻辑 block 一次。

七、分布式模拟体现消息身份 把本机 worker 换成 MessageBus 后，driver 发送带 request、attempt、deadline 和 chunk 身份的消息。worker 收到后执行同一套解析、分区和写出逻辑。若响应丢失，driver 用同一 request 重试；worker 通过幂等表返回旧结果，不重复提交。若 worker 慢到 deadline 过期，driver 可以放弃等待，但迟到响应仍要被识别和释放。这个模拟让读者看到，分布式不是把函数搬远，而是把身份、提交和不确定性写进协议。

八、恢复阶段体现状态机完整性 作业重启后，系统先读取 manifest，确认已提交输出；扫描临时目录，清理未提交文件；读取任务日志，重建未完成 chunk；读取幂等表，避免重复执行已提交 request。每个对象都要有确定命运：已提交、可重做、可删除、需人工检查。若某个文件既不在 manifest，也无法从任务日志解释，就是状态机缺口。恢复阶段是最终项目最严格的考试，因为它会暴露所有“正常运行时看不见”的边界。

九、性能报告体现证据链 端到端报告不能只给总耗时。它要列阶段耗时、输入大小、输出大小、队列峰值、内存峰值、线程数、I/O in-flight 数、重试次数、错误行数量、cache 或带宽证据、失败注入规则和未验证边界。读者应根据报告判断下一步优化方向：如果 parse 占比高，回到布局和分支；如果 write 占比高，回到 I/O 和背压；如果 worker 空闲，回到任务图和切分；如果重试多，回到消息模型和 deadline。

十、这个案例如何进入第三册 第三册实现 CPU 推理引擎时，输入文件变成权重和 token，chunk 变成 tensor block，partition 变成 batch 或算子切分，kernel 变成矩阵乘、量化反量化和激活函数，manifest 变成模型加载和输出提交记录，MessageBus 可以模拟多 worker 推理。第二册端到端案例已经训练了身份、布局、并行、I/O、调度和报告，第三册只需要把计算 kernel 换成 AI 算子，并继续沿用同一套系统纪律。这样三册之间是递进关系，不是主题跳跃。

18.21 移动端和受限 Linux 环境的实验边界

用户可能在 Termux、容器、云主机或权限受限的 Linux 环境中学习第二册。这样的环境不一定能使用全部性能计数器，不一定能设置 CPU 频率，不一定有 NUMA，也不一定允许 `io_uring` 或某些 tracing 工具。教材不能因此把实验全部取消，而要教读者区分“语义实验”和“硬件定量实验”。语义实验在受限环境中仍然能做，硬件定量结论则要诚实标注边界。

一、Termux 适合训练状态机和恢复 手机环境很适合跑小规模但严格的实验：有界队列、线程池关闭、短写模拟、manifest 恢复、MessageBus 丢包、重复请求和幂等表。这些实验不依赖高权限计数器，却能覆盖系统正确性。读者可以用小输入和固定随机种子反复运行，观察状态日志是否符合预期。不要因为设备不是服务器，就放弃状态机训练。

二、性能结论要降低表述强度 在移动端上，温控、调频、后台进程和系统限制会让性能数字波动更大。报告可以记录 wall time、阶段耗时、输入大小、线程数和内存峰值，但不要把它宣称为通用硬件结论。更合适的写法是：“在当前设备和当前温度策略下，这个输入显示写出阶段占比最高”。如果要得到更强结论，需要换到可控机器上复测。

三、工具缺失时保留替代证据 没有 perf，可以保留反汇编、阶段计时和输入族；没有 NUMA 工具，可以保留线程数、绑定限制和共享写实验；没有真实异步 I/O，可以用延迟写线程和短写包装器模拟完成事件；没有多机集群，可以用 MessageBus 模拟消息乱序和丢失。替代证据不能伪装成完整证据，但它能继续训练推导能力。

四、文件路径和产物要固定 移动端阅读时，产物目录应稳定，比如 `/sdcard/STU/BOOKS` 下按书名放 PDF 和 EPUB。项目实验也应把输入、输出、manifest、临时文件和报告放在可清理的固定目录。路径稳定能减少调试噪声，也方便读者把书和实验材料对应起来。工程学习中，文件组织本身就是降低认知负担的一部分。

五、受限环境更要求小而完整 设备资源有限时，不要追求巨大输入，而要追求小输入覆盖完整状态。三个 chunk、两个 worker、一个响应丢失、一个短写、一次重启，就能覆盖端到端恢复。输入小，时间线更容易画；状态完整，原理更容易验证。等语义稳定后，再把同一套实验搬到更强机器上测性能。这个顺序比直接追求大规模数字更可靠。

18.22 验收失败后的返工规则

最终项目如果没有通过验收，不能用“再调一调”含糊处理。返工应按失败类型回到对应章节，而不是在最后一章临时堆补丁。这样做能保护系统结构，也能让读者知道每个知识点在项目里的责任。一次失败报告至少要写出失败现象、触发输入、所属阶段、可能原因、下一步验证和回退方案。

一、结果不一致先回 reference 如果优化版本和 reference 输出不一致，先停止性能优化。检查输入解析、输出顺序、浮点误差、错误行处理和重复记录规则。若 reference 本身没有覆盖该边界，就先补 reference 和测试；若 reference 已经明确，才去查并行切分、scan offset、merge 顺序或分布式重复提交。正确性失败不能靠增加日志掩盖。

二、吞吐低先回证据链 如果系统慢，先看阶段耗时和资源峰值。解析慢回到布局、分支和 SIMD；计算慢回到 kernel 和并行算法；线程空闲回到任务图和队列；写出慢回到 I/O 和背压；重试多回到消息模型。不要看到慢就同时改十个地方。一次只验证一个假设，保留优化前后报告，才能知道哪一步真的有效。

三、内存失控先回所有权 如果内存峰值超过预算，先列出 buffer、队列、临时文件、幂等表、退休节点和中间数组。确认每个对象谁创建、谁释放、什么时候可见、失败时是否清理。很多内存问题不是分配器问题，而是生命周期没有闭合。回到所有权图，比盲目换容器更可靠。

四、关闭卡住先回等待状态 如果程序退出卡住，先列出所有等待者：线程池 worker、条件变量等待者、I/O completion、MessageBus timer、future 等待者。检查 close 是否广播，取消是否传播，任务是否还持有未完成依赖，队列是否已关闭但仍有人 push。关闭失败通常说明等待状态没有显式化，不能只在析构函数里强行终止线程。

五、恢复失败先回提交点 如果重启后重复输出、丢输出或读取半成品，先检查 manifest、临时文件、checksum、request id 和提交顺序。写出、提交、响应和幂等记录必须处于一致边界。恢复失败往往不是恢复代码单独的问题，而是正常路径没有把提交点写清楚。修复时要同时改正正常路径和恢复路径，并补崩溃点测试。

18.23 最终阅读顺序和复盘方法

读完第二册后，不建议只按页码复习。更有效的方式是按一次作业的生命周期复盘：输入身份怎样建立，数据怎样进入内存，热路径怎样减少分支和搬运，线程怎样领取任务，队列怎样背压，partial

怎样合并，输出怎样提交，消息怎样重试，恢复怎样判断每个对象的命运。每复盘一步，就回到对应章节找证据和反例。这样复习不是重复阅读，而是把知识重新编排成工程路径。

一、先复盘正确性 拿一个小输入，手写 reference 输出，再让并行版本、异步写出版本和分布式模拟版本都生成同一语义结果。若结果不同，不要先看性能。先问输出顺序、重复记录、错误输入、浮点误差和提交边界是否定义一致。正确性复盘能防止读者把系统复杂性误当成优化成果。

二、再复盘资源 同一输入下记录内存峰值、buffer 数、队列长度、线程数、临时文件数量和 in-flight 请求数。资源复盘会暴露很多隐藏假设：队列容量是否随输入放大，幂等表是否有过期策略，退休节点是否长期滞留，临时文件是否清理。系统能长期运行，靠的不是一次跑通，而是资源边界稳定。

三、最后复盘证据 每个结论都要能指向证据：反汇编、阶段耗时、checksum、状态日志、故障矩阵、源码阅读笔记或工具输出。没有证据的结论保留为猜测，不能写进最终设计。这个习惯会直接带到第三册的推理引擎：算子快不快、量化对不对、调度是否合理，都必须用同样的证据链说明。

复盘完成后，读者应能把一个陌生系统的问题拆成语义、资源、等待、生命周期和观测五条线，再按证据逐步收窄，而不是依赖经验猜测。这也是第二册最终要交付的能力：面对新机器、新输入和新故障时，仍然能从稳定问题出发，逐步建立可验证的系统解释。

18.24 贯穿项目：从一个能跑的统计器改成可恢复计算引擎

最终项目最容易走偏成“把前面所有模块都列一遍”。更好的写法是从一个能跑但不可靠的统计器开始。第一版程序读取日志，解析事件，按 key 计数，输出结果。正常输入上它很快，也能通过简单测试。然后我们连续注入几个事故：某个输入 chunk 解析到一半失败，某个 worker 在写出 block 后崩溃，某个响应丢失导致 task 重试，某个热点 key 让 reduce 长尾，Driver 在 manifest 提交前后分别重启。若程序只能在正常路径跑通，这些事故会把它拆散；若它是一个真正的计算引擎，每个事故都应进入状态表、manifest、checkpoint 和报告。

这个项目的主线不是“实现很多功能”，而是让每个功能从失败中长出来。输入身份来自重复处理和错误回溯；冷热布局来自热路径和调试证据冲突；scan 来自并行输出位置；任务图来自 worker 等待 future 的饥饿；可靠写来自短写和崩溃点；MessageBus 来自响应丢失；checkpoint 来自 Driver 重启；最终报告来自用户必须相信结果。这样第 18 章不是概念汇总，而是把前 17 章改写成一个可运行、可审查、可恢复的工程。

一、第一版只允许串行 reference 项目第一步必须保留串行 reference。它可以慢，但要把语义写清：输入行如何切分，非法字段如何记录，重复 record id 如何处理，key 如何编码，输出顺序是否稳定，计数溢出如何报告。Reference 的任务不是参与性能竞争，而是给后续所有优化提供 oracle。没有 reference，并行、异步和分布式版本都无法证明自己仍在解决同一个问题。

Reference 还要输出审计材料。每个输入文件有 file id，每个 chunk 有 offset 和 length，每条记录有 record id，每个输出 key 有确定顺序和 checksum。若后续版本结果不同，先用这些材料定位差异来自解析、过滤、聚合、分区还是写出。把 reference 写得太随意，最后项目会在性能优化里失去方向。

二、第二版建立身份账本 身份账本是全书第一条工程线。FileId 记录路径、大小、修改时间和输入 checksum；ChunkId 记录文件和 offset 范围；RecordId 记录 chunk 内序号；TaskId 记录阶段和 chunk；Attempt 记录物理执行；BlockId 记录输出文件、partition 和 checksum；ManifestGeneration 记录可见输出版本。这些字段看似繁琐，但每个都回答一个事故问题。

当 worker 迟到响应时，Attempt 能区分旧尝试和新尝试；当错误报告要回溯时，RecordId 能找到原始行；当恢复扫描发现临时文件时，BlockId 和 manifest 能判断它是否有效；当输入文件被替换时，FileId 的 checksum 能拒绝旧 checkpoint。身份账本不是日志装饰，而是项目能否解释自身状态的前提。

三、第三版拆出热路径和证据路径 解析后的数据进入 HotBatch：连续的 key id、value、record id、错误 mask。原文、错误上下文和字段字符串留在 ColdStore。聚合阶段只读 HotBatch，错误报告和调试再用 record id 回到 ColdStore。这个设计把第 6 章数据布局落到项目里。热路径不搬运原文，证据路径不丢失上下文。

这里要警惕一个常见失败：为了速度删除 record id。短期看聚合更快，长期看一旦 checksum 不一致，系统无法定位哪条记录导致差异。最终项目必须把“可解释结果”列为验收项。性能优化如果

破坏证据链，就不合格。

四、第四版用 scan 定义并行输出位置 Filter 和 partition 不允许多个线程直接 push 到共享输出。每个 chunk 先统计每个 bucket 的数量，再做 prefix，得到每个 chunk 每个 bucket 的输出区间，最后并行回填。这样输出位置是确定的，线程之间不抢写，manifest 也能记录每个 bucket 的范围。第 13 章的 scan 在这里变成工程结构：它不仅提高性能，还让失败重做范围清晰。

如果某个 chunk 回填失败，系统能知道哪些输出区间无效；如果某个 bucket 热点严重，报告能看到 bucket 大小和长尾；如果需要稳定输出，chunk 顺序和 bucket 顺序成为合同。并行算法不再是一个孤立 kernel，而是 manifest 和恢复的一部分。

五、第五版引入任务图而不是阻塞 future 读取、解析、局部聚合、partition、写出和 manifest commit 都是任务。任务之间用依赖边连接：chunk 读取完成后解析 ready，解析完成后聚合 ready，聚合完成后 partition ready，partition 完成后写出 ready，写出完成后 commit ready。任何 worker 都不能在任务内部阻塞等待同池 future。等待必须变成 continuation，事件到来后再入队。

这个设计能复现并修复第 14 章的饥饿事故。Trace 应记录 task submit、ready、start、finish、fail、cancel、resume。若项目慢了，报告能判断 worker 是空闲、阻塞、偷取过多还是依赖链太长。任务图不是为了炫技，而是让执行资源和等待状态可见。

六、第六版把写出改成临时文件加 manifest 写出阶段先写临时文件，可靠写循环处理短写，完成后计算 checksum，再尝试提交 manifest。Manifest 接受后，block 才对 reduce 或最终输出可见。临时文件存在不等于提交，completion 成功不等于提交，响应成功也不等于提交。这个边界贯穿 I/O 和分布式章节。

崩溃点矩阵必须进入测试。写前崩溃，任务可重做；写一半崩溃，临时文件 checksum 不匹配；写完未提交崩溃，临时文件仍是候选而非结果；manifest 提交后崩溃，恢复应接受该 block；清理前崩溃，恢复扫描再次清理。若某一格写不清，说明提交协议还不完整。

七、第七版加入 MessageBus 和故障注入 本机函数调用改成 MessageBus 后，Driver 和 worker 不共享业务对象。Driver 发送带 request、attempt、deadline、epoch、chunk id 的消息；worker 返回 block id、checksum 和状态。MessageBus 可以丢请求、丢响应、延迟、重复、乱序和返回 Busy。每个故障规则必须可复现，不能只靠随机概率。

响应丢失实验是最小验收。Worker 写出 block，reply 被 drop，Driver timeout 后创建新 attempt，新 attempt 提交，旧 reply 迟到。期望 manifest 只接受一个 block，旧 reply 被标记 LateAttempt，临时文件按规则清理。这个实验把第 16 章的事故放进最终项目，而不是停在文字。

八、第八版加入 checkpoint 和恢复 Checkpoint 保存 driver epoch、task table、manifest generation、partition assignment、retry budget、幂等表摘要和输入身份。恢复时先读 checkpoint，再读 manifest，扫描临时目录，对账输入文件，重建未完成任务。每个对象必须落入四类之一：已提交、可重做、可删除、需人工检查。恢复程序不能用文件修改时间猜测正确性。

恢复还要处理版本。若输入文件 checksum 变了，旧 checkpoint 不能继续使用；若协议版本不兼容，程序应拒绝恢复并保留旧文件；若 manifest 引用的 block 缺失，作业不能假装成功。最终项目的恢复语义要保守而清楚，宁可明确失败，也不能静默给出错误结果。

九、第九版加入热点 key 和分片策略 构造一个 key 占据输入的大部分。普通 hash partition 会让一个 reduce partition 变成长尾。第一步做 map-side combine，减少同一 key 的重复传输；第二步把热点 key 拆成多个子 key，分给多个 reduce；第三步二级 reduce 合并子 key。每一步都要保持 manifest 和 record id 证据链，不能让热点拆破坏最终精确计数。

热点策略也要进入报告。记录最大 key 占比、前十 key 占比、每个 partition 字节数、每个 reduce 耗时、二级合并成本和最终 checksum。这样分片不是“hash 一下”，而是从输入分布、负载均衡和提交语义推导出来。

十、第十版生成可审查 run report 最终项目每次运行都生成 run report。报告包含输入摘要、reference checksum、并行 checksum、阶段耗时、队列高水位、内存峰值、任务数、attempt 数、retry amplification、late reply 数、Busy 数、manifest generation、checkpoint generation、故障规则、恢复动作和未验证边界。报告本身也有版本和 checksum，方便比较多次运行。

这份报告是最终项目最重要的用户界面。没有它，用户只能相信程序；有了它，用户能审查程序为什么可信。高质量系统教材不应只教读者写快代码，还要教读者给出能被别人复查的证据。

十一、从一次失败演练看完整链路 把所有机制合在一起，做一次小输入演练：三个 chunk，两个 worker，一个热点 key，一个响应丢失，一个短写，一个 Driver 重启。时间线如下。Chunk 0 正常完成；Chunk 1 的 first attempt 写出后响应丢失；Chunk 2 触发热点拆分；写出阶段对某个 block 注入短写，可靠写循环补齐；Driver 在 manifest commit 后重启；恢复程序读取 manifest，接受已提交 block，拒绝旧 attempt 响应，重做未提交 task，最终 checksum 与 reference 一致。

这个演练比大规模 benchmark 更能说明项目质量。输入小，时间线能画出来；故障多，状态机能被验证；结果可比，reference 能兜底。等这个演练稳定后，再扩大输入、线程数和 worker 数，性能数据才有意义。

十二、项目代码边界 最终项目可以按模块组织：input 负责文件身份和 chunk；parse 负责 reference 和 HotBatch；runtime 负责任务图；partition 负责 scan 和 bucket；io 负责可靠写与 buffer 所有权；manifest 负责提交；bus 负责消息和故障注入；recovery 负责 checkpoint 和对账；report 负责证据输出。模块边界来自事故链，而不是文件夹审美。

C++ 示例也要遵守这个边界。跨模块传递固定宽度 id，不传裸指针；view 不拥有内存，owner 管理生命周期；状态枚举表达合法转移；错误返回区分 Retryable、Fatal、Busy、Stale 和 Cancelled；协议字段不要使用含糊的整型写法。最终项目既是教材内容，也是代码规范训练。

十三、验收标准 最终验收按顺序进行。第一，reference 和优化版在无故障下 checksum 一致。第二，故障注入矩阵每一格有预期和实际。第三，恢复后不重复提交、不读取半成品、不丢已提交结果。第四，热点输入下报告能解释长尾和改善。第五，受限环境中能跑语义实验，并诚实标注硬件结论边界。第六，PDF 和 EPUB 中保留代码、表述和路径，不出现原始 LaTeX 泄漏。验收顺序体现本书价值：正确性、恢复、可解释性在性能口号之前。

18.25 最终项目里程碑：每一步都要能失败、能解释、能回退

最终项目如果一次性要求实现完整计算引擎，读者很容易迷失。更好的推进方式是里程碑。每个里程碑只引入一个新的系统边界，同时保留上一版可运行结果和失败测试。这样项目从小到大增长，复杂度有来源，出错时也能回退。里程碑不是进度表，而是知识结构：每一步都说明为什么要加一个概念、它解决哪个事故、它带来什么新风险。

一、里程碑一：串行 reference 和输入身份 目标是读取输入、解析记录、统计 key、输出结果，并生成 reference report。新增概念只有输入身份和 record id。测试包括空文件、单行、非法字段、重复行、超长行。验收看输出 checksum、错误记录和 record id 回溯。这个里程碑不追求性能，它回答“我们到底在算什么”。若第一步语义含糊，后面所有优化都没有地基。

失败注入可以很简单：输入文件在两次运行之间被修改。程序应通过 file id 或 checksum 发现 reference 不可比较，而不是把旧报告和新输入混在一起。这个小事故让读者理解输入身份为什么要早早存在。

二、里程碑二：HotBatch 和地址报告 目标是把解析结果拆成热字段和冷字段。新增概念是 HotBatch、ColdStore、RecordId 索引和地址报告。测试要求聚合结果和 reference 一致，错误记录仍能回溯原文。报告比较 AoS 和 SoA 的阶段耗时、分配次数、内存峰值和错误回溯成功率。这个里程碑回答“如何让热路径少搬数据，同时不丢证据”。

失败注入是删除或损坏 ColdStore 中的一条记录。程序应在错误报告阶段发现 record id 无法回溯，并把它记为诊断失败，而不是输出错误的原文。这样布局优化和可诊断性同时被验收。

三、里程碑三：并行 scan 和 partition manifest 目标是把 filter、partition、local combine 改成并行版本。新增概念是 chunk count、exclusive scan、bucket range、partition manifest。测试包括共享 push 反例、倾斜 bucket、稳定输出和某 chunk 写入失败。验收看每个 chunk 的输出区间、最终 checksum、bucket 大小和重做范围。这个里程碑回答“多个 worker 如何写同一个逻辑输出而不互相踩”。

失败注入是让第三个 chunk 在回填中途失败。恢复程序应根据 manifest 或区间记录知道第三个 chunk 的范围无效，其他 chunk 不受影响。若系统只能全量重跑，也可以接受，但报告要说明当前恢复粒度。不要假装已经支持细粒度恢复。

四、里程碑四：线程池、队列和关闭 目标是把并行任务交给可解释线程池。新增概念是 Queue-Contract、worker state、task lifecycle、drain close、cancel close。测试包括上游快下游慢、关闭时

队列有元素、消费者等待空队列时关闭、任务抛异常。验收看没有悬挂，异常能回到 driver，队列高水位进入报告。这个里程碑回答“等待和关闭怎样不靠运气”。

失败注入是只通知一个条件变量方向，复现消费者永远睡眠；修复后，关闭能唤醒所有等待者。这个小测试比大压力测试更能说明条件变量谓词。

五、里程碑五：任务图和 continuation 目标是把读取、解析、聚合、写出、提交改成任务图。新增概念是 DependencyCounter、ReadyQueue、Continuation、CancellationSource。测试包括父任务等待子任务的饥饿反例、菱形依赖、任务失败后取消后继、关闭时仍有未完成任务。验收看 trace 能解释每个任务状态。这个里程碑回答“为什么任务不能把等待藏在线程栈里”。

失败注入是两个 worker、两个父任务、两个子任务的 future 饥饿。修复后，父任务后半段变成 continuation，子任务能运行。这个测试把任务图的必要性讲清楚。

六、里程碑六：可靠写、短写和 manifest 目标是把输出写到临时文件，完成后提交 manifest。新增概念是 WriteRequest、offset、in-flight bytes、temporary block、ManifestEntry。测试包括短写、写出延迟、写完未提交崩溃、提交后崩溃、取消后 completion 迟到。验收看恢复后只读取 manifest 接受的 block，不读半成品。这个里程碑回答“文件存在为什么不等于结果有效”。

失败注入是 write 只写一部分。同步版本应继续写剩余部分；异步版本应生成 PartialWritten completion 并维护 offset。若任何版本把短写当成功，manifest checksum 会失败。这个测试直接训练 I/O 语义。

七、里程碑七：MessageBus、retry 和幂等提交 目标是把本机 worker 改成消息驱动。新增概念是 MessageEnvelope、request id、attempt、deadline、retry budget、CommitRecord。测试包括丢请求、丢响应、重复请求、迟到响应、Busy、旧 epoch。验收看 manifest 不重复提交，retry amplification 可控，late reply 被分类。这个里程碑回答“远端执行成功为什么不是外部生效”。

失败注入是 worker 执行成功但 response 被 drop。Driver 创建新 attempt 并最终提交一个结果，旧 attempt 迟到后不改变 manifest。这个测试是第 16 章的核心事故，必须进入自动回归。

八、里程碑八：多进程边界 目标是让 Driver 和 worker 运行在不同进程，通过 Unix domain socket 或管道通信。新增概念是 wire format、framing、partial read/write、process crash、connection state。测试包括半 header 关闭、响应写一半关闭、worker 崩溃后临时文件残留、旧进程恢复发送旧响应。验收看协议解析严格，恢复规则不依赖共享内存。这个里程碑回答“本机模拟是否真的遵守消息边界”。

失败注入是 worker 在写出 block 后进程退出。Driver 应重试 task，恢复扫描应保留或清理旧临时文件但不能自动提交。这个测试会暴露单进程版本里隐藏的全局状态和指针假设。

九、里程碑九：分片、热点 key 和读写语义 目标是处理倾斜 key 和 partition route。新增概念是 partition version、route epoch、hot key split、二级 reduce、stale route。测试包括热点 key 长尾、旧 epoch 请求、子 key 合并、迁移中旧响应。验收看输出精确、长尾改善、旧 owner 无法提交新结果。这个里程碑回答“分片如何从负载问题变成状态归属问题”。

失败注入是 epoch 41 的旧请求在 epoch 42 后到达旧 owner。旧 owner 必须返回 StaleRoute 或转发，不能执行提交。报告应按 key 查询这条历史。

十、里程碑十：checkpoint、恢复和审计报告 目标是把所有状态纳入恢复。新增概念是 checkpoint generation、task table、manifest generation、idempotency table、recovery audit。测试包括 checkpoint 前崩溃、checkpoint 后响应丢失、manifest 缺失、临时目录脏文件、输入 checksum 改变。验收看恢复报告能解释每个对象命运。这个里程碑回答“系统重启后谁说了算”。

失败注入是 Driver 在 manifest 提交后、内存状态更新前崩溃。恢复时必须从 manifest 重建 committed 状态，而不是重做并重复提交。这个测试如果失败，说明提交权威仍然不清。

十一、里程碑十一：性能证据和边界声明 目标是生成可信性能报告。新增概念是阶段耗时、输入族、字节账本、队列高水位、线程状态、未验证边界。测试包括规则输入、随机输入、错误密集输入、热点输入、小输入、大输入。验收看报告能解释性能变化，而不是只给总时间。这个里程碑回答“快在哪里，为什么快，什么时候不保证快”。

受限环境中，报告要降级但不失真。没有 PMU，就不写确定的 branch miss 结论；没有 NUMA，就不声称 NUMA 优化；手机温控明显，就记录多轮样本和设备状态。诚实边界是高质量教材的一部分。

十二、里程碑十二：发布工件和阅读体验 最终里程碑不是代码，而是书和实验材料的发布。PDF 和 EPUB 要使用统一字体和代码样式，EPUB 不应包含解析错误的图片化图示，正文不应泄漏原始 LaTeX 符号，目录只保留章级入口。书籍产物放到固定手机目录，实验报告也放在项目输出目录。学习材料如果自己不可读，读者就很难相信它讲的工程质量。

这个里程碑回应一个现实问题：系统工程不只在代码里，也在文档、构建、发布和复查里。最终项目的产物要能被手机阅读、能被 git 追踪、能被重新构建、能被别人审查。可重复发布是可重复实验的延伸。

18.26 验收报告样例：一次小输入故障演练应该写什么

为了避免报告变成空泛模板，本节给出一份小输入故障演练的文字形态。输入包含三个文件，每个文件十行；其中一行非法字段，一个 key 是热点。系统启动两个 worker，一个 Driver，一个 MessageBus。故障规则为：丢弃 task 2 attempt 0 的成功响应；让 task 1 的写出第一次短写；在 manifest 提交 task 0 后重启 Driver。演练规模很小，但覆盖输入、解析、写出、消息、提交和恢复。

一、输入摘要 报告先写输入事实：文件数量、总字节数、总记录数、非法记录数、key 基数、最大 key 占比、reference checksum。非法字段的位置通过 file id、offset、record id 表达。这样后续任何阶段都能回到同一输入身份。不要一上来写性能数字，因为此时还不知道程序是否处理了同一批数据。

二、故障规则 故障规则必须可复现。写成“随机丢包”没有审计价值。应写：request 2007、task 2、attempt 0、TaskReply 第一次到达 MessageBus 时 drop；WriteRequest 1003 第一次 completion 返回 4096 bytes，total 为 16384 bytes；Driver 在 manifest generation 5 append 完成后退出。每条规则都有目标、触发次数和预期影响。

三、阶段结果 报告按阶段列出 read、parse、partition、write、commit、recover。Parse 阶段输出错误记录数和 HotBatch checksum；partition 阶段输出 bucket range 和热点拆分信息；write 阶段输出短写重试次数和 block checksum；commit 阶段输出 manifest entries；recover 阶段输出接受、重做、删除和人工检查对象数量。阶段报告让用户知道错误没有被总 checksum 掩盖。

四、状态机事件 事件表列出关键状态转移：task 0 committed before restart；task 1 partial write then completed；task 2 attempt 0 output written but reply dropped；task 2 attempt 1 committed；late reply from attempt 0 rejected。每个事件带 trace id、request id、attempt、epoch 和 reason。这样响应丢事故不再是文字描述，而是可审查数据。

五、最终判断 最终判断按顺序写。Correctness：parallel checksum 等于 reference checksum，非法记录一致。Recovery：Driver 重启后未重复提交 task 0，task 2 只有 attempt 1 进入 manifest。Resources：in-flight bytes 峰值低于预算，queue high watermark 可接受。Performance：本次小输入不用于硬件结论，只用于语义演练。Open boundary：未验证真实多机网络、掉电持久化和 NUMA。这个结论比“测试通过”更有信息量。

六、报告失败时如何返工 若最终 checksum 不一致，先比较 reference 和 HotBatch；若 HotBatch 一致，比较 partition range；若 range 一致，比较 block checksum；若 block 一致，比较 manifest；若 manifest 一致，检查 reduce 合并。报告应给出这种排查路径。没有路径，失败就会变成翻日志。最终项目要把调试路线也作为交付物。

18.27 源码级实现切片：把书里的概念落到文件和接口

最终项目如果只停留在文字，会让读者觉得所有原则都很正确，却不知道如何开始写代码。本节给出一个源码级切片，不要求一次实现完整系统，但要求每个目录、接口和测试都对应前面的事故。代码组织不是审美问题，而是责任边界。一个目录如果无法说明自己维护什么状态、拥有什么资源、处理什么失败，就会逐渐变成杂物箱。

一、input 模块只负责身份和字节范围 `src/input/` 负责 FileId、ChunkId、输入扫描、chunk 切分和输入 checksum。它不负责解析字段，也不负责业务统计。这样输入模块可以独立测试：给定文件和 chunk size，输出稳定 chunk 列表；文件改变后，FileId 改变；非法路径或读取失败返回明确错误。若 input 模块顺手解析业务字段，后面 HotBatch 和 reference 的责任就会混乱。

二、parse 模块同时提供 reference 和 HotBatch `src/parse/` 包含串行 reference parser 和

优化 parser。Reference parser 生成完整语义结果和错误记录；优化 parser 生成 HotBatch、ColdStore 和 error index。测试先比较两者语义，再测性能。这个模块的核心接口不是“parse 一行字符串”，而是“从 ChunkId 生成带 RecordId 的批次”。只有这样，错误回溯、分区和恢复才能共享身份。

三、runtime 模块不理解业务 key `src/runtime/` 负责 Task、TaskState、DependencyCounter、ReadyQueue、Worker、CancellationSource 和 TraceEvent。它不应该知道 event key、manifest 或 checksum 的业务含义。任务函数可以产生业务结果，但运行时只管理状态、依赖、取消、异常和调度。这样运行时可以用人工任务图测试，而不需要真实日志输入。若运行时直接操作 manifest，模块边界就会倒向一个巨大的 driver。

四、partition 模块负责输出位置合同 `src/partition/` 负责局部 histogram、exclusive scan、bucket range、回填和 PartitionManifest。它接收 HotBatch 和 partition function version，输出每个 bucket 的区间和 checksum。这个模块不写文件，也不提交最终结果。它的测试应覆盖共享 push 反例、热点 bucket、稳定顺序、predicate 版本不一致。这样 scan 的正确性和恢复边界可以独立验证。

五、io 模块负责 buffer 所有权和可靠写 `src/io/` 负责 BufferPool、WriteRequest、WriteState、可靠写循环、短写模拟、completion 和临时文件。它不决定 block 是否最终有效，只报告写出候选是否完整、checksum 是否匹配。Manifest 提交属于更高层。这个边界能避免 completion handler 直接写业务状态。I/O 模块测试包括短写、延迟完成、取消后完成、buffer 归还、队列字节高水位。

六、manifest 模块是提交权威 `src/manifest/` 负责 ManifestEntry、ManifestGeneration、CommitRecord、append、load、rebuild 和 duplicate rejection。它接收候选 block 和 task identity，决定是否接受。它不读原始输入，也不执行计算。恢复时，manifest 模块提供权威条目，其他模块按它清理或重做。测试包括重复 attempt、旧 epoch、半写 manifest、checksum mismatch 和 generation 恢复。

七、bus 模块只模拟传输不决定业务 `src/bus/` 负责 MessageEnvelope、framing、故障规则、inbox 容量、timer event、delivery event。它可以丢消息、延迟消息、重复消息、返回 Busy，但不应知道日志统计语义。业务层收到事件后进入状态机。这样 MessageBus 能用合成消息测试，也能迁移到多进程 socket。若 bus 直接调用 worker 函数并返回结果，它就退化成本地调用包装。

八、driver 模块编排状态机 `src/driver/` 负责 JobState、TaskRuntimeState、retry budget、deadline、checkpoint、recovery 和 run report 汇总。Driver 是控制面，负责把 input、runtime、partition、io、manifest、bus 连接起来。它不是所有代码的垃圾桶。每个复杂动作都应委托给对应模块，Driver 只保存跨模块状态和决策。Driver 测试是端到端故障矩阵，而不是所有单元测试的替代。

九、fault 模块让失败可复现 `src/fault/` 负责确定性故障注入：drop reply、partial write、worker crash、driver restart、busy response、stale epoch、checksum corruption。故障规则有 id、匹配条件、触发次数和 action。所有事故演练都引用 fault rule id。这样失败报告能重放，不依赖“随机有时发生”。随机压力可以作为后期测试，但不能替代确定性单点故障。

十、report 模块输出人能读的证据 `src/report/` 负责 RunReport、StageReport、FaultReport、RecoveryAudit、PerformanceSummary。它读取其他模块生成的结构化事件，输出文本或 JSON。报告模块不应重新推断业务事实，而是格式化已记录事实。若报告里需要某个字段，说明生产该字段的模块应在事件中记录。报告是系统的观察面，不是事后补注释。

十一、测试目录按事故组织 `tests/` 不只按模块组织，也按事故组织。比如 `tests/fault/drop_reply_retries_once`、`tests/recovery/crash_after_manifest_append`、`tests/queue/close_wakes_consumers`、`tests/partition/shared_push_counterexample`。事故测试能防止概念回退。每个测试都写输入、故障规则、预期状态、预期报告字段。这样读者看到测试名就知道它守住哪条教材原则。

十二、接口评审清单 每个公共接口评审时间八个问题。它是否暴露稳定身份；是否说明所有权；是否区分 view 和 owner；是否返回结构化错误；是否可取消；是否可重试；是否能写入 trace；是否有恢复语义。若某个接口只能在正常路径解释，就不应进入最终项目。这个清单能把全书质量要求压到代码层。

18.28 实现工单样例：不是一次写完整，而是逐张工单收敛

为了让读者真正能动手，本节把一个阶段拆成工单。每张工单都包含动机、修改、测试和报告字段。这样项目管理也体现教材方法：从问题出发，做最小改动，验证一个边界，再进入下一步。不要用“实现分布式系统”这种巨大工单吓住自己。

工单一：为输入 chunk 增加身份 动机：恢复和错误报告无法定位输入。修改：增加 FileId、ChunkId、RecordId；reference report 输出这些身份。测试：同一文件两次运行 id 稳定，修改文件后 FileId 改变，错误行能回溯 offset。报告字段：file count、chunk count、record count、input checksum。完成标准：没有性能要求，只要求身份稳定。

工单二：把共享 push 改成 count-scan-write 动机：共享输出锁竞争，失败后不知道写入范围。修改：filter 先 count，再 exclusive scan，再写区间。测试：共享 push 反例、稳定顺序、chunk 写失败。报告字段：chunk count、output offset、output count、scan checksum。完成标准：结果与 reference 一致，输出区间可审计。

工单三：给有界队列补关闭合同 动机：取消时消费者可能永远等待。修改：QueueContract 返回 Success、Closed、Cancelled；close drain 和 close cancel 分开；关闭时 notify all。测试：消费者等待空队列时关闭，生产者等待满队列时关闭，drain 保留已有元素。报告字段：queue high watermark、closed mode、waiter wake count。完成标准：关闭测试不悬挂。

工单四：写出层处理短写 动机：write 返回部分字节时可能误提交坏文件。修改：WriteRequest 保存 offset 和 remaining；completion 区分 PartialWritten、Completed、Failed。测试：注入第一次只写一部分，最终文件 checksum 正确；注入 fatal 错误时 manifest 不提交。报告字段：partial write count、retry count、block checksum。完成标准：短写不破坏输出。

工单五：manifest 拒绝重复 attempt 动机：响应丢失后重试会产生两个候选 block。修改：CommitRecord 按 task id 保存 accepted attempt；重复或迟到 attempt 返回 LateAttempt 或 Duplicate。测试：attempt 0 写出后响应丢失，attempt 1 提交，attempt 0 迟到。报告字段：accepted attempt、late reply count、duplicate count。完成标准：manifest 每个 task 只有一条有效记录。

工单六：Driver 重启后从 manifest 重建状态 动机：内存状态丢失导致重复提交。修改：启动时读取 manifest 和 checkpoint，重建 task table；旧 epoch 响应拒绝。测试：manifest append 后崩溃，恢复后不重做；append 前崩溃，恢复后重做。报告字段：recovered committed tasks、rescheduled tasks、stale replies。完成标准：两个崩溃点行为不同且符合预期。

工单七：热点 key 拆分和二级合并 动机：单个 reduce partition 长尾。修改：检测热 key，生成子 key，多个 reducer 处理，最终二级合并回业务 key。测试：分散强 key、单热 key、tie-break、重复 attempt。报告字段：max key ratio、subkey count、reduce p99、final merge checksum。完成标准：输出精确，长尾解释清楚。

工单八：生成 run report 动机：用户无法审查程序结果。修改：汇总输入、阶段、故障、恢复、性能和未验证边界。测试：每个故障演练报告包含指定字段；缺字段测试失败。报告字段本身即产物。完成标准：读报告能复现事故时间线，不需要翻源码猜测。

工单九：构建手机阅读产物 动机：教材需要稳定交付到手机目录。修改：构建 PDF 和 EPUB，复制到书名目录，校验 checksum，扫描 EPUB 不含图片和原始 LaTeX。测试：打开 PDF 文本抽取无明显宏泄漏，EPUB 解包无图片资源，目录章级可读。完成标准：产物路径固定，可重复生成。

工单十：提交和推送 动机：工程变更必须可追踪。修改：git status 确认只包含本次书稿和必要产物，commit 信息说明案例驱动重写，push 到远端。测试：远端推送成功，最终 commit hash 记录到回复。完成标准：用户不需要手动提醒提交推送。

18.29 Linux 源码对照阅读工单：从项目现象回到内核路径

用户反复强调 Linux 源码解读，这件事不能写成“列几个内核函数”。源码阅读必须从项目现象出发：线程为什么被唤醒，写文件为什么只是进入页缓存，队列等待为什么可能进入 futex，进程崩溃后文件为什么还在，网络连接关闭为什么只告诉我们传输断了而不是业务失败。读源码的目标不是覆盖内核全貌，而是让读者能把项目里的状态转移和 Linux 提供的机制对应起来。每次阅读都要有输入现象、用户态证据、内核入口、源码角色、未验证边界和回到项目的设计结论。

一、futex 对照有界队列等待 现象是消费者在队列为空时等待，生产者 push 后唤醒；关闭时必须唤醒所有等待者。用户态先看条件变量谓词：有元素、关闭、取消。Linux 层面，很多 pthread 条件变量和 mutex 的慢路径会落到 futex 思路：无竞争时尽量在用户态修改原子状态，竞争时进入内核睡眠，状态变化后唤醒等待者。阅读时不需要把 futex 每个分支背下来，而要理解它回答的问题：线程什么时候从可运行变成睡眠，谁负责唤醒，唤醒后为什么还要重新检查谓词。

回到项目，结论是条件变量不能等待通知本身，必须等待状态谓词。即使内核正确唤醒，用户态状态如果没有在同一锁保护下更新，等待者仍可能看到不一致事实。源码阅读证明的是“睡眠和唤醒只是执行机制”，不替代队列合同。报告应写：本次阅读解释了为什么低竞争锁可以很快，也解释了为什么高竞争或错误关闭会进入内核等待；它没有证明任何业务状态正确。

二、调度器对照 worker 空闲和抢占 现象是任务运行时报告某 worker 执行一个短任务用了很久。用户态 trace 显示任务 start 到 finish 时间长，但 CPU time 可能不长。Linux 调度器看到的是线程，不知道任务图。阅读调度路径时，重点不是完整理解所有调度类，而是理解可运行线程、睡眠线程、抢占、时间片、CPU 亲和和迁移这些角色。线程池以为 worker 正在运行，实际它可能被抢占；线程池以为任务在等待，操作系统只知道线程睡眠。

回到项目，结论是 run report 要区分 wall time 和 CPU time，并记录实际 CPU。若 worker 任务耗时长但 CPU time 小，问题可能是调度延迟或阻塞；若 CPU time 也长，才更可能是计算路径慢。受限环境没有完整调度跟踪时，报告至少保留 worker 时间线和系统可见 CPU 列表。源码阅读帮助读者理解“线程池不是操作系统调度器”，二者分层不同。

三、页缓存对照可靠写出 现象是 write 返回成功后，进程崩溃或系统异常时，恢复发现文件内容或目录项并不符合预期。用户态先看可靠写合同：临时文件完整写入、checksum 校验、必要同步、manifest 提交。Linux 阅读从 VFS、file、inode、address space、dirty page、writeback 的角色进入。普通 write 往往把数据放进页缓存并标记 dirty，设备写回稍后发生；fsync 试图把相关数据和元数据推到持久化边界；rename 改变目录项，但目录持久化也有自己的顺序问题。

回到项目，结论是文件存在不等于提交成功，write 返回不等于掉电后可见。教学项目可以在受限环境中不承诺完整掉电语义，但必须用进程崩溃点矩阵训练顺序：写中、写完未提交、manifest 后、清理前。源码阅读让读者知道为什么要使用临时文件、checksum 和 manifest，而不是扫描目录当作结果。

四、page fault 对照 mmap 和输入读取 现象是第一次扫描输入慢，第二次扫描快；或者 mmap 版本在某些输入上出现长尾。Linux 源码阅读从缺页处理、VMA、页缓存和文件映射关系进入。第一次访问映射页时，内核可能需要建立页表、读取文件页或关联页缓存；后续访问命中页缓存和 TLB，成本不同。这个路径能解释为什么冷运行和热运行不能混为一个数字。

回到项目，结论是性能报告要区分冷读和热读。若优化只是在热页缓存下测得，不能宣称改善了真实首次读取；若 mmap 简化了拷贝，但增加了缺页长尾，也要写入边界。源码阅读不是让读者背页表层级，而是帮助解释“为什么同一个顺序扫描第一次和第二次不一样”。

五、内存分配对照 HotBatch 和 arena 现象是节点式对象版本分配次数多、内存碎片明显，HotBatch 版本更稳。Linux 和运行库层面的完整分配器很复杂，教学阅读可从用户态 allocator 和系统调用边界入手：小对象可能来自分配器缓存，大块内存可能通过映射或堆扩展获得，释放也不一定立刻还给系统。内核只看到页和映射，不理解 LogRecord 语义。

回到项目，结论是 arena 适合生命周期一致的 batch，但不适合跨任务长期持有的对象。HotBatch 的 owner 必须活到所有 view 结束，错误报告如果需要冷数据，就不能提前释放 arena。源码阅读帮助读者把“分配快”翻译成“生命周期和页行为更可控”，而不是把 arena 当万能优化。

六、网络字节流对照 MessageBus framing 现象是多进程版本中 worker 收到半个消息，或响应发送一半后连接关闭。Linux socket 层提供字节流语义，应用层消息边界不自动保留。阅读时关注 read、write、poll 或 epoll 的用户可见行为：可读不等于完整消息，可写不等于全部写完，连接关闭不等于业务失败。TCP 或 Unix domain socket 的内部路径很深，本书只需追到“字节流和就绪事件”这条线。

回到项目，结论是 MessageEnvelope 必须有 framing，连接层必须维护 ReadingHeader 和 ReadingPayload 状态，业务层必须根据 request 和 attempt 判断连接关闭后如何恢复。源码阅读把“RPC 不是函数调用”进一步落到系统调用边界：一次 write 和一次 read 不是一条消息。

七、epoll 对照事件循环 现象是多个连接、timer 和 I/O completion 同时存在，业务代码如果

在回调里直接提交状态，很快混乱。Linux epoll 或类似机制提供 ready event，不提供业务状态机。阅读时关注事件来源：文件描述符可读、可写、错误、关闭；timerfd 或用户态 timer 触发；事件可能合并，处理后需要再次检查状态。内核告诉你“某个资源可能推进”，不告诉你“某个 task 应该提交”。

回到项目，结论是事件循环只生成 DriverEvent，业务状态由 Driver 状态机处理。TimeoutEvent、ReplyEvent、BusyEvent、CancelEvent 都统一进入同一判定流程。不要在 epoll 回调里直接写 manifest。源码阅读让读者理解系统事件和业务事件之间的边界。

八、进程退出对照 worker crash 现象是 worker 进程退出，Driver 看到连接断开或 wait 状态，但临时文件仍在。Linux 会回收进程内存、关闭文件描述符，却不会理解业务临时文件是否有效。阅读进程退出路径时，只需抓住资源回收范围：内存、描述符、进程状态由内核处理；业务文件、manifest、checkpoint 仍由应用协议处理。不要指望操作系统替你回滚业务状态。

回到项目，结论是 worker crash 只产生一个故障事件。恢复仍要通过 manifest、checksum、attempt 和 task table 判断临时文件命运。进程死亡不是业务事务回滚。这个结论能防止读者把系统崩溃恢复想得过于简单。

九、文件锁和进程锁的边界 现象是多个进程同时访问同一 manifest 或输出目录。Linux 提供文件锁和原子 rename 等机制，但它们不是分布式共识。文件锁可以协调同机进程，不能自动解决多机脑裂；rename 可以提供某些文件系统上的原子目录项切换，不能证明数据已经按业务语义提交。源码阅读或手册阅读要把这些边界写清。

回到项目，结论是教学版可以用单 Driver 管理 manifest，多进程 worker 不直接提交全局 manifest。若未来多 Driver，就需要 leader term 或共识。不要把本地文件锁包装成集群一致性。边界诚实比功能包装更重要。

十、cgroup 和移动环境对照性能数字 现象是同一线程数在 Termux、容器和服务器的表现差异很大。Linux cgroup、CPU quota、调度策略、后台限制和移动设备温控都会影响可运行时间。源码级深入不是本书重点，但读者应知道：可见 CPU 数、实际可用算力和调度权限可能不同。报告中记录环境比强行解释数字更重要。

回到项目，结论是性能报告要有环境段。手机上可完成状态机和恢复实验，但硬件结论要降级；容器内要记录 CPU quota；服务器上要记录 NUMA 和频率策略。源码阅读帮助读者理解“操作系统也是实验条件”。

十一、内核源码阅读报告格式 每份源码阅读报告都按六段写。第一，项目现象，例如短写、等待、缺页、连接关闭。第二，用户态最小复现。第三，内核入口或机制角色。第四，源码或文档能解释的事实。第五，仍未验证或依赖文件系统、平台、权限的边界。第六，回到项目的设计修改。这样的报告能防止源码阅读变成函数名堆砌。

十二、源码阅读不替代测试 读懂某条内核路径，不等于项目正确。futex 解释睡眠，不能证明队列谓词正确；页缓存解释 write 返回，不能证明 manifest 恢复正确；epoll 解释 ready event，不能证明业务状态机正确。源码阅读给我们物理和系统机制，测试验证我们的协议。两者互补，不可互相替代。

18.30 第三册交接前检查：CPU 推理引擎不能突然从零开始

第二册结束时，第三册还没有真正展开 AI，但交接准备必须清楚。第三册的大项目是 CPU 版本推理引擎，加载量化模型，在本地 Linux 上跑推理。它不应该突然从模型名词开始，而应该继承第二册的工程骨架：输入身份、可靠 I/O、HotBatch、任务图、buffer 所有权、manifest、trace、checkpoint、性能报告。不同的是，日志记录变成 token 和 tensor，统计 kernel 变成矩阵乘、量化反量化、归一化和采样，shuffle block 变成权重块、KV cache 和中间激活。

一、模型文件是更严格的输入 模型文件不是普通二进制 blob。它包含权重张量、量化参数、tokenizer、配置、版本和校验。第二册的 FileId、checksum 和 manifest 可以直接迁移。加载时先建立 ModelId，记录文件路径、大小、格式版本、权重 checksum、tokenizer checksum 和配置摘要。若任何一项变化，旧缓存和旧 benchmark 不能直接复用。这样第三册一开始就有输入身份，不会靠文件名猜模型。

二、权重布局继承 HotBatch 思路 量化权重的热路径只需要紧凑的整数块、scale、zero point 或 group 参数；调试路径需要张量名、原始维度、来源层和校验。权重布局应像 HotBatch 一样区

分热数据和证据数据。矩阵乘内核读紧凑块，报告和错误诊断通过 TensorId 回到模型元数据。这样性能和可解释性不冲突。

三、算子调度继承任务图 一次推理不是一个函数，而是一串算子依赖：embedding、attention、MLP、norm、sampling。单 batch 可以串行，多个请求可以并发，某些层内可以分块并行。任务图能表达算子依赖、buffer 生命周期和取消。若用户取消请求，已经提交的算子如何停止，KV cache 如何释放，输出 token 如何标记，都和第二册任务图一致。

四、KV cache 是生命周期考试 KV cache 会跨 token 存活，又随请求结束释放；它很大，不能随便复制；它可能按层、头、位置分块；并发请求共享权重但不共享 KV。第二册的 owner/view、BufferPool、in-flight bytes、checkpoint 思路都能迁移。第三册不能只讲 attention 公式，还要讲 KV cache 的内存布局、所有权、回收和观测。

五、量化结果要有 reference 第三册也必须先有 reference。可以先用简单标量实现或小矩阵高精度版本，定义量化反量化误差、token 输出差异和可接受范围。优化版的 SIMD、线程和缓存布局都和 reference 比较。没有 reference，量化推理很容易变成“能输出文字就算对”。第二册的 correctness first 原则必须延续。

六、算子性能报告继承 Roofline 和阶段报告 矩阵乘、softmax、layer norm、sampling 的瓶颈不同。报告要拆阶段，记录 token latency、prefill latency、decode latency、内存带宽、线程数、batch size、权重字节、激活字节。若某算子声称优化成功，要说明是减少访存、增加 SIMD 利用、改善线程分块，还是减少临时分配。第二册测量章的证据链直接复用。

七、推理请求继承 deadline 和背压 推理服务即使只在本地跑，也会遇到请求排队。用户可能提交多个 prompt，某些请求很长，decode 阶段占用时间多。系统需要 request id、deadline、queue capacity、cancel、Busy 和 run report。不要等到第三册讲服务化才引入这些概念。第二册的 MessageBus 和背压已经提供了模型。

八、模型输出也需要可复现边界 采样可能使用随机数，temperature、top-k、top-p、seed 都影响输出。若测试要求可复现，就要固定 seed 和采样策略；若允许随机，就要比较概率边界或中间 logits。第二册并行确定性章节已经训练了这件事：输出顺序和随机性必须成为接口合同。第三册只是在 AI 场景中重新应用。

九、Linux 观察仍然服务具体问题 推理引擎慢时，不要先猜模型太大。先看加载慢、prefill 慢、decode 慢、内存分配慢、线程空闲还是 cache miss。Linux 工具仍然只服务问题：缺页解释首次加载，perf 解释热算子，线程 trace 解释调度，I/O trace 解释模型读取。源码阅读仍然不堆函数名，而是解释现象。

十、第三册开篇应复用第二册最终项目 第三册第一章可以先把日志统计引擎中的 input、runtime、io、report 骨架保留下来，把 parse 和 kernel 替换成 tokenizer 和 tensor operator。这样读者会看到新主题不是另一本孤立书，而是在同一系统骨架上引入更复杂计算负载。第二册最终项目写得越扎实，第三册越能把篇幅用于 AI 模型和算子本身，而不是重新补系统基础。

十一、交接检查表 进入第三册前，第二册项目至少应具备：稳定输入身份，可靠写出，任务图 trace，buffer 所有权，manifest，run report，故障注入，手机产物发布。若这些还没有，第三册做推理引擎时会在模型加载、权重缓存、算子调度和输出报告中反复踩同样的坑。交接检查不是拖延，而是保证新内容能建立在已有工程能力上。

十二、读者能力目标 第二册结束时，读者不一定已经会写高性能推理引擎，但应该能面对一个推理引擎设计提出正确问题：模型文件身份是什么，权重怎样布局，KV cache 谁拥有，算子输出如何校验，线程池是否被阻塞 I/O 占住，deadline 如何传播，报告如何复现一次慢请求。能提出这些问题，第三册才有可能把 AI 算子讲深，而不是把读者淹没在新名词里。

18.31 端到端设计评审：把所有问题按顺序问完

最终项目交付前，应模拟一次设计评审会议。评审不问“用了哪些高级技术”，而按作业生命周期逐项追问。每个问题都要求指向代码、状态表、测试和报告字段。答不上来，就不是表达不够漂亮，而是系统边界还没写清。设计评审的价值在于它把全书知识变成一套固定问法，让读者以后面对任何系统都能复用。

一、输入评审 评审先问输入身份。文件被替换后，系统如何发现？同一路径下内容变化，旧 checkpoint 是否会被拒绝？非法行如何记录，错误报告能否回到 offset？若答案只是“输入文件一般不会变”，评审不通过。最终项目必须承认外部世界会变化，并用 FileId、ChunkId、RecordId 和 checksum 把变化写进证据链。

二、语义评审 接着问 reference。串行 reference 是否覆盖空输入、非法字段、重复记录、超长行和热点 key？优化版本改变输出顺序是否被允许？浮点或近似算法有没有误差规则？若 reference 只是为了跑通 demo，而不是语义合同，所有性能数字都不能进入评审。正确性是第一道门。

三、布局评审 然后问热路径。聚合阶段实际读取哪些字段？冷字段能否回溯？HotBatch 生命周期由谁拥有？View 是否可能在 arena 释放后继续使用？如果某个指针跨过异步边界，评审要求指出 owner。数据布局优化不是只看 cache，而要同时看证据和生命周期。

四、并行评审 并行阶段要问输出位置。多个 worker 是否写共享 vector？如果用了 scan, count 和 write 是否使用同一 predicate version？某 chunk 失败后，输出区间如何作废？Partition manifest 是否记录 bucket 范围和 checksum？若并行输出没有可审计区间，恢复和调试都会变成全局猜测。

五、线程评审 线程池要问任务分类。Compute、I/O、completion、driver 是否共用同一 worker 集合？若磁盘慢，解析任务是否会被写出任务占住？关闭时已提交任务是 drain 还是 cancel？等待者如何唤醒？设计评审不接受“析构时 join 一下”这种答案，必须看到状态机。

六、同步评审 锁和条件变量要问谓词。队列等待的是通知，还是等待“有元素、关闭、取消”这样的状态？多个锁的获取顺序是否固定？锁内是否调用用户回调或 I/O？指标变量是否偷偷参与关闭协议？同步代码最怕看起来短，评审要看不变量，而不是看 API 数量。

七、原子评审 原子变量要问角色。它是统计、发布、线性化、生命周期还是等待？每个 release 发布了哪些普通字段？每个 acquire 后允许读取哪些字段？Relaxed 变量是否可能被后续维护者拿来判断释放或关闭？如果不能给每个原子变量写角色，宁可用锁保留清晰语义。

八、I/O 评审 写出阶段要问提交点。Submit 成功、completion 成功、checksum 通过、manifest 提交分别是什么状态？短写如何推进 offset？Buffer completion 前能否复用？崩溃发生在写中、写完未提交、提交后、清理前分别怎样恢复？若答案依赖“文件应该已经写完”，评审不通过。

九、消息评审 分布式模拟要问未知状态。Timeout 是否被当成失败？重试是否有预算？旧 attempt 响应迟到如何分类？Busy 是否触发退避而不是立即重试？消息里是否带 request、attempt、epoch、deadline、trace、payload length 和 checksum？如果响应处理函数收到 Ok 就直接提交，说明第 16 章没有落地。

十、恢复评审 恢复阶段要问权威记录。重启后先信 manifest、checkpoint、临时目录还是 worker 响应？每个临时文件如何归类？Manifest 指向的 block 缺失怎么办？Checkpoint 版本不兼容怎么办？恢复不是正常路径的附录，而是系统语义的最终裁判。评审必须能看到恢复矩阵。

十一、性能评审 性能报告要问证据链。总耗时下降来自哪个阶段？输入族是否覆盖规则、随机、错误密集、热点和小输入？没有 PMU 权限时用了什么替代证据？数字是否超过字节账本能解释的上限？性能评审不接受“快了很多”，必须给出可复查的阶段数据和边界声明。

十二、可读性评审 最后问产物。PDF 目录是否只保留章级入口？代码块是否可读？EPUB 是否没有图片化图示和原始 LaTeX 泄漏？手机目录是否按书名组织？这些看似排版问题，实际影响学习路径。一本系统教材如果自己的构建产物不可审查，就很难说服读者相信它的工程纪律。

十三、评审结论写法 设计评审结论不要写“基本通过”。它应列出通过项、阻塞项、风险项和后续工单。通过项说明已有测试；阻塞项必须在发布前修复；风险项说明当前边界，例如未验证真实掉电、未验证多机网络、未验证 NUMA；后续工单进入第三册或后续版本。评审结论本身也应提交到仓库，成为项目历史的一部分。

十四、为什么这不是形式主义 这些问题看起来多，但每个都来自前面的具体事故：重复 partial、解析器变慢、共享计数器、队列关闭悬挂、ready 发布错误、Treiber 栈回收、共享 push、future 饥饿、短写、响应丢失、旧 owner 写入、Driver 重启。设计评审只是把事故经验固化成问法。真正的形式主义是背术语却不问事故。

18.32 常见反例库：用小事故防止系统退化

最终项目维护到后期，最容易退化的不是大架构，而是一个个小反例被遗忘。新人看到代码，会觉得某个字段太麻烦、某个状态太保守、某个测试太小，于是删掉 `attempt`、合并错误码、把 `manifest` 改成扫描目录、把 `queue close` 简化成一个布尔值。短期看代码少了，长期看同一类事故会重新出现。因此项目应保留一组常见反例库。每个反例都很小，但都守住一条系统原则。回归测试不是为了覆盖率数字，而是为了防止知识退化。

反例一：重复 `partial` 让结果翻倍 输入只有两行，`split 0` 统计出 `key A` 的计数为二。Worker 写出 `partial` 后，Driver 在写 `manifest` 前崩溃。错误恢复扫描临时目录，把旧 `partial` 读入；随后又重做 `split 0`，生成新 `partial`；`reduce` 合并两份，结果变成四。这个反例守住 `manifest` 权威性。修复不是禁止重试，而是规定只有 `manifest` 接受的 `block` 才能进入 `reduce`，临时目录只是候选池。测试应检查临时目录里可以有多份候选，但 `manifest` 每个逻辑 `task` 只有一条有效记录。

反例二：错误行被性能优化吞掉 解析器优化后跳过了慢错误路径。规则输入 `checksum` 一致，错误密集输入中错误行数量变少。这个反例守住 `reference` 和错误语义。修复不是把错误路径重新塞回热循环，而是保留快路径检测和慢路径精确报告。测试应同时比较成功记录、错误记录和错误位置。只比较最终计数，可能完全看不到错误记录丢失。

反例三：共享计数器让扩展曲线停住 每条记录处理后都更新一个全局原子计数器。八线程以内还能接受，线程继续增加后吞吐不升反降。这个反例守住共享写热点分析。修复是把高频计数拆成线程本地或节点本地 `partial`，低频合并。测试不只看最终计数，还看扩展曲线、`cache line` 争用或替代指标。`Atomic` 语义正确，不代表它适合高频热路径。

反例四：队列关闭只通知生产者 消费者在空队列上等待，主线程关闭队列，只唤醒等待空间的生产者，没有唤醒等待元素的消费者。程序退出卡住。这个反例守住条件变量谓词。修复是让关闭状态进入所有等待谓词，并在关闭时唤醒所有方向的等待者。测试要用容量为一的小队列复现，不需要大压力。小反例越小，越能说明问题本质。

反例五：发布状态早于字段写入 生产者先把 `ready` 置为 `true`，再写 `payload` 字段。消费者看到 `ready` 后读字段，偶尔读到旧值或未初始化值。这个反例守住发布协议。修复是先写普通字段，最后 `release` 发布状态；消费者 `acquire` 观察状态后再读字段。测试不一定能稳定复现，所以代码评审必须写出发布了哪些字段。内存序正确性不能完全交给运行结果。

反例六：无锁栈释放了仍被观察的节点 线程一读到栈顶节点后暂停，线程二弹出并释放该节点，分配器复用地址，线程一恢复后继续使用旧指针。这个反例守住无锁生命周期。修复可以是 `hazard pointer`、`epoch` 或先保留锁版结构。测试要故意暂停读者线程，观察退休列表和释放时机。无锁结构的关键不是 `CAS` 写得短，而是节点什么时候可以安全释放。

反例七：共享 `push` 让输出顺序和恢复都失控 多个线程过滤记录后直接向共享 `vector push`。加锁版本慢，去锁版本错，本地 `vector` 合并版本顺序不稳定，失败后不知道哪些输出有效。这个反例守住 `scan` 的系统意义。修复是 `count`、`scan`、`write` 三阶段，让每个 `chunk` 拥有确定输出区间。测试要在某个 `chunk` 写到一半时失败，检查恢复能定位该区间。

反例八：`future` 等待把线程池全部占住 两个 `worker` 分别运行父任务，父任务提交子任务后同步等待。子任务已经 `ready`，但没有 `worker` 执行。这个反例守住任务图和 `continuation`。修复是把父任务后半段注册为子任务完成后的 `continuation`，释放 `worker` 去跑 `ready` 任务。测试只需要两个 `worker` 和四个任务。若这个反例不在回归中，运行时很容易被重新写成阻塞线程池。

反例九：短写被当作完整成功 写出 16 KiB `block`，第一次 `write` 只返回 4 KiB，代码却把它当成功并提交 `manifest`。Reduce 读取坏文件，`checksum` 不一致。这个反例守住可靠写循环。修复是 `WriteRequest` 保存 `offset` 和 `remaining`，直到所有字节完成后才进入 `checksum` 和 `commit`。测试要注入短写，而不是等待真实文件系统偶然出现。可靠 I/O 必须可模拟。

反例十：响应丢失后两个 `attempt` 都提交 `Attempt 0` 执行成功并写出 `block`，成功响应丢失。Driver 超时后启动 `attempt 1`，`attempt 1` 提交。随后 `attempt 0` 的旧响应迟到，错误代码再次提交。这个反例守住分布式模型的核心：执行成功不等于外部生效。修复是 `CommitRecord` 按逻辑 `task` 接受一个 `attempt`，旧响应只进入 `late` 事件。测试要检查 `manifest generation` 不变，`late reply` 指标增加。

反例十一：旧 owner 在新 epoch 后继续写 Partition 7 从 worker A 迁移到 worker B, epoch 从 41 切到 42。A 收到旧请求并按旧路由写出结果，B 也按新路由提交。这个反例守住状态归属和 fencing。修复是请求、响应、manifest 都带 route epoch，旧 epoch 写入被拒绝或转发。测试要按一个 key 的时间线检查旧 owner 是否失去提交权。

反例十二：恢复时相信内存而不是日志 Driver 在 manifest append 成功后崩溃，内存状态还没更新。恢复程序只看 checkpoint 中旧状态，认为 task 未提交，于是重做并重复输出。这个反例守住恢复权威。修复是恢复时从 manifest 重建 committed 状态，再用 checkpoint 补充未完成任务。测试要区分 append 前崩溃和 append 后崩溃，两者行为必须不同。

反例十三：性能报告只给总时间 优化版声称快三倍，但没有输入摘要、checksum、阶段耗时、环境和失败输入。这个反例守住测量纪律。修复是生成 run report：输入、正确性、阶段、资源、故障和边界。测试可以要求报告字段存在，缺字段即失败。性能报告如果不能被审查，速度数字就不能进入设计决策。

反例十四：手机环境数字被当成通用结论 Termux 上某个版本更快，就把结论写成“该算法更快”。后来在服务器上结果相反。这个反例守住环境边界。修复是报告设备、温度、权限、可见 CPU、是否有 PMU、是否有 NUMA，并把移动端结论降级为当前环境现象。受限环境可以训练状态机，不应假装验证全部硬件结论。

反例十五：源码阅读变成函数名堆砌 报告列了一串 Linux 函数，却没有解释项目现象。这个反例守住源码阅读方法。修复是每次阅读从现象开始：短写、缺页、等待、连接关闭、进程崩溃。然后写用户态复现、内核机制、能解释的事实和不能证明的边界。源码阅读服务工程判断，不服务炫耀阅读量。

反例库的使用方式 这些反例应进入测试、设计评审和章节复习。每次改 manifest，跑重复 partial 和响应迟到；每次改队列，跑关闭唤醒；每次改运行时，跑 future 饥饿；每次改 I/O，跑短写和崩溃点；每次改分片，跑旧 epoch 写入。反例库让系统记住过去犯过的错。它也让教材质量可持续：不是靠作者不断提醒，而是靠测试和评审把原则固定下来。

反例库和第三册的关系 第三册推理引擎会出现新名词，但这些反例仍然有效。模型文件被替换对应输入身份；量化误差对应 reference；KV cache 悬垂对应生命周期；请求取消对应队列关闭；算子 completion 对应异步完成；权重 manifest 对应提交权威；热 token 或长 prompt 对应热点 key；性能报告仍要写环境边界。反例库是跨主题的系统思维，不是第二册一次性材料。

18.33 一次失败排查日记：从错误结果回到单个提交点

最终再给一段排查日记。假设小输入只有一百条记录，reference 输出 key A 为三十七，优化版输出三十九。这个差异很小，最容易被误判成并行顺序、浮点误差或随机噪声。正确做法不是先改代码，而是沿着报告一层一层收窄。第一步看输入摘要：文件 checksum 一致，记录数一致，非法记录数一致，说明输入和解析边界大概率没有变。第二步看 HotBatch checksum：reference parser 和优化 parser 的 key 序列一致，说明冷热布局没有丢字段。第三步看 partition manifest：bucket 2 的输出 count 比 reference 多二。到这里，问题已经从全系统缩小到一个 bucket 的输出位置或重复提交。

一、先排除解析和布局 排查人员打开错误记录表，确认非法字段数量一致；再按 record id 抽查 key A 的记录，发现每条记录都能回溯到原始 offset。若这一步失败，应该回第 6 章的 HotBatch 和 ColdStore，而不是继续查分布式提交。很多系统调试浪费时间，就是因为没有先排除上游语义。报告能让我们用证据说“解析阶段暂时不是嫌疑人”。

二、再排除 scan 写区间 查看 count、scan、write 三阶段。Count 阶段显示 chunk 3 对 bucket 2 的 count 为五，write 阶段也写了五个元素；chunk 4 的 count 为七，write 阶段却写了九个元素。这个差异说明 predicate 在 count 和 write 两次执行中不一致，或者 chunk 4 被执行了两次。若是 predicate 不一致，应回到配置版本；若是执行两次，应继续查 task attempt。排查不靠猜，而靠阶段中间量。

三、再看 task attempt Task table 显示 chunk 4 有两个 attempt。Attempt 0 写出成功，但响应被故障规则丢弃；Attempt 1 后来写出并提交。Manifest 却包含 attempt 0 和 attempt 1 两条 block。至此根因出现：manifest 提交表没有按逻辑 task 去重，而是按 block 文件名接受。这个错误和第 16 章响应丢失事故完全一致。输出多二不是并行算法问题，而是提交权威问题。

四、修复要落在提交层 错误修复不应在 reduce 阶段过滤重复文件名，也不应在恢复扫描时按时间选择较新文件。真正修复是 ManifestCommit 检查 task id、stage id、attempt 和 generation：若逻辑 task 已有 accepted attempt，新 block 只能进入 late 或 duplicate 分类。修复后，临时目录里仍可以保留两个 block，但 reduce 只读取 manifest 接受的一个。这样系统承认重复执行存在，同时阻止重复生效。

五、补回归测试 新增回归测试叫“响应丢失后旧 attempt 不重复提交”。输入仍是一百条记录，故障规则固定丢 attempt 0 的 reply。测试断言三件事：parallel checksum 等于 reference，manifest 中 chunk 4 只有一条 accepted entry，late reply count 为一。还要断言临时目录可以存在旧 block，因为临时文件存在不是错误，错误是把它当有效结果。这个测试把排查日记固化成反例库的一员。

六、更新报告字段 这次事故还暴露报告缺口：旧报告只有最终 checksum，没有每个 task 的 accepted attempt。修复后，run report 增加 task commit summary：task id、accepted attempt、candidate blocks、late replies、duplicate replies。这样下次看到 checksum 差异，可以更快定位到提交层。报告字段不是越多越好，而是每次事故后补上能缩短排查路径的证据。

七、复盘结论 这个排查日记展示了第二册最重要的方法：错误结果先回 reference，再沿阶段证据缩小范围，最后把根因落到一个明确提交点。不要被并行、异步、分布式这些大词吓住。系统再复杂，也可以拆成输入、布局、输出位置、任务 attempt、写出、manifest 和恢复。只要每层有身份、状态和 checksum，排查就不是玄学。

18.34 第二册最终交付清单

第二册交付时，读者不应只拥有一本 PDF 和几个零散实验，而应拥有一套能复查的材料。第一，书稿本身讲清了从硬件、单机性能、多线程、并行算法、I/O 到分布式计算的推导链。第二，贯穿项目有 reference、任务图、可靠写出、manifest、故障注入和报告。第三，每个关键概念至少有一个反例测试守住。第四，构建产物能在手机上阅读。第五，仓库历史能说明这次改动为什么发生。这个清单看似普通，却能防止项目回到“概念很多、证据很少”的状态。

一、正文交付 正文必须以案例推动概念。看到流水线，要能回到解析循环；看到 TLB，要能回到地址序列；看到锁，要能回到队列关闭；看到原子，要能回到 ready 发布；看到 scan，要能回到共享 push；看到分布式，要能回到响应丢失。若某一章只能背名词，说明还没有达到第二册标准。

二、实验交付 实验必须小而完整。每个实验都要有 reference、输入、故障规则、预期状态、实际报告。大输入 benchmark 可以作为补充，但不能替代小反例。小反例让读者理解为什么概念必要，大 benchmark 只说明某台机器上的性能现象。两者顺序不能反。

三、报告交付 Run report 至少包含输入摘要、正确性 checksum、阶段耗时、资源高水位、故障规则、恢复动作和未验证边界。报告如果不能解释一次失败，就只是日志集合。报告如果能让别人重放事故，它就是工程材料。第二册最终项目必须把报告当成一等产物。

四、源码阅读交付 Linux 源码阅读笔记必须绑定项目现象。futex 对应队列等待，页缓存对应可靠写，epoll 对应事件循环，调度器对应 worker 时间线，进程退出对应 worker crash。每份笔记都写能解释什么和不能证明什么。源码阅读不是函数名索引，而是系统现象的解释工具。

五、发布交付 PDF 和 EPUB 要放到固定目录，文件名稳定，字体和代码风格统一，EPUB 不含图片化图示，正文不泄漏 LaTeX 符号。发布不是最后的机械步骤，而是读者学习体验的一部分。能稳定发布，才说明构建系统、排版和内容结构都进入可维护状态。

六、第三册入口 第三册开始前，第二册应留下清楚接口：模型文件继承输入身份，权重布局继承 HotBatch，算子调度继承任务图，KV cache 继承 buffer 所有权，推理请求继承 deadline 和背压，推理报告继承 run report。这样第三册不是另起炉灶，而是在第二册计算系统上加入 AI 算子和推理语义。

18.35 发布前人工复读

自动检查能发现很多格式和构建问题，但最后仍要人工复读几个关键段落。复读不是从第一页逐字校对，而是按风险点抽查。先读第 16 章响应丢失事故，看 timeout 是否仍被解释成未知状态，旧 attempt 是否不会重新提交。再读第 17 章单 key 迁移，看旧 owner、route epoch、leader term 和 manifest generation 是否串成一条时间线。最后读第 18 章故障演练，看 reference、HotBatch、

scan、任务图、可靠写、MessageBus、checkpoint 和 run report 是否能组成同一个项目，而不是八个并列模块。

一、复读概念来源 每遇到一个术语，都问它从哪个事故出来。Deadline 来自本地等待不能证明远端停止；attempt 来自同一 task 多次执行；manifest 来自临时文件不能直接代表结果；route epoch 来自旧 owner 迟到写；continuation 来自 worker 等待 future 的饥饿；release 和 acquire 来自 ready 发布普通字段。若某个术语找不到事故，就要改写或删除。这样能防止正文重新退化成概念表。

二、复读代码边界 代码示例要检查三件事。第一，协议字段使用固定宽度整数或清楚类型，不用含糊写法。第二，发送、写出、提交、恢复这些跨边界动作都带身份和状态。第三，示例不暗示可以直接把 C++ 对象布局当作持久格式或网络格式。代码是教材可信度的一部分，示例若偷懒，读者会把偷懒当成建议。

三、复读排版体验 移动端阅读时，章节标题不应挤成目录噪声，代码块要能横向理解，正文第一行要有缩进，彩色只用于强调和代码，不应抢正文。EPUB 中若图示被转换成图片，会影响微信读书等阅读器，所以本册尽量用文字事件表和代码表达图示。排版不是装饰，它决定读者能否长时间读下去。

四、复读质量边界 如果某段文字放到任何系统书里都成立，就说明它太空。要把它改成本书对象：日志记录、HotBatch、partition、manifest、attempt、worker、Driver、checkpoint。具体对象越清楚，读者越能把知识落到实验。第二册的标准不是“讲过这个词”，而是“读者能用这个词解释一个失败运行”。

五、发布后回归策略 发布不是结束。后续每次改书稿、改项目代码或改构建脚本，都要先跑反例库，而不是只重新生成 PDF。最小回归包括：错误行回溯、共享 push 改 scan、队列关闭唤醒、ready 发布、短写、响应丢失、旧 epoch 写入、Driver 重启恢复。若时间有限，优先跑这些小反例，而不是跑一个大 benchmark。小反例守住语义，大 benchmark 只说明当前机器上的速度。这个策略能让第二册长期保持清晰，不会随着后续第三册加入推理引擎而把基础边界弄乱。

六、后续版本如何扩展 以后继续扩写时，不要直接往章节末尾堆新专题。每次新增内容都要先选一个事故：某个算子结果不一致，某个 buffer 悬垂，某个模型文件版本错，某个请求取消后 completion 迟到。先写事故，再推概念，再给代码和实验。这样本书可以继续增长，但不会变成术语百科。内容越多，越要坚持从具体失败推导抽象原则。

七、读者复查路线 读者复查第二册时，可以只拿一个小输入反复走完整链路。先看它在 reference 中怎样得到正确答案，再看 HotBatch 怎样保存热字段和 record id，再看 scan 怎样给它分配输出位置，再看 task attempt 怎样写出候选 block，再看 manifest 怎样接受或拒绝它，最后看恢复报告怎样解释它的命运。这个复查路线能把全书压成一条记录的旅程，避免复习时重新变成概念背诵。

八、最小闭环 最小闭环只有一句话：同一条记录在任何阶段都能说明自己从哪里来、现在归谁管、失败后如何处理、最终是否已经生效。硬件、并发、I/O 和分布式知识都服务这句话。只要这个闭环不断，系统扩大以后仍然可解释；一旦闭环断开，代码再快也只是暂时能跑。

九、最终提醒 第二册不追求把所有系统技巧一次讲完，而是要求每个技巧都有来源、有边界、有证据。读者以后遇到陌生框架，也可以先问同样的问题：输入身份是否稳定，状态是否发布，等待是否可见，提交点是否唯一，恢复是否能解释每个残留对象，报告是否能让别人复查。能把这些问题问顺，系统学习就不再依赖记忆名词。最终交付要让读者相信两件事：程序为什么得到这个结果，以及结果出错时应该从哪里开始查。这就是闭环。

18.36 案例深化：最终项目的故障演练脚本

最终工程章要把前面所有原则收束成一套可运行演练。演练不是随机制造错误，而是按系统边界逐个切开：输入、解析、局部聚合、shuffle、写出、manifest、checkpoint、恢复。每个脚本都要有明确预期和验收指标。这样项目不只是“能跑 word count”，而是能证明自己在故障下仍然解释得清楚。

一、输入损坏脚本 构造包含非法字段、超长行、缺失分隔符和重复 record id 的输入。Reference 给出错误记录和可接受输出。并行版本必须报告同样错误，不因为分块而丢失错误位置。这个脚本连接第一章的 record id、第二章的解析热循环和第六章的冷热字段回溯。

二、shuffle 丢 block 脚本 让某个 map attempt 写出 block 后，删除临时文件或让响应丢失。系统应检测 manifest 缺失或 checksum 不匹配，重试相应 task，而不是让 reduce 静默少读。验收看最终 checksum、manifest 条目和重试次数。这个脚本连接 I/O 可靠写入和分布式幂等提交。

三、Driver 重启脚本 在 manifest 写入前、写入后、checkpoint 写入前、写入后四个位置分别终止 Driver。恢复后，已提交 task 不重做，未提交 task 重做，旧 attempt 响应被分类。验收看 generation、task table、临时目录和 trace。这个脚本证明最终项目真的有恢复语义。

四、热点 key 脚本 构造一个 key 占据大部分输入。普通 hash partition 会让一个 reduce partition 长尾。加入本地聚合和热 key 拆分后，观察尾延迟和输出正确性。验收不只看速度，还要看子 key 合并是否精确，manifest 是否防止重复 attempt 计数。这个脚本连接数据结构、并行算法和分片调度。

五、报告生成脚本 最终项目应能自动输出一份 run report：输入摘要、reference checksum、并行 checksum、任务数、attempt 数、retry amplification、queue high watermark、manifest generation、checkpoint generation、故障规则和未验证边界。报告生成本身是工程能力。没有报告，用户只能相信程序；有报告，用户可以审查程序。

18.37 案例复盘：最终引擎靠故障演练收束

最终项目不能只展示正常 word count。丢一个 shuffle block、Driver 在 commit 后重启、旧 attempt 晚到、热点 key 拖慢 reduce，都会让系统暴露真实质量。没有 reference、manifest、checkpoint 和 trace，程序即使输出数字，也无法解释这个数字是否可信。

从演练脚本到验收报告 演练按输入损坏、shuffle 丢 block、Driver 重启、热点 key、慢 worker 的顺序打开。每一步检查 reference checksum、parallel checksum、attempt 数、checkpoint generation、queue high watermark、临时文件清理和故障规则。性能报告放在正确性和恢复报告之后。

给第三册留下工程骨架 第三册 AI 推理引擎会遇到相同问题：模型 artifact 需要 manifest，算子执行需要 trace，量化结果需要 checksum，推理请求需要 deadline 和 backpressure。第二册最终项目不是结束，而是为后续系统保留一套可复查的工程骨架。

18.38 本章习题

习题与作业

习题一：为日志统计系统写端到端数据流图。要求包含输入分片、map-side combine、shuffle partition、reduce merge、输出提交和 checkpoint。

习题二：设计 shuffle manifest。说明它记录哪些 map 输出、partition、attempt、文件位置或消息范围，以及恢复时如何使用。

习题三：构造热点 key 输入。比较普通 hash partition、map-side combine、热 key 拆分和二级 reduce。说明每种方案的网络流量和 reduce 长尾变化。

习题四：设计 task attempt 提交协议。要求处理 attempt A 成功但响应丢失、attempt B 后完成、driver 重启三种情况。

习题五：为本机模拟系统设计故障注入表。至少包含丢消息、重复消息、慢 worker、慢 reduce、提交失败和 checkpoint 损坏。每个故障写出预期恢复行为。

附录 A

实验路线

第二册的实验围绕 Compute Systems Labs 展开。第一组实验是硬件与测量基础，包括 `lscpu` 拓扑记录、缓存层级观察、TLB 与缺页计数、`perf stat`、顺序归约和并行归约。第二组实验是数据布局，包括 AoS、SoA、false sharing、顺序访问、随机访问和预取效果。第三组实验是单机高性能计算，包括多累加器、SIMD 自动向量化、Roofline 分析和典型内核模式。

第四组实验是多线程同步，包括 mutex counter、atomic counter、bounded queue、条件变量、信号量和线程亲和性。第五组实验是无锁结构，包括 SPSC ring buffer、CAS 循环、ABA 现象、版本标签和内存回收方案分析。第六组实验是并行算法，包括 reduce、scan、partition、thread-local histogram 和任务粒度比较。第七组实验是运行时，包括线程池、任务队列、工作窃取模型、异步 I/O 和背压。第八组实验是分布式计算，包括本机多进程 RPC 模拟、超时重试、幂等请求、分片、复制和 shuffle。

每个实验必须记录环境、输入、命令、输出、正确性检查和结论。性能数据没有上下文就不能进入正文。若实验受手机、Termux 或虚拟化环境限制，也要明确记录限制。

附录 B

源码阅读索引

本附录保存后续源码专题的入口。

Linux 内核源码用于理解调度、NUMA、页表、透明大页、perf 事件、futex、epoll、内存分配和网络栈。阅读时不要从全局搜索函数名开始，而要围绕问题进入：线程为什么迁移、缺页为什么发生、futex 怎样把用户态锁和内核等待连接起来。

glibc 和 musl 用于阅读 pthread、mutex、condition variable、malloc 和系统调用封装。重点观察用户态 fast path 和进入内核 slow path 的分界。

Intel TBB、oneTBB 或 LLVM OpenMP runtime 用于阅读任务调度、线程池、工作窃取和并行算法运行时。阅读时关注任务粒度、队列结构、窃取策略和 shutdown 语义。

Folly、Abseil 和 Boost.Lockfree 用于阅读并发容器、原子封装、hazard pointer、RCU 风格回收和高性能队列。阅读时先搞清进度保证和内存回收，不要只看 CAS。

Redis、RocksDB、Kafka 和 Spark 用于阅读分布式计算和存储工程：事件循环、后台线程、LSM compaction、分区、复制、日志、shuffle、背压和故障恢复。阅读时抓数据流和失败路径，不要只列模块名。

术语表

算术强度 操作数与数据移动字节数的比值，用来判断一个内核更可能受计算吞吐还是内存带宽限制。

Roofline 用计算峰值、内存带宽和算术强度估算性能上界的模型。

SIMD 一条指令处理多个数据 lane 的执行方式，是现代 CPU 单核吞吐的重要来源。

乱序执行 CPU 在保持单线程可观察结果不变的前提下，提前执行已经准备好的指令以隐藏延迟。

分支预测 CPU 在分支结果确定前预测执行路径，预测失败会清空错误路径上的工作。

cache line 缓存一致性和缓存填充的基本粒度，常见大小为 64 字节。

TLB 地址翻译缓存，用于缓存虚拟页到物理页的映射。

NUMA 非统一内存访问结构，不同核心访问不同内存节点的成本不同。

false sharing 多个线程写不同变量，但变量落在同一 cache line 上，导致一致性流量和扩展性下降。

内存序 原子操作对可见性和重排施加的约束，例如 relaxed、acquire、release 和 `memory_order_seq_cst`。

lock-free 一类进度保证，表示系统整体能持续前进，但不保证每个线程有限步完成。

背压 下游处理不过来时向上游施加的流量控制机制。

共识 分布式系统中多个节点在故障下对日志顺序或状态达成一致的协议问题。

索引

instruction

add, 34